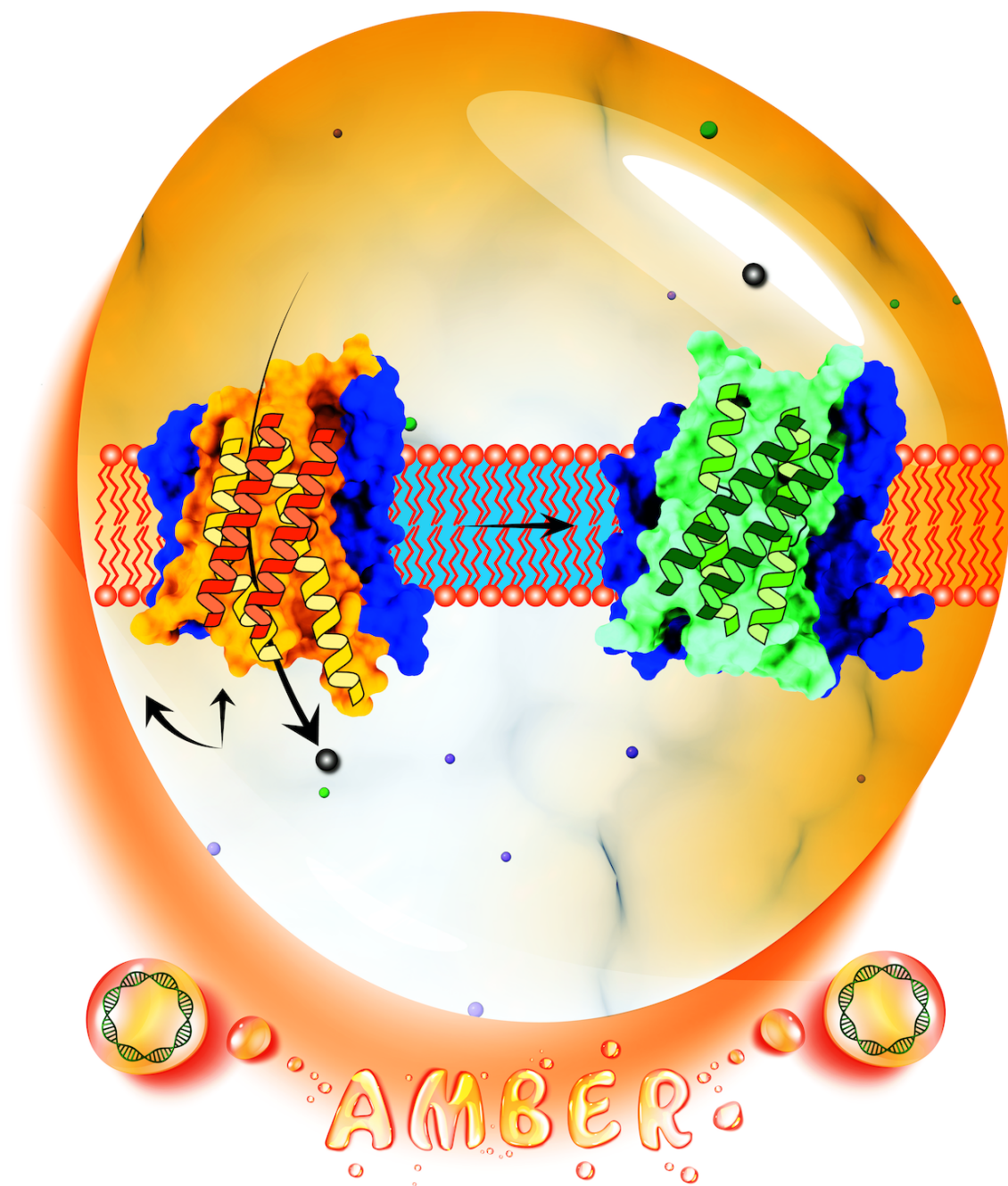


Amber 2025

Reference Manual

(Covers Amber24 and AmberTools25)



Amber 2025

Reference Manual

(Covers Amber24 and AmberTools25)

Principal contributors to the current codes:

David A. Case (Rutgers)
Thomas E. Cheatham III (Utah)
Carlos Simmerling (Stony Brook)
Adrian Roitberg (Florida)
Kenneth M. Merz (Michigan State)
Ross C. Walker (Independent Consultant)
Ray Luo (UC Irvine)
Pengfei Li (Loyola University Chicago)
Tom Darden (OpenEye)
Celeste Sagui (NCSU)
Feng Pan (FSU)
Junmei Wang (Pitt)
Daniel R. Roe (NIH)
Jason Swails (Entos, Inc.)
Andreas W. Götz (UC San Diego)
Jamie Smith (Relativity Space)
David Cerutti (Psivant Discovery)
Taisung Lee (Rutgers)
Darrin York (Rutgers)
Timothy Giese (Rutgers)
Tyler Luchko (CSU Northridge)
Felipe Silva Carvalho (CSU Northridge)
Negin Forouzesh (CSU LA)
Vinícius Wilian D. Cruzeiro (Architect Therapeutics)
Gérald Monard (U. Lorraine)
Yinglong Miao (UNC)
Jinan Wang (UNC)
Charles Lin (TandemAI)
G. Andrés Cisneros (UNT)
Ali Rahnamoun (Michigan State)
Zhen Li (Michigan State)
Akhil Shajan (Michigan State)
Madushanka Manathunga (Atmos Inc)
Joshua T. Berryman (Uni. Luxembourg)

Mahdieh Ghazimirsaeed (AMD)
István Kolossváry (Budapest, Boston U.)
Nikolai R. Skrynnikov (Purdue, SPbU)
Oleg Mikhailovskii (Purdue, SPbU)
Yi Xue (Tsinghua)
Sergei A. Izmailov (SPbU)
Koushik Kasavajhala (Stony Brook)
Kellon Belfon (Roivant Discovery)
Jana Shen (U Maryland)
Julie Harris (U Maryland)
Alexey Onufriev (Virginia Tech)
Saeed Izadi (Virginia Tech, Genentech)
Xiongwu Wu (NIH)
Holger Gohlke (Düsseldorf/FZ Jülich)
Stephan Schott-Verdugo (FZ Jülich)
Yongxian Wu (UC Irvine)
Zhen Huang (UC Irvine)
Qiang Zhu (UC Irvine)
Cizhang Zhao (UC Irvine)
George Giambasu (Merck)
Jian Liu (Peking Univ.)
Hai Nguyen (Schrodinger)
Scott R. Brozell (Rutgers)
Andriy Kovalenko (NINT)
Mike Gilson (UC San Diego)
Ido Ben-Shalom (UC San Diego)
Tom Kurtzman (CUNY)
Sergio Pantano (Inst. Pasteur, Uruguay)
Matias Machado (Inst. Pasteur, Uruguay)
H. Metin Aktulga (Michigan State)
Mehmet Cagri Kaymak (Michigan State)
Kurt A. O'Hearn (Michigan State)
Peter A. Kollman (UC San Francisco)

For more information, please visit <https://ambermd.org/contributors.html>

- When citing Amber 2025 (comprised of AmberTools25 and Amber24) in the literature, the following citation should be used:
D.A. Case, H.M. Aktulga, K. Belfon, I.Y. Ben-Shalom, J.T. Berryman, S.R. Brozell, F.S. Carvalho, D.S. Cerutti, T.E. Cheatham, III, G.A. Cisneros, V.W.D. Cruzeiro, T.A. Darden, N. Forouzeshe, M. Ghazimirsaeed, G. Giambasu, T. Giese, M.K. Gilson, H. Gohlke, A.W. Goetz, J. Harris, Z. Huang, S. Izadi, S.A. Izmailov, K. Kasavajhala, M.C. Kaymak, I. Kolossváry, A. Kovalenko, T. Kurtzman, T.S. Lee, P. Li, Z. Li, C. Lin, J. Liu, T. Luchko, R. Luo, M. Machado, M. Manathunga, K.M. Merz, Y. Miao, O. Mikhailovskii, G. Monard, H. Nguyen, K.A. O'Hearn, A. Onufriev, F. Pan, S. Pantano, A. Rahnamoun, D.R. Roe, A. Roitberg, C. Sagui, S. Schott-Verdugo, A. Shajan, J. Shen, C.L. Simmerling, N.R. Skrynnikov, J. Smith, J. Swails, R.C. Walker, J. Wang, J. Wang, X. Wu, Y. Wu, Y. Xiong, Y. Xue, D.M. York, C. Zhao, Q. Zhu, and P.A. Kollman (2025), Amber 2025, University of California, San Francisco.
- Peter Kollman died unexpectedly in May, 2001. We dedicate Amber to his memory.
- *Cover illustration:* The figure represents the transition of a ZIP metal transporter between an inward-facing state and an outward-facing state during the transport cycle. The stationary region of the protein is shown as a solid blue surface. Green and purple spheres represent Cl^- and K^+ ions, respectively, while the zinc ion is shown as a grey sphere. The structures were captured using enhanced sampling simulations, demonstrating that metal transport is coupled with elevator-like motions, where the transport domain moves upward relative to the stationary domain. During a transport cycle, the transport domain undergoes a hinge motion (curved arrow on the bottom left) and vertical sliding (straight arrow on the bottom left). This figure is based on work published by Prof. Kenneth Merz Jr. and Prof. Jian Hu (*Nature Commun.* 2024, **15**, 9665). Figure by Majid Jafari.
- This manual was created on Mon Jul 28 09:41:34 AM MDT 2025 .

Contents

Contents	5
I. Introduction and Installation	13
1. Introduction	15
1.1. Information flow in Amber	15
2. Installation	19
2.1. Basic installation guide	19
2.2. The cmake build system in Amber	21
2.3. Applying Updates	26
2.4. Contacting the developers	28
II. Amber force fields	29
3. Molecular mechanics force fields	31
3.1. Proteins	32
3.2. Nucleic acids	37
3.3. Modified amino acids and nucleotides	39
3.4. Carbohydrates	41
3.5. Lipids	48
3.6. Solvents	50
3.7. Ions	53
3.8. Force fields related to semi-empirical QM	55
3.9. The GAL17 force field for water over platinum	55
3.10. Fluorescent dyes: AMBER-DYES in AMBER force field files	57
3.11. Coarse-grained and multiscale simulations using the SIRAH force field	58
3.12. Obsolete force field files	61
4. The Generalized Born/Surface Area Model	63
4.1. GB/SA input parameters	65
4.2. Implicit Solvent with Explicit Ions model (GBION)	68
4.3. Effect of Pressure, Osmolytes and Temperature in Implicit Solvent: the Group Transfer Free Energy (GTFE) model	72
4.4. ALPB (Analytical Linearized Poisson-Boltzmann)	73
5. GBNSR6	75
5.1. GB equations available in gbnsr6	75
5.2. Numerical implementation of the R6 integral	75
5.3. Usage	76
6. PBSA	79
6.1. Introduction	79
6.2. Usage and keywords	82

CONTENTS

6.3. Torch PBSA runtime	91
6.4. Example inputs and demonstrations of functionalities	91
6.5. Visualization functions in <i>pbsa</i>	94
6.6. <i>pbsa</i> in <i>sander</i> and NAB	103
6.7. GPU accelerated <i>pbsa</i>	104
7. Torch PBSA	108
7.1. LibTorch installation	108
7.2. Usage	109
8. Reference Interaction Site Model	111
8.1. Introduction	111
8.2. Practical Considerations	117
8.3. Work Flow	120
8.4. <i>rismld</i>	123
8.5. 3D-RISM in <i>sander</i>	126
8.6. <i>rism3d.snglpnt</i>	136
8.7. RISM File Formats	140
9. sqm: Semi-empirical quantum chemistry	147
9.1. Available Hamiltonians	147
9.2. Dispersion and hydrogen bond correction	149
9.3. Usage	150
10. QUICK: <i>ab initio</i> quantum chemistry	156
10.1. Features and limitations	157
10.2. Installation	157
10.3. Usage	158
11. QM/MM calculations	159
11.1. Built-in semiempirical NDDO methods and SCC-DFTB	159
11.2. Interface for <i>ab initio</i> and DFT methods	168
11.3. QM/MM simulations with QUICK	184
11.4. QM/MM simulations with TeraChem	187
11.5. QM/MM simulations with xTB	191
11.6. QM/MM simulations with DFTB+	193
11.7. QM/MM simulations with DPRc machine learning corrections	196
11.8. Adaptive solvent QM/MM simulations	198
11.9. Adaptive buffered force-mixing QM/MM	203
11.10 SEBOMD: SemiEmpirical Born-Oppenheimer Molecular Dynamics	210
11.11 ReaxFF/AMBER	215
12. Using energies and forces from an external library	222
12.1. Installation instructions	222
12.2. Simulation setup and input parameters	222
III. System preparation	224
13. Preparing PDB Files	226
13.1. Cleaning up Protein PDB Files for AMBER	226
13.2. Residue naming conventions	227
13.3. Chains, Residue Numbering, Missing Residues	228
13.4. <i>pdb4amber</i>	228

13.5. reduce	230
13.6. pdb2pqr	231
13.7. gwh (guess water hydrogens)	231
13.9. packmol-memgen	232
13.10 Building bilayer systems with AMBAT	236
14. LEaP	238
14.1. Introduction	238
14.2. Concepts	238
14.3. Running LEaP	242
14.4. Basic instructions for using LEaP to build molecules	247
14.5. Error Handling and Reporting	248
14.6. Commands	249
14.7. Building oligosaccharides, lipids and glycoproteins	267
15. Reading and modifying Amber parameter files	276
15.1. Understanding Amber parameter files	276
15.2. ParmEd	284
16. Antechamber and GAFF	313
16.1. Principal programs	313
16.2. A simple example for antechamber	318
16.3. Programs called by antechamber	321
16.4. Miscellaneous programs	326
17. paramfit	330
17.1. Usage	331
17.2. The Job Control File	332
17.3. Multiple molecule fits	338
17.4. Fitting Forces	338
17.5. Examples	339
18. Molecular Mechanics Parameter Fitting in mdgx	341
18.1. Input and Output	341
18.2. Installation	342
18.3. Partial Charge Development	342
18.4. Implicitly Polarized Charge Development	344
18.5. Customizable Virtual Site Support	346
18.6. Bonded Term Fitting in mdgx	349
18.7. Configuration Sampling	352
18.8. Parallel Generalized Born Problems on the GPU	355
19. Python Metal Site Modeling Toolbox (pyMSMT)	357
19.1. Introduction	357
19.2. Usage	358
20. Electrostatic Parameterization with py_resp.py	371
20.1. pyresp_gen.py	371
20.2. py_resp.py Usage	372
20.3. Examples for py_resp.py	376
20.4. Large Scale Electrostatic Parametrization using PyPE_RESP	380
21. Setting up crystal simulations	387
21.1. UnitCell	387

CONTENTS

21.2. PropPDB	387
21.3. AddToBox	388
21.4. ChBox	389
 IV. Running simulations	 390
22. sander	392
22.1. Introduction	392
22.2. File usage	393
22.3. Example input files	394
22.4. Namelist Input Syntax	395
22.5. Overview of the information in the input file	396
22.6. General minimization and dynamics parameters	396
22.7. Potential function parameters	419
22.8. Polarisable Gaussian Multipole Model	428
22.9. Varying conditions	429
22.10 File redirection commands	433
22.11 Getting debugging information	433
22.12 multisander (and multipmemd)	436
22.13 APBS as an alternate PB solver in Sander	437
22.14 Programmer's Corner: The sander API	439
 23. pmemd	 461
23.1. Introduction	461
23.2. Functionality	461
23.3. PMEMD-specific namelist variables	463
23.4. Slightly changed functionality	465
23.5. Parallel performance tuning and hints	465
23.6. GPU Accelerated PMEMD	466
 24. Atom and Residue Selections	 473
24.1. Amber Masks	473
24.2. "Atom Expressions" in NAB Applications	476
24.3. GROUP Specification	476
 25. Sampling configuration space	 481
25.1. Self-Guided Langevin dynamics	481
25.2. Accelerated Molecular Dynamics	484
25.3. Gaussian Accelerated Molecular Dynamics	488
25.4. Targeted MD	496
25.5. Multiply-Targeted MD (MTMD)	497
25.6. Nudged elastic band calculations	499
25.7. Adaptive String Method	502
25.8. Low-MODE (LMOD) methods	505
25.9. DL-Find Optimization	511
 26. Free energies	 516
26.1. Thermodynamic integration	516
26.2. Linear Interaction Energies	531
26.3. Replica Exchange Molecular Dynamics (REMD)	532
26.4. Adaptively Biased MD, Steered MD, Umbrella Sampling with REMD and String Method	559
26.5. Steered Molecular Dynamics (SMD) and the Jarzynski Relationship	575

26.6. Absolute Free Energies using EMIL	577
26.7. Nonequilibrium sampling and Jarzynski's equation	581
27. Constant pH calculations	591
27.1. Background	591
27.2. Preparing a system for constant pH simulation	591
27.3. Running at constant pH	594
27.4. Analyzing constant pH simulations	597
27.5. Extending constant pH to additional titratable groups	597
27.6. pH Replica Exchange MD	602
27.7. cphstats	602
28. Constant Redox Potential calculations	611
28.1. Preparing a system for constant Redox Potential simulation	611
28.2. Running at constant Redox Potential	613
28.3. Analyzing constant Redox Potential simulations	614
28.4. Extending constant Redox Potential to additional titratable groups	614
28.5. Redox Potential Replica Exchange MD	615
28.6. cestats	615
29. Continuous constant pH molecular dynamics	618
29.1. Implementation notes	618
29.2. Usage description	619
29.3. Continuous constant pH MD with pH replica exchange	624
29.4. Obtaining parameters for a novel titratable group	625
30. NMR refinement	627
30.1. Distance, angle and torsional restraints	628
30.2. NOESY volume restraints	633
30.3. Chemical shift restraints	635
30.4. Pseudocontact shift restraints	636
30.5. Direct dipolar coupling restraints	637
30.6. Residual CSA or pseudo-CSA restraints	639
30.7. Preparing restraint files for Sander	640
30.8. Getting summaries of NMR violations	647
30.9. Time-averaged restraints	647
30.10 Multiple copies refinement using LES	648
30.11 Some sample input files	648
31. Xray and cryoEM refinement	653
31.1. EMAP restraints for rigid and flexible fitting into EM maps	653
31.2. FRETrest: Förster Resonance Energy Transfer restraints	655
31.3. X-ray functionality and diffraction-based restraints for pmemd	657
32. Locally-enhanced sampling	660
32.1. Preparing to use LES with Amber	660
32.2. Using the ADDLES program	661
32.3. More information on the ADDLES commands and options	663
32.4. Using the new topology/coordinate files with SANDER	664
32.5. Using LES with the Generalized Born solvation model	665
32.6. Case studies: Examples of application of LES	665
33. gem.pmemd	669
33.1. Introduction	669

CONTENTS

33.2. Input variables	669
34. Artificial Intelligence / Machine Learning	673
34.1. KMMD: Molecular Dynamics Using a Kernel Machine	673
35. NAB, nabc and libsf	677
V. Analysis of simulations	678
36. mdout_analyzer.py and ambpdb	680
36.1. ambpdb	680
37. cpptraj	682
37.1. Introduction	682
37.2. Running Cpptraj	683
37.3. General Concepts	687
37.4. Variables and Control Structures	690
37.5. Data Sets and Data Files	693
37.6. Data File Options	697
37.7. Coordinates (COORDS) Data Set Commands	702
37.8. General Commands	716
37.9. Topology File Commands	730
37.10. Trajectory File Commands	741
37.11. Action Commands	752
37.12. Analysis Commands	845
37.13. Analysis Examples	893
38. pytraj	895
38.1. Introduction	895
38.2. Development	895
38.3. Documentation and examples	895
39. MMPBSA.py	899
39.1. Introduction	899
39.2. Preparing for an MM/PB(GB)SA calculation	899
39.3. Running MMPBSA.py	902
39.4. Python API	915
40. FEW	921
40.1. Installation	921
40.2. Overview of workflow steps and minimal input	923
40.3. Common setup of molecular dynamics simulations	924
40.4. Workflow for automated MM-PBSA & MM-GBSA calculations (WAMM)	931
40.5. Linear interaction energy workflow (LIEW)	939
40.6. Thermodynamic integration workflow (TIW)	943
41. BAR/PBSA	953
41.1. Introduction	953
41.2. Usage	953
41.3. Example for <i>bar_pbsa.py</i>	956
42. edgembar	957
42.1. edgembar	957

42.2. edgembar-amber2dats.py	960
42.3. edgembar-WriteGraphHtml.py	960
42.4. edgembar-bookend2dats.py	961
42.5. fetkutils-tischedule.py	961
43. SAXS	963
43.1. Introduction and theory	963
43.2. Usage	964
44. MoFT: analysis of volumetric data	967
44.1. Usage	967
44.2. Examples	969
45. ndfes	971
45.1. ndfes	971
45.2. ndfes-PrepareAmberData.py	973
45.3. ndfes-CheckEquil.py	977
45.4. ndfes-CombineMetafiles.py	979
45.5. ndfes-PrintFES.py	980
45.6. ndfes-AvgFESs.py	980
45.7. ndfes-path	981
45.8. ndfes-path-prepguess.py	985
45.9. ndfes-path-analyzesims.py	986
Bibliography	988
Index	1049

Part I.

Introduction and Installation

1. Introduction

Amber is the collective name for a suite of programs that allow users to carry out molecular dynamics simulations, particularly on biomolecules. None of the individual programs carries this name, but the various parts work reasonably well together, and provide a powerful framework for many common calculations.[1–4] The term *Amber* is also used to refer to the empirical force fields that are implemented here.[5, 6] It should be recognized, however, that the code and force field are separate: several other computer packages have implemented the *Amber* force fields, and other force fields can be implemented with the *Amber* programs. Further, the force fields are in the public domain, whereas the codes are distributed under a license agreement.

The Amber software suite is divided into two parts: **AmberTools** a collection of freely available programs mostly under the GPL license, and **Amber24**, which is centered around the *pmemd* simulation program, and which has a more restrictive license. Please see <https://ambermd.org> for an overview, tutorials and downloading information. You can perform many simulation tasks with **AmberTools**, and can do more extensive simulations with the combination of **AmberTools** and **Amber** itself.

Everyone should read (or at least skim) this chapter. Even if you are an experienced Amber user, there may be things you have missed, or new features, that will help. Although Amber may appear dauntingly complex at first, it has become easier to use over the past few years, and overall is reasonably straightforward once you understand the basic architecture and option choices. In particular, we have worked hard on the tutorials to make them accessible to new users. Thousands of people have learned to use Amber; don't be easily discouraged.

If you want to learn more about basic biochemical simulation techniques, there are a variety of good books to consult, ranging from introductory descriptions,[7–9] to standard works on liquid state simulation methods,[10–12] to multi-author compilations that cover many important aspects of biomolecular modelling.[13–17] Looking for "paradigm" papers that report simulations similar to ones you may want to undertake is also generally a good idea. If you are new to this field, Chapter 15 provides a basic introduction to force fields, along with details of how the parameters are encoded in Amber files.

1.1. Information flow in Amber

Understanding where to begin in **AmberTools** is primarily a problem of managing the flow of information in this package — see Fig. 1.1. You first need to understand what information is needed by the simulation programs (*sander* or *pmemd*). You need to know where it comes from, and how it gets into the form that these programs require. This section is meant to orient the new user and is not a substitute for the individual program documentation.

Information that all the simulation programs need (see the circles in Fig. 1.1):

1. Cartesian coordinates for each atom in the system. These usually come from X-ray crystallography, NMR spectroscopy, or model-building. They should generally be in Protein Data Bank (PDB) format. The program *LEaP* provides a platform for carrying out many of these modeling tasks, but users may wish to consider other programs as well. Generally, editing of these files is needed, and the *pdb4amber* script can do some of this.
2. Residue libraries: Connectivity, atom names, atom types, residue names, and charges. This information comes from the database, which is found in the `$AMBERHOME/dat/leap/lib` directory, and is described in Chapter 3. It contains topology for the standard amino acids as well as N- and C-terminal charged amino acids, DNA, RNA, and common sugars and lipids. Topology information for other molecules (not found in the standard database) is kept in user-generated “residue files”, which are generally created using *antechamber*.

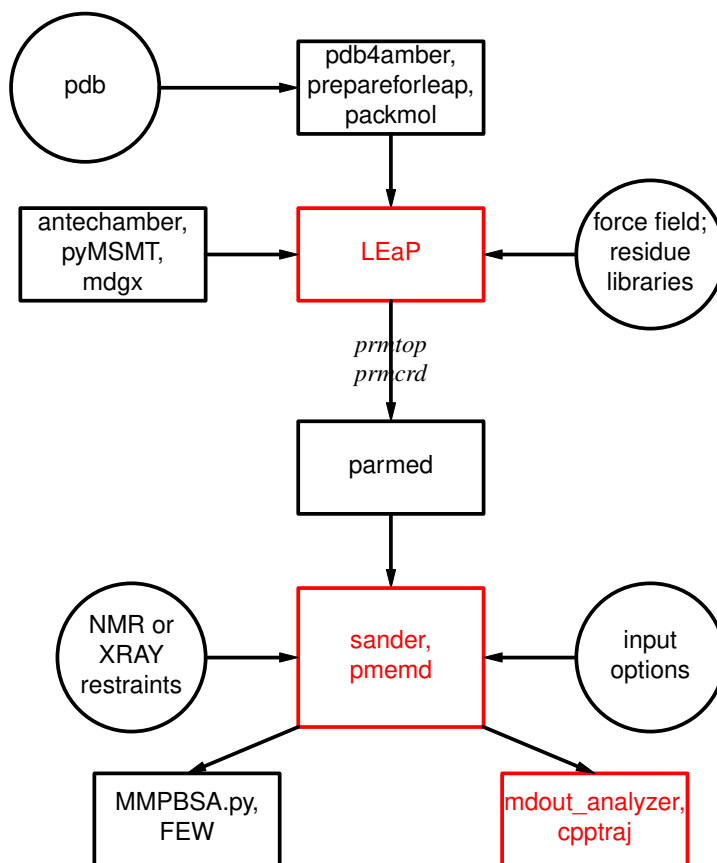


Figure 1.1.: Basic information flow in Amber. Rectangles represent programs and circles represent input files. Programs in red are the most important.

3. Force field: Parameters for all of the bonds, angles, dihedrals, and atom types in the system. The standard parameters for several force fields are found in the `$AMBERHOME/dat/leap/parm` directory; see Chapter 3 for more information. These files may be used “as is” for proteins and nucleic acids, or users may prepare their own files that contain modifications to the standard force fields.
4. Once the topology and coordinate files (often called *prmtop* and *prmcrd*, but any legal file names can be used) are created (usually by *tleap*), the *parmed* script can be used to examine and verify these, and to make modifications. In particular, the *checkValidity* action will flag many potential problems.
5. Input options: The user specifies the procedural options and state parameters desired. These are specified in input files, named *mdin* by default.
6. Other restraints, often based on experimental data, such as from NMR or Xray; other sources of restraints may also be used.

1.1.1. Preparatory programs

LEaP is the primary program to create a new system in Amber, or to modify existing systems. It is available as the command-line program *tleap* or the GUI *xleap*.

pdb4amber generally helps in preparing pdb-format files coming from other places (such as *rcsb.org*) to be compatible with LEaP.

prepareforleap is not a program, but an action inside *cpptraj*, that also helps make pdb-format files compatible with LEaP. It is particularly useful for carbohydrates.

parmed provides a simple way to extract information about the parameters defined in a parameter-topology file. It can also be used to check that the parameter-topology file is valid for complex systems (see the *checkValidity* command), and it can also make simple modifications to this file.

antechamber is the main program to develop force fields for small organic molecules (e.g., drugs, modified amino acids) using a version of the general Amber force field (GAFF). These can be used directly in LEaP, or can serve as a starting point for further parameter development.

pyMSMT provides a means to build, prototype, and validate molecular mechanical (MM) models of metallo-proteins and organometallic compounds. It uses the bonded plus electrostatics model to expand existing pairwise additive force fields. It is a reimplementation of MCPB in Python, with a more efficient workflow and many modeling processes from previous versions incorporated automatically.

mdgx allows the generation of bonded force field parameters for any molecule by fitting to quantum data.

packmol-memgen provides a simple way to generate membrane systems, with or without protein, by orienting input proteins with Memembed and using Packmol as the packing engine. It can handle complex lipid mixtures, as well as multi-bilayer systems. The output is compatible with Amber through charmm-lipid2amber.py.

1.1.2. Simulation programs

sander (part of AmberTools) is the basic energy minimizer and molecular dynamics program. This program relaxes the structure by iteratively moving the atoms down the energy gradient until a sufficiently low average gradient is obtained. The molecular dynamics portion generates configurations of the system by integrating Newtonian equations of motion. MD will sample more configurational space than minimization, and will allow the structure to cross over small potential energy barriers. Configurations may be saved at regular intervals during the simulation for later analysis, and basic free energy calculations using thermodynamic integration may be performed. More elaborate conformational searching and modeling MD studies can also be carried out using the *sander* module. This allows a variety of constraints to be added to the basic force field, and has been designed especially for the types of calculations involved in NMR, Xray or cryo-EM structure refinement.

pmemd (part of Amber) is a version of *sander* that is optimized for speed and for parallel scaling; the *pmemd.cuda* variant runs on GPUs. The name stands for “Particle Mesh Ewald Molecular Dynamics,” but this code can now also carry out generalized Born simulations. The input and output have only a few changes from *sander*.

gem.pmemd (part of AmberTools) is a (CPU-only) variant of the pmemd program that is designed for calculations using “advanced” force fields, such as AMOEBA[18] and GEM.[19]

1.1.3. Analysis programs

mdout_analyzer.py is a simple-to-run Python script that will provide summaries of information that is in the output files from *sander* or *pmemd*.

cpptraj is the main trajectory analysis utility (written in C++) for carrying out superpositions, extractions of coordinates, calculation of bond/angle/dihedral values, atomic positional fluctuations, correlation functions, analysis of hydrogen bonds, etc. See Chap. 37 for more information.

pytraj is a Python wrapper for cpptraj. It introduces additional flexibility into data analysis by combining with Python’s rich ecosystems (such as numpy, scipy, and ipython-notebook).

1. Introduction

MMPBSA.py is a Python script that automates energy analysis of snapshots from a molecular dynamics simulation using ideas generated from continuum solvent models. (There is also an older perl script, called *mm_pbsa.pl*, that has similar functionality.)

FEW (Free energy workflow) automates free energy calculations of protein-ligand binding using TI, MM/PBSA-type, or LIE calculations.

edgembarr Alchemical free energy simulation analysis. See Chap. [42](#) for more information.

ndfes Umbrella sampling free energy analysis See Chap. [45](#) for more information.

2. Installation

2.1. Basic installation guide

This chapter gives an overview of how to install and test your distribution. Note that the procedure is different from earlier versions of Amber, relying on *CMake* rather than *make*. Once you have downloaded the distribution files, do the following:

1. First, **extract the files** in some location (we use `/home/xxxx` as an example here, but you can install anywhere that you have write permissions):

```
cd /home/xxxx
tar xvfj ambertools25.tar.bz2 # (Extracts into an ambertools25.src directory.)
# OR: #
tar xvfj pmemd24.tar.bz2 # (Extracts into pmemd24.src)
```

Note: This procedure has changed from earlier releases. In the past, AmberTools and Amber resided in the same folder, and were compiled together. Starting in 2025, Amber and AmberTools are separate releases, residing in different folders. Each can be compiled on its own, without the other. One improvement is that compilation of just the *pmemd* program (say on a high-performance computer system) is now much simpler in the past. A second advantage for many users will be this: AmberTools is available as a pre-compiled conda package (see ambermd.org/GetAmber.php) which can be installed via “conda install”. Users who choose this option only need to download and compile `pmemd24.tar.bz2`.

2. Next, you may need to **install some compilers and other libraries**. Details depend on what OS you have, and what is already installed. Package managers can greatly simplify this task. For lists of requirements for Windows, macOS and for many variants of Linux, please visit ambermd.org/Installation.php. In particular, you will need to have *cmake* in your PATH. A restriction is that you cannot use the *cmake* you obtain from a conda distribution you may have; you will need to use a package manager, or download it from <https://cmake.org/>. If you have an existing miniconda distribution, please remove it from your PATH while building Amber.
3. **Apply patches** to what is in the distribution tarfile. Do the following:

For AmberTools:

```
cd ambertools25_src
./update_amber --help # to see an overview of options
./update_amber --update # to apply any available updates
```

For Amber:

```
cd pmemd24_src
./update_pmemd --help # to see an overview of options
./update_pmemd --update # to apply any available updates
```

4. **Building with cmake:** The Amber development team has moved our build system to *cmake*, with the conversion being spearheaded by Jamie Smith.

The basic rationale for the move, and instructions on using *cmake* to build Amber, are at

- ambermd.org/pmwiki/index.php/Main/CMake-Quick-Start
- ambermd.org/pmwiki/pmwiki.php/Main/CMake-Common-Options
- Section 2.2, below.

2. Installation

For most users, the options chosen in the sample script (below) should be OK. Note that with `cmake`, the “source” directory (where you extracted the files,) must be different from the installation directory. Thus, make sure that `-DCMAKE_INSTALL_PREFIX` is not set to `amber24_src` in the `run_cmake` script.

```
cd ambertools25_src/build
# optional: edit the run_cmake script to make any needed changes;
./run_cmake
```

Next, build and install the code:

```
make install
```

5. The installation step will create a resource file `amber.sh` in your installation directory. This file will set up your shell environment correctly for Amber when it is sourced:

```
source /home/xxxx/ambertools25/amber.sh # for bash, zsh, ksh, etc.
or
source /home/xxxx/pmemd24/amber.sh
```

Note that the resource file must be sourced, not executed. Adding these commands to your login resource file (e.g., `~/.bashrc`, `~/.zshrc`) will set up your environment every time you start a new shell. In particular, it sets the `AMBERHOME` or `PMEMDHOME` environment variable, which is needed for a number of workflows involving Amber. [There is a similar script, `amber.csh`, for those who use a C type shell interactively.]

6. This should be followed by a **testing phase**. If you have `-DINSTALL_TESTS=TRUE` in your `cmake` invocation, then you can do the following:

```
cd $AMBERHOME # (this was set in step 5, above)
make test.serial
```

which will run tests and will report successes or failures.

If “possible FAILURE” messages are found, go to the subdirectories of `$AMBERHOME/AmberTools/test` or `$AMBERHOME/test`, and look at the “*.dif” files. Differences should involve round-off in the final digit printed, or occasional messages that differ from machine to machine (see below for details). As with compilation, if you have trouble with individual tests, you may wish to comment out certain lines in the Makefiles (i.e., `$AMBERHOME/AmberTools/test/Makefile` or `$AMBERHOME/test/Makefile`), and/or go directly to the test subdirectories to examine the inputs and outputs in detail. For convenience, all of the failure messages and differences are collected in the `$AMBERHOME/logs` directory; you can quickly see from these if there is anything more than round-off errors.

The nature of molecular dynamics is such that the course of the calculation is very dependent on the order of arithmetical operations and the machine arithmetic implementation, *i.e.*, the method used for round-off. Because each step of the calculation depends on the results of the previous step, the slightest difference will eventually lead to a divergence in trajectories. As an initially identical dynamics run progresses on two different machines, the trajectories will eventually become completely uncorrelated. Neither of them are “wrong;” they are just exploring different regions of phase space. Hence, states at the end of long simulations are not very useful for verifying correctness. Averages are meaningful, provided that normal statistical fluctuations are taken into account. “Different machines” in this context means any difference in floating point hardware, word size, or rounding modes, as well as any differences in compilers or libraries. Differences in the order of arithmetic operations will affect round-off behavior; $(a + b) + c$ is not necessarily the same as $a + (b + c)$. Different optimization levels will affect operation order, and may therefore affect the course of the calculations.

All initial values reported as integers should be identical. The energies and temperatures on the first cycle should be identical. The RMS and MAX gradients reported in `sander` are often more precision sensitive than the energies, and may vary by 1 in the last figure on some machines. In minimization and dynamics calculations, it is not unusual to see small divergences in behavior after as little as 100-200 cycles.

7. If you are new to Amber, you should look at the **tutorials** (available at <https://ambermd.org/tutorials>) and **this manual** in order to become familiar with the *Amber* features and functionalities.

8. In order to compile and test the parallel (MPI) version of *Amber*, follow these steps (after successfully installing the serial version).

- a) You must first ensure that you have installed MPI and that *mpicc* and *mpif90* are in your PATH. Some MPI installations are tuned to particular hardware (such as InfiniBand), and you should use those versions if you have such hardware. Most people can use standard versions of either *mpich* or *openmpi* obtained from a package manager. See <https://ambermd.org/Installation.php> for information about your particular operating system. See also the information about MPI in 2.2 below.
- b) Then do the following:

```
cd /home/xxxx/ambertools25_src/build
# edit the run.cmake script to set -DMPI=TRUE
./run_cmake
make install
export DO_PARALLEL="mpirun -np 2"
cd /home/xxxx/ambertools25      # or wherever you specified for the INSTALL_PREFIX
source amber.sh
make test.parallel
# Note, some tests, like the replica exchange tests, require more
# than 2 threads, so we suggest that you test with either 4 or 8
# threads as well
export DO_PARALLEL="mpirun -np 4"
make test.parallel
```

9. If you have licensed Amber24, repeat steps 4-8 above, changing “*ambertools25*” to “*pmemd24*”. Note that running steps 4-8, and in particular sourcing the *amber.sh* file, will create an AMBERHOME environment variable (and add \$AMBERHOME/bin to your PATH) when you do this from the *ambertools25_src* folder; it will create a PMEMDHOME variable (add add \$PMEMDHOME/bin to your PATH) when done from the *pmemd24_src* folder.

2.2. The cmake build system in Amber

This section will walk you through performing certain common tasks with the CMake build system. Note: this is fairly advanced information; for a more gentle introduction, please visit these pages:

- [CMake Quick Start Guide](#)
- [CMake Common Options](#)

2.2.1. Using MPI and OpenMP

MPI and OpenMP provide different methods of parallelizing Amber -- MPI at the process level, and OpenMP at the thread level. MPI takes the form of one or more libraries that Amber needs to link with, while OpenMP requires compiler support and is activated by a specific compiler flag. If you are working in a high-performance computing environment, then there will usually be a specific system MPI installation compatible with your hardware that you are supposed to use. Make sure to find out what that is and where it's installed before going any further.

You can enable MPI in the CMake build system by passing the `-DMPI=TRUE` flag. This will enable use of MPI in all programs that support it. For each of these programs, the standard (serial) version will still be built, and an additional version with MPI support, usually identified by the “.MPI” suffix appended to the name, will be compiled.

Traditionally, MPI is integrated into programs' build systems by telling them to use special "compiler wrappers" that automatically apply the needed flags and libraries for MPI before calling the real compiler. However, Amber does not use these, since it would make it impossible to compile executables without MPI support. Instead, Amber makes use of CMake's FindMPI module, which extracts the compiler flags from the MPI wrappers and lets CMake use them only where needed. By default, FindMPI will search for MPI compiler wrappers (e.g.

2. Installation

mpicc, mpicxx, or mpif95) in your PATH and use the settings from the first one it finds. If you want to select a different MPI implementation, you can define (-D) the variables `MPI_C_COMPILER`, `MPI_CXX_COMPILER`, and `MPI_Fortran_COMPILER` to point to the MPI wrappers for their respective languages. Or, with CMake >= 3.9 installed, you can define `MPIEXEC_EXECUTABLE` to point to the location of a mpiexec executable, and CMake will attempt to find the MPI that is installed in the same directory as it. For even more information, Refer to [Cmake's FindMPI docs](#).

OpenMP can be enabled using the `-DOPENMP=TRUE`, and thankfully the process for configuring it is not as convoluted. CMake is aware of the needed OpenMP flags for all supported compilers and will automatically find one that works. If none is available, an error will be printed. Similarly to MPI, once OpenMP is enabled an alternate version of all supported programs will be made that has a ".OMP" suffix.

2.2.2. Using CUDA

CUDA is Nvidia's software development kit for creating custom applications that run on Nvidia GPUs. Amber primarily uses CUDA in *pmemd.cuda*, but it's also used to accelerate several other applications in Amber-Tools, such as *cpptraj*, *mdgx*, *pbsa*, and *QUICK*. You can enable CUDA in the CMake build system using `-DCUDA=TRUE`. This will build CUDA versions of all applications that support it. MPI CUDA versions will also be built if MPI is enabled. More details, especially for *pmemd*, are available in Section 23.6.4.

Currently Amber supports CUDA versions from 7.5 to 12.6 inclusive. However, older versions are less well tested and more likely to cause issues, and you may also run into trouble with the CUDA SDK being incompatible with newer compilers on your machine. So, it's better to use one of the newer CUDA versions if possible. Note that the compilation of complex CUDA code such as Amber's is extremely CPU and memory intensive, so CUDA builds are much slower than those of other languages.

By default, CMake will search for the CUDA compiler executable (*nvcc*) on your PATH and use the CUDA installation associated with it. To specify a certain install location, define the `CUDA_TOOLKIT_ROOT_DIR` variable, e.g. `-DCUDA_TOOLKIT_ROOT_DIR=/usr/local/cuda-11.0`. The Amber build system uses CMake's legacy FindCUDA module and will continue to for the foreseeable future. So, information related to CUDA that is for newer versions of CMake may not be accurate. Instead, refer to the [FindCUDA docs](#) for information.

Amber supports use of the Nvidia NCCL library for communications between multiple GPUs, which provides a performance improvement over plain MPI. If the library is enabled (using `-DNCCL=TRUE`), then it will be activated when *pmemd.MPI.cuda* is run on 3 or more GPUs. This is not of relevance for *QUICK* as in this case communication time is negligible.

2.2.3. Using HIP and AMD GPUs

Amber24 has support for execution on AMD GPUs using **ROCm** and **HIP** (Heterogeneous-Compute Interface for Portability). **ROCm** is an Advanced Micro Devices software stack for GPU programming. **HIP** is a C++ Runtime API that allows developers to create portable applications for AMD and NVIDIA GPUs. Here are some installation notes:

- **ROCm** support in Amber is available for *pmemd*, *cpptraj*, *mdgx* and *pbsa*. It can be enabled in the CMake build system using `-DHIP=ON`. For parallel runs with MPI, GPU-Aware MPI librarys (e.g., OpenMPI with UCX and ROCM support) must be installed on your system. Amber24 supports ROCm version 6.0 and later.
- CMake will use the HIP compiler executable (*hipcc*) on your PATH and use the ROCm installation associated with it. To specify a certain install location, define the `HIP_TOOLKIT_ROOT_DIR` variable, e.g. `-DHIP_TOOLKIT_ROOT_DIR = /opt/rocm/`.
- Amber24 supports execution on different AMD GPU architectures including MI100, MI210, MI250, and MI300. It also supports execution on RDNA2 based GPUs (e.g., RX 6900 XT). For Amber execution on RDNA2 based GPUs, you need to add `-DHIP_WARP_64=OFF` to the cmake command.
- There is an *compile_with_hip.sh* file in the `pmemd24_src` folder that provides the *cmake* commands and more detailed instructions for building Amber for AMD GPUs.

- To run the test suite for AMD GPUs, go to the `amber24/test` folder, and type `'make test.hip.serial'` or `'make test.hip.parallel'`. There are DPFP (default) and SPFP versions of these tests. As with CUDA installations, one should definitely make sure the serial code is working before trying parallel runs. As noted above, special version of MPI libraries are required for GPU-parallel execution.

Although we have done extensive tests on some AMD platforms, users should remember that this code is about a decade newer than the corresponding CUDA codes, and have not undergone as much “real-world” testing. Be sure to check short runs against CPU or CUDA codes, and be on the lookout for problems that other users might not yet have experienced.

2.2.4. Controlling External Libraries

Amber can use, for one purpose or another, a great variety of third-party libraries. Some, such as NetCDF, FFTW, and Boost, are core components of many programs and as such must be enabled for the build to succeed. Others are only optional and Amber can work just fine without them. The complete description of what these libraries do and how to use them is too complex for here and is left to the relevant sections of the manual. Instead, this section will focus on the build system’s tools for managing them.

After the configuration finishes, the build system will print a build report showing all libraries used. Here’s an example from my system:

```
--
--                               3rd Party Libraries
-- ---building bundled: -----
-- ucpp - used as a preprocessor for the NAB compiler
-- netcdf-fortran - for creating trajectory data files from Fortran
-- pnetcdf - used by cpptraj for parallel trajectory output
-- readline - used for the console functionality of cpptraj
-- xblas - used for high-precision linear algebra calculations
-- mpi4py - MPI support library for MMPBSA.py
-- ---using installed: -----
-- arpack - for fundamental linear algebra calculations
-- netcdf - for creating trajectory data files
-- fftw - used to do Fourier transforms very quickly
-- apbs - used by Sander as an alternate Poisson-Boltzmann equation solver
-- zlib - for various compression and decompression tasks
-- libbz2 - for bzip2 compression in cpptraj
-- plumed - used as an alternate MD backend for Sander
-- libm - for fundamental math routines if they are not contained in the C library
-- mkl - alternate implementation of lapack and blas that is tuned for speed
-- perlmol - chemistry library used by FEW
-- boost - C++ support library
-- nccl - NVIDIA parallel GPU communication library
-- mbx - computes energies and forces for pmemd with the MB-pol model
-- ---disabled: -----
-- blas - for fundamental linear algebra calculations
-- lapack - for fundamental linear algebra calculations
-- c9x-complex - used as a support library on systems that do not have C99 complex.h support
-- lio - used by Sander to run certain QM routines on the GPU
-- pupil - used by Sander as an alternate user interface
```

There are a lot of important details in this report. The “canonical” name of each library is listed, along with its description. You’ll also notice that each library is listed as either “bundled”, “installed”, or “disabled”. This indicates where the build system found each library.

With some exceptions, Amber will automatically find and use libraries it finds on the system, marking them as installed. You’ll see output from these detections earlier in the build, with a message explaining why it couldn’t find each library that is missing and what info it needs to locate it. If you don’t need the library active you can ignore these messages, but otherwise you can use that information to determine what variables to define. For example, if you saw this output:

2. Installation

```
-- Could NOT find PnetCDF_C (missing: PnetCDF_C_LIBRARY PnetCDF_C_INCLUDE_DIR)
```

you could help CMake find the library with the following command:

```
cmake <path to source> -DPnetCDF_C_LIBRARY=<path to libpnetcdf.so> \  
-DPnetCDF_C_INCLUDE_DIR=<path to folder containing pnetcdf.h>
```

To find libraries when the paths aren't specified directly, CMake uses a specific search path which generally contains all the system directories. But what if you have certain libraries installed to a nonstandard directory? The easiest way to help CMake find those libraries is by defining the variable `CMAKE_PREFIX_PATH`. This can be set to one path or a semicolon-separated list, and each of these paths will be searched like a standard Unix prefix: `<path>/bin` for programs, `<path>/lib` for libraries, and `<path>/include` for headers. If you've used Autoconf build systems before this is similar to the `--prefix` option, though it does not control the install directory.

For many libraries which are required and are not commonly found on people's systems, Amber provides bundled versions to make users' lives easier. These bundled versions are automatically compiled and installed along with Amber, and should work seamlessly. They also are guaranteed to get built with the same environment and settings as Amber, removing a common source of problems. However, they do increase the binary size and can cause conflicts with libraries already installed on the system, so especially if you are packaging Amber, you may wish to use the external versions.

In the past, the Amber developers have had trouble with user issues related to broken installations of certain libraries on certain common OSs. To combat this, the decision was made to prevent Amber from linking to certain libraries by default unless specifically told to. As of Amber 24, these libraries are *boost*, and *mkl*. To disable this behavior and use all found libraries, you can use the option `-DTRUST_SYSTEM_LIBS=TRUE`.

Sometimes, even more fine-grained control over 3rd party libraries is needed, such as if a specific 3rd party library is found but fails to link and you want to disable it. For this purpose, three override options are provided: `FORCE_DISABLE_LIBS`, `FORCE_INTERNAL_LIBS`, and `FORCE_EXTERNAL_LIBS`. These accept semicolon-separated lists of library names. `FORCE_DISABLE_LIBS` will force Amber to build without a given library, and will print an error if that library is required. `FORCE_INTERNAL_LIBS` will tell Amber to prefer the internal version of a bundled library. Finally, `FORCE_EXTERNAL_LIBS` will tell Amber to prefer the version of a library that is installed on the system.

One last thing: keep in mind that these variables are lists and the entire list is set at once. Suppose you had previously disabled MKL because of a link error, using `-DFORCE_DISABLE_LIBS=mkl`. Then, a build error occurs with `mpi4py` and you want to disable that too. It's fine to run CMake again without passing the `FORCE_DISABLE_LIBS` option, but when you change it you need to pass the full new value so the `mkl` entry isn't erased. So, the argument to use would be `-DFORCE_DISABLE_LIBS=mkl;mpi4py`.

2.2.5. Selecting BLAS and MKL

Almost all Amber programs require access to the BLAS (Basic Linear Algebra Subprograms) and LAPACK (Linear Algebra PACKage) libraries for computing various matrix operations. By default, Amber uses the venerable Netlib implementations of these libraries, which are widely compatible, but are not the best optimized. Over time, several optimized versions of BLAS and LAPACK have been produced, which can offer performance increases of 50%-1000% on large matrix operations. If you are building Amber for a high performance computing environment, it is highly recommended to make use of an optimized BLAS implementation. Popular options include OpenBLAS, which is free and supports a wide variety of platforms, and MKL, which is more extensive and may provide better performance on Intel chips.

Non-MKL BLAS implementations are handled using CMake's `FindBLAS` and `FindLAPACK` modules. These know about and search for a variety of BLAS and LAPACK implementations, including Netlib, OpenBLAS, and Macs' Accelerate framework. To force them to search for these specific versions of BLAS and LAPACK, you can set the `BLA_VENDOR` variable to "Generic", "OpenBLAS", or "Apple" respectively. The full list is documented here. If your BLAS is installed to a nonstandard location, you may need to add it to the CMake search path using the methods in the previous section.

MKL, however, is a special case. It is a very complicated library that is difficult to link properly on all systems, so it is not found by default to reduce the chance of errors. To enable it, either pass `-DTRUST_SYSTEM_LIBS=TRUE` or `-DFORCE_EXTERNAL_LIBS=mkl` (see above). Amber will then search for MKL in its default install location, such as `/opt/intel/mkl` on Linux. The environment variables `MKL_HOME` and `MKLROOT` will also be checked if they are defined. If MKL is installed to a different location, or if you need to select a specific version, define the `MKL_HOME` CMake variable to point to MKL's install directory. MKL can be used in two modes: threaded or serial. Threaded mode provides the option for MKL to split calculations across multiple threads internally (exactly how it does this is configured using environment variables). By default Amber will attempt to link MKL in threaded mode, but if this causes problems (it requires that your compiler have an OpenMP implementation supported by MKL) then you can use `-DMKL_MULTITHREADED=FALSE` to turn this off. Also, if you want Amber to use the MKL static libraries, you can pass the `-DMKL_STATIC=TRUE` option. Unfortunately, due to how CMake find modules work, this option only takes effect the first time CMake is run.

2.2.6. Configuring Python

A substantial amount of Amber programs either are written in or provide interfaces to Python. Unfortunately, Python installations tend to vary wildly across different systems, and Python programs are very prone to issues with dependencies on native libraries as well as other Python libraries. So, Amber supports three different Python configurations for different systems and setups.

1. The first option, and the one that is used by default, is to let Amber control the Python distribution entirely. Amber will download a self-contained Miniforge python interpreter when CMake is run for the first time and will manage it entirely itself. Once Amber is installed, you can access Amber's miniconda via the *amber.python* symlink in the install directory. Using miniconda will eliminate the chance of a conflict between Amber's binaries and dependencies and your system Python interpreter.
2. Your other option is to just use your existing system Python interpreter. Set `DOWNLOAD_MINICONDA` to `FALSE`, and let CMake find your Python interpreter on the `PATH`. To select a different interpreter, set the `PYTHON_EXECUTABLE` variable to point to it. Amber requires certain Python packages be installed: currently *numpy*, *scipy*, *matplotlib*, *cython*, *setuptools*, and *tkinter*. You can install these through your distro's package manager or through *pip*. If you don't have root access, the `pip install --user` command is your friend since it will install to your home directory instead of the system dirs. Amber's CMake build system has good support for working with your system Python, and it should work fine on most systems. However, there can still be issues, so we recommend switching to Miniconda if the system installation is not working for you.

2.2.7. Configuring Amber Settings

Here are a few other commonly used Amber build options. Ever had an Amber tool that you didn't care about fail to build, and you just wish you could make it disappear? Well now you can, with `DISABLE_TOOLS`! Just pass it a semicolon-separated list of tools (folder names under `AmberTools/src/` or `src/`) to this option, and it will prevent them from building. A note will be added at the bottom of the build report saying which tools you've disabled. It also tracks dependencies between tools, so disabling something that other things depend on will properly disable the dependers instead of causing build errors.

Another useful option is the `STATIC` flag. This will cause all Amber executables and libraries to be linked statically. This means that they don't depend on any other libraries from Amber and can be moved anywhere or to any other machine (as long as the same system libraries are present). It also may provide a performance boost to some programs by removing the overhead of resolving symbols in shared libraries, though this has not been measured.

Finally, Amber has two different ways of running tests, controlled by the `INSTALL_TESTS` option. With `INSTALL_TESTS` enabled, all Amber and AmberTools tests are installed to the install prefix, and can be run with the standard commands using the Makefile there. This makes the installation totally independent of the source dir, which is convenient for packaging or distributing Amber. However, there are some downsides: the tests are quite large, taking up a gigabyte or more of space. Copying them from the source folder will eat up even more of your disk and make the install process take quite a bit longer. If you're planning on keeping the source directory

2. Installation

around then it might make more sense to leave `INSTALL_TESTS` disabled. In this configuration, the tests will not be installed and you must run them out of the source directory after sourcing `amber.sh`.

Several other common tasks are covered with more in-depth guides:

- [Cross-compiling Amber](#)
- [Creating packages](#) (includes Linux deb/rpm packages, OS X DMG packages, and Windows installers)

2.2.8. Selecting Special Features

As of Amber 24, significantly improved performance of `pmemd.cuda.MPI` is available through the MVAPICH MPI library's MVAPICH2-GDR GPU to GPU direct communication facility[20]. The improvement is 84% for the explicit solvent subset of the Amber benchmark suite. Users must manually activate this feature: Edit `run_cmake` and add `-DMVAPICH2GDR_GPU_DIRECT_COMM=TRUE` to the Linux section. And you must use MVAPICH2-GDR version 2.3.7 or later as your MPI. If you employ this feature then, in addition to citing Amber, please also cite reference [20] and note whether this capability enabled larger simulations.

2.2.9. Selecting Compilers

Most users will not need to explicitly select their compilers because the `run_cmake` script automatically selects either CLANG for macOS or GNU for Linux. Advanced users can select alternate compilers. The complete list of compilers is CLANG, CRAY, GNU, INTEL, INTELLVM, ONEAPI, and PGI. New in Amber 24 is support for the Intel oneAPI compilers specified via INTELLVM or the alias ONEAPI.

The option `INTEL` refers to the Intel classic compilers (*icc*, *icpc* and *ifort*) which have been deprecated by Intel; these are no longer available through the normal Intel release mechanism, although old versions can probably be obtained. The new Intel oneAPI compilers (*icx*, *icpx* and *ifx*) are definitely for experienced users. It is quite rare for Amber codes to exhibit any significant performance advantage compared to GNU compilers. For a list of known issues and general instructions on building Amber with oneAPI compilers please visit ambermd.org/InstIntelOneAPI.php.

2.2.10. Debugging the Build

Last but not least, there are several options that are very useful when things go haywire in the build.

You'll notice pretty quickly when building that CMake chooses to omit the full compiler command in favor of a pretty-looking filename and progress percentage only. This is nice most of the time, but can be a problem if a compile command is failing and you aren't sure why. Luckily, CMake has a handy option for these situations: `CMAKE_VERBOSE_MAKEFILE`. Setting it to `TRUE` will cause it to print out the full compiler command for each file. As a shortcut, if you are using Makefiles, then you can run `make VERBOSE=1` to trigger the same behavior without rerunning CMake.

But what if you're sure that Amber is being compiled correctly, but it's having trouble linking to an external library? This is where `-DPRINT_PACKAGING_REPORT=TRUE` can help. This will cause Amber to print a detailed list of all the libraries that it is linking to on your system and where they are located. It's mainly meant to help analyze dependencies for packaging, but it's also convenient as a general purpose debugging tool in case Amber is linking to something it shouldn't be.

2.3. Applying Updates

This section describes the main updating script responsible for managing updates. We suggest that you at least skim the first section on the basic usage—particularly the note about the `--version` flag for if/when you ask for help on the mailing list.

2.3.1. Basic Usage

Updates to AmberTools and Amber are downloaded, applied, and managed automatically using the Python script *update_amber*. This script works on every version of Python from Python 2.4 through the latest Python 3 release. To use this command manually, you must refer to the “source” directory, i.e. the folder headed by “amber24_src” where you downloaded the codes. Here, we are going to assume that you have set your AMBERSOURCE environment variable to this directory, say by typing the command:

```
export AMBERSOURCE=/path/to/ambertools25_src
```

Now there are three basic update-related commands:

- *\$AMBERSOURCE/update_amber --check-updates*: This option will query the Amber website for any updates that have been posted that have not been applied to your installation. If you think you have found a bug, this is helpful to try first before emailing with problems since your bug may have already been fixed.
- *\$AMBERSOURCE/update_amber --version*: This option will return which patches have been applied to the current tree so far. When emailing the Amber list with problems, it is important to have the output of this command, since that lets us know exactly which updates have been applied.
- *\$AMBERSOURCE/update_amber --update*: This option will go to the Amber website, download all updates that have not been applied to your installation, and apply them to the source code. **Note that you will have to recompile any affected code for the changes to take effect!** To do this, go to your build directory and re-run the *cmake* command you used in Step 3 of Section 2.1.

2.3.2. Advanced options

update_amber has additional functionality as well that allows more intimate control over the patching process. For a full list of options, use the *--full-help* command-line option. These are considered advanced options.

- *\$AMBERSOURCE/update_amber --download-patches*: Only download patches, do not apply them
- *\$AMBERSOURCE/update_amber --apply-patch=<PATCH>*: This will apply a third-party patch
- *\$AMBERSOURCE/update_amber --reverse-patch=<PATCH>*: Reverses a third-party patch file that was applied via the *--apply-patch* option (see above).
- *\$AMBERSOURCE/update_amber --show-applied-patches*: Shows details about each patch that has been applied (including third-party patches)
- *\$AMBERSOURCE/update_amber --show-unapplied-patches*: Shows details about each patch that has been downloaded but not yet applied.
- *\$AMBERSOURCE/update_amber --remove-unapplied*: Deletes all patches that have been downloaded but not applied. This will force *update_amber* to download a fresh copy of that patch.
- *\$AMBERSOURCE/update_amber --update-to AmberTools/# , Amber/#*: This command will apply all patches necessary to bring AmberTools up to a specific version and Amber up to a specific version. Note, no updates will ever be reversed using this command. You may specify only an AmberTools version or an Amber version (or both, comma-delimited). No patches are applied to an omitted branch.
- *\$AMBERSOURCE/update_amber --revert-to AmberTools/# , Amber/#*: This command does the same as *--update-to* described above, except it will only reverse patches, never apply them.

update_amber will also provide varying amounts of information about each patch based on the verbosity setting. The verbose level can be set with the *--verbose* flag and can be any integer between 0 and 4, inclusive. The default verbosity level changes based on how many updates must be described. If only a small number of updates

2. Installation

need be described, all details are printed out. The more updates that must be described, the less information is printed. If you manually set a value on the command-line, it will override the default. These values are described below (each level prints all information from the levels before plus additional information):

- 0: Print out only the name of the update file (no other information)
- 1: Also prints out the name of the program(s) that are affected
- 2: Also prints out the description of the update written by the author of that update.
- 3: Also prints the name of the person that authored the patch and the date it was created.
- 4: Also prints out the name of every file that is modified by the patch.

2.3.3. Internet Connection Settings

If `update_amber` ever needs to connect to the internet, it will check to see if <https://ambermd.org> can be contacted within 10 seconds. If not, it will report an error and quit. If your connection speed is particularly slow, you can lengthen this timeout via the `--timeout` command-line flag (where the time is given in seconds).

Proxies By default, `update_amber` will attempt to contact the internet through the same mechanism as programs like `wget` and `curl`. For users that connect to the internet through a proxy server, you can either set the `http_proxy` environment variable yourself (in which case you can ignore the rest of the advice about proxies here), or you can configure `update_amber` to connect to the internet through a proxy. To set up `update_amber` to connect to the internet through a proxy, use the following command:

```
$AMBERSOURCE/update_amber --proxy=<PROXY ADDRESS>
```

You can often find your proxy address from your IT department or the preferences in your favorite (configured) web browser that you use to surf the web. If your proxy is authenticated, you will also need to set up a user:

```
$AMBERSOURCE/update_amber --proxy-user=<USERNAME>
```

If you have set up a user name to connect to your proxy, then you will be asked for your proxy password the first time `update_amber` attempts to utilize an online resource. (For security, your password is never stored, and will need to be retyped every time `update_amber` runs).

You can clear all proxy information using the `--delete-proxy` command-line flag—this is really only necessary if you no longer need to connect through any proxy, since each time you configure a particular proxy user or server it overwrites whatever was set before.

Mirrors If you would like to download Amber patches from another website or even a folder on a local filesystem, you can use the `--amber-updates` and `--ambertools-updates` command-line flags to specify a particular web address (must start with `http://`) or a local folder (use an absolute path). You can use the `--reset-remotes` command-line flag to erase these settings and return to the default Amber locations on <https://ambermd.org>.

If you set up online mirrors and never plan on connecting directly to <http://ambermd.org>, you can change the web address that `update_amber` attempts to connect to when it verifies an internet connection using the `--internet-check` command-line option.

2.4. Contacting the developers

Please send suggestions and questions to amber@ambermd.org. You need to be subscribed to post there; to subscribe, go to <http://lists.ambermd.org/mailman/listinfo/amber>. You can unsubscribe from this mailing list on the same site.

Part II.

Amber force fields

3. Molecular mechanics force fields

Amber is designed to work with several simple types of force fields, although it is most commonly used with parametrizations developed by Peter Kollman and his co-workers and academic “descendants”. The traditional parametrization uses fixed partial charges, centered on atoms. Less commonly used modifications add polarizable dipoles to atoms, so that the charge description depends upon the environment; such potentials are called “polarizable” or “non-additive”. An alternative is to use force fields originally developed for the CHARMM or Tinker (AMOEBA) codes; these require a different setup procedure, which is described in Sections 15.2.2.8 (for CHARMM) and Chapter 33 (for AMOEBA). Chapter 15 provides a basic introduction to force fields, along with details of how the parameters are encoded in Amber files.

There are many force fields that could be used, and users can specify the force fields they wish to use. Depending on what components are in your system, you may need to specify:

- a protein force field (recommended choice is *ff19SB*)
- a DNA force field (recommended choice is *OL24*)
- an RNA force field (recommended choice is *OL3*)
- a carbohydrate force field (recommended choice is *GLYCAM.06j*)
- a lipid force field (recommended choice is *lipid21*)
- a water model with associated atomic ions (more variable); popular choices are *spc/e*, *tip4pew*, and *OPC*. Not needed if you are using an implicit solvent model.
- a general force field, for organic molecules like ligands (recommended choice is *gaff2*)
- other components (such as modified amino acids or nucleotides, other ions), as needed

Notes:

1. You have to be careful if you try to adopt a “mix and match” strategy for different components. The recommended choices are designed to work well together, and have been fairly extensively tested. Use of other combinations requires a deeper knowledge of the nature and origin of force fields; see below and consult the original papers for more information. If you wish to combine proteins with nucleic acids, only the recommended combination above (or one where *leaprc.DNA.OL24* is replaced with *leaprc.DNA.OL21*) is allowed.
2. In general, your input file to LEaP will begin with several commands to source the relevant standard leaprc files. The standard leaprc files are in the *\$AMBERHOME/dat/leap/cmd* directory and are accessible to LEaP by default. For example the following preamble would allow you to include proteins, DNA, lipids, general components, water, and atomic ions like Na⁺ or Cl⁻, using the current recommended force fields:

```
source leaprc.protein.ff19SB
source leaprc.DNA.OL24
source leaprc.lipid21
source leaprc.water.opc
source leaprc.gaff2
```

Note that explicit solvent simulations now require you to load a *leaprc.water.xxxx* file; this is a change from earlier versions, where the TIP3P water model was loaded by default. The change reflects

3. Molecular mechanics force fields

the growing awareness[21] within the modeling community that TIP3P should no longer be assumed as appropriate for every type of biomolecular simulation, and that the use of more modern water models instead can offer clear accuracy improvements in a rapidly increasing number of situations, see below. Note the importance of the order in which the different components are loaded; in particular, the water model should be loaded after the protein force-field.

3. There are some leaprc files for older force fields in the `$AMBERHOME/dat/leap/cmd/oldff` directory. We no longer recommend these combinations, but we recognize that there may be reasons to use them, especially for comparisons to older simulations. See Section 3.12 for more information.
4. In particular, the `leaprc.ff14SB` file, in the `oldff/` directory, is identical to the file of the same name in the directory above it. In spite of its name, it is a “combined” file, with protein, DNA, RNA and water elements. This file might be of particular interest if you want to make sure that systems created the “new” way (with the leaprc files outlined above) are consistent with those using the older, “combined” method.

3.1. Proteins

In addition to the recommended file, `leaprc.protein.ff14SB`, there are a variety of alternatives for proteins; these are described in the following sections. A recent review of Amber protein force fields, past and present,[22] provides additional information and perspective.

3.1.1. The SB family of protein forcefields (ff19SB, ff14SB, and ff99SB)

```
leaprc.protein.ff19SB
leaprc.protein.ff14SB

leaprc.protein.ff14SBonly # This is the same as leaprc.protein.ff14SB, but will additionally load:
frcmod.ff99SB14          ff99SB backbone parameters with ff14SB atom types
```

ff19SB

`ff19SB` [23] is the latest model of the SB protein forcefields, developed in the Simmerling Lab at Stony Brook University. The new ff19SB forcefield has shown to improve amino acid-dependent properties such as helical propensities and reproduces the differences in amino-acid-specific PDB Ramachandran map. Users are encouraged to read the ff19SB article [23] to learn more about the motivation behind ff19SB, as well as details of the fitting and testing protocols and improved performance relative to ff14SB. Our older SB protein forcefield models utilized uncoupled phi/psi dihedral parameters for the protein backbone, and every amino acid except for glycine used the backbone dihedral parameters fit using alanine. In ff19SB, we improved the backbone dihedrals parameters for every standard amino acids. We fit coupled ϕ/ψ parameters using 2D ϕ/ψ conformational scans for multiple amino acids, using 2D QM energy surfaces in solution as reference data. These new dihedral parameters include amino-acid specific CMAPs that are based on residue name. We also zeroed the amplitudes of the old backbone phi/psi dihedral parameters (in atom name, C-N-CA-C, N-CA-C-N, C-N-CA-CB, CB-CA-C-N, HA-CA-C-O) from ff14SB that are based on the atom types. It is important that ff19SB be combined only with a parameter set that has no cosine terms for these dihedrals.

Our results [23] showed that ff19SB pairs best with the more accurate water model OPC [24], and that the older TIP3P model has serious limitations when used with the QM-based ff19SB. As a result, **we strongly recommend using ff19SB with OPC**, and we recommend against use with TIP3P.

In order to separate the new ff19SB parameters from the original ff14SB parameters, a new atom type XC was created for C-alpha for all non-terminal residues. All the bonds, angles, non-bonded parameters (except S, see below), and dihedral parameters not involving C-alpha were retained from ff14SB. The old backbone dihedral parameters for C-alpha were modified to use atom type XC for C-alpha (instead of the old CX), and the amplitudes were set to zero since it will use CMAP instead.

How to use ff19SB:

To use ff19SB users can execute the following command in tleap:

```
source leaprc.protein.ff19SB
```

This will load the following files:

1. **parm19.dat** is similar to parm10.dat. It has the new atom type XC parameters, which are identical to CX parameters, except for the dihedral H1-CX-C-O parameters.
2. **frmod.ff19SB** contains the parameters from frmod.ff14SB, where the CX atom type was replaced with the XC atom types. The dihedral H1-CX-C-O was copied over from parm10.dat. CX is also replaced with XC for this dihedral. The magnitude of the backbone dihedrals with XC is zeroed. This is done since the residue-based CMAP is used instead to calculate the backbone dihedral energies. The Lennard-Jones parameters for S, SH were both obtained from atom type "s" (sulfur with one connected atom) from gaff2.dat, while Lennard-Jones parameters for HS were obtained from atom type "hs" (hydrogen-bonded to sulphur) in gaff2.dat. The CMAP parameters were updated for all non-terminal versions of the 20 standard amino acids, as well as alternate protonation states for these residues.
3. **amino19.lib** All parameters from amino12.lib were copied over. Then, CX (alpha carbon atom type in ff14SB) was replaced with XC for the entire file. None of the amino acids here should use atom type CX for the alpha carbon.
4. **aminont12.lib and aminoct12.lib** is the same file as used for ff14SB, and is not changed in ff19SB. ff19SB CMAP parameters are not applied to terminal amino acids since they do not have both phi and psi. Instead, ff14SB is applied using parameters contained in aminont12.lib for N-terminal amino acids and aminoct12.lib for the C-terminal amino acids.

Instructions for implementing ff19SB for a new amino acid (residue)

The situation often arises when a user may want to modify parameters for a standard amino acid or may want to create a new parameters set for a modified amino acid. If the user wants to implement ff19SB on their new amino acid, they should be cautious about the C-alpha atom type. In ff14SB, CX is used for the C-alpha atom type, and hence all the ff14SB backbone parameters specify the CX atom type. In ff19SB, CX is replaced by XC, and hence all the ff19SB backbone parameters specify the XC atom type. Additionally, the ff19SB backbone dihedral parameters are zeroed, since CMAPS are used to define the energy of phi and psi. Importantly, if the CX atom type is used, then ff14SB backbone dihedral parameters will be applied to all residues that use the CX atom type, and if the XC atom type is used, then all backbone dihedral parameters will be zeroed. Care must be taken not to mix these two protocols. When implementing ff19SB for a new amino acid, the user has the option to build their topology file via tleap using pure ff19SB including a generic CMAP for the new residue, or a mixture of ff14SB/ff19SB using ff19SB for everything except the new residue. Therefore we urge the user to follow the procedure described in one of the scenarios below.

Scenario 1: In order to apply ff14SB parameters to a non-standard amino acid or a specific standard amino acid and apply ff19SB to every other amino acid in the protein, please follow these steps:

```
source leaprc.protein.ff19SB
loadoff user-defined-file.lib
loadamberparams user-defined-file.frcmod
```

The user-defined library and frcmod files for the new residue must use the CX atom type for C-alpha. Since the ff19SB CMAP is applied based on residue name, it is important that new residue using CX for C-alpha does not match the existing residue names for the standard amino acids, or else the CMAP will be applied in addition to the ff14SB backbone parameters, giving incorrect results.

Scenario 2: In order to apply ff19SB parameters to a non-standard amino acid or a specific standard amino acid and also apply ff19SB to every other amino acid in the protein, please follow these steps:

3. Molecular mechanics force fields

```
source leaprc.protein.ff19SB
loadoff user-defined-file.lib
loadamberparams user-defined-file.frcmod
loadamberparams frcmod.ff19SB.XXX
```

The user-defined library file and frcmod files for the new residue must use the XC atom type for C-alpha. Ensure the amplitudes of the phi/psi dihedrals are zeroed since you will be applying a CMAP for phi/psi. To apply a CMAP for the phi/psi dihedral of the modified amino acid, the user must modify the provided file frcmod.ff19SB.XXX by replacing XXX in the CMAP_TITLE and CMAP_RESLIST shown below, with the new residue name matching that defined in the user-defined library file. frcmod.ff19SB.XXX can be found in \$AMBERHOME/dat/leap/parm/ directory.

```
%FLAG CMAP_TITLE
XXX CMAP
%FLAG CMAP_RESLIST 1
XXX
```

frcmod.ff19SB.XXX will apply the LEU CMAP backbone parameters which we recommend as a generic model for modified amino acids. Next, the user can load the new frcmod.ff19SB.XXX.

ff14SB

ff14SB [25] was a continuing evolution of the earlier *ff99SB* force field.[26] Several groups had noticed that the older *ff94* and *ff99* parameter sets did not provide a good energy balance between helical and extended regions of peptide and protein backbones. Another problem is that many of the *ff94* variants had incorrect treatment of glycine backbone parameters. *ff99SB* improved this behavior, presenting a careful reparametrization of the backbone torsion terms in *ff99* and achieves much better balance of four basic secondary structure elements (PP II, β , α_L , and α_R). Briefly, dihedral term parameters were obtained through fitting the energies of multiple conformations of glycine and alanine tetrapeptides to high-level *ab initio* QM calculations. We have shown that this force field provides much improved proportions of helical versus extended structures. In addition, it corrected the glycine sampling and should also perform well for β -turn structures, two things which were especially problematic with most previous Amber force field variants. The changes mainly involve torsional parameters for the backbone and side chains. For backbones, experimental scalar coupling data for small solvated peptides became available [27] against which *ff99SB* was compared.[28] As *ff99SB* backbone dihedrals were fit based on gas-phase quantum data, we felt that slight empirical adjustments were worth pursuing. This was done to improve agreement with scalar coupling data, and we observed that this also improved stabilities of helical peptides.

ff14SBonlysc

ff14SBonlysc, where *sc* stands for side chains, includes *ff99SB* backbone parameters with updated side chain parameters that were derived from *ab initio* quantum mechanics calculations (as were the *ff99SB* backbone corrections). This model is slightly different from *ff14SB*, which includes the *ff14SBonlysc* parameters as well as a small empirical correction to backbone parameters that was designed to improve agreement between NMR data and simulations in TIP3P water for short peptides. We are currently exploring whether this empirical correction also improves simulations in other water models, such as the *GBneck2* (igb=8) model. [29] Currently, it appears that igb=8 may work best with the fully quantum mechanics-based dihedral parameters included in *ff14SBonlysc*. Simulations performed in explicit water most likely benefit from the empirical corrections included in *ff14SB* or *ff19SB*.

3.1.2. The ff15ipq protein force field

<pre>leaprc.protein.ff15ipq parm15ipq_10.3.dat amino15ipq_10.0.lib</pre>	This will load the files listed below force field parameters topologies and charges for amino acids
--	--

<code>aminont15_ipq10.0.lib</code>	same, for N-terminal amino acids
<code>aminont15ipq_10.0.lib</code>	same, for C-terminal amino acids

ff15ipq [30] continues the development begun with the *ff14ipq* force field [31, 32], but offers new, we hope better, parameter choices, data fitting, and validation. The physical assumptions behind the model are the same, but problems with *ff14ipq*, most generally the "stickiness" of polar groups in simulations, led to sweeping parameter changes. The pair-specific Lennard-Jones terms in *ff14ipq* were the problem, introducing an imbalance of protein:water and protein:protein interactions. They have been replaced by modified polar hydrogen radii and a consistent Lorentz-Berthlot combining rule as found in other Amber force fields. As a consequence, the entire charge set has changed, albeit slightly, and the torsion parameters have been expanded and rederived. To further improve the internal potential energy surface, refitted angle parameters are included for the protein backbone. The new version comprises nearly 1,200 unique parameters, and *ff14ipq* is archived (use `oldff/leaprc.ff14ipq`) for backwards compatibility and comparisons.

The extended IPolQ charge derivation anticipates a workflow in which the final model must have charges roughly consistent with the polarization molecules experience in water, but also new torsion parameters which are often derived with quantum calculations of the system in vacuum. In the extended methodology, two sets of charges are fitted: one for the systems in vacuum and the other for systems in the condensed phase. The original IPolQ method [31] derives the appropriate condensed phase charges by fitting to the average electrostatic potential of polarized and unpolarized molecules: a process that harkens to linear response theory and implicitly accounts for the energetic cost of polarizing the system away from its gas phase equilibrium. The extended scheme draws on the vacuum phase electrostatic data a second time to make an alternative set of charges appropriate to describe the vacuum potential energy surface—the IPolQ charges themselves are, in fact, re-expressed as a perturbation of this gas phase charge set. Both sets of charges are derived in the same linear least squares fitting problem, with restraint equations weakly coupling the corresponding charges together. This creates charge sets for each phase related by a minimal perturbation, which can be assumed to be the effective, average polarization of the molecules when they enter solution. The charge set appropriate to the vacuum phase is then used when fitting torsion potentials to vacuum phase quantum mechanical energies, and the torsion potentials are transferred directly for use with the condensed-phase charge set in actual simulations, following the earlier assumption that the effective polarization of the molecules, and thereby any energetic consequences of entering the condensed phase, are captured in the charge perturbation.

All parameter optimization in *ff15ipq*, like its predecessor *ff14ipq*, is iterative: a generational learning scheme whereby the results of previous simulations and force field manipulations are submitted to quantum single point energy calculations and then added to the training data. As with *ff14ipq*, charges and gas-phase conformational energies are all taken at the MP2/cc-pVTZ level; *ff15ipq* takes the *ff14ipq* conformational energies as its starting point and expands the space nearly four-fold. We find that this crude form of machine learning is a good substitute for human intervention. As with *ff14ipq*, the iterative process led to an evolution in simulation performance over a variety of systems. We utilized these benchmarks to determine when the parameter set was ready for general release.

The new *ff15ipq* model [30] was derived with the SPC/E-b water model of Takemura and Kitao [33]. Returning to three-point water models improves performance of most Amber protein simulations on GPUs by about 30% due to the reduction in the overall number of particles; a smaller improvement can be seen on CPUs. While SPC/E-b is the recommended water model, the solvent reaction field potential observed in our IPolQ studies is consistent across three- and even some four-point waters: combinations of *ff15ipq* with TIP3P, the original SPC/E, and other water models are reasonable to try. One issue that may arise in some circumstances is the compatibility of the water model with ion parameters: we have set *ff15ipq* to reference ion parameters appropriate for the nearest water model available, SPC/E. However, for highly charged or dense ionic solutions this combination may be sub-optimal. With respect to compatibility with other macromolecular force fields such as sugars, lipids, or nucleic acids, we note that while the charge set is novel, the MP2/cc-pVTZ solution-phase IPolQ charges [31] are in fact quite similar to the Cornell charges derived at the HF/6-31G* level [34]. This result may support the long lifespan of that charge set, and makes it likely that *ff15ipq* will be compatible with other force fields designed at the common HF/6-31G* level.

ff15ipq has been validated on a larger number of test systems than its predecessor, and for much longer timescales. Multiple alpha-helical and beta-sheet peptides have been tested at a variety of temperatures, and numerous small

3. Molecular mechanics force fields

proteins (the largest including lysozyme and the p53/MDM2 complex) have been simulated for timescales ranging from 4 to 10 microseconds, displaying excellent stability and also instability in cases where loops of the proteins or isolated peptides are known to be disordered. Various teething problems in the *ff14ipq* force field were solved by improvements to the data set or the fitting protocol itself, so we are increasingly confident that *ff15ipq* and future products of the IPolQ workflow will be reliable straight out of the automated parameter development phase. The entire data set and *mdgx* input file for deriving the torsion and angle parameters of *ff15ipq* will be released as supporting information in the upcoming publication on the force field. In the future we hope to build on the lineage of ff-ipq protein models to include other important areas of biological chemistry.

Modified amino acids. An expanded version of the ff15ipq protein force field, denoted as ff15ipq-m[35], can be obtained with the following command in LEaP:

```
source leaprc.mimetic.ff15ipq
```

This expanded force field enables the modeling of four classes of artificial backbone units that are commonly used alongside natural α residues in blended or “heterogeneous” backbones of protein mimetics: chirality-reversed D- α -residues, the C $_{\alpha}$ -methylated α -residue Aib, homologated β -residues (β^3) bearing proteinogenic side chains, and two cyclic β residues (β^{cy} ; aminopyrrolidine carboxylic acid (APC) and trans-2-aminocyclopentane-1-carboxylic acid (ACPC)). A tutorial is available for getting started with this force field (<http://ambermd.org/tutorials/advanced/tutorial36/index.php>).

Parameters for fluorinated, aromatic amino acids [36] to be used with the ff15ipq protein forcefield can be obtained with the following command in LEaP:

```
source leaprc.fluorine.ff15ipq
```

This includes parameters for 4-, 5-, 6-, and 7-fluoro-tryptophan (W4F, W5F, W6F, W7F), 3-fluoro- and 3,5-difluoro-tyrosine (Y3F, YDF), as well as 4-fluoro- and 4-trifluoromethyl-phenylalanine (F4F, FTF).

3.1.3. The fb15 (“force balance”) protein force field

<code>leaprc.protein.fb15</code>	This will load the files listed below
<code>frcmod.fb15</code>	force field parameters
<code>frcmod.tip3pfb</code>	parameters for the force balance 3-point model
<code>all_aminofb15.lib</code>	topologies and charges for amino acids
<code>all_aminontfb15.lib</code>	same, for N-terminal amino acids
<code>all_aminocfb15.lib</code>	same, for C-terminal amino acids

The files can be used for protein-water simulations using the “force-balance” approach described in Ref. [37, 38]. There is also a 4-point water model available, as described in section 3.6. For alkali and halide ions, the Joung-Cheatham parameters for TIP3P (or TIP4PEW) are recommended; see Section 3.7.

3.1.4. The Duan et al. (2003) force field

<code>leaprc.protein.ff03.r1</code>	loads the following files:
<code>frcmod.ff03</code>	For proteins: changes to parm99.dat, primarily in the phi and psi torsions.
<code>all_amino03.in</code>	Charges and atom types for proteins
<code>all_aminont03.in</code>	For N-terminal amino acids
<code>all_aminoc03.in</code>	For C-terminal amino acids

The **ff03** force field [39, 40] is a modified version of *ff99* (described below). The main changes are that charges are now derived from quantum calculations that use a continuum dielectric to mimic solvent polarization, and that the ϕ and ψ backbone torsions for proteins are modified, with the effect of decreasing the preference for helical configurations. The changes are just for proteins; nucleic acid parameters are the same as in *ff99*.

The original model used the old (*ff94*) charge scheme for N- and C-terminal amino acids. This was what was distributed with Amber 9, and can still be activated by using *oldff/leaprc.ff03*. More recently, new libraries for the

terminal amino acids have been constructed, using the same charge scheme as for the rest of the force field. This newer version (which is recommended for all new simulations) is accessed by using *leaprc.protein.ff03.r1*.

3.1.5. The Yang et al. (2003) united-atom force field

<code>frcmod.ff03ua</code>	For proteins: changes to <code>parm99.dat</code> , primarily in the introduction of new united-atom carbon types and new side chain torsions.
<code>uni_amino03.in</code>	Amino acid input for building database
<code>uni_aminont03.in</code>	NH3+ amino acid input for building database.
<code>uni_aminoc03.in</code>	COO- amino acid input for building database.

The **ff03ua** force field [41] is the united-atom counterpart of *ff03*. This force field uses the same charging scheme as *ff03*. In this force field, the aliphatic hydrogen atoms on all amino acid side-chains are united to their corresponding carbon atoms. The aliphatic hydrogen atoms on all alpha carbon atoms are still represented explicitly to minimize the impact of the united-atom approximation on protein backbone conformations. In addition, aromatic hydrogens are also explicitly represented. Van der Waals parameters of the united carbon atoms are refitted based on solvation free energy calculations. Due to the use of an all-atom protein backbone, the ϕ and ψ backbone torsions from *ff03* are left unchanged. The sidechain torsions involving united carbon atoms are all refitted. In this parameter set, nucleic acid parameters are still in all atom and kept the same as in *ff99*.

3.1.6. Options for intrinsically disordered proteins.

Intrinsically disordered proteins (IDPs) and intrinsically disordered regions (IDRs) are proteins or parts (regions) of protein that lack stable secondary and tertiary structures under specific physiological conditions[42]. Compared to globular proteins in their native states, atomistic modeling of IDPs and IDRs is inherently more demanding: these structures are represented by multiple inter-converting conformations, often within $k_B T$ of each other. Thus, while a simulation that focuses on the unique native state of a globular protein may be robust to errors in the force-field that over-stabilize the native state, the same errors of just 1 or $2k_B T$ may lead to a completely wrong relative abundance of conformations representing the IDP. Long time-scale simulations have demonstrated[43] that several popular water models, in combination with any of several widely accepted force-fields, lead to overly compact IDP conformations. Efforts to improve force fields and water models for IDPs are on-going[43–47]; recently, OPC water model in combination with the *ff99SB* was found to improve, significantly, accuracy of atomistic simulations of IDPs[48].

3.2. Nucleic acids

As with proteins, many features of the current force fields, including partial atomic charges, Lennard-Jones parameters, and most bond and angle terms, date back to force fields developed in the 1990's, and overviews of this work are available.[49, 50] The next breakthroughs in the Amber nucleic acid force field development came from observations on relatively longer simulations, 50-100 ns time scale, in the early 2000's.[51, 52] These simulations found systematic over-population of $\gamma = trans$ backbone geometries in nucleic acids. High level QM calculations were performed on models of sugars and phosphates, specifically a sugar-phosphate model[53] and a sugar-phosphate-sugar model,[54] which ultimately led to the *ff99-bsc0* parameterization.[53] For simulation of canonical DNA and RNA structures, the *ff99-bsc0* parameterization has proven rather successful. For non-canonical structures, particularly those with loops or bulges, or χ flips, some anomalies have been noted.

3.2.1. RNA

With RNA, incorrect loop geometries, backbone sub-state populations, and sugar pucker populations were observed in longer simulations. In addition to occasional non-conservation of south puckers, multiple groups noticed a tendency for the RNA backbone to shift, putting χ into the high-*anti* region which leads to an opening of the

3. Molecular mechanics force fields

Desired Behavior	Source these files	Notes
RNA		
<i>ff99OL3</i>	<i>leaprc.RNA.OL3</i>	<i>parmbsc0 α/γ [53] + χOL3 [55] to ff99</i>
<i>ff99OL3 + backbone phosphate</i>	<i>leaprc.RNA.LJbb</i>	<i>ff99OL3 + backbone phosphate modifications[56, 57]</i>
<i>ff99χ + bsc0</i>	<i>leaprc.RNA.YIL</i>	<i>parmbsc0 α/γ [53]+ Yildirim [58]χ mods to ff99.</i>
<i>ff99bsc0</i>	<i>oldff/leaprc.ff99bsc0</i>	<i>Contains parmbsc0 α/γ mods[53] to ff99.</i>
“Rochester” torsions	leaprc.RNA.ROC	[59]
“DE Shaw” modifications	leaprc.RNA.Shaw	[60]
Modified nucleotides	leaprc.modrna08	parameters for modified nucleosides [61]

Table 3.1.: How to specify RNA force fields in LEaP. Recommended variants are listed in italics.

duplex structure into a ladder-like configuration. Again, QM methods at various levels were employed to improve the χ distribution using relevant model systems. The most tested χ modifications are the “OL” modifications used in `leaprc.RNA.OL3`. [55, 62]

On top of the OL modifications, Bergonzo & Cheatham [57] found that with modified phosphate parameters from Steinbrecher et al. [56] and an improved water model (OPC), better agreement with NMR data for RNA tetranucleotide populations was observed. In this parameter set, a new atom type for O4’ was created named OR (previously type OS). This allowed modification of O2 and OS atom types to LJ=1.7493, 0.2100 and 1.7718, 0.1700; previous values were LJ=1.6612, 0.2100 and 1.6837, 0.1700. These changes are incorporated into `leaprc.RNA.LJbb`.

An alternative available with Amber is the Yildirim χ modifications (and also related modifications called TOR which alter ϵ/ζ as well) [58, 63, 64], and a systematic assessment and validation of these newer χ modifications is underway on a large series of RNA tetraloop structures. Note that small changes to a particular dihedral may lead to alteration in properties of related dihedrals, and may have unintended consequences. For example, the *ff99-bsc0* modifications tend to lock RNA sugar puckers mainly in the north, even with nucleotides in particular sequence contexts that prefer southern conformations. Moreover, the χ modifications tend to further destabilize γ = trans. This suggests that to reliably improve the nucleic acid dihedrals, a more systematic approach across many dihedrals with simultaneous fitting may be more appropriate. Moreover, we no longer fully support the idea that parameters are transferable between DNA and RNA, or between purines and pyrimidines. For example, the *ff99-OL* modifications (with or without *ff99-bsc0*) improve the modeling of RNA, but lead to issues with DNA, most notably with quadruplex structures. Therefore recent work has focused on separate χ modifications for DNA. [65]

An alternative set of torsions for RNA, fit to quantum calculations has been developed by the Rochester group, [59] and can be loaded with the `leaprc.RNA.ROC` file. More extensive modifications are contained in the “DE Shaw” force field, [60], which can be loaded with `leaprc.RNA.Shaw`.

3.2.2. DNA

As noted in Table 3.2, most current DNA force fields are based on parameters and charges that go back to Amber’s *ff99*. A new set of parameters for the δ/τ_1 pucker dihedrals [73], the ϵ/ζ dihedral [66] and for the β dihedral [67] torsion for DNA have been developed using QM methods that include the solvation effects implicitly. This set of parameters have been tested with several double-stranded DNA systems including the Dickerson-Drew dodecamer, A-tracs, CG-rich duplexes, DNA-protein interactions, Z-DNA and G-quadruplexes. The most recent OL24 combines the previous OL15 and OL21 versions and adds refinements to the sugar pucker for DNA. More details about the OL24 force field development and test cases are available in http://fch.upol.cz/ff_ol/ and is now our recommended DNA force field, available in `leaprc.DNA.OL24`. [68, 74]. The reader should pay careful attention to the use of the χ modifications, since the naming convention of the authors is the same for RNA and DNA. In addition, the *tumuc1* DNA force field (based on *parm94*) provides refined QM-based electrostatic parameters, available in `leaprc.DNA.tumuc1`

Details of the different modifications available for DNA are presented in Table 3.2. Regarding the performance of *bsc1*, *tumuc1* and OL24 for DNA, preliminary testing comparing these force fields strongly suggests that all the variations perform similarly and are improvements over the previous *bsc0* modification. [74–76] We refer the

Name	Modification	Notes
ff94	Original force field file	Obsolete
ff98	Modified charge set	Obsolete
ff99	Updated charge set	Foundation for all current ff's
bsc0	Barcelona α/γ backbone modification	[53]
ϵ/ζ OL1	ϵ/ζ modification for DNA	improvement for DNA, no effects for RNA [66]
χ OL4	χ modification tuned for DNA	[65]
β OL1	β dihedral modification tuned for DNA	improvement for DNA, no effects for RNA[67]
OL15	(ϵ/ζ OL1 + χ OL4 + β OL1)	[68]
bsc1	Update to bsc0	[69]
tumuc1	Refined electrostatic terms	[70, 71]
OL21	OL15 + α/γ modification	[72]
OL24	OL21 + δ/τ_1 modifications	[73]

Table 3.2.: Force field name and modifications for simulating nucleic DNA. Recommended variants are listed in *italics*.

reader to the original articles of each force field to better understand the details and performance between each variant.

3.2.3. Single nucleotides

Nucleic acid residues use the new (version 3) PDB nomenclature: “DC” is used for deoxy-cytosine, and “C” for cytosine in RNA, etc. Earlier force fields (which are *not* recommended!) use “RC” for the RNA version. If you want a single, nucleoside, use “CN”, etc. For a single nucleotide, use the following command in LEaP:

```
cnuc = sequence { OHE C3 }
```

and analogs for other bases. Note that this will construct a protonated 5' phosphate group, which may not be what you want.

3.3. Modified amino acids and nucleotides

3.3.1. Phosphorylated amino acids

Parameters for phosphorylated amino acids [56, 77] to be used for ff99SB and older forcefields can be obtained with the following command in LEaP:

```
source leaprc.phosaa10
```

Updated parameters have been developed for newer versions of the Stony Brook (SB) family of forcefields, with new forcefield parameters for the side chains of phosphorylated amino acids [78], in addition to modified amino acids [79] that are commonly used in experimental studies such as FRET and EPR. These side-chain parameters are optimized for use with ff14SB and ff19SB by fitting against relative QM energies at the MP2/6-311+G** level using our inhouse torsion fitting protocol[80]. Currently, side-chain parameters for phosphorylated serine, histidine (deprotonated, protonated), tyrosine, and threonine are provided. For ff14SB, parameters for phosphorylated amino acids [78] can be obtained with the following command in LEaP:

```
source leaprc.phosaa14SB
```

For ff19SB, parameters for phosphorylated amino acids [78] can be obtained with the following command in LEaP:

```
source leaprc.phosaa19SB
```

3.3.2. Other modified amino acids

The modified amino acids selenomethionine, cyano-phenylalanine, and azido-phenylalanine are used as FRET quenchers. We also added parameters for acetylated lysine and for the nitroxide spin-label methanesulfonylthioate (MTSL), which is often used in EPR experiments to probe distances. For selenomethionine, we fit new LJ parameters for selenium, as well as bond, angle, and dihedral parameters for the C-Se bond. To use these parameters for ff14SB, the user can run the following command in LEaP:

```
source leaprc.protein.ff14SB_modAA
```

To use these parameters for ff19SB, the user can run the following command in LEaP:

```
source leaprc.protein.ff19SB_modAA
```

The ff19SB_modAA leaprc will load lib and frcmod files that have the CX to XC atom type conversion, the backbone phi/psi dihedrals will be zeroed, and the LEU CMAP will be applied to all five residues.

The residue names for these modified amino acids are MSE (selenomethionine), AZF (azido-phenylalanine), CYF (cyano-phenylalanine), CNX (MTSL) and ALY (acetylated-lysine). These residue names should match those in the loaded file with the coordinates (e.g. PDB file). The residue names can also be used with the sequence command in LEaP to create XYZ coordinates. Since the modifications for the phosphorylated and modified amino acids are on the side chains and not the backbone, users can use these modifications with ff19SB.

3.3.3. 5' phosphorylation in nucleic acids

The 5' end of many DNA and RNA chains have one or more phosphate groups. These can be accessed by loading the following library, after loading the standard DNA or RNA libraries:

```
loadOff terminal_monophosphate.lib
```

See Ref. [56] for details on how these were constructed. After loading this file, the 5' residues for DNA and RNA (e.g. DA5 and A5) will contain a terminal (mono-)phosphate group with a single negative charge.

A more recent set of 5' phosphorylated RNA residues [57] is included in `leaprc.RNA.LJbb`. Here, the somewhat non-standard residue names AMP, GMP, UMP and CMP are used to indicate 5' phosphorylation. You may need to edit input PDB files to match this nomenclature.

3.3.4. Generation of new nucleotides using modXNA

The modXNA tool [81] allows users to quickly create new Amber compatible nucleotides in the form of a general *frcmod* and modification-specific LIB files that can be loaded directly into LEaP. This tool consists of a collection of base, sugar and backbone fragments (mol2 files) that can be stitched together to create the required nucleotide. The fragments are located in \$AMBERHOME/dat/modXNA path and can be browsed at the *modxna.chpc.utah.edu* website (<https://modxna.chpc.utah.edu/catalog/>). modXNA requires a text input file (i. e. *in.modxna*) specifying what fragments to stitch together. The fragments are 3-letter codes for each fragment available: one for the backbone, one for the sugar and one for the base.

The fragments must be specified in the above order. For example, to create a nucleotide including a phosphorothioate backbone (code PS1 according to the modXNA website), a sugar with a C2'-O-methyl (code OME) and a 5-methylcytosine (code M5C), the input file for modXNA will be:

```
PS1 OME M5C
```

The first column declares what backbone to grab from the DAT library of backbone fragments, the second column declares the base fragment and the third column refers to the base. To run the script use:

```
modxna -i in.modxna
```

The modXNA tool will look into the \$AMBERHOME/dat/modXNA library for the specified residues, remove the required capping groups from each fragment, adjust the charge and create a new structure; it will run a small optimization on the structure and generate a LIB file that can be used in LEaP, along with the global frcmod.modxna file, to build systems with modified nucleotides. The script will create multiple tmp files to track any errors in the structure generation. The *tmp.opt.pdb* file is generated from the GB optimization and can be used to quickly check that the requested structure was generated.

For more information about the use of this tool refer to the original publication [81] and the website *mod-xna.chpc.utah.edu* to see the catalog of available fragments and more examples on how to use this tool.

3.4. Carbohydrates

GLYCAM06 is a consistent and transferable parameter set for modeling carbohydrates,[82] and glycoconjugates.[83, 84] The core philosophy of the force field development process is that parameters should be: (1) be transferable to all carbohydrate ring formations and sizes, (2) be self-contained and therefore readily transferable to many quadratic force fields, (3) not require specific atom types for α - and β -anomers, (4) be readily extendable to carbohydrate derivatives and other biomolecules, (5) be applicable to monosaccharides and complex oligosaccharides, and (6) be rigorously assessed in terms of the relative accuracy of its component terms.

When combining GLYCAM06 with AMBER parameters for other biomolecules, parameter orthogonality is ensured by assigning unique atom types for GLYCAM. In order to facilitate combining GLYCAM06 with other AMBER parameter sets for other biomolecules, a variation on the GLYCAM atom types has been introduced in which the new name consists of an uppercase letter followed by second character, either a number or lowercase letter. For example the GLYCAM "CG" atom type has been changed to "Cg"; "HO" is now represented as "Ho", and so forth.

As soon as new parameters are generated, or alterations are made to existing parameters, a new version of GLYCAM is released. Updated versions that introduce new functionality are denoted using a letter suffix (i.e. GLYCAM06a, 06b, etc.). Each release is accompanied with an associated text file that summarizes the new functionality or alteration. For example, a particularly important update, released in GLYCAM06e, altered the endo-anomeric torsion term (Cg-Os-Cg-Os) in order to more accurately reproduce the populations arising from ring flips (4C_1 to 1C_4 etc.). This particular case suggested the need to be able to independently characterize the exo- and endo-anomeric effect, which was achieved by assigning different atom types (Oa and Oe) to represent the endo-anomeric and exo-anomeric oxygen atoms, respectively.

In another important update (GLYCAM06g), a small van der Waals term was applied to all hydroxyl hydrogen atoms (Ho) to address a rare, but catastrophic, situation that can arise during MD simulations. In certain carbohydrate (and potentially other) configurations, a hydroxyl proton may be structurally constrained to being very close to a carboxylate moiety. During an MD simulation of such a system, an oscillatory motion can begin between the hydroxyl proton and the negative charge site, leading ultimately to failure of the simulation as the proton collapses onto the negatively charged moiety. The small van der Waals term (Ho, $R^* = 0.2000 \text{ \AA}$, $\epsilon = 0.0300 \text{ kcal/mol}$) is just large enough to add sufficient repulsion to prevent this behavior, while not being large enough to perturb properties such as hydrogen bond lengths.

The GLYCAM force field family, especially, GLYCAM06, has been extensively employed in simulations of biomolecules by the larger scientific community.[85–88] The updated GLYCAM parameters and documentation are available for download at the GLYCAM-Web site (*www.glycam.org*). Also available on the website are tools for simplifying the generation of structure and topology files for performing simulations of oligosaccharides, glycoconjugates and glycoproteins. GLYCAM-Web has been integrated into several glycomics databases, such as the Consortium for Functional Glycomics (*www.functionalglycomics.org*).

GLYCAM06 force field

Always check *glycam.org/params* for more recent versions and new functionalities.

```
leaprc.GLYCAM_06j-1      LEaP configuration file for use of GLYCAM06
                           with carbohydrates alone or in combination
                           with the ff14SB force field.
```

3. Molecular mechanics force fields

GLYCAM_06j.dat	Parameters for oligosaccharides
GLYCAM_06j-1.prep	Structures and charges for glycosyl residues
GLYCAM_lipids_06h.prep	Structures and charges for some lipid residues
GLYCAM_amino_06j_12SB.lib	Glycoprotein libraries compatible with ff14SB.
GLYCAM_aminoc_06j_12SB.lib	
GLYCAM_aminont_06j_12SB.lib	

GLYCAM06EP force field using lone pairs (extra points)

GLYCAM_06EPb.dat	Parameters for oligosaccharides
GLYCAM_06EPb.prep	Structures and charges for glycosyl residues
leaprc.GLYCAM_06EPb	LEaP configuration file for GLYCAM-06EP

GLYCAM Force Field Parameters Download Page

<https://glycam.org/docs/forcefield/parameters/index.html>

GLYCAM_06j-1.prep contains prep entries for all carbohydrate residues and GLYCAM_lipids_06h.prep contains prep entries for some lipid residues (although for lipid membrane simulations we recommend you use the Amber Lipid 21 force field). GLYCAM_06EPb.prep contains prep entries for all carbohydrate residues available for modeling with extra points.

For linking glycans to proteins, libraries containing modified amino acid residues (Ser, Thr, Hyp, and Asn) must be loaded. To build a glycoprotein using ff14SB, GLYCAM_amino_06j_12SB.lib GLYCAM_aminont_06j_12SB.lib and GLYCAM_aminoc_06j_12SB.lib must be loaded and the desired protein force field must also be loaded. Amino acid libraries designed for linking carbohydrates modeled with extra points are not currently available.

3.4.1. File versioning

Beginning on 15 September, 2011, a new versioning system was implemented for Glycam parameters. Files produced before that date will not necessarily conform to the new system. In the new system, all files containing parameters are versioned. Users should check their contents and replace them with recent versions as appropriate.

The new versioning system employs letters and numbers. If a parameter set contains new functionality (e.g., the addition of new parameters) or fundamental changes (e.g., atom type name reassignments), a letter will be appended to its name. If the new version contains corrections (e.g., for typographical errors), its name will be appended with a number. See glycam.org/params for more documentation and examples.

Researchers are also encouraged to read the version change documentation available on the GLYCAM Parameters download page under "Documents." In this document, the changes specific to each version release are detailed. The changes are also summarized here in Table 3.3.

3.4.2. Atom type name changes

Beginning with versions g, Glycam atom type names will adopt a standard designed to keep them from overlapping with other force fields. In most cases, Glycam's type names will consist of two characters, one upper-case followed by one lower-case. Because of this, leaprc files, lib files and prep files from versions prior to g will be incompatible with current versions.

Note that some type names will not reflect the new Glycam type standard, despite being present in the Glycam force field files, for example in the files for linking glycans to amino acid residues. In these cases, Glycam will use the type name appropriate to the external force field. Parameters will be introduced only to the extent necessary to provide a link between the force fields. Since the associated parameters will also include Glycam types, they should only affect the intersections between the two force fields.

Beginning with versions j, atom type names for linking to amino acids are compatible with ff14SB. Older versions of protein and nucleic acid force fields might not be compatible.

Version	Release Date	Contributors	Change Summary
j	15 Feb., 2014	BLF	<i>Modified all parameters to be compatible with ff14SB. These files may not be compatible with older protein and nucleic acid force fields.</i>
i	27 Aug., 2013	AKN	Added two new monosaccharides to the prep file.
h	20 Oct., 2010	MBT, BLF	<i>*Changed atom type naming to be orthogonal to other force fields.</i> Added HO van der Waals parameters. Set protein-related parameter values to their parm99 counterparts. Updated N-sulfation parameters.
g	20 Oct., 2010	MBT	<i>* 1,4-scaling terms added to parameter file.</i> Angle and torsion updates for pyranose rings, N-sulfate, phosphate and sialic acid.
f	3 Feb., 2009	MBT	<i>* Corrected a typo in O-Acetyl term</i>
e	28 May, 2008	MBT	<i>* Updated glycosidic linkage terms to optimize ring puckering in pyranoses</i>
d	12 May, 2008	SPK, MBT, ABY	Terms for thiol glycosidic linkages
c	21 Feb., 2008	MBT, ABY	<i>* Additional (published) terms for some lipid simulations[89]</i>
b	10 Jan., 2008	MBT, ABY	Alkanes, alkenes, amide and amino groups for some lipid simulations[89]
a	24 Apr., 2005	ABY	Sulfates & phosphates for carbohydrates

Table 3.3.: Version change summary for the GLYCAM-06 force field. **Previously released parameters were changed. See full release notes at glycam.org/params. SPK: Sameer P. Kawatkar. MBT: Matthew B. Tessier. ABY: Austin B. Yongye. BLF: B. Lachele Foley. AKN: Anita K. Nivedha*

3.4.3. General information regarding parameter development

In GLYCAM-06,[82] the torsion terms have now been entirely developed by fitting to quantum mechanical data (B3LYP/6-31++G(2d,2p)//HF/6-31G(d)) for small-molecules. This has converted GLYCAM-06 into an additive force field that is extensible to diverse molecular classes including, for example, lipids and glycolipids. The parameters are self-contained, such that it is not necessary to load any AMBER parameter files when modeling carbohydrates or lipids. To maintain orthogonality with AMBER parameters for proteins, notably those involving the CT atom type, tetrahedral carbon atoms in GLYCAM are called Cg (C-GLYCAM, CG in previous releases). Thus, GLYCAM and AMBER may be combined for modeling carbohydrate-protein complexes and glycoproteins. More information on atom type names is available in 3.4.2. Because the GLYCAM-06 torsion terms were derived by fitting to data for small, often highly symmetric molecules, asymmetric phase shifts were not required in the parameters. This has the significant advantage that it allows one set of torsion terms to be used for both α - and β -carbohydrate anomers regardless of monosaccharide ring size or conformation. A molecular development suite of more than 75 molecules was employed, with a test suite that included carbohydrates and numerous smaller molecular fragments. The GLYCAM-06 force field has been validated against quantum mechanical and experimental properties, including: gas-phase conformational energies, hydrogen bond energies, and vibrational frequencies; solution-phase rotamer populations (from NMR data); and solid-phase vibrational frequencies and crystallographic unit cell dimensions.

3.4.4. Development of partial atomic charges

As in previous versions of GLYCAM, the atomic partial charges were determined using the RESP formalism, with a weighting factor of 0.01,[82, 90] from a wavefunction computed at the HF/6-31G(d) level. To reduce artifactual fluctuations in the charges on aliphatic hydrogen atoms, and on the adjacent saturated carbon atoms, charges on aliphatic hydrogens (types HC, H1, H2, and H3) were set to zero while the partial charges were fit to the remaining atoms.[91] It should be noted that aliphatic hydrogen atoms typically carry partial charges that

3. Molecular mechanics force fields

Carbohydrate	Pyranose	Furanose
	α/β , D/L	α/β , D/L
Arabinose	yes	yes
Lyxose	yes	yes
Ribose	yes	yes
Xylose	yes	yes
Allose	yes	
Altrose	yes	
Galactose	yes	<i>a</i>
Glucose	yes	<i>a</i>
Gulose	yes	
Idose	<i>a</i>	
Mannose	yes	
Talose	yes	
Fructose	yes	yes
Psicose	yes	yes
Sorbose	yes	yes
Tagatose	yes	yes
Fucose	yes	
Quinovose	yes	
Rhamnose	yes	
Galacturonic Acid	yes	
Glucuronic Acid	yes	
Iduronic Acid	yes	
N-Acetylgalactosamine	yes	
N-Acetylglucosamine	yes	
N-Acetylmannosamine	yes	
Neu5Ac	yes, <i>b</i>	yes, <i>b</i>
KDN	<i>a, b</i>	<i>a, b</i>
KDO	<i>a, b</i>	<i>a, b</i>

Table 3.4.: Current Status of Monosaccharide Availability in GLYCAM. (*a*) Currently under development. (*b*) Only one enantiomer and ring form known.

fluctuate around zero when they are included in the RESP fitting, particularly when averaged over conformational ensembles.[82, 92] In order to account for the effects of charge variation associated with exocyclic bond rotation, particularly associated with hydroxyl and hydroxymethyl groups, partial atomic charges for each sugar were determined by averaging RESP charges obtained from 100 conformations selected evenly from 10-50 ns solvated MD simulations of the methyl glycoside of each monosaccharide, thus yielding an ensemble averaged charge set.[82, 92]

3.4.5. Carbohydrate parameters for use with the TIP5P water model

In order to extend GLYCAM to simulations employing the TIP-5P water model, an additional set of carbohydrate parameters, GLYCAM-06EP, has been derived in which lone pairs (or extra points, EPs) have been incorporated on the oxygen atoms.[93] The optimal O-EP distance was located by obtaining the best fit to the HF/6-31g(d) electrostatic potential. In general, the best fit to the quantum potential coincided with a negligible charge on the oxygen nuclear position. The optimal O-EP distance for an sp³ oxygen atom was found to be 0.70 Å; for an sp² oxygen atom a shorter length of 0.3 Å was optimal. When applied to water, this approach to locating the lone pair positions and assigning the partial charges yielded a model that was essentially indistinguishable from TIP-5P. Therefore, we believe this model is well suited for use with TIP-5P.[93] The new files are named 06EP (originally 04EP), as they have been corrected for numerous typographical errors and updated to match current naming and

residue structure conventions.

3.4.6. Carbohydrate Naming Convention in GLYCAM

Note: the *prepareforleap* action in *cpptraj* automates many of the difficult steps in taking an input PDB file containing carbohydrates, and adapting it for Amber. You should generally try this first before delving into the details of GLYCAM nomenclature discussed below.

In order to incorporate carbohydrates in a standardized way into modeling programs, as well as to provide a standard for X-ray and NMR protein database files (pdb), we have developed a three-letter code nomenclature. The restriction to three letters is based on standards imposed on protein data bank (PDB) files by the RCSB PDB Advisory Committee (www.rcsb.org/pdb/pdbac.html), and for the practical reason that all modeling and experimental software has been developed to read three-letter codes, primarily for use with protein and nucleic acids.

As a basis for a three-letter PDB code for monosaccharides, we have introduced a one-letter code for monosaccharides (Table 3.5).[94] Where possible, the letter is taken from the first letter of the monosaccharide name. Given the endless variety in monosaccharide derivatives, the limitation of 26 letters ensures that no one-letter (or three-letter) code can be all encompassing. We have therefore allocated single letters firstly to all 5- and 6-carbon, non-derivatized monosaccharides. Subsequently, letters have been assigned on the order of frequency of occurrence or biological significance.

Using three letters (Tables 3.6 to 3.8), the present GLYCAM residue names encode the following content: carbohydrate residue name (Glc, Gal, etc.), ring form (pyranosyl or furanosyl), anomeric configuration (α or β , enantiomeric form (D or L) and occupied linkage positions (2-, 2,3-, 2,4,6-, etc.). Incorporation of linkage position is a particularly useful addition, since, unlike amino acids, the linkage cannot otherwise be inferred from the monosaccharide name. Further, the three-letter codes were chosen to be orthogonal to those currently employed for amino acids.

3. Molecular mechanics force fields

	Carbohydrate ^a	One letter code ^b	Common Abbreviation
1	D-Arabinose	A	Ara
2	D-Lyxose	D	Lyx
3	D-Ribose	R	Rib
4	D-Xylose	X	Xyl
5	D-Allose	N	All
6	D-Altrose	E	Alt
7	D-Galactose	L	Gal
8	D-Glucose	G	Glc
9	D-Gulose	K	Gul
10	D-Idose	I	Ido
11	D-Mannose	M	Man
12	D-Talose	T	Tal
13	D-Fructose	C	Fru
14	D-Psicose	P	Psi
15	D-Sorbose	B ^d	Sor
16	D-Tagatose	J	Tag
17	D-Fucose (6-deoxy D-galactose)	F	Fuc
18	D-Quinovose (6-deoxy D-glucose)	Q	Qui
19	D-Rhamnose (6-deoxy D-mannose)	H	Rha
20	D-Galacturonic Acid	O ^d	GalA
21	D-Glucuronic Acid	Z ^d	GlcA
22	D-Iduronic Acid	U ^d	IdoA
23	D-N-Acetylgalactosamine	V ^d	GalNac
24	D-N-Acetylglucosamine	Y ^d	GlcNAc
25	D-N-Acetylmannosamine	W ^d	ManNAc
26	N-Acetyl-neuraminic Acid	S ^d	NeuNAc, Neu5Ac
	KDN	KN ^{c,d}	KDN
	KDO	KO ^{c,d}	KDO
	N-Glycolyl-neuraminic Acid	SG ^{c,d}	NeuNGc, Neu5Gc

Table 3.5.: The one-letter codes that form the core of the GLYCAM residue names for monosaccharides ^aUsers requiring prep files for residues not currently available may contact the Woods group (www.glycam.org) to request generation of structures and ensemble averaged charges. ^bLowercase letters indicate L-sugars, thus L-Fucose would be “f”, see Table 3.8. ^cLess common residues that cannot be assigned a single letter code are accommodated at the expense of some information content. ^dNomenclature involving these residues will likely change in future releases.[94] Please visit www.glycam.org for the most updated information.

	α -D-Glcp	β -D-Galp	α -D-Arap	β -D-Xylp
Linkage Position	Residue Name	Residue Name	Residue Name	Residue Name
Terminal ^b	0GA ^b	0LB	0AA	0XB
1- ^c	1GA ^c	1LB	1AA	1XB
2-	2GA	2LB	2AA	2XB
3-	3GA	3LB	3AA	3XB
4-	4GA	4LB	4AA	4XB
6-	6GA	6LB		
2,3-	ZGA ^d	ZLB	ZAA	ZXB
2,4-	YGA	YLB	YAA	YXB
2,6-	XGA	XLB		
3,4-	WGA	WLB	WAA	WXB
3,6-	VGA	VLB		
4,6-	UGA	ULB		
2,3,4-	TGA	TLB	TAA	TXB
2,3,6-	SGA	SLB		
2,4,6-	RGA	RLB		
3,4,6-	QGA	QLB		
2,3,4,6-	PGA	PLB		

Table 3.6.: Specification of linkage position and anomeric configuration in D-hexo- and D-pentopyranoses in three-letter codes based on the GLYCAM one-letter code ^aIn pyranoses A signifies α -configuration; B = β . ^bPreviously called GA, the zero prefix indicates that there are no oxygen atoms available for bond formation, i.e., that the residue is for chain termination. ^cIntroduced to facilitate the formation of a 1-1' linkage as in α -D-Glc-1-1'- α -D-Glc {1GA 0GA}. ^dFor linkages involving more than one position, it is necessary to avoid employing prefix letters that would lead to a three-letter code that was already employed for amino acids, such as ALA.

	α -D-Glcf	β -D-Manf	α -D-Araf	β -D-Xylf
Linkage position	Residue name	Residue name	Residue name	Residue name
Terminal	0GD	0MU	0AD	0XU
1-	1GD	1MU	1AD	1XU
2-	2GD	2MU	2AD	2XU
3-	3GD	3MU	3AD	3XU
...	...	...	...	...
etc.	etc.	etc.	etc.	etc.

Table 3.7.: Specification of linkage position and anomeric configuration in D-hexo- and Dpentofuranoses in three-letter codes based on the GLYCAM one-letter code. In furanoses D (down) signifies α ; U (up) = β .

	α -L-Glcp	β -L-Manp	α -L-Arap	β -L-Xylp
Linkage position	Residue name	Residue name	Residue name	Residue name
Terminal	0gA	0mB	0aA	0xB
1-	1gA	1mB	1aA	1xB
2-	2gA	2mB	2aA	2xB
3-	3gA	3mB	3aA	3xB
...	...	...	...	...
etc.	etc.	etc.	etc.	etc.

Table 3.8.: Specification of linkage position and anomeric configuration in L-hexo- and Lpentofuranoses in three-letter codes.

3.5. Lipids

Biological processes in the human body are dependent on highly specific molecular interactions. The vast majority of the interactions take place in compartments within the cell, and an understanding of the behavior of the membranes that compartmentalize and enclose the cell is therefore critical for rationalizing these processes. Biological membranes are complex structures formed mostly by lipids and proteins. For this reason lipid bilayers have received a lot of attention both computationally and experimentally for many years.[95, 96] The vital role of cell membranes is underlined by the estimation that over half of all proteins interact with membranes, either transiently or permanently.[97] Further, G protein-coupled receptors embedded in the membrane account for 50–60% of present day drug targets, and membrane proteins as a whole make up around 70%.[98] Even so, only 685 resolved unique structures of membrane embedded proteins, out of a total of 65 500 searchable entries (after removing redundant structures), exist in the Protein Data Bank (April 2017) reflecting the difficulties in studying membrane-associated proteins experimentally, making them prime targets for simulation.

Prior to 2012, the only force field parameters for lipids distributed with AmberTools were part of the Glycam force field and were limited in scope.[89] Traditionally, lipid simulations with Amber have either employed the Charmm parameters, via support for the Charmm force fields through the Chamber package[99] or through attempts to adapt the General Amber Force Field (GAFF) with limited success[100].

In 2012, Amber greatly expanded support for simulation of lipids. This included the development of a modular framework for lipid simulations and initial parameterization within the *LIPID11* force field[101] as well as a careful refinement of the non-bonded parameters and associated torsion terms within the GAFF force field for specific application to lipids.[102] The latter, *GAFFLipid*, was the first lipid parameter set based on the Amber force field equation to support simulation of lipid bilayers in the tensionless NPT ensemble while the former, *LIPID11*, provided the first modular framework for constructing lipid simulations analogous to the Amber amino and nucleic acid force fields. Together these developments have made simulation of phospholipids with AMBER substantially easier. *LIPID14* was released in 2014 [103] and represented a major advancement over the previous Amber compatible lipid force fields for lipid bilayer simulations in the NPT ensemble without the need for an artificial constant surface tension term. Validation of the *LIPID14* parameters were provided through extensive self-assembly simulations [104, 105]. Inclusion and validation of parameters for cholesterol [106] represented an important addition to the lipid parameter set, allowing even more complex lipid containing systems to be simulated. *LIPID17* built upon the modularity of *LIPID14* and provided an extension of modular phospholipid residues to include anionic head groups and polyunsaturated tails. The latest *LIPID* force field for AMBER is *LIPID21*[107] which builds upon the modularity of *LIPID14* and *LIPID17* and provides an extension of modular phospholipid residues to include anionic head groups, polyunsaturated tails and sphingomyelin. As part of the refinement from *LIPID14* the bonded alkane parameters have been revised and updated by fitting to quantum energies. Furthermore, new partial charges have been generated for all the head group residues in order to accommodate the anionic head groups whilst maintaining consistency in the charge derivation approach. Details regarding the parameterization are given in Dickson et al. [107]. The modular nature of the force field allows for many combinations of lipid head and tail groups as well as rapid and standardized parameterization of additional lipids. *LIPID21* was validated through bilayer simulations of twenty different phospholipid types, for a total of 0.9 microseconds each without applying a surface tension or constant area term. The lipid bilayer structural features compare favorably with experimental measures such as area per lipid, bilayer thickness, NMR order parameters and scattering data.

A word of caution regarding barostat and cut off selection with lipid simulations. It is well known that lipids can be very sensitive to simulation conditions. Previous advice has been to use the Berendsen barostat and a 10Å cutoff when simulating lipids with the AMBER Lipid force fields. This was due to bilayer deformation that was regularly seen with simulations run using the Monte Carlo Barostat. Recent work by Gomez et al.[108] investigated this behavior and determined that there are issues when using the MC barostat with hard Lennard-Jones cutoffs. The issue affects all simulations but is most obvious when simulating lipid bilayers. Gomez et al recommend use of an LJ force switch when running simulations with the MC barostat.

3.5.1. LIPID21: The Amber lipid force field

```
leaprc.lipid21    defines atom types and loads the files below
```

	Description	LIPID21 Residue Name
Acyl chain	Lauroyl (12:0)	LAL
	Myristoyl (14:0)	MY
	Palmitoyl (16:0)	PA
	Sphingosine (16:1)	SA
	Oleoyl (18:1 n-9)	OL
	Stearoyl (18:0)	ST
	Arachidonoyl (20:4)	AR
	Docosaheptaenoyl (22:6)	DHA
Head group	Phosphatidylcholine	PC
	Phosphatidylethanolamine	PE
	Phosphatidylserine	PS
	Phosphatidylglycerol (R-)	PGR
	Phosphatidylglycerol (S-)	PGS
	Phosphatidic acid	PH-
	Sphingomyelin	SPM
Other	Cholesterol	CHL

Table 3.9.: *LIPID21 residue names.*

<code>lipid21.lib</code>	atoms, charges, and topologies for LIPID21 residues
<code>lipid21.dat</code>	LIPID21 force field parameters

The LIPID21 force field represents the logical next step in the development of an Amber lipid force field building on the modular nature of LIPID11[101], LIPID 14 [103] and LIPID 17 to allow for tensionless lipid bilayer simulations in Amber. LIPID21[107] has been designed to be fully compatible with the other pairwise-additive protein, nucleic acid, carbohydrate, and small molecule Amber force fields.

LIPID21 is a modular force field for the simulation of phospholipids and cholesterol. To achieve this modularity phospholipids are divided into interchangeable head group and tail group "residues."

Currently, there are eight tail group residues and six head group residues supported, as well as cholesterol, and LEaP supports any combination of these lipid residues. The supported LIPID21 residues and their residue names are listed in Table 3.9. LIPID21 can be used alone or in conjunction with other Amber force fields. The order with which the various AMBER force fields are loaded along with LIPID21 should not matter. For example, to load ff19SB and LIPID21 in LEaP use:

```
source leaprc.protein.ff19SB
source leaprc.lipid21
```

Due to the significant improvements in fidelity offered by LIPID21 we do not recommend the continued use of previous LIPID force fields. As such LIPID11, 14 and 17 have been deprecated to the oldff/ directory.

LIPID21 PDB format

LIPID21 atom names and types are defined in Skjevik, et al[101], Dickson, et al[103], Madej et al[106] and Dickson, et al.[107]

A properly formatted lipid PDB can be loaded into LEaP. Each phospholipid molecule in LIPID21 is made up of three residues. Atoms from each residue must be in contiguous blocks and ordered as described below in each molecule. A TER card must be appended after all the atoms for each molecule. Table 3.10 specifies the residue format for the PDB file loaded by LEaP in order to correctly define linker atoms.

The connectivity (CONNECT records) section of the PDB is redundant and should be removed prior to loading into LEaP. The head group and tail residues are linked together by the LEaP program after loading the lipid PDB file.

PDB formatted structure files with alternative residue and atom names (such as Charmm C36) may be converted to the LIPID21 naming convention by way of the script called *charmmlipid2amber.py* which is supplied with AmberTools to convert Charmm C36 residue and atom names to LIPID21 nomenclature.

Lipid 1	sn-1 tail residue head group residue sn-2 tail residue TER card
Lipid 2	sn-1 tail residue head group residue sn-2 tail residue TER card
...	...

Table 3.10.: *LIPID21 PDB format for LEaP*

```
charmm lipid2amber.py -i charmm_c36.pdb -o output lipid21.pdb
```

Additionally, membrane systems can be prepared by means of the *packmol-memgen* included software (13.9).

3.6. Solvents

```
leaprc.water.<type> loads solvents.lib and the appropriate frcmod file
solvents.lib        library for water, methanol, chloroform, NMA, urea
frcmod.tip4p         Parameter changes for TIP4P.
frcmod.tip4pew       Parameter changes for TIP4PEW.
frcmod.tip5p         Parameter changes for TIP5P.
frcmod.spce          Parameter changes for SPC/E.
frcmod.spceb         Parameter changes for SPC/Eb.
frcmod.opc           Parameter changes for OPC.
frcmod.opc3          Parameter changes for OPC3.
frcmod.opc3pol       Parameter changes for OPC3-pol.
frcmod.pol3          Parameter changes for POL3.
frcmod.tip3pfb       Parameter changes for the force-balance TIP3P model
frcmod.tip4pfb       Parameter changes for the force-balance TIP4P model

frcmod.meoh          Parameters for methanol.
frcmod.chcl3         Parameters for chloroform.
frcmod.nma           Parameters for N-methylacetamide.
frcmod.urea          Parameters for urea (or urea-water mixtures).
```

Amber provides direct support for several water models.

There is no default, but TIP3P[109] will be used for residues with names HOH or WAT, following a long tradition. Despite the fact that many properties of this old water model deviate significantly from those of real water, the model has an impressive track record and is still a popular choice in biomolecular simulations. There is more than one good reason behind this tenacity other than simple inertia[21]. In particular, many older force fields were parametrized in simulations that used TIP3P as the solvent: errors in the solvent part of the total energy are compensated, to an extent, by fitted parameters of the gas phase (solute) part. As a result, many existing force fields are inherently biased towards TIP3P to various degrees. Replacing TIP3P with another water model without re-parametrizing the underlying gas-phase force field may not necessarily lead to better accuracy of the biomolecular simulation that might be expected to benefit from the more accurate water model. Fortunately, AMBER force fields are not very strongly biased towards any specific water model, which makes the task of testing new models easier. In recent years several new models appeared that describe the state of liquid water much more accurately than TIP3P, these models showed significant improvements in outcomes of many types of biomolecular simulations, even with older force fields. A recent addition to AMBER family of protein force fields, ff19SB[23], was developed without an inherent bias towards a water model; OPC is recommended for use with this force field[23].

If you want to use water models other than TIP3P, execute the following LEaP commands after loading your leaprc file:

```
WAT = OPC (residues named WAT in pdb file will be OPC)
source leaprc.water.opc
```

(The above is obviously for the OPC model.) The *solvents.lib* file contains TIP3P,[109] TIP3P/F,[110] TIP4P,[109, 111] TIP4P/Ew,[112, 113] TIP5P,[114] OPC,[24] OPC3,[115] OPC3-pol,[116] POL3,[117] SPC/E,[118] SPC/Eb,[33] TIP3PFB,[37] and TIP4PFB[37] models for water; these are called TP3, TPF, TP4, T4E, TP5, OPC, OP3, O3P, PL3, SPC, SPC, FB3 and FB4, respectively. (The SPC/E and SPC/Eb models are both called SPC: you just have to be sure to load the appropriate frcmod file.) By default, the residue name in the prmtop file will be WAT, regardless of which water model is used.

The “standard” leaprc files for *tip3p*, *spce*, *tip4pew* and *opc* also load the Joung/Cheatham monovalent ion parameters (see below). If you wish to use other parameters, or to deal with divalent or other ions, you will need to load the appropriate frcmod files.

Amber has two flexible water models, one for classical dynamics, SPC/Fw[119] (called “SPF”) and one for path-integral MD, qSPC/Fw[120] (called “SPG”). You would use these in the following manner:

```
WAT = SPG
loadAmberParams frcmod.qspc/fw
set default FlexibleWater on
```

Then, when you load a PDB file with residues called WAT, they will get the parameters for qSPC/Fw. (Obviously, you need to run some version of quantum dynamics if you are using qSPC/Fw water.)

The *solvents.lib* file, which is automatically loaded with many leaprc files, also contains pre-equilibrated boxes for many of these water models. These are called POL3BOX, QSPCFWBOX, SPCBOX, SPCFWBOX, TIP3PBOX, TIP3PFBBOX, TIP4PBOX, TIP4PEWBOX, OPCBOX, OPC3BOX, OPC3POLBOX, and TIP5PBOX. These can be used as arguments to the *solvateBox* or *solvateOct* commands in LEaP.

In addition, non-polarizable models for the organic solvents methanol, chloroform and N-methylacetamide are provided,[121] along with a box for an 8M urea-water mixture. The input files for a single molecule are in *\$AMBERHOME/dat/leap/prep*, and the corresponding frcmod files are in *\$AMBERHOME/dat/leap/parm*. Pre-equilibrated boxes are in *\$AMBERHOME/dat/leap/lib*. For example, to solvate a simple peptide in methanol, you could do the following:

```
source leaprc.protein.ff14SB (get a standard force field)
loadAmberParams frcmod.meoh (get methanol parameters)
peptide = sequence { ACE VAL NME } (construct a simple peptide)
solvateBox peptide MEOHBOX 12.0 0.8 (solvate the peptide with meoh)
saveAmberParm peptide prmtop prmcrd
quit
```

Similar commands will work for other solvent models.

3.6.1. The OPC family of water models

OPC is a non-polarizable, 4-point, 3-charge rigid water model.[24] Geometrically, it resembles TIP4P-like models, although the values of OPC point charges and charge-charge distances are quite different. The model has a single VDW center on the oxygen nucleus. The model is constructed based on the concept of optimal point charge approximation; [122] the central idea of OPC is to distribute the point charges to best reproduce the 3 lowest order multipole moments of water molecule in liquid phase. The optimal values for the dipole μ and the square quadrupole moment Q_T [123] are determined as best fit values that reproduce key experimental properties of water in liquid phase. The low dimensionality of the parameter space μ - Q_T permits a virtually exhaustive search for the global optimum; the global optimization sets this water model family apart from the others. The linear quadrupole and the octupole moments[124] are fixed to values obtained from high quality QM calculations.[123]

A full description of OPC and its properties can be found in Ref.[24]. For 11 key liquid state properties against which water models are most often benchmarked, OPC is on average within 0.76% of the experiment (relative

3. Molecular mechanics force fields

error). This accuracy is dramatically better compared to the commonly used rigid models. For example, the dielectric constant of TIP3P and TIP4PEw is 94 and 63.9 respectively, while OPC predicts it to be 78.4 ± 0.6 (the experimental value is 78.4). The reported OPC properties were computed using Amber 12 on GPUs with a time-step of 2 fs, periodic boundary conditions, an 8 angstrom cut-off for nonbonded interactions, and PME for long range electrostatics. SHAKE was used to constrain hydrogens. The rest of parameters are set to current Amber defaults; note that these include accounting for the van der Waals interactions beyond the cut-off via a continuum model (vdwmeth=1).

OPC in biomolecular simulations: Because of the improved accuracy in bulk properties, OPC delivers noticeable accuracy improvement in practical biomolecular simulations, even with existing force-fields. Specifically, OPC was found to yield quantitative agreement with NMR experiment for conformational populations of small RNA fragments,[57, 125, 126] and therefore is a commonly used water model for RNA simulations. [127–129] OPC has been shown to improve structural description of DNA duplex,[68] DNA G-quadruplex, [130] thermodynamics of ligand binding,[131] small molecule hydration,[24] rotational dynamics of proteins, [132] simulations of lipid monolayer, [133] and intrinsically disordered proteins.[48, 134]

Ion parameters for OPC: Four nonbonded parameter sets (the 12-6 normal usage set, 12-6 HFE set, 12-6 IOD set, and 12-6-4 set) for various ions in conjunction with the OPC water model have been developed by Li, Merz and co-workers;[135–137] see Section 3.7 for the definition and important usage suggestions. Additional OPC-specific ion parameters have also been reported recently.[138]

Based on our limited experience, it appears that the Joung/Cheatham ion parameters for TIP4P-EW (jc_tip4pew)[139] may also be acceptable for OPC water model, especially when accurate reproduction of IODs is critical. This set has already been tested in practice with OPC model.[57, 131]

OPC3 water model: OPC3 – a 3-point rigid non-polarizable water model – is the latest addition to the family, constructed using the same philosophy as OPC. Further details are available in Ref.[115]. Briefly, OPC3 is significantly more accurate than the commonly used water models of same class (TIP3P, SPC/E) in reproducing a comprehensive set of liquid bulk properties, over a wide range of temperatures. Relative to the 4-point OPC, OPC3 is somewhat less accurate compared to experiment. Similar to the OPC water model, four nonbonded parameter sets for various ions in conjunction with the OPC3 water model (the 12-6 normal usage set, 12-6 HFE set, 12-6 IOD set, and 12-6-4 set) have also been developed by Li, Merz and co-workers;[135–137] see Section 3.7 for the definition and important usage suggestions. Moreover, the Joung/Cheatham ion parameters previously developed for TIP3P may also be used with OPC3.

3.6.1.1. OPC3-pol: fast polarizable water model

OPC3-pol is a new classical 3-point water model that explicitly accounts for electronic polarizability with minimal impact on computational efficiency.[116] The model is based on the Drude oscillator concept, with several significant modifications compared to existing models [140] of this kind. Parameters of OPC3-Pol have been globally optimized to match experiment, just like other water models of the globally optimal OPC family.[24, 115] OPC3-pol reproduces five key bulk water properties at room temperature, with an average relative error of 0.6%.

At the moment, the intended use of OPC3-Pol is in long classical atomistic simulations where water polarization effects are known or expected to be very important. Likewise, studies that aim at investigating effects of water polarization itself can benefit from the computationally efficient OPC3-pol. Based on our limited experience, globular proteins and dsDNA should be fine.

Efficiency: Compared to existing polarizable water models employed in atomistic biomolecular simulations, OPC3-pol has two key advantages. First, its relative speed: OPC3-pol's computational efficiency is near that of fixed-charge TIP3P (in-between that of TIP3P and TIP4P-Ew); OPC3-Pol supports increased (4 fs) integration time step with HMR. Second, OPC3-pol is intended to be used with existing non-polarizable force-fields, e.g. ff14SB or ff19SB; no specialized polarizable force-field is required.

Use in MD simulations: So far, OPC3-pol has been tested in simulations of a globular protein (ubiquitin) and a B-DNA dodecamer with recent AMBER force-fields, ff99SB, ff14SB, ff19SB, and OL15, respectively, demonstrating structure stability close to X-ray reference on multi-microsecond time-scale. With ff14SB, the ubiquitin structure was marginally more stable than with ff19SB, with an average RMSD at 1.0 Å for ff14SB compared to 1.2 Å for ff19SB. On a B-DNA dodecamer with OL15, the simulated structure matched the crystal structure within 1.5 Å backbone RMSD (excluding terminal nucleotides), and the widths of its major and minor

grooves agreed well with the experimental reference.

Known limitations: The main limitations of the model stem from its main design choice of keeping it as simple as possible. The goal was to deliver a fast and reasonably accurate polarizable water model, easily accessible for use in biomolecular simulations; thus compromises were made. In particular, this is a 3-point model, which inherits a number of incorrigible ills of these simplest models.[21, 141] For example, the accuracy of OPC3-pol in reproducing liquid water properties is lower than that of 4-point OPC, one should not expect accuracy miracles from simulations that employ OPC3-pol. The model parameters have been optimized for ambient conditions, some of its liquid water properties deviate from experiment away from the 300 K target. In particular, the density maximum is off; we do not recommend using the model far outside the ~300 K, ~1 bar ambient conditions. OPC3-pol has not yet been tested nearly as extensively in biomolecular simulations as other OPC family models. In particular, while its performance on an intrinsically disordered protein showed some promise, more extensive testing is needed for a definitive conclusion on whether OPC3-pol should be used outside of the domain of globular structures.

SETUP: The basic set-up is the same as for any other water model. Model residue name: **O3P**. To load it in LEaP:

```
WAT = O3P
source leaprc.water.opc3pol
```

In the absence of OPC3-pol specific ion models, we cautiously recommend Li/Merz ion parameters for +1 and -1 ions in OPC water (12-6 HFE set):

```
loadAmberParams frcmod.ions11m_126_hfe.opc
```

which we have used in simulations described above.

USE WITH 4 fs time-step: For that, hydrogen mass repartitioning (HMR)[142] is required, see the two steps below.

1) Load the extra frcmod file in LEaP (after the "source leaprc.water.opc3pol" command):

```
loadAmberParams frcmod.opc3pol_HMR4fs
```

Loading this frcmod file sets up HMR for OPC3-pol water in the system. In detail, it increases H atom mass to 2.008 Da and reduces O atom and Drude atom mass to 7.0 Da in all OPC3-pol water molecules.

2) After the LEaP step, call "HMR" command in *ParmEd* to set up HMR for non-water molecules in the system. Note that the "dowater" argument for "HMR" should **not** be invoked, otherwise OPC3-pol water's HMR configuration set up in the first step will be overwritten.

3.7. Ions

frcmod.ionsjc_tip3p	Joung/Cheatham ion parameters for TIP3P water
frcmod.ionsjc_spce	same, but for SPC/E water
frcmod.ionsjc_tip4pew	same, but for TIP4P/EW water
frcmod.ions11m_126_tip3p	Li/Merz ion parameters for +1 and -1 ions in TIP3P water (12-6 normal usage set)
frcmod.ions11m_126_spce	same, but in SPC/E water
frcmod.ions11m_126_tip4pew	same, but in TIP4P/EW water
frcmod.ions11m_iod	Li/Merz ion parameters for +1 and -1 ions in TIP3P, SPC/E, and TIP4P/EW waters (12-6 IOD set)
frcmod.ions2341m_126_tip3p	Li/Merz ion parameters for +2 to +4 ions in TIP3P water (12-6 normal usage set)
frcmod.ions2341m_126_spce	same, but in SPC/E water
frcmod.ions2341m_126_tip4pew	same, but in TIP4P/EW water
frcmod.ions2341m_hfe_tip3p	Li/Merz ion parameters for +2 to +4 ions in TIP3P water (12-6 HFE set)
frcmod.ions2341m_hfe_spce	same, but in SPC/E water
frcmod.ions2341m_hfe_tip4pew	same, but in TIP4PEW water
frcmod.ions2341m_iod_tip3p	Li/Merz ion parameters for +2 to +4 ions in TIP3P water (12-6 IOD set)
frcmod.ions2341m_iod_spce	same, but in SPC/E water
frcmod.ions2341m_iod_tip4pew	same, but in TIP4P/EW water
frcmod.ions11m_1264_tip3p	Li/Merz ion parameters for -1 and +1 ions in TIP3P water (12-6-4 set)

3. Molecular mechanics force fields

frcmmod.ions11m_1264_spce	same, but in SPC/E water
frcmmod.ions11m_1264_tip4pew	same, but in TIP4PEW water
frcmmod.ions2341m_1264_tip3p	Li/Merz ion parameters for +2 to +4 ions in TIP3P water (12-6-4 set)
frcmmod.ions2341m_1264_spce	same, but in SPC/E water
frcmmod.ions2341m_1264_tip4pew	same, but in TIP4PEW water
frcmmod.ions1m_126_opc3	Li/Merz ion parameters for -1 to +4 in OPC3 water (12-6 normal usage set)
frcmmod.ions1m_126_opc	same, but in OPC water
frcmmod.ions1m_126_fb3	same, but in TIP3P-FB water
frcmmod.ions1m_126_fb4	same, but in TIP4P-FB water
frcmmod.ions1m_hfe_opc3	Li/Merz ion parameters for -1 to +4 in OPC3 water (12-6 HFE set)
frcmmod.ions1m_hfe_opc	same, but in OPC water
frcmmod.ions1m_hfe_fb3	same, but in TIP3P-FB water
frcmmod.ions1m_hfe_fb4	same, but in TIP4P-FB water
frcmmod.ions1m_iod_opc3	Li/Merz ion parameters for -1 to +4 in OPC3 water (12-6 IOD set)
frcmmod.ions1m_iod_opc	same, but in OPC water
frcmmod.ions1m_iod_fb3	same, but in TIP3P-FB water
frcmmod.ions1m_iod_fb4	same, but in TIP4P-FB water
frcmmod.ions1m_1264_opc3	Li/Merz ion parameters for -1 to +4 in OPC3 water (12-6-4 set)
frcmmod.ions1m_1264_opc	same, but in OPC water
frcmmod.ions1m_1264_fb3	same, but in TIP3P-FB water
frcmmod.ions1m_1264_fb4	same, but in TIP4P-FB water
atomic.ions.lib	topologies for monoatomic ions (new naming scheme)
ions94.lib	topologies for ions with the old naming scheme

In 2008, Joung and Cheatham created a consistent set of parameters for alkali halide ions, fitting solvation free energies, radial distribution functions, ion-water interaction energies and crystal lattice energies and lattice constants for non-polarizable spherical ions.[139, 143] These have been separately parametrized for each of three popular water models, as indicated above.

Li, Merz and co-workers subsequently developed ion parameters for the monovalent, divalent, trivalent and tetravalent ions for the 12-6 LJ nonbonded model and the 12-6-4 LJ-type nonbonded model in conjunction with seven different water models (TIP3P, SPC/E, TIP4PEW, OPC3, OPC, TIP3P-FB, and TIP4P-FB) for PME simulations.[135–137, 144–147] The experimental values they tried to reproduce are the experimental Hydration Free Energy (HFE) values, Ion-Oxygen Distance (IOD) values and Coordination Number (CN) values of the first solvation shell. It was found that it is hard to reproduce the three experimental values simultaneously by using the 12-6 LJ nonbonded model. Since the charge-induced dipole interaction is proportional to r^{-4} , a new term with format $(C/r)^4$ was added to the 12-6 LJ potential, yielding a 12-6-4 LJ-type potential. The new potential with designed parameters could reproduce the experimental HFE, IOD and CN values at the same time without significant compromise. Especially for the highly charged metal ions, the 12-6-4 LJ-type nonbonded model performs much better than the 12-6 one overall. Similar to Joung and Cheatham’s work, water models were treated separately for the parameter design, as indicated in the name of frcmmod files. Users can check the notes in the frcmmod files to see the reference of each parameter.

For the 12-6 LJ nonbonded model, three different parameter sets are available for each water model to meet different requirements:

1. 12-6 normal usage set. This contains the HFE set of the monovalent ions (which could reproduce the experimental HFE),[136, 147] the Compromise (CM) set of divalent ions (which could reproduce the experimental relative HFE and CN values),[135, 145] and the IOD set (which could reproduce the experimental IOD) for the trivalent and tetravalent ions.[137, 146] **These parameters are recommended to be used in the normal MD simulations.** This is because for the monovalent ions the error of the 12-6 LJ nonbonded model is pretty small (a CM set may not be needed since the HFE or IOD sets are pretty close to each other) while for the trivalent and tetravalent metal ions the 12-6 LJ nonbonded model has relatively big errors (a CM set could have big errors for both HFE and IOD at this moment).
2. 12-6 HFE set to reproduce experimental HFE.[135–137, 144, 146, 147] The HFE parameter set has limited error for monovalent ions, while could have remarkable error for highly charged ions. Since we use the HFE set for monovalent ions in the 12-6 normal usage set, we don’t have a specific HFE set parameter file

for monovalent ions. In addition, an old HFE set of parameters for OPC water models were developed by Pengfei Li previously only for Na^+ , K^+ , and Cl^- ions (personal communications with Alexey Onufriev), and this parameter set can be found in a recent publication by the Onufriev group.[148] This parameter set is not available in the current version of AMBER, and has a small difference from the current HFE parameter set for OPC in AMBER. Herein the reference is provided in case anyone wants to use this old parameter set for reproducibility sake or other purposes.

3. 12-6 IOD set to reproduce experimental IOD.[135–137, 144, 146, 147] Since the ion with certain parameter could reproduce similar IOD values in the three water models, so the IOD set parameters of three water models were designed identical (for the monovalent and divalent metal ions, while for the trivalent and tetravalent ions, the IOD set are estimated for each water model separately). **The IOD parameter set are recommended to be used in the structural refinement or for structural property orientated investigation.**

For the 12-6-4 LJ-type nonbonded model, only one parameter set (12-6-4 set) designed for each of the three water models. The 12-6-4 model has also been tested in mixed systems (such as nucleic acids, proteins and ionic solutions) and have shown excellent transferability.[135–137, 145–147] The 12-6-4 model has been shown to give greatly improved structural, thermodynamic, kinetic and mass transport properties for Mg^{2+} in water relative to the 12-6 model..[149] The 12-6-4 model with the SPC/E water model performed exceptionally well for simulating all properties in these benchmark calculations.[149] The parameters which are specifically designed for the divalent metal ions with 12-6-4 LJ-type nonbonded model are shown as the 12-6-4 set above. These frcmod files can be used to generate an original prmtop file. **After obtaining the original prmtop file, one should use the `add12_6_4` command in `parmed` to generate a prmtop with the additional C_4 terms with the flag `LENNARD_JONES_CCOEF`.** Please see the `add12_6_4` command 15.2.2.6 in Subsection 15.2.2 in the manual for detailed information. After obtaining the prmtop with the additional C_4 term, you can use `sander` or `pmemd` to run the simulation.

Panteva *et al.* have also fine-tuned the C_4 terms between several divalent metal ions (Mg^{2+} , Mn^{2+} , Zn^{2+} , and Cd^{2+}) and nucleic acid systems[150] while keeping the C_4 terms between metal ions and water designed by Li and Merz.[145] The new parameter set arguably provides a better balance the interaction types in nucleic acid systems, has been shown to be predictive in identifying metal ion binding sites in nucleic acids[151], and is recommended to use in related modeling. Moreover, the 12-6-4 model showed its ability to well simulate the chelate effect[152] and the thermodynamics of metal ion binding in a metalloprotein.[153] A tutorial about the 12-6-4 model is here: ambermd.org/tutorials/advanced/tutorial20/12_6_4_.php.

Moreover, Li has shown that it is possible to simultaneously reproduce the HFE, IOD, and coordination number (CN) values of metal ions by using a 12-6 model with adjusting the atomic charges of the first solvation shell water molecules.[154] The relationship between the C_4 parameter and induced dipole moment was also derived. This study used a strategy similar to the fluctuating charge model, hence it can be considered as a work bridging the 12-6-4 model and the fluctuating charge model.

3.8. Force fields related to semi-empirical QM

ParmAM1 and **parmPM3** are classical force field parameter sets that reproduce the geometry of proteins minimized at the semi-empirical AM1 or PM3 level, respectively.[155] These new force fields provide an inexpensive, yet reliable, method to arrive at geometries that are more consistent with a semi-empirical treatment of protein structure. These force fields are meant only to reproduce AM1 and PM3 geometries (warts and all) and were not tested for use in other instances (e.g., in classical MD simulations, etc.) Since the minimization of a protein structure at the semi-empirical level can become cost-prohibitive, a “preminimization” with an appropriately parametrized classical treatment will facilitate future analysis using AM1 or PM3 Hamiltonians.

3.9. The GAL17 force field for water over platinum

<code>leaprc.music</code>	Adds atom types and loads <code>music.lib</code> and <code>music.dat</code>
<code>music.lib</code>	Library for metal surface atoms, virtual sites, and Drude rod particles.

3. Molecular mechanics force fields

```
music.dat Parameters for metal surface, Drude rod particles and LJ terms with water.
```

The GAL17 force field[156] was developed as part of the MuSiC project (Multiscale Simulations in Catalysis) to describe the interaction of water and a Pt(111) surface. The GAL17 force field is implemented in the *sander* program and can be combined with any water model. It provides a significant improvement over previously existing force fields for Pt(111)/water interactions. Its well-balanced performance suggests that it is an ideal candidate to generate relevant geometries for the metal/water interface, paving a way to a representative sampling of the equilibrium distribution at the interface and to predict solvation free energies at the solid/liquid interface. At present only parameters for water over Pt(111) are available, however, the force field is extensible to other metal surface and solutes such as alcohols or sugar molecules that are typical substrates in catalytic upgrading of biomass extracts. The GAL17 force field consists of

- A Lennard-Jones term between Pt atoms and water oxygen atoms that describes physisorption of water at the surface.
- A polarized Gaussian term between Pt surface atoms and water oxygen atoms that describes chemisorption at Pt top sites.
- Two terms that describe the angular dependence of the water/Pt surface interaction energy.

The GAL17 force field thus does not include explicit terms to describe image charge interactions, that is electrostatic interactions between charged particles and a metallic conductor, explicitly. Instead these effects are included implicitly. In addition, it has been shown that image charge interactions account for less than 10% of the interaction energy for water adsorbed at a Pt(111) surface[157]. Although not employed in GAL17, the music force field library does contain parameters for a symmetric Drude rod model[157] that can be employed to investigate image charge effects.

In GAL17 the platinum surface atoms have atom name Pt and residue name MET. The platinum surface must be perpendicular to one of the Cartesian coordinate axes. Water molecules must be above the surface (coordinate values larger than the metal atoms). Given a properly formatted pdb file that contains a platinum metal surface and water molecules, one would use the GAL17 force field with TIP3P water in the following manner:

```
source leaprc.music
source leaprc.water.tip3p
ptwat = loadpdb ptwat.pdb
saveAmberParm ptwat prmtop inpcrd
```

This will load the correct LJ parameters between platinum and water oxygen atoms. In addition, one needs to activate the Gaussian and angle adsorption correction terms via the *&music* namelist. This namelist also provides an option to define the orientation of the surface plane. All force field parameters can be controlled via this namelist, advanced users may want to look into the source code file *music_module.F90* for all available options. At present there are no good parameters for platinum metal and simulations must therefore constrain the position of the platinum atoms. This can be conveniently achieved with *belly* dynamics. A typical input would thus contain

```
&cntrl
...
ibelly = 1,                ! constrain atom positions
bellymask = '@O,H1,H2'    ! let water molecules move
/
&music
pt_plane = 'yz'           ! default is 'xy', i.e. surface in xy plane
/
```

When running simulations with *sander* in parallel, it may be advisable to orient the metal surface in the yz plane to achieve better load balancing with the algorithm that is used by *sander* to distribute work across MPI tasks. Tests that may serve as examples how to build input files and run simulations with GAL17 are contained in directory *\$AMBERHOME/test/sander_music/*.

3.10. Fluorescent dyes: AMBER-DYES in AMBER force field files

<code>leaprc.amberdyes</code>	defines atom types and loads the files below
<code>amberdyes.lib</code>	atoms, charges, and topologies for dye and linker residues
<code>amberdyes.dat</code>	AMBER-DYES in AMBER force field parameters

The AMBER-DYES force field parameters[158] were modified and implemented into the AMBER Software Suite[159]. The modifications were performed for all Cystein-ending linkers to fix an issue [160] existing in the original dye parameters[158]. The chirality of the Cystein-ending linkers is in R-configuration, but can be easily changed via the “flip” command in cpptraj. Further modifications were performed for the dye “Alexa Fluor 647” to remove the discordance between the structure used in the original parameters [158] and the commercially available one [161]. The original dye structure can still be manually loaded by using the `amberdyes.org.lib` file. Fluorescence ligands, so-called dyes, are widely used to investigate protein structures and dynamics, such as conformational changes, folding, association and dissociation of complexes, and enzymatic cycles. Dyes are usable with multi-protein and single-protein systems. MD simulations with explicit dyes can improve the interpretation of experimental results. Especially in Forster Resonance Energy Transfer (FRET) experiments, it is of utmost importance to obtain precise information about the position and orientation of the dyes.

At the moment AMBER-DYES in AMBER covers 22 commonly used dyes and 6 linkers (see table below):

Dye	Residue name	Linker residue
Alexa Fluor 350	A35	C1R, L1R
Alexa Fluor 488	A48	B1R, C1R, L1R
Alexa Fluor 532	A53	C1R, L1R
Alexa Fluor 568	A56	C1R, L1R
Alexa Fluor 594	A59	C1R, L1R
Alexa Fluor 647	A64	B1R, C2R, L1R
Lumiprope Cy3	C3N	C2R, L1R
Lumiprope Sulfo-Cy3	C3W	L1R
Lumiprope Cy5	C5N	C2R, L1R
Lumiprope Sulfo-Cy5	C5W	L1R
Lumiprope Cy5.5	C55	C2R, L1R
Lumiprope Cy7	C7N	L1R
Lumiprope Cy7.5	C75	L1R

Dye	Residue name	Linker residue
ATTO 390	T39	C2R, L1R
ATTO 425	T42	C2R, L1R
ATTO 465	T46	C2R, L1R
ATTO 488	T48	C3R, L2R
ATTO 495	T49	C2R, L1R
ATTO 514	T51	C3R, L2R
ATTO 520	T52	C2R, L1R
ATTO 610	T61	C2R, L1R
ATTO Thio12	Tth	C3R, L2R

Table 3.11.: AMBER-DYES in AMBER residue names.

To attach a linker / dye combination to your structure, hand-edit your PDB file, similarly to 3.2.3, and choose an attachment point (e.g. residue 3):

ATOM	16	ND2	ASN	E	2	3.872	30.857	39.020	1.00	13.86	N
ATOM	17	N	ILE	E	3	5.739	34.298	36.056	1.00	14.08	N
ATOM	18	CA	ILE	E	3	4.144	36.258	39.575	1.00	7.14	C
ATOM	19	C	ILE	E	3	5.305	36.089	40.541	1.00	9.18	C
ATOM	20	O	ILE	E	3	5.662	37.000	41.282	1.00	12.86	O
ATOM	21	CB	ILE	E	3	4.933	36.389	35.001	1.00	13.23	C
ATOM	22	CG1	ILE	E	3	5.138	37.899	35.089	1.00	11.53	C
ATOM	23	CG2	ILE	E	3	3.449	36.064	35.230	1.00	12.95	C
ATOM	24	CD1	ILE	E	3	6.522	38.291	34.603	1.00	11.29	C
ATOM	25	N	PHE	E	4	4.507	35.854	38.224	1.00	11.91	N

Change the residue name (ILE) of the CA atom to the linker residue name (e.g. C1R) and delete the rest of the residue:

3. Molecular mechanics force fields

ATOM	16	ND2	ASN	E	2	3.872	30.857	39.020	1.00	13.86	N
ATOM	18	CA	C1R	E	3	4.144	36.258	39.575	1.00	7.14	C
ATOM	25	N	PHE	E	4	4.507	35.854	38.224	1.00	11.91	N

Append your PDF file with the C99 atom of your dye (e.g. Alexa Fluor 488) after the TER card:

ATOM	1317	N	ASN	E	163	19.398	31.025	41.679	1.00	38.17	N
TER	1318		ASN	E	163						
ATOM	1319	C99	A48	E	164						

Use LEaP to load the AMBER-DYES in AMBER force field (at best by sourcing `leaprc.amberdyes`, load your updated PDB file, set a bond between the dye (always atom C99) and linker (always atom N99), and relax the structure:

```
source leaprc.amberdyes
pdb = loadpdb 1481.pdb
bond pdb.A48.C99 pdb.C1R.N99
select pdb.A48
select pdb.C1R
relax pdb
saveAmberParm pdb prmtop inpcrd
```

Additional settings are subject to personal preference. LEaP will produce a structure with a bonded dye usable for MD simulations. Do, however, check the generated structure for sanity before using it.

3.10.1. Fluorescent chromophores

Additional parameters for six common fluorescent protein chromophores—eGFP, eBFP, eYFP, eCFP, DsRed, and mCherry—are available[162] by sourcing `leaprc.xFPchromophores` after sourcing the main force field `leaprc` file (e.g. `leaprc.protein.ff19SB`). This will allow seamless loading of PDB files containing fluorescent proteins provided they follow standard residue naming of the chromophore: eGFP=CRO, eBFP=IIC, eYFP=CR2, eCFP=CRF, DsRed=CRQ, and mCherry=CH6. The chromophore parameters are based on parm10 with the ff19SB modifications, but also borrow heavily from GAFF. Both uppercase and lowercase atom types are utilized, so users should take caution if mixing ff19SB with GAFF. See original reference[162] for details of implementation.

3.11. Coarse-grained and multiscale simulations using the SIRAH force field

The Southamerican Initiative for a Rapid and Accurate Hamiltonian (SIRAH) is a residue-based Coarse-Grained (CG) force field compatible with multiscale simulations. It was fitted to reproduce structural properties of biomolecules, granting a speed up of nearly 2 orders of magnitude in comparison to all-atom simulations, with a reasonable compromise on accuracy. Currently, it includes parameters for DNA,[163] phospholipids,[164] and proteins, [165] the most frequent post-translational modifications,[166] metal ions,[167] and glycans.[168] Additionally, SIRAH uses its own water model for explicit solvent called WatFour (WT4 for shortness), which also includes monovalent electrolytes (Na⁺, K⁺, and Cl⁻).[169] As the beads constituting the WT4 water molecule carry partial charges, the model creates its own dielectric permittivity, while the use of explicit electrolytes allows setting the ionic strength in the solution.

SIRAH uses the standard two-body classical Hamiltonian implemented in Amber. Hence, common concepts as partial charges, atom types, and equilibrium distances/angles can be straightforwardly transferred from atomistic to CG simulations. In this way, simulations performed with SIRAH can fully profit from GPU acceleration and analysis programs included in AmberTools.

Mapping from fully atomistic structures uses the position of real atoms to place interacting beads. Therefore, equilibrium values in the bonded terms of the Hamiltonian are directly extracted from experimental or canonical structures, reducing free parameters and facilitating the backmapping from CG to all-atoms.[170] Because of this, conformational preferences (i.e., helical, extended beta or coil conformations in proteins, and the B-form in DNA) are introduced in the bonded part of the Hamiltonian, obviating topological biases or the need to impose elastic network models to fix secondary structures.

Polar CG beads carry a partial charge, and electrostatic interactions are calculated at long range via the Particle Mesh Ewald method.

Conceptually, the main difference between SIRAH and a fully atomistic force field is in the calculation of the Lennard-Jones potential. In SIRAH, some interactions are not calculated using normal combination rules, but set to specific values between pairs of beads. This provides a flexible and convenient option to fix interactions that only apply to certain pairs of beads without modifying the entire force field. In particular, this feature is used in SIRAH to fine-tune the balance between electrostatic and Lennard-Jones interactions.

3.11.1. Available simulation schemes

Currently, the following CG and multiscale simulation schemes are available in SIRAH:

1. Explicit solvent CG simulations: they may include complex systems (Protein, DNA, membranes, glycans, water, and ions).
2. Implicit solvent CG simulations: Currently available only for DNA using generalized Born model with `igb=1`. [163, 171]
3. Multiscale simulations: These can be performed in three fashions:
 - a) - Multiscale solvation: fine grain (FG, or fully atomistic) solute solvated with atomistic water + CG water + supra CG water. This scheme is particularly well suited for highly solvated systems as virus capsids [172, 173] and is transferable to different force fields. Indeed, the WT4 water model has been tested to work in combination with TIP3P, SPC and SPC/e water models. [174]
 - b) - Dual scale DNA simulations: this scheme can deal with single or double-stranded DNA in which a certain number of nucleotides are defined at the atomistic level, while the rest is treated at the CG level. Simulations can be performed in explicit or implicit solvent (see point 2). SIRAH parameters have been developed to work with the bsc0 FG force field, [175, 176] and successfully checked for compatibility with the newer bsc1 version.
 - c) - QM/(FG/CG) simulations: this scheme profits from the possibility to run QM/MM simulations in AMBER. The current implementation has been only tested in a Russian-doll fashion with a quantum region surrounded by FG nucleotides nested in a CG double helix. [177]

3.11.2. Preparing your system for a CG simulation

In a nutshell, SIRAH is provided simply as another force field, plus a set of tools. In principle, all you need to get started is previous knowledge on how to run an MD simulation with AMBER and a fully protonated structure. Schematically, you can set up a CG simulation in three very simple steps.

1. Create a symbolic link in your working directory to ensure you will find the required files:

```
ln -s $AMBERHOME/dat/SIRAH/ .
```

2. Map the FG structure to CG. In its simplest form just type:

```
./SIRAH/tools/CGCONV/cgconv.pl -i your-protonated-FG-file.pdb -o your.CG-file.pdb
```

This will return a CG PDB file with standard mapping options. All options are shown by typing:

```
./SIRAH/tools/CGCONV/cgconv.pl -h
```

3. Molecular mechanics force fields

3. In your LEaP input file include:

```
AddPath SIRAH
source leaprc.sirah
```

For instance, a typical LEaP file for the protein 1CRN would look like:

```
# Load SIRAH force field
addPath ./sirah.amber
source leaprc.sirah
# Load model
protein = loadpdb 1CRN_cg.pdb
# Info on system charge
charge protein
# Set S-S bridges
bond protein.3.BSG protein.40.BSG
bond protein.4.BSG protein.32.BSG
bond protein.16.BSG protein.26.BSG
# Add solvent, counterions and 0.15M NaCl
# Tuned solute-solvent closeness for best hydration
solvateOct protein WT4BOX 20 0.7
addIonsRand protein NaW 22 ClW 22
# Save Parm
saveAmberParmNetcdf protein 1CRN_cg.prmtop 1CRN_cg.ncrst
# EXIT
quit
```

Notice that three disulfide bonds are created. For this to work, the Cysteine names in your PDB file must be edited from their thiol name (see comment on residue naming below).

Thereafter it is just normal Amber stuff!

Noteworthy, the SIRAH topologies and coordinate files generated with Leap are compatible with NAMD to run simulations using that MD engine.^[178]

Step-by-step tutorials on different cases of interest are available at <https://sirahff.github.io/documentation/>, while input files and initialization protocols are located in \$AMBERHOME/dat/SIRAH/tutorial/. We strongly advise you to complete these basic tutorials before utilizing the force field.

Parameters for ATP, GTP, ADP, GTP, cAMP, and cGMP are also available. New parameters for small molecules and cofactors can be developed upon request to sirahff@gmail.com.

3.11.3. Tips and tricks.

Answers to frequently asked questions can be found at \$AMBERHOME/dat/SIRAH/tutorial/SIRAH_FAQs.pdf.

1. The FG to CG mapping in SIRAH is intended to preserve physicochemically important interaction points (for example, Watson-Crick interactions in DNA). Therefore, the positions of Hydrogen atoms are needed in some residues, for instance, in Serine. Because of this, the starting point for CG simulation is a properly protonated PDB file. Amber naming is fully supported.
2. An important point to keep in mind is that the use of a 12-6 term for the Lennard-Jones interaction in a generally flatten CG surface may be potentially troublesome. Large steric repulsions in the absence of topological restraints could produce spurious structural distortions particularly sensitive to steric clashes. Hence, it is always a good idea (although not strictly necessary) to start with a well-relaxed set of starting coordinates.
3. Solvation may be a potential source of problems. SIRAH uses LEaP tools solvateBox or solvateOct to solvate CG solutes. However, the relatively large size of a CG water molecule may create vacuum holes nearby the solute that can lead to strong (unscreened) electrostatic interactions in the solute's surface. Similarly, when adding electrolytes, the use of addIons or addIonsRand, which substitute one water molecule by one ion,

might be problematic if the ionic positions lie very close to the solute's surface. Most likely, these problems will be fixed during the initialization protocol described in the tutorials. However, as in any simulation, the user should carefully check the initial set up.

4. In proteins, residues are named with lower "s" and the one-letter-code for amino acids (i.e., Alanine is sA). A third letter may indicate a residue modification. For instance, sE or sD stands for a Glutamate or Aspartate, respectively, while sEh or sDh correspond to protonated versions of those amino acids. Besides standard amino acids, the following modifications are available.
 - a) sX: Cysteine in S-S bond
 - b) sCp: Palmitoylated cysteine.
 - c) sEh, sDh: protonated acidic residues
 - d) sHe, sHd: Histidine protonated in epsilon and delta positions
 - e) sHp: positively charged Histidine (double protonated).
 - f) sSp, sTp, sYp: phosphorylated aminoacids.
 - g) sKa, sKm: Acetylated and methylated Lysine, respectively.
5. Zwitterionic and non-zwitterionic terminals are available. However, unlike the protein force fields included in AMBER, ACE and NME residues do not exist in SIRAH. Zwitterionic terminals are the default option but neutral terminals can be set by renaming the corresponding residues from s[one-letter-code] to a[one-letter-code] (Nt-acetylated) or m[one-letter-code] (Ct-amidated) after mapping. For example, to set a neutral N-terminal Histidine protonated at N ϵ rename it from "sHe" to "aHe".
6. Analysis: The Tcl script `sirah_vmdtk.tcl` provided in `$AMBERHOME/dat/SIRAH/tools/` contains a series of analysis and visualization tools to be used in VMD including backmapping, calculation of secondary structures. Additionally, it provides visualization macros to obtain the right connectivity, sizes, etc.[170] A Graphical User Interface for analysis of CG trajectories is available from <https://github.com/SIRAHFF/SIRAH-Tools-GUI>.
7. The backmapping to atomistic coordinates involves the reconstruction from CG coordinates using internal coordinates and an all-atoms minimization. If the system includes phospholipids, AMBERTools22 or higher is required.

3.12. Obsolete force field files

Some files are still included in the distribution for historical interest. We do *not* recommend that these be used any more for molecular simulations. The leaprc files that load these files have been moved to `$AMBERHOME/dat/leap/cmd/oldff`. We have removed this section from the current manual, but you can find this information in the [Amber24 Reference Manual](#).

4. The Generalized Born/Surface Area Model

Implicit solvent methods can speed up atomistic simulations by approximating the discrete solvent as a continuum, thus drastically reducing the number of particles in the system. An additional effective speedup often comes from much faster sampling of the conformational space afforded by these methods.[179–183] The generalized Born (GB) solvation model is the most commonly used implicit solvent model for atomistic MD simulation; it has been most widely tested on ff99SB and ff14SBonlysc, but in principle could be used with other non-polarizable force fields, such as ff03. A recent (2019) review gives a good overview.[184] To estimate the total solvation free energy of a molecule, ΔG_{solv} , one typically assumes that it can be decomposed into the "electrostatic" and "non-electrostatic" parts:

$$\Delta G_{solv} = \Delta G_{el} + \Delta G_{nonel} \quad (4.1)$$

where ΔG_{nonel} is the free energy of solvating a molecule from which all charges have been removed (i.e. partial charges of every atom are set to zero), and ΔG_{el} is the free energy of first removing all charges in the vacuum, and then adding them back in the presence of a continuum solvent environment. Generally speaking, ΔG_{nonel} comes from the combined effect of two types of interaction: the favorable van der Waals attraction between the solute and solvent molecules, and the unfavorable cost of breaking the structure of the solvent (water) around the solute. In the current Amber codes, this is taken to be proportional to the total solvent accessible surface area (SA) of the molecule, with a proportionality constant derived from experimental solvation energies of small non-polar molecules, and uses a fast LCPO algorithm [185] to compute an analytical approximation to the solvent accessible area of the molecule.

The Poisson-Boltzmann approach described in the next section has traditionally been used in calculating ΔG_{el} . However, in molecular dynamics applications, the associated computational costs are often very high, as the Poisson-Boltzmann equation needs to be solved every time the conformation of the molecule changes. Amber developers have pursued an alternative approach, the analytic generalized Born (GB) method, to obtain a reasonable, computationally efficient estimate to be used in molecular dynamics simulations. The methodology has become popular,[186–193] especially in molecular dynamics applications,[194–197] due to its relative simplicity and computational efficiency, compared to the more standard numerical solution of the Poisson-Boltzmann equation. Within Amber GB models, each atom in a molecule is represented as a sphere of radius R_i with a charge q_i at its center; the interior of the atom is assumed to be filled uniformly with a material of dielectric constant 1. The molecule is surrounded by a solvent of a high dielectric ϵ (80 for water at 300 K). The GB model approximates ΔG_{el} by an analytical formula,[186, 198]

$$\Delta G_{el} \approx -\frac{1}{2} \sum_{ij} \frac{q_i q_j}{f_{GB}(r_{ij}, R_i, R_j)} \left(1 - \frac{\exp[-\kappa f_{GB}]}{\epsilon} \right) \quad (4.2)$$

where r_{ij} is the distance between atoms i and j , the R_i are the so-called *effective Born radii*, and $f_{GB}()$ is a certain smooth function of its arguments. The electrostatic screening effects of (monovalent) salt are incorporated [198] via the Debye-Huckel screening parameter κ .

A common choice [186] of f_{GB} is

$$f_{GB} = [r_{ij}^2 + R_i R_j \exp(-r_{ij}^2/4R_i R_j)]^{1/2} \quad (4.3)$$

although other expressions have been tried.[189, 199] The effective Born radius of an atom reflects the degree of its burial inside the molecule: for an isolated ion, it is equal to its van der Waals (VDW) radius ρ_i . Then one obtains the particularly simple form:

4. The Generalized Born/Surface Area Model

$$\Delta G_{el} = -\frac{q_i^2}{2\rho_i} \left(1 - \frac{1}{\epsilon}\right) \quad (4.4)$$

where we assumed $\kappa = 0$ (pure water). This is the famous expression due to Born for the solvation energy of a single ion. The function $f_{GB}()$ is designed to interpolate, in a clever manner, between the limit $r_{ij} \rightarrow 0$, when atomic spheres merge into one, and the opposite extreme $r_{ij} \rightarrow \infty$, when the ions can be treated as point charges obeying the Coulomb's law.[192] For deeply buried atoms, the effective radii are large, $R_i \gg \rho_i$, and for such atoms one can use a rough estimate $R_i \approx L_i$, where L_i is the distance from the atom to the molecular surface. Closer to the surface, the effective radii become smaller, and for a completely solvent exposed side-chain one can expect R_i to approach ρ_i .

The effective radii depend on the molecule's conformation, and so have to be re-computed every time the conformation changes. This makes the computational efficiency a critical issue, and various approximations are normally made that facilitate an effective estimate of R_i . With the exception of GBNSR6 (see Section 5.1), the so-called *Coulomb field approximation*, or *CFA*, is used for Amber GB models, which replaces the true electric displacement around the atom by the Coulomb field. Within this assumption, the following expression can be derived:[192]

$$R_i^{-1} = \rho_i^{-1} - \frac{1}{4\pi} \int \theta(|\mathbf{r}| - \rho_i) r^{-4} d^3\mathbf{r} \quad (4.5)$$

where the integral is over the solute volume surrounding atom i . For a realistic molecule, the solute boundary (molecular surface) is anything but trivial, and so further approximations are made to obtain a closed-form analytical expression for the above equation, *e.g.* the so-called pairwise de-screening approach of Hawkins, Cramer and Truhlar,[200] which leads to a GB model implemented in Amber with *igb=1*. The 3D integral used in the estimation of the effective radii is performed over the van der Waals (VDW) spheres of solute atoms, which implies a definition of the solute volume in terms of a set of spheres, rather than the complex molecular surface,[201] commonly used in the PB calculations. For macromolecules, this approach tends to underestimate the effective radii for buried atoms,[192] arguably because the standard integration procedure treats the small vacuum-filled crevices between the van der Waals (VDW) spheres of protein atoms as being filled with water, even for structures with large interior.[199] This error is expected to be greatest for deeply buried atoms characterized by large effective radii, while for the surface atoms it is largely canceled by the opposing error arising from the Coulomb approximation, which tends [187, 191, 202] to overestimate R_i .

The deficiency of the model described above can, to some extent, be corrected by noticing that even the optimal packing of hard spheres, which is a reasonable assumption for biomolecules, still occupies only about three quarters of the space, and so "scaling-up" of the integral by a factor of four thirds should effectively increase the underestimated radii by about the right amount, without any loss of computational efficiency. This idea was developed and applied in the context of pH titration,[192] where it was shown to improve the performance of the GB approximation in calculating pKa values of protein sidechains. However, the one-parameter correction introduced in Ref. [192] was not optimal in keeping the model's established performance on small molecules. It was therefore proposed [197] to re-scale the effective radii with the re-scaling parameters being proportional to the degree of the atom's burial, as quantified by the value I_i of the 3D integral. The latter is large for the deeply buried atoms and small for exposed ones. Consequently, one seeks a well-behaved re-scaling function, such that $R_i \approx (\rho_i^{-1} - I_i)^{-1}$ for small I_i , and $R_i > (\rho_i^{-1} - I_i)^{-1}$ when I_i becomes large. The following simple, infinitely differentiable re-scaling function was chosen to replace the model's original expression for the effective radii:

$$R_i^{-1} = \tilde{\rho}_i^{-1} - \rho_i^{-1} \tanh(\alpha\Psi - \beta\Psi^2 + \gamma\Psi^3) \quad (4.6)$$

where $\Psi = I_i \tilde{\rho}_i$, and α, β, γ are treated as adjustable dimensionless parameters which were optimized using the guidelines mentioned earlier (primarily agreement with the PB). Currently, Amber supports two GB models (termed OBC) based on this idea. These differ by the values of α, β, γ , and are invoked by setting *igb* to either *igb=2* or *igb=5*. The details of the optimization procedure and the performance of the OBC model relative to the PB treatment and in MD simulations on proteins is described in Ref. [197]; an independent comparison to the PB in calculating the electrostatic part of solvation free energy on a large data set of proteins can be found in Ref. [203].

Our experience with generalized Born simulations is mainly with *ff99SB*, *ff14SBonlysc* or *ff03*; the current GB

1	2	5	7	8
<i>mbondi</i>	<i>mbondi2</i>	<i>mbondi2</i>	<i>bondi</i>	<i>mbondi3</i>

Table 4.1.: *Recommended radii sets for various GB models. For values of igb given in the top row, the string in the second row should be entered in LEaP as “set default PBRadii xxx”.*

models are not compatible with polarizable force fields. Replacing explicit water with a GB model is equivalent to specifying a different force field, and users should be aware that none of the GB options (in Amber or elsewhere) is as mature as simulations with explicit solvent; user discretion is advised. For example, it was shown that salt bridges are too strong in some of these models [204, 205] and some of them provide secondary structure distributions that differ significantly from those obtained using the same protein parameters in explicit solvent, with GB having too much α -helix present.[206, 207] The combination of the *ff14SBonlysc* force field with *igb*=8 gives the best results for proteins [29][208], nucleic acids and protein-nucleic acid complexes. [209]

Despite these limitations, implicit treatment of solvent is widely used in molecular simulations for two main reasons: algorithmic/computational speed and conformational sampling. [183, 210] Implicit solvent methods can be algorithmically/computationally faster, as measured by simulation time steps per processor (CPU) time, because the vast number of individual interactions between the atoms of individual solvent molecules do not need to be explicitly computed. Implicit-solvent simulations can also sample conformational space faster in the low viscosity regime afforded by the implicit solvent model.[179–183] To some extent, the interest in implicit-solvent-based simulations is motivated by the need to sample very large conformational spaces for problems such as protein folding, binding-affinity calculations, or large-scale fluctuations of nucleosomal DNA fragments. The speedup of conformational change can vary considerably, depending on the details of the transition, and can range from no speedup at all to almost a 100-fold speedup. [183] In general, the larger the conformational change, the higher the speedup one may expect, but this tendency is not universal or uniform. These speedup values are also expected to vary by the specific flavour of GB model used, a detailed analysis for *igb*5 can be found in Ref. [183].

The generalized Born models used here are based on the "pairwise" model introduced by Hawkins, Cramer and Truhlar,[200, 211] which in turn is based on earlier ideas by Still and others.[186, 191, 202, 212] The so-called overlap parameters for most models are taken from the Tinker molecular modeling package (<http://tinker.wustl.edu>). The effects of added monovalent salt are included at a level that approximates the solutions of the linearized Poisson-Boltzmann equation.[198] The original implementation was by David Case, who thanks Charlie Brooks for inspiration. Details of our implementation of generalized Born models can be found in Refs. [213, 214].

4.1. GB/SA input parameters

As outlined above, there are several "flavors" of GB available, depending upon the value of *igb*. The version that has been most extensively tested corresponds to *igb*=1; the "OBC" models (*igb*=2 and 5) are newer, but appear to give significant improvements and are recommended for most projects (certainly for peptides or proteins). The newest, most advanced, and least extensively tested model, *GBn* (*igb*=7), yields results in considerably better agreement with molecular surface Poisson-Boltzmann and explicit solvent results than the "OBC" models under many circumstances.[207] The *GBn* model was parameterized for peptide and protein systems and is not recommended for use with nucleic acids. A modification on the *GBn* model (*igb*=8) further improves agreement between Poisson-Boltzmann and explicit solvent data compared to the original formulation (*igb*=7).[29] Users should understand that all (current) GB models have limitations and should proceed with caution. Generalized Born simulations can only be run for non-periodic systems, *i.e.* where *ntb*=0. Unlike its use in explicit solvent PME simulations, short nonbonded cutoff values have much stronger impact on accuracy of the GB calculations. Essentially, any cutoff values other than *cut* > *structure size* can lead to artifacts. Current GPU implementation of the GB can not use cutoffs. If the nonbonded cutoff is used in GB calculations, it should be greater than that for PME calculations, perhaps *cut*=16. The slowly-varying forces generally do not have to be evaluated at every step for GB, either *nrespa*=2 or 4, although that option may lead to some artifacts as well.

igb = 0 No generalized Born term is used. (Default)

4. The Generalized Born/Surface Area Model

- = 1 The Hawkins, Cramer, Truhlar[200, 211] pairwise generalized Born model is used, with parameters described by Tsui and Case.[213] This model uses the default radii set up by LEaP. It is slightly different from the GB model that was included in Amber6. If you want to compare to Amber 6, or need to continue an ongoing simulation, you should use the command "set default PBradii amber6" in LEaP, and set *igb*=1 in *sander*. For reference, the Amber6 values are those used by an earlier Tsui and Case paper.[195] Note that most nucleic acid simulations have used this model, so you take care when using other values. Also note that Tsui and Case used an offset (see below) of 0.13 Å, which is different from its default value.
- = 2 Use a modified GB model developed by A. Onufriev, D. Bashford and D.A. Case; the main idea was published earlier,[192] but the actual implementation here[197] is an elaboration of this initial idea. Within this model, the effective Born radii are re-scaled to account for the interstitial spaces between atom spheres missed by the GB^{HCT} approximation. In that sense, GB^{OBC} is intended to be a closer approximation to true molecular volume, albeit in an average sense. With *igb*=2, the inverse of the effective Born radius is given by:

$$R_i^{-1} = \bar{\rho}_i^{-1} - \tanh(\alpha\Psi - \beta\Psi^2 + \gamma\Psi^3) / \rho_i$$
 where $\bar{\rho}_i = \rho_i - offset$, and $\Psi = I\rho_i$, with *I* given in our earlier paper. The parameters α , β , and γ were determined by empirical fits, and have the values 0.8, 0.0, and 2.909125. This corresponds to model I in Ref [197]. With this option, you should use the LEaP command "set default PBradii mbondi2" to prepare the *prmtop* file.
- = 3 or 4 These values are unused; they were used in Amber 7 for parameter sets that are no longer supported.
- = 5 Same as *igb*=2, except that now α, β, γ are 1.0, 0.8, and 4.85. This corresponds to model II in Ref [197]. With this option, you should use the command "set default PBradii mbondi2" in setting up the *prmtop* file, although "set default PBradii bondi" is also OK. When tested in MD simulations of several proteins,[197] both of the above parameterizations of the "OBC" model showed equal performance, although further tests [203] on an extensive set of protein structures revealed that the *igb*=5 variant agrees better with the Poisson-Boltzmann treatment in calculating the electrostatic part of the solvation free energy.
- = 6 With this option, there is no continuum solvent model used at all; this corresponds to a non-periodic, "vacuum", model where the non-bonded interactions are just Lennard-Jones and Coulomb interactions.
- = 7 The GB_n model described by Mongan, Simmerling, McCammon, Case and Onufriev[215] is employed. This model uses a pairwise correction term to GB^{HCT} to approximate a molecular surface dielectric boundary; that is to eliminate interstitial regions of high dielectric smaller than a solvent molecule. This correction affects all atoms and is geometry-specific, going beyond the geometry-free, "average" re-scaling approach of GB^{OBC} , which mostly affects buried atoms. With this method, you should use the bondi radii set. The overlap or screening parameters in the *prmtop* file are ignored, and the model-specific GB_n optimized values are substituted. The model carries little additional computational overhead relative to the other GB models described above.[215] This method is not recommended for systems involving nucleic acids.
- = 8 Same GB functional form as the GB_n model (*igb*=7), but with different parameters. The offset, overlap screening parameters, and *gbneckscale* are changed. In addition, individual α , β , and γ parameters can be specified for each of the elements H, C, N, O, S, P. Parameters for other elements have not been optimized, and the default values used are the ones from *igb*=5, which were not element-dependent. Default values were optimized for H, C, N, O and S atoms in protein systems.[29] Although the parameters for P in proteins can be specified, the default values were not optimized and are the *igb*=5 values. Nucleic acids have separate

parameters from those used for proteins, and default values were optimized for H, C, N, O and P atoms in nucleic acid systems.[209]

The following are the default parameters sander uses with *igb*=8:

```
Sh=1.425952, Sc=1.058554, Sn=0.733599,
So=1.061039, Ss=-0.703469, Sp=0.5,
offset=0.195141, gbneckscale=0.826836,
gbalphaH=0.788440, gbbetaH=0.798699, gbgammaH=0.437334,
gbalphaC=0.733756, gbbetaC=0.506378, gbgammaC=0.205844,
gbalphaN=0.503364, gbbetaN=0.316828, gbgammaN=0.192915,
gbalphaOS=0.867814, gbbetaOS=0.876635, gbgammaOS=0.387882,
gbalphaP=0.41836, gbbetaP=0.29005, gbgammaP=0.10642
screen_hnu=1.69654, screen_cnu=1.26890,
screen_nnu=1.425974, screen_onu=0.18401, screen_pnu=1.54506,
gb_alpha_hnu=0.53705, gb_beta_hnu=0.36286, gb_gamma_hnu=0.11670,
gb_alpha_cnu=0.33167, gb_beta_cnu=0.19684, gb_gamma_cnu=0.09342,
gb_alpha_nnu=0.68631, gb_beta_nnu=0.46319, gb_gamma_nnu=0.13872,
gb_alpha_onu=0.60634, gb_beta_onu=0.46301, gb_gamma_onu=0.14226,
gb_alpha_pnu=0.41836, gb_beta_pnu=0.29005, gb_gamma_pnu=0.10642
```

Parameters for proteins and for nucleic acids were optimized separately and can be independently specified. Protein parameters: Sh, Sc, Sn, So, Ss and Sp are scaling parameters, gbalphax, gbbetax, gbgammaX are the α , β , γ set for element X. gbalphaos, gbbetaos, gbgammaos is the α , β , γ set applied to both O and S. The phosphorus parameters (in proteins) were taken from GBneck2nu parameters. Nucleic acid parameters (end with "nu"): screen_Xnu (X=h, c, n, o, p) are scaling parameters, gb_alpha_Xnu (X=h, c, n, o, p) are the α , β , γ set for element X.

Since parameters are assigned for each atom based on its residue name (hard-coded in "sander/egb.F90" (subroutine isnucat)), users need to update the residue table in the sander source code if nucleic acids with different names are simulated using this GB model.

The default values for offset=0.195141, gbneckscale=0.826836 are recommended for both proteins and nucleic acids.

mbondi3 radii are recommended with *igb*=8 and can be employed with the LEaP command "set default PBradii mbondi3". The mbondi3 radii were adjusted based on protein simulations, and optimization of these radii for nucleic acids is currently underway.

- =10** Calculate the reaction field and nonbonded interactions using a numerical Poisson-Boltzmann solver. This option is described in the Chapter 6. Note that this is *not* a generalized Born simulation, in spite of its use of *igb*; it is rather an alternative continuum solvent model.

intdiel	Sets the interior dielectric constant of the molecule of interest. Default is 1.0. Other values have not been extensively tested.
extdiel	Sets the exterior or solvent dielectric constant. Default is 78.5.
saltcon	Sets the concentration (M) of 1-1 mobile counterions in solution, using a modified generalized Born theory based on the Debye-Hückel limiting law for ion screening of interactions.[198] Default is 0.0 M (<i>i.e.</i> no Debye-Hückel screening.) Setting <i>saltcon</i> to a nonzero value does result in some increase in computation time.
rgbmax	This parameter controls the maximum distance between atom pairs that will be considered in carrying out the pairwise summation involved in calculating the effective Born radii. Atoms whose associated spheres are farther away than <i>rgbmax</i> from given atom will not contribute to that atom's effective Born radius. This is implemented in a "smooth" fashion (thanks mainly to W.A. Svrcek-Seiler), so that when part of an atom's atomic sphere lies inside <i>rgbmax</i> cutoff, that part contributes

4. The Generalized Born/Surface Area Model

to the low-dielectric region that determines the effective Born radius. The default is 25 Å, which is usually plenty for single-domain proteins of a few hundred residues. Even smaller values (of 10-15 Å) are reasonable, changing the functional form of the generalized Born theory a little bit, in exchange for a considerable speed-up in efficiency, and without introducing the usual cut-off artifacts such as drifts in the total energy.

The *rgbmax* parameter affects only the effective Born radii (and the derivatives of these values with respect to atomic coordinates). The *cut* parameter, on the other hand, determines the maximum distance for the electrostatic, van der Waals and "off-diagonal" terms of the generalized Born interaction. The value of *rgbmax* might be either greater or smaller than that of *cut*: these two parameters are independent of each other. However, values of *cut* that are too small are more likely to lead to artifacts than are small values of *rgbmax*; therefore one typically sets *rgbmax* ≤ *cut*.

rbornstat	If <i>rbornstat</i> = 1, the statistics of the effective Born radii for each atom of the molecule throughout the molecular dynamics simulation are reported in the output file. Default is 0.
offset	The dielectric radii for generalized Born calculations are decreased by a uniform value "offset" to give the "intrinsic radii" used to obtain effective Born radii. Default is 0.09 Å.
gbsa	Option to carry out GB/SA (generalized Born/surface area) simulations. For the default value of 0, surface area will not be computed and will not be included in the solvation term. If <i>gbsa</i> = 1, surface area will be computed using the LCPO model.[185] If <i>gbsa</i> = 2, surface area will be computed by recursively approximating a sphere around an atom, starting from an icosahedra. Note that no forces are generated in this case, hence, <i>gbsa</i> = 2 only works for a single point energy calculation and is mainly intended for energy decomposition in the realm of MM-GBSA. If <i>gbsa</i> = 3, surface area will be computed using a fast pairwise approximation [216] suitable for GPU computing in pmemd.cuda program; the acceleration in pmemd.cuda compared with <i>gbsa</i> = 2 is ~30 times faster [216]. Note that <i>gbsa</i> = 3 is currently not supported in MM-GBSA, QM/MM or libsff. Although <i>gbsa</i> = 3 is supported in pmemd, its general usage is not recommended as the speed gain is insignificant because its fast approximation of surface area algorithm was specifically designed for GPU-accelerated GB simulations. Therefore, we recommend users use <i>gbsa</i> =3 with pmemd.cuda.
surften	Surface tension used to calculate the nonpolar contribution to the free energy of solvation (when <i>gbsa</i> = 1), as $E_{np} = \text{surften} \times SA$. The default is 0.005 kcal/mol/Å ² . [217] For <i>gbsa</i> = 3, <i>surften</i> works comparably with <i>gbsa</i> = 1 given the same value. [216]
rdt	This parameter is only used for GB simulations with LES (Locally Enhanced Sampling). In GB+LES simulations, non-LES atoms require multiple effective Born radii due to alternate de-screening effects of different LES copies. When the multiple radii for a non-LES atom differ by less than RDT, only a single radius will be used for that atom. See Chapter 32 for more details. Default is 0.0 Å.

4.2. Implicit Solvent with Explicit Ions model (GBION)

Aqueous solvation plays a critical role in the structure, stability, and function of biomolecules. The Generalized Born (GB) model offers a computationally efficient way to simulate solvent by treating it as a continuous dielectric medium. However, the traditional GB model approximates only monovalent ions, and only as a density cloud distributed according to the Debye–Hückel approximation. Localized effects of discrete ions around the solute are missing, the model does not distinguish K⁺ from Na⁺, and does not provide a suitable approximation for multivalent ions. Differences between ion types interacting with biomolecules, and their discrete nature, are crucial for most of biologically relevant systems.

The GBION model

the Implicit Solvent with EXplicit Ions model (GBION) extends the canonical GB model to include explicit ions in simulations, providing a more nuanced view of ion-solute interactions. The core innovation lies in modifying the GB equation to account for disconnected topology[218, 219], which is achieved by accommodating additional coefficients that help approximate correct charge-charge interactions between the solute atoms and explicit ions, and between the explicit ions. Interactions between solute atoms are calculated using the canonical GB model. The whole electrostatic energy of interaction of i th and j th atoms with modified GB equation takes the form:

$$E_{ij}(d_{ij}) = \frac{q_i q_j}{K_\epsilon(a, b) \epsilon_{in} d_{ij}} - \left(\frac{1}{K_\epsilon(a, b) \epsilon_{in}} - \frac{1}{\epsilon_{out}} \right) \times \frac{q_i q_j}{\sqrt{d_{ij}^2 + R_i R_j \exp\left(-d_{ij}^2 / 4 K_{GB}(a, b) (R_i R_j)\right)}} \quad (4.7)$$

This equation captures the general nature of charge-charge interactions mediated by the solute or solvent, introducing $K_{GB}(a, b)$ and $K_\epsilon(a, b)$ coefficients for tuning the interaction energy. The parameter $K_{GB}(a, b)$ controls how the function approximates the interaction between the solvent-separated charges: it was shown previously[218] that a pure Coulomb interaction works well for solvent separated charges, including ions. That regime is approximated by setting $K_{GB}(a, b) \ll 1$, see specific recommended values below. Note that when $K_{GB}(a, b) \rightarrow 0 \Rightarrow E(d_{ij}) \rightarrow \frac{q_i q_j}{\epsilon_{out} d_{ij}}$. Additional flexibility to fine-tune solute-ion and ion-ion interactions is afforded by the coefficient $K_\epsilon(a, b)$, which scales the whole electrostatic energy. The introduction of specific coefficients $K_{GB}(a, b)$ and $K_\epsilon(a, b)$ for interactions between solute atoms and ions (both anions and cations) enables the model to mimic, fairly accurately, the localized effects of ions on solvation energetics. For solute-solute interactions, $K_{GB}(a, b) = 1$ and $K_\epsilon(a, b) = 1$, meaning that the canonical GB is used.

The current version of GBION model was optimized and tested against explicit solvent distributions of Na^+ , K^+ , Cl^- and CoHex^{3+} ions around DNA. The model was also tested in simulations of the nucleosome with NaCl and KCl . We recommend to use the model for simulating DNA duplexes in the presence of these ions. We also cautiously recommend using GBION for simulations of the nucleosome in the presence of ions with the optimized parameters, especially if the focus on the DNA behavior. We do not recommend using the current version of GBION model for pure protein simulations, especially for positively charged proteins, without additional parameter optimization. So far, the GBION model was parametrized and used in combination with GBneck2 model (igb=8). We do not use Analytical Linearized Poisson-Boltzmann approximation here (alpb=0). To calculate the non-polar term of solution energy one can apply the widely used approximation based on the assumption that non-polar term is proportional to the solvent-accessible surface area (SASA) (gbsa=3 in the AMBER input file).

4.2.1. Usage

The setup of an GBION simulation resembles that of a canonical GB simulation, but with an additional step. First, set up the topology/coordinate files and input parameter files as you would for a simulation using GBneck2 (igb=8). Then, add the explicit ions that correspond to the desired salt concentration, see below. The ions are confined within a spherical volume, see Fig. 4.1. The simulation volume is defined by standard distance restraints (nmropt=1 in AMBER input file), which keep the ions from drifting beyond the pre-defined distance R from the center of mass of a certain group of atoms (see Fig. 4.1). In the example directory you can find a script (disang.py) that produces the restraint file *disang_NaCl.txt*.

To switch on the corresponding restraints, add the lines below to the input file:

```
/
&wt type='END' ,
/
DISANG=disang_NaCl.txt
&end
```

Where *disang_NaCl.txt* is the file that includes a description of the restraints.

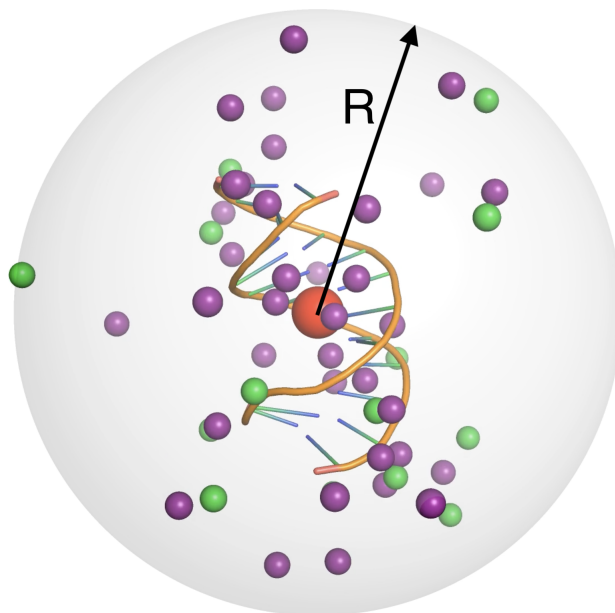


Figure 4.1.: Example of the restraining sphere used to set the simulation volume and ion concentration in GBION model. Purple and green spheres are Na^+ , and Cl^- ions, respectively. The grey sphere of radius R with its center near the center of mass of the DNA fragment illustrates the simulation volume. The red sphere denotes the center of the restraining sphere.

To set up the ion concentration we use SLTCAP method here. It utilizes the equation below to calculate the numbers of anions and cations that should be added to a simulating system:

$$N_{\pm} = V_w c_0 e^{\mp \text{arsinh}\left(\frac{Q}{2V_w c_0}\right)}, \quad (4.8)$$

where N_+ is the number of cations to be added, N_- – the number of anions to be added, Q is the total charge of the molecule (solute), V_w denotes the volume of the solvent in the system, and c_0 is the desired salt concentration in units of particles per $\text{\AA}^3$; to convert from the common [M/L] one should multiply the M/L value by 6.02×10^{-4} . The solvent volume V_w is defined as the volume of the enclosing sphere less the volume of the solute, Fig.4.1. The solute volume can be estimated via e.g. ProteinVolume server; however, in practice, it is often the case that $V_{\text{solute}} \ll V_w$, in which case one can simply neglect the former and calculate the solvent volume V_w as the sphere volume. Therefore, in most cases, one can use the approximate equation below to calculate the numbers of cations and anions:

$$N_{\pm} = \frac{4\pi R^3}{3} * 6.02 * 10^{-4} * c_0 e^{\mp \text{arsinh}\left(\frac{Q}{2\frac{4\pi R^3}{3} * 6.02 * 10^{-4} * c_0}\right)}, \quad (4.9)$$

where R denotes the radius of the confining sphere, and c_0 is the desired salt concentration in [M/L]. If the formulae return non-integer N_+ and N_- , one should take closest integer for either N_+ or N_- , and subtract it from the absolute value of Q to find N_- or N_+ , respectively. The GBION model has so far been optimized against Joung/Cheatham ion parameters (*frmod.ionsjc_tip3p*).

4.2.2. GBION input parameters

gbion Specifies the version of GBION model to be used
= 0 GBION model is turned off (default)

- = 3 both $K_{GB}(a,b)$ and K_ϵ depend of the respective atom types (currently recommended)
- = 1 only $K_{GB}(a,b)$ coefficient is available, which does not depend on the ion type. (under development, not recommended)
- = 2 both $K_{GB}(a,b)$ and K_ϵ coefficients are available, but the latter does not depend on the ion type (under development, not recommended)

Currently, all $K_{GB}(a,b)$ and K_ϵ defaults are 1.0, which effectively means GBION is not invoked unless the corresponding non-default values are specified explicitly. That is to run GBION, set the following non-default parameters (copy and paste into input file)

Parameters for NaCl:

```
gi_coef_1_n=0.05, gi_coef_2_pn=0.05,
intdiel_ion_1_p=54, intdiel_ion_1_n=8,
intdiel_ion_2_pp=54, intdiel_ion_2_pn=8,
intdiel_ion_2_nn=8,
```

Parameters for KCl:

```
gi_coef_1_n=0.05, gi_coef_2_pn=0.05,
intdiel_ion_1_p=36, intdiel_ion_1_n=8,
intdiel_ion_2_pp=36, intdiel_ion_2_pn=8,
intdiel_ion_2_nn=8,
```

Parameters for CoHex:

```
gi_coef_1_p=0.01, gi_coef_1_n=0.01,
gi_coef_2_pp=0.06, gi_coef_2_pn=0.06,
gi_coef_2_nn=0.06, intdiel_ion_2_pp=2,
intdiel_ion_2_pn=2, intdiel_ion_2_nn=2,
```

Below is the general description of all the parameters of the model:

`gi_coef_1_p` Sets the $K_{GB}(a,b)$ coefficient for the interaction between solute atoms and cations. The default is 1.0.

`gi_coef_1_n` Sets the $K_{GB}(a,b)$ coefficient for interactions between solute atoms and anions. The default is 1.0.

`gi_coef_2_pp` Sets the $K_{GB}(a,b)$ coefficient for cation-cation interactions. The default is 1.0.

`gi_coef_2_pn` Sets the $K_{GB}(a,b)$ coefficient for cation-anion interactions. The default is 1.0.

`gi_coef_2_nn` Sets the $K_{GB}(a,b)$ coefficient for anion-anion interactions. The default is 1.0.

`intdiel_ion_1_p` Sets the internal dielectric constant $K_\epsilon(a,b)$ for solute-cation interactions. The default is 1.0.

`intdiel_ion_1_n` Sets the internal dielectric constant $K_\epsilon(a,b)$ for solute-anion interactions. The default is 1.0.

`intdiel_ion_2_pp` Sets the internal dielectric constant $K_\epsilon(a,b)$ for cation-cation interactions. The default is 1.0.

`intdiel_ion_2_pn` Sets the internal dielectric constant $K_\epsilon(a,b)$ for cation-anion interactions. The default is 1.0.

`intdiel_ion_2_nn` Sets the internal dielectric constant $K_\epsilon(a,b)$ for anion-anion interactions. The default is 1.0.

4. The Generalized Born/Surface Area Model

The parameters listed below are unused in the current version of the model, but that may change in the future. These parameters scale the “neck” correction used to eliminate interstitial regions of high dielectric smaller than a solvent molecule. In GBneck models, the contribution of this neck correction is defined by the parameter *gbneckscale*, which can be scaled by $K_{NS}(a,b)$. In the current parametrization $K_{NS}(a,b) = 1$ for all atoms and ions, but that may change when new ions are parametrized.

`gb_neckscale_ion_1_p=1` Sets the $K_{NS}(a,b)$ coefficient for the interaction between solute atoms and cations. The default is 1.0.

`gb_neckscale_ion_1_n=1` Sets the $K_{NS}(a,b)$ coefficient for the interaction between solute atoms and anions. The default is 1.0.

`gb_neckscale_ion_2_pp=1` Sets the $K_{NS}(a,b)$ coefficient for cation-cation interactions. The default is 1.0.

`gb_neckscale_ion_2_pn=1` Sets the $K_{NS}(a,b)$ coefficient for cation-anion interactions. The default is 1.0.

`gb_neckscale_ion_2_nn=1` Sets the $K_{NS}(a,b)$ coefficient for anion-anion interactions. The default is 1.0.

4.3. Effect of Pressure, Osmolytes and Temperature in Implicit Solvent: the Group Transfer Free Energy (GTFE) model

The term ΔG_{solv} (Eq. 4.1) describes the free energy of solvation of a compound in water. The implicit solvent description that results from the application of this energetic term is suitable to simulate the behavior of molecules at ambient temperature and pressure, and in pure water (the effect of ions can additionally be included either using the Debye-Hückel approximation, or via the GBION model). However, the effect of pressure, or osmolytes, cannot be taken into account when using Eq. 4.1. Similarly, Eq. 4.1 fails at describing the cold denaturation behavior of proteins.

The objective of the GTFE (Group Transfer Free Energies) model is to extend the application range of implicit solvent models for proteins to account for the effect of pressure, temperature (e.g., cold denaturation) and the presence of osmolytes. This is achieved by modifying Eq. 4.1 to the following form:

$$\Delta G_{solv} = \Delta G_{el} + \Delta G_{nonel} + \Delta G_{tr} \quad (4.10)$$

where ΔG_{tr} represents the free energy of transfer of a protein molecule from pure water, at ambient temperature and pressure, to an osmolyte-water solution at any desired value of temperature and pressure.

The transfer free energy term is modelled according to an additive framework [220]:

$$\Delta G_{tr} = \sum_{k=1}^n \Delta g_{tr,sc}^k \alpha_{k,sc} + \Delta g_{tr,bb}^k \sum_{k=1}^n \alpha_{k,bb} \quad (4.11)$$

where the summations run over the k residues in a protein molecule, the Δg_{tr}^k terms are group transfer free energies representing the contributions of each sidechain (subscript sc) or backbone (subscript bb) group within the protein, and α_k is a fractional solvent accessible surface area.

The application of Eq. 4.10 allows the description of protein cold- [221] and pressure-induced [222] unfolding in implicit solvent simulations. The effect of common osmolytes on protein conformational [223] and colloidal [224] stability can also be implicitly described via Eq. 4.10.

4.3.1. Usage

The setup of a GTFE implicit solvent simulation resembles that of a canonical GB/SA simulation. The only additional step needed is the calculation of the group transfer free energies (Δg_{tr}^k) terms corresponding to the desired temperature, pressure and osmolyte concentration. A python script is publicly available to compute group transfer free energies, for any desired temperature/ pressure combination, and for some common osmolytes. The script can be freely downloaded from GitHub

(<https://github.com/andrea-arsiccio/DeltaG-calculation-amber>). The output of the python script is a list of group transfer free energies for every side chain, and for the backbone group, in kJ/mol. These group transfer free energies can directly be inserted into an input file for a sander or pmemd implicit solvent calculations. An exemplary input file for a GTFE implicit solvent calculation is shown below:

```
/
GTFE simulation
&cntrl
imin = 0,
igb = 5, gbsa = 1, surften = 0.005, extdiel = 78.4, ntpc = 1000,ntwx = 1000, ntwr =
ntt = 3, gamma_ln = 1.0, ig = -1, nscm = 500,
tempi = 298.0, temp0 = 298.0,
nstlim = 50000, dt = 0.002, ntc = 2, ntf = 2,
cut = 9999.0, ala = -0.20, arg = -0.89, asn = -1.62, asp = 0.15,
cys = 0.0, gln = -2.29, glu = 0.026, gly = 0.0, his = -2.11,
hip = -2.11, ile = -1.61, leu = -2.28, lys = -0.95, met = -2.02,
phe = -3.48, pro = -0.74, ser = -0.86, thr = -0.92, triptophan = -5.92,
tyr = -1.89, valine = -0.91, bb = -1.63
/
```

where the last entries (from *ala* to *bb*) are the group transfer free energies for the side chains and backbone (*bb*) groups, in kJ/mol.

The fractional solvent accessible surface area values (α_k in Eq. 4.11) can be computed using the LCPO algorithm (*gbsa* = 1) in both sander and pmemd, or using a pairwise approximation (*gbsa* = 3) in pmemd.

4.4. ALPB (Analytical Linearized Poisson-Boltzmann)

Like the GB model, the ALPB approximation [225, 226] can be used to replace the need for explicit solvent, with similar benefits (such as enhanced conformational sampling) and caveats. The basic ALPB equation that approximates the electrostatic part of the solvation free energy is

$$\Delta G_{el} \approx \Delta G_{alpb} = -\frac{1}{2} \left(\frac{1}{\epsilon_{in}} - \frac{1}{\epsilon_{ex}} \right) \frac{1}{1 + \alpha\beta} \sum_{ij} q_i q_j \left(\frac{1}{f_{GB}} + \frac{\alpha\beta}{A} \right) \quad (4.12)$$

where $\beta = \epsilon_{in}/\epsilon_{ex}$ is the ratio of the internal and external dielectrics, $\alpha=0.571412$, and A is the so-called *effective electrostatic size* of the molecule, see the definition of *Arad* below. Here f_{GB} is the same smooth function as in the GB model. The GB approximation is then just the special case of the ALPB when the solvent dielectric is infinite; however, for finite values of solvent dielectric the ALPB tends to be more accurate. For aqueous solvation, the accuracy advantage offered by the ALPB is still noticeable, and becomes more pronounced for less polar solvents. Statistically significant tests on macromolecular structures [226] have shown that ALPB is more likely to be a better approximation to PB than the GB. At the same time, the ALPB has virtually no additional computational overhead relative to GB. However, users should realize that at this point the new model has not yet been tested nearly as extensively as the canonical GB model. The ALPB can potentially replace the GB in the energy analysis of snapshots via the MM-GB/SA scheme. The electrostatic screening effects of monovalent salt are currently introduced into the ALPB in the same manner as in the GB, and are determined by the parameter *saltcon*.

alpb	Flag for using ALPB to handle electrostatic interactions within the implicit solvent model.
= 0	No ALPB (default).
= 1	ALPB is turned on. Requires that one of the analytical GB models is also used to compute the effective Born radii, that is one must set <i>igb</i> =1,2,5, or 7. The ALPB uses the same sets of radii as required by the particular GB model.
arad	Effective electrostatic size (radius) of the molecule. Characterizes its over-all dimensions and global shape, and is not to be confused with the effective Born radius of an atom. An appropriate

4. The Generalized Born/Surface Area Model

value of *Arad* must be set if *alpb=1*: this can be conveniently estimated for your input structure with the utility *elsize* that comes with the main distribution. The default is 15 Å. While *Arad* may change during the course of a simulation, these changes are usually not very large; the accuracy of the ALPB is found to be rather insensitive to these variations. In the current version of Amber *Arad* is treated as constant throughout the simulation, the validity of this assumption is discussed in Ref. [226]. Currently, the effective electrostatic size is only defined for "single-connected" molecules. However, the ALPB model can still be used to treat the important case of complex formation. In the docked state, the compound is considered as one, with its electrostatic size well defined. When the ligand and receptor become infinitely separated, each can be assigned its own value of *Arad*.

4.4.1. *elsize*

NAME

***elsize* - Given the structure, estimates its effective electrostatic size (parameter *Arad*) need by the ALPB model.**

SYNOPSIS

Usage: *elsize* input-pqr-file [-options]
-det an estimate based on structural invariants. DEFAULT.
-ell an estimate via elliptic integral (numerical).
-elf same as above, but via elementary functions.
-abc prints semi-axes of the effective ellipsoid.
-tab prints all of the above into a table without header.
-hea prints same table as -tab but with a header.
-deb prints same as -tab with some debugging information.
-xyz uses a file containing only XYZ coordinates.

DESCRIPTION

elsize is a program originally written by G. Sigalov to estimate the effective electrostatic size of a structure via a quick, analytical method. The algorithm is presented in detail in Ref. [226] You will need your structure in a pqr format as input, which can be easily obtained from the prmtop and inpcrd files using *ambpdb* utility described above:

ambpdb -p prmtop -pqr -c inpcrd > input-file-pqr

After that you can simply do: *elsize input-file-pqr* , the value of electrostatic size in Angstroms will be output on stdout. The source code is in the src/etc/ directory, its comments contain more extensive description of the options and give an outline of the algorithm. A somewhat less accurate estimate uses just the XYZ coordinates of the molecule and assumes the default radius size of for all atoms:

elsize input-file-xyz

This option is not recommended for very small compounds. The code should not be used on structures made up of two or more completely disjoint" compounds – while the code will still produce a finite value of *Arad* , it is not very meaningful. Instead, one should obtain estimates for each compound separately.

5. GBNSR6

GBNSR6 is an implementation of the Generalized Born (GB) model in which the effective Born radii are computed numerically, via the so-called “R6” integration[227, 228] over molecular surface of the solute:

$$\mathbf{R}_i^{-1} = \left(-\frac{1}{4\pi} \oint_{\partial V} \frac{\mathbf{r} - \mathbf{r}_i}{|\mathbf{r} - \mathbf{r}_i|^6} \cdot d\mathbf{S} \right)^{1/3} \quad (5.1)$$

For most structures, GB solvation based on the numerical R6 radii are virtually as accurate[215] as the GB energies based on the “gold standard” perfect effective radii, which can in principle be obtained from numerical solution of the PB equation[199]. As a result, the numerical R6 formulation is generally more accurate than the fast analytical approaches described above. In contrast to most GB practical models, GBNSR6 model is parameter-free in the same sense as the numerical PB framework is. Thus, accuracy of GBNSR6 relative to the PB standard is virtually unaffected by the choice of input atomic radii. However, unlike the analytical GB models in AMBER, GBNSR6 can not yet be used in dynamics. Recent benchmarks show that electrostatic binding energies computed by GBNSR6 are in good agreement with the numerical PB reference[229, 230].

Within GBNSR6, any of the following three versions of the pairwise GB equation can be used for computation of the solvation energies: (1) the canonical (Still 1990) GB[186], (2) the canonical GB with the ALPB correction[225, 226], and (3) the charge hydration asymmetric generalized Born (CHAGB) model[231]. The models are listed below; the first two are described in more detail in the GB section of the main manual, a brief introduction into CHAGB is below. For more information about these models please refer to the original references.

5.1. GB equations available in gbnsr6

- Canonical GB: the original equation due to Still et al, Eqs.4.2, 4.3.
- ALPB: an inexpensive correction, Eq. 4.12, to Still’s equation that restores correct dependence on dielectric constants. The correction is recommended in all cases except small molecules with decidedly non-spherical topology (e.g., rings) or structures that are topologically not singly-connected, e.g., two molecules not in contact with each other. The electrostatic size is computed automatically, no need to specify it in GBNSR6.
- CHAGB: The effect of charge hydration asymmetry (CHA)[124] – non-invariance of solvation free energy upon solute charge inversion – is incorporated into the Generalized Born framework[231]. The CHA is added to the GB equation (with or without the ALPB correction) to emulate asymmetric response to solvated charge of the specified explicit water model, e.g. TIP3P; the asymmetric response, which can be very strong, is ultimately determined by the charge distribution within the water model. Note that in contrast to standard GB or PB, CHAGB employs a novel definition of the dielectric boundary that does not subsume the CHA effects into the intrinsic atomic radii, therefore a special input radii set is used with this model. This model has so far been tested on a diverse set of neutral small molecules, charged and uncharged amino acid analogs and small proteins. Noticeable accuracy improvement over the uncorrected GB was reported for individual solvation energies. The optimum radii set for CHAGB available in this implementation shows better transferability between different classes of molecules. However, the model has not been tested in the context of protein-ligand binding, which may require a different radii set for optimum performance.

5.2. Numerical implementation of the R6 integral

- The R6 integral for computing the effective Born radius, Eq. 5.1, is performed for each atom over grid-based molecular surface of the solute. The molecular surface is based on the field-view method[232] also used in

5. GBNSR6

the PBSA tool. A uniform Cartesian grid is utilized to discretize a rectangular box containing the molecular structure. By exploiting the conservation of “electric flux” through the surface, the resulting finite difference grid surface elements traverse the same solid angle as the spherical surface elements obtained from the Lee and Richards molecular surface. More details of this implementation can be found in Ref.[232].

5.3. Usage

Just like other GB models available in AMBER, GBNSR6 can be used for efficient estimates of solvation free energy in situation where numerical PB estimates are too expensive. In addition to the value of the total solvation free energy, ΔG , its pairwise decomposition ΔG_{ij} can be obtained without significant additional computational expense typically associated with such estimates within the PB formalism. Options to output components of the non-polar solvation energy are available as well.

5.3.1. Input files

gbnsr6 has a similar usage as *amber/sander*:

gbnsr6 -i mdin -o mdout -p prmtop -c inpcrd

mdin input control data for the computations.

mdout output of the program in a user readable state info and diagnostics. “-o stdout” will send the output to the terminal.

prmtop input molecular topology file.

inpcrd input initial coordinate file.

5.3.2. Basic input options

The input file is very similar to the Amber/sander format. There are two namelist `&cntrl` and `&gb`. The only flag available in `&cntrl` is `inp`, the rest of the flags are in the namelist `&gb`. The following is a description of the available flags:

B	Specifies the value of uniform offset [215] to the (inverse) effective radii, the default value is $0.028 \text{ \AA}^{-1}$ which gives better agreement with the PB model, regardless of the structure size. For best agreement with the explicit solvent (TIP3P) solvation energies, optimal value of B depends on the structure size: for small molecules (number of atoms less than 50), we recommend $B=0$. With -chagb option, B is calculated automatically based on the solute size.
alpb	Specifies if ALBP correction is to be used. = 0 Canonical GB is used. = 1 ALPB is used (default)
epsin	Sets the dielectric constant of the solute region, default is 1.0. The solute region is defined to be the solvent excluded volume.
epsout	Sets the implicit solvent dielectric constant for the solvent, the default value is 78.5.
istrng	Sets the ionic strength (in mM) for the GB equation. Default is 0 mM. Physiological monovalent salt would correspond to 145 mM. Note the unit is different from that (in M) used by the other generalized Born methods implemented in Amber.
Rs	Sets the value of the dielectric boundary shift compared to the molecular surface, default value is $0.52 \text{ \AA}$ (only relevant for the -chagb option).
dprob	Sets the radius of the solvent robe, default is $1.4 \text{ \AA}$.

space	Sets the grid spacing that determines the resolution of the solute molecular surface, default is 0.5 Å. Note that memory footprint of this grid-based implementation of GBNSR6 may become large for large structures, e.g. the nucleosome (about 25,000 atoms) will take close to 2 GB of RAM when the default grid spacing is used. For very large structures, one may consider increasing the value of space, which will reduce the memory footprint and execution time; however, the accuracy will also decrease.
arccres	Sets the arc resolution used for numerical integration over molecular surface, the default value is 0.2 Å.
rbornstat	= 0 values of the inverse effective Born radii are not printed (default). = 1 print the inverse effective Born radii to the outfile.
dgi j	This flag is used for printing pairwise electrostatic energies. The values will be found in the output file, starting with the label “DGij”. The second and third columns of these lines specify the atom indexes of the respective atomic pair. Energy units are kcal/mol. = 0 does not print pairwise terms (default). = 1 prints polar component only of the solvation energy between all pairs of atoms.
radiopt	Specifies the set of intrinsic atomic radii to be used with the chagb option. = 0 uses hardcoded intrinsic radii optimized for small drug like molecules, and single amino acid dipeptides[231] (default) = 1 intrinsic radii are read from the topology file. Note that the dielectric surface defined using these radii is then shifted outwards by R_s relative to the molecular surface. The option is not recommended unless you are planning to re-optimize the input radii set for your problem.
chagb	= 0 Do not use CHAGB (default). = 1 Use CHAGB.
ROH	Sets the value of R_{OH}^z for CHA GB model, the default is 0.586Å. This parameter defines which explicit water model is being mimicked with respect to its propensity to cause CHA, the default corresponds to TIP3P and SPC/E. For OPC, $R_{OH}^z = 0.699\text{Å}$, for TIP4P $R_{OH}^z = 0.734\text{Å}$, and 0.183Å for TIP5P/E. A perfectly tetrahedral water, which can not cause charge hydration asymmetry, would have $R_{OH}^z = 0$.
tau	Sets the value of τ in the CHAGB model, the default is 1.47. This dimensionless parameter controls the effective range of the neighboring charges (j) affecting the CHA of atom (i), see Ref.[231] for details.
inp	= 0 do not compute nonpolar solvation energy. = 1 compute nonpolar solvation energies.
cavity_surften	Sets the surface tension parameter for nonpolar solvation calculation, the default value is 0.005 (kcal/mol/Å ²). This will be read only if the inp=1.

More options are available in a stand-alone version of GBNSR6 code not based on Cartesian grid [227].

5.3.3. Examples of input files

Compute electrostatic energy using default parameters.

```
&cntrl
  inp=0
/
```

5. GBNSR6

Compute electrostatic energies including nonpolar solvation energies and print the inverse effective Born radii

```
&cntrl
  inp=1
/
&gb
  epsin=1.0, epsout=78.5, istrng=0, dprob=1.4, space=0.5,
  arcres=0.2, B=0.028, alpb=1, rbornstat=1, cavity_surften=0.005
/
```

Use chagb to compute solvation energy, include ALPB correction.

```
&cntrl
  inp=1
/
&gb
  alpb=1, chagb=1
/
```

6. PBSA

Several efficient finite-difference numerical solvers, both linear [233, 234] and nonlinear,[235] are implemented in *pbsa* for various applications of the Poisson-Boltzmann method. The GPU support of those solvers is also implemented in *pbsa.cuda*. [236, 237] In the following, a brief introduction is given to the method, numerical solvers, and numerical energy and force calculations. This is followed by a detailed description of the usage and keywords. Example input files are explained for typical *pbsa* applications. The GPU-enabled *pbsa.cuda* is illustrated in section 6.7. For more information on the background and how to use the method, please consult the cited references and online *Amber* tutorial pages.

6.1. Introduction

Solvation interactions, especially solvent-mediated dielectric screening and Debye-Hückel screening, are essential determinants of the structure and function of proteins and nucleic acids.[238] Ideally, one would like to provide a detailed description of solvation through explicit simulation of a large number of solvent molecules and ions. This approach is frequently used in molecular dynamics simulations of solution systems. In many applications, however, the solute is the focus of interest, and the detailed properties of the solvent are not of central importance. In such cases, a simplified representation of solvation, based on an approximation of the mean-force potential for the solvation interactions, can be employed to accelerate the computation.

The mean-force potential averages out the degrees of freedom of the solvent molecules, so that they are often called implicit or continuum solvents. The formalism with which implicit solvents can be applied in molecular mechanics simulations is based on a rigorous foundation in statistical mechanics, at least for additive molecular mechanics force fields. Within the formalism, it is straightforward to understand how to decompose the total mean-field solvation interaction into electrostatic and non-electrostatic components that scale quite differently and must be modeled separately (see for example [239]).

The Poisson-Boltzmann (PB) solvents are a class of widely used implicit solvents to model solvent-mediated electrostatic interactions.[238] They have been demonstrated to be reliable in reproducing the energetics and conformations as compared with explicit solvent simulations and experimental measurements for a wide range of systems.[238] In these models, a solute is represented by an atomic-detail model as in a molecular mechanics force field, while the solvent molecules and any dissolved electrolyte are treated as a structure-less continuum. The continuum treatment represents the solute as a dielectric body whose shape is defined by atomic coordinates and atomic cavity radii.[240] The solute contains a set of point charges at atomic centers that produce an electrostatic field in the solute region and the solvent region. The electrostatic field in such a system, including the solvent reaction field and the Coulombic field, may be computed by solving the PB equation:[241, 242]

$$\nabla \cdot [\epsilon(\mathbf{r}) \nabla \phi(\mathbf{r})] = -4\pi\rho(\mathbf{r}) - 4\pi\lambda(\mathbf{r}) \sum_i z_i c_i \exp(-z_i \phi(\mathbf{r})/k_B T) \quad (6.1)$$

where $\epsilon(\mathbf{r})$ is the dielectric constant, $\phi(\mathbf{r})$ is the electrostatic potential, $\rho(\mathbf{r})$ is the solute charge, $\lambda(\mathbf{r})$ is the Stern layer masking function, z_i is the charge of ion type i , c_i is the bulk number density of ion type i far from the solute, k_B is the Boltzmann constant, and T is the temperature; the summation is over all different ion types. The salt term in the PB equation can be linearized when the Boltzmann factor is close to zero. However, the approximation apparently does not hold in highly charged systems. Thus, it is recommended that the full nonlinear PB equation solvers be used in such systems.

The non-electrostatic or non-polar (NP) solvation interactions are typically modeled with a term proportional to the solvent accessible surface area (SASA).[217] An alternative and more accurate method to model the non-polar solvation interactions is also implemented in *pbsa*. [243] The new method separates the non-polar solvation interactions into two terms: the attractive (dispersion) and repulsive (cavity) interactions. Doing so significantly

6. PBSA

improves the correlation between the cavity free energies and solvent accessible surface areas or molecular volumes enclosed by SASA for branched and cyclic organic molecules.[244] This is in contrast to the commonly used strategy that correlates total non-polar solvation energies with solvent accessible surface areas, which only correlates well for linear aliphatic molecules.[217] In the alternative method, the attractive free energy is computed by a numerical integration over the solvent accessible surface area that accounts for solvation attractive interactions with no cutoff.[245]

6.1.1. Numerical solutions of the PB equation

In *pbsa* both the linear form and the full nonlinear form of the PB equation are supported. Many strategies may be used to discretize the PB equation, but only the finite-difference (FD) method, or more rigorously, the finite-volume method [246–248] is fully supported in *pbsa* for both the linear and nonlinear PB equations. A FD method involves the following steps: mapping atomic charges to the FD grid points (termed grid charges below); assigning non-periodic/periodic boundary conditions, i.e., electrostatic potentials on the boundary surfaces of the FD grid; and applying a dielectric model to define the high-dielectric (i.e., water) and low-dielectric (i.e., solute interior) regions and mapping it to the FD grid edges.

These steps allow the partial differential equation to be converted into a linear or nonlinear system with the electrostatic potential on grid points as unknowns, the charge distribution on the grid points as the source, and the dielectric constant on the grid edges (and the salt-related term for the linear case) wrapped into the coefficient matrix, which is a seven-banded symmetric matrix. In *pbsa*, four common linear FD solvers are implemented: modified ICCG, geometric multigrid, conjugate gradient, and successive over-relaxation (SOR).[234] In addition, we have also implemented six nonlinear FD solvers: Inexact Newton(NT)/modified ICCG, NT/geometric multigrid, conjugate gradient, and SOR and its improved versions - adaptive SOR and damped SOR.[235]

In addition to the FD method, a new discretization strategy is also introduced to solve the linear PB equation.[249] The Immersed Interface method (IIM) is a second-order accurate numerical method developed for systems with interface, i.e. solute/solvent boundary in this case. In the IIM discretization scheme, the linear equations on regular grid points, i.e. grid points away from the interface, are the same as the standard finite-difference method, but the linear equations on irregular grid points, i.e. grid points nearby the interface, are constructed by minimizing the magnitude of the local truncation error in the discretization of the PB equation.[250] It can be proven that the errors of calculated potentials are at the order of $O(h^2)$ on the regular grid points and $O(h)$ on the irregular grid points.[250]

6.1.2. Numerical interpretation of energy and forces

PB solvents approximate the solvent-induced electrostatic mean-force potential by computing the reversible work in the process of charging the atomic charges in a solute molecule or complex. The charging free energy is a function of the electrostatic potential ϕ , which can be computed by solving the linear or nonlinear system.

It has been shown (see for example [239]) that the total electrostatic energy of a solute molecule can be approximated through the FD approach by subtracting the self FD Coulombic energy ($G_{coul,shelf}^{FD}$) and the short-range FD Coulombic energy ($G_{coul,short}^{FD}$) from the total FD electrostatic energy ($G_{coul,total}^{FD}$), and adding back the analytical short-range Coulombic energy ($G_{coul,short}^{ana}$). The self FD Coulombic energy is due to interactions of grid charges within one single atom. The self energy exists even when the atomic charge is exactly positioned on one grid point. It also exists in the absence of solvent and any other charges. It apparently is a pure artifact of the FD approach and must be removed. The short-range FD Coulombic energy is due to interactions between grid charges in two different atoms that are separated by a short distance, usually less than 14 grid units. The short-range Coulombic energy is inaccurate because the atomic charges are mapped onto their eight nearest FD grids, thus causing deviation from the analytical Coulomb energy. The correction of $G_{coul,shelf}^{FD}$ and $G_{coul,short}^{FD}$ is made possible by the work of Luty and McCammon's analytical approach to compute FD Coulombic interactions.[251]

Therefore, the PB electrostatic interactions include both Coulombic interactions and reaction field interactions for all atoms of the solute. The total electrostatic energy is given in the energy component EEL in the output file. The term that is reserved for the reaction field energy, EPB, is zero if this method is used. If you want to know how much of EEL is the reaction field energy, you can set the BCOPT keyword (to be explained below) to compute the reaction field energy only by using a Coulombic field (or singularity) free formulation.[252]

When the full nonlinear Poisson-Boltzmann equation is used, an additional energy term, the ionic energy, should also be included. This energy term disappears in the symmetrical linear system because the effects due to opposite ions cancel out. It is currently approximated by calculation up to the space boundary of the FD grid. It should be noted that the NBUFFER keyword may need increasing to obtain good precision in the ionic energy for small molecules with a large FILLRATIO.

An alternative method of computing the electrostatic interactions is also implemented in *pbsa*. In this method, the reaction field energy is computed directly after the induced surface charges are first computed at the dielectric boundary (i.e., the surface that separates solute and solvent). These surface charges are then used to compute the reaction field energy,[238] and is given as the EPB term. It has been shown that doing so improves the convergence of reaction field energy with respect to the FD grid spacing. However, a limitation of this method is that the Coulombic energy has to be recomputed analytically with a pairwise summation procedure. When this method is used, the EEL term only gives the Coulombic energy with a cutoff distance provided in the input file. The two ways of computing electrostatic interactions are controlled by the keywords ENEOPT and FRCOPT to be described below.

The non-polar solvation free energy is returned by the ECAVITY term, which is either the total non-polar solvation free energy or the cavity solvation free energy in the two different models described above. The EDISPER term returns the dispersion solvation free energy. Of course it is zero if the total non-polar solvation free energy has been returned by ECAVITY. The word INP can be used to choose one of the two treatments of non-polar solvation interactions.[243] Specifically, you can use SASA to correlate total non-polar solvation free energy, i.e., $G_{np} = NP_TENSION \times SASA + NP_OFFSET$ as in PARSE.[217] You can also use SASA to correlate the cavity term only and use a surface-integration approach to compute the dispersion term.[243] i.e., $G_{np} = G_{disp} + G_{cavity}$, with $G_{cavity} = CAVITY_TENSION \times SASA + CAVITY_OFFSET$. See the discussion of keywords in 8.2.8. These options are described in detail in Ref. [243].

Finally, in this release, the PB forces are now correctly interpreted for the widely used SES molecular surface definition, i.e., the partition of dielectric boundary pressure/force can now reproduce the virtual work principle. This is achieved by proper decomposition of the dielectric boundary force on the reentrant portion of the molecular surface. Specifically, the molecular surface is computed more accurately by considering the cases when the solvent probe touches three atoms simultaneously. Next the reentrant force is also distributed onto the three atoms forming the reentrant surface following the virtual work principle.[253]

6.1.3. Numerical accuracy and related issues

Note that the accuracy of any numerical PB procedure is determined by the discretization resolution specified in the input, i.e., the grid spacing. The convergence criterion for the iteration procedures also plays some role for the numerical PB solvers. Finally the accuracy is highly dependent upon the methods used for computing total electrostatic interactions. In Lu and Luo,[239] the accuracy of the first method for total electrostatic interactions is discussed in detail. In Ref.[253] the accuracy of the second method is discussed.

It is recommended that the second method for total electrostatic interactions be used for most calculations. Apparently the cutoff distance for charge-charge interactions strongly influences the accuracy of electrostatic interactions. The default setting is infinity, i.e., no cutoff is used. In this method, the convergence of the reaction field energy with respect to the grid spacing is much better than that of the first method. Our experience shows that the reaction field energies converge to within ~2% for tested proteins at the grid spacing of 0.5 Å when the weighted harmonic average of dielectric constants is used at the solute/solvent interface (when SMOOTHOPT = 1, see below).[254]

The reaction field energies computed with the second method (when SMOOTHOPT = 2) are also in excellent agreement (differences in the order of 0.1%) with those computed with the *Delphi* program which uses the same method for energy calculation. For example, see the computational set up documented in test case *pbsa_delphi* in this release.[255]

The accuracy of non-polar solvation energy depends on the quality of SASA which is computed numerically by representing each atomic surface by spherically distributed dots. Thus a higher dot density gives more accurate atomic surface and molecular surface. However, it is found that the default setting for the dot density is quite sufficient for typical applications.[243] Should you encounter any memory allocation error for surface calculation, you are advised to use a coarser surface dot resolution if the physical memory of your computer is limited.

6. PBSA

Numerical solvation calculations are memory intensive for macromolecules due to the fine grid resolution required for sufficient accuracy. Thus, the efficiency of *pbsa* depends on how much memory is allocated for it and the performance of the memory subsystem. The option that is directly related to its memory allocation is the FD grid spacing for the PB equation and the surface dot resolution for molecular surface. Apparently the geometric dimension and the number of atoms are also important for predicting the memory usage. In general for a typical computer configuration with 8GB memory, the geometric dimension can be as large as $180 \times 180 \times 180 \text{ \AA}^3$ at the default grid spacing of $0.5 \text{ \AA}$ before the computer responds too slowly.

6.2. Usage and keywords

6.2.1. File usage

pbsa has a very similar user interface as the *Amber/sander* program, though much simpler.

```
pbsa [-O] -i mdin -o mdout [-p prmtop -c inpcrd]/[-pqr pqr]
```

Starting from the 2014 release, *pbsa* supports the free format pqr file. Once the pqr reading is enabled, the default Amber file reading and processing would be bypassed. Here is a brief description of the files mentioned above.

mdin *input* control data for the run.

mdout *output* user readable state info and diagnostics “-o stdout” will send output to stdout (to the terminal) instead of to a file.

prmtop *input* molecular topology, force field, atom and residue names, and (optionally) periodic box type.

inpcrd *input* initial coordinates and (optionally) velocities and periodic box size.

pqr *input* initial coordinates, atomic charges and radii in the free format pqr.

Here are a few comments on the “free-formatted” pqr file used by *pbsa*. First all fields are delimited by spaces only. Second there is no strict format requirement as in a standard pdb file. This more liberal style is to accommodate pqr files of different origins. *pbsa* reads data on a per-line basis using the following format:

```
Tag AtomNumber AtomName ResidueName ChainID ResidueNumber XYZ Charge Radius
```

Tag A string specifying either ATOM or HETATM. Lines with other strings are ignored.

AtomNumber The sequence no of the atom, which is reset to start from 1.

AtomName The atom name.

ResidueName The residue name.

ChainID The chain ID of the atom, optional, which is ignored.

ResidueNumber The sequence no. of the residue, which is ignored.

XYZ The floating numbers representing the atomic coordinates (in Angstrom).

Charge A float number providing the atomic charge (in electron).

Radius A float number providing the atomic radius (in Angstrom).

Finally it is worth to point out that it is apparently very hard to know whether the charge and radius fields are swapped as in the *Delphi* generated pqr file. Here we have assumed that the data are in the plain P.Q.R. order. Please make sure you are following the same convention in generating the pqr files.

6.2.2. Basic input options

The layout of the input file is in the same way as that of *Amber/sander* for backward compatibility with previous releases in *Amber*. The keywords are put in the the namelist of `&cntrl` for basic controls and `&pb` for more detailed manipulation of the numerical procedures. This subsection discusses the basic keywords, either retained from *sander* or newly created to invoke different energetic analyses. To reduce confusion most keywords from *sander* have been removed from the namelist so they can no longer be read since the current implementation in *pbsa* only performs single-structure calculations with the coordinates from **inpcrd** and exits. However, the current release is compatible with the **mdin** file generated with the *mmpbsa* script in previous releases in *Amber*. Users interested in energy minimization and molecular dynamics with the PB implementation are referred to *sander* in the release of *Amber*. Nevertheless, for purposes of validation and development, the atomic forces can be dumped out in a file when requested as described below.

The numerical electrostatic procedures can be turned on by setting IPB to either 1, 2 or 4. The flag IGB = 10 is phased out in this release. The numerical non-polar procedures can be turned on by setting INP to either 1 or 2. The backward compatible flag NPOPT is also phased out in this release.

<code>imin</code>	Flag to run minimization. Both options give the same output energies though the output formats are slightly different. This option is retained from previous releases in the <i>Amber</i> package for backward compatibility. The current release of <i>pbsa</i> only supports single point energy calculation. = 0 No minimization. Dynamics is available with <i>sander</i> and NAB. = 1 Single point energy calculation. Default. Multiple-step PB minimization is also available with <i>sander</i> and NAB.
<code>ntx</code>	Option to read the coordinates from the “inpcrd” file. Only options 1 and 2 are supported in this releases. Other options will cause <i>pbsa</i> to issue a warning though it does not affect the energy calculation. = 1 X is read formatted with no initial velocity information. Default. = 2 X is read unformatted with no initial velocity information.
<code>ipb</code>	Option to set up a dielectric model for all numerical PB procedures. IPB = 1 corresponds to a classical geometric method, while a level-set based algebraic method is used when $IPB \geq 2$. The default IPB is 2. = 0 No electrostatic solvation free energy is computed. = 1 The dielectric interface between solvent and solute is built with a geometric approach. [233] = 2 The dielectric interface is implemented with the level set function. Use of a level set function simplifies the calculation of the intersection points of the molecular surface and grid edges and leads to more stable numerical calculations. Default.[255] = 4 The dielectric interface is also implemented with the level set function. However, the linear equations on the grid points nearby the dielectric boundary are constructed using the IIM. In this option, The dielectric constant do not need to be smoothed, that is, SMOOTHOPT is useless. Only the linear PB equation is supported, that is, NPBOPT = 0. Starting from the Amber 2018 release, SOLVOPT is no longer relevant as only one stable solver is supported.[249] = 6 The dielectric interface is implemented analytically with the revised density function approach (SASOPT=2). The linear equations on the irregular points are constructed using the IIM and fully utilizing the analytical surface. Otherwise, it is exactly the same as IPB=4.[256] = 7 The dielectric interface is implemented analytically with the revised density function approach (SASOPT=2). The linear equations on the irregular points are constructed using the X-factor harmonic average method.[257]

6. PBSA

- = 8** The dielectric interface is implemented analytically with the revised density function approach (SASOPT=2). The linear equations on the irregular points are constructed using the second-order harmonic average method.[257]

`inp` Option to select different methods to compute non-polar solvation free energy.

- = 0** No non-polar solvation free energy is computed.
- = 1** The total non-polar solvation free energy is modeled as a single term linearly proportional to the solvent accessible surface area, as in the PARSE parameter set, that is, if INP = 1, USE_SAV must be equal to 0. See Introduction. [243]
- = 2** The total non-polar solvation free energy is modeled as two terms: the cavity term and the dispersion term. The dispersion term is computed with a surface-based integration method [243] closely related to the PCM solvent for quantum chemical programs.[245] Under this framework, the cavity term is still computed as a term linearly proportional to the molecular solvent-accessible-surface area (SASA) or the molecular volume enclosed by SASA. Default.

Once the above basic options are specified, *pbsa* can proceed with the default options to compute the solvation free energies with the input coordinates. Of course, this means that you only want to use default options for default applications. More PB options described below can be defined in the `&pb` namelist, which is read immediately after the `&cntrl` namelist. We have tried hard to make the defaults for these parameters appropriate for calculations of solvated molecular systems. Please use caution when changing any default options. Also note that the default options may have changed over time. For a detailed discussion of all related options on the quality of the calculations, please refer to our recent publication [258].

6.2.3. Options to define physical constants

`epsin` Sets the dielectric constant of the solute region, default to 1.0. The solute region is defined to be the solvent excluded volume.

`epsout` Sets the implicit solvent dielectric constant, default to 80. The solvent region is defined to be the space not occupied the solute region. i.e., only two dielectric regions are allowed in the current release.

`epsmem` Sets the membrane dielectric constant. Only used if `membraneopt > 0`, does nothing otherwise. Value used should be between `epsin` and `epsout` or there may be errors. Previously spelled as `epsmemb`, which is being phased out. Defaults to 1.0.

`smoothopt` Instructs PB how to set up dielectric values for finite-difference grid edges that are located across the solute/solvent dielectric boundary.

- = 0** The dielectric constants of the boundary grid edges are always set to the equal-weight harmonic average of EPSIN and EPSOUT.
- = 1** A weighted harmonic average of EPSIN and EPSOUT is used for boundary grid edges. The weights for EPSIN and EPSOUT are fractions of the boundary grid edges that are inside or outside the solute surface.[259] Default.
- = 2** The dielectric constants of the boundary grid edges are set to either EPSIN or EPSOUT depending on whether the midpoints of the grid edges are inside or outside the solute surface.

`istrng` Sets the ionic strength (in mM) for the PB equation. Default is 0 mM. Note the unit is different from that (in M) in the generalized Born methods implemented in *Amber*. Note also that we are only dealing with symmetrical solution, so the ionic strength should be equal to the square of the valence of the symmetrical ions times the ion concentration (in mM).

`pbtemp` Temperature (in K) used for the PB equation, needed to compute the Boltzmann factor for salt effects; default is 300 K.

<code>radiopt</code>	Option to set up atomic radii. = 0 Use radii from the <code>prmtop</code> file for both the PB calculation and for the NP calculation (see INP). = 1 Use atom-type/charge-based radii by Tan and Luo [260] for the PB calculation. Note that the radii are optimized for <i>Amber</i> atom types as in standard residues from the <i>Amber</i> database. If a residue is built by <i>antechamber</i>, i.e., if GAFF atom types are used, radii from the <code>prmtop</code> file will be used. Please see [260] on how these radii are optimized. The procedure in [260] can also be used to optimize radii for nonstandard residues. These optimized radii can be read in if they are incorporated into the radii section of the <code>prmtop</code> file (of course via RADIOPT = 0). Default.
<code>dprob</code>	Solvent probe radius for molecular surface used to define the dielectric boundary between solute and solvent. DPROB = 1.4 by default.
<code>iprob</code>	Mobile ion probe radius for ion accessible surface used to define the Stern layer. Default to 2.0 Å.
<code>sasopt</code>	Option to determine which kind of molecular surfaces to be used in the Poisson-Boltzmann implicit solvent model. Default is 0. = 0 Use the solvent excluded surface (SES) as implemented by [255]. = 1 Use the solvent accessible surface (SAS). Apparently, this reduces to the van der Waals surface (VDW) when the <code>dprobe</code> is set to zero. = 2 Use the smooth surface defined by a revised density function. [261] This must be combined with <code>IPB</code> ≥ 2. = 3 Use the solvent excluded surface inferred by the machine-learned solvent excluded surface (MLSES) model. This must be combined with <code>IPB</code> = 2 (See 6.2.9 for details).
<code>saopt</code>	Option to compute the surface area of a molecule. Default is 0. Once the computation is enabled, the surface area will be reported in the output file with the subtitle “Total molecular surface”. Note that only the surface areas for the solvent excluded surface and the solvent accessible surface are supported in this release. = 0 Do not compute any surface area. = 1 Use the field-view method to compute the surface area. [232]
<code>triopt</code>	Option to add trimer arc dots for a more accurate and lower memory mapping method of the analytical solvent excluded surface. See [255] = 0 Trimer arc dots are not used. = 1 Trimer arc dots are used. Default.
<code>arcres</code>	<i>pbsa</i> uses a numerical method to compute solvent accessible arcs. See [255]. The ARCREs keyword gives the resolution (in the unit of Å) of dots used to represent these arcs, default to 0.25 Å. These dots are first checked against nearby atoms to see whether any of the dots are buried. The exposed dots represent the solvent accessible portion of the arcs and are used to define the dielectric constants on the grid edges. It should be pointed out that ARCREs should be reduced to (0.125 Å) when the TRIOPT option is turned off to achieve a similar accuracy in the reaction field energies. More generally, ARCREs should be set to $\max(0.125 \text{ Å}, 0.5h)$ when the TRIOPT option is turned on, or $\max(0.0625 \text{ Å}, 0.25h)$ when the TRIOPT option is turned off (h is the grid spacing).

6.2.4. Options for implicit membrane models

<code>membraneopt</code>	Option to turn the implicit membrane on and off. The membrane is implemented as a slab like region with a uniform or heterogeneous dielectric constant depth profile. = 0 No implicit membrane used (default). = 1 Use a uniform membrane dielectric constant in a slab-like implicit membrane.[262] = 2 Use a heterogeneous membrane dielectric constant in a slab-like implicit membrane. The dielectric constant varies with depth from a value of 1 in the membrane center to 80 at the membrane periphery. The dielectric constant depth profile was implemented using the PCHIP fitting.[263] = 3 Use a heterogeneous membrane dielectric constant in a slab-like implicit membrane. The dielectric constant varies with depth from a value of 1 in the membrane center to 80 at the membrane periphery. The dielectric constant depth profile was implemented using the Spline fitting.[263]
<code>mprob</code>	Membrane probe radius in Å, default to 2.70. This is used to specify the highly different lipid molecule accessibility versus that of the water.[264]
<code>mthick</code>	Membrane thickness in Å, default to 40.0. This is different from the previous default of 20 Å.
<code>mctrdz</code>	Membrane center in Å in the z direction. Default is 0 - membrane centered at the center of the protein.
<code>poretype</code>	Turn on and off the automatic depth-first search method to identify the pore.[264] = 0 Do not turn on the pore searching algorithm. = 1 Turn on the pore searching algorithm.
<code>poreradius</code>	Controls the radius, in Å, of the cylindrical exclusion region. This is no longer needed given the automatic pore searching algorithm.

6.2.5. Options to select numerical procedures

<code>npbopt</code>	Option to select the linear [234] or the full nonlinear PB equation.[235] = 0 Linear PB equation is solved. Default. = 1 Nonlinear PB equation is solved.
<code>solvopt</code>	Option to select iterative solvers. = 1 Modified ICCG or Periodic (PICCG) if <code>bcopt = 10</code> is. Default. = 2 Geometric multigrid. A four-level v-cycle implementation is applied by default. = 3 Conjugate gradient (Periodic version available under <code>bcopt = 10</code>). This option requires a large MAXITN to converge. = 4 SOR. This option requires a large MAXITN to converge. = 5 Adaptive SOR. This is only compatible with <code>NPBOPT = 1</code> . This option requires a large MAXITN converge.[235] = 6 Damped SOR. This is only compatible with <code>NPBOPT = 1</code> . This option requires a large MAXITN to converge.[235]
<code>accept</code>	Sets the iteration convergence criterion (relative to the initial residue). Default to 0.001.
<code>maxitn</code>	Sets the maximum number of iterations for the finite difference solvers, default to 100. Note that MAXITN has to be set to a much larger value, e.g. 10,000, for the less efficient solvers, such as conjugate gradient and SOR, to converge.

<code>fillratio</code>	The ratio between the longest dimension of the rectangular finite-difference grid and that of the solute. Default is 2.0. It is suggested that a larger FILLRATIO, for example 4.0, be used for a small solute, such as a ligand molecule. Otherwise, part of the small solute may lie outside of the finite-difference grid, causing the finite-difference solvers to fail.
<code>space</code>	Sets the grid spacing for the finite difference solver; default is 0.5 Å.
<code>nbuffer</code>	Sets how far away (in grid units) the boundary of the finite difference grid is away from the solute surface; default is 0 grids, i.e., automatically set to be at least a solvent probe or ion probe (diameter) away from the solute surface.
<code>nfocus</code>	Set how many successive FD calculations will be used to perform an electrostatic focussing calculation on a molecule. Default to 2, the maximum. When NFOCUS = 1, no focusing is used. It is recommended that NFOCUS = 1 when the multigrid solver is used.
<code>fscale</code>	Set the ratio between the coarse and fine grid spacings in an electrostatic focussing calculation. Default to 8.
<code>nbggrid</code>	Sets how often the finite-difference grid is regenerated; default is 1 step. For molecular dynamics simulations, it is recommended to be set to at least 100. Note that the PB solver effectively takes advantage of the fact that the electrostatic potential distribution varies very slowly during dynamics simulations. This requires that the finite-difference grid be fixed in space for the code to be efficient. However, molecules do move freely in simulations. Thus, it is necessary to regenerate the finite-difference grid occasionally to make sure a molecule is well within the grid.

6.2.6. Options to compute energy and forces

ENEOPT is the option to set a method to compute electrostatic energy and forces, and DBFOPT is phased out in this release.

<code>bcopt</code>	Boundary condition options. = 1 Boundary grid potentials are set as zero, i.e. conductor. Total electrostatic potentials and energy are computed. = 5 Computation of boundary grid potentials using all grid charges. Total electrostatic potentials and energy are computed. Default. = 6 Computation of boundary grid potentials using all grid charges. Reaction field potentials and energy are computed with the charge singularity free formalism.[252] = 10 Periodic boundary condition is used. Total electrostatic potentials and energy are computed. Can be used with SOLVOPT = 1, 2, 3, or 4 and IPB = 1 or 2. It should only be used on charge-neutral systems. If the system net charge is detected to be nonzero, it will be neutralized by applying a small neutralizing charge on each grid (i.e. a uniform plasma) before solving.
<code>eneopt</code>	Option to compute total electrostatic energy and forces. = 1 Compute total electrostatic energy and forces with the particle-particle particle-mesh (P3M) procedure outlined in Lu and Luo.[239] In doing so, energy term EPB in the output file is set to zero, while EEL includes both the reaction field energy and the Coulombic energy. The van der Waals energy is computed along with the particle-particle portion of the Coulombic energy. The electrostatic forces and dielectric boundary forces can also be computed.[239] This option requires a nonzero CUTNB and BCOPT = 5. = 2 Use dielectric boundary surface charges to compute the reaction field energy. Default. Both the Coulombic energy and the van der Waals energy are computed via summation of pairwise atomic interactions. Energy term EPB in the output file is the reaction field energy. EEL is the Coulombic energy.

6. PBSA

- = 3** Similar to the first option above, a P3M procedure is applied for both solvation and Coulombic energy and forces for larger systems.
- = 4** Similar to the third option above, a P3M procedure for the full nonlinear PB equation is applied for both solvation and Coulombic energy and forces for larger systems. A more robust and clean set of routines were used for the P3M and dielectric surface force calculations.
- frcopt** Option to compute and output electrostatic forces to a file named *force.dat* in the working directory.
- = 0** Do not compute or output atomic and total electrostatic forces. This is default.
- = 1** Reaction field forces are computed by trilinear interpolation. Dielectric boundary forces are computed using the electric field on dielectric boundary. The forces are output in the unit of kcal/mol·Å.
- = 2** Use dielectric boundary surface polarized charges to compute the reaction field forces and dielectric boundary forces [253] The forces are output in the unit of kcal/mol·Å.
- = 3** Reaction field forces are computed using dielectric boundary polarized charge. Dielectric boundary forces are computed using the electric field on dielectric boundary. [265] The forces are output in the unit of kcal/mol·Å.
- scalec** Option to compute reaction field energy and forces.
- = 0** Do not scale dielectric boundary surface charges before computing reaction field energy and forces. Default.
- = 1** Scale dielectric boundary surface charges using Gauss's law before computing reaction field energy and forces.
- cutfd** Atom-based cutoff distance to remove short-range finite-difference interactions, and to add pairwise charge-based interactions, default is 5 Å. This is used for both energy and force calculations. See Eqn (20) in Lu and Luo.[239]
- cutnb** Atom-based cutoff distance for van der Waals interactions, and pairwise Coulombic interactions when ENEOPT = 2. Default to 0. When CUTNB is set to the default value of 0, no cutoff will be used for van der Waals and Coulombic interactions, i.e., all pairwise interactions will be included. When ENEOPT = 1, this is the cutoff distance used for van der Waals interactions only. The particle-particle portion of the Coulombic interactions is computed with the cutoff of CUTFD.
- nsnba** Sets how often atom-based pairlist is generated; default is 1 step. For molecular dynamics simulations, a value of 5 is recommended.

6.2.7. Options for visualization and output

- phiout** *pbsa* can be used to output spatial distribution of electrostatic potential for visualization. [262]
- = 0** No potential file is printed out. Default.
- = 1** Electrostatic potential is printed out in a file named *pbsa.phi* in the working directory. Please refer to examples in the next section on how to display electrostatic potential on molecular surface.
- phiform** Controls the format of the electrostatic potential file.
- = 0** The electrostatic potential (kT/mol·e) is printed in the *Delphi* binary format. Default.
- = 1** The electrostatic potential (kcal/mol·e) is printed in the *Amber* ASCII format.
- = 2** The electrostatic potential (kcal/mol·e) is printed in the DX volumetric data format for use with *VMD*.

- `outlvlset` *pbsa* can be set to write the total level set, used in locating interfaces between regions of differing dielectric constant, to a DX format volumetric data file. This option will control printing of the total level set (i.e. both solute-solvent and membrane level sets combined if membrane present)
- = false** No level set file printed out. Default.
- = true** Level set printed out in a file named *pbsa_lvlset.dx*
- `outmlvlset` *pbsa* can be set to write the membrane level set, used in locating interfaces between regions of differing dielectric constant, to a DX format volumetric data file. This option controls printing a separate file for the membrane level set. Does nothing if *membraneopt* is not turned on.
- = false** No level set file printed out. Default.
- = true** Level set printed out in a file named *pbsa_lvlset.dx*
- `npbverb` When set to 1, turns on verbose mode in *pbsa*; default is 0.

6.2.8. Options for non-polar solvation models

- `decompopt` Option to select different decomposition schemes when *INP* = 2. See [243] for a detailed discussion of the different schemes. The default is 2, the σ decomposition scheme, which is the best of the three schemes studied.[243] As discussed in Ref. [243], *DECOMPOPT* = 1 is not a very accurate approach even if it is more straightforward to understand the decomposition.
- = 1** The 6/12 decomposition scheme.
- = 2** The σ decomposition scheme. Default
- = 3** The WCA decomposition scheme.
- `use_rmin` The option to set up van der Waals radii. The default is to use *rmin* to improve the agreement with TIP3P [243].
- = 0** Use atomic van der Waals σ values.
- = 1** Use atomic van der Waals *rmin* values. Default.
- `sprob` Solvent probe radius for solvent accessible surface area (SASA) used to compute the dispersion term, default to 0.557 Å in the σ decomposition scheme as optimized in Ref. [243] with respect to the TIP3P solvent and the PME treatment. Recommended values for other decomposition schemes can be found in Table 4 of [243]. If *USE_SAV* = 0 (see below), *SPROB* can be used to compute SASA for the cavity term as well. Unfortunately, the recommended value is different from that used in the dispersion term calculation as documented in Ref. [243] Thus two separate *pbsa* calculations are needed when *USE_SAV* = 0, one for the dispersion term and one for the cavity term. Therefore, please carefully read Ref. [243] before proceeding with the option of *USE_SAV* = 0. Note that *SPROB* was used for ALL three terms of solvation free energies, i.e., electrostatic, attractive, and repulsive terms in previous releases in *Amber*. However, it was found in the more recent study [243] that it was impossible to use the same probe radii for all three terms after each term was calibrated and validated with respect to the TIP3P solvent. [243, 260]
- `vprob` Solvent probe radius for molecular volume (the volume enclosed by SASA) used to compute non-polar cavity solvation free energy, default to 1.300 Å, the value optimized in Ref. [243] with respect to the TIP3P solvent. Recommended values for other decomposition schemes can be found in Tables 1-3 of Ref. [243].
- `rhow_effect` Effective water density used in the non-polar dispersion term calculation, default to 1.129 for *DECOMPOPT* = 2, the σ scheme. This was optimized in Ref. [243] with respect to the TIP3P solvent in PME. Optimized values for other decomposition schemes can be found in Table 4 of Ref. [243].

6. PBSA

<code>use_sav</code>	The option to use molecular volume (the volume enclosed by SASA) or to use molecular surface (SASA) for cavity term calculation. The default is to use the molecular volume enclosed by SASA. Recent study shows that the molecular volume approach transfers better from small training molecules to biomacromolecules. = 0 Use SASA to estimate cavity free energy. = 1 Use the molecular volume enclosed by SASA. Default.
<code>cavity_surften</code>	The regression coefficient for the linear relation between the total non-polar solvation free energy (INP = 1) or the cavity free energy (INP = 2) and SASA/volume enclosed by SASA. The default value is for INP = 2 and set to the best of three tested schemes as reported in Ref. [243], i.e. DECOMPOPT = 2, USE_RMIN = 1, and USE_SAV = 1. See recommended values in Tables 1-3 for other schemes.
<code>cavity_offset</code>	The regression offset for the linear relation between the total non-polar solvation free energy (INP = 1) or the cavity free energy (INP = 2) and SASA/volume enclosed by SASA. The default value is for INP = 2 and set to the best of three tested schemes as reported in Ref. [243], i.e. DECOMPOPT = 2, USE_RMIN = 1, and USE_SAV = 1. See recommended values in Tables 1-3 for other schemes.
<code>maxsph</code>	<i>pbsa</i> uses a numerical method to compute solvent accessible surface area.[243] MAXSPH variable gives the approximate number of dots to represent the maximum atomic solvent accessible surface, default to 400. These dots are first checked against covalently bonded atoms to see whether any of the dots are buried. The exposed dots from the first step are then checked against a non-bonded pair list with a cutoff distance of 9 Å to see whether any of the exposed dots from the first step are buried. The exposed dots of each atom after the second step then represent the solvent accessible portion of the atom and are used to compute the SASA of the atom. The molecular SASA is simply a summation of the atomic SASA's. A molecular SASA is used for both PB dielectric map assignment and for NP calculations.

6.2.9. Options to enable machine-learned solvent excluded surface models

The Machine-Learned Solvent Excluded Surface (MLSES) model, GENIUSES [266] offers a significantly more efficient generation of molecular surfaces, with over 26 times faster performance on GPUs and 4 times faster on CPUs while retaining ~95% accuracy compared to the classical solvent-excluded surface implementation referenced in the classical implementation of SES in AMBER.[255] Con2SES-3D offers improved surface construction accuracy and comparable speed performance to GENIUSES on GPUs, although it typically requires more GPU memory. An alternative, Con2SES-2D, is also available; however, Con2SES-3D is recommended due to its superior runtime efficiency. Options explicitly marked as requiring the Torch PBSA runtime necessitate the installation of LibTorch. Further details regarding the Torch PBSA runtime are provided in 6.3. A sample input file for single point calculation is provided in 6.4.4.

<code>mlses_opt</code>	Option to select the runtime for the MLSES model. To enable this feature, SASOPT = 3 is required. Users can choose from different runtime options: = 0 Use the customized Fortran function/CUDA kernel to run the GENIUSES model on a CPU/GPU. Default. = 1 Use the Torch PBSA runtime to run the GENIUSES model on a CPU or GPU. = 2 Use the Torch PBSA runtime to run the Con2SES-2D model on a CPU or GPU. = 3 Use the Torch PBSA runtime to run the Con2SES-3D model on a CPU or GPU.
------------------------	--

6.2.10. Options to enable active site focusing

Active site focusing is an extension to the electrostatic focusing method. Electrostatic focusing can be regarded as a multi-level FDPB calculation (two levels currently implemented) in which a coarse-grid solution is conducted to set up the boundary condition for the requested fine-grid solution. In the original implementation of electrostatic focusing, the fine grid always covers all the solute atoms. However in the enhanced implementation, the fine grid is allowed to cover only a local region of interest, such as an enzyme active site or ligand docking site. In such applications, most or all of the protein atoms are held frozen during a calculation while only the active site side chain and the substrate ligand are allowed to move. In principle, energies computed with the local electrostatic focusing method should correlate with those computed with the original electrostatic focusing method if the movable substrate/ligand atoms are well within the local region of interest. The “active site” or the local region is specified as a rectangular box by the following six variables:[267]

<code>xmax</code>	The upper boundary of the box in x direction.
<code>xmin</code>	The lower boundary of the box in x direction, XMAX has to be greater than XMIN.
<code>ymax</code>	The upper boundary of the box in y direction.
<code>ymin</code>	The lower boundary of the box in y direction, YMAX has to be greater than YMIN.
<code>zmax</code>	The upper boundary of the box in z direction.
<code>zmin</code>	The lower boundary of the box in z direction, ZMAX has to be greater than ZMIN.

Of course, these keywords are zero by default, i.e. the original electrostatic focusing would be invoked if these keywords remain to be the default value of zero.

6.2.11. Options to enable multiblock focusing

This option is no longer supported starting in the Amber 2018 release.

6.3. Torch PBSA runtime

LibTorch is the C++ distribution of the PyTorch deep learning framework, developed by the PyTorch team.[268] It supports flexible tensor operations and dynamic deep neural network modeling in C++, enabling developers to deploy trained PyTorch models on C++ platforms without relying on the Python interpreter. LibTorch also offers a high-performance heterogeneous tensor abstraction that facilitates array computations across various accelerators, including CPUs, GPUs, and NPUs.

PBSA aims to integrate machine learning models to accelerate several performance-critical components, such as surface construction and PB energy calculations, while also leveraging tensor computing to enhance the efficiency of numerical procedures. To this end, PBSA adopts LibTorch to achieve this objective within a pure C++ runtime environment, resulting in a dedicated runtime referred to as Torch PBSA. Details of the Torch PBSA runtime can be found in 7. All methods in PBSA that depend on Torch PBSA are explicitly labeled.

6.4. Example inputs and demonstrations of functionalities

6.4.1. Single-point calculation of solvation free energies

Normally the default *pbsa* options are capable of dealing with most situations. Users should be fully aware of the meaning of an option before they change its default value. In all the following example inputs, only the options that are different from their default values will be shown, and the explanations on the changes will be given in detail. Here is a sample input file that might be used to perform single structure calculations.

6. PBSA

```
Sample single point PB calculation
&cntrl
/
&pb
npbverb=1, istrng=150, fillratio=1.5, saopt=1,
/
```

Note that NPBVERB = 1 above. This generates much detailed information in the output file for the PB and NP calculations. A useful printout is atomic SASA data for both PB and NP calculations which may or may not use the same atomic radius definition. Since the FD solver for PB is called twice to perform electrostatic focus calculations, two PB printouts are shown for each single point calculation. For the PB calculation, a common error message can be generated when FILLRATIO is set to the default value of 2.0 for small molecules. This may cause a solute to lie outside of the focusing finite-difference grid.

In this example INP is not set and equal to the default value of 2, which calls for non-polar solvation calculation with the new method that separates cavity and dispersion interactions. The EDISPER term gives the dispersion solvation free energy, and the ECAVITY term gives the cavity solvation free energy. The default options for the NP calculation are set to the recommended values for the σ decomposition scheme and to use molecular volume to correlate with cavity free energy. You can find recommended values for other decomposition schemes and other options in Tables 1-4 of Ref. [243]. If INP is set to 1, the ECAVITY term would give the total non-polar solvation free energy.

Finally, a few words on the RADIOPT option, set to the default value of 1 instructing PB to use the optimized values instead of reading the radii from the prmtop file. Starting this release, the RADIOPT option only controls the radius definition for the PB calculation. The INP=2 calculation automatically uses the default values, such as atomic radii and solvent probes as optimized in Ref. [243]. On the other hand, the INP=1 calculation is allowed to use whatever radii that a user decides to use.

The ion strength option ISTRNG is set to 150 in unit mM, a typical value for a physiological environment. The FILLRATIO option is set to 1.5 because the biomolecule is relatively large. We set saopt to 1 because we need the information of the molecular surface area (the molecular surface is defined as the solvent excluded surface since SASOPT is set to its default value 0).

6.4.2. Implicit membrane model

pbsa now supports inclusion of an implicit membrane region in implicit solvation calculations. This feature is enabled by setting MEMBRANEOPT to 1 (default value is 0, for off). The membrane will extend the solute dielectric region to include a slab-like planar region running parallel to the xy plane. The thickness is controlled by the MTHICK option. The default is 40 Å. The membrane region will be centered on the protein center by default, and can be set to a user-provided value using the MCTRDZ option (default is 0). Neither option will have any effect unless MEMBRANEOPT is set to 1. The dielectric constant can be controlled using epsmem. We set the membrane interior dielectric constant to a value of 4.0 in this example. This is four times that of the solute which defaults to 1 (same as vacuum). The value of epsmem should always be set to a value greater than or equal to EPSIN (solute dielectric constant) and less than EPSOUT (solvent dielectric constant). These default to 1.0 and 80.0 respectively.

When using the implicit membrane model, we recommend SASOPT=0, i.e. the classical solvent excluded surface, due to its better numerical behavior. When running with the default options, the program will compute solvent excluded surfaces both with the water probe (DPROB=1.40 by default) and the membrane probe (MPROB=2.70 by default). This setting was found to be consistent with the explicit solvent MD simulations. It is also suggested that periodic boundary conditions be used to avoid unphysical edge effects. This is supported in all linear solvers. In the following we choose Periodic Incomplete Cholesky Conjugate Gradient (PICCG). So we set IPB = 1, BCOPT = 10, and SOLVOPT = 1 (default). In addition, ENEOPT needs to be set to 1 because the charge-view method (ENEOPT = 2) is not supported for this application.

```
Sample single point PB calculation with membrane region
&cntrl
ipb=1, inp=0
```



```

/
&pb
radiopt=0, nfocus=1, maxitn=200,
bcopt=10, eneopt=1, solvopt=1,
sasopt=0, membraneopt=1, epsmem = 4.0
outlvlset=true, outmlvlset=true
/

```

The MAXITN option is set to a bigger value, 200, than the default one, 100, because the linear solvers, when applied to periodic boundary conditions, seem to require slightly more iterations than non-periodic solvers to converge.

To aid in visualization of the dielectric model, the level set function, which is used to locate the interfacial surfaces between regions of differing dielectric constant, can be written to output files. Output of the total level set function, including both the solute-solvent and membrane contributions, can be written to a DX formatted volumetric data file by setting the OUTLVLSET option to “true”. The membrane contribution can be written to a separate file by setting the OUTMLVLSET option to “true”. This may take a good deal of extra time, so be sure to leave it off if you don’t want / need to visualize the levelset surface. Accordingly, NFOCUS is set to 1 because of the use of periodic boundary condition.

Finally, if calculations need to be performed on a protein with a solvent-filled channel region, this region would be identified automatically by setting PORETYPE=1.

6.4.3. Single point calculation of forces

Since *pbsa* is released for single point calculations in *AmberTools*, no energy minimization or molecular dynamics is supported. However, the PB procedure can be invoked to print out the numerical electrostatic forces for developmental purposes. Here is a sample input:

```

Sample PB force computation
&cntrl
inp=0
/
&pb
npbverb=1, radiopt=0, frcopt=2
/

```

Note that INP is set to 0 to turn off non-polar solvation interactions. RADIOPT = 0 means the atomic radii from the topology files will be used. FRCOPT is set to 2, *i.e.*, induced surface charges are used to compute the electrostatic energy and forces. Since CUTNB is equal to the default value of zero, an infinite cutoff distance is used for both Coulombic and van der Waals interactions.

6.4.4. Single point calculation with MLSES

Here is a sample input file to turn on the MLSES surface in PB calculations. Note that IPB=2 and SASOPT=3 must be set together for this model to work.

```

Sample PB input to use MLSES
&cntrl
ntx=1, imin=1, ipb=2, inp=0
/
&pb
npbverb=1, istrng=0,
epsout=80.0, epsin=1.0, dprob=1.4, radiopt=0, sasopt=3,
fillratio=1.25, nfocus=1, space=0.5,
accept=0.000001, maxitn=10000, solvopt=3,
npbopt=0, bcopt=6,

```

6. PBSA

```
eneopt=1, frcopt=0, cutnb=15, cutsa=8, cutfd=7
/
```

6.4.5. Comparing with *Delphi* results

Under identical condition, *pbsa* is highly consistent with *Delphi* in term of computed reaction field energies. In this subsection, we briefly go over the details on how you can obtain comparable energies from both programs. Apparently, you need coordinates, atomic charges, and atomic radii that have exactly the same numerical values in both the *Amber* format and the *Delphi* format, i.e., the pqr format.

For a *Delphi* computation with the following input parameters:

```
salt=0.150
ionrad=2.0
exdi=80.0
indi=1.0
scale=2.0
prbrad=1.5
perfil=50
bndcon=4
linit=1000
```

A comparable computation in *pbsa* can be obtained by using the following input file:

```
Sample PB for delphi comparison
&cntrl
ipb=1, inp=0
/
&pb
istrng=150, ivalence=1, iprob=2.0, dprob=1.5,
radiopt=0, bcopt=5, smoothopt=2, nfocus=1,
/
```

IPB is set to 1 to make sure *pbsa* uses exactly the same surface definition as *Delphi*. Note that the values of *exdi*, *indi*, *prbrad*, and *ionrad* in *Delphi* should be consistent with the values of EPSOUT, EPSIN, DPROB, and IPROB in *pbsa*, respectively. In *Delphi* *salt*=0.150 is set in the unit of M, while in *pbsa* ISTRNG = 150 is in the unit of mM. In *Delphi* the grid spacing is set as the number of grids per Å, i.e., *scale*=2.0, while in *pbsa* the grid spacing is set straight in Å as SPACE = 0.5. In *Delphi* the grid dimension is set as percentage of the solute dimension over the grid dimension, i.e., *perfil*=50, which is equivalent to the ratio of solute dimension over grid dimension set as FILLRATIO = 2 in *pbsa*. Finally, *Delphi* sets the boundary condition by *bndcon*=4 and *pbsa* sets the boundary condition as BCOPT = 5; both programs mean to use the Debye-Huckel limiting behavior for each atomic charged sphere. There are additional options in *pbsa* that do not have corresponding counterparts in *Delphi*. For example, SMOOTHOPT is used to instruct the program to use a specific dielectric boundary smoothing option, which is equivalent to that used in *Delphi* when set to 2. (see Section 6.2.3).

6.5. Visualization functions in *pbsa*

pbsa can produce volumetric data files to allow visualization of electrostatic potential and level set maps.[\[262\]](#) There are two points to note before continuing.

1. The data files generated can become quite large if small grid spacings are used since they will scale as the cube of the inverse of grid spacing
2. Unless singularity removal methods are used, the potential at grid nodes corresponding to atom centers may be quite large when compared to the potential at the molecular / atomic surface. This will often result in poor contrast during visualization of the potential map, particularly when it is used as a color map for a molecular surface.

These two points should be kept in mind when determining grid spacing. For visualization purposes, a grid spacing of about one angstrom should provide good results. If finer spacing is needed, singularity removal (BCOPT = 6) can be used to prevent poor contrast that could result from the presence of singularities. Lastly, when using grid spacings of 0.5 Å or lower, the output files may become quite large (tens, or even hundreds of megabytes each) and may take a significant amount of time (up to several seconds each) to generate.

6.5.1. Visualization of electrostatic potential using *PyMol*

pbsa can produce an electrostatic potential map for visualization in *PyMol* when setting PHIOUT = 1.[262] By default, *pbsa* outputs a file pbsa.phi in the *Delphi* binary format. The sample input file is listed below:

```
Sample PB visualization input
&cntrl
inp=0
/
&pb
npbverb=1, space=1.,
phiout=1, phiform=0
/
```

To be consistent with the surface routine of *PyMol*, the option PHIOUT = 1 instructs *pbsa* to use the radii as defined in *PyMol*. The finite-difference grid is also set to be cubic as in *Delphi*. The default DPROB value is equal to that used in *PyMol*, 1.4 Å. A large grid spacing, e.g. 1 Å or higher, is recommended for visualization purposes, as commented above.

Here is an example of loading the potential map in *PyMol*. First load the molecule in the form of prmtop and inpcrd. In our case we need to rename our prmtop file to molecule.top and inpcrd file to molecule.rst and load the molecule with commands

```
PyMol> load molecule.top
PyMol> load molecule.rst
```

The molecule will appear as an object “molecule”. Next display the surface of the molecule in the *PyMol* menu by clicking “S” and then select surface. Now import the potential map generated by *pbsa* with the command in *PyMol*

```
PyMol> load pbsa.phi
```

to create a value map object called “pbsa”. After this, create a value ramp called e_lv1 from the potential map with the command

```
PyMol> ramp_new e_lv1, pbsa, [-7, 0, 7]
```

You can assign surface_color to the e_lv1 ramp with the command

```
PyMol> set surface_color, e_lv1, molecule
```

This will display the surface with the color scale according to the potential. You can adjust the value scale, such as [-5, 0, 5], to change the color scale and use “rebuild” command to redraw the surface.

6.5.2. Writing electrostatic potential to DX format volumetric data file

To visualize the *pbsa* potential using *VMD*, you will need to set the output to DX format by changing PHIFORM = 0 to PHIFORM = 2.[262]

```
Sample PB visualization input
&cntrl
inp=0
```

6. PBSA

```
/
&pb
npbverb=1, space=1., sasopt=2,
phiout=1, phiorm=2
/
```

The program will now generate a file called `pbsa_phi.dx`. This format should be automatically recognized by VMD. It can be either loaded directly into your molecule or as a separate file.

6.5.3. Loading DX format electrostatic potential data in VMD

1. go to the “File” menu in the *VMD* Main window.
2. Select “New Molecule...”.
 - This will bring up the “Molecule File Browser” window
3. Click on the “Browse...” button in the “Molecule File Browser” window
4. Select the file “`pbsa_phi.dx`” that was generated by *pbsa* using the file selection dialogue that pops up.
 - The “Determine file type:” drop down menu should now read “DX”.
5. Click the “Load” button.

VMD will, by default, display the data with an isosurface representation.

6.5.4. Visualization Example of MLSES

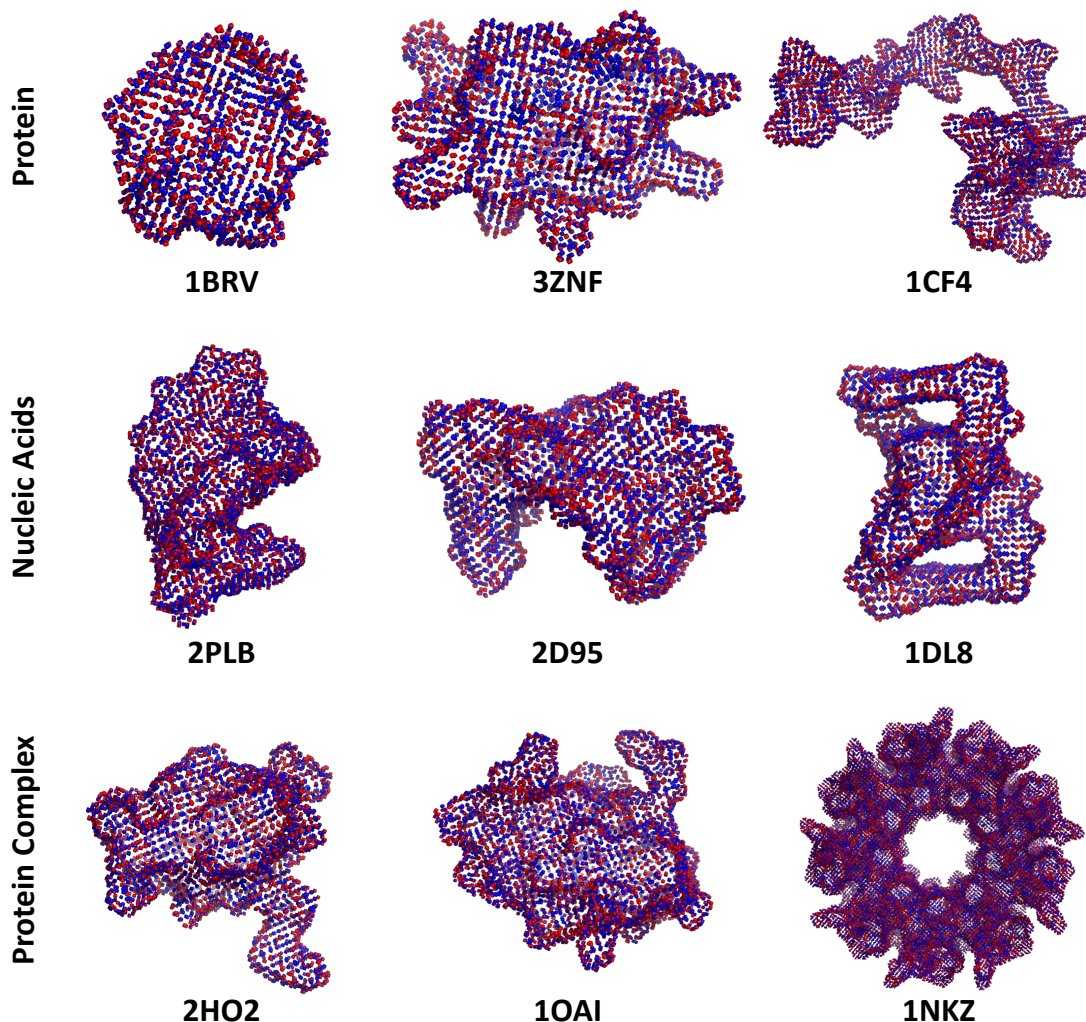
To visualize the solvent surface generated by the MLSES model, you will need to set the output file to an XYZ format data file.

```
Sample PB visualization input
&cntrl
ntx=1, imin=1, ipb=2, inp=0
/
&pb
npbverb=1, istrng=0,
epsout=80.0, epsin=1.0, dprob=1.4, radiopt=0, sasopt=3,
fillratio=1.25, nfocus=1, space=0.5, mlses_bench=0,
accept=0.000001, maxitn=10000, solvopt=3,
npbopt=0, bcopt=6,
eneopt=1, frcopt=0, cutnb=15, cutsa=8, cutfd=7
/
```

The program will now generate a file called `interface.dot`. After adding the total atom number as the first line of the file, rename it to `interface.xyz`. This format should be automatically recognized by VMD.

Rendering Examples of GENIUSES

The following image displays the validation of our GENIUSES [266](blue) on protein, unseen nucleic acids, and protein complex datasets, compared with the classical SES [255](red). The nearly indistinguishable superimposed surfaces suggest that GENIUSES closely mirrors SES across various shapes and conformations while significantly reduced computation time, particularly on GPUs.



6.5.5. Changing the representation model

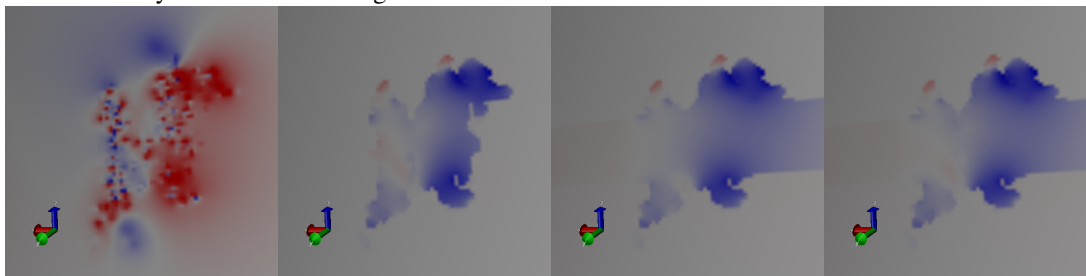
1. Select “Representations...” from the “Graphics” menu in the “VMD Main” window
 - The “Graphical Representations” window should pop up
2. Select the object corresponding to the volumetric data you loaded from the “Selected Molecule” pull down menu
3. Click on the representation you wish to change
 - There should be one present for the isosurface being displayed
4. Click on the “Draw style” tab if it is not already selected
5. Select “Volume” from the “Coloring Method” pull down menu if it is not already chosen
 - Another pull down menu will appear next to it.
 - If you have multiple data files loaded for the same object you can choose which is used to color your chosen draw method representation here
6. The “Drawing Method” pull down menu will let you choose a different visual representation model.

6. PBSA

- To directly visualize potential data, use either “Isosurface” or “Volume Slice”
- *VMD* can also be used to visualize the corresponding electric field by choosing “Field Lines”.

Displayed below are Volume Slice representations of electrostatic potential maps generated for an aquaporin system. Computations were run using the periodic conjugate gradient solver for a 1 Å grid spacing, and FILLRATIO of 2.0. For the systems using implicit water, the charge singularity removal methodology was used.

From Left to right: Vacuum, Water only, Water and 20 Å slab-like membrane, Water and 20 Å slab-like membrane with 6 Å cylindrical channel region removed.



Often, the data ranges will not be consistent between potential distributions for different implicit solvent setups. E.g. the range of the electrostatic values seen for vacuum will likely be larger than the range for implicit water. The range of values displayed can be set manually to provide consistent color scaling for comparison.

6.5.6. Adjusting the color scale of the color map

1. Select “Colors...” from the “Graphics” menu in the “*VMD* Main” window
 - This should cause the “Color Controls” window to pop up
2. Select the “Color Scale” tab
 - The color scheme can be selected from the “Method” pull down menu
 - The “Offset” and “Midpoint” sliders can be used to adjust the scaling of the color map.
 - If singularities are present, it may be difficult to get a good scaling for volume maps generated with fine grid spacings. In this case, either re-run with singularity removal on, or set the color scale range manually as shown in the next section.

When singularity removal is not employed, the presence of singularities will cause the range of the electrostatic potential distribution near the atom centers to be much wider than near the molecular surface. This typically results in very poor contrast particularly for implicit solvent since the high dielectric constant in the solvent region will amplify the effect. This can be compensated for by manually setting the Color Scale Data Range.

6.5.7. Changing the color scale range

1. Select desired representation to modify
2. Select “Volume” Coloring Method and Select the desired volumetric map to rescale from the pull down menu.
 - Each time you change the volumetric map being displayed, you will need to repeat this, so it is a good idea to make multiple representations for each potential data set rather than switching between them on the same representation.
3. Select the “Trajectory” tab
4. You should see the automatically computed range in the “Color Scale Data Range:” boxes. The left hand box controls the minimum value for the range, the right hand box controls the maximum value for the range.

5. Set the minimum and maximum values as needed to improve the contrast. Often the inner 10% to 30% of the total (automatic) range will give good contrast for a one angstrom grid spacing.
6. Click on the “Set” button when you are finished
7. To return to the automatic scaling that was originally calculated by *VMD*, click the “Autoscale” button.

Electrostatic potential data can also be used as a color map for other drawing methods. You will need to first load the data into the molecule you wish to display.

6.5.8. Loading electrostatic potential data into an existing molecule

The names of the files are used as labels, so it is useful to rename them from “pbsa_phi.dx” to something more descriptive before loading.

1. Select the molecule you wish to display the potential color map on in the “*VMD* Main” window
2. Go to the “File” menu in the *VMD* Main window.
3. Select “Load Data Into Molecule...”.
 - This will bring up the “Molecule File Browser” window
4. Click on the “Browse...” button in the “Molecule File Browser” window
5. Select the file “pbsa_phi.dx” that was generated by pbsa using the file selection dialogue that pops up.
 - The “Determine file type:” drop down menu should now read “DX”.
6. Click the “Load” button.

The data should now be loaded into the molecule you selected.

6.5.9. Using the electrostatic potential data as a color map

Once you have loaded a volumetric data file into a molecule, it can be used to generate a color map for any representations of that molecules model.

1. Open the “Graphical Representations” window if it is not already open
 - Select “Representations...” from the “Graphics” menu in the “*VMD* Main” window
2. Select the molecule you loaded the data into from the “Selected Molecule” pull down menu
3. Select the representation you wish to map the potential color map onto
4. Select the “Draw Style” tab if it is not already selected
5. Select “Volume” from the “Coloring Method” pull down menu
 - Another pull down menu should appear next to it
 - Choose the selection that corresponds to the data you just loaded, it should be the last one on the list if it is the last one that was loaded.

VMD will attempt to automatically scale the color mapping used for Volumetric data that you load. The color scale may be manually adjusted if needed (see previous section)

6.5.10. Loading and displaying the level set map

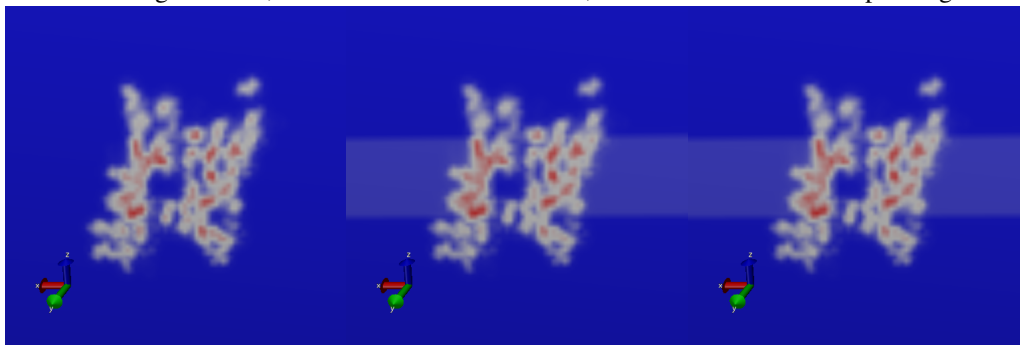
The level set used by *pbsa* to model the solute - solvent interface can be written to an output file in DX format by setting OUTLVLSET to “true” in the input file.

```
Sample PB visualization input
&cntrl
inp=0
/
&pb
npbverb=1, space=1., sasopt=2,
phiout=1, phiform=2,
outlvlset=true
/
```

The level set will be written to a DX format volumetric data file named “pbsa_lvlset.dx”. This file can be used to visualize the corresponding molecular surface. The level set file is loaded into *VMD* in the same manner as an electrostatic potential data file. Cross sections can be viewed using the “Volume Slice” representation.

Shown below are the level sets for the aquaporin systems shown previously (no level set is shown for vacuum as there is no dielectric interface being modeled in that system)

From left to right: Water, Water + Slab-like membrane, Water + Membrane with pore region



6.5.11. Visualizing the molecular surface as an isosurface of the level set

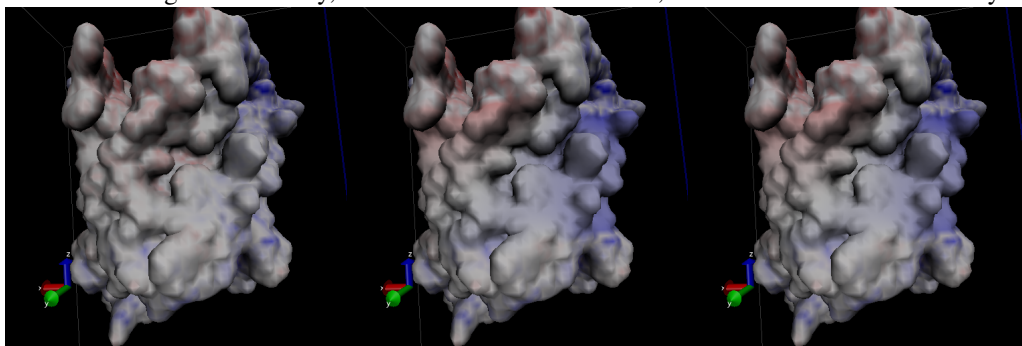
The level set is constructed such that the molecular surface is the locus of all points where the level set is zero. This allows us to use the Isosurface representation in *VMD* to display the solvent excluded surface by setting the “Isovalue” to 0. Alternatively, if we wish to view the potential just outside the surface, we can set the “Isovalue” to a number slightly higher than 0. E.g. 0.1 or 0.01.

1. Load the level set data file into the molecule.
 - This is done using the same procedure as loading an electrostatic potential data file, but the level set data file will be chosen instead of the potential data file.
2. Create a new Isosurface representation in the “Graphical Representations” window.
3. Select the volume map for the level set from the pull down menu
4. Choose an “Isovalue” at or slightly above 0.
5. Using the “Coloring Method” pull down menu, you may also use a previously loaded electrostatic potential data file as a color map by selecting “Volume” and then selecting the appropriate volume map from the pull down menu that appears.
 - *VMD* will automatically assign color scale range every time.

- To compare multiple potential maps, it is often desirable to use the same color scale range for each. The best way to do this is to make a new representation for each potential map and manually assign the same color scale range to be identical for each (see previous section)

The examples below were generated for Aquaporin (1IH5 in the protein data bank) under various implicit solvent options using a FILLRATIO of 2.0, grid spacing of 1Å. For each calculation, the periodic conjugate gradient solver with singularity removal was used. The level set for the system modeling implicit water was used to build the isosurfaces. The electrostatic potential data files were then overlaid as color maps with the color scale ranges set to [-80000,80000].

From Left to right: Water only, Water + Slab Like Membrane, Water + Membrane with 6Å cylindrical pore.



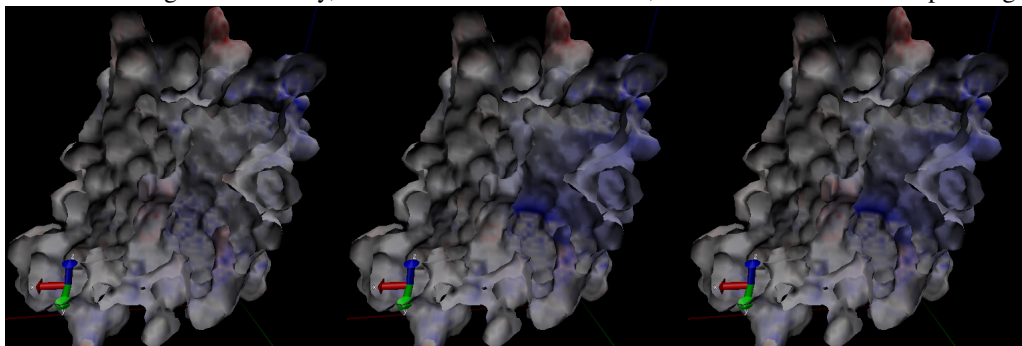
6.5.12. Visualizing interior channels, voids, and solvent pockets

One of the common roles for membrane proteins is to act as a transmembrane channel, to allow specific substance to pass from one side of a membrane to another. Features such as solvent / ion channels or internal voids will often be occluded from view by the exterior surface. One option that can allow these to be viewed is to use the clipping plane tool in VMD.

1. Open the “Extensions” pull down menu in the “VMD Main” window and go to the “Visualization” submenu and select “Clipping Plane Tool”.
2. The “Clip Tool” window should pop up.
3. The “Distance” slider allows clipping to be set
4. The “Normal” slider sets the normal of the clipping plane.
 - The “flip” button on the right will let you clip from front to back, which will be useful to clip the occluding exterior surface from the view and reveal the interior.

The clipping tool was used to reveal the internal pore region for the aquaporin system setups used in the previous section.

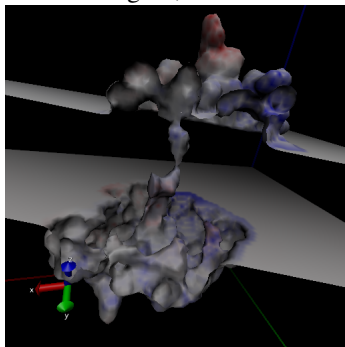
From Left to right: Water only, Water + Slab like Membrane, Water + Membrane with pore region excluded.



6. PBSA

As an alternative, the level set map generated using `PORTYPE=1` with the implicit membrane option will allow a cylindrical region to be excluded from the membrane level set. The corresponding isosurface will show any interior cavities or voids which fall within this region for isovalues at or slightly above 0 (since the level set at the membrane-solute interface will be below 0). See the previous section for details on writing and loading the level set file.

Shown below is the level set isosurface for the aquaporin system with implicit water plus a membrane with a cylindrical region removed. The corresponding potential data was again overlayed as a color map. The surface of the channel region, and the membrane-solvent interface planes are now clearly visible.



6.5.13. Importing / Modifying Atomic Radii to VMD from the prmtop file

Currently, VMD does not support loading radii for atoms directly from the prmtop file when it loads a molecule. These values can be loaded relatively easily using the tkconsole, however. To do so:

1. select "Tk Console" from the "Extensions" menu in the "VMD Main" window.
 - The "VMD TkConsole" window will then open
2. Be sure that the atom you want to import radii for is the top molecule on the list in the VMD Main window. If it is not, you will need to replace "top" with the appropriate ID
3. Type or copy and paste the following lines, but DO NOT hit enter yet.

```
set prot [atomselect ``top`` all]
$prot set radius {#RadiiList#}
```

4. You will now need to replace #RadiiList# with the one from the prmtop file.
 - a) Open the prmtop file for the molecule using a text editor
 - b) find the section that starts with "%FLAG RADII"
 - c) Highlight/Select the list of numbers that follows "%FORMAT(5E16.8)"
 - d) Copy the list (usually done by selecting "Copy" from the "Edit" menu in your text editor)
 - e) Go back to the "VMD TkConsole" window
 - f) Highlight/Select #RadiiList#
 - g) Select "Paste Ctrl-v" from the "Edit" menu in the "VMD TkConsole" window
5. Now hit return
 - If this was successful, you should now have the correct radii for each atom in the molecule.
 - you can have the console print the list of all radii by typing:

```
$prot get radius
```

- For a more human readable printout, use:

```
for {set ind 0} {$ind<[llength $rad]} {incr ind} \
{puts "Atom $ind radius is [lindex $rad $ind]}
```

These radii are used by VMD to display the VDW surface (made by selecting “VDW” from the “Drawing Method” pull down menu in the “Graphical Representations” window). One useful trick is to set them to be a small amount larger (say .01 Å) than those used to generate the surface. This will ensure that the color map will represent the external field just outside of the molecule. To modify the radii type or copy the following in the Tk Console:

```
set rad [$prot get radius]
for {set ind 0} {$ind<[llength $rad]} {incr ind} \
{lset rad $ind [expr [lindex $rad $ind] +.01]}
```

The above code will increase all atomic radii by .01 angstroms. This can be changed if a different amount is desired. (The code assumes you already followed steps 1 through 5 otherwise \$prot will be undefined!)

6.6. *pbsa* in *sander* and NAB

6.6.1. Electrostatic forces/gradients in *pbsa*

Force calculation in the finite-difference Poisson-Boltzmann method is straightforward, though not a trivial issue. It can be shown, by using the variation of the electrostatic free energy, that the electrostatic force density consists of three components, viz., the reaction field force, the dielectric boundary force, and the ionic force. [269] Since the ionic force is much smaller in absolute value than the other two components, we only include the reaction field force and the dielectric boundary force in this release.

The reaction field force only exists where there are atomic charges, so that it is straightforward to be mapped onto atoms. In contrast, the dielectric boundary force exists on the molecular surface where the dielectric constant changes. The surface force, or pressure, cannot be easily mapped onto atoms. This is because a force-mapping procedure from the molecular surface to atoms apparently needs the derivatives of molecular surface with respect to atomic positions. However such derivatives do not exist for the widely used molecular surface definition, i.e., the solvent excluded surface (SES). We are actively developing an analytical molecular surface definition that is consistent with the widely used SES definition for the numerical PB methods so that this difficulty will be overcome in future releases.

Temporarily, a partial solution in the mapping of dielectric boundary force as described by Gilson et al[269] is implemented for PB dynamics and minimization when the SES definition is used. The stability of the MD simulation has been much improved with a more accurate mapping method of analytical SES.

6.6.2. Example for *pbsa* in *sander*

All *pbsa* functionalities are available in *sander* and all input options are exactly the same as in the standalone *pbsa*. An apparent exception is IPB: you need to really set IPB to nonzero in order to invoke *pbsa* functionalities. All other default values of PB options in *sander* are same as those in *pbsa* for single point calculations, whereas there are some options that have different recommended or default values when PB minimization or dynamics is enabled. These options are

```
space=0.25
arcres=0.125
fscale=4
eneopt=2
bcopt=6
frcopt=2
```

The SPACE, ARCREC and FSCALE are all set for higher resolution of the grid so that the force calculation can be more accurate. The charge view method (ENEOPT = 2, FRCOPT = 2) is used here because it has been tested

6. PBSA

to be able to run stable molecular dynamics simulations. Plus, BCOPT is set to 6 to remove charge singularity for the same stability purpose. An example input for PBMD is given as follows

```
Sample PB visualization input
&cntrl
imin=0, ntx=1, irest=0,
ipb=2, ntb=0,
ntc=2, ntf=2,
tempi=100, temp0=100, ntt=3, gamma_ln=1,
nstlim=100000, dt=0.002,
ntpr=100, ntwr=100000, ntwx=100,
/
&pb
npbgrid=500, nsnba=5,
/
```

IPB is explicitly set to 2 to enable PB dynamics. The NPBGRID option is set to 500, which means the finite difference grid is regenerated every 500 dynamics steps. NSNBA = 5 means the atom-based pairlist is generated every 5 steps. Please refer to Chapter 22 for the other &cntrl options. Note that the above input can be used with *sander* only.

6.7. GPU accelerated *pbsa*

The GPU version of *pbsa* is called *pbsa.cuda*. Starting from the Amber 2019 release, some bottleneck setup routines of *pbsa* are also ported into the GPU code. A new biconjugate gradient (BiCG) GPU solver is added for solving the linear system using the second-order IIM (IPB=6)[256] or improved harmonic average methods (IPB=7/8),[257] which generate unsymmetrical matrices. Together with the GPU-supported solvers, *pbsa.cuda* is fully GPU-enabled. The workflow and additional bottlenecks are still in the process of optimization. Based on the *pbsa.cuda*, a GPU-supported MMPBSA is under development.

For the numerical solver phase, our test shows that Geometric Multigrid (MG), Jacobi-preconditioned CG, and Red-black SOR are among the optimal ones.[236][237] Our analysis shows that a speedup ratio of about 7 can be achieved for the overall time, while depending on the solvers and tested systems. Note that the timing measurement is preliminary and we expect more efficiency gain as the optimization is ongoing.

While the GPU code is considered to be production ready, it is still evolving and has not been tested to the same extent as the CPU code. Users should exercise caution when using *pbsa.cuda*. The error checking on the GPU is not as verbose as it is on the CPU. In particular, simulation failures such as failed PB setup or other simulation instabilities, may manifest themselves as CUDA launch errors or GPU download failures. These are not informative error messages. If you encounter problems during a simulation on the GPU you should first try to run the identical simulation on the CPU to ensure that it is not your simulation setup causing the problems. Feedback and questions should be posted to the Amber mailing list (see <http://lists.ambermd.org/>). Future development will aim for more robust code and user-friendly interface, and more performance-boost.

This section of the manual describes supported features, accuracy and memory considerations, installation and other aspects of *pbsa.cuda* at the time of the release. Note that the rapidly changing nature of this field means the frequent updates are likely. You should refer to the AmberTools update page (see <https://ambermd.org/bugfixesat.html>) for the most up to date information.

6.7.1. Supported features

pbsa.cuda supports only linear FDPB solvers. The available solver options for this release are MG, Jacobi-preconditioned CG, Red-black SOR. The BiCG solver is also available for solving linear systems with unsymmetrical matrices. While among the available solvers, MG is clearly the best solver for large systems as shown in our analysis. To use this feature, the solver option of *pbsa.cuda* must be specified as:

```
solvopt=2 (for MG)
```

or

```
solvopt=3 (for Jacobi-preconditioned CG)
```

or

```
solvopt=4 (for Red-black SOR)
```

MG solver is very fast to converge, usually in a few steps with the acceptance criterion of 10^{-4} . For a higher criterion such as 10^{-6} for very large systems, the MG solver may fail to converge due to the single precision used. To overcome this issue, we have hooked up the MG solver to the Jacobi-preconditioned CG when the residual norm no longer decreases rapidly, to utilize both the efficiency of MG and the stability of Jacobi-preconditioned CG. Make sure you reset MAXITN to a much larger number, i.e. 5000 (versus the default value of 100 for the default solver). This is to prevent premature termination of the Jacobi-preconditioned CG solver. Currently, the free boundary condition or the conductor boundary condition (NBC) is supported for all GPU solvers. In addition, the periodic boundary condition (PBC) is also supported for the Jacobi-preconditioned CG solver or the BiCG solver. The latter option is useful when simulating periodic systems such as those with membranes. The boundary condition options to use are:

```
bcopt=5, or 1 (for NBC)
```

or

```
bcopt=10 (for PBC)
```

We strongly recommend BCOPT=1 for NBC. This is the conductor boundary and has zero cost to set up, but its solvation energies are very close to those with BCOPT=5 for typical proteins that we have tested. Once SOLVOPT and BCOPT options are set as above, all other standard serial *pbsa* features are supported as usual; you should refer to the previous sections on the usage of the CPU version of *pbsa*. An example input of single point solvation free energy calculation using the MG solver in *pbsa.cuda* is as follows:

```
&cntrl
  ntx=1, imin=1, ipb=2, inp=0
/  
&pb
  npbverb=1, istrng=0, epsout=80.0, epsin=1.0, space=.5,  

  accept=0.0001, dprob=1.4, radiopt=1, fillratio=1.5,  

  smoothopt=0, arcres=0.0625, nfocus=1,  

  bcopt=1, solvopt=2, maxitn=3000
/  
&end
```

6.7.2. Advanced PB algorithms on the GPU platform

A set of novel PB algorithms have also been developed and implemented into the GPU platform, i.e., IPB=6 for the analytical IIM,[256] IPB=7 for the X-factor harmonic average method, and IPB=8 for the second order harmonic average method.[257] These new methods are more elaborative than the classic harmonic average method (IPB=1/2) in handling the solute/solvent interface conditions, and thus give more accurate results. The analytical IIM (IPB=6) is now a recommended substitution of the old IIM algorithm (IPB=4), which employs analytical routines for setting up the linear system and is more stable. Between IPB=7 and IPB=8, the former is recommended for most situations, as it has a better balance between efficiency and accuracy. It can also reproduce the most accurate analytical IIM very well while requiring only third of its executing time. An example input of using these PB algorithms in *pbsa.cuda* is as follows, notice that all three methods require *sasopt=2* and all uses the BiCG solver only.

```
&cntrl
  ntx=1, imin=1, ipb=6/7/8, inp=0
/  
&end
```

6. PBSA

```
&pb
  npbverb=0, istrng=0, epsout=80.0, epsin=1.0, space=.5,
  accept=0.0001, dprob=1.4, radiopt=0, fillratio=1.5,
  smoothopt=0, nfocus=1, sasopt=2,
  bcopt=2, maxitn=3000, cutnb=15, cutsa=8, cutfd=7
/
```

6.7.3. Supported GPUs

pbsa.cuda has been developed based on the NVIDIA CUDA environment and thus only runs on NVIDIA GPUs at present. Since the GPU code is written in the single precision mode thus there is no requirement for GPU hardware to support double precision calculations. Consistent with the Amber CUDA requirements, compute capability 3.0 or above is required. We tested the released code and found it functions well on multiple NVIDIA GPUs, including Quadro P5000, TITAN Xp, GeForce GTX 1080, and GeForce RTX 2080. We expect that most mid- to high-end GPUs are supported.

Currently selection of which GPU is used for single GPU runs is automatic if the GPUs are set to process-exclusive mode (`nvidia-smi -c 3`), but the recommended approach is to use the `CUDA_VISIBLE_DEVICES` environment variable to select which GPU should be used. More details are provided in the section [6.7.5](#).

6.7.4. Accuracy consideration and memory usage

pbsa.cuda was developed in single precision as single precision operations are widely supported with high efficiency on most consumer-grade GPUs. Nevertheless, double precision operations are possible, but are at a significant performance disadvantage. Specifically we adopted a hybrid precision scheme: the linear system solution uses single precision, while the linear system setup (i.e. molecular surface and mapping of dielectric constants etc) and the post-processing of energy and force use double precision, except that with `IPB=2`, the reaction energy calculation, the level set density evaluation and the surface area non-bonded list determination use single precision as they have been ported to GPUs. Extensive tests of electrostatic solvation energy shows that correlation coefficients between hybrid and double precision codes are 1.0 for both 10^{-3} and 10^{-6} convergence criteria. Maximum relative errors are 3.9×10^{-3} and 5.8×10^{-6} , respectively.

Memory usage is crucial for GPU implementations since memory is often limited on most consumer-grade GPUs. In the Jacobi-preconditioned CG implementation, typical GPU memory usage is about $92 \times N_{grid}$ bytes, where N_{grid} is the number of grid nodes when discretizing the system with the finite difference method. While in the MG implementation, where the unified memory is used, the typical GPU memory usage is about $75 \times N_{grid}$ bytes. If the MG-Jacobi-PCG hybrid solver is involved in the computation with tighter convergence criteria, the typical GPU memory usage is about $135 \times N_{grid}$ bytes. Our analysis of the MG solver showed that NVIDIA Titan Xp cards, which have 12 GB GPU memory, are sufficient to successfully run all our 144 stress tests until host memory hit the limit first. On the older NVIDIA GTX 980 Ti cards with ~ 6 GB GPU memory, the MG implementation is able to successfully complete calculations with ~ 75.0 million grid points given sufficient host memory. Worth noting is that for extremely large grids, for example those with at least one billion grid points, the MG implementation generally requires about 70 GB memory, which is far beyond the available memory on most consumer-grade GPU cards. You can refer to NVIDIA hardware manage tool *nvidia-smi* to obtain the runtime memory allocation information.

6.7.5. Installation and testing

pbsa.cuda must be built separately from the standard serial *pbsa* installation. Before attempting to build the GPU version of *pbsa*, we recommend you first build and test at least the serial version of Amber and AmberTools. This would help to ensure that issues related to standard compilation on your hardware and operating system are resolved before you work with the more demanding GPU-related compilation and testing issues. Of course, you should also be familiar with the Amber compilation and test procedures.

It is assumed that you have already correctly installed and tested the CUDA environment. Additionally the environment variable `CUDA_HOME` should be set to point to your NVIDIA Toolkit installation and `$CUDA_HOME/bin/`

should be in your \$PATH. We recommend users to use CUDA 9.x or CUDA 10.x to use the MG solver, which relies on advanced data managements, such as unified memory, which are only available in CUDA 8.0 or higher.

To build and install *pbsa.cuda*, please follow the general instructions for installing CUDA programs, in Sec. 23.6.4. Next you can run the tests using the default GPU with:

```
cd $AMBERHOME/AmberTools/test
export CUDA_VISIBLE_DEVICES=1    # choose the device you wish to test
make test.cuda
```

Note on some intel platforms, you need to use a larger stack size other than the system default setting to avoid stack overflow fails when running *pbsa.cuda*. The following command should do the trick:

```
ulimit -s unlimited
```

To determine the device ID for the available hardware in your system, you can run NVIDIA's *deviceQuery* executable included in the CUDA SDK, after unsetting *CUDA_VISIBLE_DEVICES* environment variable:

```
unset CUDA_VISIBLE_DEVICES
deviceQuery
```


7. Torch PBSA

Yongxian Wu

Recent advances in machine learning have enabled a new class of efficient and accurate probabilistic models, accelerating electrostatic calculations in implicit solvent models. LibTorch is a C++ runtime library developed by the PyTorch team.[268] It facilitates flexible tensor computations and dynamic deep neural network modeling. With LibTorch, developers can seamlessly deploy their trained neural network models built on PyTorch to their desired C++ platforms, bypassing the overhead of a Python interpreter. Additionally, the LibTorch library offers general-purpose, high-performance numerical computing capabilities with the abstraction of tensors. PBSA aims to integrate machine learning models to accelerate several performance-critical components, such as surface construction and PB energy calculations, while also leveraging tensor computing to enhance the efficiency of numerical procedures. To this end, PBSA adopts LibTorch to achieve this objective within a pure C++ runtime environment, resulting in a dedicated runtime referred to as Torch PBSA. Currently, Torch PBSA introduces support for neural network-based surface construction within the Poisson–Boltzmann Surface Area (PBSA) framework. Two models, GENIUSES and Con2SES, are implemented in Torch PBSA to generate solvent-excluded surfaces (SES) for downstream electrostatic and solvation energy calculations. These models are seamlessly integrated into the PBSA through Torch PBSA runtime, offering GPU-accelerated, high-throughput performance with minimal loss of accuracy. We expect more features will be implemented by properly utilizing the tensor computing abstraction provided by LibTorch.

7.1. LibTorch installation

Torch PBSA requires LibTorch as the backend to support array computations and neural network inference. LibTorch is a C++ runtime library developed by the PyTorch team.[268] It facilitates flexible tensor computations and dynamic deep neural network modeling. With LibTorch, developers can seamlessly deploy their trained neural network models built on PyTorch to their desired C++ platforms, bypassing the overhead of a Python interpreter. Additionally, the LibTorch library offers general-purpose, high-performance numerical computing capabilities with the abstraction of tensors, which is beneficial for AMBER developers seeking to create efficient numerical algorithms and integrate custom machine learning models into the AMBER software suite.

There are two methods to enable LibTorch in AMBER:

- **Built-in mode**

- In this mode, LibTorch is automatically installed during the AMBER configuration process. It is integrated into the AMBER project through carefully defined configuration settings to ensure compatibility with other AMBER components. Therefore, the built-in LibTorch is the recommended option, unless a specific use case requires an older version—for instance, due to unsupported CUDA versions. Currently, LibTorch v2.5.1 is included as the latest supported version. It supports computation on CPUs as well as CUDA versions ≥ 11.8 and ≥ 12.1 . The specific CUDA version is automatically determined by the user’s configuration. To enable the built-in LibTorch, specify the following flags prior to running the CMake configuration:

```
-DLIBTORCH=ON    # Enable LibTorch
```

- **User-installed mode**

- In this mode, users can include custom LibTorch manually and point CMake to the local installation path. For installation instructions, refer to the official PyTorch website: <https://pytorch.org/>. Then configure CMake with:


```
-DLIBTORCH=ON      # Enable LibTorch
-DTORCH_HOME=<PATH_TO_LIBTORCH>  # Path to LibTorch root directory
```

7.2. Usage

7.2.1. Machine-learned solvent excluded surface model (MLSES)

Surface definitions such as solvent-accessible surface (SAS), solvent-excluded surface (SES), and van der Waals (vdW) surface play a critical role in modeling solvation effects in biomolecular simulations. Among these, the SES is widely used in PB-based models for defining the dielectric boundary between solute and solvent. However, classical SES construction methods rely on geometric algorithms that are computationally intensive and sensitive to grid resolution, posing limitations in large-scale simulations and high-throughput applications such as virtual screening.

To address these challenges, Torch PBSA introduces two machine learning-based methods for SES construction:

- **GENIUSES** (Grid-robust Efficient Neural Interface for Universal Solvent-Excluded Surface) [266] is a neural framework based on point cloud processing, optimized for grid robustness and cross-platform efficiency.
- **Con2SES** introduces a convolutional neural network framework with learnable 2D and 3D kernels, explicitly modeling grid-context interactions, making it highly accurate in complex molecular interiors.

These models replace traditional geometric algorithms with trained neural networks that learn surface-level set distributions from data. The neural models are deployed using LibTorch, the C++ backend of PyTorch, enabling efficient inference without requiring a Python interpreter. When compiled with GPU support, Torch PBSA offers up to 28× speedup over classical AMBER SES while maintaining high accuracy.

7.2.1.1. GENIUSES

GENIUSES [266] is a platform-agnostic surface generation framework that combines point cloud-based molecular representation with neural network inference. The model operates using dense matrix multiplications, which can be efficiently executed on both CPU and GPU platforms. GENIUSES is designed to be robust to variations in grid spacing and has been benchmarked on multiple datasets, achieving ~95% fidelity in SES reconstruction compared to the classical AMBER implementation.

Key features of GENIUSES include:

- Matrix-multiplication-based inference for efficient execution on diverse hardware.
- Robustness to grid resolution.
- High scalability for large biomolecular systems.
- A custom CUDA implementation to reduce runtime overhead beyond the LibTorch framework.

GENIUSES has been fully integrated into AMBER and is accessible via Torch PBSA. Its speed advantage (~26–33× on GPU) makes it suitable for high-throughput workflows and large-scale PBSA calculations. To enable GENIUSES in PBSA, please follow the option description in [6.2.9](#).

7.2.1.2. Con2SES

Con2SES is a convolutional neural network model that incorporates grid-context interactions to improve SES construction accuracy, especially in complex interior regions of molecules. In contrast to grid-independent methods that treat each grid point in isolation, Con2SES leverages 2D and 3D convolutional kernels to capture local neighborhood information.

Key features of Con2SES include:

7. Torch PBSA

- Explicit modeling of many-body grid-context interactions.
- Sliding voxel window strategy for scalable inference.
- High accuracy in interior surface modeling and cavity representation.
- ~99% accuracy and ~28× speedup compared to classical SES.

Con2SES includes both a 2D and 3D variant, with the 3D model demonstrating superior performance across diverse molecular test sets. Despite incorporating more contextual information, the runtime performance of Con2SES-3D remains competitive due to its efficient GPU-accelerated implementation, which is recommended over Con2SES-2D. To enable Con2SES in PBSA, please follow the option description in [6.2.9](#).

7.2.2. Example Input

An example input for a single-point solvation free energy calculation using the MLSES modules in *pbsa* is shown below:

```
&cntrl
  ntx=1, imin=1, ipb=2, inp=0
/
&pb
  npbverb=0, istrng=0,
  epsout=80.0, epsin=1.0, dprob=1.4, radiopt=0, sasopt=3,
  fillratio=1.5, nfocus=1, space=0.95,
  accept=0.000001, maxitn=100000, solvopt=3,
  npbopt=0, bcopt=6,
  eneopt=1, frcopt=0, cutnb=15, cutsa=8, cutfd=7,
  mlses_opt=${mlses_opt},
# See Section 6.2.9 for the runtime option number.
/
```

8. Reference Interaction Site Model

In addition to explicit and continuum dielectric implicit solvation models, Amber also has a third type of solvation model for molecular mechanics simulations, the reference interaction site model (RISM) of molecular solvation[270–283]. 3D-RISM may be used with both open [284] and periodic boundaries [285]. In AmberTools, 1D-RISM is available as `rism1d`. 3D-RISM is available as an option in `MMPBSA.py` and `sander`. `rism3d.snglpnt` is a simplified, standalone interface, ideal for calculating solvation thermodynamics on individual structures and trajectories. Details specific to using `sander` and `sander.MPI` can be found in Chapter 22.

When using 3D-RISM, please cite references [270, 280–282]. Additional references are provided for some options (see 8.5.2 and 8.6.1).

8.1. Introduction

RISM is an inherently microscopic approach, calculating the equilibrium distribution of the solvent, from which all thermodynamic properties are then determined. Specifically, RISM is an approximate solution to the Ornstein-Zernike (OZ) equation[271, 280, 281, 286, 287]

$$h(r_{12}, \Omega_1, \Omega_2) = c(r_{12}, \Omega_1, \Omega_2) + \rho \int d\mathbf{r}_3 d\Omega_3 c(r_{13}, \Omega_1, \Omega_3) h(r_{32}, \Omega_3, \Omega_2), \quad (8.1)$$

where r_{12} is the separation between particles 1 and 2 while Ω_1 and Ω_2 are their orientations relative to the vector $\mathbf{r}_{12}$. The two functions in this relation are h , the total correlation function, and c , the direct correlation function. The total correlation function is defined as

$$h_{ab}(r_{ab}, \Omega_a, \Omega_b) \equiv g_{ab}(r_{ab}, \Omega_a, \Omega_b) - 1,$$

where g_{ab} is the pair-distribution function, which gives the conditional density distribution of species b about a . In cases where only radial separation is considered, for example by orientational averaging over site α of species a and site γ of species b , gives the familiar one dimensional site-site radial distribution function, $g_{\alpha\gamma}(r_{\alpha\gamma})$.

For real mixtures, it is often convenient to speak in terms of a solvent, V, of high concentration and a solute, U, of low concentration. A generic case of solvation is infinite dilution of the solute, i.e., $\rho^U \rightarrow 0$. We can rewrite Equation (8.1), in the limit of infinite dilution, as a set of three equations:

$$h^{VV}(r_{12}, \Omega_1, \Omega_2) = c^{VV}(r_{12}, \Omega_1, \Omega_2) + \rho^V \int d\mathbf{r}_3 d\Omega_3 c^{VV}(r_{13}, \Omega_1, \Omega_3) h^{VV}(r_{32}, \Omega_3, \Omega_2), \quad (8.2)$$

$$h^{UV}(r_{12}, \Omega_1, \Omega_2) = c^{UV}(r_{12}, \Omega_1, \Omega_2) + \rho^V \int d\mathbf{r}_3 d\Omega_3 c^{UV}(r_{13}, \Omega_1, \Omega_3) h^{VV}(r_{32}, \Omega_3, \Omega_2), \quad (8.3)$$

$$h^{UU}(r_{12}, \Omega_1, \Omega_2) = c^{UU}(r_{12}, \Omega_1, \Omega_2) + \rho^V \int d\mathbf{r}_3 d\Omega_3 c^{UV}(r_{13}, \Omega_1, \Omega_3) h^{VU}(r_{32}, \Omega_3, \Omega_2). \quad (8.4)$$

Equation (8.3) is directly relevant for biomolecular simulations where we are often interested in the properties of a single, arbitrarily complex solute in the solution phase. Solutions to Equation (8.3) can be obtained using 3D-RISM. However, a solution to Equation (8.2) for pure solvent is a necessary prerequisite and is readily obtained from 1D-RISM.

To obtain a solution to the OZ equations it is necessary to have a second equation that relates h and c or uniquely defines one of these functions. The general closure relation is[286]

$$g(r_{12}, \Omega_1, \Omega_2) = \exp[-\beta u(r_{12}, \Omega_1, \Omega_2) + h(r_{12}, \Omega_1, \Omega_2) - c(r_{12}, \Omega_1, \Omega_2) + b(r_{12}, \Omega_1, \Omega_2)] \quad (8.5)$$

8. Reference Interaction Site Model

u is the potential energy function for the two particles and b is known as the bridge function (a non-local functional, representable as infinite diagrammatic series in terms of h [286]). It should be noted that u is the only point at which the interaction potential enters the equations. Depending on the method used to solve the OZ equations, u is generally an explicit potential. In principle, it should now be possible to solve our two equations. For example, we may wish to use SPC/E as a water model. Inputting the relevant aspects of the SPC/E model into u , 1D-RISM can be used to calculate the equilibrium properties of the SPC/E model. A different explicit water model will yield different properties.

A fundamental problem for all OZ-like integral equation theories is the bridge function, which contains multiple integrals that are readily solved only in special circumstances. In practice, an approximate closure relation must be used. While many closures have been developed, at this time only three are implemented in 3D-RISM: hypernetted-chain approximation (HNC), Kovalenko-Hirata (KH) and the partial series expansion of order- n (PSE- n).

For HNC, we set $b = 0$, giving[286]

$$\begin{aligned} g^{\text{HNC}}(r_{12}, \Omega_1, \Omega_2) &= \exp(-\beta u(r_{12}, \Omega_1, \Omega_2) + h(r_{12}, \Omega_1, \Omega_2) - c(r_{12}, \Omega_1, \Omega_2)) \\ &= \exp(t^*(r_{12}, \Omega_1, \Omega_2)) \end{aligned} \quad (8.6)$$

where t^* is the renormalize-indirect correlation function. HNC works well in many situations, including charged particles, but has difficulties when the size ratios of particles in the system are highly varied and may not always converge on a solution when one should exist. Also, as the bridge term is generally repulsive, HNC allows particles to approach too closely, overestimating non-Coulombic interactions[281].

KH is a combination of HNC and the mean spherical approximation (MSA), the former being applied to the spatial regions of solvent density depletion ($g < 1$), including the repulsive core, and the latter to those of solvent density enrichment ($g > 1$), such as association peaks[280, 281]

$$g^{\text{KH}}(r_{12}, \Omega_1, \Omega_2) = \begin{cases} \exp(t^*(r_{12}, \Omega_1, \Omega_2)) & \text{for } g(r_{12}, \Omega_1, \Omega_2) \leq 1 \\ 1 + t^*(r_{12}, \Omega_1, \Omega_2) & \text{for } g(r_{12}, \Omega_1, \Omega_2) > 1 \end{cases} \quad (8.7)$$

Like HNC, KH handles Coulombic systems well but overestimates non-Coulombic interactions. Unlike HNC, it does not have difficulties with highly asymmetric particle sizes and readily converges to stable solutions for almost all systems of practical interest. The reliability of the KH closure makes it particularly suitable for molecular mechanics calculations.

PSE- n offers the ability to interpolate between KH and HNC. Here, the exponential regions of solvent density enrichment are treated as a Taylor expansion,

$$g^{\text{PSE-}n}(r_{12}, \Omega_1, \Omega_2) = \begin{cases} \exp(t^*(r_{12}, \Omega_1, \Omega_2)) & \text{for } g(r_{12}, \Omega_1, \Omega_2) \leq 1 \\ \sum_{i=0}^n (t^*(r_{12}, \Omega_1, \Omega_2))^i / i! & \text{for } g(r_{12}, \Omega_1, \Omega_2) > 1 \end{cases} \quad (8.8)$$

In the case of $n = 1$, the KH closure is obtained, while in the limit of $n \rightarrow \infty$ HNC is recovered. This allows a balance between the numerical stability of KH and the often better accuracy of HNC.

8.1.1. 1D-RISM

1D-RISM is used to calculate bulk properties of the solvent and is a prerequisite for 3D-RISM, for which the primary result is the bulk solvent site-site susceptibility in reciprocal space, $\chi^{\text{VV}}(k)$. As its name would suggest, 1D-RISM is a one-dimensional calculation. The six-dimensional OZ equations are reduced to one dimension (radial separation) via the fundamental RISM approximation[271–274, 286, 287], which produces the intramolecular pair correlation matrix,

$$\omega_{\alpha\gamma}(k) = \sin(kr_{\alpha\gamma}) / (kr_{\alpha\gamma}) \quad (8.9)$$

where α and γ label the different atom types in the model. Note that atoms of the same type in RISM theory have the same Lennard-Jones and Coulomb parameters. For example, most three site water models have two RISM types, oxygen and hydrogen. Depending on the model, propane, C_3H_8 , may have two carbon types and two

hydrogen types. Equation (8.2) then becomes

$$\begin{aligned}
 h_{\alpha\gamma}(r) &= \sum_{\mu\nu} \int d\mathbf{r}' d\mathbf{r}'' \omega_{\alpha\mu}(|r-r'|) c_{\mu\nu}(|r'-r''|) [\omega_{\nu\gamma}(r'') + \rho_\nu h_{\nu\gamma}(r'')] \\
 &= \frac{1}{(2\pi)^3} \int e^{i\mathbf{k}\cdot\mathbf{r}} d\mathbf{k} \left[\omega \mathbf{c} [\mathbf{1} - \rho \omega \mathbf{c}]^{-1} \omega \right]_{\alpha\gamma} \\
 &= \sum_0^\infty \omega(k) \mathbf{c}(k) \omega(k) [\rho \mathbf{c}(k) \omega(k)]^n.
 \end{aligned} \tag{8.10}$$

Equation (8.10) must be complemented with one of the five closures currently supported by `rism1d` (see Subsection 8.4.1). In 1d, these are site-site closures and there is no orientational dependence. For example, the HNC closure (Eq. (8.6)) becomes,

$$g_{\alpha\gamma}^{\text{HNC}}(r) = \exp \left[-\beta u_{\alpha\gamma}(r) + h_{\alpha\gamma}(r) - c_{\alpha\gamma}(r) \right]. \tag{8.11}$$

Equation (8.10), with KH, HNC or PSE- n closures, is readily applicable to liquid mixtures, with site indices of the site-site correlation functions enumerating interaction sites on all (different) species in the solution and the intramolecular matrix (8.9) set equal to zero for sites α, γ belonging to different species.

A dielectrically consistent version of 1D-RISM theory (DRISM) enforces the proper dielectric asymptotics of the site-site correlation functions, and so provides the self-consistent dielectric properties of electrolyte solution with polar solvent and salt in a range of concentrations, including the given dielectric constant of the solution [288].

The 1D-RISM integral equations are then solved for the site-site direct correlation function in an iterative manner, accelerated by the modified direct inversion of the iterative subspace (MDIIS) [281, 289]. All correlation functions are represented as one-dimensional grids and the convolution integrals in Equation (8.10) are performed in reciprocal space by making use of a fast Fourier transform applied to the short-range parts of all the correlations, while the electrostatic asymptotics are separated out and Fourier transformed analytically [281–283].

1D-RISM is a general method and not restricted to water or pure solvents. For example, 1D-RISM may be used to treat solutions of aqueous alkali and halide ions at various concentrations [290]. The output from 1D-RISM can then be used for complex solutes, such as DNA [291], in 3D-RISM.

8.1.2. 3D-RISM

With the results from 1D-RISM, a 3D-RISM calculation for a specific solute can be carried out. For 3D-RISM calculations, only the solvent orientational degrees of freedom are averaged over and Equation (8.3) becomes [279, 280]

$$h_\gamma^{\text{UV}}(\mathbf{r}) = \sum_\alpha \int d\mathbf{r}' c_\alpha^{\text{UV}}(\mathbf{r}-\mathbf{r}') \chi_{\alpha\gamma}^{\text{VV}}(r'), \tag{8.12}$$

where $\chi_{\alpha\gamma}^{\text{VV}}(r)$ is the site-site susceptibility of the solvent, obtained from 1D-RISM and given by

$$\chi_{\alpha\gamma}^{\text{VV}}(r) = \omega_{\alpha\gamma}^{\text{VV}}(r) + \rho_\alpha h_{\alpha\gamma}^{\text{VV}}(r).$$

3D-RISM supports HNC, KH and PSE- n closures (see Sections 8.6.1 and 39.3.1). As with the 1D-RISM closures, these are constructed by analogy from Eqs. 8.6–8.8. For example, HNC becomes

$$g_\gamma^{\text{HNC,UV}}(\mathbf{r}) = \exp \left(-\beta u_\gamma^{\text{UV}}(\mathbf{r}) + h_\gamma^{\text{UV}}(\mathbf{r}) - c_\gamma^{\text{UV}}(\mathbf{r}) \right). \tag{8.13}$$

As with 1D-RISM, correlation functions are represented on (3D) grids, convolution integrals are performed in reciprocal space and a self-consistent solution is iteratively converged upon using the MDIIS accelerated solver. There is one 3D grid for each solvent type for each correlation function. For example, for a solute in SPC/E water there will be both $g_{\text{H}}^{\text{UV}}(\mathbf{r})$ and $g_{\text{O}}^{\text{UV}}(\mathbf{r})$ grids. Each point on the $g_{\text{H}}^{\text{UV}}(\mathbf{r})$ will give the fractional density of water hydrogen at that location of real-space.

8. Reference Interaction Site Model

To properly treat electrostatic forces in electrolyte solution with polar molecular solvent and ionic species, the electrostatic asymptotics of all the correlation functions (both the 3D and radial ones) are treated analytically [281, 282, 292]. The non-periodic electrostatic asymptotics are separated out in the direct and reciprocal space and the remaining short-range terms of the correlation functions are discretized on a 3D grid in a non-periodic box large enough to ensure decay of the short-range terms at the box boundaries [292]. The convolution of the short-range terms in the integral equation (8.12) is calculated using 3D fast Fourier transform [293, 294]. Accordingly, the electrostatic asymptotics terms in the thermodynamics integral (8.15) below are handled analytically and reduced to one-dimensional integrals easy to compute [292].

With a converged 3D-RISM solution for h^{UV} and c^{UV} , it is straightforward to calculate solvation thermodynamics. From the perspective of molecular simulations, the most important thermodynamic values are the excess chemical potential of solvation (solvation free energy), μ^{ex} and the mean solvation force, $\mathbf{f}_i^{\text{UV}}(\mathbf{R}_i)$, on each solute atom, i . μ^{ex} can be obtained through analytical thermodynamic integration for HNC,

$$\mu^{\text{ex,HNC}} = k_B T \sum_{\alpha} \rho_{\alpha}^{\text{V}} \int d\mathbf{r} \left[\frac{1}{2} (h_{\alpha}^{\text{UV}}(\mathbf{r}))^2 - c_{\alpha}^{\text{UV}}(\mathbf{r}) - \frac{1}{2} h_{\alpha}^{\text{UV}}(\mathbf{r}) c_{\alpha}^{\text{UV}}(\mathbf{r}) \right], \quad (8.14)$$

KH,

$$\mu^{\text{ex,KH}} = k_B T \sum_{\alpha} \rho_{\alpha}^{\text{V}} \int d\mathbf{r} \left[\frac{1}{2} (h_{\alpha}^{\text{UV}}(\mathbf{r}))^2 \Theta(-h_{\alpha}^{\text{UV}}(\mathbf{r})) - c_{\alpha}^{\text{UV}}(\mathbf{r}) - \frac{1}{2} h_{\alpha}^{\text{UV}}(\mathbf{r}) c_{\alpha}^{\text{UV}}(\mathbf{r}) \right], \quad (8.15)$$

and PSE- n ,

$$\mu^{\text{ex,PSE-}n} = k_B T \sum_{\alpha} \rho_{\alpha}^{\text{V}} \int d\mathbf{r} \left[\frac{1}{2} (h_{\alpha}^{\text{UV}}(\mathbf{r}))^2 - c_{\alpha}^{\text{UV}}(\mathbf{r}) - \frac{1}{2} h_{\alpha}^{\text{UV}}(\mathbf{r}) c_{\alpha}^{\text{UV}}(\mathbf{r}) - \frac{(t^*(\mathbf{r}))^{n+1}}{(n+1)!} \Theta(h_{\alpha}^{\text{UV}}(\mathbf{r})) \right], \quad (8.16)$$

where Θ is the Heaviside function.

Analogous versions of Eqns. 8.6, 8.15 and 8.16 are used in 1D-RISM. While these are used for DRISM they have been derived for XRISM. Furthermore, these equations have been derived a number of different ways with slightly different functional forms of the $-\frac{1}{2}hc$ term [280, 295–298]. These different functional forms are equivalent in XRISM but not in DRISM. The form introduced by Pettitt and Rossky [296] is the most popular in the literature and the default selection in `rismld`. It is possible to have `rismld` evaluate and output all three functional forms (see [Output](#)) but, for DRISM, none of these expressions are strictly correct.

The force equation

$$\mathbf{f}_i^{\text{UV}}(\mathbf{R}_i) = -\frac{\partial \mu^{\text{ex}}}{\partial \mathbf{R}_i} = -\sum_{\alpha} \rho_{\alpha} \int d\mathbf{r} g_{\alpha}^{\text{UV}}(\mathbf{r}) \frac{\partial u_{\alpha}^{\text{UV}}(\mathbf{r} - \mathbf{R}_i)}{\partial \mathbf{R}_i}$$

is valid for all closures with a path independent expression for the excess chemical potential, such as HNC, KH and PSE- n closures implemented in 3D-RISM [270, 299–301].

In addition to closure specific expressions for the solvation free energy, other approximations also exist. The Gaussian fluctuation (GF) approximation[302, 303] is given as

$$\mu^{\text{ex,GF}} = k_B T \sum_{\alpha} \rho_{\alpha}^{\text{V}} \int d\mathbf{r} \left[-c_{\alpha}^{\text{UV}}(\mathbf{r}) - \frac{1}{2} h_{\alpha}^{\text{UV}}(\mathbf{r}) c_{\alpha}^{\text{UV}}(\mathbf{r}) \right] \quad (8.17)$$

and has been shown to yield improved absolute solvation free energies for both polar and non-polar solutes[303, 304] but not necessarily for relative free energies[305]. It is not associated with a particular closure but is typically used in place of the expression for a given closure.

Eqs. (8.14)-(8.16) give the total solvation free energy, ΔG_{sol} , but it is often useful to decompose this into electrostatic (solvent polarization), ΔG_{pol} , and non-electrostatic (dispersion and cavity formation), $(\Delta G_{\text{dis}} + \Delta G_{\text{cav}})$, terms. Conceptually, we can divide the path of the thermodynamic integration into two steps: first the solute without partial charges is inserted into the solvent (dispersion and cavity formation) and then partial charges are

introduced, which polarize the solvent,

$$\mu^{\text{ex}} = \Delta G_{\text{sol}} = \Delta G_{\text{pol}} + \Delta G_{\text{dis}} + \Delta G_{\text{cav}}.$$

ΔG_{sol} is produced by a 3D-RISM calculation on the charged solute. ΔG_{pol} is then the difference of the two calculations. As a point of reference, generalized-Born and Poisson-Boltzmann methods calculate only ΔG_{pol} and, typically, use a calculation involving solvent accessible surface area to predict $\Delta G_{\text{dis}} + \Delta G_{\text{cav}}$.

8.1.3. Analytic Temperature Derivatives

For the thermodynamic analysis of solvation, it is often useful to calculate the energetic and entropic contributions, ϵ^{solv} and $-TS^{\text{solv}}$ respectively, to the solvation free energy. It has been shown that it is possible to analytically decompose the solvation free energy into these two contributions when the solvation free energy has a closed analytical form, such as with HNC and KH closure [306]. In what follows, the analytical expression of energetic and entropic contributions to the solvation free energy are derived in the framework of 1D-RISM theory with HNC closure. The similar derivation can be applied to other closures as well as to the framework of 3D-RISM theory. At this time, temperature derivatives are implemented for with HNC, KH and PSE- n closures in both 1D- and 3D-RISM [307].

The solvation free energy of species U in a solution consisting of N total species is expressed in the RISM-HNC framework as

$$\mu_{\text{HNC}}^{\text{ex,U}} = k_{\text{B}}T \sum_{\alpha}^{\text{on U}} \sum_{M=1}^N \sum_{\gamma}^{\text{on M}} \rho_{\gamma} \int d\mathbf{r} \left[\frac{1}{2} (h_{\alpha\gamma}(r))^2 - c_{\alpha\gamma}(r) - \frac{1}{2} h_{\alpha\gamma}(r) c_{\alpha\gamma}(r) \right].$$

The differentiation of the solvation free energy with respect to the temperature T leads to

$$\delta_T \mu_{\text{HNC}}^{\text{ex,U}} = \mu_{\text{HNC}}^{\text{ex,U}} + k_{\text{B}}T \sum_{\alpha}^{\text{on U}} \sum_{M=1}^N \sum_{\gamma}^{\text{on M}} \rho_{\gamma} \int d\mathbf{r} \left[h_{\alpha\gamma}(r) \cdot \delta_T h_{\alpha\gamma}(r) - \delta_T c_{\alpha\gamma}(r) - \frac{1}{2} \delta_T h_{\alpha\gamma}(r) \cdot c_{\alpha\gamma}(r) - \frac{1}{2} h_{\alpha\gamma}(r) \cdot \delta_T c_{\alpha\gamma}(r) \right].$$

where δ_T is $T \frac{\partial}{\partial T}$. Since $\mu_{\text{HNC}}^{\text{ex,U}} = \epsilon^{\text{solv,U}} - TS^{\text{solv,U}}$, we have $\delta_T \mu_{\text{HNC}}^{\text{ex,U}} = -TS^{\text{solv,U}}$ and therefore the above equation can be rearranged as

$$\epsilon^{\text{solv,U}} = -k_{\text{B}}T \sum_{\alpha}^{\text{on U}} \sum_{M=1}^N \sum_{\gamma}^{\text{on M}} \rho_{\gamma} \int d\mathbf{r} \left[h_{\alpha\gamma}(r) \cdot \delta_T h_{\alpha\gamma}(r) - \delta_T c_{\alpha\gamma}(r) - \frac{1}{2} \delta_T h_{\alpha\gamma}(r) \cdot c_{\alpha\gamma}(r) - \frac{1}{2} h_{\alpha\gamma}(r) \cdot \delta_T c_{\alpha\gamma}(r) \right]. \quad (8.18)$$

It is noted that the solvation energy $\epsilon^{\text{solv,U}}$ can be viewed as consisting of two contributions: one arising from creation of a polarized cavity (in pure solvent) and the other corresponding to the energy of embedding the solute molecule into the cavity. The former is the solvent reorganization energy and the latter is the average solute-solvent interaction energy that is obtained as $\sum_{\alpha} \sum_{\gamma} \rho_{\gamma} \int d\mathbf{r} u_{\alpha\gamma} g_{\alpha\gamma}$.

The temperature derivatives of correlation functions $\delta_T h(r)$ and $\delta_T c(r)$ can be obtained by solving the temperature derivative of RISM-HNC equations

$$\delta_T \mathbf{h}(k) = \mathbf{w}(k) \delta_T \mathbf{c}(k) \mathbf{w}(k) + \rho \mathbf{w}(k) \delta_T \mathbf{c}(k) \mathbf{h}(k) + \rho \mathbf{w}(k) \mathbf{c}(k) \delta_T \mathbf{h}(k)$$

and

$$\delta_T h_{\alpha\gamma}(r) = \left[\frac{u_{\alpha\gamma}(r)}{k_{\text{B}}T} + \delta_T h_{\alpha\gamma}(r) - \delta_T c_{\alpha\gamma}(r) \right] (h_{\alpha\gamma}(r) + 1).$$

Some practical examples can be found in [308], [309] and [307].

8.1.4. Treecode Summation for Electrostatic Interactions

One of the most computationally expensive parts of the non-periodic 3D-RISM calculation is computing Coulomb potential between the solute sites and solvent grid in real-space and the related long-range asymptotics of the direct and total correlation functions in both real- and reciprocal-space [305]. These functions must be computed on N_{box} grid points from M solute atoms, which is an $O(MN_{\text{box}})$ operation and can become prohibitively expensive for large systems. While the cost of reciprocal-space calculations can be mitigated using a simple cutoff in wavelength (see the `asymptKSpaceTolerance` option in Sections 8.6.1 and 8.5.2.1), such a treatment would lead to large errors for real-space calculations. Instead, we employ cluster-particle treecodes, which are a class of fast summation methods that can be used to reduce the cost of computing the interactions between the N_{box} grid point targets and M solute atom sources to $O((M + N_{\text{box}}) \log(N_{\text{box}}))$. [284]

To speed up computation, the treecode replaces a collection of far-field particle-particle interactions with one particle-cluster interaction, where the clusters are nodes within a hierarchical octree. This treecode requires three parameters: a multipole acceptance criterion (MAC), θ , a Taylor series expansion order parameter, p , and a maximum target number per leaf, N_0 [310]. The MAC determines if the cluster and particle are well-separated and the interaction is evaluated, or if further children in the tree of target clusters are traversed. If the ratio of the radius of the cluster of targets to the distance between the cluster center and a source particle is less than θ , then the interaction is evaluated. Otherwise, we traverse the children clusters of the target cluster. The Taylor series expansion order parameter p specifies the order of the Taylor expansion for evaluating the cluster-particle interaction. A recurrence relation is used to calculate the Taylor coefficients. N_0 determines the maximum number of targets in a leaf target cluster, i.e., a node at the lowest level of the octree. If a target leaf-source particle interaction fails the MAC, then the interactions are evaluated directly.

When such a procedure is used, the potential, V , at a target site, $\mathbf{x}_i$, due to a collection of M source particles, $\mathbf{y}_j$, with associated charges, q_j , can be written as the sum of the direct interactions for the leaf and the Taylor series expansions that may be computed at each level,

$$V(\mathbf{x}_i) = \sum_{\mathbf{y}_j \in D} q_j \phi(\mathbf{x}_i, \mathbf{y}_j) + \sum_{l=1}^L \sum_{\mathbf{y}_j \in I_l} q_j \phi(\mathbf{x}_i, \mathbf{y}_j),$$

where ϕ is a general potential function. L is the number of tree levels, where level 1 is the root cluster and level L denotes the leaves. A target site will then belong to a nested sequence of clusters, $\mathbf{x}_i \in C_L \subseteq \dots \subseteq C_1$, where cluster C_l is at level l . The direct calculation is only performed for source terms not well-separated from the targets, as determined by the MAC.

When the targets in a cluster, C_l , are well-separated from a set of source sites, a Taylor expansion is used to approximate the potential. Here, the cluster's geometric center is denoted $\mathbf{x}_c^l$ and I_l denotes the list of all source particles $\mathbf{y}_j$ that are well separated from cluster C_l but not from cluster $C_1, \dots, C_{l-1}$. Expanding the second term $\phi(\mathbf{x}_i, \mathbf{y}_j)$ about $\mathbf{x}_c^l$, the center of cluster l , gives

$$\begin{aligned} \sum_{\mathbf{y}_j \in I_l} q_j \phi(\mathbf{x}_i, \mathbf{y}_j) &\approx \sum_{\mathbf{y}_j \in I_l} q_j \sum_{\|\mathbf{k}\|=0}^p \frac{1}{\mathbf{k}!} \partial_{\mathbf{x}}^{\mathbf{k}} \phi(\mathbf{x}_c^l, \mathbf{y}_j) (\mathbf{x}_i - \mathbf{x}_c^l)^{\mathbf{k}} \\ &= \sum_{\|\mathbf{k}\|=0}^p m_{\mathbf{k}}(\mathbf{x}_c^l) (\mathbf{x}_i - \mathbf{x}_c^l)^{\mathbf{k}}, \end{aligned}$$

where the coefficients $m_{\mathbf{k}}$ are

$$m_{\mathbf{k}}(\mathbf{x}_c^l) = \sum_{\mathbf{y}_j \in I_l} q_j (-1)^{\|\mathbf{k}\|} a_{\mathbf{k}}(\mathbf{x}_c^l, \mathbf{y}_j),$$

and the Taylor coefficients $a_{\mathbf{k}}$ are

$$a_{\mathbf{k}}(\mathbf{x}_i, \mathbf{y}_j) = \frac{1}{\mathbf{k}!} \partial_{\mathbf{y}}^{\mathbf{k}} \phi(\mathbf{x}_i, \mathbf{y}_j).$$

Note that this is a Taylor series in three dimensions, where $\|\mathbf{k}\| = k_1 + k_2 + k_3$, $\mathbf{k}! = k_1! k_2! k_3!$, $\partial_{\mathbf{y}}^{\mathbf{k}} = \partial_{y_1}^{k_1} \partial_{y_2}^{k_2} \partial_{y_3}^{k_3}$, $(\mathbf{x}_i - \mathbf{x}_c)^{\mathbf{k}} = (x_{i1} - x_{c1})^{k_1} (x_{i2} - x_{c2})^{k_2} (x_{i3} - x_{c3})^{k_3}$, and 1, 2, 3 denote the three respective Cartesian directions.

Previous work [311, 312] established recurrence relations for Coulomb and screened Coulomb interactions. The cluster-particle treecode in 3D-RISM employs recurrence relations to calculate Taylor coefficients for Coulomb interactions as well as the asymptotic direct correlation and total correlation functions. The Taylor series for the Coulomb potential and the asymptotic direct correlation function converge exactly to their respective interactions; the Taylor series for the asymptotic total correlation function, however, uses an additional far field approximation which does not exactly approach the underlying interaction.

See section 8.2.3 and Table 8.2 for suggested settings.

8.1.5. Molecular Reconstruction

3D spatial distributions of solvation thermodynamics can provide insights into the role of water in a binding site, potentially identifying waters that can or cannot be easily displaced. Such maps can be easily obtained from the integrands of the relevant functions, such as the excess chemical potential, Eq. (8.16), or solvation energy, Eq. (8.18). However, since 3D-RISM is a site-site theory, separate distribution grids are produced for the each solvent site; e.g., one for each of hydrogen and oxygen. The result of simply adding these together is messy and difficult to interpret.

To obtain molecule thermodynamic distributions, qualitatively similar to those produced by grid inhomogeneous solvation theory, we use the intramolecular correlation function, Eq. (8.9), to reconstruct the molecular spatial distribution [313]. We begin by considering an arbitrary thermodynamic quantity, $A(\mathbf{r})$, and identifying a central site, α , such as oxygen in water. Then the molecular distribution is approximated by

$$A(\mathbf{r}) \approx A_\alpha(\mathbf{r}) + g_\alpha(\mathbf{r}) \sum_{\gamma \neq \alpha} \omega_{\alpha\gamma}(\mathbf{r}) * A_\gamma(\mathbf{r}).$$

Since the intramolecular correlation function contains the distance between two sites in the same molecule, the convolution, $*$, radially projects $A_\gamma(\mathbf{r})$ the bond length distance, $r_{\alpha\gamma}$. The result is then multiplied by the pair distribution function of the central site, which weights the contributions by the relative density of the central site. For example, the molecular excess chemical potential of water would be calculated as

$$\mu^{\text{ex}}(\mathbf{r}) \approx \mu_{\text{O}}^{\text{ex}}(\mathbf{r}) + g_{\text{O}}(\mathbf{r}) \omega_{\text{OH}}(\mathbf{r}) * \mu_{\text{H}}^{\text{ex}}(\mathbf{r}).$$

The excluded volume voxels are zeroed out in this approach, so integrating the molecular reconstruction does not yield the same result as integrating the site distribution grids, though it may be close to the value provided by UC or PC+ corrections.

At this time, the method is only implemented for water and assumes that oxygen is the first site. The method can be turned on using the `molReconstruct` flag in `sander` or `rism3d.snlgpnt`, in which case the molecular reconstruction is output in addition to any requested site-based thermodynamic distributions, such as the excess chemical potential or entropy.

8.2. Practical Considerations

8.2.1. Computational Requirements and Parallel Scaling

Calculating a 3D-RISM solution for a single solute conformation typically requires about 100 times more computer time than the same calculation with explicit solvent or PB. While there are other factors to consider, such as sampling confined solvent or overall efficiency of sampling in the whole statistical ensemble at once, this can be prohibitive for many applications. Memory is also an issue as the 3D correlation grids require anywhere from a few megabytes for the smallest solutes to gigabytes for large complexes. A lower bound and very good estimate

for the total memory required is

$$\text{Total memory} \geq 8 \text{ bytes} \times \left[N_{\text{box}} N^V \left(\underbrace{2N_{\text{MDIIS}}}_{c, \text{residual}} + \underbrace{1}_u + \underbrace{N_{\text{decomp}}}_{\text{polar decomp}} \underbrace{N_{\text{propagate}}}_{\text{past solutions}} \right) \right. \\ \left. (N_{\text{box}} + 2N_y N_z) \left\{ \underbrace{4}_{\text{asymptotics}} + \underbrace{1}_{\text{FFT scratch}} + \underbrace{2}_{g,h} N^V \right\} \right]$$

where $N_{\text{box}} = N_x \times N_y \times N_z$ is the total number of grid points, N^V is the number of solvent atom species and N_{MDIIS} is the number of MDIIS vectors used to accelerate convergence. u^{UV} , c^{UV} and the residual of c^{UV} are stored in real-space only and require a full grid for each solvent. c^{UV} and its residual also require N_{MDIIS} grids for the MDIIS routine (see the `mdiis_nvec` keyword) and $N_{\text{propagate}}$ grids to make use of solutions from previous solute configurations to improve the initial guess (see the `npropagate` keyword). If a polar/non-polar decomposition is requested (see the `polardecomp` keyword) an additional set of grids for past solutions with no solute charges is kept ($N_{\text{decomp}} = 2$); by default this is turned off ($N_{\text{decomp}} = 1$). The full real space grid plus an additional $2N_y N_z$ grid points are needed (due to the FFT) for g and h for each solvent species and for the four grids required to compute the long range asymptotics. Memory, therefore, scales linearly with N_{box} while computation time scales as $O(N_{\text{box}} \log(N_{\text{box}}))$ due to the requirements of calculating the 3D fast Fourier transform (3D-FFT). To overcome these requirements, two options are available beyond optimizations already in place, multiple time steps and parallelization. Multiple time step methods are available only in `sander` (Chapter 22) and are applicable to molecular dynamics calculations only. Parallelization is available for all calculations but is limited by system size and computational resources.

Both `sander` and `rism3d.snglpnt` have MPI implementations of 3D-RISM that distribute both memory requirements and computational load. As memory is distributed, the aggregate memory of many computers can be used to perform calculations on very large systems. Memory distribution is handled by the FFTW 3.3 library so decomposition is done along the z -axis. If a variable solvation box size is used, the only consideration is to avoid specifying a large, prime number of processes (≥ 7). For fixed box sizes, the number of grids points in each dimension must be divisible by two (a general requirement) and the number of grid points in the z -axis must be divisible by the number of processes. `sander.MPI` also has the additional consideration that the number of processes cannot be larger than the number of solute residues if SHAKE is used; `rism3d.snglpnt` does not suffer from this limitation.

8.2.2. Output

g^{UV} , h^{UV} and c^{UV} files can be output for 3D-RISM calculations and are useful for visualization and calculation of thermodynamic quantities. As all file formats save only one density per file (see <https://ambermd.org/FileFormats.php>), there is one file for each solvent atom type for each requested frame. For the default MRC format, each file is $(256 + N_{\text{box}} \times 4)$ bytes, which can quickly fill disk space. Note that these file format use single precision floating point numbers.

8.2.3. Numerical Accuracy

Numerical accuracy depends on the residual tolerance specified for the numerical solution at runtime and the solvation box physical size and grid spacing. In most cases, you will need to test these parameters to ensure you have the accuracy required. As a rough guide, the numerical error in the solvation free energy is related to the tolerance by

$$\epsilon_{\Delta G_{\text{solv}}} \approx 10 \times \text{tolerance}. \quad (8.19)$$

Molecular dynamics [270], minimization and trajectory post-processing [305] have different requirements for the maximum residual tolerance. Molecular dynamics does well with a tolerance of 10^{-5} and `npropagate=5`. Minimization requires tolerances of 10^{-11} or lower and is typically limited to `drms` $\geq 10^{-4}$. Trajectory post-processing for MM/RISM should use enough digits to obtain the necessary accuracy when differences in solvation free energy are computed. For example, if a error $< 0.2 \text{ kcal/mol}$ is required for $\Delta \Delta G_{\text{solv}}$, then ΔG_{solv} should

		ljTolerance		
		< 0	0	> 0
buffer	< 0	Fixed box size with dimensions of solvbox. LJ cutoff fit to box size and correction applied.	Fixed box size with dimensions of solvbox. No LJ cutoff or correction applied.	Fixed box size with dimensions of solvbox. LJ cutoff with ljTolerance applied. Correction applied if the box size is large enough.
	0	ljTolerance=tolerance/10 and the box size is selected to fit the cutoff. Correction applied.	Error.	Box size is selected to fit the cutoff. Correction applied if the box size is large enough.
	> 0	Box size determined by buffer. LJ cutoff fit to box size and correction applied.	Box size determined by buffer. No LJ cutoff or correction applied.	Box size determined by buffer. Correction applied if the box size is large enough.

Table 8.1.: The relationship between *ljTolerance*, *tolerance*, *buffer*, and *solvbox* in determining 3D-RISM solvent box and Lennard-Jones cutoff values.

be computed with an absolute error of 0.1 kcal/mol. The relative error required to achieve this depends on the magnitude of ΔG_{solv} .

Almost all applications should use a grid spacing of 0.3 to 0.5 Å or smaller. A larger grid spacing quickly leads to severe errors in thermodynamic quantities. Smaller grid spacing may be necessary for some applications (e.g., mapping potentials of mean force).

The size of the solvation box can be set in a number of ways; e.g., setting the box size directly, setting a buffer distance between the solute and the edges of the solvent box or should typically be at least 14 Å for water or larger for ionic solutions. The solvation box size should be increased until the thermodynamic properties converge (see Section 8.3.2). Systems with a neutral solute or non-ionic solvent are the simplest case, as solvent box size associated errors are primarily due to the truncation of the Lennard-Jones potential. Fortunately, this error can be corrected for if a cutoff is applied and the cutoff does not extend beyond the solvent box. In general, when using this correction, a cutoff where

$$u_{\alpha}^{\text{LJ}}(r_{\text{cut}}) \leq \text{tolerance}/10 \quad (8.20)$$

does not affect numerical precision of the calculation. Since long range Coulomb interactions are handled analytically by the long range asymptotics functions [282, 305], the solvent box size can be determined by the cutoff distance in many cases, which is calculated from the maximum error in the Lennard-Jones calculation and is determined at run time by the combination of *ljTolerance*, *tolerance*, *buffer*, and *solvbox* values used. The behavior is summarized in Table 8.1 on page 119.

For calculations with charged solutes in ionic solvent, the absolute size of the box required for sufficient numerical accuracy will depend on the absolute charge of the concentration of ions. Generally, lower ion concentrations require larger solvent boxes. Here, we recommend experimenting with different buffer sizes and setting the Lennard-Jones tolerance according to Eq. (8.20).

Independent of solvent-box size and grid spacing, time can be saved by truncating the reciprocal space expressions for the long range asymptotics. In general, a cutoff where

$$\hat{c}_{\alpha}^{(as)}(k_{\text{cut}}) \leq \text{tolerance}/10 \quad (8.21)$$

does not affect numerical precision of the calculation. The cutoff in reciprocal space is determined by *asymptKSpaceTolerance*.

For solutes with more than 1000 atoms, it becomes beneficial to replace the direct sum, real-space calculations of the Coulomb and long-range asymptotic interactions with treecode fast summation. Table 8.2 contains suggested parameter choices for treecode summation based off experience. Some calculated values are more sensitive than others, so we recommend experimenting with these settings for your system.

	treecodeMAC	treecodeOrder	treecodeN0
Total Correlation Function	0.3	$\max\left(2, \frac{\log_{10}(\text{tolerance})+5.7}{-0.7}\right)$	500
Direct Correlation Function	0.3	$\max\left(2, \frac{\log_{10}(\text{tolerance})+1.9}{-0.8}\right)$	500
Coulomb	0.3	$\max\left(2, \frac{\log_{10}(\text{tolerance})+1.4}{-0.8}\right)$	500

Table 8.2.: Suggested 3D-RISM treecode parameters.

8.2.4. Solvation Free Energy Corrections

3D-RISM with HNC-like closures is known to overestimate the non-polar component of the solvation free energy. Several alternate expressions for the solvation free energy have been developed to correct this and are based all, or in part, on the partial molar volume (PMV) of the solute. These include the Universal Correction (UC) [314], Ng Bridge Correction (NgB) [315] and the Pressure Correction Plus (PC+/3D-RISM) correction [316]. 3D-RISM currently implements UC and PV+/3D-RISM as runtime options. NgB results can be calculated from the standard thermodynamic output if the `polarDecomp` option is used but is not implemented directly. UC and NgB are both parameterized corrections. So, parameters for these corrections must be used only with the `.xvv` file used to create them. Our implementation of UC uses the excess chemical potential of the closure rather than the GF functional, as we have found this provides better results in general [307]. All of these corrections have been almost exclusively used with pure water under ambient conditions, though there are promising results for UC with non-polar liquids.[317] Using these methods with different solvents and co-solvents is a subject of on-going research.

8.3. Work Flow

Using 3D-RISM through `sander` or `rism3d.snglpnt` for molecular dynamics, minimization or snapshot analysis is very similar to using implicit solvent models like GBSA or PBSA. However, some additional preliminary setup is required, the extent of which depends on the solvent to be used.

3D-RISM requires detailed information of the bulk solvent in the form of the site-site susceptibility, χ^{VV} , and properties such as the temperature and partial charges. This is read in as an `.xvv` file, which is produced by a 1D-RISM calculation. These `.xvv` files are independent of the solute molecule and may be reused with any solute, any number of times. However, if another 3D-RISM calculation is to be preformed with any details of the bulk solvent changed (e.g., temperature or pressure), a new `.xvv` file must be produced. Examples of precomputed `.xvv` files for SPC/E and TIP3P water can be found in `$AMBERHOME/AmberTools/test/rism1d`.

8.3.1. Computing bulk solvent properties with `rism1d`

Special care must be taken when producing `.xvv` files for use with 3D-RISM, particularly with respect to grid parameters. It is important that the spatial extent of the grid be large enough to capture the essential long range features of the solvent while the spacing must be fine enough to sample the short-range structure. A grid spacing of 0.025 Å is sufficient for most applications. The number of grid points required, which will determine the physical length of the grid in Å, generally depends on the properties of the solvent. Low concentration aqueous salt solutions typically require much larger grids than pure bulk water. A good indicator that the grid is large enough is convergence of `delhv0` in the `.xvv` file. When converged, `delhv0` should retain four to five digits of precision when the number of grid points is doubled.

The ability of 3D-RISM to perform temperature derivatives and calculate solvation energy and entropy requires `.xvv` files with with temperature dependence information. `rism1d` must be run with `entropicDecomp` option turned on (Section 8.4.1). The version number in the `.xvv` file header indicates the maximum information available. Version 1.001 (since AmberTools 14) allows temperature derivatives and solvation entropies and energies for all reported quantities. Version 1.000 (AmberTools 12 and 13) does not allow temperature derivatives of the PMV or solvation energies and entropies of PMV-based corrections. Version 0.001 does not have information for any temperature derivatives.

1D-RISM calculations require details of the some bulk properties of the solvent, such as temperature and dielectric constant, and an explicit model of the molecular components. These are read in from one or more `.mdl` files, depending on the composition of the solvent. Several `.mdl` files are included in the Amber distribution and can be found in `$AMBERHOME/dat/rism1d/mdl`. These include many of the explicit models for solvent and ions used with the Amber force fields. Other solvents models may be used by creating appropriate MDL files. See Section 8.7 for format details.

8.3.2. Selecting the Solvation Box Size

The non-periodic solvation box super-cell can be defined as variable or fixed in size. When a variable box size is used, the box size will be adjusted to maintain a minimum buffer distance between the atoms of the solute and the box boundary. This has the advantage of maintaining the smallest possible box size while adapting to changes of solute shape and orientation. Alternatively, the box size and grid spacing can be explicitly specified at run-time and used for the duration of the calculation. Generally, the box should be large enough to provide the desired numerical accuracy. See section 8.2.3 for details on how to best achieve this.

For calculations with periodic boundaries, the unit cell is taken from the input coordinate file, whether it is a restart file or trajectory. You should select a unit cell size that is appropriate for you system, as you would for an explicit solvent calculation.

For both periodic and open boundary calculations, the grid spacing is an input parameter. Generally, a grid spacing of 0.5 Å is the largest that will provide useable results. The grid spacing should be decreased to obtain better numerical precision.

Solvent box dimensions have a strong effect on the numerical precision of 3D-RISM. See Subsection 8.2.3 for recommendation on selecting an appropriate box size and resolution.

8.3.3. Selecting Solute Centering

Regardless of how the solvation box is defined, the “center” of the solute is placed in the middle of the box. The center of the solute and how it is placed in the solvent box is controlled with the `centering` keyword. Generally, `centering=1` (center=center-of-mass) is the default for open boundary and should be used for MD and `centering=2` (center=center-of-geometry) should be used for minimization. Center-of-mass and center-of-geometry are conserved quantities in each method respectively. For periodic boundaries, the default is `centering=0` (no centering). For visualization purposes, `centering=1` or `centering=2` may be better choices.

Other options for solute centering are available for special situations. To restrict the absolute position of grid-points to be integer multiples of the grid-spacing (e.g., (2.5 Å, 3.0 Å) for a grid spacing of 0.5 Å) use `centering=3` for center-of-mass and `centering=4` for center-of-geometry. To perform centering only on the first calculation (i.e., first step of MD or minimization or first frame of a trajectory analysis), use the negative integer corresponding to the desired center definition. This allows the solute to drift in the solvent box. Finally, with some care, it is possible to achieve custom centering using `centering=0`. Here, no solute centering is performed and the solvent grid has an origin of (0,0,0) and a center of $(\frac{x\text{-length}}{2} + dx, \frac{y\text{-length}}{2} + dy, \frac{z\text{-length}}{2} + dz)$. If you use `centering=0` with open boundaries, it is advisable to use a fixed-size solvent box.

8.3.4. Solution Convergence

The default parameters for 3D-RISM are selected to provide the best performance for the majority of systems. In cases where a convergence is not achieved, the strategies below may be useful.

8.3.4.1. Closure Bootstrap

When a PSE-*n* or HNC closure is desired, the most effective method to overcome convergence issues is to use a low order closure solution as a starting guess. The KH closure should be the starting point as it is numerically robust and, typically, converges easily in the vast majority of case. After this, higher orders of PSE-*n* can be used until the desired closure is reached. The procedure for 1D-RISM and 3D-RISM differs slightly in practice.

8. Reference Interaction Site Model

1D-RISM `rismld` can use restart files to implement this approach (see Section Subsection 8.4.1). First, run `rismld` with the KH closure to convergence. Then use the `.sav` file as input for the next highest closure. The root name of the `.sav` file must be the same as your `.inp` file. To avoid overwriting lower order solutions, name the files by closure or use separate directories. You will have to rename the `.sav` files as you go.

3D-RISM All 3D-RISM interfaces have closure bootstrapping builtin via the *closure* and *tolerance* keywords. Closures should be specified as an ordered list with last closure being the highest order closure. The solutions of the intermediate closures can have a high tolerance. The default tolerance for intermediate closures is 1 and there is no observed benefit to tolerances less than 1e-2. See details in Subsection 8.6.1 and Subsection 8.5.2.1.

8.3.4.2. MDIIS Settings

MDIIS default setting are appropriate for most cases. Should your residual diverge or the solver get stuck on a particular value, you can try modest adjustments.

Decrease *mdiis_del* *mdiis_del* controls the step size of MDIIS. A smaller step size can help convergence but if this is set too small it can cause convergence problems. For `rismld`, this should be no lower than 0.1 or 0.2. For 3D-RISM, it should be 0.5 at the lowest.

Increase *mdiis_nvec* This is the number of trial solutions that are saved for predicting a new solution. The optimal number for rapid convergence is typically 10 for 3D-RISM and 20 for 1D-RISM. However, for 3D-RISM, the default choice of 5 requires much less memory and is computationally faster even though more iterations are required. Increasing the *mdiis_nvec* may help for 3D-RISM but is unlikely to help for 1D-RISM.

Increase *mdiis_restart* Occasionally, the MDIIS routine goes in the wrong direction and the residual increases significantly. If it increases more than *mdiis_restart* then the MDIIS routine selects the solution with the lowest residual and purges the other trial solutions. The default value of 10 can be too aggressive and cause the solver to cycle. Increasing the value to 100 or 1000 sometimes allows the solver to recover from a misstep.

8.3.4.3. Parameter Annealing

Chargeless, hot gases are the easiest systems to converge. For 1D-RISM, this can be used to bootstrap a solution in a similar manner to closure bootstrapping. By slowly turning on charges, lowering the temperature or increasing the density, a converged solution may be reached. This only works for 1D-RISM because it requires restarting from a previous solution. As with closure bootstrapping, files should be carefully renamed during the procedure. There is no general protocol but the parameter increment should be reduced as the target value is approached. E.g., turning on charges in a linear fashion usually isn't helpful.

8.3.4.4. Forcefield selection

The forcefield may affect convergence due to the number of solvent sites involved or the particular parameters of the forcefield.

Number of Sites Molecules with more sites are more difficult to converge. Six or more sites is already difficult to converge and more than 10 may not be possible under any circumstances. One solution is to use a united atom or coarse grained forcefields to reduce the number of sites.

Alternate Parameterization Some parameter sets simply yield a stiffer set of equations to solve. Choosing an alternate parameter set may allow convergence with only small differences in the numerical results. For example, the cSPC/E water model with SPC/E Joung/Cheatham ions is easier to converge at higher ion concentrations in 1D-RISM than cTIP3P water with TIP3P Joung/Cheatham ions. Both models give nearly identical results in RISM at lower concentrations but NaCl in cTIP3P water will not converge above 0.5 M for the PSE-3 closure despite using all of the above methods.

8.3.5. Thermodynamic Output

When `nptrism` $\neq$ 0 thermodynamic data about the solvent is output as a table of solute and solvent information. When using the `rism3d.snglpnt` interface, units are indicated in the key table or as indicated below. The `sander` interface provides the same output, but does not provide a reference table at the beginning of the calculation.

8.3.5.1. Solute Information

solutePotentialEnergy [kcal/mol] provides the total potential energy of the solute and its decomposition into the potential energy terms. The solvation free energy for the current 3D-RISM closure is included as this corresponds to the solvation forces the solute would experience. The energy terms, in order, are Total, LJ, Coulomb, Bond, Angle, Dihedral, H-Bond, LJ-14, Coulomb-14, Restraints, and 3D-RISM.

8.3.5.2. Solvent Information

Solvent information consists of core set of thermodynamic information and optional solvation free energy corrections. Temperature derivatives and polar/non-polar decomposition is performed when `entropicDecomp` and `polarDecomp` options are used. Temperature derivatives names have a postfix of `_dT`, except for free energies, which are decomposed into `solvationEnergy` and `-TS`. Polar/non-polar components have `polar` or `apolar` added to the front of the quantity name.

rism.excessChemicalPotential [kcal/mol] Excess chemical potential or solvation free energy for the selected closure (see Section 8.1.2).

rism.excessChemicalPotentialGF [kcal/mol] (Optional) Excess chemical potential or solvation free energy using the Gaussian fluctuation functional (see Eq. (8.17)).

rism.excessChemicalPotentialPCPLUS [kcal/mol] (Optional) Excess chemical potential or solvation free energy using the PC+/3D-RISM functional (see Section 8.2.4).

rism.excessChemicalPotentialUC [kcal/mol] (Optional) Excess chemical potential or solvation free energy using the UC functional (see Section 8.2.4).

rism.solventPotentialEnergy [kcal/mol] Interaction energy between the solute and solvent, calculated from

$$\Delta U_{\text{sol}} = \sum_{\alpha} \rho_{\alpha} \int d\mathbf{r} g_{\alpha}^{\text{UV}}(\mathbf{r}) u_{\alpha}^{\text{UV}}(\mathbf{r}).$$

rism.excessParticlesCorrected [#] Excess number of solvent particles compared to a uniform distribution at bulk density.

rism.excessChargeCorrected [e] Excess charge of solvent particles compared to a uniform distribution at bulk density.

rism.KirkwoodBuff [$\text{\AA}^3$] All space integral of the total correlation function.

rism.DCFintegral [$\text{\AA}^3$] All space integral of the direct correlation function.

8.4. *rism1d*

1D-RISM calculations are carried out with `rism1d`, and require only one input file with an `.inp` suffix. The input file is listed on the command line without this suffix.

```
rism1d inputfile
```

Parameters for the calculation are read in from `parameters` name list.

8.4.1. Parameters

Note that these keywords are not case sensitive.

Theory

theory	[DRISM] The 1D-RISM theory to use. DRISM Dielectrically consistent RISM (recommended). XRISM Extended RISM.
closure	[KH] The type of closure to use. KH Kovalenko-Hirata (recommended). PSE_n Partial serial expansion of order n. E.g., “PSE3”. HNC Hyper-netted chain equation. PY Percus-Yevick.
entropicDecomp	[1] Solve another set of integral equations to calculate the temperature derivative. This typically adds less than 50% to the compute time and yields an energy/entropy decomposition of the excess chemical potential for all species and sites. [307] 0 Do not calculate the temperature derivative. 1 Calculate the temperature derivative.

Grid Size

dr	[0.025] Grid spacing in real space in Å.
nr	[16384] Number of grid points. Should be a product of small prime factors (2, 3 and 5).

Output

outlist	[] Indicates what output files to produce. Output file names use the root name of the input file with an extension listed below. This is a list of any combination of the following characters in any order, upper or lower case. U $U^{VV}(r)$ Solvent site-site potential in real space, <code>inputfile.uvv</code> (see https://ambermd.org/FileFormats.php). X $\chi^{VV}(k)$ Solvent site-site susceptibility in reciprocal space. Required input for 3D-RISM, <code>inputfile.xvv</code> (see https://ambermd.org/FileFormats.php). G $G^{VV}(r)$ Solvent site-site pair distribution function in real-space, <code>inputfile.gvv</code> (see https://ambermd.org/FileFormats.php). B $B^{VV}(r)$ Solvent site-site bridge correction in real space, <code>inputfile.bvv</code> (see https://ambermd.org/FileFormats.php). T Thermodynamic properties of the solvent, <code>inputfile.therm</code> (see https://ambermd.org/FileFormats.php). E $exN^{VV}(r)$, exN^{VV} Solvent site-site running, <code>inputfile.exnvv</code>, and total, <code>inputfile.n00</code> (see https://ambermd.org/FileFormats.php), excess coordination numbers in real space. N $N^{VV}(r)$ Solvent site-site running coordination numbers in real space, <code>inputfile.nvv</code> (see https://ambermd.org/FileFormats.php). Q exQ^{VV} Solvent site-site excess total charge of site γ about α, <code>inputfile.q00</code> (see https://ambermd.org/FileFormats.php).
---------	---

S $S^{VV}(k)$ Solvent site-site structure factor in reciprocal space, `inputfile.svv` (see <https://ambermd.org/FileFormats.php>).

<code>ROUT</code>	[0] Largest real space separation in Å for output files. If 0 then all grid points will be output.
<code>KOUT</code>	[0] Largest reciprocal space separation in Å ⁻¹ for output files. If 0 then all grid points will be output.
<code>KSAVE</code>	[-1] Output an intermediate solution every <code>KSAVE</code> steps. If <code>KSAVE</code> ≤ 0 then no intermediate restart files are written. If any restart files are present at run time (<code>.sav</code> suffix) they are automatically used. However, such files are non-portable binary files.
<code>PROGRESS</code>	[1] Write the current residue to standard output every <code>PROGRESS</code> iteration. If <code>PROGRESS</code> ≤ 0 then residue is not reported.
<code>SELFTEST</code>	[0] If '1', perform a self-consistency check and output the results to <code>inputfile.self.test</code> . Only tests applicable to the input parameters and system are performed. The results will depend on the input parameters (e.g., 'tolerance') used.

Species keywords

For each molecular species in the solvent mixture, a `species` name list should be provided.

density	[] (Required.) Density of the species in M. See 'units' below.
units	['M'] Units for density value. Options are 'M' (molar), 'mM' (millimolar), '1/Å^3' (number per Å ³), 'g/cm^3' (g/cm ³) or 'kg/m^3' (kg/m ³).
model	[] (Required.) Relative or absolute path to and name of the <code>.mdl</code> file with the parameters for this solvent molecule.

Solution Convergence

rism1d uses MDIIS to accelerate convergence. The default parameters for this method are usually near optimal but some systems can be difficult to converge. In such cases it may be useful to use a small step size (`mdiis_del`=0.1 or 0.2). Occasionally, the target tolerance of 10⁻¹² can not be achieved. A tolerance of 10⁻¹⁰ to 10⁻¹¹ is often sufficient but it is advisable to check how sensitive your calculations are to this.

<code>mdiis_nvec</code>	[20] Number of MDIIS vectors to use.[289]
<code>mdiis_del</code>	[0.3] MDIIS step size.[289]
<code>mdiis_restart</code>	[10] If the current residual is <code>mdiis_restart</code> times larger than the smallest residual in memory, then the MDIIS procedure is restarted using the lowest residual solution stored in memory. Increasing this number can sometimes help convergence.[289]
<code>tolerance</code>	[1e-12] Target residual tolerance for the self-consistent solution.
<code>maxstep</code>	[10000] Maximum number of iterations to converge to a solution.
<code>extra_precision</code>	[1] Controls the use of extra precision routines at key points in the 1D-RISM solver. This can be useful for achieving low tolerances or for very large box lengths but increases computational cost. Strongly recommended for solutions with charged particles (e.g., salts).
0	No extra precision routines are used.
1	Sensitive matrix multiplication and addition routines are done in extra precision. A small computational cost is incurred.

8. Reference Interaction Site Model

Solvent Description

temperature [298.15] Temperature in Kelvin.

dieps [] (Required.) Dielectric constant of the solvent.

nsp [] (Required.) Number of species (molecules) in the solutions. Also indicates the number of species name lists to follow.

Other

smear [1.0] Charge smear parameter in Å for long range asymptotics corrections.

adbcor [0.5] Numeric parameter for DRISM.

8.4.2. Example

Mixed ionic solvent.

```
&PARAMETERS
THEORY='DRISM', CLOSURE='KH',           !Theory
  NR=16384, DR=0.025,                   !Grid size and spacing
  OUTLIST='x', ROUT=384, KOUT=0,        !Output
  MDIIS_NVEC=20, MDIIS_DEL=0.3, TOLERANCE=1.e-12, !MDIIS
  KSAVE=-1,                             !Check pointing
  PROGRESS=1,                           !Output frequency
  MAXSTEP=10000,                        !Maximum iterations
  SMEAR=1, ADBCOR=0.5,                 !Electrostatics
  TEMPERATURE=310, DIEPS=78.497, NSP=3 !bulk solvent properties
/
&SPECIES
!SPC/E water
  DENSITY=55.296d0,                    !very close to 0.0333 1/A3
  MODEL=" ../../dat/rism1d/model/SPC.mdl"
/
&SPECIES
!Sodium
  units='mM'
  DENSITY=100,
  MODEL=" ../../dat/rism1d/model/Na+.mdl"
/
&SPECIES
!Chloride
  units='g/cm^3'
  DENSITY=35.45e-4,
  MODEL=" ../../dat/rism1d/model/Cl-.mdl"
/
```

8.5. 3D-RISM in sander

3D-RISM functionality is available in *sander* and is built as part of the standard install procedure. Some features specific to *sander* are discussed here.

8.5.1. Multiple Time Step Methods for 3D-RISM

At this time, the computational cost of 3D-RISM is still prohibitive for performing calculations at each step of molecular dynamics calculations. One of the most effective ways to reduce this computational burden is to reduce the number of solutions calculated by using multiple time step (MTS) methods. Two MTS methods, r-RESPA and force-coordinate extrapolation (FCE), are implemented for 3D-RISM in *sander* and can be combined such that solutions are only calculated once every 4 ps [318].

r-RESPA[319, 320] and I-Verlet[321] impulse MTS algorithms are widely used methods to reduce the computational load of long-range interactions while maintaining the desirable properties of energy conservation and time reversibility. Impulse MTS can be invoked for 3D-RISM independent of the existing r-RESPA implementation using the `RISMnRESPA` variable. For typical biomolecular simulations, impulse MTS is limited to a maximum step size of 8 fs if using the optimized Nose-Hoover thermostat (`ntt=9`) and 5 fs[322] for the Langevin thermostat. Since the computational load of calculating all internal interactions of the solute is small compared to the 3D-RISM calculation, it is recommend to use `dt=0.001`, `nrespa=1` and `RISMnRESPA=2` or 5, depending on the integrator.

To overcome the stability limitation of impulse MTS, FCE uses one of several available extrapolation methods to efficiently predict the forces for some time steps rather than computing a full 3D-RISM solution[270, 323]. In the simplest extrapolation scheme, corresponding to `FCEnttrans=0`, forces, $\{\mathbf{F}\}$, on N^U solute atoms for the current time step t_k are approximated as a linear combination of forces from the n previous time steps obtained from 3D-RISM calculations,

$$\{\mathbf{F}\}^{(k)} = \sum_{l=1}^n a_{kl} \{\mathbf{F}\}^{(l)}, \quad l \in \text{3D-RISM steps.} \quad (8.22)$$

The weight coefficients a_{kl} are obtained by expressing the current set of coordinates, $\{\mathbf{R}\}^{(k)}$, as a linear combination of coordinates from the n previous time steps for which 3D-RISM calculations were performed. That is, the current set of coordinates is projected onto the basis of n previous solute arrangements by minimizing the norm of the difference between the current $3 \times N^U$ matrix of coordinates $\{\mathbf{R}\}^{(k)}$ and the corresponding linear combination of the previous ones $\{\mathbf{R}\}^{(l)}$,

$$\text{minimize} \left| \{\mathbf{R}\}^{(k)} - \sum_{l=1}^n a_{kl} \{\mathbf{R}\}^{(l)} \right|^2.$$

Coefficients a_{kl} are then used in Equation (8.22) to extrapolate forces at the current intermediate time step. Similarly, the known coordinates for the current time step can be approximated from previous time steps as

$$\{\mathbf{R}\}^{(k)} = \sum_{l=1}^N a_{kl} \{\mathbf{R}\}^{(l)}.$$

Five extrapolation methods are available (`FCEnttrans=0-4`, see below) and each differs in computational cost along with the largest permitted outer time step, ranging from 20 fs (`FCEnttrans=4` with Langevin dynamics, `ntt=3`) all the way up to 4 ps (`FCEnttrans=6` using OIN, `ntt=9`). The latter procedures utilize a more complex extrapolation protocol than pictured above, involving a rotation of the outer basis coordinates and coefficient weight normalization and minimization. For a detailed description of these methods, please refer to [323] and [318]. Note that FCE MTS does not conserve energy and is not time reversible.

Combined impulse FCE MTS calculations (see Figure 8.1) start the simulation using impulse MTS, where full RISM-3D solutions are computed every `RISMnRESPA` time steps until the requested size for the basis set, `FCEnbasis`, is achieved. After a large enough basis set is collected, 3D-RISM calculations are only performed once every `FCEstride` $\times$ `RISMnRESPA` time steps, and `FCEnbasis` of `FCEnbasis` saved coordinates are used for one of the above extrapolation procedures every `RISMnRESPA` intermediate time steps. The `FCEnbasis` coordinates represent an optimized subset of `FCEnbasis`, found through distance minimization with the current solute coordinate. Note that large inaccuracies in the force extrapolation can ensue if `FCEnbasis` is equal to the number of solute degrees freedom.

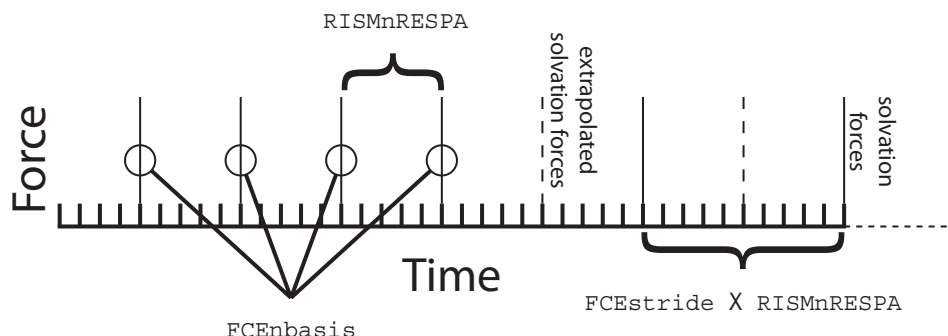


Figure 8.1.: *Multiple time step methods in 3D-RISM. $\text{RISMnRESPA}(= 5)$ is the number of base time steps between application of solvation forces (exact or extrapolated). $\text{FCEnbasis}(= 4)$ is the number of previous solutions used to extrapolate forces, in this case four previous solutions. Once FCEnbasis solutions have been calculated, exact 3D-RISM forces are calculated every $\text{FCEstride}(= 2) \times \text{RISMnRESPA}$ time steps; solvation forces are otherwise obtained through extrapolation.*

8.5.2. Usage

Full 3D-RISM functionality is available in `sander` as part of the standard install procedure. However, some methods available in `sander` are not compatible with 3D-RISM, such as QM/MM simulations. At this time, only standard molecular dynamics, minimization and trajectory post-processing with non-polarizable force fields are supported. With the exception of multiple time step features, 3D-RISM keywords in `sander` are identical to those in `rism3d.snglplt` and `MMPBSA.py`.

3D-RISM specific command line options for `sander` are

```
sander [standard options] -xvv xvfile -guv guvroot -huv huvroot
-cuv cuvroot -uuv uuvroot -asyp asympfile
-quv quvroot -chgdist chgdistroot
-exchem exchemroot -solvene solveneroot
-entropy entropyroot -potUV potUVroot
```

xvvfile *input* description of bulk solvent properties, required for 3D-RISM calculations. Produced by `rismld`.

guvroot *output* root name for solute-solvent 3D pair distribution function, $G^{UV}(\mathbf{R})$. This will produce one file for each solvent atom type for each frame requested.

huvroot *output* root name for solute-solvent 3D total correlation function, $H^{UV}(\mathbf{R})$. This will produce one file for each solvent atom type for each frame requested.

cuvroot *output* root name for solute-solvent 3D total correlation function, $C^{UV}(\mathbf{R})$. This will produce one file for each solvent atom type for each frame requested.

uuvroot *output* root name for solute-solvent 3D potential energy function, $U^{UV}(\mathbf{R})$, in units of kT . This will produce one file for each solvent atom type for each frame requested.

asympfile *output* root name for solute-solvent 3D long-range real-space asymptotics for C and H . This will produce one file for each of C and H for each frame requested and does not include the solvent site charge. Multiply the distribution by the solvent site charge to obtain the long-range asymptotics for that site.

quvroot *output* root name for solute-solvent 3D charge density distribution [$e/\text{\AA}$]. This will produce one file that combines contributions from all solvent atom types for each frame requested.

chgdistroot *output* root name for solute-solvent 3D charge distribution [e]. This will produce one file that combines contributions from all solvent atom types for each frame requested.

exchemroot *output* root name for 3D excess chemical potential distribution files.

solvenroot *output* root name for 3D solvation energy distribution files.

entropyroot *output* root name for 3D solvation entropy distribution files.

potUVroot *output* root name for 3D solute-solvent potential energy distribution files.

Generated output files can be large and numerous. For each type of correlation, a separate file is produced for each solvent atom type. The frequency that files are produced is controlled by the `ntwrism` parameter. Every time step that output is produced, a new set of files is written with the time step number in the file name. For example, a molecular dynamics calculation using an SPC/E water model with `ntwrism=2` and `-guv guv` on the command line will produce two files on time step ten: `guv.O.10.mrc` and `guv.H1.10.mrc`.

8.5.2.1. Keywords

With the exception of `irism`, which is found in the `&cntrl` name list, all 3D-RISM options are specified in the `&rism` name list.

irism [0] Use 3D-RISM. Found in `&cntrl` name list.

= 0 Off.

= 1 On.

Closure Approximation

closure [KH] Comma separate list of closure approximations. If more than one closure is provided, the 3D-RISM solver will use the closures in order to obtain a solution for the last closure in the list when no previous solutions are available. The solution for the last closure in the list is used for all output.

= **KH** Kovalenko-Hirata (KH).[\[282\]](#)

= **HNC** Hyper-netted chain equation (HNC).[\[295, 324\]](#)

= **PSE n** Partial series expansion of order- n (PSE- n), where “ n ” is a positive integer.[\[301\]](#)

Solvation Free Energy Corrections

gfCorrection [0] Compute the Gaussian fluctuation excess chemical potential functional (see §8.1.2). [\[302, 303, 307\]](#)

= 0 Off.

= 1 On.

pcpluscorrection [0] Compute the PC+/3D-RISM excess chemical potential functional (see §8.2.4). [\[307, 325\]](#)

= 0 Off.

= 1 On.

uccoeff [0,0,0,0] Compute the UC excess chemical potential functional with the provided coefficients (see §8.2.4). a and b are the coefficients for the original UC functional, though using the closure excess chemical potential functional. $a1$ and $b1$ are optional and provide temperature dependence to the correction (UCT in [\[307\]](#)).

Periodic boundaries While 3D-RISM uses open boundaries by default, periodic boundaries may be employed (section 8.3.2). [285] The unit cell dimension are read from the coordinate file, but the grid spacing is defined in the `&rism` namelist.

`periodic` Use periodic boundaries instead of open boundaries.[285] Options for calculating the periodic potential are

- `= pme` Particle mesh Ewald summation (recommended)
- `= ewald` Ewald summation

`solvcut` Sets Lennard-Jones cutoff distance for periodic calculations.

`grdspc` [0.5,0.5,0.5] Linear grid spacing in Å.

Open boundary long-range interactions For open boundary calculations, long-ranged Coulomb interactions and asymptotic corrections may be calculated using direct summation or treecode summation (section 8.1.4). [284] Long-range asymptotics are used to analytically account for solvent distribution beyond the solvent box. Long-range asymptotics are always used when calculating a solution but can be omitted for the subsequent thermodynamic calculations, though it is not recommended.

`asympcorr` [.true.] Use long-range asymptotic corrections for thermodynamic calculations.

- `= .true.` Use the long-range corrections.
- `= .false.` Do not use long-range corrections.

`treeDCF` [.true.] Use direct sum or the treecode approximation to calculate the direct correlation function long-range asymptotic correction. [284]

- `= .false.` Use direct sum.
- `= .true.` Use treecode approximation.

`treeTCF` [.true.] Use direct sum or the treecode approximation to calculate the total correlation function long-range asymptotic correction. [284]

- `= .false.` Use direct sum.
- `= .true.` Use treecode approximation.

`treeCoulomb` [.false.] Use direct sum or the treecode approximation to calculate the Coulomb potential energy. [284]

- `= .false.` Use direct sum.
- `= .true.` Use treecode approximation.

`treeDCFMAC` [0.1] Treecode multipole acceptance criterion for the direct correlation function long-range asymptotic correction.

`treeTCFMAC` [0.1] Treecode multipole acceptance criterion for the total correlation function long-range asymptotic correction.

`treeCoulombMAC` [0.1] Treecode multipole acceptance criterion for the Coulomb potential energy.

`treeDCFOrder` [2] Treecode Taylor series order for the direct correlation function long-range asymptotic correction.

`treeTCFOrder` [2] Treecode Taylor series order for the total correlation function long-range asymptotic correction. Note that the Taylor expansion used does not converge exactly to the TCF long-range asymptotic correction, so a very high order will not necessarily increase accuracy.

`treeCoulombOrder` [2] Treecode Taylor series order for the Coulomb potential energy.

`treeDCFNO` [500] Maximum number of grid points contained within the treecode leaf clusters for the direct correlation function long-range asymptotic correction. This sets the depth of the hierarchical octree.

`treeTCFNO` [500] Maximum number of grid points contained within the treecode leaf clusters for the total correlation function long-range asymptotic correction. This sets the depth of the hierarchical octree.

`treeCoulombNO` [500] Maximum number of grid points contained within the treecode leaf clusters for the Coulomb potential energy. This sets the depth of the hierarchical octree.

Open Boundary Solvation Box The open boundary solvation box super-cell can be defined as variable or fixed in size. When a variable box size is used, the box size will be adjusted to maintain a minimum buffer distance between the atoms of the solute and the box boundary. This has the advantage of maintaining the smallest possible box size while adapting to changes of solute shape and orientation. Alternatively, the box size can be specified at run-time. This box size will be used for the duration of the sander calculation.

Solvent box dimensions have a strong effect on the numerical precision of 3D-RISM. See Subsection 8.2.3 for recommendation on selecting an appropriate box size and resolution.

Variable Box Size

buffer [14] Minimum distance in Å between the solute and the edge of the solvent box. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, and `tolerance`.

`< 0` Use fixed box size (`ng3` and `solvbox`).

`>= 0` Buffer distance.

grdspc [0.5,0.5,0.5] Linear grid spacing in Å.

Fixed Box Size

ng3 [] Sets the number of grid points for a fixed size solvation box. This is only used if `buffer < 0`.

`nx, ny, nz` Points for *x*, *y* and *z* dimensions.

solvbox [] Sets the size in Å of the fixed size solvation box. This is only used if `buffer < 0`. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, and `tolerance`.

`lx, ly, lz` Box length in *x*, *y* and *z* dimensions.

Solution Convergence

tolerance [1e-5] A list of maximum residual values for solution convergence. When used in combination with a list of closures it is possible to define different tolerances for each of the closures. This can be useful for difficult to converge calculations (see Subsection 8.4.1 for details). For the sake of efficiency, it is best to use as high a tolerance as possible for all but the last closure. For minimization a tolerance of 1e-11 or lower is recommended. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, `buffer`, and `solvbox`. Three formats of list are possible.

`one tolerance` All closures but the last use a tolerance of 1. The last tolerance in the list is used by the last closure. In practice this, is the most efficient.

`two tolerances` All closures but the last use the first tolerance in the list. The last tolerance in the list is used by the last closure.

8. Reference Interaction Site Model

- `n` `tolerances` Tolerances from the list are assigned to the closure list in order.
- `ljTolerance` [-1] Determines the Lennard-Jones cutoff distance based on the desired accuracy of the calculation. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `tolerance`, `buffer`, and `solvbox`. [284]
- `asymptKSpaceTolerance` [-1] Determines the reciprocal space long range asymptotics cutoff distance based on the desired accuracy of the calculation. See §8.2.3 for details on how this affects numerical accuracy. [284] Possible values are
- < 0 `asymptKSpaceTolerance=tolerance/10`,
 - 0 no cutoff, and
 - > 0 given value determines the maximum error in the reciprocal-space long range asymptotics calculations.
- `mdiis_del` [0.7] “Step size” in MDIIS. [289]
- `mdiis_nvec`** [5] Number of vectors used by the MDIIS method. Higher values for this parameter can greatly increase memory requirements but may also accelerate convergence. [289]
- `mdiis_restart` [10] If the current residual is `mdiis_restart` times larger than the smallest residual in memory, then the MDIIS procedure is restarted using the lowest residual solution stored in memory. Increasing this number can sometimes help convergence. [289]
- `mdiis_method` [2] Specify implementation of the MDIIS routine.
- = 0 Original. For small systems (e.g. < 64³ grid points) this implementation may be faster than the BLAS optimized version.
 - = 1 BLAS optimized.
 - = 2 BLAS and memory optimized.
- `maxstep` [10000] Maximum number of iterations allowed to converge on a solution. `nrespa`
- `npropagate` [5] Number of previous solutions propagated forward to create an initial guess for this solute atom configuration.
- = 0 Do not use any previous solutions
 - = 1..5 Values greater than 0 but less than 4 or 5 will use less system memory but may introduce artifacts to the solution (e.g., energy drift).

Minimization and Molecular Dynamics

- `centering`** [1/0] Controls how the solute is centered/re-centered in the solvent box. Defaults to `centering=1` for open boundaries and `centering=0` for periodic boundaries.
- = -4 Center-of-geometry with grid-point rounding. Center on first step only.
 - = -3 Center-of-mass with grid-point rounding. Center on first step only.
 - = -2 Center-of-geometry. Center on first step only.
 - = -1 Center-of-mass. Center on first step only.
 - = 0 No centering. Default for periodic boundaries. Not recommended for open boundaries.
 - = 1 Center-of-mass. Center on every step. Recommended for molecular dynamics.
 - = 2 Center-of-geometry. Center on every step. Recommended for minimization.
 - = 3 Center-of-mass with grid-point rounding.
 - = 4 Center-of-geometry with grid-point rounding.

zerofrc [1] Redistribute solvent forces across the solute such that the net solvation force on the solute is zero.

= 0 Unmodified forces.

= 1 Zero net force.

Trajectory Post-Processing

apply_rism_force [1] Calculate and use solvation forces from 3D-RISM. Not calculating these forces can save computation time and is useful for trajectory post-processing.

= 0 Do not calculate forces.

= 1 Calculate forces.

Multiple Time Steps Multiple time step features are only available in *sander*.

rismnrespa [1] $\text{rismnrespa} \times \text{dt} = \text{RISM RESPA multiple time step}$. 8 fs is the maximum time step if using optimized-isokinetic integrator ($\text{nnt}=9$), and 5 fs using Langevin dynamics ($\text{nnt}=3$). “1” corresponds to no multiple time stepping.

fcestride [0] $\text{fcestride} \times \text{rismnrespa} \times \text{dt} = \text{FCE multiple time step}$, also called outer time step, i.e., full 3D-RISM solutions are performed every $\text{fcestride} \times \text{rismnrespa}$ steps. In between full solutions extrapolated force impulses are applied every rismnrespa steps. “1” corresponds to no multiple time stepping.

= 0 No FCE multiple time stepping.

= 1 Invokes the FCE code but yields the same trajectories as 0.

>= 1 Invoke FCE with 3D-RISM solutions every $\text{fcestride} \times \text{rismnrespa}$ steps.

fcenbasis [20] Number of previous full solutions to store, fcenbase of these are used for the force extrapolation. If FCE is not desired this can be set to 1 to reduce memory usage.

fcenbase [20] The number of previous solutions to use for the force extrapolation. This is a subset of fcenbasis and must be $\leq \text{fcenbasis}$. If $\text{fcenbase} < \text{fcenbasis}$, then an optimized subset of fcenbasis is found through minimization of the square distances with the current coordinate - the fcenbase closest solutions are chosen. Options for this selection can be found in the commands that follow.

fcесort [0] Sort the fcenbase basis vectors for the extrapolation according to increasing distance from the current coordinate. May decrease roundoff errors.

= 0 No sorting is performed (default).

= 1 Sorting is performed.

fcecrd [0] The coordinates used for the FCE method.

= 0 The absolute x, y, z position of each neighbor atom (with translations due to *centering*).

= 1 For predicting the forces on atom i , use the distance of each neighbor atom as the “coordinate”. This has one third the number of coordinates to use in the prediction. Also, directional information is lost.

= 2 For predicting the forces on atom i , use the x, y, z position of each neighbor atom with atom i as the origin. Recommended.

fcеweigh [0] Use weighted coordinates for the force extrapolation. Works with $\text{fcetrans} = [1], [2], \text{ or } [3]$.

= 0 No weighting of the coordinates is performed (default).

- = 1** Weighting of basis coordinates in the extrapolation. Expensive but more precise.
- `fceenormsw` [0] Balancing minimization of the squared norm of the basis expansion coefficients from least squares fitting. Specifies the magnitude of the parameter ε^2 of an additional constraint added to the least squares fitting problem that balances the equations and resulting coefficients, improving the quality and stability of the force extrapolation. Used only if `fcettrans=2`.
- = 0** No weight minimization is performed (default).
- > 0** Minimization is performed with specified balancing parameter `fceenormsw`. This parameter should in general be small as the squared norm is being minimized, and should be optimized to the value that produces the most accurate results from simulation.
- `fcettrans` [0] The method of transformation of the outer basis coordinates and the method of finding expansion coefficients in the least squares minimization problem. It can significantly affect the permitted size of the outer time step. Transformations involve a non-Eckhart rotation of all `fcenbasis` coordinates. In the least squares minimization problem, for the QR decomposition method, normalization is used if `fcenbase > solute degrees of freedom`.
- = 0** (Default) No coordinate transformation of the outer basis coordinates. Fast but not precise and should only be invoked if using small outer time steps (up to 200fs). Method of QR decomposition is used for finding expansion coefficients from least squares minimization.
- = 1** Transformation of basis coordinates with respect to the first (most recent) basis coordinate, from these the `fcenbase` subset is selected by minimum distance from current (also rotated) coordinate. QR decomposition is used for the least squares minimization. Permits large outer time steps on the order of several picoseconds. Fastest with regard to [2] and [3].
- = 2** ASFE extrapolation: like [1], transformation of basis coordinates with respect to first basis point, but normal equations method is utilized instead of QR, with additional squared norm minimization, specified by `fceenormsw`. An extra precision and stability is gained with small, positive values of `fcernormsw`. Most advanced method in Amber 15. This represents the ASFE extrapolation scheme as laid out in [323].
- = 3** (place holder, same as 2 above)
- = 4** Basic force extrapolation - no coordinate transformation, weighting, selecting, and sorting. Only small outer time steps, on the order of tens of fs, are permitted. This is the method as implemented in Amber 11.
- = 5** GSFE extrapolation 1: Individual transformation and selecting with respect to the current coordinate of each atom using a neighbouring scheme complemented by the e-minimization and *ifreq*-scheme (see *fceifreq* below) as well as all other developed techniques. It is recommended for large macromolecules of greater than 10 Å in size and can be used with very large outer steps (up to order of several picoseconds). See [326] for detailed explanation. This represents the one of the two new GSFE extrapolation schemes (Generalized Solvent Force-coordinate Extrapolation) as presented in [326].
- = 6** GSFE extrapolation 2: Individual transformation and selecting with respect to the post coordinate of each atom using a neighbouring scheme complemented by the e-minimization and the full *ifreq*-support. It is recommended for large macromolecules and can be used with huge outer steps (up to order of several picoseconds). It appears to be better than the above case `fcettrans=5` (partial *ifreq*-support version) because it can be exploited with larger number (up to N~100-200) of basic points providing a higher accuracy (with nearly the same computational efforts as the `fcettrans=5`-version at N~30), but may require more memory. Note that at any values of *fceifreq*, both the approaches have the same scheme for building the index mask which maps the extended set to the best subset and differ in the way of constructing the transformation matrix. At *fceifreq*=1, these two approaches are equivalent. This is the second GSFE scheme presented in [326] and [318] and represents the most advanced 3D-RISM solvent force extrapolation scheme available in AMBER to date.

- fceifreq** Extended to basic mapping list updating frequency used in the GSFE FCE extrapolation schemes above. If `fceifreq=1` then `fcetrans=6` is equivalent to `fcetrans=5`. See [326] for detailed explanation. Default value is 1.
- fcenfrcor** Net force correction flag for GSFE force extrapolation (`fcetrans=5` and `fcetrans=6`). If `fcenfrcor > 0`, a correction factor is subtracted from the extrapolated forces. See [326] for in depth explanation. Default is 0.

Output

- ntwrism** [0] Indicates that solvent density grid should be written to file every `ntwrism` iterations.
= 0 No files written.
>= 1 Output every `ntwrism` time steps.
- molReconstruction** [0] For any thermodynamic distributions requested, also out the molecular reconstruction (see section 8.1.5). [313]
- volfmt** ['mrc'] Format of volumetric data files. May be `mrc`, `ccp4`, `dx` or `xyzv` (see section 8.7).
- verbose** [0] Indicates level of diagnostic detail about the calculation written to the log file.
= 0 No output.
= 1 Print the number of iterations used to converge.
= 2 Print details for each iteration and information about what FCE is doing every `progress` iterations.
- write_thermo** [1] Print solvation thermodynamics in addition to standard `sander` output. The format is the same as that found in `rism3d.snglpnt`.
- polarDecomp** [0] Decomposes solvation free energy into polar and non-polar components. Note that this typically requires 80% more computation time.
= 0 No polar/non-polar decomposition.
= 1 Polar/non-polar decomposition.
- entropicDecomp** [0] Decomposes solvation free energy into energy and entropy components. Also performs temperature derivatives of other calculated quantities. Note that this typically requires 80% more computation time and requires a `.xv` file version 1.000 or higher (see §8.1.3 and 8.3). [307]
= 0 No entropic decomposition.
= 1 Entropic decomposition.
- progress** [1] Display progress of the 3D-RISM solution every `kshow` iterations. 0 indicates this information will not be displayed. Must be used with `verbose > 1`.

8.5.2.2. Example

Molecular Dynamics (`imin=0`)

```
molecular dynamics with 3D-RISM and impulse MTS
&cntrl
  ntx=1, ntp=100, ntwx=1000,ntwr=10000,
  nstlim=10000,dt=0.001,                !No shake or r-RESPA
  ntt=3, temp0=300, gamma_ln=20,        !Langevin dynamics
  ntb=0,                                !Non-periodic
  cut=999.,                             !Calculate all
```

```

                                !solute-solute
                                !interactions
    irism=1,
/
&rism
    rismnrespa=5,                !r-RESPA MTS
    fcenbasis=10, fcestride=2, fcecrd=2    !FCE MTS
/

```

Minimization (imin=1)

```

Default XMIN minimization with 3D-RISM
&cntrl
    imin=1, maxcyc=200,
    drms=1e-3,                !RMS force. Can be as low as 1e-4
    ntmin=3,                  !XMIN
    ntpr=5,
    ntb=0,                    !Non-periodic
    cut=999.,                !Calculate all
                                !solute-solute interactions
    irism=1
/
&rism
    tolerance=1e-11,          !Low tolerance
    solvcut=9999,             !No cut-off for
                                !solute-solvent interactions
    centering=2               !Solvation box centering
                                !using center-of-geometry
/

```

Trajectory Post-Processing (imin=5)

```

Trajectory post-processing with 3D-RISM
&cntrl
    ntx=1, ntpr=1, ntwx=1,
    imin=5, maxcyc=1,          !Single-point energy calculation
                                !on each frame
    ntb=0,                    !Non-periodic
    cut=9999.,                !Calculate all
                                !solute-solute interactions
    irism=1
/
&rism
    tolerance=1e-4,           !Saves some time compared to 1e-5
    apply_rism_force=0,        !Saves some time. Forces are not used.
    npropagate=1              !Saves some time and 4*8*Nbox bytes
                                !of memory compared to npropagate=5.
/

```

8.6. rism3d.snglpnt

3D-RISM functionality is also available in the command line tools `rism3d.snglpnt` and `rism3d.snglpnt.MPI` installed at compile time. These programs perform single point 3D-RISM calculations on trajectories and indi-

vidual solute snapshots. No other processing is done to the structures, so unwanted solvent molecules should be removed before hand. Except for minimization and molecular dynamics, all 3D-RISM features are available. Thermodynamic data is always output (see Section 8.3.5).

8.6.1. Usage

3D-RISM specific command line keywords generally are generally identical to keyword options in *sander*. If run without input, *rism3d.snglpnt* prints default settings for all parameters.

Unlike *sander*, three input files for the system are required: a PDB, parameter-topology file, and a restart or trajectory file. An appropriate PDB file can be created with *ambpdb* (Section 36.1).

- pdb *PDB file* (Required, input.) PDB file for the solute. In addition, a restart or trajectory file must be supplied. Coordinates from the PDB are not used.
- prmtop *prmtop file* (Required, input.) Parameter-topology file for the solute.
- rst *restart file* (Optional, input.) Coordinates for the solute in restart format. Not required if a trajectory file is provided.
- y|traj *trajectory file* (Optional, input.) Trajectory for the solute in NetCDF or ASCII format. Not required if a restart file is provided.
- xvv X^{VV} *file* (Required, input.) Bulk solvent susceptibility file from 1D-RISM (see section 8.7).
- guv G^{UV} *root* (Optional, output.) Root name for 3D solvent pair distribution files.
- cuv C^{UV} *root* (Optional, output.) Root name for 3D solvent direct correlation files.
- huv H^{UV} *root* (Optional, output.) Root name for 3D solvent total correlation files.
- uuv U^{UV} *root* (Optional, output.) Root name for 3D solvent potential [kT] files.
- asympt *asymptotics root* (Optional, output.) Root name for 3D real-space long range asymptotics for total and direct correlation files. This will produce one file for each of C and H for each frame requested and does not include the solvent site charge. Multiply the distribution by the solvent site charge to obtain the long-range asymptotics for that site.
- quv Q^{UV} *root* (Optional, output.) Root name for 3D solvent charge density distribution files. This is the charge density [$e/\text{\AA}$] at each grid point with contributions from all solvent types.
- chgdist *charge distribution root* (Optional, output.) Root name for 3D solvent charge distribution files. This gives a point charge [e] at each grid point with contributions from all solvent types.
- exchem (Optional.) Root name for 3D excess chemical potential distribution files.
- solvene (Optional.) Root name for 3D solvation energy distribution files.
- entropy (Optional.) Root name for 3D solvation entropy distribution files.
- potUV (Optional.) Root name for 3D solute-solvent potential energy distribution files.
- molReconstruct (Optional.) For any thermodynamic distributions requested, also out the molecular reconstruction (see section 8.1.5).[313]
- volfmt (Optional.) Format of volumetric data files. May be mrc (default), ccp4, dx or xyzv (see section 8.7).

8. Reference Interaction Site Model

- `--closure` *closure name* (Optional.) A whitespace separated list of one or more of KH [282], HNC [295, 324] or PSE_n [301] where “*n*” is a positive integer. If more than one closure is provided, the 3D-RISM solver will use the closures in order to obtain a solution for the last closure in the list when no previous solutions are available. The solution for the last closure in the list is used for all output. This can be useful for difficult to converge calculations (see §8.3.4).
- `--periodic` *periodic potential* (Optional.) Use periodic boundaries instead of open boundaries.[285]
Options for calculating the periodic potential are
- | | |
|--------------------|---|
| <code>pme</code> | Particle mesh Ewald summation (recommended) |
| <code>ewald</code> | Ewald summation |
- `--noasympcorr` (Optional.) Turn off long range asymptotic corrections for thermodynamic output only. Long-range asymptotics are still used to calculate the solution.
- `--buffer` *distance* (Optional.) Minimum distance between the solute and the edge of the solvent box. Use this with `--grdspc`. Incompatible with `--ng` and `--solvbox`. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, and `tolerance`.
- `--solvcut` *distance* (Optional.) Sets Lennard-Jones cutoff distance for periodic calculations. If ‘-1’ or no value is specified then the buffer distance is used.
- `--grdspc` *3D grid spacing* (Optional.) Comma separated linear grid spacings for *x*, *y* and *z* dimensions. Use this with `--buffer`. Incompatible with `--ng` and `--solvbox`.
- `--ng` *3D grid points* (Optional.) Comma separated number of grid points for *x*, *y* and *z* dimensions. Use this with `--solvbox`. Incompatible with `--buffer` and `--grdspc`.
- `--solvbox` *3D box length* (Optional.) Comma separated solvation box side length for *x*, *y* and *z* dimensions. Use this with `--ng`. Incompatible with `--buffer` and `--grdspc`. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, and `tolerance`.
- `--tolerance` *residual target* (Optional.) A whitespace separated list of maximum residual values for solution convergence. When used in combination with a list of closures it is possible to define different tolerances for each of the closures. This can be useful for difficult to converge calculations (see §8.3.4). For the sake of efficiency, it is best to use as high a tolerance as possible for all but the last closure. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `ljTolerance`, `buffer`, and `solvbox`. Three formats of list are possible.
- | | |
|-----------------------------|--|
| <code>one tolerance</code> | All closures but the last use a tolerance of 1. The last tolerance in the list is used by the last closure. In practice this, is the most efficient. |
| <code>two tolerances</code> | All closures but the last use the first tolerance in the list. The last tolerance in the list is used by the last closure. |
| <code>n tolerances</code> | Tolerances from the list are assigned to the closure list in order. |
- `--ljTolerance` *Lennard-Jones accuracy* (Optional.) Determines the Lennard-Jones cutoff distance based on the desired accuracy of the calculation. See §8.2.3 for details on how this affects numerical accuracy and how this interacts with `tolerance`, `buffer`, and `solvbox`. [284]
- `--asymptKSpaceTolerance` *reciprocal space long range asymptotics accuracy* (Optional.) Determines the reciprocal space long range asymptotics cutoff distance based on the desired accuracy of the calculation. See §8.2.3 for details on how this affects numerical accuracy. [284]
Possible values are
- | | |
|---------------------|--|
| <code>< 0</code> | <code>asymptKSpaceTolerance=tolerance/10,</code> |
| <code>0</code> | no cutoff, and |

> 0 given value determines the maximum error in the reciprocal-space long range asymptotics calculations.

--treeDCF *flag* (Optional.) Use direct sum or the treecode approximation to calculate the direct correlation function long-range asymptotic correction.[284]

0 Use direct sum.

1 Use treecode approximation.

--treeTCF *flag* (Optional.) Use direct sum or the treecode approximation to calculate the total correlation function long-range asymptotic correction.[284]

0 Use direct sum.

1 Use treecode approximation.

--treeCoulomb *flag* (Optional.) Use direct sum or the treecode approximation to calculate the Coulomb potential energy.[284]

0 Use direct sum.

1 Use treecode approximation.

--treeDCFMAC *acceptance criterion* (Optional.) Treecode multipole acceptance criterion for the direct correlation function long-range asymptotic correction.

--treeTCFMAC *acceptance criterion* (Optional.) Treecode multipole acceptance criterion for the total correlation function long-range asymptotic correction.

--treeCoulombMAC *acceptance criterion* (Optional.) Treecode multipole acceptance criterion for the Coulomb potential energy.

--treeDCFOrder *order* (Optional.) Treecode Taylor series order for the direct correlation function long-range asymptotic correction.

--treeTCFOrder *order* (Optional.) Treecode Taylor series order for the total correlation function long-range asymptotic correction. Note that the Taylor expansion used does not converge exactly to the TCF long-range asymptotic correction, so a very high order will not necessarily increase accuracy.

--treeCoulombOrder *order* (Optional.) Treecode Taylor series order for the Coulomb potential energy.

--treeDCFNO *leaf size* (Optional.) Maximum number of grid points contained within the treecode leaf clusters for the direct correlation function long-range asymptotic correction. This sets the depth of the hierarchical octtree.

--treeTCFNO *leaf size* (Optional.) Maximum number of grid points contained within the treecode leaf clusters for the total correlation function long-range asymptotic correction. This sets the depth of the hierarchical octtree.

--treeCoulombNO *leaf size* (Optional.) Maximum number of grid points contained within the treecode leaf clusters for the Coulomb potential energy. This sets the depth of the hierarchical octtree.

--mdiis_del *step size* (Optional.) MDIIS step size.[289]

--mdiis_nvec *# of vectors* (Optional.) Number of previous iterations MDIIS uses to predict a new solution.[289]

--mdiis_restart *# of vectors* (Optional.) If the current residual is `mdiis_restart` times larger than the smallest residual in memory, then the MDIIS procedure is restarted using the lowest residual solution stored in memory. Increasing this number can sometimes help convergence.[289]

8. Reference Interaction Site Model

`--maxstep` *step number* (Optional.) Maximum number of iterative steps per solution.

`--npropagate` *# old solutions* (Optional.) Number of previous solutions to use in predicting a new solution.

`--polarDecomp` (Optional.) Decomposes solvation free energy into polar and non-polar components. Note that this typically requires 80% more computation time.

`--entropicDecomp` (Optional.) Decomposes solvation free energy into energy and entropy components. Also performs temperature derivatives of other calculated quantities. Note that this typically requires 80% more computation time and requires a .xv file version 1.000 or higher (see §8.1.3 and 8.3). [307]

`--gf` (Optional.) Compute the Gaussian fluctuation excess chemical potential functional (see §8.1.2). [302, 303, 307]

`--pc+` (Optional.) Compute the PC+/3D-RISM excess chemical potential functional (see §8.2.4). [307, 325]

`--uccoeff` *a, b[, a1, b1]* (Optional.) Compute the UC excess chemical potential functional with the provided coefficients (see §8.2.4). *a* and *b* are the coefficients for the original UC functional, though using the closure excess chemical potential functional. *a1* and *b1* are optional and provide temperature dependence to the correction (UCT in [307]).

`--centering` *method* (Optional.) Select how solute is centered in the solvent box.

- 4** Center-of-geometry with grid-point rounding. Center on first step only.
- 3** Center-of-mass with grid-point rounding. Center on first step only.
- 2** Center-of-geometry. Center on first step only.
- 1** Center-of-mass. Center on first step only.
- 0** No centering. Dangerous.
- 1** Center-of-mass. Center on every step. Recommended for molecular dynamics.
- 2** Center-of-geometry. Center on every step. Recommended for minimization.
- 3** Center-of-mass with grid-point rounding.
- 4** Center-of-geometry with grid-point rounding.

`--verbose` *level* (Optional.)

- 0** No output.
- 1** Print the number of iterations required to converge.
- 2** Print convergence details for each iteration.

8.7. RISM File Formats

8.7.1. MDL

Solvent MoDeL (MDL) files use the prmtop specification. Each of the following sections may appear in the file in any order. The Fortran string format specifications can be different from the recommend values below.

```
%VERSION VERSION_STAMP = Vxxxx.yyy DATE = mm:dd:yy hh:mm:ss
```

The current version of the format is 0001.000. Date should be the date and time the file is created.

```
%FLAG TITLE  
%FORMAT (20a4)
```


Optional description of the file.

%FLAG POINTERS
%FORMAT(10I8)

Defines the lengths of arrays in the file.

NATOM Number of physical atoms in the model.

NSITE Number of unique solvent sites (share common Lennard-Jones parameters and partial charges).

%FLAG ATMNAME
%FORMAT(20a4)

CHARACTER(len=4) (NSITE) Four character name of each solvent site.

%FLAG MASS
%FORMAT(5e16.8)

REAL*8 (NSITE) Mass of each solvent site (amu).

%FLAG CHG
%FORMAT(5e16.8)

REAL*8 (NSITE) Partial charge for each solvent site, $18.2223e (\sqrt{kT\text{\AA}})$.

%FLAG LJEPSILON
%FORMAT(5e16.8)

REAL*8 (NSITE) Lennard-Jones ϵ for each solvent site (kcal/mol).

%FLAG LJSIGMA
%FORMAT(5e16.8)

REAL*8 (NSITE) Lennard-Jones $r_{\min}/2$ (sometimes called $\sigma^*/2$) for each solvent site ($\text{\AA}$)

$$U_{\alpha\gamma}^{\text{LJ}} = \sqrt{\epsilon_{\alpha}\epsilon_{\gamma}} \left(\left(\frac{r_{\min,\alpha} + r_{\min,\gamma}}{2r} \right)^{12} - 2 \left(\frac{r_{\min,\alpha} + r_{\min,\gamma}}{2r} \right)^6 \right).$$

Note that this is related to the commonly used σ as

$$\sigma = r_{\min} 2^{-1/6}.$$

%FLAG MULTI
%FORMAT(10I8)

INTEGER*4 (NSITE) Multiplicity of each solvent site. This should sum to NATOM.

%FLAG COORD
%FORMAT(5e16.8)

REAL*8 (3*NATOM) xyz-coordinates of each atom ($\text{\AA}$).

8.7.2. XVV

The .xvv file provides all of the bulk-solvent information required for 3D-RISM. This includes information about the solvent model, thermodynamic state and the necessary correlation functions. .xvv files use the prmtop specification. Each of the following sections may appear in the file in any order. The format specifications can be different from the recommend values below.

1D- and 3D-RISM now use version 1.000 of the file format. Differences include

- additional information about solvent, such as mass, number of sites per species, coordinates;
- RISM's internal system of units is now used;
- temperature derivative, DELHVO_DT and XVV_DT, are included when available (see 8.4.1);
- and SIGV has been replaced by RMIN2V.

All 3D-RISM interfaces still support the original 0.001 version of the format. For detailed information on version 0.001, please see the AmberTools 1.5 manual.

```
%VERSION VERSION_STAMP = V0001.000 DATE = mm:dd:yy hh:mm:ss
```

The current version of the format is 0001.000. Date should be the date and time the file is created.

```
%FLAG POINTERS
```

```
%FORMAT(10I8)
```

Defines the lengths of arrays in the file.

NR Number of 1D grid points in $\chi_{ab}^{VV}(k)$.

NV Number of total solvent sites.

NSP Number of solvent species (molecules).

```
%FLAG THERMO
```

```
%FORMAT(1PE24.16)
```

REAL (8) (6) Temperature [K], dielectric constant, inverse Debye length (κ) [$\text{\AA}$], compressibility [$\text{\AA}^{-3}$], grid spacing [$\text{\AA}$], charge smear [$\text{\AA}$].

```
%FLAG ATOM_NAME
```

```
%FORMAT(20A4)
```

CHARACTER (len=4) (NSITE) Four character name of each solvent site.

```
%FLAG MTV
```

```
%FORMAT(10I8)
```

INTEGER (4) (NSITE) Multiplicity of each solvent site.

```
%FLAG NVSP
```

```
%FORMAT(10I8)
```

INTEGER (4) (NSP) Number of sites for each solvent species.

```
%FLAG MASS
```

```
%FORMAT(1P5E16.8)
```

REAL (8) (NSITE) Mass of each solvent site (g/mol).

%FLAG RHOV
%FORMAT(1P5E16.8)

REAL(8) (NSITE) Number density of each solvent site ($\text{\AA}^{-3}$).

%FLAG QV
%FORMAT(1P5E16.8)

REAL(8) (NSITE) Partial charge for each solvent site multiplied by the square root of the Coulomb constant, $\sim 18.2223 (\sqrt{kT\text{\AA}})$.

%FLAG QSPV
%FORMAT(1P5E16.8)

REAL(8) (NSPECIES) Net charge for each solvent species multiplied by the square root of the Coulomb constant, $\sim 18.2223 (\sqrt{kT\text{\AA}})$.

%FLAG EPSV
%FORMAT(1P5E16.8)

REAL(8) (NSITE) Lennard-Jones ε for each solvent site (kT).

%FLAG RMIN2V
%FORMAT(1P5E16.8)

REAL(8) (NSITE) Lennard-Jones $r_{\min}/2$ ($\sigma^*/2$) for each solvent site ($\text{\AA}$).

%FLAG DELHV0
%FORMAT(1P5E16.8)

REAL(8) (NSITE) Long range Coulomb correction for each solvent site ($\sqrt{kT\text{\AA}}$).

%FLAG DELHV0_DT
%FORMAT(1P5E16.8)

REAL(8) (NSITE) (Optional) Temperature derivative long range Coulomb correction for each solvent site ($\sqrt{kT\text{\AA}}$).

%FLAG COORD
%FORMAT(1P5E16.8)

REAL(8) (3*sum(MTV)) Coordinates of all atoms (not sites) for each solvent species with the dipole moment aligned with the z-axis ($\text{\AA}$).

%FLAG XVV
%FORMAT(1P5E16.8)

REAL(8) (NR, NSITE, NSITE) $\chi_{ab}^{VV}(k)$. This array is stored in column major order. That is, the NR index varies fastest.

%FLAG XVV_DT
%FORMAT(1P5E16.8)

REAL(8) (NR, NSITE, NSITE) (Optional) $\delta_T \chi_{ab}^{VV}(k)$. This array is stored in column major order. That is, the NR index varies fastest.

8.7.3. Site-site functionals

All *.vv files, except .xvv (see §8.7.2), provide the separation dependence of all site-site pairings for a particular functional and use the same format. The first four lines have a “#” in the first character column, provide a description of the contents of the file and indicate site-site pairs. The first data column is the site-site separation and the remaining columns provide the value of the functional for the site-site pair at this separation.

The following example is for the direct correlation function (.cvv) for pure water. A standard, ‘two-site’ water model is used, consisting of oxygen (O) and hydrogen (H1). This gives one solvent species with two atoms.

```
#RISM1D ATOM-ATOM INTERACTIONS: DIRECT CORRELATION VS. SEPARATION [A]
#S=SPECIES, A=ATOM
# SEPARATION S1A1:S1A1 S1A1:S1A2 S1A2:S1A2
# SEPARATION O:O H1:O H1:H1
0.00000000E+000 -3.81875841E+002 1.64156197E+002 -9.24562553E+001
2.50000000E-002 -3.81695327E+002 1.64139031E+002 -9.24384608E+001
```

8.7.4. Thermodynamics

Thermodynamic output is divided into global, species and site properties sections. Global properties are generally not decomposable into species or site contributions (e.g., pressure). Species properties are the values for individual molecular species, for example, the excess chemical potential of a single molecule. Some of these properties, such as the partial molar volume, may not be decomposable into individual sites. Site properties are contributions from individual sites. Values for sites from the same species will sum to give the total value for the species.

The file format is white-space delimited with the first three columns giving a description, variable name and units of the property calculated. The remaining columns contain the calculated values for the system, species or site. Descriptive lines are indicated with a leading “#”.

The following example is for a standard, ‘two-site’ water model is used, consisting of oxygen (O) and hydrogen (H1), at standard temperature and density. In this calculation, energy/entropy free energy decomposition is also performed. I.e., $EXCHEM_{sp} = ESOLV_{sp} - TS_{sp}$.

#Global properties				
#Description	Variable	Units	Value	
Compressibility	xi	[10e-4/MPa]	4.73552130E+000	
Pressure (Virial)	Pvir	[MPa]	2.51627507E+003	
Excess free energy	FE	[kcal/mol]	-1.03698038E+003	
#Species properties				
#Description	Variable	Units	SPC	
Excess chemical potential	EXCHEM _{sp}	[kcal/mol]	-2.79190339E+000	
Solvation energy	ESOLV _{sp}	[kcal/mol]	-1.16421825E+001	
-Temperature*solvation entropy	-TS _{sp}	[kcal/mol]	8.85027911E+000	
Partial molar volume	PMV	[Å ⁻³]	3.00300236E+001	
#Site properties				
#Description	Variable	Units	O	H1
Excess chemical potential	EXCHEM _v	[kcal/mol]	-6.47897321E+000	3.68706981E+000
Solvation energy	ESOLV _v	[kcal/mol]	-1.19565867E+001	3.14404192E-001
-Temperature*solvation entropy	-TS _v	[kcal/mol]	5.47761350E+000	3.37266562E+000

8.7.5. Total excess values

.n00 and .q00 files provide the total excess coordination number and charge about each solvent site. The total excess of site γ around site α is

$$n_{\alpha\gamma}^{\text{extot}} = \rho_{\gamma} \int_0^{\infty} h_{\alpha\gamma}(r) dr,$$

while the total excess charge is

$$q_{\alpha\gamma}^{\text{extot}} = q_{\gamma} n_{\alpha\gamma}^{\text{extot}}.$$

These values are presented in their respective files as $n_{\text{site}} \times n_{\text{site}}$ arrays. Any asymmetry in these arrays is due to numerical error. .q00 files additionally provided the total excess charge from all sites.

The following example gives the total excess charge for a standard, ‘two-site’ water model is used, consisting of oxygen (O) and hydrogen (H1), at standard temperature and density.

```
#Total excess coordinated charge [e] of column site about row site
      O          H1          Total charge
O      7.92607232E-001 -7.92607230E-001  1.67181313E-009
H1     7.92607231E-001 -7.92607229E-001  2.44386922E-009
```

8.7.6. MRC and CCP4 volumetric data

Both MRC and CCP4 file formats use the MRC 2014 format (https://www.ccpem.ac.uk/mrc_format/mrc2014.php). This is a binary format that is much faster to read and write than DX and XYZV formats and requires approximately one quarter of the disk space. MRC is a newer format that is compatible with CCP4 and provides additional information, such as the grid origin coordinate, necessary to align the volumetric data with the solute molecular coordinates. Some CCP4 readers may ignore this additional data and MRC is recommended.

8.7.7. DX volumetric data

By default, 3D correlation functions from 3D-RISM calculations use the ASCII version of the Data Explorer (DX) file format for volumetric data on regular grids as defined in the DX user manual: <http://opendx.informatics.jax.org/docs/html/pages/usrgu068.htm#HDREDF>.

Header

```
object 1 class gridpositions counts Nx Ny Nz

Nx          INTEGER*4. Number of grid points in the x dimension.
Ny          INTEGER*4. Number of grid points in the y dimension.
Nz          INTEGER*4. Number of grid points in the z dimension.

origin Ox Oy Oz

Ox          REAL*8. x coordinate of grid origin in Cartesian space.
Oy          REAL*8. y coordinate of grid origin in Cartesian space.
Oz          REAL*8. z coordinate of grid origin in Cartesian space.

delta dx 0 0
delta 0 dy 0
delta 0 0 dz

dx          REAL*8. Linear grid size between in the x dimension.
dy          REAL*8. Linear grid size between in the y dimension.
dz          REAL*8. Linear grid size between in the z dimension.

object 2 class gridconnections counts Nx Ny Nz
object 3 class array type double rank 0 items N data follows

N           INTEGER*4. N = Nx × Ny × Nz.
```

Data

```
data(i,j,k) data(i,j,k+1) data(i,j,k+2)
```

data(i,j,k) REAL*8. Three data values per line with the last (z) index varying fastest for a total of N values.

Footer

```
object "Untitled" call field
```

8.7.8. XYZV volumetric data

An alternate format for volumetric data is the simple ASCII x-y-z-value (XYZV) format. The x-, y- and z-coordinates each grid point is written on a line followed by the value of the grid point. There is no header or footer. For example,

```
⋮  
-7.10789855E+000 -1.12570084E+001 -1.61284113E+001 1.35771922E-006  
-2.10789855E+000 -1.12570084E+001 -1.61284113E+001 -5.32279347E-006  
2.89210145E+000 -1.12570084E+001 -1.61284113E+001 -1.58802759E-005
```

9. sqm: Semi-empirical quantum chemistry

AmberTools contains its own quantum chemistry program, called *sqm*. This is code extracted from the QM/MM portions of *sander*, but is limited to “pure QM” calculations. A principal current use is as a replacement for MOPAC for deriving AM1-bcc charges, but the code is much more general than that. Presently, it is limited to single point calculations and energy minimizations (geometry optimizations) for closed-shell systems. It supports a wide variety of semi-empirical Hamiltonians, including many recent ones. An external electric field generated by a set of point charges can be included for single point calculations. Our plan is to add capabilities to subsequent versions. The major contributors are as follows:

- The original semi-empirical support was written by Ross Walker, Mike Crowley, and Dave Case,[327] based on public-domain MOPAC codes of J.J.P. Stewart.
- DFTB2 (SCC-DFTB) support was written by Gustavo Seabra, Ross Walker and Adrian Roitberg,[328] and is based on earlier work of Marcus Elstner.[329, 330]
- Support for diagonal third-order corrections to SCC-DFTB was written by Gustavo Seabra and Josh McClellan.
- DFTB3 was added by Andreas Goetz.
- Various SCF convergence schemes were added by Tim Giese and Darrin York.
- The PM6 Hamiltonian was added by Andreas Goetz and dispersion and hydrogen bond corrections were added by Andreas Goetz and Kyoyeon Park.
- The extension for MNDO type Hamiltonians to support d orbitals was written by Tai-Sung Lee, Darrin York and Andreas Goetz.
- The charge-dependent exchange-dispersion corrections of vdW interactions[331] was contributed by Tai-Sung Lee, Tim Giese, and Darrin York.
- Support for reading user-defined parameters for NDDO methods was added by Tai-Sung Lee and Darrin York.

The DFTB/DFTB2 code was originally based on the DFT/DYLAX code by Marcus Elstner *et al.*, but has since been extensively re-written and optimized. The DFTB3 implementation is an extension of this code.

9.1. Available Hamiltonians

Available MNDO-type semi-empirical Hamiltonians are PM3,[332] AM1,[333] RM1,[334] MNDO,[335] PDDG/PM3,[336] PDDG/MNDO,[336] PM3CARB1,[337], PM3-MAIS[338, 339], MNDO/d[340–342], AM1/d (Mg from AM1/d[343] and H, O, and P from AM1/d-PhoT[344]) and PM6[345].

Also available is the density functional theory-based tight-binding (DFTB) Hamiltonian[328, 346, 347] and its self-consistent-charge version with Taylor expansion up to second order (SCC-DFTB or DFTB2)[329] and third-order (DFTB3)[348]. If you use the mio-1-1 parameters for DFTB2, you can add an empirical correction for dispersion effects[349] and calculate CM3 charges[350] (both only for elements H, C, N, O, S, P). Diagonal third-order corrections are available for DFTB2[351] with mio-1-1 parameters but it is recommended to perform full DFTB3 simulations instead. Neither dispersion corrections nor halogen corrections are implemented for DFTB3.

The elements supported by each QM method are:

9. *sqm*: Semi-empirical quantum chemistry

- MNDO: H, Li, Be, B, C, N, O, F, Al, Si, P, S, Cl, Zn, Ge, Br, Cd, Sn, I, Hg, Pb
- MNDO/d: H, Li, Be, B, C, N, O, F, Na, Mg, Al, Si, P, S, Cl, Zn, Ge, Br, Sn, I, Hg, Pb
- AM1: H, C, N, O, F, Al, Si, P, S, Cl, Zn, Ge, Br, I, Hg
- AM1/d: H, C, N, O, F, Mg, Al, Si, P, S, Cl, Zn, Ge, Br, I, Hg
- PM3: H, Be, C, N, O, F, Mg, Al, Si, P, S, Cl, Zn, Ga, Ge, As, Se, Br, Cd, In, Sn, Sb, Te, I, Hg, Tl, Pb, Bi
- PDDG/PM3: H, C, N, O, F, Si, P, S, Cl, Br, I
- PDDG/MNDO: H, C, N, O, F, Cl, Br, I
- RM1: H, C, N, O, P, S, F, Cl, Br, I
- PM3CARB1: H, C, O
- PM3-MAIS: H, O, Cl
- PM6: H, He, Li, Be, B, C, N, O, F, Ne, Na, Mg, Al, Si, P, S, Cl, Ar, K, Ca, Sc, Ti, V, Cr, Mn, Fe, Co, Ni, Cu, Zn, Ga, Ge, As, Se, Br, Kr, Rb, Sr, Y, Zr, Nb, Mo, Tc, Ru, Rh, Pd, Ag, Cd, In, Sn, Sb, Te, I, Xe, Cs, Ba, La, Lu, Hf, Ta, W, Re, Os, Ir, Pt, Au, Hg, Tl, Pb, Bi
- DFTB/DFTB2/DFTB3: (Any atoms for which parameters are available from www.dftb.org)

The PM6 implementation has not been extensively tested for all available elements. Please check your results carefully, possibly by comparison to other codes that implement PM6, if transition metal elements are present. SCF convergence may be more difficult to achieve for transition metal elements with partially filled valence shells.

If the PM6 Hamiltonian is used in a QM/MM simulation with *sander* using electrostatic embedding (see Section 11) or if an electric field of external point charges is used, then the electrostatic interactions between QM and MM atoms are modeled using the MNDO type core repulsion function for interactions between QM and MM atoms. Parameters for the exponents α of the QM atoms are taken from PM3 (a default value of five is used for the exponents α of the MM atoms as is the case for MNDO, AM1 and PM3). Since PM3 does not have parameters for all elements that are supported by PM6, the missing exponents were defined in an ad hoc manner (see the source code in \$AMBERHOME/AmberTools/src/sqm/qm2-parameters.F90, variable `alp_pm6`). The magnitude of the coefficients α is probably not critical for the accuracy of QM/MM calculations but this should be tested on a case by case basis. This does not affect QM calculations with *sqm*.

9.1.1. DFTB parameter files

In order to use DFTB2 or DFTB3 (*qm_theory*=DFTB2 or DFTB3) a set of integral parameter files is required. The mio-1-1 parameter files for DFTB2 and 3ob-3-1 parameter files are distributed with Amber under a Creative Commons Attribution-ShareAlike 4.0 International License, see <http://creativecommons.org/licenses/by-sa/4.0/>. The parameters were obtained from the website www.dftb.org on February 22, 2017. You may want to check if there are any updates to the parameters. If you perform DFTB simulations, in addition to Amber please cite the publications describing the QM/MM and DFTB implementations as well as the following references for the DFTB parameters:

When using DFTB2 with mio-1-1 and following elements:

- O, N, C, H: M. Elstner, D. Porezag, G. Jungnickel, J. Elsner, M. Haugk, Th. Frauenheim, S. Suhai, G. Seifert, *Phys. Rev. B* **58** (1998) 7260.
- S: T. A. Niehaus, M. Elstner, Th. Frauenheim, S. Suhai, *J. Molec. Struct. (THEOCHEM)* **541** (2001) 185.
- P: M. Gaus, Q. Cui, M. Elstner, *J. Chem. Theory Comput.* **7** (2011) 931-948.

When using DFTB3 with 3ob-3-1 and following elements:

- O, N, C, H: M. Gaus, A. Goez, M. Elstner, *J. Chem. Theory Comput.* **9** (2013) 338-354.
- P, S: M. Gaus, X. Lu, M. Elstner, Q. Cui, *J. Chem. Theory Comput.* **10** (2014) 1518-1537.
- Mg, Zn: X. Lu, M. Gaus, M. Elstner, Q. Cui, *J. Phys. Chem. B* **119** (2015) 1062-1082.
- Na, F, K, Ca, Cl, Br, I: M. Kubillus, T. Kubar, M. Gaus, J. Rezac, M. Elstner, *J. Chem. Theory Comput.* **11** (2015) 332-342.

Additional parameter files can be obtained from the website www.dftb.org. By default it is assumed that DFTB2 uses the mio-1-1 parameter set and DFTB3 the 3ob-3-1 parameter set and that the corresponding files with extension *.skf* reside in the directories *\$AMBERHOME/dat/slko/mio-1-1* and *\$AMBERHOME/dat/slko/3ob-3-1*. If you want to use other parameter sets and/or put the parameter files in other directories then you have to specify the location in the input file (keyword *dftb_slko_path*, see section 9.3 for details).

Following parameter files for use with DFTB2 and the mio-1-1 parameter set are also distributed with AmberTools: Dispersion parameters for H, C, N, O, P and S are available in the file *\$AMBERHOME/dat/slko/mio-1-1/DISPERSION.INP_ONCHSP*, CM3 parameters for the same atoms are in the file *\$AMBERHOME/dat/slko/mio-1-1/CM3_PARAMETERS.DAT* file, and two parametrizations for diagonal third-order SCC-DFTB terms (SCC-DFTB-PA and SCC-DFTB-PR) are in the files *DFTB_3RD_ORDER_PA.DAT* and *DFTB_3RD_ORDER_PR.DAT*, both located in the same directory.

9.2. Dispersion and hydrogen bond correction

An empirical dispersion and hydrogen bonding correction is implemented for the MNDO type Hamiltonians AM1 and PM6[352]. The empirical dispersion correction follows the formalism for DFT-D[353] and consists of a physically sound r^{-6} term that is damped at short distances to avoid the short-range repulsion which can be written as

$$E_{dis} = -s_6 \sum_{ij} f_{damp}(r_{ij}, R_{ij}^0) C_{6,ij} r_{ij}^{-6}, \quad (9.1)$$

where r_{ij} is the distance between two atoms i and j , R_{ij}^0 is the equilibrium van der Waals (vdW) separation derived from the atomic vdW radii, $C_{6,ij}$ the dispersion coefficient, and s_6 a general scaling factor. The damping function is given as

$$f_{damp}(r_{ij}, R_{ij}^0) = \left[1 + \exp \left(-\alpha \frac{r_{ij}}{s_R R_{ij}^0} - 1 \right) \right]^{-1}. \quad (9.2)$$

Bondi vdW radii[354] are used and for a pair of unlike atoms we have

$$R_{ij}^0 = \frac{R_{ii}^{0^3} + R_{jj}^{0^3}}{R_{ii}^{0^2} + R_{jj}^{0^2}}. \quad (9.3)$$

For the C_6 coefficients the following equation is used,

$$C_{6,ij} = 2 \frac{(C_{6,ii}^2 C_{6,jj}^2 N_{eff,i} N_{eff,j})^{1/3}}{(C_{6,ii} N_{eff,j}^2)^{1/3} + (C_{6,jj} N_{eff,i}^2)^{1/3}}, \quad (9.4)$$

where the Slater-Kirkwood effective number of electrons $N_{eff,i}$ and the C_6 coefficients can easily be found in the literature[353].

An empirical hydrogen bonding correction[352] that is transferable among different semiempirical Hamiltonians and has been parametrized for use with the dispersion correction described above is also available. This correction does not make the assumption of a specific acceptor/hydrogen/donor binding situation. Instead it considers the hydrogen bond as a charge-independent atom-atom term between two atoms capable of serving as an acceptor or donor (for example, O, N) and weights this by a function that accounts for the steric arrangement of the two

atoms and the favorable positioning of a hydrogen atom inbetween. A damping function corrects for long- and short-range behavior,

$$E_{\text{H-bond}} = \frac{C_{AB}}{r_{AB}^2} f_{\text{geom}} f_{\text{damp}}, \quad (9.5)$$

$$f_{\text{geom}} = \cos(\theta_A)^2 \cos(\phi_A)^2 \cos(\psi_A)^2 \cos(\phi_B)^2 \cos(\phi_B)^2 \cos(\psi_B)^2 f_{\text{bond}}, \quad (9.6)$$

$$f_{\text{bond}} = 1 - \frac{1}{1 + \exp[-60(r_{XH}/1.2 - 1)]}, \quad (9.7)$$

$$f_{\text{damp}} = \left(\frac{1}{1 + \exp[-100(r_{AB}/2.4 - 1)]} \right) \left(1 - \frac{1}{1 + \exp[-10(r_{AB}/7.0 - 1)]} \right), \quad (9.8)$$

$$C_{AB} = \frac{C_A + C_B}{2}. \quad (9.9)$$

Here, C_A and C_B are the atomic hydrogen bonding correction parameters and the (torsion) angles in the function f_{geom} are defined similarly to an earlier hydrogen bond correction[355].

The hydrogen bond correction can be used both for single point energy calculations or geometry optimizations with SQM and for molecular dynamics simulations with SANDER. However, we do not recommend the use for molecular dynamics at present since cutoffs needed to be implemented for the calculation of f_{geom} of equation (9.6). This and some other conditional evaluations give rise to discontinuities in the potential energy surface and thus make this method unattractive for MD simulations.

9.3. Usage

The *sqm* program uses the following simple command line:

```
sqm [-O] -i <input-file> -o <output-file>
```

mdin is the default input-file name, and *mdout* is the default output-file name. As in other Amber programs, the “-O” flag allows the program to over-write the output file.

An example input file for running a simple minimization is shown here:

```
Run semi-empirical minimization
&qmmm
  qm_theory='AM1',   qmcharge=0,
/
  6      CG      -1.9590      0.1020      0.7950
  6      CD1      -1.2490      0.6020     -0.3030
  6      CD2      -2.0710      0.8650      1.9630
  6      CE1      -0.6460      1.8630     -0.2340
  6      C6       -1.4720      2.1290      2.0310
  6      CZ       -0.7590      2.6270      0.9340
  1      HE2      -1.5580      2.7190      2.9310
 16      S15      -2.7820      0.3650      3.0600
  1      H19      -3.5410      0.9790      3.2740
  1      H29      -0.7870     -0.0430     -0.9380
  1      H30       0.3730      2.0450     -0.7840
  1      H31      -0.0920      3.5780      0.7810
  1      H32      -2.3790     -0.9160      0.9010
```

The *&qmmm* namelist contains variables that allow you to control the options used. Following that is one line per atom, giving the atomic number, atom name, and Cartesian coordinates (free format). The variables in the *&qmmm* namelist are these:

qm_theory Level of theory to use for the QM region of the simulation (Hamiltonian). Default is to use the semi-empirical Hamiltonian PM3. Options are AM1, RM1, MNDO, PM3-PDDG, MNDO-PDDG,

PM3-CARB1, MNDO/d (same as MNDOD), AM1/d (same as AM1D), PM6, DFTB2 (same as DFTB), and DFTB3. The dispersion correction can be switched on for AM1 and PM6 by choosing AM1-D* and PM6-D, respectively. The dispersion and hydrogen bond correction will be applied for AM1-DH+ and PM6-DH+.

dftb_slko_path Path to the DFTB Slater-Koster parameter files. Defaults to '\$AMBERHOME/dat/slko/mio-1-1/' for DFTB2 and '\$AMBERHOME/dat/slko/3ob-3-1/' for DFTB3. You can specify a different directory here, which is assumed to be a subdirectory of '\$AMBERHOME/dat/slko/' unless you specify an absolute path.

dftb_disper Flag turning on (1) or off (0) the use of a dispersion correction to the DFTB2 energy (only for mio-1-1 parameters). Requires *qm_theory*=DFTB2. It is assumed that you have the file DISPERSION.INP_ONCHSP in your \$AMBERHOME/dat/slko/mio-1-1 directory. This file must be downloaded from the website www.dftb.org, as described in the beginning of this chapter. Only available for elements H, C, O, N, P, S. (Default = 0)

dftb_3rd_order Third order diagonal corrections to DFTB2 with mio-1-1 parameters. Default="" (the empty string which means no third order correction).

= **'PA'** Use the SCC-DFTB-PA parametrization, which was developed for proton affinities. The parameters will be read from the \$AMBERHOME/dat/slko/DFTB_3RD_ORDER_PA.DAT file.

= **'PR'** Use the SCC-DFTB-PR parametrization, which was developed for phosphate hydrolysis reactions. The parameters will be read from the \$AMBERHOME/dat/slko/DFTB_3RD_ORDER_PR.DAT file.

= **'READ'** Parameters will be read from the *mdin* file, in a separate "dftb_3rd_order" namelist, which must have the same format as the files above.

= **'filename'** Parameters will be read from the file specified by *filename*, in the "dftb_3rd_order" namelist, which must have the same format as the files above.

dftb_chg Flag to choose the type of charges to report when doing a DFTB calculation.

= **0** (default) - Print Mulliken charges.

= **2** Print CM3 charges. Only available for DFTB2 with mio-1-1 parameters for elements H, C, N, O, S and P.

dftb_telec Electronic temperature, in K, used to accelerate SCC convergence in DFTB calculations. The electronic temperature affects the Fermi distribution promoting some HOMO/LUMO mixing, which can accelerate the convergence in difficult cases. In most cases, a low *telec* (around 100K) is enough. Should be used only when necessary, and the results checked carefully. Default: 0.0K

dftb_maxiter Maximum number of SCC iterations before resetting Broyden in DFTB calculations. (default: 70)

qmcharge Charge on the QM system in electron units (must be an integer). (Default = 0)

spin Multiplicity of the QM system. Currently only singlet calculations are possible and so the default value of 1 is the only available option. Note that this option is ignored by DFTB/SCC-DFTB, which allows only ground state calculations. In this case, the spin state will be calculated from the number of electrons and orbital occupancy.

qmqmdx Flag for whether to use analytical or numerical derivatives of the semiempirical electron repulsion integrals. The default (and recommended) option is to use ANALYTICAL QM-QM derivatives.

= **1** (default) - Use analytical derivatives for QM-QM forces.

9. *sqm*: Semi-empirical quantum chemistry

- = 2** Use numerical derivatives for QM-QM forces. Note: the numerical derivative code has not been optimised as aggressively as the analytical code and as such is significantly slower. Numerical derivatives are intended mainly for testing purposes.
- verbosity** Controls the verbosity of QM/MM related output. *Warning:* Values of 2 or higher will produce a lot of output.
- = 0** (default) - only minimal information is printed - Initial QM geometry and link atom positions as well as the SCF energy at every ntp steps.
- = 1** Print SCF energy at every step to many more significant figures than usual. Also print the number of SCF cycles needed on each step.
- = 2** As 1 and also print info about memory reallocations, number of pairs per QM atom, QM core - QM core energy, QM core - MM atom energy, and total energy.
- = 3** As 2 and also print SCF convergence information at every step.
- = 4** As 3 and also print forces on the QM atoms due to the SCF calculation and the coordinates of the link atoms at every step.
- = 5** As 4 and also print all of the info in kJ/mol as well as kcal/mol.
- tight_p_conv** Controls the tightness of the convergence criteria on the density matrix in the SCF.
- = 0** (default) - loose convergence on the density matrix (or Mulliken charges, in case of a SCC-DFTB calculation). SCF will converge if the energy is converged to within `scfconv` and the largest change in the density matrix is within $0.05 \times \sqrt{\text{scfconv}}$.
- = 1** Tight convergence on density (or Mulliken charges, in case of a SCC-DFTB calculation). Use same convergence (`scfconv`) for both energy and density (charges) in SCF. Note: in the SCC-DFTB case, this option can lead to instabilities.
- scfconv** Controls the convergence criteria for the SCF calculation, in kcal/mol. In order to conserve energy in a dynamics simulation with no thermostat it is often necessary to use a convergence criterion of 1.0d-9 or tighter. Note, the tighter the convergence the longer the calculation will take. Values tighter than 1.0d-11 are not recommended as these can lead to oscillations in the SCF, due to limitations in machine precision, that can lead to convergence failures. Default is 1.0d-8 kcal/mol. Minimum usable value is 1.0d-14.
- pseudo_diag** Controls the use of 'fast' pseudo diagonalisations in the SCF routine. By default the code will attempt to do pseudo diagonalisations whenever possible. However, if you experience convergence problems then turning this option off may help. Not available for DFTB/SCC-DFTB.
- = 0** Always do full diagonalisation.
- = 1** Do pseudo diagonalisations when possible (default).
- pseudo_diag_criteria** Float controlling criteria used to determine if a pseudo diagonalisation can be done. If the difference in the largest density matrix element between two SCF iterations is less than this criteria then a pseudo diagonalisation can be done. This is really a tuning parameter designed for expert use only. Most users should have no cause to adjust this parameter. (Not applicable to DFTB/SCC-DFTB calculations.) Default = 0.05
- diag_routine** Controls which diagonalization routine will be used during the SCF procedure. This is an advanced option to fine-tune performance which has negligible effect on energies (and generally little effect on geometries in the case of SQM energy minimizations). The speed of each diagonalizer is a function of the number and type of QM atoms as well as the LAPACK library that the program was linked to. As such there is not always an obvious choice to obtain the best performance. The simplest option is to set `diag_routine = 0` in which case the program will test each diagonalizer in turn, including the pseudo diagonalizer, and select the one that gives optimum performance. As of AmberTools 15 `diag_routine = 0` is the default for both SQM and QMMM in Sander. Not available for DFTB/SCC-DFTB.

- = 0** Automatically select the fastest routine (default).
 - = 1** Use internal diagonalization routine.
 - = 2** Use lapack dspev.
 - = 3** Use lapack dspevd.
 - = 4** Use lapack dspevx.
 - = 5** Use lapack dsyev.
 - = 6** Use lapack dsyevd.
 - = 7** Use lapack dsyevr.
- printcharges** **= 0** Don't print any info about QM atom charges to the output file (default)
- = 1** Print Mulliken QM atom charges to output file every *ntpr* steps.
- print_eigenvalues** Controls printing of MO eigenvalues.
- = 0** Do not print MO eigenvalues
 - = 1** Print MO eigenvalues at the end of a single point calculation or geometry optimization (default)
 - = 2** Print MO eigenvalues at the end of every SCF cycle (only NDDO methods, not DFTB)
 - = 3** Print MO eigenvalues during each step of the SCF cycle (only NDDO methods, not DFTB)
- qxd** Flag to turn on (=true.) or off (=false., default) the charge-dependent exchange-dispersion corrections of vdW interactions[331].
- parameter_file** = 'PARAM.FILE' Read user-defined parameters from the file 'PARAM.FILE'. The first three space-separated entries (case insensitive) of each line will be interpreted as a user-modified parameter in the sequence of *parameter name*, *element name*, and *value*. For example, a line contains "USS Cl -111.6139480D0 " will cause the USS parameter of the Cl element changed to -111.6139480. A line beginning with "END" will stop the reading. This function currently only works for MNDO, AM1, PM3, MNDO/d, and AM1/d. Also, when new nuclear core-core parameters (FN, in PM3, AM1, and AM1/d) are re-defined, the number of FNN parameter sets (NUM_FN) also needs to be defined. For example, if FNn3 (*n* = 1, 2, or 3) is defined, then NUM_FN needs to be set to 3 or 4.
- peptide_corr** **= 0** Don't apply MM correction to peptide linkages. (default)
- = 1** Apply a MM correction to peptide linkages. This correction is of the form $E_{scf} = E_{scf} + h_{type}(i_{type}) \sin^2 \phi$, where ϕ is the dihedral angle of the H-N-C-O linkage and h_{type} is a constant dependent on the Hamiltonian used. (Recommended, except for DFTB/SCC-DFTB.)
- itrmax** Integer specifying the maximum number of SCF iterations to perform before assuming that convergence has failed. Default is 1000. Typically higher values will not do much good since if the SCF hasn't converged after 1000 steps it is unlikely to. If the convergence criteria have not been met after itrmax steps the SCF will stop and the minimisation will proceed with the gradient at itrmax. Hence if you have a system which does not converge well you can set itrmax smaller so less time is wasted before assuming the system won't converge. In this way you may be able to get out of a bad geometry quite quickly. Once in a better geometry SCF convergence should improve.
- maxcyc** Maximum number of minimization cycles to allow, using the *xmin* minimizer with the TNCG method. Default is 9999. Single point calculations can be done with *maxcyc* = 0.
- ntpr** Print the progress of the minimization every *ntpr* steps; default is 10.
- grms_tol** Terminate minimization when the gradient falls below this value; default is 0.02

9. sqm: Semi-empirical quantum chemistry

- `ndiis_attempts` Controls the number of iterations that DIIS (direct inversion of the iterative subspace) extrapolations will be attempted. Not available for DFTB/SCC-DFTB. The SCF does not even begin to exhaust its attempts at using DIIS extrapolations until the end of iteration 100. Therefore, for example, if `ndiis_attempts=50`, then DIIS extrapolations would be performed at end of iterations 100 to 150. The purpose of not performing DIIS extrapolations before iteration 100 is because the existing code base performs quite well for most molecules; however, if convergence is not met after 100 iterations, then it is presumed that further iterations will not yield SCF convergence without doing something different, i.e., DIIS. Thus, the implementation of DIIS in SQM is a mechanism to try and force SCF convergence for molecules that are otherwise difficult to converge. Default 0. Maximum 1000. Minimum 0. Note that DIIS will automatically turn itself on for 100 attempts at the end of iteration 800 even if you did not explicitly set `ndiis_attempts` to a nonzero value. This is done as a final effort to achieve convergence.
- `ndiis_matrices` Controls the number of matrices used in the DIIS extrapolation. Including only one matrix is the same as not performing an extrapolation. Including an excessive number of matrices may require a large amount of memory. Not available for DFTB/SCC-DFTB. Default 6. Minimum 1. Maximum 20.
- `vshift` Controls level shifting (only NDDO methods, not DFTB). Virtual orbitals can be shifted up by `vshift` (in eV) to improve SCF convergence in cases with small HOMO/LUMO gap. Default 0.0 (no level shift).
- `errconv` SCF tolerance on the maximum absolute value of the error matrix, i.e., the commutator of the Fock matrix with the density matrix. The value has units of hartree. The default value of `errconv` is sufficiently large to effectively remove this tolerance from the SCF convergence criteria. Not available for DFTB/SCC-DFTB. Default 1.d-1. Minimum 1.d-16. Maximum 1.d0.
- `qmmm_int` When running QM calculations in the `sqm` program, an electric field of external point charges can be added. In this way, the electrostatic effect outside of the QM region can be modeled, making the calculation a simplified QM/MM calculation without QM/MM vdW's contribution. Like QM/MM calculations (see Section 11), the method to couple QM and MM electrostatic interactions for external charges and semiempirical Hamiltonians can be specified via the `qmmm_int` namelist variable.
- The current implementation limits use of external charges to only single point energy calculations. To run such a calculation, an additional field, which begins with `#EXCHARGES` and ends with `#END`, is required to specify the external point charges in the input. Each external point charge must include atomic number, atom name, X, Y, Z coordinates and the charge in units of the electron charge. An example input looks like:

```
single point energy calculation (adenine), with external charges (thymine)
&qmmm
  qm_theory = 'PM3',
  qmcharge = 0,
  maxcyc = 0,
  qmmm_int = 1,
/
7  N   1.0716177  -0.0765366   1.9391390
1  H   0.0586915  -0.0423765   2.0039181
1  H   1.6443796  -0.0347395   2.7619159
6  C   1.6739638  -0.0357766   0.7424316
7  N   0.9350155  -0.0279801  -0.3788916
6  C   1.5490760   0.0012569  -1.5808009
1  H   0.8794435   0.0050260  -2.4315709
```

```

7  N   2.8531510   0.0258031  -1.8409596
6  C   3.5646109   0.0195446  -0.7059872
6  C   3.0747955  -0.0094480   0.5994562
7  N   4.0885824  -0.0054429   1.5289786
6  C   5.1829921   0.0253971   0.7872176
1  H   6.1882591   0.0375542   1.1738824
7  N   4.9294871   0.0412404  -0.5567274
1  H   5.6035368   0.0648755  -1.3036811
#EXCHARGESwill be
6  C  -4.7106131   0.0413373   2.1738637  -0.03140
1  H  -4.4267056   0.9186178   2.7530256   0.06002
1  H  -4.4439282  -0.8302573   2.7695655   0.05964
1  H  -5.7883971   0.0505530   2.0247280   0.03694
6  C  -3.9917387   0.0219348   0.8663338  -0.25383
6  C  -4.6136833   0.0169051  -0.3336520   0.03789
1  H  -5.6909220   0.0269347  -0.4227183   0.16330
7  N  -3.9211729  -0.0009646  -1.5163659  -0.47122
1  H  -4.4017172  -0.0036078  -2.4004924   0.35466
6  C  -2.5395897  -0.0149474  -1.5962357   0.80253
8  O  -1.9416783  -0.0291878  -2.6573783  -0.63850
7  N  -1.9256484  -0.0110593  -0.3638948  -0.58423
1  H  -0.8838255  -0.0216168  -0.3784269   0.35404
6  C  -2.5361367   0.0074651   0.8766724   0.71625
8  O  -1.8674730   0.0112093   1.9120833  -0.60609
#END

```

10. QUICK: *ab initio* quantum chemistry

AmberTools now distributes the *QUICK* (QUantum Interaction Computational Kernel) *ab initio* quantum chemistry program.[356–362] *QUICK* is a GPU enabled *ab initio* and density functional theory software capable of performing electronic structure calculations on general organic/biomolecular systems. *QUICK* is capable of performing efficient Hartree-Fock (HF) and density functional theory (DFT) energy and gradient calculations. The standalone version of *QUICK* is available in four different types: serial, MPI parallel, CUDA serial, and CUDA MPI parallel; giving rise to four respective executables: *quick*, *quick.MPI*, *quick.cuda*, and *quick.cuda.MPI*.

QUICK is also available as the QM engine for QM/MM simulations with SANDER. Furthermore, the functionalities of the GPU- and multi-GPU-accelerated versions of *QUICK* can be accessed directly from SANDER executables called *sander.quick.cuda* and *sander.quick.cuda.MPI*; these executables are identical to *sander* and *sander.MPI* in all SANDER functionalities, except they are capable of performing efficient QM/MM calculations with the *QUICK* library through an API. The serial and parallel functionalities of *QUICK* can be accessed from the *sander* and *sander.MPI* executables. More information about the QM/MM functionalities is provided in section 11.3.

If you use *QUICK* in your work, please cite the following reference:

- Manathunga, M.; O’Hearn, K. A.; Shajan, A.; Smith, J.; Miao, Y.; He, X.; Ayers, K.; Brothers, E.; Götz, A. W.; Merz, K. M. *QUICK-24.03* University of California San Diego, CA and Michigan State University, East Lansing, MI, 2024.

If you perform DFT calculations please also cite:

- Manathunga, M.; Miao, Y.; Mu, D.; Götz, A. W.; Merz, K. M. Parallel Implementation of Density Functional Theory Methods in the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.* **16**, 4315–4326 (2020). DOI: 10.1021/acs.jctc.0c00290

If you use the GPU accelerated version of *QUICK* please also cite:

- Manathunga, M.; Aktulga, H. M.; Götz, A. W.; Merz, K. M. Quantum Mechanics/Molecular Mechanics Simulations on NVIDIA and AMD Graphics Processing Units. *J. Chem. Inf. Model.* **63**, 711–717 (2023). DOI: 10.1021/acs.jcim.2c01505
- Manathunga, M.; Jin, C.; Cruzeiro, V. W. D.; Miao, Y.; Mu, D.; Arumugam, K.; Keipert, K.; Aktulga, H. M.; Merz, K. M., Jr.; Götz, A. W. Harnessing the Power of Multi-GPU Acceleration into the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.* **17**, 3955–3966 (2021). DOI: 10.1021/acs.jctc.1c00145
- Miao, Y.; Merz, K. M., Jr. Acceleration of High Angular Momentum Electron Repulsion Integrals and Integral Derivatives on Graphics Processing Units. *J. Chem. Theory Comput.* **11**, 1449–1462 (2015). DOI: 10.1021/ct500984t

If you perform geometry optimizations please also cite:

- Shajan, A.; Manathunga, M.; Götz, A.W.; Merz, K.M. Geometry optimization: A comparison of different open-source geometry optimizers. *J. Chem. Theory Comput.* **19**, 7533–7541 (2023). DOI: 10.1021/acs.jctc.3c00188

More details about *QUICK* can be found in the *QUICK* user manual, at the following link: <https://quick-docs.readthedocs.io/en/24.3.0>. This also contains recommended citations for third party software used in *QUICK*.

10.1. Features and limitations

The current version of *QUICK*, 24.03, shipped with AmberTools contains the following features and limitations:

Features:

- Hartree-Fock energy calculations
- Density functional theory calculations with dispersion corrections (LDA, GGA and Hybrid-GGA functionals available)
- Grimme type dispersion corrections
- Restricted closed-shell and unrestricted open-shell wavefunctions
- Gradient and geometry optimization calculations
- Mulliken charge analysis
- Exports Molden format for visualization of geometry and orbital data
- Wide range of popular Gaussian basis sets included
- Supports QM/MM calculations with Amber
- Message Passing Interface (MPI) distributed parallelization for CPU platforms
- Massively parallel, single GPU implementation via CUDA and HIP for Nvidia and AMD GPUs
- Distributed, multi-GPU support via MPI+CUDA/MPI+HIP, also across multiple compute nodes

Limitations:

- Supports energy/gradient calculations with basis functions up to d (f function support available on CUDA versions)
- GPU f function code is not highly optimized, requires large amount of RAM (may fail on consumer GPUs)
- No open shell gradients with f functions on GPUs
- Supports only Cartesian basis functions (no spherical harmonics)
- Effective core potentials (ECPs) are not supported
- DFT calculations are performed exclusively using SG1 grid system
- No meta-GGA nor range-separated hybrid functionals
- HIP/MPI+HIP support disabled for this release due to GPU code rewrites (f basis function support), please use AmberTools 2023 / QUICK version 23.08b for HIP support

10.2. Installation

QUICK is installed by default, including the standalone executables and the *sander* executables with corresponding features for QM/MM simulations. Support for f functions is disabled by default and can be enabled by setting the flag `-DENABLEF=TRUE` when invoking `cmake`. Please note that the two-electron repulsion integral (ERI) code is very complex. As a consequence the compilation of the CUDA/HIP code for GPUs can take a long time. It will take several minutes for a single GPU architecture (several hours if ERI f-function support is enabled). By default, the Amber build system generates executables that work for all Nvidia GPU architectures that are supported by the available CUDA Toolkit. As a consequence the build can take very long, upwards of half an hour (or several hours if f functions are enabled) - be patient, the compiler is working hard to generate lightning fast code for HF and DFT calculations! In the case of the HIP version, the code will be compiled for gfx908 architecture (MI100 GPU) by default.

Amber installation instructions can be found at section [2.1](#).

10.3. Usage

Examples of *QUICK* input files can be found at: `$AMBERHOME/AmberTools/src/quick/test`.

Here is an example of a gradient calculation for a single water molecule at the B3LYP/cc-pVDZ level of theory:

```
B3LYP BASIS=cc-pVDZ CHARGE=0 MULT=1 GRADIENT
O          -0.06756756  -0.31531531  0.00000000
H           0.89243244  -0.31531531  0.00000000
H          -0.38802215   0.58962052  0.00000000
```

Before running *QUICK*, users must source `$AMBERHOME/amber.sh` (or `$AMBERHOME/amber.csh`, depending on your environment).

Assuming the input file above is called *water.in*, in order to run the calculation with the serial version of *QUICK*:

```
quick water.in
```

Or to run with the CUDA parallel version of *QUICK* with 2 GPUs:

```
mpirun -np 2 quick.cuda.MPI water.in
```

HIP parallel version can be run in the same way except the executable name will be *quick.hip.MPI*. In all cases, an output file called *water.out* will be generated. If you use multiple GPUs, please make sure to use one MPI process per GPU.

More information on how to perform calculations can be found in the *QUICK* user manual, at following URL: link: <https://quick-docs.readthedocs.io/en/24.3.0/user-manual.html>.

11. QM/MM calculations

Sander supports the option of describing part of the system quantum mechanically in an approach known as a hybrid (or coupled potential) QM/MM simulation. There are three basic ways in which QM/MM simulations are enabled in *sander*:

1. Semi-empirical neglect of diatomic overlap (NDDO)-type and density functional tight binding (DFTB) Hamiltonians are supported natively by *sander* via the sqm software library. The basic documentation (e.g. what Hamiltonians are implemented, description of the input parameters) can be found in Chapter 9. In section 11.1 below we limit our description to those features that are unique to the QM/MM interface implemented in *sander*.
2. More advanced Hamiltonians based on *ab initio* wave function theory (WFT) and density functional theory (DFT) are supported via an interface to external QM software packages the use of which is described in section 11.2.
3. Seamless *ab initio* QM/MM simulations with Hartree-Fock (HF) and density functional theory (DFT) methods are possible via the GPU enabled QM code QUICK, which is distributed with AmberTools. QUICK can be used either as QM program external to *sander* or via the QUICK library linked to *sander* (recommended). Details about QUICK are in chapter 10 and QM/MM simulations via *sander* and QUICK are described in section 11.3 below.
4. *ab initio* QM/MM simulations are also available using the GPU-accelerated TeraChem code as the QM engine. TeraChem supports HF, DFT, and post-Hartree-Fock calculations such as CI and CASSCF, including at the excited states. Details about running QM/MM simulations with TeraChem are provided at section 11.4.

The built-in semi-empirical QM/MM support was written by Ross Walker and Mike Crowley, [327] based originally on public-domain MOPAC codes of J.J.P. Stewart. The QM/MM generalized Born implementation uses the model described by Pellegrini and Field[363] while regular QM/MM Ewald support is based on the work of Nam *et al.*[364] with QM/MM PME support based on the work of Walker *et al.*[327]. SCC-DFTB support was written by Gustavo Seabra, Ross Walker and Adrian Roitberg,[328] and is based on earlier work of Marcus Elstner.[329, 330] The extension for DFTB3 was written by Andreas Goetz. The interface to external QM packages [365] was originally developed by Andreas Goetz but many others have contributed in the meantime. Vinicius Cruzeiro has coupled Amber to QUICK [366] via the API that was developed by Madushanka Manathunga. Vinicius Cruzeiro implemented a faster QM/MM interface with TeraChem using TCPB-cpp (see section 11.4).

11.1. Built-in semiempirical NDDO methods and SCC-DFTB

When running a QM/MM simulation in *sander* the system is partitioned into two regions, a QM region consisting of the atoms defined by either the *qmmask* or *iqmatoms* keyword, and a MM region consisting of all the atoms that are not part of the QM region. For a typical protein simulation in explicit solvent the number of MM atoms will be much greater than the number of QM atoms. Either region can contain zero atoms, giving either a pure QM simulation or a standard classical simulation. For periodic simulations, the quantum region must be *compact*, so that the extent (or diameter) of the QM region (in any direction) plus twice the QM/MM cutoff must be less than the box size. Hence, you can define an "active site" to be the QM region, but in most cases could not ask that all cysteine residues (for example) be quantum objects. The restrictions are looser for non-periodic (gas-phase or generalized Born) simulations, but the codes are written and tested for the case of a single, compact quantum region.

11. QM/MM calculations

The partitioned system is characterized by an effective Hamiltonian which operates on the system's wavefunction Ψ , which is dependent on the position of the MM and QM nuclei, to yield the system energy E_{eff} :

$$H_{eff}\Psi(x_e, x_{QM}, x_{MM}) = E_{eff}(x_{QM}, x_{MM})\Psi(x_e, x_{QM}, x_{MM}) \quad (11.1)$$

The effective Hamiltonian consists of three components - one for the QM region, one for the MM region and a term that describes the interaction of the QM and MM regions, implying that likewise the energy of the system can be divided into three components. If the total energy of the system is re-written as the expectation value of H_{eff} then the MM term can be removed from the integral since it is independent of the position of the electrons:

$$E_{eff} = \langle \Psi | H_{QM} + H_{QM/MM} | \Psi \rangle + E_{MM} \quad (11.2)$$

In the QM/MM implementation in *sander*, E_{MM} is calculated classically from the MM atom positions using the Amber or CHARMM force field equation and parameters, whereas H_{QM} is evaluated using the chosen QM method.

The interaction term $H_{QM/MM}$ is more complicated. By default, *sander* uses an electrostatic embedding scheme (also referred to as additive scheme) in which the interaction of the MM point charges with the electrons of the QM system as well as the interaction between the MM point charges and the QM nuclei (atomic cores for semi-empirical methods) is explicitly taken into account. In other words, the MM region polarizes the QM electron density. For the case where there are no covalent bonds between the atoms of the QM and MM regions the interaction Hamiltonian is thus the sum of an electrostatic term and a Lennard-Jones (VDW) term and can be written as

$$H_{QM/MM} = \sum_q \sum_m \left[Q_m h_{electron}(x_e, x_{MM}) - Q_m Z_q h_{core}(x_{QM}, x_{MM}) + \left(\frac{A}{r_{qm}^{12}} - \frac{B}{r_{qm}^6} \right) \right] \quad (11.3)$$

where the subscripts e , m and q refer to the electrons, the MM nuclei and the QM nuclei respectively. Here Q_m is the charge on MM atom m , Z_q is the core charge (nucleus minus core electrons) on QM atom q , r_{qm} is the distance between atoms q and m , and A and B are Lennard-Jones interaction parameters. For systems that have covalent bonds between the QM and MM regions, the situation is more complicated, as discussed later.

A more approximate form of the interaction term $H_{QM/MM}$ is referred to as mechanical embedding (or subtractive QM/MM scheme). In this case the interactions between the QM and the MM region are obtained within the same classical approximation that is used for the MM region, that is

$$H_{QM/MM} = \sum_q \sum_m \left[\frac{Q_m Q_q}{r_{qm}} + \left(\frac{A}{r_{qm}^{12}} - \frac{B}{r_{qm}^6} \right) \right] \quad (11.4)$$

where Q_q is the classical MM point charge assigned to an atom in the QM region. Mechanical embedding is useful to impose steric constraints on the embedded QM system, however, the electron density is not polarized by the MM environment. An additional complication of this approach is that the point charges that are assigned to the atoms in the QM region have to represent the electrostatic potential of the QM region during the whole course of a QM/MM simulation.

If one evaluates the expectation values in Eq. 11.2 over a single determinant built from molecular orbitals

$$\phi_i = \sum_j c_{ij} \chi_j \quad (11.5)$$

where the c_{ij} are molecular orbital coefficients and the χ_j are atomic basis functions, the total energy depends upon the c_{ij} and on the positions x_{MM} and x_{QM} of the atoms. The energy is obtained by setting $\partial E_{eff} / \partial c_{ij}$ to zero which leads to a self-consistent (SCF) procedure to determine the c_{ij} , (with a modified Fock matrix that contains the electric field arising from the MM charges in the case of electrostatic embedding). Once the energy is known, the forces on the atoms can be obtained by taking the derivative of the energy expression with respect to the positions of the QM and MM atoms.

The main subtlety that arises in the case of electrostatic embedding is that, for a periodic system, there are formally an infinite number of QM/MM interactions; even for a non-periodic system, the (finite) number of such

interactions may be prohibitively large. These problems are addressed in a manner analogous to that used for pure MM systems: a PME approach is used for periodic systems, and a (large) cutoff may be invoked for non-periodic systems. Some details are discussed below.

11.1.1. The QM/MM interface and link atoms

The sections above dealt with situations where there are no covalent bonds between the QM and MM regions. In many protein simulations, however, it is necessary to have the QM/MM boundary cut covalent bonds, and a number of additional approximations have to be made. There are a variety of approaches to this problem, including hybrid orbitals, capping potentials, and explicit link atoms. The last option is the method available in *sander*.

There are a number of ways to implement a link atom approach that deal with the way the link atom is positioned, the way the forces on the link atom are propagated, and the way non-bonding interactions around the link atom are treated. Each time an energy or gradient calculation is to be done, the link atom coordinates are re-generated from the current coordinates of the QM and MM atoms making up the QM-MM covalent pair. The link atom is placed along the bond vector joining the QM and MM atom, at a distance d_{L-QM} from the QM atom. By default d_{L-QM} is set to the equilibrium distance of a methyl C-H atom pair (1.09 Å) but this can be set in the input file. The default link atom type is hydrogen, but this can also be specified as an input.

Since the link atom position is a function of the coordinates of the "real" atoms, it does not introduce any new degrees of freedom into the system. The chain rule is used to re-write forces on the link atom itself in terms of forces on the two real atoms that define its position. This is analogous to the way in which "extra points" or "lone-pairs" are handled in MM force fields.

The remaining details of how the QM-MM boundary is treated are as follows: for the interactions surrounding the link atom, the MM bond term between the QM and MM atoms is calculated classically using the classical force field parameters, as are any angle or dihedral terms that include at least one MM atom. The Lennard-Jones interactions between QM-MM atom pairs are calculated in the same way as described in the section above with exclusion of 1-2 and 1-3 interactions and scaling of 1-4 interactions. What remains is to specify the electrostatic interactions between QM and MM atoms around the region of the link atom.

A number of different schemes have been proposed for handling link-atom electrostatics. Many of these have been tested or calibrated on (small) gas-phase systems, but such testing can neglect some considerations that are very important for more extended, condensed-phase simulations. In choosing our scheme, we wanted to ensure that the total charge of the system is rigorously conserved (at the correct value) during an MD simulation. Further, we strove to have the Mulliken charge on the link atom (and the polarity of its bond to the nearest QM atom) adopt reasonable values and to exhibit only small fluctuations during MD simulations. Link atoms interact with the MM field in exactly the same way as regular QM atoms. That is they interact with the electrostatic field due to all the MM atoms that are within the cutoff, with the exception of the MM link pair atoms (MM atoms that are bound directly to QM atoms). VDW interactions are not calculated for link atoms. These are calculated between all real QM atoms and all MM atoms, including the MM link pair atoms. For Generalized Born simulations the effective Born radii for the link atoms are calculated using the intrinsic radii for the MM link pair atoms that they are replacing.

In the case of electrostatic embedding the atoms that make up the QM region (including the MM link pair atom) have their charges from the prmtop file essentially replaced with Mulliken charges. Hence it is important to consider the issue of charge conservation. The QM region (including the link atoms) by definition must have an integer charge. This is defined by the &qmmm namelist variable *qmcharge*. If the MM atoms (including the MM link pair atoms) that make up the QM region have prmtop charges that sum to the value of *qmcharge* then there is no problem. If not, there are two options for dealing with this charge, defined by the namelist variable *adjust-q*. A value of 1 will distribute the difference in charge equally between the nearest *nlink* MM atoms to the MM link pair atoms. A value of 2 will distribute this charge equally over all of the MM atoms in the simulation (excluding MM link pair atoms).

11.1.2. A reformulated QM/MM interface for PM3

In the current version of Amber, a reformulated QM-MM core-charge potential (denoted as PM3/MM*) has been implemented. This reformulated potential scales the interaction between a QM core and a MM charge for the

11. QM/MM calculations

purpose of better description of the geometry and energy at the QM-MM interface:[367]

$$E_{QM/MM}^{core} = Z_a q_m (s_a s_a, s_m s_m) \left[1 + \frac{|q_m|}{q_m} \cdot \left(-e^{-f_1^a \cdot R_{am}} + e^{-f_2^a \cdot R_{am}} \right) \right] \quad (11.6)$$

where Z_a is the effective core charge of QM atom a , q_m is the partial charge on MM atom m , s_a is an s orbital on the QM atom, s_m is a notional s orbital on the MM atom, R_{am} is the QM-MM interatomic distance, and f_1^a and f_2^a are exponential scale factors which depend on the QM atom only. Optimal values for f_1^a and f_2^a were determined based on the PM3 Hamiltonian, and are available for H, C, N and O atoms (so the QM region is limited to these four atoms; but the MM region is not restricted). Application of this reformulated potential shows improved prediction of geometry and interaction energy at the QM-MM interface for hydrogen bonded small molecule complexes typical of biomolecular interactions, without significantly impacting the modeling of other interaction types, such as dispersion dominant complexes.[367] In a QM/MM calculation, giving `qmmm_int=3` along with `qm_theory=PM3` will invoke this potential.

Based on PM3/MM*, further developments to the semi-empirical QM/MM coupling method have been introduced – PM3/MMX2 (`qmmm_int=4` and `qm_theory=PM3`) – which shares the same QM core-MM charge equation with the PM3/MM* model. In addition, a QM parameter, ρ_{mm} , is introduced to each type of QM atoms in order to "fine-tune" the QM electron-MM charge interaction (Eq. 11.7). Although ρ_{mm} is a parameter for QM atom, the subscript *mm* emphasizes that it is a MM-related property (eqn 3.xx). Parameters are currently available for H, C, N, O and S QM atoms (manuscript in preparation).

$$E_{QM/MM}^{electron} = -q_m (\mu_a v_a, s_m s_m) = \sum_{\ell_a} \sum_{\ell_m} [M_{\ell_a k}^a M_{\ell_m k}^m] \quad (11.7)$$

where

$$[M_{\ell_a k}^a M_{\ell_m k}^m] = \frac{e^2}{2^{l_a+l_m}} \sum_{i=1}^{l_a} \sum_{j=1}^l \left[r_{ij}^2 + (\rho_{l_a}^a + \rho_{mm}^a)^2 \right]^{-1/2} \quad (11.8)$$

11.1.3. Generalized Born implicit solvent

The implementation of Generalized Born (GB) for QM/MM calculations is based on the method described by Pellegrini and Field.[363] Here, the total energy is taken to be E_{eff} from Eq. 11.2 plus E_{gb} from Eq. 4.2. In E_{gb} , charges on the QM atoms are taken to be the Mulliken charges determined from the quantum calculation; hence these charges depend upon the molecular orbital coefficients c_{ij} as well as the positions of the atoms.

As with conventional QM/MM simulations, one then solves for the c_{ij} by setting $\partial E_{eff} / \partial c_{ij} = 0$. This leads to a set of SCF equations with a Fock matrix modified not only by the presence of MM atoms (as in "ordinary" QM/MM simulations), but also modified by the presence of the GB polarization terms. Once self-consistency is achieved, the resulting Mulliken charges can be used in the ordinary way to compute the GB contribution to the total energy and forces on the atoms.

11.1.4. Ewald and PME

The support for long range electrostatics in QM/MM calculations using electrostatic embedding is based on a modification of the Nam, Gao and York Ewald method for QM/MM calculations.[364] This approach works in a similar fashion to GB in that Mulliken charges are used to represent long range interactions. Within the cutoff, interactions between QM and MM atoms are calculated using a full multipole treatment. Outside of the cutoff the interaction is based on pairwise point charge interactions. For semiempirical NDDO-type methods this leads to a slight discontinuity at the QM/MM cutoff boundary and thus a small energy drift during QM/MM MD simulations in the NVE ensemble. This energy drift can be avoided by using a switching function at the cutoff (see below).

The implementation in Ref [364] uses an Ewald sum for both QM/QM and QM/MM electrostatic interactions. This can be expensive for large MM regions, and thus *sander* uses a modification of this method by Walker and Crowley[327] that uses a PME model (rather than an Ewald sum) for QM/MM interactions. This is controlled by the `qm_pme` variable discussed below.

When running QM/MM Ewald or PME simulations in *sander*, if QM multipoles are involved in QM-MM interactions (NDDO methods), a discontinuity in the QM-MM electrostatic potential occurs at the cutoff distance due to the sudden change in the potential function (the difference between Eqs. 11.9 and 11.10), thus resulting in energy conservation problems in the simulation.

$$E_{QM/MM}^{r < cutoff} = -q_m(\mu_a v_a, s_m s_m) + Z_a q_m(s_a s_a, s_m s_m)(1 + scale) \quad (11.9)$$

$$E_{QM/MM}^{r > cutoff} = \frac{q_m(Z_a - \sum c_{\mu\mu})}{r} \quad (11.10)$$

This problem can be avoided by applying a switching function to smoothly connect the two different potentials. The QM/MM electrostatic potential using a switching function can thus be written as:

$$E_{QM/MM} = E_{QM/MM}^{r < cutoff} s(r) + E_{QM/MM}^{r > cutoff} (1 - s(r))$$

The switching function can be turned on or off via the *&qmmm* namelist variable *qmmm_switch*, for details see section 11.1.6 below.

11.1.5. Hints for running successful QM/MM calculations

Required Parameters and Prmtop Creation

QM/MM calculations without link atoms require mass, charges, van der Waals and GB radii in the *prmtop* file. All bonds, angles, and dihedrals parameters involving QM atoms are neglected. In the case of electrostatic embedding the charges are also neglected. (Note that when SHAKE is applied to the QM reg, the bonds are constrained to the ideal MM values, even when these are part of a QM region; hence, for this case, it is important to have correct bond parameters in the QM region.) The simplest general prescription for setting things up is to use *antechamber* and *LEaP* to create a reference force field, since "placeholders" are required in the *prmtop* file even for things that will be neglected. This also allows you to run comparison simulations between pure MM and QM/MM simulations, which can be helpful if problems are encountered in the QM/MM calculations.

The use of *antechamber* to construct a pure MM reference system is even more useful when there are link atoms, since here MM parameters for bonds, angles and dihedrals that cross the QM/MM boundary are also needed.

Choosing the QM region

There are no good universal rules here. Generally, one might want to have as large a QM region as possible, but having more than 80-100 atoms in the QM region will lead to simulations that are very expensive. One should also remember that for many features of conformational analysis, a good MM force field may be better than a semiempirical or DFTB quantum description. In choosing the QM/MM boundary, it is better to cut non-polar bonds (such as C-C single bonds) than to cut unsaturated or polar bonds. Link atoms are not placed between bonds to hydrogen. Thus cutting across a C-H bond will NOT give you a link atom across that bond. (This is not currently tested for in the code and so it is up to the user to avoid such a situation.) Furthermore, link atoms are restricted to one per MM link pair atom. This is tested for during the detection of link atoms and an error is generated if this requirement is violated. This would seem to be a sensible policy otherwise you could have two link atoms too close together. See the comments in *qm.link.atoms.f* for a more in-depth discussion of this limitation.

Choice of electrostatic cutoff

The implementation of the non-bonded cut off in QM/MM simulations is slightly different than in regular MM simulations. The cut off between MM-MM atoms is still handled in a pairwise fashion. However, for QM atoms any MM atom that is within *qmcut* of ANY QM atom is included in the interaction list for all QM atoms. This means that the value of *qmcut* essentially specifies a shell around the QM region rather than a spherical shell around each individual QM atom. Ideally the cut off should be large enough that the energy as a function of the cutoff has converged. For non-periodic, generalized Born simulations, a cutoff of 15 to 20 Å seems sufficient in some tests. (Remember that long-range electrostatic interactions are reduced by a factor of 80 from their gas-phase

11. QM/MM calculations

counterparts, and by more if a nonzero salt concentration is used.) For periodic simulations, the cutoff only serves to divide the interactions between "direct" and "reciprocal" parts; as with pure MM calculations, a cutoff of 8 or 9 Å is sufficient here.

Parallel simulations

The built-in QM/MM implementation currently supports execution in parallel via the message passing interface (MPI), however, the implementation is not fully parallel. At present all parts of the QM simulation are parallel except the density matrix build and the matrix diagonalisation. For small QM systems these two operations do not take a large percentage of time and so acceptable scaling can be seen to around 8 CPU cores (depending on type of CPU and/or interconnect speed between compute nodes). For large QM systems the matrix diagonalisation time will dominate and so the scaling will not be as good. In this case it may be beneficial to choose a LAPACK diagonalization routine in combination with a threaded library such as the Intel Math Kernel Library (MKL). For details on how to choose the diagonalization routine see Section 9.3. The number of threads to be used for the diagonalization is set via an environment variable of the operating system (typically OMP_NUM_THREADS).

11.1.6. General QM/MM &qmmm Namelist Variables

An example input file for running a simple QM/MM MD simulation is shown here:

```
&cntrl
  imin=0, nstlim=10000,      ! Perform MD for 10,000 steps
  dt=0.002,                  ! 2 fs time step
  ntt=1, tempi=0.1, temp0=300.0, ! Berendsen temperature control
  ntb=1,                     ! Constant volume periodic boundaries
  ntf=2, ntc=2,              ! Shake hydrogen atoms
  cut=8.0,                   ! 8 angstrom classical non-bond cut off
  ifqnt=1                    ! Switch on QM/MM coupled potential
/
&qmmm
  qmmask=':753',            ! Residue 753 should be treated using QM
  qmcharge=-2,               ! Charge on QM region is -2
  qm_theory='PM3',           ! Use the PM3 semi-empirical Hamiltonian
  qmcut=8.0                  ! Use 8 angstrom cut off for QM region
/
```

The *&qmmm* namelist contains variables that allow you to control the options used for a QM/MM simulation. This namelist must be present when running QM/MM simulations and at the very least must contain either the *iqmatoms* or *qmmask* variable which define the region to be treated quantum mechanically. If *ifqnt* is set to zero then the contents of this namelist are ignored.

For the QM region definition specify one of either *iqmatoms* or *qmmask*. Link atoms will be added automatically along bonds (as defined in the prmtop file) that cross the QM/MM boundary.

<i>iqmatoms</i>	comma-separated integer list containing the atom numbers (from the prmtop file) of the atoms to be treated quantum mechanically.
<i>qmmask</i>	Mask specifying the quantum atoms. E.g. :1-2, = residues 1 and 2. See mask documentation for more info.
<i>qmcut</i>	Specifies the size of the electrostatic cutoff in Angstroms for QM/MM electrostatic interactions. By default this is the same as the value of cut chosen for the classical region, and the default generally does not need to be changed. Any classical atom that is within qmcut of <i>any</i> QM atom is included in the pair list. For PME calculations, this parameter just affects the division of forces between direct and reciprocal space. <i>Note:</i> this option only effects the electrostatic interactions between the QM and MM regions. Within the QM region all QM atoms see all other QM atoms

regardless of their separation. QM-MM van der Waals interactions are handled classically, using the cutoff value specified by *cut*.

- qm_ewald** This option specifies how long range electrostatics for the QM region should be treated.
- = 0** Use a real-space cutoff for QM-QM and QM-MM long range interactions. In this situation QM atoms do not see their images and QM-MM interactions are truncated at the cutoff. This is the default for non-periodic simulations.
 - = 1** (default) Use PME or an Ewald sum to calculate long range QM-QM and QM-MM electrostatic interactions. This is the default when running QM/MM with periodic boundaries and PME.
 - = 2** This option is similar to option 1 but instead of varying the charges on the QM images as the central QM region changes the QM image charges are fixed at the Mulliken charges obtained from the previous MD step. This approach offers a speed improvement over *qm_ewald=1*, since the SCF typically converges in fewer steps, with only a minor loss of accuracy in the long range electrostatics. This option has not been extensively tested, although it becomes increasingly accurate as the box size gets larger.
- kmaxqx, y, z** Specifies the maximum number of kspace vectors to use in the x, y and z dimensions respectively when doing an Ewald sum for QM-MM and QM-QM interactions. Higher values give greater accuracy in the long range electrostatics but at the expense of calculation speed. The default value of 8 should be optimal for most systems.
- ksqmaxq** Specifies the maximum number of K squared values for the spherical cut off in reciprocal space when doing a QM-MM Ewald sum. The default value of 100 should be optimal for most systems.
- qm_pme** Specifies whether a PME approach or regular Ewald approach should be used for calculating the long range QM-QM and QM-MM electrostatic interactions.
- = 0** Use a regular Ewald approach for calculating QM-MM and QM-QM long range electrostatics. Note this option is often much slower than a pme approach and typically requires very large amounts of memory. It is recommended only for testing purposes.
 - = 1** (default) Use a QM compatible PME approach to calculate the long range QM-MM electrostatic energies and forces and the long range QM-QM forces. The long range QM-QM energies are calculated using a regular Ewald approach.
- qmmm_switch** Specifies whether a switching function shall be used at the cutoff for long range electrostatics (applies only to NDDO methods). The lower and higher boundaries of the switching function are user definable, see *r_switch_lo* and *r_switch_hi*.
- = 0** (default). Do not use a switching function. This leads to slight discontinuities in the potential at the cut off and thus an energy drift in NVE simulations.
 - = 1** Use a switching function. See also variables *r_switch_hi* and *r_switch_lo*.
- r_switch_hi** Specifies the upper boundary of the switching function in Å (see *qmmm_switch*). Defaults to *qmcut*.
- r_switch_lo** Specifies lower boundary of the switching function in Å (see *qmmm_switch*). Defaults to *r_switch_hi* - 2.
- qmgb** Specifies how the QM region should be treated with generalized Born.
- = 2** (default) As described above, the electrostatic and "polarization" fields from the MM charges and the exterior dielectric (respectively) are included in the Fock matrix for the QM Hamiltonian.

11. QM/MM calculations

- = 3** This is intended as a debugging option and should only be used for single point calculations. With this option the GB energy is calculated using the Mulliken charges as with option 2 above but the fock matrix is NOT modified by the GB field. This allows one to calculate what the GB energy would be for a given structure using the gas phase quantum charges. When combined with a simulation using *qmgb=2*, this allows the strain energy from solvation to be calculated.
- qm_theory** Level of theory to use for the QM region of the simulation. (Hamiltonian). Default is to use the semi-empirical hamiltonian PM3. See the [Section 9.3](#) for details.
- qmmm_int** Controls the way in which QM/MM interactions are handled in the direct space QMMM sum. This controls only the electrostatic interactions. VDW interactions are always calculated classically using the standard 6-12 potential. Note: with the exception of *qmmm_int=0* DFTB calculations (*qm_theory=DFTB*) always use a simple Mulliken charge - resp charge interaction and the value of *qmmm_int* has no influence.
- = 0** This turns off all electrostatic interaction between QM and MM atoms in the direct space sum. Note QM-MM VDW interactions will still be calculated classically.
- = 1** (default) QM-MM interactions in direct space are calculated in the same way for all of the various semi-empirical hamiltonians. The interaction is calculated in an analogous way to the the core-core interaction between QM atoms. The MM resp charges are included in the one electron hamiltonian so that QMcore-MMResp and QMelectron-MMResp interactions are calculated.
- = 2** This is the same as for 1 above except that when AM1, PM3 or Hamiltonians derived from these are in use the extra Gaussian terms that are introduced in these methods to improve the core-core repulsion term in QM-QM interactions are also included for the QM-MM interactions. This is the equivalent to the QM-MM interaction method used in CHARMM and DYNAMO. It tends to slightly reduce the repulsion between QM and MM atoms at small distances. For distances above approximately 3.5 angstroms it makes almost no difference.
- = 3** Using this along with *qm_theory=PM3* invokes a reformulated QM core-MM charge potential at the QM-MM interface (Eq. 11.6). Current parametrization limits the QM region to H, C, N and O atoms only; MM region is not restricted.[367]
- = 4** Currently not in use.
- = 5** Mechanical embedding: The electrostatic interaction between QM and MM atoms is treated on the same level as within the MM region using the classical force field point charges also for the QM atoms. The electronic Hamiltonian does not contain the field generated by the MM region point charges and thus the electron density is not polarized by the MM environment. Does not work with GB. Not extensively tested in presence of link atoms.
- qmshake** Controls whether SHAKE is applied to QM atoms. Using SHAKE on the QM region will allow you to use larger time steps such as 2 fs with *NTC=2*. If, however, you expect bonds involving hydrogen to be broken during a simulation you should not SHAKE for the QM region. WARNING: the SHAKE routine uses the equilibrium bond lengths as specified in the prmtop file to reset the atom positions. Thus while bond force constants and equilibrium distances are not used in the energy calculation for QM atoms the equilibrium bond length is still required if QM SHAKE is on.
- = 0** Do not shake QM H atoms.
- = 1** Shake QM H atoms if SHAKE is turned on (*NTC>1*) (default).
- printdipole** Controls whether the dipole moment shall be printed every *ntpr* steps.
- = 0** Do not print the dipole moment (default).
- = 1** Print the dipole moment of the QM region.

- = 2** Print the total dipole moment of the QM and MM region.
- `writpdb` **= 0** Do not write a PDB file of the selected QM region. (default).
= 1 Write a PDB file of the QM region. This option is designed to act as an aid to the user to allow easy checking of what atoms were included in the QM region. When this option is set a crude PDB file of the atoms in the QM region will be written on the very first step to the file *qmmm_region.pdb*.
- `vsolv` Controls whether solvent molecules shall be included into the QM region (requires settings in the *&vsolv* namelist; see also section 11.8 on adaptive solvent QM/MM simulations, in particular the namelist information in section 11.8.2.2).
= 0 Do not include solvent molecules into the QM region (default).
= 1 Include solvent molecules via simple solvent switching (requires *&vsolv* namelist).
= 2 Adaptive solvent QM/MM with fixed number of solvent molecules in A and T regions (requires *&vsolv* and *&adqmmm* namelists).
= 3 Adaptive solvent QM/MM with fixed size of A and T regions (requires *&vsolv* and *&adqmmm* namelists).

In addition to the above parameters, the following variables may be set, as described in Section 9.3:

`qm_theory`, `dftb_disper`, `dftb_3rd_order`, `dftb_chg`, `dftb_telec`, `dftb_maxiter`, `qmcharge`, `spin`, `qmqmdx`, `verbosity`, `tight_p_conv`, `scfconv`, `pseudo_diag`, `pseudo_diag_criteria`, `diag_routine`, `printcharges`, `qxd`, `parameter_file`, `peptide_corr`, and `itmax`.

11.1.7. Link Atom Specific QM/MM &qmmm Namelist Variables

The following options go in the *&qmmm* namelist and control the link atom behaviour.

- `lnk_dis` Distance in Å from the QM atom to its link atom. Currently all link atoms must be placed at the same distance. A negative value of `lnk_dis` specifies that the link atom should be placed directly on top of the MM link pair atom. In this case the distance of the link atom from the QM region changes as a function of time and the actual value of `lnk_dis` is ignored. Additionally this means that not all link atoms will be placed at the same distance. Negative values of `lnk_dis` will work with regular link atoms, such as hydrogen, but are really intended for use with pseudo atom / capping approaches. Default = 1.09Å.
- `lnk_method` This defines how classical valence terms that cross the QM/MM boundary are dealt with.
=1 (Default) in this case any bond, angle or dihedral that involves at least one MM atom, including the MM link pair atom is included. This means the following (where QM = QM atom, MM = MM atom, MML = MM link pair atom.):
Bonds = MM-MM, MM-MML, MML-QM
Angles = MM-MM-MM, MM-MM-MML, MM-MML-QM, MML-QM-QM
Dihedrals = MM-MM-MM-MM, MM-MM-MM-MML, MM-MM-MM-MML-QM, MM-MML-QM-QM, MML-QM-QM-QM
=2 Only include valence terms that include a full MM atom, that is, count the MM link pair atom as effectively being a QM atom. This option is designed to be used in conjunction with a pseudo atom / capping type approach where the link atom is parameterized specifically to behave like a uni-valent version of the MM atom it replaces. This option gives the following interactions:
Bonds = MM-MM, MM-MML
Angles = MM-MM-MM, MM-MM-MML, MM-MML-QM

11. QM/MM calculations

Dihedrals = MM-MM-MM-MM, MM-MM-MM-MML, MM-MM-MML-QM, MM-MML-QM-QM

`lnk_atomic_no` The atomic number of the link atoms. This selects what element the link atoms are to be. Default = 1 (Hydrogen). Note this must be an integer and an atomic number supported by the chosen QM theory.

`adjust_q` This controls how charge is conserved during a QMMM calculation involving link atoms. When the QM region is defined the QM atoms and any MM atoms involved in link bonds have their RESP charges zeroed. If the sum of these RESP charges does not exactly match the value of *qmcharge* then the total charge of the system will not be correct.

= 0 No adjustment of the charge is done.

= 1 The charge correction is applied to the nearest *nlink* MM atoms to MM atoms that form link pairs. Typically this will be any MM atom that is bonded to a MM link pair atom (a MM atom that is part of a QM-MM bond). This results in the total charge of QM+QMLink+MM equaling the original total system charge from the *prmtop* file. Requires *natom-nquant-nlink* $\geq nlink$ and *nlink* > 0 .

= 2 (default) - This option is similar to option 1 but instead the correction is divided among all MM atoms (except for those adjacent to link atoms). As with option 1 this ensures that the total charge of the QM/MM system is the same as that in the *prmtop* file. Requires *natom-nquant-nlink* $\geq nlink$.

11.1.8. Charge-dependent exchange-dispersion corrections of vdW interactions

The *sqm* program provides a new charge-dependent energy model consisting of van der Waals (vdW) and polarization interactions between the quantum mechanical (QM) and molecular mechanical (MM) regions in a combined QM/MM calculation. vdW interactions are commonly treated using empirical Lennard-Jones (L-J) potentials, whose parameters are often chosen based on the QM atom type (e.g., based on hybridization or specific covalent bonding environment). This strategy for determination of QM/MM nonbonding interactions becomes tedious to parametrize and lacks robust transferability. Problems occur in the study of chemical reactions where the "atom type" is a complex function of the reaction coordinate. This is particularly problematic for reactions, where atoms or localized functional groups undergo changes in charge state and hybridization.

In *sqm*, this charge-dependent energy model was implemented based on a scaled overlap model for repulsive exchange and attractive dispersion interactions that is a function of atomic charge. The model is chemically significant since it properly correlates atomic size, softness, polarizability, and dispersion terms with minimal one-body parameters that are functions of the atomic charge[331].

This "Charge-dependent exchange-dispersion corrections of vdW interactions" can be invoked by the "`qxd=true.`" switch in the *&qmmm* namelist. Note that this model currently does not have any effect on pure quantum calculations through *sqm*, the *qxd* correction is only added to QM/MM interactions in *sander*. The default values of *qxd* parameters are set to reproduce the regular L-J interactions of typical atom types (HC for H, C* for C, N for N, OW for O, and parameters for F and Cl are optimized[331]) when the charge dependence parameters are zero. There are eight *qxd* parameters (symbols used in the reference[331] are indicated in the parentheses): *qxd_s* (*s*), *qxd_z0* ($\zeta(0)$), *qxd_zq* (ζ_q), *qxd_d0* (α_1), *qxd_dq* ($3 \times B$), *qxd_q0* (α_2), *qxd_qq* ($3 \times B$), and *qxd_neff* ($N_{eff}(0)$). All parameters can be modified through external user-defined parameter files (see the usage of 'parameter_file' in Section 9.3).

11.2. Interface for *ab initio* and DFT methods

In addition to the built-in semi-empirical methods *sander* also supports QM/MM simulations with *ab initio* wave function theory (WFT) and density functional theory (DFT) potentials via an interface to external QM software packages[365]. The implementation makes use of the existing QM/MM infrastructure that has been developed

earlier for the semi-empirical methods. Thus, much of AMBER's previous QM/MM functionality such as the user-friendly link atom approach are available and the implementation remains simple and transparent to use without any significant additional steps in the simulation setup as compared to semi-empirical QM/MM simulations. At present the interface supports several well-known and widely used QM software packages. Mechanical embedding is available for

- ADF (Amsterdam Density Functional) [368, 369]
- GAMESS-US [370, 371]
- NWChem [372]

Mechanical and electrostatic embedding is available for

- Gaussian [373]
- Orca [374]
- Q-Chem[375][375]
- TeraChem [376]
- QUICK [356–362]
- MRCC [377, 378]
- Fireball [379]

While ADF, Gaussian, Q-Chem and TeraChem are commercial programs, GAMESS-US, NWChem, Orca, QUICK, MRCC and Fireball are available at no cost for academic research. TeraChem has a demo version freely available at <http://www.petachem.com>. In this version, each QM calculation can be ran up to 15 minutes using up to 2 GPUs; it supports QM/MM simulations with AMBER (see section 11.4). QUICK is available as standalone QM package but also distributed with AmberTools (see chapter 10). The QUICK library can be linked against *sander* and we recommend using this API based version over the file based interface (FBI), for details see section 11.3. Fireball, which implements a density functional theory-based tight binding approach, requires compilation of *sander* with special flags, see the section on Fireball below for details. The interface has been written in a modular fashion and is easily extensible to support other QM software packages. It is our intention to keep adding support for other software packages. If you are interested in interfacing a specific program, please do not hesitate to contact us.

The interface was developed by Andreas Goetz (SDSC, UCSD) with help of Matthew Clark (SDSC) and support by Ross Walker (SDSC, UCSD). Thanks are due to Christine Isborn and Todd Martinez (Stanford University) for modifications to the TeraChem code to support this interface, to Mark Williamson (University of Cambridge) for an initial version of the module that supports NWChem, Bence Hégely for contributing code that supports MRCC, and Jesús Mendieta and José Ortega Mateo for contributing code that supports Fireball. If you make use of this interface, please cite the following work:

- A. W. Götz, M. A. Clark, R. C. Walker, *An extensible interface for QM/MM molecular dynamics simulations with AMBER*, J. Comput. Chem. **35**, 95-108 (2014), DOI: 10.1002/jcc.23444

If you are doing QM/MM simulations with QUICK, please cite in addition the following work:

- V. W. D. Cruzeiro, M. Manathunga, K. M. Merz, A. Götz, *Open-Source Multi-GPU-Accelerated QM/MM Simulations with AMBER and QUICK*. J. Chem. Inf. Model. **61**, 2019 (2021), DOI: 10.1021/acs.jcim.1c00169
- Manathunga, M.; Aktulga, H. M.; Götz, A. W.; Merz, K. M. *Quantum Mechanics/Molecular Mechanics Simulations on NVIDIA and AMD Graphics Processing Units*. J. Chem. Inf. Model. **63**, 711-717 (2023), DOI: 10.1021/acs.jcim.2c01505

11. QM/MM calculations

If you are using the client/server interface with TeraChem (using TCPB-cpp, see section 11.4), the recommended interface, please cite in addition the following work:

- V. W. D. Cruzeiro, Y. Wang, E. Pieri, E. G. Hohenstein, T. J. Martínez, *TeraChem protocol buffers (TCPB): Accelerating QM and QM/MM simulations with a client–server model*, J. Chem. Phys. **158**, 044801 (2023), DOI: 10.1063/5.0130886

If you are using the file-based interface with TeraChem, please cite in addition the following work:

- C. M. Isborn, A. W. Götz, M. A. Clark, R. C. Walker, T. J. Martínez, *Electronic Absorption Spectra from MM and ab initio QM/MM Molecular Dynamics: Environmental Effects on the Absorption Spectrum of Photoactive Yellow Protein*, J. Chem. Theory Comput. **8**, 5092-5106 (2012), DOI: 10.1021/ct3006826

If you are using the interface with the MRCC code, please cite in addition the following work:

- B. Hégyel, F. Bogár, G. G. Ferenczy, M. Kállay, *A QM/MM program for calculations with frozen localized orbitals based on the Huzinaga equation*, Theoret. Chem. Acc. **134**, 132 (2015), DOI: 10.1007/978-3-662-49825-5_16

If you are using the interface with the Fireball code, please cite in addition the following work:

- J. I. Mendieta-Moreno, R. C. Walker, J. P. Lewis, P. Gómez-Puertas, J. Mendieta, J. Ortega, *FIREBALL/AMBER: An efficient local-orbital DFT QM/MM method for biomolecular systems*, J. Chem. Theory Comput. **10**, 2185-2193 (2014), DOI: 10.1021/ct500033w

Access to QM methods not available within Amber is also possible via the Amber interface to the PUPIL simulation framework. For details, see refs. 380, 381. In what follows we will describe the QM/MM interface that is native to *sander*.

11.2.1. Theory

As described in section 11.1, the Hamiltonian of a system that is partitioned into a QM region that is treated with WFT and a classical region that is treated with MM consists of three components and the energy associated with this Hamiltonian is obtained as the corresponding expectation value

$$E = \langle \Psi | \mathcal{H}_{QM} + \mathcal{H}_{QM/MM} | \Psi \rangle + E_{MM}. \quad (11.11)$$

A QM/MM calculation therefore requires not only to choose the WFT used in the QM region and the MM model used for the MM region, but in addition also the form of the QM/MM Hamiltonian which describes the interaction between the quantum and the classical region. The most simple approach is to neglect any electronic coupling between the QM and the MM system and include only the classical non-bonded van der Waals (vdW) and electrostatic interactions between the QM and the MM atoms. This is useful to impose steric constraints on the embedded QM system and commonly referred to as mechanical embedding. In most cases, however, it is better to allow for an explicit polarization of the QM system due to the presence of the point charges on the MM atoms. This is referred to as electronic embedding and the resulting interaction energy becomes

$$\begin{aligned} E_{QM/MM}^{electronic} = & \sum_{A \in MM} \int \rho(\mathbf{r}) \frac{Q_A}{|\mathbf{r} - \mathbf{R}_A|} d\mathbf{r} + \sum_{A \in QM, B \in MM} \frac{Z_A Q_B}{R_{AB}} \\ & + \sum_{A \in QM, B \in MM} \epsilon_{AB} \left[\left(\frac{\sigma_{AB}}{R_{AB}} \right)^{12} - \left(\frac{\sigma_{AB}}{R_{AB}} \right)^6 \right]. \end{aligned} \quad (11.12)$$

This QM/MM energy expression also holds for DFT and the terms represent, in order, the electrostatic interaction between the QM electron density and the MM point charges, the electrostatic interaction between the QM point charge nuclei and the MM point charges, and the van der Waals repulsion between the QM and MM atoms.

The forces acting on an atom A in a QM/MM calculation are given in terms of derivatives of the total energy expression (11.11) with respect to the Cartesian coordinates of the atom,

$$\mathbf{F}_A = -\nabla_A E_{QM} - \nabla_A E_{QM/MM} - \nabla_A E_{MM}, \quad (11.13)$$

where $\nabla_A = \partial/\partial \mathbf{R}_A = (\partial/\partial R_A^x, \partial/\partial R_A^y, \partial/\partial R_A^z)$. If a QM and an MM program are coupled for QM/MM calculations, the QM program will calculate the QM forces $-\nabla_A E_{QM}$ acting on QM atoms and the MM program the MM forces $-\nabla_A E_{MM}$ acting on the MM atoms. All that remains, is to calculate the forces acting on QM and MM atoms due to the QM/MM interaction energy, $-\nabla_A E_{QM/MM}$. For mechanical embedding this will be entirely handled by the MM program. For electronic embedding the forces are given as

$$\begin{aligned} \nabla_A E_{QM/MM}^{electronic} &= Z_A \sum_{B \in MM} \frac{Q_B(\mathbf{R}_A - \mathbf{R}_B)}{R_{AB}^3} + \sum_{B \in MM} \int \frac{\partial \rho(\mathbf{r})}{\partial \mathbf{R}_A} \frac{Q_B}{|\mathbf{r} - \mathbf{R}_B|} d\mathbf{r} + \sum_{B \in MM} \nabla_A V_{AB}^{LJ} \\ &= -Z_A \mathbf{E}_{MM}(\mathbf{R}_A) - \int \rho(\mathbf{r}) \mathbf{E}_{MM}(\mathbf{r}) d\mathbf{r} + \sum_{B \in MM} \nabla_A V_{AB}^{LJ} \end{aligned} \quad (11.14)$$

for the derivatives with respect to the positions of the QM atoms A where $\mathbf{E}_{MM}$ is the electric field generated by the MM point charges and V_{AB}^{LJ} is the Lennard-Jones potential from (11.12) and

$$\begin{aligned} \nabla_B E_{QM/MM}^{electronic} &= Q_B \sum_{A \in QM} \frac{Z_A(\mathbf{R}_B - \mathbf{R}_A)}{R_{AB}^3} + \int \rho(\mathbf{r}) \frac{Q_B(\mathbf{R}_B - \mathbf{r})}{|\mathbf{r} - \mathbf{R}_B|^3} d\mathbf{r} + \nabla_B E_{QM/MM}^{mechanic} \\ &= -Q_B \mathbf{E}_{QM}(\mathbf{R}_B) + \sum_{A \in QM} \nabla_B V_{AB}^{LJ} \end{aligned} \quad (11.15)$$

for the derivatives with respect to the positions of the MM atoms B where $\mathbf{E}_{QM}$ is the electric field due to the QM charge distribution. The contributions to the gradient due to the point charge interactions and due to the interaction between the MM point charges and the QM electrons is evaluated by the QM program. Some QM programs do not calculate the forces acting on the MM atoms (point charges) due to the presence of the QM system but in general are able to return the electric field $\mathbf{E}_{QM}$ at arbitrary points in space which is then used to obtain these forces. The van der Waals repulsion (Lennard-Jones interaction) between QM and MM atoms is treated by AMBER in the same way as for semiempirical NDDO-type and DFTB methods.

11.2.2. General Remarks

When using the AMBER interface to external QM software packages for performing WFT or DFT based QM or QM/MM MD simulations, it is absolutely critical to be aware of the capabilities and limitations of the QM method to be employed. In particular, QM based MD can be more tricky than MM based MD in the sense that it is more likely that the QM program can fail for example due to SCF convergence problems. This can be the case if the geometry of the QM region is far from its ground state equilibrium, for example because a simulation is started from a bad geometry or performed at high temperature.

We have gone to large efforts and analyzed a large set of test simulations to provide the best default parameters for the supported QM programs such that forces are computed with sufficient accuracy to guarantee energy conservation for constant energy MD simulations. Of particular importance are SCF convergence and associated integral neglect thresholds and the size of the grid used for the numerical quadrature of the exchange-correlation (XC) potential and energy for DFT calculations. However, other than providing appropriate input parameters, AMBER does not have any control over the external program and it is at the user's discretion to employ sensible input parameters for the QM program and to prepare the system such that the simulations are started at a reasonable starting structure.

In any case we highly recommend to write restart files frequently so that a simulation can be restarted without loss of much computational time in the case that a simulation should crash. The interface also stores the last in- and output files of the external QM program during each MD step. Should there be any problems with the QM program, it is therefore possible to analyze the reasons and take appropriate countermeasures.

11. QM/MM calculations

The interface requires data to be exchanged between *sander* and the QM program. The default operation of the interface is based on file exchange and system calls and, during each step of a geometry optimization or an MD simulation, writes an input file for the external program, starts a single point gradient calculation with the external program, and reads the energy and forces from the external program's output file (binary ADF checkpoint or formatted GAMESS, Gaussian, ORCA, QUICK, Q-Chem, MRCC or TeraChem output files). Data communication using a client/server model is also implemented and currently supported by TeraChem (see section 11.4); this is the recommended TeraChem interface. An exception is Fireball, which is interfaced as a linked library against *sander* (see below). Similarly QUICK is also interfaced as a linked library against *sander*, which is built by default (see section 11.3).

11.2.3. Limitations

In principle, all types of simulations that are possible with *sander* are supported. There are, however, some restrictions for simulations that require *sander* to run in parallel, in particular path integral molecular dynamics (PIMD) and replica exchange molecular dynamics (REMD), see the discussion of Parallelization below. The interface to external QM programs also lacks some features regarding solvent models in comparison to the semiempirical MNDO and DFTB QM/MM implementation that is available in AMBER, the most critical ones are listed here.

Generalized Born Generalized Born (GB) implicit solvent models are not supported if external QM programs are used for the QM region.

Particle Mesh Ewald (PME) and Periodic Boundary Conditions The PME approach for treating long-range electrostatic QM/MM and QM/QM interactions in periodic systems is currently not supported. It is possible to use periodic boundary conditions but a cutoff is used for the point charges to be included in the QM Hamiltonian (determined by *&qmmm* namelist variable *qmcut*) thus truncating the long-range QM/MM electrostatic interactions in (11.12). This leads to discontinuities in the potential energy surface and poor energy conservation for MD runs in the NVE ensemble. The user may consider running non-periodic simulations with a cutoff that is larger than the system size thus effectively including all interactions.

11.2.4. Performance Considerations

The computational cost of DFT is comparable to Hartree–Fock (HF) theory which is the simplest WFT method that serves as zeroth order approximation for more elaborate correlated WFT methods such as Møller–Plesset perturbation theory, configuration interaction theory and coupled cluster theory. The calculations can be accelerated by using density fitting approaches, sometimes called resolution-of-identity (RI) approximation, which in the case of DFT with exchange-correlation (XC) functionals that do not require admixture of exact HF-exchange, leads to speedups of roughly one order of magnitude without compromising the accuracy of the results. Nevertheless, the computational cost of DFT is in general two to three orders of magnitude higher than that of semiempirical QM models. We recommend to carefully test the performance of the QM program to choose an optimal number of processor counts for parallelized QM calculations. QUICK can be executed efficiently on one or multiple GPUs, providing high performance for QM/MM MD simulations. Typical simulation performance for typical QM system sizes of tens of atoms will be on the order of a few to tens of picoseconds per day, depending on the underlying QM model chosen.

11.2.5. Parallelization

The MPI parallel executable *sander.MPI* can be used to run QM/MM MD simulations with external QM software in which the MM portion of the calculation is parallelized. However, the computational cost of the MM part is usually small compared to the cost of the QM part. In order to execute the QM part of the calculation in parallel, the external QM program has to be instructed to do so, as described in the sections below.

In the case of PIMD or REMD simulations that require a separate energy and force evaluation for each group at each time step, the parallelized executable *sander.MPI* has to be used. Multiple processes can be launched per

group to parallelize the MM calculations. Care has to be taken to choose the right number of parallel threads in the external QM program. For example, on a machine with 32 cores, a simulation with 16 beads or replicas can run the external QM program with 2 threads in parallel to make maximum use of the available processing cores. If the available processors are spread over multiple nodes, special care has to be taken to ensure that the different instances of the external QM program are launched on the correct nodes.

It is possible to execute *sander.MPI* in parallel via MPI while also running MPI or OpenMP parallel versions of the external QM program. Depending on the MPI implementation, this can, however, fail. In our experience, MPICH and MVAPICH work well while OpenMPI does not work.

11.2.6. Usage

All that is required to use the interface is a working installation of AMBER and one or more of the supported QM programs. In order to use the external program from within *sander*, the *&cntrl* namelist variable *ifqnt* = 1 must be set to enable QM calculations and the *&qmmm* namelist variable *qm_theory* = 'EXTERN' must be set to enable the external interface. The *&qmmm* namelist variable *qmmask* or *iqmatoms* is used for selecting the QM region just as for QM/MM calculations with the semiempirical NDDO-type and DFTB approaches that are natively available in AMBER. Charge and spin multiplicity for the QM region need to be defined via the variables *qmcharge* and *spin*, respectively, in the *&qmmm* namelist. For a QM MD simulation, the *sander* input file therefore needs to contain

```
! example input for QM simulation with external QM program
&cntrl
...
  ifqnt = 1,                ! switch on QM/MM
/
&qmmm
  qmmask = '@*',           ! select QM atoms (here: make all QM)
  qmcharge = 0,            ! charge on QM region (default = 0)
  spin = 1,                ! spin multiplicity of QM region (default = 1)
  qm_theory = 'EXTERN',    ! use external QM program
/
```

For QM/MM simulations with electronic embedding (this is the default) we recommend to include all MM point charges as external electric field in the QM Hamiltonian to avoid problems with energy conservation. For non-periodic simulations this can be achieved by setting the *&qmmm* namelist variable *qmcut* to a value larger than the system size.

In addition either the *&adf*, *&gms*, *&nw*, *&gau*, *&orc*, *&qc*, *&mrcc* or *&tc* namelist must be present to use either ADF, GAMESS, NWChem, Gaussian, ORCA, Q-Chem, MRCC or TeraChem, respectively, and to assign parameters for the external QM program. Please refer to the ADF, GAMESS, NWChem, Gaussian, ORCA, Q-Chem, MRCC or TeraChem user manual for details on settings for the *ab initio* or DFT calculations. A list of namelist variables and their default setting is given below. The defaults have been chosen such that energy conserving MD simulations in the NVE ensemble are possible. NWChem has not been extensively tested.

Properties that are calculated along the trajectory are printed to property files with names *adf-job.ext*, *gms-job.ext*, *gau-job.ext*, *orc-job.ext*, *qc-job.ext* and *tc-job.ext*, where *ext* is either *dip* for dipole moment (x, y, z component and absolute value) or *chg* for atomic charges, where supported. These property files are only written if requested and will be deleted at the beginning of a run, so back them up in case a trajectory needs to be restarted.

All calculations with a spin multiplicity larger than one will automatically be performed in the framework of an unrestricted formalism (as opposed to restricted open shell), that is with unrestricted HF (UHF), unrestricted DFT (UDFT) and MP2 with a UHF reference wave function (UMP2).

In addition to controlling the external programs via the *sander* input file, you may supply a template input file for the external program in order to provide input that is not supported via the program specific namelists. To enable this option, you must set *use_template* = 1 in the program specific namelist. The format, name, and

11. QM/MM calculations

input requirements for the template file vary with the external program as detailed in the corresponding program's documentation below. If you are using your own template, please make sure that the parameters of the QM method (like SCF convergence threshold and XC quadrature grid size) yield sufficiently accurate forces. Please note that program settings supplied via the program specific namelist are ignored if a template input file is used.

11.2.6.1. AMBER/ADF

This interface has been developed with older versions of ADF and last tested with ADF 2019. To use ADF with the external interface, ADF must be properly installed on the working machine. In particular, the executable `adf` must be in the search path. By default the Becke integration grid with quality "good" and the ZLM fit method with quality "good" is employed. If you prefer to use the old pair fit method (or are using an older ADF version that does not support the ZLM fit), we recommend to use "ZORA/QZ4P" basis set for the density fit for sufficiently accurate forces.

Limitations At present only mechanical embedding is supported.

&adf Namelist variables

<code>basis</code>	Basis set type to be used in the DFT calculation. Valid standard basis set types are: SZ, DZ, DZP, TZP, TZ2P, TZ2P+ and ZORA/QZ4P. (Default: <code>basis = 'DZP'</code>)
<code>core</code>	Type of frozen core to use. Allowed values are: None, Small, Medium, Large. (Default: <code>core = 'None'</code>)
<code>zlmfit</code>	Quality of density fit with the ZLM fit method. (Default: <code>zlmfit = 'good'</code>)
<code>fit_type</code>	Fit basis set type to be used for density fitting with the old pair fit method. Valid values are identical to the available basis sets (SZ, DZ etc) in which case the fit basis corresponding to the AO Basis will be used. By default the ZLM fit method will be used (Default: <code>fit_type = ''</code>)
<code>xc</code>	Exchange-correlation functional to be used. Popular choices are 'LDA VWN', 'GGA BLYP', 'GGA PBE', 'HYBRID B3LYP' and 'HYBRID PBE0'. Consult the ADF manual for all available options. (Default: <code>xc = 'GGA BLYP'</code>)
<code>scf_iter</code>	Maximum number of SCF cycles allowed. (Default: <code>scf_iter = 50</code>)
<code>scf_conv</code>	Threshold upon which to stop the SCF procedure. The tested error is the commutator of the Fock matrix and the density matrix. Convergence is considered to be achieved if the maximum element of the commutator (which is zero for an optimized wave function) is smaller than <code>scf_conv</code> . (Default: <code>scf_conv = 1.0d-06</code>)
<code>beckegrid</code>	Quality of Becke integration grid. Allowed values are: Normal, Good, VeryGood. (Default: <code>core = 'Good'</code>)
<code>integration</code>	Numerical integration accuracy for integration with olde teVelde-Baerends integration grid (Voronoi cells). By default the Becke grid will be used. The old integration grid can be used by specifying a number larger than 0, we recommend at least 5.0. (Default: <code>integration = -1.0</code>)
<code>num_threads</code>	Number of threads (and thus CPU cores) for ADF to use. Note that this is not required if you are running in a queuing system as ADF will automatically use the full number of reserved cores. (Default: <code>num_threads = 0</code> [this causes ADF to use all available cores on a machine])
<code>use_dftb</code>	Specifies whether DFTB shall be used with ADF's DFTB program <code>dftb</code> . If <code>use_dftb = 1</code> then DFTB will be used and only variables <code>charge</code> and <code>scf_conv</code> will be considered. (Default: <code>use_dftb = 0</code> [do not use DFTB, regular DFT calculation]) - works only with older DFTB versions (prior to 2011).

exactdensity The exact (as opposed to fitted) electron density is used for the evaluation of the exchange-correlation potential if `exactdensity = 1`. (Default: `exactdensity = 0`)

use_template Determine whether or not to use a user-provided template file for running external programs. (Default: `use_template = 0`)

ntpr Controls frequency of printing for dipole moment to file `adf_job.dip` (Defaults to `&cntrl` namelist variable `ntpr`)

dipole Toggles writing of dipole moment to file `adf_job.dip` (Default: `dipole = 0`)

Example An input file for QM or (mechanical embedding) QM/MM MD with ADF using the PBE functional and the TZP basis set therefore would have to contain

```
&adf
  xc = 'GGA PBE',
  basis = 'TZP',
/
```

This would execute a simulation in which the Beckgrid with quality quality good and the ZLM fit with quality good are used (see default values above).

Template input file The template file for ADF should be named `adf_job.tpl` and must contain the following keywords:

```
BASIS ... END
SAVE TAPE21
```

You should not include the following (block) keywords in the template file as these are taken care of by *sander*:

```
UNITS
FRAGMENTS ... END
RESTART
GRADIENT
ATOMS ... END
```

11.2.6.2. AMBER/GAMESS-US

To use GAMESS with the external interface, GAMESS must be compiled on the target system. Make note of the version number you specify during the GAMESS compilation process (default is 00 which makes the GAMESS execution script `runrms` look for the executable `gamess.00.x`). If you use a different version number you must specify it with the `gms_version` namelist variable. `$GMS_PATH` should be set to the path where the script `runrms` is located (for example `/opt/gamess/`). We assume that the `runrms` script copies the output `.dat` files to the directory from which GAMESS is invoked. If this is not the case, please modify the script `runrms` accordingly.

Limitations Only mechanical embedding is supported with GAMESS. The available QM models are limited to HF, DFT and MP2 since only for these analytical gradients are available in GAMESS.

&gms Namelist variables

basis Basis set type to be used in the calculation. Presently supported are the Pople type basis sets STO-3G, 6-31G, 6-31G*, 6-31G**, 6-31+G*, 6-31++G*, 6-311G, 6-311G* and 6-311G**. Also supported are the Karlsruhe valence triple zeta basis sets KTZV, KTZVP and KTZVPP (with none, one and two polarization functions, respectively) and the Dunning-type correlation consistent basis sets CCn (n = D, T, Q, 5, 6; officially called cc-pVnZ) and ACCn (as CCn but augmented with a set of diffuse function, officially called aug-cc-pVnZ). (Default: `basis = "6-31G"`)

11. QM/MM calculations

method	QM method to be used. At present, we support 'HF' for Hartree-Fock, 'MP2' for second order Møller-Plesset perturbation theory and any of the supported DFT functionals. Popular choices for DFT functionals include BP86, BLYP, PBE, B3LYP or PBE0. (Default: method = "BP86")
nrad	Number of radial points in the Euler-MacLaurin quadrature of the XC potential and energy density. (Default: nrad = 96)
nleb	Number of angular points in the Lebedev grids for the numerical quadrature of the XC potential and energy density. (Default: nleb = 590 [The GAMESS default of 302 is not accurate enough to conserve energy])
scf_conv	SCF convergence threshold. Convergence is reached when the absolute density change between two consecutive SCF cycles is less than scf_conv}. (Default: scf_conv = 1.0D-06)
maxit	Maximum number of SCF iterations. (Default: maxit = 50)
gms_version	This is the version number specified when building GAMESS. (Default: gms_version = 00)
num_threads	Number of threads (and thus CPU cores) for GAMESS to use. Note that GAMESS may require a special setup in the run_gms script to be able to run using multiple threads. Unless num_threads is explicitly specified, GAMESS will only use one thread (run on one core). (Default: num_threads = 1)
mwords	The maximum replicated memory which your job can use, on every node. This is given in units of 1,000,000 words (as opposed to 1024*1024 words), where a word is defined as 64 bits. You may need to increase this value if GAMESS crashes due to not having enough memory allocated. (Default: mwords = 50)
use_template	Determine whether or not to use a user-provided template file for running external programs. (Default: use_template = 0)
ntpr	Controls frequency of printing for dipole moment and atomic charges to files gms_prop.ext (Defaults to &cntrl namelist variable ntp)
chelpg	CHELPG charges are calculated if chelpg = 1. These charges are written to the file gms_prop.chg (Default: chelpg = 0)
dipole	Toggles writing of dipole moment to file gms_prop.dip (Default: dipole = 0)

Example An input file for QM or (mechanical embedding) QM/MM MD with GAMESS using the PBE functional and the 6-31G** basis set that should run GAMESS on 16 CPU cores therefore would have to contain

```
&gms
  method = 'DFT',
  dfttyp = 'PBE',
  basis   = '6-31G**',
  num_threads = 16,
/
```

Template input file The template file for GAMESS should be named gms_job.tpl and the \$CONTROL card must contain the following keywords:

```
RUNTYP=GRADIENT
UNIT=ANGS
COORD=UNIQUE
```

You should not include the \$DATA card in the template file as it is taken care of by *sander*.

11.2.6.3. AMBER/Gaussian

To use Gaussian with the interface, Gaussian 16, Gaussian 09, or Gaussian 03 must be properly installed on the system and a g16, g09, or g03 executable must be in the path.

Limitations A cutoff is applied to QM/MM interactions in QM/MM simulations using electrostatic embedding with and without PBCs. This leads to discontinuities in the potential energy surface and poor energy conservation. In the case of QM/MM simulations without PBCs, this cutoff (*qmcut* variable in the *&qmmm* namelist) can be set to a number that is larger than the simulated system, thus effectively not applying a cutoff. This is recommended.

&gau Namelist variables

<code>basis</code>	Basis set type to be used in the calculation. Any basis set that is natively supported by Gaussian can be used. Examples are the single zeta, split valence or triple zeta Pople type basis sets STO-3G, 3-21G, 6-31G and 6-311G. The split-valence or triple zeta basis sets can be augmented with diffuse functions on heavy atoms or additionally hydrogen by adding one or two plus signs, respectively, as in 6-31++G. Polarization functions on heavy atoms or additionally hydrogens are used by adding one or two stars, respectively, as in 6-31G**. (Default: <code>basis = "6-31G**"</code>)
<code>method</code>	Method to be used in the calculation. Can either be one of the WFT models for which Gaussian supports gradients, for example RHF or MP2, or some supported DFT functional. Popular choices are BLYP, PBE and B3LYP. (Default: <code>method = "BLYP"</code>)
<code>scf_conv</code>	Threshold upon which to stop the SCF procedure. The tested error is the commutator of the Fock matrix and the density matrix. Convergence is considered to be achieved if the maximum element of the commutator (which is zero for an optimized wave function) is smaller than <code>scf_conv</code> . Set in the form of 10^{-N} . (Default: <code>scf_conv = 8</code>)
<code>num_threads</code>	Number of threads (and thus CPU cores) for Gaussian to use. Unless <code>num_threads</code> is explicitly specified, Gaussian will only use one thread (run on one core). (Default: <code>num_threads = 1</code>)
<code>executable</code>	Optional name of the Gaussian executable. (Default: If a string for this namelist variable is not specified then g16, g09, and g03 are tried in that order producing a fatal error if none are found. Note that if a string is specified then it is a fatal error if that executable is not found.)
<code>use_template</code>	Determine whether or not to use a user-provided template file for running external programs. (Default: <code>use_template = 0</code>)
<code>ntpr</code>	Controls frequency of printing for dipole moment to file <code>gau_job.dip</code> (Defaults to <code>&cntrl</code> namelist variable <code>ntpr</code>)
<code>dipole</code>	Toggles writing of dipole moment to file <code>gau_job.dip</code> (Default: <code>dipole = 0</code>)
<code>mem</code>	String that specifies how much memory Gaussian should be allowed to use. (Default: <code>'256MB'</code>)

Example An input file for QM or QM/MM MD with Gaussian using the BP86 functional and the 6-31G** basis set and running in parallel on 8 threads (using 1 GB of memory) therefore would have to contain

```
&gau
  method = 'BP86',
  basis   = '6-31G**',
  num_threads = 8,
  mem='1GB',
/
```

11. QM/MM calculations

Template input file The template file for Gaussian should be named `gau_job.tpl` and should only contain the route section of a Gaussian input file. The route section defines the method to be used and SCF convergence criteria. Charge and spin multiplicity are specified via the `&qmmm` namelist. For example for a B3LYP calculation with 6-31G* basis set, the route section would be:

```
#P B3LYP/6-31G* SCF=(Conver=8)
```

Do not include any information about coordinates or point charge treatment since this will all be handled by *sander*. Also, do not include any *Link 0 Commands* (line starting with %) since these are handled by *sander*. If you want to run Gaussian in parallel, specify the number of processors via the `num_threads` variable in the `&gau` namelist.

11.2.6.4. AMBER/Orca

To use Orca with the interface, Orca must be properly installed on the system, the Orca executables need to reside in a directory that is in the search path. For convenience of use, namelist parameters in general correspond to Orca keywords, see the Orca manual for details.

Limitations A cutoff is applied to QM/MM interactions in QM/MM simulations with and without PBCs. This leads to discontinuities in the potential energy surface and poor energy conservation. In the case of QM/MM simulations without PBCs, this cutoff (`qmcut` variable in the `qmmm` namelist) can be set to a number that is larger than the simulated system, thus effectively not applying a cutoff. This is recommended.

Also note that ORCA only supports OpenMPI for parallel calculations.

&orc Namelist variables

basis	Basis set type to be used in the calculation. Possible choices include <code>svp</code> , <code>6-31g</code> , etc. See Orca manual for a complete list. (Default: <code>basis = "SV(P)"</code>)
cbasis	Auxiliary basis set for correlation fitting. See Orca manual for a complete list. (Default: <code>basis = "NONE"</code>)
jbasis	Auxiliary basis set for Coulomb fitting. See Orca manual for a complete list. (Default: <code>basis = "NONE"</code>)
method	Method to be used in the calculation. Popular choices include <code>hf</code> , <code>pm3</code> , <code>blyp</code> , and <code>mp2</code> . (Default: <code>method = "blyp"</code>)
convkey	General SCF convergence setting for simplified Orca input. Can take values <code>'TIGHTSCF'</code> , <code>'VERYTIGHTSCF'</code> , etc. (Default: <code>convkey='VERYTIGHTSCF'</code>)
scfconv	SCF convergence threshold for the energy. (Default: <code>scfconv = -1</code> , that is, not in use since we use the general convergence settings keyword <code>convkey</code> . Otherwise this would lead to SCF energy convergence of 10^{-N} au, if set to <code>N</code> .)
grid	Grid type used during the SCF for the XC quadrature in DFT. (Default: <code>grid = 4</code> , this corresponds to <code>Intacc = 4.34</code> for the radial grid and an angular Lebedev grid with 302 points. Conservatively chosen together with <code>finalgrid</code> to conserve energy.)
finalgrid	Grid type used for the energy and gradient calculation after the SCF for the XC quadrature in DFT. (Default: <code>finalgrid = 6</code> , this corresponds to <code>Intacc = 5.34</code> for the radial grid and an angular Lebedev grid with 590 points. Conservatively chosen together with <code>grid</code> to conserve energy.)
maxiter	Maximum number of SCF iterations. (Default <code>maxiter = 100</code>)
maxcore	Global scratch memory (in MB) used by Orca. You may need to increase this when running larger jobs. See Orca manual for more information. (Default <code>maxcore = 1024</code>)

`num_threads` Number of threads (and thus CPU cores) for Orca to use. Note that Orca only supports OpenMPI. (Default: `num_threads = 1`)

`use_template` Determine whether or not to use a user-provided template file for running external programs. (Default: `use_template = 0`)

`ntpr` Controls frequency of printing for the dipole moment to file `orca-job.dip` (Defaults to `&cntrl` namelist variable `ntpr`)

`dipole` Toggles writing of the dipole moment to file `orca-job.dip` (Default: `dipole = 0`)

Example An input file for QM or QM/MM MD with Orca using the BLYP functional, the SVP basis set therefore would have to contain

```
&orc
  method = 'blyp',
  basis   = 'svp',
/
```

Template input file The template file for Orca should be named `orca-job.tpl` and must at least contain keywords specifying the method and basis set to be used in the calculation, for example:

```
# ORCA input file for BLYP/SVP simulation
! BLYP SVP
```

You should not include the following keywords in the template file as these are taken care of by sander (like setting the runtime and adding coordinates):

```
# NOT to be included in ORCA input file
!engrad
!energy # (or any run type)
%pointcharges
*xyzfile # (or any coordinates)
```

11.2.6.5. AMBER/Q-Chem

To use Q-Chem with the interface, Q-Chem must be properly installed on the system. The `q-chem` executable needs to reside in a directory that is in the search path. For convenience of use, namelist parameters in general correspond to Q-chem keywords, see the Q-Chem manual for details. The interface has been tested with Q-Chem versions 4.0.0.1 and 4.1.1 for HF, DFT and MP2. Other methods have not been tested and could cause problems - please be careful and verify that forces/energies used by sander are correct in this case.

Limitations A cutoff is applied to QM/MM interactions in QM/MM simulations with and without PBCs. This leads to discontinuities in the potential energy surface and poor energy conservation. In the case of QM/MM simulations without PBCs, this cutoff (`qmcut` variable in the `qmmm` namelist) can be set to a number that is larger than the simulated system, thus effectively not applying a cutoff. This is recommended.

&qc Namelist variables

`basis` Basis set type to be used in the calculation. Possible choices include '6-31g**', 'cc-pVDZ' etc. See the Q-chem manual for a complete list. (Default: `basis = '6-31G*' for DFT calculations and basis = 'cc-pVDZ' for MP2`

`auxbasis` Auxiliary basis set for density fitting / RI methods. See Q-Chem manual for a complete list. (Default: `basis = 'rimp2-cc-pVDZ' for RI-MP2 calculations, otherwise none`)

11. QM/MM calculations

<code>method</code>	Method to be used in the calculation. Popular choices include 'BLYP' or other density functionals, 'MP2' and 'RIMP2'. Alternatively, the keywords exchange and correlation can be employed. (Default: <code>method = 'BLYP'</code>)
<code>exchange</code>	Exchange method. Can be specified together with the correlation keyword in place of the combined method keyword. (Default: <code>exchange = ''</code>)
<code>correlation</code>	Correlation method. Can be specified together with the exchange keyword in place of the combined method keyword. (Default: <code>correlation = ''</code>)
<code>scf_conv</code>	SCF convergence threshold. (Default: <code>scfconv = 6</code>)
<code>num_mpi_procs</code>	Number of MPI processes for Q-Chem to use. The total number of CPUs to be used is <code>num_mpi_procs</code> times <code>num_threads</code> . (Default: <code>num_mpi_procs = 1</code>)
<code>num_threads</code>	Number of threads for Q-Chem to use for each MPI process. Really this is number of threads. The total number of CPUs to be used is <code>num_mpi_procs</code> times <code>num_threads</code> . (Default: <code>num_threads = 1</code>)
<code>use_template</code>	Determine whether or not to use a user-provided template file for running external programs. (Default: <code>use_template = 0</code>)
<code>ntpr</code>	Controls frequency of printing for the dipole moment to file <code>qc_job.dip</code> (Defaults to <code>&cntrl</code> namelist variable <code>ntpr</code>)
<code>dipole</code>	Toggles writing of the dipole moment to file <code>qc_job.dip</code> . This is currently not supported. (Default: <code>dipole = 0</code>)
<code>guess</code>	Toggles use of MOs from previous step as initial guess to accelerate SCF convergence. Any string different from 'read' will disable this. (Default: <code>guess = 'read'</code>)

Example An input file for QM or QM/MM MD with Q-Chem using MP2 with the cc-pVTZ basis set therefore would have to contain

```
&qc
  method = 'mp2',
  basis   = 'cc-pVTZ',
/
```

Template input file The template file for Q-chem must be named `qc_job.tpl` and must only contain keywords in the Q-Chem `$rem` input section that specify the QM method and basis set to be used in the calculation, for example:

```
EXCHANGE becke
CORRELATION lyp
BASIS 6-311G**
SCF_CONVERGENCE 7
```

The interface will take care of adding other keywords to the `$rem` section such as `JOBTYP` and writing the `$molecule` input file sections.

11.2.6.6. AMBER/MRCC

To use MRCC with the interface, the MRCC program suite must be properly installed on the system. The MRCC driver program `dmrcc` needs to reside in a directory that is in the search path. For convenience of use, namelist parameters in general correspond to MRCC keywords, see the MRCC manual for details. The interface has been tested with the MRCC release from July 15, 2016 for HF and DFT. Other methods have not been tested but should also work - please be careful and verify that forces/energies used by sander are correct in this case.

Limitations A cutoff is applied to QM/MM interactions in QM/MM simulations with and without PBCs. This leads to discontinuities in the potential energy surface and poor energy conservation. In the case of QM/MM simulations without PBCs, this cutoff (*qmcut* variable in the *qmmm* namelist) can be set to a number that is larger than the simulated system, thus effectively not applying a cutoff. This is recommended.

&mrcc Namelist variables

<code>basis</code>	Basis set type to be used in the calculation. Possible choices include '6-31g**', 'cc-pVDZ' etc. See the MRCC manual for a complete list. (Default: <code>basis = '6-31G*'</code>)
<code>calc</code>	Type of calculation, e.g. 'SCF', 'B3LYP', 'MP2', 'CCSD(T)', etc. (Default: <code>calc = 'SCF'</code>)
<code>dft</code>	Can be specified to request a DFT calculation and specify the DFT method. (Default: <code>dft = 'off'</code>)
<code>mem</code>	Memory that will be allocated for the calculation. (Default: <code>mem = '256MB'</code>)
<code>verbosity</code>	Controls the verbosity of the MRCC output file. (Default: <code>verbosity = 2</code>)
<code>ntpr</code>	Controls frequency of printing for the dipole moment to file <code>mrcc_job.dip</code> (Defaults to <code>&cntrl</code> namelist variable <code>ntpr</code>)
<code>do_dipole</code>	Toggles writing of the dipole moment to file <code>mrcc_job.dip</code> . (Default: <code>dipole = 0</code>)
<code>nprintlog</code>	Frequency of storing MRCC output files during a minimization of molecular dynamics run. (Default: keep only last output file, <code>nprintlog = 0</code>)
<code>debug</code>	Toggles debug mode, which prints subroutine calls and additional information about the AMBER/MRCC interface. (Default: no debugging, <code>debug = 0</code>)
<code>use_template</code>	Requests use of a template file to generate MRCC input files to utilize all the capabilities of that are not available through <code>&mrcc</code> namelist keywords. The template file is basically a truncated MINP file (the default input file for MRCC) which only includes the MRCC keywords. (Default: do not use a template input file, <code>use_template = 0</code>)

The following `&mrcc` namelist variables control multilayer calculations (i.e. QM/QM/MM or QM/QM/QM/MM embedding[382]; the region highlighted in bold is controlled by the keyword). Only single point calculations are currently possible with such multilayer calculations.

<code>embed</code>	Specifies the method of the embedding QM region (2. layer) in a QM/QM/MM (3 layer) calculation or specifies the method of the 3. layer in a QM/QM/QM/MM (4 layer) calculation. Please read the MRCC manual for available options. (Default: <code>embed = 'off'</code>)
<code>embedatoms</code>	Specifies the active atoms of the embedded QM region (1. layer) in a QM/QM/MM (3 layer) calculation or specifies the active atoms of the 1. and 2. layer in a QM/QM/QM/MM (4 layer) calculation. Comma separated list of integers (Default: <code>embedatoms = 0</code>)
<code>nmo_embed</code>	Specifies the number of active MOs of the embedded QM region (1. layer) in a QM/QM/MM (3 layer) calculation or specifies the number of active MOs of the 1. and 2. layer in a QM/QM/QM/MM (4 layer) calculation. = 0 The program automatically determines the MOs of the active region with the Boughton-Pulay (BP) algorithm. (default) > 0 Number of MOs that will be selected based on the Mulliken charges of the active atoms.
<code>corembed</code>	Specifies the low-level correlation method of the embedding QM region (2. layer) in a QM/QM/MM (3 layer) calculation or specifies the low-level correlation method of the 2. layer in a QM/QM/QM/MM (4 layer) calculation. Please read the MRCC manual for available options. (Default: <code>corembed = 'off'</code>)

11. QM/MM calculations

corembedatoms Specifies the active atoms of the embedded QM region (1. layer) in a **QM/QM/MM** (3 layer) calculation or specifies the active atoms of the 1. layer in a **QM/QM/QM/MM** (4 layer) calculation. Please note that the corembedatoms have to be a subset of the embedatoms if a 4 layer calculation is requested. Comma separated list of integers (Default: corembedatoms = 0)

nmo_corembed Specifies the number of active MOs of the embedded QM region (1. layer) in a **QM/QM/MM** (3 layer) calculation or specifies the number of active MOs of the 1. layer in a **QM/QM/QM/MM** (4 layer) calculation.

= 0 The program automatically determines the MOs of the active region with the Boughton-Pulay (BP) algorithm. (default)

> 0 Number of MOs that will be selected based on the Mulliken charges of the active atoms.

Examples An input file for QM or QM/MM MD with MRCC using DFT with the BLYP functional and the cc-pVTZ basis set therefore would have to contain

```
&mrcc
  calc = 'blyp',
  basis = 'cc-pVTZ',
/
```

An example input for a multilayer QM/QM/MM calculation with LCCSD(T) for a subset of QM atoms 7 to 12 embedded into the remainder of the QM region described by PBE (i.e. LCCSD(T)/PBE/MM) would be

```
&mrcc
  calc = 'LCCSD(T)',
  basis = 'cc-pVTZ',
  embed = 'PBE',
  embedatoms = 7,8,9,10,11,12
/
```

This assumes that atoms 7-12 are part of the QM region. A 4-layer QM/QM/QM/MM calculation with LCCSD(T) for atoms 7 to 12 embedded into LMP2 for atoms 13 to 16 and the remainder described by PBE (i.e. LCCSD(T)/LMP2/DFT/MM) would be requested with

```
&mrcc
  calc = 'LCCSD(T)',
  basis = 'cc-pVTZ',
  embed = 'PBE',
  embedatoms = 7,8,9,10,11,12,13,14,15,16,
  corembed = 'LMP2',
  corembedatoms = 7,8,9,10,11,12,
/
```

Template input file The template file for MRCC must be named `mrcc_job.tpl` and must only contain keywords that specify the QM method and basis set to be used in the calculation. Not to be included are following keywords: qmmm, qmreg, dens, pointcharges, geom, embed, corembed, scfiguess. The interface will take care of adding other keywords and writing the coordinate input file section.

11.2.6.7. AMBER/Fireball

To use Fireball with the QM/MM interface, a special version of *sander* must be compiled and linked against the Fireball library (libfireball.a). The Fireball library can be obtained from the fireball-qmd web site at <https://fireball-qmd.github.io>. Compilation requires the Intel compilers and Intel MKL library. You can compile a version of *sander* that supports Fireball as follows (bash assumed):

```
export FIREBALLHOME=/path/to/fireball.a
export MKL_HOME=/path/to/Intel/MKL/library
cd $AMBERHOME
./configure -fireball intel
make install
```

It is possible to compile the MPI parallel version of *sander* in the same fashion. However, only the MM part of the calculation will execute in parallel.

Limitations A cutoff is applied to QM/MM interactions in QM/MM simulations with and without PBCs. This leads to discontinuities in the potential energy surface and poor energy conservation. In the case of QM/MM simulations without PBCs, this cutoff (*qmcut* variable in the *&qmmm* namelist) can be set to a number that is larger than the simulated system, thus effectively not applying a cutoff.

Basis set Fireball requires a basis set, commonly provided in an “Fdata” directory. This directory contains all the interactions (different contributions to the electronic Hamiltonian matrix elements) for the different types of atoms (C, H, O, N, etc.) appearing in the QM region. In principle, the Fdata directory should be placed in the working directory. Alternatively, the path where the Fdata directory is located can be defined using the variable *basis* in the *&fb* namelist variables (see below).

This Fdata directory can be downloaded from the fireball-qmd web (<https://fireball-qmd.github.io>). Advanced users can also calculate their own Fdata using the *create* set of programs that can be found in the fireball-qmd github repository.

***&fb* Namelist variables**

<i>basis</i>	Path to the Fdata directory. (Default: <i>basis</i> = <i>./Fdata</i>)
<i>max_scf_iterations</i>	Maximum number of iterations in the loop for the calculation of the self-consistent charges. (Default: <i>max_scf_iterations</i> = 70)
<i>sigmatol</i>	Threshold for self-consistency in the electronic structure calculations. (Default: <i>sigmatol</i> = 1.0E-08)
<i>idftd3</i>	DFTD3 dispersion correction. (No correction: <i>idftd3</i> = 0; Dispersion correction for BLYP: <i>idftd3</i> = 1; Default: <i>idftd3</i> = 0)
<i>iwr charges</i>	Writes atomic charges in fireball output. (Default: <i>iwr charges</i> = 0)
<i>iwrteigen</i>	Writes energy levels in fireball output. (Default: <i>iwrteigen</i> = 0)

For a complete list of all *&fb* Namelist variables, please visit <http://nanosurf.fzu.cz/wiki/doku.php?id=fireball>

Example An input file for QM or QM/MM MD using AMBER/FIREBALL with all the default values would just have to contain an empty *&fb* namelist

```
&fb
/
```

11. QM/MM calculations

As another example, a simulation using DFTD3 dispersion corrections for BLYP that also writes out the atomic charges with Fdata in a central location of the user's home directory would need the following input:

```
&fb
  basis = '/home/fireball/Fdata',
  idftd3 = 1,
  iwrtcharges = 1
/
```

To launch the simulation, simply run *sander* as follows:

```
sander -O -i mdin -o mdout -p prmtop -c inpcrd -x mdcrd -r rstrt > amberfireball.out
```

11.3. QM/MM simulations with QUICK

The *sander* program has the capability to run QM/MM simulations with the quantum chemical code *QUICK* (QUantum Interaction Computational Kernel),^[356–362, 366] shipped along with AmberTools. If you use QM/MM simulations with *QUICK* in your work, please cite the following references:

- Manathunga, M.; O'Hearn, K. A.; Shajan, A.; Cruzeiro, V. W. D.; Giese, T. J.; Smith, J.; Miao, Y.; He, X.; Ayers, K.; Brothers, E.; Götz, A. W.; Merz, K. M. QUICK-24.03. University of California San Diego, CA and Michigan State University, East Lansing, MI, 2024
- Cruzeiro, V. W. D.; Manathunga, M.; Merz, K. M.; Götz, A. W.; Open-Source Multi-GPU-Accelerated QM/MM Simulations with AMBER and QUICK. *J. Chem. Inf. Model.* **61**, 2109–2115 (2021).

If you perform DFT calculations please also cite:

- Manathunga, M.; Miao, Y.; Mu, D.; Götz, A. W.; Merz, K. M. Parallel Implementation of Density Functional Theory Methods in the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.* **16**, 4315–4326 (2020). DOI: 10.1021/acs.jctc.0c00290

If you use the GPU accelerated version of QUICK please also cite:

- Manathunga, M.; Aktulga, H. M.; Götz, A. W.; Merz, K. M. Quantum Mechanics/Molecular Mechanics Simulations on NVIDIA and AMD Graphics Processing Units. *J. Chem. Inf. Model.* **63**, 711–717 (2023). DOI: 10.1021/acs.jcim.2c01505
- Manathunga, M.; Jin, C.; Cruzeiro, V. W. D.; Miao, Y.; Mu, D.; Arumugam, K.; Keipert, K.; Aktulga, H. M.; Merz, K. M., Jr.; Götz, A. W. Harnessing the Power of Multi-GPU Acceleration into the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.* **17**, 3955–3966 (2021). DOI: 10.1021/acs.jctc.1c00145
- Miao, Y.; Merz, K. M., Jr. Acceleration of High Angular Momentum Electron Repulsion Integrals and Integral Derivatives on Graphics Processing Units. *J. Chem. Theory Comput.* **11**, 1449–1462 (2015). DOI: 10.1021/ct500984t

The *QUICK* QM/MM features are available in two options: 1) Directly through the *sander* program as a linked library via an application programming interface (API) or 2) via a file-based interface (FBI) (see also section 11.2). As shown in reference ^[366], the API option provides faster simulations because it does not require repeated initialization of the QM program or I/O operations for data exchange. Therefore, we recommend to use the API interface. The *QUICK* QM/MM features are compiled by default and immediately available after installation of AmberTools. Please refer to section 10.2 for installation instructions. Additionally, a list of *QUICK* features and limitations has been presented in section 10.1.

Important note: Both the FBI and API interfaces of *QUICK* support QM/MM simulations with mechanical embedding and electrostatic embedding. At present the same limitations and caveats apply with respect to QM/MM cutoffs as for external QM codes (see section 11.2).

11.3.1. Usage

As discussed in chapter 10, *SANDER* can access different *QUICK* installation types for QM/MM simulations: serial, MPI parallel, CUDA, HIP, CUDA + MPI parallel and HIP + MPI parallel. If using the file-based interface (FBI), the *sander* executable is capable calling any of the different *QUICK* executables: *quick*, *quick.MPI*, *quick.cuda*, *quick.hip*, *quick.cuda.MPI* or *quick.hip.MPI*. If using the application programming interface (API), a different *SANDER* executable must be used for different *QUICK* types: the serial and MPI parallel versions of *QUICK* can be accessed from the *sander* and *sander.MPI* executables, respectively; furthermore, the single-GPU-accelerated and multi-GPU-accelerated versions of *QUICK* can be accessed from *sander.quick.cuda/sander.quick.hip* and *sander.quick.cuda.MPI/sander.quick.hip.MPI*; these executables are identical to *sander* and *sander.MPI* in all *SANDER* functionalities, except they perform QM/MM calculations with *QUICK* using the GPU-accelerated code through the API.

Examples for how to use both the API and FBI functionalities can be found at the test suites in the following locations within AMBER's source: *\$AMBERHOME/test/qmmm_Quick* and *\$AMBERHOME/test/qmmm_EXTERN/*Quick* for, respectively, API and FBI.

Important note: Before running any QM/MM simulations with *QUICK*, users must make sure to source *\$AMBERHOME/amber.sh* (or *\$AMBERHOME/amber.csh*, depending on your environment). This step ensures that the location of the necessary executables and libraries are set in the environmental variables of the operating system.

11.3.1.1. File-based interface (FBI)

Below is an example of the modifications necessary in the *SANDER* input file to perform a mechanical embedding QM/MM simulation. In this example, the first two residues of the system are assigned to the QM region, and the simulation is executed at the B3LYP/def2-SVP level with *quick.cuda.MPI* using 2 GPUs:

```
&cntrl
...
ifqnt = 1,
/
&qmmm
qmmask = ':1-2',
qm_theory = 'extern',
qmmm_int = 5,
/
&quick
method = 'B3LYP',
basis = 'def2-svp',
executable = 'quick.cuda.MPI',
do_parallel = 'mpirun -np 2',
/
```

where *ifqnt* set to 1 activates the QM/MM functionality, *qmmask* specifies the QM region, *qm_theory* set as 'extern' indicates that the FBI will be used, and *qmmm_int* set to 5 specifies the use of mechanical embedding. The executable flag can be set to any of the different *QUICK* executables, and the *do_parallel* flag must be specified only if using one of the MPI parallel versions of *QUICK*. It is important to emphasize that some machines may require a command other than *mpirun*, depending on the MPI library being used. In general the serial *sander* executable must be used because of limitations to the system calls from within MPI programs (i.e. you cannot use *sander.MPI* if you want to call an external MPI program). This means that the MM portion of the calculation will be executed in serial.

11.3.1.2. Application programming interface (API)

In the example below, we present the modifications necessary in the *SANDER* input file to perform a QM/MM simulation with electrostatic embedding. Unlike for the FBI case, simulations using serial, MPI parallel, single-GPU-accelerated, and multi-GPU-accelerated *QUICK* functionalities can all use the same input file.

```

&cntrl
...
  ifqnt = 1,
/
&qmmm
  qmmask = ':1-2',
  qm_theory = 'quick',
  qmmm_int = 1,
  qm_ewald = 0,
/
&quick
  method   = 'B3LYP',
  basis     = 'def2-svp',
/

```

where *ifqnt* set to 1 activates the QM/MM functionality, *qmmask* specifies the QM region, *qm_theory* set as 'quick' makes use of API, *qmmm_int* set to 1 specifies the use of electrostatic embedding and *qm_ewald* set to 0 indicates that the QM/MM interactions should be truncated at a given cutoff (the *cut* variable is specified in the &cntrl namelist). This is currently required in the same way as when external QM codes are used via the FBI. An approach for an Ewald based treatment of long-range QM/MM electrostatics with *ab initio* QM methods is currently under development and being tested in a development version of AmberTools.

Note: when the simulation is executed with the API, an output file called *quick.out* (the prefix name for this file can be modified; see below) is generated containing the *QUICK* output information for all MD steps.

11.3.1.3. &quick namelist variables

Below we show a list of all variables that can be specified in the &quick namelist. Please notice that some variables are specific to only the API or the FBI.

method = **String** Method to be used in the calculation, can be either 'HF' or some supported DFT functional. (Default: BLYP).

basis = **String** Basis set type to be used in the calculation. (Default: 6-31G).

executable = **String (FBI only)** *QUICK* executable to be used in the simulation with the FBI. Options are: *quick*, *quick.MPI*, *quick.cuda*, *quick.hip*, *quick.cuda.MPI* or *quick.hip.MPI* (Default: quick).

do_parallel = **String (FBI only)** Portion of the command to be placed right before the executable specification for activating the parallelization. Example: 'mpirun -np 2'. (Default: none).

scf_cyc = **Integer** Number of SCF cycles. (Default: 200).

reuse_dmx = **Integer (API only)** Reuse the density matrix from previous MD step.
 = **0** OFF.
 = **1** (Default) ON.

denserms = **Float (API only)** User defined density matrix maximum RMS for convergence. (Default: 1.0E-6).

intcutoff = **Float (API only)** User defined integral cutoff. (Default: 1.0E-8).

xccutoff = **Float (API only)** User defined threshold for grid pruning in exchange correlation algorithm. (Default: 1.0E-8).

basiscutoff = **Float (API only)** Cutoff for neglecting insignificant basis functions. (Default: 1.0E-6).

gradcutoff = **Float (API only)** User defined gradient cutoff. (Default: 1.0E-7).

`export=` **String (API only)** Export molecular orbitals and other QM data into a separate file, currently supports 'molden' only. (Default: none).

`keywords` = **String (API only)** Instead of specifying the *QUICK* input variables separately with the flags above, users can use this flag instead to specify the full keywords line that would go on the top of a *QUICK* input file. Example in a simulation with electrostatic embedding: 'B3LYP BASIS=cc-pVDZ CHARGE=0 MULT=1 GRADIENT EXTCHARGES'. (Default: none).

`outfprefix` = **String (API only)** Prefix to be used in the *QUICK* output file. The name chosen here will be followed by a '.out' suffix. (Default: quick).

`debug` Debugging information.

 = **0** (Default) No debugging information is printed.

 = **1** Debugging information is printed.

 = **2** Extra debugging information is printed if using the FBI.

`use_template` (**FBI only**) Use a template input file.

 = **0** (Default) No template file is used.

 = **1** Template file is used.

11.4. QM/MM simulations with TeraChem

TeraChem has a demo version freely available at <http://www.petachem.com>. In this version, each QM calculation can be ran up to 15 minutes using up to 2 GPUs. So QM/MM simulations (with both interfaces described below) will execute continuously as long as each MD step takes less than 15 minutes.

QM/MM simulations with TeraChem are available in two options: 1) through a file-based interface as described in section 11.2 or 2) using a interface based on TeraChem's client/server model. The TeraChem client is called TCPB-cpp (TeraChem Protocol Buffers, C++ version) and is shipped with AmberTools. TeraChem's client/server model users Google's Protocol Buffers for data communication, and can be used over the internet. **We recommend users to use the client/server interface (using TCPB-cpp)** since it is faster than the file-based interface because it saves time with GPU startup time and I/O operations. In order to use the recommended interface, Amber needs to be compiled with TCPB-cpp support, which is an optional feature. To install TCPB-cpp along with the Amber installation (see instructions at section 2.1), add `-DBUILD_TCPB=TRUE` into your `cmake` command at `amber24_src/build/run_cmake` and (re)compile AMBER.

If you are using the client/server interface with TeraChem (using TCPB-cpp), please cite the following work:

- V. W. D. Cruzeiro, Y. Wang, E. Pieri, E. G. Hohenstein, T. J. Martínez, *TCPB: Accessing TeraChem as an External Library for Faster QM or QM/MM Simulations*. Submitted.

If you are using the file-based interface with TeraChem, please cite the following work:

- C. M. Isborn, A. W. Götz, M. A. Clark, R. C. Walker, T. J. Martínez, *Electronic Absorption Spectra from MM and ab initio QM/MM Molecular Dynamics: Environmental Effects on the Absorption Spectrum of Photoactive Yellow Protein*, J. Chem. Theory Comput. **8**, 5092-5106 (2012), DOI: 10.1021/ct3006826

11.4.1. Usage

Examples for how to use both the client/server and file-based interfaces can be found, respectively, at the test suites in the following locations within AMBER's source: `$AMBERHOME/test/qmmm_TeraChem` and `$AMBERHOME/test/qmmm_EXTERN/*TeraChem`.

To start TeraChem in server mode, for example, using port number 12345, run:

```
terachem -s 12345
```

11. QM/MM calculations

The server can run on the same machine that will run AMBER, or on a remote machine. By default, TeraChem will use all GPUs in the machine, but users can control which GPUs are accessible by setting the `CUDA_VISIBLE_DEVICES` environmental variable before running the command above.

If using the file-based interface (not recommended), the `terachem` executable needs to be in the search path.

More information about the options in the TeraChem input file can be found at the TeraChem manual, which can be downloaded at <http://www.petachem.com>.

11.4.1.1. Client/server interface with TCPB-cpp

Make sure the TeraChem server is up. *sander* will print an error message in case the TeraChem server is not found. The number of GPUs to be used is controlled when the server is started, not in Amber.

In the example below, we present the modifications necessary in the SANDER input file to perform a QM/MM simulation with electrostatic embedding. This example assumes that the server is running on the local machine (i.e., localhost) and on port 12345:

```
&cntrl
...
  ifqnt = 1,
/
&qmmm
  qmmask = ':1-2',
  qm_theory = 'terachem',
  qmmm_int = 1,
  qm_ewald = 0,
/
&tc
  host      = 'localhost',
  port      = 12345,
  method    = 'B3LYP',
  basis     = 'def2-svp',
/
```

where *ifqnt* activates QM/MM, *qmmask* specifies the QM region (residues 1 and 2), *qm_theory* with a value of 'terachem' specifies that the TCPB interface will be used, *qmmm_int* as 1 requests electrostatic embedding, and *qm_ewald* as 0 means that a hard QM/MM cutoff will be employed, whose value can be controlled by the cut variable in the *&cntrl* namelist.

In the example above SANDER will create a TeraChem input file that will be passed to TCPB-cpp. Alternatively, users can directly provide the TeraChem input file as follows:

```
&cntrl
...
  ifqnt = 1,
/
&qmmm
  qmmask = ':1-2',
  qm_theory = 'terachem',
  qmmm_int = 1,
  qm_ewald = 0,
/
&tc
  host      = 'localhost',
  port      = 12345,
  tcfile    = 'terachem.inp',
/
```


11.4.1.2. File-based interface

In the example below, we present the modifications necessary in the SANDER input file to perform a QM/MM simulation with mechanical embedding. This example assumes that the server is running on the local machine (i.e., localhost) and on port 12345: Here we consider the first two residues as the QM region, explicitly specify a location for TeraChem's scratch folder, and use 2 GPUs in TeraChem:

```
&cntrl
...
  ifqnt = 1,
/
&qmmm
  qmmask = ':1-2',
  qm_theory = 'extern',
  qmmm_int = 5,
/
&tc
  method   = 'B3LYP',
  basis     = 'def2-svp',
  guess     = 'scr/c0',
  scrdir    = 'scr',
  keep_scr  = 'yes',
  ngpus     = 2,
/
```

where *ifqnt* activates QM/MM, *qmmask* specifies the QM region, *qm_theory* with a value of 'extern' specifies that the file-based interface will be used, and *qmmm_int* as 5 requests mechanical embedding.

11.4.1.3. &tc namelist variables

Below we show a list of all variables that can be specified in the &tc namelist. Please notice that some variables are specific to only the client/server interface (TCPB) or the file-based interface (FBI).

host	(TCPB only) Address to the machine where the TeraChem server is hosted. (Default: host = none)
port	(TCPB only) Port number used by the TeraChem server. (Default: port = none)
tcfile	(TCPB only) TeraChem input file to be passed to TCPB-cpp. (Default: tcfile = none)
method	Method to be used in the calculation. A few examples are 'RHF', 'BLYP', 'PBE' and 'B3LYP'. (Default: method = none in the client/server interface, and method = 'BLYP' in the file-based interface)
basis	Basis set type to be used in the calculation. A few examples are 'STO-3G', '3-21G', '6-31G' and '6-311G', '3-21++G' and '6-31++G' (Default: basis = '6-31G')
dftd	Determines whether dispersion corrections are applied in the case of DFT calculations. (Default: dftd = 'no')
precision	Precision model setting (single vs double precision). (Default: precision = 'mixed')
guess	(FBI only) Path to file with initial guess for the wavefunction. (Default: guess = 'scr/c0')
scrdir	(FBI only) Path to the scratch directory. (Default: scrdir = 'scr')
keep_scr	(FBI only) Keep only a single scratch directory. (Default: keep_scr = 'yes')
threall	Determines a variety of thresholds. (Default: threall = 1.0E-11)

11. QM/MM calculations

<code>convthre</code>	SCF convergence threshold for the wavefunction. (Default: <code>convthre = 3.0E-05</code> , which leads to SCF energy convergence of approximately 10^{-7} au or 10^{-4} kcal/mol)
<code>maxit</code>	Maximum number of SCF iterations. (Default: <code>maxit = 100</code>)
<code>cis</code>	Perform CIS calculation. (Default: <code>cis = 'no'</code>)
<code>cisnumstates</code>	Number of CIS states. (Default: <code>cisnumstates = 1</code>)
<code>cistarget</code>	Target CIS state. (Default: <code>cistarget = 1</code>)
<code>dftgrid</code>	DFT grid to be employed for the numerical XC quadrature in DFT calculations. (Default: <code>dftgrid = 1</code>)
<code>ngpus</code>	(FBI only) Determines how many GPUs are to be used. (Default: <code>ngpus = 0</code> , which uses all available GPUs)
<code>gpuids</code>	(FBI only) If <code>ngpus</code> has a value other than zero, this determines the IDs of the GPUs to be used for the calculation. (Default: <code>gpuids = 0, 1, 2</code> , etc.)
<code>executable</code>	(FBI only) Name of the TeraChem executable. (Default: <code>executable = terachem</code>)
<code>use_template</code>	(FBI only) Determine whether or not to use a user-provided template file for running external programs. (Default: <code>use_template = 0</code>)
<code>ntpr</code>	(FBI only) Controls frequency of printing for dipole moment and atomic charges to files <code>tc_job.ext</code> . (Defaults to <code>&cntrl</code> namelist variable <code>ntpr</code>)
<code>charge_analysis</code>	(FBI only) Toggles writing of atomic charges to file <code>tc_job.chg</code> (Options: <code>'none'</code> or <code>'Mulliken'</code> . Default: <code>dipole = 'none'</code>)
<code>dipole</code>	(FBI only) Toggles writing of dipole moment to file <code>tc_job.dip</code> (Default: <code>dipole = 0</code>)
<code>recycleinitguess</code>	Controls if we will try to use the wavefunction from previous MD step as initial guess. (Default: <code>true</code> , <code>recycleinitguess = 1</code>)
<code>debug</code>	Debugging information. = 0 (Default) No debugging information is printed. = 1 Debugging information is printed. = 2 Extra debugging information is printed if using the file-based interface.
<code>use_template</code>	(FBI only) Use a template input file. = 0 (Default) No template file is used. = 1 Template file is used.

11.4.1.4. If specifying your own TeraChem input file

If using the client/server interface, users should pass the TeraChem input file using the `tcfile` keyword in the `&tc` namelist as shown above, and should not specify the `method` keyword. **Note:** if you specify both the `tcfile` and `method` keywords, then the `tcfile` will be an output file written by *sander*.

If using the file-based interface, as for other programs, a template file containing the input options should be specified with the name `tc_job.tpl`.

The TeraChem input file should contain at least the following keywords:

```
basis
method
```

You should not include the following keywords in the TeraChem input file as these are taken care of by either TCPB-cpp or *sander* if using the file-based interface.

```
run
coordinates
pointcharges
pointcharges_self_interaction
gpus
```

Note: If using the file-based interface, You should also not specify charge and spinmult in the template file. Instead, specify these via the *&qmmm* namelist:

11.5. QM/MM simulations with xTB

The xTB software[383] implements the GFN1-xTB and GFN2-xTB semiempirical Hamiltonians.[384, 385] The *sander* QM/MM module includes an interface to the xTB library that fully supports QM/MM Ewald electrostatics within the self-consistent field procedure. The xTB software is not distributed with AmberTools; however, one can download the xTB source code from <https://github.com/grimme-lab/xtb>, compile the xTB library, and then configure Amber to link to the xTB library. An overview of the installation is provided below. When running QM/MM calculations with a QM method provided by xTB, one should first type “ulimit -s unlimited” on the command line to avoid seemingly random segmentation faults caused by the heavy use of stack memory in the xTB library.

1. Download xTB.

```
mkdir $HOME/software && cd $HOME/software
git clone https://github.com/grimme-lab/xtb.git
cd xtb
```

2. Configure xTB. Please refer to the installation instructions provided with the xTB software. Take special note, however, that one will want to compile a shared library and install the fortran modules by using the CMake configuration options `-DBUILD_SHARED_LIBS=TRUE` `-DINSTALL_MODULES=TRUE`. For example, to compile xTB with GCC compilers and the OpenBLAS LAPACK/BLAS libraries, one would type the following:

```
export XTBHOME=${HOME}/software/xtb/local
export FC=gfortran
export CC=gcc
cmake -B _build \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DBUILD_SHARED_LIBS=TRUE -DINSTALL_MODULES=TRUE \
  -DCMAKE_EXE_LINKER_FLAGS="-lm" \
  -DBLAS_LIBRARIES="-lopenblas" -DLAPACK_LIBRARIES="-lopenblas" \
  -DWITH_TBLITE=OFF \
  -DCMAKE_INSTALL_PREFIX=${XTBHOME}
```

3. Compile and install xTB.

```
make -C _build
make -C _build install
```

4. You should now have the following files:

11. QM/MM calculations

- a) `${XTBHOME}/lib64/libxtb.so`,
- b) `${XTBHOME}/include/xtb.h`,
- c) `${XTBHOME}/include/xtb_xtb_calculator.mod`.

If you are missing the shared library, then you may need to downgrade your CMake to version 3.26.0 (or increase it to 3.29.0, 3.30.0, or 3.31.0) because the xTB developers have disabled support for some versions due to a bug in CMake.

5. Change to the Amber build directory.

```
cd /path/to/amber/build
```

6. Edit `run_cmake` to define `USE_XTB`, `XTB_LIBRARY`, and `XTB_INCLUDE_DIR`, where `XTB_LIBRARY` is the full path to the `libxtb` shared library, and `XTB_INCLUDE_DIR` is the full path to the installed xTB header and module files.

```
-DUSE_XTB=ON \  
-DXTB_LIBRARY=${XTBHOME}/lib64/libxtb.so \  
-DXTB_INCLUDE_DIR="${XTBHOME}/include"
```

7. Configure and install Amber.

```
bash ./run_cmake && make install
```

An example input that uses the xTB software within a QM/MM simulation with Ewald electrostatics is shown.

```
&cntrl  
...  
  ifqnt = 1,  
/  
&qmmm  
  qmmask = ':1-2',  
  qmcharge = 0  
  qm_theory = 'XTB',  
  qm_ewald = 1,  
/  
&xtb  
  qm_level = 'GFN2-xTB',  
  tfermi = 300.0,  
  accuracy = 1.d-3  
  maxiter = 250  
  mmhardness = 0.0  
  debug = F  
/
```

11.5.1. &xtb namelist variables

Below we show a list of all variables that can be specified in the `&xtb` namelist. Note that SCF information will be printed to the `mdout` file if one sets `verbosity=1` in the `&qmmm` namelist.

- | | |
|-----------------------|---|
| <code>qm_level</code> | = String Method to be used in the calculation, can be either <code>'GFN1-xTB'</code> or <code>'GFN2-xTB'</code> . (Default: <code>GFN2-xTB</code>). |
| <code>tfermi</code> | = Float The electronic Fermi temperature, in K. (Default: 300.0). |
| <code>accuracy</code> | = Float Controls that SCF convergence criteria. The tolerance on the energy will be <code>1.d-6 * accuracy</code> . (Default: <code>1.d-3</code>). |

`maxiter` = **Integer** Maximum number of SCF iterations. (Default: 250).

`mmhardness` = **Float** The chemical hardness of the MM atoms used to tune the nearby QM/MM electrostatics in atomic units. (Default: 0.0). If this value is a non-negative number, then all MM atoms will be treated as having the same chemical hardness. Setting it to a large positive value will effectively treat the MM atoms as point charges. If the value is 0.0, then all MM atoms will be given the hardness of hydrogen; i.e., this is the same as setting `mmhardness` = 0.405771 au. If the value is negative, then the atomic hardness of each MM atom is chosen from its atomic number (dummy particles will be treated like hydrogens), and the atomic hardness' will be scaled by `abs(mmhardness)`. In other words, if you want each MM atom to be based on its atomic number and you don't want to scale the hardness values, then set `mmhardness=-1`.

`debug` = **Logical** Turn on debug printing.

11.6. QM/MM simulations with DFTB+

The DFTB+ software[386] implements the DFTB2 and DFTB3 semiempirical Hamiltonians. The sander QM/MM module includes an interface to the DFTB+ library that supports QM/MM Ewald electrostatics within the self-consistent field procedure using the QM Mulliken charges. The DFTB+ software is not distributed with AmberTools; however, one can download the DFTB+ source code from <https://github.com/dftbplus/dftbplus.git>, compile the DFTB+ library, and then configure Amber to link to the DFTB+ library. An overview of the installation is provided below. When running QM/MM calculations with a QM method provided by DFTB+, one should first type “`ulimit -s unlimited`” on the command line to avoid seemingly random segmentation faults caused by the heavy use of stack memory in the DFTB+ library.

1. Download DFTB+.

```
mkdir $HOME/software && cd $HOME/software
git clone https://github.com/dftbplus/dftbplus.git
cd dftbplus
```

2. Configure DFTB+. Please refer to the installation instructions provided with the DFTB+ software. Take special note, however, that one will want to compile a shared library and install the fortran modules by using the CMake configuration options `-DWITH_API=TRUE` `-DBUILD_SHARED_LIBS=TRUE`. The examples below compile DFTB+ with GCC compilers and the OpenBLAS LAPACK/BLAS libraries.

- a) If you want to link DFTB+ into sander, *but you do not want to link xTB into sander*, then configure DFTB+ with the following cmake options.

```
export DFTBPLUSHOME=${HOME}/software/dftbplus/local
export FC=gfortran
export CC=gcc
cmake -DCMAKE_INSTALL_PREFIX=${DFTBPLUSHOME} \
      -DBLAS_LIBRARIES="-lopenblas" \
      -DWITH_API=TRUE \
      -DBUILD_SHARED_LIBS=TRUE \
      -DWITH_SDFTD3=TRUE \
      -B.build .
```

- b) If you want to link both DFTB+ *and* xTB into sander, then first configure and install xTB (following the instructions in 11.5). After xTB is installed, configure DFTB+ with the options below. The option `-DHYBRID_CONFIG_METHODS="Find"` tells DFTB+ to locate the required xTB libraries (rather than compiling a second version of them). The `-DCMAKE_PREFIX_PATH="${XTBHOME}"` tells DFTB+ the directory to search for the xTB libraries.

11. QM/MM calculations

```
export DFTBPLUSHOME=${HOME}/software/dftbplus/local
export FC=gfortran
export CC=gcc
cmake -DCMAKE_INSTALL_PREFIX=${DFTBPLUSHOME} \
      -DHYBRID_CONFIG_METHODS="Find" \
      -DCMAKE_PREFIX_PATH="${XTBHOME}" \
      -DBLAS_LIBRARY="-lopenblas" \
      -DWITH_API=TRUE \
      -DBUILD_SHARED_LIBS=TRUE \
      -DWITH_SDFTD3=TRUE \
      -B_build .
```

On Mac OS, you may need to include the configuration flag `-DWITH_OMP=FALSE` to disable OpenMP parallelization. Note: The DFTB3-D3H5, DFTB3-D42B, or DFTB3-D43B Hamiltonians will only be available if DFTB+ is configured with `-DWITH_SDFTD3=TRUE`. If you want to link both DFTB+ and xTB into sander but forget to set `-DHYBRID_CONFIG_METHODS="Find"` when installing DFTB+, then both semiempirical packages will compile incompatible versions of the `s-dftd3` library, resulting in compiler errors when linking sander.

3. Compile and install DFTB+.

```
make -C _build
make -C _build install
```

4. You should now have the following files and directories:

- a) `${DFTBPLUSHOME}/lib64/libdftbplus.so`,
- b) `${DFTBPLUSHOME}/lib64/cmake/dftbplus/dftbplus-config.cmake`,
- c) `${DFTBPLUSHOME}/include/dftbplus/modfiles/`

On Mac OS, these may be named slightly differently; e.g., `${DFTBPLUSHOME}/lib/libdftbplus.dylib`.

5. Change to the Amber build directory.

```
cd /path/to/amber/build
```

6. Edit `run.cmake` to define `USE_DFTBPLUS` and `CMAKE_PREFIX_PATH`, where `CMAKE_PREFIX_PATH` includes the full path to the DFTB+ installation. Note that `CMAKE_PREFIX_PATH` can list multiple paths, if necessary, but each path is delimited with `;` rather than `:`.

```
-DUSE_DFTBPLUS=ON \
-DCMAKE_PREFIX_PATH="${DFTBPLUSHOME}"
```

7. Configure and install Amber.

```
bash ./run.cmake && make install
```

An example input that uses the DFTB+ software within a QM/MM simulation with Ewald electrostatics is shown.

```
&cntrl
...
ifqnt = 1,
/
&qmmm
qmmask = ':1-2',
qmcharge = 0
```

```

qm_theory = 'DFTBPLUS',
qm_ewald = 1,
/
&dftbplus
qm_level = 'DFTB2',
tfermi = 0.0,
scftol = 1.d-7
maxiter = 250
hcorrection = -1
silent = T
debug = F
mixer = "BROYDEN"
/

```

11.6.1. &dftbplus namelist variables

Below we show a list of all variables that can be specified in the &dftbplus namelist. Note that SCF information will be printed to the mdout file if one sets verbosity=1 in the &qmmm namelist.

qm_level	= String Method to be used in the calculation, can be either 'DFTB2', 'DFTB3', or 'DFTB3-D3'. (Default: DFTB2). The 'DFTB3' method includes “full third order corrections”, and the DFTB3-D3 further includes Grimme’s D3 dispersion correction with Becke-Johnson damping. [387, 388] If the DFTB+ library was configured with the -DWITH_SDFTD3=TRUE cmake option, then DFTB+ will also support the Hamiltonians: 'DFTB3-D3H5', 'DFTB3-D42B', 'DFTB3-D43B'. The DFTB3-D3H5 model uses the D3H5 correction. [389] The D42B correction is the pairwise D4 dispersion model. [390] The D43B correction is the nonadditive D4 dispersion model. [390]
tfermi	= Float The electronic Fermi temperature, in K. (Default: 0.0).
scftol	= Float The SCF convergence criteria (Default: 1.d-7).
maxiter	= Integer Maximum number of SCF iterations. (Default: 250).
hcorrection	= Integer Turn on the hydrogen-bond correction (Default: -1). If the value is -1 and qm_level="DFTB3" or "DFTB3-D3", then the correction is turned on. If the value is -1 and qm_level="DFTB2", then correction is turned off. To manually turn the correction on, set the value to 1. To manually turn the correction off, set the value to 0.
debug	= Logical Turn on debug printing. (Default: F)
silent	= Logical If true, then do not write detailed output from the DFTB+ software to disk. (Default: T) This option is primarily used for debugging purposes. When activated, the following files will be written to the current working directory at every timestep: dftb.pin.hsd, charges.bin, and detailed.out. The dftb.pin.hsd file is the DFTB+ input file, whose contents are described in the DFTB+ User’s Guide. The other files are the atomic charges and output produced by DFTB+.
mixer	= String Selects the SCF iteration mixing algorithm; either “BROYDEN”, “DIIS”, or “SIMPLE”. (Default: BROYDEN)
d3a1	= Float The a1 parameter (unitless) in the Becke-Johnson dispersion damping function (Default: -1). If the value is less than 0, then the appropriate value, as listed in the DFTB+ User’s Manual, will be chosen based on the method specified with qm_level.
d3a2	= Float The a2 parameter (Bohr) in the Becke-Johnson dispersion damping function (Default: -1). If the value is less than 0, then the appropriate value, as listed in the DFTB+ User’s Manual, will be chosen based on the method specified with qm_level.

11. QM/MM calculations

d3s8 = **Float** The C8 dispersion scale factor (unitless) in the D3 dispersion model (Default: -1). If the value is less than 0, then the appropriate value, as listed in the DFTB+ User's Manual, will be chosen based on the method specified with `qm_level`.

11.7. QM/MM simulations with DPRc machine learning corrections

Sander includes an interface to the DeePMD-kit software[391], which implements the Deep Potential Range Corrected (DPRc) neural network correction.[392, 393] The DPRc correction is a nonelectrostatic contribution to the energy that is trained to improve the intra- and inter-atomic interactions of a QM/MM potential. The corrected potential is often referred to as a QM/MM+ Δ MLP model, and the neural network parameters must be parametrized for each underlying QM/MM base model. Furthermore, reparametrization of the DPRc correction is necessary when the training data does not adequately reflect the molecules and/or configurations in the current application. The training of transferable Δ MLP neural network parameters is a topic of active research; the current strategy is to train the neural network parameters for the system of interest to extend the sampling beyond what can be afforded from ab initio QM/MM simulation. Documentation for training the DPRc model can be found on the DeePMD-kit website <https://docs.deepmodeling.com/projects/deepmd/en/latest/model/dprc.html>. The DeePMD-kit software is not distributed with Amber; however, its source code and pre-compiled libraries can be downloaded from <https://github.com/deepmodeling/deepmd-kit>. Upon downloading the precompiled DeePMD-kit libraries, one can configure Amber to link to these libraries during compilation. An overview of installing Amber with DPRc support is provided below.

1. Download DeepMD-Kit.

```
mkdir $HOME/software && cd $HOME/software
wget https://github.com/deepmodeling/deepmd-kit/releases/latest/download/libdeepmd.c.tar.gz
tar xzf libdeepmd.c.tar.gz # this will unpack the directory ./libdeepmd_c
export DEEPMDDHOME=$HOME/software/libdeepmd_c
```

2. Verify that you have the following files.

- a) `${DEEPMDDHOME}/lib/libdeepmd.c.so`
- b) `${DEEPMDDHOME}/include/deepmd/c_api.h`
- c) `${DEEPMDDHOME}/include/deepmd/deepmd.hpp`

3. Change to the Amber build directory.

```
cd /path/to/amber/build
```

4. Edit `run_cmake` to define `USE_DEEPMDDKIT` and `CMAKE_PREFIX_PATH`, where `CMAKE_PREFIX_PATH` includes the full path to the DeePMD-kit installation. Note that `CMAKE_PREFIX_PATH` can list multiple paths, if necessary, but each path is delimited with `;` rather than `:`.

```
-DUSE_DEEPMDDKIT=ON \
-DCMAKE_PREFIX_PATH="${DEEPMDDHOME}"
```

5. Configure and install Amber.

```
bash ./run_cmake && make install
```

An example input that uses the AM1/d QM/MM+DPRc potential in a simulation is shown below.

```
&cntrl
...
  ifqnt = 1,
/
&qmmm
```



```

qmmask = ':1-2',
qmcharge = 0
qm_theory = 'AM1/d'
/
&dprc
idprc = 1
mask = ':1-2'
rcut = 6.0
interfile(1) = 'model.pb'
interfile(2:4) = ''
intrafile(1:4) = ''
avg = F
/

```

11.7.1. &dprc namelist variables

The following is a list of all variables that can be specified in the &dprc namelist.

- idprc** = **Integer** A value of 1 activates the DPRc correction. (Default: 0)
- mask** = **String** The Amber selection of those atoms whose energies and forces should be corrected with the DPRc potential. One would normally set this to be the same as qmmask in the &qmmm namelist. (Default: ")
- rcut** = **Float** A cutoff used to dynamically select nearby atoms for correction. The interactions between the selected and dynamically selected atoms will be corrected with the neural network model; however, the interactions between the dynamically selected atoms with other dynamically selected atoms are not corrected. For example, if the "mask" selects the QM atoms and rcut=6, then the QM/QM interactions and the QM/MM interactions within 6 Angstroms of the QM region will be corrected. Alternatively, if rcut=0.0, then only the QM/QM interactions would be corrected. The value of rcut should be consistent with the value used to define the QM/MM descriptor within the neural network model file. (Default: 0.0)
- interfile** = **List of String** A list of (up to 4) DeePMD-kit neural network parameter files. The networks defined within each file are expected to contain descriptors for both QM/QM and QM/MM interactions. When applying the neural network model to a simulation (inference), one would typically only include 1 neural network parameter file; i.e., interfile(1), while leaving all other filenames blank. During an active learning training procedure, one should provide 4 parameter sets. The dynamics will be performed using the correction defined in interfile(1), and after every ntwx steps the deviation between the 4 models will be compared. The model deviation will be written to the mdout file. The output will read "Active learning frame written with max. frc. std.: X.XXXX kcal/mol/A". The models are compared by calculating the force correction of each atom is computed with each of the 4 models. The standard deviation between the 4 models is calculated, and the maximum standard deviation is reported.
- intrafile** = **List of String** A list of (up to 4) DeePMD-kit neural network parameter files. The networks defined within each file are expected to contain descriptors for the QM/QM interactions only. This is analogous to using interfile and setting rcut=0.0. The intrafile option is useful when a neural network parameter set has been designed to reproduce ab initio gas phase energies and forces, and then a second (interfile) can be read to further correct the QM/QM and QM/MM interactions in a condensed phase environment. When applying the neural network model to a simulation (inference), one would typically only include 1 neural network parameter file; i.e., intrafile(1), while leaving all other filenames blank. During an active learning training procedure, one should provide 4 parameter sets. The dynamics will be performed using the correction defined in interfile(1), and after every ntwx steps the deviation between the 4 models will be compared. The model deviation will be written to the mdout file.

11. QM/MM calculations

avg = **Logical** If true and 4 interfile (or intrafile) parameter filenames have been provided, then the dynamics is performed on the average of the 4 corrections. The default is propagate the dynamics using the energy and forces of the first correction. (Default: F).

11.7.2. Neural network atom type descriptors

The DPRc potential uses neural networks to introduce an energy correction for each atom, such that the total energy correction is the sum of atom energy corrections. The neural network parameters further depend on the “atom type” of each particle. If the atom is selected by the “mask” argument (within the &dprc namelist), then the “atom type” is the 2 character element symbol of the atom; e.g., “Br” for bromine. If the atom is not selected by the mask option, but it is within rcut of any selected atom, then the “atom type” is the letter “m” followed by the 2 character element symbol; e.g., “mBr” for bromine. If the mask option is the same as the qmmask, then the QM atom types are simply the element symbols, and the MM atom types are similarly element-based with the “m” prefix. In this manner, the QM/QM correction of an element can be different from a QM/MM correction involving the same elements. A special exception is made for waters: the “MM atom types” (from the perspective of DeepMD-Kit) of water oxygen and water hydrogen is OW and HW, respectively.

11.7.3. Further information

The DeepMD-Kit website contains reference documentation <https://docs.deepmodeling.org/> and tutorials <https://tutorials.deepmodeling.com/en/latest/>. Model training is simplified using the DP-GEN <https://github.com/deepmodeling/dpgen> and the DP-Amber <https://github.com/njzjz/dpamber> utilities. Example mdin files used for simulation and active learning can be found at <https://gitlab.com/RutgersLSBR/AmberDPRc>. Amber-specific tutorials for training neural network parameters using these tools are actively being written.

11.8. Adaptive solvent QM/MM simulations

Traditional QM/MM approaches are based on a static QM/MM partitioning in which atoms belonging to the QM and MM regions are selected at the beginning of a molecular dynamics simulation. Such a static partitioning cannot be applied if part of the bulk solvent in the vicinity of a region of interest needs to be included in the QM region. Examples include cases in which the bulk solvent participates directly in a chemical reaction or in which important interactions between the solute and the bulk solvent, such as polarization and charge transfer, are not well parameterized at the QM/MM level and thus need to be described quantum mechanically. Due to molecular diffusion, solvent molecules will constantly exchange between the QM and MM regions and thus require a special treatment.

Several approaches have been developed that allow molecules to change their QM or MM character when crossing the boundaries between the QM and MM regions. A good overview and comparison of these approaches is available in the work by Buló *et al.*[394]. One of the most accurate approaches is the difference-based adaptive solvation (DAS) method[395], in the following simply called adaptive QM/MM (adQM/MM). This method is available in Amber through a parallelized implementation that has been developed by Andreas Goetz (SDSC) with help from Ross Walker (SDSC), Rosa Buló (Utrecht University) and Kyoyeon Park (UCSD). The usefulness of this adQM/MM approach for aqueous systems has been demonstrated with a development version of this implementation[396]. If you publish work that results from using this implementation, please cite the following work:

- A. W. Götz, K. Park, R. E. Buló, F. Paesani, R. C. Walker, *Efficient adaptive QM/MM implementation: Application to ion binding by peptides in solution*, in preparation.
- R. E. Buló, B. Ensing, J. Sikkema, L. Visscher, *Toward a practical method for adaptive QM/MM simulations*, J. Chem. Theory Comput. **9**, 2212-2221 (2009), DOI: 10.1021/ct900148e

In what follows we will describe the theoretical background of this implementation and how to perform adQM/MM simulations. For an alternative approach, see section 11.9.

11.8.1. Theoretical background

In adQM/MM simulations, we distinguish three different regions, an active region (A), a transition region (T), and the environment region (E). The active region contains both the part of the system that is permanently treated quantum mechanically (similar to the QM region in regular QM/MM simulations) and the solvent molecules in its vicinity that are also treated quantum mechanically. The E region is the part of the system that is treated at the MM level. Within the T region, molecules change their character from purely QM to purely MM, that is, molecules in the T region have partial QM and MM character, depending on their position within the T region. The T region that connects the A and E regions is required to guarantee that the potential energy surface or forces remain continuous throughout the simulation.

11.8.1.1. System partitioning

In the adQM/MM method[395], a partial MM character λ is assigned to each molecule in the T region. The value of λ depends on the distance of the molecule from the center of the A region according to

$$\lambda(r) = \begin{cases} 0 & r \leq R_A \\ \frac{(r-R_A)^2(3R_T-R_A-2r)}{(R_T-R_A)^3} & R_A < r < R_T \\ 1 & r \geq R_T \end{cases}, \quad (11.16)$$

where R_A and R_T are the inner and outer radii delimiting the T region. The switching function thus interpolates smoothly between QM (A region) and MM (E region).

The adQM/MM energy can be constructed as a weighted average of regular QM/MM energies due to all possible 2^{N_T} partitionings in which the N_T molecules in the T region are assigned either to the QM or the MM region,

$$E^{\text{adQM/MM}} = \sum_a \sigma_a E_a^{\text{QM/MM}}. \quad (11.17)$$

The statistical coefficients σ_a for the QM/MM partitionings are defined on basis of the λ values defined above,

$$\sigma_a = \begin{cases} 0 & \text{if } \max(\{\lambda\}_a^{\text{QM}}) > \min(\{\lambda\}_a^{\text{MM}}) \\ \min(\{\lambda\}_a^{\text{MM}}) - \max(\{\lambda\}_a^{\text{QM}}) & \text{if } \max(\{\lambda\}_a^{\text{QM}}) \leq \min(\{\lambda\}_a^{\text{MM}}) \end{cases}, \quad (11.18)$$

where $\{\lambda\}_a^{\text{QM}}$ and $\{\lambda\}_a^{\text{MM}}$ are the sets of λ values for a given QM/MM partitioning a that are assigned to the QM and MM regions, respectively. Due to this choice of coefficients, the weight σ_a of a QM/MM partitioning is zero if the partition contains one or more MM molecules closer to the A region than any of the QM molecules. The total number of nonzero QM/MM partitionings in an adQM/MM simulations is thus $N_T + 1$. In addition it is guaranteed that the weight of each partition varies smoothly from 0 to 1, removing discontinuities in the system dynamics that would appear in standard QM/MM simulations if a molecule would change its character by diffusing in or out of the QM region.

11.8.1.2. Force interpolation

The forces resulting from the adQM/MM energy 11.17 are a weighted sum of the force from each nonzero QM/MM partitioning and also contain a term that depends on the derivatives of the weight functions,

$$\mathbf{F}^{\text{adQM/MM}} = - \sum_a \left[\sigma_a \frac{\partial E_a^{\text{QM/MM}}}{\partial \mathbf{R}} + \frac{\partial \sigma_a}{\partial \mathbf{R}} E_a^{\text{QM/MM}} \right]. \quad (11.19)$$

This introduces an artificial dependence on the relative energies of the different QM/MM partitionings. Thus, in place of the energy interpolation scheme, a force interpolation is applied in which the forces are given as

$$\tilde{\mathbf{F}}^{\text{adQM/MM}} = - \sum_a \sigma_a \frac{\partial E_a^{\text{QM/MM}}}{\partial \mathbf{R}}. \quad (11.20)$$

11. QM/MM calculations

The force interpolation does not conserve the energy from equation 11.17 but it is possible to define a conserved quantity according to

$$\tilde{E}^{adQM/MM} = E^{adQM/MM} + W, \quad (11.21)$$

where the correction term W is defined through

$$\frac{\partial W}{\partial \mathbf{R}} = - \sum_a \frac{\partial \sigma_a}{\partial \mathbf{R}} E_a^{QM/MM}. \quad (11.22)$$

The quantity $\tilde{E}^{adQM/MM}$ is not a potential energy since it is only defined along the path taken by the system during the simulation. It is nevertheless useful to monitor this quantity to determine whether the simulation settings lead to numerical stability. The correction term W can be expressed as the path integral of its force vector from equation 11.22, which can be discretized. For step n of an MD simulation it is given as

$$W_n = \sum_i^n \sum_a E_a^{QM/MM}(i) \frac{\sigma_a(i+1) - \sigma_a(i-1)}{2}. \quad (11.23)$$

The Amber implementation uses exclusively the force interpolation scheme from equation 11.20 and optionally computes the correction term W from equation 11.23 to enable monitoring of the conserved quantity $\tilde{E}^{adQM/MM}$ from equation 11.21.

11.8.1.3. Alternative definitions of active, transition and environment regions

So far we have defined the boundaries between the A, T, and E regions with the distances R_A and R_T from the center of the active region. In this case both the A and the T regions have fixed volumes but the number of solvent molecules inside each region can vary during the simulation. Alternatively, we can fix the number of solvent molecules N_A and N_T within the A and T regions, respectively. In this case the volume of the A and T regions as well as the radii R_A and R_T will vary during the course of a simulation. The advantage of fixing the number of solvent molecules in the T region is that the number of QM/MM partitionings that needs to be considered also remains constant ($N_T + 1$). This is useful to optimize load balancing in a parallel adQM/MM implementation. The downside is that expression 11.23 does not strictly hold any more since the coefficients σ_a depend on the λ values which in turn depend on R_A and R_T . However, in practice, this is usually not an issue since the conserved quantity $\tilde{E}^{adQM/MM}$ needs monitoring only during simulation setup to choose settings that afford sufficient numerical stability. One thus can test simulation settings with fixed radii R_A and R_T and then switch to fixed molecule numbers N_A and N_T for production runs.

11.8.2. Running adQM/MM simulations with sander

Performing simulations with the adQM/MM approach described above requires the MPI parallelized *sander* executable `sander.MPI`. The implementation features a dual layer parallelization in which the calculations for all individual QM/MM partitionings are performed in parallel. Each of these QM/MM calculations can in turn be run in parallel. The parallelization across QM/MM partitionings is based on the *multisander* code infrastructure which effectively runs independent copies of *sander* for each QM/MM partitioning (similar to the replica exchange, path integral and thermodynamic integration implementations).

In order to run an adQM/MM simulation, the `mdin` input file needs to be set up similar to a regular QM/MM simulation. The QM region as defined in the `&qmmm` namelist defines the atoms that are in the permanent QM region. In addition, the `&qmmm` namelist variable `vsolv` needs to be set to 2 or 3 for fixed number of molecules in the A and T region or fixed size of A and T region, respectively. The following shows the minimum additions to the `mdin` input file that are required to perform an adQM/MM simulation as compared to a traditional QM/MM simulation with fixed QM and MM regions:

```
# mdin file - minimum additional content for adaptive solvent QM/MM
&qmmm
...
```

```

adjust_q = 0, ! required, charge cannot be redistributed
vsolv = 2,    ! switch on adQM/MM with fixed molecule numbers
              !                               in A and T region
/
&vsolv
  nearest_qm_solvent = 6, ! number of solvent molecules in A region
/
&adqmmm
  n_partition = 4, ! number of QM/MM partitionings
                ! = number of molecules in T region + 1
/

```

In this example, a fixed number of solvent molecules is contained in the A region (6) and in the T region (3, since the number of QM/MM partitionings is $N_T + 1$). Thus, the volume of the A and T regions changes during the simulation. Details of all namelist variables are collected below.

In addition, a groupfile for *multisander* is required. This groupfile should point all *sander* copies to the same mdin input file, inpcrd coordinate file and prmtop parameter and topology file:

```

# groupfile for adaptive solvent QM/MM run with n_partition = 4
-O -i mdin -c inpcrd -p prmtop
-O -i mdin -c inpcrd -p prmtop
-O -i mdin -c inpcrd -p prmtop
-O -i mdin -c inpcrd -p prmtop

```

If you explicitly specify output file names, make sure to give separate names to each group (for example mdout.000, mdout.001 etc), see also the *multisander* documentation. The *multisander* adQM/MM simulation can then be executed with

```
mpirun -np 4 sander.MPI -rem 0 -ng 4 -groupfile groupfile
```

In this example, 4 MPI processes will be launched for 4 process groups (*sander* copies). The individual QM/MM calculations for each partitioning would thus run in serial. To run the individual QM/MM calculations in parallel, the number of MPI processes must be a multiple of the number of process groups.

Adaptive solvent QM/MM simulations can be performed both with the semiempirical NDDO-type and DFTB methods that are native to *sander* or with QM methods that are available via the interface to external QM programs. In the latter case, each process group will launch only one instance of the external QM program and the parallelization of the QM part of the QM/MM calculations is determined by the settings for the external QM program.

11.8.2.1. Important notes for system preparation and adQM/MM simulations

At the time of writing (release of Amber 16) there is only a limited body of experience with adQM/MM simulations documented in the literature. Running adQM/MM simulations requires careful simulation setup, in particular regarding the size of the A and T regions. The A region needs to be sufficiently large to correctly describe the physics of the system of interest. The T region on the other hand needs to be sufficiently large to minimize force interpolation errors between the QM and MM regions. Since the cost of an adQM/MM simulation scales linearly with the number of molecules in the T region, a tradeoff between accuracy and cost often needs to be made. This in turn might lead to simulations that behave nicely for many time steps but eventually experience sudden, large (unphysical) forces on atoms at the T region boundaries. Similarly, whether it is more appropriate to define the center of the A region via an atom or the center of mass of the permanent QM region will affect the numerical stability of a simulation, depending on the particular system. Likewise for determining the distances of the solvent molecules via an atom or the center of mass of the solvent. In the case of water as solvent, problems can arise due to autoprotolysis which can lead to the formation of hydroxide and hydronium ions in the A region. Since the MM force field is not parameterized for hydroxide or hydronium ions, these will experience strong (unphysical) forces upon entering the T region. Careful monitoring of adQM/MM simulations and a bit of patience is thus advisable.

11. QM/MM calculations

It is a good idea to monitor the size of the A and T region and to check coordinates of atoms in the QM regions of all partitionings.

11.8.2.2. Namelist parameters for adaptive solvent QM/MM simulations

Adaptive solvent QM/MM simulations require setting the *vsolv* variable in the *&qmmm* namelist and setting variables in the *&vsolv* and *&adqmmm* namelists.

&vsolv namelist parameters The *&vsolv* namelist contains parameters that describe which solvent molecules are contained in the A region in addition to the permanent QM region that is defined in the *&qmmm* namelist. This namelist can be used without the *&adqmmm* namelist in a regular QM/MM simulation with *sander* if the variable *vsolv* in the *&qmmm* namelist is set to 1 instead of 2 or 3 (see 11.1.6). In this case there is no transition region and solvent molecules entering / leaving the QM region during the simulation would switch abruptly between QM and MM description. This is not recommended since it will result in large unphysical forces whenever such a switch occurs. However, this option is useful for post-processing of trajectories with single point QM/MM calculations in which the solvent molecules closest to the permanent QM region are treated quantum mechanically.

nearest_qm_solvent_resname Residue name of the solvent that can exchange between QM and MM region (Default: *nearest_qm_solvent_resname* = 'WAT')

nearest_qm_solvent Number of solvent molecules in the A region (Default: *nearest_qm_solvent* = 0)

nearest_qm_solvent_fq Frequency of updating of the A region. Should be set to 1 (every MD step) for adQM/MM simulations. (Default: *nearest_qm_solvent_fq* = 1)

nearest_qm_solvent_center_id Determines the atom(s) of the solvent molecules that is used to calculate the distance to the QM region.

= 0 Use the atom that is closest to the QM region. (default)

= -1 Use the center of mass.

> 0 Use this atom number within the solvent residue.

qm_center_atom_id Determines the atom of the permanent QM region that is used to calculate the distance to the solvent molecules.

= 0 Use the atom of the permanent QM region that is closest to a solvent molecule. Not supported for adQM/MM since the radii of the A and T region would remain undefined - a common point of reference is required for all solvent molecules. Useful only for post-processing of trajectories. (default)

= -1 Use the center of mass of the permanent QM region.

> 0 Use this atom number. Note that this is an absolute atom number - obviously, you should choose an atom that is in the permanent QM region.

verbosity Controls verbosity of *vsolv* output in the *mdout* file.

= 0 Standard verbosity. (default)

> 1 Increase verbosity.

&adqmmm namelist parameters If the *&qmmm* namelist variable *vsolv* is set to 2 or 3, an adQM/MM simulation with a fixed number of molecules in the A and T regions or fixed size of the A and T regions, respectively, is requested. Details of the adQM/MM simulation are set in the *&adqmmm* namelist as follows.

<code>n_partition</code>	Defines the number of QM/MM partitions to be used. For <code>vsolv=2</code> this also determines the number of solvent molecules in the transition region, which is <code>n_partition - 1</code> . For <code>vsolv=3</code> it has to be set to the largest number of QM/MM partitionings that will be encountered for the chosen values of <code>RA</code> and <code>RT</code> . (Default: <code>n_partition = 1</code>)
<code>RA</code>	Defines the radius R_A of the A region in Angstrom. Only relevant for <code>vsolv=3</code> . Needs to be changed from the default value and requires setting of <code>RT</code> . (Default: <code>RA = -1.0</code>)
<code>RT</code>	Defines the radius R_T of the T region in Angstrom. Only relevant for <code>vsolv=3</code> . Needs to be changed from the default value and requires setting of <code>RA</code> . (Default: <code>RT = -1.0</code>)
<code>calc_wbk</code>	Controls whether the book-keeping term W is calculated. = 0 Do not calculate W . (default) = 1 Calculate W via one-sided difference approximation (not recommended). = 2 Calculate W via central-difference approximation, see equation 11.23. Requires additional computations for (dis)appearing partitionings. (recommended if W is desired).
<code>verbosity</code>	Controls verbosity of adQM/MM output in the <code>mdout</code> file. = 0 Standard verbosity. (default) = 1 Increase verbosity - write distances of residues in T region from center of A region to file <code>adqmmm_res_distances.dat</code> and σ_a values to file <code>adqmmm_weights.dat</code> . These files get overwritten at each program start. = 2 Increase verbosity - write distances and σ_a values also to the <code>mdout</code> file. Also write λ values.
<code>print_qm_coords</code>	Controls whether coordinates of the QM atoms in each partitioning are written to file. = 0 Do not write coordinates. (default) = 1 Write QM coordinates for all QM/MM partitionings in xyz format to files <code>QM_coords.001</code> etc. Files are overwritten upon each program call.

11.9. Adaptive buffered force-mixing QM/MM

11.9.1. Introduction

In hybrid quantum mechanical – molecular mechanical (QM/MM) methods the reactive part of the system (i.e. where a significant change of the charge density distribution is expected) is described using a quantum mechanical model while the rest of the system is treated using molecular mechanics. Conventional (“energy-mixing”) QM/MM methods (convQM/MM) define a unique total energy function for the whole system that consists of three terms: the energy of the QM model applied to the atoms in the QM region, the energy MM model applied to atoms in the MM region and the interaction energy between the two regions:

$$E^{QM/MM}(QM+MM) = E^{QM}(QM) + E^{MM}(MM) + E^{QM \leftrightarrow MM}(QM+MM), \quad (11.24)$$

where the superscript represents the level of theory, while the region to which they are applied are indicated in parentheses. The coupling between the quantum region and the surrounding atoms ($E^{QM \leftrightarrow MM}(QM+MM)$) can be taken into account in several ways. For example, in the more sophisticated approaches, the effects of the MM charges are included in the quantum mechanical SCF calculation in the form of an externally applied field. Given a total energy, performing Hamiltonian or any other standard dynamics is straightforward. However, several uncontrolled errors could potentially be introduced by such schemes. Representing the environment by a set of point charges can over-polarise the QM region, and conversely the electrostatic effect of the ever-changing quantum mechanical charge density on atoms at the edge of the MM region is quite different from what is assumed when the MM force field parameters are determined. The delicate balance that exists between the various non-bonded MM terms is therefore no longer maintained across the QM-MM boundary. Furthermore, if adaptivity,

11. QM/MM calculations

i.e. transitions of atoms between the two regions, is allowed, a new problem appears: in general there can be chemical potential differences between the QM and MM regions for various species, and this results in a net flow between the regions, leading to unphysical density differences, structure and dynamics. Allowing adaptivity in this sense can be important when the active region itself is mobile (e.g. penetration, adhesion, crack propagation), or diffusional processes in the environment are relevant (e.g. water molecules, ions, residues enter and exit the QM region during the dynamics). To overcome these problems the adaptive buffered “force-mixing” QM/MM (abfQM/MM) method was introduced [397, 398]. The implementation of abfQM/MM was carried out by Letif Mones (University of Cambridge) and Gabor Csanyi (University of Cambridge) with help from many others (see the article below). When using this implementation in your work please cite the following papers:

- Noam Bernstein, Csilla Várnai, Iván Solt, Steven A. Winfield, Mike C. Payne, István Simon, Mónika Fuxreiter and Gábor Csányi, *QM/MM simulation of liquid water with an adaptive quantum region*, Phys. Chem. Chem. Phys., **14**, 646–656 (2012), DOI: 10.1039/c1cp22600b
- Csilla Várnai, Noam Bernstein, Letif Mones and Gábor Csányi, *Tests of an Adaptive QM/MM Calculation on Free Energy Profiles of Chemical Reactions in Solution*, J. Phys. Chem. B, **117**, 12202–12211 (2013), DOI: 10.1021/jp405974b
- Letif Mones, Andrew Jones, Andreas W. Götz, Teodoro Laino, Ross C. Walker, Ben Leimkuhler, Gábor Csányi and Noam Bernstein, *Implementation of the Adaptive Buffered Force QM/MM method into CP2K and Amber program packages*, in preparation.

11.9.2. Technical details of abfQM/MM

In the abfQM/MM method two independent force calculations are performed at each MD step. The first and more time consuming calculation is an extended conventional QM/MM calculation, which is used for calculating the forces of atoms treated quantum mechanically during the dynamics. We start with a *core* QM region, which comprises atoms that will always be treated quantum mechanically throughout the simulation. This region is enlarged (using a distance criterion, see below) to obtain the *dynamical* QM region which contains the atoms that follow QM forces. The dynamical QM region is surrounded by a buffer region whose size can be determined by simple force convergence tests [399, 400] and its construction in practice is based on geometrical considerations: atoms or molecules that are within a specified distance from the dynamical QM region are added to the buffer region. From this first calculation only the forces of the atoms in the dynamical QM region are kept and the rest (namely the forces on atoms in the buffer region) are discarded. The second calculation is used for obtaining good forces on MM atoms, especially important near the QM/MM boundary. For this, either fully MM representation of the whole system is used or, alternatively, another QM/MM force calculation, but this time using a smaller (*reduced*) QM region consisting of only the atoms in the *core* QM region. The abfQM/MM method is an abrupt force mixing method, which means that the forces are not derived from a total energy expression but a simple combination of the forces of the two calculations described above

$$\mathbf{F}_i^{\text{abfQM/MM}}(\text{QM+MM}) = \begin{cases} \mathbf{F}_i^{\text{Extended}} & \text{if } i \text{ is in the dynamical QM region,} \\ \mathbf{F}_i^{\text{Reduced}} & \text{otherwise,} \end{cases} \quad (11.25)$$

where the superscripts Extended and Reduced denote that the forces are taken from the first and second calculations described above, respectively. The selection of the QM and buffer atoms is controlled by distance criteria. Using a single distance criterion measured from some key atoms in the QM region, however, would lead to rapid fluctuation in the region definitions because atoms may cross and re-cross repeatedly. To reduce this effect, a hysteretic algorithm can be applied using an inner (r_{in}) and an outer (r_{out}) radius [397]. Thus, an MM atom is redesignated to be QM if its distance measured from the QM region (as defined by a set of atoms *always* treated quantum mechanically) is less than r_{in} and a QM atom is redesignated to be MM if this distance is larger than r_{out} . Similar hysteretic algorithms are applied for the definition of the dynamical QM region as well as the buffer region.

The above definitions may lead to QM atoms that have covalent chemical bonds with MM atoms. This is not necessarily a problem, as these bonded interactions can be treated in several ways from the point of view of carrying

out the the QM/MM calculation (e.g. link atoms, special pseudopotentials, frozen localized orbitals etc.). However, none of these schemes are general, i.e. cutting some type of QM-MM bonds in this way might not yield reasonable forces. For example, highly polarized bonds, bonds with bond order larger than 1 and delocalized bonds such as those in aromatic rings should be protected from being cut. In the conventional, nonadaptive QM/MM scheme it is easy to handle this problem, because the QM region is specified at the beginning of the simulation and the user can pick a chemically sensible set of atoms. For our dynamically varying QM (and buffer) regions, chemically sensible decisions have to be made algorithmically. Our implementation allows the user to specify a list of the breakable types of bonds which the software then uses to build the regions automatically.

Finally, as in all force mixing schemes, the abfQM/MM scheme uses dynamical forces that are not conservative, that is they are not the derivatives of a total energy function. This is the price we pay for adaptivity. The nonconservative nature of the dynamics necessitates the use of a thermostat to maintain the correct kinetic temperature throughout the system. The strength of the thermostat we need to use in practice is similar to those that are conventionally used in biomolecular simulations, which suggests that no ill effects will arise purely from the use of a thermostat – the only caveat is that since the use of a thermostat is mandatory, strictly microcanonical simulations cannot be performed. A simple Langevin thermostat is not appropriate in the presence of net heat generation (and would lead to a steady state temperature deviation of several tens of degrees near the QM/MM boundary), so a special *adaptive* thermostat (a combination of Langevin and Nose-Hoover thermostats) that is able to maintain the correct temperature even in the presence of intrinsic heating or cooling is used [401].

11.9.3. Relation to other adaptive QM/MM methods

It is worth noting that the current implementation of abfQM/MM supports the use of several other adaptive QM/MM methods. For example, setting **r_qm_in**, **r_qm_out**, **r_buffer_in** and **r_buffer_out** variables to 0 (for definitions see the next section) leads to the adaptive conventional QM/MM (adconvQM/MM) technique that can be considered also as the zero limit of the adaptive solvent QM/MM (adQM/MM) method [395] without a transition region (see also section 11.8). In this case the *extended* and *reduced* systems are identical and the dynamics is propagated by forces of a convQM/MM calculation whose QM region is adaptive. To save computational time for adconvQM/MM the program first performs the corresponding convQM/MM calculation and then a dummy full MM calculation whose forces are discarded. Another limit can be obtained when **r_buffer_in** and **r_buffer_out** variables are set to 0 (and all other radii are not). This method can be called unbuffered force mixing QM/MM (unbuffQM/MM). It has been observed that the applicability of both adconvQM/MM and unbuffQM/MM depends on several factors (system, QM method, size of *core* / *qm* regions etc.) and it is advised to perform a force convergence test [399, 400] before using them.

11.9.4. Technical glossary

11.9.4.1. Systems

- *extended* system: the first (QM/MM) calculation, which is used for calculating the forces on atoms in the dynamical QM region. To get converged forces on these atoms, a buffer region is added, leading to an extended QM region.
- *reduced* system: the second calculation, which is used for obtaining the MM forces. Either a full MM representation can be used or a QM region that is smaller than the dynamical QM region.

11.9.4.2. Atom types

There are basically four regions in the abfQM/MM method depending on their role during the dynamics: the *core*, the *qm*, the *buffer* and the *mm* regions. These sets are disjoint by definition. There are atoms which are permanent members of a given region and there are others that can change their identity by moving from one region to another. This section describes the different atom types and also gives their name and id used in the implementation. Please note the distinction between the labels “QM” and “qm” atoms: the former indicates the QM region used in the actual extended or reduced QM/MM calculations, while the latter is a label used to describe those atoms that, together with the *core* atoms, follow dynamics using quantum mechanical forces.

11. QM/MM calculations

- *core* atoms (*id* = 1-2): those atoms that constitute the QM region for the reduced system calculation. (The QM atoms in the extended calculation are the *core*, the *qm* and *buffer* atoms together.)
- *user specified core* atoms (*id* = 1, *tag* = CORE_USER): *core* atoms specified by the user. These atoms are permanent *core* atoms during the whole simulation.

- *core extension* atoms (*id* = 2, *tag* = CORE_EXT): *core* atoms selected by geometrical criteria around the user specified *core* atoms. These atoms belong temporarily to the *core* region.

$$atom_i \in \{\text{core extension atoms}\} \iff atom_i = f(r_{\text{core.in}}, r_{\text{core.out}}, \{\text{user specified core}\})$$

$$\{\text{core atoms}\} = \{\text{user specified core atoms}\} \cup \{\text{core extension atoms}\}$$

- *qm* atoms (*id* = 3-4): atoms, whose QM forces are used in the MD simulation similarly to *core* atoms but *qm* atoms are excluded from the QM region in the reduced calculation. Their forces are calculated in the extended QM/MM calculation.

- *user specified qm* atoms (*id* = 3, *tag* = QM_USER): *qm* atoms specified by the user. These atoms are *qm* atoms during the whole simulation or occasionally can become *core extension* atoms.
- *qm extension* atoms (*id* = 4, *tag* = QM_EXT): *qm* atoms selected by geometrical criteria around the *core* and *user specified qm* atoms. These atoms belong temporarily to the *qm* region.

$$atom_i \in \{\text{qm extension atoms}\} \iff atom_i = f(r_{\text{qm.in}}, r_{\text{qm.out}}, \{\text{user specified qm}\} \cup \{\text{core atoms}\})$$

$$\{\text{qm atoms}\} = \{\text{user specified qm atoms}\} \cup \{\text{qm extension atoms}\}$$

- *buffer* atoms (*id* = 5-6): these atoms are in the buffer region. Although they are treated as QM atoms in the extended calculation, forces on them from this calculation are discarded and they move with forces coming from the reduced calculation in which they are treated with MM.

- *user specified buffer* atoms (*id* = 5, *tag* = BUFFER_USER): *buffer* atoms specified by the user. These atoms are permanent *buffer* atoms during the whole simulation or occasionally can become *qm* or even *core extension* atoms.
- *buffer extension* atoms (*id* = 6, *tag* = BUFFER_EXT): *buffer* atoms selected by geometrical criteria around the *qm* and *core* atoms. These atoms belong temporarily to the *buffer* region.

$$atom_i \in \{\text{buffer extension atoms}\} \iff atom_i = f(r_{\text{buffer.in}}, r_{\text{buffer.out}}, \{\text{qm atoms}\} \cup \{\text{core atoms}\})$$

$$\{\text{buffer atoms}\} = \{\text{user specified buffer atoms}\} \cup \{\text{buffer extension atoms}\}$$

- *mm* atoms (*id* = 7, *tag* = MM): they are MM atoms in both the extended and reduced calculations. For the MD the forces are obtained from the reduced calculation.
- QM atom selections in the reduced and extended QM/MM calculations:

$$\{\text{QM atoms in the reduced system}\} = \{\text{core atoms}\}$$

$$\{\text{QM atoms in the extended system}\} = \{\text{core atoms}\} \cup \{\text{qm atoms}\} \cup \{\text{buffer atoms}\}$$

11.9.5. Namelist parameters for adaptive buffer-forced QM/MM simulations

The abfQM/MM implementation requires only two calculations for each MD step, which are performed sequentially (first the computationally more expensive extended then the reduced calculations are carried out). Consequently, unlike adaptive solvent QM/MM (adQM/MM, section 11.8) the subroutines of abfQM/MM are called directly from *sander* and no groupfile is needed. All abfQM/MM related variables should be specified in the *&qmmm* namelist. An example of an abfQM/MM dynamics is shown below:

```
# mdin file - example for adaptive buffered-force QM/MM dynamics
&cntrl
  ...
  ntt=6,      ! adaptive Langevin thermostat is used
  ...
  ifqnt=1,
/

&qmmm
  ...
  abfqmmm=1,      ! activate abf QM/MM
  r_core_in=3.0,   ! inner radius for extended core region
  r_core_out=3.5,  ! outer radius for extended core region
  r_qm_in=3.0,     ! inner radius for extended qm region
  r_qm_out=3.5,    ! outer radius for extended qm region
  r_buffer_in=4.0, ! inner radius for buffer region
  r_buffer_out=4.5, ! outer radius for buffer region
  coremask=':1',   ! core region mask
  qmmask=':112, 1129, 1824, 2395', ! qm region mask
  buffermask='',   ! buffer region mask
  corecharge=0,    ! core region charge
  qmcharge=0,      ! qm region charge
  buffercharge=0,  ! buffer region charge
/
```

11.9.5.1. Basic namelist parameters

- abfqmmm** 1 activates the adaptive buffered force-mixing method, default is 0 (no abf-QM/MM method is applied).
- coremask** *core* atom list specification (in *ambmask* format). Optional, by default (when it is missing or **coremask=' '**) it is an empty set and in this case the reduced system is the full MM representation. Note that at least one of the **coremask** or **qmmask** sets has to be specified.
- qmmask** *qm* atom list specification (in *ambmask* format). Optional, by default (when it is missing or **qmmask=' '**) it is an empty set and in this case only atoms in the core region will be treated as QM atoms during the dynamics. Note that at least one of the **coremask** or **qmmask** sets has to be specified.
- buffermask** *buffer* atom list specification (in *ambmask* format). Optional, by default (when it is missing or **buffermask=' '**) it is an empty set.
- corecharge** Total charge of core atom list defined in **coremask**, default is 0.
- qmcharge** Total charge of qm atom list defined in **qmmask**, default is 0.
- buffercharge** Total charge of buffer atom list defined in **buffermask**, default is 0.
- r_core_in** Inner radius for determining core extension region around user specified core atoms. Default is 0.
- r_core_out** Outer radius for determining core extension region around the user specified core atoms. Default is the value specified for **r_core_in**. If **r_core_out** < **r_core_in** then **r_core_out** = **r_core_in**.
- r_qm_in** Inner radius for determining qm extension region around the core and user specified qm atoms. Default is 0.
- r_qm_out** Outer radius for determining qm extension region around the core and user specified qm atoms. Default is the value specified for **r_qm_in**. If **r_qm_out** < **r_qm_in** then **r_qm_out** = **r_qm_in**.

11. QM/MM calculations

`r_buffer_in` Inner radius for determining buffer extension region around the qm and core atoms. Default is 0.

`r_core_out` Outer radius for determining buffer extension region around the qm and core atoms. Default is the value specified for `r_buffer_in`. If `r_buffer_out` < `r_buffer_in` then `r_buffer_out` = `r_buffer_in`.

11.9.5.2. Adaptive thermostats' namelist parameters

`ntt` Besides the original thermostats in sander, new adaptive ones are also introduced to be able to absorb the heat production due to the nonconservative force-mixing dynamics. The corresponding thermostat can be activated using the `ntt` command. In general, 5 activates the Nose-Hoover (chain)-Langevin, 6 the adaptive Langevin, 7 the adaptive Nose-Hoover chain and 8 the adaptive Nose-Hoover (chain)-Langevin thermostat. For adaptive QM/MM `ntt`=6 or 8 should be used.

`gamma_ln` Collision frequency in ps^{-1}

`nchain` Number of thermostats in each Nose-Hoover chain of thermostats (default is 1)

11.9.5.3. Miscellaneous namelist parameters

`selection_type` Type of selection of the different regions. Default is the atom-atom distance based selection (`selection_type` = 1). In this case a given atom is going to belong to an outer region if the distance between the atom in question and any atom in the inner region is less or equal than the corresponding criterion. Option 2 is the flexible sphere selection: for each inner region the radius of the region is calculated (as the largest distance between the centre of mass of the region and any atom belonging to that region), and the distance between the edge of the inner region and the atom in question will determine whether the atom belongs to the outer region or not. Option 3 is fixed sphere based selection: it is the same as option 2 except that only the edge of the innermost region is calculated based on its atoms and then all the other region's borders are calculated geometrically as concentric spheres. For option 2 and 3 the radii of spheres are calculated using the centre region, which is either defined by the user (`centermask`) or it is the `coremask` if specified, otherwise it is `qmmask`. Note that option 2 and 3 selects significantly larger number of atoms than option 1.

`initial_selection_type` Type of initial selection type. This command controls the initial selection if not an abfqmmm restart is performed (i.e. `read_idrst_file` is not specified). Default is 0, which is the middle sphere selection (i.e. the mean of the corresponding inner and outer radii). Option -1 uses the inner and option 1 applies the outer radius for the first selection.

`center_type` Type of calculation of center for `selection_type` = 2 and 3. Default is center of mass (option 1), while option 2 is geometric center.

`gamma_ln_qm` Collision frequency of actual *core* and *qm* atoms in ps^{-1} when adaptive massive Langevin thermostat is applied. Default value is the same as `gamma_ln` defined in &cntrl session.

`mom_cons_type` Type of force correction for momentum conservation. Default is 1 when the extra force is distributed among the corresponding atoms as equal accelerations. Option 2 applies equal forces on each atom. Options -1/-2 apply an acceleration/force proportional to the absolute value of the current acceleration/force of each atom. The region of atoms where the force correction is distributed is specified by `mom_cons_region`. Option 0 does not apply momentum conservation.

`mom_cons_region` Specifies the region where the force correction for momentum conservation is distributed. Default is 1 that distributes the correction among only current *core+qm* atoms, option 2 distributes it among current *core+qm+buffer* atoms and option 3 distributes the forces on all atoms. When `mom_cons_region` = 0 the distribution is applied only among *core* atoms.

`fix_atom_list` > 0 activates the fixed atom list method, default is 0. In fixed atom list mode the different regions are extended only by those solvent molecules that satisfy the given geometrical criteria

and no solute atoms will be selected besides the user specified ones in the **coremask**, **qmmask** and **buffermask**. Useful when only solvent exchange is expected.

solvent_atom_number Number of atoms in solvent molecule for fixed atom list mode (**fix_atom_list** > 0), default is 3. Defining this variable is important when the solvent is other than water and the solvent molecule contains more (or less) than 3 atoms.

centermask Centre region atom list specification. Optional, if not defined then it is equal to **coremask**. If **coremask** is neither specified then **centermask** equals to **qmmask**.

oxidation_number_list_file File name of oxidation numbers. Each line in the file must be either a comment (starting by '!' or '#') or a triplet: RES ATOM OXID, where RES can be 'all' (specification for all residues), 'atom' (specification for a given atom), residue name or residue index. If RES $\neq$ 'atom' then ATOM is the atom type name that can be specified completely (e.g. HE2) or partially using '*' (e.g. H* or HE*). If RES = 'atom' then ATOM is the atom index in the topology. OXID is the integer oxidation number. Since different specifications can refer to the same atom, there is a hierarchy of the assignment and the later step always overwrites the previous one: 1. RES = 'all' with partial atom type specification (in the order of $X^* \rightarrow XY^* \rightarrow XYZ^*$), 2. RES = 'all' with complete atom type specification (XYZ1), 3. specified residue type with partial atom type specification, 4. specified residue type with complete atom type specification, 5. residue index with partial atom type specification, 6. residue index with complete atom type specification, 7. atom index specification.

ext_coremask_subset Possible core extension atom set. If specified only those atoms will be chosen according to the corresponding geometrical criteria that can be also found in this list (in the case of fixed atom list method solvent residues having at least one atom in the set will be chosen). If not defined then by default it is the all atom list.

ext_qmmask_subset Possible qm extension atom set. If specified only those atoms will be chosen according to the corresponding geometrical criteria that can be also found in this list (in the case of fixed atom list method solvent residues having at least one atom in the set will be chosen). If not defined then by default it is the all atom list.

ext_buffermask_subset Possible buffer extension atom set. If specified only those atoms will be chosen according to the corresponding geometrical criteria that can be also found in this list (in the case of fixed atom list method solvent residues having at least one atom in the set will be chosen). If not defined then by default it is the all atom list.

cut_bond_list_file File name of breakable bonds for intelligent termination of different regions (core/qm/buffer). Each line in the file must be either a comment (starting by '!' or '#') or a triplet: ATOM1 ARROW ATOM2. ATOM1 and ATOM2 are both either atom types or atom indexes. ARROW specifies the direction of bond breaking: if it is '<=>' then the bond can be split from both directions, if it is '=>' or '<=' then the bond can be cut only from ATOM1 or ATOM2 directions, respectively.

max_bonds_per_atom Maximum number of ligands around any atom in the system. This controls the size of arrays for the intelligent termination. Default is 4 that is good for most biological systems. If there are atoms having more than 4 ligands then adjustment is required.

n_max_recursive Intelligent termination scheme is a recursive subroutine to get a fast and reliable performance. However, it may happen that according to the user specified breakable bonds a very large bond network will be chosen for a given region. To avoid it this variable can be used to control the maximum number of iterations: when the number of iteration reaches the value of **n_max_recursive** the program terminates. Default value is 10000.

11. QM/MM calculations

<code>min_heavy_mass</code>	To keep low the number of atoms in each extension region, by default the geometrical region selection algorithm measures the distances between only heavy atoms, and hydrogen atoms are assigned in a second step according to the heavy atoms they are bonded to. To extend the distance based selection for H atoms as well, decrease the value from its default 4.0 below the atomic mass of hydrogen (e.g. 0.0).
<code>pdb_file</code>	File name of a special abfQM/MM related pdb file generated during the dynamics. The first 8 columns have the standard pdb format ('ATOM', atom index, atom name, residue name, residue index, Cartesian coordinates of atom), 9th column is the oxidation number, 10th and 11th columns are the id number and tag according to abfQM/MM implementation, respectively, and the possible following columns include the atom indexes of MM atoms having direct bond to the given atom treated as QM atom in the extended calculation. Default name is <i>abfqmmm.pdb</i> .
<code>ntwpdb</code>	Frequency of printing out abfQM/MM information into pdb_file . Default value is 0 (no printing). Using ntwpdb < 0 allows the user to perform a selection test. In this case neither dynamical nor even point calculations are performed, the program terminates after printing the pdb file out.
<code>read_idrst_file</code>	Name of abfQM/MM atom id restart file used for restarting simulations. In the beginning of the simulation besides the user specified atoms those become also member of a given region that are within the outer radius. For a given region if the outer radius differs from the inner one, in the beginning of the dynamics the number of atoms will change until it reaches a dynamical equilibrium fluctuation. To avoid this natural transient period in a consecutive restart calculation one can use the read_idrst_file generated in the previous run telling the program the abfQM/MM atom id's of the restart configuration. Note that the safe use of read_idrst_file requires the same region specifications as in the previous run.
<code>write_idrst_file</code>	Name of abfQM/MM atom id restart file generated during the run. Default name of the file is <i>abfqmmm.idrst</i> .
<code>ntwidrst</code>	Frequency of printing the abfQM/MM atom id restart file out. Default is 0 (no printing).
<code>hot_spot</code>	1 activates the hot spot-like adaptive calculation [402] in which the forces of atoms in the buffer region are linear combinations of the forces obtained from the extended and reduced calculations using a smoothing function. Default is 0 (no hot spot-like calculation is performed).

11.10. SEBOMD: SemiEmpirical Born-Oppenheimer Molecular Dynamics

The sander program provides the ability to run SEBOMD (SemiEmpirical Born-Oppenheimer Molecular Dynamics) simulations. During a SEBOMD simulation, all atoms are considered as quantum atoms within the NDDO semiempirical approach (e.g., AM1, PM3, etc). Therefore, unlike QM/MM methods, there is no link atom, no frontier bond, no interaction between any QM and MM atoms (since there is no MM atom). Another consequence of SEBOMD simulations is that the computational time requested to compute energy and forces at each step of a molecular dynamics run can be (very) important. To allow for the computation of “large” systems (i.e., up to a couple of thousands of atoms), an optional linear scaling divide and conquer strategy is implemented[403, 404]. Periodic boundary conditions with long-range electrostatic interactions through Ewald summation can also be applied. A detailed explanation of the implementation can be found in ref [405]. If you publish work that results from using the SEBOMD in AMBER, please cite the following work:

- Antoine Marion, Hatice Gokcan, and Gerald Monard, *SemiEmpirical Born-Oppenheimer Molecular Dynamics (SEBOMD) Within the Amber Biomolecular Package*, J. Chem. Inf. Model., **59**, 206–214 (2019), DOI: 10.1021/acs.jcim.8b00605

The SEBOMD code implemented in sander is originated from the DivCon program developed in the Merz group while at Pennsylvania State University:

- Steve L. Dixon, Arjan van der Vaart, Valentin Gogonea, James J. Vincent, Edward N. Brothers, Lance M. Westerhoff and Kenneth M. Merz, Jr. *DivCon99*, The Pennsylvania State University, 1999.

Major contributors to the SEBOMD interface are as follows:

- Maintenance, code refactoring, debugging, testing by Gerald Monard
- Original roar interface by Gerald Monard and Arjan van der Vaart[406]
- Original sander port by Jennifer Thomas
- Ewald and Particle Mesh Ewald summation by Laurent Teixidor
- PIF and MAIS semiempirical correction implementation, peptidic corrections by Antoine Marion[407]
- Divide & Conquer parallel speed enhancement by Hatice Gokcan

11.10.1. Functionalities and limitations

The current SEBOMD implementation allows to run sander simulations with the following functionalities:

- molecular dynamics or energy minimization (*imin* = 0, 1, or 5)
- gas phase or periodic boundary conditions (as defined in the topology file), no support for Generalized Born solvent effect
- For PBC runs, different long range interactions handlers are possible: none, external Particle Mesh Ewald using MM point charges as defined in the topology file, or direct Mulliken Ewald summation.
- temperature regulation as implemented in sander (*ntt* flag)
- pressure regulation: only *barostat* = 2 is supported (Monte Carlo barostat)
- parallel implementation (sander.MPI): only the Divide & Conquer approach can be used (*method* > 0)
- available hamiltonians: MNDO, AM1, AM1/d-PhoT, RM1, PM3, PM3/PDDG
- available corrections to PM3 hamiltonians: MAIS and PIF
- as *d-orbitals* are not yet implemented in the SEBOMD code, only the following elements are implemented: H, C, N, O, P, S, F, Cl, Br, I (except for AM1/d-PhoT for which the P element is not yet available because it requires the *d-orbital* implementation)
- maximum number of atoms: 1000; maximum number of residues: 1000
Note: the SEBOMD code currently uses a static memory allocation as defined in *\$AMBERHOME/Amber-Tools/src/sebomd/sebomd.dim*. Users wishing to simulation bigger systems will have to modify the SEBOMD source code and recompile.

11.10.2. Sample SEBOMD input

To run a SEBOMD calculation, a specific namelist (&sebomd) must be used. It contains all the necessary information for the run. To inform sander that a SEBOMD simulation must be run, two steps are required: 1) switch the ifqnt keyword to 1 (as for a QM/MM calculation); 2) define the *qm_theory* keyword in the &qmmm namelist to 'SEBOMD'. Here is a sample mdin file for SEBOMD:

11. QM/MM calculations

```
! example input for SEBOMD simulation
&cntrl
  ...
  ifqnt = 1,           ! switch on QM calculation
/
&qmmm
  qm_theory = 'SEBOMD', ! use specific SEBOMD routines
/
&sebomd
  hamiltonian = 'AM1',  ! Use the AM1 semiempirical hamiltonian
  charge = 0,           ! total charge on the (full) system is 0
/
```

11.10.3. &sebomd namelist variables

charge	= Integer Net charge of the system (Default = 0). Note: SEBOMD only supports closed shell molecular systems.
method	Algorithm for the SCF computation. = 0 (Default) Standard closed-shell algorithm: the Fock matrix is diagonalized at each SCF iteration. (Note: all subsetting parameters are ignored, only one subsystem containing all the atoms will be generated). = 1 Use linear scaling divide & conquer SCF algorithm. Buffer regions must be specified (dbuff1 and dbuff2). Subsystems are built on an atom-based principle. = 2 Use linear scaling divide & conquer SCF algorithm. Buffer regions must be specified (dbuff1 and dbuff2). Subsystems are built on an residue-based principle (recommended option over method=1). = 3 Use linear scaling divide & conquer SCF algorithm. Buffer regions must be specified (dbuff1 and dbuff2). Subsystems are built on an heavy-atom-based principle: each heavy atom plus its hydrogens define one subsystem and there are as many subsystems as the number of non-hydrogen atoms.
ncore	= Integer When using divide and conquer method (method > 0): specify the number of subsystems used to build the core. (default: ncore = 1)
dbuff1	= Float When using divide and conquer method (method > 0): specify the extent of the first buffer region from the core in Å. (default: dbuff1 = 6.0)
dbuff2	= Float When using divide and conquer method (method > 0): specify the extent of the second buffer region from the core in Å. (default: dbuff2 = 0.0)
hamiltonian	Semiempirical hamiltonian to be used for energy and force calculations. All atoms within the molecular system will be treated at this level of theory. Available semiempirical hamiltonians: MNDO Request the use of MNDO semiempirical hamiltonian[335] AM1 Request the use of AM1 semiempirical hamiltonian[333] PM3 Request the use of PM3 semiempirical hamiltonian (default)[332] PM3PDDG Request the use of PM3/PDDG semiempirical hamiltonian[336] RM1 Request the use of RM1 semiempirical hamiltonian[334] AM1/d-PhoT Request the use of AM1/d-PhoT semiempirical hamiltonian[344] (Note: phosphorous (P) element is not yet implemented, therefore the AM1/d-PhoT hamiltonian is available only for H, C, N, O, S, F, Cl, Br and I elements)

modif

Modification/corrections to the semiempirical energy. Some semiempirical methods have been extended to improve results, mostly in the case of intermolecular interactions. For the moment only PM3 corrections to the energy are available. Possible values are:

none (default) no correction

PIF2 PM3 hamiltonian is modified for **intermolecular** core-core interactions according to the work of Bernal-Uruchurtu et al. and Harb et al. [339, 408–410]. This correction can be applied when using PM3 hamiltonian with a molecular system composed of one (or more) organic molecule(s) in interaction with explicit water molecules. Intermolecular water-water core-core interactions are computed using specific PM3-PIF parameters for aqueous solvent, while intermolecular organic-organic and organic-water intermolecular core-core interactions are computed using another specific set of PM3-PIF parameters. The intermolecular PM3-PIF (PIF2 version) parameters are available only for the following interactions:

	Water		Organic				
	Hw	Ow	H	C	N	O	Cl
Hw	✓	✓	✓	✓	✓	✓	✓
Ow	✓	✓	✓	✓	✓	✓	✓
H	✓	✓	✓	✓	✓	✓	✓
C	✓	✓	✓	∅	✓	✓	∅
N	✓	✓	✓	✓	✓	✓	∅
O	✓	✓	✓	✓	✓	✓	✓
Cl	✓	✓	✓	∅	∅	✓	∅

(✓: intermolecular interaction parameters between the two considered atom types are available; ∅: no intermolecular parameter available)

PIF3 PIF3 is an extension of the PIF2 parameters in which organic hydrogens are distinguished between “hydrophylic” hydrogens and “hydrophobic” hydrogens[407]. In the case of hydrophylic hydrogens, intermolecular interactions between the hydrogen atom and water molecules are computed using PIF2 parameters. In the case of hydrophobic hydrogens, intermolecular interactions between these hydrogen atoms and water molecules are computed using specific parameters. The distinction between hydrophobic and hydrophylic hydrogens is performed using the atom types as specified in the topology file. Hydrogen atom types which are considered as hydrophylic are: H, HO, HS, HW, hn, ho, hp, hs, hw, Ho, hO, hN, and hR. Other hydrogen atom types are considered as hydrophobic.

MAIS1 MAIS extension of the PM3 hamiltonian in which intramolecular **and** intermolecular core-core functions are replaced by specific MAIS functions. This option corresponds to the initial work of Bernal-Uruchurtu *et al.*[338]. Parameters are only available for liquid water (H and O elements).

MAIS2 Second version of the MAIS extension. Parameters are only available for H, O, and Cl elements[339].

longrange

Select the type of long range interaction when using periodic boundary conditions:

= 0 (Default) No long range interaction. Only the minimum image convention.

= 1 Perform PME (Particle Mesh Ewald) summation using constant atomic charges extracted from the topology file.

= 2 Perform an Ewald summation using Mulliken atomic charges extracted from the semiempirical wavefunction. Long-range Ewald Mulliken charge effects are incorporated in the Fock matrix of the system to polarize the wavefunction.

dpmx

SCF convergence criteria on the density matrix:

= 1e-7 (Default) SCF is considered as converged when density matrix elements between two consecutive SCF steps have not changed more than dpmx. The default value of 1e-7 ensures

11. QM/MM calculations

the conservation of the total energy during NVE simulations. Larger values will speed-up calculations by using less SCF steps but the total energy may not be conserved during molecular dynamics.

<code>fullscf</code>	Option to enable pseudo-diagonalization routines = 0 enable pseudo-diagonalization routine when possible. This can speed-up SCF calculations. (default) = 1 turn off pseudo-diagonalization. Full diagonalization of the Fock matrix is performed at each iteration of the SCF cycle.
<code>ipolyn</code>	Option to activate polynomial interpolation of the guess density matrix = 0 Use converged density matrix of the previous step as initial (guess) density matrix for the current step. Recommended option for minimization. = 1 Use polynomial interpolation of the density matrix elements from the last three steps as initial (guess) density matrix for the current step. Recommended option for molecular dynamics runs. (default)
<code>screen</code>	verbosity option for SEBOMD calculations = 0 minimum output. (default) = 1 output semiempirical energy details at each step = 2 output semiempirical energy details + the composition of all subsystems when using method > 0.
<code>lambda</code>	= Float (default 1.0) Enable the computation of a mixed energy value between SEBOMD and full MM computations. If $\lambda \neq 1.0$, in addition to a semiempirical calculation, the energy of the full system is evaluated at the MM level. Then energy and forces are mixed according to:

$$E_{pot} = \lambda E(SEBOMD) + (1 - \lambda)E(MM)$$

Since, sometimes, semiempirical potential energy surfaces are (very) different from MM surface, the use of the `lambda` keyword permits to equilibrate MD more easily. For example, from an equilibrated MM system, it is possible to run several SEBOMD simulations using different `lambda` values from 0.0 (full MM energy) to 1.0 (full QM energy) to obtain an equilibrated SEBOMD simulation.

<code>charge_out</code>	Filename used to save atomic charges. Default = 'sebomd.chg'
<code>ntwc</code>	Every <i>ntwc</i> steps, the (Mulliken) atomic charges will be written to the <i>charge_out</i> file. If <i>ntwc</i> = 0, no atomic charge file will be written. Default = 0. The format of the <i>charge_out</i> file is the following: every <i>ntwc</i> steps, the energy of the system is first written, then one line per atom is written, containing the x, y, z coordinates and the Mulliken atomic charge of the atom.
<code>peptcorr</code>	flag to apply force field corrections on peptidic bonds Some semiempirical methods do not correctly describe peptidic bond properties, leading to a pyramidal peptide bond nitrogen. An empirical force field correction can be applied to force the planarity of a peptide bond[411]. = 0 no peptidic correction. (default) = 1 apply peptidic correction (see Ludwig et al. for details[411])

peptk = **Float** The force constant of the peptidic correction (in kcal/mol).
 AM1 default value: peptk = 5.9864
 PM3 default value: peptk = 9.8526
 MNDO default value: peptk = 6.1737

11.11. ReaxFF/AMBER

AmberTools now distributes the ReaxFF reactive molecular dynamics program as a hybrid ReaxFF/AmberMD feature. ReaxFF is a widely used reactive molecular dynamics model, which has largely been applied to many different problems and is generally used stand-alone – i.e., the full system is modeled using ReaxFF. Currently the serial and OpenMP versions of AmberMD support the ReaxFF/AmberMD tool, which introduces ReaxFF capabilities to facilitate bond breaking and formation. This tool enables the study of local reactive events in large systems at a fraction of the computational costs of comparable QM/MM models. ReaxFF is implemented through interfaces to the open source PuReMD Reactive Molecular Dynamics (PuReMD) [412, 413] package. PuReMD contains significant performance improvements over the Fortran-based reference ReaxFF implementation, and the PuReMD implementation targets several shared-memory and distributed-memory architectures. If you use ReaxFF/AmberMD in your work, please cite the follow reference:

A.Rahnamoun, M.C.Kaymak, M.Manathunga, A.W.Götz, A.C.T.van Duin, K.M.Merz, Jr., and H.M.Aktulga, “ReaxFF/AMBER - a framework for hybrid reactive/non-reactive force field molecular dynamics simulations”, Journal of Chemical Theory and Computation 16 (12), 7645-7654, 2020

11.11.1. Background on ReaxFF

ReaxFF is a classical MM model in spirit, which explicitly models chemical reactions based on the bond-length/bond-order concept and dynamic distribution of charges. Like non-reactive MM models, ReaxFF consists of two sets of terms: the bonded and nonbonded terms (van der Waals and electrostatic interactions). However, ReaxFF allows bond formation and dissociation, and, hence, has significantly different bonded terms than classical potentials. To illustrate the philosophy of the method, we describe the determination of the bond energy using bond orders for carbon and hydrogen (other elements are similarly dependent based on their specific properties), while the structure and definition of the remaining terms can be found in the original description of ReaxFF [414]. A single atom type in ReaxFF defines each element, e.g., there are no sp , sp^2 , or sp^3 hybridized carbon atoms, but only one carbon atom type. The bond energy (E_{bond} , (11.26)) is described as a function of the sigma (BO_{ij}^{σ}), first π (BO_{ij}^{π}) and second π ($BO_{ij}^{\pi\pi}$) bond orders, as well as the corresponding D_e , p_{be1} and p_{be2} parameters. The different bond orders themselves (BO_{ij}^{σ} , BO_{ij}^{π} , $BO_{ij}^{\pi\pi}$) are calculated using the pairwise distance (r_{ij} between atoms i-j, the ideal bond distances (r_0^{σ} , r_0^{π} , $r_0^{\pi\pi}$) for atom types of i and j, and the force field specific parameters $p_{be[1-6]}$ as shown in (11.27). All three terms in (11.27) are considered for a bond between two carbon atoms, while only the first term is used for the σ bond that forms between a carbon atom and a hydrogen atom. However, a pairwise distance-based representation will yield small bond orders between 1-3 atoms causing a bond order overestimation between the relevant atoms. A bond order correction (BO_{ij} in (11.28)) is applied to minimize the long-range bond orders for such situations, where Δ'_i ((11.29)) is the deviation of the uncorrected bond order summation from the valence state of an atom (e.g. carbon and hydrogen have valences of four and one, respectively).

$$E_{\text{bond}} = -D_e^{\sigma} \cdot BO_{ij}^{\sigma} \cdot [p_{be1} \cdot (1 - (BO_{ij}^{\sigma})^{p_{be2}})] - D_e^{\pi} \cdot BO_{ij}^{\pi} - D_e^{\pi\pi} \cdot BO_{ij}^{\pi\pi} \quad (11.26)$$

$$BO'_{ij} = BO_{ij}^{\sigma} + BO_{ij}^{\pi} + BO_{ij}^{\pi\pi} = \exp \left[p_{bo1} \cdot \left(\frac{r_{ij}}{r_0^{\sigma}} \right)^{p_{bo2}} \right] + \exp \left[p_{bo3} \cdot \left(\frac{r_{ij}}{r_0^{\pi}} \right)^{p_{bo4}} \right] + \exp \left[p_{bo5} \cdot \left(\frac{r_{ij}}{r_0^{\pi\pi}} \right)^{p_{bo6}} \right] \quad (11.27)$$

$$BO_{ij} = BO'_{ij} \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_4(\Delta'_i, BO'_{ij}) \cdot f_5(\Delta'_j, BO'_{ij}) \quad (11.28)$$

11. QM/MM calculations

$$\Delta'_i = -Val_i + \sum_{j=1}^{neighbors[i]} BO'_{ij} \quad (11.29)$$

During an MD simulation, bond orders are evaluated at each time-step and are used to determine the atomic connectivity within a pre-defined distance cutoff (typically 5Å). A time step of 0.25 fs can be used for most simulations, while a smaller time-step is needed for higher temperature studies (>1500K). The energy curves are continuous throughout the simulation process, even at regions involving bond formation/breaking where favorable reactions can automatically occur without any restraints. This is ensured by the inclusion of other bond order related terms (see (11.30)). Bond angles (E_{val}) and torsions (E_{tor}) are evaluated using similar bond order considerations. In a bond order potential, atoms often do not achieve their optimal coordination numbers. Therefore, ReaxFF requires additional abstractions such as lone pair (E_{lp}), over/under-coordination correction (E_{over} and E_{under}), 3-body penalty (E_{pen}) for systems with two double bonds sharing an atom, Three-body conjugation term (E_{coa}), Correction for C2 (E_{C2}), Triple bond energy correction (E_{triple}) and 4-body conjugation (E_{conj}) terms. The potential is summarized below. The detailed expressions for each term can be found in the literature [414].

$$E_{system} = E_{bond} + E_{lp} + E_{over} + E_{under} + E_{val} + E_{pen} + E_{coa} + E_{C2} + E_{triple} + E_{tors} + E_{conj} + E_{H-bond} + E_{vdWaals} + E_{Coulomb} \quad (11.30)$$

To prevent energy jumps during bond formation/dissociation, there are nonbonded interactions between each atom pair (even for 1-2, 1-3 interactions) in ReaxFF. Electrostatic interactions are represented by a shielded Coulombic term and the van der Waals interaction uses a shielded Morse potential to prevent unrealistic values at very short distances. An important and computationally expensive precursor to the electrostatic interactions is the need to dynamically determine partial charges at every MD step, which is accomplished through coupling of ReaxFF with charge models (such as QEq [415] and EE [416]). Such charge models require the solution of sparse linear system of equations. The resulting formulation is complex, but highly flexible and transferable. These approaches have allowed ReaxFF to be broadly applicable to a wide range of challenging problems. Developed originally for hydrocarbons [414], the ReaxFF method has been extensively used to investigate complex systems in a wide range of applications including biological systems [417–421], materials [422–428], catalysts [429, 430], combustion, and batteries [431].

11.11.2. ReaxFF force fields

ReaxFF force field file format consists of the following sections: General, Atoms, Bonds, Off-diagonal, Angles, Torsions and Hydrogen bonds sections. The number of parameters in each section except for the General section are a function of the number of atom types fitted for a specific force field file. In the example force field file below, the number of parameters in a section are: 41, 32, 16, 6, 7, 7 and 4.

11.2 shows for which systems ReaxFF has currently been studied and parameterized. Note that this figure only indicates the elements that have been visited by ReaxFF – it does not describe the actual materials. For example, while a ReaxFF description is available for Fe/C/H/O interactions, currently parameters for the Fe/B system aren't available.

11.11.3. Implementation

Like QM/MM methods, atoms are split into 3 categories in the ReaxFF/AMBER method: i) ReaxFF atoms, which include all atoms in the chemically reactive region and are handled by a PuReMD implementation, ii) the ReaxFF/MM transition atoms, which include all atoms within a certain cutoff of the ReaxFF region and is handled by PuReMD and AMBER collaboratively, and iii) the MM atoms, which include all remaining atoms and is handled exclusively by AMBER. AMBER is the simulation driver in the ReaxFF/AMBER MD integration. After AMBER categorizes atoms into their respective groups, it sends all relevant information for ReaxFF and ReaxFF/MM atoms to the PuReMD program. After AMBER completes transferring the data exchange files, the PuReMD program then runs a zero-step non-periodic simulation to calculate the dynamic charges, energies and

Force field identifier

General

Parameter identifiers

Atom

Bond

Off-diagonal

Angle

Torsion

H-bond

Reactive MD-force field: Hydrocarbon parameters

39 ! Number of general parameters
50.0000 !Overcoordination parameter
9.8407 !Overcoordination parameter
21.2839 !Valency angle conjugation parameter
3.0000 !Triple bond stabilisation parameter
6.5000 !Triple bond stabilisation parameter
1.0000 !Not used
0.9782 !Undercoordination parameter
1.0250 !Triple bond stabilisation parameter
6.3452 !Undercoordination parameter
11.6274 !Undercoordination parameter
0.0000 !Triple bond stabilization energy
0.0000 !Lower Taper-radius
10.0000 !Upper Taper-radius
2.8793 !Not used
33.8667 !Valency undercoordination
88.6186 !Valency angle/lone pair parameter
1.0563 !Valency angle
2.0384 !Valency angle parameter
6.1431 !Not used
7.5203 !Double bond/angle parameter
0.3989 !Double bond/angle parameter: overcoord
3.9954 !Double bond/angle parameter: overcoord
-2.4837 !Not used
4.7120 !Torsion/BO parameter
10.0000 !Torsion overcoordination
2.3170 !Torsion overcoordination
-1.2635 !Conjugation 0 (not used)
2.1645 !Conjugation
1.4553 !vdWaaals shielding
0.1000 !Cutoff for bond order (*100)
2.8921 !Valency angle conjugation parameter
7.1783 !Overcoordination parameter
1.4473 !Overcoordination parameter
3.1353 !Valency/lone pair parameter
0.5000 !Not used
20.0000 !Not used
5.0000 !Molecular energy (not used)
0.0000 !Molecular energy (not used)
1.6052 !Valency angle conjugation parameter

2 cov.r; valency; a.m; Rvdw; Evdw; gammaEEM; cov.r2; #el
alfa; gammavdw; valency; Bunder; n.u.; chiEEM; etaEEM; n.u.
cov.r3;Elp; Heat inc.;l3BO1; l3BO2; l3BO3; n.u.; n.u.
ov/un; vail; n.u.; val3; vval4; n.u.; n.u.; n.u.
C 1.3826 4.0000 12.0000 2.0195 0.0763 0.8712 1.2360 4.0000
10.6359 1.9232 4.0000 40.5154 0.0000 5.7254 6.9235 0.0000
1.1663 0.0000 200.0498 6.1551 28.6991 12.1086 0.0000 0.0000
-14.1953 3.5288 0.0000 6.2998 2.9663 0.0000 0.0000 0.0000
H 0.6510 1.0000 1.0080 1.7693 0.0244 0.7625 -0.1000 1.0000
10.0482 5.2587 1.0000 0.0000 0.0000 3.8196 9.8832 1.0000
-0.1000 0.0000 65.0500 3.7647 2.7644 1.0000 0.0000 0.0000
-13.3669 3.6915 0.0000 6.2998 2.8793 0.0000 0.0000 0.0000
3 Edis1; Edis2; Edis3; pbel; pbo5; l3corr; pbo6; kov
pbe2; pbo3; pbo4; n.u.; pbo1; pbo2; ovcorr; n.u.
1 1 152.0140 104.0507 72.1693 0.2447 -0.7132 1.0000 23.5135 0.3545
0.1152 -0.2069 9.2317 1.0000 -0.1042 5.9159 1.0000
1 2 174.2967 0.0000 0.0000 -0.5193 0.0000 1.0000 6.0000 0.4401
18.9231 1.0000 0.0000 1.0000 -0.0099 8.2733 0.0000
2 2 177.8312 0.0000 0.0000 -0.3029 0.0000 1.0000 6.0000 0.6891
10.6518 1.0000 0.0000 1.0000 -0.0191 5.4288 0.0000 0.0000

1 Evdw; Rvdw; alfa; cov.r; cov.r2; cov.r3
1 2 0.0404 1.8583 10.3804 1.0376 -1.0000 -1.0000
3 Thetao; Ka; Kb; pconj; pv2; kpenal; pv3
1 1 1 70.2140 14.0458 2.0508 0.0000 0.0000 35.9933 1.0400
1 1 2 71.6289 18.4967 8.4619 0.0000 0.0000 0.0000 1.0400
2 1 2 72.7374 18.0638 2.9517 0.0000 0.2000 0.0000 1.0400
4 V1; V2; V3; V2(BO); vconj; n.u; n.u.
1 1 1 1 0.0000 28.8256 0.1796 -4.6957 -1.3130 0.0000 0.0000
1 1 1 2 0.0000 32.8083 0.4536 -4.6087 -1.7236 0.0000 0.0000
2 1 1 2 0.0000 36.7455 0.3087 -4.7435 -0.7311 0.0000 0.0000
0 1 2 0 0.0000 00.0000 0.1000 -4.7435 0.0000 0.0000 0.0000

1 Rhb; Dehb; vhb1; vhb2
1 2 1 2.0347 0.0000 4.9076 4.2357

Figure 11.1.: Example ReaxFF force field parameter file.

H																			He
Li	Be											B	C	N	O	F	Ne		
Na	Mg											Al	Si	P	S	Cl	Ar		
K	Ca	Sc	Ti	V	Cr	Mn	Fe	Co	Ni	Cu	Zn	Ga	Ge	As	Se	Br	Kr		
Rb	Sr	Y	Zr	Nb	Mo	Tc	Ru	Rh	Pd	Ag	Cd	In	Sn	Sb	Te	I	Xe		
Cs	Ba	★Lu	Hf	Ta	W	Re	Os	Ir	Pt	Au	Hg	Tl	Pb	Bi	Po	At	Rn		
Fr	Ra	★★Lr	Rf	Db	Sg	Bh	Hs	Mt	Ds	Rg									

Available

Not Yet Available

★ La, Ce, Pr-Yb

★★ Ac-No

Figure 11.2.: Periodic table elements currently described by ReaxFF [431].

forces on the ReaxFF and ReaxFF/MM atoms. Upon completion, it sends this information back to AMBER to complete the energy and force computations for the ReaxFF/MM and MM regions.

In implementing the interface API between the PuReMD and AMBER programs, we have adopted the following procedure for a successful hybrid model:

- Dynamic charges on ReaxFF atoms are calculated under the influence of ReaxFF/MM atoms with static charges
- All ReaxFF interactions between ReaxFF-ReaxFF pairs are calculated without any modifications
- Electrostatic interactions between ReaxFF (w/dynamic charge)-ReaxFF/MM (w/static charge) atom pairs are calculated by ReaxFF
- van der Waals interactions between ReaxFF-ReaxFF/MM atom pairs are handled by AMBER (e.g., using a Lennard Jones potential)
- Interactions between MM-ReaxFF/MM and MM-MM pairs are handled by AMBER as usual.

11.11.4. Installation

To build the serial version of ReaxFF/AMBER, the flag `-DBUILD_REAXFF_PUREMD=TRUE` must be set when using CMake, while the flag `-reaxff-puremd` must be set for building with the legacy configure / Make system. To build the OpenMP version of ReaxFF/AMBER, the flags `-DBUILD_REAXFF_PUREMD=TRUE` `-BUILD_OPENMP=TRUE` must be set when using CMake, while the flag `-reaxff-puremd-openmp` must be set for building with the legacy configure / Make system.

Currently, the functionalities of the serial and parallel CPU versions of PuReMD can be accessed directly from SANDER executable. ReaxFF/AMBER calculations are performed using the PuReMD library through the established API. For running a ReaxFF/AMBER simulation, in addition to a mandatory AMBER input files, a ReaxFF force field file is required in the simulation directory; by default, the ReaxFF force field file must be named `ffield.reaxff` – see `&reaxff` namelist section for the variable `ffield` to change the default filename.

Since ReaxFF/AMBER is built based on the QM/MM structure, the AMBER input file is similar to the required input file for a QM/MM calculation with external QM program. Below is an example of an equilibration calculation input file for using hybrid ReaxFF/AMBER.

```
imin = 0,
ntf = 2,
```

```

ntc = 2,
cut = 10.0,
ntb = 2,
nstlim = 15000,
dt = 0.00025,
temp0 = 300.0,
ntt = 3,
ntp = 1,
ifqnt = 1
/

&qmmm
  qmmask = ':1',
  qmcharge = 0,
  qm_theory = 'extern',
  qmshake = 0,
  qmcut = 10.0,
  qm_ewald = 0,
  qm_pme = 0
/

&reaxff
  solvtol = 1e-6,
  thbcut = 0.001

```

11.11.5. &reaxff namelist variables

Below we show a list of all variables that can be specified in the &reaxff namelist.

control = **String** Filename for the reaxff-puremd control file. (Default: empty String [disabled]).

ffield = **String** Filename for the ReaxFF parameter file. (Default: ffield.reaxff).

numthreads = **Integer** Number of threads used in ReaxFF/AMBER (OpenMP version only). (Default: 1).

charge_method = **Integer** Charge method used for calculations.

- = **0** Charge equilibration (QEq) method (NOTE: currently unsupported with QM/MM mode).
- = **1** (Default) Electronegativity equalization (EE) method.
- = **2** Atom-condensed Kohn-Sham approximated to second order (ACKS2) method.

nbrcut = **Real** Cut-off in Angstroms for bonded interactions within the QM region (Default: 5.0).

hbondcut = **Real** Cut-off in Angstroms for hydrogen bond interactions within the QM region (Default: 7.5).

thbcut = **Real** Numeric threshold used for cutting three-body interactions (Default: 0.001).

include_polarization_energy Determine inclusion/exclusion of polarization energy for QM region.

- = **0** Do not include polarization energy.
- = **1** (Default) Include polarization energy.

solvtol = **Real** Tolerance used for the iterative solver for atomic charges (Default: 1.0e-8).

solvmaxit = **Integer** Maximum iterations to perform for the iterative solver for atomic charges (Default: 200).

solvprecond Preconditioning method to use with the iterative solver for atomic charges.

11. QM/MM calculations

= 0 No preconditioning.

= 1 (Default) Jacobi.

`char_const_contig_start` = **List of Integers** Starting atom numbers for contiguous ranges of atoms specifying additional charge constraints.

`char_const_contig_end` = **List of Integers** Ending atom numbers for contiguous ranges of atoms specifying additional charge constraints.

`char_const_contig_value` = **List of Reals** Charge constraints in Coulombs for the contiguous ranges of atoms.

`char_const_custom_count` = **List of Integers** Number of values per each custom charge constraint.

`char_const_custom_atom_index` = **List of Integers** Atom indices used for values in custom charge constraints.

`char_const_custom_coeff` = **List of Reals** Coefficients applied multiplicatively on each atomic charge in custom charge constraints.

`char_const_custom_rhs` = **List of Reals** Net charge in Coulombs for each custom charge constraint.

IMPORTANT NOTE: When using contiguous or custom charge constraints, ensure that all QM atoms are constrained (i.e., encapsulated by at least one constraint). If this is not done properly, the charge solver will either find solutions corresponding to unrealistic charge values (e.g., very large charge values) or may fail to converge (due to the underpinning sparse linear system having incorrect properties). Also, note that the conventional constraint of the sum of charges being fixed is removed when using either contiguous or custom charge constraints; you will need to encode net charge appropriately in your input (if desired).

11.11.6. reaxff-puremd control file (advanced)

Users can also specify an optional control file for additional control over reaxff-puremd. This is useful as only a subset of parameters are exposed via the APIs for ReaxFF/AMBER. By default, the control variable in the `&reaxff` namelist is the empty string, so users must provide a filename for the control file if they wish to use this feature. Below is an example of a reaxff-puremd control file — consult the PuReMD documentation for a complete list of parameters and their explanations.

<code>ensemble_type</code>	<code>0</code>	<code>! denotes the type of the ensemble: 0 = NVE</code>
<code>periodic_boundaries</code>	<code>1</code>	<code>! 0: no periodic boundaries (currently unsupported),</code> <code>! 1: periodic boundaries</code>
<code>reneighbor</code>	<code>1</code>	<code>! controls the reneighboring frequency (0 only)</code>
<code>tabulate_long_range</code>	<code>0</code>	<code>! tabulate Coulomb energy and force calculations:</code> <code>! 0 = no tabulation,</code> <code>! >0 = num. entries in lookup table</code>
<code>vlist_buffer</code>	<code>1.0</code>	<code>! set the "buffer" in Angstroms beyond the long-range</code> <code>! cut-off used during neighbor list construction</code>
<code>nbrhood_cutoff</code>	<code>5.0</code>	<code>! bond cutoff in Angstroms</code>
<code>hbond_cutoff</code>	<code>7.5</code>	<code>! H-bond cutoff in Angstroms</code>
<code>thb_cutoff</code>	<code>0.001</code>	<code>! threshold value for valence angles</code>
<code>charge_method</code>	<code>1</code>	<code>! charge method: 0 = QEq (unsupported with Amber),</code> <code>! 1 = EE, 2 = ACKS2</code>
<code>cm_q_net</code>	<code>0.0</code>	<code>! net system charge</code>
<code>cm_solver_q_err</code>	<code>1e-6</code>	<code>! tolerance for the charge solver</code>
<code>cm_solver_type</code>	<code>2</code>	<code>! iterative linear solver used for charge method:</code> <code>! 0 = GMRES, 1 = Householder GMRES,</code> <code>! 2 = CG, 3 = SDM, 4 = BiCGStab</code>
<code>cm_solver_max_iters</code>	<code>200</code>	<code>! max solver iterations</code>


```
cm_solver_pre_comp_type 1      ! method used to compute preconditioner:  
                                ! 0 = None, 1 = Jacobi
```

IMPORTANT NOTE: the AMBER variables within the &reaxff namelist section always take precedence over the variables in the reaxff-puremd control file. As such, mixing the two (beyond specifying filenames within &reaxff) is not recommended

12. Using energies and forces from an external library

From Amber20 on, it is possible to run simulations where the energies and forces are computed from an external library. This allows the use of Amber features like geometry optimization, restraints, umbrella sampling or T-REMD during the simulations, and the use of AmberTools like CPPTRAJ for the analyses of the output files. The feature of calling external libraries is available for both *sander* and *pmemd*, in serial, CUDA, and MPI versions. The *sander* or *pmemd* code of course doesn't know if the external library is using MPI or CUDA, or anything else: that is for the supplier of the library to decide.

As of release time Amber24, effective interfacing with the MBX and KMMD external libraries is tested and supported. MBX is a software developed by the Paesani group at UCSD that computes energies and forces for the MB-pol and MB-nrg many-body potentials (<http://paesanigroup.ucsd.edu/software/mbx.html>). Machine Learning Molecular Dynamics (KMMD) is a bundled library which uses the “external” interface for convenience, and for ease of customisation.

The purpose of the interface is that other external libraries can be easily used, at most requiring only minor modifications to the current Amber source code (see `src/pmemd/src/external.F90` for *pmemd* and `AmberTools/src/sander/external.F90` for *sander*) and build system. If the desired library is renamed to `libmbx.so` or `libkmmd.so` and placed on Amber's linking path, and if it follows the same interface requirements, then it can be dropped in without editing or recompiling the Amber source code at all. To use C++ or C to create a library modifying the Amber force evaluation, users are encouraged to look at the source file `AmberTools/src/KMMD/KMMDwdassw.externFortran.C` which implements the Amber interface for the KMMD library.

12.1. Installation instructions

In order to make use of a custom external library from Amber, the first step is to properly compile Amber with access to the external library. For compilation, the following steps need to be executed:

1. Add `-DCMAKE_PREFIX_PATH=[path to where you installed the external library]` into your `cmake` command at `amber20_src/build/run_cmake`
2. (Re)compile Amber

For the extremely lazy, just copy the external library to the installation `lib` directory, with the name of an existing library already using the external interface, and the smuggled-in library should be picked up at run time.

12.2. Simulation setup and input parameters

In order to use an external library users still have to set up a conventional Amber simulation with `prmtop` and coordinates files, even though the parameters will not be used during the simulation because energies and forces are provided by the external library. After this step is completed, the following changes are required to the `mdin` input file in order to activate the external library:

1. Add the flag `iextpot=1` into the main namelist `&cntrl` in your `mdin` file. If you do not set `iextpot` or if you set `iextpot=0`, then the simulation will be performed with the force field parameters in your `prmtop` file.

2. Add a new namelist called `&extpot` into the `mdin` file. This namelist contains input parameters that control which external library is in use, and parameters that are specific to a given external library. In the case of MBX, you have to specify the name of the `json` file that controls MBX. Here is an example:

```
&extpot  
  extprog='mbx' ,  
  json='mbx.json' ,  
/
```

Note: users may want to execute the minimization, heating and equilibration stages of the simulation at the force field level before activating the external library in order to save computational time.

The format of the 'json' file depends on the particular external library: this can in fact be any filename, it is opened and read by the external library code and not by Amber.

Part III.

System preparation

13. Preparing PDB Files

The only required or useful data in a PDB file to set up AMBER simulations are: atom names, residue names, and maybe chain identifiers (if more than one chain is present), and the coordinates of heavy atoms. Non-protein structures (especially low-molecular-weight ligands) will cause problems unless extra libraries are loaded; water and monatomic ions are generally recognized if their names in the PDB file correspond to the internal names in the AMBER libraries.

The upshot is that most PDB files require some modification before being used in Amber. Most of the recommended steps given below can be achieved with the *pdb4amber* program with the *reduce* option:

```
pdb4amber -i orig.pdb -o new.pdb --reduce --dry
```

This converts the original pdb file into one likely to be more suitable for input into LEaP. But these programs (which are described in Sections 13.4 and 13.5 below) cannot anticipate all situations, so you should still examine the output pdb file to consider the points below.

13.1. Cleaning up Protein PDB Files for AMBER

This is a crucial step in the preparation and many potential problems and subsequent errors arise from omitting this step! (But also note that these are guidelines for beginners: there are certainly circumstances where you may wish to modify the ideas presented here.)

- Analyze the PDB file visually in any viewer that can represent (and maybe modify) the file. Alternatively, use a text editor. Delete all parts which are judged irrelevant for the simulation. Be aware that anything not protein or water will require you to prepare and load extra library files.
- If the x-ray unit cell in the PDB file contains more than one image, choose the entity you want to use and delete the other(s).
- If there is a **ligand**, save it as an MDL standard data file (SDF). Many software packages are able to do this directly. You may also save the ligand in PDB format and then use some other tools later to convert it into a decent SDF file (**including correct bond order and all hydrogens**). It is crucial to **keep the coordinates of its heavy atoms at their original location**. Then delete it from the PDB file. The ligand must be treated separately later.
- Delete all water molecules that are not considered relevant. Some waters might be essential for ligand binding. If those waters are kept, they should be made part of the receptor (as distinct "residues"), not of the ligand. *LEaP* recognizes water if the residue name is WAT or HOH. In later simulations, they may have to be tethered (more or less strongly) to their original positions to prevent them from "evaporating".
- Apply the same delete procedure to ions, co-factors, and other stuff that has no special relevance for the planned simulation.
- **Get rid off all protein (or peptide) hydrogens that are explicitly expressed in the PDB file.** The *reduce* and *LEaP* utilities add hydrogens automatically with predefined names. Having hydrogens in PDB files with names that *LEaP* does not recognize within its residue libraries leads to a total mess.
- Eventually, **remove also all connectivity records**. These are mostly referring to ligands, or, in some cases, to disulfide links. The latter should be explicitly re-connected (see later) without relying on connectivity records in the PDB file.

- The final PDB file of the protein should only contain unique locations for heavy atoms of amino acids (and maybe oxygens of specific water molecules). (In some PDB files, the same amino acid may be represented by different states (conformations). You must decide which unique location you want to use later in the simulations. If you don't do anything, Amber will use the "A" conformation, which is generally the most highly occupied one.) Missing atoms in amino acids are mostly allowed since *LEaP* can rebuild them if the **residue names** are **correct** and if the **atoms** already present have **correct names** also.
- **Make use of "TER" records to separate parts in the PDB file which are not connected covalently.** This is especially important in protein structures in which parts are missing (gaps). Not separating the loose ends by a "TER" record may lead to strange (and wrong) behavior in *LEaP* or later in the simulations. Apply the same rule to individual water molecules which you want to keep and separate each water by a "TER" record.

13.2. Residue naming conventions

Tautomeric and protonation states are not rendered in PDB files. If a defined state for a residue is required, its **name** in the PDB file must reflect the choice. The following subsections deal with these cases. **Important:** if you change a residue name in a PDB file, make sure to change it for **all** atoms of that residue!

Note also that PDB files written by *LEaP* will keep the "special" names, which sometimes leads to annoying effects in software packages which are not prepared for amino acids called HIE, HIP, CYX, and alike. You might consider to change these names back to the standard prior to using these PDB files in other software packages. You can also use the *"-bres"* option in *ambpdb* to do that.

Histidine can exist in three forms (δ , ϵ , and protonated). The PDB file must reflect the choice of the user. In the current versions of *LEaP* command files included with AMBER, ϵ -histidine is the default, i.e., a "HIS" residue in a PDB file will be translated automatically to HIE (for ϵ -histidine). If the residue is called "HID" in the PDB file, the resulting residue for AMBER will become δ -histidine, while "HIP" will yield the protonated form.

Cysteine can exist in free form or as part of a disulfide bridge. PDB residues named "CYS" are automatically converted into a free cysteine with a SH side chain end. If the cysteine is known to be in a **S-S bridge**, the residue name in the PDB file **must** be "CYX". In that case, no hydrogen is automatically added to the side chain which ends in a bare sulfur. However, S-S bonds to pairing cysteines are not automatically made but must be specified by the user.

Asp, Glu, Lys Sometimes the usually charged residues aspartate "ASP", glutamate "GLU", and lysine "LYS" might have to be used in their uncharged form. The residue names must then be changed to "ASH", "GLH", and "LYN", respectively. A neutral form of **arginine** is not foreseen in AMBER (as the pKa of arginine is around 12, it is always considered protonated).

Terminals: ACE, NHE, NME There are special N- and C-terminal cap residues which can be used to neutralize the N- and C-terminal in peptide chains when the defaults (NH_3^+ for the N-terminal and COO^- for the C-terminal) are not appropriate.

The "ACE" residue [$-C(=O) - CH_3$] can be used to cap the N-terminal. The PDB entry of the capping residue ACE must be:

ATOM	1	CH3	ACE	resnumber	x	y	z
ATOM	2	C	ACE	resnumber	x	y	z
ATOM	3	O	ACE	resnumber	x	y	z

Note the atom name "CH3" for this special carbon: another name is not allowed. Hydrogens should be omitted. They are automatically added if the residue name and the heavy atom names are correct.

For capping the C-terminus, two possibilities are given. The first one is a simple NH_2 termination giving [$C(=O) - NH_2$]. This residue is called "NHE" in the PDB file and consists of a single atom to be named N:

ATOM	1	N	NHE	resnumber	x	y	z
------	---	---	-----	-----------	---	---	---

The second possible C-terminal cap is $NH - CH_3$, resulting in $[C(=O) - NH - CH_3]$ at the C-terminal. Its entry in the PDB file must have the residue name "NME" and has the following PDB entry:

ATOM	1	N	NME	resnumber	x	y	z
ATOM	2	C	NME	resnumber	x	y	z

The atom name for the carbon must be "C", as in the CIF format. "NHE" and "NME" residues are automatically completed with hydrogens. Do not enter them explicitly.

The "ACE" residue should be the first residue in a chain (strand) while "NHE" or "NME" should be the last. If cap residues are used to terminate gaps in incomplete protein chains, they must appear at the exact gap location, respecting N-terminal and C-terminal order. Gaps must be separated by a "TER" record in the PDB file. See section 13.3.

13.3. Chains, Residue Numbering, Missing Residues

- AMBER preparation modules assume that residues in a PDB file are connected and appear sequentially in the file. If not covalently connected (i.e., linked by an amide bond), the residues must be separated by "TER" records in the PDB file. (Alternatively, the chainid must change on going from one chain to the next chain.) Thus for example, a protein consisting of two chains should have a "TER" record after the final residue of the first chain. Similarly, if residues are missing (e.g., not detected in x-ray, or cut by the user), the gap should also be separated by a "TER" record. Terminal residues will be charged by default. If the user wants to avoid this (especially for gaps), these residues should be capped (by ACE and NHE or NME).
- In general, *LEaP* and tools using it refer to the original **input residue numbers**. Thus, residues are numbered (rather "named") according to the original PDB file for special commands like the disulfide connections.
- In output files from *LEaP*, **residues will always be numbered starting from 1**, irrespective of the original numbering. Gaps are not considered either. Thus if a protein chain runs from 21 to 80, with residues 31 to 40 (i.e., 10 residues) missing, the final numbering of residues will run from 1 to 50.

The final residue numbers are the ones that must be used in later simulations to refer to individual residues via *AMBER masks* or *NAB atom expressions*. For example, if a protein chain with residues from 30 to 110 is prepared for AMBER simulations, the final numbering will go from 1 to 81. If the original residues 35 to 40 should be fixed or tethered, the actual residues to be specified are 6 to 11. This can lead to serious errors. So be careful about residue numbers. The script *pytleap* described later will always generate a new PDB file with exact AMBER residue numbering and atom names. This PDB file should be used as reference throughout all subsequent AMBER simulations. Above all, when using atom masks or atom expressions (see Appendix 24), always check that they really refer to the desired atoms before running lengthy simulations. *Fixing or tethering wrong atoms are a common error which may easily go unnoticed.*

13.4. pdb4amber

pdb4amber analyses PDB files and cleans them for further usage, especially with the *LEaP* programs of Amber. This utility was originally written by Romain Wolf, but later modified (mainly by Hai Nguyen) to use the *parmed* tools under the hood.

Typing *pdb4amber* on the command line without options (or followed by -h) produces the following help message:

```
usage: pdb4amber [-h] [-i FILE] [-o FILE] [-y] [-d] [-s STRIP_ATOM_MASK]
                [-m MUTATION_STRING] [-p] [--constantph] [--most-populous]
                [--keep-altlocs] [--reduce] [--no-reduce-db] [--pdbname]
```



```

        [--add-missing-atoms] [--model MODEL] [-l FILE] [-v]
        [--leap-template] [--no-conect] [--noter]
        [input]

positional arguments:
  input                PDB input file (default: stdin)

optional arguments:
  -h, --help            show this help message and exit
  -i FILE, --in FILE    PDB input file (default: stdin)
  -o FILE, --out FILE   PDB output file (default: stdout)
  -y, --nohyd           remove all hydrogen atoms (default: no)
  -d, --dry             remove all water molecules (default: no)
  -s STRIP_ATOM_MASK, --strip STRIP_ATOM_MASK
                        Strip given atom mask, (default: no)
  -m MUTATION_STRING, --mutate MUTATION_STRING
                        Mutate residue
  -p, --prot            keep only Amber-compatible residues (default: no)
  --constantph         rename GLU,ASP,HIS for constant pH simulation
  --most-populous      keep most populous alt. conf. (default is to keep 'A')
  --keep-altlocs        Keep alternative conformations
  --reduce             Run Reduce first to add hydrogens. (default: no)
  --no-reduce-db       If reduce is on, skip using it for hetatoms. (default:
                        usual reduce behavior for hetatoms)
  --pdbid              fetch structure with given pdbid, should combined with
                        -i option. Subjected to change
  --add-missing-atoms  Use tleap to add missing atoms
  --model MODEL        Model to use from a multi-model pdb file (integer).
                        (default: use 1st model). Use a negative number to
                        keep all models
  -l FILE, --logfile FILE
                        log filename
  -v, --version         version
  --leap-template      write a LEaP template for easy adaption (EXPERIMENTAL)
  --no-conect          Not write S-S conect record
  --noter              Not writing TERUsage: pdb4amber [options]

```

The new output file (specified with `-o` or `--out`) is a standard PDB file with all residues sequentially re-numbered from 1 to N. In addition, several other files are created automatically:

- A text file with the output PDB file name and `_renum.txt` added. This is a table to help convert the renumbered residues into the original ones.
- A PDB file with the output PDB file name and `_nonprot.pdb` appended. This is a PDB file that contains only non-protein residues (apart from water), i.e., mainly ligands and other stuff.
- When using `-d` (`--dry`), a PDB file with the output file name plus `_water.pdb` added. This file contains exclusively the water that has been stripped from the original PDB file.
- A text file with the output PDB file name and `_sslink` attached, if disulfide bonds have been detected by `pdb4amber`. This file might be used by the `pytleap` script to generate the correct disulfide bonds between cysteines.

The following information is written to screen, but can also be captured into a text file by ending the command line with `'2>'` e.g.:

```
pdb4amber -i pdbin.pdb -o pdbout.pdb [-options] 2> some.file.name.log
```

13. Preparing PDB Files

Chains: All chain indicators in the PDB file are listed. This is useful especially in cases where the x-ray unit cell contains more than one image of a protein (or complex). In many cases, one is only interested in one main peptide chain. A long list of different chains may indicate that the PDB file should be cleaned manually prior to using `pdb4amber`.

Insertions: Insertions are mostly 'artificial' residue numbers to keep specific key residue numbers in large protein families constant. `pdb4amber` discards insertion codes and re-numbers all residues from 1 to N. But the insertions are listed to the screen and also included in the `_renum.txt` file.

Histidines: `pdb4amber` first checks if the type of each histidine (HIE, HIP, HID) can be determined from explicit hydrogens. Any histidines whose protonation state can not be determined are renamed to HIE. A message alerts the user to all histidine residues (the residue numbers refer to the renumbered scheme!) to allow for manual reassignment if so desired.

Non-standard residues: Non-standard residues (i.e., residues not automatically recognized by Amber) are listed. Mostly they are ligands (sometimes co-factors, detergent, buffer components, etc.). The user must take care of these separately. These residues are also found in the `_nonprot.pdb` file mentioned above. They are removed from the final output PDB file if the `-p (--prot)` option was chosen. Otherwise they are left also in the output PDB file.

Cysteines in disulfide bonds: `pdb4amber` locates possible (most probable) disulfide bonds by checking the distance between SG (gamma sulfur) atoms in cysteines. If a distance SG-SG less than 2.5 Angstrom is found between the SG atoms of two CYS, a disulfide bond is assumed. The respective CYS residues are renamed to CYX (required for Amber) in the final PDB output file. CONECT records are also printed in the final PDB output file which are then automatically recognized by `tleap`. The residue numbers of the CYX residues refer to the renumbered scheme!

Gaps: `pdb4amber` tries hard (and mostly succeeds) in locating 'gaps', i.e., missing residues in the PDB file. This is done by checking distances of consecutive C-N atoms. If such a distance is larger than 2 Angstrom, `pdb4amber` considers that there is a gap between the two residues and reports the gap to the screen. The listed residue numbers refer to the renumbered scheme! It is up to user to decide how to handle the gaps. *Doing nothing at all will most probably lead to trouble later!* By simply introducing a TER record at the gap, Amber (LEaP) will later introduce the charged N (NH3+) or C (COO-) terminals at the gap borders. If far from the binding site, this might be OK (except in long and unconstrained MD, where such unnatural charges will inevitably lead to unrealistic behavior). The better solution is to introduce ACE or NME caps at the correct positions (in addition to a TER record separating the gap residues). This can be done in various ways (e.g. with PyMol). The correct names of the newly introduced residues (ACE or NME) and atoms (CH3 for the methyl carbon, C, N, O for the others) must be observed!

Missing atoms: `pdb4amber` tries to determine missing heavy atoms in standard amino acids and reports these. Residue numbers refer to the renumbered sequence. Note that this has no implications on further usage of the file with LEaP since missing atoms are added automatically anyway. In some cases, this addition may lead to clashes however and it might be useful to know which residues are actually affected by LEaP.

13.5. reduce

Reduce is a program for adding hydrogens to a Protein Data Bank (PDB) molecular structure file. It was developed by J. Michael Word at Duke University in the lab of David and Jane Richardson. *Reduce* is described in: Word, et. al. (1999) Asparagine and Glutamine: Using Hydrogen Atom Contacts in the Choice of Side-chain Amide Orientation, *J. Mol. Biol.* **285**, 1733-1747.

Both proteins and nucleic acids can have hydrogens added. HET groups can also be processed as long as the atom connectivity is provided. A slightly modified version of the connectivity table provided by the PDB is included. The latest version of *reduce* is available at <http://kinemage.biochem.duke.edu/>.

In most circumstances, the recommended command when using *reduce* to add hydrogens to a PDB file and standardize the bond lengths of existing hydrogens is

```
reduce -build -nuclear coordfile.pdb > coordfileH.pdb
```

which includes the optimization of adjustable groups (OH, SH, NH₃⁺, Met-CH₃, and Asn, Gln and His sidechain orientation). Disulfides, covalent modifications, and connection of the ribose-phosphate nucleic acid backbone, are recognized and any hydrogens eliminated by bonding are skipped. When an amino acid main-chain nitrogen is not connected to the preceding residue or some other group, *reduce* treats it as the N-terminus and constructs an NH₃⁺ only if the residue number is less than or equal to an adjustable limit (1, by default). Otherwise, it considers the residue to be the observable beginning of an actually-connected fragment and does not protonate the nitrogen. *Reduce* does not protonate carboxylates (including the C-terminus) because it does not specifically consider pH, instead modeling a neutral environment.

Hydrogens are positioned with respect to the covalently bonded neighbors and these are identified by name. Nonstandard atom names are the primary cause of missing or misplaced hydrogens. If *reduce* tries to process a file which contains hydrogens with nonstandard names, the existing hydrogens may not be recognized and may interfere with the generation of new hydrogens. The solution may be to remove existing hydrogens before further processing.

There are a number of other, more advance, options for *reduce*, which can be viewed by running:

```
reduce -h
```

13.6. *pdb2pqr*

While *reduce* will add hydrogens according to the residue names, it is not able to estimate if the pK_a of a given residue is affected by its environment, nor to use a pH as desired by the user. If it is suspected that a titratable residue has a different protonation state than "conventionally", you can use tools like *pdb2pqr* [432] for obtaining PDB structures with the proper residue names according to their protonation state, using *propka3* [433] in the background to calculate the pK_a for the residues. *pdb2pqr* is installed with the included Miniconda at the moment. If you used your own python, it can be easily installed in a conda environment or by using pip:

```
conda install -c conda-forge pdb2pqr // pip install pdb2pqr
```

With *pdb2pqr* available in your path, you can add hydrogens according to estimated pK_as using the following command:

```
pdb2pqr --ffout AMBER --titration-state-method=propka --with-ph PH --pdb-output NAME
```

where "NAME.pdb" corresponds to the protein that is going to be protonated, and PH is the pH to use for the calculation (7 by default).

13.7. *gwh* (guess water hydrogens)

This is an old code (originally written by Suishi Miyamoto) that optimizes the positions of hydrogens in waters. It uses an iterative procedure that tries to find good electrostatics and hydrogen bonding possibilities. This can be especially helpful when only oxygen positions are provided in the input PDB file (as is the case for most crystal structures). But *gwh* can also improve on the rather arbitrary placement that *tleap* would make in such circumstances. The usage is this:

```
gwh -p prmtop -w watpdb [-consd] [-elstat] < input-pdb > output-watpdb
```

The *input-pdb* file should be compatible with the *prmtop* file. Any water residues that might be present there will not be modified. Water molecules in *watpdb* will have hydrogens added or optimized, with results going to *output-watpdb*. A typical use pattern would be this: remove waters from your initial *pdb* file, transferring them to *watpdb*. Use *tleap* to build a *prmtop* file from the remaining, "dry" *input-pdb* file. Then paste the updated waters from *output-watpdb* at the end of the *input-pdb* file, and rebuild the *prmtop* file.

13. Preparing PDB Files

A note: if you are planning an MD equilibration anyway, the location of the initial hydrogen positions may not matter much. Tests on fairly simple protein-water systems suggest that minimization alone produces better results (*i.e.* lower energies) if *gwh* is used than if the essentially arbitrary hydrogen positions coming from *tleap* are used. Note also that *gwh* is a complement to *reduce* (described above): *gwh* deals only with water residues, whereas *reduce* ignores them.

If the `-consd` flag is set, a constant dielectric constant will be used to compute the electrostatics that are being optimized for the hydrogens. (Most testing has been done without this flag, so tread carefully.) If the `-elstat` flag is present, the water residues will be sorted by their local electrostatic energies, which can help when looking for water residues that are less “happy” in their environment.

13.8.

13.9. packmol-memgen

packmol-memgen is a workflow to generate Amber-ready protein/membrane/ion/solvent systems by using Memembed [434] for orienting the protein, pdbremix [435] to estimate the volume, and Packmol [436, 437] as the packing engine. The software is also able to wrap simple Amber tasks, such as parametrization (“`--parametrize`”) and minimization (“`--minimize`”) of the generated system, or inclusion of solutes into the water box. The workflow and main features have been described in:

- “PACKMOL-Memgen: A Simple-To-Use, Generalized Workflow for Membrane-Protein-Lipid-Bilayer System Building” S. Schott-Verdugo and H. Gohlke **Journal of Chemical Information and Modeling** 59 (6), 2522-2528 doi:10.1021/acs.jcim.9b00269 [438].

A typical case scenario is, for example, to pack a bacterial membrane protein into a bacterial-like membrane (such as DOPE:DOPG 3:1). To fulfill such a task, the following command line is sufficient:

```
packmol-memgen --pdb NAME.pdb --lipids DOPE:DOPG --ratio 3:1
```

where “NAME.pdb” corresponds to the protein that is going to be packed, and the orders of the colon-separated lists of lipids and ratio correspond to each other. These lists can be further expanded to any complex mixture the user desires (e.g., “DOPC:DOPE:DOPS:CHL1”), specifying different compositions per leaflet (by separating with “//”, e.g. “DOPC:DOPE//DOPE:DOPS”), or even adding additional lipid bilayers every time the “`--lipids`” flag is used. The user has to be aware that, by increasing the complexity of the membrane bilayer, the packing time will increase, but more importantly, the time required to equilibrate such a system will also increase. The output pdb is made Amber readable through *charmmlipid2amber.py*. By default, all hydrogens of the protein will be trimmed and reprotonated with 13.5. You can control this behaviour with `--nottrim` and `--notprotonate`. This will not change protonation states using the residue environment, as *pdb2pqr* would. If *pdb2pqr* is available in your PATH (currently installed when using the Miniconda option), you can use it to protonate your protein using and setting the pH with `.`. Please note that *pdb2pqr* uses propka3 to estimate the protonation states in the protein structure, which should be more accurate than *reduce*.

The dimensions of the system are by default estimated based on the size of the protein to be packed. If no protein is being used, you can still pack a membrane only system by using “`--distxy_fix`” to specify the x and y box length:

```
packmol-memgen --lipids DOPE:DOPG --ratio 3:1 --distxy_fix 100
```

where a membrane of 100 x 100 Å will be generated. Alternatively, you can use the “`--dims`” flag to specify the whole box size (see below).

As of Amber24, the OPM/PPM3 orientation program can be used within the script by adding the `--ppm` flag [439], with the *immers* executable also available as a standalone. Please check the help message for further options (“`-h`”, and “`--help`” for extended help).

In AmberTools25, *packmol* > 21.0.1 with a periodic boundary condition GENCAN evaluation is included, with a first approach to make use of its functionality through the `--pbc` flag in *packmol-memgen*. It has been observed that considering PBC can make better packing results, so consider testing it for you systems.

Acyl chain full name (Acid)	Abbreviation
Lauric acid 12:0	L
Myristic acid 14:0	M
Palmitic acid 16:0	P
Stearic acid 18:0	S
Oleic acid 18:1(9)	O
Arachidonic acid 20:4(5,8,11,14)	A
Docosahexaenoic acid 22:6(4,7,10,13,16,19)	H (D)*
*D is the usual abbreviation. H is adopted to distinguish from diacyl phospholipids.	

Table 13.1.: Acyl chain abbreviations used in packmol-memgen.

Head group full name	Abbreviation
Phosphatidylcholine	PC
Phosphatidylethanolamine	PE
Phosphatidylglycerol	PG
Phosphatidic acid	PA
Phosphatidylserine	PS
Cardiolipin	CL

Table 13.2.: Head group abbreviations used in packmol-memgen.

13.9.1. Included lipids and naming conventions

The lipid names used are abbreviations of trivial names, where the first and second letters correspond to the acyl chains in positions *sn*-1 and *sn*-2, respectively, and the rest corresponds to the headgroup present (with the exception of cholesterol):

<sn-1 tail><sn-2 tail><headgroup>

In case the first letter is a D, it is assumed that both acyl chains are equal (e.g. DOPC, 1,2-dioleoyl-*sn*-glycerophosphocholine). Tables 13.1 and 13.2 show the abbreviations used. With “--available_lipids” a brief list of commonly used lipids will be printed.

Lipid extension Lipid_ext: lysophospholipids, phosphatidylinositol, cardiolipins and sterols

The list of available lipids has been extended considerably from the combinations possible from Lipid17 or Lipid21. We call this force field extension Lipid_ext. New headgroups include lysophospholipids, phosphatidylinositols, and cardiolipins; they should be handled with care, as they are still in development. If you want a full list of the available lipids, you can use “--available_lipids.all”, but be aware that about 4000 lipids are available at the moment. If you are looking for something specific, `grep` this list and see if you find the lipid you are interested in.

In addition to the lipids mentioned above, ergosterol, campesterol, sitosterol, and stigmasterol were added in AmberTools21. Depending on the user, they use Lipid17/Lipid21 bonded and LJ parameters, with charges obtained through a multiconformational RESP fit, using a capping strategy as described for both Lipid11 [101] and Lipid14 [103]. The headgroups and sterols included can be found in Table 13.3.

In the case of lysophospholipids, the 4-letter names describe the topology of the molecules. For example, for 2LPC (1-palmitoyl-2-hydroxy-*sn*-glycero-3-phosphocholine), a lysophosphatidylcholine with a C16 fatty acid chain:

```

2          // "lyso" position, or where a tail was removed. In this case the tail in position sn-2 is "missing".
L          // "lyso", indicating that the lipid corresponds to a lysophospholipid.
P          // name of the tail present. Palmitoyl in this case.
```

Description	Lipid_ext Residue Name
1,3-bis(<i>sn</i> -glycero-3'-phospho)- <i>sn</i> -glycerol	CLI
1-hydroxy- <i>sn</i> -phosphatidylcholine	PE2
2-hydroxy- <i>sn</i> -phosphatidylcholine	PE1
1-hydroxy- <i>sn</i> -phosphatidylethanolamine	PE2
2-hydroxy- <i>sn</i> -phosphatidylethanolamine	PE1
1-hydroxy- <i>sn</i> -phosphatidylglycerol	PG2
2-hydroxy- <i>sn</i> -phosphatidylglycerol	PG1
ergosterol	ERG
stigmaterol	STI
sitosterol	SIT
campesterol	CAM

Table 13.3.: *Lipid_ext* cardiolipin and lysophospholipid head group and sterol residue names. For *LIPID21* residue names, check 3.9

C // last letter of the name of the phospholipid head group. Choline in this case.

Cardiolipins follow similar rules as for phospholipids, with the name describing first positions *sn*-1 and *sn*-2 of the glycerol moiety attached in position *sn*-1 of the central glycerol, followed by positions *sn*-1 and *sn*-2 of the glycerol attached in position *sn*-3, finishing with CL. For example, PODOCL would be a POPA (1-palmitoyl-2-oleoyl PA) and a DOPA (1,2-dioleoyl PA) attached through their *sn*-3 phosphate in positions *sn*-1 and *sn*-3 of a glycerol, respectively. The exception to this comes when all tails are equal; in this case, the prefix T (from tetra-) is added. As an example, TOCL is a glycerol with two DOPA residues, each attached in positions *sn*-1 and *sn*-3 of the glycerol moiety.

Additionally, experimental head groups of phosphatidylinositols with multiple phosphorylation and protonation states are included. The parameters were derived in a similar fashion as for the lysophospholipids, including GLYCAM_06j (3.4) parameters for the inositol/phosphate part. The headgroups of phosphatidylinositols can be found in Table 13.5 with the abbreviation used for packmol-memgen.

To make use of lysophospholipids, phosphatidylinositols, cardiolipins, or additional sterols, the parameter files within packmol-memgen have to be loaded. The easiest way to do this is by using the “--parametrize” together with the “--keep” flags. For example:

```
packmol-memgen --lipids DOPE:TOPC --ratio 3:1 --distxy_fix 100 --keep --parametrize
```

This will generate a *leap.in* file, which can be edited by the user if required. **WARNING!:** In the case of cardiolipins, *packmol-memgen* specifies bonds for the acyl tails in position *sn*-3 of the headgroup explicitly! This means that if for any reason you try to use the “CLI” headgroup outside of *packmol-memgen*, you need to make sure to set these bonds in LEaP yourself.

13.9.2. Additional functionalities

If you want to use your own parametrized lipids, you can parse a memgen.parm file with “--memgen_parm”, which has the following structure (check the default file in the packmol_memgen source folder):

```
RES HEAD_A TAIL_A APL_FF APL VOLUME CHARGE HEAD_PLANE TAIL_PLANE CH3:CH2:CH HEAD_VOLUME CHARMM NAM
```

A description of the meaning of the fields can be seen in Table 13.4.

The option to provide Gaussian-shaped constraints to the membrane surface is provided. This allows to generate shapes of curved or buckled membranes with the program according to:

$$f(x,y) = he^{(-\frac{x^2}{2c^2} - \frac{y^2}{2d^2})}$$

Field	Comment
RES	Residue name and prefix of PDB file to be used.
HEAD_A	Atoms to be constrained to the membrane surface, above HEAD_PLANE. List of atom indexes separated by commas.
TAIL_A	Atoms to be constrained close to the membrane center, under TAIL_PLANE. List of atom indexes separated by commas.
APL_FF	Area per lipid (APL) in $\text{\AA}^2$ estimated in DOPC 4:1 mixture. An educated guess can be used.
APL	Experimental APL in $\text{\AA}^2$ if available, XX otherwise.
VOLUME	Experimental volume in $\text{\AA}^3$ if available, XXXX otherwise.
CHARGE	Lipid charge.
HEAD_PLANE	Distance to membrane center of plane above which head atoms have to be placed. Usually 18 $\text{\AA}$.
TAIL_PLANE	Distance to membrane center of plane under which tail atoms have to be placed. Usually 4 $\text{\AA}$.
CH3:CH2:CH	Number of CH ₃ , CH ₂ and CH carbons in lipid tails. E.g. 2:28:4 for DOPC.
HEAD_VOLUME	Molecular volume of headgroup in $\text{\AA}^3$.
CHARMM	PDB compatible with CHARMM? Y/N
NAME	Full name.

Table 13.4.: Fields in the memgen.parm file.

You can set the values of c , d , or h by respectively listing the desired values after the "--xygauss" flag and regulate the shape of the resulting system. An example of a command line to build a buckled membrane:

```
packmol-memgen --xygauss 50 5000 40 --dims 300 50 121 --tight_box --parametrize
```

where "--dims" sets the x, y, and z dimensions, and "--tight_box" is required to parametrize the system with the expected xy dimensions and avoid the curvature from relaxing. This will generate a DOPC (lipid used by default) membrane "pinched" in the x-axis. Finding the right dimensions and shape values might take some trial and error. You can check the systems while they are being packed by opening the intermediate PDB with a molecular viewer like PyMOL to decide if the generated shape is as expected. Consider checking the recently implemented MC-barostat **baroscalingdir** option for your simulations if you use curved membranes. A barostat controlling only the z-axis (baroscalingdir=3) allows to relax the simulation box while keeping the xy dimensions.

You can automatically coarse-grain your system using the SIRAH force field 3.11 by using the "--sirah" flag. Lipids with the "si" prefix in the available lipids ("--available_lipids") are usable for SIRAH simulations.

Using packmol-memgen to prepare solvated systems

Even though *packmol-memgen* was designed for membrane packing, it can also be used to solvate only. This can be particularly useful if a salt or a specific solute concentration is desired. For this, use a command line as follows:

```
packmol-memgen --pdb NAME.pdb --solvate --cubic \
--solute SOLUTE.pdb --solute_con CONCENTRATION
```

where "SOLUTE.pdb" corresponds to a pdb file that contains the solute to be added, and "CONCENTRATION" is either the number of molecules to add, the concentration in molar (by adding M as a suffix, e.g. 1M) or the volume percentage (by adding % as a suffix, e.g. 10%). The latter is estimated using a grid approach on the input

13. Preparing PDB Files

"SOLUTE.pdb". If you want to parametrize the system automatically with the script, you should pass lib and frcmod files with "--ligand_param". A distance constraint can be set between the introduced solute and a protein with "--solute_prot_dist", avoiding starting conformations close to a possible binding site.

Additionally, support for using multiple solvents has been added. You can check the included solvent options and source of parameters with "--available_solvents". You can choose the solvents to use with "--solvents" and "--solvent_ratio", in a similar way as specified for different lipids, where the ratio is in v/v:

```
packmol-memgen --pdb NAME.pdb --solvate --cubic \  
--solvents WAT:CL3 --solvent_ratio 4:1
```

where a defined PDB will be solvated in a 4:1 water:chloroform mixture. **WARNING:** The parameters are obtained from the provided references. Despite that all of them have been tested within the AMBER framework, there is no insurance that they will behave properly in complex mixtures! You are encouraged to perform your own tests, and to develop your own set of parameters. You can extend or overwrite the list of usable solvents with "--solvent_parm"; you can check the file structure in the packmol-memgen source folder, which is specified as:

```
RES DENSITY MOLECULAR_WEIGHT CHARGE NAME
```

where RES specifies the residue name and the prefix of the pdb file to be used, DENSITY is in g/ml and the MOLECULAR_WEIGHT in g/mol. Be aware that the charges are not used at the moment, but can be neutralized during parametrization in LEaP if required. You can still parametrize your system within the script by passing your lib and frcmod files with "--ligand_param".

For a complete set of available functionalities, please refer to the help included within the software by executing:

```
packmol-memgen --help
```

Note: Due to the complex packing problem and the possible initial clashes in the output, the user is encouraged to shortly minimize the system using the CPU code of pmemd or sander. This can be done directly with the "--minimize" flag.

13.10. Building bilayer systems with AMBAT

An alternative tool for building bilayer systems is AMBAT (Amber Membrane Builder and Analysis Tool), developed by Tarun Khanna and Ian Gould. This package consists of three *tcl* scripts for building and analyzing membrane models and for inserting proteins into the bilayer. Instructions and the scripts themselves are in the \$AMBERHOME/AmberTools/src/AMBAT folder.

Description	Lipid_ext Residue Name	Abbreviation	Protonated phosphate position
Phosphatidylinositol (PI)	PI	PI	-
PI-3'-phosphate	PI3	PI3	-
	PH3	PI3H	3
PI-4'-phosphate	PI4	PI4	-
	PH4	PI4H	4
PI-5'-phosphate	PI5	PI5	-
	PH5	PI5H	5
PI-3',4'-bisphosphate	P2A	PI34A	3
	P2B	PI34B	4
	H2A	PI34H	3,4
	2-A	PI34-	-
PI-3',5'-bisphosphate	P2C	PI35A	3
	P2D	PI35B	5
	H2B	PI35H	3,5
	2-B	PI35-	-
PI-4',5'-bisphosphate	P2E	PI45A	4
	P2F	PI45B	5
	H2C	PI45H	4,5
	2-C	PI45-	-
PI-3',4',5'-trisphosphate	P3A	PI345A	3,4
	P3B	PI345B	3,5
	P3C	PI345C	4,5
	P3D	PI345D	3
	P3E	PI345E	4
	P3F	PI345F	5
	P3-	PI345-	-
	P3H	PI345H	3,4,5

Table 13.5.: *Lipid_ext* phosphatidylinositol head group residue names. For *Lipid17* residue names, check [3.9](#)

14. LEaP

14.1. Introduction

LEaP is the generic name given to the programs *teLeap* and *xaLeap*, which are generally run *via* the *tleap* and *xleap* shell scripts. These two programs share a common command language but the *xleap* program has been enhanced through the addition of an X-windows graphical user interface. The name LEaP is an acronym constructed from the names of the older AMBER software modules it replaces: link, edit, and parm. Thus, LEaP can be used to prepare input for the AMBER molecular mechanics programs.

LEaP is the basic tool to construct force field files (see Fig. 1.1). Using *tleap*, the user can:

```
Read AMBER PREP input files
Read Amber PARM format parameter sets
Read and write Object File Format files (OFF)
Read and write PDB files
Construct new residues and molecules using simple commands
Link together residues and create nonbonded complexes of molecules
Modify internal coordinates within a molecule
Generate files that contain topology and parameters for AMBER and NAB
```

```
usage: tleap [ -I<dir> ] [ -f <file>|- ]
```

The command *tleap* is a simple shell script that calls *teLeap* with a number of standard arguments. Directories to be searched are indicated by one or more “-I” flags; standard locations are provided in the *tleap* script. The “-f” flag is used to tell *tleap* to take its input from a file (or from *stdin* if “-f -” is specified). If there is no “-f” flag, input is taken interactively from the terminal.

A key command for LEaP is *loadPdb*, which inputs sequence and structure information from Protein Data Bank Files. *Be sure to read Section 13 for information on how to “clean up” PDB files before loading them.*

14.2. Concepts

In order to effectively use LEaP it is necessary to understand the philosophy behind the program, especially the concepts of LEaP commands, variables, and objects. In addition to exploring these concepts, this section also addresses the use of external files and libraries with the program.

14.2.1. Commands

A researcher uses LEaP by entering commands that manipulate objects. An object is just a basic building block; some examples of objects are ATOMs, RESIDUEs, UNITs, and PARMSETs. The commands that are supported within LEaP are described throughout the manual and are defined in detail in the “Command Reference” section.

The heart of LEaP is a command-line interface that accepts text commands which direct the program to perform operations on objects. All LEaP commands have one of the following two forms:

```
command argument1 argument2 argument3 ...
variable = command argument1 argument2 ...
```

For example:

```
edit ALA trypsin = loadPdb trypsin.pdb
```

Each command is followed by zero or more arguments that are separated by whitespace. (Whitespace is blanks, tabs, and commas; and as of Amber version 21 carriage returns are also treated as whitespace.) Some commands return objects which are then associated with a variable using an assignment (=) statement. Each command acts upon its arguments, and some of the commands modify their arguments' contents. The commands themselves are case-insensitive. That is, in the above example, `edit` could have been entered as `Edit`, `eDiT`, or any combination of upper and lower case characters. Similarly, `loadPdb` could have been entered a number of different ways, including `loadpdb`. In this manual, we frequently use a mixed case for commands. We do this to enhance the differences between commands and as a mnemonic device. Thus, while we write `createAtom`, `createResidue`, and `createUnit` in the manual, the user can use any case when entering these commands into the program.

The arguments in the command text may be objects such as `NUMBERS`, `STRINGs`, or `LISTs`, or they may be variables. These two subjects are discussed next.

14.2.2. Variables

A variable is a handle for accessing an object. A variable name can be any alphanumeric string whose first character is an alphabetic character. Alphanumeric means that the characters of the name may be letters, numbers, or special symbols such as `"*"`. The following special symbols should not be used in variable names: dollar sign, comma, period (full stop), pound sign (hash), equals sign, space, semicolon, double quote, or the curly braces `{` and `}`. LEaP commands should not be used as variable names. Unlike commands, variable names are case-sensitive: `"ARG"` and `"arg"` are different variables. Variables are associated with objects using an assignment statement not unlike that found in conventional programming languages such as Fortran or C.

```
mole = 6.02E23
MOLE = 6.02E23
myName = "Joe Smith"
listOf7Numbers = { 1.2 2.3 3.4 4.5 6 7 8 }
```

In the above examples, both `mole` and `MOLE` are variable names, whose contents are the same (6.02×10^{23}). Despite the fact that both `mole` and `MOLE` have the same contents, they are not the same variable. This is due to the fact that variable names are case-sensitive. LEaP maintains a list of variables that are currently defined. This list can be displayed using the `list` command. The contents of a variable can be printed using the `desc` command.

14.2.3. Objects

The object is the fundamental entity in LEaP. Objects range from the simple, such as `NUMBERS` and `STRINGs`, to the complex, such as `UNITs`, `RESIDUEs` and `ATOMs`. Complex objects have properties that can be altered using the `set` command, and some complex objects can contain other objects. For example, `RESIDUEs` are complex objects that can contain `ATOMs` and have the properties: residue name, connect atoms, and residue type.

NUMBERS

`NUMBERS` are simple objects holding double-precision floating point numbers. They serve the same function as "double precision" variables in Fortran and "double" variables in C.

STRINGs

`STRINGs` are simple objects that are identical to character arrays in C and similar to character strings in Fortran. `STRINGs` store sequences of characters which may be delimited by double quote characters. Example strings are:

```
"Hello there"
"String with a " (quote) character"
"Strings contain letters and numbers:1231232"
```

LISTs

LISTs are made up of sequences of other objects delimited by LIST open and close characters. The LIST open character is an open curly bracket (`{`) and the LIST close character is a close curly bracket (`}`). LISTs can contain other LISTs and be nested arbitrarily deep. Example LISTs are:

```
{ 1 2 3 4 }
{ 1.2 "string" }
{ 1 2 3 { 1 2 } { 3 4 } }
```

LISTs are used by many commands to provide a more flexible way of passing data to the commands. The `zMatrix` command has two arguments, one of which is a LIST of LISTs where each subLIST contains between three and eight objects.

PARMSETs (Parameter Sets)

PARMSETs are objects that contain bond, angle, torsion, and non-bonding parameters for AMBER force field calculations. They are normally loaded from force field data files, such as *parm94.dat*, and *frmod* files.

ATOMs

ATOMs are complex objects that do not contain any other objects. The ATOM object corresponds to the chemical concept of an atom. Thus, it is a single entity that may be bonded to other ATOMs and used as a building block for creating molecules. ATOMs have many properties that can be changed using the `set` command. These properties are defined below.

name This is a case-sensitive STRING property and it is the ATOM's name. The names for all ATOMs in a RESIDUE should be unique. The name has no relevance to molecular mechanics force field parameters; it is chosen arbitrarily as a means to identify ATOMs. Ideally, the name should correspond to the PDB standard, being 3 characters long except for hydrogens, which can have an extra digit as a 4th character.

type This is a STRING property. It defines the AMBER force field atom type. It is important that the character case match the canonical type definition used in the appropriate force field data (**.dat*) or *frmod* file. For smooth operation, all atom types must have element and hybridization defined by the `addAtomTypes` command. The standard AMBER force field atom types are added by the selected leaprc file.

charge The charge property is a NUMBER that represents the ATOM's electrostatic point charge to be used in a molecular mechanics force field.

element The atomic element provides a simpler description of the atom than the type, and is used only for LEaP's internal purposes (typically when force field information is not available). The element names correspond to standard nomenclature; the character "?" is used for special cases.

position This property is a LIST of NUMBERS. The LIST must contain three values: the (X, Y, Z) Cartesian coordinates of the ATOM.

RESIDUES

RESIDUES are complex objects that contain ATOMs. RESIDUES are collections of ATOMs, and are either molecules (e.g., formaldehyde) or are linked together to form molecules (e.g., amino acid monomers). RESIDUES have several properties that can be changed using the `set` command. (Note that database RESIDUES are each contained within a UNIT having the same name; the residue GLY is referred to as GLY.1 when setting properties. When two of these single-UNIT residues are joined, the result is a single UNIT containing the two RESIDUES.)

One property of RESIDUES is connection ATOMs. Connection ATOMs are ATOMs that are used to make linkages between RESIDUES. For example, in order to create a protein, the N-terminus of one amino acid residue must be linked to the C-terminus of the next residue. This linkage can be made within LEaP by setting the N

ATOM to be a connection ATOM at the N-terminus and the C ATOM to be a connection ATOM at the C-terminus. As another example, two CYX amino acid residues may form a disulfide bridge by crosslinking a connection atom on each residue.

There are several properties of RESIDUEs that can be modified using the `set` command. The properties are described below:

connect0 This defines the first of up to three ATOMs that are used to make links to other RESIDUEs. In UNITs containing single RESIDUEs, the RESIDUE's connect0 ATOM is usually defined as the UNIT's head ATOM. (This is how the standard library UNITs are defined.) For amino acids, the convention is to make the N-terminal nitrogen the connect0 ATOM.

connect1 This defines the second of up to three ATOMs that are used to make links to other RESIDUEs. In UNITs containing single RESIDUEs, the RESIDUE's connect1 ATOM is usually defined as the UNIT's tail ATOM. (This is done in the standard library UNITs.) For amino acids, the convention is to make the C-terminal oxygen the connect1 ATOM.

connect2 This defines the third of up to three ATOMs that are used to make links to other RESIDUEs. In amino acids, the convention is that this is the ATOM to which disulfide bridges are made.

restype This property is a STRING that represents the type of the RESIDUE. Currently, it can have one of the following values: "undefined", "solvent", "protein", "nucleic", or "saccharide". Some of the LEaP commands behave in different ways depending on the type of a residue. For example, the solvate commands require that the solvent residues be of type "solvent". It is important that the proper character case be used when defining this property.

name The RESIDUE name is a STRING property. It is important that the proper character case be used when defining this property.

UNITs

UNITs are the most complex objects within LEaP, and the most important. They may contain RESIDUEs and ATOMs. UNITs, when paired with one or more PARMSETs, contain all of the information required to perform a calculation using AMBER. UNITs can be created using the `createUnit` command. RESIDUEs and ATOMs can be added or deleted from a UNIT using the `add` and `remove` commands. UNITs have the following properties, which can be changed using the `set` command:

head

tail These define the ATOMs within the UNIT that are connected when UNITs are joined together using the sequence command or when UNITs are joined together with the PDB or PREP file reading commands. The tail ATOM of one UNIT is connected to the head ATOM of the next UNIT in any sequence. (Note: a TER card in a PDB file causes a new UNIT to be started.)

box This property can either be null, a NUMBER, or a LIST. The property defines the bounding box of the UNIT. If it is defined as null then no bounding box is defined. If the value is a single NUMBER, the bounding box will be defined to be a cube with each side being *box* Å across. If the value is a LIST, it must contain three NUMBERS, the lengths of the three sides of the bounding box.

cap This property can either be null or a LIST. The property defines the solvent cap of the UNIT. If it is defined as null, no solvent cap is defined. If it is a LIST, it must contain four NUMBERS. The first three define the Cartesian coordinates (X, Y, Z) of the origin of the solvent cap in Å, while the fourth defines the radius of the solvent cap, also in Å.

Examples of setting the above properties are

```
set dipeptide head dipeptide.1.N
set dipeptide box { 5.0 10.0 15.0 }
set dipeptide cap { 15.0 10.0 5.0 8.0 }
```

14. LEaP

The first example makes the amide nitrogen in the first RESIDUE within “dipeptide” the head ATOM. The second example places a rectangular bounding box around the origin with the (X, Y, Z) dimensions of (5.0, 10.0, 15.0) in Å. The third example defines a solvent cap centered at (15.0, 10.0, 5.0) Å with a radius of 8.0 Å. Note: the `set cap` command does not actually solvate, it just sets an attribute. See the `solvateCap` command for a more practical case.

Complex objects and accessing subobjects

UNITS and RESIDUES are complex objects. Among other things, this means that they can contain other objects. There is a loose hierarchy of complex objects and what they are allowed to contain. The hierarchy is as follows:

- UNITS can contain RESIDUES and ATOMS.
- RESIDUES can contain ATOMS.

The hierarchy is loose because it does not forbid UNITS from containing ATOMS directly. However, the convention that has evolved within LEaP is to have UNITS directly contain RESIDUES which directly contain ATOMS.

Objects that are contained within other objects can be accessed using dot “.” notation. An example would be a UNIT which describes a dipeptide ALA-PHE. The UNIT contains two RESIDUES each of which contain several ATOMS. If the UNIT is referenced (named) by the variable `dipeptide`, then the RESIDUE named ALA can be accessed in two ways. The user may type one of the following commands to display the contents of the RESIDUE:

```
desc dipeptide.ALA
desc dipeptide.1
```

The first command translates to “describe some RESIDUE named ALA within the UNIT named dipeptide”. The second form translates as “describe the RESIDUE with sequence number 1 within the UNIT named dipeptide”. The second form is more useful because every subobject within an object is guaranteed to have a unique sequence number. If the first form is used and there is more than one RESIDUE with the name ALA, then an arbitrary residue with the name ALA is returned. To access ATOMS within RESIDUES, either of the following forms of command may be used:

```
desc dipeptide.1.CA
desc dipeptide.1.3
```

Assuming that the ATOM with the name CA has a sequence number 3 within RESIDUE 1, then both of the above commands will print a description of the alpha-carbon of RESIDUE dipeptide.ALA or dipeptide.1. The reader should keep in mind that dipeptide.1.CA is the ATOM, an object, contained within the RESIDUE named ALA within the variable `dipeptide`. This means that dipeptide.1.CA can be used as an argument to any command that requires an ATOM as an argument. However dipeptide.1.CA is not a variable and cannot be used on the left hand side of an assignment statement.

14.3. Running LEaP

```
xleap -h or tleap -h
```

will give a list of command-line arguments (which are very simple). Once you have started either program, typing “help” will display useful information about possible actions.

A file called *leaprc* is executed as a script file at the start of the LEaP session unless the user suppresses it with the `-s` command line option. Sample script files are in `$AMBERHOME/dat/leap/cmd`, and you may wish to copy one of these to become “your” default file. LEaP will look first for a *leaprc* file in the user’s current directory, then in any directories included with `-I` flags.

The command line interface allows the user to specify a log file that is used to log all input and output within the command line environment. The log file is named using the *logFile* command. The file has two purposes: to

allow the user to see a complete record of operations performed by LEaP, and to help recover from (and recreate) program crashes. Output from LEaP commands is written to the log file at a verbosity level of 2 regardless of the verbosity level set by the user using the *verbosity* command. Each line in the log file that was typed in by the user begins with the two characters "> " (a greater-than sign followed by a space). This allows the user to extract the commands typed into LEaP from the log file to create a script file that can be executed using the *source* command. This provides a type of insurance against program crashes by allowing the user to regenerate their interactive sessions. An example of a command that will create a script to reenact a LEaP session is:

```
cat LOGFILE | grep "^> " | sed "s/^> //" > SOURCEFILE.x
```

Note that changes via graphical and table interfaces (xleap) are not captured by command-line traces.

tleap (terminal LEaP) is the non-graphical, command-line-only interface to LEaP. It has the same functionality as the *xleap* main window (Universe Editor Command Window, described below), and uses standard text control keys. *xleap* is a windowing interface to LEaP. In addition to the command-line interface contained in the Universe Editor window, it has a Unit Editor (graphical molecule editor), an Atom Properties Editor, and a Parmset Editor. These editors are discussed in subsequent subsections.

14.3.1. Universe Editor

The window that first appears when the user starts *xleap* is called the Universe Editor. The Universe Editor is the most basic way in which users can interact with *xleap*. It has two parts, the "command window," which corresponds to the *tleap* command interface, and the "pulldown" items above the window, which provide mouse-driven methods to generate specific commands for the command window, either directly or via popped-up dialog boxes. The items in the pulldowns allow the user to generate commands using dialog boxes. To display the "File" pulldown, for example, press the left mouse button on "File;" to select an item in the pulldown, keep the button down, move the mouse to highlight the item, then release the mouse button. A dialog box will then pop up containing fields which the user can fill in, and lists from which values can be chosen; these will be used to generate commands for the command window interface.

14.3.2. Unit Editor

When the user enters the edit command from the Universe Editor Command Window, the Unit Editor will be displayed if the argument to the edit command is an existing UNIT or a nonexistent (i.e. new) object. The Parmset Editor will be activated if the argument is a PARMSET. The Parmset Editor is discussed later in this subsection.

The Unit Editor has five parts. At the top of the window is a pulldown menu bar; below it is a set of buttons titled "Manipulation" that define the mode of mouse activity in the graphics window, and below that, a list of elements to select for the manipulation "Draw" mode (selecting one automatically selects "Draw" mode). Then comes the graphical molecule-editing ("viewing") window itself, and at the very bottom a text window where status and errors are reported.

Unit Editor Menu Bar

The menu bar has three pulldowns: "Unit," "Edit," and "Display."

Unit pulldown The Unit pulldown contains commands affecting the whole UNIT.

- "Check unit" – checks the UNIT in the viewing window for improbable bond lengths, missing force field atom types, close nonbonded contacts, and a non-integral and nonzero total charge. Information is printed in the text window at the bottom of the Unit Editor.
- "Calculate charge" – the total electrostatic charge for the UNIT is displayed in the text window at the bottom of the Unit Editor.
- "Build," "Add H & Build" – the coordinates of new atoms are adjusted according to hybridization (inferred from bonds) and standard geometries. (See also the *Edit* pulldown's "Relax" selection.) Newly-drawn ATOMs are marked as "unbuilt" until they are marked otherwise by one of the Build

commands or by the *Edit* pulldown's "Mark selection (un)built." The builder *only* builds coordinates for unbuilt ATOMs. This allows users to draw molecules piecemeal and make adjustments as they draw, without worrying that the builder is going to undo their work. "Add H & Build" adds hydrogens to the ATOMs that do not have a full valence and builds coordinates for the hydrogens and any other ATOMs that are marked "unbuilt." The number of hydrogens added to each ATOM is determined by the hybridization and element type of each ATOM.

- "Import unit" – a selection window pops up for the user to incorporate a copy of another unit in the current one. The imported unit will generally superimpose on the existing one. (Hint: select all atoms in the current unit before doing this to simplify dragging them apart using the Manipulation *Move* mode.)
- "Close" – Exit the Editor.

Edit pulldown The Edit pulldown contains commands relating to the currently- selected ATOMs in the viewer window. Selection is described below in the "Manipulation buttons" section.

- "Relax selection" – performs a limited energy minimization of all selected ATOMs, leaving unselected ATOMs fixed in place, by relaxing strained bonds, angles, and torsions. If atom types have been assigned and can be found in the currently-loaded force field, force field parameters are used. If no types are available then default parameters are used that are based on ATOM hybridization. This command invokes an iterative algorithm that can take some time to converge for large systems. As the algorithm proceeds, the modified UNIT will be continuously updated within the viewing window. The user can stop the process at any time by placing the mouse pointer within the viewing window and typing control-C. Since only internal coordinates are energy minimized, steric overlap can result.
- "Edit selected atoms" – pops up an Atom Properties Editor, a tool for examining/setting the properties of the selected ATOMs. The Atom Properties Editor allows the user to edit the ATOM names, types and charges in a convenient table format. It is described in a separate subsection below.
- "Flip chirality" – This command inverts the chirality of all selected ATOMs. In order for the chirality to be inverted, the ATOM cannot be in more than one ring. The operation causes the lightest chains leaving the ATOM to be moved so as to invert the chirality. If the ATOM has only three chains attached to it, then only one of the chains will be moved.
- "Select Rings/Residues/Molecules" – expands the currently selected group of atoms to include all partially-contained rings, residues, or molecules.
- "Show everything" – causes all ATOMs to become visible.
- "Hide selection" – makes all selected ATOMs invisible.
- "Show selection only" – makes only selected ATOMs visible.
- "Mark selection unbuilt/built" - see "Unit/Build," above.

Display pulldown The Display pulldown contains commands that determine what information is displayed within the viewing window.

- "Names" – toggles display of ATOM names at each ATOM position.
- "Types" – toggles display of molecular mechanics atom types. The ATOM types are displayed within parentheses "()".
- "Charges" – toggles display of the atomic charges.
- "Residue names" – toggles display of residue names. These are displayed at the position of the first ATOM, before any of that ATOM's information that may be displayed. The residue names are displayed within angled brackets "<>".
- "Axes" – toggles display of the Cartesian coordinate axes. The origin of the axes coincides with the origin of Cartesian space.
- "Periodic box" – toggles display of the periodic box, if the UNIT has one.

Unit Editor manipulation buttons

The Manipulation buttons are Select, Twist, Move, Erase, and Draw. They determine the behavior of the mouse left-button when the mouse pointer is in the Viewing Window.

Select This button allows one to select part or all of a UNIT in anticipation of a subsequent operation or action.

In the *Select* mode, the user can highlight ATOMs within the viewing window for special operations. The mouse pointer becomes a pointing hand in the viewing window in this mode. Selected ATOMs are displayed in a different color (or different line styles on monochrome systems) from all other ATOMs. Atoms can be selected with the left-button in several ways: first, clicking on an atom and releasing selects that atom. Clicking twice in a row on an atom (at any speed) selects all atoms (this is a bug – only the residue should be selected). Keeping the button down and moving to release on another atom selects all ATOMs in the shortest chain between the two ATOMs, if such a chain exists. Finally, by first pressing the button in empty space, and holding it down as the mouse is moved, one can "drag a box" enclosing atoms of interest. Note that a current selection can be expanded by using the "Edit" menubar pulldown select option to complete any partial selection of rings, residues or molecules.

If the user holds down the SHIFT key while performing any of the above actions, the same effect will be seen, except ATOMs will be unselected.

Twist *Twist* mode operates on previously-*Selected* atoms. The intention is to allow rotation about dihedrals; if too many atoms are selected, odd transformations can occur. While in the *Twist* mode, the mouse pointer looks like a curved arrow. Twisting is driven by holding down the left-button anywhere in the viewing window and moving the mouse up and down. It is important to select a complete torsion (all four atoms) before trying to "twist" it.

Move Like *Twist*, *Move* mode operates on previously-*Selected* atoms. While in the *Move* mode, the mouse pointer looks like four arrows coming out of one central point. Holding down the left-button anywhere allows movement of these atoms by dragging in any direction in the viewing plane. (The view can be rotated by holding down the middle-button to allow any movement desired.) This option allows the user to move the selected ATOMs relative to the unselected ATOMs.

To *rotate* the selected ATOMs relative to the unselected ones, press and drag the mode (left) button while holding down the SHIFT key. The selected ATOMs will rotate around a central ATOM on a "virtual sphere" (see the subsection below on the rotate (middle) button for more information on the "virtual sphere"). The user can change which ATOM is used as the center of rotation by clicking the mode (left) button on any of the ATOMs in the window.

Erase *Erase* mode causes the mouse pointer to resemble a chalkboard eraser when it is in the viewing window. Clicking the left-button will delete any atoms or bonds under this mouse pointer, one atom or bond per click.

Draw Choosing *Draw* is equivalent to choosing the default "Elements" atom in the next array of buttons; the initial default is carbon. While in the *Draw* mode, the mouse pointer is a pencil when in the viewing window. Clicking the left-button deposits an atom of the current element, while dragging the mouse pointer with the left-button held down draws a bond: if no atom is found where the button is released, one is created.

When the mouse pointer approaches an ATOM, the end of the line connected to the pointer will "snap" to the nearest ATOM. This is to facilitate drawing of bonds between ATOMs. Any bonds that are drawn will by default be single bonds. To change the order of a bond, the user would move the mouse to any point along the bond and click the mode (left) button. This will cause the order of the bond to increase until it is reset back to a single bond. The user can cycle through the following bond order choices: single, double, triple, and aromatic.

If the user rotates a structure as it is being drawn, she will notice that all of the ATOMs that have been drawn lie in the same plane. New ATOMs are automatically placed in the plane of the screen. The fact that LEaP places the new ATOMs in the same plane is not a handicap because once a rough sketch of part of the structure is complete, the user can invoke one of LEaP's two model building facilities ("Unit/Build" and "Edit/Relax Selection" in the Unit Editor Menu bar) to build full three dimensional coordinates.

Unit Editor Elements Buttons "C, H, O, ..." These buttons put the viewing window in *Draw* mode if it is not in that mode already, and select the drawing element. The more common elements have their own buttons, and all elements are also found by pulling down the *other elements* button.

Unit Editor Viewing Window

The viewing window displays a projection of the UNIT currently being edited. The user can manipulate the structure within the viewing window with the mouse. By moving the mouse and holding down the mouse buttons, the user can rotate, scale, and translate the UNIT within the window. The functions attached to the mouse buttons are:

Rotate (Middle button) By pressing the rotate (middle) button within the viewing window and dragging the mouse, the user can rotate the UNIT around the center of the viewing window. While the rotate (middle) button is down, a circle appears within the viewing window, representing a "virtual sphere trackball." As the user drags the mouse around the outside of the circle, the UNIT will spin around the axis normal to the screen. As the user drags the mouse within the circle, the UNIT will spin around the axis in the screen, perpendicular to the movement of the mouse. The structures that are being viewed can be considered to be embedded within a sphere of glass. The circle is the projection of the edge of the sphere onto the screen. Rotating a UNIT while the mouse is within the circle is akin to placing a hand on a glass sphere and turning the sphere by pulling the hand. The rotate operation does not modify the coordinates of the ATOMs; rather, it simply changes the user's point of view.

Translate (Right button) By pressing the translate (right) button within the viewing window and dragging the mouse around the viewing window, the user can translate the UNIT within the plane of the screen. The structures will follow the mouse as it moves around the window. This operation does not modify the coordinates of the UNIT.

Scale (middle plus right button) If the scale "button" (holding the middle and right buttons down at the same time) is depressed, the user will change the size of the structures within the viewing window. Pressing the scale (middle plus right) button and dragging the mouse up and down the screen will increase and decrease the scale of the structures. This operation does not modify the coordinates of the UNIT.

Mode (left button) The function of the left button is determined by the current mode of the viewing window as described in the "Manipulation" section, above. When the mouse enters the viewing window it changes shape to reflect the current mode of the viewing window.

Spacebar Another always-available operation when the mouse pointer is in the viewing window is the keyboard spacebar. It centers and normalizes the size of the molecule in the viewing window. This is especially useful if the UNIT becomes "lost" due to some operation.

The functions of the middle and right buttons are fixed and always available to the user. This allows the user to change the viewpoint of the UNIT within the viewing window regardless of its current mode. The user might ask why there are controls to translate in the plane of the screen, but not out of the plane of the screen. This is because LEaP does not have depth-cueing or stereo projection and this makes it difficult for users to perceive changes in the depth of a structure. However, the user can rotate the entire UNIT by 90 degrees which will orient everything so that the direction that was coming out of the screen becomes a direction lying in the plane of the screen. Once the UNIT has been rotated using the rotate (middle) button, the user can translate the structure anywhere in space. While it does take some getting used to, users can become very adept at the combination of rotations and translations.

14.3.3. Atom Properties Editor

The Atom Properties Editor is popped up by the Unit Editor when the user selects the *Edit selected atoms* command from the *Edit* pulldown. The Atom Properties Editor allows the user to edit the properties of ATOMs using a convenient table format. ATOM properties are: name, type, charge, and element.

14.3.4. Parmset Editor

If the user enters the command *edit Foo* in the Universe Editor and *Foo* is a PARMSET, then a Parmset Editor is popped up. First, a window appears which contains a number of buttons. The buttons list the parameters that can be edited – Atom, Bond, Angle, Proper Torsion, Improper Torsion, and Hydrogen Bond – and an option to close the editor. Choosing one of the parameter buttons will pop up a Table Editor. This editor resembles that of the Atom Properties Editor, having three parts: the Menu Bar, Status Window, and Table Window.

14.4. Basic instructions for using LEaP to build molecules

This section gives an overview of how LEaP is most commonly used. Detailed descriptions of all the commands are given in the next section.

14.4.1. Building a Molecule For Molecular Mechanics

In order to prepare a molecule within LEaP for AMBER, three basic tasks need to be completed.

1. Any needed UNIT or PARMSET objects must be loaded;
2. The molecule must be constructed within LEaP;
3. The user must output topology and coordinate files from LEaP to use in AMBER.

The most typical command sequence is the following:

```
source leaprc.protein.ff14SB (load a force field)
x = loadPdb trypsin.pdb (load in a structure)
.... add in cross-links, solvate, etc.
saveAmberParm x prmtop prmcrd (save files)
```

There are a number of variants of this:

1. Although `loadPdb` is by far the most common way to enter a structure, one might use `loadOff`, or `loadAmberPrep`, or use the `zMatrix` command to build a molecule from a Z-matrix. For small molecules, e.g., ligand like, `loadMol2` or `loadMol3` are available. See the Commands section below for descriptions of these options. If you do not have a starting structure (in the form of a PDB file), LEaP can be used to build the molecule; you will find, however, that this is not always a straightforward process. Many experienced Amber users turn to other (commercial and non-commercial) programs to create their initial structures.
2. Be very attentive to any errors produced in the `loadPdb` step; these generally mean that LEaP has misread the file. A general rule of thumb is to keep editing your input PDB file until LEaP stops complaining. It is often convenient to use the `addPdbAtomMap` or `addPdbResMap` commands to make systematic changes from the names in your PDB files to those in the Amber topology files; see the `leaprc` files in `$AMBERHOME/dat/leap/cmd` for examples of this. *Be sure to read Section 13 for information on how to “clean up” PDB files before loading them.*
3. The `saveAmberParm` command cited above is appropriate for most force fields; for polarizable calculations you will need to use `saveAmberParmPol`.

14.4.2. Amino Acid Residues

For each of the amino acids found in the LEaP libraries, there has been created an N-terminal and a C-terminal analog. The N-terminal amino acid UNIT/RESIDUE names and aliases are prefaced by the letter N (e.g., NALA) and the C-terminal amino acids by the letter C (e.g., CALA). If the user models a peptide or protein within LEaP, they may choose one of three ways to represent the terminal amino acids. The user may use (1) standard amino

14. LEaP

acids, (2) protecting groups (ACE/NME), or (3) the charged C- and N-terminal amino acid UNITS/RESIDUES. If the standard amino acids are used for the terminal residues, then these residues will have incomplete valences. These three options are illustrated below:

```
{ ALA VAL SER PHE }  
{ ACE ALA VAL SER PHE NME }  
{ NALA VAL SER CPHE }
```

The default for loading from PDB files is to use N- and C-terminal residues; this is established by the `addPdbResMap` command in the standard `leaprc` files. To force incomplete valences with the standard residues, one would have to define a sequence (“`x = { ALA VAL SER PHE }`”) and use `loadPdbUsingSeq`, or use `clearPdbResMap` to completely remove the mapping feature.

Histidine can exist either as the protonated species or as a neutral species with a hydrogen at the δ or ϵ position. For this reason, the histidine UNIT/RESIDUE name is either HIP, HID, or HIE (but not HIS). The standard `leaprc` files assign the name HIS to HIE. Thus, if a PDB file is read that contains the residue HIS, the residue will be assigned to the HIE UNIT object. This feature can be changed within one’s own `leaprc` file.

The AMBER force fields also differentiate between the residue cysteine (CYS) and the similar residue which participates in disulfide bridges, cystine (CYX). The user will have to explicitly define, using the `bond` command, the disulfide bond for a pair of cystines, as this information is not read from the PDB file. In addition, the user will need to load the PDB file using the `loadPdbUsingSeq` command, substituting CYX for CYS in the sequence wherever a disulfide bond will be created.

14.4.3. Nucleic Acid Residues

The “D” prefix can be used to distinguish between deoxyribose and ribose units. Residue names like “A” or “DA” can be followed by a “5” or “3” (“DA5”, “DA3”) for residues at the ends of chains; this is also the default established by `addPdbResMap`, even if the “5” or “3” are not added in the PDB file. The “5” and “3” residues are “capped” by a hydrogen; the plain and “3” residues include a “leading” phosphate group. Neutral residues (nucleosides) capped by hydrogens end their names with “N”, as in “DAN”.

14.5. Error Handling and Reporting

In Amber version 18 changes were made to LEaP’s error processing. The first set of changes involve error handling. For input from a file (i.e., `tLeap` invoked with `-f`) execution is now terminated at the first occurrence of these errors: file input/output errors, illegal command syntax, illegal command arguments, and some command parsing errors. The intent is to simplify error detection and to ease troubleshooting. For interactive input there is no change in handling: LEaP continues to be forgiving of these errors in the hope that the user can recover in real time.

The final set of changes involve error reporting. LEaP produces four kinds of messages: errors, warnings, notes, and processing messages. Messages beginning with “Fatal Error!” or “Error!” or “Error:” indicate a serious problem. Messages beginning with “Warning!” or “Warning:” indicate a potential problem that should be investigated. Messages beginning with “Note.” or “Note:” provide information worth noting. Messages that are not designated by one of the above tags report processing status. Total counts of errors, warnings, and notes are outputted at the end of LEaP. The intent is to simplify error detection by emitting clear and consistent messages.

As with all computational software, LEaP’s output should be carefully examined. Some error and warning messages mention likely causes or contain suggested workarounds, but all such messages provide clues. Apply common sense and the scientific method to troubleshoot. Typical first steps are to verify input files and to search the AMBER Mail Reflector for similar reported problems. Note that LEaP normally produces a log file that contains all messages and more detailed output that can be inspected.

14.5.1. Known Issues

This is an incomplete list, started with Amber version 24, focussing on incompatibilities with standards, significant bugs, and gotchas in *LEaP*:

- LEaP does not write Mol2 files that are compliant with Sybyl bond types in the @<TRIPOS>BOND record. As of Amber version 24, single, double, and triple bond types are correctly emitted; the number 4 is used to denote aromatic bonds. In previous versions, the number 1 was always emitted.
- A workaround exists for the num lock issue in xLEaP; contact the Amber mailing list for details.

Of course other issues should be reported via the Amber mailing list.

14.6. Commands

The following is a description of the commands that can be accessed using the command line interface in *tleap*, or through the command line editor in *xleap*. Whenever an argument in a command line definition is enclosed in square brackets (e.g., [arg]), then that argument is optional. When examples are shown, the command line is prefaced by “>”, and the program output is shown without this character preface.

Some commands that are almost never used have been removed from this description to save space. You can use the “help” facility to obtain information about these commands; most only make sense if you understand what the program is doing behind the scenes.

14.6.1. add

add a b

UNIT/RESIDUE/ATOM a,b

Add the object b to the object a. This command is used to place ATOMs within RESIDUEs, and RESIDUEs within UNITs. This command will work only if b is not contained by any other object.

The following example illustrates both the add command and the way the TIP3P water molecule is created for the LEaP distribution.

```
> h1 = createAtom H1 HW 0.417
> h2 = createAtom H2 HW 0.417
> o = createAtom O OW -0.834
>
> set h1 element H
> set h2 element H
> set o element O
>
> r = createResidue TIP3
> add r h1
> add r h2
> add r o
>
> bond h1 o
> bond h2 o
> bond h1 h2
>
> TIP3 = createUnit TIP3
>
> add TIP3 r
> set TIP3.1 retype solvent
> set TIP3.1 imagingAtom TIP3.1.O
>
> zMatrix TIP3 {
> { H1 O 0.9572 }
> { H2 O H1 0.9572 104.52 }
> }
>
```

```
> saveOff TIP3 water.lib
Saving TIP3.
Building topology.
Building atom parameters.
```

14.6.2. addAtomTypes

```
addAtomTypes { { type element hybrid } { ... } ... }
```

Define element and hybridization for force field atom types. This command for the standard force fields can be seen in the standard leaprc files. The STRINGS are most safely rendered using quotation marks. If atom types are not defined, confusing messages about hybridization can result when loading PDB files.

14.6.3. addC4Pairwise

```
addC4Pairwise unit atom1 atom2 C4Value(kcal/mol*A^4)
```

Adds pairwise C4 interaction between two specific atoms. C4 interaction means the non-bonded force between *atom1* and *atom2* will follow 12-6-4 pattern. Note here this 12-6-4 pattern is atom-specific, not atom-type-specific. This is achieved by generating LENNARD_JONES_DCOEF and LENNARD_JONES_DVALUE flags in the prmtop file. Under LENNARD_JONES_DCOEF flag, each row will contain the ID of *atom1*, the ID of *atom2*, and a numeric index for debugging purpose. Under LENNARD_JONES_DVALUE flag, each row will contain the C4Value. Multiple addC4Pairwise commands are supported in a single tleap file.

14.6.4. addC4Type

```
addC4Type unit type1 type2 C4Value(kcal/mol*A^4)
```

Similar to add12_6_4 in ParmEd, this function will generate a LENNARD_JONES_CCOEF flag in the prmtop file. Under that flag, a C4 matrix will be presented with mostly zero values, except for the entries that represent non-zero C4Values between *type1* and *type2*. Multiple addC4Type commands are supported in a single tleap file.

14.6.5. addIons and addIons2

```
addIons unit ion1 numIon1 [ion2 numIon2]
addIons2 unit ion1 numIon1 [ion2 numIon2]
```

Adds counterions in a shell around *unit* using a Coulombic potential on a grid. If *numIon1* is 0 then the unit is neutralized. In this case, *ion1* must be opposite in charge to *unit* and *ion2* must not be specified. Otherwise, the specified numbers of *ion1* [*ion2*] are added [in alternating order]. If solvent is present, it is ignored in the charge and steric calculations, and if an ion has a steric conflict with a solvent molecule, the ion is moved to the center of that solvent molecule, and the latter is deleted. (To avoid this behavior, either solvate *_after_* *addIons*, or use *addIons2*.) Ions must be monatomic.

This procedure is not guaranteed to globally minimize the electrostatic energy. When neutralizing regular-backbone nucleic acids, the first cations will generally be placed between phosphates, leaving the final two ions to be placed somewhere around the middle of the molecule. The default grid resolution is 1 Å, extending from an inner radius of (*maxIonVdwRadius* + *maxSoluteAtomVdwRadius*) to an outer radius 4 Å beyond. A distance-dependent dielectric is used for speed. *addIons2* is the same as *addIons*, except solvent and solute are treated the same. Algorithms for determining the number of ions to add, based on a desired salt concentration, are given in Refs. [440, 441].

14.6.6. addIonsRand

```
addIonsRand unit ion1 numIon1 [ion2 numIon2] [separation]
```

Adds counterions in a shell around *unit* by replacing random solvent molecules. If *numIon1* is 0, the unit is neutralized (*ion1* must be opposite in charge to *unit*, and *ion2* cannot be specified). Otherwise, the specified numbers of *ion1* [*ion2*] are added [in alternating order]. If *separation* is specified, ions will be guaranteed to be more than that distance apart in Angstroms. Ions must be monoatomic. Solvent must be present. It must be possible to position the requested number of ions with the given separation in the solvent.

This procedure is much faster than `addIons`, as it does not calculate charges. The randomness depends on the invocation of the LEaP executable. In particular, if one creates three copies of a unit and performs in a single LEaP script `addIonsRand` on each copy then the three resulting solvated units will be identical. However, if one invokes LEaP three times on a script that performs one `addIonsRand` on the unit and saves a pdb of that unit then three different pdb files will be created. Algorithms for determining the number of ions to add, based on a desired salt concentration, are given in Refs.[440, 441].

14.6.7. addPath

```
addPath path
```

Add the directory in *path* to the list of directories that are searched for files specified by other commands. The following example illustrates this command.

```
> addPath /disk/howard
/disk/howard added to file search path.
```

After the above command is entered, the program will search for a file in this directory if a file is specified in a command. Thus, if a user has a library named “/disk/howard/rings.lib” and the user wants to load that library, one only needs to enter `load rings.lib` and not `load /disk/howard/rings.lib`.

14.6.8. addPdbAtomMap

```
addPdbAtomMap list
```

The atom Name Map is used to try to map atom names read from PDB files to atoms within residue UNITS when the atom name in the PDB file does not match an atom in the residue. This enables PDB files to be read in without extensive editing of atom names. Typically, this command is placed in the LEaP startup file, “leaprc”, so that assignments are made at the beginning of the session. *list* should be a LIST of LISTS. Each sublist should contain two entries to add to the Name Map. Each entry has the form:

```
{ string string }
```

where the first string is the name within the PDB file, and the second string is the name in the residue UNIT.

14.6.9. addPdbResMap

```
addPdbResMap list
```

The Name Map is used to map RESIDUE names read from PDB files to variable names within LEaP. Typically, this command is placed in the LEaP startup file, “leaprc”, so that assignments are made at the beginning of the session. The LIST is a LIST of LISTS. Each sublist contains two or three entries to add to the Name Map. Each entry has the form:

```
{ double string1 string2 }
```

where *double* can be 0 or 1, *string1* is the name within the PDB file, and *string2* is the variable name to which *string1* will be mapped. To illustrate, the following is part of the Name Map that exists when LEaP is started with a standard leaprc file:

14. LEaP

```
ADE --> DADE
: :
0 ALA --> NALA
0 ARG --> NARG
: :
1 ALA --> CALA
1 ARG --> CARG
: :
1 VAL --> CVAL
```

Thus, the residue ALA will be mapped to NALA if it is the N-terminal residue and CALA if it is found at the C-terminus. The above Name Map was produced using the following (edited) command line:

```
> addPdbResMap {
> { 0 ALA NALA } { 1 ALA CALA }
> { 0 ARG NARG } { 1 ARG CARG } : :
> { 0 VAL NVAL } { 1 VAL CVAL }
> : :
> { ADE DADE } : :
> }
```

14.6.10. alias

```
alias [ string1 [ string2 ] ]
```

This command will add or remove an entry to the Alias Table or list entries in the Alias Table. If both strings are present, then *string1* becomes the alias to *string2*, the original command. If only one string is used as an argument, then that string will be removed from the Alias Table. If no arguments are given to the command, the current aliases stored in the Alias Table will be listed.

The proposed alias is first checked for conflict with the LEaP commands and rejected if a conflict is found. A proposed alias will replace an existing alias with a warning being issued. The alias can stand for more than a single word, but also as an entire string so the user can quickly repeat entire lines of input.

14.6.11. bond

```
bond atom1 atom2 [ order ]
```

Create a bond between *atom1* and *atom2*. Both of these ATOMs must be contained by the same UNIT. By default, the bond will be a single bond. By specifying “-”, “=”, “#”, or “:” as the optional argument, order, the user can specify a single, double, triple, or aromatic bond, respectively. Example:

```
bond trx.32.SG trx.35.SG
```

14.6.12. bondByDistance

```
bondByDistance container [ maxBond ]
```

Create single bonds between all ATOMs in the UNIT *container* that are within *maxBond* Å of each other. If *maxBond* is not specified, a default distance will be used. This command is especially useful in building molecules. Example:

```
bondByDistance alkylChain
```


14.6.13. check

```
check unit [ parms ]
```

This command can be used to check *unit* for internal inconsistencies that could cause problems when performing calculations. This is a very useful command that should be used before a UNIT is saved with `saveAmberParm` or its variants. Currently it checks for the following possible problems:

- long bonds
- short bonds
- non-integral total charge of the UNIT
- missing force field atom types
- close contacts ($< 1.5 \text{ \AA}$) between nonbonded ATOMs

The user may collect any missing molecular mechanics parameters in a PARMSET for subsequent editing. In the following example, the alanine UNIT found in the amino acid library has been examined by the check command:

```
> check ALA
Checking 'ALA' ....
Checking parameters for unit 'ALA'.
Checking for bond parameters.
Checking for angle parameters.
Unit is OK.
```

14.6.14. combine

```
variable = combine list
```

Combine the contents of the UNITs within *list* into a single UNIT. The new UNIT is placed in *variable*. This command is similar to the sequence command except it does not link the ATOMs of the UNITs together. In the following example, the input and output should be compared with the example given for the sequence command.

```
> tripeptide = combine { ALA GLY PRO }
Sequence: ALA
Sequence: GLY
Sequence: PRO
> desc tripeptide
UNIT name: ALA
Head atom: .R<ALA 1>.A<N 1>
Tail atom: .R<PRO 3>.A<C 13>
Contents:
R<ALA 1>
R<GLY 2>
R<PRO 3>
```

14.6.15. copy

```
newvariable = copy variable
```

In most cases, creates an exact duplicate of the object *variable*. Since *newvariable* is not pointing to the same object as *variable*, changing the contents of one object will not alter the other object. Example:

14. LEaP

```
> tripeptide = sequence { ALA GLY PRO }
> tripeptideSol = copy tripeptide
> solvateBox tripeptideSol TIP3PBOX 8 2
```

In the above example, *tripeptide* is a separate object from *tripeptideSol* and is not solvated. Had the user instead entered

```
> tripeptide = sequence { ALA GLY PRO }
> tripeptideSol = tripeptide
> solvateBox tripeptideSol TIP3PBOX 8 2
```

then both *tripeptide* and *tripeptideSol* would be solvated since they would both refer to the same object.

Note that in a few instances, the copy command does not produce an exact copy. This is particularly relevant when making copies of oligosaccharide residues. In these, the copy command invariably inverts chirality at the anomeric carbon. The workaround for this is to use the copy command twice, where the second call inverts the chirality back.

14.6.16. createAtom

```
variable = createAtom name type charge
```

Return a new and empty ATOM with *name*, *type*, and *charge* as its atom name, atom type, and electrostatic point charge. (See the add command for an example of the `createAtom` command.)

14.6.17. createResidue

```
variable = createResidue name
```

Return a new and empty RESIDUE with the name *name*. (See the add command for an example of the `createResidue` command.)

14.6.18. createUnit

```
variable = createUnit name
```

Return a new and empty UNIT with the name *name*. (See the add command for an example of the `createUnit` command.)

14.6.19. deleteBond

```
deleteBond atom1 atom2
```

Delete the bond between the ATOMs *atom1* and *atom2*. If no bond exists, an error will be displayed.

14.6.20. desc

```
desc variable
```

Print a description of the object *variable*. In the following example, the alanine UNIT found in the amino acid library has been examined by the `desc` command:

```
> desc ALA
UNIT name: ALA
Head atom: .R<ALA 1>.A<N 1>
Tail atom: .R<ALA 1>.A<C 9>
Contents: R<ALA 1>
```

Now, the `desc` command is used to examine the first residue (1) of the alanine UNIT:

```
> desc ALA.1
RESIDUE name: ALA
RESIDUE sequence number: 1
Type: protein
Connection atoms:
Connect atom 0: A<N 1>
Connect atom 1: A<C 9>
Contents:
A<N 1>
A<HN 2>
A<CA 3>
A<HA 4>
A<CB 5>
A<HB1 6>
A<HB2 7>
A<HB3 8>
A<C 9>
A<O 10>
```

Next, we illustrate the `desc` command by examining the ATOM N of the first residue (1) of the alanine UNIT:

```
> desc ALA.1.N
ATOM Name: N
Type: N
Charge: -0.463
Element: N
Atom flags: 20000|posfxd- posblt- posdrn- sel- pert- notdisp- tchd-
             posknwn+ int - nmin- nbld-
Atom position: 3.325770, 1.547909, -0.000002
Atom velocity: 0.000000, 0.000000, 0.000000
Bonded to .R<ALA 1>.A<HN 2> by a single bond.
Bonded to .R<ALA 1>.A<CA 3> by a single bond.
```

Since the N ATOM is also the first atom of the ALA residue, the following command will give the same output as the previous example:

```
> desc ALA.1.1
```

14.6.21. groupSelectedAtoms

```
groupSelectedAtoms unit name
```

Create a group within *unit* with the name *name*, using all of the ATOMs within *unit* that are selected. If the group has already been defined then overwrite the old group. The `desc` command can be used to list groups. Example:

```
groupSelectedAtoms TRP sideChain
```

An expression like “TRP@sideChain” returns a LIST, so any commands that require LISTS can take advantage of this notation. After assignment, one can access groups using the “@” notation. Examples:

```
select TRP@sideChain
center TRP@sideChain
```

The latter example will calculate the center of the atoms in the “sideChain” group. (See the `select` command for a more detailed example.)

14.6.22. help

```
help [string]
```

This command prints a description of the command in *string*. If no argument is given, a list of help topics is provided.

14.6.23. impose

```
impose unit seqlist internals
```

The `impose` command allows the user to impose internal coordinates on *unit*. The list of RESIDUEs to impose the internal coordinates upon is in *seqlist*. The internal coordinates to impose are in *internals*, which is an object of type LIST.

The command works by looking into each RESIDUE within *unit* that is listed in *seqlist* and attempts to apply each of the internal coordinates within *internals*. The *seqlist* argument is a LIST of NUMBERS that represent sequence numbers or ranges of sequence numbers. A range of sequence numbers is represented by two element LISTS that contain the first and last sequence number in the range. The user can specify sequence number ranges that are larger than what is found in *unit*, in which case the range will stop at the beginning or end of *unit* as appropriate. For example, the range { 1 999 } will include all RESIDUEs in a 200 RESIDUE UNIT.

The *internals* argument is a LIST of LISTS. Each sublist contains a sequence of ATOM names which are of type STRING followed by the value of the internal coordinate. An example of the `impose` command would be:

```
impose peptide { 1 2 3 } { { ``N'' ``CA'' ``C'' ``N'' -40.0 } { ``C'' ``N'' ``CA'' ``C''
```

This would cause the RESIDUE with sequence numbers 1, 2, and 3 within the UNIT *peptide* to assume an α -helical conformation. The command

```
impose peptide { 1 2 { 5 10 } 12 } { { ``CA'' ``CB'' 5.0 } }
```

will impose on the residues with sequence numbers 1, 2, 5, 6, 7, 8, 9, 10, and 12 within the UNIT *peptide* a bond length of 5.0 Å between the α and β carbon atoms. RESIDUEs without an ATOM named CB, such as glycine, will be unaffected.

It is important to understand that the `impose` command attempts to perform the intended action on all residues in the *seqlist*, but does not necessarily limit itself to acting only upon *internals* contained within those residues. That is, the list does not limit the residues to consider. Rather, it is a list of all starting points to consider. In other words, to specify a *seqlist* of { 3 4 } tells *impose* to attempt to set two torsions, one starting in residue 3 and the other starting in residue 4. It does not specify that the torsion should only be set if the atoms are found within residues 3 and/or 4.

Because of this, one must be careful when setting torsions between two residues. It is necessary to know which atoms are contained in which residues. Consider the following trisaccharide:



To build it most simply in LEaP requires the following directive. Note that the build order in LEaP is the reverse of the standard order in which the residues are written above.

```
glycan = sequence { ROH 6LB 6MB OGA }
```

A proper build of a 1-6 oligosaccharide linkage often requires setting three torsions. In the manner that residues are defined in the Glycam force fields, the atoms describing two of those torsions, ϕ and ψ , span two residues. However, the atoms in the third, ω , exist entirely within one residue. In fact, they exist within all three glycan residues in the example above. The following commands will set only the three torsions in the glycosidic linkage between residues 4 (OGA) and 3 (6MB).

```
impose glycan { 4 } { { 'H1' 'C1' 'O6' 'C6' -60.0 } } # O6 & C6 are in resid
impose glycan { 4 } { { 'C1' 'O6' 'C6' 'C5' 180.0 } } # only C1 is in residu
impose glycan { 3 } { { 'O6' 'C6' 'C5' 'O5' 60.0 } } # all are in residue 3
```

The common misconception that the *seqlist* sets a limit on the residues affected can cause trouble in this case. For example, this command

```
impose glycan { 4 3 } { { 'H1' 'C1' 'O6' 'C6' -60.0 } }
```

will find all sequences beginning in residue 4 and in residue 3 that contain the serially bonded atoms H1 C1 O6 and C6. Therefore, in this case, it will set the specified torsions between residues 4 and 3 as well as between 3 and 2. Similarly, this command

```
impose peptide { 4 } { { 'O6' 'C6' 'C5' 'O5' 60.0 } }
```

will not affect any inter-residue linkage, but instead will set the C5-C6 torsion in the glucopyranoside (OGA) at the non-reducing end of the oligosaccharide.

The ordering and content within the *internals* list is important as well. For these examples, consider the simple peptide sequence:

```
peptide = sequence { ALA ALA ALA ALA }
```

The ordering of the *internals* specifies the atoms to which the torsion set is applied. The *impose* command will find the first atom in the *internals* list, check for the presence of a bonded second atom, and so forth. It will then apply the action, here a torsion, to those four atoms. For example, this command:

```
impose peptide { 3 } { { 'N' 'CA' 'C' 'N' -40.0 } } # between 3 and 4
```

will set the torsion between residues 3 and 4. However, this one:

```
impose peptide { 3 } { { 'N' 'C' 'CA' 'N' -40.0 } } # between 3 and 2
```

will set the torsion between residues 3 and 2.

If at any point, the *impose* command does not find an atom bonded to a previous atom in an *internals* list, it will silently ignore the command. This is likely to occur in two instances. One, the atom simply might not exist in the residue:

```
impose peptide { 3 } { { 'N' 'CA' 'CB' 'HB4' 10.0 } } # no effect, silent
```

Here, of course, there is no atom named HB4 in alanine. Similarly, improper torsions are ignored. For example, this command also has no effect:

```
impose peptide { 3 } { { 'N' 'HB1' 'CA' 'CB' 10.0 } } # no effect, silent
```

because HB1 is not bonded to N.

Three types of conformational change are supported: Bond length changes, bond angle changes, and torsion angle changes. If the conformational change involves a torsion angle, then all dihedrals around the central pair of atoms are rotated. The entire list of internals is applied to each RESIDUE.

It is also important to note that the *impose* command performs its actions entirely using internal coordinates. Because of this, it is difficult to predict the resulting behavior when the coordinates are translated back to cartesian, for example when writing a PDB file.

14.6.24. list

List all of the variables currently defined. To illustrate, the following (edited) output shows the variables defined when LEaP is started with a standard leaprc file:

```
> list A ACE ALA ARG ASN : : VAL W WAT Y
```

14.6.25. loadAmberParams

```
variable = loadAmberParams filename
```

Load an AMBER format parameter set file and place it in *variable*. All interactions defined in the parameter set will be contained within *variable*. This command causes the loaded parameter set to be included in LEaP's list of parameter sets that are searched when parameters are required. General proper and improper torsion parameters are modified during the command execution with the LEaP general type "?" replacing the AMBER general type "X"

```
> parm91 = loadAmberParams parm91X.dat
> saveOff parm91 parm91.lib
```

14.6.26. loadAmberPrep

```
loadAmberPrep filename [ prefix ]
```

This command loads an AMBER PREP input file. For each residue that is loaded, a new UNIT is constructed that contains a single RESIDUE and a variable is created with the same name as the name of the residue within the PREP file. If the optional argument *prefix* (a STRING) is provided, its contents will be prefixed to each variable name; this feature is used to prefix UATOM residues, which have the same names as AATOM residues with the string "U" to distinguish them.

```
> loadAmberPrep cra.in
Loaded UNIT: CRA
```

14.6.27. loadOff

```
loadOff filename
```

This command loads the OFF library within the file named *filename*. All UNITS and PARMSETs within the library will be loaded. The objects are loaded into LEaP under the variable names the objects had when they were saved. Variables already in existence that have the same names as the objects being loaded will be overwritten. Any PARMSETs loaded using this command are included in LEaP's library of PARMSETs that is searched whenever parameters are required (the old AMBER format is used for PARMSETs rather than the OFF format in the default configuration). Example command line:

```
> loadOff parm91.lib
Loading library: parm91.lib
Loading: PARAMETERS
```

14.6.28. loadMol2

```
variable = loadMol2 filename
```

Load a Sybyl MOL2 format file into *variable*, a UNIT. This command is very much like `loadOff`, except that it only creates a single UNIT.

14.6.29. loadPdb

```
variable = loadPdb filename
```

Load a Protein Data Bank (PDB) format file with the file name *filename* into *variable*, a UNIT. The sequence numbers of the RESIDUES will be determined from the order of residues within the PDB file ATOM records. This function will search the variables currently defined within LEaP for variable names that map to residue names within the ATOM records of the PDB file. If a matching variable name is found then the contents of the variable are added to the UNIT that will contain the structure being loaded from the PDB file. Adding the contents of the matching UNIT into the UNIT being constructed means that the contents of the matching UNIT are copied into the UNIT being built and that a bond is created between the connect0 ATOM of the matching UNIT and the connect1 ATOM of the UNIT being built. (This bond creation does not occur if a PDB ‘TER’ card separates the atoms. As of AmberTools21 a PDB TER record is also used to detect a new residue in the case of contiguous residues with identical residue sequence numbers.) The UNITS are combined in the same way UNITS are combined using the sequence command. As atoms are read from the ATOM records their coordinates are written into the correspondingly named ATOMs within the UNIT being built. If the entire residue is read and it is found that ATOM coordinates are missing, then external coordinates are built from the internal coordinates that were defined in the matching UNIT. This allows LEaP to build coordinates for hydrogens and lone-pairs which are not specified in PDB files. Note that the standard leaprc files include commands to establish automatic N- and C-termination of amino acid sequences and 5’ and 3’ termination of nucleic acid sequences.

```
> crambin = loadPdb 1crn
```

14.6.30. loadPdbUsingSeq

```
loadPdbUsingSeq filename unitlist
```

This command reads a PDB format file named *filename*. This command is identical to `loadPdb` except it does not use the residue names within the PDB file. Instead, the sequence is defined by the user in *unitlist*. For more details see `loadPdb`.

```
> peptSeq = { UALA UASN UILE UVAL UGLY }
> pept = loadPdbUsingSeq pept.pdb peptSeq
```

In the above example, a variable is first defined as a LIST of united atom RESIDUES. A PDB file is then loaded, in this sequence order, from the file “pept.pdb”.

14.6.31. logFile

```
logFile filename
```

This command opens the file with the file name *filename* as a log file. User input and all output is written to the log file. Output is written to the log file as if the verbosity level were set to 2. An example of this command is

```
> logfile /disk/howard/leapTrpSolvate.log
```

14.6.32. measureGeom

```
measureGeom atom1 atom2 [ atom3 [ atom4 ] ]
```

Measure the distance, angle, or torsion between two, three, or four ATOMs, respectively.

In the following example, we first describe the RESIDUE ALA of the ALA UNIT in order to find the identity of the ATOMs. Next, the `measureGeom` command is used to determine a distance (determining simple angles and dihedral angles are straightforward extensions). As shown in the example, the ATOMs may be identified using atom names or numbers.

14. LEaP

```
> desc ALA.ALA
RESIDUE name: ALA
RESIDUE sequence number: 1
Type: protein ....
> measureGeom ALA.ALA.3 ALA.ALA.CB
Distance: 1.52 angstroms
```

14.6.33. quit

Quit the LEaP program.

14.6.34. remove

remove container item

Remove the object *item* from the object *container*. If *container* does not contain *item*, an error message will be displayed. This command is used to remove ATOMs from RESIDUEs, and RESIDUEs from UNITs. If the object represented by *item* is not referenced by any other variable name, it will be destroyed.

```
> dipeptide = combine { ALA GLY }
Sequence: ALA
Sequence: GLY
> desc dipeptide
UNIT name: ALA
Head atom: .R<ALA 1>.A<N 1>
Tail atom: .R<GLY 2>.A<C 6>
Contents: R<ALA 1> R<GLY 2>
> remove dipeptide dipeptide.2
> desc dipeptide UNIT name: ALA
Head atom: .R<ALA 1>.A<N 1>
Tail atom: null
Contents: R<ALA 1>
```

14.6.35. saveAmberParm

saveAmberParm unit topologyfilename coordinatefilename

Save the Amber/NAB topology and coordinate files for *unit* into the files named *topologyfilename* and *coordinatefilename* respectively. This command will cause LEaP to search its list of PARMSETs for parameters defining all of the interactions between the ATOMs within *unit*. It produces topology files and coordinate files that are identical in format to those produced by Amber PARM and can be read into Amber and NAB for calculations. The output of this operation can be used for minimizations, dynamics, and thermodynamic perturbation calculations.

In the following example, the topology and coordinates from the all_amino94.lib UNIT ALA are generated:

```
> saveamberparm ALA ala.top ala.crd
```

14.6.36. saveMol2

saveMol2 unit filename type-flag

Write *unit* to the file *filename* as a Tripos mol2 format file. If *type-flag* is 0, the Tripos (Sybyl) atom types will be used; if *type-flag* is 1, the Amber atom types present in *unit* will be used. Generally, you would want to set *type-flag* to 1, unless you need the Sybyl atom types for use in some program outside Amber; Amber itself has no force fields that use Sybyl atom types.

14.6.37. saveOff

saveOff object filename

The `saveOff` command allows the user to save UNITS and PARMSETs to a file named *filename*. The file is written using the Object File Format (off) and can accommodate an unlimited number of uniquely named objects. The names by which the objects are stored are the variable names specified within the *object* argument. If the file *filename* already exists, the new objects will be added to it. If there are objects within the file with the same names as objects being saved then the old objects will be overwritten. The argument *object* can be a single UNIT, a single PARMSET, or a LIST of mixed UNITS and PARMSETs. (See the `add` command for an example of the `saveOff` command.)

14.6.38. savePdb

savePdb unit filename

Write *unit* to the file *filename* as a PDB format file. In the following example, the PDB file from the ALA unit is generated:

```
> savepdb ALA ala.pdb
```

Warning: The PDB-like file created with this command is primarily useful for reading back into *tleap*, or for other Amber-related uses. It is consistent with Amber, but not with other aspects of the PDB standard (e.g. in atom and residue names, etc.) Use the *ambpdb* program (see Section 36.1) if you need a file that more fully complies with the PDB standard.

14.6.39. sequence

variable = sequence list

The `sequence` command is used to combine the contents of *list*, which should be a LIST of UNITS, into a new, single UNIT. This new UNIT is constructed by taking each UNIT in *list* in turn and copying its contents into the UNIT being constructed. As each new UNIT is copied, a bond is created between the tail ATOM of the UNIT being constructed and the head ATOM of the UNIT being copied, if both connect ATOMs are defined. If only one is defined, a warning is generated and no bond is created. If neither connection ATOM is defined then no bond is created. As each RESIDUE is copied into the UNIT being constructed it is assigned a sequence number which represents the order the RESIDUES are added. Sequence numbers are assigned to the RESIDUES so as to maintain the same order as was in the UNIT before it was copied into the UNIT being constructed. This command builds reasonable starting coordinates for all ATOMs within the UNIT; it does this by assigning internal coordinates to the linkages between the RESIDUES and building the external coordinates from the internal coordinates from the linkages and the internal coordinates that were defined for the individual UNITS in the sequence.

```
> tripeptide = sequence { ALA GLY PRO }
```

14.6.40. set

This command operates in two modes. In the first, it sets default values for some parameters. In the second, it sets specific properties to containers (for example, UNITS).

Defaults can be set in LEaP for the global parameters below with this usage:

set default parameter value

For example:

```
set default PBRadii mbondi
```

14. LEaP

OldPrmtopFormat If set to “on”, the `saveAmberParm` command will write a prmtop file in the format used in Amber 6 and earlier versions; if set to “off” (the default), it will use the new format. This is discouraged for general use and is available mainly for backwards compatibility with programs that expect old-style topology files or for testing.

Dielectric If set to “distance” (the default), electrostatic calculations in LEaP will use a distance-dependent dielectric; if set to “constant”, a constant dielectric will be used.

PdbWriteCharges If set to “on”, atomic charges will be placed in the “B-factor” field of PDB files saved with the `savePdb` command; if set to “off” (the default), no such charges will be written.

PBRadii Used to choose various sets of atomic radii for generalized Born or Poisson-Boltzmann calculations. Options are: “bondi”, which gives values from Ref. [354], which should be used with *igb* = 7; “mbondi”, which is the default, and the recommended parameter set for *igb* = 1 [213]; “mbondi2”, which is a second modification of the Bondi radii set [197], and should be used with *igb* = 2 or 5; “mbondi3”, which is a third modification of the Bondi radii set [29] recommended for use with *igb* = 8; and “amber6”, which is only to be used for reproducing very early calculations that used *igb* = 1 [195].

nocenter If set to “on”, LEaP will not center the coordinates inside the box for a periodic simulation; it will leave them unchanged as it does for a non-periodic simulation (note that the various solvate commands can still rigidly translate a solute). If set to “off” (the default), centering of coordinates will occur (as it always has, in previous versions of LEaP). Avoiding coordinate translations can be useful to avoid changing reference (perhaps experimental) coordinates. This option may be especially helpful for crystal simulations.

reorder_residues If set to “off”, residues in the output will be left in the same order they were found in the input file. The default behavior (“on”) is to place non-solvent residues first, followed by solvent residues, followed by solvent cap residues (if cap exists). “off” can, for example, be useful in crystal simulations (keep residues belonging to each asymmetric unit separate), but note that turning residue ordering off is untested and may lead to unforeseen behavior. Only set to “off” if you know what you are doing!

The parameters listed below can be set for the specified *containers* within LEaP using the following syntax:

```
set container parameter object
```

Some examples:

```
set ATOM name "name"  
set RESIDUE connect0 ATOM  
my_system = loadPDB file.pdb  
set my_system box {25 30 32}
```

For ATOMS:

name A unique STRING descriptor used to identify ATOMS.

type This is a STRING property that defines the AMBER force field atom type.

charge The charge property is a NUMBER that represents the ATOM’s electrostatic point charge to be used in a molecular mechanics force field.

position This property is a LIST of NUMBERS containing three values: the (X, Y, Z) Cartesian coordinates of the ATOM.

pertName This STRING is a unique identifier for an ATOM in its final state during a Free Energy Perturbation calculation. This functionality is no longer implemented in Amber.

pertType This STRING is the AMBER force field atom type of a perturbed ATOM. This functionality is no longer implemented in Amber.

pertCharge This NUMBER represents the final electrostatic point charge on an ATOM during a Free Energy Perturbation. This function is no longer implemented in Amber.

For RESIDUEs:

connect0 This identifies the first of up to three ATOMs that will be used to make links to other RESIDUEs. In a UNIT containing a single RESIDUE, the RESIDUE's connect0 ATOM is usually defined as the UNIT's head ATOM.

connect1 This identifies the second of up to three ATOMs that will be used to make links to other RESIDUEs. In a UNIT containing a single RESIDUE, the RESIDUE's connect1 ATOM is usually defined as the UNIT's tail ATOM.

connect2 This identifies the third of up to three ATOMs that will be used to make links to other RESIDUEs. In amino acids, the convention is that this is the ATOM to which disulfide bridges are made.

restype This property is a STRING that represents the type of the RESIDUE. Currently, it can have one of the following values: "undefined", "solvent", "protein", "nucleic", or "saccharide".

name This STRING property is the RESIDUE name.

For UNITs:

head Defines the ATOM within the UNIT that is connected when UNITs are joined together: the tail ATOM of one UNIT is connected to the head ATOM of the subsequent UNIT in any sequence.

tail Defines the ATOM within the UNIT that is connected when UNITs are joined together: the tail ATOM of one UNIT is connected to the head ATOM of the subsequent UNIT in any sequence.

box This property defines the bounding box of the UNIT (*container*). If *object* is set to null then no bounding box is defined. If it is a single NUMBER, the bounding box will be defined to be a cube with each side being NUMBER Å across. If it is a LIST, it must contain three NUMBERS, the lengths (in Å) of the three sides of the bounding box. Note that this command does not allow one to set the angles for the periodic system. See the `ChBox` command to do that.

cap This property defines the solvent cap of the UNIT. If it is set to null then no solvent cap is defined. Otherwise, it should be a LIST of four NUMBERS; the first three NUMBERS define the Cartesian coordinates (X, Y, Z) of the origin of the solvent cap in Å, while the fourth defines the radius of the solvent cap, also in Å.

14.6.41. `setBox`

```
setBox solute enclosure [ distance ]
```

This command creates a periodic box around *solute*, which should be a UNIT. It does not add any solvent to the system. `setBox` creates a cuboid box. The *enclosure* parameter determines whether the box encloses entire atoms or just atom centers. The former case is specified by the STRING value "vdw" for *enclosure* and the latter case by the STRING "centers". Use "centers" if the system has been previously equilibrated as a periodic box. The minimum distance between any atom in *solute* and the edge of the periodic box is given by the *distance* parameter; see the `solvateBox` command for more details.

```
> mol = loadpdb my.pdb
> setBox mol "vdw"
```

14.6.42. solvateBox and solvateOct

```
solvateBox solute solvent distance [ "iso" ] [ closeness ]
solvateOct solute solvent distance [ "iso" ] [ closeness ]
```

These two commands create periodic solvent boxes around *solute*, which should be a UNIT. *solvateBox* creates a cuboid box, while *solvateOct* creates a truncated octahedron. *solute* is modified by the addition of copies of the RESIDUES found within *solvent*, which should also be a UNIT, such that the minimum distance between any atom originally present in *solute* and the edge of the periodic box is given by the *distance* parameter. The resulting solvent box will be repeated in all three spatial directions.

If the distance parameter is a single NUMBER then the minimum distance is the same for the x, y, and z directions, unless the STRING "iso" parameter is specified to make the box or truncated octahedron isometric. For *solvateBox* if "iso" is used, the solute is rotated to orient the principal axes, otherwise it is just centered on the origin. For *solvateOct* if the "iso" option is used, the isometric truncated octahedron is rotated to an orientation used by the PME code, and the box and angle dimensions output by the *saveAmberParm** commands are adjusted for PME code imaging. In *solvateBox*, if the distance parameter is a LIST of three NUMBERS then the NUMBERS are applied to the x, y, and z axes respectively. As the larger box is created and superimposed on the solute, solvent molecules overlapping the solute are removed. In *solvateOct*, when a LIST is given for the distance parameter, four numbers are given instead of three, where the fourth is the diagonal clearance. If 0.0 is given as the fourth number, the diagonal clearance resulting from the application of the x,y,z clearances is reported. If a non-0 value is given, this may require scaling up the other clearances, which is also reported. Similarly, if a single NUMBER is given, any scaleup of the x,y,z buffer to accommodate the diagonal clip is reported.

The optional *closeness* parameter can be used to control how close, in Å, solvent ATOMS may come to solute ATOMS. The default value of *closeness* is 1.0. Smaller values allow solvent ATOMS to come closer to solute ATOMS. The criterion for rejection of overlapping solvent RESIDUES is if the distance between any solvent ATOM and its nearest solute ATOM is less than the sum of the two ATOMS' van der Waals radii multiplied by *closeness*.

```
> mol = loadpdb my.pdb
> solvateOct mol TIP3PBOX 12.0 0.75
```

14.6.43. solvateCap

```
solvateCap solute solvent position radius [ closeness ]
```

The *solvateCap* command creates a solvent cap around *solute*, which is a UNIT. *solute* is modified by the addition of copies of the RESIDUES found within *solvent*, which should also be a UNIT. The solvent box will be repeated in all three spatial directions to create a large solvent sphere with a radius of *radius* Å.

The *position* argument defines where the center of the solvent cap is to be placed. If *position* is a UNIT, a RESIDUE, an ATOM, or a LIST of UNITS, RESIDUES, or ATOMS, then the geometric center of the ATOM or ATOMS within the object will be used as the center of the solvent cap sphere. If *position* is a LIST containing three NUMBERS, then it will be treated as a vector describing the position of the solvent cap sphere center.

The optional *closeness* parameter can be used to control how close, in Å, solvent ATOMS may come to solute ATOMS. The default value of *closeness* is 1.0. Smaller values allow solvent ATOMS to come closer to solute ATOMS. The criterion for rejection of overlapping solvent RESIDUES is if the distance between any solvent ATOM and its nearest solute ATOM is less than the sum of the two ATOMS' van der Waals radii multiplied by *closeness*.

This command modifies *solute* in several ways. First, the UNIT is modified by the addition of solvent RESIDUES copied from *solvent*. Secondly, the "cap" parameter of *solute* is modified to reflect the fact that a solvent cap has been created around the solute.

```
> mol = loadpdb my.pdb
> solvateCap mol TIP3PBOX mol.2.CA 12.0 0.75
```

14.6.44. solvateShell

```
solvateShell solute solvent thickness [ closeness ]
```

The `solvateShell` command adds a solvent shell to *solute*, which should be a UNIT. *solute* is modified by the addition of copies of the RESIDUES found within *solvent*, which should also be a UNIT. The resulting solute/solvent UNIT will be irregular in shape since it will reflect the contours of the original solute molecule. The solvent box will be repeated in three directions to create a large solvent box that can contain the entire solute and a shell *thickness* Å thick. Solvent RESIDUES are then added to *solute* if they lie within the shell defined by *thickness* and do not overlap with any ATOM originally present in *solute*. The optional *closeness* parameter can be used to control how close solvent ATOMS can come to solute ATOMS. The default value of the *closeness* argument is 1.0. Please see the `solvateBox` command for more details on the closeness parameter.

```
> mol = loadpdb my.pdb
> solvateShell mol TIP3PBOX 12.0 0.8
```

14.6.45. source

```
source filename
```

This command executes the contents of the file given by *filename*, treating them as LEaP commands. To display the commands as they are read, see the `verbosity` command.

14.6.46. transform

```
transform atoms, matrix
```

Transform all of the ATOMS within *atoms* by a symmetry operation. The symmetry operation is represented as a (3×3) or (4×4) matrix, and given as nine or sixteen NUMBERS in *matrix*, a LIST of LISTS. The general matrix looks like:

```
r11 r12 r13 -tx r21 r22 r23 -ty r31 r32 r33 -tz 0 0 0 1
```

The matrix elements represent the intended symmetry operation. For example, a reflection in the (x,y) plane would be produced by the matrix:

```
1 0 0 0 1 0 0 0 -1
```

This reflection could be combined with a 6 Å translation along the x-axis by using the following matrix:

```
1 0 0 6 0 1 0 0 0 0 -1 0 0 0 0 1
```

In the following example, wrB is transformed by an inversion operation:

```
transform wrpB { { -1 0 0 } { 0 -1 0 } { 0 0 -1 } }
```

14.6.47. translate

```
translate atoms direction
```

Translate all of the ATOMS within *atoms* by the vector given by *direction*, a LIST of three NUMBERS.

Example:

```
translate wrpB { 0 0 -24.53333 }
```

14.6.48. verbosity**verbosity level**

This command sets the level of output that LEaP provides the user. A value of 0 is the default, providing the minimum of messages. A value of 1 will produce more output, and a value of 2 will produce all of the output of level 1 and display the text of the script lines executed with the source command. The following line is an example of this command:

```
> verbosity 2
Verbosity level: 2
```

14.6.49. zMatrix**zMatrix object zmatrix**

The `zMatrix` command is quite complicated. It is used to define the external coordinates of ATOMs within *object* using internal coordinates. The second parameter of the `zMatrix` command is a LIST of LISTS; each sub-list has several arguments:

```
{ a1 a2 bond12 }
```

This entry defines the coordinate of *a1*, an ATOM, by placing it *bond12* Å along the x-axis from ATOM *a2*. *a2* is placed at the origin if its coordinates are not defined.

```
{ a1 a2 a3 bond12 angle123 }
```

This entry defines the coordinate of *a1* by placing it *bond12* Å away from *a2* making an angle of *angle123* degrees between *a1*, *a2* and *a3*. The angle is measured in a right-hand sense and in the xy plane. ATOMs *a2* and *a3* must have coordinates defined.

```
{ a1 a2 a3 a4 bond12 angle123 torsion1234 }
```

This entry defines the coordinate of *a1* by placing it *bond12* Å away from *a2*, creating an angle of *angle123* degrees between *a1*, *a2*, and *a3*, and making a torsion angle of *torsion1234* degrees between *a1*, *a2*, *a3*, and *a4*.

```
{ a1 a2 a3 a4 bond12 angle123 angle124 orientation }
```

This entry defines the coordinate of *a1* by placing it *bond12* Å away from *a2*, and making angles *angle123* degrees between *a1*, *a2*, and *a3*, and *angle124* degrees between *a1*, *a2*, and *a4*. The argument *orientation* defines whether *a1* is above or below a plane defined by *a2*, *a3* and *a4*. If *orientation* is positive, *a1* will be placed so that the triple product $((a3-a2) \times (a4-a2)) \cdot (a1-a2)$ is positive. Otherwise, *a1* will be placed on the other side of the plane. This allows the coordinates of a molecule like fluoro-chloro-bromo-methane to be defined without having to resort to dummy atoms.

The first arguments within the `zMatrix` entries (*a1*, *a2*, *a3* and *a4*) are either ATOMs, or STRINGs containing names of ATOMs that already exist within *object*. The subsequent arguments (*bond12*, *angle123*, *torsion1234* or *angle124*, and *orientation*) are all NUMBERS. Any ATOM can be placed at the *a1* position, even one that has coordinates defined. This feature can be used to provide an endless supply of dummy atoms, if they are required. A predefined dummy atom with the name "*" (a single asterisk, no quotes) can also be used.

There is no order imposed in the sub-lists. The user can place sub-lists in arbitrary order, as long as they maintain the requirement that all ATOMs *a2*, *a3*, and *a4* must have external coordinates defined, except for entries that define the coordinate of an ATOM using only a bond length. (See the `add` command for an example of the `zMatrix` command.)

14.7. Building oligosaccharides, lipids and glycoproteins

Build assistance available at GLYCAM-Web:

The approaches presented below have been automated, with many additional options available, at the GLYCAM-Web site: www.glycam.org. The capabilities of the website are being expanded. Currently, the available functionalities include:

Oligosaccharides, linear and branched
Glycoproteins, O- or N-linked, with multiple glycans
Builds of oligosaccharides via URL directive

Build assistance available in the AmberTools tests:

Examples in addition to those described below can be found in the AmberTools tests. The relevant files are located in:

```
$AMBERHOME/AmberTools/test/leap/glycam # main test directory
$AMBERHOME/AmberTools/test/leap/glycam/06EPb # extra points oligosaccharides
$AMBERHOME/AmberTools/test/leap/glycam/06j # main oligosaccharides
$AMBERHOME/AmberTools/test/leap/glycam/06j_10 # glycoprotein with ff10
$AMBERHOME/AmberTools/test/leap/glycam/06j_12SB # glycoprotein with ff12SB
```

PLEASE NOTE: The molecules in the test directories were constructed for the purpose of testing functionality in AmberTools. They might not be ready for simulations as they are. Some might be in configurations with severe clashes. Most structural issues can be resolved by manipulating appropriate torsions. The glycoprotein tests contain usage examples for torsion manipulations using the impose command.

Each sub-directory below "glycam" contains tests relevant to specific force fields. To run an individual test, saving all relevant output and intermediate files, change to the sub-directory and issue the command:

```
./Run.glycam evaluate
```

To return the directory to its previous state, run:

```
./Run.glycam clean
```

The 00_README file in the main directory contains more information about using the tests.

Additional notes about this section:

Before continuing in this section, you should review the GLYCAM naming conventions covered in Section 3.4. After that, there are two important things to keep in mind. The first is that GLYCAM is designed to build oligosaccharides, not just monosaccharides. In order to link the monosaccharides together, each residue in GLYCAM will have at least one open valence position. That is, each GLYCAM residue lacks either a hydroxyl group or a hydroxyl proton, and may be lacking more than one proton depending on the number of branching locations. Thus, none of the residues is a complete molecule unto itself. For example, if you wish to build α -D-glucopyranose, you must explicitly specify the anomeric -OH group (see Figure 14.1 for two examples).

The second thing to keep in mind is that when the `sequence` command is used in LEaP to link monosaccharides together to form a linear oligosaccharide (analogous to peptide generation), the residue ordering is opposite to the standard convention for writing the sequence. For example, to build the disaccharides illustrated in Figure 14.1, using the `sequence` command in LEaP, the format would be:

```
upperdisacc = sequence { ROH 3GB 0GB }
lowerdisacc = sequence { OME 4GB 0GA }
```

While the `sequence` command is the most direct method to build a linear glycan, it is not the only method. Alternatives that facilitate building more complex glycans and glycoproteins are presented below. For those who need to build structures (and generate topology and coordinate files) that are more complex, a convenient interface that uses GLYCAM is available on the internet (<http://glycam.ccruc.uga.edu> or <http://www.glycam.org>).

Throughout this section, sequences of LEaP commands will be entered in the following format:

14. LEaP

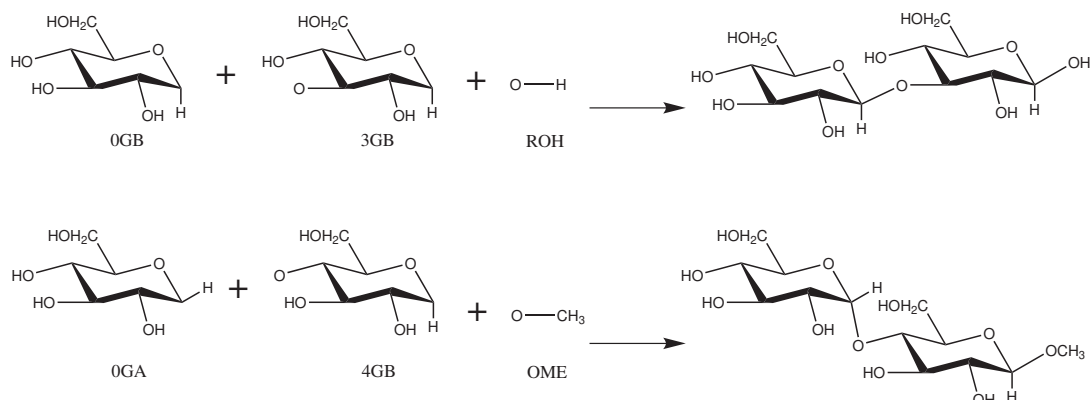


Figure 14.1.: Schematic representation of disaccharide formation, indicating the need for open valences on carbon and oxygen atoms at linkage positions.

command argument(s) # descriptive comment

This format was chosen so that the lines can be copied directly into a file to be read into LEaP. The number sign (#) signifies a comment. Comments following commands may be left in place for future reference and will be ignored by LEaP. Files may be read into LEaP either by sourcing the file or by specifying it on the command line at the time that LEaP is invoked, e.g.:

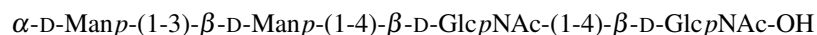
```
tLeap -f leap_input_file
```

Note that any GLYCAM parameter set shipped with Amber is likely to be updated in the future. The current version is *GLYCAM_06j.dat*. This file and *GLYCAM_06j-1.prep* are automatically loaded with the default *leaprc.GLYCAM_06j-1*. The user is encouraged to check www.glycam.org for updated versions of these files.

14.7.1. Procedures for building oligosaccharides using the GLYCAM-06 parameters

14.7.1.1. Example: Linear oligosaccharides

This section contains instructions for building a simple, straight-chain tetrasaccharide:



First, it is necessary to determine the GLYCAM residues that will be used to build it. Since the initial $\alpha\text{-D-Manp}$ residue links only at its anomeric site, the first character in its name is 0 (zero), indicating that it has no branches or other connections, i.e., it is terminal. Since it is a D-mannose, the second character, the one-letter code, is M (capital). Since it is an α -pyranose, the third character is A. Therefore, the first residue in the sequence above is OMA. Since the second residue links at its 3-position as well as at the anomeric position, the first character in its name is 3, and, being a β -pyranose, it is 3MB. Similarly, residues three and four are both 4YB. It will also be necessary to add an OH residue at the end to generate a complete molecule. Note that in Section 14.7.3, below, the terminal OH *must* be omitted in order to allow subsequent linking to a protein or lipid. Note also that when present, a terminal OH (or OME etc) is assigned its own residue number.

Converting the order for use with the sequence command in LEaP, gives:

```
Residue name sequence: ROH 4YB 4YB 3MB OMA
Residue number:      1   2   3   4   5
```

Here is a set of LEaP instructions that will build the sequence (there are, of course, other ways to do this):

```
source leaprc.GLYCAM_06j-1 # load leaprc
glycan = sequence { ROH 4YB 4YB 3MB OMA } # build oligosaccharide
```

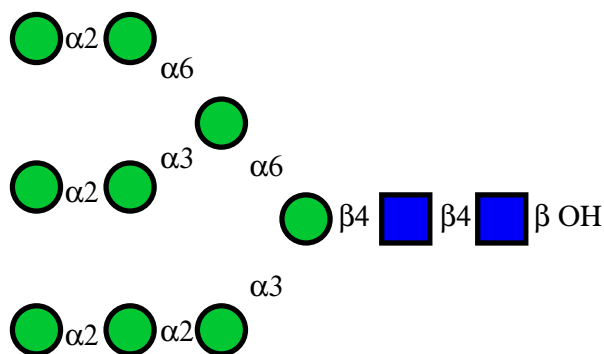



Figure 14.2.: Structure of Man-9, represented in the symbolic notation used by the Consortium for Functional Glycomics. Here, ● =D-Manp and ■ =D-GlcpNAc

```
impose branch {4} { {H1 C1 O6 C6 -60.0} } # set phi torsion and
impose branch {4} { {C1 O6 C6 H6 0.0} } # set psi OMA(6) & VMB
impose branch {4} { {H1 C1 O4 C4 60.0} } # set phi torsion and
impose branch {4} { {C1 O4 C4 H4 0.0} } # set psi 3MB & 4YB
impose branch {3} { {H1 C1 O4 C4 60.0} } # set phi torsion and
impose branch {3} { {C1 O4 C4 H4 0.0} } # set psi 4YB & 4YB
impose branch {5} { {H1 C1 O3 C3 -60.0} } # set phi torsion and
impose branch {5} { {C1 O3 C3 H3 0.0} } # set psi OMA(3) & VMB
saveamberparm branch.branch.top branch.crd # save top & crd
savepdb branch.pdb # save pdb
```

14.7.1.3. Example: Complex branched oligosaccharides

The following example builds a highly branched, high-mannose structure shown in Figure 14.2. In this example, it is especially important to note that when the branching is ambiguous, LEaP might not choose the attachment point one wants or expects. For this reason, connectivity should be specified explicitly whenever the structure branches. That is, one cannot specify the longest linear sequence and add branches later. The sequence command must be interrupted at each branch point. Otherwise, the connectivity is not assured. In this example, a branch occurs at each VMA (-3,6-D-Manp) residue.

The following set of commands, given to tleap, will safely produce the structure represented in Figure 14.2.

```
source leaprc.GLYCAM_06j-1
glycan = sequence { ROH 4YB 4YB VMB }
set glycan tail glycan.4.O6
glycan=sequence { glycan VMA }
set glycan tail glycan.5.O6
glycan=sequence { glycan 2MA OMA }
set glycan tail glycan.5.O3
glycan=sequence { glycan 2MA OMA }
set glycan tail glycan.4.O3
glycan=sequence { glycan 2MA 2MA OMA }
impose glycan {3} { {H1 C1 O4 C4 60.0} }
impose glycan {3} { {C1 O4 C4 H4 0.0} }
impose glycan {4} { {H1 C1 O4 C4 60.0} }
impose glycan {4} { {C1 O4 C4 H4 0.0} }
impose glycan {5} { {H1 C1 O6 C6 -60.0} } # 1-6 Link from (5) to (4), Phi
impose glycan {5} { {C1 O6 C6 C5 180.0} } # 1-6 Link from (5) to (4), Psi
impose glycan {4} { {O6 C6 C5 O5 60.0} } # 1-6 Link from (5) to (4), Chi
impose glycan {10} { {H1 C1 O3 C3 -60.0} }
impose glycan {10} { {C1 O3 C3 H3 0.0} }
```

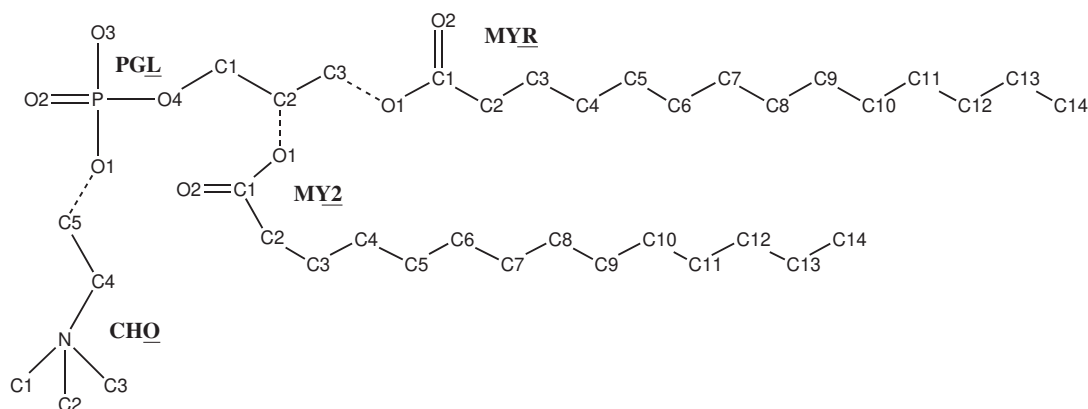


Figure 14.3.: DMPC

```

impose glycan {6} { {H1 C1 O6 C6 -60.0} }
impose glycan {6} { {C1 O6 C6 C5 180.0} }
impose glycan {5} { {O6 C6 C5 O5 -60.0} }
impose glycan {8} { {H1 C1 O3 C3 -60.0} }
impose glycan {8} { {C1 O3 C3 H3 0.0} }
impose glycan {7} { {H1 C1 O2 C2 -60.0} }
impose glycan {7} { {C1 O2 C2 H2 0.0} }
impose glycan {9} { {H1 C1 O2 C2 -60.0} }
impose glycan {9} { {C1 O2 C2 H2 0.0} }
impose glycan {11} { {H1 C1 O2 C2 -60.0} }
impose glycan {11} { {C1 O2 C2 H2 0.0} }
impose glycan {12} { {H1 C1 O2 C2 -60.0} }
impose glycan {12} { {C1 O2 C2 H2 0.0} }
saveamberparm glycan glycan.prmtop glycan.restrt

```

14.7.2. Procedures for building a lipid using GLYCAM-06 parameters

The procedure described here allows a user to produce a single lipid molecule without consideration for axial alignment. Lipid bilayers are typically built in the (x,y) plane of a Cartesian coordinate system, which requires the individual lipids to be aligned hydrophilic “head” to hydrophobic “tail” along the z-axis. This can be done relatively easily by loading a template PDB file that has been appropriately aligned on the z-axis.

The lipid described in this example is 1,2-dimyristoyl-*sn*-glycero-3-phosphocholine or DMPC. For this example, DMPC will be composed of four fragments: CHO, the choline “head” group; PGL, the phospho-glycerol “head” group; MYR, the *sn*-1 chain myristic acid “tail” group; and MY2, the *sn*-2 chain myristic acid “tail” group. See the molecular diagram in 14.3 for atom labels (hydrogens and atomic charges are removed for clarity) and bonding points between each residue (dashed lines). This tutorial will use only prep files for each of the four fragments. These prep files were initially built as PDB files and formatted as prep files using *antechamber*. GLYCAM-compatible charges were added to the prep files and a prep file database (GLYCAM_lipids.06h.prep) was created containing all four files.

14.7.2.1. Example: Building a lipid with LEaP.

One need not load the main GLYCAM prep files in order to build a lipid using the GLYCAM-06 parameter set, but it is automatically loaded with the default *leaprc.GLYCAM_06j-1*. Note that the lipid generated by this set of commands is not necessarily aligned appropriately to create a bilayer along an axis. The commands to use are:

```

source leaprc.GLYCAM_06j-1 # source the leaprc for GLYCAM-06
loadamberprep GLYCAM_06_lipids.prep # load the lipid prep file

```

```

set CHO tail CHO.1.C5 # set the tail atom of CHO as C5.
set PGL head PGL.1.O1 # set the head atom of PGL to O1
set PGL tail PGL.1.C3 # set the tail atom of PGL to C3
lipid = sequence { CHO PGL MYR } # generate the straight-chain
# portion of the lipid
set lipid tail lipid.2.C2 # set the tail atom of PGL to C2
lipid = sequence { lipid MY2 } # add MY2 to the "lipid" unit
impose lipid {2} { {C1 C2 C3 O1 163} } # set torsions for
impose lipid {2} { {C2 C3 O1 C1 -180} } # PGL & MYR
impose lipid {2} { {C3 O1 C1 C2 180} }
impose lipid {2} { {O4 C1 C2 O1 -60} } # set torsions for
impose lipid {2} { {C1 C2 O1 C1 -180} } # PGL & MY2
impose lipid {2} { {C2 O1 C1 C2 180} }
# Note that the values here may not necessarily
# reflect the best choice of torsions.
savepdb lipid DMPC.pdb # save pdb file
saveamberparm lipid DMPC.top DMPC.crd # save top and crd files

```

14.7.3. Procedures for building a glycoprotein in LEaP.

The LEaP commands given in this section assume that you already have a PDB file containing a glycan and a protein in an appropriate relative configuration. Thorough knowledge of the commands in LEaP is required in order to successfully link any but the simplest glycans to the simplest proteins, and is beyond the scope of this discussion. Several options for generating the relevant PDB file are given below (see Items 5a-5c).

The protein employed in this example is bovine ribonuclease A (PDBID: 3RN3). Here the branched oligosaccharide assembled in the second example will be attached (*N*-linked) to ASN 34 to generate ribonuclease B.

14.7.3.1. Setting up protein pdb files for glycosylation in LEaP.

1. Delete any atoms with the "HETATM" card from the PDB file. These would typically include bound ligands, non-crystallographic water molecules and non-coordinating metal ions. Delete any hydrogen atoms if present.
2. In general, check the protein to make sure there are no duplicate atoms in the file. This can be quickly done by loading the protein in LEaP and checking for such warnings. In this particular example, residue 119 (HIS) contained duplicate side chain atoms. Delete all but one set of duplicate atoms.
3. Check for the presence of disulfide bonds (SSBOND) by looking at the header section of the PDB file. 3RN3 has four disulfide bonds, between the following pairs of cysteine residues: 26—84, 40—95, 58—110, and 65—72. Change the names of these eight cysteine residues from CYS to CYX.
4. At present, it is possible to link glycans to serine, threonine, hydroxyproline and asparagine. You must rename the amino acid in the protein PDB file manually prior to loading it into LEaP. The modified residue names are OLS (for *O*-linkages to SER), OLT (for *O*-linkages to THR), OLP (for *O*-linkages to hydroxyproline, HYP) and NLN (for *N*-linkages to ASN). Libraries containing amino acid residues that have been modified for the purpose are automatically loaded when *leaprc.GLYCAM_06j-1* is sourced. See the lists of library files in 3.4 for more information.
5. Prepare a PDB file containing the protein and the glycan, with the glycan correctly aligned relative to the protein surface. There are several approaches to performing this including:
 - a) It is often the case that one or more glycan residues are present in the experimental PDB file. In this case, a reasonable method is to superimpose the linking sugar residue in the GLYCAM-generated glycan upon that present in the experimental PDB file, and to then save the altered coordinates. If you use this method, remember to delete the experimental glycan from the PDB file! It is also essential to ensure that each carbohydrate residue is separated from other residues by a TER card in the PDB file. Also

remember to delete the terminal OH or OMe from the glycan. Alternately, the experimental glycan may be retained in the PDB file, provided that it is renamed according to the GLYCAM 3-letter code, and that the atom names and order in the PDB file match the GLYCAM standard. This is tedious, but will work. Again, be sure to insert TER cards if they are missing between the protein and the carbohydrate and between the carbohydrate residues themselves.

- b) Use a molecular modeling package to align the GLYCAM-generated glycan with the protein and save the coordinates in a single file. Remember to delete the terminal OH or OMe from the glycan.
- c) Use the Glycoprotein Builder tool at <http://www.glycam.org>. This tool allows the user to upload protein coordinates, build a glycan (or select it from a library), and attach it to the protein. All necessary AMBER files may then be downloaded. This site is also convenient for preprocessing protein-only files for subsequent uploading to the glycoprotein builder.

14.7.3.2. Example: Adding a branched glycan to 3RN3 (N-linked glycosylation).

In this example we will assume that the glycan generated above (“branch.pdb”) has been aligned relative to the ASN 34 in the protein file and that the complex has been saved as a new PDB file (e.g., as “3rn3_nlink.pdb”). The last amino acid residue should be VAL 124, and the glycan should be present as 4YB 125, 4YB 126, VMB 127, OMA 128 and OMA 129.

Remember to change the name of ASN 34 from ASN to NLN. For the glycan structure, ensure that each residue in the PDB file is separated by a “TER” card. *The sequence command is not to be used here, and all linkages (within the glycan and to the protein) will be specified individually.*

Enter the following commands into *xleap* (or *tleap* if a graphical representation is not desired). Alternately, copy the commands into a file to be sourced.

```
source leaprc.GLYCAM_06j-1 # load the GLYCAM-06 leaprc for ff14SB
source leaprc.protein.ff14SB # load the protein force field
glyprot = loadpdb 3rn3_nlink.pdb # load protein and glycan pdb file
bond glyprot.125.O4 glyprot.126.C1 # make inter glycan bonds
bond glyprot.126.O4 glyprot.127.C1
bond glyprot.127.O6 glyprot.128.C1
bond glyprot.127.O3 glyprot.129.C1
bond glyprot.34.SG glyprot.125.C1 # make glycan -- protein bond
bond glyprot.26.SG glyprot.84.SG # make disulfide bonds
bond glyprot.40.SG glyprot.95.SG
bond glyprot.58.SG glyprot.110.SG
bond glyprot.65.SG glyprot.72.SG
addions glyprot CL 0 # neutralize appropriately
solvateBox glyprot TIP3P BOX 8 # solvate the solute
savepdb glyprot 3nr3_glycan.pdb # save pdb file
saveamberparm glyprot 3nr3_glycan.top 3nr3_glycan.crd # save top, crd
quit # exit leap
```

14.7.4. Solvating a system with a specific number of molecules

Sometimes it is desirable to solvate a system with a target number of waters rather than specifying a particular box size. The following script is a wrapper around LEaP which can be used for this purpose:

```
$AMBERHOME/AmberTools/src/etc/Solvate.sh
```

In addition to LEaP (really *tleap*), the script also makes use of *cpptraj* for determining molecule info.

14.7.4.1. Solvate.sh Usage

```
Solvate.sh <input_file>
Input File Options: (default)
```

target <#> Target # of waters to add.
buffer <buf> Initial buffer size (10.0).
bufx <buf> Initial buffer X size (mode 2|3 only, 10.0).
bufy <buf> Initial buffer Y size (mode 2|3 only, 10.0).
pdb <file> Solute PDB file name.
top <name> Output topology (solvated.parm7).
crd <name> Output coordinates (solvated.rst7).
leapin <file> LEaP input script for loading parameters etc.
ionsin <file> Optional LEaP input for loading ions etc (run after solvating).
templeap <name> Name of temporary leap input script (temp.leap.in).
tol <#> Number of waters > target allowed, will be removed (2).
mode <#> Solvate mode: (0)- SolvateOct 1 - SolvateBox 2 - SolvateBoxXYZ (bufx and bufy are scaled) 3 - SolvateBoxZ (bufx and bufy are fixed)
loadpdb {yes|no} If (yes), use 'loadpdb PDB'; otherwise <leapin> should set up unit <molname>.
loadcmd <cmd> Command to load solute file; default 'loadpdb'.
soluteres <#> Number of solute residues. If blank try to guess from PDB.
molname <name> Solute molecule unit name ('m').
solventunit <name> Solvent unit (TIP3PBOX). Recognized solvent units: TIP3PBOX SPCBOX OPCBOX TIP4PEWBOX

First the file specified by leapin is read by LEaP, then the system is solvated, then the file specified by 'ionsin' is read in order to add ions etc.

14.7.4.2. Solvate.sh Example

Solvate an RNA tetranucleotide with 2500 TIP3P waters and 3 Na+ ions.

Input file: solvate.in

```

# Target number of waters
target 2500
# Initial guess for buffer
buffer 10
# Input PDB name
pdb rGACC.pdb
# Output topology name
top rGACC.tip3p.parm7
# Output coordinates name
crd rGACC.nomin.rst7
# Base leap input script
leapin leap.solvate.in
# Additional script for adding ions etc
ionsin leap.ions.in
# Tolerance (# of waters off from target allowed)
tol 3
# 0 - SolvateOct
mode 0

```

LEaP input: leap.solvate.in

```
source leaprc.RNA.OL3  
set default pbradii mbondi2
```

LEaP Ions input: leap.ions.in

```
addions m Na+ 1  
addions m Na+ 1  
addions m Na+ 1
```

15. Reading and modifying Amber parameter files

This chapter describes the content of Amber parameter files, along with details about *ParmEd* (which can be used to examine and modify prmtop files) and *mdgx* (which can be used to fit force fields to quantum mechanical and other target data).

15.1. Understanding Amber parameter files

Romain M. Wolf, Jason Swails, and David A. Case

This chapter provides a short description of Amber-compatible force field parameter files is given. Only the actual data in parameter (*.dat) files are discussed. The special issue of deriving partial charges is not addressed. Also, more complex subjects dealing with parameters for implicit solvent (GB or PB) or polarisability computations are skipped. This text is meant as a documentation for users who want to understand parameter files, and in some cases might be tempted to change or add some parameters. Most of the following documentation is found in bits and pieces at various Amber-related sites and in tutorials or original Amber manuals and these various sources have been helpful to put together this hopefully concise documentation.

15.1.1. Parameter Transfers between Force Fields

Transferring parameters from one force field to another must respect the underlying functional form, the units in which parameters are expressed in the parameter files, and also the exact procedures on how individual parameters were obtained. In addition, attention must be paid to the methods used to deduce partial charges. Force fields are self-consistent, i.e., all terms are interrelated and their actual values depend on the way they were derived. Therefore, any parameter transfer between different force fields is dangerous, even when the functional form is the same (or looks as if it were...).

Torsion terms are the most critical. Many torsion barriers and profiles are not easily assessed experimentally and are often deduced from *ab initio* quantum mechanical (QM) computations on small fragments. Since QM calculations offer many possibilities, the exact nature of these calculations (basis sets, Hartree-Fock and/or density functionals, etc.) used to derive parameters should be known.

Special care must also be applied to 1-4 interactions, i.e., interactions between atoms separated by exactly three consecutive bonds. Most Amber force fields for example assume that 1-4 interactions get a special treatment. See section 15.1.6 for details. In many other force fields, the special treatment of 1-4 interactions is either different or non-existent. This has an immediate influence on the torsion terms and resulting conformation energies. Therefore, before transferring torsion terms, van der Waals parameters and partial charges from other force fields, check the special treatment of 1-4 interactions in the source and the target force field.

15.1.2. How Amber Routines Use the Parameter Files

Amber routines that perform actual calculations (sander, pmemd, etc...) do not read parameter files directly. They use a special file type, the *parameter-topology* file (*parmtop* from now on), which contains all the information required by the various energy functions in the computation routines. The *parmtop* file is specific to the molecular system for which it was created and is directly related to the second required file, the coordinate file.¹ Smallest

¹This file can be in the Amber coordinate 'crd' file format or, for some applications, also in PDB format.

changes to the system (adding or removing atoms, or even changing the order of atoms in the coordinate file) render the *parmtop* useless.

Although *parmtop* files are pure ASCII files, changing parameters directly in them by standard text editors is strongly discouraged. In the worst case, computations will run without any warnings, but results might be totally flawed. The safest way to generate *parmtop* files is to use an Amber tool like *tleap* that has been used, tested, and enhanced over a number of years and usually generates correct *parmtop* files, provided that the input is correct and that all required information is available via fragment libraries and parameter files. The latest AmberTools 12.0 version (April 2012) includes the *ParmEd* python script of Jason Swails which is very useful to examine or post-process *parmtop* files. However, only users with detailed knowledge on the exact format of *parmtop* files should dare fiddling around with this data type.

15.1.3. "*.dat" and "frcmod.*" Files

The standard parameter files with the `.dat` extension are located in the folder `$AMBERHOME/dat/leap/parm`. Adding or changing parameters directly in the parameter files delivered with an Amber distribution is not a good idea for the following reasons: (a) you might mess up the parameter file, (b) you might have trouble to remember and find your changes later and add confusion when publishing results, (c) subsequent updates or patches might overwrite your changes.

In the above mentioned folder, there are also various `frcmod.*` files. They have basically the same format as the parameter `*.dat` files. See some of the examples provided in the Amber distributions. These files can be read into *tleap* exactly like the standard `*.dat` files. They merge the default parameters in the `*.dat` file with the new parameters in the `frcmod.*` files. More important, if the same parameters already exist in the `*.dat` files, the parameters in the `frcmod.*` files overwrite the default `*.dat` parameters. This offers a handy way to add new or to change original parameters without ever touching the default parameter files. Just make sure to read the respective `frcmod.*` files in *tleap* when the new or altered parameters should be used.

15.1.4. Parameters Required for Amber Force Fields

The simplest form of the Amber force field (neglecting implicit solvent or polarisation terms) uses the following Hamiltonian:

$$\begin{aligned}
 E_{total} = & \sum_{bonds} k_b (r - r_0)^2 \\
 & + \sum_{angles} k_\theta (\theta - \theta_0)^2 \\
 & + \sum_{dihedrals} V_n [1 + \cos(n\phi - \gamma)] \\
 & + \sum_{i=1}^{N-1} \sum_{j=i+1}^N \left[\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6} + \frac{q_i q_j}{\epsilon R_{ij}} \right]
 \end{aligned}
 \tag{15.1}$$

In this equation, the terms $k_b, r_0, k_\theta, \theta_0, V_n, \gamma, A_{ij}, B_{ij}$ are parameters to be specified in the parameter files mentioned in section 15.1.3 for the various Amber force fields.² The meaning of these different parameters is outlined in the following sections.

Equation 15.1 does not have a special term for out-of-plane motions. Amber routines handle these terms through the same formulation as the torsion terms (see section 15.1.6).

Partial charges (q_i, q_j in equation 15.1), although parameters also, do not appear in parameter files, but are assigned differently (see 15.1.7).

²Note that equation 15.1 does not use the (physically more correct) $\frac{k_b}{2}$, $\frac{k_\theta}{2}$, and $\frac{V_n}{2}$ notations because it refers to the constants as they appear in the actual parameter files.

15.1.5. Atom Types

Amber atom types can be one or two characters long. Uppercase (standard protein and nucleotide force fields), lowercase (GAFF General Amber Force Field), and mixed upper-lowercase (GLYCAM sugar force field) are allowed. Obviously, atom types must have a single, unique, definition.

If considering the definition a new atom type, think about the consequences. Of course, an atom type with an identical name must **not** already exist in one of the standard force fields used in the Amber community. Depending on how often and in how many combinations the atom type might occur, be also aware of the rather large number of additional parameters that might be required. Especially for bond angles, this number can grow very rapidly.

A new atom type definition, if required, must be clear and precise. It should also be possible to treat the definition in an automatic atom-type assignment procedure. Requiring users to visually verify and to change atom types by hand will cause trouble and will make it impossible to use the force field in automatic procedures that should not require user intervention for this task.

15.1.6. Bonded Interaction Terms

Bond Stretching Terms

The first row in equation 15.1 (page 277) is the harmonic term for bond stretching. In Amber-type parameter files, the force constant k_b is given for energy values in kcal/mol, with bond lengths in Å. The following line shows an example from the GAFF force field file `gaff.dat`.

The bond between a sp^3 carbon (c3) and a hydroxyl oxygen (oh) has a default (equilibrium) value of 1.426 Å and a force constant of 314.1 kcal/mol/Å².

```
c3-oh 314.1 1.4260 SOURCE1 914 0.0129
```

The entrance in the parameter file starts with the definition of the bond (atomtype1 hyphen atomtype2), followed by the force constant k_b (in kcal/mol/Å²) and the equilibrium bond length r_0 (in Å). Only the first three fields are relevant for computations. The other fields on the line above are mainly documentation.

As stated before, atom types in Amber FFs cannot have more than two characters. But if they have only one character (e.g., a carbonyl carbon atom c), entries with a one-letter atom type must look like this:

```
c -oh 466.4 1.3060 SOURCE1 271 0.0041
```

i.e., the space is **after** the atom type, **before** the hyphen.

Starting with a space like on the next line might lead to problems.

```
c-oh 466.4 1.3060 SOURCE1 271 0.0041
```

This holds for all parameter file entries that use hyphens to separate atom types, i.e., also angle and torsion terms (see following sections).

Angle Bending Terms

Angle bending terms are parameterised by a force constant k_θ in kcal/mol/radian² and an equilibrium angle value θ_0 in degrees. They have the format as shown below:

```
c3-c3-oh 67.720 109.430 SOURCE3 48 1.5023
```

The middle atom c3 is bonded to another c3 and to a hydroxyl oxygen oh. The equilibrium bond angle θ_0 is 109.43 degrees and the force constant is 67.720 kcal/mol/radian². Note that internally, angle deviations are computed in π -radian². The *parmtop* files also express the default 'equilibrium' bond angles in radians. For example, the angle of 109.43 degrees is internally represented as 1.9099 π -radians. Using degrees in the original parameter files is obviously more convenient. Anything after the third field, the equilibrium angle, is mainly documentation and not required.

Torsion Terms

The third row in equation 15.1 is the usual Fourier-series expansion for torsional terms. In Amber parameter files, these entries require a careful explanation:

First, many torsion terms contain generic entries, using the notation 'X' for 'any atom'. These terms are used when the parameter file does not contain more specific terms for the same torsion. They are combined with explicit terms when present. Entries with generic 'X' atoms must always come **before** the more specific ones in the parameter files.

Second, Amber parameter files use a special notation for torsions that require more than one torsional term (see example towards the end of section 15.1.6).

Third, the parameter file entry not only contains the torsion barrier term V_n (in kcal/mol), the phase γ (degrees) and the periodicity n , but also a **divider** (integer) which splits the torsion term into individual contributions for each pair of atoms involved in the torsion.

Fourth, torsion entries can also contain information about the special scaling of 1-4 non-bonded interactions (see section 15.1.6 on page 281).

Consider the following example, the default term for the torsion around a C_{sp3} - C_{sp3} single bond:

```
X -c3-c3-X 9 1.400 0.000 3.000 JCC, 7, (1986), 230
```

The five relevant terms on this line are:

1. the definition (X -c3-c3-X)
2. the divider (9)
3. the barrier term (1.400)
4. the phase (0.000)
5. the periodicity (3.000)

Fields after the periodicity are mainly comment, **except for the special flags SCNB and SCEE**, that, if present, govern the special treatment of 1-4 non-bonded interaction (see section 15.1.6)

The torsional barrier term (the actual barrier divided by two) is 1.400 and the periodicity is 3. The **phase is zero** in this example, meaning that a **maximum** energy is encountered at zero degrees. A **phase of 180 degrees** on the other hand means that there is a **minimum** at 180 degrees. The divider is 9 because each C_{sp3} has three X attached to it and each X 'sees' three X attached to the other C_{sp3} ($3 \times 3 = 9$).

For a torsion angle ϕ (defined as X-c3-c3-X) of -60, 60, or 180 degrees, the torsion energy term would be zero:

$$\frac{1.4}{9} \times [1 + \cos(3 \times \phi - 0.0)] = 0 \quad (15.2)$$

This corresponds to the staggered conformation, i.e., the lowest energy state in a X_3C-CX_3 connectivity like for example ethane (H_3C-CH_3)

By rotating around the C-C bond, an eclipsed conformation where the X are exactly opposed is encountered three times (periodicity = 3), namely at $\phi = 0, 120$, or 240 (-120) degrees.

$$\frac{1.4}{9} \times [1 + \cos(3 \times \phi - 0.0)] = 0.3111 \quad (15.3)$$

Since the divider is 9, we have to multiply the value of 0.3111 by 9 to get the full torsional barrier, i.e., $9 \times 0.3111 = 2.8$ kcal/mol.³ This might be used for ethane for example and would be close to the experimental torsion barrier (ca. 3 kcal/mol).

In GAFF however, there is also a specific term for $HC-C3-C3-HC$ that would come into play for ethane. In this case, the divider is 1, because the term is fully defined.

³The actual barrier value of 2.8 kcal/mol here is twice the barrier term of 1.4 in the parameter file.

15. Reading and modifying Amber parameter files

hc-c3-c3-hc 1 0.15 0.0 3. Junmei et al, 1999

Thus, using GAFF for ethane, this term counts 9 times because there are nine [hc,hc] pairs seeing each other. Instead of equation 15.3, one would use

$$0.15 \times [1 + \cos(3 \times \phi - 0.0)] = 0.3000 \quad (15.4)$$

i.e., the total torsional term in ethane would be $9 \times 0.3 = 2.7$ kcal/mol. The experimental torsional barrier value of ca. 3 kcal/mol would be reached because of the additional van der Waals and Coulomb repulsion terms between the staggered hydrogens.

Assume a connectivity for which some terms are fully defined (all four atom types are specified) while no specific entry is given for others. In that case, the equations are combined. The specific terms are counted once (divider = 1) and the remaining general terms are added according to

$$\frac{V_{\text{barrier}}}{\text{divider}} \times [1 + \cos(\text{periodicity} \times \phi - \text{phase})] \quad (15.5)$$

Things get more complex when the Fourier series has more than one term. A typical example would be the rotation around an amide bond R1-NH-C(=O)-R2. In this case, the *trans* amide (H and O on opposite sides, $\phi = 180^\circ$) is preferred over the *cis*-amide (H and O on the same side, $\phi = 0$). The entry in the GAFF parameter file for this torsion is

hn-n -c -o 1 2.50 180.0 -2. JCC,7,(1986),230
hn-n -c -o 1 2.00 0.0 1. J.C.cistrans-NMA

If the torsion definition has a "negative" periodicity (-2 in the case above), it tells programs reading the parameter file that additional terms are present for that particular connectivity. The equation to be applied for **hn-n -c -o** is:

$$E_{\text{torsion}} = 2.00 \times [1 + \cos(1 \times \phi - 0.0)] + 2.50 \times [1 + \cos(2 \times \phi - 180.0)] \quad (15.6)$$

Equation 15.6 prefers the *trans* amide ($\phi = 180^\circ$) over the *cis* amide ($\phi = 0$) by 4 kcal/mol considering the torsion term alone. However the more favourable Coulomb term (the 1-4 attractive interaction between the negative carbonyl oxygen and the positive amide hydrogen) reduces the overall preference for the *trans* conformation close to the experimental value of ca. 2 kcal/mol.

In addition, the following general terms have to be applied for the torsions involving R1 and R2 in the peptide bond R1-NH-C(=O)-R2, in order to compute the high torsional barrier of an amide bond:

X -c -n -X 4 10.000 180.000 2.000

Torsional terms are obviously the most difficult part to parametrize in a force field. They are in a way the last rescue to get torsional barriers right, after all other terms have been adjusted. Therefore, their transfer from one force field to the other is always most risky and acceptable only if all other involved terms in two force fields are very similar. Transferability must always be validated.

Out-of-Plane Terms

Out-of-plane terms are handled via a Fourier term, similar to the torsion terms. But the four involved atoms are not serially (linearly) bonded, they are "branched". The "central" atom is the atom that is forced into the plane of the other three. For example, to keep a carbonyl group R1-C(=O)-R2 planar, the central C atom must be forced into the plane of the other three connected items R1, R2, and O. The entry in the GAFF parameter file for this term is

X -X -c -o 10.5 180. 2. JCC,7,(1986),230

Note that in Amber the central atom type (here c) is the **third** in the definition. The order of the remaining atoms should (by definition) be alphabetic in atom type. The phase is always 180° . In all-atom force fields, the periodicity is always 2.

Out-of-plane terms are the only terms that are allowed to be "missing" in Amber parameter files. Common ones are added automatically by tools like *tLeap*. In many cases, these terms are "cosmetics" that avoid "in principle" planar structures from getting distorted under the influence of other forces (e.g., fused rings, planar nitrogens with three substituents, etc...). The actual parameterisation is often intuitive and for many entries, the ("generic") parameters are identical.

1-4 Non-Bonded Interaction Scaling

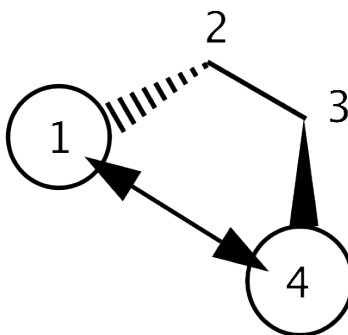


Figure 15.1.: 1-4 Interactions between atoms "1" and "4".

Non-bonded interactions between atoms separated by three consecutive bonds (as schematically shown in Figure 15.1) require a special treatment in Amber force fields. Although referring to non-bonded interactions, scaling information is included in the torsion terms part of the parameter files.

By default, vdW 1-4 interactions are divided (scaled down) by a factor of 2.0, electrostatic 1-4 terms by a factor of 1.2. These are default values for the protein force fields and GAFF, but not for sugar force fields GLYCAM.06EP and GLYCAM.06, for example, in which these interactions are not scaled at all.

Without any additional information, programs like *tLeap*, used to prepare *parmtop* files, assume that the standard scaling mentioned above is to be applied. However, this default can be overwritten in the torsion section of the parameter file. An example is shown below for torsional terms in the GLYCAM.06j force field:

```
S -Ng-Cg-H1 1 2.00 0.0 1. SC EE=1.0 SCNB=1.0 N-Sulfates
S -Ng-Cg-Cg 1 0.0 0.0 -3. SC EE=1.0 SCNB=1.0 N-Sulfates
```

The special notation `SC EE=1.0 SCNB=1.0` following the standard torsion terms⁴ will tell *tLeap* to prepare a *parmtop* file which transfers these data into a special section, as shown below:

```
%FLAG SC EE_SCALE_FACTOR
%FORMAT(5E16.8)
scaling factors are entered here....
%FLAG SCNB_SCALE_FACTOR
%FORMAT(5E16.8)
scaling factors are entered here....
```

When using standard Amber force field parameter files as delivered with AmberTools, the user does not need to care about this. However, when adding additional parameters, especially torsion terms, one should be aware of these scaling factors and decide if they should be default or altered.

15.1.7. Non-Bonded Terms

Van der Waals Parameters

The standard formulation of the 6-12 Lennard-Jones potential $V_{i,j}$ between two atoms i and j is:

⁴In this case, the fields coming after the periodicity (field 5), i.e., fields 6 and 7 are also read and are not 'just' comment!

$$V_{i,j} = 4\epsilon_{i,j} \left[\left(\frac{\sigma_{i,j}}{r_{i,j}} \right)^{12} - \left(\frac{\sigma_{i,j}}{r_{i,j}} \right)^6 \right] \quad (15.7)$$

Here, $r_{i,j}$ is the distance separating the two atoms, $\epsilon_{i,j}$ is the depth of the potential well for the interaction of atoms i and j , and $\sigma_{i,j}$ is the distance where the potential is exactly zero, i.e., where 'repulsion' starts for the two atoms. Both $\epsilon_{i,j}$ and $\sigma_{i,j}$ are specific for **the pair** of atoms (or more precisely, 'atom types').

Another possible formulation of $V_{i,j}$, relating to the concept of van der Waals radii, is:

$$V_{i,j} = \epsilon_{i,j} \left[\left(\frac{R_{min}}{r_{i,j}} \right)^{12} - 2 \left(\frac{R_{min}}{r_{i,j}} \right)^6 \right] \quad (15.8)$$

In this case, R_{min} is the sum of the van der Waals radii, $R_i + R_j$ of atoms i and j , the contact distance at which the potential is at its minimum, i.e., at a value of $-\epsilon$.

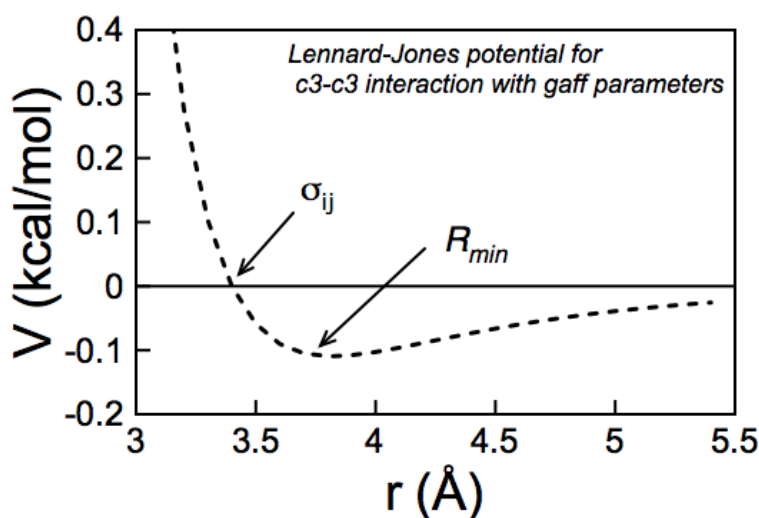


Figure 15.2.: Example of Lennard-Jones potential: the used data are those for the c3 atom type in the gaff force field (vdW radius $R_{min} = 1.908 \text{ Å}$, $\epsilon = 0.1094 \text{ kcal/mol}$)

Combining equations (15.7) and (15.8) gives for the relation between σ and R_{min} :

$$R_{min} = 2^{1/6} \sigma \text{ or } \sigma = 2^{-1/6} R_{min} \quad (15.9)$$

In force fields, the 'A,B' notation of the Lennard-Jones potential is commonly used:

$$V_{i,j} = \frac{A_{i,j}}{r_{i,j}^{12}} - \frac{B_{i,j}}{r_{i,j}^6} \quad (15.10)$$

where $A_{i,j}$ and $B_{i,j}$ are specific parameters for atom type pairs i and j . The meaning of $A_{i,j}$ and $B_{i,j}$ are easily deduced from equation (15.7):

$$A = 4\epsilon\sigma^{12} \text{ and } B = 4\epsilon\sigma^6 \quad (15.11)$$

or, in terms of R_{min} , using equation (15.8):

$$A = \epsilon R_{min}^{12} \text{ and } B = 2\epsilon R_{min}^6 \quad (15.12)$$

Van der Waals data in Amber force fields are given for each atom as a single data pair, a radius R_{min} ('van der Waals' radius in Å) and an energy ϵ (kcal/mol) representing the depth of the potential well.

These values are given at the end of the force field parameter files. In protein force fields, lines above these data show equivalences. For example the line

```
N NA N2 N* NC NB NT NY
```

indicates that all atom types following N (the amide nitrogen) inherit the same Lennard-Jones parameters. Thus, no entry for NA, N2, ... has to be given explicitly.

For Amber force fields, cross terms involving different atom types i and j are evaluated according to the Lorentz/Berthelot mixing rules:

$$\sigma_{i,j} = 0.5(\sigma_{i,i} + \sigma_{j,j}) \text{ or } R_{min,i,j} = 0.5(R_{min,i} + R_{min,j}) \quad (15.13)$$

$$\epsilon_{i,j} = \sqrt{\epsilon_{i,i} \cdot \epsilon_{j,j}} \quad (15.14)$$

The *parmtop* file entries are in 'A' and 'B' terms to be used directly with equation 15.10, transforming the $[R_{min}, \epsilon]$ data pairs from the parameter files.

As an example, consider ethanol ($\text{CH}_3\text{CH}_2\text{OH}$) with the GAFF force field. There are five different GAFF atom types. Below are shown the corresponding $[R_{min}, \epsilon]$ data pairs, as found in the *gaff.dat* parameter file:

```
h1 1.3870 0.0157 Veenstra et al JCC,8, (1992),963
hc 1.4870 0.0157 OPLS
ho 0.0000 0.0000 OPLS Jorgensen, JACS,110, (1988),1657
oh 1.7210 0.2104 OPLS c3 1.9080 0.1094 OPLS
```

Note that there are three different hydrogen types: *hc*, the default H atom connected to an aliphatic carbon, *h1*, a hydrogen type connected to an aliphatic carbon with one electronegative substituent (the oxygen in this case), and the hydroxyl hydrogen *ho* (for which van der Waals interactions are neglected in Amber).

Partial Charges

For Amber force fields, partial charges do not appear in parameter files. For proteins and nucleic acid force fields that use fragment (residue) libraries, partial charges are pre-defined and have been computed from electrostatic-potential fitting of high-level *ab initio* QM. They are automatically assigned by tools like *tLeap*. Library files are found in the folder `$AMBERHOME/dat/leap/lib`.

Below is shown the alanine (ALA) residue of the library file `all-amino94.lib`:

```
"N" "N" 0 1 131072 1 7 -0.415700
"H" "H" 0 1 131072 2 1 0.271900
"CA" "CT" 0 1 131072 3 6 0.033700
"HA" "H1" 0 1 131072 4 1 0.082300
"CB" "CT" 0 1 131072 5 6 -0.182500
"HB1" "HC" 0 1 131072 6 1 0.060300
"HB2" "HC" 0 1 131072 7 1 0.060300
"HB3" "HC" 0 1 131072 8 1 0.060300
"C" "C" 0 1 131072 9 6 0.597300
"O" "O" 0 1 131072 10 8 -0.567900
```

The partial charges for each atom are given in the last field of each line.

For the GAFF force fields, there are various options to compute partial charges; the AM1-BBC method is probably the best trade-off between quality and speed. There are other file types that can contain user-specified partial charges, e.g., SYBYL mol2 files. See the *antechamber* documentation for details.

In *parmtop* files, partial charges are not entered as fragments of the electron charge, but are multiplied by the square-root of 332.05 (= 18.22), because the factor 332.05 converts the Coulomb energy into kcal/mol when using fragments of the electron charge in the Coulomb term of equation 15.1.

15.1.8. Final Remarks

Most parameters in Amber force fields have been tested on a large variety of structures. In rare cases, situations are encountered where structures look "strange" or where results are obviously wrong. One should first look into details of the simulation conditions and settings before blaming the problem on actually flawed force field parameters. Simple test cases are often helpful to resolve the enigma.

When changing or adding parameters and later publishing results, new parameter should be mentioned. Also, the Amber developers team should be notified about possibly problematic parameters. This ensures that potential errors are corrected via patches in later versions and it will help the entire user community.

15.2. ParmEd

ParmEd (*parmed*) is a topology file editor written in Python that enables high level control of the primary force field file in Amber: the *prmtop* file. ParmEd will modify the topology file and produce a new topology file that will work with *sander*, *pmemd*, and NAB programs, and provides options unavailable otherwise. ParmEd currently supports topology files created with both *tleap* and *chamber* (but support is very limited for those created with *tinker.to.amber*).

15.2.1. Running parmed

parmed is used in a manner very similarly to *cpptraj*.

```
usage: parmed [-h] [-v] [-i FILE] [-p <prmtop>] [-c <inpcrd>] [-O]
             [-l FILE] [--prompt PROMPT] [-n] [-e] [-s] [-r]
             [<prmtop>] [<script>]

positional arguments:
  <prmtop>              Topology file to analyze.
  <script>              File with a series of ParmEd commands to execute.

optional arguments:
  -h, --help            show this help message and exit
  -v, --version         show program's version number and exit

Input Files:
  -i FILE, --input FILE
                        Script with ParmEd commands to execute. Default reads
                        from stdin. Can be specified multiple times to process
                        multiple input files.
  -p <prmtop>, --parm <prmtop>
                        List of topology files to load into ParmEd. Can be
                        specified multiple times to process multiple
                        topologies.
  -c <inpcrd>, --inpcrd <inpcrd>
                        List of inpcrd files to load into ParmEd. They are
                        paired with the topology files in the same order that
                        each set of files is specified on the command-line.

Output Files:
  -O, --overwrite      Allow ParmEd to overwrite existing files.
  -l FILE, --logfile FILE
                        Log file with every command executed during an
                        interactive ParmEd session. Default is parmed.log

Interpreter Options:
  These options affect how the ParmEd interpreter behaves in certain cases.
  --prompt PROMPT      String to use as a command prompt.
```


-n, --no-splash Prevent printing the greeting logo.
-e, --enable-interpreter Allow arbitrary single Python commands or blocks of Python code to be run. By default Python commands will not be run as a safeguard for your system. Make sure you trust the source of the ParmEd command before turning this option on.

Error Handling:

These options control how ParmEd handles various errors and warnings that appear occur during the course of Action execution

-s, --strict Prevent scripts from running past unrecognized input and actions that end with an error. In interactive mode, actions with unrecognized inputs and failed actions prevent any changes from being made to the topology, but does not quit the interpreter. This is the default behavior.
-r, --relaxed Scripts ignore unrecognized input and simply skip over failed actions, executing the rest of the script. Unrecognized input in the interactive interpreter emits a non-fatal warning.

Like with ptraj and cpptraj, if you do not supply the prmtop or the input_file, it will read the commands from STDIN as you type them.

15.2.2. ParmEd commands (they are all case-insensitive)

All actions that work on a topology file will use the “parm <idx>|<name>” input sequence to operate on a specified topology file. If present, either the topology file loaded <idx> topologies after the first one or the topology file loaded with the given <name> will be modified by that action. If absent, the LAST topology file loaded will be modified. The <idx> ranges from 0 to the total number of loaded topologies minus 1.

(Note: if you actually have a topology file named “1” that is not the second loaded topology file, you will need to address it via an index. That is, integers will always be assumed to be indices unless they are out of the topology file range.)

15.2.2.1. addAtomicNumber

Usage: *addAtomicNumber*

Adds a section in the topology file with the flag ATOMIC_NUMBER in order to identify specific elements. Elements are matched based on their atomic masses in the MASS section of the topology file. An atom is assigned an element by matching it with the element on the periodic table whose atomic mass is closest to the atom in question. This approach should work for any atom whose mass is either unchanged from the LEaP output or if that atom’s mass has only been changed to one of its isotopes.

15.2.2.2. addDihedral

Usage: *addDihedral* <mask1> <mask2> <mask3> <mask4> <phi_k> <per> <phase> <scee> <scnb> [type <type>]

Adds a dihedral term (will NOT replace an existing dihedral) between atoms in mask1, mask2, mask3, and mask4. The dihedral is defined around the bond between the atoms in mask2 and mask3. Each mask must define the same number of atoms. For mask1 defines atoms 1,2,3; mask2 defines atoms 11,12,13; mask3 defines atoms 21,22,23; and mask4 defines atoms 31,32,33, then 3 new dihedrals will be added. One between atoms 1, 11, 21, and 31, another between atoms 2, 12, 22, and 32, and a third between atoms 3, 13, 23, and 33. The dihedrals will be set with force constant *phi_k*, periodicity *per*, phase angle *phase*, 1-4 electrostatic scaling factor *scee* (this must be specified – the default Amber value is 1.2 and the default GLYCAM and CHARMM value is 1.0), the 1-4 van

15. Reading and modifying Amber parameter files

der Waals scaling factor *scnb* (this must be specified – the default Amber value is 2.0 and the default GLYCAM and CHARMM value is 1.0). The *type* is either “normal” or “improper”. If this is an improper torsion, *<mask3>* should represent the central atoms bonded to all other atoms in the improper torsion.

End-group interactions are excluded automatically if the two end atoms (atoms 1 and 4) are bonded or angled to each other, or if they appear in a different dihedral. Otherwise, they are included. These are the same rules that *tleap* uses when it creates the topology file, and correctly accounts for complex exclusion rules involving ring systems (of size 4, 5, and 6) as well as multi-term torsion parameters.

15.2.2.3. addExclusions

Usage: *addExclusions <mask1> <mask2>*

Allows you to add arbitrary exclusions to the exclusion list. Every atom in *<mask2>* is added to the exclusion list for each atom in *<mask1>* so that non-bonded interactions between those atom pairs will not be computed. NOTE that this ONLY applies to direct-space (short-range) non-bonded potentials. For PME simulations, long-range electrostatics between these atom pairs are still computed (in different unit cells).

15.2.2.4. addLJType

Usage: *addLJType <mask> [radius <new_radius>] [epsilon <new_epsilon>] [radius_14 <new_radius14>] [epsilon_14 <new_epsilon14>]*

This command will assign all atoms specified in the given mask to a new van der Waals (VDW) atom type. Note that several different Amber atom types may in fact be the same VDW type, so this command is designed to give you control over changing just a single atom’s (or single Amber atom type’s) VDW parameters. Every atom specified in the mask will be given the SAME type (but different from every other atom in the topology file), even if their original VDW types are different. The parameters [new_radius] and [new_depth] are optional parameters that specify that atom’s radius and well depth, which are combined with every other type’s radius and depth via the canonical Amber combining rules. They default to the original value of the FIRST atom that is matched by the mask.

Note that for *chamber*-created topology files (ONLY), each atom type has separate 1-4 parameters that may be specified as well. Unspecified values will be taken from the default parameters of the first atom type as described above. Any attempt to supply the 1-4 parameters on a normal topology created with LEaP will result in an error.

See the command *printLJTypes* for additional information here. You can use this command to see if *addLJType* may be necessary for what you’re trying to do.

15.2.2.5. addPDB

Usage: *addPDB <filename> [elem] [strict] [allcodes]*

This command replaces the *add_pdb* program that was released in previous AmberTools releases. It reads in a PDB file *<filename>* and adds the following new sections to the topology file:

RESIDUE_CHAINID The chain ID of each residue (if it was added by *tleap* and not in the PDB file, a * is used)

RESIDUE_ICODE PDB insertion code

RESIDUE_NUMBER The original residue number of this residue in the PDB file

ATOM_ELEMENT Atomic element. This section is redundant now that the topology file has an **ATOMIC_NUMBER** section. Therefore, this section is no longer printed by default.

The *strict* keyword turns residue mismatches (excluding solvent) into fatal errors. Note that for nucleic acids, terminal residue names often do not match the residue names in the PDB file because of the added 5 or 3 to the residue name (for the 5’ terminus and 3’ terminus, respectively).

The *elem* keyword will force the **ATOM_ELEMENT** section to be printed to the topology file, but the element will be determined from the **ATOMIC_NUMBER** section (or atomic mass if the former is not present) rather than the atom names as was done in the *add_pdb* program.

The *allicodes* keyword forces insertion codes to be printed even if every one will be blank. This allows parsers that use that section to be sure it will always be present.

Residues not in the PDB will be assigned a CHAINID of '*' and a RESIDUE_NUMBER of 0. While this action is based on, and reproduces the key results of, the historical *add_pdb* program, it is a bit more flexible.

15.2.2.6. add12_6_4

Usage: *add12_6_4* [*<mask>*] [*c4file <c4file>*] | *watermodel <watermodel>*] [*polfile <polfile>*] [*tunfactor <tunfactor>*]

The *add12_6_4* command is designed to create the prmtop files, which contain the C_4 terms between the ions and each atom type in the prmtop file. By using it together with the outparm command, there will be a new flag named "LENNARD_JONES.CCOEF" created in the end of the output prmtop file. The C_4 terms between the ions and "OW" atom type (which is the oxygen atom of the water molecule, here we assume the polarizability of "HW" is equal to zero) has been determined. Detailed information can be found in the papers of Li, Merz and co-workers.[135–137, 145–147]

The C_4 term between two different kinds of ions is calculated by following equation:

$$C_4(M-X) = tunfactor \times \left[\frac{C_4^M(H_2O)}{\alpha_0(H_2O)} \times \alpha_0(X) + \frac{C_4^X(H_2O)}{\alpha_0(H_2O)} \times \alpha_0(M) \right] \quad (15.15)$$

Where M and X mean two different kinds of ions which all have their C_4 values towards "OW" determined. Herein $C_4^M(H_2O)$ and $C_4^X(H_2O)$ are the two C_4 values. $\alpha_0(H_2O)$, $\alpha_0(M)$, and $\alpha_0(X)$ are the polarizabilities of "OW", "M", and "X", respectively.

The C_4 terms between every other atom pair including ion are calculated by following equation (including ion interact with itself, herein M represents ion, Y means the other atom type in the atom pair):

$$C_4(M-Y) = tunfactor \times \frac{C_4^M(H_2O)}{\alpha_0(H_2O)} \times \alpha_0(Y) \quad (15.16)$$

Here we explain the Usage terms:

1. The *<mask>* is the ion which was treated as the ion center when adding the C_4 terms. Please make sure its corresponding "ATOMIC_NUMBER" is correct in the prmtop file, in that this is the criterion used in the code to identify the metal ion. If you want to use 12-6-4 potentials for different kinds of ions in the prmtop file, please specify all of these ions in the *<mask>* together (i.e. *<mask>* contains at least one ion for each kind) other than performing several *add12_6_4* commands for different kinds of ions one after another, because the the current implementation doesn't support the later situation. If no *<mask>* provided, default value :ZN.

2. To add the C_4 term between the ion and the "OW" atom type (the oxygen atom in the water molecules), you can either use your own *<c4file>* (in this way you need to create a *<c4file>* where the first column is the Atom Symbol plus charge and the second column is the corresponding C_4 value) or use the C_4 values stored in *parmed* (in this way you only need to specify the watermodel you are using in the command line, either TIP3P, SPCE, TIP4PEW, OPC3, OPC, FB3, or FB4, where FB3 and FB4 represent the TIP3P-FB and TIP4P-FB water models, respectively). If nothing (neither your own *<c4file>* nor the water model you are using) is specified, the TIP3P water model will be treated as the default and the related values stored in *parmed* will be used.

3. To add C_4 terms between the ion and atom types besides "OW", you need a polarizability file of all the atom types (the default file is \$AMBERHOME/dat/leap/parm/lj.1264.pol.dat, you can also create and use your own polfile where the first column is the Amber Atom Type while the second column is polarizability), and a tunfactor (which is shown in the previous equation - the default value is set as 1.0). The best tunfactor value for each force field may be different. You can also fine tune the optimal value for a specific force field if so inclined.

After using the *add12_6_4* command in the parmed, please don't forget to use the outparm command to output the new prmtop file. One thing need to clarify: for the *<c4file>*, the first column is Atom Symbol plus charge (e.g., Na1, Mg2, Cl-1, ...) and the second column is the C_4 value between the ion and the "OW" atom type; for the *<polfile>*, the first column is the AMBER Atom Type (e.g., HC, CT, N3, OS, Be2+, Mg2+, Ca2+ ...) while the second column is the polarizability.

15.2.2.7. **cd**

This changes into the given directory (just like the UNIX `cd` command).

15.2.2.8. **chamber**

Usage: `chamber -top <RTF> -param <PAR> -str <STR> -psf <PSF> [-crd <CRD>] [-nocmap] [-box <a,b,c[, $\alpha$ , $\beta$ , $\gamma$ ]>|bounding] [-radii <radiusset>]`

CHAMBER (CHArmm $\leftrightarrow$ AMBER) is a tool which enables the use of the CHARMM force field within AMBER's molecular dynamics engines (MDEs). If you make use of the CHARMM force field in Amber, please cite Ref. [99].

AMBER[34] and CHARMM[442, 443] are two approaches to the parametrization of classical force fields that find extensive use in the modeling of biological systems. The high similarity in the functional form of the two potential energy functions used by these force fields, Eq.(15.19 and 15.20), gives rise to the possible use of one force field within the other MDE.

$$V_{\text{AMBER}} = \sum_{\text{bonds}} k(r - r_{eq})^2 + \sum_{\text{angles}} k(\theta - \theta_{eq})^2 + \sum_{\text{dihedrals}} \frac{V_n}{2} [1 + \cos(n\phi - \gamma)] / \\ + \sum_{i < j} \left[\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6} \right] + \sum_{i < j} \left[\frac{q_i q_j}{\epsilon R_{ij}} \right] \quad (15.17)$$

$$V_{\text{CHARMM}} = \sum_{\text{bonds}} k_b(b - b_0)^2 + \sum_{\text{angles}} k_\theta(\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi [1 + \cos(n\phi - \delta)] \\ + \sum_{\text{Urey-Bradley}} k_u(u - u_0)^2 + \sum_{\text{impropers}} k(\omega - \omega_0)^2 + \sum_{\phi, \psi} V_{\text{CMAP}} \\ + \sum_{\text{nonbonded}} \epsilon \left[\left(\frac{R_{\text{min}_{ij}}}{r_{ij}} \right)^{12} - \left(\frac{R_{\text{min}_{ij}}}{r_{ij}} \right)^6 \right] + \frac{q_i q_j}{\epsilon r_{ij}} \quad (15.18)$$

For the implementation of the CHARMM force field within Amber, parameters that are of the same energy term can be directly translated. However, there are differences in the functional forms of the two potentials, with CHARMM having three additional bonded terms. With respect to the 1-4 non-bonded interactions, CHARMM scales these in a different manner: the electrostatic scaling factor (*scee*) is 1.0 in CHARMM but 1.2 in Amber, while the van der Waals scaling factor (*scnb*) is 1.0 within CHARMM but 2.0 in Amber. Additionally, CHARMM uses a different set of parameters in the Lennard-Jones equation for the van der Waals interaction if the two atoms are bonded 1-4 to each other.

The first additional bonded term is CHARMM's two-body Urey-Bradley term, which extends over all 1-3 bonds. The second is a four-body quadratic improper term. The final additional term is a cross term, named CMAP, [444, 445], which is a function of two sequential protein backbone dihedrals. This term originates from differences observed between classically calculated two-dimensional ϕ/ψ peptide free energy surfaces using the CHARMM22 force field and those of experiment. CMAP is a numerical energy correction which essentially transforms the 2D ϕ/ψ classical energy map to match that of a QM calculated map.

Support for these extra terms has required the development of extra sections to Amber's extensible `prmtop` format to accommodate this new information as well as modifications of the precision of existing sections. For example, the CHARMM parameter file stores the equilibrium angle (θ_0 , Eq.15.20) parameter in degrees in its parameter file, while Amber stores it in radians in the `prmtop`. However, during the conversion with *chamber*, this becomes inexact when converted to radians. Within CHARMM this is done internally at runtime and the inexactness is determined by the variable type that will hold the result of this conversion. However, for Amber, this conversion is done at the *chamber* execution stage, and as a result is limited by the precision to which that specific parameter is written to the `prmtop` file. Hence the precision of the `ANGLE.EQUIL.VALUE` has been increased; similar changes were carried out for the `CHARGE` and `VDW` sections for the same reasons. Specifically, the modified sections of the `prmtop` format and the additions to it are as follows:

```
%FLAG CTITLE
```

The keyword CTITLE is used in place of TITLE to specify that this is a CHAMBER prmtop.

```
%FLAG FORCE_FIELD_TYPE
```

```
%FORMAT(i2,a78)
```

```
1 CHARMM 31 *>>>>>>>CHARMM22 All-Hydrogen Topology File for Proteins
```

```
<<
```

This section described the force field in use. The initial integer specifies the number of lines to be read. The keyword CHARMM here indicates that this is the CHARMM force field.

```
%FLAG CHARGE
```

```
%COMMENT Atomic charge multiplied by sqrt(332.0716D0) (CCELEC)
```

```
%FORMAT(3e24.16)
```

The default format for charge has been changed from 5e16.8 to 3e24.16

```
%FLAG CHARMM_UREY_BRADLEY_COUNT
```

```
%COMMENT V(ub) = K_ub(r_ik - R_ub)**2
```

```
%COMMENT Number of Urey Bradley terms and types
```

```
%FORMAT(2i8)
```

This additional section describes the number of CHARMM Urey-Bradley terms present and the total number of Urey-Bradley types in use.

```
%FLAG CHARMM_UREY_BRADLEY
```

```
%COMMENT List of the two atoms and its parameter index
```

```
%COMMENT in each UB term: i,k,index
```

```
%FORMAT(10i8)
```

This additional section lists the atom indexes and parameter lookup index for each of the Urey-Bradley terms.

```
%FLAG CHARMM_UREY_BRADLEY_FORCE_CONSTANT
```

```
%COMMENT K_ub: kcal/mole/A**2
```

```
%FORMAT(5e16.8)
```

This additional section lists the force constant for each of the Urey-Bradley types.

```
%FLAG CHARMM_UREY_BRADLEY_EQUIL_VALUE
```

```
%COMMENT r_ub: A
```

```
%FORMAT(5e16.8)
```

This additional section lists the equilibrium value for each of the Urey-Bradley types.

```
%FLAG CHARMM_NUM_IMPROPERS
```

```
%COMMENT Number of terms contributing to the
```

```
%COMMENT quadratic four atom improper energy term:
```

```
%COMMENT V(improper) = K_psi(psi - psi_0)**2
```

```
%FORMAT(10i8)
```

This additional section lists the number of CHARMM improper terms present.

```
%FLAG CHARMM_IMPROPERS
```

```
%COMMENT List of the four atoms in each improper term
```

```
%COMMENT i,j,k,l,index i,j,k,l,index
```

```
%COMMENT where index is into the following two lists:
```

```
%COMMENT CHARMM_IMPROPER_{FORCE_CONSTANT,IMPROPER_PHASE}
```

```
%FORMAT(10i8)
```

15. Reading and modifying Amber parameter files

This additional section lists the atom indices and index into the parameter arrays for each of the CHARMM improper terms.

```
%FLAG CHARMM_NUM_IMPR_TYPES
%COMMENT Number of unique parameters contributing to the
%COMMENT quadratic four atom improper energy term
%FORMAT (i8)
```

This additional section lists the number of types present for the CHARMM impropers.

```
%FLAG CHARMM_IMPROPER_FORCE_CONSTANT
%COMMENT K_psi: kcal/mole/rad**2
%FORMAT (5e16.8)
```

This additional section lists the force constant for each CHARMM improper types.

```
%FLAG CHARMM_IMPROPER_PHASE
%COMMENT psi: degrees
%FORMAT (5e16.8)
```

This additional section lists the equilibrium phase angle for each of the CHARMM improper types.

```
%FLAG LENNARD_JONES_ACOEF
%FORMAT (3e24.16)
```

The default format for the Lennard Jones A and B coefficients has been changed from 5e16.8 to 3e24.16.

```
%FLAG LENNARD_JONES_14_ACOEF
%FORMAT (3e24.16)
```

This additional section and the corresponding BCOEF section provide the alternative parameters for 1-4 VDW interactions in the CHARMM force field.

In concert with these prmtop additions, the appropriate modifications have to be made within *sander* and *pmemd* to enable the calculation of the energy and derivatives corresponding to these new terms. The intention behind the approach of creating a CHARMM enabled prmtop file is that the use of this prmtop file should be transparent to the user. Once a CHARMM prmtop file is produced by *chamber*, the *sander* and *pmemd* dynamics engines automatically detect the presence of CHARMM parameters in the prmtop file and automatically select the correct parameters and code paths.

WARNING: *The use of an unpatched Amber molecular dynamics engine with a chamber-generated prmtop file will give undefined behavior, leading to incorrect results. If you see the following error at runtime:*

```
ERROR: Flag "TITLE" not found in PARM file
```

it most likely means that you are using an old pmemd or sander executable.

CHAMBER (CHArmm↔AMBER) is a tool which enables the use of the CHARMM force field within AMBER's molecular dynamics engines (MDEs). If you make use of the CHARMM force field in Amber, please cite Ref. [99].

AMBER[34] and CHARMM[442, 443] are two approaches to the parametrization of classical force fields that find extensive use in the modeling of biological systems. The high similarity in the functional form of the two potential energy functions used by these force fields, Eq.(15.19 and 15.20), gives rise to the possible use of one

force field within the other MDE.

$$\begin{aligned}
 V_{\text{AMBER}} = & \sum_{\text{bonds}} k(r - r_{eq})^2 + \sum_{\text{angles}} k(\theta - \theta_{eq})^2 + \sum_{\text{dihedrals}} \frac{V_n}{2} [1 + \cos(n\phi - \gamma)] / \\
 & + \sum_{i < j} \left[\frac{A_{ij}}{R_{ij}^{12}} - \frac{B_{ij}}{R_{ij}^6} \right] + \sum_{i < j} \left[\frac{q_i q_j}{\epsilon R_{ij}} \right]
 \end{aligned} \tag{15.19}$$

$$\begin{aligned}
 V_{\text{CHARMM}} = & \sum_{\text{bonds}} k_b(b - b_0)^2 + \sum_{\text{angles}} k_\theta(\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi [1 + \cos(n\phi - \delta)] \\
 & + \sum_{\text{Urey-Bradley}} k_u(u - u_0)^2 + \sum_{\text{impropers}} k(\omega - \omega_0)^2 + \sum_{\phi, \psi} V_{\text{CMAP}} \\
 & + \sum_{\text{nonbonded}} \epsilon \left[\left(\frac{R_{\text{min}_{ij}}}{r_{ij}} \right)^{12} - \left(\frac{R_{\text{min}_{ij}}}{r_{ij}} \right)^6 \right] + \frac{q_i q_j}{\epsilon r_{ij}}
 \end{aligned} \tag{15.20}$$

For the implementation of the CHARMM force field within Amber, parameters that are of the same energy term can be directly translated. However, there are differences in the functional forms of the two potentials, with CHARMM having three additional bonded terms. With respect to the 1-4 non-bonded interactions, CHARMM scales these in a different manner: the electrostatic scaling factor (*scee*) is 1.0 in CHARMM but 1.2 in Amber, while the van der Waals scaling factor (*scnb*) is 1.0 within CHARMM but 2.0 in Amber. Additionally, CHARMM uses a different set of parameters in the Lennard-Jones equation for the van der Waals interaction if the two atoms are bonded 1-4 to each other.

The first additional bonded term is CHARMM's two-body Urey-Bradley term, which extends over all 1-3 bonds. The second is a four-body quadratic improper term. The final additional term is a cross term, named CMAP, [444, 445], which is a function of two sequential protein backbone dihedrals. This term originates from differences observed between classically calculated two-dimensional ϕ/ψ peptide free energy surfaces using the CHARMM22 force field and those of experiment. CMAP is a numerical energy correction which essentially transforms the 2D ϕ/ψ classical energy map to match that of a QM calculated map.

Support for these extra terms has required the development of extra sections to Amber's extensible prmtop format to accommodate this new information as well as modifications of the precision of existing sections. For example, the CHARMM parameter file stores the equilibrium angle (θ_0 , Eq.15.20) parameter in degrees in its parameter file, while Amber stores it in radians in the prmtop. However, during the conversion with *chamber*, this becomes inexact when converted to radians. Within CHARMM this is done internally at runtime and the inexactness is determined by the variable type that will hold the result of this conversion. However, for Amber, this conversion is done at the *chamber* execution stage, and as a result is limited by the precision to which that specific parameter is written to the prmtop file. Hence the precision of the ANGLE_EQUIL_VALUE has been increased; similar changes were carried out for the CHARGE and VDW sections for the same reasons. Specifically, the modified sections of the prmtop format and the additions to it are as follows:

```
%FLAG CTITLE
```

The keyword CTITLE is used in place of TITLE to specify that this is a CHAMBER prmtop.

```
%FLAG FORCE_FIELD_TYPE
```

```
%FORMAT(i2,a78)
```

```
1 CHARMM 31 *>>>>>>>>>CHARMM22 All-Hydrogen Topology File for Proteins
```

```
<<
```

This section described the force field in use. The initial integer specifies the number of lines to be read. The keyword CHARMM here indicates that this is the CHARMM force field.

```
%FLAG CHARGE
```

```
%COMMENT Atomic charge multiplied by sqrt(332.0716D0) (CCELEC)
```

```
%FORMAT(3e24.16)
```

15. Reading and modifying Amber parameter files

The default format for charge has been changed from 5e16.8 to 3e24.16

```
%FLAG CHARMM_UREY_BRADLEY_COUNT
%COMMENT V(ub) = K_ub(r_ik - R_ub)**2
%COMMENT Number of Urey Bradley terms and types
%FORMAT(2i8)
```

This additional section describes the number of CHARMM Urey-Bradley terms present and the total number of Urey-Bradley types in use.

```
%FLAG CHARMM_UREY_BRADLEY
%COMMENT List of the two atoms and its parameter index
%COMMENT in each UB term: i,k,index
%FORMAT(10i8)
```

This additional section lists the atom indexes and parameter lookup index for each of the Urey-Bradley terms.

```
%FLAG CHARMM_UREY_BRADLEY_FORCE_CONSTANT
%COMMENT K_ub: kcal/mole/A**2
%FORMAT(5e16.8)
```

This additional section lists the force constant for each of the Urey-Bradley types.

```
%FLAG CHARMM_UREY_BRADLEY_EQUIL_VALUE
%COMMENT r_ub: A
%FORMAT(5e16.8)
```

This additional section lists the equilibrium value for each of the Urey-Bradley types.

```
%FLAG CHARMM_NUM_IMPROPERS
%COMMENT Number of terms contributing to the
%COMMENT quadratic four atom improper energy term:
%COMMENT V(improper) = K_psi(psi - psi_0)**2
%FORMAT(10i8)
```

This additional section lists the number of CHARMM improper terms present.

```
%FLAG CHARMM_IMPROPERS
%COMMENT List of the four atoms in each improper term
%COMMENT i,j,k,l,index i,j,k,l,index
%COMMENT where index is into the following two lists:
%COMMENT CHARMM_IMPROPER_{FORCE_CONSTANT,IMPROPER_PHASE}
%FORMAT(10i8)
```

This additional section lists the atom indices and index into the parameter arrays for each of the CHARMM improper terms.

```
%FLAG CHARMM_NUM_IMPR_TYPES
%COMMENT Number of unique parameters contributing to the
%COMMENT quadratic four atom improper energy term
%FORMAT(i8)
```

This additional section lists the number of types present for the CHARMM impropers.

```
%FLAG CHARMM_IMPROPER_FORCE_CONSTANT
%COMMENT K_psi: kcal/mole/rad**2
%FORMAT(5e16.8)
```

This additional section lists the force constant for each CHARMM improper types.

```
%FLAG CHARMM_IMPROPER_PHASE
```



```
%COMMENT psi: degrees
```

```
%FORMAT (5e16.8)
```

This additional section lists the equilibrium phase angle for each of the CHARMM improper types.

```
%FLAG LENNARD_JONES_ACOEF
```

```
%FORMAT (3e24.16)
```

The default format for the Lennard Jones A and B coefficients has been changed from 5e16.8 to 3e24.16.

```
%FLAG LENNARD_JONES_14_ACOEF
```

```
%FORMAT (3e24.16)
```

This additional section and the corresponding BCOEF section provide the alternative parameters for 1-4 VDW interactions in the CHARMM force field.

In concert with these prmtop additions, the appropriate modifications have to be made within *sander* and *pmemd* to enable the calculation of the energy and derivatives corresponding to these new terms. The intention behind the approach of creating a CHARMM enabled prmtop file is that the use of this prmtop file should be transparent to the user. Once a CHARMM prmtop file is produced by *chamber*, the *sander* and *pmemd* dynamics engines automatically detect the presence of CHARMM parameters in the prmtop file and automatically select the correct parameters and code paths.

WARNING: *The use of an unpatched Amber molecular dynamics engine with a chamber-generated prmtop file will give undefined behavior, leading to incorrect results. If you see the following error at runtime:*

```
ERROR: Flag "TITLE" not found in PARM file
```

it most likely means that you are using an old pmemd or sander executable.

This command will read topology information from a CHARMM or XPLOR PSF file and write an AMBER inpcrd and chamber-style prmtop file so the CHARMM force field can be run in *sander* and *pmemd*. PSF files generated by CHARMM, CHARMM-GUI, and VMD are all supported. PSF files are always generated using a set of topology (RTF) files that define the residues (akin to library files in tleap)—these files define the connectivity and atom types of all atoms in the system. Topology and parameter (PAR) files are always paired, so you must use the parameter file here that matches the topology file you used to create the PSF originally. This is *very* important. CHARMM stream files (that define both residue and parameter sections) are also supported, but must be specified using the *-str* flag. Do not pass any topology or parameter files to the *-str* flag. The *-top* and *-param* flags can be specified multiple times.

NBFI terms defined in any stream or parameter files are read and implemented.

-top <RTF> CHARMM Residue Topology File (RTF). This is not needed if the atom types are defined in the parameter files (this seems to be true for CHARMM36 and probably later force fields).

-par <PAR> CHARMM Parameter file. This defines all of the CHARMM parameter files, and the one that corresponds to the topology file used to create the PSF must be used.

-str <STR> CHARMM stream file. Any parameters stored in the stream file will be loaded.

-toppar <RTF|PAR> CHARMM RTF, parameter, or stream file—the type is automatically detected from the name. All standard CHARMM files should work, but if you have changed the file name, you should use either the *-top*, *-par*, or *-str* flags above. Wild-cards are recognized, so you can do something like *-toppar toppar/*36_prot** to get all of the files that contain 36_prot in their name inside the *toppar* directory.

-psf <PSF> CHARMM/XPLOR/VMD Protein Structure File (PSF). This file defines the structure and topology of the system.

15. Reading and modifying Amber parameter files

-crd <CRD> File containing coordinates for the system. CHARMM coordinate, restart, and PDB files are all supported. If this is a PDB file and a CRYST1 record defines a periodic box, the unit cell dimensions will be set from this information. You can use the **-box** argument to override this.

-nocmap Ignore any CMAP terms that may be defined. This is strongly discouraged unless you have a good reason to do it.

-box <a,b,c[, α, β, γ]>|bounding Defines the periodic box dimensions (and will override any PBC defined in the coordinate file). You can either give the keyword “bounding,” which will define the smallest possible orthorhombic box that encloses the centers of all atoms or you can give the unit cell dimensions. If you provide only lengths, the box shape is assumed to be orthorhombic.

-radii <radiusset> The AMBER implicit solvent radius set. The options are equivalent to the “set PBRadii <radiusset>” options in tleap. See page 262 for more information. Available choices are amber6, bondi, mbondi, mbondi2, and mbondi3. Default choice is “mbondi” (same as tleap).

Note, after using this command, the created parm object will be the active parm. You need to use either the **parmout** or **outparm** commands to actually print a topology file (and don’t forget to also print a coordinate file!)

Validation Starting with version c36a2 of CHARMM, a command (**frcdump**) has been implemented which provides a validation route for alternate implementations of the CHARMM force fields. For a given system, this command writes the various force field potential energy contributions, as well as the energy gradient experienced by each atom, to a file using a specific format and to a high precision. The same formatted output can also be generated by the AMBER MDEs to facilitate comparison and to validate that the CHARMM force field is being implemented correctly in Amber’s MDEs.

An example section of a charmm script that will write this output to a file called **charmm_gold.c36a2** is as follows:

```
open unit 20 form write name charmm_gold.c36a2
frcdump unit 20
close unit 20
```

The analogous mdin section for Amber is as follows:

```
&debugf
  do_charmm_dump_gold = 1,
/
```

Given this directive, the Amber MDE will stop after evaluating the potential energy of a system and write the energy and forces pertaining to this to a (hardcoded) file called **charmm_gold** in the same directory as the mdin file. The reader is invited to examine the various example test calculations within the `$AMBERHOME/test/chamber/dev_tests/` directory for in depth examples of the above. For such testing, it is recommended that both the CHARMM binary and the Amber MDE binaries be compiled with the same compiler. Given that CHARMM support within Amber and the *chamber* software is still somewhat experimental, the user is advised to carry out such a comparison before running a long production run.

Known limitations / Issues This is a non-exhaustive list of the current known bugs and/or limitations with *chamber*:

- CHARMM polarization models are not supported. (**IPOL /= 0**)
- The mdout file will contain extra potential energy fields pertaining to the CHARMM terms. This may break or confuse third party scripts that parse such outputs.
- Third party scripts and/or tools which do not correctly parse the extensible prmtop format may have issues with a *chamber*-generated prmtop file.

- The potential energy decomposition components (self, reciprocal, direct, adjusted) of the Particle Mesh Ewald energy generated in the **charmm_gold** file when the **do_charmm_dump_gold = 1** mdin option in Amber do not match with the breakdown used in CHARMM, however, the summation and resulting forces do match.

If other issues are found, the *parmed* authors would be very grateful if these could be reported to them, either via the Amber mailing list and/or directly to the authors. Please ensure that prior to reporting an issue, the *chamber* binary passes the test cases provided with AmberTools. Please provide a standalone example of the problem with all input files present and a script reproducing the sequence of commands that triggers the problem. The posting of large files (> 2 MB) to the Amber mailing list is not recommended; instead one should make the files available on a website somewhere and provide a link to it with the posting to the list.

15.2.2.9. change

Usage: *change* <property> <atom_mask> <new_value>

This command allows you to change the value of an atom's property for every atom in a given mask to a new value. The allowed atomic properties you can modify are the CHARGE (given in units of elementary atomic charges), MASS (in g/mol), RADII (in Angstroms, these are the GB radii), SCREEN (the GB screening parameters), ATOM_NAME, and AMBER_ATOM_TYPE (this is NOT the van der Waals type). Every atom in the mask will be given the same new_value.

NOTE: The prmtop utility used here stores the partial CHARGE array in terms of elementary atomic charges. All charges are multiplied by 18.2223 prior to being written to any new topology file (and is divided by that number when read in from a topology file). Therefore, if you are changing specific atomic charges in this case, specify new charges in elementary atomic charges.

NOTE: This command gives you access to specific atoms. If you want to change all of the GB radii to be compatible with a specific GB model, see the changeRadii command.

15.2.2.10. changeLJPair

Usage: *changeLJPair* <mask1> <mask2> <Rmin> <epsilon>

This command changes a specific pairwise interaction between the atom type of the atoms in mask1 (these must all be the same type) and the atoms in mask2 (these must all be the same type as well). Rmin and Depth are the pre-combined values of these variables, which allows you to define your own combining rules for a specific pair of atoms.

If you want to see which atoms this command will affect, you can use the printLJTypes with either of the given masks to get a list of atoms that share the same type as the atoms in that mask.

This command is similar to NBFIX available through CHARMM.

15.2.2.11. changeLJ14Pair

Usage: *changeLJ14Pair* <mask1> <mask2> <Rmin> <epsilon>

This command is similar to changeLJPair above, except it alters the 1-4 Lennard Jones terms only. Note that this command is only available for *chamber*-created topology files, and it will result in an error if applied to a normal topology created with LEaP.

15.2.2.12. changeLJSingleType

Usage: *changeLJSingleType* <mask> <Rmin> <epsilon>

This command allows you to change the radius and well depth of particular nonbonded atom types. It will set new values for each interaction the selected type has with every other atom type (irrespective if changeLJPair altered one of these terms before).

15. Reading and modifying Amber parameter files

15.2.2.13. changeProtState

Usage: *changeProtState* <mask> <state #>

Changes the protonation state of a residue that is titratable via constant pH simulations in Amber. <mask> must match all atoms of one, and only one, pH-active titratable residue. As of Amber 18, pH-active titratable residues include AS4, GL4, CYS, TYR, HIP, LYS, and PRN.

15.2.2.14. changeRedoxState

Usage: *changeRedoxState* <mask> <state #>

Changes the redox state of a residue that is titratable via constant Redox Potential simulations in Amber. <mask> must match all atoms of one, and only one, redox-active titratable residue. As of Amber 18, the only redox-active titratable residue is HEH.

15.2.2.15. changeRadii

Usage: *changeRadii* <parameter_set>

Parameter set is one of the following: bondi, mbondi, mbondi2, mbondi3, amber6. This command will reset all of the intrinsic GB radii to the specified set without having to recreate a topology file through LEaP.

15.2.2.16. checkValidity

Usage: *checkValidity*

Thoroughly checks the topology file for a wide range of errors. It also checks for common mistakes, like missing disulfide bridges, for instance. More checks are done if a restart file is loaded prior to running this command. If you are getting a strange error from a simulation engine, it may be worth using this to check the prmtop. Note that this action, in particular, requires a version of Python 2.5 to 2.7.

15.2.2.17. defineSolvent

Usage: *defineSolvent* <residue_list>

This command will allow you to define custom solvent residues. The residue_list must be a comma-separated list with no whitespace separating the residue names. This is important for the proper determination of the SOLVENT_POINTERS and ATOMS_PER_MOLECULE sections of the topology file. By default, HOH and WAT residues are recognized as solvent.

15.2.2.18. deleteBond

Usage: *deleteBond* <mask1> <mask2> [verbose]

This command will delete all bonds in which one atom is in mask1 and the other atom is in mask2. It also deletes all other valence terms (angles, Urey-Bradleys,* torsions, impropers, and CMAPs,*) in which a deleted bond was a part. This is distinct from using setBond to assign a force constant of 0 because it also deletes other valence terms and removes those atoms from the respective nonbonded exclusion lists (since they are no longer bonded to each other).

If you use the “verbose” keyword, you will get a printout of every bond that is deleted.

*Some terms are only found in chamber-style topology files specifying a CHARMM force field.

15.2.2.19. deleteDihedral

Usage: *deleteDihedral* <mask1> <mask2> <mask3> <mask4>

Deletes the dihedral around <mask2> and <mask3> in which the end-groups are <mask1> and <mask4>. For multi-term dihedrals, it removes each term.

15.2.2.20. deletePDB

Usage: *deletePDB*

Deletes the flags that are added by *addPDB* (see description above).

15.2.2.21. energy

Usage: *energy* [*cutoff* <cut>] [*igb* <IGB>] [*saltcon* <conc>] | [*Ewald*] [*nodisper*] [*omm*] [*applayer*] [*platform* <platform>] [*precision* <precision model>] [*decompose*]

Computes the energy for a given structure. If you did not load a coordinate file on the command-line, you must use *loadRestrt* (see below) in order to load a set of coordinates (and box dimensions for periodic simulations). The options are:

cutoff <cut> The cutoff, in Angstroms, to use for the nonbonded cutoff. The default value is 1000 for non-periodic systems and 8 for periodic systems.

dumpfrfc <filename> The file name to write atomic forces to. The format is a single header line starting with '#' followed by natom lines with the x, y, and z components of the force space-delimited.

Non-periodic options These options are applied only to non-periodic simulations. If the *prmtop* indicates periodicity (i.e., *IFBOX* > 0), these options are ignored.

igb <IGB> GB model to use. Allowed values are 0, 1, 2, 5, 6, 7, and 8. The values 0 and 6 indicate vacuum electrostatics. The other values match the options available in *sander*, *pmemd*, and *NAB* (see pages 65 for more details).

saltcon <conc> Salt concentration (in Molarity) to use when using GB implicit solvent. See page 67 for more information.

Periodic options These options are applied only to periodic simulations. If the *prmtop* does not indicate periodicity (i.e., *IFBOX* == 2), these options are ignored.

Ewald Use the Ewald sum to compute long-range electrostatics instead of Particle-Mesh Ewald (this is *much* slower than PME for large systems). This is equivalent to setting *ew_type=1* in *sander*.

nodisper Do not use the long-range dispersion correction to correct for Lennard-Jones interactions that are excluded beyond the cutoff. This is equivalent to setting *vdwmeth=0* in *sander* and *pmemd*.

OpenMM-specific options Instead of using the *sander*-Python API to compute energies and forces, you can use OpenMM. OpenMM must be installed and the Python application layer must be available for import. OpenMM cannot currently handle octahedral boxes (or any non-orthorhombic box).

omm This keyword must be present in order to use the OpenMM engine instead of the *sander* Python package. All following options are ignored unless this keyword is present

platform <platform> OpenMM compute platform to use. Options are CUDA, OpenCL, Reference, and CPU. Consult the OpenMM manual for more details.

precision <precision model> OpenMM precision model to use. Options are single, double, and mixed. Reference is always double and CPU is always single. The mixed precision model (default) uses single precision for calculations and double for accumulation.

decompose By default, OpenMM does not decompose energy contributions to different terms (e.g., bond, angle, torsion, etc.). If present, this keyword will make ParmEd break the energies down as much as possible (OpenMM does not compute non-bonded energy terms separately, so Lennard-Jones, 1-4 nonbonded interactions, and electrostatics will all be conflated into a single term). Energy terms are always decomposed when not using the OpenMM API.

applayer If present, this keyword will write a temporary topology file and load an OpenMM system using the support classes bundled with OpenMM directly (rather than using ParmEd's internal OpenMM System creator). This is provided as a way to validate the agreement between OpenMM's application layer and ParmEd.

15.2.2.22. go

Usage: *go*

Stop reading commands and execute every command that has come before. This has exactly the same effect as the End Of File (EOF) character. All commands in a script after "go" will be ignored. Placing "go" as the last line of a script is the same as not including it at all (since the next line contains EOF, which executes the same behavior). Thus, you can get the same behavior from the interactive session by either typing "go" or sending the EOF character (which on unix is CTRL-D)

15.2.2.23. gromber

Usage: *gromber* <top_file> [define <DEFINE[=VAR]>] [topdir <directory>] [radii <radiusset>]

Load a Gromacs topology file with parameters as an Amber-formatted system. Note, if your Gromacs topology file requires any include topology files (as most do), you will need to have Gromacs installed for this to work.

- <top_file>: The Gromacs topology file to load
- <coord_file>: The coordinate file to load into the system. Can be any recognized format (GRO, PDB, mmCIF, inpcrd, etc.)
- *define* <DEFINE[=VAR]>: Preprocessor defines that control the processing of the Gromacs topology file.
- *topdir* <directory>: The directory containing all Gromacs include topology files. This is only necessary if Gromacs is not installed in a location that ParmEd can find.
- *radii* <radiusset>: The GB radius set to use. Can be *mbondi*, *bondi*, *mbondi2*, or *amber6*. Default is *mbondi*

Gromacs topology files do not store the unit cell information. Therefore, in order to make sure that unit cell information is properly assigned to the resulting system, the provided <coord_file> should contain unit cell information (e.g., GRO, PDB, PDBx/mmCIF, and inpcrd files can all store box information).

ParmEd will try to locate the Gromacs topology directory using either the *GMXDATA* or *GMXBIN* environment variables (which should point to the *\$PREFIX/share/gromacs* or *\$PREFIX/bin* directories, respectively, where *\$PREFIX* is the install prefix). If neither is set, the topology directory is located relative to the location of the *gmx* (Gromacs 5+) or *pdb2gmx* (Gromacs 4 or older) in the user's *PATH*. If none of the above is true, the default installation location (*/usr/local/gromacs/share/gromacs/top*) is used. Any provided *topdir* will override default choices (but only for this particular command – future *gromber* actions will use the default location again).

You can provide as many defines as you wish, and the ordering you specify them is preserved. The default value assigned to each define is "1". To provide multiple defines, use the keyword multiple times, for example:

```
define MYVAR=something define MYVAR2=something_else ...
```

It is important to note that Gromacs supports a much larger array of bonded potentials than Amber does. Gromacs supports several different bonded potentials (cubic, quartic, and Morse, just to name a few), while Amber supports only the simple harmonic bond potential. Similarly, Amber only supports quadratic angle potentials, periodic torsions, and the 12-6 Lennard-Jones potential. Through the chamber extensions (see Sec. 15.2.2.8), Amber can also support Urey-Bradley angle potentials, quadratic improper potentials, and correction map (CMAP) potentials.

Other potential energy functions Gromacs supports—like the Buckingham nonbonded potential—cannot be computed in Amber and so will result in an error. ParmEd also currently only supports the Lorentz-Berthelot combining rules, although support for the geometric combining rules is planned.

15.2.2.24. HMassRepartition

Usage: *HMassRepartition* [*<mass>*] [*dowater*]

This action implements hydrogen mass repartitioning in which the mass of each hydrogen is changed to *<mass>* (the default value is 3.024 daltons if no mass is provided). The mass of the heavy atom that the hydrogen is attached to is adjusted so that the total mass remains the same. This allows longer time steps to be taken in dynamics (see the relevant literature regarding this approach; e.g., [142]). By default, partitioning is only applied to the solute since SHAKE on water is handled analytically (via the SETTLE algorithm). Water can be forcibly repartitioned using the keyword *dowater*.

15.2.2.25. help

Usage: *help* [*action*]

This command does one of two things. If *action* is not specified, a list of available commands along with their short usage statement is displayed in a nicely formatted table. If *action* is provided and that action exists, a usage statement along with a short description is printed. This is a useful reference for quick interactive sessions. You can use a single “?” character instead of the word ‘help’.

15.2.2.26. history

Usage: *history*

This command prints a list of the previous commands that were run in ParmEd. This can be useful if you want to turn your interactive ParmEd session into a script (much like the *history* command works in the shell).

15.2.2.27. interpolate

Usage: *interpolate* *<nparm>* [*parm2 <other_parm>*] [*eleonly*] [*prefix <prefix>*] [*startnum<num>*]

This command can be useful to create topology files that are a linear combination of two topology files, specified by *<other_parm>* and the currently active parm (which can be set for this action using the *parm* keyword). If only two parms are loaded (see *listParms*, below), *<other_parm>* defaults to the inactive parm for this action.

The options are described below:

<nparm> Number of topology files that will be generated *in addition to* the two end-state parms.

parm2 <other_parm> The other topology file to use when interpolating prmtops (in addition to the active parm). The selection here works the same as the *parm* keyword for every other action.

eleonly If present, this only interpolates the charge vectors. This is currently the only supported mode, although van der Waals interpolation is planned for future versions.

prefix <prefix> The prefix of the prmtop file names that will be written by this action. Generated topologies will be written as *<prefix>.#*, where # starts from *<num>* (see below) and increases by 1 for each parm. Default is the name of the active parm.

startnum <num> The number to use as a suffix for the first generated parm. Default value is 1.

15.2.2.28. listParms

Usage: *listParms*

This command will list all of the topology file names for the topology files that have been loaded into the main list, highlighting the active one.

15.2.2.29. lmod

Usage: *lmod*

This action adjusts the Lennard Jones parameters to work with the LMOD code in Amber. It changes Lennard-Jones A-coefficients that are 0 to 1000 to improve numerical stability in LMOD conformational searches (ntmin=4) except when `lmod_verbosity=5, 6`, in which case the `parmed/lmod` command should not be used. This action replaces the *lmodprmtop* program.

15.2.2.30. loadCoordinates

Usage: *loadCoordinates* <filename>

This reads a coordinate file and loads the first ste of coordinates found into the active structure. File type is auto-detected, with supported file formats currently including:

- Amber restart file
- Amber NetCDF restart file
- CHARMM coordinate file
- CHARMM restart file
- Amber mdcrd trajectory file
- Amber NetCDF trajectory file
- PDB file
- PDBx/mmCIF file

For trajectories and PDB or mmCIF files with multiple models, the coordinates are taken from the first frame or model. Note, this is a generalization of the *loadRestrt* command, below.

15.2.2.31. loadRestrt

Usage: *loadRestrt* <restart_filename>

This command takes an `inpcrd` or a restart file to assign coordinates to each of the atoms. This is needed for any commands that require coordinates.

15.2.2.32. ls

This is supposed to emulate the Unix 'ls' program as closely as possible, and can be used inside ParmEd in the same way.

15.2.2.33. minimize

Usage: *minimize* [cutoff <cut>] [[igb <IGB>] [saltcon <conc>]] [[restrain <mask>] [weight <k>]] [norun] [script <script_file.py>] [platform <platform>] [precision <precision model>] [tol <tolerance>] [maxcyc <cycles>]

Uses OpenMM to minimize a structure. After this action, the coordinates stored in the topology file are updated with the minimized coordinates (and the minimized structure will be written if a coordinate file is provided in the *outparm* or *parmout* commands).

General options The following options apply to all systems

cutoff <cut> This is the non-bonded cutoff in Angstroms to use for the minimization. For periodic systems, the default value is 8 Angstroms. For non-periodic systems, no cutoff is applied.

restrain <mask> If provided, the given mask will have Cartesian positional restraints applied with the given force constant (see weight below)

weight <k> The restraint weight used in the positional restraints according to (15.21). Note that this force constant is not scaled by 1/2 as it is in Hooke's Law (so it is half the value of the corresponding force constant).

$$E_{restraint} = k(\mathbf{r} - \mathbf{r}_{eq})^2 \quad (15.21)$$

norun Do not run the minimization—just write the script and quit. If no script is requested, an error is raised and nothing is done.

script <script_file.py> The name of a file in which to write a Python script that will perform the desired energy minimization using OpenMM.

tol <tolerance> The tolerance to use to determine when to stop the minimization. Default is 0.001.

maxcyc <cycles> The maximum number of minimization cycles to use. By default there is no limit—the minimization will run until the tolerance is reached.

Implicit Solvent Options The following options apply only to implicit solvent simulations

igb <IGB> GB model to use. Allowed values are 0, 1, 2, 5, 6, 7, and 8. The values 0 and 6 indicate vacuum electrostatics. The other values match the options available in sander, pmemd, and NAB (see page 65 for more details).

saltcon <conc> Salt concentration (in Molarity) to use when using GB implicit solvent. See page 67 for more information.

OpenMM-specific options These options specify some computational details of the OpenMM calculation.

platform <platform> OpenMM compute platform to use. Options are CUDA, OpenCL, Reference, and CPU. Consult the OpenMM manual for more details. If you are using positional restraints, the CPU and Reference platforms will be even slower compared to the OpenCL and CUDA platforms than usual.

precision <precision model> OpenMM precision model to use. Options are single, double, and mixed. Reference is always double and CPU is always single. The mixed precision model (default) uses single precision for calculations and double for accumulation.

15.2.2.34. netCharge

Usage: *netCharge* [mask]

This command will calculate the net charge of all atoms belonging to a specific mask. If no mask is provided, it returns the net charge of all atoms in the topology file.

15.2.2.35. OpenMM

Usage: *OpenMM* [sander/pmemd options] [-platform <platform>] [-precision <precision model>] [dcd] [progress] [script <script_file.py>] [norun]

This action use OpenMM to run a molecular dynamics simulation in a mode very similarly to how sander or pmemd would run the same simulation. It recognizes all of the same command-line options as sander and pmemd in addition to the ones listed above. It will read an mdin file (given by the -i flag) and run an equivalent simulation

15. Reading and modifying Amber parameter files

(or as close to equivalent as possible) using the OpenMM Python application layer. See the OpenMM website (<https://simtk.org/home/openmm>) and manual for more details. If a simulation cannot be done, an error message is emitted.

The computational platform to use (CUDA, OpenCL, CPU, or Reference) can be provided as *<platform>*. By default, the fastest platform detected will be used. The precision model can be used to specify the precision of the variables that will be used as *<precision model>*. Currently supported options are “mixed” (single precision for calculations, double precision for accumulation), “single” (everything is done in pure single precision), and “double” (everything is done in pure double precision). As of OpenMM 6.0, only the CUDA and OpenCL platforms support multiple precision models. CPU is always single and Reference is always double.

The default prmtop that will be used is the active topology file, although either the *parm* or *-p* flags can be used to specify a different one. The *progress* keyword makes ParmEd print a message when it starts a new phase of the simulation.

The *script* keyword allows you to specify the name of a file in which a Python script that runs an equivalent calculation that ParmEd is running is printed to. This allows you to both inspect what ParmEd is doing behind-the-scenes with the OpenMM Python application layer as well as implement functionality not supported by Amber (but supported by OpenMM) without having to do the potentially laborious setup beforehand. The latter is particularly useful with the *norun* keyword, which will prevent ParmEd from running any dynamics or minimization.

The *dcd* keyword can be used to make ParmEd print the trajectory in DCD format. This is useful if you do not have a NetCDF Python package installed (any of *scipy*, *ScientificPython*, or *netCDF4* will work), but still wish to generate a binary trajectory file.

See Chapter 22 and Chapter 23 for a more thorough description of the *sander/pmemd options*.

Some caveats for this action are listed below.

- The OpenMM package and the Python application layer must be installed and importable from the Python environment. ParmEd supports only OpenMM version 6.0 or higher.
- OpenMM itself requires Python 2.6 or later, which in turn passes on this requirement to this command in ParmEd.
- A NetCDF package for Python must be installed (for the Python interpreter used during the AmberTools configure step) and available to either read or write NetCDF trajectories and restarts. Supported NetCDF-Python packages are *netCDF4*, *ScientificPython*, or *scipy* (provided that the NetCDF bindings of those packages are included in the install). The *scipy* package is recommended.
- Trajectory file and restart file writing from the Python application layer are *very* slow, especially for ASCII versions of the files. NetCDF and DCD files are notably faster to write, but still incur significant overhead. Increasing the intervals for data printouts (*ntpr*, *ntwx*, *ntwv*, *ntwf*, and *ntwr* in the *mdin* file, for instance) can significantly improve computational performance, particularly for the GPU-enabled platforms.
- Not all features in *sander* and *pmemd* are supported, and not all unsupported options may be caught currently.

15.2.2.36. outCIF

Usage: *outCIF* *<file>* [*norenumber*] [*anisou*]

This will write a PDBx/mmCIF file from the currently active system. This is the new file format used by the Protein Data Bank in preference to the traditional PDB file format. The various options are described below:

<file> The name of the PDBx/mmCIF file to write

norenumber If this keyword is given, the original atom and residue numbering from the input structure are used rather than using the internal ordering used by Amber programs. If you used *addPDB* previously to add this information to the prmtop, this keyword will respect the numbering in the *original* PDB file. This will also work if you loaded your parm file from a PSF, PDB, or CIF file that may contain non-sequential numbering.

anisou If anisotropic B-factors are present, print them to the PDBx/mmCIF file.

15.2.2.37. outparm

Usage: *outparm* <prmtop_name> [<restrt_name>]

This command is just like *parmout*, except it can occur as many times as you want it to, and that topology file is written in the order in which that command is placed in the input file or read from STDIN (similar to *outtraj* in *cpptraj*). If you provide a file name for *restrt_name*, *parmed* will also write a valid restart file from the provided initial coordinates and velocities (if present) from the restart file added via the *loadRestrt* command. It will include velocities if they were present in the initial restart file. Note this is most useful when used in conjunction with the “strip” command. If all solvent is stripped, the box information will be discarded. If you do not strip all solvent molecules, the box info will remain unchanged from the original (even if you strip a large number of solvent molecules). If you removed a large number of solvent molecules, take care to re-equilibrate the density before continuing with production dynamics.

15.2.2.38. outPDB

Usage: *outPDB* <file> [*norenumber*] [*charmm*] [*anisou*]

This will write a PDBx/mmCIF file from the currently active system. This is the new file format used by the Protein Data Bank in preference to the traditional PDB file format. The various options are described below:

<file> The name of the PDBx/mmCIF file to write

norenumber If this keyword is given, the original atom and residue numbering from the input structure are used rather than using the internal ordering used by Amber programs. If you used *addPDB* previously to add this information to the *prmtop*, this keyword will respect the numbering in the *original* PDB file. This will also work if you loaded your *parm* file from a PSF, PDB, or CIF file that may contain non-sequential numbering.

charmm If a CHARMM SEGID identifier is loaded (either from the CHARMM PSF file or a CHARMM-modified PDB file), print that to the PDB file

anisou If anisotropic B-factors are present, print them to the PDB file as ANISOU records.

15.2.2.39. parm

Usage: *parm* <filename> | *parm set* <filename>|<index>

If used with the “set” keyword, the active topology is changed to the one with the given file name or the <index>+1th topology file that was loaded. If used without the “set” keyword, it adds a new topology file to the list of available topologies from the given file name and sets that as the active topology for all future actions. (All previous actions were already applied to the previous ‘active’ topology).

15.2.2.40. parmout

Usage: *parmout* <prmtop_name> [<restrt_name>]

This command is similar to *trajout* in *cpptraj* and *ptraj*. It is ALWAYS the last command executed, and only the last *parmout* command is executed. It writes a topology file with all of the modifications made to it during the course of the whole ParmEd session. If you provide a file name for *restrt_name*, *parmed* will also write a valid restart file from the provided initial coordinates and velocities (if present) from the restart file added via the *loadRestrt* command. It will include velocities if they were present in the initial restart file. Note this is most useful when used in conjunction with the “strip” command. If all solvent is stripped, the box information will be discarded. If you do not strip all solvent molecules, the box info will remain unchanged from the original (even if you strip a large number of solvent molecules). If you removed a large number of solvent molecules, take care to re-equilibrate the density before continuing with production dynamics.

15.2.2.41. **printAngles**

Usage: *printAngles* <mask> [<mask> [<mask>]]

This will print out every angle that involves at least one atom specified by <mask>. If additional masks are given, only the angles in which the three atoms are specified in each of the given masks (with the central atom required to be in the second mask) are printed.

15.2.2.42. **printBonds**

Usage: *printBonds* <mask> [<mask>]

This will print out every bond that involves at least one atom specified by <mask>. If a second mask is given, only bonds in which one atom appears in each mask will be printed.

15.2.2.43. **printDetails**

Usage: *printDetails* <mask>

This command prints atomic details of every atom matching a given mask (atom number, residue number, residue name, atom name, atom type, van der Waals radius, van der Waals well depth, mass, and charge) in standard Amber units. This is a useful command to make sure that every atom you think belongs in a mask actually does belong in the mask (and that no atoms were missed). The mask parser implemented in Python here is (mostly) a copy of ptraj's mask parser implemented in C, but some parts had to be rewritten slightly to adjust for different syntaxes of the two languages. Note, distance-based criteria is not yet implemented in this parser.

15.2.2.44. **printDihedrals**

Usage: *printDihedrals* <mask> [<mask> [<mask> [<mask>]]]

This will print out every dihedral that involves at least one atom specified by <mask>. It labels dihedrals in which end-group interactions are omitted (either because they are in a multiterm dihedral or a ring) with an *M* and improper dihedrals with an *I* in the output. If multiple masks are given, only dihedrals that have one atom in each mask are printed. Ordering is important here, so the first atom must be in the first mask, the second atom in the second, etc. The order can be precisely reversed, but no other ordering is recognized.

15.2.2.45. **printFlags**

Usage: *printFlags*

This command prints every %FLAG present in the topology file (see <https://ambermd.org/FileFormats.php> for a description of what each section labelled with these FLAGS means).

15.2.2.46. **printInfo**

Usage: *printInfo* <flag>

This command just prints out all of the data in a given prmtop %FLAG (see <https://ambermd.org/FileFormats.php> for details)

15.2.2.47. **printLJMatrix**

Usage: *printLJMatrix* <mask>

This function prints out how every atom type interacts with the atom type(s) in <mask>.

15.2.2.48. **printLJTypes**

Usage: *printLJTypes* [mask]

This command prints out each atom's van der Waals, or Lennard-Jones type in the mask, as well as every other atom that shares the same atom type as any type in the mask. If no mask is provided, it prints out that information for every atom. This is particularly useful if you want to see if changing a particular pair interaction will affect

more atoms than you expect. If it turns out that you wish to treat some of the atoms that share the same VDW type differently from one another, you will have to “separate” them by using the `addLJType` command before modifying them.

15.2.2.49. `printPointers`

Usage: `printPointers`

This command will print every pointer along with its name and a short description in the topology file. Solvated topology files will also have their `SOLVENT_POINTERS` printed in the same manner.

15.2.2.50. `quit`

Usage: `quit`

This command will halt *parmed* in its tracks. It is effectively the same as *go* except it will NOT execute any *parmout* command (although any *outparm* command used prior to quitting has already been executed)

15.2.2.51. `scale`

Usage: `scale <FLAG> <factor>`

This action scales all numbers in the *FLAG* section of the topology file by multiplying it by the number *<factor>*. This can be used, for instance, to scale all of the torsion force constants by a particular value in a Hamiltonian replica exchange simulation. [446]

15.2.2.52. `scee`

Usage: `scee <value>`

Allows the user to set/change the value of the electrostatic scaling constant that will be used to scale 1-4 electrostatic interactions. This needs to be set in the *prmtop* since it was removed from the *sander/pmemd* input file in Amber 11. This will apply *<value>* to all dihedral terms.

15.2.2.53. `scnb`

Usage: `scnb <value>`

Allows the user to set/change the value of the VDW scaling constant that will be used to scale 1-4 VDW interactions. This needs to be set in the *prmtop* since it was removed from the *sander/pmemd* input file in Amber 11. This will apply *<value>* to all dihedral terms.

15.2.2.54. `setAngle`

Usage: `setAngle <mask1> <mask2> <mask3> <k> <THETeq>`

Changes (or adds a non-existent) angle in the topology file. Each mask must select the same number of atoms, and an angle will be placed between the atoms in *mask1*, *mask2*, and *mask3* (one angle between atom1 from *mask1*, atom1 from *mask2*, and atom1 from *mask3*, another angle between atom2 from *mask1*, atom2 from *mask2*, and atom2 from *mask3*, etc.)

15.2.2.55. `setBond`

Usage: `setBond <mask1> <mask2> <k> <Req>`

Changes (or adds a non-existent) bond in the topology file. Each mask must select the same number of atoms, and a bond will be placed between the atoms in *mask1* and *mask2* (one bond between atom1 from *mask1* and atom1 from *mask2* and another bond between atom2 from *mask1* and atom2 from *mask2*, etc.)

15.2.2.56. setMolecules

Usage: *setMolecules* [*solute_ions=True|False*]

This command uses its own algorithm to determine system molecularity (which resets SOLVENT_POINTERS and ATOMS.PER_MOLECULE to what they *should* have been set to by *LEaP*). It will also determine if there are any errors in which molecules are not represented as consecutive atoms within a topology file (which won't happen unless you modify it yourself or there is a bug in tleap that prevents it from reordering atoms properly). However, in some unusual systems, tleap has been known to set the molecularity incorrectly, leading to strange segfaults and errors in sander and pmemd. Errors of this type can be caught with *checkValidity* and corrected using this command. It will also allow you to choose whether free ions are treated as part of the solute or part of the solvent.

15.2.2.57. setOverwrite

Usage: *setOverwrite* [*True|False*]

Allows the original topology file to be overwritten. By default, the original prmtop file is protected, and you cannot overwrite it. If you provide no value on this line, then it defaults to True. Note that no check is made if you are overwriting any other existing file (just the original topology).

15.2.2.58. source

Usage: *source* <*file*>

Loads a file with a list of ParmEd commands and executes them immediately.

15.2.2.59. strip

Usage: *strip* <*mask*> [*nobox*]

This will strip every atom that corresponds to the given atom mask out of the topology file altogether. Any bond, angle, or dihedral that it is a part of will be deleted as well. The bond, angle, and dihedral types that are no longer referenced after the atoms are stripped out are deleted from the topology file. All Lennard Jones parameters are kept, however, even if they are no longer used. In this way, any LJ modifications you did before the strip command will remain intact. The nobox keyword will make ParmEd delete the unit cell information from the topology file. This is necessary if you intend to use the resulting topology file for aperiodic simulations (e.g., using GB implicit solvent).

15.2.2.60. summary

Usage: *summary*

This command prints out a summary of topology file contents. If coordinates are present, more information is given (like system density). An example of the output is shown below:

Pure water:

```
Amino Acid Residues:    0
Nucleic Acid Residues:  0
Number of cations:      0
Number of anions:       0
Num. of solvent mols:   4096
Num. of unknown atoms:  0
Total charge (e-):      0.0000
Total mass (amu):       73793.5360
Number of atoms:        12288
Number of residues:     4096
System volume (ang^3):  122023.94
System density (g/mL):  1.004222
```

Implicit solvent protein system:

```

Amino Acid Residues: 108
Nucleic Acid Residues: 0
Number of cations: 0
Number of anions: 0
Num. of solvent mols: 0
Num. of unknown atoms: 0
Total charge (e-): -4.0000
Total mass (amu): 11669.4360
Number of atoms: 1654
Number of residues: 108

```

15.2.2.61. tiMerge

Usage: *tiMerge* <mol1mask> <mol2mask> <scmask1> <scmask2> [<scmask1N>] [<scmask2N>] [<tol>]

This will remove redundant bonding terms and atoms from prmtop files for use in thermodynamic integration calculations with PMEMD. The input topology should have two molecules corresponding to V_0 and V_1 . mol1mask/mol2mask are the atom masks for the molecules that should be merged (for V_0 and V_1 respectively). scmask1/scmask2 are the atom masks that list the unique atoms within the molecules to be merged. These do not necessarily have to be soft core atoms. For instance, removing the charges on a residue in a protein requires two copies of that residue in the prmtop file. These masks can be set to that residue. All atoms not in scmask1/scmask2 but in mol1mask/mol2mask should be the same, as these are considered common atoms. Any bonding terms which involve scmask atoms will be kept, but any extra terms will be removed. scmask1N/scmask2N are only used for atoms that will not be merged. These atoms will be included in the masks for output, so that additional soft core molecules that should not be merged do not have to be manually renumbered. tol specifies how close the coordinates have to be for the atoms in V_0 and V_1 to be considered the same. See Subsection 26.1.8 for a complete description of thermodynamic integration in PMEMD as well as an example of this command.

15.2.2.62. writeFrcmod

Usage: *writeFrcmod* <frcmod_name>

This command will dump a complete frcmod file containing every parameter in your topology file. (Note that the effects of a changeLJPair command will NOT be reflected in the topology file unless the pair you choose is between two atoms with the same VDW type, in which case it will alter *all* pair interactions with that atom type). It assumes the canonical Amber combining rules for VDW terms (Lorentz-Berthelot), and uses each type's interaction with itself to extract the well depths and VDW radii.

15.2.2.63. writeOFF

Usage: *writeOFF* <OFF_File>

Writes an Amber OFF (library) file containing every residue, including terminal residues, found in a given topology file. You must have loaded a coordinate file before running this command.

15.2.3. Examples

This section outlines a couple of example input files for *parmed* with comments describing what each command does. You can try these examples on the test parameter files in \$AMBERHOME/AmberTools/test/parmed (either the normal_prmtop/trx.prmtop or the chamber_prmtop/dhfr_gas.prmtop).

Example 1

```

# This file generates a topology file with the new mbondi3 radii
# optimized for the igb = 8 GB model and changes the charge set
# of LYS 3 (trx.prmtop) to set up for a FEP-like calculation.
# In practice you would need more than just the protonated and
# deprotonated state (you would have to interpolate), but this
# is just a demonstration.

# Change to mbondi3
changeRadii mbondi3

# Output the first topology file
outparm trx_mbondi3_state0.parm7

# Change the charges of the LYS
change charge :3@N -0.3479
change charge :3@H 0.2747
change charge :3@CA -0.24
change charge :3@HA 0.1426
change charge :3@CB -0.10961
change charge :3@HB2,HB3 0.034
change charge :3@CG 0.06612
change charge :3@HG2,HG3 0.01041
change charge :3@CD -0.03768
change charge :3@HD2,HD3 0.01155
change charge :3@CE 0.32604
change charge :3@HE2,HE3 -0.03358
change charge :3@NZ -1.03581
change charge :3@HZ1 0
change charge :3@HZ2,HZ3 0.38604
change charge :3@C 0.7341
change charge :3@O -0.5894

# Output the second topology file
outparm trx_mbondi3_state1.parm7

```

Example 2

```

# This file generates a topology file in which the L-J
# interactions between atoms 10 and 28 have been removed,
# and the L-J interactions between atoms 40, 41, 42, and
# 57 with everybody else has been removed.

# Make atoms 10 and 28 new LJ types, but keep their original
# well depths and radii
addLJType @10
addLJType @28

# Zero the interaction between them
changeLJPair @10 @28 0.0 0.0

# Make atoms 40, 41, 42, and 57 a new LJ type with 0s for
# their parameters to remove all of their LJ interactions
# with every other atom
addLJType @40-42,57 radius 0.0 epsilon 0.0

```



```
# Write the final topology file. This statement could have
# been put anywhere
parmout altered_LJ.parm7
```

15.2.4. Converting Amber files to gromacs and CHARMM

Pengfei Li has prepared some simple python scripts that use ParmEd to convert Amber prmtop file to gromacs and CHARMM formats:

amb2chm_psf_crd.py

```
Usage: amb2chm_psf_crd.py -p prmtop -c inpcrd -f psf
                        -d crd -b pdb [--dict dict_file]
```

Options:

```
-h, --help      show this help message and exit
-p PRMTOP      Prmtop file
-c INPCRD      Inpcrd file
-f PSF         PSF file
-d CRD         CRD file
-b PDB         A PDB file to generate
--dict=DICF    Dictionary file name
```

The program will generate a new PDB file (the -b option). This file will have residue and atom names consistent with the generated PSF and CRD files. This file is for user's reference.

amb2chm_par.py

```
Usage: amb2chm_par.py -i input_file [-f input_file_option]
                        -o output_file [--nat use_new_attype]
```

Options:

```
-h, --help      show this help message and exit
-i INPUTF       The input file
-f FOPT         The input file is a parameter file (1) or just contains file
                names (2) [default: 2]
-o OUTPUTF      The output file
--nat=NEWTYP    Whether to perform atom type transfer [0 means no, 1 means
                yes, default: 1]
```

For the -f option, users can specify it as 1 or 2. 1 means there is only one AMBER dat/frcmod file to convert to CHARMM PAR file and this file name follows the -f option. 2 means there are multiple AMBER dat and/or frcmod files to convert to one single CHARMM PAR file and the file follows the -f option is the file containing the dat and/or frcmod file names (with each dat/frcmod file name is in an independent line). This is the default setting.

For the --nat option, users can specify it as 0 or 1. 0 means atom type transfer will not be made. Which means the PAR file will keep the AMBER atom types. 1 means atom type transfer will be made. Which means the PAR file will have the atom types compatible with the CHARMM force field. This is the default setting.

amb2gro_top_gro.py

```
Usage: amb2gro_top_gro.py -p prmtop -c inpcrd -t top
                        -g gro -b pdb
```

Options:

```

-h, --help    show this help message and exit
-p PRMTOP     Prmtop file
-c INPCRD     Inpcrd file
-t TOP        GROMACS top file
-g GRO        GROMACS gro file
-b PDB        A PDB file to generate

```

The program will generate a new PDB file (the -b option). This file will have residue and atom names consistent with the generated top and gro files. This file is for user's reference.

15.2.5. Advanced Options

This section describes some of the advanced options in *parmed*.

15.2.5.1. Interactive Python Shell

To increase ParmEd's flexibility, you can activate an limited, interactive Python interpreter to inject your own custom Python code into *parmed*'s normal execution. This brings with it the risk that custom code can be malicious if untrusted, so custom code evaluation is disallowed by default. To enable it, use the “-e” or “-enable-interpreter” command-line flag when executing *parmed*. To improve security, import statements are disallowed, although the math module has been imported for basic mathematical operations. To execute a single instruction, begin the command with a “!”. In this case, leading whitespace is eliminated (so leading tabs/spaces are ignored here). For example,

```

bash $ parmed -e -n trx.prmtop
Loaded Amber topology file trx.prmtop

Reading input from STDIN...
> !print amber_prmtop.parm.parm_data['ATOM_NAME'][0:10]
['N', 'H1', 'H2', 'H3', 'CA', 'HA', 'CB', 'HB2', 'HB3', 'OG']

```

To execute a formatted block of code that requires more than one line, use “!!” to indicate to ParmEd that you wish to drop to interpreter mode. Terminate that block of code with another “!!” line. The prompt in STDIN-mode changes to “py >>>”. For example:

```

bash$ parmed -e -n trx.prmtop
Loaded Amber topology file trx.prmtop

Reading input from STDIN...
> !!
py >>> def formatted_print(items):
py >>>     i = 0
py >>>     for item in items:
py >>>         print '%10.4f ' % item,
py >>>         i += 1
py >>>         if i % 5 == 0: print ''
py >>>     print ''
py >>>
py >>> formatted_print(amber_prmtop.parm.parm_data['CHARGE'][0:10])
py >>> !!
    0.1849      0.1898      0.1898      0.1898      0.0567
    0.0782      0.2596      0.0273      0.0273     -0.6714

> quit
Quitting.

```

The main topology class list being worked on is called `amber_prmtop`. The currently ‘active’ topology file is the ‘`parm`’ attribute of the list. You can also access specific topology files using an integer index or the original `prmtop` name. See the API documentation below if you are interested in making custom modifications. Note that it is VERY easy to break a topology file with this approach, so consider this an advanced option. A description of the topology file format can be found on <https://ambermd.org/FileFormats.php>.

WARNING: Variable declarations you make here drop onto the top-level namespace in ParmEd’s normal operating environment. That is, any variable you declare here MIGHT override a critical one for ParmEd. Variable names to avoid using include any of the Python built-in functions and types as well as *line*, *code*, *debug*, *actions*, *ParmError*, *LineToCmd*, *AmberParm*, *output_parm*, and *input*.

15.2.5.2. Extending ParmEd

This section describes what is necessary to add a new action to ParmEd.

All actions are parsed from the `actions.py` file in `$AMBERHOME/AmberTools/src/parmed/parmed/tools` directory. Each action must be its own class that inherits from `Action` and takes an `ArgumentList` as its first argument in its `init` method. All arguments should be extracted from the `ArgumentList` using its `get_next-<type>`, `get_key-<type>`, and `has_key` methods (the `get_key-<type>` and `has_key` methods should be called first). See existing methods as examples. You also need to take care to write the class doc-strings (the string immediately following every class declaration) to be as helpful as possible, because they are used in the help function. You must also add your command’s usage statement in the `Usages` dictionary found at the top of `ParmedActions.py`, or it will be invisible to the help function and interpreter tab-autocompletion. The command name is taken as the first argument from that usage string.

No further action is necessary to add your functionality to ParmEd (and you should never have to edit `parmed` directly – any class put in `actions.py` is immediately accessible by `parmed` as long as it inherits from the `Action` base class). Existing actions provide helpful examples if you choose to expand ParmEd.

15.2.5.3. ParmEd API

ParmEd is a rapidly changing program, and keeping comprehensive API documentation up-to-date is beyond the scope of this manual. Please see <https://parmed.github.io/ParmEd> for project documentation if you wish to use the ParmEd API in your own Python scripts. The documentation there is generated automatically from the source code and is kept up-to-date with the latest version. That said, you may find it useful to use some of the ParmEd commands described previously in your own Python scripts. This is described in the following paragraphs.

The actions in this version of ParmEd have been generalized to make it easy to incorporate them into your own Python scripts. To gain access to the actions, you must import them from the `ParmedTools` package. The `Action` class names are identical to the names printed in Subsection 15.2.2. When cast to a string, the action instance will output what it has done (or will do). The `execute` method bound to each `Action` instance will actually carry out the action on the specified topology file.

You can instantiate a new action in one of two ways, but the first argument must be an `AmberParm` (or `ParmList`) instance in both cases. Then, you can either load a single string with all of the options and key words (the same way as you would type it in *parmed*), or you can enter each argument independently with keywords being added appropriately.

An example showing how to add a new Lennard-Jones atom type is shown below using both techniques described above.

```
import os
import sys
from parmed.amber import AmberParm
from parmed.tools import addLJType

parm = AmberParm('trx.prmtop')

act = addLJType(parm, '@1 radius 0.0 epsilon 0.0')
act.execute()
```

15. Reading and modifying Amber parameter files

```
print 'I just did:\n%s' % act

parm.writeParm('trx_modified.prmtop')

# The following code does the same thing
parm = AmberParm('trx.prmtop')

act = addLJType(parm, '@1', radius=0.0, epsilon=0.0)
act.execute()
print 'I just did:\n%s\n\t...again.' % act

parm.writeParm('trx_modified_2.prmtop')
```

16. Antechamber and GAFF

These are a set of tools to generate files for organic molecules and for some metal centers in proteins, which can then be read into LEaP. The Antechamber suite was written by Junmei Wang, and is designed to be used in conjunction with the general AMBER force field (GAFF) (gaff.dat).[447] See Ref. [448] for an explanation of the algorithms used to classify atom and bond types, to assign charges, and to estimate force field parameters that may be missing in gaff.dat. The python Metal Site Modeling Toolbox (pyMSMT) software package was developed by Pengfei Li, and is described in Section 19.

Like the traditional AMBER force fields, GAFF uses a simple harmonic function form for bonds and angles. Unlike the traditional AMBER force fields, atom types in GAFF are more general and cover most of the organic chemical space. In total there are 33 basic atom types and 22 special atom types. The charge methods used can be HF/6-31G* RESP, AM1-BCC,[449, 450] or ABCG2.[451–453] The force field parametrization was performed entirely with HF/6-31G* RESP charges. (Note that in AM1-BCC, the QM electrostatic potentials that were used as fitting targets were created in a very slightly different manner and then compared to RESP charges, using different scaling factors (i.e. 0.001/0.01 [450] versus 0.0005/0.001 [454].)

To maintain force field integrity and to achieve the best performance, the following is the overall guidance on choosing force fields and charge models. Both gaff and gaff2 were developed using HF/6-31G* RESP charge, therefore, RESP charge is compatible with both general AMBER force fields. It was found that gaff/bcc combination achieves good performance in many scenarios including solvation free energy calculations. For the efficient charge models, bcc or abcg2, gaff/bcc and gaff2/abcg2 combinations are recommended, while gaff2/bcc or gaff/abcg2 are not.

The van der Waals parameters are the same as those used by the traditional AMBER force fields. The equilibrium bond lengths and bond angles came from *ab initio* calculations at the MP2/6-31G* level and statistics derived from the Cambridge Structural Database. The force constants for bonds and angles were estimated using empirical models, and the parameters in these models were trained using the force field parameters in the traditional AMBER force fields. General torsional angle parameters were extensively applied in order to reduce the huge number of torsional angle parameters to be derived. The force constants and phase angles in the torsional angle parameters were optimized using our PARMSCAN package,[455] with an aim to reproduce the rotational profiles depicted by high-level *ab initio* calculations (geometry optimizations at the MP2/6-31G* level, followed by single point calculations at MP4/6-311G(d,p)).

By design, GAFF is a complete force field (so that missing parameters rarely occur); it covers almost all the organic chemical space that is made up of C, N, O, S, P, H, F, Cl, Br and I. Moreover, GAFF is totally compatible with the AMBER macromolecular force fields. It should be noted that GAFF atom types, except metal types, are in lower case, while AMBER atom types are always in upper case. This feature makes it possible to load both AMBER protein/nucleic acid force fields and GAFF without any conflict. One can even merge the two kinds of force fields into one file. The combined force fields are capable of studying complicated systems that include both proteins/nucleic acids and organic molecules. We believe that the combination of GAFF with AMBER macromolecular force fields will provide a useful molecular mechanical tool for rational drug design, especially in binding free energy calculations and molecular docking studies. Since its introduction, GAFF has been used for a wide range of applications, including ligand docking,[456] bilayer simulations,[100, 457] and the study of pure organic liquids [458]; see also a recent overview of general force fields for small molecules.[459]

16.1. Principal programs

The *antechamber* program itself is the main program of Antechamber. If your molecule falls into any of several fairly broad categories, *antechamber* should be able to process your PDB file directly, generating output files suitable for LEaP. Otherwise, you may provide an input file with connectivity information, i.e., in a format such

as Mol2 or SDF. If there are missing parameters after *antechamber* is finished, you may want to run *parmchk2* to generate a frcmod template that will assist you in generating the needed parameters.

16.1.1. antechamber

This is the most important program in the package. It can perform many file conversions, and can also assign atomic charges and atom types. As required by the input, antechamber executes the following programs: *sqm* (or, alternatively, *mopac* or *divcon*), *atomtype*, *am1bcc*, *bondtype*, *espgen*, *resp* and *prepgen*. It typically produces many intermediate files; these may be recognized by their names, in which all letters are upper-case. If you experience problems while running *antechamber*, you may want to run the individual programs that are described below (to facilitate this run antechamber with the option '-s 2').

Antechamber options:

```
-help print these instructions
-i      input file name
-fi     input file format
-o      output file name
-fo     output file format
-c      charge method
-cf     charge file name
-nc     net molecular charge (int)
-a      additional file name
-fa     additional file format
-ao     additional file operation
      crd  : only read in coordinate
      crg  : only read in charge
      radius: only read in radius
      name  : only read in atom name
      type  : only read in atom type
      bond  : only read in bond type
-m      multiplicity (2S+1), default is 1
-rn     residue name, overrides input file, default is MOL
-rf     residue topology file name in prep input file,
      default is molecule.res
-ch     check file name for gaussian, default is 'molecule'
-ek     mopac or sqm keyword, inside quotes; overwrites previous ones
-gk     gaussian job keyword, inside quotes, is ignored when both -gopt and -gsp are used
-gopt   gaussian job keyword for optimization, inside quotes
-gsp    gaussian job keyword for single point calculation, inside quotes
-gm     gaussian memory keyword, inside quotes, such as "%mem=1000MB"
-gn     gaussian number of processors keyword, inside quotes, such as "%nproc=8"
-gdsk   gaussian maximum disk usage keyword, inside quotes, such as "%maxdisk=50GB"
-gv     add keyword to generate gesp file (for Gaussian 09 only)
      1    : yes
      0    : no, the default
-ge     gaussian esp file generated by iop(6/50=1), default is g09.gesp
-tor    torsional angle list, inside a pair of quotes, such as "1-2-3-4:0,5-6-7-8"
      ':1' or ':0' indicates the torsional angle is frozen or not
-df     am1-bcc precharge flag, 2 - use sqm(default); 0 - use mopac
-at     atom type
      gaff : the default
      gaff2: for gaff2 (beta-version)
      amber: for PARM94/99/99SB
      bcc  : bcc
      abcg2: abcg2
```

```

sybyl: sybyl
-du    fix duplicate atom names: yes(y) [default] or no(n)
-bk    component/block Id, for ccif
-an    adjust atom names: yes(y) or no(n)
       the default is 'y' for 'mol2' and 'ac' and 'n' for the other formats
-j     atom type and bond type prediction index, default is 4
       0    : no assignment
       1    : atom type
       2    : full bond types
       3    : part bond types
       4    : atom and full bond type
       5    : atom and part bond type
-s     status information: 0(brief), 1(default) or 2(verbose)
-eq    equalizing atomic charge, default is 1 for '-c resp' and '-c bcc' and 0 for the other charge
       0    : no use
       1    : by atomic paths
       2    : by atomic paths and structural information, i.e. E/Z configurations
-pf    remove intermediate files: yes(y) or no(n) [default]
-pl    maximum path length to determin equivalence of atomic charges for resp and bcc,
       the smaller the value, the faster the algorithm, default is -1 (use full length),
       set this parameter to 10 to 30 if your molecule is big (# atoms >= 100)
-seq   atomic sequence order changable: yes(y) [default] or no(n)
-dr    acdoctor mode: yes(y) [default] or no(n)

-i -o -fi and -fo must appear in command lines and the others are optional

Use 'antechamber -L' to list the supported file formats and charge methods

```

List of the File Formats:

file format	type	abbre.	index	file format	type	abbre.	index
Antechamber		ac	1	Sybyl Mol2		mol2	2
PDB		pdb	3	Modified PDB		mpdb	4
AMBER PREP (int)		prepi	5	AMBER PREP (car)		prepc	6
Gaussian Z-Matrix		gzmat	7	Gaussian Cartesian		gcrt	8
Mopac Internal		mopint	9	Mopac Cartesian		mopcrt	10
Gaussian Output		gout	11	Mopac Output		mopout	12
Alchemy		alc	13	CSD		csd	14
MDL		mdl	15	Hyper		hin	16
AMBER Restart		rst	17	Jaguar Cartesian		jcrt	18
Jaguar Z-Matrix		jzmat	19	Jaguar Output		jout	20
Divcon Input		divcrt	21	Divcon Output		divout	22
SQM Input		sqmcrt	23	SQM Output		sqmout	24
Charmm		charmm	25	Gaussian ESP		gesp	26
Component cif		ccif	27	GAMESS dat		gameess	28
Orca input		orcinp	29	Orca output		orcout	30
pdbqt		pdbqt	31				

AMBER restart file can only be read in as additional file

List of the Charge Methods:

charge method	abbre.	index	charge method	abbre.
---------------	--------	-------	---------------	--------

RESP	resp	1		AM1-BCC	bcc	2
CM1	cm1	3		CM2	cm2	4
ESP (Kollman)	esp	5		Mulliken	mul	6
Gasteiger	gas	7		ABCG2	abcg2	8
Read in charge	rc	9		Write out charge	wc	10
Delete Charge	dc	11				

Examples:

The basic use of *antechamber* is to pick input and output files and formats (via the `-i`, `-fi`, `-o`, `-fo` flags), and choose various options for charge models, atom types, etc. A typical use would be:

```
antechamber -i my.pdb -fi pdb -o my.mol2 -fo mol2 -c bcc -nc 1
```

The only “tricky” part is in generating resp charges, which requires interacting with the Gaussian program, and which varies depending on the version:

Using Gaussian 98 files as input:

- (1) `antechamber -i g98.out -fi gout -o sustiva.resp.mol2 -fo mol2 -c resp -eq 2`
- (2) `antechamber -i g98.out -fi gout -o sustiva.cm2.mol2 -fo mol2 -c cm2`

Using Gaussian03 files as input:

- (11) `antechamber -i g03.out -fi gout -o mtx.mol2 -fo mol2 -c resp`
`-a mtx.pdb -fa pdb -ao name`

Using Gaussian09 (version b1 and beyond):

- (12) `antechamber -i ch3I.mol2 -fi mol2 -o gcrt.com -fo gcrt -gv 1 -ge ch3I.gesp`
run Gaussian09 with gcrt.com as input
`antechamber -i ch3I.gesp -fi gesp -o ch3I.resp.mol2 -fo mol2 -c resp -eq 2`

The following is the detailed explanations of some flags

- nc** This flag specifies the net charge of the input molecule, otherwise, the net charge is read in from the input directly (such as `gout`, `mopout`, `sqmout`, `sqmcert`, `gcrt`, etc.) or calculated by summing the partial charges (such as `mol2`, `prepi`, etc).
- a,-fa,-ao** Sometimes, one wants to read additional information from another file other than the input, the `-ao` flag informs the program to read in which information from the additional file specified with `-a` flag. In Example (11), a `mol2` file is generated from a Gaussian output file with atom names read in from a `pdb` file.
- ch,-gk,-gm,-gn** Those flags specify the keywords and resource usage in Gaussian calculations
- ge,-gv** The `-ge` flag specifies the file name of `gesp` file generated using `iop(6/50=1)` with Gaussian 09; the `-gv` flag specifies the Gaussian version and the default is `'1'` for Gaussian 09. If one wants to generate Gaussian input files (`gcrt` and `gzmat`) for older Gaussian versions, `-gv` must be set to `'0'`.
- rn** The `-rn` line specifies the residue name to be used; thus, it must be one to three characters long.
- at** This flag is used to specify whether atom types are to be created for the GAFF force field or for atom types consistent with `parm94.dat` and `parm99.dat` (i.e., the AMBER force fields). If you are using *antechamber* to create a modified residue for use with the standard AMBER `parm94/parm99` force fields, you should set this flag to “amber”; if you are looking at a more arbitrary molecule, set it to “gaff”, even if the molecule is intended for use as a ligand bound to a macromolecule described by the AMBER force fields.

- j** This flag instructs the program how to run 'bondtype' and 'atom type'. '-j 1' assumes the bond types already exists; '-j 4' first predicts the connectivity table, then assigns bond and atom types sequentially; '-j 5' reads in connectivity table from the input and then run 'bondtype' and 'atomtype' sequentially. In most situations, '-j 4', the default option, is recommended. However, '-j 5' should be used if the input structure is not good enough and it includes the bond connectivity information (such as mol2, mdl, gzmat, etc.)
- eq** This flag specifies how to do charge equilibration. With '-eq 1', atomic charge equilibration is predicted only by atom paths, in another word, if two or more atoms have exactly same sets of atom paths, they are equivalent and their charges are forced to be same. While '-eq 2' predicts charge equilibration using both atom paths and some geometrical information (E/Z configuration). With the '-eq 2' option, the charges of two hydrogen atoms bonded to the No 2 carbon of chloroethene are different as they adopt different configurations to chlorine (one is cis and the other is trans). Similarly, the two amide hydrogen atoms of acetamide do not share the same partial charge as the amide bond cannot rotate freely. To back-compatible to the older versions, the default is set to '1'

In Example (12), a gcr file of iodine methane is generated and a gesp file named ch3I.gesp is produced when running Gaussian 09 with the default keyword. In Examples (13-15), RESP charges are generated for acetamide using different charge equilibration options. In the following table, the charges are listed for comparison purposes.

atom names	eq = 0	eq = 1	eq = 2
	no equalization	atomic paths	+ geometry
methyl carbon	-0.5190	-0.5516	-0.5193
methyl hydrogen	0.1412/0.1380/0.1396	0.1470	0.1397
carbonyl carbon	0.9673	0.9786	0.9673
oxygen	-0.6468	-0.6463	-0.6468
nitrogen	-1.1189	-1.1219	-1.1189
amide hydrogen	0.4556/0.4429	0.4501	0.4556/0.4429

16.1.2. parmchk2

parmchk2 reads in an ac/mol2/prepi/prepc file, an atomtype similarity index file (the default is \$AMBERHOME/dat/antechamber/PARMCHK.DAT) as well as a force field file (the default is \$AMBERHOME/dat/leap/parm/gaff.dat). It writes out a force field modification (frcmod) file containing any force field parameters that are needed for the molecule but not supplied by the force field (*.dat) file. Problematic parameters, if any, are indicated in the frcmod file with the note, "ATTN, need revision", and are typically given values of zero. This can cause fatal terminations of programs that later use a resulting prmtop file; for example, a zero value for the periodicity of the torsional barrier of a dihedral parameter will be fatal in many cases. For each atom type, an atom type corresponding file (ATCOR.DAT) lists its replaceable general atom types. By default, only the missing parameters are written to the frcmod file. When the "-a" switch is given the value "Y", *parmchk2* prints out all force field parameters used by the input molecule, whether they are already in the parm file or not. This file can be used to prepare the frcmod file used by thermodynamic integration calculations using sander.

Unlike *parmchk* which only checks several substitutions for a missing force field parameter, *parmchk2* enumerates all the possible substitutions and select the one with the best similarity score as the final substitute. Moreover, a penalty score, which measures the similarity between the missing force field parameter and the substitute is provided. The similarity scores are calculated using the similarity indexes defined in the atom type similarity index file (PARMCHK.DAT). A similarity index of a pair of atom types ('A/B') for a specific force field parameter type was generated by calculating the average percent absolute error of two set of force field parameters in gaff. The two set of force field parameters are identical except that one set has atom type 'A' and the other has 'B'. Each atom type pair ('A/B') has nine similarity indexes for nine different types of force field parameters, which are bond equilibrium length, bond stretching force constant, bond equilibrium angle ('A' and 'B' are central atoms), bond angle bending force constant ('A' and 'B' are central atoms), bond equilibrium angle ('A' and 'B' are non-central atoms), bond angle bending force constant ('A' and 'B' are non-central atoms),

torsional angle twisting force constant ('A' and 'B' are inner side atoms), torsional angle twisting force constant ('A' and 'B' are outer side atoms), and improper dihedral angle.

```
parmchk2 -i      input file name
          -o      frcmod file name
          -f      input file format (prepi, prepc, ac, mol2, frcmod, leaplog)
          -s      ff parm set, it is suppressed by "-p" option
                  1 or gaff:      gaff (the default)
                  2 or gaff2:     gaff2
                  3 or parm99:    parm99
                  4 or parm10:    parm10
                  5 or lipid14:   lipid14
          -frc    frcmod files to be loaded, the supported frcmods include
                  ff99SB, ff14SB, ff03 for proteins , bsc1, ol15, ol3 for DNA and yil for
                  eg. ff14SB+bsc1+yil, ff99SB+bsc1
          -p      parmfile, suppress '-s' flag, optional
          -pf     parmfile format
                  1: for amber FF data file (the default)
                  2: for additional force field parameter file
          -afrc   additional frcmod file, no matter using -p or not, optional
          -c      atom type corresponding score file, default is PARMCHK.DAT
          -atc    additional atom type corresponding score file, optional
                  type 'parmchk2 -l' to learn details
          -a      print out all force field parameters including those in the parmfile
                  can be 'Y' (yes) or 'N' (no) default is 'N'
          -w      print out parameters that matching improper dihedral parameters
                  that contain 'X' in the force field parameter file, can be 'Y' (yes)
                  or 'N' (no), default is 'Y'
          -fc     option of force constant calculation for '-f frcmod' or '-f leaplog'
                  1: default behavior (the default option)
                  2: do empirical calculation before using corresponding atom types
          -att    for the frcmod input format, option of performing parmchk
                  1: for all parameters (the default)
                  2: only for those with ATTN
```

Example:

```
parmchk2 -i sustiva.prep -f prepi -o frcmod
```

This command reads in *sustiva.prep* and finds the missing force field parameters listed in *frcmod*.

16.2. A simple example for antechamber

The most common use of the antechamber program suite is to prepare input files for LEaP, starting from a three-dimensional structure, as found in a PDB file. The antechamber suite automates the process of developing a charge model and assigning atom types, and partially automates the process of developing parameters for the various combinations of atom types found in the molecule.

As with any automated procedure, the output should be carefully examined, and users should be on the lookout for any unusual or incorrect program behavior.

Suppose you have a PDB-format file for your ligand, say thiophenol, which looks like this:

```
ATOM      1  CG  TP      1      -1.959   0.102   0.795
ATOM      2  CD1 TP      1      -1.249   0.602  -0.303
ATOM      3  CD2 TP      1      -2.071   0.865   1.963
```

ATOM	4	CE1	TP	1	-0.646	1.863	-0.234
ATOM	5	C6	TP	1	-1.472	2.129	2.031
ATOM	6	CZ	TP	1	-0.759	2.627	0.934
ATOM	7	HE2	TP	1	-1.558	2.719	2.931
ATOM	8	S15	TP	1	-2.782	0.365	3.060
ATOM	9	H19	TP	1	-3.541	0.979	3.274
ATOM	10	H29	TP	1	-0.787	-0.043	-0.938
ATOM	11	H30	TP	1	0.373	2.045	-0.784
ATOM	12	H31	TP	1	-0.092	3.578	0.781
ATOM	13	H32	TP	1	-2.379	-0.916	0.901

(This file may be found at `$AMBERHOME/AmberTools/test/antechamber/tp/tp.pdb`). The basic command to create a mol2 file for LEaP is just:

```
antechamber -i tp.pdb -fi pdb -o tp.mol2 -fo mol2 -c bcc
```

The output file will look like this:

```
@<TRIPOS>MOLECULE
TP
    13    13    1    0    0
SMALL
bcc
@<TRIPOS>ATOM
    1 CG      -1.9590    0.1020    0.7950 ca    1 TP    -0.132000
    2 CD1      -1.2490    0.6020   -0.3030 ca    1 TP    -0.113000
    3 CD2      -2.0710    0.8650    1.9630 ca    1 TP     0.015900
    4 CE1      -0.6460    1.8630   -0.2340 ca    1 TP    -0.137000
    5 C6       -1.4720    2.1290    2.0310 ca    1 TP    -0.132000
    6 CZ       -0.7590    2.6270    0.9340 ca    1 TP    -0.113000
    7 HE2      -1.5580    2.7190    2.9310 ha    1 TP     0.136500
    8 S15      -2.7820    0.3650    3.0600 sh    1 TP    -0.254700
    9 H19      -3.5410    0.9790    3.2740 hs    1 TP     0.190800
   10 H29      -0.7870   -0.0430   -0.9380 ha    1 TP     0.133500
   11 H30       0.3730    2.0450   -0.7840 ha    1 TP     0.134000
   12 H31      -0.0920    3.5780    0.7810 ha    1 TP     0.133500
   13 H32      -2.3790   -0.9160    0.9010 ha    1 TP     0.136500

@<TRIPOS>BOND
    1    1    2 ar
    2    1    3 ar
    3    1   13 1
    4    2    4 ar
    5    2   10 1
    6    3    5 ar
    7    3    8 1
    8    4    6 ar
    9    4   11 1
   10    5    6 ar
   11    5    7 1
   12    6   12 1
   13    8    9 1

@<TRIPOS>SUBSTRUCTURE
    1 TP          1 TEMP          0 ****    ****    0 ROOT
```

16. Antechamber and GAFF

This command says that the input format is pdb, output format is Sybyl mol2, and the BCC charge model is to be used. The output file is shown in the box titled .mol2. The format of this file is a common one understood by many programs. However, to display molecules properly in software packages other than LEaP and gleap, one needs to assign atom types using the '-at sybyl' flag rather than using the default gaff atom types.

You can now run parmchk2 to see if all of the needed force field parameters are available:

```
parmchk2 -i tp.mol2 -f mol2 -o frcmod
```

This yields the frcmod file:

```
remark goes here
MASS
BOND
ANGLE
DIHE
IMPROPER
ca-ca-ca-ha      1.1      180.0      2.0      General improper \
torsional angle (2 general atom types)
ca-ca-ca-sh      1.1      180.0      2.0      Using default value
NONBON
```

In this case, there were two missing dihedral parameters from the gaff.dat file, which were assigned a default value. (As gaff.dat continues to be developed, there should be fewer and fewer missing parameters to be estimated by parmchk2.) In rare cases, parmchk2 may be unable to make a good estimate; it will then insert a placeholder (with zeros everywhere) into the frcmod file, with the comment "ATTN: needs revision". After manually editing this to take care of the elements that "need revision", you are ready to read this residue into LEaP, either as a residue on its own, or as part of a larger system. The following LEaP input file (leap.in) will just create a system with thiophenol in it:

```
source leaprc.gaff
mods = loadAmberParams frcmod
TP = loadMol2 tp.mol2
saveAmberParm TP prmtop inpcrd
quit
```

You can read this into LEaP as follows:

```
tleap -s -f leap.in
```

This will yield a prmtop and inpcrd file. If you want to use this residue in the context of a larger system, you can insert commands after the loadAmberPrep step to construct the system you want, using standard LEaP commands.

In this respect, it is worth noting that the atom types in gaff.dat are all lower-case, whereas the atom types in the standard AMBER force fields are all upper-case. This means that you can load both gaff.dat and (say) parm99.dat into LEaP at the same time, and there won't be any conflicts. Hence, it is generally expected that you will use one of the AMBER force fields to describe your protein or nucleic acid, and the gaff.dat parameters to describe your ligand; as mentioned above, gaff.dat has been designed with this in mind, i.e., to produce molecular mechanics descriptions that are generally compatible with the AMBER macromolecular force fields.

The procedure above only works as it stands for neutral molecules. If your molecule is charged, you need to set the -nc flag in the initial antechamber run. Also note that this procedure depends heavily upon the initial 3D structure: it must have all hydrogens present, and the charges computed are those for the conformation you provide, after minimization in the AM1 Hamiltonian. In fact, this means that you must have a reasonable all-atom initial model of your molecule (so that it can be minimized with the AM1 Hamiltonian), and you may need to specify what its net charge is, especially for those molecular formats that have no net charge information, and no partial charges or the partial charges in the input are not correct. The system should really be a closed-shell molecule, since all of the atom-typing rules assume this implicitly.

Further examples of using antechamber to create force field parameters can be found in the `$AMBERHOME-test/antechamber` directory. Here are some practical tips from Junmei Wang:

1. For the input molecules, make sure there are no open valences and the structures are reasonable. All hydrogen atoms must be present. Antechamber doesn't know what to do with metal ions (see the MCPB.py program for that), or for other non-organic elements such as Boron. Look at the `$AMBERHOME/dat/leap/parm/gaff.dat` file to see what sorts of atomic environments are supported.
2. The Antechamber package produces two kinds of messages: error messages and informative messages. Informative messages begin with "Info:" and may be safely ignored, but they may be helpful for understanding and troubleshooting antechamber. For example: "Info: Bond types are assigned for valence state 1 with penalty of 1". Messages beginning with "Fatal Error!" or "Error:" indicate a problem. Some such messages may mention likely causes or contain suggested workarounds, but all such messages provide clues. Apply common sense and the scientific method to troubleshoot. Typical first steps are to verify input files and to search the AMBER Mail Reflector for similar reported problems. Additional steps are described below.
3. Failures are most often produced when antechamber infers an incorrect connectivity. In such cases, you can revise by hand the connectivity information in "ac" or "mol2" files. Systematic errors could be corrected by revising the parameters in `$AMBERHOME/dat/antechamber/CONNECT.TPL`.
4. It is a good idea to check the intermediate files in case of a program failure, and you can run separate programs one by one. Use the "-s 2" flag to antechamber to see details of what it is doing.
5. *acdoctor* can diagnose many possible problems with input molecules. If you encounter failures when running antechamber programs, it is highly recommended to let *acdoctor* perform a diagnosis. Run the *acdoctor* program or use the *acdoctor* mode in program antechamber; the latter is controlled by option '-dr' and is on by default.
6. By default, the AM1 Mulliken charges that are required for the AM1-BCC procedure are computed using the *sqm* program, with the following keyword (which is placed inside the *&qmmm* namelist):

```
qm_theory="AM1", grms_tol=0.0005, scfconv=1.d-10,
```

For some molecules, especially if they have bad starting geometries, convergence to these tight criteria may not be obtained. If you have trouble, examine the *sqm.out* file, and try changing *scfconv* to 1.d-8 and/or increase the value of *grms_tol*. If you see failures in scf convergence that are not fixed by changing *scfconv*, try adding setting *ndiis_attempts=700*. You can use the `-ek` flag to antechamber to change these: for example

```
antechamber .... -ek "qm_theory='AM1', grms_tol=0.0005, scfconv=1.d-8, ndiis_attempts=700,"
....|
```

But be aware that there may be something "wrong" with your molecule if these problems arise; *acdoctor* may help (see the previous tip).

7. The standard procedure for obtaining AM1-BCC charges calls for a geometry optimization first. [449, 450] For some molecules (especially anions like phosphates) such a vacuum minimization may be inappropriate, since it can lead to formation of intramolecular hydrogen bonds that are not representative of the expected conformations in solution. If you trust your initial geometries, you can add *maxcyc=0* to the `-ek` flag to skip the geometry minimization. You might also want to turn off geometry optimization in order try out several conformations in order to assess the sensitivity of the AM1-BCC charges to input geometry.

16.3. Programs called by antechamber

The following programs are automatically called by antechamber when needed. Generally, you should not need to run them yourself, unless problems arise and/or you want to fine-tune what antechamber does.

16.3.1. atomtype

Atomtype reads in an ac file and assigns the atom types. You may find the default definition files in \$AMBERHOME/dat/antechamber: ATOMTYPE_AMBER.DEF (AMBER), ATOMTYPE_GFF.DEF (general AMBER force field). ATOMTYPE_GFF.DEF is the default definition file. It is pointed out that the usage of atomtype is not limited to assign force field atom types, it can also be used to assign atom types in other applications, such as QSAR and QSPR studies. The users can define their own atom type definition files according to certain rules described in the above mentioned files.

```
atomtype -i input file name
        -o output file name (ac)
        -f input file format(ac (the default) or mol2)
        -p atom type set, suppressed by "-d" option
            gaff : the default
            amber : for PARM94/99/99SB
            bcc  : for AM1-BCC
            abcg2 : for ABCG2
            gas  : for Gasteiger charge
            sybyl : for atom types used in sybyl
        -d atom type definition file, optional
        -a do post atom type adjustment (it is applied with "-d" option)
        -an adjust atom names: yes(y) or no(n)
            y - fix atom names. Default.
            n - don't fix atom names.
```

Example:

```
atomtype -i sustiva_resp.ac -o sustiva_resp_at.ac -f ac -p amber
```

This command assigns atom types for sustiva_resp.ac with amber atom type definitions. The output file name is sustiva_resp_at.ac

16.3.2. am1bcc

Am1bcc first reads in an ac or mol2 file with or without assigned AM1-BCC atom types and bond types. Then the bcc parameter file (the default, BCCPARM.DAT is in \$AMBERHOME/dat/antechamber) is read in. An ac file with AM1-BCC charges [449, 450] is written out. Be sure the charges in the input ac file are AM1-Mulliken charges.

```
am1bcc -i input file name in ac format
        -o output file name
        -f output file format(pdb or ac, optional, default is ac)
        -t am1bcc type, default is 'bcc', can also be 'abcg2'
        -p bcc parm file name (optional)
        -j atom and bond type judge option, default is 0)
            0: No judgement
            1: Atom type
            2: Full bond type
            3: Partial bond type
            4: Atom and full bond type
            5: Atom and partial bond type
```

Example:

```
am1bcc -i compl.ac -o complbcc.ac -f ac -j 4
```

This command reads in `comp1.ac`, assigns both atom types and bond types and finally performs bond charge correction to get AM1-BCC charges. The `-j` option of 4, which is the default, means that both the atom and bond type information in the input file is ignored and a full atom and bond type assignments are performed. The `-j` option of 3 and 5 implies that bond type information (single bond, double bond, triple bond and aromatic bond) is read in and only a bond type adjustment is performed. If the input file is in mol2 format that contains the basic bond type information, option of 5 is highly recommended. `comp1.bcc.ac` is an ac file with the final AM1-BCC charges.

16.3.3. bondtype

`bondtype` is a program to assign six bond types based upon the read in simple bond types from an ac or mol2 format with a flag of `“-j part”` or purely connectivity table using a flag of `“-j full”`. The six bond types as defined in AM1-BCC [449, 450] are single bond, double bond, triple bond, aromatic single, aromatic double bonds and delocalized bond. This program takes an ac file or mol2 file as input and write out an ac file with the predicted bond types. After the continually improved algorithm and code, the current version of `bondtype` can correctly assign bond types for most organic molecules (>99% overall and >95% for charged molecules) in our tests.

Starting with Amber 10, bond type assignment is proceeded based upon residues. The bonds that link two residues are assumed to be single bonded. This feature allows antechamber to handle residue-based molecules, even proteins are possible. It also provides a remedy for some molecules that would otherwise fail: it can be helpful to dissect the whole molecule into residues. Some molecules have more than one way to assign bond types; for example, there are two ways to alternate single and double bonds for benzene. The assignment adopted by `bondtype` is purely affected by the atom sequence order. To get assignments for other resonant structures, one may freeze some bond types in an *ac* or *mol2* input file (appending 'F' or 'f' to the corresponding bond types). Those frozen bond types are ignored in the bond type assignment procedure. If the input molecules contain some unusual elements, such as metals, the involved bonds are automatically frozen. This frozen bond feature enables `bondtype` to handle unusual molecules in a practical way without simply producing an error message.

```
bondtype -i input file name
-o output file name
-f input file format (ac or mol2)
-j judge bond type level option, default is part
full full judgment
part partial judgment, only do reassignment according
to known bond type information in the input file
-s stop running if APS (atomic penalty score) is not available
0 - no, all the related bonds are frozen, the default
1 - yes
-an adjust atom names: yes(y) or no(n)
y - fix atom names. Default.
n - don't fix atom names.
```

Examples can be found in `$AMBERHOME/test/antechamber/bondtype` and `$AMBERHOME/test/antechamber/chemokine`.

16.3.4. prepgen

`Prepgen` generates the prep input file from an ac file. By default, the program generates a mainchain itself. However, you may also specify the main-chain atoms in the main chain file. From this file, you can also specify which atoms will be deleted, and whether to do charge correction or not. In order to generate the amino-acid-like residue (this kind of residue has one head atom and one tail atom to be connected to other residues), you need a main chain file. Sample main chain files are in `$AMBERHOME/dat/antechamber`.

```
prepgen -i input file name(ac)
-o output file name
```

```

-f  output file format (car or int, default: int)
-m  mainchain file name
-rn residue name (default: MOL)
-rf residue file name (default: molecule.res)
    -f -m -rn -rf are optional

```

Examples:

```

prepgen -i sustiva.ac -o sustiva_int.prep -f int -rn SUS -rf SUS.res
prepgen -i sustiva.ac -o sustiva_car.prep -f car -rn SUS -rf SUS.res
prepgen -i sustiva.ac -o sustiva_int.main.prep -f int -rn SUS
        -rf SUS.res -m mainchain_sus.dat
prepgen -i ala_cm2.at.ac -o ala_cm2_int.main.prep -f int -rn ALA
        -rf ala.res -m mainchain_ala.dat

```

The above commands generate different kinds of prep input files with and without specifying a main chain file.

16.3.5. espgen

Espgen reads in a gaussian (92,94,98,03) output file and extracts the ESP information. An esp file for the resp program is generated.

```

espgen -i  input file name
        -o  output file name
        -f  input format:
            1  Gaussian log file(default)
            2  Gaussian ESP file
            3  Gamess ESP file
        -p  generate esp for pGM:
            0  no, the default)
            1  yes
        -dq print out dipole and quadrupole moments:
            0  no, the default)
            1  yes
        -re print out remark line
            0  no, the default)
            1  yes

```

Example:

```

(1) espgen -i sustiva_g98.out -o sustiva.esp
(2) espgen -i ch3I.gesp -o ch3I.esp

```

Command (1) reads in sustiva_g98.out and writes out sustiva.esp, which can be used by the resp program. Command (2) reads in a gesp file generated by Gaussian 09 and outputs the esp file. Note that this program replaces shell scripts formerly found on the AMBER web site that perform equivalent tasks.

16.3.6. respgen

Respgen generates the input files for two-stage resp fitting. Starting with Amber 10, the program supports a single molecule with one or multiple conformations RESP fittings. Atom equivalence is recognized automatically. Frozen charges and charge groups are read in with '-a' flag. If there are some frozen charges in the additional input data file, a RESP charge file, QIN is generated as well. Here are flags to *respgen*:


```

-i input file name(ac)
-o output file name
-l maximum path length (default is -1, i.e. the path can be any long)
-f output file format
    resp1 - first stage resp fitting
    resp2 - second stage resp fitting
    iresp1 - first stage iresp fitting
    iresp2 - second stage iresp fitting
    resp3 - one-stage resp fitting
    resp4 - calculating ESP from point charges
    resp5 - no-equalization
-e equalizing atomic charge (default is 1)
    0 not use
    1 by atomic paths
    2 by atomic paths and geometry (such as E/Z configuration)
-a additional input data (predefined charges, atom groups etc)
-n number of conformations (default is 1)
-w weight of charge constraint
    the default values are 0.0005 for resp1/iresp1 and 0.001 for
    resp2/iresp2

```

The following is a sample of additional respgen input file

```

//predefined charges in a format of (CHARGE partial_charge atom_ID atom_name)
CHARGE -0.417500 7 N1
CHARGE 0.271900 8 H4
CHARGE 0.597300 15 C5
CHARGE -0.567900 16 O2
//charge groups in a format of (GROUP num_atom net_charge),
//more than one group may be defined.
GROUP 10 0.00000
//atoms in the group in a format of (ATOM atom_ID atom_name)
ATOM 7 N1
ATOM 8 H4
ATOM 9 C3
ATOM 10 H5
ATOM 11 C4
ATOM 12 H6
ATOM 13 H7
ATOM 14 H8
ATOM 15 C5
ATOM 16 O2

```

Example:

```

respgen -i sustiva.ac -o sustiva.respin1 -f resp1
respgen -i sustiva.ac -o sustiva.respin2 -f resp2
resp -O -i sustiva.respin1 -o sustiva.respout1 -e sustiva.esp -t qout_stage1
resp -O -i sustiva.respin2 -o sustiva.respout2 -e sustiva.esp
      -q qout_stage1 -t qout_stage2
antechamber -i sustiva.ac -fi ac -o sustiva_resp.ac -fo ac -c rc -cf qout_stage2
respgen -i acetamide.ac -o acetamide.respin1 -f resp1 -e 2
respgen -i acetamide.ac -o acetamide.respin2 -f resp2 -e 2

```

The above commands first generate the input files (sustiva.respin1 and sustiva.respin2) for resp fitting, then do two-stage resp fitting and finally use antechamber to read in the resp charges and write out an ac file, *sustiva_resp.ac*.

A more complicated example has been provided in `$AMBERHOME/test/antechamber/residuegen`. The last two 'respden' commands generate resp input files for acetamide discriminating the two amide hydrogen atoms.

16.4. Miscellaneous programs

The Antechamber suite also contains some utility programs that perform various tasks in molecular mechanical calculations. They are listed in alphabetical order.

16.4.1. acdoctor

acdoctor reads the same input file formats used by *antechamber* program and 'diagnoses' potential issues that can cause antechamber to fail. In AmberTools version 17 the *acdoctor* functionality was added to program antechamber; it is controlled by option '-dr' and is on by default. The first step is to validate some commonly-used molecular formats, such as pdb, mol2, mdl (sdf), etc. Then the presence of any unusual elements (elements other than C, O, N, S, P, H, F, Cl, Br and I) is reported; in AmberTools version 19 the unusual elements check was changed from a warning to a fatal error; please contact the Amber Mail Reflector specifying the unusual element(s) to register your interest in using antechamber on those element(s). Unfilled valences are reported and additional checks are performed when atom types and/or bond types are read for file formats ac, mol2, sdf, prepi, prepc, mdl, alc and hin. The geometry is quantified by a distance matrix and atomic clashes are reported. *acdoctor* also applies a more stringent criterion than that utilized by *antechamber* to determine whether a bond is formed or not. A warning message is printed for those bonds that fail to meet the standard as well as for weird bonds. Next *acdoctor* determines whether all atoms are linked together through atomic paths. If not, an error message is printed. This kind of error typically implies that the input molecule has one or several bonds missing. Finally, *acdoctor* tries to assign bond types and atom types for the input molecule. If no error occurs during running *bondtype* and *atomtype*, presumably the input molecule should be free from problems when running the other Antechamber programs. It is recommended to diagnose your molecules with *acdoctor* when you encounter Antechamber program suite failures.

```
Usage: acdoctor -i input file name
          -f input file format
```

Example:

```
acdoctor -i test.mol2 -f mol2
```

The program reads test.mol2 and checks for potential problems when running the Antechamber programs. Errors and warning messages are printed. (Possible file formats are listed above in Section 16.1.1).

16.4.2. parmcals

parmcals is an interactive program to calculate the bond length and bond angle parameters, according to the rules outlined in Ref. [447].

```
Please select:
1. calculate the bond length parameter: A-B
2. calculate the bond angle parameter: A-B-C
3. exit
```

16.4.3. residuegen

It can be painful to prepare a modified amino acid or nucleotide; the complication is that a residue is not a free standing molecule, and needs to be capped with extra atoms, usually at both termini. For "simple" systems, where a single conformation can be used to estimate partial charges, the *prepgen* program described above with the "-m" flag to specify which atoms to keep in the final residue. For more complex circumstances, the *residuegen*

facilitates residue topology generation. *residuegen* reads in an input file and applies a set of antechamber programs to generate residue topologies in *prepi* format. The program can be applied to generate amino-acid-like topologies for amino acids, nucleic acids and other polymers as well. An example is provided below and the file format of the input file is also explained.

Usage: *residuegen* input_file

Example:

residuegen ala.input

This command reads in *ala.input* and generate residue topology for alanine. The file format of *ala.input* is explained below.

```
#INPUT_FILE:      structure file in ac format, generated from a Gaussian output
INPUT_FILE        ala.ac
#CONF_NUM:        Number of conformations utilized
CONF_NUM          2
#ESP_FILE:        esp file generated from gaussian output with 'espgen'
#                 for multiple conformations, cat all CONF_NUM esp files onto ESP_FILE
ESP_FILE          ala.esp
#SEP_BOND:        bonds that separate residue and caps, input in a format of
#                 (Atom_Name1 Atom_Name2), where Atom_Name1 belongs to residue and
#                 Atom_Name2 belongs to a cap; must show up no more than two times
SEP_BOND          N1 C2
SEP_BOND          C5 N2
#NET_CHARGE:      net charge of the residue
NET_CHARGE        0
#ATOM_CHARGE:     predefined atom charge, input in a format of
#                 (Atom_Name Partial_Charge); can show up multiple times.
ATOM_CHARGE       N1 -0.4175
ATOM_CHARGE       H4 0.2719
ATOM_CHARGE       C5 0.5973
ATOM_CHARGE       O2 -0.5679
#PREP_FILE:       prep file name
PREP_FILE         ala.prep
#RESIDUE_FILE_NAME: residue file name in PREP_FILE
RESIDUE_FILE_NAME: ala.res
#RESIDUE_SYMBOL:  residue symbol in PREP_FILE
RESIDUE_SYMBOL:   ALA
```

16.4.4. match

The *match* program was developed to conduct least-square fittings for two molecules (one input and one reference) which are not necessarily the same in structure. Users can specify which atom or residue in the input corresponds to which in the reference in the definition file (-df). The users can also specify which atoms participating the fitting (-ds). The match matrix can be saved for translating and roating those atoms not participating the fitting procedure in separate step using '-j 2'.

Usage: *match* -i input file name
 -r reference file name
 -f format: 1-pdb (the default), 2-ac, 3-mol2, 4-sdf, 5-crd/rst
 -o output file name
 -l run log file name, default is "match.log"
 -s selection mode
 0: use all atoms (the default)
 1: specify atom names

```

2: use atom definition file
3: use residue definition file - original residue IDs
4: use residue definition file - renumbered residue IDs
-ds definition string if selection modes of '1' or '3' or '4'
    e.g. 'C,N,O,CA', or 'HET' which stands for heavy atoms for '-ds 1')
-df definition file if selection mode of '2' or '3' or '4'
    records take a form of 'ATOM atom_id.input atom_id.reference'
    or 'RES res_id.input res_id.reference'
-n number of atoms participating ls-fitting,
    default is -1, which implies to use all the selected atoms
-m matrix file, default is "match.matrix"
-t job type:
    0: calculate rms only, need -i and -r
    1: lsfit, need -i, -r and -o the default
    2: translation/rotation, need -i, -o and -m

```

Example:

```
match -f pdb -r 1be9.pdb -i 3pdz.pdb -o 3pdz_aligned.pdb -s 4 -ds "CA,C,N,O" -df 3pdz_1
```

The program runs least-square fitting for the non-hydrogen main chain atoms of residues defined in the 3pdz_1be9.corr. A part of the 3pdz_1be9.corr is shown below:

```

RES 34 35 G G
RES 35 36 I I
RES 36 37 Y F
...
RES 87 88 L I
RES 88 89 L I

```

16.4.5. match_atomname

One limitation of the Antechamber package is that the atom name information is lost after running Gaussian calculations. And a residue topology file in prepi or prepc or a mol2 file generated from the Gaussian output has atom names not matching those from the original file (usually a pdb file). Because of this glitch, one can not simply load the residue topology file to tleap, read in the pdb file, and then save to the topology. We developed match_atomname to address this problem. The match_atomname program takes an input file and a reference file in pdb, ac, prepi, prepc and mol2 format; it automatically detects the corresponding atom name in the reference for each atom name in the input. An output file in the same format as that of the input is generated using the matched atom names.

```

Usage: match_atomname -i input file name
                    -fi input format (pdb, ac, prepi, prepc, mol2)
                    -r ref file name
                    -fr ref format (pdb, ac, prepi, prepc, mol2)
                    -o output file name
                    -h include hydrogen atoms or not
                        0 not, the default
                        1 yes
                    -g geometric info (such as E/Z configuration) is considered to
                        0 no, the default
                        1 yes
                    -l maximum path length, default is -1 (full length)
                        if it takes very long time and/or core dump occur, a value b

```

Example:

```
match_atomname -i SAH.prepi -fi prepi -o SAH_matched.prepi -r SAH_XRAY.pdb -fr pdb
```

The output, SAH_matched.prepi and SAH_XRAY.pdb can be loaded to tleap directly to generate a topology for minimization or MD simulations.

17. paramfit

Robin Betz

The *paramfit* program allows specific forcefield parameters to be optimized or created by fitting to quantum energy data. *Paramfit* can be used when parameters are missing in the default force fields and *antechamber* cannot find a replacement, or when existing parameters do not describe the system to the desired level of accuracy, such as for dihedral constants on protein backbones.

Paramfit attempts to make the following statement true: **With the correct AMBER parameters, calculations performed at a quantum level over many conformations of a structures should match those calculated by AMBER.**

Paramfit can calculate the energy of each conformation and/or the force on each atom, and adjust the force field parameters so that these values correspond to input quantum data.

For energies, *Paramfit* attempts to fit the AMBER energy to the quantum energy for a variety of conformations of the input structure, minimizing the equation

$$\sum_{n=1}^N w_i \left[(E_{MM}(n) - E_{QM}(n))^2 + K \right] = 0$$

where K is a constant that adjusts for different origins in the QM and MM calculations so that minimization may be done to zero and N is the number of molecular conformations that are considered.

For forces, the equation that is optimized is

$$\sum_{n=1}^N \sum_{atom=1}^{N_{atoms}} w_i |F(n, atom)_{MM} - F(n, atom)_{QM}|^2 = 0$$

where the sum of the differences in the forces on each atom should match given the correct set of parameters. Individual structures can be assigned weights w_i to give them more or less relative importance in the fit. By default, all weights are set to 1.

The program works by altering the parameters that AMBER uses to describe the molecule, which alter the elements in the AMBER sum that is used to calculate the energy or forces. It is necessary to evaluate over many conformations of the molecule because the parameters should predict how the molecule will behave dynamically rather than statically. To get a good idea of the forces on a dihedral, for example, the energy needs to be evaluated for multiple conformations of the dihedral to see how it changes each time. *Paramfit* will fit so that the energy changes that AMBER predicts will happen when the dihedral twists match the changes predicted with quantum methods.

In order to facilitate force field development, *Paramfit* supports fitting parameters across multiple molecules (for example, fitting a single dihedral backbone term across a variety of input amino acids). Single molecule fits can also be done to generate parameters that are missing or inadequate to describe small molecules or ligands.

Paramfit provides functionality for the majority of steps in the fitting process, including writing input files for quantum packages, specifying which parameters are to be fit, determining the value of K for the system, and finally conducting the fit and saving it in a force field modification file that can be used by other programs. An external quantum program is needed to generate the energies needed for *paramfit* to conduct a fitting. Currently, the program is capable of writing input files for ADF, GAMESS, and Gaussian, although if you write your own input files instead of using *paramfit's* functionality, any quantum package will work.

Paramfit has OpenMP support for parallelization of the AMBER function evaluation over the input conformations, where each core will evaluate the energy for a subset of the conformations. Enable this by adding the *-openmp* option to configure and rebuilding *paramfit*. By default all available cores will be used. To change this,

set the `OMP_NUM_THREADS` environment variable to the number of threads to be executed. You will see a speedup directly proportional to the number of cores you are running.

Paramfit now includes several ways fitting functions to aid in parameter generation. It can fit such that the energy of each input structure matches the single-point quantum energies inputted, or can now do the same fitting only with the forces on each atom, which may produce a more accurate fit that is less sensitive to problems with the input structure, and can also fit all dihedral force constants and phases simultaneously to a small set of quantum energies using a method developed by Chad Hopkins and Adrian Roitberg. This method fits every term and requires fewer function evaluations than running the full minimization algorithm, but requires especially good sampling of each torsion angle of interest.

Fitting forces requires several additional options to specify the location of the output forces files in the job control file. The easiest way to create a job control file for any of these options is to use the wizard, which runs automatically when no job control file is specified. This will walk you through the creation of a job control file and write it for you while prompting for all necessary options for the selected fitting function.

It is highly recommended that you fit to single-point quantum energies, as fitting to forces is considerably more expensive in terms of required calculation and still somewhat experimental. The implementation of the dihedral fitting method requires a varied set of input structures, and does not allow specifying individual dihedrals to be fit. No matter which method is used, please take care to carefully validate all parameters for reasonableness—*paramfit*'s fit is dependent on the variation and quality of the input structures and the resulting parameters are not guaranteed in ill-defined areas of the input conformation set. For example, if you fit a dihedral torsion term with input structures sampling the 0-30 degree range of that dihedral, the resulting parameters cannot be expected to give a valid energy of a structure with the dihedral at 90 degrees, as the algorithm merely fits to the available data and cannot make other predictions.

17.1. Usage

Paramfit is called from the command line as follows for a single molecule fit:

```
paramfit -i Job_Control.in -p prmtop -c mdcrd -q QM_data.dat \  
-v MEDIUM --random-seed seed
```

Running *paramfit* without any options will run a wizard that assists in the creation of a job control file. It is highly recommended that you use the wizard to assist you in setting run options.

The following switches apply to single molecule fits only:

- p** prmtop The molecular topology file for the structure.
- c** mdcrd A coordinate file containing many conformations of the input structure. These may be generated by running a short simulation in solution, or by manually specifying coordinates for each atom. It is important that there be a good representation of the solution space for any parameters that are to be optimized— for example, if you want a bond force constant it would be a good idea to have input structures with a good range of values for the length of the that bond type. See Subsection [17.2.6](#)
- q** QM_data.dat A file containing the quantum energies of the structures in the coordinate file, in order, one per line. You will have to extract the energies from the output files that the quantum package produces. An example script to do this for Gaussian formatted output files can be found in `$AMBERHOME/AmberTools/src/paramfit/scripts`.

To fit multiple molecules, the following switches are used:

```
paramfit -i Job_Control.in -pf prmtop_list -cf mdcrd_list -v MEDIUM --random-seed seed
```

Here is a very brief description of the command-line arguments for a multiple molecule fit. For more information on conducting these, fits, please see [17.3](#).

17. *paramfit*

- pf** *prmtop_list* A file containing a plain-text list of input topology files and the adjustment constant K for each file separated by a space, one per line.
- cf** *mdcrd_list* A file containing a plain text list of input coordinate files, number of structures to read from each file, and directory containing quantum output from each file, separated by a space. These should be specified in the same order as the topologies in the *prmtop_list*.

The following switches apply to either type of fit:

- i** *Job_Control.in* The job control file for the program. See Section 17.2 for a description of the options and format for this file. If no job control file is specified, a wizard will be initiated that will prompt you for options and help create the file. Use of the wizard is highly recommended when running *Paramfit* for the first time.
- v** *MEDIUM* The verbosity level to run the program at, either LOW, MEDIUM, or HIGH.
- random-seed** *seed* The integer seed for the random number generator. Only specify this parameter when exactly reproducible results are needed for debugging.

17.2. The Job Control File

Similarly to *sander* and other programs, *paramfit* requires a job control file that specifies individual options for each run. The options that apply to your run vary depending on the runtype and the other settings, and they are quite numerous. To aid you in creating a job control file, a wizard has been included that will prompt you about applicable settings and create the job control file for you. Using the wizard is highly recommended, especially when running a fit for the first time. To use the wizard, simply run *paramfit* without any options. **It is highly recommended that you use the wizard to create job control files**, as it prompts for all options relevant to your run and the resulting file can then be easily edited by hand.

The format consists of variable assignments, in the format *variable=value*, with one assignment per line. Pound signs (#) will comment out lines. See the following sections for a description of what to put in the job control file for various tasks:

17.2.1. General options

paramfit requires several options be set for every run. These variables should usually appear in your job control file.

RUNTYPE Specifies whether this run will be creating quantum input files, setting parameters, or conducting a fit.

- = CREATE_INPUT** The structures in the coordinate file will be written out as individual input files for a quantum package. See 17.2.2.
- = SET_PARAMS** Provides an interactive prompt allowing you to specify which parameters will be fit for this molecule. See 17.2.3.
- = FIT** Conducts a fitting using one of the two minimization algorithms. See 17.2.4 for other options that need to be specified.

NSTRUCTURES Specifies how many structures are in the input coordinate file. If this value is less than the total number of structures in the file, only the first *n* will be read. Only applies to single molecule fits! If you are fitting multiple molecules at once, the number of structures for each molecule should be specified in the *mdcrd_list* file as described in 17.3.

17.2.2. Creating quantum input files

Given a trajectory, *Paramfit* can write input files for a variety of quantum packages. This is necessary to generate the energy values for each input conformation that *Paramfit* will fit to. You do not necessarily need to do this step and can write your own input files if desired. Currently Gaussian, ADF, and GAMESS formats are supported.

Job files will be named sequentially with filename prefix and suffix specified in the job control file. Once all the input files are written, you must run the quantum package yourself. *Paramfit* can read Gaussian output files directly, but for other packages you must extract the energies yourself into a file with one energy per line in the same order as the input structures.

Currently *Paramfit* only supports Gaussian if you are fitting forces, and will read the output files and extract the force information for you. See [17.4](#) for more information on fitting these.

To enter this mode, set RUNTYPE=CREATE_INPUT and specify the following options in your job control file:

QMHEADER File that will be prepended to all created input files for the quantum program. This specifies things on a per-system basis, such as choice of basis set, amount of memory to use, etc. These parameters will vary depending on which quantum package you are using. Sample header files for all supported quantum packages are included in `example_config_files` in *paramfit*'s source directory.

QMFILEFORMAT Specifies which quantum package the created input files should be formatted for.

= **ADF** Use the Amsterdam Density Functional Theory package.

= **GAMESS** Use the General Atomic and Molecular Electronic Structure System (GAMESS).

= **GAUSSIAN** Use Gaussian.

QM_SYSTEM_CHARGE The integral charge of the system. Defaults to 0. Note that some quantum packages may require this to also be specified in your header file.

QM_SYSTEM_MULTIPLICITY The integral multiplicity of the system. Defaults to 1 (singlet).

QMFILEOUTSTART The prefix for each of the created input files. Defaults to 'Job.' The structure number and then the suffix will be appended to this value.

QMFILEOUTEND The suffix for each of the created input files. Defaults to '.in'. With both default options, the file will be named Job.n.in.

17.2.3. Specifying parameters

In order to facilitate batch runs as well as simplify the process of running *paramfit* on larger systems, the parameters to be fit are saved and then loaded in during actual fitting so that they do not have to be specified every time. The parameter setting runtime accomplishes this by prompting whether you would like to fit bond, angle and/or dihedral parameters and then displaying a list of the specific atom types for each so that you can pick exactly what *paramfit* should optimize. This saved file does not specify a value for any of the parameters, but simply indicates which ones are to be changed during fitting.

If you do not wish to save a parameter file, you may instead fit a default set of parameters or be prompted every time. See Subsection [17.2.4](#).

To enter this mode, set RUNTYPE=SET_PARAMS and the following options:

PARAMETER_FILE_NAME Specifies the name of a file in which to store the parameters. When loading these parameters in during a fitting, this line will stay the same. Do not modify this file by hand: *paramfit* numbers each bond, angle, and dihedral in a manner that is consistent but not human-readable.

17.2.4. Fitting options

The fitting function accomplishes the actual parameter modification. It does this by minimizing the least squares difference between the quantum energy and the energy calculated with the AMBER equation over all of the input conformations. For a perfect fit, this means that over all structures, $E_{MD} - E_{QM} + K = 0$.

17. *paramfit*

K is the intrinsic difference between the quantum and the classical energies, which is represented as a parameter that is also fit. The value of K depends on the system, and should be fit once as the only parameter before fitting any other parameters.

To enter this mode, set `RUNTYPE=FIT` and set the following additional variables:

ALGORITHM The minimization algorithm to use. *paramfit* currently implements a genetic algorithm and a simplex algorithm for conduction minimization. Each algorithm requires several parameters and is suited to different problems. Please see [17.2.5](#) for descriptions of these options and a guide on choosing the appropriate algorithm.

= **GENETIC**

= **SIMPLEX**

= **BOTH** Runs the hybrid genetic algorithm followed by the simplex algorithm to fine tune results

= **NONE** No fit is performed- useful for calculating energy of each structure with the initial parameters to see their quality

FUNC_TO_FIT The fitting function to use in the calculation.

= **SUM_SQUARES_AMBER_STANDARD** Standard fit to single-point energies. Recommended selection.

= **AMBER_FORCES** Fit to the forces on atoms involved in fitted parameters. Currently only supports Gaussian output. See Section [17.4](#) for details.

= **DIHEDRAL_LEAST_SQUARES** Use Chad Hopkins and Adrian Roitberg's method to fit all dihedral terms at once. This method will fit all dihedral torsion terms simultaneously with a minimal number of function evaluations, but requires very good sampling of the relevant torsion angles.

K The intrinsic difference between the quantum and classical energies. This value needs to be determined once for each system so that the algorithm can minimize to zero instead of to a constant. See Subsection [17.5.2](#) for an example.

PARAMETERS_TO_FIT Sets how *paramfit* determines which parameters are to be fit. *paramfit* does not fit electrostatics, but is capable of fitting every other element of the AMBER sum, which include bond harmonic force constant and equilibrium length, angle harmonic force constant and equilibrium angle, and proper and improper dihedral barrier height, phase shift, and periodicity. As a general rule, the fewer parameters there are to fit, the faster and more accurate the results will be. Avoid fitting more parameters than necessary.

= **DEFAULT** Fit all bond force constants and lengths, angle force constants and sizes, and dihedral force constants. This option will usually fit a very large number of parameters, and is rarely necessary. For most cases, only a few parameters are desired, and they should be fit individually.

= **K_ONLY** Do not fit any force field parameters. Only fit the value of K (the difference between quantum and classical energies for the system). This needs to be done once per system in order to determine K before any other parameters are fit, as attempting to fit it at the same time results in inaccurate results. Since small changes in K produce a great change in the overall least squares sum, the algorithm will tend to focus on changing the value of K and will neglect the parameters.

= **LOAD** The list of parameters to be fit is contained in a file that was previously created with the parameter setting runtime. Set `PARAMETER_FILE_NAME` to the location of this file. To create this file, run *paramfit* with `RUNTYPE=SET_PARAMS`.

SCEE The value by which to scale 1-4 electrostatics for the AMBER sum. Defaults to 1.2

SCNB The value by which to scale 1-4 van der Waals for the AMBER sum. Defaults to 2.0.

QM_ENERGY_UNITS The unit of energy in the quantum data file if you are fitting to energies. This will depend on your quantum package and settings used for the single point calculations.

= HARTREE Default

= KCALMOL

= KJMOL

QM_FORCE_UNITS The unit of force in the quantum data files if you are fitting to forces. This will depend on your quantum package and settings used for the force calculations.

= HARTREE_BOHR Default

= KCALMOL_ANGSTROM

WRITE_ENERGY Saves the final AMBER energy and the quantum data for each structure to the specified file. Plotting these data is useful in verifying the results of the fitting and identifying any problem structures. See Subsection 17.5.3 for more on how to verify the accuracy of results.

WRITE_FRCMOD When the fitting is complete, the parameters will be saved in a force field modification file at this location in addition to displaying them in standard output. This file may be used with LEaP to create a new *prmtop*. If no value is specified the file will not be created.

SCATTERPLOTS Creates graphs of the bond, angles, and dihedrals found in the input files for each parameter that is being fit. These plots can be visualized using *scripts/scatterplots.sh* found in *paramfit*'s source directory. This can be helpful in assessing the quality of the input conformations. No need to specify anything after the = sign for this parameter.

SORT_MDCRDS = YES Sorts the input structures in order of increasing energy before conducting the fit. This can aid in identification of problem regions for the initial or fitted parameters, as they may be generally worse on structures in certain energy ranges.

= NO Default

COORDINATE_FORMAT The format of the input coordinate set. *Paramfit* will return an error if the file is in an unexpected format.

= TRAJECTORY Default

= RESTART

17.2.5. Algorithm options

Paramfit implements two minimization algorithms: a simplex and a hybrid simplex-genetic algorithm (GA). The current version of *paramfit* incorporates numerous refinements to the genetic algorithm that require much less input from the user— it is no longer necessary to choose between the simplex or GA. This improved algorithm means that iterative fits are no longer necessary, and the algorithm will converge very close to or at the global minimum on a single run.

The genetic algorithm starts with a randomly generated solution set, which it recombines and alters in ways similar to evolution. The GA will start with many initial randomly generated sets of parameters. It will then determine which are the best by evaluating the AMBER sum, select them for recombination to produce a new set of parameters, randomly alter a few parameters slightly to prevent premature convergence, and iterate. Once several “generations” have passed without improvement, a loosely converging simplex algorithm is run on a random subset of the population, which is then allowed to recover for several generations before further simplex iterations are conducted. This hybrid approach dramatically speeds convergence to the global minimum, while maintaining the strengths of the genetic algorithm in searching a large, complex solution space with low sampling.

The following options in the job control file will control the behavior of the genetic algorithm. In general the default values for these options is sufficient to produce good results, and alterations to them will speed convergence. Options marked *internal algorithm parameter* should not need to be altered by the vast majority of users, as they are already set to their optimum. The algorithm's results should be independent of these values if they are within reasonable ranges (run the wizard for suggestions).

OPTIMIZATIONS The integer number of possible optimizations the algorithm will use. Analogous to the population size in evolution; larger values require more function evaluations and are slower but produce better initial sampling, and smaller ones will delay convergence. Defaults to 50.

SEARCH_SPACE If positive, the algorithm will search for new parameters for everything except dihedral phases within this percentage of the original value, where 1.0 will search within $\pm 100\%$ of the value found in the input *prmtop*. Defaults to searching over the entire range of valid values and ignoring the original value in the topology file. You may wish to alter this value if you know that the original parameters are good and you wish to search in their neighborhood.

MAX_GENERATIONS The maximum number of iterations the algorithm is allowed to run before it returns the best non-converged optimization. Defaults to 50,000. If you find that you repeatedly need to increase this value compared to the default, there are likely significant problems with your system or insufficient input structures.

GENERATIONS_TO_SIMPLEX The number of iterations in a row that must pass without improvement in the best parameter set for simplex refinements to be run on a random 5% of the populations. Set to 0 for a pure genetic algorithm. Smaller values will speed convergence but may result in retrieval of local minima. Defaults to 10.

GENERATIONS_WITHOUT_SIMPLEX The number of generations that must pass between runs of simplex refinement, regardless of improvement in the best parameter set. These iterations serve as a recovery period for the population of the genetic algorithm, and allows time for the simplex results to be incorporated. If set to small or zero values, simplex refinement may run too often, resulting in convergence to a local minima and eliminating the global search properties of the genetic algorithm. Defaults to 10.

GENERATIONS_TO_CONV The number of iterations in a row that must pass without improvement in the best parameter set for the algorithm to be considered converged. Set to a larger value for a longer but potentially more accurate run. Defaults to 50, which is too large for most systems. This counter increments along with the counter to trigger simplex refinement, and at the global minimum simplex refinement will produce no improvement on the population, allowing convergence.

MUTATION_RATE *Internal algorithm parameter* The chance an allele (potential parameter) in the genetic algorithm population has to be randomly set to a new value each generation. Defaults to 0.05.

PARENT_PERCENT *Internal algorithm parameter* The percentage of each generation that is allowed to pass on alleles to the next generation. Defaults to 0.25.

The simplex algorithm is excellent at refining a good set of input parameters, but can converge on physically unreasonable values (such as negative bond force constants) if given a naive guess. For this reason, the genetic algorithm is recommended for finding the global minimum or a close approximation thereof, and the simplex algorithm may be run on the resulting parameters to confirm the results, if desired. The simplex algorithm starts at an initial set of parameters and moves “downhill” iteratively while sampling neighboring areas (much like an amoeba crawling along the function landscape), and converges when the improvement from one step to another becomes negligible. The simplex algorithm is generally faster than the GA, and excels at well-defined systems with a small number of dimensions. This algorithm requires a very well-defined sample space, and the input structures should contain a good range over all the bonds, angles, and dihedrals that are to be optimized. Otherwise, the algorithm tends to wander and will converge in badly defined areas of the sample set. In smaller, well-defined systems with only a few parameters, this algorithm will outperform the genetic algorithm.

Choose the simplex algorithm if you wish to fit only a few parameters and have a large number of input conformations. You may specify the following options to fine-tune the step sizes taken, but for the vast majority of cases the defaults should suffice:

BONDFC_dx Intrinsic length of parameter space for minimization. Used to determine the size of the steps to construct the initial simplex. Should be large enough that the steps sample a sufficiently large area but small enough to not move outside of normal parameter range. Bond force constant step size defaults to 5.0.

BONDEQ_dx Bond equilibrium length step size. Defaults to 0.02.

ANGLEFC_dx Angle force constant step size. Defaults to 1.0.

ANGLEEQ_dx Angle equilibrium step size. Defaults to 0.05.

DIHEDRALBH_dx Dihedral force constant step size. Defaults to 0.2.

DIHEDRALN_dx Dihedral periodicity step size. Defaults to 0.01.

DIHEDRALG_dx Dihedral phase step size. Defaults to 0.05.

K_dx Step size for intrinsic difference constant. Defaults to 10.0.

CONV_LIMIT Floating point number that details the convergence limit for the minimization. The smaller the number, the longer the algorithm will take to converge but the results may be more accurate. Defaults to 1.0E-15, which is very strict.

17.2.6. Bounds Checking

In order to ensure that the algorithms can return meaningful results, bounds checking routines are included in *paramfit*. The bounds checking functionality ensures that the algorithm's results are reasonable given the initial sample set, and also makes sure that the sample set is well-defined.

Since bonds and angles are approximately harmonic, the algorithm's result is reasonable if it lies within a well-defined area of the sample set. Bonds and angle values are therefore checked after the algorithm has finished running. In order to properly fit dihedrals, sample structures should span the entire range of phases for each dihedral that is to be fit. Dihedral checking is therefore accomplished before the algorithm begins to conduct the fit.

Bounds checking defaults to halting execution of the program upon reaching a failing condition. It is not recommended that this behavior be disabled, since the results of the fit are most likely inaccurate. Using the fitted parameters anyway will probably result in an inaccurate depiction of the molecule. Properly represented parameters in the input structures are crucial for a valid fit. Instead of using the parameters, fix the input structures so that data are provided in the missing ranges, which will be stated in the error message, and rerun the program twice: first in *CREATE_INPUT* mode to obtain quantum energies for the added structures and then in *FIT* mode to redo the fit.

If you **know** that your input structures describe the parameters to be fit quite well, the selectivity of the bounds checking can be altered by the specifying the following options in the job control file. Use these options with caution, and verify the generated parameters carefully.

CHECK_BOUNDS = ON The recommended and default option. This will halt execution when the bounds check fails.

= WARN Continue upon reaching a bounds failure condition, but output a warning. Do not use the parameters generated by this fit without careful verification! Use the error message and other results to determine if they are reasonable.

BOND_LIMIT Fitting results for bond lengths that are this many Angstroms away from the closest approximation in the input structures will result in a failing condition. Defaults to 0.1.

ANGLE_LIMIT Fitting results for angles that are more than this many radians away from the closes approximation in the input structures will result in a failing condition. Defaults to 0.05π .

DIHEDRAL_SPAN The entire range of valid dihedral angles, 0 to π , for each dihedral that is to be fit should be spanned by this many input structure values, otherwise a failing condition will result. Defaults to 12, meaning that there needs to be a dihedral in every $\frac{\pi}{12}$ radian interval of the valid range.

17.3. Multiple molecule fits

Paramfit supports fitting one or more parameters across multiple molecules, and contains several features to aid in force field development. The program is invoked differently, using a *prmtop* list and *mdcrd* list that specify topology and structures for each molecule to fit. Since the value of *K* is also system-dependent, you will need to fit *K* for each molecule individually.

Input topologies are specified in a *prmtop* list, which contains the filename of each topology and the value of *K* for that system, separated by a space. There are no comments permitted in this file. For example:

```
molecule1.prmtop 50.0
molecule2.prmtop 100.0
```

To obtain the value of *K* for each topology file, conduct a single-molecule fit using all the structures corresponding to that topology and put the resulting value in this file. This enables fitting to zero over multiple molecules.

Input coordinate files are stated in the *coordinate* list, which contains the filename of each coordinate set, the number of structures contained in it, and the filename containing the energy of each structure, separated by a space. Each energy file is exactly the same as single-molecule fits, containing the energy of each structure, one per line, in the same order as the corresponding coordinate file. If there are more structures available in the coordinate file than the number *N* specified, the first *N* structures will be used in the fit. An example coordinate list would be:

```
molecule1.mdcrd 200 energy1.dat
molecule2.mdcrd 100 energy2.dat
```

Parameters to fit must be present in all of the available topologies, and the parameter specification file (*PARAMETER_FILE_NAME*) should be created using a single-molecule invocation of *paramfit*. Saved output files such as energy profile will be named according to the input file name, and a single *frmod* will be written if specified. A multiple molecule invocation of *paramfit* uses the following command line options:

```
paramfit -i Job_Control.in -pf prmtop_list -cf mdcrd_list [-v MEDIUM] [--random-seed seed]
```

The only alteration to the job control file necessary for multiple molecule fits is the deletion of the *NSTRUCTURES* parameter. *NSTRUCTURES* should not be specified as it is now ambiguous and will result in a program error.

17.4. Fitting Forces

Paramfit can fit to the forces on each atom within an input structure rather than to single point energies. In theory, this provides more data to the fitting algorithm and reduces noise by considering only the forces on atoms involved in a fitted parameter in the function evaluation. This section will walk you through the process of fitting forces using *paramfit*.

Currently, force fitting can only read in Gaussian output files, so input files will be created in the format accepted by that program. Specify in the *QMHEADER* file the “force” keyword, so Gaussian will print out the forces on each atom, and run *paramfit* in the *CREATE_INPUT* mode as normal. Then run Gaussian on those input files, keeping the resulting output with the same naming scheme, for example appending “.out” to the name of an input file to indicate its input. For example, in bash:

```
for i in `ls output/Job.*.gjf`; do g09 < $i > $i.out; done
```

To run a fit with forces, you must specify the following options in the job control file, or use the wizard. *Paramfit* will read in the output files from the Gaussian job using the same order and naming scheme, so alter the *QM* filename parameters so that they match the suffix you appended to Gaussian output files.

```
# Enable force fitting function
FUNC_TO_FIT=AMBER_FORCES
# K irrelevant for force fitting
```

```

K=0.0
# Force units used by Gaussian
QM_FORCE_UNITS=HARTREE_BOHR
# Naming scheme of gaussian output files
QM_FILEOUTSTART=output/Job.
QM_FILEOUTEND=.gjf.out

```

Specify parameters to fit, algorithm and output options as described previously for fits to energy. As forces fitting is still experimental, take care to evaluate the resulting parameters.

17.5. Examples

17.5.1. Setting up to fit

The fitting process with *paramfit* follows a specific order. Example job control files for each step and a description of the step follow.

First, write a job control file to create the input structures and run *paramfit*:

```

RUNTYPE=CREATE_INPUT
# Trajectory has 50 structures
NSTRUCTURES=50
# Write in Gaussian format
QM_FILEFORMAT=GAUSSIAN
# Prepend this file to QM inputs
QM_HEADER=Gaussian.header
$AMBERHOME/bin/paramfit -i Job.Control.in -p prmtop -c mdcrd

```

After all 50 input files have been created, run the quantum program on them. Once it's finished, extract the quantum energies from the output files using the provided script, or write your own. Since the example used Gaussian:

```

$AMBERHOME/AmberTools/src/paramfit/scripts/process_gaussian.x \
output_directory energies.dat

```

Now, or while the quantum jobs are running, since neither the energies nor the structures are needed yet, determine which parameters are to be fit and save them.

```

RUNTYPE=SET_PARAMS
# File to be created
PARAMETER_FILE_NAME=saved_params
$AMBERHOME/bin/paramfit -i Job.Control.in -p prmtop

```

Now the quantum energies to fit have been obtained and the parameters to fit have been set, and the fitting process may begin.

17.5.2. Fitting K

The first step in fitting is determining the value of K for a system. A job control file that will only fit K follows:

```

RUNTYPE=FIT
PARAMETERS_TO_FIT=K_ONLY
# Use the simplex function
FITTING_FUNCTION=SIMPLEX

```

Then,

17. *paramfit*

```
$AMBERHOME/bin/paramfit -i Job_Control.in -p prmtop -c mdcrd -q energies.dat
```

Take this value of K and put it back in the job control file when conducting the actual fit.

```
RUNTYPE=FIT
# Use the parameters specified earlier
PARAMETERS_TO_FIT=LOAD
PARAMETER_FILE_NAME=saved_params
# Genetic algorithm options
FITTING_FUNCTION=GENETIC
OPTIMIZATIONS=500
GENERATIONS_TO_CONV=10
GENERATIONS_TO_SIMPLEX=2
GENERATIONS_WITHOUT_SIMPLEX=5
# Save parameters so they can be read into leap
WRITE_FRCMOD=fitted_params.frcmod
```

And call *paramfit* just as before. This example fit will create a force field modification file that can later be read into *LEaP* to create a new *prmtop* with the modified parameters for the molecule.

17.5.3. Evaluating Results

When using *paramfit*, it is important to verify the accuracy of the fitted parameters for your input structures. The `WRITE_ENERGY` option in the Job Control file is useful for this. Set it to a filename and *paramfit* will write the final AMBER energy of each structure next to the quantum energy for the same structure in a file that can be easily graphed.

If you have *gnuplot*, a script has been provided to quickly show each structure's energies. Assuming your energy file is named *energy.dat*:

```
$AMBERHOME/AmberTools/src/paramfit/scripts/plot_energy.x energy.dat
```

The resulting graph makes the identification of problem structures much easier, and gives a good visualization of the fit. In general, carefully validate parameters generated by *paramfit* against other data before conducting large simulations.

The `SCATTERPLOT` option in the job control file can also be useful in assessing the quality of the input structures. If this option is set, *paramfit* will dump a variety of data files indicating the value for all fitted bonds, angles, and dihedrals in the input conformations. These data may be visualized if you have the program *gnuplot* by running the following command in the directory where *paramfit* was run:

```
$AMBERHOME/AmberTools/src/paramfit/scripts/scatterplots.sh
```

The resulting graphs feature different colored points for each bond, angle, and dihedral type that is being fit for each of the input structures. This is useful in evaluating if the results of the fit are reasonable— for example, if the algorithm converges with an equilibrium bond length that is not similar to any of the structures, that parameter may not be accurate.

18. Molecular Mechanics Parameter Fitting in *mdgx*

The *mdgx* program has been distributed with Amber since 2012. At first, it was intended as a platform for radical redesign of the molecular dynamics algorithm, implementing a proof-of-concept multigrid technique for the particle-mesh Ewald electrostatic sum as well as a rare strategy for pair list decomposition. All of these features remain in *mdgx*, and the code retains a modest parallel CPU capability for running basic simulations. This molecular dynamics facility is critical to “IPolQ” charge development method unique to the program.

However, it soon became obvious that the needs for a simulation engine as well as an algorithmic development platform would be served by *pmemd* and its GPU extensions. The role of *mdgx* then shifted to parameter development, for which the simple C coding, the facility for reading multiple topologies, and modular extensibility have proved well suited.

Repurposed as a parameter development tool, *mdgx* is a worthy addition to the AmberTools family of programs, and one of the most powerful package-distributed tools available for this purpose. There is one exceptional functionality in the program, new to Amber20: the ability to run large numbers of simulations on small systems with the Generalized Born solvent model. At present, only standard molecular dynamics for these systems is enabled, but the standard *igb* settings present in *sander* and *pmemd* are possible (surface area terms are not yet computed). This capability enables validation runs on vast numbers of small systems with unprecedented throughput, which makes it a sensible feature to include under the theme of parameter development.

18.1. Input and Output

Input command files for *mdgx* may be similar to the *mdin* format used by *sander* and *pmemd*. One requirement of *mdgx* that is not found in *sander* is that each of the namelist segments of the input file must begin with the identifier of the namelist on its own line and end with the keyword `&end` on its own separate line. However, the namelist format is not strictly enforced in *mdgx*, not all *sander* input variables are available in *mdgx*, and some new input variables have been added. All *mdgx* input variables can also be identified by aliases that may be lengthier than their *sander* counterparts but may make the input easier for a human to parse.

All *mdgx* namelists and their associated variables may be browsed by running the *mdgx* program itself; running the program with no command line arguments will produce basic instructions for usage and a list of command-line arguments to display each namelist. For example, on the command line: “`>>mdgx -PARAM`” will show a lengthy description of all features in the *mdgx* `¶m` namelist, the parameter sampling module.

Certain directives to *mdgx* may be supplied as either part of the input file or on the command line; in particular, the names of the topology, input coordinates, and output files may be specified in either manner. Also, the random number generator seed may be specified on the command line. However, if the same variable is declared both on the command line and in the input file, the command-line input will take precedence. This predominance makes it possible to execute multiple related *mdgx* runs based on a single input file. Units of input variables follow the *sander* and *pmemd* conventions.

The *mdgx* program will read standard AMBER *prmtop* files using its own routines and, in some run modes, will perform basic tests of the topology to identify common problems such as omitted disulfide bonds or “D” to “L” chirality flips in the standard amino acids; any potential problems are reported in the *mdout* output diagnostics files, but do not immediately lead the program to halt.

Output files produced by *mdgx* follow the AMBER *.crd* and NetCDF formats for coordinates and velocities. While there is an elaborate scheme available for output file organization in traditional MD, this is not used very often and anyone interested should contact the authors. For most purposes, the standard output file types and

descriptions apply. For parameter fitting, the equivalent *mdout* contains a great deal of diagnostic information on the breadth of the training data and the improvement accomplished in the fit to the quantum energy surface.

The *mdgx* program also provides its own output format for force diagnostics. In *sander*, information relating the bond, angle, torsion, and nonbonded direct and reciprocal space forces is only available by running in “debugging” mode as specified by the `&debugf` namelist block. In *mdgx*, such output is available by setting the *sander*-related *imin* variable to 2; the output is produced in ASCII format with numerous comments to make the results comprehensible to a human, and in some cases can be convenient for analyzing forces on boxes full of atoms.

18.2. Installation

mdgx is installed as part of the AmberTools package. The program relies on the FFTW 3.3 and NetCDF libraries already distributed as part of AmberTools.

18.3. Partial Charge Development

The first parameter sets that utilized the *mdgx* apparatus were charge sets for protein force fields, namely *ff14ipq* and its successor *ff15ipq*. Before we delve into the derivation for *IPolQ*, which is somewhat involved, it is important to note that *mdgx* can perform electrostatic potential fitting using any quantum method which generates a Gaussian cubegen format file or can be converted into such a format. Once *mdgx* has data on electrostatic potentials, *mdgx* automates the tedious task of synthesizing that data into a set of partial charges.

The Restrained Electrostatic Potential (RESP) methodology is the basis for charge assignment based on quantum-mechanical electrostatic potential data, but the details differ somewhat from the implementation in *antechamber*. The basic concept of fitting charges to reproduce the electrostatic potential of a molecule, by finding the solution with least squared error in the presence of restraints, is carried over from the original Kollmann RESP. However, instead of Lagrangian constraints, equivalent charges are unified as single variables in the fit, and penalty functions are added to the fitting matrix to enforce total charge constraints. Where *mdgx* excels is in the control it gives the user over what fitting data will be used. Rather than relying on a quantum-chemistry package to select a particular surface around a molecule, *mdgx* will read the electrostatic potential due to that molecule on a regular grid and select points from that grid based on a solvent-accessible region determined by the actual Lennard-Jones parameters of the model. Because most solvent models make use of hydrogen atoms with modest or nonexistent steric properties, *mdgx* also considers points which may not be accessible to the solvent probe but might be accessible to a hydrogen atom connected to that probe. *mdgx* will read a *prmtop* describing the system and also, if required, a Virtual Sites rule file, so that partial charges may be fitted for any virtual sites that the user wishes to add. Once fitting is complete, *mdgx* can return a new Virtual Sites rule file that will apply the fitted charges to the original *prmtop* in future simulations.

Fitting is called by its own separate `&fitq` namelist, and triggers a distinct run mode in the sense that the program will terminate after the fit is complete. The options available in the `&fitq` namelist include (shorthand aliases in parentheses):

- **RespPhi** (*resp*): File names and numerical weight of an electrostatic potential to use in fitting. The format is `<string1> <string2> <real1>`, where *string1* is a Gaussian cubegen format file specifying the electrostatic potential and molecular coordinates and Z-numbers appropriate to the topology specified by *string2* and *real1* is the numerical weight to be given to this conformation in the fit.
- **IPolQPhi** (*ipolq*): File names and numerical weight of a pair of electrostatic potentials to use in *IPolQ* fitting. The format is `<string1> <string2> <string3> <real1>`, where *string1* and *string2* are Gaussian cubegen format files relating to the system in vacuum and in a condensed-phase environment (see also the section on Implicitly Polarized Charge creation). Note that the molecular coordinates in both cubegen files must match. As in the *resp* variable format, *string3* is the topology, and *real1* is the numerical weight of this conformation.

- **EPRules** (eprules): If specified, *mdgx* will output all fitted charges in the form of a Virtual Sites rule file, which can be given as input to subsequent simulations to modify the original **prmtop** and apply the fitted charge model.
- **ConfFile** (conf): If specified, *mdgx* will output the first molecular conformation, complete with any added virtual sites, in PDB format for inspection. This is useful for understanding exactly what model is being fitted.
- **TotalQ** (qtot): The total charge constraint in units of the proton charge; the sum of all fitted charges is required to equal this value. Default 0.0.
- **MinimizeQ** (minq): Restrain the charges of a group of atoms to zero by the weight given in minqwt. The groups are specified in ambmask format.
- **EqualizeQ** (equalq): Restrain the charges of a group of atoms to have the same values. Groups are specified in ambmask format.
- **MinQWeight** (minqwt): Weight used for restraining values of charges to zero; as more and more fitting data is included (either through a higher sampling density of the electrostatic potential due to each molecular conformation or additional molecular conformations) higher values of minqwt may be needed to keep the fitted charges small. However, with more data the need to restrain charges may diminish as well.
- **FitPoints** (nfpt): The number of fitting points to select from each electrostatic potential grid. The points nearest the molecule, which satisfy the limits set by the solvent probe and point-to-point distances as defined below, will be selected for the fit. Default 1000.
- **ProbeSig** (psig): The Lennard-Jones σ parameter of the solvent probe. Default 3.16435 (TIP4P oxygen).
- **ProbeEps** (peps): The Lennard-Jones ϵ parameter of the solvent probe. Default 0.16275 (TIP4P oxygen).
- **ProbeArm** (parm): The probe arm; points on the electrostatic potential grid that would be inaccessible to the solvent probe may still be included in the fit if they are within the probe arm's reach. Default 0.9572Å (TIP oxygen-hydrogen bond distance).
- **StericLimit** (pnrg): The maximum Lennard-Jones energy of the solvent probe at which a point will qualify for inclusion in the fit. Default 3.0 kcal/mol.
- **Proximity** (flim): The minimum proximity of any two points to be included in the fit. Default 0.4Å.
- **HistogramBin** (hbin): If hist is specified, *mdgx* will print a histogram reporting the number of fitting points falling within any particular distance of some atom of the molecule. This parameter controls the discretization of the histogram.
- **MaxMemory** (maxmem): Because fitting matrices can become very large in some cases (in particular, those involving multiple systems with correlated partial charges), *mdgx* offers this parameter as a safeguard against creating a matrix that may inadvertently take up too much memory. Values for this argument may be integers, or integers followed immediately (no spaces) with terms such as "GB," "Mb," or "kB" (case-insensitive) for giga/mega/kilo bytes. Default 1GB.
- **Verbose** (verbose): Unless set to zero by the user, *mdgx* will print periodic updates and record milestones from the fitting run in terminal output.

An example of a &fitq namelist is given below. In this particular problem, ECI2 and ECI3 were the names of virtual sites not in the original topology file but specified by a Virtual Sites rule file.

```
&fitq
  RespPhi    Conf12/pcm12.cube,
  RespPhi    Conf13/pcm13.cube,
  RespPhi    Conf14/pcm14.cube,
```

```

pnrq      2.0,
nfpt      15000,
minqwt    175.0,
EqualizeQ '@H1,H2'
EqualizeQ '@C12,C13'
EqualizeQ '@EC12,EC13'
MinimizeQ = '@E*'
EPRules    frag.xpt
ConfFile    f6xp.pdb
&end

```

Virtual site constructions have strong support in *mdgx* to rapidly translate between an imagined model and a practical simulation.

18.4. Implicitly Polarized Charge Development

The *mdgx* package is the workhorse used to create ff14ipq and its successor ff15ipq. At the heart of each of these protein force fields is the Implicitly Polarized Charge model (IPolQ), which *mdgx* automates.

The purpose of IPolQ is to derive an appropriate set of fixed partial charges on a molecule which account for the mean-field polarization it displays in solvent (water) while also accounting for the energetic cost of perturbing the gas phase wave function. This is handled by two separate quantum calculations on the same set of molecular coordinates: the first is performed in vacuum, the second in the presence of a time-averaged solvent charge density. Taking a page from linear response theory, the average of the two calculations provides the target electrostatic potential that the fixed partial charges should project. Collecting that charge density, particularly if the solvent contains counterions and there are infinite electrostatics in play, takes a fair amount of code, but this is why *mdgx* has a special module. It currently interfaces with two quantum packages, ORCA and Gaussian, to drive the QM calculations, and additional code in this module will write inputs and post-process the outputs of each program into material suitable for the *mdgx* electrostatic potential fitting tools. The variables available in the *&ipolq* namelist are as follows (shorthand aliases in parentheses):

- **SoluteMol** (solute): The solute molecule, specified by an ambmask string. This is the molecule of interest for charge fitting, and will be immobilized during the simulation. This must be specified by the user.
- **FrameRate** (ntqs): The rate of charge density sampling; the number of steps between successive snapshots to determine the solvent reaction field potential (SRFP). Default 1000 (the time step set in the *&cntrl* or *&ipolq* namelists should factor into the ntqs setting).
- **FrameCount** (nqframe): The number of frames used to compose the SRFP. Default 10 (this is too low for most solvent environments).
- **EqStepCount** (nsteqlim): The number of steps used to equilibrate the system before charge density collection begins. Use this part of the simulation to buffer against any artifacts that might arise from suddenly freezing the solute in place. Default 10000.
- **Blocks** (nblock): The number of blocks into which the simulation shall be divided for the purpose of estimating the convergence of the electrostatic potential. Default 4.
- **Verbose** (verbose): Default 0; set to 1 to activate step-by-step progress updates printed to the terminal window. Useful for monitoring short runs to ensure that the input successfully completes the SRFP calculation.
- **EConverge** (econv): Convergence tolerance for the SRFP (convergence checking is not yet implemented).
- **QShellCount** (nqshell): The number of additional shells of charge to place around the system in order to approximate the SRFP due to infinite electrostatics in the confines of an isolated system. Maximum (and default) is 3, minimum is 1.

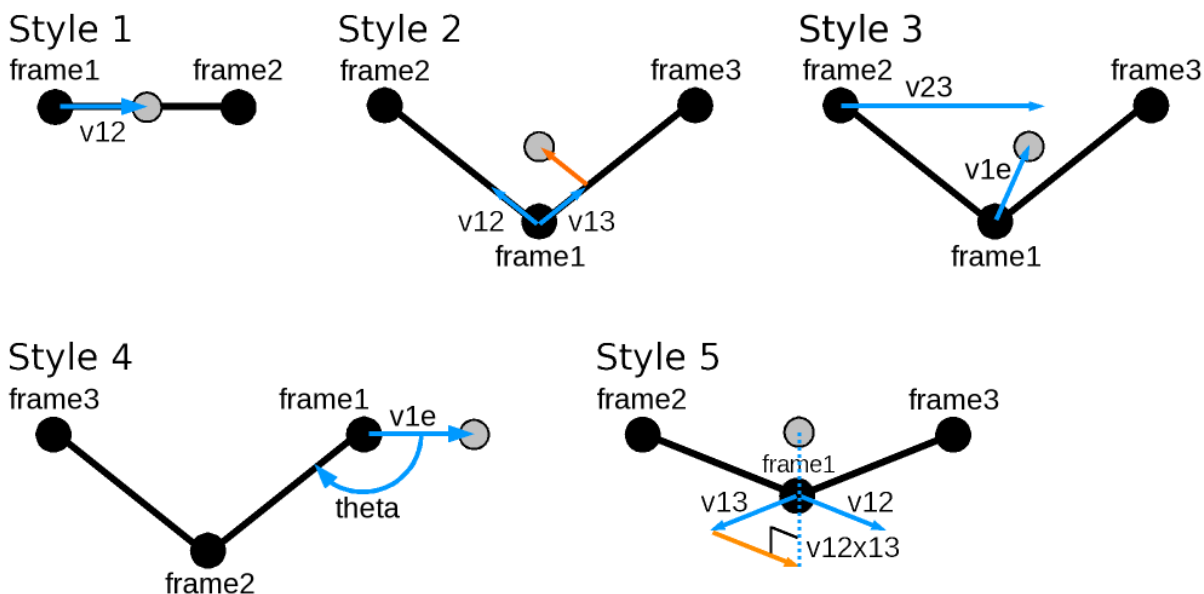
- **VShellCount** (nvshell): The number of shells around each atom on which the exact SRFP due to infinite electrostatics shall be calculated. Maximum (and default) is 3, minimum is 1.
- **QSpherePts** (nqphpt): In order to generate the surface charges that will help in approximating the SRFP, this number of points is placed equidistant on a sphere. The sphere is then rotated randomly and expanded to the radii indicated by qshell[1,2,3,x]. All points that are on the sphere due to one atom but within the sphere projected by another atom are deleted, until only points on the proper surface remain. Default 100.
- **VSpherePts** (nvphpt): Similar to nqphpt above, but for the shells of SRFP evaluation points. Default 20.
- **ExpQBoundary** (qshell1): The distance at which to locate the first surface charges, and to stop collecting charges explicitly from the simulation's non-solute (that is, solvent) atoms. Default 5.0.
- **QShell[2,3]** (qshell2, qshell3): The distance at which to locate the second and third shells of boundary charges. If engaged, each shell must be located successively further from the solute than the previous one.
- **VShell[1,2,3]** (vshell1, ...): The distances at which to locate additional shells of exact SRFP evaluation points. The SRFP is always evaluated, exactly, at the solute atom sites.
- **TimeStep** (dt): The simulation time step. This is read in the &ipolq namelist just as if it were present in the &cntrl namelist, but a value specified in &ipolq overrides the &cntrl setting. Default is 0.001ps, set in &cntrl.
- **MinQWeight** (minqwt): The stiffness of harmonic restraint by which to restrain fitted shell charges to zero. Default 0.01.
- **ModifyQ** (modq): When IPolQ is applied, it is appropriate to hyper-polarize certain molecules in the SRFP calculation. This variable may be specified as many times as necessary, followed by an ambmask string and a real number indicating the new charges to be assigned to all atoms in the mask. For example, fixed-charge water models should have their dipoles increased by an amount equal to the original model's dipole less 1.85 (the dipole of water in vacuum).
- **QuantumPrep** (prepqm): Preparatory call for QM calculations. This variable may be specified as many times as necessary. Each of these calls will be issued, in the order specified, before executing quantum calculations.
- **QuantumClean** (postqm): Post-processing calls for QM calculations. Similar to prepqm directives, called after QM calculations have been completed.
- **QMPackage** (qmprog): The quantum package to use. Supported packages are "gaussian" and "orca".
- **QMPath** (qmpath): Path to the primary QM executable. This path will be tested, taking into account prepqm calls, to be sure that the executable exists prior to running the SRFP calculation.
- **QMInputFile** (qmcomm): The base name of the QM input file. Vacuum and condensed-phase versions will be written with extensions 'vacu' and 'solv', respectively. Default 'IPolQinp'.
- **Maxmemory** (maxcore): The maximum memory that can be allocated to arrays for quantum calculations with Orca, or the maximum total memory that can be allocated for calculations with Gaussian.
- **QMOutputFile** (qmresult): The base name of the QM output file, which is given similar extensions to the input file. Default 'IPolQout'.
- **PointQFile** (ptqfi): The name of the point charges file referenced by orca for including the SRFP into the condensed-phase calculation.
- **QMSignal** (qmflag): The name of the file used to signal slave processes that the QM calculations launched by the master are complete. Default '.mdgx.finqm'.

- **QMTheory** (qmlev): The level of QM theory to use. Default MP2.
- **QMBasis** (basis): The QM basis set to use. Default cc-pvTZ.
- **WorkDirectory** (screddir): The scratch directory to use during QM calculations. Useful to reduce NFS load. If the directory exists, it will be used but not destroyed following each QM calculation. If the directory does not exist at the start of the run, it will be created and later destroyed.
- **KeepQMInput** (rqminp): Directive to retain QM input files after the run. Default 0 (OFF).
- **KeepQMCheckPt** (rqmchk): Directive to retain QM checkpoint file(s) after the run. Default 0 (OFF).
- **KeepQCloud** (rcloud): Directive to retain the solvent charge density cloud file after the run. Default 0 (OFF).
- **CheckExist** (checkex): Activates safety checks for the existence of QM executables (including electrostatic potential calculators) called at the start of the run. These checks attempt to take into account user-specified preparatory directives (see prepqm above). Default 1 (ON). Set to zero to disable this safeguard, for instance if the checks cannot find the executables but the preparatory directives, when fully implemented, are known to result in success.
- **UElec[X,Y,Z]Bin** (un[x,y,z]): The number of grid points on which to evaluate the electrostatic potential, in the X direction. Grid dimensions in Y and Z are set by similar variables.
- **UElec[X,Y,Z]Spc** (uh[x,y,z]): The grid spacing of the electrostatic potential grid in the X direction. The grid is always rectilinear. Spacings in Y and Z are set by similar variables.
- **CenterGrid** (cengrid): Directive to center the electrostatic potential grid on the location of the molecule stored in mdgx. The default behavior varies with each quantum package: 'orca' activates centering on the molecule whereas 'gaussian' calls for centering on the origin, as Orca does not reposition the molecule in its output but Gaussian will place the molecule in a 'Standard Orientation' and leave it there in the output and checkpoint files used for electrostatic potential calculations.
- **FormChkPath** (fmpath): Path to the program called for converting binary checkpoint files into formatted checkpoint files. Needed only if the QM program is 'gaussian'.
- **UEvalPath** (uvpath): Path to the program called for evaluating the electrostatic potential grid.
- **GridFile** (grid): Base name of the electrostatic potential grid to be written. As with QM input and output, this base name is appended 'vacu' or 'solv' for vacuum and condensed-phase calculations.

18.5. Customizable Virtual Site Support

It is not completely feasible to perform molecular dynamics with massless particles. However, for many useful cases in which the locations of massless particles are determined by the locations of two or more atoms with mass, it is possible to perform dynamics by using the chain rule to transfer forces from the “virtual sites” to the massive particles. These constructions, enumerated below, provide a means for breaking out of the “one atom, one site” paradigm that has dominated classical molecular dynamics. The prmtop format utilized by the sander and pmemd programs does not always provide a straightforward means of expressing the relationships between virtual sites and their parent (or “frame”) atoms. In Amber20, the sander and pmemd programs only support the most widely used cases of virtual sites (e.g. TIP4P and TIP5P water), but efforts are underway to support a broader variety of these virtual sites.

The *mdgx* program provides a means for adding any number of virtual sites to an existing force field, with custom charges and even Lennard-Jones properties. The only limitations with the virtual sites are that no new bonded terms may be added, that the virtual sites carry zero mass, and that each virtual site location be determined by two or three frame atoms on the same residue which do have mass. The constructions below follow those

Figure 18.1.: Frame styles in *mdgx*.

outlined in the GROMACS manual; a four-point frame construction devised by the GROMACS team is not yet implemented, but a “zeroth” frame type is available in *mdgx* which allows, without changing the *prmtop*, run-time modification of existing atomic non-bonded parameters.

In the Fig. Figure 18.1 on page 347, the `&rule` namelist variables for specifying each virtual site constructor are superimposed on atoms, vectors, and angles. In Style 1, the virtual site lies along the line determined by two atoms; `v12` denotes the fraction of the distance between the two atoms at which to place the virtual site. In Style 2, the virtual site lies in the plane determined by three atoms at a point determined by a combination of the displacements between atoms 1 and 2 and atoms 2 and 3. Virtual sites of Styles 1 and 2 are located by linear combinations of the positions of their frame atoms. In Style 3, the virtual site is located along the line described by frame atom 1 and a point between frame atoms 2 and 3 (`v23` denoting the fraction of this distance), at a fixed distance `v1e` from frame atom 1. Style 4, perhaps the most mathematically challenging frame type to define but very useful and intuitively comprehensible, places a virtual site at a fixed distance `v1e` from frame atom 1 such that the angle illustrated has the value `theta` (specified in radians in the `&rule` namelist). The virtual site remains in the plane of the frame atoms, and frame atom 3, which must not be colinear with the other frame atoms, orients the sign of `theta`. Virtual sites of Style 5 are defined as sites of Style 2, but projected normal to the plane according to a multiple `v12x13` of the cross product of the vectors between frame atoms 1 and 2 and frame atoms 1 and 3. Note that virtual sites of Styles 1, 2, and 5 will stretch with their frames, whereas 3 and 4 will not. The stretching will be minor if the frame atoms are bonded as shown in the figure. Due to the manner in which virtual sites are positioned in *mdgx*, frame atoms 2 and 3, and the virtual site when placed, must lie within half the van-der Waals non-bonded cutoff of frame atom 1. This should seldom if ever be a problem. A complete list of `&rule` namelist variables follows (shorthand aliases in parentheses):

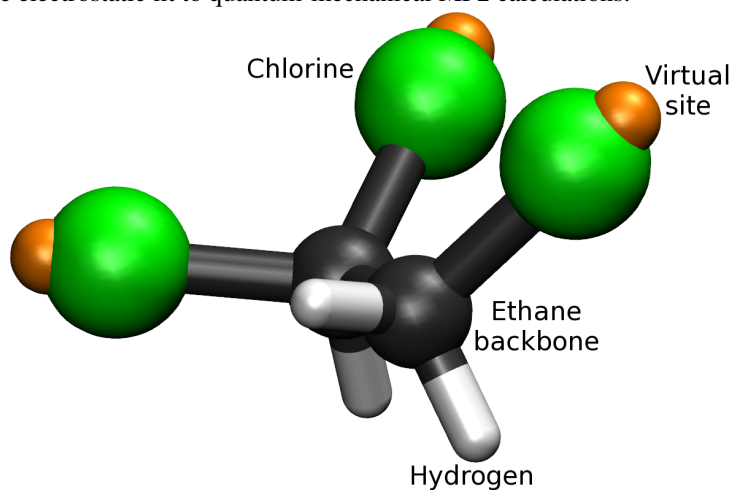
- **FrameAtom**[1,2,3] (`frame`[1,2,3]): specifies the frame atoms needed for virtual site construction
- **ExtraPoint** (`epname`): The name of the virtual site
- **AtomName** (`atom`): The name of the virtual site (alternate specifications)
- **FrameStyle** (`style`): The frame style to use (see descriptions in the preceding figure); acceptable values are 0 through 5
- **Exclude**[2,3] (`excl`[2,3]): The virtual site is definitively 1:1 bound to frame atom 1 and thereby inherits all 1:2, 1:3, and 1:4 neighbors of frame atom 1, but if ? is 2 or 3 then the virtual site will also be considered 1:1

to frame atoms 2 or 3 and inherit their bonded neighbors as well. This will not affect the 1:2, 1:3, and 1:4 neighbor lists of the frame atoms themselves.

- **Vector12** (v12): Defined according to frame type; see preceding paragraph and illustration.
- **Vector1E** (v1e): Defined according to frame type; see preceding paragraph and illustration.
- **Vector13** (v13): Defined according to frame type; see preceding paragraph and illustration.
- **Theta** (theta): Defined according to frame type; see preceding paragraph and illustration.
- **Vector23** (v23): Defined according to frame type; see preceding paragraph and illustration.
- **Vector12x13** (v12x13): Defined according to frame type; see preceding paragraph and illustration.
- **Charge** (q): Charge of the virtual site
- **Sigma** (sig): Lennard-Jones σ parameter for the virtual site
- **Epsilon** (eps): Lennard-Jones ϵ parameter for the virtual site
- **ResidueName** (residue): The residue to which extra points will be added. Because it is specified according to the four-character name, there is some possibility for ambiguity as terminal residues often have the same names as residues in the middle of a chain. Therefore, in order to add a virtual site to an the amino terminus of N-terminal alanine but skip over alanines within a polypeptide, the N-terminal alanine would have to be given a new name within the **prmtop**.

Through the `&rule` namelist, *mdgx* can fit partial charges for virtual sites. Ultimately, the `prmtop` format will be extended and `pmemd` will be able to read the *mdgx* palette of virtual sites in addition to its own, but without a means for generating parameters the sites would not be useful. For now, *mdgx* permits users to test the benefits of virtual sites for reproducing molecular electrostatics through a more elaborate distribution of monopoles. It is possible in *mdgx* (noting that the rigid geometry of the massive atoms is the same throughout all TIP water models) to simulate TIP4P or TIP5P water starting from a `prmtop` containing TIP3P water, although it is more convenient and perhaps marginally faster to simulate beginning with a `prmtop` specifying the more complex water model.

Virtual sites added in this manner follow the neighbor conventions that virtual sites are counted as “1:1” neighbors of their first parent atoms and then inherit all 1:2 (bond), 1:3 (angle), and 1:4 nonbonded neighbors of the first parent atom. It is also possible to endow virtual sites with neighbors of other parent atoms, effectively declaring the virtual sites to be 1:1 neighbors of more than one atom. The neighbor list updates implied by adding virtual sites do not get applied retroactively, however, so multiple frame atoms do not become 1:1 neighbors of each other. Because of the exclusions implied by different frame constructions, care should be taken when defining parent atoms. For instance, in the chlorinated ethane derivative below virtual sites of frame type 1 (v12 = -0.3, with chlorines being frame atom 1 and the bonded carbons being frame atom 2) can be shown to significantly improve the electrostatic fit to quantum-mechanical MP2 calculations.



In principle, the frame atom 1 may be defined as the carbon, with the chlorine (which is actually closest to the virtual site) merely defining the direction of the virtual site projection. However, this construction omits interactions between virtual sites on opposite ends of the molecule, and as a result the torsional conformations of the molecule are drastically altered (so much so that the hydration free energy in explicit solvent simulations changes by more than 3 kcal/mol). If the chlorines themselves are made frame atom 1 in each virtual site frame, the virtual sites become 1:4 neighbors to one another and interact by a slightly screened electrostatic potential.

The effects on the torsional distribution and resulting hydration free energy are then much more modest. This trichloroethane represents an extreme case, but more subtle examples abound. In general, virtual sites can change the charge distribution of a molecule to roughly the same degree that refitting an atom-centered charge model to new quantum data does. Ideally, torsional parameters would be refitted in all cases to accommodate the new electrostatic model.

18.6. Bonded Term Fitting in *mdgx*

Having the capabilities to read multiple topologies and coordinate sets, compute energies, and to optimize parameter sets made a bonded parameter fitting module a natural extension of *mdgx*. Like the RESP fitting module, the bond parameter fitting routines can read multiple systems and conformations and determine the best overall values for harmonic bond, harmonic angle, and torsion Fourier series appearing in multiple contexts. The while the RESP module is limited to 512 systems and conformations and makes its fitting matrices based on thousands of data points from each one, there is no practical limit to the number of systems and conformations that the bond parameter fitting module can muster, although it seeks only to make the total internal energy of each conformation match a single target value (presumably obtained from quantum mechanics). This duplicates some functionality in the *paramfit* program described in Chapter 17, but with improved capabilities for large data fitting problems. Results are written to several different files: the **forcedump** file (-d option on the command line or in the &files namelist) stores fitted parameters in the standard Amber parameter file or frcmod formats (i.e. parm99.dat, frcmod.ff14SB), **mdout** provides extensive analysis of the fit and sampling of each fitted parameter in the data set, and the *accrep* option described in the table below creates a complete report of the correlations, system by system, if requested.

Bonded term fitting is called by including the ¶m namelist in an input file. If detected, this namelist will send *mdgx* into a distinct run mode for parameter optimization. The goal is to take in parameter files, topologies, coordinates, and quantum single point energies, then organize this data. The first thing *mdgx* does is trace all parameters back to their sources in the parameter files (i.e. parm10.dat) and then determine which parameters are open for optimization. Then, *mdgx* computes molecular mechanics energies of all the molecular systems and conformations (as if they were isolated in the gas phase) and subtracts off the portion due to non-optimizable sources. The goal is to make the *relative* potential energy surfaces of the systems as close as possible to the corresponding quantum energy surfaces.

The *mdgx* program also offers the ability to fit CMAP surfaces alongside bond, bond angle, and torsion parameters. This unique capability treats every grid point on the CMAP surface as an independent, optimizable variable. Instead of four cosine terms in each of two dihedrals, a CMAP can contain up to 576 unique parameters—a seventy-fold increase in the amount of detail. The ¶m namelist is designed to accept trajectories and files of energy values for each frame, simplifying inputs of tens of thousands to hundreds of thousands of data points. The related &configs namelist can generate conformations and automate the QM calculations to construct such a large data set. These features can address a vast new parameter space such as that afforded by CMAPs. However, the explosion in parameter space can also be mitigated by fitting lower-resolution maps (i.e. 8 or 12 points on a side, rather than the full 24), which are then promoted (by interpolating) to the full 24-by-24 map that *sander* and *pmemd* expect.

The options available in the ¶m namelist include (shorthand aliases in parentheses):

- **System** (sys): A fitting data point. This keyword must be followed by three items: the name of a topology file, the name of a corresponding coordinate file, and the energy of this system in the stated conformation.
- **FitBonds** (bonds): Requests a linear least-squares fit for bond stiffnesses in the system.
- **FitAngles** (angles): Requests a linear least-squares fit for angle stiffnesses in the system.

18. Molecular Mechanics Parameter Fitting in mdgx

- **FitTorsions** (torsions): Requests a linear least-squares fit for torsion stiffnesses in the system.
- **FitCmaps** (cmaps): Requests a linear least-squares fit for CMAP surfaces in the system.
- **FitB** (fitb): Request that a specific bond parameter be included in the linear least-squares fitting.
- **FitA** (fita): Request that a specific angle parameter be included in the linear least-squares fitting.
- **FitH** (fith): Request that a specific torsion parameter be included in linear least-squares fitting.
- **FitBondEq** (bondeq): Requests that bond equilibrium constants be fitted alongside their spring constants.
- **FitAngleEq** (angleq): Requests that angle equilibrium constants be fitted alongside their spring constants.
- **FitLJ14** (fitscnb): Requests a linear least-squares fit for Lennard-Jones 1:4 scaling factors.
- **FitEE14** (fitscee): Requests a linear least-squares fit for electrostatic 1:4 scaling factors.
- **ReportAll** (repall): Flag to activate output of all parameters encountered during the fitting procedure, including those that were not adjusted by the fit but nonetheless contributed to the molecular mechanics energies. Default is 1 (write all parameters to the Amber parameter file), appropriate for creating a parm##.dat file to specify a new force field. Set to 0 to create files more akin to frmod files.
- **ShowProgress** (verbose): Alert the user as to the progress of the fitting procedure. Runs involving thousands of molecular conformations and hundreds of parameters can generally be completed in a few minutes. Default is 1 (ON). Set to zero to suppress output.
- **ElimOutliers** (elimsig): Flag to activate removal of molecular conformations whose energies are far outside the norm for other conformations of the same system. Default 0 (do not remove outliers).
- **ConfTol** (ctol): Tolerance for deviation from the mean energy value, specified as a function of the standard deviation for all conformations of the same system. Conformations of a system which exceed this threshold will be reported if verbose is set to 1, and removed from consideration if elimsig is set to 1. Default 5.0 sigmas.
- **EnergyUnits** (eunits): Units of the target energy values. Default Hartrees. Acceptable values include Hartree/Atomic, kJ/kilojoules, and J/joules. Case insensitive.
- **AccReport** (accrpt): Accuracy report on the fit. Contains extensive analysis on the resulting parameters, in MatLab format.
- **ParmTitle** (title): Parameter file title. This is not a file name, but rather the title appearing on the first line of the printed file named by the -d command line / &files namelist argument.
- **Vdw14Fac** (scnb): Sets a universal 1:4 scaling factor for van-der Waals interactions. Use this input to change the scaling on all systems simultaneously.
- **Elec14Fac** (scee): Sets a universal 1:4 scaling factor for electrostatic interactions. Use this input to change the scaling on all systems simultaneously.
- **BondRest** (brst): General value for harmonic restraints on bond stiffness constants.
- **AngleRest** (arst): General value for harmonic restraints on angle stiffness constants.
- **DihedralRest** (hrst): General value for harmonic restraints on dihedral cosine amplitude constants.
- **CmapRest** (mrst): General value for harmonic restraints on CMAP surfaces, coupling adjacent points to have similar values. This will push the surface towards flatness.

- **CMAPDensity** (cmapdens): Density of CMAP surfaces to use in fitting. CMAP surfaces will always be printed to the parameter file outputs as 24 x 24 objects. If CMAPDensity is a factor of 24, the outputs will have perfect fidelity to the original fitted surfaces. Otherwise, the output CMAPs will bear close resemblance to the original fitted results.
- **RestrainB** (sbrst): Applies a specific restraint stiffness to the value of a fitted bond, equivalent to changing brst for that bond alone. This command takes subdirectives of atom type names, plus 'Keq' and 'Leq' (each followed by a positive real number to denote the target stiffness and equilibrium bond length, respectively). These subdirectives may be given in free format.
- **RestrainA** (sarst): Applies a specific restraint stiffness to the value of a fitted angle, equivalent to changing arst for that angle alone. This command takes subdirectives similar to sbrst, although 'Leq' corresponds to the equilibrium angle rather than length.
- **RestrainH** (shrst): Applies a specific restraint stiffness to the value of a fitted torsion amplitude, equivalent to changing hrst for that torsion alone. This command takes subdirectives (in free format) of atom types, 'period' or 'per' followed by a real number for the periodicity, 'weight' or 'rwt' followed by a real number for the restraint strength (which scales just like hrst), and 'target' or 'trg' followed by a real number if the target value of the restraint is non-zero.
- **BondCoupling** (brstcpl): General value for pricing 1Å changes in the fitted equilibrium constant with kcal/mol-Å² changes in bond spring constants. The default is 5000.0, which penalizes 50 kcal/mol changes in the stiffness at the same rate as 0.01Å changes in the equilibrium value. Only relevant with FitBondEq = 1.
- **AngleCoupling** (arstcpl): General value for pricing 1-degree changes in the fitted equilibrium constant with kcal/mol-rad² changes in angle spring constants. The default is 114.59, which penalizes 2 kcal/mol changes in the stiffness at the same rate as 1 degree changes in the equilibrium value. Only relevant with FitAngleEq = 1.
- **BondBasisSep** (lpost): Distance from the original bond equilibrium length to place the equilibrium values of either of two basis functions for fitting a new bond term.
- **AngleBasisSep** (thpost): Distance from the original angle equilibrium length to place the equilibrium values of either of two basis functions for fitting a new angle term.
- **Spectrum** (spectrum): Request that a particular bond, angle, or dihedral from among the adjustable parameters be sampled near various values along a specified range, or otherwise included as a reoptimizable variable while others are resampled. This keyword invokes its own sort of namelist: the sub-directives can be given in any order, but they must all be given on the same line. Words that are not explicitly sub-directives or values following them may be counted as atom types, so long as they have fewer than four characters, until four such types are cataloged. Sub-directives include retain (parameters matching this request are reoptimized but not resampled), sample (parameters matching this request are resampled), order (followed by a value, 2 = bonds, 3 = angles, 4 = torsions), min and max (followed by values, the resampling range limits), spc (the resampling discretization), and break (stop adding new sub-directives to this spectrum command). This is an experimental feature, but may be useful for generating multiple force field candidates with subtly different behavior around the optimal data fit.

The data fitting capabilities in *mdgx* focus on a single linear least-squares problem, unless the experimental “spectrum” option is invoked, in which case a series of linear least squares problems are solved. This is a very active area of development in *mdgx*, and will continue to gain new features and capabilities in future patches, with the goal of leveraging high-performance computing to deliver robust parameter development to novice users.

18.7. Configuration Sampling

While *mdgx* provides lots of options for its force field applications, the charge and bonded parameter development are little more than big data fitting problems. They rely on a finely sampled and complex data set, and generating such a thing is a tedious process, prone to fatigue and human error. The `&configs` namelist in *mdgx* is designed to streamline this process by energy-minimizing hundreds to thousands of instances of a single structure, subject to different constraints, *en masse*. In principle, everything that this module does could be accomplished with a shell script executing *sander* or *pmemd*, but the execution would take tens to hundreds of times longer and things that are easily done with *mdgx* `&configs` would take complex shell scripts. Even then *mdgx* performs analyses on the data set as a whole that help users to understand whether the data set is suitable for quantum calculations, and if not, how to improve it. With a *sander* script, creating 500 energy-minimized conformations of a drug molecule that sample rotation around three critical dihedrals might take an hour, and the process may have to be repeated several times to ensure that the restraints are sufficiently stiff or that other degrees of freedom are properly relaxed or randomized. With *mdgx* `&configs`, the same process will take about a minute, making it possible to set up higher quality data sets in a single sitting.

To support the operations of configuration sampling, several *mdgx* inputs have been upgraded to support directories and regular expressions rather than simply the names of individual files. A single structure can serve as input to the configurations module to make hundreds of copies, but a trajectory, list of trajectories, or directory containing any number of single-frame files can also serve. In this manner, one structure can become a thousand, or the same minimization protocol can be applied to a thousand conformations of the same structure.

Available options for the `&configs` module fall into several categories. The minimization protocol itself is guided by parameters similar to *sander* inputs. Additionally, *mdgx* provides a feature for “shuffling” results and attempting additional minimizations. This helps solve the problem of escaping local minima without resorting to much more costly simulations with temperature to jostle small molecules around.

- **Verbose** (verbose): Sets the verbosity level (0 is silent, 1 will give frequent updates on the command line).
- **Replicas** (count): The number of configurations to generate, if starting from a single configuration in *inpcrd* or *restart* format.
- **MaxCycles** (maxcyc): The maximum number of line minimization steps to attempt in any one round of energy minimization.
- **SDSteps** (ncyc): The number of steepest descent line minimization steps to perform before switching to a conjugate gradient method. As with the eponymous keywords in *sander*, *ncyc* must be less than or equal to *maxcyc*.
- **ExclTableSize** (exclmax): The maximum number of atoms for which a table of non-bonded scaling factors will be kept. For small systems, it is faster to pre-calculate whether non-bonded interactions will be excluded or attenuated and store these values in a matrix. However, this is memory-intensive and will trash the cache for larger systems. In those cases it is better to store a different sort of data structure that will quickly determine whether two atoms constitute an exclusion.
- **ForceConverge** (frctol): Convergence criterion for the optimization. This is a quantity of force--if forces on all particles have lower magnitude than this value, the energy optimization for that configuration will be deemed converged.
- **StepConverge** (steptol): Convergence criterion for the optimization. This is a quantity of distance--if the movement of all particles along the current force vector is driven lower than this value, the energy optimization for that configuration will be deemed converged.
- **InitialStep** (step0): Initial step size for the energy optimization. This is a quantity of distance: the total magnitude of the initial step along the first computed force vector, that is the square root of the sums of squares of the displacements of all particles from their original positions, will be equal to this number (default 0.01Å). The step size will be iteratively changed throughout optimization and will be tailored to each configuration.

- **ShuffleCount** (nshuffle): The number of times to restart energy minimizations towards the specified restraint targets using different initial states.
- **ShuffleStyle** (shuffle): The type of shuffling to perform if nshuffle > 0. Available methods include "bootstrap" (new initial states will be assigned randomly from existing solutions, with replacement--one solution can serve as the initial state for more than one configuration), "jackknife" (the default--each existing solution will be assigned as the initial state for energy reoptimization to one and only one configuration), and "proximity" (every solved configuration will be evaluated in terms of the restraint targets of every other, and new initial states will be randomly chosen from among solutions whose restraint energies are within a certain threshold of the MINIMUM energy found for any existing solution with respect to the particular restraint targets of a given configuration).
- **ProximateNrg** (eprox): Threshold energy for taking existing solutions as the initial states for new attempts at energy minimization if using "proximity" reshuffling (default 5.0 kcal/mol).
- **ReplacementTol** (erep): Threshold for accepting a new solution based on a different initial state. The new solution must supplant the energy of the existing one by at least this amount. Default 1.0e-4 kcal/mol.
- **Direction** (shfdir): The direction to replace energies when reshuffling energy optimizations. Choices are "up" and "down". Default is "down," but replacement can be made to move the energies upwards, finding new local minima with higher overall energies.

The &configs module performs energy minimization subject to NMR-like restraints and supports multiple sampling strategies for each of them. Each restraint and sampling strategy commands contains its own vocabulary and multiple descriptors.

- **RandomSample** (random): Perform random sampling within a range. This keyword must be followed by a series of commands, all on the same line of the input file, but the order of the sub-commands is flexible. After seeing this keyword, mdgx will search the remainder of the line until it hits another keyword from the &configs namelist; until then it will associate any input it finds with the previous "random" or "RandomSample" keyword. The range of sampling in this case is absolute: a flat-bottom harmonic potential will be constructed, centered on the spot randomly chosen between the limits "min" and "max", or given between two { } braces. To specify that all configurations be restrained towards a single target value, the keyword "center" may be used in place of "min", "max", or { }. The potential shall be flat up to a distance "fbhw" (flat bottom half width) from the center, and thereafter rise quadratically with a coefficient "Krst" (stiffness constant K of the restraint) over a length specified by the "quadratic" keyword, or up to a point at which the restraint force would reach a limit given by the "Ftop" keyword. Beyond this limit, the force will be clamped and the restraint potential will be effectively linear, which helps to ensure that restraints to positions far from the initial configuration do not break things like chirality. Because it is more intuitive to specify a maximum restraint force than a quadratic window, "Ftop" will take priority over "quadratic" if both keywords are given. The defaults are to have 64 kcal/mol restraints applied after a 0.5Å flat bottom half width, topping out at 32 kcal/mol-Å applied force.
- **GridSample** (uniform): Perform sampling on regular intervals within a range. All of the keywords from RandomSample apply here as well.
- **RandomPerturb** (rpert): Perform sampling on regular intervals within a range based on the arrangement of atoms in each initial structure. All of the keywords from RandomSample apply here as well, except that the range now specifies minimum and maximum values relative to the initial arrangement of atoms. If multiple initial structures are read in, this will perturb each of them by similar random amounts.
- **GridPerturb** (gpert): Perform sampling on regular intervals within a range based on the initial arrangement of atoms. This is to RandomPerturb as GridSample is to RandomSample.
- **LinkOperationsCombine** (combine): Combine two operations involving grid-based, interval sampling. Without any such combinations, the interval sampling restraints in each configuration will march from one

end of their respective ranges to the other, in unison--this will generate a line of configurations in the multi-dimensional space defined by each restrained coordinate. To sample two or three dimensions of the space simultaneously at regular intervals, combine the operations. Up to three operations may be combined. For N combined operations, mdgx will take the N th root of the total number of configurations and take this many samples along each of the combined restraint dimensions.

- **MovingAtoms** (belly): Make only the atoms in the given ambmask string movable during geometry optimization.

The &configs module supports multiple output formats for the minimized structures, including the well known PDB format and outputs that can serve as input files for some popular quantum packages.

- **ShowOrigins** (showorig): Flag to have mdgx show the original files for each configuration that it solves. If all configurations start from a single state in a single file, the default behavior is to withhold this reporting. However, if there are many files, the origin of each configuration may not be so obvious, and while mdgx does attempt to alphabetize and organize long lists of files arising from directory searches or regular expressions the evolution of molecular configurations may be of interest. In these cases the default behavior is to report the origins of each configuration.
- **OutputBase** (outbase): The bases of the output file names for configurations. The format will be <base><number>.<suffix>. Multiple strings may follow this keyword, so long as they are all on the same line. Each string provided will be matched with a suffix and a style provided, in the order each is given.
- **OutputSuffix** (outsuff): Suffixes of the output file names for printed configurations.
- **OutputType** (write): The type of output to write, options being "CRD" (old Amber .crd format trajectory), "CDF" (Amber netCDF trajectory), "INPCRD" (Amber ascii 7-decimal place inpcrd file for individual configurations), "PDB" (PDB format, with descriptions of the way the configuration was generated in the REMARK section), and "ORCA", "GAUSSIAN", "MOLPRO", and "GAMESS" for input files to various quantum packages. If trajectories are being written, all configurations that pass sanity checks will be printed to the file. For the other formats, individual configurations will be printed to separate files. More than one type of output may be written after creating a set of configurations.

Sanity checking is an essential part of data set creation. Because all mdgx outputs are already energy minimized with respect to an input force field, they are already mostly "sane." However, with any minimization there can be traps, local minima, and residual strain that should be considered before submitting the configuration to quantum methods or using it as input data for making a force field. *mdgx* will automatically check for some common problems, decline to print structures that have them, and report what went wrong so that the configuration sampling can be repeated for better results.

- **BondStrain** (bstrain): The maximum bond strain (according to the input force field, as given in the topology file) that will be tolerated in any configuration that is to be printed.
- **AngleStrain** (astrain): The maximum bond angle strain (according to the input force field, as given in the topology file) that will be tolerated in any configuration that is to be printed.
- **StrainLimit** (strainlim): The maximum restraint penalty that will be tolerated in any configuration. Note that, for any of these sanity checks, convergence of the energy minimization is NOT an automatic fail--it will simply be noted in the report file summarizing the process. Rather, the sanity checks pertain to features of the structures that appear well outside the applicable range of molecular mechanics functions.

When the output structures are given as inputs for quantum calculations, there are other considerations for running the QM program itself. While advanced use of the quantum programs may require post-processing the files with a shell script, *mdgx* does interpret some basic run parameters and incorporate them into its results.

- **MaxMemory** (maxcore): When ordering mdgx to print configurations as input files to quantum packages, this states how much memory should be available for QM calculations.

- **CPUCount** (ncpu): The number of CPUs to apply in each QM calculation.
- **Multiplicity** (spin): The multiplicity to assign to this system (in all its configurations) for quantum calculations. *mdgx* is not able to calculate this on its own.
- **QMTheory** (qmlev): The level of theory to apply in quantum calculations.
- **QMBasis** (basis): The basis set to apply in quantum calculations.
- **Checkpoint** (chk): The checkpoint file to write if using Gaussian for QM calculations.

18.8. Parallel Generalized Born Problems on the GPU

The *mdgx* program's ability to read multiple topologies enabled a new strategy for conducting simulations of small, implicit solvent systems on the GPU. The benchmarks on the Amber website show how a 305 atom Trp Cage system runs at greater speed than a much larger, 2400 atom myoglobin system, but not nearly the 64-fold increase that would be expected from the system's size and the scaling of the non-bonded interaction calculation which dominates the effort. Not even the myoglobin system compares favorably to the very large nucleosome simulation, which finally tops out the GPU's bandwidth for generalized Born calculations.

The *mdgx* peptide multi-simulator is the program's first CUDA extension for implicit solvent GB and gas-phase molecular dynamics. It treats a GPU as miniature Beowulf cluster of streaming multiprocessors (SMPs) and uses the device's block scheduler as a queueing system of sorts. This paradigm shift in GPU utilization can backfill idle SMPs to reap enormous gains in total throughput on small systems (928 atoms maximum) and may even exceed the simulation rate of *pmemd.cuda* for very small systems (less than 225 atoms). While it offers RATTLE, the equivalent of SHAKE for *mdgx*'s velocity-Verlet integrator, the module also offers a velocity-Verlet I/r-RESPA multi-time stepping scheme which performs at least as well as SHAKE in most cases, and often considerably better in terms of speed and energy conservation.

The sizes of systems served by this module cover a range ideal for Generalized Born calculations. Systems with more than 928 atoms will engage the *pmemd.cuda* GB engine with reasonable efficiency. The *mdgx* engine is instead designed for maximum throughput on one or more systems by simulating independent copies on all of the GPU's SMPs.

A reliable approach for getting the best throughput on a single system is to call for a number copies of the system equal to number of SMPs on the GPU SMP, or twice that number if the system has 512 or fewer atoms, or four times that number if the system has 256 or fewer atoms. The number of SMPs will be displayed in *mdgx* output (Section 5, 'GPU Utilization'), as will the thread block and block grid sizes.

For the best throughput on an array of systems with varying sizes, the first thing to understand is that simulation time will scale as the square of the system size and cause each system to finish at a different rate. If the spread of sizes is great, this will create a lot of idle SMPs as the GPU works to finish the largest simulation. However, if *mdgx* has additional systems to run, it can backfill the idle SMPs with more work. The program will automatically arrange the systems internally in decreasing order of size, to run the largest first and the smallest last. It is therefore advantageous, if trying to simulate many systems of disparate sizes, to queue up many more systems than the size of the block grid (which will be determined by the number of SMPs and the size of the largest system). By queueing three to four times as many systems as the size of the *mdgx* block grid, the entire GPU can keep busy.

The main input for this section is the Peptide / oligomer keyword, followed by a list of subdirectives reminiscent of sander command line input. Many directives will be carried down from the &files and &cntrl namelists, such as DoRATTLE / rigidbonds, thermostat controls, and the time step. The &pptd namelist can override some of these directives for specific oligomers. Other parameters that can influence the dynamics, such as the GB style, are native to the &pptd namelist as this is the only context in which they can be used.

- **Peptide** (oligomer): A system to simulate in non-periodic conditions (implicit solvent or vacuum). After seeing this keyword, *mdgx* will search the remainder of the line until it hits another keyword from the &pptd namelist; until then it will associate any input it finds with the previous "oligomer" or "Peptide" keyword. Each oligomer requires its own topology and input coordinates, specified by the -p and -c flags to mirror sander command line input. Files for mdout, mdcrd, and mdrst can be supplied with flags -o, -x,

and -r, respectively, again like sander command line input. Multiple copies of the system can be specified by including the N-rep flag followed by the number. It is also possible to simulate replicas at a range of temperatures by providing the T-rep flag followed by an integer as well as a temperature range with the flags Tmin and Tmax (each followed by a real number). Replicas will be simulated at evenly spaced intervals of the temperature, inclusive of the two end points (i.e. Tmin 100.0 Tmax 200.0 T-rep 11 would create replicas at 100.0, 110.0, 120.0, ..., 200.0K). To simulate all replicas at one particular temperature which differs from temp0 in &cntrl, temp0 may also be supplied as a flag for a specific oligomer. Also, the -p flag may be replaced by -pi and -pf, each followed by a topology file, to create replicas based on interpolated topologies. The two topologies must have similar atom counts, names, and bonding patterns, but otherwise are just two endpoints. With two topologies, the P-rep flag followed by an integer will specify the number of copies to make at regular intervals along a linear interpolation between the topologies, again inclusive of the end points.

- **GBStyle** (igb): Type of Generalized Born solvent to use. All standard *sander* settings, including 7 and 8 (neck GB) and 6 (vacuum conditions) are available.
- **GBOffset** (offset): The offset for GB radii calculations. For igb=8 (Neck GB II), this is 0.195141. For all other models it is 0.09.
- **MinorSteps** (bondstep): The number of minor steps to use in a velocity Verlet I/r-RESPA multiple time-stepping scheme. To say "bond steps" is a bit of a misnomer: bond, angle, and 1-4 non-bonded interactions are all recalculated on each minor step in between major steps where general non-bonded and dihedral interactions are calculated.
- **Dielectric** (diel): Dielectric constant for the solvent, whether GB or some continuum homogeneous environment.

19. Python Metal Site Modeling Toolbox (pyMSMT)

19.1. Introduction

The Python Metal Site Modeling Toolbox (pyMSMT) is a python package for metal site modeling of mixed systems (especially protein systems) for ultimate use in molecular dynamics simulations. It can facilitate parameterization of both the bonded and nonbonded models. This toolbox was originally developed by Pengfei Li in Prof. Kenneth M. Merz, Jr.'s research group at Michigan State University and now who is a faculty at Loyola University Chicago. Li and Merz have written a comprehensive review about metal ion modeling, which covers a wide spectrum of models including quantum mechanics models, classical force field models, polarizable force field models, reactive force fields, and some other types of models.[460] People who are interested can check the review article for more details. Users are welcome to send suggestions and bug reports to AMBER Mailing List (amber@ambermd.org).

In the current version, six applications are supported by the pyMSMT package:

1. **MCPB.py**: a Python version for the Metal Center Parameter Builder (MCPB). MCPB.py supports various metal ions (more than 80 metal ions with partial charges/oxidation-states from +1 to +8, literally), different AMBER force fields (ff94, ff99, ff99SB, ff03, ff03.r1, ff10, ff14ipq, ff14SB, ff14SB.redq, ff14SBonlysc, ff19SB, ff15ipq, ff15ipq-vac, fb15, GAFF, and GAFF2 in the current version), different parameterization methods (Seminario, Z-matrix and empirical), and different models (bonded model and nonbonded models for metal ions). It can facilitate parameterization of both metalloproteins and organometallic compounds. The workflow is more efficient and many of the modeling processes in previous MCPB versions are automatically implemented into MCPB.py (MCPB.py uses about 10 fewer steps and many fewer scripts than MCPB). An application note of the program is published by Li and Merz.[461] The main scheme and parameters are based on previous papers published by Merz et al.[135–137, 144–147, 462] Recently, Li and Merz have published a book chapter about MCPB.py, in which they provided a series of useful tips for using the program.[463]
2. **IPMach.py**: the Python ion parameterization machine. IPMach.py can largely facilitate the parameterization for the 12-6 LJ model and 12-6-4 LJ-type model of ions. It can automatically parameterize the 12-6 LJ model for a given hydration free energy or ion-oxygen distance, and the 12-6-4 LJ-type model for given hydration free energy and ion-oxygen distance.
3. **PdbSearcher.py**: the Python version of Pdbsearcher. PdbSearcher.py better supports the automatic recognition of the metal centers in a PDB file due to better compatibility with the PDB naming scheme of metal ions.
4. **OptC4.py**: a program to optimize the C_4 terms of the 12-6-4 potential using the AMBER topology and coordinate files. It can automatically optimize the metal-site-related C_4 terms to better reproduce the experimental structure. It uses the sum of unsigned error of metal site bond lengths, angles and dihedrals as the criterion (in which the bond, angle and dihedral have different weights). For each optimization cycle, the structure will be minimized by OpenMM[464, 465] and then have the sum of unsigned error calculated. It requires OpenMM version 6.3 and an installed SciPy package in current version.
5. **CartHess2FC.py**: the program to calculate the force constants using Cartesian Hessian matrix based on Seminario method. It can calculate all the bond and angle force constants of a system based on a Gaussian fchk file or GAMESS log file that contains the Cartesian Hessian matrix.

19. Python Metal Site Modeling Toolbox (pyMSMT)

6. **espgen.py**: the Python version of espgen in the antechamber package. It can extract the electrostatic potential information from a Gaussian output file or GAMESS log file that contains this information. It supports Gaussian03, Gaussian09 and GAMESS.
7. **ProScrs.py**: the "Protein Scissors" program for cutting and capping the protein segment into clusters.
8. **car_to_files.py**: the program to generate the mol2 and PDB files based on the car file. This function is designed for users of the INTERFACE force field in AMBER, which can be checked at <https://bionanostructures.com/interface-md/>.
9. **amb2chm_psf_crd.py**: the program to generate the CHARMM PSF and CRD files based on the AMBER prmtop and inpcrd files. This function is designed for users of the AMBER force field in CHARMM.
10. **amb2chm_par.py**: the program to generate the CHARMM PAR file based on the AMBER dat/frcmod file. It can combine several AMBER dat and/or frcmod files into one CHARMM par file in one single step. This function is designed for users of the AMBER force field in CHARMM.
11. **mol2rtf.py**: the program to generate the CHARMM RTF file based on the mol2 file. This function is designed for users of the AMBER force field in CHARMM.
12. **amb2gro_top_gro.py**: the program to generate the GROMACS top and gro files based on the AMBER prmtop and inpcrd files. This function is designed for users of the AMBER force field in GROMACS.
13. **metaldpb2mol2.py**: the program to convert PDB files of metal ions to mol2 files. This function is designed for users of the MCPB.py program.

19.2. Usage

The following is a summary of the usage and options for the three applications:

19.2.1. MCPB.py

```
Usage: MCPB.py -i input_file -s/--step step.number
           [--logf Gaussian/GAMESS-US output logfile]
           [--fchk Gaussian fchk file]

Options:
  -h, --help            show this help message and exit
  -i INPUTFILE           Input file name
  -s STEP, --step=STEP  Step number
  --logf                Gaussian/GAMESS-US output logfile
  --fchk                Gaussian fchk file
```

The following is an introduction of the variables in the input file:

(Reminder: there should be no blank lines in the input_file. The values or parameters should follow the variables separated by a space.)

Required variables:

ion_ids The PDB atom ID(s) of the complex's central metal ion(s). If there is only one metal ion in the metal site, you need to put its PDB atom ID after the variable. If there are multiple metal ions in the metal site, you need to put the PDB atom IDs of all these metal ions (with these IDs are separated by space) after the variable. Each PDB atom ID should be an integer value.

ion_info This variable is only required for the nonbonded model without refitting the residue charges (step number 4n2). In all, there are four data points required for each metal ion: 1) the residue name of the metal ion in the PDB file; 2) the atom name of the metal ion in the PDB file; 3) the element symbol of the metal ion; 4) the charge (or oxidation state, which needs to be an integer) of the metal ion. For example: ZN ZN Zn 2 (the

first two are the residue and atom name of the Zn²⁺ ion in the PDB file, the third is its element symbol and the last one is its charge).

ion_mol2files The name(s) of the ion(s) in the mol2 file(s) contained in the metal center. This can be one or several name(s), depending on how many kinds of ions are included in the metal center. The user can use antechamber to transfer the single ion PDB file to a mol2 file and then manually modify the atom type and the atomic charge of the metal ion in the mol2 file.

original_pdb This is the file name of the original PDB file, which should have only one chain. The PDB file should have hydrogen atoms and metal ions in it. Users are advised to use an application like pdb4amber to clean up the PDB file first. They are also advised to add the hydrogen atoms by using a webserver such as H++ before performing the modeling in MCPB.py.

Optional variables:

add_bonded_pairs Specify the bonded atom pair(s) you want to add in the model building by MCPB.py. In default MCPB.py only detect the Metal-N/O/S/F/Cl/Br/I bond, if you have a other kind of metal ligating bond in the metal site (e.g. Metal-C bond), you need to specify the atom numbers of metal and the ligating atom in the input file. There should be dash between the numbers of two atoms bonded together. For example, if you have two Metal-C bond in the metal site, and the metal has atomic number as 1001, while the two carbon atoms have atomic numbers as 1320 and 1380 respectively, you can use add following line in the input file: "add_bonded_pairs 1001-1320 1001-1380" or use two separate lines as "add_bonded_pairs 1001-1320" and "add_bonded_pairs 1001-1380". [The default value of this variable is the null list.]

add_redcrd Specify whether additional redundant coordinates added to the Gaussian calculations for the small model. This option is designed for the Z-matrix method. If you are using Seminario method, this option can be ignored. In default Gaussian performed the geometry optimization using redundant internal coordinates, the default internal coordinates may not have the metal-ligand coordination bonds and angles (means the angles which including metal) included. If these bonds and angles are not included, it will cause users could not generate related force constants when using the Z-matrix method. 0 means do not add redundant coordinates for metal-ligand coordination bonds and angles. 1 means add redundant coordinates for metal-ligand coordinate bonds and angles to the optimization of the small model. In this way, the afterwards force constant calculation will use the same redundant internal coordinates as the optimization procedure when it reads the formerly generated chk file. Care should be taken that choosing 1 may cause convergence failure for geometry optimization, when choosing 2 is suggested. 2 means only do that for the force constant calculation of the small model. This option is suggested to use when user use option 1 but could not get a converged results for the geometry optimization procedure. In this way, Gaussian will perform the geometry optimization in a default manner, but the force constants for the Z-matrix will be based on the updated redundant coordinates. [The default is 0.]

additional_resids Specify the residues' IDs for which you want to add to the models built by MCPB.py. For example, it may be a residue in the second layer which coordinates a metal bonded residue. It will increase the computational cost for QM calculations. [The default value of this variable is the null list.]

anglefc_avg A variable used to indicate whether to make an average of angle force constants derived based on different manners of choosing the sub-matrices in Seminario method. There are A-B and B-A two ways of choosing the sub-matrix for two atoms in the parameter derivation process based on Seminario method. The angle force constant obtained based on different manners of choosing the sub-matrices may not have big differences. Two options are available: 0 or 1. 0 means not making average, using the default manner to chose the sub-matrices. 1 means making average of different manners to chose the sub-matrices. [The default is 0.]

bondfc_avg A variable used to indicate whether to make an average of bond force constants derived based on different manners of choosing the sub-matrix in Seminario method. There are A-B and B-A two ways of choosing the sub-matrix for two atoms in the parameter derivation process based on Seminario method. The bond force constant obtained based on different manners of choosing the sub-matrix would not have big

19. Python Metal Site Modeling Toolbox (pyMSMT)

differences. Two options are available: 0 or 1. 0 means not making average, using the default manner to chose the sub-matrix. 1 means making average of different manners to chose the sub-matrix. [The default is 0.]

chgfix_resids Specify the residues' IDs whose charges are going to be fixed during the charge fitting. The fixed charge values are referenced from the mol2 files used during the modeling. [The default value of this variable is the null list.]

cut_off The cutoff value is used to indicate there is a bond between the metal ion and the surrounding atoms. The unit is Angstroms. [The default is 2.8.]

force_field The user-designated name of the force field. The current version supports ff94, ff99, ff99SB, ff03, ff03.r1, ff10, ff14ipq, ff14SB, ff14SB.redq, ff14SBonlysc, ff19SB, ff15ipq, ff15ipq-vac, and fb15. [The default is ff19SB.]

frcmmod_files The variable used to indicate the parameter modification file(s) for the nonstandard residue(s) (e.g. frcmmod file generated by parmchk for a ligand molecule) in the metal complex. It can be one name or several names separated by space. [The default value of this variable is the null list.]

gaff A variable used to indicate the use of a GAFF force field during the modeling. 0 means no, 1 means using GAFF, 2 means using GAFF2. [The default is 1.]

group_name The group name the user has specified. The group name is the prefix for different kinds of modeling files e.g. PDB, fingerprint and Gaussian input files for different models. [The default is MOL.]

ion_paraset The user-designated ion parameter set to be used in the nonbonded model. (This option has no influence on the metal ion VDW parameters in the bonded model. For this variable the user choose a certain VDW parameter set for the ions using the nonbonded model, which will generate a corresponding line in the LEaP input file generated by MCPB.py.) There are five options for this variable: HFE, CM, IOD, 12_6, and 12_6_4 (reminder: there are underlines between the numbers), where the 12_6 set is equivalent to the CM set. If you use the 12-6 Lennard-Jones nonbonded model, the recommended settings are the 12_6 (or CM) set: which includes HFE set for the +1 and -1 ions, the CM set for the +2 ions, and the IOD set for the +3 and +4 ions. This is because there is no specific CM set for the +1, -1, +3, or +4 ions, while the HFE set for the +1 and -1 ions and the IOD set for the +3 and +4 ions are recommended for normal usage. They are also the default settings for these metal ions. [The default is 12_6.]

large_opt A variable used to indicate whether to do an geometry optimization in the Gaussian input file. Three options are available: 0, 1, or 2. 0 means no optimization, 1 means only optimizing the hydrogen positions, 2 means full geometry optimization. [The default is 0.]

lgmodel_chg Specify the total charge of the large model. [The default value of the charge will be assigned automatically by the program, which is not guaranteed to be right. Careful check is suggested from running the Gaussian/GAMESS-US program. If it is not right, you can add this option with right charge into the MCPB.py input file and regenerate the modeling files.]

lgmodel_spin Specify the spin of the large model. [The default value of the spin will be assigned automatically by the program as 1 or 2, based on the number of electrons. This is not guaranteed to be right. Careful check is suggested from running the Gaussian/GAMESS-US program. If it is not right, you can add this option with right spin into the MCPB.py input file and regenerate the modeling files.]

naa_mol2files The variable used to indicate non-amino acid mol2 file(s) in the metal complex if there are any nonstandard residue(s) in the metal complex. Examples of nonstandard residues include hydroxyl group and ligand molecules. For these residues, the user can use antechamber to generate the mol2 file(s) by first doing an AM1-BCC or HF/6-31G* RESP charge fit and then assigning an AMBER atom type (recommended for water or hydroxyl group) or a GAFF atom type (recommended for ligand). [The default value of this variable is the null list.]

scale_factor Specify the frequency scale factor for force constant derivation based on QM methods. This scale factor will scale all the bond and angle force constants determined from the QM calculations. *Reminder:* The force constant scale factor is usually equal to the square of the frequency scale factor. For example, if you are using the HF/6-31G* level of theory to do a calculation and its frequency scale factor is 0.9, the force constant scale factor you need to use is $0.9^2=0.81$. [The default value is 1.0 (no scaling performed).]

smmodel_chg Specify the total charge of the small model. [The default value of the charge will be assigned automatically by the program, which is not guaranteed to be right. Careful check is suggested from running the Gaussian/GAMESS-US program. If it is not right, you can add this option with right charge into the MCPB.py input file and regenerate the modeling files.]

smmodel_spin Specify the spin of the small model. [The default value of the spin will be assigned automatically by the program as 1 or 2, based on the number of electrons. This is not guaranteed to be right. Careful check is suggested from running the Gaussian/GAMESS-US program. If it is not right, you can add this option with right spin into the MCPB.py input file and regenerate the modeling files.]

software_version The version of software the user used to perform the QM calculations. Five options are available, g03 (which represents Gaussian03), g09 (which represents Gaussian09), g16 (which represents Gaussian16), gau (which represents Gaussian), and gms (which represents GAMESS-US). In the current version of MCPB.py, all the three Gaussian versions (g03, g09, and g16) are equally supported, all of them are equal to option gau, with they are kept for backward compatibility. [The default is gau.]

sqm_opt A variable used to indicate the use of SQM in AmberTools to do a simulation of the sidechain and/or large model before using Gaussian to perform the calculation. *Please note:* if 1, 2 or 3 are chosen, the first step of the modeling process will take additional time (minutes for the sidechain model and hours for the large model). [The default is 0.]

- 0 – means no use of SQM.
- 1 – means the optimization is done only for the sidechain model.
- 2 – means the optimization is done only for the large model.
- 3 – means the optimization is done for both the sidechain and large models.

water_model The user-designated water model to be used in the molecular modeling. Options are TIP3P, SPCE, TIP4PEW, OPC3, OPC, FB3, and FB4. Where FB3 and FB4 represent the TIP3P-FB and TIP4P-FB water models, respectively. [The default is OPC.]

xstru Specify whether the structure in the original PDB file is used to generate the equilibrium bond distances and angle values in the frmod file. 0 means not using, but use the QM optimized structure. 1 means using. [The default is 0.]

The following is an explanation of the step number variables:

Here are the options for the step_number:

For step1 there are three options: 1a (default, same as specifying 1), 1m and 1n.

For step2 there are five options: 2b, 2e, 2s (default, same as specifying 2), 2ms, and 2z.

For step3 there are five options: 3a, 3b (default, same as specifying 3), 3c, 3d, and 3f.

For step4 there are three options: 4b (default, same as specifying 4), 4n1 and 4n2.

The following is the detailed explanation of the steps used in the modeling procedure:

Step1. Used to generate the modeling files (e.g. PDB, fingerprint and Gaussian input files) for different models (e.g. sidechain, standard and large models). Three options are available and their explanation is shown below. Default is 1a.

- 1a – Used to automatically rename the atom types of the center metal ions and the surrounding bonded atoms in the standard fingerprint file.

19. Python Metal Site Modeling Toolbox (pyMSMT)

- 1m – Used to automatically rename only the atom type(s) of the center metal ion(s) to the AMBER atomic ion atom type style in the standard fingerprint file.
- 1n – Used to generate the standard fingerprint file without renaming the atom types. Users can rename the atom type of the metal ion(s) and its ligating atoms manually in the standard fingerprint file.

Please note: Between using Step1 and Step2, the Gaussian calculations (if needed), should be done for the sidechain model (to calculate the force constants) and the large model (to do the RESP charge calculation) using Gaussian input files. Prior to the calculation, users can change the parameters (such as the calculation method, basis set, etc.) in the Gaussian input files according to their own preferences. After finishing this procedure, the user can move on to Step2.

Step2. Used to generate the frcmod file for the modeling. In this step, a frcmod file (with pre.frcmod name at the end of the file name), will be pre-generated. This file includes all the parameters, except the bond and angle parameters related to the metal ions. Later, the final frcmod file will be generated which will include all the parameters. There are three methods to choose from: Empirical, Seminario and Z-matrix. Each of these methods generates the metal ion-related bond and angle parameters. If you don't have a QM optimized structure, you can also generate a frcmod file with metal related bond and angle parameters as zero (see step 2b) and then manually modify it later for further usage. Default is the 2s (Seminario method).

- 2b - The "blank" methodGenerate a frcmod files with metal related bonds and angles have zero as the equilibrium values and force constants. If use with option "xstru 1" in the MCPB.py input file, it will generate the equilibrium values based on the original PDB structure and force constants as zero. User can modify the generated frcmod file by manually assigned bond and angle parameters for further usage.
- 2e – The Empirical method,[466]can generate the metal ion-related bond and angle parameters efficiently without doing Gaussian calculations. It only supports Zn2+ ion in the current version.
- 2s – The Seminario method[467] generates the force field parameters based on sub-matrices of the Cartesian Hessian matrix obtained from quantum calculations. This method requires a Gaussian fchk file (which can be generated from a chk file by using the formchk command in Gaussian).*Reminder:* both the geometry optimization and force constant calculation procedures are needed to generate the final chk file and subsequent fchk file for the force constant calculations done by the Seminario method.
- 2ms – The modified Seminario method[468] generates the force field parameters in the same way as the Seminario method except that it generates the angle force constants in a modified way to solve a (potential) double-counting issue. This method requires a Gaussian fchk file (which can be generated from a chk file by using the formchk command in Gaussian).*Reminder:* both the geometry optimization and force constant calculation procedures are needed to generate the final chk file and subsequent fchk file for the force constant calculations done by the modified Seminario method.
- 2z – The Z-matrix method generates the force field parameters by using the Cartesian Hessian matrix obtained from the quantum calculations. This method requires the force constant Gaussian output file (usually named as a log file) after the geometry optimization and force constant calculations.

Step3. Used to perform charge parameterization and to generate the mol2 files for the residues within the metal ion complex. There are several fitting schemes available in this step. The five options are shown below. Options 3a-3d are for RESP charge fitting, while option 3f uses a fluctuating charge model for complexes containing 3d metals developed by Landry and Li.[469, 470] The default is 3b since it was identified as the best combination among 3a-3d in the work of Peters et al.[462]*Reminder:* the chgfix_resids variable is effective in this procedure, if the variable is specified, the charge restriction will be used as well as one of the following choices.

- 3a – Allows all the charges of the atoms in the ligating residues to change without any restrictions.
- 3b – Restrains the charges of the heavy backbone atoms in the ligating residues according to the user-chosen force field.

- 3c – Restrains the charges of the backbone atoms (both heavy and hydrogen atoms) in the ligating residues according to the user-chosen force field.
- 3d – Restrains the charges of the backbone atoms (both heavy and hydrogen atoms) and C beta atoms in the ligating residues according to the user-chosen force field.
- 3f – Using a fluctuating charge model[469, 470] to determine the charges of metal site residues, during which restrains the charges of the backbone N, H, C, and O atoms in the ligating residues according to the user-chosen force field. This method only supports 3d metal-containing metalloproteins (which contains Cr, Mn, Fe, Co, Ni, and Zn specifically) now.

Step4. Generates the LEaP input file. The default is 4b.

- 4b – Generates the LEaP input file for the bonded model.
- 4n1 – Generates the LEaP input file for the nonbonded model and refits the charge of the ligating residues.
- 4n2 – Generates the LEaP input file for the nonbonded model without refitting the charge of the ligating residues.

Here are some suggestions for the parameterization procedure:

1) For the modeling of the bonded model, the following steps are usually needed (4 steps):
1a/1n→2e/2s/2z→3a/3b/3c/3d→4b

2) For the modeling of a non-bonded model with a refitted charge, users can follow the workflow (3 steps):
1m→3a/3b/3c/3d→4n1

3) For modeling with a normal nonbonded model (without fitting any charges), users usually only need one step to perform the modeling (1 step): 4n2.

The following is an explanation of the logf and fchk variables:

These variables are optional. If provided, they are only active in step2 and/or step3. The default log file name is group_name + '_sidechain.fc.log' for step2 and group_name + '_large.mk.log' for step3. The default fchk file name is group_name + '_sidechain_opt.fchk' and it is only active for step2 when using Gaussian software and Seminario method to obtain the force constant parameters.

If you are using Gaussian software and Seminario method to generate force constant parameters, it uses the fchk file of sidechain model to store the Cartesian Hessian matrix. If you are using Gaussian software and Z-matrix method to generate force constant parameters, it uses the log file of sidechain model to store the force constant parameters. While if you are using GAMESS-US software and Seminario method to generate the force constants, it uses the log file of sidechain model to store the Cartesian Hessian matrix. In current version the software doesn't support GAMESS-US with Z-matrix method to generate the force constants.

Both the Gaussian and GAMESS-US software use the log file of large model to store the ESP charges.

19.2.2. IPMach.py

```
Usage: IPMach.py -i inputfile
Options:
  -h, --help          show this help message and exit
  -i INPUTF           Input file name
```

The following is an introduction of the variables in the input_file:

(Reminder: there should be no blank lines in the input_file. The values or parameters should follow the variables separated by a space.)

Required variables:

resname Residue name of the ion for parameterization (e.g. NA).

atname Atom name of the ion for parameterization (e.g. NA).

element Element of the ion for parameterization (e.g. Na).

atype Atom type of the ion for parameterization (e.g. Na+).

charge Charge of the ion for parameterization (e.g. 1).

Optional variables:

cpus Number of cpus to be used during the parameterization (e.g. 2). There are at least 2 cpus needed to perform TI simulation using sander program. While 1 cpu could also be used to perform TI simulation using pmemd program. [The default is 2.]

gpus Number of gpus to be used during the parameterization. It should be 0 or 1. If it equals 1, the pmemd.cuda program will be used. [The default is 0.]

timesteps Number of steps used during the thermodynamic integration simulation. Two options are available: 1 or 2. 1 means use the "one-step" method while 2 means use the "two-step" method. The "two-step" method generally needs longer simulation time but gives better accuracy. [The default is 2.]

ti_windows Number of windows used for the "one-step" TI simulation. Which is not effective when using the "two-step" TI simulation method. [The default is 7.]

vdw_windows Number of windows used for the VDW scaling step in the "two-step" simulation method. Which is not effective when using the "one-step" TI simulation method. [The default is 3.]

chg_windows Number of windows used for the charge scaling step in the "two-step" simulation method. Which is not effective when using the "one-step" TI simulation method. [The default is 7.]

rev Whether reverse TI simulation performed. There are two options available: 0 or 1. 0 means no, 1 means yes. Performing reverse TI simulation could double the simulation time but offer more valid results. [The default is 1.]

rmin VDW radius parameter (unit is Angstrom) used for the initial guessing. [The default is 1.5.]

hfe Target hydration free energy value (unit is kcal/mol) used for the parameterization. [The default is -100.0.]

hfe_tol Tolerance of target hydration free energy value (unit is kcal/mol) during the parameterization. [The default is 1.0]

iod Target ion-oxygen distance of first solvation shell (unit is Angstrom) used for the parameterization. [The default is 2.0.]

iod_tol Tolerance of target ion-oxygen distance of first solvation shell (unit is Angstrom) during the parameterization. [The default is 0.01.]

cal_type Type of the calculation: whether it is a optimization (OPT), or single point calculation (SP). [The default is OPT.]

set Parameter set to generate. Three options are available: HFE, IOD, or 1264. The HFE parameter set will treat hydration free energy as target during the parameterization. The IOD parameter set will treat ion-oxygen distance of first solvation shell as target during the parameterization. The variable should be set to 1264 for the 12-6-4 calculations. Otherwise it will be set to 1264 automatically if the c4v variable is not equal to 0.0. [The default is HFE.]

maxiter Maximum iteration steps during the parameter optimization. [The default is 100.]

mode There are three modes available for running the program: test, scan, normal. These three modes will increase the simulation time one-by-one. [The default is normal]

program The MD program used during the parameterization. Two options: sander or pmemd. Reminder: pmemd.cuda can perform TI simulations for the 12-6-4 LJ-type model starting from Amber18. However, in the current version of release, IPMach.py does not support TI simulations with the 12-6-4 LJ-type model using the pmemd program (neither pmemd, pmemd.MPI, nor pmemd.cuda). [The default is sander.]

watermodel Water model used during the parameterization. The ion parameters may vary for different water models. It is suggested to parameterize a ion model for a specific water model. Ten options: tip3p, tip4p, tip4pew, tip5p, spc, spce, opc3, opc, fb3, and fb4. [The default is tip3p.]

c4 Initial C4 values between ion and oxygen in water for parameterization of the 12-6-4 model for ions. Unit is kcal/mol * Angstrom⁴. The input value will be kept to 1 decimal place. [The default is 0.0, means only 12-6 parameters with no C4 parameters applied.]

distance The distance variable for the solvateBox command in LEaP when creating the periodic solvent box around the metal ion. Unit is Angstrom. The input value will be kept to 1 decimal place. [The default is 13.0.]

19.2.3. PdbSearcher.py

```
Usage: PdbSearcher.py -i/--ion ionname -l/--list input_file
                        -e/--env environment_file -s/--sum summary_file
                        [-c/--cut cutoff]
```

Options:

-h, --help	show this help message and exit
-i IONNAME, --ion=IONNAME	Element symbol of ion, e.g. Zn
-l INPUTF, --list=INPUTF	List file name, list file contains one PDB file name per line
-e ENVRMTF, --env=ENVRMTF	Environment file name. An environment file is used to store the metal center environment information such as ligating atoms, distance, geometry etc. For each bond, there is a record.
-s SUMF, --sum=SUMF	Summary file name. A summary file is used to store the metal center summary information such as metal center geometry, ligating residues etc. For each metal center there is a record.
-c CUTOFF, --cut=CUTOFF	Optional. The cut off value used to detect the bond between metal ion and ligating atoms. The unit is Angstroms. If there is no value specified, the default algorithm will be used. The default algorithm recognizes the bond when its distance is no less than 0.1 (smaller than 0.1 usually indicates a low quality structure) and no bigger than the covalent radius sum of the two atoms with a tolerance of 0.4.

19.2.4. OptC4.py

```
Usage: OptC4.py -m amber_mask -p topology_file -c coordinate_file -r restart_file
               [--maxsteps maxsteps] [--phase simulation_phase]
               [--size optimization_step_size] [--method optimization_method]
               [--platform device_platform] [--model metal_complex_model]
```

Options:

-h, --help	show this help message and exit
------------	---------------------------------

```

-m ION_MASK           Amber mask of the center metal ion
-p PFILE             Topology file
-c CFILE             Coordinate file
-r RFILE             Restart file
--maxsteps=MAXSTEPS  Maximum minimization steps performed by OpenMM
                    in each parameter optimization cycle.
                    [Default: 1000]
--phase=SIMUPHA      Simulation phase, either gas or liquid.
                    [Default: gas]
--size=STEPSIZE      Step size chosen by the user for the C4 value
                    during parameter searching. [Default: 10.0]
--method=MINM        Optimization method of the C4 terms, The options
                    are: powell, cg, bfgs or slsqp. [Default: bfgs]
                    Please check the website:
                    http://docs.scipy.org/doc/scipy/reference/optimize.html#m
                    for more information if interested.
--platform=PLATF     Platform used. The options are: reference, cpu,
                    gpu or opencl. [Default: cpu] Here we use the
                    OpenMM software to perform the structure
                    minimization. Please check OpenMM user guide for
                    more information if interested.
--presn=PRESN        Precision used. The options are: single, mixed, or
                    double. This option is only valid when using the CUDA
                    or OpenCL platform. [Default: single]
--model=MODEL        The metal ion complex model chosen to calculate the sum
                    of unsigned average errors of bond lengths, angles, and
                    dihedrals (the units of them are angstrom, degree and
                    degree respectively while the weights of them are
                    1/100, 1/2 and 1 respectively). This sum is the
                    criterion for the optimization (with a smaller value,
                    the better the parameters). The options are: 1 or 2.
                    1 means a small model (only contains the metal ion
                    and binding heavy atoms) while 2 means a big model
                    (contains the metal ion and heavy atoms in the
                    ligating residues). [Default: 1]

```

19.2.5. CartHess2FC.py

```

Usage: CartHess2FC.py -p PDB_file -f QM_output_file [-v software] [-m method]
                    [--scalef freq_scale_factor] [--nstpdb]
                    [--bavg] [--avg13] [--aavg]

```

Options:

```

-h, --help           show this help message and exit
-i INPUTFILE         Input PDB file name
-f HESSF             Quantum output file name (a fchk/log file for Gaussian or a file fo
                    log file for GAMESS-US).
-v SOFTV             Software version [Default is gau (means Gaussian). Other
                    options are g03 (means Gaussian03), g09 (means Gaussian09),
                    g16 (means Gaussian16), and gms (means GAMESS-US)]. The
                    options g03, g09, and g16 are all equal to gau but kept for
                    backward compatibility.
-m METHOD             Method used. [Default is sem (means Seminario, applicable

```

```

to g03, g09, gl6, gau, and gms). Other options are msem
(means modified Seminario, applicable to g03, g09, gl6,
gau, and gms) and zmx (means Z-matrix, only applicable to
g03, g09, gl6, and gau.)]
--scalef=SCALEF  Scale factor (ATTENTION: This is the scale factor of
frequency. The force constants will be scaled by
multiplying the square of scale_factor).
--nstdpdb        Non standard PDB file used. It is the PDB file which have
all the atom names as element followed by interger number.
It could be a PDB file generated by software such as
antechamber based on the Gaussian output file.
--bavg          Make average of bond force constants based on different ways
of choosing sub Hessian matrices using Seminario method.
--avg13        Make average of Urey-Bradley force constants based on
different ways of choosing sub Hessian matrices using
Seminario method.
--aavg         Make average of angle force constants based on different ways
of choosing sub Hessian matrices using Seminario method.

```

CartHess2FF.py is designed to generate force field parameters for bond, angle, Urey-Bradley (1-3 interaction), dihedral, and improper torsion terms based on quantum calculated Cartesian Hessian matrix. These terms could be used separately in force fields such as AMBER, CHARMM, CNS (Crystallography and NMR System), etc. while limits may be applicable for the dihedral and improper terms (see below).

$$\begin{aligned}
 E_{total} = & \sum_{bonds} k_b(r - r_0)^2 \\
 & + \sum_{angles} k_\theta(\theta - \theta_0)^2 \\
 & + \sum_{dihedrals} k_\phi(\phi - \phi_0)^2
 \end{aligned}
 \tag{19.1}$$

CartHess2FF.py could generate the force field parameters for the potential shown in Eq19.1 based on the log file of force constant calculation using Gaussian software and Z-matrix method. Here the dihedral term uses a harmonic potential other than a Fourier expansion. Users may try to transfer the parameters to a Fourier term based on the relationship: $V_n = 2k_\phi/n^2$ (while V_n and n are from Eq15.1) while there is no guarantee for working. This is due to 1-4 nonbonded interaction is usually coupled to dihedral potential in AMBER while it is not considered current potential formulation. Meanwhile, there is also other issues (such as the connectivities of the central two atoms) available which limits the transferability of these dihedral parameters. Therefore only qualitatively comparison between different dihedral angles (while the QM calculations should be carried out under same level of theory) are suggested by the author. The Gaussian calculation could be performed with `iop(7/33=1)`, before which the structural optimization at the same level of theory is needed.

$$\begin{aligned}
E_{total} = & \sum_{bonds} k_b (r - r_0)^2 \\
& + \sum_{\text{Urey-Bradley}} k_u (u - u_0)^2 \\
& + \sum_{angles} k_\theta (\theta - \theta_0)^2 \\
& + \sum_{dihedrals} k_\phi (\phi - \phi_0)^2 \\
& + \sum_{impropers} k_{AN} (r_{AN} - r_{AN,0})^2
\end{aligned}
\tag{19.2}$$

Moreover, CartHess2FF.py could generate the force field parameters for the potential shown in Eq19.2 based on the Cartesian Hessian matrix obtained by using Gaussian or GAMESS-US. Comparing to Eq19.1 it has an additional Urey-Bradley (1-3 harmonic interaction) term and a harmonic improper torsion term (instead of a Fourier term). In the harmonic improper term A and N represent the central atom and its projection into the plane of the other three atoms in the improper torsion. Herein k_{AN} , r_{AN} , and $r_{AN,0}$ represent the force constant, distance and equilibrium distance between A and N respectively. Similarly, parameters of the dihedral and improper torsion terms have limited transferability and are only suggested to be used for qualitative comparison. A fchk file is needed for Gaussian, and it could be obtained by using the "formchk" command on the chk file after the force constant calculation (again, before which a structural optimization at the same level of theory is needed). A log file is needed for GAMESS-US force constant calculation (same as Gaussian, a structural optimization at the same level of theory is needed before the calculation).

19.2.6. espngen.py

```
Usage: espngen.py -i input_file -o output_file [-v software]
Options:
  -h, --help          show this help message and exit
  -i INPUTFILE        Input file name
  -o OUTPUTFILE       Output file name
  -v SOFTVERSION      Software version [Default is gau (means Gaussian),
                      other option is gms (means GAMESS-US)]
```

19.2.7. ProScrs.py

```
Usage: ProScrs.py -i input_file -p PDB_file
Options:
  -h, --help          show this help message and exit
  -i INPUTFILE        Input file name
  -p PDBFILE          PDB file name
  -s PRE              File name prefix (default: MOL)
  -c CHG              Charge (default: 0)
  --symcrd=SYMCRD     Use symbolic Cartesian coordinates (default: 0)
  --fix=FIX           Fix heavy atoms or not (default: 0): 0 means no, 1 means
                      only backbone N, CA, C, and O atoms, 2 means backbone N,
                      CA, C, O, and sidechain beta atoms, 3 means all heavy
                      atoms.
  --crd0=CRD0        Reassign the coordinates with first atom as 0 (default: 0)
```

Reminder: For the following functions, some X-H distances will be adjusted based on the normal X-H distances

for the generated Gaussian input files (here X represents a heavy atom).

- ace** Specify the residue number for which residue you want to treat as ACE (CH₃CO). ProScrs.py will keep the backbone CA, HA, C, O atoms, and change beta atom in the sidechain and backbone N atoms into hydrogen atoms while omit all the atoms in the residue.
- act** Specify the residue number for which residue you want to treat as the CH₃CO₂⁻ group. This is specific for the C-terminal residue which has backbone O atom coordinated to another atom (e.g. a metal ion). ProScrs.py will keep the backbone CA, HA, C, O, OXT atoms, and change beta atom in the sidechain and backbone N atoms into hydrogen atoms while omit all the atoms in the residue.
- nme** Specify the residue number for which residue you want to treat as NME (CH₃NH). ProScrs.py will keep the backbone CA, HA, N, H atoms, and change beta atom in the sidechain and backbone C atoms into hydrogen atoms while omit all the atoms in the residue.
- ant** Similar to nme variable except keep all the backbone N atom and H atoms connected to it. This can be a N-terminal residue which has backbone N atom coordinated to another atom (e.g. a metal ion). ProScrs.py will keep the backbone CA, HA, N, H1, H2, H3 atoms, and change beta atom in the sidechain and backbone N atoms into hydrogen atoms while omit all the atoms in the residue.
- gly** Specify the residue number for which residue you want to treat as the GLY residue. This can be used for the situation that backbone atoms of the residue matters most while sidechain doesn't involve a lot. ProScrs.py will keep the backbone N, H, CA, HA, C, O atoms and change beta atom in the sidechain into a hydrogen atom while omit all the atoms in the residue.
- keep** Specify the residue number for which residue you want to keep entirely. This can be any residue (such as a metal, water, ligand, or amino acid).
- sc** Specify the residue number for which residue you only want to keep the sidechain. ProScrs.py will keep the sidechain and backbone CA, HA atoms and change the backbone N and C atoms into hydrogen atoms while omitting all the other backbone atoms.
- sc_knh** Specify the residue number for which residue you want to keep the sidechain and backbone NH group. ProScrs.py will keep the sidechain and backbone CA, HA, N, H atoms and change the backbone C atom into a hydrogen atom while omitting the O backbone atom.
- sc_kco** Specify the residue number for which residue you want to keep the sidechain and backbone CO group. ProScrs.py will keep the sidechain and backbone CA, HA, C, O atoms and change the backbone N atom into a hydrogen atom while omitting the H backbone atom.
- c2h** Specify the residue number for which residue you only want to keep the backbone C atom and change it to a hydrogen atom. This is used to have a hydrogen cap for next residue connecting to it.
- n2h** Specify the residue number for which residue you only want to keep the backbone N atom and change it to a hydrogen atom. This is used to have a hydrogen cap for former residue connecting to it.

19.2.8. car_to_files.py

```
Usage: car_to_files.py -i input_file -m mol2_file -p pdb_file -r residue_name
Options:
  -h, --help          show this help message and exit
  -i INPUT_FILE       Input file name
  -m MOL2_FILE        Output mol2 file name
  -p PDB_FILE         Output PDB file name
  -r RESNAME          Residue name
```

19.2.9. mol2rtf.py

```
Usage: mol2rtf.py -i mol2_file -o rtf_file -r residue_name
                    -n new_resname [--ref reference_rtf_file]
```

Options:

```
-h, --help          show this help message and exit
-i INPUT_FILE       Input mol2 file
-o OUTPUT_FILE      Output RTF file
-r RESNAME          Original residue name
-n NEW_RESNAME      New residue name
--ref=REF_RTF       Reference RTF file
```

The --ref option is only needed when one wants to create a RTF file for an amino acid residue which has the residue name different from the standard residue name used in the CHARMM force field. For this case, one needs to specify the reference RTF file as the RTF file for the ff14SB force field in the CHARMM software package. Which can be found as "/toppar/non_charmm/parm14sb.all.rtf" in the CHARMM software package. The metal site residues are renamed to different names in the workflow of MCPB.py (e.g. HID->HD1). For example, for the situation of HID->HD1, mol2rtf.py will generate a RTF file for HD1 based on the HID residue in the reference file. During this procedure, the -r option should be set as HID, while the -n option should be set as HD1.

19.2.10. metalpdb2mol2.py

```
Usage: metalpdb2mol2.py -i pdb_file -o mol2_file -c charge
```

Options:

```
-h, --help          show this help message and exit
-i INPUT_FILE       Input PDB file
-o OUTPUT_FILE      Output mol2 file
-c CHARGE           Charge of the metal ion
```

The program will convert a PDB file which contains an metal ion to a mol2 file. The PDB file should only have one single metal ion in it. Users need to specify the charge of the metal ion using the -c option. This program was developed specifically for the MCPB.py users to convert the metal ion PDB file to a mol2 file.

20. Electrostatic Parameterization with `py_resp.py`

Shiji Zhao and Qiang Zhu

`py_resp.py` is a Python program extending the functionalities of the ancestor program `resp`.^[471] The RESP (Restrained ElectroStatic Potential) algorithm fits the quantum mechanically calculated molecular electrostatic potential (ESP) at molecular surfaces using an atom-centered point charge model. This method was developed primarily by Bayly.^[454, 472] The RESP method is compatible with the additive Amber force fields, such as `ff19SB`,^[23] `ff14SB`,^[25] and `gaff2`.^[447] However, the additive force fields are unable to model the important atomic polarization effects. Several induced dipole based polarizable force fields have been incorporated into Amber, such as `ff02`,^[121] `ff12pol`,^[473–476] as well as the polarizable Gaussian Multipole (pGM) force field described in 22.8 that is still under active development.^[477–480] `py_resp.py` provides parameterization schemes for several electrostatic models, including the RESP model with atomic charges for the additive force fields, and the RESP-ind and RESP-perm/RESP-perm-v models with additional induced and permanent dipole moments for the polarizable force fields.^[471] `py_resp.py` has implemented all key features of the `resp` program, so we encourage current and future users of `resp` switch their workflow to `py_resp.py`.

20.1. `pyresp_gen.py`

`pyresp_gen.py` is a Python program developed for automatically generating input files for `resp` or `py_resp.py` programs. This program is easy to use and simplifies the process for electrostatic parameterization, especially for the RESP-perm/RESP-perm-v models provided by `py_resp.py` which requires the equivalence information for permanent dipole moments. Below is the usage information:

```
usage: pyresp_gen.py [-h] -i input
                  [-f1 stage1] [-f2 stage2] [-p ptype] [-d dtype] [-n nmol] [-q charge]
                  [-qwt1 qwt1] [-qwt2 qwt2] [-pwt1 pwt1] [-pwt2 pwt2]
                  [-excl2 excl2] [-excl3 excl3] [-depth depth] [-v verbose]
                  [-strategy strategy]

Options:
  -h      Show help message and exit.
  -i      Input ESP data file generated by ESPGEN.
  -f1     Output file for 1st stage PyRESP fitting, default: pyrespgen.1st.
  -f2     Output file for 2nd stage PyRESP fitting, default: pyrespgen.2nd.
  -p      Polarization type: chg, ind, perm. default: perm.
  -d      Damping function type: (1) applequist, (2) tinker, (3) exponential, (4) linear,
         (5) pGM. default: 5.
  -n      Number of molecules or conformations, default: 1. Note: For the current
         release, only one molecule is supported.
  -q      Total molecular charge, default: 0.
  -qwt1   Charge restraint weight of 1st stage, default: 0.0005 (borrowed from RESP).
  -qwt2   Charge restraint weight of 2nd stage, default: 0.001 (borrowed from RESP).
  -pwt1   Permanent dipole restraint weight of 1st stage, recommended: 0.0001.
  -pwt2   Permanent dipole restraint weight of 2nd stage, recommended: 0.0005.
  -excl2  include (0) or exclude (1) 1-2 interactions, default: 0.
  -excl3  include (0) or exclude (1) 1-3 interactions, default: 0.
  -depth  Maximum depth for searching equivalent atoms, default: 3.
```

```
-v Print verbose information or not, default: 0.
-strategy Choose Strategy for pGM-perm, default: 2 (only for test purpose).
```

20.1.1. pyresp_gen.py Example

The following commands are examples showing how to utilize *pyresp_gen.py*.

```
pyresp_gen.py -i CH4.dat
pyresp_gen.py -i CH4.dat -f1 CH4_chg.1st -f2 CH4_chg.2nd -p chg
pyresp_gen.py -i CH4.dat -f1 CH4_ind.1st -f2 CH4_ind.2nd -p ind
pyresp_gen.py -i CH4.dat -f1 CH4_perm.1st -f2 CH4_perm.2nd -p perm
pyresp_gen.py -i NH4+.dat -f1 CH4_perm.1st -f2 CH4_perm.2nd -p perm -q 1
pyresp_gen.py -i NH4+.dat -f1 CH4_perm.1st -f2 CH4_perm.2nd -p perm -q 1 -v 1
```

The first command just takes one argument *-i CH4.dat*, while other arguments are omitted and taking the default one. After running this command, two files are produced named as *pyresp_gen.1st* and *pyresp_gen.2nd*. The second to the fourth commands just illustrate how to specify the polarization type. The results from the second command are the same as the first command. The second and third command will generate the input files for RESP-ind and RESP-perm models, respectively, specifying the pGM damping function. The fifth command shows when feeding in charged molecule, you need to specify argument *-q* followed by molecular charge. The sixth command turns on the verbose information, you will see much detailed information printed on the screen. The *esp* data files and corresponding output files generated by *pyresp_gen.py* could be found in *\$AMBERHOME/AmberTools/examples/pyresp_gen*.

20.2. py_resp.py Usage

py_resp.py is a simple Python program with the following usage:

```
usage: py_resp.py [-O] -i input -o output [-q qin] [-ip polariz] -t qout -e espot
               [-s esout]
```

Options:

```
-O Overwrite output files if they exist.
-i input of general information.
-o output of results.
-q input of replacement parameters.
-ip input of atomic polarizabilities.
-t output of parameters.
-e input of ESP and coordinates.
-s output of ESP values for new parameters.
```

For the additive RESP model, the only two types of required input files for *py_resp.py* are the input file (*-i*) and the ESP data file (*-e*). For the polarizable RESP-ind and RESP-perm models, the atomic polarizability input file (*-ip*) is also required. The input file (*-i*) could be generated by the *pyresp_gen.py* (20.1) program. The ESP input file (*-e*) needs to be generated by a quantum mechanical program, such as *Gaussian*, *Jaguar*, *GAMESS*, or *Firefly*, in combination with the *espgen* program of the *antechamber* (16.3) program suite.

The format of the input file (*-i*):

-1st line-

```
TITLE      a character string
```

-2nd section-

Begin namelist with " &cntrl"

```

nmol      = the number of structure(s) for fitting (default 1)
            structure(s): orientation(s), conformation(s) or molecule(s)

iqopt     = 1  reset all initial parameters to zero (default)
            = 2  read in new initial parameters from -q unit

ihfree    = 0  all atoms are restrained
            = 1  hydrogens not restrained (default)

irstrnt   = 0  harmonic restraints (old style)
            = 1  hyperbolic restraint to parameters of zero (default)
            = 2  only analysis of input parameters; no parameterization is
                  carried out

qwt       = restraint weight for charges; default is 0.0005

ioutopt   = 0  normal run
            = 1  write restart info of new esp to -s unit (default)

ireornt   = 0  normal run (default)
            = 1  reorient molecule to standard reorientation in Gaussian
                  definition before calculating molecular dipole and
                  quadrupole moments

iquad     = 0  report molecular quadrupole moment in Buckingham definition
            = 1  report molecular quadrupole moment in Gaussian definition
                  (default)

ipol      = 0  additive RESP model; no atomic dipole calculations
            1  Applequist scheme without damping
            2  Tinker-exponential damping scheme
            3  exponential damping scheme
            4  linear damping scheme
            5  pGM damping scheme (default)

igdm      = 0  normal run
            = 1  use distributed pGM charges and dipoles in ESP fitting
                  only use with ipol = 5 (default)

exc12     = 0  include 1-2 interactions for electric field calculations
                  (default)
            = 1  exclude 1-2 interactions

exc13     = 0  include 1-3 interactions for electric field calculations
                  (default)
            = 1  exclude 1-3 interactions

ipermdip  = 0  RESP-ind model; do not calculate permanent dipole
            = 1  RESP-perm model; calculate permanent dipoles (default)

pwt       = restraint weight for permanent dipoles; default is 0.0005

```

```
virtual = 0 normal run (default)
        = 1 enable permanent dipoles for 1-3 virtual bonds
```

End namelist with " &end"

-3rd line-

```
wtmol    relative weight for the structure if multiple structure fit
         (1.0 otherwise)
```

-4th line-

```
subtitle for the structure (a character string)
```

-5th line-

```
charge   iuniq   (iuniq.p)
charge   = total charge for this structure (-99 if no total charge
         constraint)
iuniq    = total number of atoms for this structure
iuniq.p  = total number of permanent dipoles for this structure
```

-6th section-

```
one line for each atom:
element number = element number in periodic table
ivary          = control charge variations of each center
(ivary.p       = control permanent dipole variations of each center)
Note: The permanent dipoles of each atom are ordered with the atom
      number of reference atoms. If virtual = 1, real permanent
      dipoles come before all virtual dipoles for each atom.
```

```
ivary & ivary.p
      = 0 current parameter fitted independently of other centers
      = -1 current parameter frozen at initial value read from -q unit
      = n current parameter fitted and equivalenced to the center "n"
```

```
*blank to end only if the number of structure(s) "nmol" is greater than 1
```

-7th section-

```
intra-molecular charge constraint(s); blank line if no constraint
```

```
ngrp = the number of charge centers in the atom group of this constraint
grpchg = charge value to which the associated group of atoms (given on the
        next section) is to be constrained
```

-7.1th section-

```
imol, iatom (repeat if more than 8 centers)
the list ("ngrp" long) of the atoms to be constrained to the charge
specified on the previous line.
```

```
*blank to end
```

-8th section-

inter-molecular charge constraint(s)
 same format as intra-molecular charge constraint(s) - see the 7th & 7.1th sections

*blank to end

-9th section-

multiple structure atom charge equivalencing

ngrp = the number of charge centers in the group of atoms for equivalencing

-9.1th section-

imol iatom (repeat if more than 8 centers)
 the list ("ngrp" long) of the atoms to be equivalenced

*blank to end

-10th section-

multiple structure permanent dipole equivalencing

ngrp = the number of permanent dipole centers in the group of permanent dipoles for equivalencing

-10.1th section-

imol idip (repeat if more than 8 centers)
 the list ("ngrp" long) of the permanent dipoles to be equivalenced

*blank to end

The format of the ESP input file (-e):

-1st line-

natom nesp (total number of atoms & ESP points)

-2nd line up to natom+1 line-

atom coordinates X Y Z (in Bohrs) & element number & atom type
 Note: atom type can be generated by the espgen program with -p 1

-natom+2 line up to natom+2+nesp line-

ESP & coordinates
 espot X Y Z (in a.u. & Bohrs)

The format of the replacement parameters input file (-q):

(note: same format as that produced by -t unit)

%FLAG TITLE: a character string

subtitle for the structure

%FLAG ATOM CRD: I4,3E16.7

atm.no: atom number
 X, Y, Z: atom coordinates X Y Z

%FLAG ATOM CHRG: 2(I4,X7),I4,X2,E16.7

```
atm,no: atom number
element.no: element number in periodic table
ivary: charge variations
q(opt): optimized atomic charge
```

%FLAG PERM DIP LOCAL: 3(I4,X5),I4,X2,E16.7 (for ipermdip=1)

```
dip.no: dipole number
atm.no: atom number
ref.no: reference atom number
ivary: permanent dipole variations
p(opt): optimized permanent dipole in local frame
```

%FLAG PERM DIP GLOBAL: I4,3E16.7 (for ipermdip=1)

```
atm.no: atom number
X, Y, Z: optimized permanent dipole in X Y Z directions of global frame
```

%FLAG IND DIP GLOBAL: I4,3E16.7 (for ipol>0)

```
atm.no: atom number
X, Y, Z: induced dipole in X Y Z directions of global frame
```

The format of the atomic polarizabilities input file (-ip):

-1st line-

```
Some comments
```

-2st section-

```
atom type & atomic polarizability (a.u.) & damping factor (a.u.)
```

Note: For each of the provided damping schemes (Tinker-exponential, exponential, linear, and pGM), the damping factor has different meanings. See PyRESP publication for reference.[\[471\]](#)

```
end with a line starting with 'a' (unused)
```

-3rd section-

```
atom type equivalence information
EQ & list of atome types with same polarizabilities and damping factors
```

20.3. Examples for *py_resp.py*

Two examples are used below to demonstrate the usage of *py_resp.py*. As with any automated procedure, the output should be carefully examined, and users should pay attention to any unusual or incorrect program behavior. The input and output files of these and more examples can be found in *\$AMBERHOME/AmberTools/examples/PyRESP*. These should be consulted by those interested in testing the *py_resp.py* program.

20.3.1. Single conformation parameterization

In the first example let's parametrize a water molecule. Here are the input files (named as *wat.in*) for the four electrostatic models, with explanations of the parameter variation information in the 6th section of the input file. (See [20.2](#))

RESP

```

resp for water
&cntrl
  nmol = 1,
  iqopt = 1,
  ihfree = 1,
  qwt = 0.0005,
  ioutopt = 1,
  ipol = 0,
  ipermdip = 0
&end
1.0
water
  0    3
  8    0
  1    0
  1    2

```

RESP-ind

```

resp-ind for water
&cntrl
  nmol = 1,
  iqopt = 1,
  ihfree = 1,
  qwt = 0.0005,
  ioutopt = 1,
  ipol = 5,
  igdm = 1,
  exc12 = 0,
  exc13 = 0,
  ipermdip = 0,
&end
1.0
water
  0    3
  8    0
  1    0
  1    2

```

The RESP and RESP-ind models only require the variation information for charges. *ivary* = 0 indicates the current charge will be fitted independently, and this is true for the first (oxygen) and second (hydrogen) atoms. The third atom is a hydrogen equivalent to the second atom, so that its *ivary* is set to 2, indicating the two hydrogen atoms are equivalent and will be fit together.

RESP-perm

```

resp-perm for water
&cntrl
  nmol = 1,
  iqopt = 1,
  ihfree = 1,
  qwt = 0.0005,
  ioutopt = 1,
  ipol = 5,

```

```

    igdm = 1,
    exc12 = 0,
    exc13 = 0,
    ipermdip = 1,
    pwt = 0.0005,
    virtual = 0
&end
1.0
water
    0    3    4
    8    0    0    1
    1    0    0
    1    2    3

```

For the RESP-perm model, the variation information *ivary_p* for permanent dipoles (3+ columns) is also needed. The water molecule has 4 permanent dipoles in total, and the permanent dipoles of each atom are ordered with the atom number of reference atoms. The permanent dipoles from the oxygen to the two hydrogens are equivalent, so are the permanent dipoles from the two hydrogens to the oxygen. Therefore, we have *ivary_p* = 0 and 1 for the oxygen atom, and *ivary_p* = 0 and *ivary_p* = 3 for the permanent dipoles of the two hydrogen atoms.

RESP-perm-v

```

resp-perm-v for water
&cntrl
    nmol = 1,
    iqopt = 1,
    ihfree = 1,
    qwt = 0.0005,
    ioutopt = 1,
    ipol = 5,
    igdm = 1,
    exc12 = 0,
    exc13 = 0,
    ipermdip = 1,
    pwt = 0.0005,
    virtual = 1
&end
1.0
water
    0    3    6
    8    0    0    1
    1    0    0    0
    1    2    3    4

```

Compared with the RESP-perm model, water molecule has two extra virtual permanent dipoles between the two hydrogens for the RESP-perm-v model. In this model, the variation information of the real permanent dipoles appears before all virtual permanent dipoles for each atom. Therefore, for the hydrogen atoms, the permanent dipoles to the oxygen atom come before that to another hydrogen atom, and we have *ivary_p* = 0 and *ivary_p* = 4 for the virtual permanent dipoles of the two hydrogen atoms.

Next are the first several lines of the ESP input file, named as *esp_wat.dat*:

```

3   295   0
      -0.9982123E-32   0.0000000E+00   0.2313846E+00   8   ow
      -0.2610123E-32   0.1494187E+01  -0.9255383E+00   1   hw

```

```

-0.1829851E-15 -0.1494187E+01 -0.9255383E+00 1 hw
-0.4207115E-01 -0.9982123E-32 0.0000000E+00 0.3935248E+01
-0.4040255E-01 0.1851931E+01 0.0000000E+00 0.3439024E+01
-0.3659220E-01 0.9259657E+00 0.1603820E+01 0.3439024E+01
-0.3659220E-01 -0.9259657E+00 0.1603820E+01 0.3439024E+01
...

```

The first line shows this file contains the coordinates of 3 atoms, as well as the ESP value and coordinates of 295 points at the surface of the water molecule. In the section of atom information, the last column (gauss atom types) is required for the RESP-ind and RESP-perm/RESP-perm-v models.

With required input files prepared, the parameterization using *py_resp.py* could be performed using the following command. The unit -ip can be found in *\$AMBERHOME/AmberTools/examples/PyRESP/polarizability*,[\[481\]](#) and it is only required for the RESP-ind and RESP-perm/RESP-perm-v models.

```

py_resp.py -O -i wat.in -o wat.out -t wat.chg [-ip PGM-pol-2016-09-01] \
-s wat_calc.esp -e esp_wat.dat

```

The output files (-o) named as *wat.out*, the output parameter files (-t) named as *wat.chg*, and the output ESP files (-s) named as *wat_calc.esp* for each model can be found in corresponding subfolders under *\$AMBERHOME/AmberTools/examples/PyRESP/test/water*.

20.3.2. Double conformations charge fitting

Next, we show how to parameterize a *bis-(naphthyl-1-methyl) acetic acid* “super” molecule from two *2-methyl-3-naphthylpropionic acid* monomer molecules (*nmol* = 2) using the standard two-stage parameterizations.[\[454, 471, 472\]](#)

The input files of the two stages named as *bis_1.in* and *bis_2.in* for the RESP model can be found in *\$AMBERHOME/AmberTools/examples/PyRESP/test/bis-naphthyl/resp*. In the first stage, atom pairs (22, 23) are equivalenced to be fitted together. In the 8th section of the input file, (See [20.2](#)) intermolecular charge constraints (-1.000) are applied to atoms 19, 20, 21, 22, 23, 24, 25, 26, 27 of conformation 1, and atoms 20, 25, 26, 27 of conformation 2. These atoms will be removed in the final “super” molecule. In the 9th section of the input file, (See [20.2](#)) identical atoms between the two conformations are equivalenced. In the second stage, two control parameters are changed: (1) *iqopt* is set to 2, since the charges generated from the first stage will be used as initial charges. (2) *qwt* is set to 0.001, which is the recommended restraint weight. In addition, all atoms except atoms 18, 28 and 29 are set frozen. Atom pair (28, 29) are equivalenced to be fitted together. Note that there is no intermolecular charge constraint in this stage, and only the three unfrozen atoms need to be equivalenced between the two conformations.

The input files of the two stages named as *bis_1.in* and *bis_2.in* for the RESP-ind model can be found in *\$AMBERHOME/AmberTools/examples/PyRESP/test/bis-naphthyl/resp-ind*, which are identical to those of the RESP model except for the section of the control parameters.

The input files of the two stages named as *bis_1.in* and *bis_2.in* for the RESP-perm model can be found in *\$AMBERHOME/AmberTools/examples/PyRESP/test/bis-naphthyl/resp-perm*. There are 60 permanent dipoles (come from 30 covalent bonds) in the monomer molecule. Compared with the RESP and RESP-ind models, the charge variation information (2nd columns of 6th section, see [20.2](#)), intermolecular charge constraints (8th section, see [20.2](#)) and intermolecular charge equivalence (9th section, see [20.2](#)) stay the same. The variation information for permanent dipoles (3+ columns of 6th section, see [20.2](#)) and intermolecular permanent dipole equivalence (10th section, see [20.2](#)) are needed.

Next are several lines of the ESP input file, named as *bis_esp.dat*. Note that there is no line break between the ESP input information between the two conformations:

```

29 1250
-.3419687E+00 -.3824524E+00 -.2237947E+00 6 ca
-.3013712E+01 -.6673325E+00 -.1723714E+00 6 ca

```

```

                -.4165565E+01  -.3077458E+01  -.4524195E+00  6  ca
                -.6721855E+01  -.3333760E+01  -.4098062E+00  6  ca
                ...
                .7835869E+01  .2863310E+01  -.2994926E+01  1  hc
                .4242550E+01  .2648473E+01  .1954913E+01  1  hc
                .3940033E+01  .4124677E+01  -.9782360E+00  1  hc
    -.1457416E+00  -.3419687E+00  -.3824524E+00  .3744632E+01
    -.1619179E+00  .1642244E+01  -.3824524E+00  .3212963E+01
    -.1370663E+00  -.1334075E+01  -.2100831E+01  .3212963E+01
    -.1568089E+00  .6501378E+00  -.2100831E+01  .3212963E+01
    ...
    -.9008716E-01  .2956127E+01  .5828852E+01  -.5064440E+01
    -.9859980E-01  .3940033E+01  .4124677E+01  -.5513580E+01
29 1250
                -.3419687E+00  -.3824524E+00  -.2237947E+00  6  ca
                -.3013712E+01  -.6673325E+00  -.1723714E+00  6  ca
                -.4165565E+01  -.3077458E+01  -.4524195E+00  6  ca
                -.6721855E+01  -.3333760E+01  -.4098062E+00  6  ca
                ...
                .7835869E+01  .2863310E+01  -.2994926E+01  1  hc
                .4242550E+01  .2648473E+01  .1954913E+01  1  hc
                .3940033E+01  .4124677E+01  -.9782360E+00  1  hc
    -.1457416E+00  -.3419687E+00  -.3824524E+00  .3744632E+01
    -.1619179E+00  .1642244E+01  -.3824524E+00  .3212963E+01
    -.1370663E+00  -.1334075E+01  -.2100831E+01  .3212963E+01
    -.1568089E+00  .6501378E+00  -.2100831E+01  .3212963E+01
    ...
    ...
    ...
    ...

```

The first line shows this file contains the coordinates of 29 atoms, as well as the ESP value and coordinates of 1250 points at the surface of the molecule. The ESP input information of the second conformation is followed immediately after the end of the ESP input information of the first conformation.

With required input files prepared, the standard two-stage RESP charge fitting using *py_resp.py* could be performed using the following two commands (-ip is not needed for the RESP model):

```

py_resp.py -O -i bis_1.in -o bis_1.out [-ip pGM-pol-2016-09-01] \
-t bis_1.chg -s bis_1.calc.esp -e bis.esp.dat
py_resp.py -O -i bis_2.in -o bis_2.out [-ip pGM-pol-2016-09-01] \
-t bis_2.chg -s bis_2.calc.esp -e bis.esp.dat -q bis_1.chg

```

The output files (-o) named as *bis_1.out* and *bis_2.out*, the output parameter files (-t) named as *bis_1.chg* and *bis_2.chg*, and the output ESP files (-s) named as *bis_1.calc.esp* and *bis_2.calc.esp* for each model can be found in corresponding subfolders under \$AMBERHOME/AmberTools/examples/PyRESP/test/bis-naphthyl.

20.4. Large Scale Electrostatic Parametrization using PyPE_RESP

Marco Lapsien and Holger Gohlke

pype-resp.py is a pipeline that aims to facilitate the derivation of atomic partial charges by enabling easy setup of multi-conformer and multi-molecule RESP [454] fits.

```

usage: pype-resp.py [-ih] [-t] [-dir] [-i input] [-o output] [-ts]
[-eq [EQUIVALENCES ...]] [-c [CONSTRAINT ...]] [-def [MOL_DEF ...]]
[-m MOL_NAME] [-molgen] [-noresp] [-no2] [-pymol] [-ncpus NCPUS]
[-at {gaff,gaff2,sybyl,amber,bcc,abcg2}] [-p {chg,ind,perm}]

```



```

[-d {1,2,3,4,5}] [-ip poldat] [-depth DEPTH] [-exc12 {0,1}] [-exc13 {0,1}]
[-qwt1 QWT1] [-qwt2 QWT2] [-pwt1 PWT1] [-pwt2 PWT2]
Options:
  -ih          Display help for input file syntax and exit
  -t           Generate a template PyPE_RESP input file and exit
  -dir         Absolute or relative path to working directory
               (Default: Current directory)
  -i           Absolute or relative path to a PyPE_RESP input file
               (Overwrites all other command line settings)
  -o           Prefix for log and csv file created (Default: RESP_charge_summary)
  -ts         Carry out test run to check substructure matches of SMARTS patterns
  -eq         Group of atoms to be fitted equivalently.
               Can be supplied multiple times
  -c           Group of atoms to be part of a constraint group.
               Can be supplied multiple times
  -def         List of substrings corresponding to molecules
               that should be considered for the fit.
  -m           Three-letter Residue name to be inserted into all mol2 and mol3 files
               (Default: MOL)
  -molgen      Generate mol2 file and exit (Default: False)
  -noresp      Stop before RESP fitting (Default: False)
  -no2         Carry out RESP fit in only one stage instead of two (Default: False)
  -pymol.      Generate PyMOL session highlighting constrained atoms (experimental!)
  -ncpus       Number of cores to use for ESP file and PyRESP input generation.
               Does not influence fitting speed (Default: 1)
  -at          Atom type to implement in mol2 and mol3 files
               (Available: gaff2 (default), gaff, amber, bcc, abcg2)
  -p           Charge model to use during the RESP fit
               (Available: chg (default), ind, perm)
  -d           Damping scheme (integer) to use (Available: 1 (Applequist), 2 (tinker),
               3 (exponential), 4 (linear), 5 (pgm). Default is 5
               (Only needed, when ind or perm charge models are used)
  -ip          Absolute or relative path to file containing atomic polarizabilities
               (Only needed, when ind or perm charge models are used)
  -depth       Maximum depth (int) for searching equivalent atoms (Default: 3)
  -exc12       Include (False) or exclude (True) 1-2 interactions
               for electric field calculations (Default: False)
               (Only needed, when ind or perm charge models are used)
  -exc13       Include (False) or exclude (True) 1-3 interactions
               for electric field calculations (Default: False)
               (Only needed, when ind or perm charge models are used)
  -qwt1        Charge restraint weight (float) for the first-stage RESP fit.
               (Default: 0.0005)
  -qwt2        Charge restraint weight (float) for the second-stage RESP fit
               (Default: 0.001)
  -pwt1        Dipole restraint weight (float) for the first-stage RESP fit
               (Default: 0.0001)
  -pwt2        Dipole restraint weight (float) for the second-stage RESP fit
               (Default: 0.0005)

```

It utilizes *antechamber* (16.1.1) and the *RDKit* [482] Python Package to generate and identify conformers based on the InChi-Code of the respective molecule. Furthermore *pype-resp.py* calls *espgen* (19.2.6) to extract ESP information from *Gaussian* [373] output files, *pyresp-gen.py* (20.1) to generate input files for *py-resp.py* (20.2) and carry out subsequent RESP fitting including all functionalities of *py-resp.py*. A detailed description of the *pype-resp.py* workflow and its features can be found in:

- "PyPE.RESP: A tool to facilitate and standardize derivation of RESP charges" M. Lapsien, M. Bonus, L. Gahan, A. Raguin, H. Gohlke **bioRxiv** 2025.2001.2007.631713. <https://doi.org/10.1101/2025.01.07.631713>

[483].

20.4.1. General Usage

The simplest use case for *pype-resp.py* is the derivation of RESP charges for an ensemble of conformers of one or multiple molecules without applying charge constraints apart from adhering to the total charge of each molecule. This can be achieved by the following command:

```
pype-resp.py -dir $WRKDIR
```

where “\$WRKDIR” corresponds to the absolute or relative path to the directory containing each of the *Gaussian* output files.

By default, *pype-resp.py* will be able to distinguish between different conformers and molecules and will write atom equivalency statements to the *py_resp.py* input files accordingly. However, *pype-resp.py* also allows the definition of charge constraints using the `-c/--constraint` flag, and additional atom equivalency statements using the `-eq/--equate` flag of the command line interface. Both flags accept similar structured arguments of the form “MOL_IDENTIFIER : ATOM_IDENTIFIER”, where MOL_IDENTIFIER can be:

- an unambiguous substring that matches the filename of the *Gaussian* output files of the conformer/molecule the statement should be applied to. E.g. if a file is named “Example_molecule1.gout”, suitable identifiers could be “Example”, “molecule”, or even “ampl”, assuming that the identifier does not match any other filename of a different molecule in the working directory.
- an integer (1-based indexing) corresponding to the internal indexing of molecules in ascending alphabetical order. Assuming three *Gaussian* output files of three different molecules that are called “Example_molecule.gout”, “Trial_molecule.gout”, and “Test_molecule.gout”, their corresponding integer identifiers would be 1, 3, and 2 respectively.

whereas ATOM_IDENTIFIER can be:

- an unambiguous SMARTS pattern matching the atoms the statement should be applied to (see 20.4.2 for details on the usage of SMARTS patterns in *pype-resp.py*), e.g. “C (H) =O”
- a space-separated list of atom indices (1-based indexing) corresponding to the atoms the statement should be applied to, e.g. “1 2 27 15”

An arbitrary example command to equate atoms 2 and 3 of MOL_1 and atom 17 of MOL_2, while defining atoms 5, 6 and 7 of MOL_1 as an **intramolecular** constraint group and atoms 10, 12 and 14 of MOL_1 with atoms 24, 15 and 29 of MOL_2 as an **intermolecular** charge constraint group is:

```
pype-resp.py -dir . -c "MOL_1:5 6 7" -c "MOL_1:10 12 14" "MOL_2:24 15 29" \  
-eq "MOL_1:2" "MOL_1:3" "MOL_2:17"
```

(Note: \$WRKDIR has been swapped with “.”, which corresponds to executing *pype-resp.py* in the current directory).

This command highlights that the `-c` flag can be used multiple times, with each argument corresponding to a new constraint group definition (The same is true for the `-eq` flag). Additionally, it also highlights how *pype-resp.py* distinguishes between intra- and intermolecular charge constraints. By default, an intramolecular charge constraint is assumed if only one argument is supplied (here: MOL_1 : 5 6 7), whereas submission of two arguments (here: MOL_1 : 10 12 14 and MOL_2 : 24 15 29) leads to definition of an intermolecular charge constraint. In the case of charge constraint definitions, if more than two arguments are registered, *pype-resp.py* will exit with an error.

In the above example, both charge constraint groups would be constrained to a total charge of 0, which is the default. The charge can be changed by appending it to the constraint group definition, e.g.:

```
-c "MOL.1:5 6 7:-1" -c "MOL.1:10 12 14" "MOL.2:24 15 29:1"
```

constrains the first group to a total charge of -1 and the second group to a total charge of 1 independent of whether a charge was also specified in the first argument.

20.4.2. Usage of SMARTS patterns

By convention, RESP charge constraint groups are defined using atom indices, which are based on the atom order read from the ESP input files. However, in some cases, it may be more convenient to identify the relevant atoms based on substructure matching instead of looking up the respective indices. In such cases *pype-resp.py* allows the definition of `ATOM_IDENTIFIERS` using unambiguous SMARTS patterns to be matched against the target molecule using the *RDKit* Python package. Therefore, all features of the SMARTS language can be utilized to target a desired group of atoms. It should be noted that hydrogen atoms need to be included explicitly in the SMARTS pattern, if they are meant to be included in the constraint group. However, since writing even short expressions such as “`C ([H]) ([H]) ([H])`” to match a methyl group can be tedious, *pype-resp.py* allows users to omit the brackets around hydrogen atoms and will add them automatically before substructure matching. Hence, “`C (H) (H) (H)`” is a valid expression to match a methyl group.

Given a SMARTS pattern, it might not be immediately obvious whether it leads to an unambiguous substructure match or not. To combat this, *pype-resp.py* can perform test runs to determine whether SMARTS patterns used as `ATOM_IDENTIFIERS` lead to one or multiple substructure matches in the respective molecule. It can be invoked by passing the `-ts/--test-smarts` flag along with valid definitions for charge constraint groups or atom equivalency statements, e.g.

```
pype-resp.py -dir . -c "MOL.1:C(H) (H) (H)" \
-eq "MOL.1:[N;H1]" "MOL.2:[N;H2]" -ts
```

(Note: Additional constraint group and atom equivalency statements using tom indices as `ATOM_IDENTIFIERS` will be disregarded for testing).

Once it is known that a SMARTS pattern is ambiguous, it can be difficult to modify to ensure unambiguity. To facilitate this, *pype-resp.py* allows the slicing of SMARTS patterns by enclosing the parts to neglect after substructure matching in pipe (“|”) symbols. This can be utilized to rewrite complex SMARTS sequences such as “`[C; $ (C-[O; $ (O-C) ]) ] (H) (H) (H)`”, which will match a methyl group that is attached to oxygen attached to an aromatic carbon, to “`C (H) (H) (H) |Oc|`”, thereby encapsulating the same information in a more concise expression.

20.4.3. Input file generation

Parametrizing many molecules with multiple constraints and atom equivalency statements can lead to long commands and reduce clarity. As an alternative, *pype-resp.py* offers to use input files instead, which can be passed using the `-i/--input` flag:

```
pype-resp.py -i $INPUT_FILE
```

where “`$INPUT_FILE`” corresponds to the absolute or relative path to the input file. Note that if an input file is used, all other command line options will be ignored, as all settings are now read from the input file. Input files cover the same options as the command line interface (see below) and will fall back to their default values if not specified in the input file. Detailed information about input files can be obtained by running *pype-resp.py* with the `-ih/--input_help` flag set, and a template input file is obtained by passing the `-t/--template` flag.

Input file format (order is arbitrary)

-Molecule definition section-

MOL.n = substring

where *n* is an integer (1-based indexing) corresponding to the assigned molecule index and *substring* is a substring matching the *Gaussian* output filename of the target molecule.

-Constraint group definition section-

MOL.n_CON.m = ATOM_IDENTIFIER : GROUP_CHARGE (float)

where *n* is an integer (1-based indexing) corresponding to the index assigned in the molecule definition section and *m* is an integer (1-based indexing) corresponding to the assigned constraint group in molecule *n*. **ATOM_IDENTIFIER** can either be a list of space-separated atom indices or a SMARTS pattern (see above). **GROUP_CHARGE** is an optional argument and must only be set if the group should be constrained to a charge other than zero.

-Atom equivalency definition section-

EQUATE = MOL.n ATOM_IDENTIFIER ; MOL.m ATOM_IDENTIFIER ; ...

where *n* and *m* are integers (1-based indexing) corresponding to the indices assigned in the molecule definition section and **ATOM_IDENTIFIER** can either be a single atom index or a SMARTS pattern matching a single atom in the target molecule.

-General options section-

ATOM.TYPE	= gaff2	Use gaff2 atom types (default)
	= gaff	Use gaff atom types
	= amber	Use amber atom types
	= bcc	Use bcc atom types
	= abcg2	Use abcg2 atom types
WORK.DIR	= absolute or relative path to the working directory. Defaults to current directory.	
OUTPUT.NAME	= Name for log and csv file created. Defaults to "RESP_charge_summary".	
MOL2.ONLY	= True	Only generate mol2 files and exit. Defaults to False.
NORESP	= True	Stop before carrying out RESP fit. Defaults to False.
FIRST.STAGE.ONLY	= True	Only carry out first stage RESP fitting. Defaults to False.
MOL.NAME	= Three letter Residue name implemented in all generated mol2 and mol3 files. Defaults to "MOL".	

-pyresp_gen.py and pyresp.py related options-

CHARGE.MODEL	= chg	additive point-charge model (default)
	= ind	induced dipole model
	= perm	induced dipole with permanent dipoles model
DAMP	= 1	Applequist scheme without damping
	= 2	Tinker-exponential damping scheme
	= 3	Exponential damping scheme

	= 4	Linear damping scheme
	= 5	pGM damping scheme
SEARCH_DEPTH	= Maximum depth (integer) for searching equivalent atoms. Defaults to 3.	
CHARGE_WEIGHT_1	= Charge restraint weight (float) for the first-stage RESP fit. Defaults to 0.0005.	
CHARGE_WEIGHT_2	= Charge restraint weight (float) for the second-stage RESP fit. Defaults to 0.001.	
POLAR_WEIGHT_1	= Dipole restraint weight (float) for the first-stage RESP fit. Defaults to 0.0001.	
POLAR_WEIGHT_2	= Dipole restraint weight (float) for the second-stage RESP fit. Defaults to 0.0005.	
EXC12	= True Exclude 1-2 interactions for electric field calculations. Defaults to False.	
EXC13	= True Exclude 1-3 interactions for electric field calculations. Defaults to False.	

20.4.4. Output files

Within the working directory, *pype-resp.py* will create a collection of temporary files that will be deleted once the RESP fit is finished. They have the following extensions:

- *.PyPERESPesp*: Created by *espgen*; contains ESP information per conformer
- *.PyPERESPMol2*: Created by *antechamber*; mol2 file per conformer
- *.PyPERESPTEMP*: Created by *antechamber*; mol2 file with Sybyl atom types to be read by *RDKit*

Additionally, a file named “*_PyPE_RESP.log*” is created to outline the individual steps performed by *pype-resp.py*, and a subfolder named “*_PyPE_RESP*” is generated, which contains three subdirectories named “*Mol3_files*”, “*Run_statistics*” and “*RESP_RUN_FILES*” respectively. The latter contains all files read and created by *py_resp.py*. These are:

- *Concatenated.esp*: Contains ESP information for all conformers per molecule
- *First_stage.in*: First stage *py_resp.py* input file generated by *pype-resp.py*
- *First_stage.out*: First stage result file generated by *py_resp.py*
- *First_stage.chg*: First stage parameter file generated by *py_resp.py*
- *First_stage.espout*: First stage ESP output file generated by *py_resp.py*

The latter three files are also created for the second stage RESP fit (prefix: “*Second_stage*”) if it is carried out. In “*Run_statistics*” a log file can be found (default name: “*RESP_charge_summary.log*”) that contains fit statistics provided by *py_resp.py* (ssvpot, RSS, RMSE, RRMSE) as well as information about the derived RESP charges per constraint group. This information includes:

- The sum of charges per constraint group

20. Electrostatic Parameterization with *py_resp.py*

- The sum of charges of all intra- and intermolecular charge constraint groups per molecule
- The sum of charges of all atoms not assigned to any constraint group per molecule
- The difference between the latter two sums

This information is additionally stored in a CSV file (default name: “*RESP_charge_summary.csv*”) to support subsequent processing by data analysis software.

Finally, “*Mol3_files*” stores the mol3 files created for each molecule processed by *pype-resp.py*. Within the mol3 files, the derived RESP charges are mapped to each atom and it also contains information about the `connect0` to `connect5` Records, which facilitates subsequent processing by *LEaP*.

20.4.5. Additional functionalities

As stated above, *pype-resp.py* includes all functionalities provided by *py_resp.py*, including the RESP-ind and REPS-perm charge models, along with the options related to these models. They can be set using flags named identical to those in *py_resp.py*.

The RESP fitting can be reduced to first stage fitting by adding the `-no2/--no2stage` flag or can be prohibited entirely by adding the `-noresp/--no-pyresp` flag, which enables manual inspection of the generated RESP inputs before fitting.

Finally, *py_resp.py* can use more than one CPU core by setting the number of cores to use *via* the `-ncpus/--ncpus` flag. **This does not affect the fitting process**, but rather the creation of individual ESP files and *py_resp.py* input files.

21. Setting up crystal simulations

David S. Cerutti

Simulations of biomolecular crystals are in principle no different than any of the simulations that AMBER does in periodic boundary conditions. However, the setup of these systems is not trivial and probably cannot be accomplished with the LEaP software. Of principal importance are the construction of the solvent conditions (packing precise amounts of multiple solvent species into the simulation cell), and tailoring the unit cell dimensions to accommodate the inherently periodic nature of the system. The LEaP software, designed to construct simulations of molecules in solution, will overlay a pre-equilibrated solvent mask over the (biomolecular) solute, tile that mask throughout the simulation cell, and then prune solvent residues which clash with the solute. The result of this procedure is a system which will likely contract under constant pressure dynamics as the pruning process has left vacuum bubbles at the solute:solvent interface. Simulations of biomolecular crystals require that the simulation cell begin at a size corresponding to the crystallographic unit cell, and deviate very little from that size over the course of equilibration and onset of constant pressure dynamics. This demands a different strategy for placing solvent in the simulation cell. Four programs in the *AmberTools* release are designed to accomplish this. An example of their use is given in a web-based tutorial at <https://ambermd.org/tutorials/advanced/tutorial13/XtalTutor1.html>. A recent (2018) review of crystal simulations is also worth consulting.^[484]

For brevity, only basic descriptions of the programs are given in this manual. All of the programs may be run with command line input; the input options to each program may be listed by running each program with no arguments.

21.1. UnitCell

A macromolecular crystal contains many repeating unit cells which stack like blocks in three dimensional space just as simulation cells do in periodic boundary conditions. Each unit cell, in turn, may contain multiple symmetry-related clusters of atoms. A PDB file contains one set of coordinates for the irreducible unit of the crystal, the “asymmetric unit,” and also information about the crystal space group and unit cell dimensions. The *UnitCell* program reads PDB files, seeking the SMTRY records within the REMARKs to enumerate the rotation and translation operations which may be applied to the coordinates given in the PDB file to reconstruct one complete unit cell.

Usage of the *UnitCell* program is as follows. The simple command rests on a critical assumption, that the PDB file contains an accurate CRYST1 record and that the REMARK 290 SMTRY records provide its space group symmetry operations.

```
UnitCell -p MyProtein.pdb -o UnitCell.pdb
```

21.2. PropPDB

Simulations in periodic boundary conditions require a minimum unit cell size: the simulation cell must be able to enclose a sphere of at least the nonbonded direct space cutoff radius plus a small buffer region for nonbonded pairlist updates. Many biomolecular crystal unit cells come in “shoebox” dimensions that may have one very short side; many unit cells are also not rectangular but triclinic, meaning that the size of the largest sphere they can enclose is further reduced. The workhorse simulation engine, pmemd.cuda, even requires that the simulation cell be at least three times as thick as the cutoff plus some buffer margin in order to run safely: for typical sum conditions this thickness is about 30Å. For these reasons, and perhaps to ensure that the rigid symmetry imposed by periodic boundary conditions does not create artifacts (crystallographic unit cells are equivalent when averaged over all time and space, but are not necessarily identical at any given moment), it may be necessary to include

21. Setting up crystal simulations

multiple unit cells within the simulation cell. This is the purpose of the *PropPDB* program: to propagate a unit cell in one or more directions so that the complete simulation cell meets minimum size requirements.

Drawing on the hypothetical example above, if the unit cell is too small we can extend it in the *x* and *z* dimensions:

```
PropPDB -p UnitCell -o ExpandedCell.pdb -ix 2 -iy 1 -iz 2
```

21.3. AddToBox

The *AddToBox* program handles placement of solvent within a crystal unit cell or supercell (as may be created by *PropPDB*). As described in the introduction, the basic strategy is to place solvent such that added solvent molecules do not clash with biomolecule solutes, but *may* clash with one another initially. This compromise is necessary because enough solvent must be added to the system to ensure that the correct unit cell dimensions are maintained in the long run, but it is not acceptable to place solvent within the interior of a biomolecule where it might not belong and never escape.

The *AddToBox* program takes a PDB file providing the coordinates of a complete biomolecular unit cell or supercell (argument -c), the dimensions by which that supercell repeats in space (the unit cell dimensions are taken from the CRYST1 record of this file), a PDB file describing the solvent residue to add (argument -a), and the number of copies of that solvent molecule to add (argument -na). *AddToBox* inherently assumes that the biomolecular unit cell it is initially presented may contain some amount of solvent already, and according to the AMBER convention of listing macromolecular solute atoms first and solvent last assumes that the first -P atoms in the file are the protein (or biomolecule). *AddToBox* will then color a very fine grid “black” if the grid point is within a certain distance of a biomolecular atom (argument -RP, default 5.0Å) or other solvent atom (argument -RW, default 1.0Å); the grid is “white” otherwise (the grid is stored in binary for memory efficiency). *AddToBox* will then make a copy of the solvent residue and randomly rotate and translate it somewhere within the unit cell. If all atoms of the solvent residue land on “white” grid voxels, the solvent molecule will become part of the system and the grid around the newly added solvent will be blacked out accordingly. If the solvent molecule cannot be placed, this process will be repeated until a million consecutive failures are encountered, at which point the program will terminate. If *AddToBox* has not placed the requested number of solvent molecules by the time it terminates, the -V option can be used to order the program to recursively call itself with progressively smaller solvent buffer distances until all the requested solvent can be placed. The output of the *AddToBox* program is another PDB named by the -o option.

Successful operation of *AddToBox* may take practice. If multiple solvent species are required, as is the case with heterogeneous crystallization solutions, *AddToBox* may be called repeatedly with each input molecular cell being the previous call’s output. When considering crystal solvation, the order of addition is important! It is recommended that rare species, such as trace buffer reagents, be added first, with large -RW argument to ensure that they are dispersed throughout the available crystal void zones. Large solvent species such as MPD (an isohexane diol commonly used in crystallization conditions) should be added second, and with a sufficiently large -RW argument that methyl groups and ring systems cannot become interlocked (which will likely lead to SHAKE / vlimit errors). Small and abundant species such as water should be added last, as they can go anywhere that space remains.

Below is an example of the usage for a hypothetical protein with 5431 atoms and a net charge of +6 that is to be neutralized with ammonium sulfate:

```
AddToBox -c ExpandedCell.pdb -a Sulfate.pdb -na 18 -RP 3.0 -P 5431 -o System.pdb
AddToBox -c System.pdb -a Ammonium.pdb -na 30 -RP 3.0 -P 5431 -o System.pdb
AddToBox -c System.pdb -a Water.pdb -na 1089 -RP 3.0 -P 5431 -o System.pdb
```

The use of the -V flag ensures that the desired amounts of each species are included. The protein clipping radius of 3Å is lower than the default, but safe (remember, this radius stipulates that no solvent atom, regardless of the size of the solvent molecule, come within 3Å of the protein). Note how the original protein PDB file serves as the base for system, but thereafter we work with the System.pdb to accumulate more solvent particles. Here, the

ammonium sulfate serves both to neutralize the system and replicate a salty bath, perhaps from a crystallization mother liquor, hence the break from the usual 2:1 stoichiometry of ammonium sulfate ions.

It is likely that the unobservable “void” regions between biomolecules in most crystals *do not* contain solvent species in proportion to their abundance in the crystallization solution—the vast majority of these regions are within a few Ångströms of some biomolecular surface, and different biomolecular functional groups will preferentially interact with some types of solvent over others. Also, in many crystals some solvent molecules *are* observed; in many of these, the amount of solvent observed is such that it would be impossible to pack other species into the unit cell in proportion to their abundances in the crystallization fluid. In these cases, we recommend estimating the amount of volume that must be filled with solvent *apart from solvent which has already been observed in the crystal*, and filling this void with solvent in proportion to the composition of the crystallization fluid. For example, if a crystal were grown in a 1:1 mole-to-mole water/ethanol mixture, and the crystal coordinates as deposited in the PDB contained 500 water molecules and 3 ethanol molecules, we would use *AddToBox* to add water and ethanol in a 1:1 ratio until the system contained enough solvent to maintain the correct volume during equilibrium dynamics at constant pressure.

Finally, it is difficult to estimate exactly how much solvent will be needed to maintain the correct equilibrium volume; the advisable approach is simply to make an initial guess and script the setup so that, over multiple runs and reconstructions, the correct system composition can be found. We recommend matching the equilibrium unit cell volume to within 0.3% to keep this simulation parameter within the error of most crystallographic measurements. While errors of 0.5-1% will show up quickly after constant pressure dynamics begin, a 10 to 20ns simulation may be needed to ensure that the correct equilibrium volume has been achieved.

21.4. ChBox

After the complex process of adding solvent, the LEaP program may be used to produce a topology and initial set of coordinates based on the PDB file produced by *AddToBox*. By using the *SetBox* command, LEaP will create a periodic system without adding any more solvent on its own. The only problem with using LEaP at this point is that the program will fail to realize that the system *does* tile in three dimensions if only the box dimensions are set properly. If visualized, the output of UnitCell / PropPDB will likely look jagged, but the output of *AddToBox*, containing lots of added water, will make it obvious how parts of biomolecules jutting out one face of the box fit neatly into open spaces on an opposite face. The topology produced by LEaP needs no editing; only the last line of the coordinates does. This can be done manually, but the *ChBox* program automates the process, taking the same coordinates supplied to *AddToBox* and grafting them into the input coordinates file.

The program is even unnecessary in the case of orthorhombic (rectangular) unit cells, as this the *tleap* command will substitute:

```
set [unit] box { <x> <y> <z> }
```

For cells that do not have only 90-degree box angles, *ChBox* will do the trick.

Part IV.

Running simulations

22. sander

22.1. Introduction

This is a guide to *sander*, an Amber module which carries out energy minimization, molecular dynamics, and NMR refinements. The acronym stands for **S**imulated **A**nnealing with **N**MR-**D**erived **E**nergy **R**estraints, but this module is used for a variety of simulations that have nothing to do with NMR refinement. Some of the functionality of *sander* is available with better computational performance in the *pmemd* module. In general, *sander* and *pmemd* are input compatible. *sander* inputs for features not supported by *pmemd* should be properly parsed by *pmemd* and *pmemd* should report that the requested feature is not supported. There are a few features available in *pmemd* that are not supported by *sander*, see Sections 23.3 and 23.4. Some general features are outlined in the following paragraphs:

1. *Sander* provides direct support for several force fields for proteins and nucleic acids, and for several water models and other organic solvents. The basic force field implemented here has the following form, which is about the simplest functional form that preserves the essential nature of molecules in condensed phases:

$$\begin{aligned} V(\mathbf{r}) = & \sum_{bonds} K_b(b-b_0)^2 + \sum_{angles} K_\theta(\theta-\theta_0)^2 \\ & + \sum_{dihedrals} (V_n/2)(1+\cos[n\phi-\delta]) \\ & + \sum_{nonbij} (A_{ij}/r_{ij}^{12}) - (B_{ij}/r_{ij}^6) + (q_i q_j / r_{ij}) \end{aligned}$$

"Non-additive" force fields based on atom-centered dipole polarizabilities can also be used. These add a "polarization" term to what was given above:

$$E_{pol} = -2 \sum_i \mu_i \cdot \mathbf{E}_{io}$$

where μ_i is an induced atomic dipole. In addition, charges that are not centered on atoms, but are off-center (as for lone-pairs or "extra points") can be included in the force field.

2. The particle-mesh Ewald (PME) procedure (or, optionally, a "true" Ewald sum) is used to handle long-range electrostatic interactions. Long-range van der Waals interactions are estimated by a continuum model. Biomolecular simulations in the NVE ensemble (*i.e.* with Newtonian dynamics) conserve energy well over multi-nanosecond runs without modification of the equations of motion.
3. Two periodic imaging geometries are included: rectangular parallelepiped and truncated octahedron (box with corners chopped off). (*Sander* itself can handle many other periodically-replicating boxes, but input and output support in *LEaP* and *ptraj* is only available right now for these two.) The size of the repeating unit can be coupled to a given external pressure, and velocities can be coupled to a given external temperature by several schemes. The external conditions and coupling constants can be varied over time, so various simulated annealing protocols can be specified in a simple and flexible manner.
4. It is also possible to carry out non-periodic simulations in which aqueous solvation effects are represented *implicitly* by a generalized Born/ surface area model by adding the following two terms to the "vacuum" potential function:

$$\Delta G_{sol} = \sum_{ij} \left(1 - \frac{1}{\epsilon}\right) (q_i q_j / f_{GB}(r_{ij})) + A \sum_i \sigma_i$$

The first term accounts for the polar part of solvation (free) energy, designed to provide an approximation for the reaction field potential, and the second represents the non-polar contribution which is taken to be proportional to the surface area of the molecule.

5. Users can define internal restraints on bonds, valence angles, and torsions, and the force constants and target values for the restraints can vary during the simulation. The relative weights of various terms in the force field can be varied over time, allowing one to implement a variety of simulated annealing protocols in a single run.
6. Internal restraints can be defined to be "time-averaged", that is, restraint forces are applied based on the averaged value of an internal coordinate over the course of the dynamics trajectory, not only on its current value. Alternatively, restraints can be "ensemble-averaged" using the locally-enhanced-sampling (LES) option.
7. Restraints can be directly defined in terms of NOESY intensities (calculated with a relaxation matrix technique), residual dipolar couplings, scalar coupling constants and proton chemical shifts. There are provisions for handling overlapping peaks or ambiguous assignments. In conjunction with distance and angle constraints, this provides a powerful and flexible approach to NMR structural refinements.
8. Replica exchange calculations can allow simultaneous sampling at a variety of conditions (such as temperature), and allow the user to construct Boltzmann samples in ways that converge more quickly than standard MD simulations. Other variants of biased MD simulations can also be used to improve sampling.
9. Restraints can also be defined in terms of the root-mean-square coordinate distance from some reference structure. This allows one to bias trajectories either towards or away from some target. Free energies can be estimated from non-equilibrium simulations based on targetting restraints.
10. Free energy calculations, using thermodynamic integration (TI) with a linear or non-linear mixing of the "unperturbed" and "perturbed" Hamiltonian, can be carried out. Alternatively, potentials of mean force can be computed using umbrella sampling.
11. The empirical valence bond (EVB) scheme can be used to mix "diabatic" states into a potential that can represent many types of chemical reactions that take place in enzymes.
12. QMMM Calculations where part of the system can be treated quantum mechanically allowing bond breaking and formation during a simulation. Semi-empirical and DFTB Hamiltonians are provided directly within *sander*. More advanced *ab initio* and DFT Hamiltonians are available via an interface to external QM software packages.
13. Nuclear quantum effects can be included through path-integral molecular dynamics (PIMD) simulations, and estimates of quantum time-correlation functions can be computed.

22.2. File usage

```
sander [-help] [-O] [-A] -i mdin -o mdout -p prmtop -c inpcrd -r restrt
-ref refc -mtmd mtmd -x mdcrd -y inptraj -v mdvel -frc mdfrc -e mden
-inf mdinfo -radii radii -cpin cpin -cpout cpout -cprestrt cprestrt
-cein cein -ceout ceout -cerestrt cerestrt -evbin evbin -suffix suffix
-O Overwrite output files if they exist.
-A Append output files if they exist (used mainly for replica exchange).
```

Here is a brief description of the files referred to above; the first five files are used for every run, whereas the remainder are only used when certain options are chosen.

mdin *input* control data for the min/md run

mdout *output* user readable state info and diagnostics -o stdout will send output to stdout (to the terminal) instead of to a file.

mdinfo *output* latest mdout-format energy info

prmtop *input* molecular topology, force field, periodic box type, atom and residue names

inpcrd *input* initial coordinates and (optionally) velocities and periodic box size

refc *input* (optional) reference coords for position restraints; also used for targeted MD

mtmd *input* (optional) containing list of files and parameters for targeted MD to multiple targets

mdcrd *output* coordinate sets saved over trajectory

inptraj *input* coordinate sets in trajectory format, when imin=5 or 6

mdvel *output* velocity sets saved over trajectory

mdfrc *output* force sets saved over trajectory

mden *output* extensive energy data over trajectory (not synchronized with mdcrd or mdvel)

restrt *output* final coordinates, velocity, and box dimensions if any - for restarting run

inpdip *input* polarizable dipole file, when indmeth=3

rstddip *output* polarizable dipole file, when indmeth=3

cpin *input* protonation state definitions

cprestrt protonation state definitions, final protonation states for restart (same format as cpin)

cpout *output* protonation state data saved over trajectory

cein *input* redox state definitions

cerestrt redox state definitions, final redox states for restart (same format as cein)

ceout *output* redox state data saved over trajectory

evbin *input* input for EVB potentials

suffix *output* this string will be added to all unspecified output files that are printed (for *multisander* runs, it will append this suffix to all output files)

22.3. Example input files

Here are a couple of sample files, just to establish a basic syntax and appearance. There are more examples of NMR-related files later in this chapter.

1. Simple restrained minimization

```
Minimization with Cartesian restraints
&cntrl
imin=1, maxcyc=200, (invoke minimization)
ntpr=5, (print frequency)
ntr=1, (turn on Cartesian restraints)
restraint_wt=1.0, (force constant for restraint)
restraintmask=':1-58', (atoms in residues 1-58 restrained)
/
```

2. "Plain" molecular dynamics run

```
molecular dynamics run
&cntrl
imin=0, irest=1, ntx=5, (restart MD)
ntt=3, temp0=300.0, gamma_ln=5.0, (temperature control)
ntp=1, taup=2.0, (pressure control)
ntb=2, ntc=2, ntf=2, (SHAKE, periodic bc.)
nstlim=500000, (run for 0.5 nsec)
ntwx=1000, ntpr=200, (output frequency)
/
```

3. Self-guided Langevin dynamics run

```
Self-guided Langevin dynamics run
&cntrl
imin=0, irest=0, ntx=1, (start LD)
ntt=3, temp0=300.0, gamma_ln=1.0, (temperature control)
ntc=3, ntf=3, (SHAKE)
nstlim=500000, (run for 0.5 nsec)
ntwx=1000, ntpr=200, (output frequency)
isgld=1, tsgavg=0.2, sgft=1.0, (SGLD)
/
```

22.4. Namelist Input Syntax

Namelist provides list-directed input, and convenient specification of default values. It dates back to the early 1960's on the IBM 709, but was regrettably not part of Fortran 77. It is a part of the Fortran 90 standard, and is supported as well by most Fortran 77 compilers (including g77).

Namelist input groups take the form:

```
&name
var1=value, var2=value, var3(sub)=value,
var4(sub,sub,sub)=value,value,
var5=repeat*value,value,
/
```

The variables must be names in the Namelist variable list. The order of the variables in the input list is of no significance, except that if a variable is specified more than once, later assignments may overwrite earlier ones.

Blanks may occur anywhere in the input, except embedded in constants (other than string constants, where they count as ordinary characters).

It is common in older inputs for the ending "/" to be replaced by "&end"; this is non-standard-conforming.

Letter case is ignored in all character comparisons, but case is preserved in string constants. String constants must be enclosed by single quotes ('). If the text string itself contains single quotes, indicate them by two consecutive single quotes, e.g. C1' becomes 'C1'' as a character string constant.

Array variables may be subscripted or unsubscripted. An unsubscripted array variable is the same as if the subscript (1) had been specified. If a subscript list is given, it must have either one constant, or exactly as many as the number in the declared dimension of the array. Bounds checking is performed for ALL subscript positions, although if only one is given for a multi-dimension array, the check is against the entire array size, not against the first dimension. If more than one constant appears after an array assignment, the values go into successive locations of the array. It is NOT necessary to input all elements of an array.

Any constant may optionally be preceded by a positive (1,2,3,...) integer repeat factor, so that, for example, 25*3.1415 is equivalent to twenty-five successive values 3.1415. The repeat count separator, *, may be preceded and followed by 0 or more blanks. Valid LOGICAL constants are 0, F, .F., .FALSE., 1, T, .T., and .TRUE.; lower case versions of these also work.

22.5. Overview of the information in the input file

General minimization and dynamics input One or more title lines, followed by the (required) &cntrl and (optional) &pb, &ewald, &qmmm, &amoeba or &debugf namelist blocks. Described in Sections 22.6 and 22.7.

Varying conditions Parameters for changing temperature, restraint weights, etc., during the MD run. Each parameter is specified by a separate &wt namelist block, ending with &wt type="END", /. Described in Section 22.9.

File redirection TYPE=*filename* lines. Section ends with the first non-blank line which does not correspond to a recognized redirection. Described in Section 22.10.

Group information Read if *ntr*, *ibelly* or *idecomp* are set to nonzero values, and if some other conditions are satisfied; see sections on these variables, below. Described in Appendix 24.3.

22.6. General minimization and dynamics parameters

Each of the variables listed below is input in a namelist statement with the namelist identifier &cntrl.cmmu can enter the parameters in any order, using keyword identifiers. Variables that are not given in the namelist input retain their default values. Support for namelist input is included in almost all current Fortran compilers, and is a standard feature of Fortran 90. A detailed description of the namelist convention is given in Appendix A.

In general, namelist input consists of an arbitrary number of comment cards, followed by a record whose first seven characters after a "&" (e.g. "&cntrl") name a group of variables that can be set by name.cmsys is followed by statements of the form " maxcyc=500, diel=2.0, ... ", and is concluded by an "/" token. The first line of input contains a title, which is then followed by the &cntrl namelist. Note that the first character on each line of a namelist block must be a blank.

Some of the options and variables are much more important, and commonlycmrdified, than are others. We have denoted the "common" options by printing them in **boldface** below. In general, you can skip reading about the non-bold options on a first pass, and you should change these from their defaults only if you think you know what you are doing.

22.6.1. General flags describing the calculation

imin Flag to run minimization.

- = 0** (default) Run molecular dynamics without any minimization.
- = 1** Perform an energy minimization.
- = 5** Read in a trajectory for analysis using the minimization algorithms.

Although sander will write energy information in the output files (using *ntpr*), it is often desirable to calculate the energies of a set of structures at a later point. In particular, one may wish to post-process a set of structures using a different energy function than was used to generate the structures. An example of this is MM-PBSA analysis, where the explicit water is removed and replaced with a continuum model.

If *imin* is set to 5, sander will read a trajectory file (the “inptraj” argument, specified using *-y* on the command line), and will perform the functions described in the *mdin* file (e.g., an energy minimization) for each of the structures in this file. The final structure from each minimization will be written out to the normal *mdcrd* file. If you wish to read in a binary (i.e., NetCDF format) trajectory, be sure to set *ioutfm* to 1 (see below). Note that this will result in the output trajectory having NetCDF format as well.

For example, when *imin* = 5 and *maxcyc* = 1000, sander will minimize each structure in the trajectory for 1000 steps and write a minimized coordinate set for each frame to the *mdcrd* file. If *maxcyc* = 1, the output file can be used to extract the energies of each of the coordinate sets in the *inptraj* file.

Trajectories containing box coordinates can be post-processed. In order to read trajectories with box coordinates, *ntb* should be greater than 0.

IMPORTANT CAVEAT: The initial coordinates input file used (*-c* <inpcrd>) should be the same as the initial coordinates input file used to generate the original trajectory. This is because sander sets up parameters for PME from the box coordinates in the initial coordinates input file.

- = 6** Read in a trajectory for analysis using the molecular dynamics driver

Like *imin*=5, this option reads a trajectory file for analysis (the “inptraj” argument, specified using *-y* on the command line). Instead of minimizing the potential energy of each coordinate set, it instead initiates dynamics from each frame as if it were read as a restart file. That is, this option is equivalent to outputting each frame as a restart file and starting the dynamics with the given options. If *nstlim*=0, then this effectively performs a single point energy for each frame. If the input trajectory file contains velocities (from a previous simulation that set *ntwv*=-1), then the stored values are used as the initial velocities of each simulation. If the input trajectory file does not contain velocities, then each simulation assigns initial velocities based on the value of *tempi*. If the random number seed (*ig*) is set to -1, then a new random seed value is generated for each frame; otherwise all frames use the same random number seed (i.e., the specified value of *ig*). The values of *ig*, *irest*, *ntx*, *nstlim*, and *tempi* needed to reproduce the results are written to the *mdout* file before each simulation is initiated. One should be able to reproduce the results by extracting the appropriate frame as a restart file and using the provided values of *ig*, *rest*, *ntx*, *nstlim*, and *tempi*.

- = 7** Listen to the selected internet socket and return energies and forces when instructed by an external server

When this option is set, sander does not perform MD; instead, it listens for messages from a server instructing it to compute the potential energy and forces of a system. The server IP address and port number are provided as command line arguments *-host* and *-port*. The default values are *-host* 127.0.0.1 and *-port* 31415. The communication pattern follows the protocol implemented in the i-PI software. The [i-PI program](#) is a molecular dynamics driver used to perform classical and centroid path integral molecular dynamics. When i-PI performs classical MD, one can instantiate a single sander process to evaluate the potential. However, when i-PI is used to perform PIMD, which involves calculating potential energies for several

“beads” at each time step, multiple instances of sander can be launched to simultaneously evaluate the required potential energies. The current implementation of the interface is limited to simulations in the NVE and NVT ensembles; therefore, one should launch sander with a restart file whose unit cell lattice vectors are consistent with the input structure supplied to i-PI.

nmropt = 0 (default) No nmr-type analysis will be done.
 = 1 NMR restraints and weight changes will be read.
 = 2 NMR restraints, weight changes, NOESY volumes, chemical shifts and residual dipolar restraints will be read.

22.6.2. Nature and format of the input

ntx Option to read the initial coordinates, velocities, and box size from the inpcrd file. Option 1 must be used when one is starting from minimized or model-built coordinates. If an MD restrt file is specified for inpcrd then option 5 is generally used (unless you explicitly wish to ignore the velocities that are present).
 = 1 (default) Coordinates, but no velocities, will be read; either formatted (ASCII) files or NetCDF files can be used, as the input file type will be auto-detected.
 = 5 Coordinates and velocities will be read from either a NetCDF or a formatted (ASCII) coordinate file. Box information will be read if *ntb* > 0. The velocity information will only be used if *irest* = 1 (see below).

irest Flag to restart a simulation.
 = 0 (default) Do not restart the simulation; instead, run as a new simulation. Velocities in the input coordinate file, if any, will be ignored, and the time step count will be set to 0 (unless overridden by *t*; see below).
 = 1 Restart the simulation, reading coordinates and velocities from a previously saved restart file. The velocity information is necessary when restarting, so *ntx* (see above) must be 5 (for Amber versions much older than 20, *ntx* must be greater than or equal to 4), if *irest* = 1.

22.6.3. Nature and format of the output

ntxo Format of the final coordinates, velocities, and box size (if a constant volume or pressure run) written to file "restrt".
 = 1 Formatted (ASCII)
 = 2 (default) NetCDF file (recommended, unless you have a workflow that requires the formatted form.)

ntpr Every *ntpr* steps, energy information will be printed in human-readable form to files "mdout" and "mdinfo". "mdinfo" is closed and reopened each time, so it always contains the most recent energy and temperature. Default 50.

ntave Every *ntave* steps of dynamics, running averages of average energies and fluctuations over the last *ntave* steps will be printed out. A value of 0 disables this printout. Setting *ntave* to a value 1/2 or 1/4 of *nstlim* provides a simple way to look at convergence during the simulation. Default = 0 (disabled).

ntwr Every *ntwr* steps during dynamics, the "restrt" file will be written, ensuring that recovery from a crash will not be so painful. No matter what the value of *ntwr*, a restrt file will be written at the end of the run, i.e., after *nstlim* steps (for dynamics) or *maxcyc* steps (for minimization). If *ntwr* < 0, a unique copy of the file, "restrt.<nstep>", is written every abs(*ntwr*) steps. This option is

useful if for example one wants to run free energy perturbations from multiple starting points or save a series of *restrt* files for minimization. Default = *nstlim*.

- iwrap** If *iwrap* = 1, the coordinates written to the restart and trajectory files will be "wrapped" into a primary box. This means that for each molecule, its periodic image closest to the middle of the "primary box" (with x coordinates between 0 and a, y coordinates between 0 and b, and z coordinates between 0 and c) will be the one written to the output file. This often makes the resulting structures look better visually, but has no effect on the energy or forces. Performing such wrapping, however, can mess up diffusion and other calculations. If *iwrap* = 0, no wrapping will be performed, in which case it is typical to use *cpptraj* as a post-processing program to translate molecules back to the primary box. For very long runs, setting *iwrap* = 1 may be required to keep the coordinate output from overflowing the trajectory and restart file formats, especially if trajectories are written in ASCII format instead of NetCDF (see also the *ioutfm* option). Default = 0.
- ntwx** Every *ntwx* steps, the coordinates will be written to the *mdcrd* file. If *ntwx* = 0, no coordinate trajectory file will be written. Default = 0.
- ntwv** Every *ntwv* steps, the velocities will be written to the *mdvel* file. If *ntwv* = 0, no velocity trajectory file will be written. If *ntwv* = -1, velocities will be written to *mdcrd*, which then becomes a combined coordinate/velocity trajectory file, at the interval defined by *ntwx*. This option is available only for binary NetCDF output (*ioutfm* = 1). Most users will have no need for a velocity trajectory file and so can safely leave *ntwv* at the default. Default = 0. Note that dumping velocities frequently, like forces or coordinates, will introduce potentially significant I/O and communication overhead, hurting both performance and parallel scaling.
- ionstepvelocities** Controls whether to print the half-step-ahead velocities (0, default) or on-step velocities (1). The half-step-ahead velocities can potentially be used to restart calculations, but the on-step velocities correspond to calculated kinetic energy/temperature.
- ntwf** Every *ntwf* steps, the forces will be written to the *mdfrc* file. If *ntwf* = 0, no force trajectory file will be written. If *ntwf* = -1, forces will be written to the *mdcrd*, which then becomes a combined coordinate/force trajectory file, at the interval defined by *ntwx*. This option is available only for binary NetCDF output (*ioutfm* = 1). Most users will have no need for a force trajectory file and so can safely leave *ntwf* at the default. Default = 0. Note that dumping forces frequently, like velocities or coordinates, will introduce potentially significant I/O and communication overhead, hurting both performance and parallel scaling.
- ntwe** Every *ntwe* steps, the energies and temperatures will be written to file "mden" in a compact form. If *ntwe* = 0 then no *mden* file will be written. Note that energies in the *mden* file are not synchronized with coordinates or velocities in the *mdcrd* or *mdvel* file(s). Assuming identical *ntwe* and *ntwx* values the energies are one time step before the coordinates (as well as the velocities which are synchronized with the coordinates). Consequently, an *mden* file is rarely written. Default = 0.
- ioutfm** The format of coordinate and velocity trajectory files (*mdcrd*, *mdvel* and *inptraj*). As of Amber 9, the binary format used in previous versions is no longer supported; binary output is now in NetCDF trajectory format. Binary trajectory files have many advantages: they are smaller, higher precision, much faster to read and write, and able to accept a wider range of coordinate (or velocity) values than formatted trajectory files.
- = 0 Formatted ASCII trajectory
- = 1 (default) Binary NetCDF trajectory
- ntwpri** The number of atoms to include in trajectory files (*mdcrd* and *mdvel*). This flag can be used to decrease the size of these files, by including only the first part of the system, which is usually of greater interest (for instance, one might include only the solute and not the solvent). If *ntwpri* = 0, all atoms will be included.

= 0 (default) Include all atoms of the system when writing trajectories.

> 0 Include only atoms 1 to *ntwpri* when writing trajectories.

idecomp Perform energy decomposition according to a chosen scheme. In former distributions this option was only really useful in conjunction with *mm_pbsa*, where it is turned on automatically if required. Now, a decomposition of $\langle \partial V / \partial \lambda \rangle$ on a per-residue basis in thermodynamic integration (TI) simulations is also possible.^[485] The options are:

= 0 (default) Do not decompose energies.

= 1 Decompose energies on a per-residue basis; 1-4 EEL + 1-4 VDW are added to internal (bond, angle, dihedral) energies.

= 2 Decompose energies on a per-residue basis; 1-4 EEL + 1-4 VDW are added to EEL and VDW.

= 3 Decompose energies on a pairwise per-residue basis; otherwise equivalent to *idecomp* = 1. Not available in TI simulations.

= 4 Decompose energies on a pairwise per-residue basis; otherwise equivalent to *idecomp* = 2. Not available in TI simulations.

If energy decomposition is requested, residues may be chosen by the RRES and/or LRES card. The RES card is used to select the residues about which information is written out. See chapters 26.1 for more information. Use of *idecomp* > 0 is incompatible with *ntr* > 0 or *ibelly* > 0.

22.6.4. Frozen or restrained atoms

ibelly Flag for belly type dynamics. If set to 1, a subset of the atoms in the system will be allowed to move, and the coordinates of the rest will be frozen. The *moving* atoms are specified with *bellymask*. This option is not available when *igb*>0. When belly type dynamics is in use, bonded energy terms, vdW interactions, and direct space electrostatic interactions are *not calculated* for pairs of frozen atoms. Note that this does *not* provide any significant speed advantage. Freezing atoms can be useful for some applications but is maintained primarily for backwards compatibility with older versions of Amber. Most applications should use the *ntr* variable instead to restrain parts of the system to stay close to some initial configuration. Default = 0.

ntr Flag for restraining specified atoms in Cartesian space using a harmonic potential, if *ntr* = 1. The restrained atoms are determined by the *restraintmask* string. The force constant is *restraint_wt*. The reference coordinates are read in "restrt" format from the "refc" file. Default = 0. If *ntr*=1, you should also set *netfrc*=0; see the *netfrc* variable for more information.

restraint_wt The weight ($\text{kcal} \cdot \text{mol}^{-1} \cdot \text{\AA}^{-2}$) for Cartesian restraints when *ntr* = 1. The restraint is of the form $k(\Delta x)^2$, where *k* is the force constant of value given by this variable, and Δx is the difference between one of the Cartesian coordinates of a restrained atom and its reference position. There is a term like this for each Cartesian coordinate of each restrained atom. Note that this variable does not have anything to do with NMR restraints, and there is no way to have *restraint_wt* depend upon the time step.

restraintmask String that specifies the *restrained* atoms when *ntr* = 1.

bellymask String that specifies the *moving* atoms when *ibelly*=1.

The syntax for both *restraintmask* and *bellymask* is given in Section 24.1.1. Note that these mask strings are limited to a maximum of 256 characters.

22.6.5. Energy minimization

maxcyc	The maximum number of cycles of minimization. Default = 1.
ncyc	If NTMIN is 1 then the method of minimization will be switched from steepest descent to conjugate gradient after NCYC cycles. Default 10.
ntmin	Flag for the method of minimization. = 0 Full conjugate gradient minimization. The first 4 cycles are steepest descent at the start of the run and after every nonbonded pairlist update. = 1 For NCYC cycles the steepest descent method is used then conjugate gradient is switched on (default). = 2 Only the steepest descent method is used. = 3 The XMIN method is used, see Section 25.8.4. = 4 The LMOD method is used, see Section 25.8.5. = 5 The DL-Find module is used, see Section 25.9
dx0	The initial step length. If the initial step length is too big then will give a huge energy; however the minimizer is smart enough to adjust itself. Default 0.01.
drms	The convergence criterion for the energy Derivative: minimization will halt when the Root-Mean-Square of the Cartesian elements of the gradient of the energy is less than this. Default is $10^{-4} \text{kcal} \cdot \text{mol}^{-1} \cdot \text{\AA}^{-1}$.

22.6.6. Molecular dynamics

nstlim	Number of MD-steps to be performed. Default 1.
nscm	Flag for the removal of translational and rotational center-of-mass (COM) motion at regular intervals (default is 1000). For non-periodic simulations, after every NSCM steps, translational and rotational motion will be removed. For periodic systems, just the translational center-of-mass motion will be removed. This flag is ignored for belly simulations. For Langevin dynamics, the <i>position</i> of the center-of-mass of the molecule is reset to zero every NSCM steps, but the velocities are not affected. Hence there is no change to either the translation or rotational components of the momenta. (Doing anything else would destroy the way in which temperature is regulated in a Langevin dynamics system.) The only reason to even reset the coordinates is to prevent the molecule from diffusing so far away from the origin that its coordinates overflow the format used in restart or trajectory files.
t	The time at the start (psec) this is for your own reference and is not critical. Start time is taken from the coordinate input file if IREST=1. Default 0.0.
dt	The time step (psec). Recommended MAXIMUM is .002 if SHAKE is used, or .001 if it isn't. Note that for temperatures above 300K, the step size should be reduced since greater temperatures mean increased velocities and longer distance traveled between each force evaluation, which can lead to anomalously high energies and system blowup. Default 0.001. The use of Hydrogen Mass Repartitioning (HMR) (see [142] and references therein for more information), together with SHAKE, allows the time step to be increased in a stable fashion by about a factor of two (up to .004) by slowing down the high frequency hydrogen motion in the system. To use HMR, the masses in the topology file need to be altered before starting the simulation. ParmEd can do this automatically with the HMassRepartition option; see Section 15.2.

nrespa This variable allows the user to evaluate slowly-varying terms in the force field less frequently. For PME, "slowly-varying" (now) means the reciprocal sum. For generalized Born runs, the "slowly-varying" forces are those involving derivatives with respect to the effective radii, and pair interactions whose distances are greater than the "inner" cutoff, currently hard-wired at 8 Å. If $NRESPA > 1$ these slowly-varying forces are evaluated every *nrespa* steps. The forces are adjusted appropriately, leading to an impulse at that step. If $nrespa * dt$ is less than or equal to 4 fs then the energy conservation is not seriously compromised. However if $nrespa * dt > 4$ fs then the simulation becomes less stable. Note that energies and related quantities are only accessible every *nrespa* steps, since the values at other times are meaningless.

22.6.7. Temperature regulation

Note: Flag "ntt" is used for the temperature regulation in the default thermostat scheme as shown below. The "middle" thermostat scheme [Section 22.6.10] is much more efficient than the default scheme to accurately sample the configuration/conformation space in the molecular dynamics simulation for the NVT ensemble. Please read Section 22.6.10 for more details.

ntt Switch for temperature scaling. Note that setting $ntt=0$ corresponds to the microcanonical (NVE) ensemble (which should approach the canonical one for large numbers of degrees of freedom). Some aspects of the "weak-coupling ensemble" ($ntt=1$) have been examined, and roughly interpolate between the microcanonical and canonical ensembles.[486, 487] The $ntt=2$ and 3 options correspond to the canonical (constant T) ensemble.

- = 0** Constant total energy classical dynamics (assuming that $ntb < 2$, as should probably always be the case when $ntt=0$).
- = 1** Constant temperature, using the weak-coupling algorithm.[488] A single scaling factor is used for all atoms. Note that this algorithm just ensures that the total kinetic energy is appropriate for the desired temperature; it does nothing to ensure that the temperature is even over all parts of the molecule. Atomic collisions will tend to ensure an even temperature distribution, but this is not guaranteed, and there are many subtle problems that can arise with weak temperature coupling.[489] Using $ntt=1$ is especially dangerous for generalized Born simulations, where there are no collisions with solvent to aid in thermalization.) Other temperature coupling options (especially $ntt=3$) should be used instead.
- = 2** Andersen-like temperature coupling scheme,[490] in which imaginary "collisions" randomize the velocities to a distribution corresponding to *temp0* every *vrand* steps. Note that in between these "massive collisions", the dynamics is Newtonian. Hence, time correlation functions (etc.) can be computed in these sections, and the results averaged over an initial canonical distribution. Note also that too high a collision rate (too small a value of *vrand*) will slow down the speed at which the molecules explore configuration space, whereas too low a rate means that the canonical distribution of energies will be sampled slowly. A discussion of this rate is given by Andersen.[491] Note that this option is not equivalent to the original thermostat described by Andersen[491].
- = 3** Use Langevin dynamics with the collision frequency γ given by *gamma_ln*, discussed below. Note that when γ has its default value of zero, this is the same as setting $ntt = 0$. Since Langevin simulations are highly susceptible to "synchronization" artifacts,[492, 493] you should explicitly set the *ig* variable (described below) to a different value at each restart of a given simulation.
- = 9** Optimized Isokinetic Nose-Hoover chain ensemble (OIN) [323, 494]. Constant temperature simulation utilizing Nose-Hoover chains and an isokinetic constraint on the particle and thermostat velocities, implemented for use in multiple time-stepping methods, namely for 3D-RISM and RESPA. Stabilizes and smooths particle dynamics and mitigates resonance instabilities, allowing for larger intermediate time steps, up to 16 fs for RESPA ($nrespa=16$ for $dt=0.001$) and 8 fs for 3D-RISM MTS size ($rismnrespa=8$). Each atom is coupled to

three Nose-Hoover chains per atom and the thermostat coupling constant (relaxation time) is determined from $1/\text{gamma_ln}$, hence gamma_ln must be > 0 if $\text{ntt}=9$ invoked. Variable nkija specifies the number of substeps of dt to use for integrating the equations of motion and idistr specifies the frequency at which the thermostat velocity distribution functions are accumulated (if > 0). Such functions are written at frequency ntpr . Two additional files containing the thermostat and chain restart velocities, tfreeze.rst and vfreez.rst , are written at frequency ntwr .

- = 10** Stochastic Isokinetic Nose-Hoover RESPA integrator [495]. A novel isokinetic integrator developed by Tuckerman and co-workers that invokes an isokinetic constraint on the particle velocities combined with nkija (see below) auxiliary thermostat velocities v_1 and v_2 . The integrator includes a stochastic component in the equations of motion, which introduces white noise into the system, for the purpose of minimizing resonance instabilities in the velocities, ultimately allowing for larger RESPA steps. The isokinetic constraint has the form $mv^2 + \frac{L}{L+1} \sum_{i=1}^L Q_1 v_{1i}^2 = Lk_B T$. Here L is the number of additional thermostat degrees of freedom, defined in AMBER as nkija (see below), and Q_1 is the thermostat mass, determined from sinrtau (below), v is the particle velocity and v_1 is one of two auxiliary velocities (e.g. thermostat velocities), and m , k_B , and T , are the particle mass, Boltzmann constant, and system temperature (temp0), respectively. In using this integrator, the system is placed in the isokinetic ensemble, as such the velocities are NOT canonical and no thermodynamic observables can be derived from them. This will lead to anomalous temperature readings throughout the simulation - for 1 thermostat degree of freedom ($L = \text{nkija} = 1$) the temperature will appear about one-half the specified temperature (temp0), and with additional thermostat DOF, the temperature will approach, but never exceed, the desired temperature, temp0 . However, the particle coordinates ARE canonical and it can be said the configurations obtained from a simulation were sampled from a Boltzmann distribution at the specified temperature (temp0).
- = 11** Stochastic version of Berendsen thermostat, also known as Bussi thermostat [496]. This thermostat samples canonical distribution by scaling all velocities to a random temperature probed from canonical distribution. Collision frequency with thermostat is controlled by tautp .

temp0	Reference temperature at which the system is to be kept, if $\text{ntt} > 0$. Note that for temperatures above 300K, the step size should be reduced since increased distance traveled between evaluations can lead to SHAKE and other problems. Default 300.
temp0les	This is the target temperature for all LES particles (see Chapter 6). If $\text{temp0les} < 0$, a single temperature bath is used for all atoms, otherwise separate thermostats are used for LES and non-LES particles. Default is -1, corresponding to a single (weak-coupling) temperature bath.
tempi	Initial temperature. For the initial dynamics run, ($\text{ntx} = 1$ or for Amber versions much older than 20, $\text{ntx} < 3$) the velocities are assigned from a Maxwellian distribution at tempi K. If $\text{tempi} = 0.0$, the velocities will be calculated from the forces instead. tempi has no effect if $\text{ntx} = 5$ (for Amber versions much older than 20, if $\text{ntx} > 3$). Default 0.0.
ig	The seed for the pseudo-random number generator. The MD starting velocity is dependent on the random number generator seed if tempi is nonzero and $\text{ntx} = 1$ (for Amber versions much older than 20, if $\text{ntx} < 3$). The value of this seed also affects the set of pseudo-random values used for Langevin dynamics or Andersen-like coupling, and hence should be set to a different value on each restart if $\text{ntt} = 2$ or 3. If $\text{ig} = -1$ (the default) then the random seed will be based on the current date and time, and hence will be different for every run. Unless you specifically desire reproducibility, it is recommended that you set $\text{ig} = -1$ for all runs involving $\text{ntt} = 2$ or 3.
tautp	Time constant, in ps, for heat bath coupling for the system, if $\text{ntt} = 1$. Default is 1.0. Generally, values for TAUTP should be in the range of 0.5-5.0 ps, with a smaller value providing tighter coupling to the heat bath and, thus, faster heating and a less natural trajectory. Smaller values of

	TAUTP result in smaller fluctuations in kinetic energy, but larger fluctuations in the total energy. Values much larger than the length of the simulation result in a return to constant energy conditions.
gamma_ln	The collision frequency γ , in ps^{-1} , when $ntt = 3$. Default is 0. A simple Leapfrog integrator is used to propagate the dynamics, with the kinetic energy adjusted to be correct for the harmonic oscillator case.[497, 498] Note that it is not necessary that γ approximate the physical collision frequency, which is about 50 ps^{-1} for liquid water. In fact, it is often advantageous, in terms of sampling[498, 499] or stability of integration[500], to use much smaller values, around 2 to 5 ps^{-1} . [498, 500] For implicit solvent (GB), even much lower values may be useful: for example, setting gamma_ln to 0.01 ps^{-1} can lead to significant, up to 100-fold in some cases, speedup of conformational sampling.[183] Also used to determine thermostat coupling constant for the Optimized Isokinetic Nose-Hoover chain integrator (OIN, $ntt=9$), which is equal to $1/\text{gamma_ln}$ [323], so the specified gamma_ln must be > 0 . A gamma_ln of 10 ps^{-1} represents a coupling constant of 100 fs. For $ntt=10$, this is the friction constant associated with the stochastic component of the integrator, essentially serving the same role as in the Langevin integrator [495]. This parameter is required for $ntt=10$ and must be > 0 .
vrand	If $vrand > 0$ and $ntt=2$, the velocities will be randomized to temperature TEMP0 every vrand steps. Default is 1000.
vlimit	If not equal to 0.0, then any component of the velocity that is greater than abs(VLIMIT) will be reduced to VLIMIT (preserving the sign). This can be used to avoid occasional instabilities in molecular dynamics runs. VLIMIT should generally be set to a value like 20 (the default), which is well above the most probable velocity in a Maxwell-Boltzmann distribution at room temperature. A warning message will be printed whenever the velocities are modified. Runs that have more than a few such warnings should be carefully examined.
nkija	For use with $ntt=9$ and $ntt=10$., For $ntt=9$, this the number of substeps of dt when integrating the thermostat equations of motion, for greater accuracy. For $ntt=10$, this specifies the number of additional auxiliary velocity variables $v1$ and $v2$, which will total $nkija \times v1 + nkija \times v2$ [495]. Default is 1 for both integrators.
idistr	For the isokinetic integrator ($ntt=9$), the frequency at which the thermostat velocity distribution functions are accumulated.
sinrtau	For the SINR (Stochastic Isokinetic Nose-Hoover RESPA) integrator ($ntt=10$), this specifies the time scale for determining the masses associated with the two auxiliary velocity variables $v1$ and $v2$ (e.g. thermostat velocities) and hence the magnitude of the coupling of the physical velocities with the auxiliary velocities. Generally this should be related to the time scale of the system. See [495] for more explanation. Default is 1.0.

22.6.8. Pressure regulation

In "constant pressure" dynamics, the volume of the unit cell is adjusted (by small amounts on each step) to make the computed pressure approach the target pressure, *pres0*. Equilibration with $ntp > 0$ is generally necessary to adjust the density of the system to appropriate values. Note that fluctuations in the instantaneous pressure on each step will appear to be large (several hundred bar), but the average value over many steps should be close to the target pressure. Pressure regulation only applies when Constant Pressure periodic boundary conditions are used ($ntp > 0$). The two available pressure coupling algorithms available in Amber are of the "weak-coupling" variety, analogous to temperature coupling,[488] and the use of the Monte Carlo barostat. While the Berendsen barostat yields the correct target density, it does not strictly sample from the isothermal-isobaric ensemble and typically yields volume fluctuations that are too low. The Monte Carlo barostat, on the other hand, samples rigorously from the isobaric-isothermal ensemble and does not necessitate computing the virial. Please note: in general you will need to equilibrate the temperature to something like the final temperature using constant volume ($ntp=0$) before switching on constant pressure simulations to adjust the system to the correct density. If you fail to do this, the program will try to adjust the density too quickly, and bad things (such as SHAKE failures) are likely to happen.

ntp	Flag for constant pressure dynamics. This option should be set to 1 or 2 when Constant Pressure periodic boundary conditions are used. = 0 No pressure scaling (Default) = 1 md with isotropic position scaling = 2 md with anisotropic (x-,y-,z-) pressure scaling: this should only be used with orthogonal boxes (i.e. with all angles set to 90 degrees). Anisotropic scaling is primarily intended for non-isotropic systems, such as membrane simulations, where the surface tensions are different in different directions; it is generally not appropriate for solutes dissolved in water. [323] Anisotropic pressure scaling can also be applied to just one specified direction (x, y or z) with the directional pressure scaling option (<code>baroscalingdir > 0</code>). In this case the box scales along the one chosen direction only, and its dimensions along the other two directions remain fixed. This type of directional pressure control option can be useful in situations where one needs to keep the solvent box unchanged along two direction, while still maintaining a constant pressure in the system. For example, a phase boundary can be created by placing two boxes from different simulations in contact with each other along a common face, which can be useful for simulating phase transitions such as water to ice[501, 502]. = 3 md with semiisotropic pressure scaling: this is only available with constant surface tension (<code>csurften > 0</code>) and orthogonal boxes. This links the pressure coupling in the two directions tangential to the interface. = 4 md towards a targeted volume. This is not for production but for modifying the volume of the system, particularly useful for preparing REMD simulations where the shape of each replica needs to be the same. When <code>ntp=4</code>, the following variables in the “ewald” namelist should be set: “<code>target_n</code>”: Number of target volume iterations to reach the target volume (default 100). “<code>target_a</code>”, “<code>target_b</code>”, “<code>target_c</code>”: the cell dimension of the target volume.
barostat	Flag used to control which barostat to use in order to control the pressure. = 1 Berendsen (Default) = 2 Monte Carlo barostat
mcbarint	Number of steps between volume change attempts performed as part of the Monte Carlo barostat. Default is 100.
pres0	Reference pressure (in units of bars, where 1 bar $\approx$ 0.987 atm) at which the system is maintained (when <code>NTP > 0</code>). Default 1.0.
comp	compressibility of the system when <code>NTP > 0</code>. The units are in $1.0 \times 10^{-6} \text{ bar}^{-1}$; a value of 44.6 (default) is appropriate for water.
taup	Pressure relaxation time (in ps), when <code>NTP > 0</code>. The recommended value is between 1.0 and 5.0 psec. Default value is 1.0, but larger values may sometimes be necessary (if your trajec)[323]tories seem unstable).
baroscalingdir	Flag for pressure scaling direction control. Applicable when using Monte Carlo barostat (<code>barostat = 2</code>) with anisotropic pressure scaling (<code>ntp = 2</code>). = 0 box size scales randomly (x, y or z) each scaling step (default) = 1 box scales only along x-direction, dimensions along y-, z-axes are fixed = 2 box scales only along y-direction, dimensions along x-, z-axes are fixed = 3 box scales only along z-direction, dimensions along x-, y-axes are fixed

Surface tension regulation

Constant surface tension is used in statistical ensembles for simulating liquid interfaces. This is primarily intended for lipid membrane simulations with two or more interfaces. Constant surface tension is only available for simulations with anisotropic pressure or semiisotropic scaling. This algorithm is an extension to the Berendsen pressure scaling algorithm that adjusts the tangential pressure evaluation in order to maintain a “constant” surface tension.[503] Since the surface tension is a function of the pressure tensor, fluctuations of the surface tension will be large.

In order to use constant surface tension, periodic boundary conditions ($ntb = 2$), anisotropic or semiisotropic pressure scaling ($ntp = 2$ or $ntp = 3$), and an orthogonal box must be used.

csurften Flag for constant surface tension dynamics.

- = 0** No constant surface tension (default)
- = 1** Constant surface tension with interfaces in the yz plane
- = 2** Constant surface tension with interfaces in the xz plane
- = 3** Constant surface tension with interfaces in the xy plane

gamma.ten Surface tension value in units of dyne/cm. Default value is 0.0 dyne/cm.

ninterface Number of interfaces in the periodic box. There must be at least two interfaces in the periodic box. Two interfaces is appropriate for a lipid bilayer system and is the default value.

22.6.9. SHAKE bond length constraints

ntc Flag for SHAKE to perform bond length constraints.[504] (See also NTF in the **Potential function** section. In particular, typically $NTF = NTC$.) The SHAKE option should be used for most MD calculations. The size of the MD timestep is determined by the fastest motions in the system. SHAKE removes the bond stretching freedom, which is the fastest motion, and consequently allows a larger timestep to be used. For water models, a special "three-point" algorithm is used.[505] Consequently, to employ TIP3P set $NTF = NTC = 2$.

Since SHAKE is an algorithm based on dynamics, the minimizer is not aware of what SHAKE is doing; for this reason, minimizations generally should be carried out without SHAKE. One exception is short minimizations whose purpose is to remove bad contacts before dynamics can begin.

For parallel versions of *sander* only intramolecular atoms can be constrained. Thus, such atoms must be in the same chain of the originating PDB file.

- = 1** SHAKE is not performed (default)
- = 2** bonds involving hydrogen are constrained
- = 3** all bonds are constrained (not available for parallel or qmmm runs in *sander*)

tol Relative geometrical tolerance for coordinate resetting in shake. Recommended maximum: < 0.00005 Angstrom Default 0.00001.

jfastw Fast water definition flag. By default, the system is searched for water residues, and special routines are used to SHAKE these systems.[505]

- = 0** Normal operation. Waters are identified by the default names (given below), unless they are redefined, as described below.
- = 4** Do not use the fast SHAKE routines for waters.

The following variables allow redefinition of the default residue and atom names used by the program to determine which residues are waters.

WATNAM The residue name the program expects for water. Default 'WAT'.

OWTNM The atom name the program expects for the oxygen of water. Default 'O'.

HWTNM1 The atom name the program expects for the 1st H of water. Default 'H1'.

HWTNM2 The atom name the program expects for the 2nd H of water. Default 'H2'.

noshakemask String that specifies atoms that are not to be shaken (assuming that $ntc > 1$). Any bond that would otherwise be shaken by virtue of the *ntc* flag, but which involves an atom flagged here, will *not* be shaken. The syntax for this string is given in Chap. 13.5. Default is an empty string, which matches nothing. A typical use would be to remove SHAKE constraints from all or part of a solute, while still shaking rigid water models like TIPnP or SPC/E. Another use would be to turn off SHAKE constraints for the parts of the system that are being changed with thermodynamic integration, or which are the EVB or quantum regions of the system.

If this option is invoked, then all parts of the potential must be evaluated, that is, *ntf* must be one. The code enforces this by setting *ntf* to 1 when a *noshakemask* string is present in the input.

If you want the *noshakemask* to apply to all or part of the water molecules, you must also set *jfastw=4*, to turn off the special code for water SHAKE. (If you are not shaking waters, you presumably also want to issue the "set default FlexibleWater on" command in LEaP; see that chapter for more information.)

22.6.10. The “middle” scheme

22.6.10.1. Introduction

The “middle” scheme offers a unified framework to develop efficient thermostating algorithms for configurational sampling for the canonical ensemble, as described in Refs. [506–511]. It can be implemented for performing molecular dynamics (MD) or path integral molecular dynamics (PIMD), either with or without holonomic constraints. The “middle” scheme allows the use of much larger time intervals (i.e., timestep sizes) Δt to maintain the same accuracy, which significantly improves the configurational sampling efficiency. That is, it is efficient for calculating structural properties and thermodynamic observables that depend on coordinate variables. Most thermostats control the temperature by updating momenta of the system. Some prevailing thermostats include stochastic ones (such as the Andersen thermostat and Langevin dynamics) and deterministic ones (such as the Nosé-Hoover thermostat and Nosé-Hoover chain). In the “middle” scheme, immediately after the coordinate-updating step for half a time interval, the thermostat process for a full time interval takes place, which is then followed by the coordinate-updating step for another half time interval [506, 510].

Here we present a brief introduction to the “middle” scheme. For many thermostats, the integration in one time step Δt can be splitted into three parts, the steps for updating coordinates, momenta and thermostat, denoted as “x”, “p” and “T”, respectively. In this case the “equation of motion” may be expressed as

$$\begin{bmatrix} d\mathbf{x}_t \\ d\mathbf{p}_t \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{M}^{-1}\mathbf{p}_t \\ 0 \end{bmatrix}}_{\mathbf{x}} dt + \underbrace{\begin{bmatrix} 0 \\ -\nabla_{\mathbf{x}_t} U(\mathbf{x}_t) \end{bmatrix}}_{\mathbf{p}} dt + \underbrace{[\text{thermostat}]}_{\mathbf{T}} \quad (22.1)$$

Here, U is the potential energy, $\mathbf{M}$ is the diagonal mass matrix, $\mathbf{x}$ and $\mathbf{p}$ are the vectors of coordinate and momentum, respectively. Equation (22.1) is, however, not convenient to do the analysis.

A more useful approach is to employ the forward Kolmogorov equation to express the evolution of the density distribution in the phase space $\rho(\mathbf{x}, \mathbf{p})$.

$$\begin{aligned} \frac{\partial}{\partial t} \rho &= \mathcal{L} \rho \\ &= (\mathcal{L}_x + \mathcal{L}_p + \mathcal{L}_T) \rho \end{aligned} \quad (22.2)$$

The relevant Kolmogorov operators for the 1st and 2nd terms of the right-hand side (RHS) are

$$\mathcal{L}_x \rho = -\mathbf{p}^T \mathbf{M}^{-1} \nabla_{\mathbf{x}} \rho \quad (22.3)$$

$$\mathcal{L}_p \rho = \nabla_{\mathbf{x}} U(\mathbf{x}) \cdot \nabla_{\mathbf{p}} \rho \quad (22.4)$$

respectively. The definition of $\mathcal{L}_T$ depends on the specific thermostat. The phase space propagators for a time interval Δt for the three parts are $e^{\mathcal{L}_x \Delta t}$, $e^{\mathcal{L}_p \Delta t}$, and $e^{\mathcal{L}_T \Delta t}$, respectively.

The propagation in each time step with the velocity Verlet (VV) algorithm is performed as

$$e^{\mathcal{L} \Delta t} \approx e^{\mathcal{L}_{\text{middle}}^{\text{VV}} \Delta t} = e^{\mathcal{L}_p \Delta t / 2} e^{\mathcal{L}_x \Delta t / 2} e^{\mathcal{L}_T \Delta t} e^{\mathcal{L}_x \Delta t / 2} e^{\mathcal{L}_p \Delta t / 2} \quad (22.5)$$

The phase space propagator for the thermostat part $e^{\mathcal{L}_T \Delta t}$ is designed in the middle. Equation (22.5) is denoted as “VVMiddle”. The numerical algorithm reads

$$\begin{aligned} \text{Update Momenta for half a step: } & \mathbf{p} \leftarrow \mathbf{p} - \frac{\partial U}{\partial \mathbf{x}} \frac{\Delta t}{2} \\ \text{Update Coordinates for half a step: } & \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2} \\ \text{Thermostat for a full time step: } & \text{thermostat_step} \\ \text{Update Coordinates for another half step: } & \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2} \\ \text{Update Momenta for another half step: } & \mathbf{p} \leftarrow \mathbf{p} - \frac{\partial U}{\partial \mathbf{x}} \frac{\Delta t}{2} \end{aligned} \quad (22.6)$$

where *thermostat_step* is the subroutine for the thermostat process, which is determined according to the thermostat method of choice.

The stationary state distribution of “VVMiddle” for a harmonic system $U(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{x}_{\text{eq}})^T \mathbf{A}(\mathbf{x} - \mathbf{x}_{\text{eq}})$ is

$$\begin{aligned} \rho_{\text{middle}}^{\text{VV}}(\mathbf{x}, \mathbf{p}) = \frac{1}{Z_N} \exp \left\{ -\beta \left[\frac{1}{2} \mathbf{p}^T \left(\mathbf{M} - \mathbf{A} \frac{\Delta t^2}{4} \right)^{-1} \mathbf{p} \right. \right. \\ \left. \left. + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{\text{eq}})^T \mathbf{A} (\mathbf{x} - \mathbf{x}_{\text{eq}}) \right] \right\} \end{aligned} \quad (22.7)$$

as long as the thermostat process keeps the Maxwell (or Maxwell-Boltzmann) momentum distribution unchanged, i.e.

$$e^{\mathcal{L}_T \Delta t} \rho_{\text{MB}}(\mathbf{p}) = \rho_{\text{MB}}(\mathbf{p}) \quad (22.8)$$

where the Maxwell momentum distribution is

$$\rho_{\text{MB}}(\mathbf{p}) = \left(\frac{\beta}{2\pi} \right)^{3N/2} |\mathbf{M}|^{-1/2} \exp \left[-\beta \left(\frac{1}{2} \mathbf{p}^T \mathbf{M}^{-1} \mathbf{p} \right) \right] \quad (22.9)$$

Here $\beta = \frac{1}{k_B T}$ with k_B as the Boltzmann constant, T is the temperature of the system. (N is the number of particles.) It is then easy to verify that the marginal distribution of coordinates for “VVMiddle” is exact in the harmonic limit. Many types of thermostats satisfy the criteria (thermostat process keeps the Maxwell momentum distribution unchanged, Equation (22.8)), which include, but not limited to, the thermostats listed below.

- Andersen thermostat (real dynamics case)

In this thermostat, each particle of the system stochastically collides with a fictitious heat bath, and once the collision occurs, the momentum of this particle is chosen afresh from the Maxwell-Boltzmann momentum distribution. The explicit form for the thermostat process can be expressed as

$$\begin{aligned} \mathbf{p}^{(j)} & \leftarrow \sqrt{\frac{m_j}{\beta}} \boldsymbol{\theta}_j, \quad (j = \overline{1, N}) \\ \text{if } \mu_j & < v \Delta t \quad (\text{or more precisely } \mu_j < 1 - e^{-v \Delta t}) \end{aligned} \quad (22.10)$$

Here v is the collision frequency, $\boldsymbol{\theta}_j$ is a vector of independent Gaussian-distributed random numbers with

zero mean and unit variance, m_j the mass for the j th atom, μ_j is a uniformly distributed random number in the range (0,1). Here μ_j may be different for each particle ($j = \overline{1, N}$). In the current version of AMBER μ_j is the same for all particles. (I.e., the global Andersen thermostat is employed.)

The phase space propagator for the thermostat process is

$$e^{\mathcal{L}_T \Delta t} \rho = e^{-\nu \Delta t} \rho(\mathbf{x}, \mathbf{p}) + (1 - e^{-\nu \Delta t}) \rho_{\text{MB}}(\mathbf{p}) \int_{-\infty}^{\infty} \rho(\mathbf{x}, \mathbf{p}) d\mathbf{p} \quad (22.11)$$

- Andersen thermostat (virtual dynamics case)

The explicit form for the virtual dynamics case of the Andersen thermostat is expressed as

$$\left. \begin{aligned} \mathbf{p}^{(j)} &\leftarrow \sqrt{\frac{m_j}{\beta}} \boldsymbol{\theta}_j, \text{ if } \mu_j < 1 - e^{-\nu \Delta t} \\ \mathbf{p}^{(j)} &\leftarrow -\mathbf{p}^{(j)}, \text{ otherwise} \end{aligned} \right\} (j = \overline{1, N}) \quad (22.12)$$

The phase space propagator for the thermostat process is

$$e^{\mathcal{L}_T \Delta t} \rho = e^{-\nu \Delta t} \rho(\mathbf{x}, -\mathbf{p}) + (1 - e^{-\nu \Delta t}) \rho_{\text{MB}}(\mathbf{p}) \int_{-\infty}^{\infty} \rho(\mathbf{x}, \mathbf{p}) d\mathbf{p} \quad (22.13)$$

- Langevin dynamics (real dynamics case)

The thermostat process is the solution to the Ornstein-Uhlenbeck (OU) process

$$\mathbf{p} \leftarrow e^{-\boldsymbol{\gamma} \Delta t} \mathbf{p} + \sqrt{\frac{1}{\beta}} \mathbf{M}^{1/2} (\mathbf{1} - e^{-2\boldsymbol{\gamma} \Delta t})^{1/2} \boldsymbol{\eta} \quad (22.14)$$

Here, $\boldsymbol{\eta}$ is a vector of independent Gaussian-distributed random numbers with zero mean and unit variance, $\boldsymbol{\gamma}$ is the diagonal friction coefficient matrix. In the current version of AMBER all diagonal elements of $\boldsymbol{\gamma}$ are set to be the same. (That is, the friction coefficient is the same for all particles. The global Langevin thermostat is used.)

The phase space propagator for the thermostat process is

$$\begin{aligned} e^{\mathcal{L}_T \Delta t} \rho &= \left(\frac{\beta}{2\pi} \right)^{3N/2} |\mathbf{M}(\mathbf{1} - e^{-2\boldsymbol{\gamma} \Delta t})|^{-1/2} \\ &\cdot \int d\mathbf{p}_0 \rho(\mathbf{x}, \mathbf{p}_0) \exp \left[-\frac{\beta}{2} (\mathbf{p} - e^{-\boldsymbol{\gamma} \Delta t} \mathbf{p}_0)^T \right. \\ &\quad \left. \cdot \mathbf{M}^{-1} (\mathbf{1} - e^{-2\boldsymbol{\gamma} \Delta t})^{-1} (\mathbf{p} - e^{-\boldsymbol{\gamma} \Delta t} \mathbf{p}_0) \right] \end{aligned} \quad (22.15)$$

- Langevin dynamics (virtual dynamics case)

The virtual dynamics case represents another type of discrete evolution that may not correspond to a continuous, real dynamical counterpart of the Langevin equation.

$$\mathbf{p} \leftarrow -e^{-\boldsymbol{\gamma} \Delta t} \mathbf{p} + \sqrt{\frac{1}{\beta}} \mathbf{M}^{1/2} (\mathbf{1} - e^{-2\boldsymbol{\gamma} \Delta t})^{1/2} \boldsymbol{\eta} \quad (22.16)$$

The virtual dynamics case is also able to produce the desired stationary distribution. The phase space prop-

agator for the thermostat process is then

$$\begin{aligned}
 e^{\mathcal{L}_T \Delta t} \rho = & \left(\frac{\beta}{2\pi} \right)^{3N/2} |\mathbf{M}(\mathbf{1} - e^{-2\gamma \Delta t})|^{-1/2} \\
 & \cdot \int d\mathbf{p}_0 \rho(\mathbf{x}, \mathbf{p}_0) \exp \left[-\frac{\beta}{2} (\mathbf{p} + e^{-\gamma \Delta t} \mathbf{p}_0)^T \right. \\
 & \left. \cdot \mathbf{M}^{-1} (\mathbf{1} - e^{-2\gamma \Delta t})^{-1} (\mathbf{p} + e^{-\gamma \Delta t} \mathbf{p}_0) \right]
 \end{aligned} \tag{22.17}$$

- Nosé-Hoover (NH) thermostat and Nosé-Hoover chain (NHC)
See Ref. [506] for more detailed discussions.

The “middle” scheme of a thermostat includes both real and virtual dynamics cases. (See Refs. [508, 509].) It is proved in Ref. [508] that, while the Langevin equation algorithm (BAOAB) proposed in Ref. [512] is simply only the real dynamics case of “VVMiddle”, another Langevin equation algorithm proposed (without employing the Lie-Trotter splitting) in Ref. [513] is equivalent to “VVMiddle” for Langevin dynamics. The real dynamics case for the Andersen thermostat and that for Langevin dynamics have been implemented in the current version of AMBER.

When the leapfrog algorithm, rather than the velocity-Verlet algorithm, is employed in the “middle” scheme, it is denoted as “LFMiddle”[510]. The propagation in each time step with the leapfrog (LF) algorithm is performed as

$$e^{\mathcal{L} \Delta t} \approx e^{\mathcal{L}_{\text{middle}}^{\text{LF}} \Delta t} = e^{\mathcal{L}_x \Delta t / 2} e^{\mathcal{L}_T \Delta t} e^{\mathcal{L}_x \Delta t / 2} e^{\mathcal{L}_p \Delta t} \tag{22.18}$$

The numerical algorithm of “LFMiddle” reads

$$\begin{aligned}
 \text{Update Momenta for a full time step: } & \mathbf{p} \leftarrow \mathbf{p} - \frac{\partial U}{\partial \mathbf{x}} \Delta t \\
 \text{Update Coordinates for half a step: } & \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2} \\
 \text{Thermostat for a full time step: } & \text{thermostat_step} \\
 \text{Update Coordinates for another half step: } & \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2}
 \end{aligned} \tag{22.19}$$

For any general systems, “LFMiddle” shares the same accuracy as “VVMiddle” for sampling the marginal distribution of coordinates. In addition, “LFMiddle” leads to the exact marginal distribution of momenta in the harmonic limit[510]. For simplicity and compatibility, only “LFMiddle” is integrated into AMBER.

The “middle” scheme with holonomic constraints (such as bond length constraints) is also implemented. While MD with holonomic constraints are widely used in biological simulations, PIMD with holonomic constraints may help understand nuclear quantum effects of different motions in molecular systems. For instance, help assign spectral features as shown in Ref. [514]. In the “middle” scheme, when holonomic constraints are applied, the SHAKE [504] and RATTLE [515] algorithms are used for fixing coordinates and velocities, respectively. Particularly for the molecular system that contains water molecules, the analytical SETTLE algorithm [505] may be used to apply the constraint to the water molecule.

The full “LFMiddle” with holonomic constraints [511] reads

$$\begin{aligned}
 &\text{Update Momenta for a full time step: } \mathbf{p} \leftarrow \mathbf{p} - \frac{\partial U}{\partial \mathbf{x}} \Delta t \\
 &\quad \text{Solve velocity constraints: RATTLE} \\
 &\text{Update Coordinates for half a step: } \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2} \\
 &\quad \text{Thermostat for a full time step: } thermostat_step \\
 &\text{Update Coordinates for another half step: } \mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \mathbf{p} \frac{\Delta t}{2} \\
 &\quad \text{Solve coordinate constraints: SHAKE} \\
 &\quad \text{Solve velocity constraints: RATTLE}
 \end{aligned} \tag{22.20}$$

Refs. [506–511, 516] show theory and applications of “middle” scheme.

While the “middle” scheme for PIMD with the staging transformation (staging PIMD) was first demonstrated in Ref. [507], that for PIMD with the normal-mode transformation (normal-mode PIMD) was first proposed in the supplemental material of Ref. [507] in 2016, which can be found via the URLs provided by the publisher:

- https://aip.scitation.org/doi/suppl/10.1063/1.4954990/suppl_file/supplemental+material-submitted.docx
- ftp://ftp.aip.org/epaps/journ_chem_phys/E-JCPSA6-145-007626

In addition, the arXiv preprint (that includes Ref. [507] and its supplemental material) is also available (<https://arxiv.org/ftp/arxiv/papers/1611/1611.06331.pdf>). In the current version, the “middle” scheme is implemented for the primitive version of PIMD (PRIMPIMD) of AMBER. The staging PIMD or normal-mode PIMD algorithms in the “middle” scheme will also be implemented into AMBER soon.

22.6.10.2. Input parameters

In order to perform MD or PRIMPIMD simulations with the “middle” scheme, three additional flags should be added in the *mdin* file, which are used to distinguish different methods.

- ischeme** Flag for choosing an integration scheme for molecular dynamics.
=0 (default) conventional scheme in AMBER.
=1 “middle” scheme based on the leapfrog algorithm.
- ithermostat** Flag for different thermostats when the “middle” scheme is employed. Two types of thermostats are currently available.
=1 Langevin dynamics.
=2 Andersen thermostat.
- therm_par** The parameter used in a thermostating method of the “middle” scheme, in the unit of ps^{-1} , which should always be a positive number. It refers to the friction coefficient for Langevin dynamics (*ithermostat* = 1) or the collision frequency for the Andersen thermostat (*ithermostat* = 2).

The recommended value for *therm_par* is related to the characteristic frequency ($\tilde{\omega}$) of the specific system. The characteristic time of the potential energy autocorrelation function is

$$\tau_{UU} = \int_0^\infty \frac{\langle U(0)U(t) \rangle - \langle U \rangle^2}{\langle U^2 \rangle - \langle U \rangle^2} dt \tag{22.21}$$

The optimal value of the thermostat parameter that produces the minimum correlation time of the potential is $\xi^{opt} \approx \tilde{\omega}$ for Langevin dynamics and $\xi^{opt} \approx \sqrt{2}\tilde{\omega}$ for the Andersen thermostat, as the time interval Δt approaches

zero. E.g. for a HO molecule, the frequency of the O-H stretch is around 3600 cm^{-1} (680 ps^{-1}), so one can choose 680 ps^{-1} as the value of *therm_par* when Langevin dynamics is used, or 960 ps^{-1} when the Andersen thermostat is employed. When the time interval Δt is finite in the two thermostating methods, while the characteristic correlation time goes to infinity as the thermostat parameter approaches zero, the characteristic correlation time gradually reaches a plateau as the thermostat parameter increases. (Please see Refs. [508–510] for more discussions.) When condensed phase systems are simulated, it is not straightforward to estimate the optimal thermostat parameter(s) that could be related to the mixing of frequencies (or time scales) of the system [491]. Some numerical tests are necessary for obtaining the reasonable region for the thermostat parameter such that the characteristic time divided by the time interval is relatively small. (This is true not only for the “middle” scheme but for all thermostat algorithms.) For a liquid water system (216 water molecules in a cell with periodic boundary conditions) with no holonomic constraints, the thermostat parameter is usually chosen to be $2 - 50\text{ ps}^{-1}$.

22.6.10.3. Middle scheme using pmemd and pmemd.cuda

The Langevin thermostat in the “middle” scheme is available for pmemd and pmemd.cuda. (Other thermostat types will be integrated later.) One can run both serial and parallel jobs for the “middle” scheme using pmemd. In pmemd.MPI simulations, flag *midpoint* = 1 should be used for a more reasonable CPU parallelism. The GPU-accelerated “middle” scheme can be performed for the serial job with a single GPU with pmemd.cuda_DPFP or pmemd.cuda_SPFP. (The multi-GPU support for the “middle” scheme will be integrated later.)

The “middle” scheme in pmemd or pmemd.cuda supports classical MD simulations with or without holonomic constraints. The SHAKE [504] and RATTLE [515] algorithms are available in pmemd and pmemd.cuda. The analytical SETTLE algorithm [505] is applied by default for the water molecule with constraints.

22.6.10.4. Examples

Examples include a liquid water system (216 water molecules in a cell with the periodic boundary conditions) with the q-SPC/Fw model, an alanine dipeptide (ACE-ALA-NME) solved in a box with 401 methol molecules, and a peptide chain ACE-ALA-ALA-ALA-NME in vacuum. One is also encouraged to check the test cases in *\$AMBERHOME/test/middle-scheme*. In AMBER the analytical SETTLE algorithm is the default (*jfastw*=0) for applying the constraints for the water molecule. (Note that in some liquid water models, the intramolecular H-H is specified as a bond in the topology file, so all the intramolecular O-H and H-H distances are constrained when *ntc*=2 is employed in AMBER.)

Examples performed with pmemd and pmemd.cuda are all Classical MD simulations. Three typical examples are a liquid water system (4096 water molecules in a cell with the periodic boundary conditions), a DNA $A_7 - T_7$ duplex in solution and a Deep Eutectic Solvent system (1:2 ratio choline chloride and ethylene glycol).

Molecular dynamics (for classical statistics)

(1) MD input using the Langevin thermostat with the “LFMiddle” scheme for liquid water.

Test: *\$AMBERHOME/test/middle-scheme/MD_Unconstr_Langevin_water*

```
MD: NVT simulation of liquid water
&cntrl
ipimd = 0, nstlim = 10      ! MD for 10 steps
ntx = 1, irest = 0         ! read coordinates
temp0 = 300, tempi = 300    ! temperature: target and initial
dt = 0.001                 ! time step in ps
cut = 7.0                  ! non-bond cut off
ischeme = 1                !! leapfrog middle scheme
ithermostat = 1            !! Langevin thermostat
therm_par = 5.0            !! thermostat parameter in 1/ps
ig = 1000                  ! random seed
ntc = 1, ntf = 1           ! no constraints
ntpr = 1, ntwr = 5, ntwx = 5 ! output settings
/
```


One can run either a serial job (using *sander*):

```
$ sander -O -i md_LGV.in -p qspcfw216.top -c nvt.rst -o md_LGV.out \
-r lgv.rst -info lgv.info
```

or a parallel job (using *sander.MPI*):

```
$ mpirun -np 4 sander.MPI -O -i md_LGV.in -p qspcfw216.top -c nvt.rst \
-o md_LGV.out -r lgv.rst -info lgv.info
```

(2) MD input using Langevin dynamics with the “LFMiddle” scheme for the liquid water. Lengths of the bonds having hydrogen atoms are constrained.

Test: \$AMBERHOME/test/middle-scheme/MD_Constr_Langevin_water

```
MD: NVT simulation of liquid water
&cntrl
ipimd = 0, nstlim = 10      ! MD for 10 steps
ntx = 1, irest = 0         ! read coordinates
temp0 = 300, tempi = 300    ! temperature: target and initial
dt = 0.004                 ! time step in ps
cut = 7.0                  ! non-bond cut off
ischeme = 1,               !! leapfrog middle scheme
ithermostat = 1,           !! Langevin thermostat, random seed is default value
therm_par = 5.0            !! thermostat parameter, in 1/ps
ntc = 2, ntf = 2           ! constrain lengths of the bonds having hydrogen atoms
ntpr = 1, ntwr = 5, ntwx = 5 ! output settings
/
```

Run either a serial way (using *sander*):

```
$ sander -O -i md_LGV.in -p qspcfw216.top -c nvt.rst -o md_LGV.out \
-r lgv.rst -info lgv.info
```

or a parallel job (using *sander.MPI*):

```
$ mpirun -np 4 sander.MPI -O -i md_LGV.in -p qspcfw216.top -c nvt.rst \
-o md_LGV.out -r lgv.rst -info lgv.info
```

Path integral molecular dynamics (for quantum statistics)

(1) PRIMPIMD input using the Andersen thermostat with the “LFMiddle” scheme for the liquid water. Lengths of the bonds having hydrogen atoms are constrained.

Test: \$AMBERHOME/test/middle-scheme/PIMD_Constr_Andersen_water

```
PRIMPIMD: NVT simulation of liquid water
&cntrl
ipimd = 1, nstlim = 10     ! PRIMPIMD for 10 steps
ntx = 5, irest = 0         ! read coordinates
temp0 = 300, tempi = 300    ! target temperature and initial temperature
dt = 0.002                 ! time step in ps
cut = 7.0                  ! non-bond cut-off
ischeme = 1,               !! leapfrog middle scheme
ithermostat = 2,           !! Andersen thermostat
therm_par = 8.0            !! thermostat parameter, in 1/ps
ig = 777                   ! random seed
ntc = 2, ntf = 2           ! constrain lengths of the bonds having hydrogen atoms
ntpr=1, ntwr=5, ntwx=5     ! output settings
/
```

(2) PRIMPIMD input using Langevin dynamics with the “LFMiddle” scheme for the liquid water. No constraints are applied.

Test: \$AMBERHOME/test/middle-scheme/PIMD_Langevin_water

```
PRIMPIMD: NVT simulation of liquid water
&cntrl
ipimd = 1, nstlim = 10      ! PRIMPIMD for 10 steps
ntx = 5, irest = 0         ! read coordinates,
                           ! and run as a new simulation.
temp0 = 300, tempi = 300    ! target and initial temperature
dt = 0.001                 ! time step, in ps
cut = 7.0                  ! non-bond cut off
ischeme = 1,               !! leapfrog middle scheme
ithermostat = 1,           !! Langevin thermostat
therm_par = 5.0            !! thermostat parameter, in 1/ps
ntc = 1                    ! no constraints, default
ntpr=1, ntwr=5, ntwx=5     ! output settings
/
```

When one runs PIMD in AMBER, a groupfile is needed. The groupfile *gf_pimd* may look like:

```
-O -i pimd.in -p qspcwf216.top -c nvt1.rst -o bead1.out -r bead1.rst
-x bead1.mdcrd -inf bead1.mdinfo
-O -i pimd.in -p qspcwf216.top -c nvt2.rst -o bead2.out -r bead2.rst
-x bead2.mdcrd -inf bead2.mdinfo
-O -i pimd.in -p qspcwf216.top -c nvt3.rst -o bead3.out -r bead3.rst
-x bead3.mdcrd -inf bead3.mdinfo
-O -i pimd.in -p qspcwf216.top -c nvt4.rst -o bead4.out -r bead4.rst
-x bead4.mdcrd -inf bead4.mdinfo
```

Note that each line starts with “-O” and ends with “-inf <info>”. The groupfile above contains 4 lines, which means 4 path integral beads are used.

sander.MPI is executed via the following command:

```
$ mpirun -np 8 sander.MPI -ng 4 -groupfile gf_pimd
```

The number of processes (8) that is specified by “-np” is a multiple of the number of groups (4). In this case 2 CPU processes are used on each path integral bead.

QM/MM molecular dynamics

QM/MM MD input using the Langevin thermostat with the “LFMiddle” scheme for the alanine dipeptide solved in methol box. Lengths of the bonds having hydrogen atoms are constrained for the MM part, while no constraints are applied to the QM part.

Test: \$AMBERHOME/test/middle-scheme/QMMM_Constr_ALA_Methol

```
constrained MD NVT: Alanine dipeptide in meoh (explicit solvent).
&cntrl
ipimd = 0, nstlim = 10,    ! MD for 10 steps
irest = 0, ntx = 1,       ! read coordinates
temp0 = 300               ! target temperature
tempi = 300               ! initial temperature
dt = 0.002,               ! time step, in ps
cut = 8,                  ! non-bond cut off
ig = 6666,                ! random seed for reproducing results
ischeme = 1,              !! leapfrog middle scheme
ithermostat = 1           !! Langevin thermostat
therm_par = 5.0,          !! thermostat parameter, in 1/ps
```

```

ntc=2,ntf=2           ! constrain lengths of bonds having hydrogen atoms
ntpr=1, ntwr=1, ntwx=1 ! output settings
ifqnt=1              ! switch on QM/MM coupled potential
/
&qmmm qmmask=':ACE,ALA,NME', ! residues treated using QM
qmcharge=0,          ! charge on QM region is 0
qmshake=0,           ! no SHAKE for QM region
qm_theory='PM3',      ! use the PM3 semi-empirical Hamiltonian
qmcut=8.0            ! use 8 angstrom cut off for QM region
/

```

One can run either a serial job (using *sander*):

```

$ sander -O -i qmmm.in -p ala.top -c ala.crd -o qmmm.out -r qmmm.rst \
-x qmmm.crd -info qmmm.info

```

or a parallel job (using *sander.MPI*):

```

$ mpirun -np 4 sander.MPI -O -i qmmm.in -p ala.top -c ala.crd \
-o qmmm.out -r qmmm.rst -x qmmm.crd -info qmmm.info

```

Replica exchange molecular dynamics

REMD input using the Langevin thermostat with the “LFMiddle” scheme for the ACE-ALA-ALA-ALA-NME in vacuum. Lengths of the bonds having hydrogen atoms are constrained.

Test: \$AMBERHOME/test/middle-scheme/REMD_Constr_ALA

Below is the input file for one of the replicas. The target temperatures are 300, 325, 350, and 400K for the 4 replicas, respectively.

```

REMD test with 4 replicas
&cntrl
imin = 0, nstlim = 100,      ! MD for 100 steps
irest=1, ntx = 5,           ! read coordinates and velocities
tempi = 0.0, temp0 = 300.0, ! initial and target temperature
ischeme= 1,                 !! leapfrog middle scheme
ithermostat = 1,            !! Langevin thermostat
therm_par = 1.0,            !! thermostat parameter, in 1/ps
dt = 0.002,                 ! time step, in ps
ig=6666,                    ! random seed
ntc = 2, ntf = 2,           ! constrain lengths of the bonds having hydrogen atoms
ntwx = 50, ntwr = 50, ntpr = 50, ! output setting
ntb=0,                       ! no periodicity
cut = 99.0,                  ! non bond cut off
numexchg=5,                  ! exchange frequency
&end

```

When one runs REMD in AMBER, a groupfile is needed. The groupfile *groupfile* may look like:

```

-O -rem 1 -remlog rem.log -i rem.in.000 -p ala3.top -c mdrestrt -o rem.out.000
-r rem.r.000 -inf reminfo.000
-O -rem 1 -remlog rem.log -i rem.in.001 -p ala3.top -c mdrestrt -o rem.out.001
-r rem.r.001 -inf reminfo.001
-O -rem 1 -remlog rem.log -i rem.in.002 -p ala3.top -c mdrestrt -o rem.out.002
-r rem.r.002 -inf reminfo.002
-O -rem 1 -remlog rem.log -i rem.in.003 -p ala3.top -c mdrestrt -o rem.out.003
-r rem.r.003 -inf reminfo.003

```

Note that each line starts with “-O” and ends with “-inf <info>”. The groupfile has 4 lines, which means 4 replicas are employed in REMD.

sander.MPI is executed via the following command:

```
$ mpirun -np 4 sander.MPI -ng 4 -groupfile groupfile
```

The number of processes (4) that is specified by “-np” can be replaced by any multiple of the number of replicas used in REMD (4 in this case).

pmemd/pmemd.cuda examples

(1) *Liquid water with 4096 water molecules in a cell*

Test: \$AMBERHOME/test/middle-scheme/4096wat

```
MD: NVT simulation of liquid water
&cntrl
ntx = 5, irest = 1,          ! read coordinates
ntc = 2, ntf = 2,          ! constrain lengths of bonds
tol = 0.0000001,          ! having hydrogen atoms
nstlim = 10,              ! MD for 10 steps
ntpr = 1, ntwr = 10000     ! output settings
dt = 0.001,               ! timestep in ps
ig = 71277,               ! random seed
cut = 7.0,                ! non-bond cut off
ischeme = 1,              !! Leapfrog middle scheme
ithermostat = 1,          !! Langevin thermostat
therm_par = 5.0,          !! thermostat parameter
midpoint = 1              ! use midpoint method, only for pmemd.MPI;
                           ! remove this flag otherwise
/
```

(2) *DNA A₇ – T₇ duplex solvated in a cell with 3351 water molecules and 12 sodium ions as counter-ions*

Test: \$AMBERHOME/test/middle-scheme/DNA7

```
MD: NVT simulation of DNA duplex
&cntrl
ntx = 5, irest = 1,          ! read coordinates
ntc = 2, ntf = 2,          ! constrain lengths of bonds
tol = 0.0000001,          ! having hydrogen atoms
nstlim = 10,              ! MD for 10 steps
ntpr = 1, ntwr = 10000     ! output settings
dt = 0.001,               ! timestep in ps
ig = 71277,               ! random seed
cut = 9.0,                ! non-bond cut off
ischeme = 1,              !! Leapfrog middle scheme
ithermostat = 1,          !! Langevin thermostat
therm_par = 5.0,          !! thermostat parameter
midpoint = 1              ! use midpoint method, only for pmemd.MPI;
                           ! remove this flag otherwise
/
```

(3) *Ethaline Deep Eutectic Solvent(512 choline chloride and 1024 ethylene glycol molecules)*

Test: \$AMBERHOME/test/middle-scheme/ETH

```
MD: NVT simulation of Ethaline Deep Eutectic Solvent
&cntrl
ntx = 5, irest = 1,          ! read coordinates
ntc = 2, ntf = 2,          ! constrain lengths of bonds
tol = 0.0000001,          ! having hydrogen atoms
nstlim = 10,              ! MD for 10 steps
ntpr = 1, ntwr = 10000     ! output settings
dt = 0.001,               ! timestep in ps
```

```

ig = 71277,          ! random seed
cut = 10.0,          ! non-bond cut off
ischeme = 1,         !! Leapfrog middle scheme
ithermostat = 1,     !! Langevin thermostat
therm_par = 5.0,     !! thermostat parameter
midpoint = 1         ! use midpoint method, only for pmemd.MPI;
                    ! remove this flag otherwise
/

```

All the three pmemd examples can be executed in a serial job(using sander or pmemd):

```
$ sander -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

or

```
$ pmemd -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

or a parallel job(using sander.MPI or pmemd.MPI):

```
$ sander.MPI -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

or

```
$ pmemd.MPI -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

or a GPU-accelerated job(using pmemd.cuda):

```
$ pmemd.cuda_DPFP -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

```
$ pmemd.cuda_SPFP -O -i mdin -o mdout -p prmtop -c inpcrd -r restrt
```

One is also encouraged to access the tutorial for the “middle” scheme on the webpage

<http://jianliugroup.pku.edu.cn/tutorials.html>

22.6.11. Water cap

ivcap Flag to control cap option. The "cap" refers to a spherical portion of water centered on a point in the solute and restrained by a soft half-harmonic potential. For the best physical realism, this option should be combined with *igb=10*, in order to include the reaction field of waters that are beyond the cap radius.

= 0 Cap will be in effect if it is in the *prmtop* file (default).

= 1 With this option, a cap can be excised from a larger box of water. For this, cutcap (i.e., the radius of the cap), xcap, ycap, and zcap (i.e., the location of the center of the cap) need to be specified in the &cntrl namelist. Note that the cap parameters must be chosen such that the whole solute is covered by solvent. Solvent molecules (and counterions) located outside the cap are ignored. Although this option also works for minimization and dynamics calculations in general, it is intended to post-process snapshots in the realm of MM-PBSA to get a linear-response approximation of the solvation free energy, output as 'Protein-solvent interactions'.

= 2 Cap will be inactivated, even if parameters are present in the *prmtop* file.

= 5 With this option, a shell of water around a solute can be excised from a larger box of water. For this, cutcap (i.e., the thickness of the shell) needs to be specified in the &cntrl namelist. Solvent molecules (and counterions) located outside the cap are ignored. This option only works for a single-step minimization. It is intended to post-process snapshots in the realm of MM-PBSA to get a linear-response approximation of the solvation free energy, output as 'Protein-solvent interactions'.

fcap The force constant for the cap restraint potential.

cutcap Radius of the cap, if *ivcap=1* is used.

xcap, ycap, zcap Location of the cap center, if *ivcap=1* is used.

22.6.12. NMR refinement options

(Users should consult the section NMR refinement to see the context of how the following parameters would be used.)

<code>iscale</code>	Number of additional variables to optimize beyond the 3N structural parameters. (Default = 0). At present, this is only used with residual dipolar coupling and CSA or pseudo-CSA restraints.
<code>noeskp</code>	The NOESY volumes will only be evaluated if $\text{mod}(\text{nstep}, \text{noeskp}) = 0$; otherwise the last computed values for intensities and derivatives will be used. (default = 1, i.e. evaluate volumes at every step)
<code>ipnlty</code>	This parameter determines the functional form of the penalty function for NOESY volume and chemical shift restraints. = 1 the program will minimize the sum of the absolute values of the errors; this is akin to minimizing the crystallographic R-factor (default). = 2 the program will optimize the sum of the squares of the errors. = 3 For NOESY intensities, the penalty will be of the form $\text{awt}[I_c^{1/6} - I_o^{1/6}]^2$. Chemical shift penalties will be as for <code>ipnlty=1</code> .
<code>mxcsub</code>	Maximum number of submolecules that will be used. This is used to determine how much space to allocate for the NOESY calculations. Default 1.
<code>scaln</code>	"Mass" for the additional scaling parameters. Right now they are restricted to all have the same value. The larger this value, the slower these extra variables will respond to their environment. Default 100 amu.
<code>pencut</code>	In the summaries of the constraint deviations, entries will only be made if the penalty for that term is greater than PENCUT. Default 0.1.
<code>tausw</code>	For noesy volume calculations (<code>NMROPT = 2</code>), intensities with mixing times less than TAUSW (in seconds) will be computed using perturbation theory, whereas those greater than TAUSW will use a more exact theory. See the theory section (below) for details. To always use the "exact" intensities and derivatives, set TAUSW = 0.0; to always use perturbation theory, set TAUSW to a value larger than the largest mixing time in the input. Default is TAUSW of 0.1 second, which should work pretty well for most systems.

22.6.13. EMAP restraints

EMAP restraints are used to perform targeted conformational search (TCS)[517]. EMAP uses maps to define restraints to maintain conformations and/or to induce simulation systems to the target conformations. The restraint map can be either obtained from electron microscopy experiments or derived from known protein structures, or defined from initial simulation coordinates. EMAP can be used to do rigid docking of molecules into maps and to do flexible fitting to obtain conformations defined by experimental maps. EMAP can also be used to maintain conformations of protein domains when studying large scale conformational change. One useful application of EMAP restraints is using a map as a boundary for finite systems. A boundary map can be created around the simulation system or read in from a map file. Users should consult the section 31.1 to see how to define EMAP restraints.

<code>iemap</code>	Turn on EMAP restrained simulation when <code>iemap</code> >0. (Default = 0). EMAP restraint information must be input from <code>&emap</code> namelists in the input file.
<code>gammamap</code>	Friction constant for the EMAP restraint maps when allowed to move. (Default=1/ps). (See Section 31.1)

22.7. Potential function parameters

The parameters in this section generally control what sort of force field (or potential function) is used for the simulation.

22.7.1. Generic parameters

ntf Force evaluation. Note: If SHAKE is used (see NTC), it is not necessary to calculate forces for the constrained bonds.

- = 1 complete interaction is calculated (default)
- = 2 bond interactions involving H-atoms omitted (use with NTC=2)
- = 3 all the bond interactions are omitted (use with NTC=3)
- = 4 angle involving H-atoms and all bonds are omitted
- = 5 all bond and angle interactions are omitted
- = 6 dihedrals involving H-atoms and all bonds and all angle interactions are omitted
- = 7 all bond, angle and dihedral interactions are omitted
- = 8 all bond, angle, dihedral and non-bonded interactions are omitted

ntb This variable controls whether or not periodic boundaries are imposed on the system during the calculation of non-bonded interactions. Bonds spanning periodic boundaries are not yet supported. There is no longer any need to set this variable, since it can be determined from *igb* and *ntp* parameters. The “proper” default for *ntb* is chosen (*ntb*=0 when *igb* > 0, *ntb*=2 when *ntp* > 0, and *ntb*=1 otherwise). This behavior can be overridden by supplying an explicit value, although this is discouraged to prevent errors. The allowed values for NTB are

- = 0 no periodicity is applied and PME is off (default when *igb* > 0)
- = 1 constant volume (default when *igb* and *ntp* are both 0, which are their defaults)
- = 2 constant pressure (default when *ntp* > 0)

If NTB is nonzero then there must be a periodic boundary in the topology file. Constant pressure is not used in minimization (IMIN=1, above).

For a periodic system, constant pressure is the only way to equilibrate density if the starting state is not correct. For example, the solvent packing scheme used in LEaP can result in a net void when solvent molecules are subtracted which can aggregate into “vacuum bubbles” in a constant volume run. Another potential problem are small gaps at the edges of the box. The upshot is that almost every system needs to be equilibrated at constant pressure (*ntb*=2, *ntp*>0) to get to a proper density. But be sure to equilibrate first (at constant volume) to something close to the final temperature, before turning on constant pressure.

dielc Dielectric multiplicative constant for the electrostatic interactions. Default is 1.0. Please note this is NOT related to dielectric constants for generalized Born or Poisson-Boltzmann calculations. It should only be used for quasi-vacuum simulations.

cut This is used to specify the nonbonded cutoff, in Angstroms. For PME, the cutoff is used to limit direct space sum, and 8.0 is usually a good value. When *igb*>0, the cutoff is used to truncate nonbonded pairs (on an atom-by-atom basis); here a larger value than the default is generally required. A separate parameter (**RGBMAX**) controls the maximum distance between atom pairs that will be considered in carrying out the pairwise summation involved in calculating the effective Born radii, see the generalized Born section below.
When *igb* > 0, the default is 9999.0 (effectively infinite)
When *igb*=0, the default is 8.0.

fswitch	When off, <i>fswitch</i> ≤ 0, uses a truncation cutoff. When on <i>fswitch</i> > 0, sets a force switching region where the force cutoff smoothly approaches 0 between the region of the fswitch value to the cut value. Force values below the fswitch value follow the standard Lennard-Jones force. Default is -1. This option is not supported for use with GB (i.e., only <i>igb</i> =0 and <i>ntb</i> >0), nor is it compatible with the 12-6-4 and pairwise 12-6-4 Lennard-Jones models (<i>lj1264</i> =1 and <i>plj1264</i> =1). Due to performance regressions (about 20%) with running with the force switching on, it is recommended that simulations run with fswitch off unless using a force field that requires or recommends using the force switch.
nsnb	Determines the frequency of nonbonded list updates when <i>igb</i> =0 and <i>nbflag</i> =0; see the description of <i>nbflag</i> for more information. Default is 25.
ipol	When set to 1, use a polarizable force field. See Section 22.7.5 for more information. Default is 0.
ipgm	When set to 1, use the polarizable Gaussian Multipole force field. See Section 22.8 for more information. Default is 0.
ifqnt	Flag for QM/MM run; if set to 1, you must also include a &qmmm namelist. See Section 6.4 for details on this option. Default is 0.
igb	Flag for using the generalized Born implicit solvent models. See Chapter 4 for information about using this option. Default is 0.
ipb	Flag for using the Poisson-Boltzmann implicit solvent models. See Chapter 6 for information about using this option. Default is 0.
irism	Flag for 3D-reference interaction site model (RISM) molecular solvation method. See Section 8.5 for information about this option. Default is 0.
ievb	If set to 1, use the empirical valence bond method to compute energies and forces. See Section 6.3 for information about this option. Default is 0.
iamoeba	Flag for using the <i>amoeba</i> polarizable potentials of Ren and Ponder.[518, 519] When this option is set to 1, you need to prepare an amoeba namelist with additional parameters. Also, the <i>prmtop</i> file is built in a special way. See Section 33 for more information about this option. Default is 0.
lj1264	In general, you should rarely have to set this variable. When the Lennard-Jones C-coefficient is found in your prmtop file, the default value is set to 1 (meaning it is active). When this flag is <i>not</i> present in the prmtop file, the default value is set to 0 (meaning the 12-6-4 potential [145] is inactive). Setting this to 0 when the C-coefficient is present will forcibly turn off the 12-6-4 potential. Setting <i>lj1264</i> to 1 when no C-coefficient is present will result in a fatal error. Therefore, this flag can be used to quickly disable the r^{-4} term. However, the remaining L-J parameters will still be optimized for the 12-6-4 potential, so this should only be done when testing! It supports <i>sander</i> , <i>pmemd</i> (both the serial and MPI versions) and also <i>pmemd.cuda</i> . However, it is currently only compatible with the Particle Mesh Ewald method for long-range electrostatics. For more information please see Section 3.7. For adding it to your topology file, see Subsection 15.2.2.6 for simple systems (when only one atom type has C_4) and also this tutorial for complex systems (when several atom types have C_4 s).
plj1264	Similar to <i>lj1264</i> above, if not present in the input file, this keyword will still automatically turn to 1 as long as D-coefficient is found in the prmtop file.
efx	This sets the x component of the electric field in kcal/(mol*A*e). Electric fields are naturally off if <i>efx</i> , <i>efy</i> , <i>efz</i> are 0. Default value is 0. It currently only supports <i>pmemd</i> (both the serial and MPI versions).

<code>efy</code>	This sets the y component of the electric field in kcal/(mol*Å*e). Electric fields are naturally off if <code>efx</code> , <code>efy</code> , <code>efz</code> are 0. Default value is 0. It currently only supports <i>pmemd</i> (both the serial and MPI versions).
<code>efz</code>	This sets the z component of the electric field in kcal/(mol*Å*e). Electric fields are naturally off if <code>efx</code> , <code>efy</code> , <code>efz</code> are 0. Default value is 0. It currently only supports <i>pmemd</i> (both the serial and MPI versions).
<code>efn</code>	If <code>efn</code> is on (<code>efn=1</code>), the x, y, z (<code>efx</code> , <code>efy</code> , <code>efz</code>) components are scaled to box size. For example <code>efx/x</code> length of box size, <code>efy/y</code> length of box size, <code>efz/z</code> length of box size. This normalizes the electric field charge to your box size. It is off when it is 0. It currently only supports <i>pmemd</i> (both the serial and MPI versions).
<code>efphase</code>	<code>efphase</code> sets the timestep phase for the electric field using the equation $\cos((2\pi \times \text{effreq}/1000)(dt \times \text{step}) - (\pi \times \text{efphase}/180))$. It currently only supports <i>pmemd</i> (both the serial and MPI versions).
<code>effreq</code>	<code>effreq</code> sets the timestep frequency for the electric field using the equation $\cos((2\pi \times \text{effreq}/1000)(dt \times \text{step}) - (\pi \times \text{efphase}/180))$. It currently only supports <i>pmemd</i> (both the serial and MPI versions).
<code>mcwat</code>	<code>mcwat</code> determines whether the Monte Carlo (MC) water equilibrium feature is used. Set 1 to run, 0 otherwise. <code>nmc</code> , <code>nmd</code> , <code>mcwatmaxdif</code> , <code>mcwatmask</code> , <code>mcligshift</code> , and <code>mcboxshift</code> are variables which control the frequency and functionality of this feature. Currently only supported on <i>pmemd</i> , <i>pmemd.cuda</i> , and REMD runs of <i>pmemd.cuda.MPI</i> .
<code>nmd</code>	<code>nmd</code> is the number MD steps per MC/MD cycle. Default values is 1,000.
<code>nmc</code>	<code>nmc</code> is the number of MC steps that are performed per MC/MD cycle. Default value is 100,000
<code>mcwatmask</code>	<code>mcwatmask</code> is an amber mask selection of atoms which we calculate the center of mass to define the center of the MC region. The MC region is the region in which water molecules can do translational MC moves. It is a rectangular prism with user-defined dimensions (using <code>mcligshift</code>) and centered at the center of coordinates of a user-defined set of atoms selected with the AMBER masking function.
<code>mcligshift</code>	<code>mcligshift</code> is used to define the length of the grid along each axis starting from the center of mass of <code>mcwatmask</code> , in Å. For example, a value of 10 means that the grid will extend 10 Å from the center along each axis. Thus, one may enhance water exchange in a buried binding pocket by centering the MC region on a residue bordering the pocket and making the region large enough that it extends into the water outside the protein. If the MC region is not specified, it will be set to the entire simulation box.
<code>ntwd</code>	(default 0), set to 1 to turn on Thermodynamic Integration (TI) Free Energy Decomposition in the <i>pmemd.decomp</i> executable. When on, the command line flag <code>-decomp</code> can be set to determine the log file that outputs decomposition data (default set to <code>decomp.log</code> if unspecified). This feature is best used with <code>reweight=1</code> where rather than running a simulation from scratch the simulation just runs on specified frames fed in via the <code>-c</code> command flag. Otherwise, the program will score every step of the simulation with the decomposition on. To save on disk space the code outputs the sum of the dV/dL of all steps rather than output each step individually. The following masks are used to adjust the output of decomposition: <code>ligmask</code> , <code>proteinmask</code> , <code>cofacmask</code> , <code>decompmask</code> . The <code>decomp.log</code> output is organized in that it lists the number of steps ran to get the average dV/dL (divide any dV/dL value by number of steps ran). Then the next four numbers determine what is considered "region decomposition" where you can see what portion of the dV/dL is attributed to ligand-ligand interactions (strain energy), ligand-protein interactions, ligand-cofactor interactions, and ligand-bulk interactions (desolvation), these regions are specified by the <code>ligmask</code> , <code>proteinmask</code> ,

and cofactormask. Anything not in these masks is specified as water/bulk by the code. The next portion is a list of all atoms that you have specified you want the individual dV/dL values as specified by the decompmask also separated by the type of interaction that atom has first to the ligand, to the protein, to the water/bulk, and then the total dV/dL of the atom. The decomposition dV/dL value doesn't contain constraints, restraints, and vdw long range energy calculations, so results may differ from an actual binding free energy dV/dL value. Since the methodology is usually used with reweight=1, the orientation matters more than the dynamics, so constraint/restraint energies are not needed.

- decompmask** an amber mask string that sets the atoms to be displayed in the decomp.log file (this is to help with potential file size issues). You can just set to all atoms to get the decomposition of all atoms.
- ligmask** amber mask of atoms that are considered the ligands in your calculation. Generally these should be set to your timask1 and timask2 atoms. This is used in the calculation for region decomposition (ligand-ligand contributions, ligand-protein contribution, ligand-cofactor contributions, and ligand-water contributions). If this is not set the region decompositions will be incorrect but the overall dV/dL per atom will still be correct. If ligand atoms are not set in this mask, those atoms will be considered part of the ligand-water/bulk calculation.
- proteinmas** amber mask of atoms that are considered part of your protein. This is used in the calculation for region decomposition specifically the ligand-protein contribution. If protein atoms are not set in this mask, those atoms will be considered part of the ligand-water/bulk calculation. Leave blank if you don't have a protein.
- cofactormask** amber mask of atoms that are considered part of your cofactor. This is used in the calculation for region decomposition specifically the ligand-cofactor contribution. If cofactor atoms are not set in this mask, those atoms will be considered part of the ligand-water/bulk calculation. Leave blank if you don't have a cofactor.
- ramdboost** Sets default random boost acceleration for ramd (default 1). This boost is multiplied by the mass of each atom in the ligand to determine the force each atom receives. This value is in internal acceleration units refer to the Amber units. Ramd is a pmemd and pmemd.cuda only feature and does not support MPI.
- ramdboostfreq** Sets number of steps between each time ramd boost strength is increased (default 0).
- ramdboostrate** Sets the amount to increase the ramdboost acceleration each time ramdboostfreq (default is 0).
- ramdint** Sets the time step interval to apply ramd boost on to the ligand (default is 0).
- ramdmaxdist** Determines the end condition for the simulation (ramd terminates when nstlim is reached or when ramdmaxdist is satisfied). ramdmaxdist is the amount of angstrom displacement from initial center of mass distance of protein and ligand to when this displacement increases by ramdmaxdist.
- ramdligmask** Amber selection mask for what is considered the ligand that needs to be boosted in ramd.
- ramdprotmask** Amber selection mask for what is considered the protein that is used to calculate the distance the ligand has moved.
- reweight** Allows the re-evaluation of trajectories (usually with a new parameter file). Set 1 to turn on. When running this command, in the topology command of the run file (-c) place the trajectory instead of the topology file. This supports netcdf only. Do note if matching against an older run, this does not capture step 0 because step 0 usually evaluates off of the topology which the trajectory generally does not contain. It is recommended to rerun with the same parameter file to check if the feature is working as intended before proceeding with a modified parameter topology file as Amber has a lot of features and not all of them were tested for this feature (was primarily written for TI calculation reweighting). Reweight is supported in pmemd and pmemd.cuda, and does not support MPI.

`midpoint` Turns on midpoint optimizations (usage of 3-D spatial decomposition). 1 is on, 0 is off (default). This switch is currently experimental. Please consult ambermd.org/intel/midpoint.htm for currently supported features and advanced user compilations. Currently only supported on *pmemd.MPI*.

22.7.2. Particle Mesh Ewald

The Particle Mesh Ewald (PME) method is always "on", unless *ntb* = 0. PME is a fast implementation of the Ewald summation method for calculating the full electrostatic energy of a unit cell (periodic box) in a macroscopic lattice of repeating images. The PME method is fast since the reciprocal space Ewald sums are B-spline interpolated on a grid and since the convolutions necessary to evaluate the sums are calculated via fast Fourier transforms (FFTs). Note that the accuracy of the PME method is related to the density of the charge grid (NFFT1, NFFT2, and NFFT3), the spline interpolation order (ORDER), and the direct sum tolerance (DSUM.TOL); see the descriptions below for more information.

The PME method was implemented originally in Amber 3a by Tom Darden and has been developed in subsequent versions by many people, in particular by Tom Darden, Celeste Sagui, Tom Cheatham and Mike Crowley.[520–523] Generalizations of this method to systems with polarizable dipoles and electrostatic multipoles are described in Refs. [524, 525].

The `&ewald` namelist is read immediately after the `&cntrl` namelist. We have tried hard to make the defaults for these parameters appropriate for solvated simulations. *Please take care in changing any values from their defaults.* The `&ewald` namelist has the following variables:

`nfft1`, `nfft2`, `nfft3` These give the size of the charge grid (upon which the reciprocal sums are interpolated) in each dimension. Higher values lead to higher accuracy (when the DSUM.TOL is also lowered) but considerably slow the calculation. Generally it has been found that reasonable results are obtained when NFFT1, NFFT2 and NFFT3 are approximately equal to A, B and C, respectively, leading to a grid spacing (A/NFFT1, etc.) of 1.0 Å. Significant performance enhancement in the calculation of the fast Fourier transform is obtained by having each of the integer NFFT1, NFFT2 and NFFT3 values be a *product of powers* of 2, 3, and/or 5. If the values are not given, the program will chose values to meet these criteria.

`order` The order of the B-spline interpolation. The higher the order, the better the accuracy (unless the charge grid is too coarse). The minimum order is 3. An order of 4 (the default) implies a cubic spline approximation which is a good standard value. Note that the cost of the PME goes as roughly the order to the third power.

`verbose` Standard use is to have VERBOSE = 0. Setting VERBOSE to higher values (up to a maximum of 3) leads to voluminous output of information about the PME run.

`ew_type` Standard use is to have EW_TYPE = 0 which turns on the particle mesh ewald (PME) method. When EW_TYPE = 1, instead of the approximate, interpolated PME, a *regular* Ewald calculation is run. The number of reciprocal vectors used depends upon RSUM.TOL, or can be set by the user. The exact Ewald summation is present mainly to serve as an accuracy check allowing users to determine if the PME grid spacing, order and direct sum tolerance lead to acceptable results. Although the cost of the exact Ewald method formally increases with system size at a much higher rate than the PME, it may be faster for small numbers of atoms (< 500). For larger, macromolecular systems, with > 500 atoms, the PME method is significantly faster.

`dsum_tol` This relates to the width of the direct sum part of the Ewald sum, requiring that the value of the direct sum at the Lennard-Jones cutoff value (specified in CUT as during standard dynamics) be less than DSUM.TOL. In practice it has been found that the relative error in the Ewald forces (RMS) due to cutting off the direct sum at CUT is between 10.0 and 50.0 times DSUM.TOL. Standard values for DSUM.TOL are in the range of 10^{-6} to 10^{-5} , leading to estimated RMS deviation force errors of 0.00001 to 0.0005. Default is 10^{-5} .

<code>rsum_tol</code>	This serves as a way to generate the number of reciprocal vectors used in an Ewald sum. Typically the relative RMS reciprocal sum error is about 5-10 times <code>RSUM_TOL</code> . Default is 5×10^{-5} .
<code>mlimit(1, 2, 3)</code>	This allows the user to explicitly set the number of reciprocal vectors used in a regular Ewald run. Note that the sum goes from <code>-MLIMIT(2)</code> to <code>MLIMIT(2)</code> and <code>-MLIMIT(3)</code> to <code>MLIMIT(3)</code> with symmetry being used in first dimension. Note also the sum is truncated outside an automatically chosen sphere.
<code>ew_coeff</code>	Ewald coefficient, in $\text{\AA}^{-1}$. Default is determined by <code>dsum_tol</code> and <code>cutoff</code> . If it is explicitly inputted then that value is used, and <code>dsum_tol</code> is computed from <code>ew_coeff</code> and <code>cutoff</code> .
<code>nbflag</code>	If <code>nbflag = 0</code> , construct the direct sum nonbonded list in the "old" way, <i>i.e.</i> update the list every <code>nsnb</code> steps. If <code>nbflag = 1</code> (the default when <code>imin = 0</code> or <code>ntb > 0</code>), <code>nsnb</code> is ignored, and the list is updated whenever any atom has moved more than $1/2$ <code>skinnb</code> since the last list update.
<code>skinnb</code>	Width of the nonbonded "skin". The direct sum nonbonded list is extended to <code>cut + skinnb</code> , and the van der Waals and direct electrostatic interactions are truncated at <code>cut</code> . Default is $2.0 \text{ \AA}$. Use of this parameter is required for energy conservation, and recommended for all PME runs.
<code>skin_permit</code>	(<i>pmemd.cuda</i> only) The threshold, as a fraction of <code>skinnb</code> , at which particle migration will trigger a non-bonded pair list rebuild. Enter values between 0.5 (minimum, default) and 1.0 (maximum). Once a particle has traveled more than half the non-bonded pair list margin <code>skinnb</code> , it is possible, although improbable, that another particle has also traveled this distance towards the first, and the pair is then within the non-bonded cutoff but not counted in the pair list. However, as the system gets larger, the probability that any one particle will travel 0.5 times the margin grows linearly, while the likelihood of a pair of nearby particles causing a violation remains constant and low. The frequency of pair list updates is a major factor in the moderate decrease in performance seen in very large systems (the scaling of the FFT is a smaller factor). Furthermore, if an interaction is missing, it will be at the periphery of the cutoff---the threshold at which non-bonded interactions are omitted by construction. Other codes have already implemented "sloppy pair lists," so Amber is following suit and letting the user control the level of risk. By permitting particles to travel up to 0.75 times the pair list margin, pair list updates can be reduced by approximately half and miss one interaction in tens of millions. The most aggressive setting, 1.0, will see the pair list rebuilt at a third of the original rate and omit about one of every million valid interactions. A setting of 0.75 is recommended for the best tradeoff of performance to safety.
<code>nbtell</code>	If <code>nbtell = 1</code> , a message is printed when any atom has moved far enough to trigger a list update. Use only for debugging or analysis. Default of 0 inhibits the message.
<code>netfrc</code>	The basic "smooth" PME implementation used here does not necessarily conserve momentum. If <code>netfrc = 1</code> , (the default) the total force on the system is artificially removed at every step. This parameter is set to 0 if minimization is requested, which implies that the gradient is an accurate derivative of the energy. It should also be set to 0 if coordinate restraints are used (<code>ntr > 0</code>); note: setting <code>netfrc=0</code> when <code>ntr > 0</code> is not now the default, but will be in future versions.
<code>vdwmeth</code>	Determines the method used for van der Waals interactions beyond those included in the direct sum. A value of 0 includes no correction; the default value of 1 uses a continuum model correction for energy and pressure.
<code>eedmeth</code>	Determines how the switch function for the direct sum Coulomb interaction is evaluated. The default value of 1 uses a cubic spline. A value of 2 implies a linear table lookup. A value of three implies use of an "exact" subroutine call.
<code>eedtdns</code>	Density of spline or linear lookup table, if <code>eedmeth</code> is 1 or 2. Default is 500 points per unit.

`column_fft` 1 or 0 flag to turn on or off, respectively, column-mode fft for parallel runs. The default mode is slab mode which is efficient for low processor counts. The column method can be faster for larger processor counts since there can be more columns than slabs and the communications pattern is less congested. This flag has no effect on non-parallel runs. Users should test the efficiency of the method in comparison to the default method before performing long calculations. Default is 0 (off).

22.7.3. Using IPS for the calculation of nonbonded interactions

Isotropic Periodic Sum (IPS) is a method for long-range interaction calculation.[526–531] Unlike the Ewald method, which uses periodic boundary images to calculate long range interactions, IPS uses isotropic periodic images of a local region to calculate the long-range contributions.

The IPS method in the current version is different from that implemented in Amber10. All IPS potentials use rationalized polynomial forms and the electrostatic interaction is calculated using the polar IPS potential. [530] In addition, the 3D IPS/DFFT algorithm [529] is implemented to handle heterogeneous systems as well as finite systems. A homogeneous system is defined as the one where a cutoff region (with *cut* as its radius) has similar composition throughout the system, such as small molecular solutions. Otherwise, a system is defined as a heterogeneous system, such as interfacial systems or finite systems. For heterogeneous systems, a local region larger than the cutoff region, normally equal or larger than the periodic boundary box, must be used to produce accurate long range interactions. For homogeneous systems, it is recommended to use the 3D IPS method (*ips*≤3), which uses the cutoff distance, *cut*, to define the local region radius. *cut* is typically around 10 Å. The 3D IPS/DFFT method (*ips*≥4) can be used for any type of systems, but is recommended for heterogeneous systems only due to the extra discrete fast Fourier transform (DFFT) expense.

For the amoeba polarizable potentials in *sander*, 3D IPS is implemented for interactions between charges, dipoles, and multipoles. The local region radius takes the value of *ee_dsum_cut* in the amoeba namelist, typically, 7 Å.

`ips` Flag to control nonbonded interaction calculation method. The *cut* value will be used to define the local region radius for *ips*≤3. When IPS is used for electrostatic interaction, PME will be turned off. When using the amoeba polarizable potentials, *iamoeba*=1, *ips*>0 (same as *ips*=2) will turn on 3D IPS for all charge, dipole, and quadrupole interactions and the *ee_dsum_cut* value will be used to define the local region radius.

= 0 IPS will not be used (default).

= 1 3D IPS will be used for both electrostatic and L-J interactions.

= 2 3D IPS will be used only for electrostatic, including all multipole, interactions.

= 3 3D IPS will be used only for L-J interactions.

= 4 3D IPS/DFFT will be used for both electrostatic and L-J interactions.

= 5 3D IPS/DFFT will be used only for electrostatic interactions.

= 6 3D IPS/DFFT will be used only for L-J interactions.

`raips` Local region radius. *raips* is automatically set to *cut* for 3D IPS calculations (*ips*≤3) and should be set larger than *cut* for 3D IPS/DFFT calculations (*ips*≥4). A negative value indicates that it is set to the longest box side of a simulation system. For finite systems, i.e., system without periodic boundary conditions, *raips*=∞, which corresponding no image interaction. The default value is -1 Å.

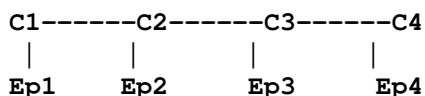
`mipsex, mipsy, mipsz` Number of grids along three periodic boundary sides when using 3D IPS/DFFT method (*ips*≥4). Negative values indicate they are calculated based on the grid size, *gridips*. Typical numbers are the lengths of box sides (in Å) divided by 2 Å. Default values are -1. When *ips*=6 and PME is used for electrostatic interaction, they are set to *nfft1*, *nfft2*, and *nfft3* defined for PME, respectively.

mipso	The order of the B-spline interpolation ($ips \geq 4$). The higher the order, the better the accuracy (unless the charge grid is too coarse). The minimum order is 3. An order of 4 (the default) implies a cubic spline approximation which is a good standard value. The cost for the DFFT calculation goes as roughly the order to the third power. For $ips=6$ and PME is used to electrostatic interaction, it is set to <i>order</i> defined for PME.
gridips	Grid size for 3D IPS/DFFT calculation ($ips \geq 4$). The default value is 2 Å.
dvbips	Volume tolerance for updating IPS function grids ($ips \geq 4$). When volume changes like in <i>NPT</i> simulations, the grid size changes and IPS function on grid points need be updated. The updating only happens when the volume change ratio is more than <i>dvbips</i> . The default value is 1×10^{-8} .

22.7.4. Extra point options

Several parameters deal with "extra-points" (sometimes called lone-pairs), which are force centers that are not at atomic positions. These are currently defined as atoms with "EP" in their names. These input variables are really only for the convenience of force-field developers; *do not change the defaults unless you know what you are doing, and have read the code*. These variables are set in the &ewald namelist.

frameon	If <i>frameon</i> is set to 1, (default) the bonds, angles and dihedral interactions involving the lone pairs/extra points are removed except for constraints added during parm. The lone pairs are kept in ideal geometry relative to local atoms, and resulting torques are transferred to these atoms. To treat extra points as regular atoms, set <i>frameon</i> =0.
chngmask	If <i>chngmask</i> =1 (default), new 1-1, 1-2, 1-3 and 1-4 interactions are calculated. An extra point belonging to an atom has a 1-1 interaction with it, and participates in any 1-2, 1-3 or 1-4 interaction that atom has. For example, suppose (excusing the geometry) C1,C2,C3,C4 form a dihedral and each has 1 extra point attached as below



The 1-4 interactions include C1-C4, Ep1-C4, C1-Ep4, and Ep1-Ep4. (To see a printout of all 1-1, 1-2, 1-3 and 1-4 interactions set *verbose*=1.) These interactions are masked out of nonbonds. Thus the amber mask list is rebuilt from these 1-1, 1-2, 1-3 and 1-4 pairs. A separate list of 1-4 nonbonds is then compiled. This list does not agree in general with the above 1-4, since a 1-4 could also be a 1-3 if its in a ring. See the *ephi()* routine for the precise algorithm involved here. The list of 1-4 nonbonds is printed if *verbose*=1.

22.7.5. Polarizable potentials

The following parameters are relevant for *polarizable potentials*, that is, when *ipol* is set to 1 in the &cntrl namelist. These variables are set in the &ewald namelist.

indmeth	If <i>indmeth</i> is 0, 1, or 2 then the nonbond force is called iteratively until successive estimates of the induced dipoles agree to within DIPTOL (default 0.0001 debye) in the root mean square sense. The difference between <i>indmeth</i> = 0, 1, or 2 have to do with the level of extrapolation (1st, 2nd or 3rd-order) used from previous time steps for the initial guess for dipoles to begin the iterative loop. So far 2nd order (<i>indmeth</i> =1) seems to work best. If <i>indmeth</i> = 3, use a Car-Parinello scheme wherein dipoles are assigned a fictitious mass and integrated each time step. This is much more efficient and is the current default. Note that this method is unstable for $dt > 1$ fs.
diptol	Convergence criterion for dipoles in the iterative methods. Default is 0.0001 Debye.

<code>maxiter</code>	For iterative methods (<code>indmeth<3</code>), this is the maximum number of iterations allowed per time step. Default is 20.
<code>dipmass</code>	The fictitious mass assigned to dipoles. Default value is 0.33, which works well for 1 fs time steps. If <code>dipmass</code> is set much below this, the dynamics are rapidly unstable. If set much above this the dynamics of the system are affected.
<code>diptau</code>	This is used for temperature control of the dipoles (for <code>indmeth=3</code>). If <i>diptau</i> is greater than 10 (ps units) temperature control of dipoles is turned off. Experiments so far indicate that running the system in NVE with no temperature control on induced dipoles leads to a slow heating, barely noticeable on the 100ps time scale. For runs of length 10ps, the energy conservation with this method rivals that of SPME for standard fixed charge systems. For long runs, we recommend setting a weak temperature control (e.g. 9.99 ps) on dipoles as well as on the atoms. Note that to achieve good energy conservation with iterative method, the <code>diptol</code> must be below 10^{-7} debye, which is much more expensive. Default is 11 ps (<i>i.e.</i> default is turned off).
<code>irstdip</code>	If <code>indmeth=3</code> , a restart file for dipole positions and velocities is written along with the restart for atomic coordinates and velocities. If <code>irstdip=1</code> , the dipolar positions and velocities from the <code>inpdip</code> file are read in. If <code>irstdip=0</code> , an iterative method is used for step 1, after which Car-Parrinello is used.
<code>scaldip</code>	To scale 1-4 charge-dipole and dipole-dipole interactions the same as 1-4 charge-charge (<i>i.e.</i> divided by <code>scee</code>) set <code>scaldip=1</code> (default). If <code>scaldip=0</code> the 1-4 charge-dipole and dipole-dipole interactions are treated the same as other dipolar interactions (<i>i.e.</i> divided by 1).

22.7.6. Dipole Printing

By including a `&dipoles` namelist containing a series of groups, at the end of the input file, the printing of permanent, induced and total dipoles is enabled.

The X, Y and Z components of the dipole (in debye) for each group will be written to `mdout` every NTPR steps. In order to avoid ambiguity with charged groups all of the dipoles for a given group are calculated with respect to the centre of mass of that group.

It should be noted that the permanent, inducible and total dipoles will be printed regardless of whether a *polarizable potential* is in use. However, only the permanent dipole will have any physical meaning when *non-polarizable potentials* are in use.

It should also be noted that the groups used in the dipole printing routines are not exclusive to these routines and so the dipole printing procedure can only be used when group input is *not* in use for something else (*i.e.* restraints).

22.7.7. Detailed MPI Timings

`profile_mpi` Adjusts whether detailed per thread timings should be written to a file called `profwile_mpi` when running `sander` in parallel. By default only average timings are printed to the output file. This is done for performance reasons, especially when running *multisander* runs. However for development it is useful to know the individual timings for each mpi thread. When running in serial the value of `profile_mpi` is ignored.

= 0 No detailed MPI timings will be written (default).

= 1 A detailed breakdown of the timings for each MPI thread will be written to the file: `profile_mpi`.

22.8. Polarium Gaussian Multipole Model

22.8.1. Introduction

The polarizable Gaussian Multipole (pGM) force field is a polarizable Amber force field currently under active development.[477–480, 532–536] The pGM model retains the valence terms—bond, angle, and dihedral— as well as the nonbonded van der Waals interactions from the standard AMBER additive force fields. Its primary advancement lies in the treatment of electrostatic interactions. The difference is only in its treatment of electrostatic interactions. Specifically, the pGM model represents atomic charge distributions as Gaussian-shaped multipole expansions at atomic centers. In the current version, electrostatic interactions are modeled with both permanent and induced multipoles, and both of which are truncated at the dipole level. Therefore the pGM model has a more sophisticated electrostatic framework than previous Amber polarizable force fields for which only induced dipoles are added to atomic charges. The theoretical framework in the use of pGM electrostatics in molecular dynamics is rigorously derived, including the interface of PME and pGM for liquid phase molecular simulations.[477] The electrostatic parameters of the pGM model can be derived with the *py_resp.py* program described in 20.[471]

The induced dipoles are computed using a self-consistent field (SCF) approach, where the electric field from permanent multipoles is first calculated, followed by iterative updates of the induced dipoles. An enhanced numerical algorithm has been implemented, reducing convergence to just 2–3 iterations in most NVE simulations. Comprehensive testing has demonstrated that pGM provides accurate results and maintains stable total energy under NVE simulations.[534]

The pGM model supports MPI parallelization, making it feasible for small- to medium-sized systems.[536] Simulations under various thermodynamic ensembles (NVT, NPT) are also supported.[478, 536] A three-point rigid water model compatible with pGM has also been developed as a good starting point to prove the pGM model has good properties in accurately describing the atomic interactions.[536, 537]

The *sander* program is capable of conducting pGM simulations directly. The *pmemd* program with the pGM support is still under active development and is available upon request. The input parameters and the namelist are listed below. A test case is also provided for a 512-water box for NVE, NVT, and NPT simulations, and all test cases support MPI parallelization. Similar to other polarizable dipole models, the energy conservation can be achieved if the induction convergence criterion is set to 10^{-4} or tighter. For practical simulations, a criterion of 10^{-2} achieves satisfactory energy conservation while significantly reducing computational time.[534]

The PME parameters *ew_coeff*, *nfft1*, *nfft2*, *nfft3*, and *order* from the *&ewald* namelist are all related to the accuracy of the overall pGM/PME electrostatics in *sander*. The *ew_coeff* together with the direct sum cutoff (*ee_dsum_cut*, see below) controls the accuracy in the Ewald direct sum, and *ew_coeff* together with the PME grid dimensions *nfft1/2/3* and *order* control the accuracy in the reciprocal sum. Since pGM model requires higher accuracy than classic point charge model, we recommend values exceed those in typical PME simulations with point-charge models. Typical values from our testing are *ew_coeff* = 0.35, *order* = 5, and *nfft1/2/3* are approximately equal to the cell length in the relevant direction, i.e. particle mesh grid spacing ≤ 1 Ångstrom.[477, 536]

In addition to PME, the pGM model also supports the Isotropic Periodic Sum (IPS) method, which replaces the PME reciprocal term with a pairwise IPS potential. To enable this, set both the *ips* and *pol_gauss_ips* flags to 1.[535] One test case is provided for the pGM simulation with IPS method under *'/test/pgm_512wat'*.

22.8.2. Input Variables

&cntrl Namelist input:

ipgm Set to 1 to use the pGM force field. When pGM is used, a *&pol_gauss* namelist is required (see below).

&pol_gauss Namelist input:

pol_gauss_verbose In addition to the usual *sander* output, by setting *pgm_verbose*=1, extra printing of energy and forces can be found in the output file. Default to 0.

`pol_gauss_ips` To use isotropic periodic sum method for pGM model, by setting `pol_gauss_ips=1`, `ips=1` under `&cntrl` Namelist required. Default to 0.

`ee_dsum_cut` The ewald direct sum cutoff. It is recommend to be set to at least 9 Ångstrom.

`dipole_scf_tol` The induced dipoles in the pGM model are solutions to a set of linear equations. These equations are solved iteratively by a linear system solver. `dipole_scf_tol` is the convergence criterion for the iterative solution to the linear equations. To achieve good energy conservation in NVE simulations (i.e. similar to that observed for additive force fields at otherwise identical conditions), a convergence criterion of 10^{-2} is needed. Starting from 2023, the convergence is measured with the maximum relative error on individual dipoles instead of overall residue relative error, so the numerical tolerance appears to be very different, but the convergence quality requirement is similar to previous releases.[\[477\]](#)

`dipole_solv_opt` Set the induction iteration solver. Default to 3, preconditioned conjugate gradient solver. Set it to 4 to choose the previous default SOR solver.

`scf_cg_niter` The maximum iterations when solving the induction equations with a conjugate gradient solver. Default to 50.

`scf_sor_coefficient` This is the successive relaxation parameter in the SOR solver, which can be adjusted to balance the efficiency and stability of the solver. Default value is 0.65.

`scf_sor_niter` The maximum iterations when solving the induction equations with the SOR solver. Default to 100.

22.9. Varying conditions

This section of information is read (if `NMROPT > 0`) as a series of namelist specifications, with name "&wt". This namelist is read repeatedly until a namelist `&wt` statement is found with `TYPE=END`.

`TYPE` Defines quantity being varied; valid options are listed below.

`ISTEP1, ISTEP2` This change is applied over steps/iterations `ISTEP1` through `ISTEP2`. If `ISTEP2 = 0`, this change will remain in effect from step `ISTEP1` to the end of the run at a value of `VALUE1` (`VALUE2` is ignored in this case). (*default= both 0*)

`VALUE1, VALUE2` Values of the change corresponding to `ISTEP1` and `ISTEP2`, respectively. If `ISTEP2=0`, the change is fixed at `VALUE1` for the remainder of the run, once step `ISTEP1` is reached.

`IINC` If `IINC > 0`, then the change is applied as a step function, with `IINC` steps/iterations between each change in the target `VALUE` (ignored if `ISTEP2=0`). If `IINC =0`, the change is done continuously. (*default=0*)

`IMULT` If `IMULT=0`, then the change will be linearly interpolated from `VALUE1` to `VALUE2` as the step number increases from `ISTEP1` to `ISTEP2`. (*default*) If `IMULT=1`, then the change will be effected by a series of multiplicative scalings, using a single factor, `R`, for all scalings. i.e.

$$\text{VALUE2} = (\text{R}^{**}\text{INCREMENTS}) * \text{VALUE1}.$$

`INCREMENTS` is the number of times the target value changes, which is determined by `ISTEP1`, `ISTEP2`, and `IINC`.

The remainder of this section describes the options for the `TYPE` parameter. For a few types of cards, the meanings of the other variables differ from that described above; such differences are noted below. Valid Options for `TYPE` (you must use uppercase) are:

22. *sander*

BOND	Varies the relative weighting of bond energy terms.
ANGLE	Varies the relative weighting of valence angle energy terms.
TORSION	Varies the relative weighting of torsion (and J-coupling) energy terms. Note that any restraints defined in the input to the PARM program are included in the above. Improper torsions are handled separately (IMPROP).
IMPROP	Varies the relative weighting of the "improper" torsional terms. These are not included in TORSION.
VDW	Varies the relative weighting of van der Waals energy terms. This is equivalent to changing the well depth (epsilon) by the given factor.
HB	Varies the relative weighting of hydrogen-bonding energy terms.
ELEC	Varies the relative weighting of electrostatic energy terms.
NB	Varies the relative weights of the non-bonded (VDW, HB, and ELEC) terms.
ATTRACT	Varies the relative weights of the attractive parts of the van der waals and h-bond terms.
REPULSE	Varies the relative weights of the repulsive parts of the van der waals and h-bond terms.
RSTAR	Varies the effective van der Waals radii for the van der Waals (VDW) interactions by the given factor. Note that this is done by changing the relative attractive and repulsive coefficients, so ATTRACT/REPULSE should not be used over the same step range as RSTAR.
INTERN	Varies the relative weights of the BOND, ANGLE and TORSION terms. "Improper" torsions (IMPROP) must be varied separately.
ALL	Varies the relative weights of all the energy terms above (BOND, ANGLE, TORSION, VDW, HB, and ELEC; does not affect RSTAR or IMPROP).
REST	Varies the relative weights of *all* the NMR restraint energy terms.
RESTS	Varies the weights of the "short-range" NMR restraints. Short- range restraints are defined by the SHORT instruction (see below).
RESTL	Varies the weights of any NMR restraints which are not defined as "short range" by the SHORT instruction (see below). When no SHORT instruction is given, RESTL is equivalent to REST.
NOESY	Varies the overall weight for NOESY volume restraints. Note that this value multiplies the individual weights read into the "awt" array. (Only if NMROPT=2; see Section 4 below).
SHIFTS	Varies the overall weight for chemical shift restraints. Note that this value multiplies the individual weights read into the "wt" array. (Only if NMROPT=2; see section 4 below).
SHORT	Defines the short-range restraints. For this instruction, ISTEP1, ISTEP2, VALUE1, and VALUE2 have different meanings. A short-range restraint can be defined in two ways. (1) If the residues containing each pair of bonded atoms comprising the restraint are close enough in the primary sequence:

$$\text{ISTEP1} \leq \text{ABS}(\text{delta_residue}) \leq \text{ISTEP2},$$

where delta_residue is the difference in the numbers of the residues containing the pair of bonded atoms.

(2) If the distances between each pair of bonded atoms in the restraint fall within a prescribed range:

$$\text{VALUE1} \leq \text{distance} \leq \text{VALUE2}.$$

Only one SHORT command can be issued, and the values of ISTEP1, ISTEP2, VALUE1, and VALUE2 remain fixed throughout the run. However, if IINC>0, then the short-range interaction list will be re-evaluated every IINC steps.

TGTRMSD	Varies the RMSD target value for targeted MD.
TEMP0	Varies the target temperature TEMP0.
TEMPOLES	Varies the LES target temperature TEMPOLES.
TAUTP	Varies the coupling parameter, TAUTP, used in temperature scaling when temperature coupling options NTT=1 is used.
CUT	Varies the non-bonded cutoff distance.
NSTEP0	If present, this instruction will reset the initial value of the step counter (against which ISTEP1/ISTEP2 and NSTEP1/NSTEP2 are compared) to the value ISTEP1. This only affects the way in which NMR weight restraints are calculated. It does not affect the value of NSTEP that is printed as part of the dynamics output. An NSTEP0 instruction only has an effect at the beginning of a run. For this card (only) ISTEP2, VALUE1, VALUE2 and IINC are ignored. If this card is omitted, NSTEP0 = 0. This card can be useful for simulation restarts, where NSTEP0 is set to the final step on the previous run.
STPMLT	If present, the NMR step counter will be changed in increments of STPMLT for each actual dynamics step. For this card, only VALUE1 is read. ISTEP1, ISTEP2, VALUE2, IINC, and IMULT are ignored. Default = 1.0.

DISAVE, ANGAVE, TORAVE If present, then by default time-averaged values (rather than instantaneous values) for the appropriate set of restraints will be used. DISAVE controls distance data, ANGAVE controls angle data, TORAVE controls torsion data. See below for the functional form used in generating time-averaged data.

For these cards: $\text{VALUE1} = \tau$ (characteristic time for exponential decay) $\text{VALUE2} = \text{POWER}$ (power used in averaging; the nearest integer of value2 is used) Note that the range (ISTEP1→ISTEP2) applies only to TAU; The value of POWER is not changed by subsequent cards with the same ITYPE field, and time-averaging will always be turned on for the entire run if one of these cards appears.

Note also that, due to the way that the time averaged internals are calculated, changing τ at any time after the start of the run will only affect the relative weighting of steps occurring after the change in τ . Separate values for τ and POWER are used for bond, angle, and torsion averaging.

The default value of τ (if it is 0.0 here) is 1.0D+6, which results in no exponential decay weighting. Any value of $\tau \geq 1.0D+6$ will result in no exponential decay.

If DISAVE, ANGAVE, or TORAVE is chosen, one can still force use of an instantaneous value for specific restraints of the particular type (bond, angle, or torsion) by setting the IFNTYP field to "1" when the restraint is defined (IFNTYP is defined in the DISANG file).

If time-averaging for a particular class of restraints is being performed, all restraints of that class that are being averaged (that is, all restraints of that class except those for which IFNTYP=1) *must* have the same values of NSTEP1 and NSTEP2 (NSTEP1 and NSTEP2 are defined below). (For these cards, IINC and IMULT are ignored) See the discussion of time-averaged restraints following the input descriptions.

DISAVI, ANGAVI, TORAVI **ISTEP1:** Ignored.

ISTEP2: Sets IDMPAV. If IDMPAV > 0, *and* a dump file has been specified (DUMPAVE is set in the file redirection section below), then the time-averaged values of the restraints will be written every IDMPAV steps. Only one value of IDMPAV can be set (corresponding to the first DISAVI/ANGAVI/TORAVI card with ISTEP2 > 0), and *all* restraints (even those with IFNTYP=1) will be "dumped" to this file every IDMPAV steps. The values reported reflect the current value of τ .

VALUE1: The integral which gives the time-averaged values is undefined for the first step. By default, for each time-averaged interval, the integral is assigned the current value of the internal on the first step. If VALUE1 $\neq$ 0, this initial value of internal r is reset as follows:

```
-1000. < VALUE1 < 1000.: Initial value = r.initial + VALUE
VALUE1 <= -1000.: Initial value = r.target + 1000.
1000. <= VALUE1 : Initial value = r.target - 1000.
```

r_{target} is the target value of the internal, given by R2+R3 (or just R3, if R2 is 0). VALUE1 is in angstroms for bonds, in degrees for angles.

VALUE2: This field can be used to set the value of τ used in calculating the time-averaged values of the internal restraints reported at the end of a simulation (if LISTOUT is specified in the redirection section below). By default, no exponential decay weighting is used in calculating the final reported values, regardless of what value of τ was used during the simulation. If VALUE2>0, then $\tau = \text{VALUE2}$ will be used in calculating these final reported averages. Note that the value of VALUE2 = τ specified here only affects the reported averaged values in at the end of a simulation. It does not affect the time-averaged values used during the simulation (those are changed by the VALUE1 field of DISAVE, ANGAVE and TORAVE instructions).

IINC: If IINC = 0, then forces for the class of time-averaged restraints will be calculated exactly as $(dE/dr_{\text{ave}}) (dr_{\text{ave}}/dx)$. If IINC = 1, then forces for the class of time-averaged restraints will be calculated as $(dE/dr_{\text{ave}}) (dr(t)/dx)$. Note that this latter method results in a non-conservative force, and does not integrate to a standard form. But this latter formulation helps avoid the large forces due to the $(1+i)$ term in the exact derivative calculation—and may avert instabilities in the molecular dynamics trajectory for some systems. See the discussion of time-averaged restraints following the input description. Note that the DISAVI, ANGA VI, and TORAVI instructions will have no affect unless the corresponding time average request card (DISAVE, ANGAVE or TORAVE, respectively) is also present.

DUMPFREQ Istep1 is the only parameter read, and it sets the frequency at which the coordinates in the distance or angle restraints are dumped to the file specified by the DUMPAVE command in the I/O redirection section. (For these cards, ISTEP1 and IMULT are ignored).

END END of this section.

NOTES:

1. All weights are relative to a default of 1.0 in the standard force field.
2. Weights are not cumulative.
3. For any range where the weight of a term is not modified by the above, the weight reverts to 1.0. For any range where TEMP0, SOFTR or CUTOFF is not specified, the value of the relevant constant is set to that specified in the input file.
4. If a weight is set to 0.0, it is set internally to 1.0D-7. This can be overridden by setting the weight to a negative number. In this case, a weight of exactly 0.0 will be used. *However*, if any weight is set to exactly 0.0, it cannot be changed again during this run of the program.

5. If two (or more) cards change a particular weight over the same range, the weight given on the last applicable card will be the one used.
6. Once any weight change for which NSTEP2=0 becomes active (i.e. one which will be effective for the remainder of the run), the weight of this term cannot be further modified by other instructions.
7. Changes to RSTAR result in exponential weighting changes to the attractive and repulsive terms (proportional to the scale factor**6 and **12, respectively). For this reason, scaling RSTAR to a very small value (e.g. ≤ 0.1) may result in a zeroing-out of the vdw term.

22.10. File redirection commands

Input/output redirection information can be read as described here. Redirection cards must follow the end of the weight change information. Redirection card input is terminated by the first non-blank line which does not start with a recognized redirection TYPE (e.g. LISTIN, LISTOUT, etc.).

The format of the redirection cards is

TYPE = filename

where TYPE is any valid redirection keyword (see below), and filename is any character string. The equals sign ("=") is required, and TYPE must be given in *uppercase* letters.

Valid redirection keywords are:

LISTIN	An output listing of the restraints which have been read, and their deviations from the target distances <i>before</i> the simulation has been run. By default, this listing is not printed. If POUT is used for the filename, these deviations will be printed in the normal output file.
LISTOUT	An output listing of the restraints which have been read, and their deviations from the target distances <i>after</i> the simulation has finished. By default, this listing is not printed. If POUT is used for the filename, these deviations will be printed in the normal output file.
DISANG	The file from which the distance and angle restraint information described below (Section 30.1) will be read.
NOESY	File from which NOESY volume information (Section 30.2) will be read.
SHIFTS	File from which chemical shift information (Section 30.3) will be read.
PCSHIFT	File from which paramagnetic shift information (Section 30.3) will be read.
DIPOLE	File from which residual dipolar couplings (Section 30.5) will be read.
CSA	File from which CSA or pseudo-CSA restraints (Section 30.6) will be read.
DUMPAVE	File to which the time-averaged values of all restraints will be written. If DISAVI / AN-GAVI / TORAVI has been used to set IDMPAV $\neq$ 0, then averaged values will be output. If the DUMPFREQ command has been used, the instantaneous values will be output.

22.11. Getting debugging information

The debug options in *sander* are there principally to help developers test new options or to test results between two machines or versions of code, but can also be useful to users who want to test the effect of parameters on the accuracy of their ewald or pme calculations. If the debug options are set, *sander* will exit after performing the debug tasks set by the user.

To access the debug options, include a &debug namelist. Input parameters are:

22. sander

`do_debugf` Flag to perform this module. Possible values are zero or one. Default is zero. Set to one to turn on debug options.

One set of options is to test that the atomic forces agree with numerical differentiation of energy.

`atomn` Array of atom numbers to test atomic forces on. Up to 25 atom numbers can be specified, separated by commas.

`nranatm` number of random atoms to test atomic forces on. Atom numbers are generated via a random number generator.

`ranseed` seed of random number generator used in generating atom numbers default is 71277

`neglgdel` negative log of delta used in numerical differentiating; e.g. 4 means delta is 10^{-4} Angstroms. Default is 5. *Note:* In general it does no good to set `neglgdel` larger than about 6. This is because the relative force error is at best the square root of the numerical error in the energy, which ranges from 10^{-15} up to 10^{-12} for energies involving a large number of terms.

`chkvir` Flag to test the atomic and molecular virials numerically. Default is zero. Set to one to test virials.

`dumpfrc` Flag to dump energies, forces and virials, as well as components of forces (bond, angle forces etc.) to the file "forcedump.dat" This produces an ascii file. Default is zero. Set to one to dump forces.

`rmsfrc` Flag to compare energies forces and virials as well as components of forces (bond, angle forces etc.) to those in the file "forcedump.dat". Default is zero. Set to one to compare forces.

Several other options are also possible to modify the calculated forces.

`zerochg` Flag to zero all charges before calculating forces. Default zero. Set to one to remove charges.

`zerovdw` Flag to remove all van der Waals interactions before calculating forces. Default zero. Set to one to remove van der Waals.

`zerodip` Flag to remove all atomic dipoles before calculating forces. Only relevant when polarizability is invoked.

`do_dir`, `do_rec`, `do_adj`, `do_self`, `do_bond`, `do_cbond`, `do_angle`, `do_ephi`, `do_xconst`, `do_cap`
These are flags which turn on or off the subroutines they refer to. The defaults are one. Set to zero to prevent a subroutine from running. For example, set `do_dir=0` to turn off the direct sum interactions (van der Waals as well as electrostatic). These options, as well as the `zerochg`, `zerovdw`, `zerodip` flags, can be used to fine tune a test of forces, accuracy, etc.

EXAMPLES:

This input list tests the reciprocal sum forces on atom 14 numerically, using a delta of 10^{-4} .

```
&debugf
neglgdel=4, nranatm = 0, atomn = 14,
do_debugf = 1, do_dir = 0, do_adj = 0, do_rec = 1, do_self = 0,
do_bond = 1, do_angle = 0, do_ephi = 0, zerovdw = 0, zerochg = 0,
chkvir = 0,
dumpfrc = 0,
rmsfrc = 0,
/
```

This input list causes a dump of force components to "forcedump.dat". The bond, angle and dihedral forces are not calculated, and van der Waals interactions are removed, so the total force is the Ewald electrostatic force, and the only nonzero force components calculated are electrostatic.

```

&debugf
neglgdel=4, nranatm = 0, atomn = 0,
do_debugf = 1,do_dir = 1,do_adj = 1,do_rec = 1, do_self = 1,
do_bond = 0,do_angle = 0,do_ephi = 0, zerovdw = 1, zerochg = 0,
chkvir = 0,
dumpfrc = 1,
rmsfrc = 0,
/

```

In this case the same force components as above are calculated, and compared to those in "forcedump.dat". Typically this is used to get an RMS force error for the Ewald method in use. To do this, when doing the force dump use ewald or pme parameters to get high accuracy, and then normal parameters for the force compare:

```

&debugf
neglgdel=4, nranatm = 0, atomn = 0,
do_debugf = 1,do_dir = 1,do_adj = 1,do_rec = 1, do_self = 1,
do_bond = 0,do_angle = 0,do_ephi = 0, zerovdw = 1, zerochg = 0,
chkvir = 0,
dumpfrc = 0,
rmsfrc = 1,
/

```

For example, if you have a 40x40x40 unit cell and want to see the error for default pme options (cubic spline, 40x40x40 grid), run 2 jobs—— (assume box params on last line of inpcrd file)

Sample input for 1st job:

```

&cntrl
dielc =1.0,
cut = 11.0, nsnb = 5, ibelly = 0,
ntx = 5, irest = 1,
ntf = 2, ntc = 2, tol = 0.0000005,
ntb = 1, ntp = 0, temp0 = 300.0, tautp = 1.0,
nstlim = 1, dt = 0.002, maxcyc = 5, imin = 0, ntmin = 2,
ntpr = 1, ntwx = 0, ntt = 0, ntr = 0,
jfastw = 0, nmrmx=0, ntave = 25,
/
&debugf
do_debugf = 1,do_dir = 1,do_adj = 1,do_rec = 1, do_self = 1,
do_bond = 0,do_angle = 0,do_ephi = 0, zerovdw = 1, zerochg = 0,
chkvir = 0,
dumpfrc = 1,
rmsfrc = 0,
/
&ewald
nfft1=60,nfft2=60,nfft3=60,order=6, ew_coeff=0.35,
/

```

Sample input for 2nd job:

```

&cntrl
dielc =1.0,
cut = 8.0, nsnb = 5, ibelly = 0,
ntx = 5, irest = 1,
ntf = 2, ntc = 2, tol = 0.0000005,
ntb = 1, ntp = 0, temp0 = 300.0, tautp = 1.0,

```

```

nstlim = 1, dt = 0.002, maxcyc = 5, imin = 0, ntmin = 2,
ntpr = 1, ntwx = 0, ntt = 0, ntr = 0,
jfastw = 0, nmrmax=0, ntave = 25,
/
&debugf
do_debugf = 1, do_dir = 1, do_adj = 1, do_rec = 1, do_self = 1,
do_bond = 0, do_angle = 0, do_ephi = 0, zeroovdw = 1, zerochg = 0,
chkvir = 0,
dumpfrc = 0,
rmsfrc = 1,
/
&ewald
ew_coeff=0.35,
/

```

Note that an Ewald coefficient of 0.35 is close to the default error for an 8 Angstrom cutoff. However, the first job used an 11 Angstrom cutoff. The direct sum forces calculated in the 2nd job are compared to these, giving the RMS error due to an 8 Angstrom cutoff, with this value of `ew_coeff`. The reciprocal sum error calculated in the 2nd job is with respect to the pme reciprocal forces in the 1st job considered as "exact".

Note further that if in these two jobs you had not specified "`ew_coeff`" *sander* would have calculated `ew_coeff` according to the cutoff and the direct sum tolerance, defaulted to 10^{-5} . This would give two different ewald coefficients. Under these circumstances the direct, reciprocal and adjust energies and forces would not agree well between the two jobs. However the total energy and forces should agree reasonably, (forces to within about 5×10^{-4} relative RMS force error) Since the totals are invariant to the coefficient.

Finally, note that if other force components are calculated, such as van der Waals, bond, angle, etc., then the total force will include these, and the relative RMS force errors will be with respect to this total force in the denominator.

22.12. multisander (and multipmemd)

The *multisander* and *multipmemd* functionality are available in the parallel versions of the programs (i.e., *sander.MPI* and *pmemd.MPI*). This mode allows multiple independent simulations, or replicas, to be run in the same program instance. It is particularly useful for computer clusters in which priority is given to large CPU-count jobs. In this case, the command-line usage of *sander* and *pmemd* is slightly altered, as shown below:

```
mpirun -np <#proc> sander.MPI -ng <#groups> -groupfile groupfile
```

In this case, `#proc` processors will be evenly divided among `#groups` individual simulations (`#proc` must be a multiple of `#group`!). The groupfile consists of a number of lines which is the command-line for each of the `#groups` simulations you wish to run. Comment lines (i.e., those with `#` in the first column) are ignored, after which the first `#groups` lines are read as the command-line flags of the N^{th} simulation.

The *multisander* and *multipmemd* mechanisms are also utilized for methods requiring multiple simulations to communicate with one another, such as thermodynamic integration in *sander* and replica exchange molecular dynamics (both described later). An example groupfile and program call are shown below.

Groupfile:

```

# Comment lines must start with a pound sign
# and there can be as many comment lines as you
# want, wherever you want them.
-O -p prmtop1 -c inpcrd1 -i replica1.mdin -suffix replica1
-O -p prmtop2 -c inpcrd2 -i replica2.mdin -suffix replica2
-O -p prmtop3 -c inpcrd3 -i replica3.mdin -suffix replica3
-O -p prmtop4 -c inpcrd4 -i replica4.mdin -suffix replica4

```

The `-suffix` flag behaves slightly differently than it does for classical use. In standard simulations (i.e., without *multisander* or *multipmemd*), the provided suffix will be applied only to output files that are printed but were not

given names on the command-line. With multisander, however, each thread has to produce different output files so that different replicas do not try to write to the same file. As a result, a default suffix of 000, 001, 002, etc. is given to the replicas and is added to every unspecified output file. If a `-suffix` is specified in the groupfile, as shown above, every output file—including those given an explicit name for that replica—are given the additional suffix.

The four simulations shown in the groupfile above can be run on 8 processors each with the following command (note, running *sander.MPI* may differ on your system).

```
mpirun -np 32 sander.MPI -ng 4 -groupfile groupfile
```

The *multisander* and *multiplatform* concepts are implemented via the use of MPI communicators. Each replica is assigned a replica-wide communicator along which all communications required for standard MD simulations are performed (called `commsander` and `pmemd_comm` in *sander* and *pmemd*, respectively). Each replica communicator has a master thread (rank 0 in that communicator), and the master thread of each replica are joined in another MPI communicator of replica masters (called `commmaster` and `pmemd_master_comm` in *sander* and *pmemd*, respectively). All inter-replica communication is performed via `commmaster` or `pmemd_master_comm`.

By default, all N threads are allocated to each of the M groups by dividing the threads sequentially. That is, the first N/M threads are assigned to replica 0, the second group of N/M threads are assigned to replica 1, etc. The `-ng-nonsequential` flag will stripe the thread assignments. Replica 0 will receive threads 0, $N - 1$, $2N - 1$, etc., while replica 1 receives threads 1, N , $2N$, etc.

22.13. APBS as an alternate PB solver in Sander

APBS is a robust, numerical Poisson-Boltzmann solver with many features (for more details see <http://apbs.sourceforge.net/>). APBS can be used as an alternative PB solver in sander when compiled with sander using iAPBS.^[538] sander.APBS can be then used for implicit solvent MD simulations, calculation of solvation energies and electrostatic properties and to generate electrostatic potential maps for visualization. It can also be used in the MM-PBSA approach to estimate solvation and apolar (GAMMA * SASA) energy contributions to free energies of binding.

Please see APBS documentation (<http://apbs.sourceforge.net/doc/user-guide/index.html>) for definition of APBS input parameters and iAPBS documentation (<http://mccammon.ucsd.edu/iapbs/>) on how to build sander.APBS and how to use it.

To use `mm_pbsa.pl` script with sander.APBS the following is necessary:

- - sander.APBS must be installed in \$AMBERHOME/bin directory.
- - @GENERAL and @PB sections in input file need to be modified.
- - PQR files for ligand, receptor and complex need to be prepared if an
- alternate charge/radius scheme is used (which is recommended).

Input file description

The `mm_pbsa.in` input file which is included in the Amber distribution can be used with the following modifications:

- (1) Turn on PB and turn off GB and MS calculations in the @GENERAL section of the input file:

```
@GENERAL
MM 1
GB 0
PB 1
MS 0
```

- (2) Input file @PB section:

```

#
@PB
#
#
# PROC = 3 uses sander.APBS as the PB solver
# REFE - REFE = 0 is always used with sander.APBS
# INDI and EXDI are solute and solvent dielectric constants
# SCALE - grid spacing in number of grid points per A
# LINIT - no effect
# PRBRAD - solvent probe radius in A
# ISTRNG - ionic strength in mM
#
# RADIOPT - option to set up radii and charges for PB calculation:
# 0: uses the radii from prmtop files
# 2: reads in PQR files with radii/charges information from
# lig.pqr, rec.pqr and com.pqr PQR files
#
# APBS options:
# BCFL, SRFM, CHGM, SWIN, GAMMA - see APBS and iAPBS documentation for details
# GAMMA is surface tension for apolar energies (in kJ/mol/A**2),
# defaults to 0.105 (Please note the units!)
#
PROC 3
REFE 0
INDI 1.0
EXDI 80.0
SCALE 2
LINIT 1000
PRBRAD 1.4
ISTRNG 0.0
#
RADIOPT 0
#
BCFL 2
SRFM 1
CHGM 1
SWIN 0.3
GAMMA 0.105
#

```

PQR files

With RADIOPT=2 three PQR files are required: lig.pqr, rec.pqr and com.pqr with charge/radius information for the ligand, receptor and complex, respectively. This is the recommended option to get better estimates of solvation energies.

The PQR files can be created with pdb2pqr utility:

```

pdb2pqr.py --assign-only --ff=amber com.pdb com.pqr
pdb2pqr.py --assign-only --ff=amber rec.pdb rec.pqr
pdb2pqr.py --assign-only --ff=amber lig.pdb lig.pqr

```

where --ff=amber is the requested force field charge/radius parameters. Several options are available (Amber, CHARMM, PARSE, etc.) and also a user defined charge/radius scheme is supported (with --ff=myff option).

pdb2pqr.py can be obtained from <http://pdb2pqr.sourceforge.net/>. PDB2PQR service is also available on the web at <http://nbcn.net/pdb2pqr/>. The PDB files (com.pdb, rec.pdb and lig.pdb) can be generated using ambpdb utility.

22.14. Programmer's Corner: The sander API

By Jason M. Swails

This section describes a new feature of *sander*—an application programmer interface (API) that encapsulates some of *sander*'s basic functionality into a library that can be included in your own programs. *sander* was originally written in Fortran as a standalone program that made extensive use of common blocks (i.e., global variables) and uses MPI for the parallel implementation rather than a type of shared-memory parallelization scheme like OpenMP or pthreads. This design conferred a number of constraints on the resulting API.

1. Only one system can be set up for use with the API at a time. Switching Hamiltonians or input parameters requires a lot of deallocation and reallocation and will be inefficient if done very frequently.
2. Only serial execution is supported.
3. LES and non-LES functionality cannot be combined in the same library.
4. File names have a fixed maximum length (256 characters). This can be extended only by adjusting the *sander* source code and recompiling.

Despite these limitations, the *sander* API provides functionality unavailable in other libraries, including QM/MM forces and energies, PB and GB energies, and LES functionality (through a separate library). Although originally written in Fortran, an API has been provided for four languages: Fortran, C, C++, and Python.

The next sections describe the general API design and then the Fortran, C, C++, and Python APIs specifically. **Note:** the python version of this API is sometimes referred to as *pysander*. However, there is no program called *pysander*; that term is rather a shorthand for using “import sander” within a python driver script.

22.14.1. General API Design

This section describes the functions that are available in each variant of the *sander* API. The exact syntax for how the various functions and subroutines are called—and what, if anything, they return—is listed in the following sections for each API.

22.14.1.1. Data Structures

The following data structures are provided as part of the API. These data structures provide a way to provide input or query output from the API. They are the equivalent of C structs (Fortran type and Python class). All floating point data types are double precision (double in C and C++, double precision in Fortran, and float in Python). All integer data types are standard integers (int in C and C++, integer in Fortran, and int in Python).

sander_input

This contains variables used to provide input for the sander API. The attributes that are exposed here have the same name, options, and function as the input options with the same name described earlier in this chapter (and some in earlier chapters). These attributes are listed below, with their data type (float or integer) listed in parentheses at the end.

extdiel External dielectric constant for GB calculations. (float)

intdiel Internal dielectric constant for GB calculations. (float)

- rgbmax** Distance cutoff in Angstroms to use when computing effective GB radii. (float)
- saltcon** Salt concentration, in Molarity, to use when modeling ionic strength effects in a GB calculation. (float)
- cut** Nonbonded cutoff in Angstroms. (float)
- dielc** Dielectric constant to use for all electrostatic interactions. You should use `extdiel` or `intdiel`, described above, for GB calculations (this option should only be used if you are sure it is what you want—it is usually not what you want). (float)
- rdt** This is an option specific to GB calculations with LES (and only has an effect when using the `sanderles` library). When using GB with LES, non-LES atoms require multiple effective radii due to alternate de-screening effects from the different copies. When the multiple radii differ by less than `rdt`, only a single radius will be used for this atom. Default is 0.0. See Chapter 32 for more information. (float)
- igb** GB model to use for GB calculations. Allowable values are 0 (no GB), 1, 2, 5, 6 (vacuum), 7, 8, and 10 (PB). More information is available on page 65. (integer)
- alpb** If 1, use the analytical linearized Poisson-Boltzmann approximation. See Section 4.4 for more information. (integer)
- gbsa** If set to 0, no SASA-based nonpolar free energy of solvation correction is used. If set to 1, the SASA is approximated using the linear combination of pairwise overlaps method (LCPO). If set to 2, the SASA is approximated using a recursive algorithm constructing spheres around each atom. Note, gradients (forces) are not available from this model, so the forces returned by the API will be incorrect if this option is used. (integer)
- lj1264** If 1, use the 12-6-4 Lennard Jones potential form designed for divalent metal ions. If 0, do not use the 12-6-4 Lennard-Jones model. The topology file must be set up correctly to use the 12-6-4 model first! (integer)
- plj1264** If 1, use the pairwise 12-6-4 Lennard Jones potential form designed for divalent metal ions. If 0, do not use the pairwise 12-6-4 Lennard-Jones model. The topology file must be set up correctly to use the pairwise 12-6-4 model first! (integer)
- ipb** Option to compute the solvation free energy using the Poisson-Boltzmann equation. Allowable values are 0 (no PB equation), 1, 2, and 4. See Chapter 6 for more information. (integer)
- vdwmeth** For periodic simulations only (i.e., when `ntb`, below, is set to 1). When set to 1, a long-range dispersion correction based on an analytical integral assuming an isotropic, uniform bulk particle distribution beyond the cutoff is added to the van der Waals energy. When set to 0, no correction is used. (integer)
- ew_type** For periodic simulations only (i.e., when `ntb`, below, is set to 1). When set to 0, the particle-mesh Ewald method is used to compute long-range electrostatics. When set to 1, a traditional Ewald method is used to compute long-range electrostatics (PME is *much* faster for systems with more than 500 atoms or so). (integer)
- ntb** If set to 0, periodic boundaries are not applied. If set to 1, periodic boundaries are used. The value of 2 does not (yet) apply to the API (i.e., constant pressure), as this is an MD-specific option. (integer)
- ifqnt** If set to 1, a QM/MM potential is used (and you must provide a set of valid QM options as well). If set to 0, no QM/MM potential is used. (integer)
- jfastw** Fast water definition flag. By default, the system is searched for water residues, and special routines are used to SHAKE these systems (i.e., they are constrained using the analytical SETTLE algorithm). If set to 0, this default behavior is triggered. If set to 4, the numerical SHAKE routines are used. (integer)
- ntf** Flag to determine which, if any, interactions to omit from the energy calculation. (integer)

Table 22.1.: Summary of default values assigned to `sander_input` variables by the two initialization subroutines provided in the sander API. When alternate values are given for `gas_sander_input`, the latter corresponds to the value assigned if a GB model is requested.

sander_input variable	gas_sander_input default	pme_sander_input default
extdiel	1 or 78.5	0
intdiel	1	0
rgbmax	25.0	25.0
saltcon	0	0
cut	1000.0	8.0
dielc	1	1
rdt	0	0
igb	6 or input value	0
alpb	0	0
gbsa	0	0
lj1264	0	0
plj1264	0	0
ipb	0	0
inp	0	0
vdwmeth	0	1
ew_type	0	0
ntb	0	1
ifqnt	0	0
jfastw	0	0
ntc	1	1
ntf	1	1

ntc Flag to determine whether to use the SHAKE algorithm to constrain bond distances. (integer)

There are two subroutines that will initialize a `sander_input` instance with default values—one that prepares the input for a periodic simulation and one that prepares the input for an aperiodic system (either gas phase or GB calculation). The default values assigned are summarized in table 22.1.

qmmm.input.options

This struct contains a set of options for controlling what portion of the system is treated using quantum mechanics (QM), which QM Hamiltonian is used to treat the QM portion of the system, how the boundary between the QM and MM portions of the system are handled, and how the QM and MM portions interact.

The variables in this data structure have the same name and function as the variables defined in the `&qmmm` namelist of the input file. You can find more information about QM/MM in Chapter 11 on page 159 and about the options specifically in Chapter 9 and Subsection 11.1.6.

There are three types of data types in this struct. Floating point, integer, and character array (i.e., string) values. Like with `sander_input` above, floating point numbers are represented in full double precision and integers as standard integers. The strings in this section are fixed-size arrays of characters. The type of each variable is indicated in parentheses after the variable is defined followed by the fixed-size length of the array if it is an array value. The standard value for the maximum number of QM atoms (`MAX_QUANTUM_ATOMS`) is 10,000.

Note that strings are treated by the API as Fortran strings, *not* C-style strings. The main difference is that Fortran strings do not have a null terminal character (`'\0'`), which means that every character after the final “letter” of the string must contain a space (or null character). As a result, the typical string routines defined in the C `string.h` header file (e.g., `strcpy` and `strncpy`) may not assign the strings correctly if they are not properly initialized entirely with spaces first. That is why the `qm_sander_input` function is provided as part of the API, so I suggest

that you always initialize a `qmmm_input_options` data structure using this method when using the C or C++ APIs.

The defaults listed below are those assigned by the `qm_sander_input` function in the API (and are the same as the defaults defined in Subsection 11.1.6).

qmcut Nonbonded cutoff in Angstroms used for QM/MM nonbonded interactions (note there is no such thing as a cutoff *within* the QM region, since it is the wavefunction of the entire system we are optimizing). The default value is the MM cutoff being used (i.e., cut from `sander_input`, above). (float)

lnk_dis Distance in Angstroms of the QM atom to its link atom. Default is 1.09. (float)

scfconv Controls the convergence of the SCF calculation. The SCF terminates when the energy difference between the last two steps is smaller than the value given here. Default is 10^{-8} and the smallest value that can practically be used within the limits of double precision floating point arithmetic is 10^{-14} . (float)

errconv SCF tolerance on the maximum absolute value of the error matrix (i.e., the commutator of the Fock matrix with the density matrix). The value is in units of Hartrees. The default value is large enough that `scfconv` will always be more strict. (float)

dftb_telec Electronic temperature, in K, used to accelerate SCC convergence in DFTB calculations. The electronic temperature affects the Fermi distribution promoting some HOMO/LUMO mixing, which can accelerate the convergence in difficult cases. In most cases, a low *telec* (around 100K) is enough. Should be used only when necessary, and the results checked carefully. Default: 0.0 (float)

dftb_telec_step The size of the step to take when reducing the electronic temperature in a DFTB calculation. The smaller the step, the longer it will take to get the electronic temperature to zero. (float)

fockp_d1 First prefactor for the Fock matrix prediction. Default is 2.4. Changing this is not recommended. (float)

fockp_d2 Second prefactor for the Fock matrix prediction. Default is -1.2. Changing this is not recommended. (float)

fockp_d3 Third prefactor for the Fock matrix prediction. Default is -0.8. Changing this is not recommended. (float)

fockp_d4 Fourth prefactor for the Fock matrix prediction. Default is 0.6. Changing this is not recommended. (float)

damp SCF damping factor. Default is 1.0. Changing this is not recommended. (float)

vshift Controls level shifting for NDDO methods (not DFTB). Virtual orbitals can be shifted up by `vshift` (in eV) to improve SCF convergence in cases with a small HOMO/LUMO gap. Default is 0.0. (float)

kappa Related to the Debye salt concentration for GB models. This is set automatically from `saltcon` in the `sander_input` data structure. (float)

pseudo_diag_criteria Controls whether a pseudo-diagonalization of the Fock matrix can be performed (not applicable for DFTB). Default is 0.05. (float)

min_heavy_mass The smallest value, in atomic mass units, that an atomic mass can have and still be considered a “heavy-atom” (i.e., anything besides Hydrogen). Default is 4.0. (float)

r_switch_hi If `qmmm_switch` (below) is turned on, this is the distance, in Angstroms, at which the switch goes to zero. By default, it is the same as `qmcut`. (float)

r_switch_lo If `qmmm_switch` (below) is turned on, this is the distance, in Angstroms, at which the switch turns on. By default, it is 2 Angstroms smaller than `r_switch_hi`. (float)

iqmatoms List of atom indexes, starting from 1, that will be treated using QM. This is one way, along with `qmmask`, of specifying the QM region. Default is an empty list. (integer array, `MAX_QUANTUM_ATOMS`).

- qmgb** Specifies how the QM region should be treated with Generalized Born. (integer)
- = 2** (default) As described above, the electrostatic and “polarization” fields from the MM charges and the exterior dielectric, respectively, are included in the Fock matrix for the QM Hamiltonian.
 - = 3** This is intended for debugging and is only useful for single-point calculations. This computes the GB energy by treating every atom in the QM region as a point charge equal to its Mulliken charge. This can be compared to the result when qmgb is set to 2 to evaluate the “strain” energy from the GB solvation.
- lnk_atomic_no** The atomic number of the element you wish to use as the link atom. Default is 1 (Hydrogen). (integer)
- ndiis_matrices** The number of error vectors to use for the DIIS convergence algorithm. Default is 6. (integer)
- ndiis_attempts** The number of iterations that DIIS extrapolation will be attempted. Not available for DFTB. Default value is 0, maximum is 1000. (integer)
- lnk_method** The method used to define how classical valence terms across the QM/MM boundary will be treated. See Subsection 11.1.7 for more information. Default is 1. (integer)
- qmcharge** The net charge of the QM region. Default is 0. (integer)
- corecharge** The net charge of the core QM region. Default is 0. (integer)
- buffercharge** The net charge of the buffer QM region. Default is 0. (integer)
- spin** Spin multiplicity of the QM region. Default is 1 (singlet). (integer)
- qmqmdx** Controls whether QM-QM derivatives are computed analytically or pseudo-numerically. The default (and recommended) is to use analytical QM-QM derivatives. Set to 1 for analytical derivatives, 2 for pseudo-numerical derivatives. Default is 1. (integer)
- verbosity** This has no effect on the API, since output is suppressed. Keep the default value of 0. (integer)
- printcharges** This has no effect on the API since output is suppressed. Keep the default value of 0. (integer)
- printdipole** This has no effect on the API, since output is suppressed. Keep the default value of 0. (integer)
- print_eigenvalues** This has no effect on the API, since output is suppressed. Keep the default value of 0. (integer)
- peptide_corr** If set to 0, (default), do not apply a correction to peptide linkages. If set to 1, apply a MM correction to peptide linkforages. (integer)
- itrmax** Maximum number of SCF iterations to perform before deciding that the convergence has failed. Default is 1000. (integer)
- printbondorders** This has no effect on the API, since output is suppressed. Keep the default value of 0. (integer)
- qmshake** Controls whether SHAKE is applied to QM atoms. If 0, no SHAKE. If 1 (default), SHAKE QM atoms if MM SHAKE is turned on. By default, MM SHAKE is not turned on. This really has no effect, anyway, since the API does not currently support dynamics. (integer)
- qmmrij_incore** If set to 1 (default), store QM-MM pairs and related equations in memory. If set to 0, do not. (integer)
- qmqm_erep_incore** If set to 1 (default), store QM-QM 1-electron repulsion integrals to memory. If set to 0, calculate them on-the-fly. (integer)
- pseudo_diag** If set to 1 (default), allow the use of pseudo-diagonalization of the Fock matrix as long as the pseudo_diag_criteria is met. (integer)

- qm_ewald** Specifies how the long-range electrostatics for the QM region should be treated. See the description in Subsection 11.1.6 for more information. (integer)
- qm_pme** If 0, use a regular Ewald sum for computing QM-QM and QM-MM long-range electrostatic interactions. If 1 (default), use PME instead. (integer)
- kmaxqx** Number of K-space vectors to use in the Ewald/PME calculations in the X-dimension. Default value is 8. (integer)
- kmaxqy** Same as above, but in the Y-dimension. (integer)
- kmaxqz** Same as above, but in the Z-dimension. (integer)
- ksqmaxsq** Specifies the maximum number of K^2 values for the spherical cutoff in reciprocal space when doing a QM-MM Ewald sum. The default value of 100 should be optimal for most systems. (integer)
- qmmm_int** Controls the way in which the QM-MM interaction is handled. See Subsection 11.1.6 for more information. Default is 1. (integer)
- adjust_q** Controls how charge is conserved during a QM/MM calculation with respect to link atoms. See Subsection 11.1.6 for more information. Default is 2. (integer)
- tight_p_conv** Controls the tightness of the convergence criteria on the density matrix in the SCF. If 0 (default), the convergence is loose. If set to 1, convergence is tight. See Chapter 9 for more information. (integer)
- diag_routine** The diagonalization routine to use to diagonalize the Fock matrix. By default (`diag_routine = 0`), the fastest routine is chosen. See the description in Chapter 9 for more details. (integer)
- density_predict** If 1, use the density matrix from the previous MD step. Since MD is not currently supported in the API, do not deviate from the default value of 0. (integer)
- fock_predict** If set to 0, do not attempt to predict the Fock matrix. (Default). If set to 1, try to. (integer)
- vsolv** If set to 1, use variable solvent QM/MM. If set to 0 (default), do not. This option is irrelevant to the API since it does not support QM/MM. (integer)
- dftb_maxiter** The maximum number of SCF iterations to be used in SCC-DFTB calculations. Default is 70. (integer)
- dftb_disper** If set to 1, use a dispersion correction for DFTB/SCC-DFTB. If set to 0 (default), do not. (integer)
- dftb_chg** Has no effect on the API, since printing is disabled. (integer)
- abfqmmm** Toggles the adaptive biased force QM/MM. Since the API does not support MD, this option has no effect. Default is 0. (integer)
- hot_spot** If set to 1, activates hot spot-like adaptive calculation in which the forces of atoms in the buffer region are linear combinations of the forces obtained from the extended and reduced calculations using a smoothing function. If set to 0 (default), disable this behavior. (integer)
- qmmm_switch** If set to 1, use a switching function defined by `r_switch_lo` and `r_switch_hi`. If set to 0 (default), do not. (integer)
- core_iqmatoms** A list of atom indices (starting at 1) that are selected for inclusion in the core QM/MM region in adaptive simulations. (integer array, `MAX_QUANTUM_ATOMS`)
- buffer_iqmatoms** A list of atom indices (starting at 1) that are selected for inclusion in the buffer QM/MM region in adaptive simulations. (integer array, `MAX_QUANTUM_ATOMS`)
- qmmask** An Amber selection mask that provides another way of defining the QM region instead of `iqmatoms`. (character array, 8192)

coremask An Amber selection mask that provides another way of defining the core QM region in adaptive simulations instead of `core_iqmatoms`. (character array, 8192)

buffermask An Amber selection mask that provides another way of defining the buffer QM region in adaptive simulations instead of `buffer_iqmatoms`. (character array, 8192)

centermask An Amber selection mask that defines the center region. If not set, it defaults to `coremask`. (character array, 8192)

dftb_3rd_order Specifies the 3rd-order DFTB correction. Default ('NONE') means no 3rd order correction is used. See Chapter 9 for more information. (character array, 256)

qm.theory String that defines which level of QM theory to use. There is no default and this must be supplied. Available options are defined in Chapter 9. (character array, 12)

pot_ene

This data structure is populated when the energy and forces are computed for the positions that are currently set. All elements of this data structure are double-precision floating point numbers and are given in kilocalories per mole.

tot The total potential energy

vdw The van der Waals contribution to the total energy (not including 1-4 interactions)

elec The electrostatic contribution to the total energy (not including 1-4 interactions)

gb Polar solvation free energy from GB calculations

bond The energy contribution from valence bonds.

angle The energy contribution from valence angles.

dihedral The energy contribution from valence torsions.

vdw_14 The energy contribution from 1-4 van der Waals interactions

elec_14 The energy contribution from 1-4 electrostatic interactions

constraint Really misnamed, this is the total restraint energy if NMR or positional restraints are used.

polar Polarization energy if you are using a polarizable force field.

hbond The 10-12 contribution to the total energy (not used in modern force fields)

surf The non-polar solvation free energy contribution from GB and PB calculations.

scf The QM energy contribution (includes charge-charge interactions between MM and QM atoms, but not dispersion interactions—those are added to the vdw component).

disp Dispersion energy contribution (?? not really sure what this is)

dvdI Not really applicable to the API, since it is used for constant pH MD calculations. This should always be 0.

angle_ub For CHARMM force field, this is the Urey-Bradley contribution to the total energy.

imp For CHARMM force field, this is the improper torsion contribution to the total energy.

cmap For CHARMM force field, this is the correction map energy contribution for coupled torsions.

emap When fitting to an electron density map, this is the restraint energy derived from violations to the map.

les The total energy contributed by the LES copies.

noe The energy penalty for NOE violations.

pb The total polar solvation free energy from PB calculations.

rism The total solvation free energy from 3D-RISM calculations.

ct Charge transfer energy (for `crg_reloc`)

amd_boost This is the AMD boosting energy. It is not applicable for the API since molecular dynamics is not currently supported.

22.14.1.2. Basic subroutines

This section describes the functions and subroutines that are defined by the API and explains what they do. Since their exact behavior (e.g., their arguments and return values) differ depending on which API you are using, the exact usage is deferred to later sections. However, what they *do* is described here.

There are very strong similarities between the C/C++ and Fortran function calls. While the Python function calls are also similar, the Python behavior often differs the most.

gas_sander_input This function will initialize a `sander_input` data structure with the appropriate defaults for carrying out either a gas-phase calculation or an implicit solvent GB calculation. It takes an integer argument defining the GB model to use. See the `igb` variable in the `sander_input` data structure above for allowable values.

It is recommended that you initialize your `sander_input` instance using either this routine or `pme_sander_input` to make sure that all variables are initialized. Uninitialized variables in any of the compiled languages (i.e., not Python) take on undefined behavior and could result in strange bugs.

This can be called regardless of whether or not a system is currently set up.

pme_sander_input This function will initialize a `sander_input` data structure with the appropriate defaults for carrying out a PME calculation on a periodic system. It is recommended that you initialize your `sander_input` instance using either this routine or `gas_sander_input` to make sure that all variables are initialized. Uninitialized variables in any of the compiled languages (i.e., not Python) take on undefined behavior and could result in strange bugs.

This can be called regardless of whether or not a system is currently set up.

qm_sander_input This function will initialize a `qmmm_input_options` data structure with the defaults listed in Subsection 22.14.1.1. This is the recommended method for initializing QM input options, particularly in the C and C++ interfaces where string handling is fragile.

sander_setup These functions take a topology file, coordinates, box dimensions, and a set of input options (`sander_input` and `qmmm_input_options`) and sets up the *sander* API so that energies and forces can be calculated.

These functions can only be called if no system is currently set up. You must call `sander_cleanup` before setting up a different system (or changing input parameters).

set_positions This function takes an array of double precision particle positions ($3 \times \text{natom}$) and sets them as the active conformation.

This function can only be called if there is currently a system set up.

set_box This function takes three box lengths and the angles between them and sets the unit cell (and reciprocal unit cell) vectors from these values.

This function can only be called if there is currently a system set up.

sander.natom This function returns the number of atoms present in the system that is currently set up. This function can only be called if there is currently a system set up.

get.positions This function returns the currently active atomic coordinates for the system that is currently set up. This function can only be called if there is currently a system set up.

get.inpcrd.natom This function takes the name of an inpcrd file and reads the number of atoms that are defined in this file. If this file is not present or its format cannot be determined, the number of atoms is set to -1, which indicates an error. This function can be called regardless of whether or not a system is currently set up.

read.inpcrd.file This function takes the name of an inpcrd file, an array of length $3 \times \text{natom}$ double precision floating point numbers, and an array of 6 double precision floating numbers and fills them with the atomic coordinates and box dimensions, respectively. The box dimensions are stored as a, b, c, α , β , and γ .

Since the two arrays must already be allocated, the typical workflow is to call `get.inpcrd.natom` to determine how large the coordinate array must be made. Then call `read.inpcrd.file` after allocating the coordinate array.

This function can be called regardless of whether or not a system is currently set up.

is.setup This function returns whether a system is currently set up or not. This function can be called regardless of whether or not a system is currently set up.

energy.forces This function computes the energy and forces for the current coordinates of the system that is currently set up and returns them in the potential energy data structure and $(3 \times \text{natom})$ -length double precision array that is passed to this routine. This function can only be called if a system is currently set up.

sander.cleanup This function clears all of the internal memory initialized and allocated by the `sander.setup` routines. This function can only be called if a system is set up, but after this function completes, a system is no longer set up.

22.14.2. The Fortran API

The Fortran API is implemented with a Fortran module. The module is compiled when AmberTools is built and the modulefile is deposited in `$AMBERHOME/include`.

One of the limitations of Fortran modules is that you *must* use the same compiler to build your program as you used to compile AmberTools in the first place. If you wish to change compilers (in many cases, this also includes compiler versions as well), then you need to recompile AmberTools with that same compiler as well. The available API modules are `sander_api` for the standard *sander* functionality and `sanderles_api` for the LES capabilities.

22.14.2.1. Data structures

The Fortran data structures are all different sequence types. The sequence descriptor simply means that they are layed out sequentially in memory in exactly the same way that a struct is in C or C++. Variables within a type are accessed using the `%` operator.

The `sander` input options are available as `type(sander_input)`, the QM/MM input options are available as `type(qmmm_input_options)`, and the potential energy data structure is available as `type(potential_energy_rec)`. The names of the variables that make up each of these types are the same as those defined in Subsection 22.14.1.1.

An example of using the `sander_input` type is shown in the small code fragment below

```

use sander_api, only : sander_input
type(sander_input) :: inp
inp%cut = 9999.d0
inp%ifqnt = 0
inp%igb = 5

```

22.14.2.2. Function call syntax

This section details the function calls for the various subroutines available in the Fortran API. All of these subroutines and functions are public members of both `sanderapi_mod` and `sanderlesapi_mod`.

```
subroutine gas_sander_input(sander_input inp, int igb)
```

This subroutine takes a `sander_input` instance and optionally an integer corresponding to the GB model you want to use (see the description for `igb` above with regards to permissible values). If an illegal `igb` value is provided, a warning is printed to `stderr` and a value of 6 (corresponding to vacuum) is given to `inp%igb`. See table 22.1 for a list of the default values assigned to each variable.

```
subroutine pme_sander_input(sander_input inp)
```

This subroutine takes a `sander_input` instance and initializes every variable inside with the value listed in table 22.1.

```
subroutine qm_sander_input(qmmm_input_options inp)
```

This subroutine takes a `qmmm_input_options` instance and initializes all of the variables to the values given in Subsection 22.14.1.1. This is the recommended way to initialize the QM/MM options type.

```

subroutine sander_setup(character(len=*) top,
double precision, dimension(3*natom) crd,
double precision, dimension(6) box,
sander_input inp,
qmmm_input_options qm_inp,
integer ierr)

```

This subroutine sets up *sander* with the given topology file name, given coordinates, given box dimensions, input options, and QM/MM input options. The `ierr` variable is an error flag and will come back with a value of 0 if the setup succeeded or a value of 1 if it failed. Every other variable is input and guaranteed not to change.

No checking is done to make sure that the number of coordinates provided is correct compared to the number of atoms defined in the topology file. Note that answers will be ridiculous if the coordinate order does not match the atom order in the topology file. Segfaults and other memory violations are possible if the provided coordinate array or box array are too small.

The box array is given in the format $(a, b, c, \alpha, \beta, \gamma)$, which is the same as the format used at the bottom of the input coordinate and restart files. This argument is required even if the system is not periodic, but the values are not used (so they can be initialized to anything).

The `qmmm_input_options` variable is optional, but must be present if `inp%ifqnt` is 1. If `qm_inp` is not provided but QM/MM is requested, an error message will be printed to `stderr` and `ierr` will return with a value of 1.

The error flag `ierr` is required. If `qm_inp` is omitted, then `ierr` must be specified via keyword. See the examples at the end of this section. This function should never be called if a system is already set up.

```
subroutine set_positions(double precision, dimension(3*natom) crd)
```

This subroutine sets the current positions of the active system (and so can only be called if a system is currently set up). Note that the onus is on the programmer to make sure that the coordinate array is large enough. No error checking is done. The input parameter is guaranteed not to change.

```

subroutine set_box(double precision a, double precision b,
                  double precision c,
                  double precision alpha, double precision beta,
                  double precision gamma)

```

This subroutine sets the box dimensions and angles from the input parameters (which are guaranteed not to change).

```

subroutine get_positions(double precision, dimension(3*natom) positions)

```

This subroutine stores the currently active positions inside the passed array.

```

subroutine energy_forces(type(potential_energy_rec) ener,
                        double precision, dimension(3*natom) forces)

```

This subroutine will compute the energies and forces from the current conformation of the system that is currently set up and populate the `ener` type and `forces` array with the resulting values. Those parameters are purely output. The energies are all given in units of kilocalories per mole and forces are given in $\text{kcal/mol}/\text{\AA}$. This subroutine can only be called if a system is currently set up.

```

subroutine sander_cleanup()

```

This subroutine will deallocate all memory used by the *sander* API and return it to a state where no system is set up and a new one can be initialized.

```

logical function is_setup()

```

This function can be called to query whether there is currently a system set up for the *sander* API. It returns `.true.` if a system is set up and `.false.` otherwise.

```

subroutine sander_natom(integer natom)

```

This subroutine will query the currently set up system and return the number of atoms defined by the topology file used during setup. The input parameter will return with the number of atoms in the system or 0 if no system is currently set up.

```

subroutine get_inpcrd_natom(character(len=*) filename, integer natom)

```

This subroutine will open the specified file and try to read how many atoms are defined in that coordinate file. It supports both NetCDF and standard ASCII-formatted inpcrd and restart files. If there was an error in reading the file—either because the file does not exist, read permissions are not set, or the format is unrecognized—`natom` will return with a value of -1. Otherwise, `natom` returns with the number of atoms defined in the inpcrd file, `filename`.

```

subroutine read_inpcrd_file(character(len=*) filename,
                           double precision, dimension(3*natom) crd,
                           double precision, dimension(6) box,
                           integer ierr)

```

This subroutine will read the specified coordinate file and fill the `crd` and `box` arrays with the coordinates and box defined in the file. Both NetCDF restart files and ASCII restart files are supported. If no box is defined in the specified file, the `box` array is initialized to 0. The coordinate array is expected to be allocated with the appropriate amount of space. You can call `get_inpcrd_natom` (described above) to determine how large the coordinate array must be.

If there is a problem reading the file—either because the file does not exist, read permissions are not set, or the format is unrecognized—`ierr` will come back with a value of 1 and the `crd` array will be uninitialized (the `box` array will still be set to 0). If reading succeeded, `ierr` will come back as 0. This function can be called regardless of whether a system is currently set up or not.

22.14.2.3. Example uses of the Fortran API

In this section we show a series of example programs that use the *sander* Fortran API. At the end of this section, we show how to compile your program using the same Fortran compiler you used to build Amber. We will assume that you created a file with the same name as the program name using the `.F90` suffix. You are recommended to use this suffix for your own programs.

We do not do any error checking in these programs since it adds considerably to the length of the example programs. However, you are encouraged to make use of the error reporting in your own programs to avoid program crashes. Syntax highlighting is applied to make the code easier to read.

The first example we provide below shows a sample program that computes purely MM energies for a non-periodic system using one of the GB models available in Amber.

```

program sample1
  use sander_api, only: sander_input, gas_sander_input, &
                        sander_setup, energy_forces, &
                        sander_cleanup, potential_energy_rec, &
                        get_inpcrd_natom, read_inpcrd_file, &
                        sander_natom

  implicit none
  double precision, allocatable, dimension(:) :: crd, frc
  double precision, dimension(6) :: box
  type(sander_input) :: inp
  type(potential_energy_rec) :: ene
  integer :: natom, ierr
  ! Find how many atoms are in our inpcrd file
  call get_inpcrd_natom("inpcrd", natom)
  allocate(crd(natom*3), stat=ierr)
  ! Parse the inpcrd file
  call read_inpcrd_file("inpcrd", crd, box, ierr)
  ! Set up input options to use igb=5 with 0.2M salt
  call gas_sander_input(inp, 5)
  inp%saltcon = 2.0d-1
  ! Set up our system
  call sander_setup("prmtop", crd, box, inp, ierr=ierr)
  ! Coordinate array is no longer needed
  deallocate(crd)
  ! Find out how big our force array must be
  call sander_natom(natom)
  allocate(frc(natom*3), stat=ierr)
  call energy_forces(ene, frc)
  ! Do whatever you want with the energies and forces
  ! ...
  ! Free up our memory
  call sander_cleanup
  deallocate(frc)
  return
end program sample1

```

The second example we provide shows how to use the Fortran API to compute the energy for a periodic system using a multiscale QM/MM Hamiltonian. We will treat residues 10, 11, 12, and 20 using the PDDG-PM3 Hamiltonian.

```

program sample2
  use sander_api, only: sander_input, pme_sander_input, &
                        sander_setup, energy_forces, &

```

```

sander_cleanup, potential_energy_rec, &
get_inpcrd_natom, read_inpcrd_file, &
sander_natom, qmmm_input_options, &
qm_sander_input

implicit none
double precision, allocatable, dimension(:) :: crd, frc
double precision, dimension(6) :: box
type(sander_input) :: inp
type(qmmm_input_options) :: qm_inp
type(potential_energy_rec) :: ene
integer :: natom, ierr
! Find how many atoms are in our inpcrd file
call get_inpcrd_natom("inpcrd", natom)
allocate(crd(natom*3), stat=ierr)
! Parse the inpcrd file
call read_inpcrd_file("inpcrd", crd, box, ierr)
! Set up input options to use PME with a 10A cutoff
call pme_sander_input(inp)
inp%cut = 10.d0
inp%ifqnt = 1
call qm_sander_input(qm_inp)
qm_inp%qmmask = ":10-12,20"
qm_inp%qm_theory = "PDDG-PM3"
! Set up our system
call sander_setup("prmtop", crd, box, inp, qm_inp, ierr)
! Coordinate array is no longer needed
deallocate(crd)
! Find out how big our force array must be
call sander_natom(natom)
allocate(frc(natom*3), stat=ierr)
call energy_forces(ene, frc)
! Do whatever you want with the energies and forces
! ...
! Free up our memory
call sander_cleanup
deallocate(frc)
return
end program sample2

```

To compile Fortran programs using the *sander* API, the compiler must be able to find the *sander_api* (or *sanderles_api*) module files, which are deposited in `$AMBERHOME/include` when you build AmberTools. You must also link `libsander.so` (or `libsanderles.so`) when you link your program. On Mac OS X, these shared libraries are named `libsander.dylib` and `libsanderles.dylib` instead.

The programs we've written above are simple enough that they can be compiled and linked at the same time. The following command should compile the `sample1` program above, assuming it was saved to a file called `sample1.F90`. Note, make sure you use the same compiler you used to build AmberTools in the first place.

```
gfortran -I$AMBERHOME/include -L$AMBERHOME/lib -o sample1 sample1.F90 -lsander
```

This command will create a program called *sample1* that you can run from the command-line. Of course as it is written, the program will require that the files `prmtop` and `inpcrd` be present in the current directory. It will initialize the *sander* API, compute the energy, and quit without printing anything. Feel free to experiment with your own modifications to these programs.

22.14.2.4. Using the LES Fortran API

To use the LES functionality, you need to use the `sanderles_api` module instead of `sander_api` and you have to link to the `sanderles` library instead of the `sander` library (i.e., change `-lsander` to `-lsanderles` in the above compilation step). Since both libraries define most of the same symbols, you unfortunately cannot link both libraries to the same program. For example:

```
gfortran -I$AMBERHOME/include -L$AMBERHOME/lib -o sample1 sample1.F90 -lsanderles
```

22.14.3. The C and C++ APIs

This section describes how to use the C and C++ APIs. These two APIs are the same, and operate very much like a prototypical C API. This is because C and Fortran are both procedural languages (as opposed to object-oriented, like C++). Therefore, Fortran functionality maps more completely onto C than it does onto C++.

The function prototypes and data structures used for the C and C++ APIs are defined in the `sander.h` header file that is installed to `$AMBERHOME/include` when you build AmberTools.

22.14.3.1. Data Structures

The C and C++ data structures are all different `structs`. Variables within a `struct` are accessed using the `.` operator.

The `sander` input options are available as the type `sander_input`, the QM/MM input options are available as the type `qmmm_input_options`, and the potential energy data structure is available as the type `pot_ene`.

An example of using the `sander_input` type is shown in the small code fragment below.

```
#include ``sander.h``
sander_input inp;
inp.cut = 9999.0;
inp.ifqnt = 0;
inp.igb = 5;
```

22.14.3.2. Function call syntax

This section details the function calls for the various functions defined in the `sander.h` header file. The syntax is almost identical to the Fortran syntax, except that error codes are typically returned by the function rather than set in the final input parameter.

```
void gas_sander_input(sander_input *inp, int igb)
```

Unlike the Fortran API, the GB parameter is not optional. This subroutine takes a pointer to a `sander_input` instance and the GB model you wish to use (0 or 6 for vacuum). If an illegal `igb` value is provided, a warning is printed to `stderr` and a value of 6 is given to `inp->igb`. See table 22.1 for the default values assigned to each variable.

```
void pme_sander_input(sander_input *inp)
```

This subroutine takes a pointer to a `sander_input` instance and initializes every variable inside with the value listed in table 22.1.

```
void qm_sander_input(qmmm_input_options *inp)
```

This subroutine takes a `qmmm_input_options` instance and initializes all of the variables to the values given in Subsection 22.14.1.1. This is the recommended way to initialize the QM/MM options type.

```
int sander_setup_mm(const char* top, double *crd,
                  double *box, sander_input *inp)
```


This function sets up *sander* with the given topology file name, given coordinates, given box dimensions, and input options. Since overloading is not permitted in C, the QM/MM input struct cannot be made optional. Therefore, this function can only be used when `inp->ifqnt` is 0. This function returns 0 upon success or 1 upon failure. A system is only considered set up if this function returns 0.

No checking is done to make sure that the number of coordinates provided is correct compared to the number of atoms defined in the topology file. Note that answers will be ridiculous if the coordinate order does not match the atom order in the topology file. Segfaults and other memory violations are possible if the provided coordinate array or box array are too small.

The box array is given in the format $(a, b, c, \alpha, \beta, \gamma)$, which is the same as the format used at the bottom of the input coordinate and restart files. This argument is required even if the system is not periodic, but the values are not used (so they can be initialized to anything).

This function should never be called if a system is already set up.

```
int sander_setup(const char* top, double *crd,
                double *box, sander_input *inp,
                qmmm_input_options *qm_inp)
```

This function does the same thing as `sander_setup_mm` described above, but it also requires a pointer to a `qmmm_input_options` instance. If `inp->ifqnt` is set to 0, the contents of `qm_inp` are ignored and a standard MM system is set up. If successful, this function returns 0. Otherwise, it returns 1.

This function should never be called if a system is already set up.

```
void set_positions(double *crd)
```

This function sets the current positions of the active system (and so can only be called if a system is currently set up). Note that the onus is on the programmer to make sure that the coordinate array is large enough. No error checking is done. The input parameter is guaranteed not to change.

```
void set_box(double a, double b, double c,
             double alpha, double beta, double gamma)
```

This function sets the box dimensions and angles from the input parameters (which are guaranteed not to change).

```
void get_positions(double *positions)
```

This function gets the “active” positions for the system that is currently set up.

```
void energy_forces(pot_ene *ener, double *forces)
```

This function will compute the energies and forces from the current conformation of the system that is currently set up and populate the `ener` type and `forces` array with the resulting values. Those parameters are purely output. The energies are all given in units of kilocalories per mole and forces are given in $kcal/mol/\text{\AA}$. This subroutine can only be called if a system is currently set up.

```
void sander_cleanup(void)
```

This function will deallocate all memory used by the *sander* API and return it to a state where no system is set up and a new one can be initialized.

```
int is_setup(void)
```

This function can be called to query whether there is currently a system set up for the *sander* API. It returns 0 if no system is set up and 1 if a system is set up.

```
int sander_natom(void)
```

This function will query the currently set up system and return the number of atoms defined by the topology file used during setup. If no system is set up, this function returns 0.

```
int get_inpcrd_natom(const char *filename)
```

This function will open the specified file and try to read how many atoms are defined in that coordinate file. It supports both NetCDF and standard ASCII-formatted inpcrd and restart files. If there was an error in reading the file—either because the file does not exist, read permissions are not set, or the format is unrecognized—the return value will be -1. Otherwise, this function returns the number of atoms defined in the inpcrd file, `filename`.

```
int read_inpcrd_file(const char* filename, double *crd, double *box)
```

This subroutine will read the specified coordinate file and fill the `crd` and `box` arrays with the coordinates and box defined in the file. Both NetCDF restart files and ASCII restart files are supported. If no box is defined in the specified file, the `box` array is initialized to 0. The coordinate array is expected to be allocated with the appropriate amount of space. You can call `get_inpcrd_natom` (described above) to determine how large the coordinate array must be.

If there is a problem reading the file—either because the file does not exist, read permissions are not set, or the format is unrecognized—this function will return 1 and the `crd` array will be uninitialized (the `box` array will still be set to 0). If reading succeeded, this function will return 0. This function can be called regardless of whether a system is currently set up or not.

22.14.3.3. Examples and uses of the C and C++ APIs

In this section, we show examples of how to use the C and C++ API. These samples do exactly the same thing as the two examples in Subsection 22.14.2.3. At the end of this section, we show how to compile your C or C++ program.

We do not do any error checking in these programs since it adds considerably to the length of the example programs. However, you are encouraged to make use of the error reporting in your own programs to avoid program crashes. Syntax highlighting is applied to make the code easier to read.

The first example we provide below shows a sample C program that computes purely MM energies for a non-periodic system using one of the GB models available in Amber.

```
#include <stdlib.h>
#include "sander.h"
int main() {
    sander_input inp;
    double *crd, *frc;
    double box[6];
    pot_ene ene;
    int natom, ierr;
    // Find out how many atoms are in our inpcrd file
    natom = get_inpcrd_natom("inpcrd");
    crd = (double*) malloc(natom*3*sizeof(double));
    ierr = read_inpcrd_file("inpcrd", crd, box);
    // Set up input options to use igb=5 with 0.2M salt
    gas_sander_input(&inp, 5);
    inp.saltcon = 0.2;
    // Set up our system
    ierr = sander_setup_mm("prmtop", crd, box, &inp);
    // Coordinate array is no longer needed
    free(crd);
    // Find out how big our force array must be
    frc = (double*) malloc(sander_natom()*3*sizeof(double));
    energy_forces(&ene, frc);
    /* Do whatever you want with the energies and forces
    * ...
    * Free up our memory
```

```

    */
    sander_cleanup();
    free(frc);
    return 0;
}

```

The second example we provide shows how to use the C API to compute the energy for a periodic system using a multiscale QM/MM Hamiltonian (in a C++ program this time). We will treat residues 10, 11, 12, and 20 using the PDDG-PM3 Hamiltonian.

```

#include "sander.h"
#include <cstring>
int main() {
    sander_input inp;
    double *crd, *frc;
    double box[6];
    pot_ene ene;
    int natom, ierr;
    // Find out how many atoms are in our inpcrd file
    natom = get_inpcrd_natom("inpcrd");
    crd = new double[natom*3];
    ierr = read_inpcrd_file("inpcrd", crd, box);
    // Set up input options to use igb=5 with 0.2M salt
    pme_sander_input(&inp);
    inp.cut = 10.0;
    qm_sander_input(&qm_inp);
    strncpy(qm_inp.qmmask, ``:10-12,20'', 9);
    strncpy(qm_inp.qm_theory, ``PDDG-PM3'', 8);
    // Set up our system
    ierr = sander_setup("prmtop", crd, box, &inp, &qm_inp);
    // Coordinate array is no longer needed
    delete[] crd;
    // Find out how big our force array must be
    frc = new double[sander_natom()*3];
    energy_forces(&ene, frc);
    /* Do whatever you want with the energies and forces
     * ...
     * Free up our memory
     */
    sander_cleanup();
    delete[] frc;
    return 0;
}

```

To compile C or C++ programs using the *sander* API, the compiler must be able to find the *sander.h* header file, which are deposited in `$AMBERHOME/include` when you build AmberTools. You must also link *libsander.so* (or *libsanderles.so*) when you link your program. On Mac OS X, these shared libraries are named *libsander.dylib* and *libsanderles.dylib* instead.

The programs we've written above are simple enough that they can be compiled and linked at the same time. The following command should compile the `sample1` program above, assuming it was saved to a file called `sample1.F90`. Note, make sure you use the same compiler you used to build AmberTools in the first place.

```
gcc -I$AMBERHOME/include -L$AMBERHOME/lib -o sample1 sample1.c -lsander
```

This command will create a program called *sample1* that you can run from the command-line. Of course as it is written, the program will require that the files *prmtop* and *inpcrd* be present in the current directory. It will initialize the *sander* API, compute the energy, and quit without printing anything. Feel free to experiment with your own modifications to these programs. For the second sample, you need to use a C++ compiler instead of the C compiler.

22.14.3.4. Using the LES C/C++ API

There is only one header file for the *sander* C/C++ API. The LES and standard functionalities are differentiated using the LES preprocessor directive. To use the LES functionality, you need to define the LES macro. You can either do this in the source code (by putting “`#define LES 1`” before `#include ‘sander.h’`) or by compiling with the `-DLES` flag. If you use the LES symbol (either as a variable or a preprocessor macro), you will have to implement this in the source code and undefine the macro after *sander.h* is included. For example, on the command line this would look like:

```
gcc -DLES -I$AMBERHOME/include -L$AMBERHOME/lib -o sample1 sample1.c -lsanderles
```

22.14.4. The Python API

This section describes how to use the Python API so that you can use *sander* functionality inside your own Python scripts. Building the Python bindings requires that the Python development headers and libraries be installed. As long as you install the recommended packages listed on https://ambermd.org/amber_install.html for your Linux, Mac or Windows distribution, the necessary prerequisites will be installed.

The *sander* functionality is implemented in the *sander* Python module. The *sander* LES functionality is implemented in the *sanderles* module. While the Python API implements the functions described on page 446, the semantics of how these functions are used in Python differs more than the difference between the C/C++ and Fortran APIs.

The Python API has numerous advantages over the other options. First, processing strings is handled correctly by the boilerplate that interfaces Python with C, meaning that the programmer does not have to worry about how strings map to the underlying Fortran code. Second, data is always initialized, so the programmer does not have to worry about bugs arising from uninitialized variables. Finally, array sizes are determined automatically and no allocation or deallocation is required.

Furthermore, the Python API provided here interacts with other Python packages provided as part of Amber-Tools—specifically several of the classes provided by ParmEd. See Section 15.2 for more information (specifically Subsection 15.2.5.3 for the ParmEd Python API documentation).

22.14.4.1. Data Structures

The data structures in the Python API are all “restricted” classes, where restricted means setting new attributes is not supported and will raise an `AttributeError`. The data types for the *sander* input options, QM/MM input options, and potential energy terms are the classes `InputOptions`, `QmInputOptions`, and `EnergyTerms`, respectively. The last class is part of the private *sander*.*pys* namespace since it is only produced as output and never needed as input, whereas the first two are members of the *sander* package namespace.

Unlike C and Fortran, the Python classes have default constructors that will initialize all of the variables for the different classes. An example of using the `InputOptions` class is shown below.

```
import sander
inp = sander.InputOptions()
inp.cut = 9999.0
inp.extdiel = 78.5
inp.intdiel = 1
```

22.14.4.2. Function call syntax

This section details the function calls for the various functions defined in the `sander` package.

```
inp = sander.gas_input(6)
```

The `igb` argument is an optional integer that defaults to 6 (vacuum). This function returns an initialized `InputOptions` instance whose values are listed in table 22.1. If an illegal `igb` value is provided, a `ValueError` is raised.

```
inp = sander.pme_input()
```

This subroutine returns a `InputOptions` instance and initializes every member with the value listed in table 22.1.

```
qm_inp = sander.qm_input()  
qm_inp = sander.QmInputOptions()
```

These two commands both return a `QmInputOptions` instance and initializes all of the variables to the values given in Subsection 22.14.1.1. The function (`sander.qm_input()`) is redundant, since the `QmInputOptions` constructor does the same thing. The function was provided only for consistency with the Fortran and C/C++ APIs.

```
sander.setup(prmtop, coordinates, box, mm_options, qm_options=None)
```

This function sets up `sander` with the given topology file, coordinates, box dimensions, and input options. If `mm_options.ifqnt` is 1 and `qm_options` is not provided, a `ValueError` is raised. The topology file can be either an `AmberParm` instance (see Subsection 15.2.5.3 for more information) or a string filename pointing to a valid Amber topology file.

The coordinate array can either be a `numpy.ndarray` instance an `array.array` instance, or a `list`. The array must be 1-dimensional with a length equal to $3 \times \text{natom}$. In particular, the coordinate array taken from a `Rst7` instance can be used. Alternatively, the `coordinates` argument can be a string that is the filename of a coordinate or restart file.

The box array, too, can be a `numpy.ndarray`, `array.array`, or `list` instance of length 6. If it is not one of those data types, a `TypeError` will be raised. If it does not have 6 elements, a `ValueError` will be raised. If no box is needed, the `box` argument can be set to `None`. Alternatively, if `box` is set to `None` and a filename was passed to the `coordinates` argument that contains box dimensions, the box will be set from the information in that file. However, any box dimensions passed using the `box` argument will take precedence.

The `mm_options` must be a `InputOptions` instance or a `TypeError` will be raised. The `qm_options` must be a `QmInputOptions` instance or `None`. Otherwise, a `TypeError` will be raised.

If there is any problem setting the system up, or if a system is already set up, a `RuntimeError` will be raised. This “function” is actually a class that implements the context manager protocol via the `with` statement (Python 2.5 or greater, only—Python 2.4 users must use the syntax above).

```
with sander.setup(prmtop, coordinates, box, mm_options, qm_options=None):  
... do stuff
```

The return value of `sander.setup` is a reference to the class (which itself can be used in a context manager). Upon exiting the context manager, `sander.cleanup` is called (but only if `sander.setup` succeeded).

```
sander.set_positions(crd)
```

This function sets the current positions of the active system. The `crd` argument can be a `numpy.ndarray`, `array.array`, or `list` instance and must be either 1-dimensional with a length $3 \times \text{natom}$ or 2-dimensional with a shape of `natom, 3`. If the array is not the correct length, a `ValueError` will be raised. If it is not one of the aforementioned types, a `TypeError` will be raised. If a system is not currently set up, a `RuntimeError` will be raised. This function returns `None`.

```
sander.set_box(a, b, c, alpha, beta, gamma)
```

This function sets the box dimensions and angles from the input parameters. If the incorrect number of arguments are given, or if the arguments are not all numbers, a `TypeError` is raised. If no system is currently set up, a `RuntimeError` is raised. This function returns `None`.

```
positions = sander.get_positions()
```

This function returns the coordinates as a one-dimensional list for the currently active system. If no system is currently set up, a `RuntimeError` is raised.

```
ene, frc = sander.energy_forces()
```

This function will compute the energies and forces from the current conformation of the system that is currently set up and returns a two-element tuple in which the first element is an `EnergyTerms` instance with the attributes listed in Subsection 22.14.1.1 and the second attribute is a $3 \times \text{natom}$ -length list with the atomic forces. The energies are all given in units of kilocalories per mole and forces are given in $\text{kcal/mol}/\text{\AA}$. A `RuntimeError` is raised if no system is currently set up.

```
sander.cleanup()
```

This function will deallocate all memory used by the *sander* API and return it to a state where no system is set up and a new one can be initialized. If no system is set up, a `RuntimeError` is raised. This function returns `None`.

```
bool = sander.is_setup()
```

This function can be called to query whether there is currently a system set up for the *sander* API. It returns `False` if no system is set up and `True` if a system is set up.

```
natom = sander.natom()
```

This function will query the currently set up system and return the number of atoms defined by the topology file used during setup. If no system is set up, this function raises a `RuntimeError`.

Coordinate file parsing No functions are provided to parse and query coordinate and restart files, since the `Rst7` class from the `parmed.amber` package already does that. Examples using this class are shown in the next section.

22.14.4.3. Examples and uses of the Python API

In this section, we show examples of how to use the Python API. These samples do exactly the same thing as the two examples in Subsection 22.14.2.3 and Subsection 22.14.3.3.

Unlike the previous APIs, the Python API has built-in error checking through the utilization of the Exception mechanism. The various exceptions that can be raised and the circumstances in which they will be raised are described in the previous section. You may wish to catch some of the exceptions in your own Python scripts to implement more elaborate error handling. Notice that the Python program here is much simpler than the equivalent Fortran and C programs presented earlier.

These programs also make use of the `AmberParm` class in the chemistry package that is part of the `ParmEd` program (see Section 15.2).

```
import sander
from parmed.amber.readparm import AmberParm
# Initialize the topology object with coordinates
parm = AmberParm("prmtop", "inpcrd")
# Set up input options to use igb=5 with 0.2M salt
inp = sander.gas_input(5)
```

```

inp.saltcon = 0.2
sander.setup(parm, parm.coordinates, None, inp)
# Compute the energies and forces
ene, frc = sander.energy_forces()
# Do whatever you want with the energies and forces
# ...
# Free up our memory
sander.cleanup()

```

The second example we provide shows how to use the Python API to compute the energy for a periodic system using a multiscale QM/MM Hamiltonian. We will treat residues 10, 11, 12, and 20 using the PDDG-PM3 Hamiltonian. Also, rather than loading the inpcrd file directly into the AmberParm object, we use the `open` constructor of the `Rst7` class to read in the coordinate file. While this is exactly what the `AmberParm` class does under the hood, this approach is presented here to show how to use the `Rst7` class in your own programs.

```

import sander
from parmed.amber.readparm import AmberParm, Rst7
# Initialize the topology object with coordinates
parm = AmberParm("prmtop")
rst = Rst7.open("inpcrd")
# Set up input options to use PME with a 10A cutoff
inp = sander.gas_input(5)
inp.cut = 10.0
qm_inp = sander.QmInputOptions()
qm_inp.qmmask = ``:10-12,20``
qm_inp.qm_theory = ``PDDG-PM3``
sander.setup(parm, rst.coords, rst.box, inp, qm_inp)
# Compute the energies and forces
ene, frc = sander.energy_forces()
# Do whatever you want with the energies and forces
# ...
# Free up our memory
sander.cleanup()

```

One final thing we will mention is that the sander Python API supports the context manager protocol! The previous example can be rewritten as

```

import sander
from parmed.amber.readparm import AmberParm, Rst7
# Initialize the topology object with coordinates
parm = AmberParm("prmtop")
rst = Rst7.open("inpcrd")
# Set up input options to use PME with a 10A cutoff
inp = sander.gas_input(5)
inp.cut = 10.0
qm_inp = sander.QmInputOptions()
qm_inp.qmmask = ``:10-12,20``
qm_inp.qm_theory = ``PDDG-PM3``
with sander.setup(parm, rst.coords, rst.box, inp, qm_inp):
    # Compute the energies and forces
    ene, frc = sander.energy_forces()
# Do whatever you want with the energies and forces
# ...
# Free up our memory

```

22. *sander*

When the context manager is exited (i.e., when program execution is no longer inside the `with` block), `sander` is automatically cleaned up. This occurs regardless of whether or not an error was raised during the execution of the code within the `with` block. Notice how `sander.cleanup()` is no longer necessary.

22.14.4.4. Using the LES Python API

To use the LES functionality in Python, you need to import the `sanderles` package instead of the `sander` package. Note that while nothing stops you from importing both the `sander` and `sanderles` packages in the same Python script, both packages will not work correctly in the same script.

23. pmemd

23.1. Introduction

PMEMD (Particle Mesh Ewald Molecular Dynamics) is the primary molecular dynamics engine within the AMBER Software suite. Begun by Dr. Robert E. Duke with the goal of improving performance in the most frequently used methods of sander, the code has since diverged into multiple integrated programs, offering massively parallel CPU and highly performant GPU [539–541] capabilities for common particle simulations as well as sophisticated CPU implementations of advanced models for electronic polarization. PMEMD supports Particle Mesh Ewald simulations, Generalized Born simulations, Isotropic Periodic Sums, ALPB (Analytical Linearized Poisson-Boltzmann) solvent, middle thermostat scheme and even gas phase simulations using both the AMBER and CHARMM Force fields. Most of these capabilities are also supported on the GPU accelerators, as detailed in 23.6.

For the supported functionality, the input required and output produced are intended to replicate sander. The agreement goes as far as the limits of machine roundoff differences for the CPU code, which performs essentially all of its arithmetic in 64-bit precision. Likewise, the GPU code offers a double-precision variant for quality assurance during code testing and after installation, but perfect agreement with CPU results is not guaranteed in cases where the GPU and CPU must generate their own random number sequences with different routines. The production GPU code, which performs most of its arithmetic in 32-bit precision, will necessarily diverge from the CPU code, but maintains a high degree of numerical reproducibility thanks to fixed-precision accumulation of forces and energies. PMEMD simply runs more rapidly, scales better in parallel using MPI, can make use of NVIDIA GPUs and Intel Xeon Phis for acceleration, and uses less resident memory than the more general sander engine. Dynamic memory allocation is used so memory configuration is not required. Benchmark data is available on the Amber website, ambermd.org. Given the improvements in performance in both serial and parallel as well as the incredible performance offered by GPU acceleration, it is advisable to always use PMEMD in place of sander if the simulation requirements are within the functionality envelope provided by PMEMD.

PMEMD accepts sander input files (*mdin*, *prmtop*, *inpcrd*, *refc*). All options documented in the sander section of this manual should be properly parsed and an error message generated if a requested feature is not supported. PMEMD is also backward compatible in regard to input to the same extent as sander.

23.2. Functionality

New functionality that has been added to *pmemd* includes:

- Thermodynamic Integration, FEP and MBAR support on GPUs
- Support for Adaptively biased MD
- Improved CPU performance and scaling
- 12-6-4 ionic dispersion potentials are now supported on GPUs
- middle thermostat scheme [See 22.6.10 for details]
- All-atom PME continuous constant pH MD supported on GPUs

The following functionality is also supported by the GPU version of *pmemd* in addition to the CPU version:

- Support for gas phase simulations (through *igb=6*)

23. pmemd

- Support for external electric fields
- Support for the Charmm VDW Force switch
- Support for Gaussian accelerated molecular dynamics
- Semi-Isotropic pressure scaling
- Enhanced NMR restraints and R⁶ averaging support
- Expanded umbrella sampling support
- Constant pH and REMD Constant pH

As mentioned above, PMEMD is not a complete implementation of sander. Instead, it is intended to be a fast implementation of the functionality most likely to be used by someone doing long time scale explicitly or implicitly solvated systems. It also includes some additional functionality of it's own.

Specifically the following functionality in sander is missing entirely:

imin=5 In &cntrl. Trajectory analysis is not supported.

nmropt=2 In &cntrl. A variety of NMR-specific options such as NOESY restraints, chemical shift restraints, pseudocontact restraints, and direct dipolar coupling restraints are not supported.

idecomp!=0 In &cntrl. Energy decomposition options, used in conjunction with mm_pbsa, are not supported.

ipol!=0 In &cntrl. Polarizable force field simulations are not supported.

igb==10 In &cntrl. Poisson-Boltzmann simulations are not supported.

ntmin>2 In &cntrl. XMIN and LMOD minimization methods are not supported.

Solvent Caps Solvent cap simulations are not supported.

itgtmd!=0 In &cntrl. Targeted molecular dynamics is not supported.

ievb!=0 In &cntrl. Empirical Valence Bond methods are not supported.

ifqnt!=0 In &cntrl. QM/MM methods are not supported.

&debugf namelist Use of the &debugf namelist is only supported in a very limited way. Specifically only the *do_charmm_dump_gold* option is supported.

LES The Locally Enhanced Sampling method is not supported.

REM==2 The partial REMD method (for LES) is not supported

iamoeba!=0 In &cntrl. The amoeba polarizable potentials of Ren and Ponder are not supported in pmemd, although support is provided through a special pmemd.amoeba implementation provided as part of Amber 16.

One niche feature of the CPU code that is currently missing on the GPU code is the ability to employ different cutoffs for electrostatics and van-der Waals non-bonded interactions. In particle-mesh Ewald electrostatics, the results are not necessarily more accurate for longer values of the cutoff—the mesh grid spacing can be reduced to compensate for a short cutoff on charge:charge interactions, and there are often fewer van-der Waals than electrostatic interactions in a simulation with explicit solvent. Advanced users of the CPU code can therefore tune performance for small numbers of CPUs by reducing the cutoff on electrostatics while keeping a long van-der Waals cutoff. This feature may be added to the GPU code in the near future for performance enhancements and as a stepping stone to other functionality. The following &ewald options are supported generally in PMEMD, but only with the indicated default values:

<i>ew_type=0</i>	Only Particle Mesh Ewald calculations are supported. <i>ew_type</i> = 1 (regular Ewald calculations) must be done in sander.
<i>nbflag=1</i>	The <i>nbflag</i> option is ignored for MD, and all nonbonded list updates are scheduled based on "skin" checks. This is more reliable and has little cost. The variable <i>nsnb</i> still can be set and has an influence on minimizations but is ignored during MD. For PME calculations, list building may also be scheduled based on heuristics to suit load balancing requirements in multiprocessor runs.
<i>nbtell=0</i>	The <i>nbtell</i> option is not particularly useful and is ignored.
<i>eedmeth=1</i>	Only a cubic spline switch function (<i>eedmeth</i> = 1) for the direct sum Coulomb interaction is supported. This is the default, and most widely used setting for <i>eedmeth</i> . On some machine architectures, energies and forces are actually splined as a function of r^{**2} to a higher precision than the cubic spline switch.
<i>column_fft=0</i>	This is a sander-specific performance optimization option. PMEMD uses different mechanisms to enhance performance, and ignores this option.

It is suggested that new PMEMD users simply take an existing sander mdin file and attempt a short 10-30 step run. The output will indicate whether or not PMEMD will handle the particular problem at hand for all the functionality that is supported by "standard" sander. For functionality that requires special builds of sander or sander-derived executables (LES), there may be failures in namelist parsing.

23.3. PMEMD-specific namelist variables

The following namelist options are specific to PMEMD and generally relate to PMEMD specific performance optimizations: default values:

mdout_flush_interval In &cntrl, this variable can be used to control the minimum time in integer seconds between "flushes" of the mdout file. PMEMD DOES NOT use file flush() calls at all because flush functionality does not work for all Fortran compilers used in building pmemd. Thus, pmemd does an open/close cycle on mdout at a default minimum interval of 300 seconds. This interval can be changed with this variable if desired in the range of 0-3600. If *mdout_flush_interval* is set to 0, then mdout will be reopened and closed for each printed step. This functionality is provided in pmemd because some large systems have such large file i/o buffers that mdout will have 0 length on the disk through 100's of psec of simulated time. The default of 300 seconds provides a good compromise between efficiency and being able to observe the progress of the simulation.

mdinfo_flush_interval In &cntrl, this variable can be used to control the minimum time in integer seconds between "flushes" of the mdinfo file. PMEMD DOES NOT use file flush() calls at all because flush functionality does not work for all Fortran compilers used in building pmemd. Thus, pmemd does an open/close cycle on mdinfo at a default minimum interval of 60 seconds. This interval can be changed with this variable if desired in the range of 0-3600. Note that mdinfo under pmemd simply serves as a heartbeat for the simulation at *mdinfo_flush_interval*, and mdinfo probably will not be updated with the last step data at the end of a run. If *mdinfo_flush_interval* is set to 0, then mdinfo will be reopened and closed for each printed step.

es_cutoff, *vdw_cutoff* In &cntrl, these variables can be used to control the cutoffs for vdw and electrostatic direct force interactions in PME calculations separately. If you specify these variables, you should not specify the cut variable, and there is a requirement that *vdw_cutoff* >= *es_cutoff*. These were introduced anticipating the need to support force fields where the direct force calculations are more expensive. For the current force fields, one can get slightly improved performance and about the same accuracy as one would get using a single cutoff. A good example would be using *vdw_cutoff*=9.0, *es_cutoff*=8.0. For this scenario, one gets about the accuracy in calculations associated with 9.0 angstrom cutoffs, but at a cost intermediate between an 8.0 and a 9.0 angstrom

23. pmemd

cutoff. As stated above, this feature is not currently available on the GPU, and the *cut* variable should be used exclusively.

no_intermolecular_bonds In &cntrl. New variable controlling molecule definition. If 1, any molecules (ie., molecules as defined by the prmtop) joined by a covalent bond are fused to form a single molecule for purposes of pressure and virial-related operations; if 0 then the old behaviour (use prmtop molecule definitions) pertains. The default is 1; a value of 0 is not supported with force-fields using extra points. This option was necessitated in order to efficiently parallelize model systems with extra points. This redefinition of molecules actually allows for a more correct treatment of molecules during pressure adjustments and should produce better results with less strain on covalent bonds joining prmtop-defined molecules, but if the default value is used for a NTP simulation, results will differ slightly relative to sander if any intermolecular bonding was applied in forming the prmtop (eg., a cyx-cyx bridge was added between two peptides that originated in a PDB file, with each peptide having its own "TER" card). If consistency with sander is more important to you, and you are not using extra points, then you may want to set *no_intermolecular_bonds* to 0.

ene_avg_sampling In &cntrl. New variable controlling the number of steps between energy samples used in energy averages. If not specified, then ntp is used (default). To match the behaviour of sander or PMEMD v9 or earlier, this variable should be set to 1. This variable is only used for MD, not minimization and will also effectively be turned off if *ntave* is in use (non-0) or RESPA is in use (*nrespa* > 1). It is a fairly common situation that it is completely unnecessary to sample the energies every step to get a good average during production, and this is costly in terms of performance. Thus, performance can be improved (with greatest improvements for the ensembles in the order NVE > NVT > NTP) without really losing anything of value by using the new default for energy average sampling (specify nothing).

use_axis_opt In &ewald. For parallel runs, the most favorable orientation of an orthogonal unit cell is with the longest side in the Z direction. Starting with pmemd 3.00, internal coordinates were actually re-oriented to take advantage of this, and in high processor count runs on oblong unit cells, using axis optimization can improve performance on the order of 10%. However, if a system has hotspots, the results produced with axes oriented differently may vary by on the order of 0.05% relatively quickly. This effect has to do with the fact that axis optimization changes the order of LOTS of operations and also the fft slab layout, and under mpi if the system has serious hotspots, shake will come up with slightly different coordinate sets. This is really only a problem in pathological situations, and then it is probably mostly telling you that the situation is pathological, and neither set of results is more correct (typically the ewald error term is also high). In routine regression testing with over a dozen tests, axis reorientation has no effect on results. Nonetheless, the defaults are now selected to be in favor of higher reproducibility of results. Axis optimization is only done for mpi runs in which an orthogonal unit cell has an aspect ratio of at least 3 to 2. It is turned off for all minimization runs and for runs in which velocities are randomized (*ntt* = 2 or 3). If you want to force axis optimization, you may set *use_axis_opt* = 1 in the &ewald namelist. If you set it to 0, you will force it off in scenarios where it would otherwise be used.

fft_grids_per_ang In &ewald. This variable may be used to set the desired reciprocal space fft grid density in terms of fft grids/angstrom. The nearest grid dimensions, given the prime factors supported by the underlying fft implementation, that meet or exceed this density will be used (ie., *nfft1,2,3* are set based on this specification). The default value is 1.0 grids/angstrom and gives very reasonable accuracy. PMEMD is actually more stringent now than sander in that it will meet or exceed the desired density instead of just approximating it. Thus, to get identical results with sander, one may have to specify grid dimensions to be used with the *nfft1,2,3* variables.

23.4. Slightly changed functionality

An I/O optimization has been introduced into PMEMD. If the user does not specify a value of NTWR then it defaults to NSTLIM. In general, frequent writes of restrt, especially in runs with a high processor count or on GPUs, is wasteful. Also, if the mden file is being written, it is always written as formatted output, regardless of the value of *ioutfm*. SANDER now conforms to this convention regarding *ioutfm* and mden.

In thermodynamic integration calculations, the input format is different from SANDER. The differences are explained in section 26.1 of the manual.

In addition, there are two command-line options unique to pmemd:

-l <logfile name> A name may be assigned to the log file on the command line.

-gpes <process_map_file> This option controls the distribution of threads in a *multipmemd* simulation and allows you to allocate threads to various processes however you want (rather than dividing up all threads equally between each line of the groupfile). By default, the threads are allocated sequentially (that is, for N groups given M threads per group, or N*M threads total, threads 0 to M-1 will be assigned to the first group, threads M to 2M-1 will be assigned to the second group, etc.). Using the -ng-nonsequential flag, threads will be allocated with a one-at-a-time approach. For instance, given the same setup as above, the first group gets threads 0, M, 2M, 3M, ... etc, the second group gets threads 1, M+1, 2M+1, 3M+1, ... etc. The process_map_file is a file that contains as many lines as you have groups (although the last line can be omitted, and the remaining unspecified threads will be assigned to the final group). Each line must contain space-delimited integers that correspond to the thread numbers you want assigned to that group. Each group is assigned the threads listed in that line of the process_map_file. Every thread must be specified in the process_map_file, and no thread can be specified more than once.

23.5. Parallel performance tuning and hints

As of Amber 18, PMEMD now supports a neutral territory method (by turning on the midpoint variable) that allows for 3-D domain decomposition of the simulation space. This means it is possible to divide the work each process does in the non-bond space by dividing the simulation space into equivalent smaller boxes whereby each process updates and keeps track of atoms it owns. The midpoint method improves memory usage (ongoing) and dramatically improves the communication bottleneck from O(N) in the Amber16 pmemd.MPI to O(N/P) allowing for stronger scaling across processors. As the midpoint method is still undergoing development and is considered experimental, it is recommended to consult <https://ambermd.org/intel/midpoint.htm> to check on currently supported features and information on compiling for further optimizations such as openmp and spdq mixed precision models.

In order to achieve higher scaling in other contexts, pmemd has implemented several performance algorithms, the most notable of which is the option of using a "block" or pencil fft rather than the usual slab fft algorithm. The block fft algorithm allows the reciprocal space and fft workload to be distributed to more processors, but at a cost of higher communications overhead, both in terms of the distributed fft transpose cost and in terms of communication of the data necessary to set up the fft grids in the first place. A number of variables in the &ewald namelist can be used to control whether the slab or block fft algorithm is used, how the block division occurs, whether direct force work is also assigned to tasks doing reciprocal space and fft work, whether the master is given any force and energy computation work to do, as opposed to being reserved strictly for handling output and loadbalancing, and the frequency of atom ownership reassignment, an operation that counteracts rising communications costs caused by diffusion. The various namelist variables involved have all been assigned defaults that adapt to run conditions, and in general it is probably best that the user just use the defaults and not attempt to make adjustments. However, in some instances, fine tuning may yield slightly better performance. The variables involved include *block_fft*, *fft_blk_y_divisor*, *excl_recip*, *excl_master*, and *atm_redist_freq*. These are described further in the README under pmemd/src as well as in the sourcecode itself.

Performance depends not only on proper setup of hardware and software, but also on making good choices in simulation configuration. There are many tradeoffs between accuracy and cost, as one might expect, and understanding all of these comes with experience. However, I would like to suggest a couple of good choices for your

simulations, if you have facilities where you can routinely run at high processor count, say 32 processors or more. First of all, there is an implementation of binary trajectory files in *pmemd* and *sander*, based on the netCDF binary file format. This is invoked now using *ioutfm* == 1, assuming you have built either *pmemd* or *sander* with "bintraj" support. Using this output format, i/o from the master process will be more efficient and your filesize will be about half what it would otherwise be. In Amber, *ptraj* can read these new netCDF trajectory files and can convert them to ASCII format if needed. At really high processor count using the netCDF format can be on the order of 10% more efficient than using the standard formatted trajectory output. Secondly, other simulation packages typically use multiple timestepping (*respa*) methods as an efficiency measure. These methods typically sample reciprocal space forces for PME less frequently. Due to the limited use of such methods by Amber users this approach has not been optimized in *pmemd* and hence while this can slightly improve performance for *pmemd* at low processor count, at higher processor counts using *respa* typically makes loadbalancing less efficient leading to a net loss of performance. If you wish to use *respa* for pme simulations (done typically by setting *nrespa* to 2 or 4), then you should check whether you actually get better performance. You may well not, and it will be at a cost of a loss in accuracy. The GPU PME code requires *nrespa* to be 1, and for highly tuned runs making use of a 4fs time step with SHAKE and hydrogen mass repartitioning, *nrespa* = 2 would be dangerous, in principle, with any version of the code. Using *nrespa* for generalized Born simulations is fine in all cases, however.

23.6. GPU Accelerated PMEMD

One of the important features of PMEMD, is the ability to use NVIDIA GPUs to accelerate both explicit solvent PME and implicit solvent GB simulations [539–541]. This aspect of the code base is currently maintained by Taisung Lee and others, with contributions from David Cerutti and others, and is based on the solid foundation of Scott Le Grand and Ross Walker in collaboration with NVIDIA. While GPU acceleration is a longstanding feature, the error checking is not as verbose in the GPU code as it is on the CPU and some aspects remain more constrictive. In particular, simulation failures, such as atom collisions or other simulation instabilities, will manifest themselves as CUDA launch errors or GPU download failures and not as informative error messages. Due to certain aspects of our GPU pair list operation, we have found that it is unsafe to run simulations that are less than three times the non-bonded cutoff in any given direction, and these cases will be trapped with an error message until a fix can be implemented. Furthermore, the pair list is only designed once on the current GPU implementation and may become invalid if the simulation box shrinks too drastically, as may happen in systems that have not undergone pressure equilibration. (The code will trap such simulations with an error in these circumstances.) If you encounter problems during a simulation on the GPU you should first try to run the identical simulation on the CPU to ensure that it is not your simulation setup which is causing problems. Feedback and questions should be posted to the Amber mailing list (see <http://lists.ambermd.org/>).

This section of the manual describes the feature set, installation, performance and accuracy considerations and other aspects of GPUs at the time of Amber's release. However, the rapidly changing nature of this field means that frequent updates are likely. You should refer to the web page <https://ambermd.org/gpus/> for the most up to date information.

23.6.1. Supported Features

The GPU accelerated version of PMEMD supports both explicit solvent PME simulations in all three canonical ensembles (NVE, NVT and NPT) and implicit solvent Generalized Born simulations. It has been designed to support as many of the standard PMEMD features as possible, however, there are some current limitations that are detailed below. Some of these may be addressed in the future, and patches released, with the most up to date list posted on the web page. The following options are **NOT** supported:

1. *ibelly* $\neq 0$ Simulations using belly style constraints are not supported.
2. *igb* $\neq 0$ && *cut* < *systemsize* GPU accelerated implicit solvent GB simulations do not support a cutoff.
3. *nmropt* > 1 Support is not currently available for *nmropt* > 1. In addition for *nmropt* = 1 only features that do not change the underlying force field parameters are supported. For example umbrella sampling restraints

are supported as is Jarzynski sampling as well as simulated annealing functions such as variation of Temp0 with simulation step. However, varying the VDW parameters with step is not supported.

4. *nrespa* $\neq$ 1 No multiple time stepping is supported.
5. *imin* = 1 with MPI Minimization is currently only supported for single GPU runs.
6. *es_cutoff* $\neq$ *vdw_cutoff* Independent cutoffs for electrostatics and van der Waals are not supported on GPUs.
7. *order* > 4 PME interpolation orders of greater than 4 are not supported at present.
8. *imap* > 0 EMAP restraints are not supported on GPUs.

Additionally there are some minor differences in the output format. For example the Ewald error estimate is *not* calculated when running on a GPU, in fact a completely different spline table format is used. However, for the purposes of nearly all investigators, the GPU diagnostic output will be sufficient, and numerical accuracy will track the CPU code in that the errors inherent non-bonded forces for most run settings will dwarf the errors incurred by 32-bit calculation, accumulation, and integration.

23.6.2. Supported GPUs

GPU accelerated PMEMD has been implemented using CUDA and thus will only run on NVIDIA GPUs at present. Due to accuracy concerns with pure single precision the code uses a custom designed hybrid single/double/integer-precision model termed SPFP. This places the requirement that the GPU hardware supports both double precision and integer atomics meaning only GPUs with hardware revision 3.0 or later (Kepler architecture, GTX 680 and higher) can be used. Support is provided for Tesla, Quadro and GeForce GPUs and almost all mid to high end cards are supported. The new Volta architecture is supported, with the option of a special configuration flag (*-volta*) which optimizes critical kernels for even more speed on the latest NVIDIA hardware. You are encouraged to visit the AMBER website (ambermd.org/gpus/) for an update to date list of supported hardware and performance numbers.

You should ensure that all GPUs on which you plan to run PMEMD are connected to PCI-E 2.0 x16 lane slots or better. For peer to peer communication (required for multi-GPU scaling) the cards to be used need to be in the same PCI-E domain. If this is not the case then you will likely see significantly degraded performance in parallel. For more information, and details of optimum hardware designs, please see Recommended Hardware under the GPU section of the AMBER website (<https://ambermd.org/gpus/>).

Support is provided for single GPU and multiple GPU runs. Employing multiple GPUs in a single simulation requires MPI and the `pmemd.cuda.MPI` executable. If you have multiple simulations to run then the recommended method is to use one GPU per job. The `pmemd` GPU code has been developed in such a way that for single GPU runs the PCI-E bus is only used for I/O. This sets AMBER apart from other MD packages since it means the CPU specs do not feature in the GPU code performance. As such low end economic CPUs can be used. Additionally it means that in a system containing 4 GPUs 4 individual calculations can be run at the same time without interfering with each other's performance. Selection of which GPU is used for single GPU runs is automatic if the GPUs are set to process exclusive mode (`nvidia-smi -c 3`) but the recommended approach is to use the `CUDA_VISIBLE_DEVICES` environment variable to select which GPU should be used. For parallel runs you specify the same number of MPI threads as you have GPUs you want to use. The code will automatically use peer to peer communication if available. Multi-GPU runs require the GPUs to be set to default mode (`nvidia-smi -c 0`). More details are provided in section 23.6.5.

23.6.3. Accuracy Considerations

The nature of current generation GPUs is such that single precision arithmetic is considerably faster than double precision arithmetic, particularly on commodity gaming cards with a paucity of double-precision registers. This poses an issue when trying to obtain good performance from GPUs. Traditionally the CPU code in Amber has always used double precision throughout the calculation. While this full double precision approach has been implemented in the GPU code it gives very poor performance and so the default precision model used when

running on GPUs is a combination of single and fixed precision, termed hybrid precision (SPFP), that is discussed in further detail in references [539–541]. This approach uses single precision for individual calculations within the simulation but fixed scaled integer precision for all accumulations. It also uses fixed precision for shake calculations and for other parts of the code where loss of precision was deemed to be unacceptable. Tests have shown that energy conservation is equivalent to the full double precision code and specific ensemble properties, such as order parameters, match the full double precision CPU code. Previous acceleration approaches, such as the MDGRAPE-accelerated *sander*, have used similar hybrid precision models and thus we believe that this is a reasonable compromise between accuracy and performance. The user should understand though that this approach leads to rapid divergence between GPU and CPU simulations, similar to that observed when running the CPU code across different processor counts in parallel but occurring much more rapidly. Ultimately though this is simply a cosmetic consideration since any statistical mechanical property should converge to the same value.

While the default precision model is currently the hybrid SPFP model two additional precision models have been implemented within version 16 of the GPU code to facilitate advanced testing and comparison. The choice of default precision model may change in the future based on the outcome of detailed validation tests of the different approaches. The precision models supported, all of which are built automatically at compile time are:

- **SPFP** - (*Default*) Use a combination of single precision for calculation and fixed precision for accumulation. This approach is believed to provide the optimum tradeoff between accuracy and performance and hence at the time of release is the default model invoked when using the executable *pmemd.cuda*.
- **DPFP** - Use double precision (and double precision equivalent fixed precision) for the entire calculation. This provides for careful regression testing against the CPU code. It makes no additional approximations above and beyond the CPU implementation and would be the model of choice if performance was not a consideration. On v2.0 NVIDIA hardware (e.g. M2090) the performance is approximately half that of the SPFP model while on v3.0 NVIDIA hardware (e.g. K10) the performance is substantially less than the SPFP model.
- **SPXP** - (*Experimental*) Use single precision for calculation and a combination of 32 bit integer accumulation strategies to approximate 48 bit precision in the summation stage. This precision model has been designed to provide future proofing of performance on next and later generation hardware designs. It is considered experimental at present and should not be used for production simulations except as a way to test how the model performs.

23.6.4. Installation and Testing

The GPU version of PMEMD is called *pmemd.cuda* (or *pmemd.cuda.MPI* for the multi GPU version). Before attempting to build the GPU version of PMEMD you should have built and tested at least the serial version of Amber. This will help to ensure that basic issues relating to standard compilation on your hardware and operating system do not lead to confusion with GPU related compilation and testing problems. You should also be familiar with Amber's compilation and test procedures.

It is assumed that you have already correctly installed and tested CUDA support on your GPU. Additionally the environment variable `CUDA_HOME` should be set to point to your NVIDIA Toolkit installation and `$CUDA_HOME/bin/` should be in your path. Information about supported GPU's and NVIDIA Toolkits is available at ambermd.org/GPUSupport.php. Note that the instructions below will also install CUDA versions of *cpptraj* and *pbsa*.

Building and Testing the GPU code

Assuming you have a working CUDA installation you can build all three precision models (*pmemd.cuda_SPFP*, *pmemd.cuda_DPFP* and *pmemd.cuda_SPXP*) with *pmemd.cuda* linked to *pmemd.cuda_SPFP* as follows:

```
export CUDA_HOME=/usr/local/cuda      (or other appropriate location)
cd /home/xxxx/amber20_src/build        (again, you must replace
                                       /home/xxxx with your chosen location)
# edit the run_cmake script to set -DCUDA=TRUE (you should also be using the GNU compi
```



```
./run_cmake
make install
```

Next you can run the tests using the default GPU (the one with the largest memory followed by lowest GPU ID) with:

```
cd $AMBERHOME
export CUDA_VISIBLE_DEVICES=0    (choose the GPU id you wish to test)
make test.cuda.serial
```

The majority of these tests should pass. However, given the parallel nature of GPUs, the synchrony of multiple threads is not guaranteed and certain features of the code rely on GPU kernels to generate random number sequences that may not always agree. It is not uncommon for there to be several possible failures. You may also see some tests fail with minute differences in one or two output values, even when running the tests with the double precision GPU code. The single precision GPU code will encounter higher numbers of failures—expect roughly 40% of the tests to fail—but again the differences should still be small. Extract lines containing “maximum relative error” or “maximum absolute error” from the compiled diffs file created in the *\$AMBERHOME/logs/test_amber_cuda/* directory to quickly assess the fidelity of your build. You can inspect the diff file manually to verify any possible failures. Differences which occur on only a few lines and are minor in nature can be safely ignored. Any large differences, or if you are unsure, should be posted to the Amber mailing list for comment.

Building pmemd.cuda.MPI

The GPU version of pmemd can be run in parallel on multiple GPUs using the executable pmemd.cuda.MPI. Some simulations, particularly replica exchange simulations, require a parallel executable in order to operate. Please note, however, that most users do not need a parallel GPU version, since parallel scaling across many GPUs is still very poor.

Assuming you have a working CUDA and MPI installation you can build *pmemd.cuda.MPI* as follows:

```
cd /home/xxxx/amber20_src/build
# edit the run_cmake script to have -DMPI=TRUE -DCUDA=TRUE
./run_cmake
make install
```

Next you can run the tests using the default GPUs (the one with the largest memory in descending order) with:

```
cd $AMBERHOME
export DO_PARALLEL="mpirun -np 2" # for bash/sh
make test.cuda.parallel
```

The majority of these tests should pass. However, as described above it is not uncommon for there to be some possible failures. You should inspect the diff file created in the *\$AMBERHOME/logs/test_amber_cuda_parallel/* directory to manually verify any possible failures. Differences which occur on only a few lines and are minor in nature can be safely ignored. Any large differences, or if you are unsure, should be posted to the Amber mailing list for comment.

Building pmemd.cuda.MPI with NCCL support

The NVIDIA Collective Communications Library (NCCL) is a library of multi-GPU collective communication primitives that are topology-aware. NCCL can be enabled when running on more than 2 GPUs in the same node. This may improve multi-GPU scaling, especially on systems with NVLINKs between GPUs. NCCL requires glibc 2.17 or higher CUDA 10.0 or higher, and runs on GPU's with a compute capability of 3.5 (K80 equivalent) and higher.

To enable NCCL, first install NCCL on your system. There are two ways to install NCCL. To install NCCL from source:

23. pmemd

```
git clone https://github.com/NVIDIA/nccl.git
cd nccl
git checkout `git tag | tail -n1`
make src.build CUDA_HOME=/path.to.cuda.toolkit/
```

This installs NCCL to the directory nccl/build.

Alternatively, pre-built NCCL packages can be downloaded from NVIDIA's website. See <https://docs.nvidia.com/deeplearning/sdk/nccl-install-guide/index.html> for details.

Next, the environment variable NCCL_HOME should be set to point to NCCL install path.

Finally, to enable NCCL in Amber, add `-DNCCL=TRUE` to the cmake configure options. Note NCCL build requires both MPI and CUDA to be enabled.

23.6.5. Running GPU Accelerated Simulations

In order to run a GPU accelerated MD simulation the only change required is to use the executable `pmemd.cuda` in place of `pmemd`. E.g.

```
$AMBERHOME/bin/pmemd.cuda -O -i mdin -o mdout -p prmtop \
-c inpcrd -r restrt -x mdcrd
```

This will automatically run the calculation on the GPU with the most memory even if that GPU is already in use. If you have only a single CUDA capable GPU in your machine then this is fine; however if you want to control which GPU is used, or you want to run multiple independent simulations using different GPUs, then you manually need to specify the GPU to use with the `CUDA_VISIBLE_DEVICES` environment variable.

`CUDA_VISIBLE_DEVICES` Specifies which GPU should be used for running a GPU accelerated PMEMD calculation. This is based on the hardware ID of the GPU card which can be obtained by unsetting the variable (unset `CUDA_VISIBLE_DEVICES`) and running the `deviceQuery` command from the NVIDIA CUDA SDK. Valid values are a list of integers from 0 to 32. Multiple GPUs may be listed with commas in between them, and the one with the most memory will be selected. For example:

```
export CUDA_VISIBLE_DEVICES=1,3
$AMBERHOME/bin/pmemd.cuda -O -i mdin -o mdout -p prmtop \
-c inpcrd -r restrt -x mdcrd
```

In this way it is possible to make use of multiple GPUs in a single node for multiple simultaneous calculations. When running a single calculation across multiple GPUs using `pmemd.cuda.MPI` it also allows the selection of specific GPUs on specific nodes. For example running a 2 GPU job with `mpirun -np 2 pmemd.cuda.MPI` with the above listed `CUDA_VISIBLE_DEVICES` would automatically use the second (id=1) and fourth (id=3) GPU in the node. The multi GPU code will avoid assigning MPI GPU tasks to the same GPU if sufficient GPUs are visible. For a more indepth explanation of running GPU accelerated calculations, including how to utilize the peer to peer support in parallel please refer to the GPU section of the AMBER website (<https://ambermd.org/gpus/>).

23.6.6. Considerations for Maximizing GPU Performance

There are a number of considerations above and beyond those typically used on a CPU for maximizing the performance achievable for a GPU accelerated PMEMD simulation. The following provides some tips for ensuring good performance.

1. Avoid using small values of NTPR, NTWX, NTWV, NTWE and NTWR. Writing to the output, restart and trajectory files too often can hurt performance even on CPU runs; however, this is more acute for GPU accelerated simulations because there is a substantial cost in copying data to and from the GPU. Performance is maximized when CPU to GPU memory synchronizations are minimized. This is achieved by computing as much as possible on the GPU and only copying back to CPU memory when absolutely necessary. There

is an additional overhead in that performance is boosted by only calculating the energies when absolutely necessary, hence setting NTPR or NTWE to low values will result in excessive energy calculations. You should typically not set any of these values to less than 100 (except 0 to disable them) and ideally use values of 1000 or more. >100000 for NTWR is ideal, or even better let it just default to NSTLIM.

2. Avoid setting $n_{\text{tave}} \neq 0$. Turning on the printing of running averages results in the code needing to calculate both energy and forces on every step. This can lead to a performance losses of 20% or more when running on the GPU. This can also affect performance on CPU runs although the difference is not as marked. Similar arguments apply to setting the value of `ene_avg_sampling` to small values.
3. Avoid using the NPT ensemble ($n_{\text{tb}}=2$) when it is not required; if needed make use of the Monte Carlo barostat (`barostat=2`). Performance will generally be $\text{NVE} > \text{NVT} > \text{NPT}$ ($\text{NVT} > \text{NPT}$ for `barostat=2`).
4. Use the GPU-suitable GBSA term ($g_{\text{bsa}} = 3$) in implicit solvent GB simulations. Avoid the use of $g_{\text{bsa}} = 1$ unless required. The $g_{\text{bsa}} = 1$ term is calculated on the CPU and thus requires a synchronization between GPU and CPU memory on every MD step, while $g_{\text{bsa}} = 3$ calculates all energy terms on GPU without extra I/O burdens.
5. Use the Berendsen Thermostat ($n_{\text{tt}}=1$) or Anderson Thermostat ($n_{\text{tt}}=2$) instead of the Langevin Thermostat ($n_{\text{tt}}=3$). Langevin simulations require very large numbers of random numbers which slows performance slightly. The "middle" scheme significantly improves the sampling accuracy. The "middle" scheme supports only the Langevin Thermostat now (`ithermostat = 1`). The "middle" scheme with the Anderson Thermostat will be integrated soon.
6. Set `netfrc=0` in the `&ewald` namelist to get the 'legacy' operation on `pmemd.cuda`, which did not calculate or remove the net force during PME simulations. The net force arises from the mesh calculation, but due to other algorithmic decisions can only be properly removed at the end of the force calculation. Generally the effect is to randomly nudge the system as a whole very slightly in different directions with each PME grid calculation. The `nscm` setting can be used to remove any net momentum on a much less frequent time scale. Overall, the performance cost of this net force removal is a fraction of 1% of the total time, however, so meticulous researchers can use net force removal conveniently. Note that one should also set `netfrc=0` when $n_{\text{tr}} > 0$.
7. Do not assume that for small systems the GPU will always be faster. Typically for GB simulations of less than 150 atoms and PME simulations of less than 5,000 atoms it is not uncommon for the CPU version of the code to outperform the GPU version on a single node. Typically the performance differential between GPU and CPU runs will increase as atom count increases. Additionally the larger the non-bond cutoff used the better the GPU to CPU performance gain will be.
8. When running in parallel across multiple GPUs you should use a `pmemd.cuda.MPI` that is built with the MVAPICH MPI library's MVAPICH2-GDR GPU to GPU direct communication capability. See Section 2.2.8 for details. If you are not using that capability then you should restrict jobs to a single node and select GPUs that are on the same PCIe domain and thus can communicate via peer to peer. For a discussion of PCIe topologies in modern hardware see the following writeup <http://tinyurl.com/h469f73>. For most budget 4 GPU nodes `gpus 0` and `1` can typically communicate via peer to peer and `gpus 2` and `3` can communicate via peer to peer. Thus you should not attempt to run a simulation across GPU combinations `0` and `2`, or `0` and `3` or `1` and `2` or `1` and `3`. The `mdout` file contains a section that indicates if peer to peer support is enabled.
9. If you see that performance when running multiple - multi-GPU runs is bad, that is to say you run 2×2 GPU jobs and they don't both run at full speed as if the other job was never running, then make sure you turn off thread affinity within your MPI implementation or at least set each MPI thread to use a different core. In our experience MPICH does not have this on by default and so no special settings are needed; however, both MVAPICH and OpenMPI may set thread affinity by default. This could be useful, but it seems that no attention is paid to load or even the presence of other MVAPICH or OpenMPI runs, and they always assign from core 0. So 2×2 GPU jobs may be, rather foolishly, assigned to cores 0 and 1 in both cases. The simplest solution is to just disable thread affinity as follows:

23. *pmemd*

- a) MVAPICH: `export MV2_ENABLE_AFFINITY=0; mpirun -np 2 ...`
 - b) OpenMPI: `mpirun -bind-to none -np 2 ...`
10. Turn off ECC (Tesla models C2050 and later). ECC can cost you up to 10% in performance. You should verify that your GPUs are working correctly, and not giving ECC errors for example before attempting this. You can turn this off on Fermi based cards and later by running the following command for each GPU ID as root, followed by a reboot:

```
nvidia-smi -g 0 --ecc-config=0 (repeat with -g x for each GPU ID)
```

Extensive testing of AMBER on a wide range of hardware has established that ECC has little to no benefit on the reliability of AMBER simulations. This is part of the reason it is acceptable (see recommended hardware) to use the GeForce gaming cards for AMBER simulations. For more details of ECC and MD simulations see the following paper [\[542\]](#).

24. Atom and Residue Selections

There are three ways to select atoms and residues in AMBER-related routines: the **AMBER "mask"** notation, used by most programs, the **NAB "atom expressions"**, which work only with NAB-compiled applications, and an older "GROUP" specification used in *sander* and *pmemd*. Information about these is collected in this chapter.

24.1. Amber Masks

A "mask" is a notation which selects atoms or residues for special treatment. A frequent usage is fixing or tethering selected atoms or residues during minimization or molecular dynamics.

The following lines are partially copied from the original AMBER documentation. For more details, refer to the entire section of that documentation describing the *ambmask* utility.

The "mask" selection expression is composed of "elementary selections". These start with ":" to select by residues, or "@" to select by atoms. Residues can be selected by numbers (given as numbers separated by commas, or as ranges separated by a dash) or by names (given as a list of residue names separated by commas). The same holds true for atom selections by atom numbers or atom names. In addition, atoms can be selected by AMBER atom type, in which case "@" must be immediately followed by "%". The notation ":*" means all residues and "@*" means all atoms. The following examples show the usage of this syntax.

Residue Number List Examples

```
:1-10      = "residues 1 to 10"  
:1,3,5     = "residues 1, 3, and 5"  
:1-3,5,7-9 = "residues 1 to 3 and residue 5 and residues 7 to 9"
```

Residue Name List Examples

```
:LYS       = "all lysine residues"  
:ARG,ALA,GLY = "all arginine and alanine and glycine residues"
```

Atom Number List Examples

Note that these masks use the **actual sequential numbers of atoms** in the file. This is tricky and a serious source of error. You must know these numbers correctly. Using the atom numbers of a PDB file written out by an AMBER tool is an appropriate way to avoid pitfalls. **Do not use the original atom numbers from the raw PDB file you started with.**

```
@12,17     = "atoms 12 and 17"  
@54-85     = "all atoms from 54 to 85"  
@12,54-85,90 = "atom 12 and all atoms from 54 to 85 and atom 90"
```

Atom Name List Examples

```
@CA           = all atoms with the name CA (i.e., all C-alpha atoms)
@CA,C,O,N,H   = all atoms with names CA or C or O or N or H
                (i.e., the entire protein backbone)
```

Atom Type List Examples

This last mask type is only used by specialists and mentioned here for completeness. It allows the selection of AMBER atom types and requires detailed knowledge of AMBER force fields.

```
@%CT          = all atoms with the force field type CT
                (the standard sp3 aliphatic carbon)
@%N*,N3       = all atoms with the force field type N* or N3
                (N* is a special sp2 nitrogen, N3 is an sp3 nitrogen)
```

Note that in the above example, N* is actually an atom type. The * is **not** a wild card meaning "all N-something types"!

Logical Combinations

The selections above can be combined by various logical operators, including selections like "all atoms within a certain distance from...". The use of such combinations goes beyond this introductory script. Interested users should refer to the next section for details.

24.1.1. ambmask

NAME

ambmask - test group input FIND mask (or mask string given in the &cntrl section) and dump the resulting atom selection in a given format

SYNOPSIS

```
ambmask -p prmtop -c inpcrd -prnlev [0-3] -out [short|pdb|amber] -find [maskstr]
```

DESCRIPTION

ambmask acts as a filter that inputs an Amber topology file and an Amber coordinate file and applies the "maskstr" selection string to select specific atoms or residues. (The "maskstr" selection string is similar syntactically to UCSF Chimera/Midas.) Residues can be selected by their numbers or names. Atoms can be selected by numbers, names, or Amber (forcefield) type. Selections are case insensitive. The selected atoms are printed to **stdout** (by default, in Amber-style PDB format). Atom and residue names and numbers are taken from the Amber topology. Beware that the selection string works on those names and not the ones from the original PDB file. If you are not sure how atoms or residues are named or numbered in the Amber topology, use **ambmask** with a selection string ":" (which is the default) to dump the whole PDB file with corresponding Amber atom/residue names and numbers.

The "maskstr" selection expression is composed of "elementary selections". These start with ":" to select by residues, or "@" to select by atoms. Residues can be selected by numbers (given as numbers separated by commas, or as ranges separated by a dash) or by names (given as a list of residue names separated by commas). The same holds true for atom selections by atom numbers or atom names. In addition, atoms can be selected by Amber atom type, in which case "@" must be immediately followed by "%". ":" means all residues and "@*" means all atoms. The following examples show the usage of this syntax. Square brackets should not be used in actual expressions, they are only used below to denote individual selection string examples:

```

:{residue numlist} [:1-10] [:1,3,5] [:1-3,5,7-9]
:{residue namelist} [:LYS] [:ARG,ALA,GLY]
@{atom numlist} [@12,17] [@54-85] [@12,54-85,90]
@{atom namelist} [@CA] [@CA,C,O,N,H]
@%{atom typelist} [%CT] [%N*,N3]

```

These "elementary selections" can be combined into more complex selections using binary operators "&" (and) and "|" (or), unary operator "!" (negation), distance binary operators "<:", ">:", "<@", ">@", and parentheses. Spaces around operators are irrelevant. Parentheses have the highest priority, followed by distance operators ("<:", ">:", "<@", ">@"), "!" (negation), "&" (and) and "|" (or) in order of descending priority. A wildcard "=" in an atom or residue name matches any name starting with a given character (or characters). For example, [:AS=] would match all aspartic acid residues (ASP), and asparagines (ASN); [@H=] would match all atom names starting with H (which are effectively all hydrogens). It cannot be used to match the end part of names (such as [:=A]). Some examples of more complex selections follow:

```
[@C= & !@CA,C]
```

.. all carbons except backbone alpha and carbonyl carbon

```
[(:1-3@CA | :5-7@CB)]
```

.. alpha carbons in residues 1-3 and beta carbons in residues 5-7

```
[:CYS,ARG & !(:1-10 | @CA,CB)]
```

.. all CYS and ARG atoms except those which are in residues 1-10 and which are CA or CB

```
[:* & !@H=] or [!@H=]
```

.. all heavy atoms (i.e. except hydrogens)

```
[:5 <@4.5]
```

.. all atoms within 4.5A from residue 5

```
[(:1-55 <:3.0) & :WAT]
```

.. all water molecules within 3A from residues 1-55

Compound expressions of the following type are also allowed:

```

:{residue numlist|namelist}@{atom numlist|namelist|typelist}
[:1-10@CA] is equivalent to [:1-10 & @CA]
[:LYS@H=] is equivalent to [:LYS & @H=]

```

OPTIONS

The program needs an Amber topology file and coordinates (restrt format). The filename specified with the *-p* option is Amber topology, while the filename given with the *-c* option is a coordinate file. If *-p* or *-c* options are not given, the program expects that files "prmtop" and/or "inpcrd" exist in the current directory, which will be taken as topology and coordinate files correspondingly. If no command line options are given, the program prints the usage statement.

The option *-prnlev* specifies how much (debugging) information is printed to **stdout**. If it is 0, only selected atoms are printed. More verbose output (which might be useful for debugging purposes) is achieved with higher values: 1 prints original "maskstr" in its tokenized (with operands enclosed in square brackets) and postfix (or Reverse Polish Notation) forms; number of atoms and residues in the topology file and number of selected atoms are also printed to **stdout**. 2 prints the resulting mask array, which is an array of integer values, with '1' representing a selected atom, and '0' an unselected one. Value of 3, in addition, prints mask arrays as they are pushed or popped from the stack (this is really only useful for tracing the problems occurring during stack operations). The *-prnlev* values of 0 or 1 should suffice for most uses.

The option *-out* specifies the format of printed atoms. "short" means a condensed output using residue (:) and atom (@) designators followed by residue ranges and atom names. "pdb" (default) prints atoms in Amber-style PDB format with the original "maskstr" printed as a REMARK at the top of the PDB file, and "amber" prints atom/residue ranges in the format suitable for copying into group input section of Amber input file.

The option *-find* is followed by "maskstr" expression. This is a string where some characters have a special meaning and thus express what parts (atoms/residues) of the molecule will get selected. The syntax of this string is explained in the section above (DESCRIPTION). If this option is left out, it defaults to ":", which selects all atoms in the given topology file. The length of "maskstr" is limited to 80 characters. If the "maskstr" contains spaces or special characters (which would be expanded by the shell), it should be protected by single or double quotes (depending on the shell). In addition, for C-shells even a quoted exclamation character may be expanded for history substitution. Thus, it is recommended that the operand of the negation operator always be enclosed in parentheses so that "!" is always followed by a "(" to produce "(!)" which disables the special history interpretation. For example, [*@C= & !(@CA,C)*] selects all carbons except backbone alpha and carbonyl carbon; the parentheses are redundant but shell safe. The man page indicates further ways to disable history substitution.

FILES

Assumes that *prmtop* and *inpcrd* files exist in the current directory if they are not specified with *-p* and *-c* options. Resulting (i.e. selected) atoms are written to **stdout**.

BUGS

Because all atom names are left justified in Amber topology and the selections are case insensitive, there is no way to distinguish some atom names: alpha carbon CA and a calcium ion Ca are a notorious example of that.

24.2. "Atom Expressions" in NAB Applications

NAB applications do not use the AMBER mask scheme outlined in the previous sections. They use simpler (but less powerful) selection criteria. The scheme is:

chains (or "strands") : residues : atoms

For example, *A:GLU:CA* would select all C α carbons of all glutamate residues in chain A. A plain *::* would select all atoms in all residues and all chains (not very useful). *::H** would select all hydrogen atoms in any chain and any residue, the *** being a wild card for any sequence of characters. Similarly, *::*C** would select all atoms which contain at least one "C" character, i.e., the wild card can be used in any position. The *?* can be used as a wild card for a single character. Thus, *::H?* would select any atom starting with H plus one additional character (e.g., HC, HI, HN, but **not** HG11).

The wild card can also be used in residue names. *:A*:* would select all alanines, asparagines, and arginines.

Selections can be combined separated by a vertical bar "|". *:1-3,ALA:C*|:2-5:N** would select all carbon atoms in residues 1 to 3, in all alanines **and** all nitrogen atoms in all residues from 2-5. If you would like to tether all C α atoms of a protein and the oxygen atom of explicit water molecules (with residue names 'WAT'), you would use *::CA|:WAT:O**.

Output from NAB applications always tells how many atoms have been selected for a special treatment. If you are not sure that your selection is correct, this number might at least be a hint. If you run a simulation with a protein having 200 residues and want to tether all C α carbons, *::CA* should result in 200 selected atoms (provided that all residues have a well-defined CA atom, which they should).

24.3. GROUP Specification

This section describes the format used to define groups of atoms in various Amber programs. In *sander*, a group can be specified as a movable "belly" while the other atoms are fixed absolutely in space (aside from scaling caused by constant pressure simulation), and/or a group of movable atoms can independently restrained (held by

a potential) at their positions. In *anal*, groups can be defined for energy analysis. In *sander* and *pmemd*, GROUP input comes at the end of the *mdin* input file, as discussed in Section 22.5.

Except in the analysis module where different groups of atoms are considered with different group numbers for energy decomposition, in all other places the groups of atoms defined are considered as marked atoms to be included for certain types of calculations. In the case of constrained minimization or dynamics, the atoms to be constrained are read as groups with a different weight for each group.

Reading of groups is performed by the routine RGROUP, and you are advised to consult it if there is still some ambiguity in the documentation.

Input description:

```
- 1 - Title format(20a4)
ITITL Group title for identification.
Setting ITITL = 'END' ends group input.
-----

- 1A - Weight format(f)
This line is only provided/read when using GROUP input to
define restrained atoms.
WT The harmonic force constants in kcal/mol-A**2 for the group
of atoms for restraining to a reference position.
-----

- 1B - Control to define the group
KTPG , (IGRP(I) , JGRP(I) , I = 1,7) format(a,14i)
KTPG Type of atom selection performed. A molecule can be
defined by using only 'ATOM' or 'RES', or part of the
molecule can be defined by 'ATOM' and part by 'RES'.
'ATOM' The group is defined in terms of atom numbers. The atom
number list is given in igrp and jgrp.
'RES' The group is defined in terms of residue numbers. The
residue number list is given in igrp and jgrp.
'FIND' This control is used to make additional conditions
(apart from the 'ATOM' and 'RES' controls) which a given
atom must satisfy to be included in the current group.
The conditions are read in the next section (1C) and are
terminated by a SEARCH card.
Note that the conditions defined by FIND filter any set(s) of atoms
defined by the following ATOM/RES instructions. For example,
-- group input: select main chain atoms --
FIND
* * M *
SEARCH
RES 1 999
END
END
'END' End input for the current group. Followed by either another
group definition (starting again with line 1 above), or by a second
'END' "card", which terminates all group input.
IGRP(I) , JGRP(I)
The atom or residue pointers. If ktpg .eq. 'ATOM' all
atoms numbered from igrp(i) to jgrp(i) will be put into
the current group. If ktpg .eq. 'RES' all atoms in the
residues numbered from igrp(i) to jgrp(i) will be put
into the current group. If igrp(i) = 0 the next control
card is read.
It is not necessary to fill groups according to the
numerical order of the residues. In other words, Group 1
could contain residues 40-95 of a protein, Group 2 could
contain residues 1-40 and Group 3 could contain residues
```

96-105.

If ktypg .eq. 'RES', then associating a minus sign with igrp(i) will cause all residues igrp(i) through jgrp(i) to be placed in separate groups.

In the analysis modules, all atoms not explicitly defined as members of a group will be combined as a unit in the (n + 1) group, where the (n) group in the last defined group.

- 1C - Section to read atom characteristics

***** Read only if KTYPG = 'FIND' *****

JGRAPH(I) , JSYMBL(I) , JTREE(I) , JRESNM(I) format(4a)

A series of filter specifications are read. Each filter consists of four fields (JGRAPH,JSYMBL,JTREE,JRESNM), and each filter is placed on a separate line. Filter specification is terminated by a line with JGRAPH = 'SEARCH'. A maximum of 10 filters may be specified for a single 'FIND' command.

The union of the filter specifications is applied to the atoms defined by the following ATOM/RES cards. I.e. if an atom satisfies any of the filters, it will be included in the current group. Otherwise, it is not included. For example, to select all non main chain atoms from residues 1 through 999:

-- group input: select non main chain atoms --

FIND

* * S *

* * B *

* * 3 *

* * E *

SEARCH

RES 1 999

END

END

'END' End input for the current group. Followed by either another

The four fields for each filter line are:

JGRAPH(I) The atom name of atom to be included. If this and the following three characteristics are satisfied the atom is included in the group. The wild card '*' may be used to indicate that any atom name will satisfy the search.

JSYMBL(I) Amber atom type of atom to be included. The wild card '*' may be used to indicate that any atom type will satisfy the search.

JTREE(I) The tree name (M, S, B, 3, E) of the atom to be included. The wild card '*' may be used to indicate that any tree name will satisfy the search.

JRESNM(I) The residue name to which the atom has to belong to be included in the group. The wild card '*' may be used to indicate that any residue name will satisfy the search.

Examples:

The molecule 18-crown-6 will be used to illustrate the group options. This molecule is composed of six repeating (-CH₂-O-CH₂-) units. Let us suppose that one created three residues in the PREP unit: CRA, CRB, CRC. Each of these is a (-CH₂-O-CH₂-) moiety and they differ by their dihedral angles. In order to construct 18-crown-6, the residues CRA, CRB, CRC, CRB, CRC, CRB are linked together during the LINK module with the ring closure being between CRA(residue 1) and CRB(residue 6).

Input 1:

Title one

```

RES 1 5
END
Title two
RES 6
END
END

```

Output 1: Group 1 will contain residues 1 through 5 (CRA, CRB, CRC, CRB, CRC) and Group 2 will contain residue 6 (CRB).

Input 2:

```

Title one
RES 1 5
END
Title two
ATOM 36 42
END
END

```

Output 2: Group 1 will contain residues 1 through 5 (CRA, CRB, CRC, CRB, CRC) and Group 2 will contain atoms 36 through 42. Coincidentally, atoms 36 through 42 are also all the atoms in residue 6.

Input 3:

```

Title one
RES -1 6
END
END

```

Output 3: Six groups will be created; Group 1: CRA, Group 2: CRB,..., Group 6: CRB.

Input 4:

```

Title one
FIND
O2 OS M CRA
SEARCH
RES 1 6
END
END

```

Output 4: Group 1 will contain those atoms with the atom name 'O2', atom type 'OS', tree name 'M' and residue name 'CRA'.

Input 5:

```

Title one
FIND
O2 OS * *
SEARCH
RES 1 6
END
END

```

Output 5: Group 1 will contain those atoms with the atom name 'O2', atom type 'OS', any tree name and any residue name.

Input 6:

```

Title one
RES 1 3 6 6
END
END

```

24. Atom and Residue Selections

Output 6: One group is created containing residues 1 to 3 and 6. Up to seven ranges of contiguous residues can be specified per group. (In this case there are two ranges).

Input 7:

```
First restraint group
10.0
FIND
CA * * *
SEARCH
RES 1 17    25 36
END
Second restraint group, with a different restraint weight
1.0
FIND
CA * * *
SEARCH
RES 61 127
END
END
```

Output 7: CA atoms in residues 1-17 and 25-36 will be restrained to their initial positions with a strong weight of $10.0 \text{ kcal} \cdot \text{mol}^{-1} \cdot \text{\AA}^{-2}$; CA atoms in residues 61 to 127 will have a weaker restraint force constant.

25. Sampling configuration space

The "middle" scheme [Section 22.6.10](available in sander, pmemd and pmdmd.cuda) offers an efficient approach to accurately sample configuration space in standard molecular dynamics simulations. There are many instances when standard molecular dynamics simulations get “stuck” near the starting configuration, and fail to adequately sample the available low-energy configurational space. This chapter describes a variety of techniques that can partially overcome such problems. The following chapter (on Free Energies) continues many of these ideas, adapting them to the calculation of alchemical or configurational free energy differences. There is no good distinction between these two chapters, because good sampling of the canonical distribution and estimation of free energies go hand-in-hand. But the present chapter covers methods that are *primarily* devoted to enhanced or accelerated sampling, whereas the following chapter considers methods that explicitly estimate free energy differences.

25.1. Self-Guided Langevin dynamics

Self-guided Langevin dynamics (SGLD) is designed to enhance conformational search efficiency in either a molecular dynamics (MD) simulation (when $\gamma_{ln}=0$) or a Langevin dynamics (LD) simulation (when $\gamma_{ln}>0$). This method accelerates low frequency motion to enhance conformational sampling. [543–547]

Overview: The input parameter, *tsgavg*, defines the lower limit period of the low frequency motion. Typically, *tsgavg* 0.2 ps is suitable to define bond stretching and bending motions as high frequency motion and is recommended to enhance motions like phase separation, secondary structure folding, and ligand docking, while *tsgavg*=1.0 ps is suitable to include more local motions such as bond rotation and solvent relaxation and is recommended for protein domain motion, and protein-protein docking. The input parameter, *sgft* or *sgff*, defines the strength of the guiding effect. *sgft* between -1 and $+1$ sets the momentum guiding effect, with 0 for regular LD or MD simulations. A larger *sgft* will enhance low frequency motion to accelerate conformational search. *sgff* between -0.32 and $+0.32$ defines a force guiding factor to target energy barriers. A negative value will flatten energy barriers. Normally, *sgft* and *sgff* have opposite effects on conformational distribution and there exist balanced values to conserve the canonical ensemble. Alternatively, one can set *tempsg* to define the effective guiding temperature. SGLD accelerates slow events to an affordable time scale while minimizing the perturbation to the conformational distribution. The guiding force can be applied to a part of a simulation system between atom *isgsta* and atom *isgend*, and/or defined with *sgmask*.

The conformational distribution of SGLD can be reweighted to produce canonical ensemble averages[544, 547, 548]. The most recent reformulation named the generalized self-guided molecular simulation method (SGMDg or SGLDg)[547] replace all previous SGMD/SGLD methods. The previous method[548, 549], SGLDfp is no longer available. As an alternative to Langevin dynamics (LD), self-guided Langevin dynamics via generalized Langevin equation (SGLD-GLE) [546] can exactly preserve canonical ensemble while avoiding the slow down by friction forces. SGLD-GLE method can be turned on by setting *sgfg* between -1 and $+1$. SGLD-GLE can be used with SGMD/SGLD by setting *sgft*, *sgff*, and *sgfg*.

SGLD can be used for replica exchange simulations (RXSGLD)[550] to achieve enhanced sampling with or without elevating temperature. *sgft* or *sgff* can be used to define different replicas. See Section 26.3.6 for a detailed description of RXSGLD.

SGMD/SGLD and RXSGLD are all implemented in PMEMD GPU.

isgld SGLD algorithm index. Default *isgld*= 0, SGLD is disabled; *isgld*=1 will run with uniformed guiding forces; *isgld*=2 will run SGLD/SGMD with atomic specific guiding forces; and *isgld*=3 will balance *sgft* and *sgff* to maintain canonical ensemble distribution, but enhancement in conformational search is limited.

25. Sampling configuration space

tsgavg	Local averaging time (<i>psec</i>) for the guiding force calculation. Default 0.2 <i>psec</i> . A larger value defines slower motion to be enhanced.
sgft	Momentum guiding factor. Defines the strength of the guiding effect. Default 0.0. Suggested value is 1.0 when <i>sgff</i> =0. Its value range is -1~1 with larger value leads to stronger low frequency motion. When <i>sgft</i> is set <-1 or >1, it is reset to the balanced value of <i>sgff</i> to preserve canonical ensemble.
sgff	Force guiding factor. <i>sgff</i> is used to scale down low frequency energy surface by a factor, (1+ <i>sgff</i>). <i>sgff</i> is suggested to take values between -0.32 and 0.32, with default value of 0. Suggested value is -0.3 when <i>sgft</i> =0. When <i>sgff</i> is set <-1 or >1, it is reset to the balanced value of <i>sgft</i> to preserv canonical ensemble.
sgfg	momentum guiding factor for SGLD-GLE. Degault is 0. Its value range is -1~1 with larger value resulting in faster low frequency motion. Suggested value is 1 when <i>sgft</i> =0 and <i>sgff</i> =0.
tempsg	Effective guiding temperature. Default value is 0. <i>tempsg</i> is related to <i>sgft</i> and <i>sgff</i> . If <i>tempsg</i> is set other than 0, when <i>sgft</i> =0, <i>sgft</i> will be calculated based on <i>tempsg</i> and <i>sgff</i> , and when <i>sgft</i> ≠0, <i>sgff</i> will be calculated based on <i>sgft</i> and <i>tempsg</i> . If <i>tempsg</i> is 0, <i>tempsg</i> will be calculated based on <i>sgft</i> and <i>sgff</i> . Actual <i>tempsg</i> is printed out in the output.
isgsta	The first atom index of SGLD region. Default is 1.
isgend	The last atom index of SGLD region. Default is <i>natom</i> .
sgmask	The atom mask for selecting atoms between <i>isgsta</i> and <i>isgend</i> to apply guiding forces (default=':*').
sgtype	type of spatial average to calculate guiding forces. 1 for no bonded structure is considered; 2 for all atoms connected through bonds or angles ; 3 for all atoms connected through bonds, angles, and dihedral angles; 4 for all atoms within a cutoff distance given by <i>sgsize</i> . Default is 1.
sgsize	cutoff distance for local spatial average. Default is 3.0 angstroms.

The output of SGMD/SGLD simulations contains the following properties related to the enhancement in conformational search and reweighting of conformational distribution:

METHOD: GAMM TEMPLF TEMPHF EPOTLF EPOTHF EPOTLLF SGWT

These quantities are instantaneous values defined as below:

METHOD: SGLD when *gamma_ln*>0 or SGMD when *gamma_ln*=0

GAMM: Average atomic friction constant based on atom interactions.

TEMPLF: low frequency temperature

TEMPHF: high frequency temperature. Apparent temperature=TEMPLF+TEMPHF

EPOTLF: low frequency potential energy

EPOTHF: high frequency potential energy, EPOT=EPOTLF+EPOTHF

EPOTLLF: Average of low frequency potential energy. It is needed for reweighting.

SGWT: Weighting number. exp(SGWT) is the weighting factor of current frame.

The weight of a conformation is calculated by

$$\text{Weight} = \exp(\text{SGWT}) = \exp\left(\left(p(\text{sgft}) - \text{sgff}\right) * (\text{EPOTLF} - \text{EPOTLLF}) / (\text{KBOLTZ} * \text{Temp})\right)$$

$$\text{where: } \text{sgft} = (1 + p(\text{sgft}))^{2-1} / (1 + p(\text{sgft}))$$

or:

$$w_i = \exp\left(\frac{\lambda_{FP} - \lambda_F}{kT}(E_{LF} - E_{LLF})\right)$$

where $p(\text{sgft})$ or λ_{FP} is called the balanced force guiding factor of λ_P . The relation between sgft , λ_P , and $p(\text{sgft})$, λ_{FP} , is: $\lambda_P = (1 + \lambda_{FP})^2 - \frac{1}{1 + \lambda_{FP}}$. Similarly, λ_{PF} is called the balanced momentum guiding factor of λ_F , where $\lambda_{PF} = (1 + \lambda_F)^2 - \frac{1}{1 + \lambda_F}$. When both λ_P and λ_F are used in a SGMDg/SGLDg simulation, it is recommended that $-1 \leq \lambda_P - \lambda_{PF} \leq 1$.

For convenience, two scripts, *sgldinfo.sh* and *sgldwt.sh*, are provided in the AMBERHOME/bin directory to extract SGLD properties and weighting factors from sander output files. For example, one can run:

```
sgldinfo.sh mdout
```

to examine the SGLD properties, and run:

```
sgldwt.sh mdout
```

to print weighting factors at each print time frame, averages and reweighted averages of the SGMD/SGLD properties. Ensemble average properties can be calculated through reweighting:

$$\langle P \rangle = \frac{\sum_{i=N_0}^N w_i P_i}{\sum_{i=N_0}^N w_i}$$

Below are example input files to run SGMD/SGLD simulations.

1. a SGLD simulation with *sgft* only:

```
Sample SGLD simulation for enhanced conformational search
&cntrl
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,
ntpr=100, ntwr=100000, ntt=3, gamma_ln=10.0,
ntx=5, irest=1,
ntc=2, ntf=2, tol=0.000001,
dt=0.002, ntb=0, tempi=300., temp0=300.,
isgld=1, tsgavg=0.2, sgft=1.0,
/
```

2. a SGLD simulation using bonded substructure(*nsqsize*=2) to estimate low frequency motion, which reduces noises due to bond stretching and bending.

```
Sample SGLD simulation using bonded substructures
&cntrl
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,
ntpr=100, ntwr=100000, ntt=3, gamma_ln=10.0,
ntx=5, irest=1,
ntc=2, ntf=2, tol=0.000001,
dt=0.002, ntb=0, tempi=300., temp0=300.,
isgld=1, tsgavg=0.2, sgft=1.0, nsqsize=2,
/
```

3. a SGLDg simulation using both *sgft* and *sgff* factors. When both guiding factors are used, it is recommended that $-1 \leq \lambda_P - (1 + \lambda_F)^2 + \frac{1}{1 + \lambda_F} \leq 1$.

```
Sample SGLDg simulation using both sgft and sgff
&cntrl
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,
ntpr=100, ntwr=100000, ntt=3, gamma_ln=10.0,
ntx=5, irest=1,
ntc=2, ntf=2, tol=0.000001,
```

25. Sampling configuration space

```
dt=0.002, ntb=0, tempi=300., temp0=300.,  
isgld=1,tsgavg=0.2,sgft=0.5,sgff=-0.1,  
/  

```

4. a SGMDg simulation when $\gamma_{ln}=0$:

```
Sample SGMDg simulation using both sgft and sgff  
&cntrl  
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,  
ntpr=100, ntwr=100000, ntt=1, tautp=1.0,  
ntx=5, irest=1,  
ntc=2, ntf=2, tol=0.000001,  
dt=0.002, ntb=0, tempi=300., temp0=300.,  
isgld=1,tsgavg=0.2,sgft=0.5,sgff=-0.1,  
/  

```

5. a SGLD-GLE simulation using sgfg to achieve enhanced conformational search while conserve canonical ensemble exactly:

```
Sample SGLD-GLE simulation to conserve canonical ensemble  
&cntrl  
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,  
ntpr=100, ntwr=100000, ntt=3, gamma_ln=10.0,  
ntx=5, irest=1,  
ntc=2, ntf=2, tol=0.000001,  
dt=0.002, ntb=0, tempi=300., temp0=300.,  
isgld=1,tsgavg=0.2,sgfg=1.0,  
/  

```

6. a SGLD simulation using local spatial average to achieve enhanced concerted moment:

```
Sample SGLD simulation to enhance concerted motion  
&cntrl  
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,  
ntpr=100, ntwr=100000, ntt=3, gamma_ln=10.0,  
ntx=5, irest=1,  
ntc=2, ntf=2, tol=0.000001,  
dt=0.002, ntb=0, tempi=300., temp0=300.,  
isgld=1,tsgavg=0.2,sgtype=4,sgsize=4.0,sgft=1.0,  
/  

```

25.2. Accelerated Molecular Dynamics

25.2.1. Introduction

Many systems of interest in chemistry, physics and biology are characterized by the presence of a number of metastable states separated by large barriers. Correctly sampling these systems is challenging for methods based on Molecular Dynamics, Monte Carlo sampling or any other type of dynamic simulation. For most biological systems of interest, the simulation time is limited to the nanosecond-microsecond time scale, so simple molecular dynamics cannot be used to adequately explore portions of the energy landscape separated by high barriers from the initial minimum. Furthermore, for most biological molecules, the energy landscape has multiple minima or potential energy wells with high free energy barriers, and during a molecular dynamics simulation the system is trapped in one or another local minimum for long periods of simulation time. Consequently, thermodynamics and many other properties of interest for large biological systems cannot be simulated directly because of the nonergodic nature of the present state of the molecular dynamics methodology for systems with high free energy barriers.

Accelerated Molecular Dynamics (aMD) is a bias potential introduced by the McCammon group at UCSD [551]. It is a modification to the potential that in practice reduces the height of local barriers, allowing the calculation to evolve much faster. A number of methods have been suggested to aid this problem, like replica exchange, metadynamics, etc. AMD represents an interesting option as it only requires the evolution of a single copy of the system, plus it doesn't require any previous knowledge of the shape of the potential, i.e. aMD doesn't require information of where are the barriers, saddle points or even what type of configuration changes are expected or necessary to traverse through a particular barrier. Moreover, an interesting feature of aMD is that the shape of the added potential conserves the underlying shape of the real one, such that minima are maintained as minima and barriers are preserved as barriers. In result, adding the aMD potential in practice simply modifies the relation between energy differences, so the distribution of sampling of different structures is still related to the original potential distribution and can be recovered exactly by reweighing.

The aMD modification of the potential is defined by the following equation:

$$V(r)* = V(r) + \Delta V(r) \quad (25.1)$$

$$\Delta V(r) = \frac{(E_p - V(r))^2}{(\alpha P + E_p - V(r))} + \frac{(E_d - V_d(r))^2}{(\alpha D + E_d - V_d(r))} \quad (25.2)$$

where $V(r)$ is the normal potential and $V_d(r)$ is the normal torsion potential. E_p and E_d are average potential and dihedral energies that serve as a reference energy from which to compare the present position of the calculation and therefore the relationship to the boosting factor to be applied. The terms αP and αD are factors that determine inversely the strength with which the boost is applied. For large values of alpha, the potential felt at any point will essentially be the same as the true potential. For values of alpha close to zero, the potential felt becomes constant, in this limit, the sampling becomes a random walk. The amount of boost felt at a particular point in the calculation, therefore, depends on the present value of the potential and dihedral energy, which is in direct correlation to how low in the energy surface the configuration is positioned at that moment. The boosting potential will be proportionally bigger for deeper regions of the potential energy surface, while it will be smaller for higher points, which in result conserves the underlying shape of the potential, as previously mentioned.

AMD has been applied to a vast diversity of interesting problems [552–556]. We have recently applied the implementation of aMD in Amber to the Bovine Pancreatic Trypsin Inhibitor and compared with an unbiased millisecond MD simulation, showing aMD is able to recover the right population distribution and shows excellent agreement with the MD simulation as with experimental data [555].

25.2.2. AMD implementation in Amber

AMD has been implemented in both *sander* and *pmemd* by Romelia Salomon-Ferrer. The implementation includes the possibility of boosting independently only the torsional terms of the potential (*iamd*=2) or the whole potential at once (*iamd*=1). It also allows the possibility to boost the whole potential with an extra boost to the torsions (*iamd*=3). All the information generated by aMD, necessary for reweighing is stored at each step into a vector which is flushed to a log file (*amd.log* by default) every time the coordinates are written to disk, i.e. every *ntwx* steps. This is done for performance reasons, since writing to disk is always time consuming and it is not advisable to do it every step. The name of the log file can be set to a user defined name by using the command line option *-amdlog* when running Amber. Our present implementation also allows the user to delay (or lag) the boosting a number of steps, i.e. only boost with a particular frequency defined by the variable *amdlag*. Additional parameters are specified by the following variables: *EthreshD* (E_d), *alphaD* (αD), *EthreshP* (E_p) and *alphaP* (αP).

AMD output is saved the *amd.log* this file contains all the information needed for reweighing the results obtained to recover the unperturbed distributions. The *amd.log* file gets written with the same frequency at which the configurations are saved to disk in the trajectory file (*mdcrd*). Each line corresponds to the information of a corresponding snapshot being saved on the *mdcrd* file. Regardless of what *iamd* value is used, the number of columns in the *amd.log* file are always the same, they just have 0 or 1 (correspondingly) if no boost is being added to dihedral or total energy

The *amd.log* file has the following header:

```
#All energy terms stored in units of kcal/mol
```

25. Sampling configuration space

```
#ntwx,total_nstep,Unboosted-Potential-Energy,Unboosted-Dihedral-Energy,Total-Force-Weight,
Dihedral-Force-Weight,Boost-Energy-Potential,Boost-Energy-Dihedral
```

The description for the main columns is as follows:

- **Unboosted-Potential-Energy:** Total Potential Energy without boost added, kcal/mol.
- **Unboosted-Dihedral-Energy:** dihedral energy without boost added, kcal/mol.
- **Total-Force-Weight:** The force scaling factor calculated from the boost to the Total Potential Energy
- **Dihedral-Force-Weight:** The dihedral force scaling factor from dihedral boost
- **Boost-Energy-Potential:** The boost energy in kcal/mol
- **Boost-Energy-Dihedral:** The dihedral boost energy in kcal/mol

IMPORTANT NOTE: Before Amber 14 the boost energy for the dihedral and total potential energy (last two columns) was given in units of kT . This decision was made at the beginning with the idea that the user could read and use these values directly for reweighting without any further work, but later it was decided it was much better and more consistent to have all energy output in kcal/mol as the rest of AMBER's energy output. As of AMBER 14, the last two columns of the amd.log file are given in units of kcal/mol.

Reweighting

For reweighting aMD results we would like to add the link to a great tutorial (<http://mccammon.ucsd.edu/computing/amdReweighting/>) which also provides a small python script to perform the reweighting. The script is compatible with the newer versions of AMBER, and can be used to reweight 1D and 2D distributions. A simple C code is also provided in the aMD tutorial that performs reweighting based on the Kernel Density Estimation algorithm. This algorithm also performs very well and reduces the amount of noise without using a truncated expression for the exponential and can be used as an alternative for reweighting. To extract the energies from the amd.log file into a file, weights.dat, to use with this script, something like the following could be done:

```
# Column 1: dV in units of kbT; column 2: timestep; column 3: dV in units of kcal/mol
# For AMBER14: # awk 'NR%1==0' amd.log | awk '{print ($8+$7)/(0.001987*300)}' > weights.dat

" " $2 " " ($8+$7)}' > weights.dat

# For AMBER12: # awk 'NR%1==0' amd.log | awk '{print ($8+$7)" " $3
" " ($8+$7)*(0.001987*300)}' > weights.dat
```

For reweighting a 2D distribution, for instance a Phi Psi distribution, you would need to extract the values for Phi and Psi for each frame in the mdcrd file using AmberTools and generate the file Phi_Psi file and then use the python tool provided in the website to get the reweighted surface.

```
python PyReweighting-2D.py -input Phi_Psi -Emax 100 -discX 6 -discY 6

-job amdweight_MC -order 10 -weight weights.dat | tee -a reweight_variable.log
```

For reweighting using a Maclaurin series expansion as an approximation for the exponential weight.

25.2.3. Preparing a system for aMD

As mentioned before, running aMD requires the definition of few parameters. AMD parameters are determined based on previous knowledge of the system, which is easily acquirable by a short regular MD simulation, from which the average values of the potential and torsion energy can be estimated. From there, a given amount of energy per degree of freedom is added those values, in the form of multiples of alpha, setting the values of Ep and Ed to be used. The following example should help clarify this procedure.

```
Average Dihedral : 611.5376 (based on MD simulations)
Average EPtot : -53155.3104 (based on MD simulations)
total ATOMS=16950
protein residues=64
```

For the dihedral potential:

```
Approximate energy contribution per degree of freedom.
3.5*64= 224           The value of 3.5 kcal/mol/residue seems to work well
alphaD = (1/5)*224 = 45 The value of .2 seems to work well
EthreshD = 224+611 = 835
For the total potential
alphaP = 16950*(1/5)=3390
For a lower boost you can also use a value between 0.15-0.19 instead of 0.20 (0.16 w
EthreshP = -53155.3104 + 3390 = -49765.3104
With these parameters, the aMD parameters in the input file should then be set to
iamd=3,EthreshD=835,alphaD=45,EthreshP=-49765,alphaP=3390,
For a higher acceleration it is common to simply add to Eb(dih) multiples of alpha.
iamd=3,EthreshD=880,alphaD=45,EthreshP=-49765,alphaP=3390,
Two levels higher would be then defined by:
iamd=3,EthreshD=925,alphaD=45,EthreshP=-49765,
```

After the aMD parameters to be used are defined, an MD run with aMD can be set using those parameters. Depending on the progress of the simulation, a higher boost can be applied as specified in the above example.

25.2.4. Sample input file for aMD

An example of an input file would be the following:

```
AVP dt=2.0fs with SHAKE, NPT aMD boost pot and dih
&cntrl
  imin=0, irest=1, ntx=5,
  dt=0.002, ntc=2, ntf=2, tol=0.000001,iwrap=1,
  ntb=2, cut=12.0, ntp=1,igb=0,ntwpwt = 3381,ioutfm = 1,
  ntt=3, temp0=310.0, gamma_ln=1.0, ig=-1,
  ntp=1000, ntwx=1000, ntwr=2000000, nstlim=2000000,
  iamd=3,EthreshD=835,
  alphaD=45,EthreshP=-49765,
  alphaP=3390,
/
&ewald
dsum.tol=0.000001,
/
```

25.2.5. Further information

Test cases have been included into the distribution of Amber, also a tutorial based on a study we performed on BPTI [555], showing the power of aMD and its validation versus a millisecond run on the same system performed

on Anton is now present on the Amber website. We encourage the user to read the paper, as well as follow the tutorial for more information.

25.3. Gaussian Accelerated Molecular Dynamics

25.3.1. Introduction

Gaussian Accelerated Molecular Dynamics (GaMD) is a biomolecular enhanced sampling method that works by adding a harmonic boost potential to smooth the system potential energy surface. The boost potential follows Gaussian distribution, which allows for accurate reweighting using cumulant expansion to the second order. GaMD has been demonstrated on simulations of alanine dipeptide, chignolin folding and ligand binding to the T4-lysozyme [557]. GaMD enables unconstrained enhanced sampling of these biomolecules without the need to set predefined reaction coordinates. Furthermore, the free energy profiles obtained from reweighting of the GaMD simulations help identify distinct low energy states of the biomolecules and characterize the protein folding and ligand binding pathways quantitatively.

The basic theory of GaMD can be found in References [557, 558], or at <http://miao.compbio.ku.edu/GaMD>.

25.3.2. Ligand Gaussian Accelerated Molecular Dynamics (LiGaMD)

A new algorithm called ligand GaMD or “LiGaMD” has been developed to simulate ligand binding and unbinding[559]. It works by selectively boosting the ligand non-bonded interaction potential energy. Another boost potential could be applied to the remaining potential energy of the entire system in a dual-boost algorithm (LiGaMD_Dual) to facilitate ligand binding. LiGaMD has been demonstrated on host-guest and protein-ligand binding model systems. Repetitive guest binding and unbinding in the β -cyclodextrin host were observed in hundreds-of-nanosecond LiGaMD simulations. The calculated binding free energies of guest molecules with sufficient sampling agreed excellently with experimental data (< 1.0 kcal/mol error). In comparison with previous microsecond-timescale conventional molecular dynamics simulations, accelerations of ligand kinetic rate constants in LiGaMD simulations were properly estimated using Kramers’ rate theory. Furthermore, LiGaMD allowed us to capture repetitive dissociation and binding of the benzamidine inhibitor in trypsin within 1 μ s simulations. The calculated ligand binding free energy and kinetic rate constants compared well with the experimental data. Therefore, LiGaMD provides a promising approach for characterizing ligand binding thermodynamics and kinetics simultaneously.

Next, one can add multiple ligand molecules in the solvent to facilitate ligand binding to proteins in MD simulations. This is based on the fact that the ligand binding rate constant k_{on} is inversely proportional to the ligand concentration. The higher the ligand concentration, the faster the ligand binds, provided that the ligand concentration is still within its solubility limit. In addition to selectively boosting the bound ligand, another boost potential could thus be applied on the unbound ligand molecules, protein and solvent to facilitate both ligand dissociation and rebinding.

25.3.3. Peptide Gaussian Accelerated Molecular Dynamics (Pep-GaMD)

Peptides often undergo large conformational changes during binding to the target proteins, being distinct from small-molecule ligand binding or protein-protein interactions. We have developed another algorithm called peptide GaMD or “Pep-GaMD” that enhances sampling of peptide-protein interactions [560]. See <http://miao.compbio.ku.edu/GaMD> for more information.

Presumably, peptide binding mainly involves both the bonded and non-bonded potential energies of the peptide since peptides often undergo large conformational changes during binding to the target proteins. Thus, the total potential energy of the peptide can be set as the essential peptide potential energy for applying the boost potential. Alternatively, we could include only the peptide dihedral energy in the essential peptide potential energy as it plays a more important role in peptide conformational changes than the other bonded energy terms. In addition to selectively boosting the peptide, another boost potential is applied on the protein and solvent to enhance conformational sampling of the protein and facilitate peptide rebinding.

25.3.4. Ligand Gaussian Accelerated Molecular Dynamics 3 (LiGaMD3)

A new algorithm called ligand GaMD 3 or “LiGaMD3” has been developed to simulate binding and unbinding of both small-molecule and highly flexible peptides. In LiGaMD3, three distinct boosts are applied: one on the non-bonded interaction energy of the substrate, the second one on the remaining non-bonded potential energy of the system, and a third one on the system bonded potential energy. These boosts are designed to accelerate the substrate dissociation, facilitate substrate rebinding, and promote the system conformational changes, respectively

25.3.5. Protein-Protein interaction - Gaussian Accelerated Molecular Dynamics (PPI-GaMD)

Protein-protein binding and unbinding processes often occur in over significantly longer timescales with higher binding affinity than protein-ligand interactions. Particularly, protein-protein dissociation could take place over minutes, hours, and even days. Thus, a boost potential can be selectively added to the non-bonded interaction potential energy between two proteins in PPI-GaMD[561]. In addition, another boost potential could thus be applied on the remaining potential energy to facilitate protein rebinding. More details of PPI-GaMD can be found in Reference[561].

25.3.6. Implementations of GaMD, LiGaMD, Pep-GaMD and PPI-GaMD algorithms in Amber

GaMD has been implemented in pmemd, both the serial and parallel versions on CPU (*pmemd* and *pmemd.MPI*) and GPU (*pmemd.cuda* and *pmemd.cuda.MPI*) by Yinglong Miao. Note that GaMD is not available in Sander. Similar to aMD, GaMD provides options about what energies to boost (see the *igamd* variable.) The dual-boost simulation generally provides higher acceleration than the other single-boost simulations for enhanced sampling.

LiGaMD has been implemented by Yinglong Miao in only the serial GPU version of pmemd (*pmemd.cuda*). It provides options to boost only non-bonded potential energy of the bound ligand (LiGaMD, *igamd=10*) and in addition the total system potential energy other than the non-bonded potential energy of bound ligand (LiGaMD_Dual, *igamd=11*). LiGaMD_Dual generally provides higher acceleration than LiGaMD for enhanced sampling. The simulation parameters comprise of settings for calculating the threshold energy values and the effective harmonic force constants of the boost potentials.

Pep-GaMD has been implemented by Jinan Wang in only the serial GPU version of pmemd (*pmemd.cuda*). It provides options to boost only the peptide potential energy (Pep-GaMD, *igamd=14*) and in addition the total system potential energy other than the peptide potential energy (Pep-GaMD_Dual, *igamd=15*). One additional variant of Pep-GaMD_Dual (*igamd=18*) is also available, where only the peptide dihedral potential energy was included in the peptide essential potential. Pep-GaMD_Dual generally provides higher acceleration than Pep-GaMD for enhanced sampling. The simulation parameters comprise of settings for calculating the threshold energy values and the effective harmonic force constants of the boost potentials.

LiGaMD3 has been implemented by Jinan Wang in only the serial GPU version of pmemd (*pmemd.cuda*). The simulation parameters comprise of settings for calculating the threshold energy values and the effective harmonic force constants of the three boost potentials.

PPI-GaMD has been implemented by Jinan Wang in only the serial GPU version of pmemd (*pmemd.cuda*). It provides options to boost only the protein protein interaction potential energy (PPI-GaMD, *igamd=16*) and in addition the total system potential energy other than the interaction potential energy (PPI-GaMD_Dual, *igamd=17*). PPI-GaMD_Dual generally provides higher acceleration than PPI-GaMD for enhanced sampling. The simulation parameters comprise of settings for calculating the threshold energy values and the effective harmonic force constants of the boost potentials.

All the information generated by GaMD, necessary for reweighing is stored at each step into a vector which is flushed to a log file (*gamd.log* by default) every time the coordinates are written to disk, i.e. every *ntwx* steps. The name of the log file can be set to a user defined name by using the command line option *-gamd* when running Amber. Additional parameters are specified by the following variables:

igamd Flag to apply boost potential

25. Sampling configuration space

	= 0 (default) no boost is applied = 1 boost on the total potential energy only = 2 boost on the dihedral energy only = 3 dual boost on both dihedral and total potential energy =4 boost on the non-bonded potential energy only =5 dual boost on both dihedral and non-bonded potential energy =10 boost on non-bonded potential energy of selected region (defined by timask1 and scmask1) as for a ligand (LiGaMD) =11 dual boost on both non-bonded potential energy of the bound ligand and remaining potential energy of the rest of the system (LiGaMD_Dual) =14 boost on the total potential energy of selected region (defined by timask1 and scmask1) as for a peptide (Pep-GaMD) =15 dual boost on both the peptide potential energy and the total system potential energy other than the peptide potential energy (Pep-GaMD_Dual) =16 boost on the interaction between protein partners (The first protein is defined by timask1 and scmask1 and the second one defined by bgpro2atm (first atom number of the protein) and edpro2atm (the end atom number of the protein)) for protein-protein interaction GaMD (PPI-GaMD) =17 dual boost on both the protein-protein interactions and the remaining potential energy of the entire system (PPI-GaMD_Dual) =18 dual boost on both the essential peptide potential energy (only the peptide dihedral energy term was included) and the total system potential energy other than the peptide potential energy (variant of Pep-GaMD_Dual) =28 Triple boost on the ligand essential potential energy, the remaining non-bonded potential energy and the system bonded potential energy
iE	Flag to set the threshold energy E = 1 (default) set the threshold energy to the lower bound $E = V_{max}$ = 2 set the threshold energy to the upper bound $E = V_{min} + (V_{max} - V_{min})/k_0$
iEP	Flag to overwrite iE and set the threshold energy E for applying the first boost potential in dual-boost schemes =1 (default) set the threshold energy to the lower bound $E = V_{max}$ =2 set the threshold energy to the upper bound $E = V_{min} + (V_{max} - V_{min})/k_0$
iED	Flag to overwrite iE and set the threshold energy E for applying the second boost potential in dual-boost schemes = 1 (default) set the threshold energy to the lower bound $E = V_{max}$ = 2 set the threshold energy to the upper bound $E = V_{min} + (V_{max} - V_{min})/k_0$
iEB	Flag to overwrite iE and set the threshold energy E for applying the third boost potential in the triple-boost schemes like LiGaMD3 =1 (default) set the threshold energy to the lower bound $E = V_{max}$ =2 set the threshold energy to the upper bound $E = V_{min} + (V_{max} - V_{min})/k_0$
ntcmdprep	The number of preparation conventional molecular dynamics steps. This is used for system equilibration and the potential energies are not collected for calculating their statistics. The default is 200,000 for a simulation with 2 fs timestep.

<code>ntcmd</code>	The number of initial conventional molecular dynamics simulation steps. Potential energies are collected between <i>ntcmdprep</i> and <i>ntcmd</i> to calculate their maximum, minimum, average and standard deviation (V_{max} , V_{min} , V_{avg} , σ_V). The default is 1,000,000 for a simulation with 2 fs timestep.
<code>ntebprep</code>	The number of preparation biasing molecular dynamics simulation steps. This is used for system equilibration after adding the boost potential and the potential statistics (V_{max} , V_{min} , V_{avg} , σ_V) are not updated during these steps. The default is 200,000 for a simulation with 2 fs timestep.
<code>nteb</code>	The number of biasing molecular dynamics simulation steps. Potential statistics (V_{max} , V_{min} , V_{avg} , σ_V) are updated between the <i>ntebprep</i> and <i>nteb</i> steps and used to calculate the GaMD acceleration parameters, particularly E and k_0 . The default is 1,000,000 for a simulation with 2 fs timestep. A greater value may be needed to ensure that the potential statistics and GaMD acceleration parameters level off before running production simulation between the <i>nteb</i> and <i>nstlim</i> (total simulation length) steps. Moreover, <i>nstlim</i> can be set to <i>ntcmd+nteb</i> , by which the potential statistics and GaMD acceleration parameters are updated adaptively throughout the simulation. This in some cases provides more appropriate acceleration.
<code>ntave</code>	The number of simulation steps used to calculate the average and standard deviation of potential energies. This variable has already been used in Amber. The default is set to 50,000 for GaMD simulations. It is recommended to be updated as about 4 times of the total number of atoms in the system. Note that <i>ntcmdprep</i> , <i>ntcmd</i> , <i>ntebprep</i> and <i>nteb</i> need to be multiples of <i>ntave</i> .
<code>irest_gamd</code>	Flag to restart GaMD simulation = 0 (default) new simulation. A file "gamd-restart.dat" that stores the maximum, minimum, average and standard deviation of the potential energies needed to calculate the boost potentials (depending on the <i>igamd</i> flag) will be saved automatically after GaMD equilibration stage. = 1 restart simulation (<i>ntcmd</i> and <i>nteb</i> are set to 0 in this case). The "gamd-restart.dat" file will be read for restart.
<code>sigma0P</code>	The upper limit of the standard deviation of the first potential boost that allows for accurate reweighting. The default is 6.0 (unit: kcal/mol).
<code>sigma0D</code>	The upper limit of the standard deviation of the second potential boost that allows for accurate reweighting in dual-boost simulations (e.g., <i>igamd</i> = 2, 3, 5, 11 and 15). The default is 6.0 (unit: kcal/mol).
<code>sigma0B</code>	The upper limit of the standard deviation of the third potential boost that allows for accurate reweighting in triple-boost simulations (e.g., <i>igamd</i> = 28). The default is 6.0 (unit: kcal/mol).
<code>timask1</code>	Specifies atoms of the first (bound) ligand, peptide or the first protein in ambmask format when <i>igamd</i> = 10, 11, 14, 15, 16, 17 or 18. The default is an empty string.
<code>scmask1</code>	Specifies atoms of the first (bound) ligand that will be described using soft core in ambmask format in LiGaMD when <i>igamd</i> = 10 or 11. In Pep-GaMD with <i>igamd</i> = 14, 15 or 18, this flag was only used to specify atoms of peptide in ambmask format, but the peptide atoms will not be described using soft core. In PPI-GaMD with <i>igamd</i> =16 or 17, this flag was used to specify atoms of the first protein in ambmask format, but the protein atoms will not be described using soft core. The default is an empty string.
<code>bgpro2atm</code>	Start atomic number of the second protein in PPI-GaMD (<i>igamd</i> =16,17) or the start atomic number of the protein region in LiGaMD3 (<i>igamd</i> =28) .
<code>edpro2atm</code>	The final atomic number of the second protein in PPI-GaMD (<i>igamd</i> =16,17) or the end atomic number of the protein region in LiGaMD3(<i>igamd</i> =28).
<code>nlig</code>	The total number of ligand molecules in the system. The default is 0.

25. Sampling configuration space

ibblig	The flag to boost the bound ligand selectively with $n_{\text{lig}} > 1$ =0 (default) no selective boost =1 boost the bound ligand selectively out of n_{lig} ligand molecules in the system
atom_p	Serial number of a protein atom (starting from 1 for the first protein atom) used to calculate the ligand distance. It is used only when $\text{ibblig} = 1$. The default is 0.
atom_l	Serial number of a ligand atom (starting from 1 for the first ligand atom) used to calculate the ligand distance to the protein. It is used only when $\text{ibblig} = 1$. The default is 0.
dblig	The cutoff distance between atoms atom_p and atom_l for determining whether the ligand is bound in the protein. It is used only when $\text{ibblig} = 1$. The default is 4.0 Å.

25.3.7. Algorithms used

The GaMD algorithm is summarized as follows:

```
GaMD {
  If (irest_gamd == 0) then
    For i = 1, ..., ntcmd // run initial conventional molecular dynamics
      If (i >= ntcmdprep) Update Vmax, Vmin
      If (i >= ntcmdprep && i%ntave == 0) Update Vavg, sigmaV
    End
    Save Vmax,Vmin,Vavg,sigmaV to gamd_restart.dat file
    Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV)

    For i = ntcmd+1, ..., ntcmd+nteb // Run biasing molecular dynamics
      // simulation steps
      deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
      V = V + deltaV
      If (i >= ntcmd+ntebprep) Update Vmax, Vmin
      If (i >= ntcmd+ntebprep && i%ntave == 0) Update Vavg, sigmaV
      Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV)
    End
    Save Vmax,Vmin,Vavg,sigmaV to gamd_restart.dat file
  else if (irest_gamd == 1) then
    Read Vmax,Vmin,Vavg,sigmaV from gamd_restart.dat file
  End if

  For i = ntcmd+nteb+1, ..., nstlim // run production simulation
    deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
    V = V + deltaV
  End
}

Subroutine Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV) {
  if iE = 1 :
    E = Vmax
    k0' = (sigma0/sigmaV) * (Vmax-Vmin)/(Vmax-Vavg)
    k0 = min(1.0, k0')
  else if iE = 2 :
    k0'' = (1-sigma0/sigmaV) * (Vmax-Vmin)/(Vavg-Vmin)
    if 0 < k0'' <= 1 :
      k0 = k0''
      E = Vmin + (Vmax-Vmin)/k0
    else
      E = Vmax
    end if
  end if
}
```



```

        k0' = (sigma0/sigmaV) * (Vmax-Vmin)/(Vmax-Vavg)
        k0 = min(1.0, k0')
    end
end
}

```

The LiGaMD algorithm is summarized as the following:

```

LiGaMD {
  If (irest_gamd == 0) then
    For i = 1, ..., ntcmd // run initial conventional molecular dynamics
      If (i >= ntcmdprep) Update Vmax, Vmin
      If (i >= ntcmdprep && i%ntave ==0) Update Vavg, sigmaV
    End
    Save Vmax,Vmin,Vavg,sigmaV to gamd_restart.dat file
    Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV)
    For i = ntcmd+1, ..., ntcmd+nteb // Run biasing molecular dynamics simulation steps
      deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
      V = V + deltaV
      If (i >= ntcmd+ntebprep) Update Vmax, Vmin
      If (i >= ntcmd+ntebprep && i%ntave ==0) Update Vavg, sigmaV
      Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV)
    End
    Save Vmax,Vmin,Vavg,sigmaV to gamd_restart.dat file
  else if (irest_gamd == 1) then
    Read Vmax,Vmin,Vavg, sigmaV from gamd_restart.dat file
  End if

  lig0=1 // ID of the bound ligand

  For i = ntcmd+nteb+1, ..., nstlim // run production simulation
    If (ibblig>0 && i%ntave ==0) then // swap the bound ligand with lig0 for selective boost
      For ilig = 1, ..., nlig
        dlig = distance(atom_p, atom_l)
        If (dlig <= dblig) blig=ilig
      End
      If (blig != lig0) Swap atomic coordinates, forces and velocities of ligands blig with lig0
    End if

    deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
    V = V + deltaV
  End
}

```

The Pep-GaMD algorithm is summarized as the following:

```

Pep-GaMD {
  If (irest_gamd == 0) then
    For i = 1, ..., ntcmd // run initial conventional molecular dynamics
      If (i >= ntcmdprep) Update Vmax, Vmin
      If (i >= ntcmdprep && i%ntave ==0) Update Vavg, sigmaV
    End
    Save Vmax,Vmin,Vavg,sigmaV to gamd_restart.dat file
    Calc_E_k0(iE,sigma0,Vmax,Vmin,Vavg,sigmaV)
    For i = ntcmd+1, ..., ntcmd+nteb // Run biasing molecular dynamics simulation steps
      deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
      V = V + deltaV
      If (i >= ntcmd+ntebprep) Update Vmax, Vmin
      If (i >= ntcmd+ntebprep && i%ntave ==0) Update Vavg, sigmaV
    End
  End
}

```

```

        Calc_E_k0(iE, sigma0, Vmax, Vmin, Vavg, sigmaV)
    End
    Save Vmax, Vmin, Vavg, sigmaV to gamd_restart.dat file
else if (irest_gamd == 1) then
    Read Vmax, Vmin, Vavg, sigmaV from gamd_restart.dat file
End if

For i = ntcmd+nteb+1, ..., nstlim // run production simulation
    deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
    V = V + deltaV
End
}

```

The PPI-GaMD algorithm is summarized as the following:

```

PPI-GaMD {
    If (irest_gamd == 0) then
        For i = 1, ..., ntcmd // run initial conventional molecular dynamics
            If (i >= ntcmdprep) Update Vmax, Vmin of interaction potential energy
            If (i >= ntcmdprep && i%ntave == 0) Update Vavg, sigmaV of interaction potential energy
        End
        Save Vmax, Vmin, Vavg, sigmaV to gamd_restart.dat file
        Calc_E_k0(iE, sigma0, Vmax, Vmin, Vavg, sigmaV)
        For i = ntcmd+1, ..., ntcmd+nteb // Run biasing molecular dynamics simulation steps
            deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
            V = V + deltaV
            If (i >= ntcmd+ntebprep) Update Vmax, Vmin of interaction potential energy
            If (i >= ntcmd+ntebprep && i%ntave == 0) Update Vavg, sigmaV of interaction potential energy
            Calc_E_k0(iE, sigma0, Vmax, Vmin, Vavg, sigmaV)
        End
        Save Vmax, Vmin, Vavg, sigmaV to gamd_restart.dat file
    else if (irest_gamd == 1) then
        Read Vmax, Vmin, Vavg, sigmaV from gamd_restart.dat file
    End if

    For i = ntcmd+nteb+1, ..., nstlim // run production simulation
        deltaV = 0.5*k0*(E-V)**2/(Vmax-Vmin)
        V = V + deltaV
    End
}

```

25.3.8. Sample input files

Here is a sample input file for GaMD:

```

&cntrl
    imin = 0, irest = 0, ntx = 1,
    nstlim = 17000000, dt = 0.002,
    ntc = 2, ntf = 2, tol = 0.000001,
    iwrap = 1, ntb = 1, cut = 8.0,
    ntt = 3, temp0 = 300.0, tempi = 300.0,
    ntpr = 50, ntwx = 50, ntwr = 500,
    nt xo = 1, ioutfm = 1, ig = -1, ntwprt = 22,

    igamd = 3, iE = 1, irest_gamd = 0,

```

```

ntcmd = 1000000, nteb = 1000000, ntave = 50000,
ntcmdprep = 200000, ntebprep = 200000,
sigma0P = 6.0, sigma0D = 6.0,
&end

```

Add the following for LiGaMD_Dual simulations:

```

igamd = 11, irest_gamd = 0,
ntcmd = 700000, nteb = 27300000, ntave = 140000,
ntcmdprep = 280000, ntebprep = 280000,
sigma0P = 4.0, sigma0D = 6.0, iEP = 2, iED=1,

icfe = 1, ifsc = 1, gti_cpu_output = 0, gti_add_sc = 1,
timask1 = ':225', scmask1 = ':225',
timask2 = '', scmask2 = '',

ibblig = 1, nlig = 10, atom_p = 2472, atom_l = 4, dblig = 3.7

```

Add the following for LiGaMD3 simulations:

```

igamd = 28, iEP = 2, iED = 1, iEB=1, irest_gamd = 0,
ntcmd = 1000000, nteb = 1000000, ntave = 50000,
ntcmdprep = 200000, ntebprep = 200000,
sigma0P = 6.0, sigma0D = 6.0, sigma0B = 6.0,
timask2 = '', scmask2 = '', icfe = 1, ifsc = 1,
gti_cpu_output = 0, timask1 = ':84-91', scmask1 = ':84-91, bgpro2atm=1, edpro2atm=1453,

```

Add the following parameters for Pep-GaMD simulations:

```

icfe = 1, ifsc = 1, gti_cpu_output = 0, gti_add_sc = 1,
timask1 = ':1-3', scmask1 = ':1-3',
timask2 = '', scmask2 = '',

igamd = 15, iE = 1, iEP = 1, iED = 1, irest_gamd = 0,
ntcmd = 1000000, nteb = 1000000, ntave = 50000,
ntcmdprep = 200000, ntebprep = 200000,
sigma0P = 6.0, sigma0D = 6.0,

```

or

```

icfe = 1, ifsc = 1, gti_cpu_output = 0, gti_add_sc = 1,
timask1 = ':1-3', scmask1 = ':1-3',
timask2 = '', scmask2 = '',

igamd = 18, iE = 1, iEP = 1, iED = 1, irest_gamd = 0,
ntcmd = 1000000, nteb = 1000000, ntave = 50000,
ntcmdprep = 200000, ntebprep = 200000,
sigma0P = 6.0, sigma0D = 6.0,

```

Add the following parameters for PPI-GaMD simulations:

```

icfe = 1, ifsc = 1, gti_cpu_output = 0, gti_add_sc = 1,
timask1 = ':1-56&!@H=', scmask1 = ':1-56&!@H=',
bgpro2atm=869, edpro2atm=1736,
timask2 = '', scmask2 = '',

igamd = 17, iE = 1, iEP = 1, iED = 1, irest_gamd = 0,
ntcmd = 1000000, nteb = 1000000, ntave = 50000,
ntcmdprep = 200000, ntebprep = 200000,
sigma0P = 6.0, sigma0D = 6.0,

```

25.3.9. Further information

Reweighting analysis of GaMD simulations is similar to that of previous aMD simulations, except that the *amd.log* file name needs to be replaced by *gamd.log*. Test cases have been included into the distribution of Amber, which are saved in folder `${AMBERHOME}/test/cuda/gamd`. The latest updates, examples and simulation tips of GaMD can be found at: <https://www.med.unc.edu/pharm/miaolab/resources/GaMD>. A tutorial based on a study we performed on alanine dipeptide[557], demonstrating the usage of GaMD on unconstrained enhanced sampling and free energy calculation of biomolecules is also available on the GaMD website.

Energetic reweighting: A toolkit of python scripts "PyReweighting" has been developed to facilitate reweighting analysis of aMD and GaMD simulations. PyReweighting implements a list of commonly used reweighting methods, including (1) exponential average that reweights trajectory frames by the Boltzmann factor of the boost potential and then calculates the ensemble average for each bin, (2) Maclaurin series expansion that approximates the exponential Boltzmann factor, and (3) cumulant expansion that expresses the reweighting factor as summation of boost potential cumulants. Notably, MacLaurin series expansion is equivalent to cumulant expansion on the first order. Cumulant expansion to the 2nd order ("Gaussian approximation") normally provides the most accurate reweighting results. The PyReweighting scripts and tutorial can be downloaded at: <https://www.med.unc.edu/pharm/miaolab/resources/gamd/analysis/>.

Kinetic reweighting: Reweighting of biomolecular kinetics from GaMD simulations can be obtained by applying Kramers rate theory. The curvatures and energy barriers of the reweighted and modified free energy profiles, as well as the apparent diffusion coefficients, are calculated and used in Kramers' rate equation to determine accelerations of biomolecular kinetics and recover the original biomolecular kinetic rate constants from the GaMD simulations. In addition to "PyReweighting" that facilitates calculations of free energy profiles, a Smoluchowski equation solver coded in C++ ("smol_solver" shared by Prof. Donald Hamelberg) can be used to calculate kinetic rates across PMF free energy barriers as needed to estimate the apparent diffusion coefficients. The source code and test examples, along with compiling and usage instructions included in a README file can be downloaded at: https://github.com/MiaoLab20/smol_solver.

25.4. Targeted MD

The targeted MD option adds an additional term to the energy function based on the mass-weighted root mean square deviation of a set of atoms in the current structure compared to a reference structure. The reference structure is specified using the *-ref* flag in the same manner as is used for Cartesian coordinate restraints (NTR=1). Targeted MD can be used with or without positional restraints. If positional restraints are not applied (ntr=0), *sander* performs a best-fit of the reference structure to the simulation structure based on selection in *tgfitmask* and calculates the RMSD for the atoms selected by *tgtrmsmask*. The two masks can be identical or different. This way, fitting to one part of the structure but calculating the RMSD (and thus restraint force) for another part of the structure is possible. If targeted MD is used in conjunction with positional restraints (ntr=1), only *tgtrmsmask* should be given in the control input because the molecule is 'fitted' implicitly by applying positional restraints to atoms specified in *restraintmask*.

The energy term has the form:

$$E = 0.5 * TGTMDFRC * NATTGTRMS * (RMSD - TGT RMSD)^2$$

The energy will be added to the RESTRAINT term. Note that the energy is weighted by the number of atoms that were specified in the *tgtrmsmask* (NATTGTRMS). The RMSD is the root mean square deviation and is mass weighted. The force constant is defined using the *tgtdmfrc* variable (see below). This option can be used with molecular dynamics or minimization. When targeted MD is used, *sander* will print the current values for the actual and target RMSD to the energy summary in the output file.

```
itgtmd      = 0 no targeted MD (default)
              = 1 use targeted MD
              = 2 use targeted MD to multiple targets (Multiply-targeted MD, or MTMD, see next section below)
```

<code>tgtrmsd</code>	Value of the target RMSD. The default value is 0. This value can be changed during the simulation by using the weight change option.
<code>tgtdmfr</code>	This is the force constant for targeted MD. The default value is 0, which will result in no penalty for structure deviations regardless of the RMSD value. Note that this value can be negative, which would force the coordinates AWAY from the reference structure.
<code>tgtfmask</code>	Define the atoms that will be used for the rms superposition between the current structure and the reference structure. Syntax is in Chapter 24.1.1.
<code>tgtrmsmask</code>	Define the atoms that will be used for the rms difference calculation (and hence the restraint force), as outlined above. Syntax is in Chapter 24.1.1.

One can imagine many uses for this option, but a few things should be kept in mind. In this implementation of targeted MD, there is currently only one reference coordinate set, so there is no way to force the coordinates to any specific structure other than the one reference. To move a structure toward a reference coordinate set, one might use an initial *tgtrmsd* value corresponding to the actual RMSD between the input and reference (*inpcrd* and *refc*). Then the weight change option could be used to decrease this value to 0 during the simulation. To move a structure away from the reference, one can increase *tgtrmsd* to values larger than zero. The minimum for this energy term will then be at structures with an RMSD value that matches *tgtrmsd*. Keep in mind that many different structures may have similar RMSD values to the reference, and therefore one cannot be sure that increasing *tgtrmsd* to a given value will result in a particular structure that has that RMSD value. In this case it is probably wiser to use the final structure, rather than the initial structure, as the reference coordinate set, and decrease *tgtrmsd* during the simulation. To address this, multiply-targeted MD is now available in Amber (*sander only*), and is described in the next section. As an additional note, a negative force constant *tgtdmfr* can be used, but this can cause problems since the energy will continue to decrease as the RMSD to the reference increases.

Also keep in mind that phase space for molecular systems can be quite complex, and this method does not guarantee that a low energy path between initial and target structures will be followed. It is possible for the simulation to become unstable if the restraint energies become too large if a low-energy path between a simulated structure and the reference is not accessible.

Note also that the input and reference coordinates are expected to match the *prmtop* file and have atoms in the same sequence. No provision is made for symmetry; rotation of a methyl group by 120° would result in a nonzero RMSD value.

25.5. Multiply-Targeted MD (MTMD)

In Amber (*sander only*), the user may perform targeted MD calculations using multiple reference structures. Each reference may have its own associated target RMSD value and force constant, each of which can evolve independently in time. Additionally, the masks for each defined target may differ, and targeting to any given reference structure can be activated for some or part of the simulation. The energy term for MTMD is simply the sum of the energies that would be calculated for the molecule calculated relative to each target given the target RMSD and force constant for that target. The energy will then be added to the RESTRAINT term.

To use MTMD, the MTMD input file is specified using the *-mtmd* flag in the command line arguments for *sander*. The MTMD input file will contain one instance of the *tgt* namelist (“&tgt”) for each reference structure used. The user may specify any number of reference structures.

25.5.1. Variables in the &tgt namelist:

<code>refin</code>	The file name of the reference structure used. The input and reference coordinates are expected to match the <i>prmtop</i> file and have atoms in the same sequence. <i>Default for refin is “”, no reference structure given.</i>
<code>mtmdform</code>	If <i>MTMDFORM</i> > 0, then the reference coordinate file is formatted. Otherwise, the reference coordinate file is an unformatted (binary) file. <i>Default for MTMDFORM is the value assigned to</i>

25. Sampling configuration space

MTMDFORM in the most recent namelist where *MTMDFORM* was specified. If *MTMDFORM* has not been specified in any namelist, it defaults to 1.

`mtmdstep1`, `mtmdstep2` Targeted MD for this structure is run for steps/iterations *MTMDSTEP1* through *MTMDSTEP2*. If *MTMDSTEP2* = 0, then TMD will be run through the end of the run, and the values of the target RMSD and the force constant will not change with time. Note that the first step/iteration is considered step 0. *Defaults for MTMDSTEP1 and MTMDSTEP2 are the values assigned to them in the most recent namelist where MTMDSTEP1 and MTMDSTEP2 were specified. If MTMDSTEP1 and MTMDSTEP2 have not been specified in any namelist, they default to 0.*

`mtmdvari` If *MTMDVARI* > 0, then the force constant and target RMSD will vary with step number. Otherwise, they are constant throughout the run. If *MTMDVARI* > 0, then the values *MTMDSTEP2*, *MTMDRMSD2*, and *MTMDFORCE2* must be specified (see below). *Default for MTMDVARI is the value assigned to MTMDVARI in the most recent namelist where MTMDVARI was specified. If MTMDVARI has not been specified in any namelist, it defaults to 0.*

`mtmdrmsd`, `mtmdrmsd2` The target RMSD for this reference. If *MTMDVARI* > 0, then the value of *MTMDRMSD* will vary between *MTMDSTEP1* and *MTMDSTEP2*, so that, e.g. *MTMDRMSD(MTMDSTEP1)* = *MTMDRMSD* and *MTMDRMSD(MTMDSTEP2)* = *MTMDRMSD2*. *Defaults for MTMDRMSD and MTMDRMSD2 are the values assigned to them in the most recent namelist where MTMDRMSD and MTMDRMSD2 were specified. If MTMDRMSD and MTMDRMSD2 have not been specified in any namelist, they default to 0.0.*

`mtmdforce`, `mtmdforce2` The force constant for this reference. If *MTMDVARI* > 0, then the value of *MTMDFORCE* will vary between *MTMDSTEP1* and *MTMDSTEP2*, so that, e.g. *MTMDFORCE(MTMDSTEP1)* = *MTMDFORCE* and *MTMDFORCE(MTMDSTEP2)* = *MTMDFORCE2*. *Defaults for MTMDFORCE and MTMDFORCE2 are the values assigned to them in the most recent namelist where MTMDFORCE and MTMDFORCE2 were specified. If MTMDFORCE and MTMDFORCE2 have not been specified in any namelist, they default to 0.0.*

`mtmdninc` If *MTMDVARI* > 0 and *MTMDNINC* > 0, then the changes in the values of *MTMDRMSD* and *MTMDFORCE* are applied as a step function, with *NINC* steps/iterations between each change in the target values. If *MTMDNINC* = 0, the change is effected continuously (at every step). *Default for MTMDNINC is the value assigned to MTMDNINC in the most recent namelist where MTMDNINC was specified. If MTMDNINC has not been specified in any namelist, it defaults to 0.*

`mtmdmult` If *MTMDMULT*=0, and the values of *MTMDFORCE* changes with step number, then the changes in the force constant will be linearly interpolated from *MTMDFORCE*→*MTMDFORCE2* as the step number changes. If *MTMDMULT*=1 and the force constant is changing with step number, then the changes in the force constant will be effected by a series of multiplicative scalings, using a single factor, *R*, for all scalings. *i.e.*

$$\text{MTMDFORCE2} = R^{**\text{INCREMENTS}} * \text{MTMDFORCE}$$

INCREMENTS is the number of times the target value changes, which is determined by *MTMDSTEP1*, *MTMDSTEP2*, and *MTMDNINC*. *Default for MTMDMULT is the value assigned to MTMDMULT in the most recent namelist where MTMDMULT was specified. If MTMDMULT has not been specified in any namelist, it defaults to 0.*

`mtmdmask` Define the atoms that will be used for both the rms superposition between the current structure and the reference structure and the rms difference calculation (and hence the restraint force), as outlined above. Syntax is in Chapter 24.1.1. *Default for MTMDMASK is the value assigned to MTMDMASK in the most recent namelist where MTMDMASK was specified. If MTMDMASK has not been specified in any namelist, it defaults to '*', use all atoms in the fit and force calculations.*

Namelist >gt is read for each reference structure. Input ends when a namelist statement with `refin = "` (or `refin` not specified) is found. Note that comments can precede or follow any namelist statement, allowing comments and reference definitions to be freely mixed.

25.6. Nudged elastic band calculations

25.6.1. Background

In the nudged elastic band method (NEB),[\[562, 563\]](#) the path for a conformational change is approximated with a series of images describing the molecule at discrete points along the path. A simultaneous energy minimization of the total system, while keeping the endpoint images fixed in space, provides a minimum energy path. Each image in-between the two endpoints, is connected to their nearest neighbors by "springs" along the path that serve to keep them from sliding down the energy landscape, and onto adjacent images. NEB is derived from the plain elastic band method, pioneered by Elber and Karplus,[\[564\]](#) who added the spring forces to the potential of energy surface and minimized the energy of the system. The plain elastic band method found low energy paths, but tended to cut corners in the energy landscape. NEB prevents corner cutting by truncating the spring forces in directions perpendicular to the tangent of the path. Furthermore, the forces from the molecular potential are truncated along the path, for the images to remain evenly spaced. Therefore, only the perpendicular component of the potential force ($\mathbf{F}^\perp$) and the parallel component of the spring force ($\mathbf{F}^\parallel$) are considered in the equations of motion. This leads to:

$$\begin{aligned}\mathbf{F} &= \mathbf{F}^\perp + \mathbf{F}^\parallel \\ \mathbf{F}^\perp &= -\nabla V(\mathbf{P}) + ((\nabla V(\mathbf{P}) \cdot \boldsymbol{\tau})\boldsymbol{\tau}) \\ \mathbf{F}^\parallel &= [(k_{i+1}|\mathbf{P}_{i+1} - \mathbf{P}_i| - k_i|\mathbf{P}_i - \mathbf{P}_{i-1}|) \cdot \boldsymbol{\tau}]\boldsymbol{\tau}\end{aligned}\tag{25.3}$$

where $\mathbf{F}$ is the force on image i , $\mathbf{P}_i$ is the 3N-dimensional position vector of image i with N atoms, k_i is the spring constant between image $i-1$ and image i , V is the potential described by the force field, and $\boldsymbol{\tau}$ is the 3N-dimensional tangent unit vector that describes the path.

The simplest definition of $\boldsymbol{\tau}$ is:

$$\boldsymbol{\tau} = (\mathbf{P}_i - \mathbf{P}_{i-1})/|\mathbf{P}_i - \mathbf{P}_{i-1}|\tag{25.4}$$

This definition leads to instability in the path caused by kinks that occur where the magnitude of $\mathbf{F}^\parallel$ is much larger than the magnitude of $\mathbf{F}^\perp$. A more stable tangent definition was derived to prevent kinks in the path that depends upon the energies, E , of adjacent images.[\[565\]](#) The spring constants can be the same between all images or they can be scaled to move the images closer together in the regions of transition states:[\[566\]](#)

$$\begin{aligned}\text{If } (E_i > E_{ref}) & \quad \text{then} \quad k_i = k_{max} - \Delta k(E_{max} - E_i)/(E_{max} - E_{ref}) \\ \text{otherwise} & \quad k_i = k_{max} - \Delta k\end{aligned}\tag{25.5}$$

Here E_{max} is the energy of the replica with the highest energy along the path, E_{ref} is the energy of the higher energy endpoint, and k_{max} and Δk are parameters with units of force per length. Because the spring force applies only in directions along the path and the potential of the energy surface is zeroed along the path, the calculation is relatively insensitive to the magnitude of the spring constants. Care must be taken, however, to select a spring constant that does not result in higher frequency motions than those found in the system of interest.[\[567\]](#) At each step, before calculating the spring forces that compose $\mathbf{F}^\parallel$, each image's neighbor is rotated and translated onto the image itself to find the minimum RMSD, based on a subset of the system's atoms which the user can define. In this way, each image remains a continuous MD simulation, and the communication of coordinates can be greatly reduced.

Energy minimization of the path is complicated because the forces are truncated according to the tangent direction, making it impossible to define a Lagrangian.[\[567\]](#) Conjugate gradient minimization, therefore, cannot be

used to find the minimum energy path. An algorithm for quenched molecular dynamics has been used instead.[563] With this method, the component of the velocity parallel to the force is kept, but perpendicular components are scaled:

$$\begin{aligned} \text{If } (\mathbf{v} \cdot \mathbf{f} > 0) & \quad \text{then} \quad \mathbf{v} = (\mathbf{v} \cdot \mathbf{f})\mathbf{f} \\ & \quad \text{otherwise} \quad \mathbf{v} = x(\mathbf{v} \cdot \mathbf{f})\mathbf{f} \end{aligned} \quad (25.6)$$

where $\mathbf{f}$ is the 3N-dimensional unit force vector, $\mathbf{v}$ is the 3N-dimensional velocity vector, and x is a scaling factor less than one. Recently, a super-linear minimization method was described using an adopted basis Newton-Raphson minimizer.[567]

A partial NEB (PNEB) implementation is available both in sander and pmemd, and is the only form of NEB that is currently supported in Amber [568]. This implementation allows the NEB method to be applied to a user defined subset of the system. It is required that user defines the part of the system to which NEB force decoupling is applied, as well as the part of the system to which an RMS fit of the neighboring images is performed in order to remove rotational and translational motion. PNEB enables the efficient use of NEB in large systems where a local transition is desired, or in explicitly solvated systems in which the solvent atoms need to be excluded from NEB calculations. In pmemd, PNEB simulations can be performed using the GPU accelerated executables.

As with the previous implementation of NEB [569], minimization of the energies of the system along the lowest potential energy path is achieved by simulated annealing. This requires no hypothesis for a starting path, but careful judgment of the temperature and simulation time is necessary to populate the minimum energy path. The initial coordinates can have multiple copies of the structure superimposed on the two endpoints. When adjacent structures are superimposed, the tangent, τ , is 0 in every direction. This case is explicitly handled so that the calculation is stable.

25.6.2. Preparing input files for NEB

Input *prmtop* and *inpcrd* files for NEB should be generated using *LEaP*. To perform NEB simulations, the minimum requirements are a *prmtop* file of a single image of the molecule and two *inpcrd* files representing each end of the pathway.

The following are some notes for preparing NEB input files:

1. Always check that the *prmtop* files generated for the endpoint coordinates are the same. This can be done by comparing the files using the *diff* command. Identical *prmtop* files must be used to describe both endpoints' *inpcrd* files.
2. If you have intermediate structures along the path, you must make sure the *prmtop* is appropriate for these structures as well.
3. The endpoint images serve as coordinate reference points, and remain fixed in coordinate and energy space along the path. No simulation is performed on these replicas during NEB optimization, so they must initially be well minimized to prevent the rest of the images from migrating to a local minimum before the conformational transition occurs. Take this into consideration when choosing the number of images to connect along the path.

Multisander/multiplmpemd requires a groupfile input, in which each line is a sander/pmpemd command for individual image's MD simulation. Multiple copies of each endpoint image are used for the initial simulation. When preparing the initial groupfile, the first half of the images can use copies of the initial endpoint *inpcrd*, while the other half uses copies of the final endpoint *inpcrd*. If intermediates are available and user wishes to include them, they should be added sequentially in between the endpoint conformations in the order in which these structures are thought to appear along the transition path.

Notes for running NEB using multisander or multiplmpemd:

1. If using multisander, the number of CPUs specified must be a multiple of the number of images. You can run this on a standard desktop computer, but it will generally be more efficient to run it on a minimum of one processor per image.

2. If using `multipmemd`, the number of CPUs must be a multiple, and at least twice, of the number of images. In case `pmemd.cuda.MPI` is used, it is best that the number of GPUs is equal to the number of images.
3. If the user has access to parallel computing resources, multiple processors per image may be used. Careful benchmarking should be done to gauge the best balance between computational efficiency in calculating the dynamics of each image and slowdown caused by communications overhead at each step.

25.6.3. Input Variables

<code>ineb</code>	Flag for nudged elastic band. A value of 0 (default) means that no nudged elastic band will be used. A value of 1 means that NEB simulation is being performed.
<code>tgtfitmask</code>	Flag which sets atoms to RMS fit each image's neighbor to itself. This mask must not include solvent atoms, which due to diffusion, overlapping proves impossible. The more atoms you choose, the more communication has to be done by each MPI thread. Syntax for this is here: 24.1.1
<code>tgtrmsmask</code>	Flag which sets atoms to decouple NEB forces for PNEB. This can be set to all atoms of the solute, or a subset of atoms which best describes the area of the system which undergoes the conformational change you wish to see. Syntax for this is here: 24.1.1
<code>skmax</code>	Spring constant or k_{max} mentioned above (100 by default).
<code>skmin</code>	If <code>skmin = skmax</code> , a fixed spring constant is used. Otherwise, <code>skmin</code> is taken from above for scaled spring constants (50 by default).
<code>tmode</code>	If 1 (default), use the revised tangent definition that prevents kinks. For any other value, use the simple (original) tangent definition.
<code>vv</code>	If this is 1, use the quenched velocity Verlet minimization; otherwise, do not.
<code>vfac</code>	Scaling factor for quenched velocity Verlet algorithm. (0.0 by default).

Sample input file for running initial heating along the path.

Below is an example input file that can be used to perform the initial heating step of an NEB run. Note that the input and topology files must be identical for each replica; while the names of the output, trajectory, restart and info files should not be the same between replicas.

```

Alanine NEB initial MD with small K
&cntrl
  imin = 0, irest = 0,
  ntc=1, ntf=1,
  ntpr=1, ntwx=500,
  ntb = 0, cut = 999.0, rgbmax=999.0,
  igb = 1, saltcon=0.2,
  nstlim = 40000, nscm=0,
  dt = 0.0005, ig=42,
  ntt = 3, gamma_ln=1000.0,
  tempi=0.0, temp0=300.0,
  tgtfitmask=":1,2,3",
  tgtrmsmask=":1,2,3@N,CA,C",
  ineb = 1, skmin = 10, skmax = 10,
  nmropt=1,
/
&wt type='TEMP0', istep1=0, istep2=35000,
  value1=0.0, value2=300.0
/
&wt type='END'
/

```

25. Sampling configuration space

tgfitmask variable denotes the atoms that will be used to RMS fit each replica onto its neighbor images at each step. In this case all atoms of residues 1, 2, and 3 are specified. The **tgtrmsmask** variable denotes the atoms that the NEB forces will be applied to. In this case the backbone atoms of residues 1, 2, and 3 are specified. In general, the atoms that have NEB forces applied to them should be those involved in the transition of interest. If the specific transition is not known, or there are many degrees of freedom involved in the transition, one can simply specify all solute atoms. It is *not* recommended to apply NEB forces to solvent atoms. For more examples, please refer to the runs in the `$AMBERHOME/test/neb-testcases` and `$AMBERHOME/test/cuda/neb-testcases` directories, or see reference [568].

25.6.4. Important Considerations for NEB Simulations

With the implementation of PNEB, it is important to understand some limitations of the method. Only part of the system is simulated with NEB forces, indicating this part of the system is moving along the minimum potential energy landscape of the transition path. However, the part of the system to which NEB is not applied is not necessarily forced along this minimum potential energy path, and attention must be paid to the convergence of this part of the system. The conformational change in this part of the system is with no doubt accelerated, since it responds to the part of the system to which NEB forces are applied. Further equilibration of the system may be required if the user wishes to examine changes not local to the area the NEB forces are applied to.

Careful attention must be paid to optimization methods, to assure that conformational space is explored for the NEB part of the system, while the integrity of the non-NEB part remains intact. As in all NEB implementations, a general caveat is that as the system size increases, the degrees of freedom increase and conformational changes become more difficult to quantify. While NEB is a method which does not necessitate a reaction coordinate, care should be taken when analyzing the resulting minimum energy path. Statistically relevant number of simulations must be performed to ensure reproducibility (and convergence) of the results.

25.7. Adaptive String Method

Adaptive string method [570] (ASM) aims at finding the Minimum Free Energy Path (MFEP) in the space of selected collective variables (CVs) connecting initial and final states of a given process (usually reactants and products of a chemical reaction). It is heavily based on on-the-fly string method by Maragliano and Vanden Eijnden [571], but provides some improvements that reduce the number of parameters and facilitate its usage:

- The biasing potential in each string node depends on the path direction and is defined using only two parameters: force constants along (K_l) and orthogonal (K_d) to the path.
- Instead of keeping the string nodes strictly equidistant, their positions and K_l values are optimized to ensure uniform sampling along the path.
- Once the MFEP converges, the calculation enters the "PMF stage": a path collective variable (pathCV) [572, 573] is defined along the optimized path and Umbrella Sampling is performed to obtain the Potential of Mean Force (PMF) using Umbrella Integration [574]. The optimized positions and force constants are used to ensure good overlap of the adjacent windows.

Before using ASM it is highly recommended to go through the ASM tutorials: a simple example of an SN2 reaction in water, [575] and a more complex simulation of inhibition of SARS-CoV-2 main protease by the inhibitor PF-00835231 [576, 577].

25.7.1. Input files and execution

To use ASM, three additional files must be provided.

1. **CV file** - provides the list of CVs used. It should contain the total number of CVs followed by one COLVAR namelist for each CV with the following options:

COLVAR_type type of the CV. Can be one of "BOND", "ANGLE", "DIHEDRAL" or "PPLANE" (signed point plane distance, can be used to control hybridization change between sp^3 and sp^2).

<code>atoms</code>	list of atoms defining the CV. Contains 2, 3, 4 and 4 items for bonds, angles, dihedrals and point-plane distances, respectively. In case of <code>PPLANE</code> , the first atom defines the point and the remaining three define the plane to which the distance is measured. The distance is signed, so swapping any two of the three atoms defining the plane will change the sign of the CV.
<code>box</code>	(optional) defines the minimum and maximum values this CV can have in the string nodes. Note that this is not a constraint for the value of the CV in the structures, but only affects the positions of the string nodes during optimization.

Example:

```
3

$COLVAR
COLVAR_type = "BOND"
atoms = 1, 5
box = 0, 5.0
$END

$COLVAR
COLVAR_type = "BOND"
atoms = 1, 6
box = 0, 5.0
$END

$COLVAR
COLVAR_type = "PPLANE"
atoms = 1, 3, 2, 4
$END
```

2. **guess file** - provides the initial guess for the string in the CV space. Constrains the total number of guess points followed by the values of the CVs for each point. The number of points does not have to coincide with the number of string nodes, the CV values for each node will be automatically assigned by interpolating the provided guess.

Example:

```
2
1.76510e+00 2.99074e+00 -3.48126e-01
2.97581e+00 1.76765e+00 3.13631e-01
```

3. **STOP.STRING file** - the convergence of the path must be determined manually by plotting the *convergence.dat* file. When the convergence curve forms a plateau, the beginning and the end of the plateau (in timesteps) should be written to *STOP.STRING* file in the results directory. This indicates the code to average the path over the provided interval and switch to the PMF stage.

Example:

```
10000
15000
```

ASM calculation is controlled by setting `asm=1` in the input file and providing `&asm` namelist with the ASM options. The default values should work well for most of the cases, so the namelist can be left empty. The most common options are given below. χ stands for the units of the pathCV ($\text{a.m.u}^{1/2} \cdot \text{\AA}$).

<code>dir</code>	the directory for the ASM results (default: "results")
<code>CV_file</code>	path to the CVs file (default: "CVs")
<code>guess_file</code>	path to the guess file. If left empty, the guess is constructed from the CV values in the initial structures.

25. Sampling configuration space

- `only_PMF` start the calculation directly from the PMF stage. When on, `0_final.string` and `final_parameters.dat` from a previously converged string calculation must be available in the working directory (default: `.false.`).
- `REX_period` period (in timesteps) for Replica Exchange attempts (default: number of timesteps in 50fs).
- `output_period` how often (in timesteps) to write out result files (default: 100).
- `preparation_steps` duration (in timesteps) of the preparation stage, during which the force constants are gradually increased from zero to the target value before starting the string optimization (default: number of timesteps in 1ps).
- `step` initializes the internal step counter to a given value. Negative values correspond to the preparation stage. Mainly used to start the preparation with non-zero forces in cases when the initial structures are already roughly aligned with the string nodes. For example, if `preparation_steps = 1000` and `step = -500` the bias in each node will start with half of the corresponding target values of K_l and K_d (default: `-preparation_steps`).
- `buffer_size` (timesteps) how often the PMFs should be written out. During the optimization stage the sampling from all string nodes is stored in a buffer and is integrated once the buffer is full. During the PMF stage the buffer is not used, but the frequency is kept the same for consistency (default: number of timesteps in 2ps).
- `z_bias` whether to restraint the distance from the path during the PMF stage (default: `.true.`).
- `dump_pathCV_def` whether to write out the definition of the pathCV (`pathCV.def` file) after string convergence. Can be used to restart the PMF stage with exactly the same definition or to calculate the pathCV over an arbitrary trajectory (default: `.true.`).
- `read_pathCV_def` (for `only_PMF`) whether to read the pathCV definition from the dumped file. If yes, `pathCV.def` file must be available in the working directory. (default: `.false.`).
- `minimize` use the bias from the PMF stage with energy minimization (`imin=1`). The calculation must be set up exactly as if `only_PMF` is used and the bias from the given node (see below) will be added to the total energy. Note that since ASM is only implemented in the MPI versions of `sander/pmemd`, in the minimization mode 1 group with multiple threads per group must be used (default: `.false.`).
- `nodes` total number of string nodes. Is only needed in the minimization mode since in that case it can not be inferred from the number of MPI threads.
- `node` node that defines the bias in the minimization mode.
- `force_constant_d` K_d ($\text{kcal} \cdot \text{mol}^{-1} \cdot \chi^{-1}$), the force constant for the orthogonal bias during both string optimization and the PMF stage. If not provided it is set to half of K_l (determined automatically). The default usually works OK but can be set to lower/larger value during the PMF stage if the system becomes unstable or drifts away from the path.
- `string_move_period` how often (number of steps) to update the string nodes, node positions and force constants. The changes are averaged over the provided period (default: 1).
- `grote_hynes` turn on damping of reduced mass and force projection on the pathCV for Grote-Hynes theory analysis [578].
- `gh_period` (timesteps) how often to write out Grote-Hynes data (default: 1).

The rest of the options rarely (or never) have to be used/changed, so change them only if you really know what you are doing:

<code>fix_ends</code>	whether the first and the last nodes are fixed. It is highly recommended to determine the values of the CVs in reactants and products in advance, use them as the first and the last points in the guess and do not allow them to change during the string optimization. It is also often a good idea to slightly adjust the values deeper towards reactants/products to make sure that the corresponding minima in the resulting PMF are well resolved (default: <code>.true.</code>).
<code>string_move</code>	controls whether the entire string is allowed to change (default: <code>.true.</code>).
<code>force_constant</code>	defined the initial value of K_l (default: inferred based on the string arc length to ensure uniform sampling assuming flat free energy profile).
<code>nbins</code>	number of bins used to integrate the PMF (default: 100).
<code>points_per_node</code>	number of points in the pathCV definition per each string node (default: $100 / \text{number of nodes} + 1$).
<code>points_extra</code>	number of points to extrapolate beyond the start/end of the converged path in the pathCV definition (default: <code>points_per_node * 5</code>).
<code>gamma</code>	(ps ⁻¹) string friction (default: 2000).
<code>position_gamma</code>	(kcal·mol ⁻¹ · χ^{-2} ·ps) friction of the node positions along the string (default: $2 \cdot 10^5$).
<code>force_gamma</code>	(ps) friction of the force constants (default: 5).
<code>force_kappa</code>	(χ^{-4}) factor that increases the force constants when the CV values in the structure are too far from the corresponding string node (default: 1000).
<code>Mav_damp</code>	damping coefficient for metric tensor averaging (default: 0.001).
<code>write_M</code>	whether to write the metric tensors to the <code>*.string</code> files (default: <code>.false.</code>).
<code>read_M</code>	whether to read the metric tensors from the guess file (default: <code>.false.</code>).
<code>params_file</code>	if provided, where to read the node positions and force constant from (default: "", the values are calculated automatically).
<code>reweight_z</code>	whether to reweight the samples for preliminary PMF integration during the optimization stage (default: <code>.true.</code>).

ASM takes advantage of MPI versions of sander/pmemd, but has its own implementation of Replica Exchange. So, a groupfile should be provided with the number of groups equal the number of string nodes and sander/pmemd should be executed with `-rem 0`. The most common approach is to initialize the first half of the groups with reactants structures and the rest with products structures.

25.8. Low-MODE (LMOD) methods

István Kolossváry's LMOD methods for minimization, conformational searching, and flexible docking [579–583] are fully implemented in Amber. The centerpiece of LMOD is a conformational search algorithm based on eigenvector following of low frequency vibrational modes. It has been applied to a spectrum of computational chemistry domains including protein loop optimization and flexible active site docking. The search method is implemented without explicit computation of a Hessian matrix and utilizes the Arnoldi package (ARPACK, <https://github.com/opencollab/arpac-ng>) for computing the low-frequency modes. LMOD optimization can be thought of as an advanced minimization method. LMOD can not only energy minimize a molecular structure in the local sense, but can generate a series of very low energy conformations. The LMOD capability resides in a single, top-level calling function `lmod()`, which uses fast local minimization techniques, collectively termed XMIN that can also be accessed directly through the function `xmin()`.

In the Amber24 release, the LMOD optimization code has been updated with major improvements and new features including more accurate flexible docking, the option to visualize normal modes, utilization of random mixtures of low-frequency modes, and the option to work with a range of modes anywhere in the spectrum and not just the lowest frequency modes. The latter is particularly useful for docking where the modes relevant to binding a ligand molecule are usually not the lowest frequency modes. The interface of the new LMOD has not changed, everything works exactly the same way as in earlier versions, a few parameters simply have additional options as documented below. The new features are demonstrated with production quality examples.

There are four “real-life” examples of performing LMOD searches and (in *lmod.vib.anim*) three examples of LMOD-vibration visualization, plus an additional co-vibrational analysis to reproduce the results in [583]. This analysis consists of two steps included in the *lmodvib* and *covib* sub-directories, respectively. A 3rd directory called *publication* contains the publication pdf and associated video files. These are available at <https://ambermd.org/Manuals.php>. Each directory in the tar file has a README file with more information.

25.8.1. LMOD conformational searching

The LMOD conformational search procedure is based on gentle, but very effective structural perturbations applied to molecular systems in order to explore their conformational space. LMOD perturbations are derived from low-frequency vibrational modes representing large-amplitude, concerted atomic movements. Unlike essential dynamics where such low modes are derived from long molecular dynamics simulations, LMOD calculates the modes directly and utilizes them to improve Monte Carlo sampling.

LMOD has been developed primarily for macromolecules, with its main focus on protein loop optimization. However, it can be applied to many kinds of molecular systems, including complexes and flexible docking where it has found widespread use. The LMOD procedure starts with an initial molecular model, which is energy minimized. The minimized structure is then subjected to an ARPACK calculation to find a user-specified number of low-mode eigenvectors of the Hessian matrix. The Hessian matrix, H , is never computed; ARPACK makes only implicit reference to H through its product with a series of vectors Hv , where v is an arbitrary unit vector. Hv is calculated via a finite-difference formula as follows:

$$Hv = [\nabla(x_{min} + h) - \nabla(x_{min})] / h \quad (25.7)$$

where x_{min} is the coordinate vector at the energy minimized conformation and h denotes machine precision. The computational cost of Eq. 25.7 requires a single gradient calculation at the energy minimum point and one additional gradient calculation for each new vector. Note that ∇x is never 0, because minimization is stopped at a finite gradient RMS, which is typically set to 0.1-1.0 kcal/mol-Å in most calculations.

The low-mode eigenvectors of the Hessian matrix are stored and can be re-used throughout the LMOD search. Note that although ARPACK is very fast in relative terms, a single ARPACK calculation may take up to a few hours on an absolute CPU time scale with a large protein structure. Therefore, it would be impractical to recalculate the low-mode eigenvectors for each new structure. Visual inspection of the low-frequency vibrational modes of different, randomly generated conformations of protein molecules showed very similar, collective motions clearly suggesting that low-modes of one particular conformation were transferable to other conformations for LMOD use. This important finding implies that the time limiting factor in LMOD optimization, even for relatively small molecules, is energy minimization, not the eigenvector calculation.

25.8.2. LMOD procedure

Given the energy-minimized structure of an initial protein model, protein- ligand complex, or any other molecular system and its low-mode Hessian eigenvectors, LMOD proceeds as follows. For each of the first n low-modes repeat steps 1-3 until convergence:

1. Perturb the energy-minimized starting structure by moving along the i th ($i=1-n$) Hessian eigenvector in either of the two opposite directions to a certain distance. The 3N-dimensional (N is equal to the number of atoms) travel distance along the eigenvector is scaled to move the fastest moving atom of the selected mode in 3-dimensional space to a randomly chosen distance between a user-specified minimum and maximum value.

Note: A single LMOD move inherently involves excessive bond stretching and bond angle bending in Cartesian space. Therefore the primarily torsional trajectory drawn by the low-modes of vibration on the PES is severely contaminated by this naive, linear approximation and, therefore, the actual Cartesian LMOD trajectory often misses its target by climbing walls rather than crossing over into neighboring valleys at not too high altitudes. The current implementation of LMOD employs a so-called ZIG-ZAG algorithm, which consists of a series of alternating short LMOD moves along the low-mode eigenvector (ZIG) followed by a few steps of minimization (ZAG), which has been found to relax excessive stretches and bends more than reversing the torsional move. Therefore, it is expected that such a ZIG-ZAG trajectory will eventually be dominated by concerted torsional movements and will carry the molecule over the energy barrier in a way that is not too different from finding a saddle point and crossing over into the next valley like passing through a mountain pass.

Barrier crossing check: The LMOD algorithm checks barrier crossing by evaluating the following criterion: IF the current endpoint of the zigzag trajectory is lower than the energy of the starting structure, OR, the endpoint is at least lower than it was in the previous ZIG-ZAG iteration step AND the molecule has also moved farther away from the starting structure in terms of all-atom superposition RMS than at the previous position THEN it is assumed that the LMOD ZIG-ZAG trajectory has crossed an energy barrier.

2. Energy-minimize the perturbed structure at the endpoint of the ZIG-ZAG trajectory.
3. Save the new minimum-energy structure and return to step 1. Note that LMOD saves only low-energy structures within a user-specified energy window above the then current global minimum of the ongoing search.

After exploring the modes of a single structure, LMOD goes on to the next starting structure, which is selected from the set of previously found low-energy structures. The selection is based on either the Metropolis criterion, or simply the than lowest energy structure is used. LMOD terminates when the user-defined number of steps has been completed or when the user-defined number of low-energy conformations has been collected.

Note that for flexible docking calculations LMOD applies explicit translations and rotations of the ligand(s) on top of the low-mode perturbations.

25.8.3. Co-vibrational low-mode procedure

The new version of LMOD included in Amber24 has significant new capabilities but the existing options and parameters are exactly the same as in earlier versions. Instead, new behavior can be triggered by the presence of certain files in the work directory.

1) LMOD eigenvectors no longer have to be recomputed whenever a LMOD job needs to be restarted. LMOD is now saving the computed eigenvectors with high accuracy at the end of a run in a file called `eigvecs.dat`, and in a new run the saved eigenvectors can be read instead of recomputing them. If the file `eigvecs.dat` exists in the work directory, LMOD will check if it is compatible with the current calculation and if so, it'll read the eigenvectors from the file. Note that LMOD is picky about the formatting of the eigenvector file, always use one that was written by LMOD.

2) The original LMOD used Cartesian eigenvectors with no weighting, which was/is perfectly fine for conformational searches, but if physically correct normal modes are required (especially for visualization via the setting `lmod_verbosity = 5` or `6`) the Hessian matrix must be mass weighted. The user can now include a file called `atom_weights.dat` in the work directory, in which case LMOD will mass-weight the Hessian eigenvectors. The first line of `atom_weights.dat` contains the number of atoms, and subsequent lines list the atomic masses, one per line. If no `atom_weights.dat` file can be found in the work directory, the Hessian will not be weighted.

3) At the heart of a new LMOD application is the concept termed allosteric co-vibrational modes [583]. Covib modes are mixed/superposed normal modes where the individual modes contribute with specific amplitudes, and can be used to model protein conformational transitions. The user can provide these amplitudes in a file called `amplitudes.dat`. The way it works is that LMOD already has the `lmod_verbosity=5` option to generate a pseudo-trajectory of an animation of normal modes, and this trajectory also includes a random mixture of the modes as the last "mode." With the new version of LMOD, if an amplitude file is provided in the work directory, LMOD will compute the corresponding mixed mode and include it as the last mode in the pseudo-trajectory instead of

25. Sampling configuration space

the random mixture. In subsequent work, after the Amber24 release but before AmberTools25, the computational performance enhancement option `lmod_verbosity=6` was added to visualize only the co-vibrational mode. The *amplitudes.dat* file has the number of modes in the first line, followed by the user-defined mode amplitudes, printed one amplitude per line. Note that both *eigvecs.dat* and *amplitudes.dat* include the trivial modes if there are any. This is intentional (for legacy code structure reasons), but LMOD will exclude them from the mixing.

25.8.4. XMIN

The XMIN methods for minimization are traditional and manifold in the field of unconstrained optimization: PRCG is a Polak-Ribiere nonlinear Conjugate Gradient algorithm,[584] LBFGS is a Limited-memory Broyden-Fletcher-Goldfarb-Shanno quasi-Newton algorithm,[585] and TNCG is a Truncated Newton linear Conjugate Gradient method with optional LBFGS preconditioning.[586]

Some of the `&cntrl` namelist variables that control Amber's other minimization facilities also control XMIN. Consequently, non-experts can employ the default XMIN method merely by specifying `ntmin = 3`.

<code>maxcyc</code>	The maximum number of cycles of minimization. Default is 1 to be consistent with Amber's other minimization facilities although it may be unrealistically short.
<code>ntmin</code>	The flag for the method of minimization. = 3 The XMIN method is used. = 4 The LMOD method is used. The LMOD procedure employs XMIN for energy relaxation and minimization.
<code>drms</code>	The convergence criterion for the energy gradient: minimization will halt when the root-mean-square of the Cartesian elements of the gradient is less than this. Default is $10^{-4} \text{ kcal} \cdot \text{mol}^{-1} \cdot \text{\AA}^{-1}$. This is consistent with Amber's other minimization facilities. In Amber18 and earlier this default may have been unrealistically strict. In Amber20 this criterion refers to the minimization of the input structure for which the normal modes are computed, and to avoid unnatural vibrational modes it should be set to even stricter values, e.g., 10^{-8} . Compare with input parameter <code>lmod_minimize_grms</code> below.

Other options that control XMIN are in the scope of the `&lmod` namelist. These parameters enable expert control of XMIN.

<code>lbfgs_memory_depth</code>	The depth of the LBFGS memory for LBFGS minimization, or LBFGS preconditioning in TNCG minimization. Default is 3. Suggested alternate value is 5. The value 0 turns off LBFGS preconditioning in TNCG minimization.
<code>matrix_vector_product_method</code>	The finite difference Hv matrix-vector product method: "forward" = forward difference, "central" = central difference. Default is forward difference.
<code>xmin_method</code>	The minimization method: "PRCG" = Polak-Ribiere Conjugate Gradient, "LBFGS" = Limited-memory Broyden-Fletcher-Goldfarb-Shanno, and "TNCG" = Optionally LBFGS-preconditioned Truncated Newton Conjugate Gradient. Default is LBFGS.
<code>xmin_verbosity</code>	The verbosity of the internal status output from the XMIN package: 0 = none, 1 = minimization details, and 2 = minimization and line search details plus CG details in TNCG. Currently, the XMIN status output may be disordered with respect to Amber's output. Default is 0, no output of the XMIN package internal status.

25.8.5. LMOD

Some of the options that control LMOD have the same names as Amber's other minimization facilities. See the XMIN section immediately above. Other options that control LMOD are in the scope of the `&lmod` namelist. These parameters enable expert control of LMOD.

`arnoldi_dimension` The dimension of the ARPACK Arnoldi factorization. Zero specifies the whole space, that is, three times the number of atoms. Default is 0, the whole space. Basically, the ARPACK package used for the eigenvector calculations solves multiple "small" eigenvalue problems instead of a single "large" problem, which is the diagonalization of the three times the number of atoms by three times the number of atoms Hessian matrix. This parameter is the user specified dimension of the "small" problem. The allowed range is $\text{total_low_modes} + 1 \leq \text{arnoldi_dimension} \leq \text{three times the number of atoms}$. The default means that the "small" problem and the "large" problem are identical. This is the preferred, i.e., fastest, calculation for small to medium size systems, because ARPACK is guaranteed to converge in a single iteration. The ARPACK calculation scales with three times the number of atoms times the `arnoldi_dimension` squared and, therefore, for larger molecules there is an optimal `arnoldi_dimension` much less than three times the number of atoms that converges much faster in multiple iterations (possibly thousands or tens of thousands of iterations). The key to good performance is to select an `arnoldi_dimension` such that all the ARPACK storage fits in memory. For proteins, `arnoldi_dimension=1000` is generally a good value, but often a very small 50-100 Arnoldi dimension provides the fastest net computational cost with very many iterations.

`conflib_filename` The user-given filename of the LMOD conformational library. The file format is Amber standard formatted trajectory output regardless of the value of `&cntrl` namelist variable `ioutfm`. The conformations are stored in energetic order (global minimum energy structure first), the number of conformations $\leq \text{conflib_size}$. The default filename is *conflib*.

`conflib_size` The number of conformations to store in *conflib*. Default is 3.

`energy_window` The energy window for conformation storage; the energy of a stored structure will be in the interval $[\text{global_min}, \text{global_min} + \text{energy_window}]$. Default is 0, only storage of the global minimum structure.

`explored_low_modes` The number of low frequency vibrational modes used per LMOD iteration. Default is 3.

`frequency_eigenvector_recalc` The frequency, measured in LMOD iterations, of the recalculation of eigenvectors. Default is 3.

`frequency_ligand_rotrans` The frequency, measured in LMOD iterations, of the application of rigid-body rotational and translational motions to the ligand(s). At each `frequency_ligand_rotrans`-th LMOD iteration number `ligand_rotrans` rotations and translations are applied to the ligand(s). Default is 1, ligand(s) are rotated and translated at every LMOD iteration.

`lmod_job_title` The user-given title for the job that goes in the first line of the *conflib* and *lmod_trajectory* files. The default job title is "job_title_goes_here".

`lmod_minimize_grms` In Amber18 and earlier the gradient root-mean-square convergence criterion of structure minimization. In Amber 20 this was specified to be the criterion to minimize low-energy conformations; such conformations do not require as strict a convergence criterion as does the first minimization whose convergence is now controlled with input parameter `drms`, see above. Default is 0.1.

`lmod_relax_grms` The gradient RMS convergence criterion of structure relaxation. Default is 1.0.

`lmod_restart_frequency` The frequency, in LMOD iterations, of *conflib* updating and LMOD restarting with a randomly chosen structure from the pool. Default is 5.

`lmod_step_size_max` The maximum length of a single LMOD ZIG move. Default is 5.0 Å.

`lmod_step_size_min` The minimum length of a single LMOD ZIG move. Default is 2.0 Å.

25. Sampling configuration space

- `lmod_trajectory_filename` The filename of the LMOD pseudo trajectory. The file format is standard Amber trajectory file. The conformations in this file show the progress of the LMOD search. The number of conformations = number_lmod_iterations + 1. The default filename is *lmod_trajectory*.
- `lmod_verbosity` The verbosity of the internal status output from the LMOD package: 0 = none, 1 = some details, 2 = more details, 3 = everything including ARPACK information, 4 = ARPACK only, 5 = visualize normal modes and optionally the co-vibrational mode, 6 = visualize only the co-vibrational mode. Currently, the LMOD status output may be disordered with respect to Amber's output. Default is 0, no output of the LMOD package internal status. Note that LMOD is also available in AmberTools, in the SFF package as well as in the now separate NAB package. An annotated example output corresponding to LMOD.VERBOSITY=2 can be found in the NAB documentation.
- `monte_carlo_method` The Monte Carlo method: "Metropolis" = Metropolis Monte Carlo, "Total.Quench" = the LMOD trajectory always proceeds towards the lowest lying neighbor of a particular energy well found after exhaustive search along all of the low modes, and "Quick.Quench" = the LMOD trajectory proceeds towards the first neighbor found, which is lower in energy than the current point on the path, without exploring the remaining modes. Default is Metropolis Monte Carlo.
- `number_free_rottrans_modes` In Amber18 and earlier this was solely the number of rotational and translational degrees of freedom (dof) which is related to the number of frozen or tethered atoms in the system: 0 atoms dof=6, 1 atom dof=3, 2 atoms dof=1, >=3 atoms dof=0. In Amber20 the input domain was extended to any non-negative integer, and it represents the number of modes for LMOD to skip. In this way LMOD can now explore a range of modes instead of simply modes starting with the lowest frequency. Note that it is recommended to set this to 0 once in order to examine the ro-translational modes. Default is 6.
- `number_ligand_rottrans` The number of rigid-body rotational and translational motions applied to the ligand(s). Such applications occur at each frequency_ligand_rottrans-th LMOD iteration. Default is 0, no rigid-body motions applied to the ligand(s).
- `number_ligands` The number of ligands for flexible docking. Default is 0, no ligand(s).
- `number_lmod_iterations` The number of LMOD iterations. Default is 10. Note that setting number_lmod_iterations = 0 will result in a single energy minimization.
- `number_lmod_moves` The number of LMOD ZIG-ZAG moves. Zero means that the number of ZIG-ZAG moves is not pre-defined, instead LMOD will attempt to cross the barrier in as many ZIG-ZAG moves as it is necessary. The criterion of crossing an energy barrier is stated above in the "LMOD Procedure" background section. number_lmod_moves > 0 means that multiple barriers may be crossed and LMOD can carry the molecule to a large distance on the potential energy surface without severely distorting the geometry. Default is 0, LMOD will determine automatically where to stop the ZIG-ZAG sequence.
- `random_seed` The seed of the random number generator. Default is 314159.
- `restart_pool_size` The size of the pool of lowest-energy structures to be used for restarting. Default is 3.
- `rtemperature` The value of RT in Amber energy units. This is utilized in the Metropolis criterion. Default is 1.5.
- `total_low_modes` The total number of low frequency vibrational modes to be used. Default is the minimum of 10 and three times the number of atoms minus the number of rotational and translational degrees of freedom (number_free_rottrans_modes).

The following commands are part of the `&lmod` namelist. These commands control the way LMOD applies explicit translations and rotations to one or more ligands and take effect only if `number_ligands` $\geq$ 1. All commands are lists in square brackets, separated by commas such as [1, 33, 198], however, the list is read by Sander as a string and, therefore, it should be enclosed in single quotes.

`ligstart_list`, `ligend_list` The serial number(s) of the first/last atom(s) of the ligand(s). Type integer. The number(s) should correspond to the numbering in the Amber input files `prmtop` and `inpcrd/restart`. For example, if there is only one ligand and it starts at atom 193, the command should be `ligstart_list = '[193]'`. If there are three ligands, the command should be, e.g., `'[193, 244, 1435]'`. The same format holds for all of the following commands. Note that the ligand(s) can be anywhere in the atom list, however, a single ligand must have continuous numbering between the corresponding `ligstart_list` and `ligend_list` values. For example, `ligstart_list = '[193, 244, 1435]'` and `ligend_list = '[217, 302, 1473]'`.

`ligcent_list` The serial number(s) of the atom(s) of the ligand(s), which serves as the center of rotation. Type integer. The value zero means that the center of rotation will be the geometric center of gravity of the ligand.

`rotmin_list`, `rotmax_list` The range of random rotation of a particular ligand about the origin defined by the corresponding `ligcent_list` value is specified by the commands `rotmin_list` and `rotmax_list`. The angle is given in +/- degrees. Type float. For example, in case of a single ligand and `ligcent_list = '[0]'`, `rotmin_list = '[30.0]'` and `rotmax_list = '[180.0]'` means that random rotations by an angle +/- 30-180 degrees about the center of gravity of the ligand, will be applied. Similarly, with `number_ligands = 2`, `ligcent_list = '[120.0]'` means that the first ligand will be rotated like in the single ligand example in this paragraph, but a second ligand will be rotated about its atom number 201, by an angle +/- 60-120 degrees.

`trmin_list`, `trmax_list` The range of random translation(s) of ligand(s) is defined by the same way as rotation. For example, with `number_ligand = 1`, `trmin_list = '[0.1]'` and `trmax_list = '[1.0]'` means that a single ligand is translated in a random direction by a random distance between 0.1 and 1.0 Angstroms.

25.9. DL-Find Optimization

The [DL-Find software](#) is a popular, open-source suite of geometry optimization and transition state search algorithms. Sander has been modified to directly link to [libdlfind](#), which is a fork of DL-Find taken from PyChemShell version v21.0.1. The DL-Find optimizer is useful in the context of QM/MM applications in which one searches for potential energy stationary points along a chemical reaction. To enable DL-Find optimization, set the following options in the `&cntrl` namelist: `"imin=1, ntmin=5, ntc=1, ntf=1"`. The DL-Find options are read from the `&dlfind` namelist.

25.9.1. &dlfind Namelist Options

`refcrd` (string) A restart coordinate filename, default="". If present, then Amber masks specified in the `&dlfind` namelist will be interpreted using the coordinates within `refcrd`. If `refcrd` is not specified, then the input coordinates (as specified with the `-c` command line option) will be used.

`active` (string) An amber mask indicating which atoms are allowed to move, default="*". The geometry optimization will only move the selected atoms in the fixed environment of all other atoms. In QM/MM applications, one would often set `active="QMMASK"` or `active="QMMASK<:3.0"`, where `QMMASK` is the Amber selection specified in the `&qmmm` namelist. The second form selects the QM atoms and any MM residues within 3 Angstroms of the QM region.

`crdrep` (string) The coordinate system used to optimize the geometry, default="HDLC". The DLC and TC options are impractical unless the active selection is very small.

25. Sampling configuration space

	= "CART" Cartesian coordinates = "MASS" Mass weighted Cartesian coordinates = "DLC" Delocalized coordinates (e.g., redundant internal coordinates) = "HDLC" Hybrid delocalized coordinates (intraresidue-DLC) = "TC" Total connection ($N*(N-1)/2$ bonds) = "HDLC-TC" Hybrid delocalized/total connection (intraresidue-TC)
optalg	(string) The optimization algorithm, default="LGFBS". = "LBFGS" Limited memory Broyden-Fletcher-Goldfarb-Shanno minimization. See also the lbfgsmem option, below. = "PRFO" Partitioned rational function optimization for a transition state. Note that this algorithm requires Hessians and is only practical for small active selections. See the hessupd and maxupdate options, below. The maxupdate option controls how often an explicit rebuild of the Hessian is calculated, and hessupd controls how the Hessian is approximated when an explicit rebuild is not performed. = "CG" Minimum search with the conjugate gradient method. = "SD" Minimum search with the steepest descent method. = "NR" Newton-Raphson/quasi-Newton optimizer. = "HESS" Do not perform an optimization; instead, calculate a Hessian and write it to a NetCDF file for later analysis. See the options: hess_out, hess_iseg, and hess_nseg.
trustrad	(string) The trust radius approach, default="CONST". = "CONST" Constant trust radius (specified with the maxstep option). = "ENE" Adjust trust radius based on the energy change. = "GRAD" Adjust the trust radius based on the projected gradient.
hessini	(string) How to approximate the initial Hessian, default="IDENTITY". If optalg="HESS", then only "ONEPOINT" and "TWOPOINT" are valid options. If optalg="PRFO", then one usually sets hessini="ONEPOINT". = "ONEPOINT" Finite difference of forces calculated from a single displacement. = "TWOPOINT" Central finite difference approximation of the forces. = "DIAGONAL" Only the diagonal elements are approximated from ONEPOINT. = "IDENTITY" Use the identity matrix.
hessupd	(string) Method for updating the approximate Hessian, default="BOFILL". = "NONE" Do not update the Hessian, explicitly recalculate it when needed. This option is rarely practical. = "POWELL" Powell update. = "BOFILL" Bofill update. = "BFGS" BFGS update.
maxupdate	(integer) The frequency between explicit recalculation of the Hessian, default=5. This option is most relevant for optalg="PRFO". If maxupdate is "large", then the PRFO steps will take less time, but it is more likely to fail to locate a transition state.
delta	(float) The finite difference displacement (Bohr) used to approximate the Hessian from the forces, default=0.005.

<code>lbfgsmem</code>	(integer) The number of previous LBFGS Hessians to store ! when performing LBFGS optimizations, default=5.
<code>verbosity</code>	(integer) Verbosity level, default=2. =0 Minimal output =1 Medium output =2 Full output =3 Debug output
<code>tol</code>	(float) Convergence tolerance (Hartree/Bohr) on maximum gradient component, default=4.5e-4.
<code>tole</code>	(float) Convergence criterion (Hartree) on the maximum predicted energy change, default=1.e-6.
<code>maxcycle</code>	(integer) Maximum number of optimization cycles, default=1000.
<code>maxstep</code>	(float) Maximum stepsize in internal coordinates (Bohr), default=0.5
<code>minstep</code>	(float) Do not update Hessian if predicted step size is less than minstep (Bohr), default=1.e-7.
<code>scalestep</code>	(float) A constant factor use to scale the step size, default=1.0.

Options that control the handling of constraints

<code>allow_shake</code>	(logical) By default (<code>allow_shake=.false.</code>), if the active selection contains shaken bonds, then the program will stop with an error message. It is recommended to turn shake off because it appears to optimize to unintended high-order saddle points. For debug purposes, you can set this to <code>.true.</code> to turn the error messages into warnings.
<code>disang</code>	(string) Filename containing nmropt-style restraint definitions, default="". The biasing potentials are ignored, however, the bonds, angles, and torsions defined within this file will be treated as constrained coordinates. The constraint condition values are determined from the input coordinates. In other words, the specified bonds, angles, and/or torsions will not change during the optimization.
<code>merge_residues</code>	(logical) By default (<code>merge_residues=.false.</code>), DL-Find will terminate if a bond, angle, or torsion constraint (as read from the <code>disang</code> option, above) crosses a residue boundary and <code>crdrep="HDLC"</code> or <code>"HDLC-TC"</code> . If <code>merge_residues=.true.</code> , then inter-residue constraints will cause the residues to be interpreted as a merged, super-residue.
<code>wtmask</code>	(string) An Amber atom selection mask, default="". Only used if <code>dimer=.true.</code> or <code>neb /= "NO"</code> . The dimer method and NEB algorithms involve aligning images from a weighted root mean square coordinate overlay. The weight of all QM atoms is 1. The weight of all MM atoms is <code>"wtmm"</code> . All other active atoms not contained within <code>wtmask</code> have a weight of 0.0. If <code>wtmask</code> is blank, then all active atoms are selected.
<code>wtmm</code>	(string) The weight associated with MM atoms, default=1.0. Only used if <code>dimer=.true.</code> or <code>neb /= "NO"</code> .

Options specific to `optalg="HESS"`

<code>hess_out</code>	(string) Filename of the output Netcdf Hessian file, default="hess_out.nc". Only used if <code>optalg="HESS"</code> . If <code>hess_nseg > 1</code> , then the output filename will be <code>hess_out.nc.XXX</code> , where <code>XXX</code> is a zero-padded integer: the value of <code>hess_iseg</code> .
<code>hess_nseg</code>	(integer) By default (<code>hess_nseg=1</code>), the entire Hessian is computed. Only used if <code>optalg="HESS"</code> . For large Hessians or expensive Hamiltonians, it may be faster to split the columns of the Hessian into segments that can be separately computed, and then recombine the Netcdf files into a single file. This option is the number of segments the Hessian is split into.

25. Sampling configuration space

hess_iseg (integer) The segment that this Sander instance will calculate, default=1. Only used if **optalg**="HESS". This value must be $1 \leq \text{hess_iseg} \leq \text{hess_nseg}$.

Options specific to the dimer method for locating transition states

dimer (logical) Perform the dimer method, default=.false..

dimer_mode (integer) Rotation line search method, default=1.

=0 All rotations covered by the optimizer. Requires 2 energy evaluations per iteration.

=1 Rotations by linesearch; No extrapolation of gradient. Two energy calculations are used per rotation. Requires at least 2 energy evaluations per iteration.

=2 Rotations by linesearch; Extrapolation of gradient. One energy calculation is done per iteration, the other one is interpolated. Requires at least 2 energy evaluations per iteration.

dimer_tolrot (float) Angle tolerance (degrees) in the dimer rotations, default=2.0.

dimer_maxrot (integer) Maximum number of rotations in each dimer step, default=20.

coords2 (list of string) Only the first element, **coords2(1)**, is required; it is the name of the restart coordinate file containing the displaced coordinates along the transition state mode. For example, **coords2(1)**="tsdisp.rst7". The input coordinates (specified with **-c** command line option) is the best guess at the transition state, and **coords2(1)** displaces the coordinates in the direction of the TS motion. One can create **tsdisp.rst7** by performing a restrained optimization that uses **nmropt=1** to push the relevant reaction coordinates toward the product state.

Options specific to nudged elastic band

neb (string) Chooses the nudge elastic band algorithm, default="NO".

= "NO" Do not perform nudged elastic band.

= "FREE" Allow the end-states to minimize.

= "FROZEN" Freeze the end-state coordinates.

= "PERP" Allow the end-states to move only in directions perpendicular to the path.

coords2 (list of string) The initial images used to interpolate a path. At least 1 image is required. Up to 12 images can be provided. The reactant state is input from the command line (the **-c** option). If one wanted to make a "straight line" guess at the path, then one would set **coords2(1)**="product.rst7". If one also had a guess at the transition state, then one would set **coords2(1)**="ts.rst7" and **coords2(2)**="product.rst7". If the mechanism traverses an intermediate state (that is, it is a multi-step mechanism), then it is recommended to break the NEB calculations into several steps, such that each step traverses a single transition state.

neb_nimage (integer) The number of images to optimize, default=1. This should be greater or equal to the number of input images.

neb_climb_test (float) Threshold scale factor for spawning climbing image (expressed as a multiple of the gradient tolerance for convergence), default=3.0. Setting **neb_climb_test** = **neb_freeze_test** = 0 may help if convergence issues are encountered during the later stage of the optimization.

neb_freeze_test (float) Threshold scale factor for freezing NEB images (expressed as a multiple of the gradient tolerance for convergence), default=1.0.

nebk (float) The NEB spring force constant (atomic units), default=0.01.

`neb_cart` (logical) If true (default), then interpolate the intermediate images using the specified coordinate system to create initial geometries, but perform the optimization in Cartesians. The initial guess interpolation can be performed in CART, DLC, or TC coordinate systems, but not HDLC nor HDLC-TC. Considering that DLC and TC coordinate systems can only be applied to small active regions, one will likely need to use `crdrep="CART"` to perform NEB.

26. Free energies

26.1. Thermodynamic integration

In a free energy calculation, the system evolves according to a mixed potential (such as in Eqs. 26.3 or 26.4, below). The essence of free energy calculations is to record and analyze the fluctuations in the values of V_0 and V_1 (that is, what the energies *would have been* with the endpoint potentials) as the simulation progresses. For thermodynamic integration (which is a very straightforward form of analysis) the required averages can be computed "on-the-fly" (as the simulation progresses), and printed at the end of a run. For more complex analyses (such as the Bennett acceptance ratio scheme), one needs to write the history of the values of V_0 and V_1 to a file, and later post-process this file to obtain the final free energy estimates.

There is not room here to discuss the theory of free energy simulations, and there are many excellent discussions elsewhere.[11, 587, 588] There are also plenty of recent examples to consult.[589, 590] Such calculations are demanding, both in terms of computer time, and in a level of sophistication to avoid pitfalls that can lead to poor convergence. Since there is no one "best way" to estimate free energies, *sander* and *pmemd* primarily provide the tools to collect the statistics that are needed. Assembling these into a final answer, and assessing the accuracy and significance of the results, generally requires some calculations outside of what Amber provides, *per se*. The discussion here will assume a certain level of familiarity with the basis of free energy calculations.

Both *sander* and *pmemd* have the capability of doing simple thermodynamic free energy calculations, using either PME or generalized Born potentials. When *icfe* is set to 1, information useful for doing thermodynamic integration estimates of free energy changes will be computed. The implementation is different between *sander* and *pmemd*. For *sander*, you must use the *multisander* capability to create two groups, one corresponding to the starting state, and a second corresponding to the ending state (see Section 22.12 for information); you will need a *prmtop* file for each of these two endpoints. For *pmemd*, you use a single *prmtop* file which contains both the starting and ending states. For both *sander* and *pmemd* a mixing parameter λ is used to interpolate between the "unperturbed" and "perturbed" potential functions.

26.1.1. Thermodynamic integration using Sander

There are now two different ways to prepare a thermodynamic integration free energy calculation in Sander. The first is unchanged from previous versions of Amber: Here, the two *prmtop* files that you create must have the same number of atoms, and the atoms must appear *in the same order* in the two files. This is because there is only one set of coordinates that are propagated in the molecular dynamics algorithm. If there are more atoms in the initial state than in the final, "dummy" atoms must be introduced into the final state to make up the difference. Although there is quite a bit of flexibility in choosing the initial and final states, it is important in general that the system be able to morph "smoothly" from the initial to the final state. Alternatively, you can set up your system to use the softcore potential algorithm described below. This will remove the requirement to prepare "dummy" atoms and allows the two *prmtop* files to have different numbers of atoms.

The basics of the *multisander* functionality are given in Section 22.12, but the mechanics are really quite simple. You start a free energy calculation as follows:

```
mpirun -np 4 sander.MPI -ng 2 -groupfile <filename>
```

Since there are 4 total cpu's in this example, each of the two groups will run in parallel with 2 cpu's each. The number of processors must be a multiple of two. The *groups* file might look like this:

```
-O -i mdin -p prmtop.0 -c eq1.x -o md1.o -r md1.x -inf mdinfo  
-O -i mdin -p prmtop.1 -c eq1.x -o md1b.o -r md1b.x -inf mdinfo
```


The input (*mdin*) and starting coordinate files must be the same for the two groups. Furthermore, the two *prmtop* files must have the same number of atoms, in the same order (since one common set of coordinates will be used for both.) The simulation will use the masses found in the first *prmtop* file; in classical statistical mechanics, the Boltzmann distribution in coordinates is independent of the masses so this should not represent any real restriction.

On output, the two restart files should be identical, and the two output files should differ only in trivial ways such as timings; there should be no differences in any energy-related quantities, except if energy decomposition is turned on (*idecomp* > 0); then only the output file of the first group contains the per residue contributions to $\langle \partial V / \partial \lambda \rangle$. For our example, this means that one could delete the *mdl.b.o* and *mdl.b.x* files, since the information they contain is also in *mdl.o* and *mdl.x*. (It is a good practice, however, to check these file identities, to make sure that nothing has gone wrong.)

26.1.2. Thermodynamic integration using PMEMD

In *pmemd*, there is only a single input topology file which contains the atoms corresponding to both the start and end states. As explained in Ref. [591] this removes redundant calculations, greatly improving the efficiency of the code. In order to accommodate these changes, some input flags have been modified compared to *sander*. These are marked in the sections below. Also, simulations at the endpoints, $\lambda = 0$ or $\lambda = 1$, will work even for soft core simulations.

The *prmtop* file needs to be carefully prepared in order to be compatible with the *pmemd* TI implementation. A number of examples for setting up the *prmtop* file are given below in section 26.1.8. This is not a complete tutorial on TI calculations, but explains how to prepare the new *prmtop* format for various types of TI calculations.

Performance of the PME TI *pmemd* implementation is approximately 75% that of a regular PME MD simulation with roughly the same parallel scaling. The difference in absolute performance comes from the fact that a PME calculation is not pairwise decomposable and therefore the reciprocal space calculation needs to be carried out twice per time step, once for V_0 and V_1 . For GB TI the performance difference is approximately 50% since the GB radii calculation is not pairwise decomposable and thus two non-bond calculations are carried out per time step.

The exception to this performance difference is when one is running just vdW only soft core transformations. In this situation there are no charges on the TI atoms and thus the charges for all of the atoms in both V_0 and V_1 are the same. Hence the long range electrostatics calculation only needs to be done once per step, rather than twice (for V_0 and V_1). This results in performance roughly equivalent to a standard MD simulation. This optimization is determined automatically and can be seen in the *mdout* file – ‘No charge on TI atoms. Skipping extra recip sum.’ To determine the total free energy change it is necessary to carry out additional simulations to determine the free energy of removing the charges from the molecules. It is up to the user to decide which path through the thermodynamic cycle will be more efficient for their system of interest.

26.1.3. Thermodynamic integration using PMEMD.cuda

The TI implementation of the GPU version of *pmemd* (*pmemd.cuda*) uses the same input files as the CPU version of *pmemd* TI implementations, with some additional *pmemd.cuda*-specific input parameters (details given below in Section 26.1.7). Performance of the *pmemd.cuda* TI implementation is approximately 70% that of a regular *pmemd.cuda* MD simulation,[592, 593] and has been applied to a wide array of relative binding free energy calculations for protein-ligand systems.[594] The current version of *pmemd.cuda* can be compiled with MPI to perform replica exchange simulations using multiple GPUs, and the currently it does not support a single TI simulation using multiple GPUs. The current version of *pmemd.cuda* TI does not support GB or PB calculations.

26.1.4. Basic inputs for thermodynamic integration

icfe	The basic flag for free energy calculations. The default value of 0 skips such calculations. Setting this flag to 1 turns them on, using the mixing rules in Eq. 26.3, below.
clambda	The value of λ for this run, as in Eqs. 26.3 and 26.4, below. Zero corresponds to the unperturbed Hamiltonian V_0 . $\lambda=1$ corresponds to the perturbed Hamiltonian V_1 .

26. Free energies

<code>klambda</code>	The exponent in Eq. 26.4, below.
<code>tishake</code>	Flag that determines how SHAKE is handled (Note that this flag has different implementation in <code>pmemd.cuda</code>): = 0 Coordinates are synchronized after SHAKE, no constraints removed. Caution: this could cause problems when a SHAKE bond containing one common and softcore atom(s). The potential problem is solved in the <code>pmemd.cuda</code> implementation. = 1 (default) SHAKE is removed between bonds containing one common and softcore atom(s). Note that disabling SHAKE requires the use of a 1 fs timestep.

26.1.4.1. Input flags specific to Sander

<code>idecomp</code>	Flag that turns on/off decomposition of $\langle \partial V / \partial \lambda \rangle$ on a per-residue level. The default value of 0 turns off energy decomposition. A value of 1 turns the decomposition on, and 1-4 nonbonded energies are added to internal energies (bond, angle, torsional). A value of 2 turns the decomposition on, and 1-4 nonbonded energies are added to EEL and VDW energies, respectively. The frequency by which values of $\langle \partial V / \partial \lambda \rangle$ are included into the decomposition is determined by the NTPR flag. This ensures that the sum of all contributions equals the average of all total $\langle \partial V / \partial \lambda \rangle$ values output every NTPR steps. All residues, including solvent molecules, have to be chosen by the RRES card to be considered for decomposition. The RES card determines which residue information is finally output. The output comes at the end of the <code>mdout</code> file. For each residue contributions of internal -, VdW-, and electrostatic energies to $\langle \partial V / \partial \lambda \rangle$ are given as an average over all (NSTLIM/NTPR) steps. In a first section total per residue values are output followed below by further decomposed values from backbone and sidechain atoms.
----------------------	---

26.1.4.2. Input flags specific to PMEMD

<code>tmask1</code>	Specifies the atoms unique to V_0 in ambmask format.
<code>tmask2</code>	Specifies the atoms unique to V_1 in ambmask format.

26.1.5. Background theory of thermodynamic integration

The *sander* and *pmemd* programs do not compute free energies; it is up to the user to combine the output of several runs (at different values of λ) and to numerically estimate the integral:

$$\Delta A = A(\lambda = 1) - A(\lambda = 0) = \int_0^1 \langle \partial V / \partial \lambda \rangle_{\lambda} d\lambda \quad (26.1)$$

If you understand how free energies work, this should not be at all difficult. However, since the actual values of λ that are needed, and the exact method of numerical integration, depend upon the problem and upon the precision desired, we have not tried to pre-code these into the program.

The simplest numerical integration is to evaluate the integrand at the midpoint:

$$\Delta A \simeq \langle \partial V / \partial \lambda \rangle_{1/2}$$

This might be a good first thing to do to get some picture of what is going on, but is only expected to be accurate for very smooth or small changes, such as changing just the charges on some atoms. Gaussian quadrature formulas of higher order are generally more useful:

$$\Delta A = \sum_i w_i \langle \partial V / \partial \lambda \rangle_i \quad (26.2)$$

Some weights and quadrature points are given in the accompanying table; other formulas are possible,[595] but the Gaussian ones listed there are probably the most useful. The formulas are always symmetrical about $\lambda = 0.5$,

so that λ and $(1 - \lambda)$ both have the same weight. For example, if you wanted to use 5-point quadrature, you would need to run five jobs, setting λ to 0.04691, 0.23076, 0.5, 0.76923, and 0.95308 in turn. (Each value of λ should have an equilibration period as well as a sampling period; this can be achieved by setting the *ntave* parameter.) You would then multiply the values of $\langle \partial V / \partial \lambda \rangle_i$ by the weights listed in the Table, and compute the sum.

When *icfe*=1 and *klambda* has its default value of 1, the simulation uses the mixed potential function:

$$V(\lambda) = (1 - \lambda)V_0 + \lambda V_1 \quad (26.3)$$

where V_0 is the potential with the original Hamiltonian, and V_1 is the potential with the perturbed Hamiltonian. The program also computes and prints $\langle \partial V / \partial \lambda \rangle$ and its averages; note that in this case, $\langle \partial V / \partial \lambda \rangle = V_1 - V_0$. This is referred to as linear mixing, and is often what you want unless you are making atoms appear or disappear. If some of the perturbed atoms are "dummy" atoms (with no van der Waals terms, so that you are making these atoms "disappear" in the perturbed state), the integrand in Eq. 26.1 diverges at $\lambda = 1$; this is a mild enough divergence that the overall integral remains finite, but it still requires special numerical integration techniques to obtain a good estimate of the integral.[588] *Sander* and *pmemd* implement one simple way of handling this problem: if you set *klambda* > 1, the mixing rules are

$$V(\lambda) = (1 - \lambda)^k V_0 + [1 - (1 - \lambda)^k] V_1 \quad (26.4)$$

where k is given by *klambda*. Note that this reduces to Eq. 26.3 when $k = 1$, which is the default. If $k \geq 4$, the integrand remains finite as $\lambda \rightarrow 1$. [588] We have found that setting $k = 6$ with disappearing groups as large as tryptophan works, but using the softcore option (*ifsc*>0) instead is generally preferred.[596] Note that the behavior of $\langle \partial V / \partial \lambda \rangle$ as a function of λ is not monotonic when *klambda* > 1. You may need a fairly fine quadrature to get converged results for the integral, and you may want to sample more carefully in regions where $\langle \partial V / \partial \lambda \rangle$ is changing rapidly.

Notes:

1. This is implemented in *sander* by calling the force() routine independently for each *multisander* group and then combining the forces on each step. For a fixed number of processors this increases the cost of the calculation compared with the *pmemd* code, which only calculates the differences between V_0 and V_1 .
2. It is rather easy to make mistakes when running TI calculations. It is generally good to carry out a short run (say 50 steps) setting *ntpr*=1. Then check the following; if either test fails, be sure to fix the problem before proceeding.
 - a) The restart files from V_0 and V_1 should be identical for *sander* (for *pmemd* there will only be a single restart file).
 - b) If you diff the output files for *sander*, there should only be simple differences (for *pmemd* there will only be a single combined output file). All energies, temperatures, pressures, etc. should be the same in the two files. Simulations with *sander* using the QM/MM facility may show differences in the SCF energies, but be sure that the total energies, and all the MM components, are the same.
3. Eq. 26.4 is designed for having dummy atoms in the perturbed Hamiltonian, and "real" atoms in the regular Hamiltonian. You must ensure that this is the case when you set up the system in LEaP. (See the softcore section, below, for a more general way to handle disappearing atoms, which does not require dummy atoms at all.)
4. One common application of this model is to pKa calculations, where the charges are mutated from the protonated to the deprotonated form. Since H atoms bonded to oxygen already have zero van der Waals radii (in the Amber force fields and in TIP3P water), once their charge is removed (in the deprotonated form) they are really then like dummy atoms. For this special situation, there is no need to use *klambda* > 1: since the van der Waals terms are missing from both the perturbed and unperturbed states, the proton's position can never lead to the large contributions to $\langle \partial V / \partial \lambda \rangle$ that can occur when one is changing from a zero van der Waals term to a finite one.

26. Free energies

5. The implementation requires that the masses of all atoms be the same on all threads. To enforce this, the masses found for V_0 are used for V_1 as well. In classical statistical mechanics, the canonical distribution of configurations (and hence of potential energies) is unaffected by changes in the masses, so this should not pose a limitation. Since the masses for V_1 are ignored, they do not have to match those found for V_0 .
6. Special care needs to be taken when using SHAKE for atoms whose force field parameters differ in the two end points. The same bonds must be SHAKEN in both cases, and the equilibrium bond lengths must also be the same. By default, the coordinates from V_0 are synchronized with those from V_1 after SHAKE. This will work for small perturbations, but if there is a significant change in bond length, it may be necessary to use the *noshakemask* input to remove SHAKE from the regions that are being perturbed. If this is done, be sure to set *tishake*=1 and to use a 1 fs timestep. This limitation is only valid in pmem and has been solved in the pmemd.cuda implementation.
7. Special care needs to be taken when water molecules are part of the region that is changing. You need to make sure that the “number of 3-point waters” is the same in both V_0 and V_1 . This may require setting *jfastw* and/or building the structure so that *sander* or *pmemd* do not think that the water molecules involved are actually rigid waters. Also, just setting *noshakemask* might not be enough, since this flag does not affect the *settle* routine that handles rigid waters.

n	λ_i	$1 - \lambda_i$	w_i
1	0.5		1.0
2	0.21132	0.78867	0.5
3	0.1127 0.5	0.88729	0.27777 0.44444
5	0.04691 0.23076 0.5	0.95308 0.76923	0.11846 0.23931 0.28444
7	0.02544 0.12923 0.29707 0.5	0.97455 0.87076 0.70292	0.06474 0.13985 0.19091 0.20897
9	0.01592 0.08198 0.19331 0.33787 0.5	0.98408 0.91802 0.80669 0.66213	0.04064 0.09032 0.13031 0.15617 0.16512
12	0.00922 0.04794 0.11505 0.20634 0.31608 0.43738	0.99078 0.95206 0.88495 0.79366 0.68392 0.56262	0.02359 0.05347 0.08004 0.10158 0.11675 0.12457

Table 26.1.: Abscissas and weights for Gaussian integration.

26.1.6. Softcore Potentials in Thermodynamic Integration

Softcore potentials provide an additional way to perform thermodynamic integration calculations in Amber. The system setup has been simplified so that appearing and disappearing atoms can be present at the same time and no dummy atoms need to be introduced. For *sander*, two prmtop files, corresponding to the start and end states (V_0 and V_1) of the desired transformation need to be used. The common atoms that are present in both states need to appear in the same order in both prmtop files and must have identical starting positions. In addition to the common

atoms, each process can have any number of unique soft core atoms, as specified by *scmask*. For *pmemd*, a single *prmtop* file is used, containing the unique atoms for both the start and end states. The soft core atoms are specified by *scmask1* and *scmask2* for V_0 and V_1 respectively.

A modified version of the vdW equation is used to smoothly switch off non-bonded interactions of these atoms with their common atom neighbors:

$$V_{V_0,disappearing} = 4\epsilon(1-\lambda) \left[\frac{1}{\left[\alpha\lambda + \left(\frac{r_{ij}}{\sigma}\right)^6\right]^2} - \frac{1}{\alpha\lambda + \left(\frac{r_{ij}}{\sigma}\right)^6} \right] \quad (26.5)$$

$$V_{V_1,appearing} = 4\epsilon\lambda \left[\frac{1}{\left[\alpha(1-\lambda) + \left(\frac{r_{ij}}{\sigma}\right)^6\right]^2} - \frac{1}{\alpha(1-\lambda) + \left(\frac{r_{ij}}{\sigma}\right)^6} \right] \quad (26.6)$$

Please refer to Ref [596] for a description of the implementation and performance testing when compared to the TI methods described above using *sander*. For similar information pertaining to *pmemd* please see Ref [591]. Note that the term “disappearing” is used here, but it would probably be better to say that atoms present in V_0 but not in V_1 are “decoupled” from their environment: the interactions among the “disappearing” atoms are not changed, and do not contribute to $\langle \partial V / \partial \lambda \rangle$. If the disappearing atoms are a separate molecule (say a non-covalently-bound ligand), this can be viewed as a transfer to the gas-phase.

Note that a slightly different setup is required for using soft core potentials compared to older TI implementations. Specifically, the difference is that to add or remove atoms without soft core potentials, they are transformed into interactionless dummy particles, so both end state *prmtop* files have the same number of atoms. When using soft core potentials instead, no dummy atoms are needed and the end states should be built without them. Therefore *prmtop* files for non soft core simulations may have to be adapted to be used with soft core potentials and vice versa.

All bonded interactions of the unique atoms are recorded separately in the output file (see below). Any bond, angle, dihedral or 1-4 term that involves at least one appearing or disappearing atom is not scaled by λ and does not contribute to $\langle \partial V / \partial \lambda \rangle$. Therefore, output from both processes will not be identical when soft core potentials are used. Softcore transformations avoid the origin singularity effect and therefore linear mixing can (and should) always be used with them. Since the unique atoms become decoupled from their surroundings at high or low lambdas and energy exchange between them and surrounding solvent becomes inefficient, a Berendsen type thermostat should not be used for SC calculations. Unlike in previous versions, SHAKE constraints are not automatically removed from bonds between common and unique atoms. Instead, the coordinates corresponding to common atoms in V_0 are synchronized with those of V_1 . The original behavior can be restored using *tishake*. The *icfe* and *klambda* parameters should be set to 1 for a soft core run and the desired lambda value will be specified by *clambda*. When using softcore potentials with *sander*, λ values should be picked so that $0.01 < \text{clambda} < 0.99$. The *pmemd* implementation allows lambda to be set to any value between 0.0 and 1.0, thus simulations at the endpoints are possible.

Additionally, the following parameters are available to control the TI calculation:

<code>ifsc</code>	Flag for soft core potentials = 0 SC potentials are not used (default) = 1 SC potentials are used. Be sure to use <i>prmtop</i> files that are suitable for this, i.e. not-containing dummy atoms (see above)
<code>scalpha</code>	The α parameter in 26.5 and 26.6, its default value is 0.5. Other values have not been extensively tested
<code>logdvd1</code>	If set to .ne. 0, a summary of all $\partial V / \partial \lambda$ values calculated during every step of the run will be printed out at the end of the simulation for postprocessing.
<code>dvd1_norest</code>	This option is now deprecated. Restraints involving soft core atoms are now decoupled from the rest of the system. The energy is listed separately and does not contribute to $\partial V / \partial \lambda$.

26. Free energies

<code>dynlmb</code>	If set to a value .gt. zero, <code>clambda</code> is increased by <code>dynlmb</code> every <code>ntave</code> steps. This can be used to perform simulations with dynamically changing <code>lambdas</code> .
<code>crgmask</code>	Specifies a number of atoms (in <code>ambmask</code> format) that will have their atomic partial charges set to zero. This is mainly for convenience because it removes the need to build additional <code>prmtop</code> files with uncharged atoms for TI calculations involving the removal of partial charges.

26.1.6.1. Input flags specific to Sander

<code>scmask</code>	Specifies the unique (soft core) atoms for this process in <code>ambmask</code> format. This, along with <code>crgmask</code> , is the only parameter that will frequently be different in the two <code>mdin</code> files for V_0 and V_1 . It is valid to set <code>scmask</code> to an empty string. A summary of the atoms in <code>scmask</code> is printed at the end of <code>mdout</code> .
---------------------	---

26.1.6.2. Input flags specific to PMEMD

<code>scmask1</code>	Specifies the unique (soft core) atoms for V_0 in <code>ambmask</code> format. It is valid to set <code>scmask1</code> to an empty string.
<code>scmask2</code>	Specifies the unique (soft core) atoms for V_1 in <code>ambmask</code> format. It is valid to set <code>scmask2</code> to an empty string.

The force field potential energy contributions for the unique atoms in each process will be evaluated separately during the simulation and are recorded after the complete system energy is given:

```

Softcore part of the system:    15 atoms, TEMP (K)      = 316.69
SC_Etot=      24.3248 SC_EKtot=   11.6426 SC_EPtot      = 12.6822
SC_BOND=       4.7723 SC_ANGLE=    2.1411 SC_DIHED      =  1.6096
SC_14NB=       4.2947 SC_14EEL=    0.0000 SC_VDW        = -0.1355
SC_EEL =       0.0000
SC_RES_DIST=  0.0000 SC_RES_ANG=  0.0000 SC_RES_TORS=    0.0000
SC_RES_PLPT=  0.0000 SC_RES_PLPL=  0.0000 SC_RES_GEN =    0.0000
SC_EEL_DER=   0.0000 SC_VDW_DER=-11.1533 SC_DERIV      = -11.1533

```

The temperatures reported are calculated for the SC atoms only and fluctuate strongly for small numbers of unique atoms. The energies in the first six lines include all terms that involve at least one unique atom, but `SC_VDW` gives the vdW energy for pairs of unique atoms only which are subject to the standard 12-6 LJ potential. The vdW potential between soft core / non soft core atoms (as given by equation 26.5) is part of the regular VDWAALS term and is counted for $dV/d\lambda$. The same applies to `SC_EEL`, which gives only the electrostatic interactions between unique atoms, since electrostatics between soft core / non soft core atoms (for which equation 26.7 is used) are part of regular EEL-energy. Note that the total potential energy, `SC_EPtot`, does not include contributions from the restraint energies.

`SC_EEL_DER`, `SC_VDW_DER`, and `SC_DERIV` are additional λ -dependent contributions to $\langle \partial V / \partial \lambda \rangle$ that arise from the form of the SC-potentials. For more information on how to perform and setup calculations, please consult the tutorials provided at <https://ambermd.org>.

26.1.6.3. One step transformations using soft core electrostatics

Alternatively to the two-step process of removing charges from atoms first and then changing the vdW parameters of chargeless atoms in a second TI calculation, *sander* and *pmemd* also have a soft core version of the Coulomb equation implemented for single step transformations under periodic boundary conditions. This is automatically applied to all atoms in `scmask` and their interactions with common atoms are given by:

$$V_{0,disappearing} = (1 - \lambda) \frac{q_i q_j}{4\pi\epsilon_o \sqrt{\beta\lambda + r_{ij}^2}} \quad (26.7)$$

for disappearing atoms. Replace λ by $(1 - \lambda)$ and vice versa for the form for appearing atoms. This introduces a new parameter β which controls the 'softness' of the potential. This is set in the input file via:

scbeta The parameter β in 26.7. Default value is $12\text{\AA}^2$, other values have not been extensively tested.

With the use of soft core vdW and electrostatics interactions, arbitrary changes between systems are possible in single TI calculations. However, due to the unusual potential function forms introduced, it is not always clear that a single-step calculation will converge faster than one broken down into several steps. Ref. [597] contains detailed information on the performance of such single step TI calculations.

26.1.7. pmemd.cuda-specific functionalities

26.1.7.1. Smoothstep function implementation of softcore potential and λ -scheduling

The softcore potentials theoretically can avoid the so-called "end-point catastrophe" in the cases with appearing and/or disappearing atoms. Nevertheless, there are still some practical issues to be solved. Recently the incorporation of the smoothstep function into the current AMBER softcore potential has been implemented in the pmemd.cuda TI module[598]. The smoothstep function $S_p(x)$ is a function which functional values and its derivatives up to P^{th} vanish at the boundaries $x = 0$ and $x = 1$. Such properties deliver much smooth and numerically integrable $\langle \partial V / \partial \lambda \rangle$ curves. The smoothstep function implementation of the current AMBER softcore potential is only available on GPU (pmemd.cuda). Here is a brief description:

The smoothstep functions are monotonically increasing functions that have the desirable endpoint values:

For $0 \leq x \leq 1$:

$$\begin{aligned} S_0(x) &= x \\ S_1(x) &= -2x^3 + 3x^2 \\ S_2(x) &= 6x^5 - 15x^4 + 10x^3 \\ S_3(x) &= -20x^7 + 70x^6 - 84x^5 + 35x^4 \\ S_4(x) &= 70x^9 - 315x^8 + 540x^7 - 420x^6 + 126x^5 \end{aligned} \quad (26.8)$$

For all p , $S_p(x < 0) = 0$; $S_p(x > 1) = 1$. Their derivatives, up to p^{th} , are zero at the endpoints. The softcore potentials can be re-written in forms of modified interatomic distances (r_{ij} to $r_{ij}^{VDW}(\lambda; \alpha)$ for VDW; and r_{ij} to $r_{ij}^{Elec}(\lambda; \alpha)$ for Elec):

$$\begin{aligned} V_{V_0}^{VDW}(r_{ij}) &= 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \\ V_{V_0}^{Elec}(r_{ij}) &= \left(\frac{q_i q_j}{4\pi\epsilon_0} \right) \frac{1}{r_{ij}} \\ r_{ij}^{VDW}(\lambda; \alpha) &= [r_{ij}^n + \lambda \alpha \sigma_{ij}^n]^{1/n} \\ r_{ij}^{Elec}(\lambda; \beta) &= [r_{ij}^m + \lambda \beta]^{1/m} \end{aligned} \quad (26.9)$$

where $m = 2$ and $n = 6$. The smoothstep functions can be incorporated into the softcore potentials for the vDW and electrostatic interactions by simply modifying $r_{ij}^{VDW}(\lambda; \alpha)$ and $r_{ij}^{Elec}(\lambda; \alpha)$

$$\begin{aligned} r_{ij}^{VDW}(\lambda; \alpha) &= [r_{ij}^n + \alpha S_p(\lambda) \sigma_{ij}^n]^{1/n} \\ r_{ij}^{Elec}(\lambda; \beta) &= [r_{ij}^m + \beta S_p(\lambda)]^{1/m} \end{aligned} \quad (26.10)$$

and into the mixing scheme: with the weights of the two TI region to be complimentary summed to 1

$$V(\lambda) = (1 - S_p(\lambda))V_0 + S_p(\lambda)V_1 \quad (26.11)$$

or with symmetric weights

$$V(\lambda) = (1 - S_p(\lambda))V_0 + (1 - S_p(1 - \lambda))V_1 \quad (26.12)$$

Furthermore, the boundaries of smoothstep functions can be defined to be any range within [0,1]. For example, one can utilize a smoothstep function with boundaries at [0.2, 0.5], which effectively will start the mixing at $\lambda = 0.2$ and finish at $\lambda = 0.5$. This “ λ -scheduling” can be applied to individual interactions and gives the users a very flexible way to control the mixing of the softcore potentials.

The smoothstep function incorporation is controlled by the following extra input control parameters:

`gti_lam_sch` Flag for lambda-scheduling

- = 1** lambda-scheduling is enabled, i.e., the alchemical parameter λ is replaced by $S_p(\lambda)$ (26.11 and 26.12) Note: when `gti_lam_sch=1`, the default `scalpha` is set to 0.2 and the default `scbeta` is $50 \text{ \AA}^2$.
- = 0** lambda-scheduling is disabled (default)

Lambda-scheduling control file format: When lambda-scheduling is enabled, the scheduling control will be read from the control file named by the command line option “`-lambda_sch filename`” or “`lambda.sch`” (the default name). If the file does not exist, the default scheduling behavior will be utilized. Each line in the scheduling control file should be in the following format

LambdaType, FunctionType, Matchtype, parameter1, parameter2

LambdaType The interaction type where the lambda-scheduling is applied. Valid values: “TypeGen” (for all general usage), “TypeBAT” (for the bonded terms), “TypeRestBA” (for the restraint bond/angle terms), “TypeEleRec” (for the reciprocal space terms), “TypeEleCC” (for direct space common atom terms), “TypeEleSC” (for the direct space SC atom terms), and “TypeVDW” (for the vdw terms)

FunctionType The smoothstep function to be used. Valid values: “linear”, “smooth_step0” (the same as linear), “smooth_step1”, “smooth_step2”, “smooth_step3”, and “smooth_step4”

Matchtype The mixing “matching” style, either “complementary”(26.11) or “symmetric” (26.12).

parameter1,parameter2 Real numbers: the λ range where the lambda-scheduling is applied. Must be in [0,1].

For example, an entry of “TypeVDW, smooth_step2, complementary, 0.5, 1.0” means the smoothstep function S_2 will be used for the vDW interactions, starting at $\lambda = 0.5$ and ending at $\lambda = 1.0$. When `gti_lam_sch=1` but the control file is missing, the following default will be utilized:

```
TypeGen, linear, complementary, 0.0, 1.0
TypeBAT, linear, symmetric, 0.0, 1.0
TypeEleRec, linear, symmetric, 0.0, 1.0
TypeEleCC, smooth_step2, symmetric, 0.0, 1.0
TypeEleSC, smooth_step2, symmetric, 0.0, 1.0
TypeVDW, smooth_step2, symmetric, 0.0, 1.0
```

Control of the softcore potentials:

`gti_ele_sc` Flag for the electrostatic softcore potentials

- = 0** smoothstep function is not utilized (default when `gti_lam_sch=0`).

- = **1** smoothstep function is utilized (26.10) according to the TypeEleSC type defined in the lambda-scheduling control file (default when gti_lam_sch=1).

gti_vdw_sc Flag for the vDW softcore potentials

- = **0** smoothstep function is not utilized (default when gti_lam_sch=0).
- = **1** smoothstep function is utilized (26.10) according to the TypeVDW type defined in the lambda-scheduling control file (default when gti_lam_sch=1).

Alternative forms of softcore potentials: The electrostatic part of 26.10 can be re-expressed so that these two are in identical form[599, 600]:

$$r_{ij}^{\text{VDW}}(\lambda; \alpha^{\text{VDW}}) = [r_{ij}^n + \alpha^{\text{VDW}} S_P(\lambda) \sigma_{ij}^n]^{1/n}$$

$$r_{ij}^{\text{Elec}}(\lambda; \alpha^{\text{Coul}}) = [r_{ij}^m + \alpha^{\text{Elec}} S_P(\lambda) \sigma_{ij}^m]^{1/m} \quad (26.13)$$

The new form of softcore potential can be enabled by the following input control keywords:

gti_scale_beta Flag to enable the new form of softcore potential:

- = **0** default. The new form in 26.13 is disabled.
- = **1** The new form in 26.13 is enabled and the default value of scalpha is set to 0.5 and scbeta to 1.0.

gti_vdw_exp The n value in 26.13. Default :6 ; accepted values: 2,4,6.

gti_ele_exp The m value in 26.13. Default :2 ; accepted values: 2,4,6.

scbeta: The value of scbeta is the unit-less constant α^{Elec} in 26.13 when gti_scale_beta=1; scbeta is β in 26.10 when gti_scale_beta=0.

Furthermore, since the application of softcore potentials could cause discontinuity at the cutoff boundary, 26.13 is modified:

$$r_{ij}^{\text{VDW}}(\lambda; \alpha^{\text{VDW}}) = [r_{ij}^n + W(r_{ij}) \alpha^{\text{VDW}} S_P(\lambda) \sigma_{ij}^n]^{1/n}$$

$$r_{ij}^{\text{Elec}}(\lambda; \alpha^{\text{Coul}}) = [r_{ij}^m + W(r_{ij}) \alpha^{\text{Elec}} S_P(\lambda) \sigma_{ij}^m]^{1/m} \quad (26.14)$$

where $W(r_{ij})$ is a switching function with value of 1 when $r_{ij} < \text{gti_cut_sc_on}$ and 0 when $r_{ij} > \text{gti_cut_sc_off}$, and smoothly changing from 1 to 0 in between through the second order smooth-step function. The utilization of this switching function is enabled with gti_cut_sc=2. The default value of gti_cut_sc_off is set to “cut” and gti_cut_sc_on is set to “cut”-2.

gti_cut_sc Flag to enable tail smoothing to softcore potential:

- = **0** default. no tail smoothing to SC, i.e., $W(r_{ij})$ is set to 1 in 26.14.
- = **1** add smoothing to SC-vDW, beginning at gti_cut_sc_on and ending at gti_cut_sc_off; using the second order smooth-step function.
- = **2** in addition to 1, add smoothing to SC-elec, at gti_cut_sc_on and ending at gti_cut_sc_off.

26.1.7.2. Treatment of the interactions between the common and softcore regions and within softcore regions

Regarding the treatment of the interactions between the common and softcore regions, no much attention has been put on the previous versions of AMBER, including sander, pmemd, and pmemd.cuda. While most of time such ignorance will not cause significant deviations of the calculated free energy differences, it should be treated in a more theoretically rigorous ways when applicable.

			gti_add_sc switch						
			0	1	2	3	4	5	6
1	vdw	SC/CC	S						
2	ele	SC/CC	S						
3	1-4 vdw	SC/CC	P	S	S	S	S	S	S
4	1-4 ele	SC/CC	P	S	S	S	S	S	S
5	ele	SC internal	P	P	S	S	P	S	S
6	1-4 ele	SC internal	P	P	S	S	S	S	S
7	vdw	SC internal	P	P	P	S	P	P	S
8	1-4 vdw	SC internal	P	P	P	S	S	S	S
9	torsion	SC/CC	P	P	P	P	S	S	S
10	torsion	SC internal	P	P	P	P	S	S	S

Table 26.2.: Summary of the effect of the `gti_add_sc` switch: (SC: Softcore (Dummy) region, CC: common core part, S: Scaled with lambda: not present in the dummy state, P: Not scaled with lambda: present in the dummy state)

The non-bonded terms: The non-bonded terms between the common region and the softcore regions should be always scaled with the alchemical variable λ . Nevertheless, the 1-4 non-bonded terms were not treated properly in the previous versions of AMBER, A fix has been implemented in AMBER20. The non-bonded terms within the softcore regions can be treated in either ways, provided that the conformational sampling of the softcore regions at the end point states are properly done. The following input control has been added:

`gti_add_sc` Flag to control the non-bonded interactions between the common and softcore regions, and within the softcore regions.

- = 0 the behavior of the versions prior to AMBER20. Note that this option is only for back-compatible and would produce theoretically incorrect results hence should be avoid[601].
- = 1 default. the 1-4 non-bonded terms between the common and softcore regions are scaled with the alchemical variable λ .
- = 2-6 The dependence of λ for various interactions are listed in 26.2. Note that an enhanced sampling through alchemical dimension can be realized through utilizing `gti_add_sc` >=5.[599, 602]

The behavior is summarized in Table 26.2

The bonded terms: It has been proved that [603–605] that only certain ways to handle the bonded terms across the common region and the softcore regions are theoretically correct. Briefly, there should be only one bond length term involving one softcore atom, one angle term involving one softcore atom, and one torsion term involving two softcore atoms can be not scaled with the alchemical variable λ . These terms, however, seem not having significant effect in most cases. The following input control has been added:

`gti_bat_sc` Flag to control the bonded interactions between the common and softcore regions, and within the softcore regions.

- = 0 the behavior of the versions prior to AMBER20 (default)
- = 1 the program will automatically decide the terms to be scaled with the alchemical variable λ .
- = 2 the user can decide the terms not to be scaled with the alchemical variable λ , others will be scaled. (see below)

When `gti_bat_sc`=2, The selection is done by the following masks. All must be in the AMBER standard mask language. The program will automatic determine the terms involving in the selected atoms.

`sc_bond_mask1`: The mask to select the cross common-softcore bond length terms of the softcore region 1.

gti_bat_sc	Bonded terms at the SC/CC boundary		
0	Any terms involving any dummy atom(s)		All P
	bond	R-D	Only one P; all others S
		R-R-D	Only one P; all others S
	angle	R-D-D	All P
1		R-D-R	All S
		D-R-D	All S
		R-R-D-D	Only one P; all others S
	torsion	R-D-D-D	All P
2		R-R-R-D	All S
	bond	sc_bond_mask1,2	Select atoms to be P, all others S
	angle	sc_angle_mask1,2	Select atoms to be P, all others S
	torsion	sc_torsion_mask1,2	Select atoms to be P, all others S

Table 26.3.: Summary of the effect of the `gti_bat_sc` switch: (SC: Softcore (Dummy) region, CC: common core part, S: Scaled with lambda: not present in the dummy state, P: Not scaled with lambda: present in the dummy state). Here lists only the boundary terms, all internal SC bonded terms are P.

`sc_bond_mask2`: The mask to select the cross common-softcore bond length terms of the softcore region 2.

`sc_angle_mask1`: The mask to select the cross common-softcore angle terms of the softcore region 1.

`sc_angle_mask2`: The mask to select the cross common-softcore angle terms of the softcore region 2.

`sc_torsion_mask1`: The mask to select the cross common-softcore torsion terms of the softcore region 1.

`sc_torsion_mask2`: The mask to select the cross common-softcore torsion terms of the softcore region 2.

The behavior is summarized in Table 26.3

26.1.7.3. Extra input controls (pmemd.cuda only)

The `tishake` flag has different implementation in `pmemd.cuda`:

`tishake` Flag that determines how SHAKE is handled:

- = 0 (default) Coordinates are synchronized after SHAKE, no constraints removed.
- = 1 SHAKE is removed between bonds containing one common and softcore atom(s). Note that disabling SHAKE requires a 1 fs timestep.
- = 2 SHAKE is kept in bonds containing one common and softcore atom(s). This experimental option maintains the physical bond length as specified by the forcefield across all lambda windows in a relative binding free energy calculation. This is especially important for transformations such as N-H to C-H, where the N and C are mapped, but the hydrogens are not. It is suggested to keep `gti_syn_mass = 1` for this method.

Some input controls have been implemented to more flexibly control the TI simulations:

`gti_syn_mass`: Flag to control the synchronization of common atoms in both TI regions:

- = 0 (default) The synchronization of masses and coordinates are based on the linear combination of the two end states.
- = 1 The synchronization of masses and coordinates are direct copy from V0 to V1 (the CPU version default behavior).
- = 2 The synchronization of masses and coordinates are direct copy from V1 to V0.

- = 3** This is the default when the program detects `tishake=0` and there are SHAKE bonds containing one common and softcore atom(s) with the same behavior as 0 with additional synchronization of SHAKE bonds performed: For bonds containing one common and softcore atom(s), the SHAKE-bond length at a lambda windows is defined as the linear combination of the corresponding SHAKE-bond lengths from the two end states. If a bond is a SHAKen in only one end state, the SHAKE-bond length will be copied to the corresponding non-SHAKen bond.

`ti_vdw_mask`: Mask selection to zero out vDW interactions. (In the standard AMBER mask language, similar to `crgmask`)

`gti_output`: **= 0** default;

- = 1** output the term-by-term detailed TI results.

`gti_cpu_output`: **= 1** default: The option ensures matching with the CPU-version output: The softcore λ -derivative terms from two TI regions will be combined and shown in both regions. The non-scaled contributions from softcore regions are not included in the system total potential energies. The output results of non-SC part for each TI region will be exactly the same.

- = 0** The softcore λ -derivative terms will be output separately for each TI region. The non-scaled contributions from softcore regions are included in the system total potential energies. Hence output results of non-SC part for each TI region could be different. This option is required when running REAF type enhanced sampling (Section 26.3.9).

`gti_cut`: **= 1** default: the internal softcore non-bonded terms will not be cut and the non-bond cutoff will not have effect on them.

- = 0** the old behavior of the versions prior to AMBER20, where the internal softcore non-bonded terms will be cut according to the non-bond cutoff.

Since the non-bonded terms within the softcore regions could be scaled differently with the alchemical variable λ . The default non-bond cutoff should be applied to them. One should always use `gti_cut=1` unless a comparison with the results from a previous version is desired.

`gti_chg_keep`: **= 1** default: the charges of the softcore region atoms will not be neutralized.

- = 0** the old behavior of the versions prior to AMBER20: the charges of the softcore region atoms will be neutralized if the net charge is smaller than 0.01.

The following two flags are experimental and only for testing purposes hence users should not use them:

`gti_ele_gauss`

`gti_auto_alpha`

26.1.8. Preparing TI simulations for use in PMEMD

Since the generation of the *prmtop* file required for *pmemd* TI calculations is slightly more complex, than the generation of two independent *prmtop* files as required by *sander*, so we provide here a number of examples specific to *pmemd*.

26.1.8.1. Free Energy using linear scaling

For this type of simulation, the molecule is perturbed between the start and end states using linear scaling (Eq. 26.3). This means that V_0 and V_1 must have the same number of atoms. Start by parameterizing the molecule as usual. This may include the addition of dummy atoms as needed. Then, create a *pdb* which contains both molecules separated by a TER card. Also, update the residue number for the second molecule. If the molecules are different, be sure to use a different residue name for each one. The coordinates for corresponding atoms in the *pdb*

should be the same. The *prmtop* can then be prepared as usual, using *LEaP*. Note that *LEaP* sees both molecules, so it may report a net charge in the *prmtop*, even though there is no net charge for V_0 or V_1 , even after the addition of neutralizing counterions. See Chapter 14 for a complete description of *LEaP*. The input flags for this system are:

```
icfe = 1, timask1 = ':1', timask2 = ':2'
```

Where the first molecule is unique to V_0 and the second molecule is unique to V_1 . There may be any number of other molecules, which are treated as common atoms and are part of both V_0 and V_1 .

26.1.8.2. Absolute free energy using soft core

For this type of simulation, a molecule is decoupled from the rest of the system using soft core potentials (Eqs. 26.5,26.6). Set up the *prmtop* as you would to run a simulation of the system. The end state is the system with a fully decoupled molecule, so this *prmtop* will also work for TI. The input flags for this system are:

```
icfe = 1, ifsc = 1,
timask1=':1', scmask1=':1',
timask2='', scmask2='',
```

Where the first molecule is the one that is decoupled from the rest of the system at the end state.

26.1.8.3. Relative free energy using soft core

For this type of simulation, a molecule is mutated from one to another using soft core potentials (Eqs. 26.5,26.6). This can be done as a single step using soft core electrostatics (Eq. 26.7), or part of a multistep TI calculation. The *prmtop* is prepared in the same way for both cases. First, parameterize both molecules as usual. Then, create a *pdb* containing both molecules, separated by a TER card. Additional molecules may be present and will be treated as common atoms. Using this *pdb*, prepare the system using *LEaP*. The resulting *prmtop* can be used for TI calculations. The input flags for this system are:

```
icfe = 1, ifsc = 1,
timask1=':1', scmask1=':1',
timask2=':2', scmask2=':2',
```

Where the first molecule corresponds to the starting state, and the second molecule corresponds to the ending state. This will set up a single step transformation using soft core electrostatics. To set up a soft core vdW transformation, the flag *crgmask*=':1 |:2' can be added.

26.1.8.4. Mutation of a protein residue

For this type of simulation, a single residue is mutated in a protein. First, take the *pdb* for the wildtype and the mutant proteins and concatenate the one after the other, separating them by a TER card. This is necessary because *LEaP* must deal with full molecules. The atoms in the common residues should all have the same coordinates. Any changes to the common residues, such as the addition of disulfide bonds or changing the protonation of HIS, must be done for both copies of the protein in *LEaP*. The output *prmtop* and *inpcrd* files now have two copies of the protein, with one including the mutated residue. Consider a system where residues ':1-5' represent the wildtype protein and residues ':6-10' represent the mutant protein. Furthermore, assume that residue ':3' in the wildtype is mutated, so the corresponding residue in the mutant is residue ':8'. The input flags for this system are:

```
icfe = 1, ifsc = 1,
timask1=':1-5', scmask1=':3',
timask2=':6-10', scmask2=':8',
```

This will do a single step transformation from the wildtype to the mutant protein.

There are a large number of redundant bonding terms that are being calculated, since there are two proteins in the prmtop file. These additional bonding terms can be eliminated, improving the efficiency of the calculation. This is an advanced technique, which is not needed to run a TI simulation, but to have the most efficient calculation. In order to do this, a command has been added to *parmed* to remove these extra terms and atoms as described in Section 15.2.

To run *parmed*:

```
parmed -p ti.prmtop -i merge.in
```

The input for *parmed* (merge.in) looks like this:

```
loadRestrt ti.inpcrd
setOverwrite True
tiMerge :1-5 :6-10 :3 :8
outparm ti.merged.prmtop ti.merged.inpcrd
quit
```

This will output ti.merged.prmtop and ti.merged.inpcrd which have had redundant bonding terms removed, as well as the masks that should be used in the simulation. The *parmed* output gives:

```
Loaded Amber topology file ti.prmtop
Reading actions from merge.in
Loading restart file ti.inpcrd
Prmtop is overwritable
Merging molecules :1-5 and :6-10 into the same molecule.
Use softcore mask:
timask1='@41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59',
timask2='@77,78,79,80,81,82,83,84,85,86,87,88,89,90',
scmask1='@41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59',
scmask2='@77,78,79,80,81,82,83,84,85,86,87,88,89,90',
Outputting Amber topology file ti_merged.prmtop
Done!
```

Now the input flags for *pmemd* are:

```
icfe = 1, ifsc = 1,
timask1='@41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59',
timask2='@77,78,79,80,81,82,83,84,85,86,87,88,89,90',
scmask1='@41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59',
scmask2='@77,78,79,80,81,82,83,84,85,86,87,88,89,90',
```

Another possible use is to remove redundant bonding terms in a non soft core simulation. The set up is very similar to that described above, except that the number of atoms in both molecules must be the same. When using *parmed*, ignore the scmask1/scmask2 output, as these are not used in non soft core simulations.

26.1.8.5. gas phase calculations using GB

alchemical free energy calculations (TI/FEP/MBAR) may now be run in gas phase in the *pmemd* module. This is accomplished by using the *igb=6* (no implicit solvent) option for a generalized born simulation. The only other unique input is *cut = 9999.0*, because there is particle mesh ewald to account for long range contributions for atoms farther apart than *cut*. An example input file for changing the lennard-jones of 2-methylfuran (resname 2MF) to methane in the gas phase using TI is shown below:

```
&cntrl
  imin = 0, nstlim = 500000, irest = 0, ntx = 1, dt = 0.001,
  ntt = 3, temp0 = 298.0, gamma_ln = 2.0, ig = -1,
  ntb = 0, cut = 9999.0, igb = 6,
```

```

ioutfm = 1, iwrap = 0,
ntwe = 10000, ntwx = 10000, ntp = 1000, ntwr = 500000, ntave = 500000,
ntc = 2, ntf = 1, tishake = 1,
noshakemask = ':1,2',
icfe = 1, ifsc = 1, clambda = 0.5, scalpha = 0.5, scbeta = 12.0,
timask1 = ':1', timask2 = ':2',
scmask1 = ':2MF@O3,C4,C5,C6,H10,H11,H12',      scmask2 = ''
/

```

26.1.9. Collecting potential energy differences for FEP calculations

In addition to the Thermodynamic Integration capabilities described above, *sander* can also collect potential energy values during free energy simulation runs for postprocessing by e.g. the Bennett acceptance ratio scheme. This will make *sander* calculate at given points during the simulation the total potential energy of the system as it would be for different λ -values at this conformation. This functionality is controlled by:

ifmbar If set to 1 (Default = 0), additional output is generated for later postprocessing.

mbar_states number of lambda windows considered.

mbar_lambda lambda windows simulated.

For example, if you want to run *mbar* with 15 lambda windows at 0.00, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.85, 0.90, 0.95, 1.00, you would use the following options:

```

ifmbar = 1,
mbar_states = 15,
mbar_lambda = 0.00, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.85, 0.90, 0.95,

```

The options below have been deprecated. They are here for anyone using AMBER 16 or older, but will not work in AMBER 18.

bar_intervall Compute potential energies every *bar_intervall* steps (Default = 100)

bar_l_min Minimum λ -value (Default = 0.1)

bar_l_max Maximum λ -value (Default = 0.9)

bar_l_incr The increment to increase λ by between the minimum and maximum (Default = 0.1)

Such energy collection will normally be part of a regular free energy calculation (using *icfe*=1 and *ifsc*=1) involving simulations at various λ -values. Activating this functionality will not have any influence on the simulation trajectory which will evolve according to the preset *clambda* value, it is merely a bookkeeping scheme that removes the necessity of postprocessing output files later.

26.2. Linear Interaction Energies

sander contains rudimentary facilities to compute binding free energies using the linear interaction energy model.

ilrt if set to 1, turns on the computation of LIE contributions (default=0)

lrt_interval Computer LIE contributions every *lrt_interval* MD steps (default=50)

lrtmask The 'solute'. Interaction Energies between the atoms in *lrtmask* and the remainder of the system are computed.

The LIE facilities work by computing the system energies several times using different charge and vdW-parameter sets. This results in reduced performance if `lrt_interval` is set to less than approx. 10. The LIE output at the end of the `mdout` file gives the electrostatic interaction energy between the solute and rest of the system *times 0.5*, i.e. in accordance with the original formulation of LIE theory. The solute SASA and vdW-interaction energy with its surroundings is calculated unscaled.

26.3. Replica Exchange Molecular Dynamics (REMD)

Replica exchange molecular dynamics (REMD) is an *expanded ensemble* method—it samples from an ensemble (significantly) larger than a typical statistical mechanical ensemble defined by the Hamiltonian governing the system (e.g., microcanonical, canonical, grand-canonical, etc.). This section will briefly describe the general theory of REMD-based techniques, after which the later subsections will cover the details of Amber’s REMD implementation as well as the various allowable exchange types.

26.3.1. Introduction

‘Sampling’ in expanded ensemble techniques can be broadly decomposed into two different types of sub-sampling within the total ensemble. The first type is the common conformational sampling that can be realized through methods like molecular dynamics. The second type samples from thermodynamic state-space, in which the core Hamiltonian (together with its thermodynamic constraints) that defines the ensemble is allowed to change. Thus, expanded ensemble techniques broaden the sampling space of simulations by allowing a system to move through both conformational space (which is typically continuous) and state space (which is typically discrete). A point in the phase space of the expanded ensemble is defined by specific value(s) of the state parameter(s) in addition to the $3N$ particle positions and conjugate momenta. To simplify terminology, I will refer to all of the points in phase space that have the same value of the state parameter(s) as a ‘sub-ensemble’ since it can be interpreted as the statistical ensemble defined by that thermodynamic state, whereas ‘the ensemble’ will refer to the full, expanded ensemble containing all state indices.

To ensure that the ensemble is constructed properly, the simulation must generate a reversible Markov chain of states. Standard MD obeys this requirement under the (ubiquitous) assumption that the system is ergodic. For Monte Carlo-based methods, a reversible Markov chain implies that the condition of detailed balance (Eq. 26.15) is satisfied. Detailed balance is effectively an equilibrium condition, in which the probability of being in state i (π_i) multiplied by the transition probability of moving from state i to j ($P_{i \rightarrow j}$) is equal to the probability of being in state j multiplied by the transition probability of moving from state j to i . By relating probabilities with concentrations and transition probabilities with chemical rate constants, it is easy to see that Eq. 26.15 is a simple equilibrium equation between two species.

$$\pi_i P_{i \rightarrow j} = \pi_j P_{j \rightarrow i} \quad (26.15)$$

To sample from the ensemble, REMD employs a set of non-interacting replicas (i.e., the forces on the particles are unaffected by the particles in other replicas) that attempt to swap their positions in state space during the course of the simulation. In Amber, the conformational sampling in each replica (i.e., sub-ensemble) is performed with MD while replica swaps in state space are performed using Metropolis Monte Carlo in which the exchange probability is computed from Eq. 26.15.

The general workflow used by Amber for replica exchange simulations is illustrated in Figure 26.1. Each replica runs a pre-specified number of MD steps before stopping to attempt exchanges with one of its nearest neighbor (its exchange partner alternates every other exchange attempt). Restricting exchange attempts to pairs is not required (exchanges can involve 3 replicas, for instance), but it greatly simplifies the resulting exchange probability when solving Eq. 26.15, and allows $N/2$ consecutive exchanges to be attempted independently. Furthermore, replica exchange attempts need not be made deterministically or synchronously (i.e., each replica evaluates exchange attempts at the same time relative to each other), but doing so significantly simplifies the programming requirements. The following sections will describe how REMD is implemented and performed in Amber for each of the supported types of exchanges—temperature, solution pH, solution Redox Potential, and generalized Hamiltonian.

If you are not already familiar with the technique and its theoretical underpinnings, we recommend that you study the literature, particularly of the type of replica exchange you plan on using.[606–612]

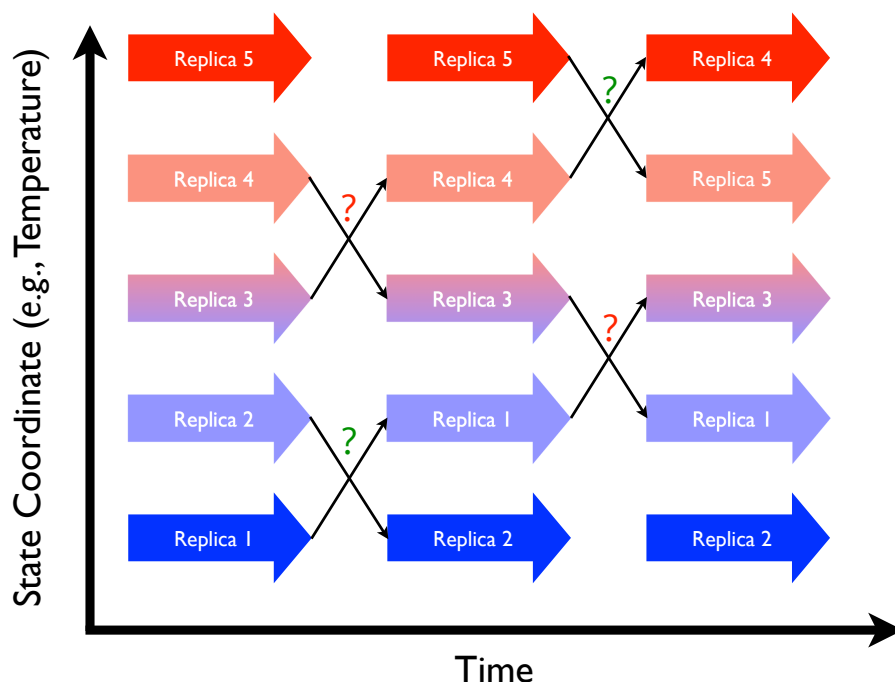


Figure 26.1.: *Replica exchange schematic showing 5 replicas combined in an expanded ensemble. Large arrows represent MD trajectories of sub-ensembles while the smaller arrows represent attempted swaps between replicas in state space. Question marks represent Monte Carlo exchange attempts (green for successful, red for failed).*

26.3.2. Running REMD simulations

In order to run REMD simulations, *sander* and *pmemd* use the *multisander* (and *multiplmemd*) machinery that allows multiple MD trajectories to be run in the same simulation. This mode of *sander* and *pmemd* is used slightly differently than their normal operation, and is described in Section 22.12.

There are two new variables that must be present in the `&cntrl` section of the `mdin` files of each replica for all REMD simulations.

numexchg This is the number of exchange attempts that will be performed between replica pairs

nstlim This is the number of MD steps that will be performed between exchange attempts

gremd_acyc (H-REMD only) When set to 1, the exchange between the first and the last replicas will not be considered and odd numbers of replicas are allowed. Default: 0.

There are also additional command-line flags that should be placed on the command-line (with the groupfile), described below:¹

-rem <#> This flag defines the type of replica exchange that will be run for 1-D REMD. The allowed values are 1 (T-REMD), 3 (H-REMD), 4 (pH-REMD), and 5 (E-REMD). Each approach is described in later sections.

¹Some specialized types of REMD simulations have additional command-line flags that will be described in the later sections.

-remlog <remlog_file> This flag specifies the name of the log file that contains information about each of the replicas during each exchange attempt. The default value is `rem.log`.

-remtype <remtype_file> This flag specifies a filename for the remtype file; this file provides helpful information about the current replica run. For reservoir REMD runs it also prints reservoir information. Default is `rem.type`

-remrandompartner <#> This flag specifies if a random partner will be picked at each replica exchange attempt instead of picking one of the neighbors. This feature is desired for some H-REMD simulations. Any value other than 0 will trigger the random selection, and a value of 0 will keep selecting only neighbors. Default: 0

Some specialized types of REMD simulations have additional command-line flags that will be described in future sections.

Note the change in the meaning of `nstlim`. For standard MD, `nstlim` is the total number of MD steps that will be performed. For REMD simulations, on the other hand, `nstlim` is the number of steps between replica exchange attempts and the total number of steps is equal to $nstlim \times numexchg$. The `nstlim` variable, then, is related to the inverse of the exchange attempt frequency (EAF) in REMD simulations. The value of `numexchg` *must* be the same for each input file, or the program will hang indefinitely as those replicas assigned more exchange attempts wait to exchange data with replicas that have already finished. We also strongly suggest keeping `nstlim` the same as well to avoid making replicas with fewer steps wait for those with more steps to finish.

REMD simulations can be run in any thermodynamic ensemble (NVE, NVT, and NPT as of AmberTools 19/Amber 20). When running replica exchange simulations with explicit solvent in the NVE or NVT ensemble, all replicas must have the same volume. Therefore, the equilibration stage of each replica should begin *after* the original system was run at constant pressure to stabilize the density (and volume). There is no such restriction for NPT REMD simulations.

Amber currently supports 5 types of exchange attempts: Temperature REMD (T-REMD), Hamiltonian REMD (H-REMD), constant pH REMD (pH-REMD), constant Redox Potential REMD (E-REMD), and replica exchange self-guided Langevin dynamics (RXSGLD). Multi-dimensional REMD simulations (defined by 2 or more state parameters) are also supported. The instructions given above apply to all REMD simulations, but instructions for running REMD simulations in general strongly depend on the type of exchange being attempted. Additional details for running REMD simulations are provided in the following sections for each type of exchange attempt.

26.3.3. Generating REMD input files with `genremdinputs.py`

Preparing all the necessary REMD input files, which are the `mdin` files and `groupfile`, and also the `remd-file` if doing Multidimensional REMD (see Subsection 26.3.10.1), can be time consuming and in some cases confusing. The `genremdinputs.py`, a Python tool written by Vinícius Cruzeiro, helps to make this task much easier [613]. This tool can be used to prepare the input files for *any* REMD simulation, one-dimensional or Multidimensional. You can access a list and description of all available command-line flags using the `--help` flag, whose output is shown below.

```
usage: genremdinputs.py [Options]
optional arguments:
  -h, --help            show this help message and exit
  -v, --version          show the program's version and exit
  --author              show the program's author name and exit
  -O, --overwrite       Allow existing outputs to be overwritten. Default:
                        False
Required Arguments:
  -inputs [FILE [FILE ...]]
                        Input files containing pH, Redox Potential,
                        Temperature, or Hamiltonian values. Each file must
                        state the type of exchange on the first line (same as
                        in the exch.type flag of the remd-file for M-REMD
                        simulations), a description in the second line, and
```

all variable values on the following lines (one value per line). As the number of replicas on each REMD dimension needs to be even, the number of values needs to be even.

-groupfile [FILE [FILE ...]]

Reference groupfiles. Each reference groupfile must contain only a single block referring to a single replica. In this block the replica number must be replaced by REPNUM (the program will replace this flag later in order to create the whole groupfile file). If doing a REMD simulation with the Hamiltonian dimension, the prmtop file name may be replaced by the same flag entered in the first line of the hamiltonian file given in the -inputs flag. The reference groupfiles must be given in the same order as their corresponding reference mdin files.

-i [FILE [FILE ...]]

Reference mdin files. Each reference mdin file must contain the variable(s) being exchanged replaced by the same flag entered in the first line of the file given in the -inputs flag. Examples: solvph=PH, solve=REDOX, temp0=TEMPERATURE. Also, each reference mdin file must contain ig=RANDOMNUM. The reference mdin files must be given in the same order as their corresponding reference groupfiles.

Non-required Arguments:

-remd-file-out FILE

Name of the -remd-file output file. Default: remd.dim.[REMD dimensions types]remd

-randomseed INTEGER

Seed for the random number generator. Default: 10

-nosort

If stated, the replica ordering per dimension will not be sorted. If not stated, sorting will be done if the input values are float or integer.

-verbose, --verbose

If stated, prints information on the screen while the program is executed.

This program generates the input files for any REMD simulations (including MultiD-REMD). It generates: the groupfile, mdin files, and the -remd-file.

genremdinputs.py requires one reference mdin file, one reference groupfile containing only a single block referring to one replica, and one simple input file for each REMD dimension. We now go over a few examples. The first example is more detailed and should be used as a reference before moving to the other examples.

26.3.3.1. Generating input files for T-REMD

Here is an example of an input file, that we call here `temperatures.dat`, to be passed to `-inputs`:

```
TEMPERATURE
Temperature Replica Exchange
260.0
280.0
300.0
320.0
```

The first line in this file is an identifier for `genremdinputs.py` and should not be changed (as of Amber18, the allowed options are: TEMPERATURE, HAMILTONIAN, PH and REDOX). The second line is a description line and you may enter any description you want. The following lines correspond to the temperatures to be used for each replica, therefore in this example our T-REMD simulation would have 4 replicas. Please remember that Replica Exchange simulations require an even number of replicas. Adding more replicas is simple and only requires one to add more temperature values inside `temperatures.dat`.

The reference mdin file consist on the mdin file for a single replica with some simple adaptations. Here is an example of a mdin.ref file:

```
&cntrl
  imin=0, irest=1, ntwx=10000, ntp=10000,
  ntwx=10000, nstlim=10, numexch=500000,
  dt=0.002, ntt=3, temp0=TEMPERATURE,
  gamma_ln=1.0, ig=RANDOMNUM,
  ntc=2, ntf=2, cut=8, iwrap=1, ioutfm=1,
  saltcon=0.1,
/
```

The value of temperature in temp0 has to be replaced by the exact same string that goes into the first line of temperatures.dat, in this case TEMPERATURE. In Replica Exchange simulations the random seed ig should be different for the different replicas, and by replacing the value of ig by RANDOMNUM the genremdinputs.py tool will automatically place random numbers for this seed in each replica. Placing ig=RANDOMNUM is always required by genremdinputs.py. The random seed used by genremdinputs.py to generate these random numbers can be changed by the flag -randomseed. In this example, as the reference mdin file ends with the suffix .ref the following mdin files would be generated: mdin.rep.001, mdin.rep.002, mdin.rep.003, and mdin.rep.004. The suffix .ref in the reference mdin file name is optional.

Here is an example of a reference groupfile which we call groupfile.ref:

```
# Replica REPNUM
-O -i mdin.rep.REPNUM -p prmtop -c min.x -o mdout.rep.REPNUM -r rst7.rep.REPNUM
```

You must replace the replica number by the flag REPNUM every time it appears. In this example, as the reference groupfile ends with the suffix .ref, this suffix will be removed and the output groupfile would be groupfile. The suffix .ref in the reference groupfile name is optional; if it is not provided the output file name will have the same name as the reference group file plus a suffix .final.

Here is an example of the execution of the program:

```
genremdinputs.py -inputs temperatures.dat -groupfile groupfile.ref -i mdin.ref -O
```

The options -O overwrites any existing files with the same name as any output files. The reference mdin file and groupfile should not be used in your REMD simulations, only the output files generated by genremdinputs.py. Please notice that genremdinputs.py will also generate the remd-file, however you don't need to provide this file in your simulation (unless you want to execute your one-dimensional REMD simulation using Amber's multidimensional REMD module, which should give equivalent results).

26.3.3.2. Generating input files for pH-REMD

The procedure here is very similar to what has been shown for T-REMD in Subsection 26.3.3.1. Here is an example of a phs.dat input file to be passed to -inputs:

```
PH
pH Replica Exchange
2.0
2.5
3.0
3.5
```

In your reference mdin file, in addition to ig=RANDOMNUM, the only other modification needed is solvph=PH.

26.3.3.3. Generating input files for E-REMD

The procedure here is very similar to what has been shown for T-REMD in Subsection 26.3.3.1. Here is an example of a `redoxes.dat` input file to be passed to `-inputs`:

```
REDOX
Redox Potential Replica Exchange
0.75
0.78
0.81
0.85
```

In your reference `mdin` file, in addition to `ig=RANDOMNUM`, the only other modification that would have to be done is `solve=REDOX`.

26.3.3.4. Generating input files for pH,T-REMD

In this Multidimensional REMD example, let's consider the `phs.dat` file from Subsection 26.3.3.2 and the `temperatures.dat` from Subsection 26.3.3.1. The `mdin.ref` file would look like this:

```
&cntrl
  imin=0, irest=1, ntwx=10000, ntp=10000,
  ntwx=10000, nstlim=200, numexch=2500,
  dt=0.002, ntt=3, temp0=TEMPERATURE,
  gamma_ln=1.0, ig=RANDOMNUM,
  ntc=2, ntf=2, cut=8, iwrap=1, ioutfm=1,
  icnstph=2, solvph=PH, ntcnstph=100,
  saltcon=0.1,
/
```

The example reference groupfile shown in Subsection 26.3.3.1 would also work here. The command would then be:

```
genremdinputs.py -inputs phs.dat temperatures.dat -groupfile groupfile.ref -i mdin.ref
```

The `remd-file` generated will now have to be considered for your Multidimensional REMD simulation (see Subsection 26.3.10.1 for more details about `remd-file`). The order of the dimensions exchanging will be the same as the order of the input files provided to the flag `-inputs`. For example, in order to generate input files for T,pH-REMD the command would be:

```
genremdinputs.py -inputs temperatures.dat phs.dat -groupfile groupfile.ref -i mdin.ref
```

26.3.3.5. Generating input files for H-REMD

Apart from the random seed in the `mdin` files, in Hamiltonian REMD the replicas may differ by either their topology files or a parameter inside their `mdin` files. The file `hamiltonians1.dat` below shows an example for the situation in which the topology files are different for each replica:

```
HAMILTONIAN
Hamiltonian Replica Exchange with different topologies
prmtop.1
prmtop.2
prmtop.3
prmtop.4
```

26. Free energies

In this case, your reference mdin file should still contain `ig=RANDOMNUM`, however the HAMILTONIAN pointer should not be in your reference mdin file but in your reference groupfile, as the example file `groupfile1.ref` shows below:

```
# Replica REPNUM
-O -i mdin.rep.REPNUM -p HAMILTONIAN -c min.x -o mdout.rep.REPNUM -r rst7.rep.REPNUM
```

Let's now considering the situation in which the different replicas differ by a parameter inside their mdin files (a parameter other than `ig`). An example of this situation is shown in the file `hiltonians2.dat` below:

```
HAMILTONIAN
Hamiltonian Replica Exchange with different mdins
0.0000
0.3333
0.6667
1.0000
```

Similarly to what has been done for `temp0` in T-REMD (see Subsection 26.3.3.1) or for `solvph` in pH-REMD (see Subsection 26.3.3.2), you need to place the pointer HAMILTONIAN in the flag inside the reference mdin file corresponding to the values listed in the file `hiltonians2.dat`. A reference groupfile like the `groupfile.ref` file shown in Subsection 26.3.3.1 could be used together with this input file.

26.3.3.6. Generating input files for H,H-REMD

`genremdinputs.py` supports generating files for Multidimensional REMD simulations that contain two or more different Hamiltonian dimensions. In this H,H-REMD example, we will adapt the files `hiltonians1.dat` and `hiltonians2.dat` from the Subsection 26.3.3.5 to be respectively our first and second Hamiltonian dimensions. The changes required in these files are quite simple: the first line in `hiltonians1.dat` need to be changed to HAMILTONIAN1 and the first line in `hiltonians2.dat` need to be changed to HAMILTONIAN2. The reference groupfile is `groupfile.hhremd.ref`:

```
# Replica REPNUM
-O -i mdin.rep.REPNUM -p HAMILTONIAN1 -c min.x -o mdout.rep.REPNUM -r rst7.rep.REPNUM
```

You need to place the pointer HAMILTONIAN2 in the flag inside the reference mdin file corresponding to the values listed in the file `hiltonians2.dat`. The `genremdinputs.py` tool can then be executed in the following way:

```
genremdinputs.py -inputs hiltonians1.dat hiltonians2.dat -groupfile \
groupfile.hhremd.ref -i mdin.ref -O
```

If none of Hamiltonian dimensions in the H,H-REMD simulation involves changing the topology file, then the pointers HAMILTONIAN1 and HAMILTONIAN2 should be both inside the reference mdin file.

26.3.4. Running Temperature REMD simulations

In temperature REMD (T-REMD), replicas are distinguished based on the temperature of their temperature bath. In general, each replica should differ from each other *only* by their target temperature, `temp0`, specified in the mdin file for each replica. The N replicas are first sorted in an array by their target temperatures, so the ordering of the replicas in the groupfile is irrelevant. Neighboring residues attempt to exchange every `nstlim` MD steps, with the exchange partners alternating each replica exchange attempt. For example, if replicas 2 and 3 attempt to swap the first time then replicas 1 and 2 will attempt to swap the next time (as will replicas 3 and 4). Topologically, the N temperature-sorted replicas form a loop, in which the first and the last replicas are neighbors. Therefore, $N/2$ exchanges are attempted every `nstlim` steps. The exchange success rate is computed via a Metropolis Monte Carlo move shown in Eq. 26.16 that satisfies detailed balance for swapping temperatures. If the exchange is

allowed between the pair, the temperature between the replicas is swapped before MD resumes. The velocities of each replica involved in successful exchange are then adjusted by the scaling factor $\sqrt{T_{new}/T_{old}}$ where T_{old} is the temperature before the exchange and T_{new} is the temperature after. This velocity scaling is done to ensure that each structure is immediately adjusted to its new target temperature. After the exchange calculation, the MD resumes for `nstlim` steps until the next exchange attempt (in which the exchange partners alternated with respect to the previous exchange attempt).

$$P_{i,j} = \min \{1, \exp [-(\beta_i - \beta_j) (E_j - E_i)]\} \quad (26.16)$$

Before starting a replica exchange simulation, an optimal set of temperatures should be determined so that the exchange ratio is roughly a constant. This spacing of the replicas in temperature-space determines the probability of exchange among the replicas, and the user is referred to the literature for a more complete description of the influence of various factors on the exchange probability. A useful resource for generating a series of temperatures with a specific exchange success probability has been developed by Patriksson and van der Spoel[614] and can be found online at <http://folding.bmc.uu.se/remd>.

Each replica requires (for input files) or generates (for output files) its own *mdin*, *inpcrd*, *mdout*, *mdcrd*, *restrt*, *mdinfo*, and associated files. The names are provided through the specification of a *groupfile* on the command line with the *-groupfile groupfile* option. The *groupfile* file contains a separate command line for each of the replicas or multisander instances, as described in Section 22.12. To choose the number of replicas or multisander instances, the *-ng N* command line option is used (in this case to specify *N* separate instances.)

For example, an 4-replica REMD job will need 4 *mdin* and 4 *inpcrd* files. Then, the groupfile might look like this:

```
#
# multisander or replica exchange group file
#
-O -i mdin.rep1 -o mdout.rep1 -c inpcrd.rep1 -r restrt.rep1 -x mdcrd.rep1
-O -i mdin.rep2 -o mdout.rep2 -c inpcrd.rep2 -r restrt.rep2 -x mdcrd.rep2
-O -i mdin.rep3 -o mdout.rep3 -c inpcrd.rep3 -r restrt.rep3 -x mdcrd.rep3
-O -i mdin.rep4 -o mdout.rep4 -c inpcrd.rep4 -r restrt.rep4 -x mdcrd.rep4
```

Note that for T-REMD the *mdin* and *inpcrd* files are *not* required to be ordered by their target temperatures since they will not remain sorted during the simulation. Sorting is performed automatically at each REMD iteration as described above. Thus one can restart REMD simulations without modifying the restart files from the previous REMD run (see below for more information about restarting REMD).

It is important when running T-REMD to ensure that each topology file is equivalent and the input files differ only in the temperature (`temp0`), and that all explicit solvent calculations are run at constant volume. Because Eq. 26.16 was derived under the assumption that exchanging replicas only swapped temperatures, only the temperature can vary between replicas. Satisfying this requirement is left to the user, and no warnings or checks are implemented if this assumption is violated.

26.3.4.1. Restarting REMD simulations

It is recommended that each REMD run generate a new set of output files (such as *mdcrd*), but for convenience one may use *-A* in the command line in order to append output to existing output files. This can be a useful option when restarting REMD simulations. If *-A* is used, files that were present before starting the REMD simulation are appended to throughout the new simulation. If *-O* is used, any files present are overwritten. The recommended input file settings for restarting a REMD simulation are *ig=-1* to use the wall clock for the pseudo-random number generator seed, *ntxo=2* to write a NetCDF restart file, *ioutfm=1* to write NetCDF trajectories, *irest=1* to restart, and *ntx=5* to read velocities; the first three should be used in the initial calculation.

At the end of a REMD simulation, the target temperature of each replica is most likely not the same as it was at the start of the simulation (due to successful exchanges). If one wishes to continue this simulation, *sander* or *pmemd* will need to know how the target temperature has changed. Since the target value is normally specified in the *mdin* file (via `temp0`), the previous *mdin* files would all need to be modified to reflect changes in target

temperature of each replica. In order to simplify this process, *sander* and *pmemd* write the final target temperature as additional information in the restart files during a T-REMD simulation. When a T-REMD simulation is started, the program will check to see if the target temperature is present in the restart file. If it is present, this value will override the value in the mdin file. In this manner, one can restart the simulation from the set of restart files and *sander* or *pmemd* will automatically update the target temperature of each replica to correspond to the final value from the previous run. If the target temperature is not present (as would be the case for the first REMD run), the correct values must be present in the mdin files.

26.3.4.2. Content of the output files

It is important to note that in the current implementation of T-REMD *all output is by replica only, not by temperature!* To facilitate post processing of trajectory data by temperature, the temperature must be specified for each snapshot. For NetCDF trajectories, adding this information is simple because NetCDF is an extensible format. We strongly recommend that you always use NetCDF trajectories, especially for REMD simulations. For ASCII formatted trajectories, a header line is written to each frame just before the coordinates. This header line has the format:

```
REMD <replica#> <exchange#> <step#> <Temperature>
```

PTRAJ and CPPTRAJ are able to read trajectories with this format.

The rem.log file for T-REMD simulations has the following format:

```
# Replica Exchange log file
# numexchg is          5
# REMD filenames:
#   remlog= rem.log
#   remtype= rem.type
# Rep#, Velocity Scaling, T, Eptot, Temp0, NewTemp0, Success rate (i,i+1), ResStruct#
# exchange             1
1      1.15      0.00    -10.46    300.00    400.00      0.00     -1
2      1.04      0.00    -10.46    325.00    350.00      2.00     -1
3      0.96      0.00    -10.46    350.00    325.00      0.00     -1
4      0.87      0.00    -10.46    400.00    300.00      2.00     -1
# exchange             2
1      0.94    312.03     -6.81    400.00    350.00      1.00     -1
2      1.07    280.77     -3.95    350.00    400.00      1.00     -1
3     -1.00    247.11    -10.58    325.00    325.00      1.00     -1
4     -1.00    271.12    -14.15    300.00    300.00      0.00     -1
# exchange             3
1      0.96    305.31    -11.02    350.00    325.00      0.67     -1
2      0.87    288.89    -12.45    400.00    300.00      1.33     -1
3      1.04    290.99    -13.30    325.00    350.00      1.33     -1
4      1.15    256.19    -12.83    300.00    400.00      0.00     -1
```

The columns, listed in order, are the replica number, velocity scaling factor, the instantaneous temperature, the potential energy of the structure, the target temperature before the exchange attempt, the target temperature after the exchange attempt, the average success rate, and the reservoir structure number. The replica number never changes since replicas swap target temperatures. When the velocity scaling factor is -1, the exchange attempt failed and velocities are not altered. For successful exchange attempts, velocities are either scaled up (when the new target temperature is higher than the old one) or down (when the new target temperature is lower than the old one). Success rates are calculated as $\#successes/\#tries \times 2$, where the factor of 2 is used because each pair of neighboring replicas attempts to exchange every other exchange attempt. In the beginning of the log file, this may lead to unusual success rates, but after a large number of exchange attempts the values will normalize. Success rates are computed as the exchange success rate between the original temperature (Temp0 column) and the next highest temperature in the temperature ladder (not necessarily the temperature it just attempted to exchange with). The success rate for the highest temperature is often 0 since it reflects the success rate between the highest and lowest temperatures.

All temperatures are reported in Kelvin and all energies in kcal/mol.

26.3.4.3. Cautions when using replica exchange

While many variations of replica exchange have been tested with sander, all possible variations have not been tested and the option is intended for use by advanced researchers that already have a comprehensive understanding of standard molecular dynamics simulations. Caution should be used when creating REMD input files. Amber will check for the most obvious errors but due to the nature of the multiple output files the reason for the error may not be readily apparent. The following is only a subset of things that users should keep in mind:

1. The number of replicas must be an even number (so that all replicas have a partner for exchange), except the case when running H-REMD with `gremd.acyc` set to 1.
2. Temp0 values for each replica must be unique for Temperature-based REMD.
3. REMD-related namelist variables (`numexchg`, `nstlim`) should be identical in the `mdin` files.
4. Temp0 values should not be changed in the `nmropt=1` weight change section.
5. A groupfile is required.
6. If high temperatures are used, it may be necessary to use a smaller time step and possibly restraints to prevent cis/trans isomerization or chirality inversion.
7. Due to increased diffusion rates at high temperature, it may be good to use `iwrap=1` to prevent coordinates from becoming too large to fit in the restart format. An alternative to this is to use the default NetCDF restart files (`ntxo=2`) which are far less likely to overflow.
8. Note that the optimal temperature range and spacing will depend on the system. The user is strongly recommended to read the literature in this area.
9. As of AmberTools 19/Amber 20, constant pressure (NPT) REMD is supported.
10. `pmemd.MPI` requires at least 2 threads per replica, whereas `sander.MPI` will work with just 1.

26.3.4.4. Replica exchange example

Below is an example of an 8-replica REMD run on 16 processors, (note that launching a MPI program varies from computer to computer).

```
mpirun -np 16 sander.MPI -ng 8 -groupfile groupfile -rem 1
```

Here is the groupfile:

```
#
# multisander or replica exchange group file
#
-o -i mdin.rep1 -o mdout.rep1 -c inpcrd.rep1 -r restrt.rep1 -x mdcrd.rep1
-o -i mdin.rep2 -o mdout.rep2 -c inpcrd.rep2 -r restrt.rep2 -x mdcrd.rep2
-o -i mdin.rep3 -o mdout.rep3 -c inpcrd.rep3 -r restrt.rep3 -x mdcrd.rep3
-o -i mdin.rep4 -o mdout.rep4 -c inpcrd.rep4 -r restrt.rep4 -x mdcrd.rep4
-o -i mdin.rep5 -o mdout.rep5 -c inpcrd.rep5 -r restrt.rep5 -x mdcrd.rep5
-o -i mdin.rep6 -o mdout.rep6 -c inpcrd.rep6 -r restrt.rep6 -x mdcrd.rep6
-o -i mdin.rep7 -o mdout.rep7 -c inpcrd.rep7 -r restrt.rep7 -x mdcrd.rep7
-o -i mdin.rep8 -o mdout.rep8 -c inpcrd.rep8 -r restrt.rep8 -x mdcrd.rep8
```

This input specifies that T-REMD should be used (`-rem 1`), with 8 replicas (`-ng 8`) and 2 processors per replica (`-np 16`). Note that the total number of processors should always be a multiple of the number of replicas.

26.3.4.5. Replica exchange using a hybrid solvent model

This section describes an advanced feature of Amber.[205, 206] Users that are not already comfortable with standard replica exchange simulations should likely get more experience with them before attempting hybrid solvent REMD calculations.

For large systems, REMD becomes intractable since the number of replicas needed to span a given temperature range increases roughly with the square root of the number of degrees of freedom in the system. Recognizing that the main difficulty in applying REMD with explicit solvent lies in the number of simulations required, rather than just the complexity of each simulation, we recently developed a new approach in which each replica is simulated in explicit solvent using standard methods such as periodic boundary conditions and inclusion of long-range electrostatic interactions using PME. However, the calculation of exchange probabilities (which determines the temperature spacing and thus the number of replicas) is handled differently. Only a subset of closest water molecules is retained, with the remainder temporarily replaced by a continuum representation. The energy is calculated using the hybrid model, and the exchange probability is determined. The original solvent coordinates are then restored and the simulation proceeds as a continuous trajectory with fully explicit solvation. This way the perceived system size for evaluation of exchange probability is dramatically reduced and fewer replicas are needed.

An important difference from existing hybrid solvent models is that the system is fully solvated throughout the entire MD simulation, and thus the distribution functions and solvent properties should not be affected by the use of the hybrid model in the exchange calculation. In addition, no restraints of any type are needed for the solvent, and the solute shape and volume may change since the solvation shells are generated for each replica on the fly at every exchange calculation. Nearly no computational overhead is involved since the calculation is performed infrequently as compared to the normal force evaluations. Thus the hybrid REMD approach can employ more accurate continuum models that are too computationally demanding for use in each time step of a standard molecular dynamics simulation. However, since the Hamiltonian used for the exchange differs from that employed during dynamics, these simulations are approximate and are not guaranteed to provide correct canonical ensembles.

At each exchange calculation sander will create the hybrid system based on the current coordinates for the fully solvated system. This is done by calculating the distance of each water oxygen to the nearest solute atom, and sorting the water by increasing shortest distance. The closest *numwatkeep* are retained and the potential energy is calculated using the GB model specified by *hybridgb*. After the energy calculation the fully solvated system is restored.

For a more complete example, users are directed to the hybridREMD test case (in the *rem_hybrid* subdirectory) in the Amber test directory.

numwatkeep The number of explicit waters that should be retained for the calculation of potential energy to be used for the exchange calculation. Before each exchange attempt, the closest *numwatkeep* waters will be retained (closest to the solute) and the rest will be temporarily removed and then replaced after the exchange probability has been calculated. The default value is -1, indicating that all waters should be retained (standard REMD). A value of 0 would direct Amber to remove all of the explicit water (as in MM-PBSA) while a nonzero value will result in some water close to the solute being retained while the rest is removed. Currently it is not possible to select a subset of solute atoms for determining which waters are "close". Determining the optimal *numwatkeep* value is a topic of current research.

hybridgb Specifies which GB model should be used for calculating the PE of the stripped coordinates, equivalent to the *igb* variable. Currently *hybridgb* values of 1, 2, 5, 7 and 8 are supported.

Cautions: Hybrid-REMD has not been extensively tested. The following would not be expected to work without further modification of the code:

1. Only the water is imaged for the creation of the stripped system. Care should be taken with dimers (such as DNA duplexes) to ensure that the imaging is correct.
2. Explicit counterions should probably not be used.
3. The choice of implicit solvent model will likely have a large effect on the resulting ensemble.

26.3.4.6. Reservoir REMD

The ability to perform REMD with a structure reservoir [615, 616] has been implemented in Amber as of version 10. Although REMD can significantly increase the efficiency of conformational sampling, obtaining converged data can still be challenging. This is particularly true for larger systems, as the number of replicas needed to span a given temperature range increases with the square root of the number of degrees of freedom in the system. Another consideration is that the folding rate of a peptide tends not to be as dependent on temperature as the unfolding rate, making the search for native peptide structures in higher temperature replicas more problematic; in the case where a native-like structure is found it will almost always be exchanged to a lower temperature replica, requiring a repeat of the search process. In addition, the exchange criterion in REMD assumes a Boltzmann-weighted ensemble of structures, which is typically not the case at the start of a REMD simulation. Although the exchange criterion will eventually drive each replica toward a Boltzmann-weighted ensemble of structures, this essentially means that until all of the replicas are converged, none of the replicas are converged.

Reservoir REMD is a method which can significantly enhance the rate of convergence and reduce the high computational expense of standard REMD simulations. An ensemble of structures (or reservoir) is generated at high temperature, then linked to lower temperatures via REMD. Periodic exchanges are attempted between randomly chosen structures in the reservoir and the highest temperature replica. If the structure reservoir is already Boltzmann-weighted,[615] convergence is significantly enhanced as the lower temperature replicas simply act to re-weight the reservoir ensemble - in essence all of the searching has been accomplished from the start. This is in contrast to standard REMD where all the replicas are run simultaneously, and the computational expense for running long simulations must be paid for each of the replicas even though only a few high-temperature ones may be contributing to sampling of new basins.

One major advantage of this approach is that a converged ensemble of conformations needs to be generated only once and only for one temperature. Typically this temperature should be high enough to facilitate crossing of energy barriers, but low enough that there is still a measurable fraction of native structure present. Another advantage is that exchanges with the reservoir do not need to be time-correlated with the replica simulations; folding events sampled during reservoir generation can provide multiple native structures for the other replicas.

It may not always be possible however to generate a Boltzmann-weighted ensemble of structures (e.g. for a large molecule in explicit solvent). In such cases it is possible to use a non-Boltzmann weighted reservoir by modifying only the exchange criterion between the reservoir and the highest temperature replica. If the weight of all structures in the reservoir is set to 1, this corresponds to a completely flat distribution across the free energy landscape. Alternatively, weights can be assigned to structures based on various structural properties. In the current implementation, weights are assigned to structures via dihedral bin clustering, wherein clusters are identified by unique configurations of user-defined dihedral angles.

There are several new command line arguments that pertain to Reservoir REMD:

-rremd Type of reservoir to use.

= 0 No reservoir (Default)

= 1 Boltzmann-weighted reservoir

= 2 Non-Boltzmann weighted reservoir where the weight of each structure in the reservoir is assumed to be $1/N$

= 3 Non-Boltzmann weighted reservoir with weights defined by dihedral angle binning.

-reservoir Specifies the file name prefix for reservoir structures. Reservoir structure files should be in the restart file format *MDRESTR*, and are expected to be named according to the format <name>.XXXXXX, where XXXXXX is a 6 digit integer, e.g. frame.000001. Default is "reserv/frame". **IMPORTANT NOTE:** Structure numbering should begin at 1. Reservoirs can be created using the *cpptraj* command **createreservoir**.

-saveene specifies the file containing energies of the structures in the reservoir (default filename is "saveene"). This file must contain a header line with format:

```
<# reservoir structures> <reservoir T> <#atoms>
<random seed> <velocity flag>
```

If the velocity flag =1 then velocity information will be read from the reservoir structure files, otherwise (if velocity flag =0) velocities will be assigned to the structure based on the reservoir temperature. After the header line there should be a line containing the potential energy of each reservoir structure. **IMPORTANT NOTE:** For reservoir REMD with dihedral bin clustering (rremd==3) each potential energy should be followed by the cluster # that reservoir structure belongs to.

-clusterinfo For reservoir REMD with dihedral bin clustering (rremd==3) this file specifies what dihedrals are used and the binsize, as well as what cluster each reservoir structure belongs to. Default is "cluster.info". File has the following format:

```
<# Dihedral Angles>
<atom# 1> <atom# 2> <atom# 3> <atom# 4> [Dihedral 1]
. .
. .
. .
<atom# 1> <atom# 2> <atom# 3> <atom# 4> [# Dihedral Angles]
<Total # Clusters>
<Cluster #> <Weight>
<Bin1><Bin2>...<Bin #Dihedral Angles> [Cluster 1]
. .
. .
. .
<Cluster #> <Weight>
<Bin1><Bin2>...<Bin #Dihedral Angles> [# Clusters]
```

The first line is the number of dihedral angles that will be binned, following the definition of those dihedral angles (4 atoms using sander atom #s, starting from 1) and the bin size for each dihedral angle. Next is the total # of clusters followed by lines providing information about each cluster: the cluster number, weight and ID as defined by dihedral binning. The ID is composed of consecutive 3 digit integers, 1 for each dihedral angle. For example, a structure belonging to cluster 7 with a weight of 2 with 2 dihedral angles that fall in bins 3 and 8 would look like:

```
7 2 003008
```

26.3.5. Hamiltonian replica exchange

Instead of spacing replicas throughout temperature space, you can also space replicas throughout “Hamiltonian space.” That is, every replica has a different Hamiltonian, or energy function, and exchange attempts occur between adjacent Hamiltonians. With *sander* and *pmemd*, Hamiltonian replica exchange is implemented by exchanging coordinates between replicas and evaluating the energy of that new structure. The corresponding detailed balance equation that is used to compute the exchange probability is shown in Eq. 26.17. This option is enabled by using *-rem 3* on the command-lines in the groupfile.

$$P_{i \rightarrow j} = \min\{1, \exp(-\beta_1 [H_1(x_2) - H_1(x_1)] - \beta_2 [H_2(x_1) - H_2(x_2)])\} \quad (26.17)$$

Here, state *i* refers to the replica combination $[\beta_1 H_1(x_1), \beta_2 H_2(x_2)]$ and state *j* refers to the replica combination $[\beta_1 H_1(x_2), \beta_2 H_2(x_1)]$. Eq. 26.17 assumes that only coordinates are traded between exchanging replicas, but allows for the temperatures to differ. The temperature does not exchange upon a successful attempt, but velocities are swapped following successful exchange attempts and scaled by $\sqrt{T_{new}/T_{old}}$ to match the target temperature of their new replica.

26.3.5.1. Free Energy Perturbation

Upon closer inspection of Eq. 26.17, we can see a close resemblance to Free Energy Perturbation[609, 617]

$$\Delta G_{a \rightarrow b} = -k_B T \ln [\exp(-\beta(E_b - E_a))] \quad (26.18)$$

We can see that for every exchange attempt, the required ΔE is calculated in both directions. The value for the free energy (in both directions) is accumulated and reported in the `rem.log` file each time an exchange is attempted.

For replica exchange free energy perturbation (REFEP), multiple topology files are often needed that correspond to a value of an alchemical parameter, λ , similar to thermodynamic integration. The ParmEd program included with AmberTools can be used to generate the intermediate topology files by scaling charges and/or van der Waals parameters. In this case, because coordinates are exchanged, each replica tracks a particular Hamiltonian and set of control variables, rather than a sequence of configurations. Note this is the opposite behavior of T-REMD in which replicas change temperatures but keep the same sequence of configurations.

26.3.5.2. Umbrella Sampling

Hamiltonian exchange can be used to perform replica exchange umbrella sampling [618] using the NMR flat well restraints. In this case, every line of the group file needs a different restraint file in which the center of the biasing umbrella changes. Each replica tracks a particular umbrella location rather than a replica trajectory. Note this is the opposite behavior of T-REMD in which replicas change temperatures but keeps the same replica trajectory.

26.3.5.3. Steps for running H-REMD simulations

Note: before running Hamiltonian replica exchange (H-REMD), you should be familiar with Temperature replica exchange (T-REMD) simulations. H-REMD simulations are set up similarly to T-REMD simulations. Each replica is specified on a line of a groupfile and is run with *multisander*. Each replica differs either by simulation control parameters in the input file (e.g., for umbrella sampling replica exchange [618] or REXAMD [619, 620]) or parameters in the topology file (e.g., REFEP).

- The majority of H-REMD settings are similar to T-REMD. A groupfile is needed. The number of replicas must be an even number (so that all replicas have a partner for exchange). Constant pressure is not supported for REMD simulations. This means `ntp` must be 0.
- Depending on the type of H-REMD, all replicas may have different force fields/control variables (if the differences are too large, the exchange probability may suffer)
- The order of the replicas in the groupfile is very important. As a general rule in all H-REMD simulations, the least different Hamiltonians (replicas) should be neighbors. Because this method is relatively new, there are very limited discussions in the literature about the optimum positions of replicas in the Hamiltonian ladder [621, 622]. Exchange neighbors are defined by adjacent lines in the groupfile (i.e., each replica exchanges ‘right’ or ‘up’ with the replica defined by the line above and exchanges ‘left’ or ‘down’ with the replica defined by the line below in the groupfile).
- For editing the prmtop, e.g., in the case of REFEP, there is a python script in AmberTools, *parmed*, which facilitates the modifications of Amber topology files. See Section 15.2 for details.
- In H-REMD, each replica has a different Hamiltonian. In contrast to T-REMD, neighbor replicas exchange their conformations, which means each replica keeps its initial Hamiltonian and there is no need for post-processing (i.e., using *ptraj* or *cpptraj*) to extract sub-ensembles. However, you will have to post-process in order to reconstruct replica-based time series.

To enable H-REMD the `-rem` flag on the command-line must be given the value 3. H-REMD simulations require the same input files as T-REMD simulations and generates the same output files. The output printed in the `remlog` file differs significantly from that found in the `remlog` file for T-REMD, however. Example `remlog` output for H-REMD is shown below:

```
# Replica Exchange log file
# numexchg is 10000
# REMD filenames:
#   remlog= remlog
#   remtype= rem.type
```

26. Free energies

```
# Rep#, Neibr#, Temp0, PotE(x.1), PotE(x.2), left_fe, right_fe, Success, Success rate (i,i+1)
# exchange 1
1 8 300.00 -12783.23 -12755.40 -16.39 0.00 F 0.00
2 3 300.00 -12839.84 -12802.56 0.00 -0.05 T 2.00
3 2 300.00 -12802.60 -12839.79 -0.04 0.00 T 0.00
4 5 300.00 -12847.41 -12858.37 0.00 -0.78 F 0.00
5 4 300.00 -12858.19 -12846.63 0.18 0.00 F 0.00
6 7 300.00 -12859.65 -12833.42 0.00 0.30 T 2.00
7 6 300.00 -12833.63 -12859.95 -0.21 0.00 T 0.00
8 1 300.00 -12771.63 -12766.84 0.00 -16.23 F 0.00
# exchange 2
1 2 300.00 -12825.03 -13147.73 0.00 -0.62 F 0.00
2 1 300.00 -13148.20 -12824.42 -0.47 0.00 F 1.00
3 4 300.00 -13136.97 -12823.77 0.00 0.62 T 1.00
4 3 300.00 -12823.32 -13137.59 0.44 0.00 T 0.00
5 6 300.00 -12919.25 -13181.18 0.00 -0.41 T 1.00
6 5 300.00 -13180.48 -12918.84 0.70 0.00 T 1.00
7 8 300.00 -13162.39 -12775.37 0.00 0.16 T 1.00
8 7 300.00 -12775.59 -13162.55 -0.22 0.00 T 0.00
...
```

The columns, in order, are the replica number, the exchange partner for this attempt, the target temperature, the potential energy of the current structure, the potential energy of the proposed structure, the free energy difference calculated via Eq. 26.18 for all exchanges to the ‘left’ (or ‘up’ in the Hamiltonian ladder), all free energies for exchanges to the ‘right’ (or ‘down’ in the Hamiltonian ladder), whether the exchange attempt succeeded (T) or not (F), and the average success rate. For each step, the only free energy values printed are those between replicas that attempted to exchange. All free energies between non-exchanging pairs are set to 0 for that step. Therefore, the ‘final’ free energies can be found by summing the respective terms from the last two exchanges in the *remlog* file. All energies have units of kcal/mol, and temperatures have units of Kelvin.

26.3.5.4. An example

When running H-REMD, the format of the groupfile is very similar to that in T-REMD, but specific details depend on the type of simulation being performed (see Subsection 26.3.3 for information about how you can use the `genremdinputs.py` tool to prepare your input files). In the case of REFEP, the *groupfile* may look like the following:

```
-O -i mdin -p prmtop.0 -c inpcrd.0 -suffix 000
-O -i mdin -p prmtop.1 -c inpcrd.1 -suffix 001
-O -i mdin -p prmtop.2 -c inpcrd.2 -suffix 002
-O -i mdin -p prmtop.3 -c inpcrd.3 -suffix 003
-O -i mdin -p prmtop.4 -c inpcrd.4 -suffix 004
-O -i mdin -p prmtop.5 -c inpcrd.5 -suffix 005
-O -i mdin -p prmtop.6 -c inpcrd.6 -suffix 006
-O -i mdin -p prmtop.7 -c inpcrd.7 -suffix 007
```

Notice how the topology file differs in each case, but the input file remains the same. An example *groupfile* for umbrella sampling may look like the following:

```
-O -i mdin.0 -p prmtop -c inpcrd.0 -suffix 000
-O -i mdin.1 -p prmtop -c inpcrd.1 -suffix 001
-O -i mdin.2 -p prmtop -c inpcrd.2 -suffix 002
-O -i mdin.3 -p prmtop -c inpcrd.3 -suffix 003
-O -i mdin.4 -p prmtop -c inpcrd.4 -suffix 004
-O -i mdin.5 -p prmtop -c inpcrd.5 -suffix 005
-O -i mdin.6 -p prmtop -c inpcrd.6 -suffix 006
-O -i mdin.7 -p prmtop -c inpcrd.7 -suffix 007
```

Notice in this case how the topology file is the same but the input file differs in each case (which is where the center of the umbrella is defined). Like T-REMD, *sander.MPI* (or *pmemd.MPI*) are executed via the following command:

```
mpirun -np 16 sander.MPI -ng 8 -groupfile groupfile -rem 3
```

Note that the particular method for launching an MPI program may depend on your MPI implementation. Also, *pmemd* requires at least 2 threads per replica, whereas *sander* will work with just 1.

26.3.6. RXSGLD: Replica exchange using Self-Guided Langevin Dynamics

RXSGLD utilizes the guiding force, *sgft* or *sgff*, to define replicas. SGLD simulations are performed for replicas[550]. Please refer to Section 25.1 about how to set up SGLD simulations. When temperature is the same for all replicas, the replica exchange ratios are high and so is the conformational search efficiency. RXSGLD is an alternative to SGMD/SGLD, to achieve efficient conformational search while being able to obtain the canonical ensemble distribution. RXSGLD is turned on if *isgld* is set to >0 in the *sander* or *pmemd* input file when performing replica exchange simulations (i.e., *rem* > 0).

For the convenience of reference, we define replicas as non-interacting identical simulation systems and define stages as simulation conditions between which replicas transit. In T-REMD, stages are different by temperatures, while in RXSGLD, stages are different by the strength of guiding forces, as defined by *sgft* and/or *sgff*. For example, we can set *sgft*=0, 0.5, 0.8, and 1.0 for stages 1 to 4, respectively, while *temp0*=300K for all stages. In RXSGLD, temperatures in different stages can be the same or different from each other; however, it is preferred to keep all temperatures the same to achieve high replica exchange efficiency.

Because the temperatures of different stages may be the same, the stages are given a ID from 1 to Nrep. Like T-REMD, each RXSGLD trajectory file is for each replica. Unlike T-REMD, the frames of RXSGLD trajectories are preceded by the following information:

```
RXSGLD <replica#> <exchange#> <step#> <stage ID>
```

The output from RXSGLD contains the following lines:

```
TEMP0= <temp0> SGFT= <sgft> SGFF= <sgff> STAGE= <stag ID> REPNUM= <rep#> EXCHAN
```

RXSGLD trajectories can be processed with PTRAJ analogously to T-REMD trajectories: merely replace temperatures with stage IDs. For example, to extract the trajectory on stage 1, we can use the following command:

```
ptraj rxsgld.top <<EOF
trajin rxsgld.trj.000 rxsgldtraj rxsgldid 1
trajout rxsgld.trj.stag 1
EOF
```

26.3.7. pH-REMD

In constant pH REMD, replicas attempt to exchange their solution pH values in much the same way as temperatures are exchanged in T-REMD. The idea of swapping pH in the discrete protonation state implementation (described in Section 27) was proposed by [Itoh et al. 610] and later implemented and evaluated in Amber. [611] The implementation here works very similarly to T-REMD, except each replica is given a different value of *solvpH* in the *mdin* file instead of *temp0*. The exchange probability, shown in Eq. 26.19, is derived under the assumption that all replicas have the same temperature.

$$P_{i \rightarrow j} = \min \left\{ 1, \exp \left[\ln 10 \left(N_i^{H^+} - N_j^{H^+} \right) (pH_i - pH_j) \right] \right\} \quad (26.19)$$

Where $N_i^{H^+}$ is the number of titratable protons currently ‘active’ in state i .

Before running pH-REMD simulations, you should first be familiar with running constant pH MD described in Section 27, since it will help you set up each replica. Aside from the changes required to run at constant pH, setting up pH-REMD simulations is quite similar to setting up T-REMD simulations. Each replica should have the same topology file, and all mdin files should be identical except for the value of `solvph` and the random seed `ig`. Furthermore, each replica should be titrating the same residues (this is very important). For instance, you cannot turn ‘off’ carboxylate titrations at basic pH if your pH-REMD spans both acidic and basic conditions.

26.3.7.1. Analyzing Output

The output from pH-REMD simulations is analyzed in the same way as standard constant pH simulations, with some preprocessing required. Because the pH of each replica changes upon successful replica exchange attempts, each replica trajectory contains members from ensembles at all pHs. Therefore, you must use `ptraj` or `cpptraj` to extract ensembles at each pH. To simplify the coding required, the pH is stored as the ‘temperature’ in each trajectory, so the T-REMD machinery should be used in `ptraj/cpptraj` to extract desired ensembles. Please refer to Subsection 37.10.4 and make use of the `remdtrajtemp` command replacing temperature by pH values.

The `cpout` files have additional information added to them to indicate which protonation states belong to which target pH ensembles. This is done by printing the pH next to each record, as shown below.

```
Solvent pH:  3.00000
Monte Carlo step size:      5
Time step:      5
Time:      10.008
Residue    0 State:  3 pH:   3.000

Residue    0 State:  3 pH:   3.000

Residue    0 State:  3 pH:   3.500

Residue    0 State:  3 pH:   3.500

Residue    0 State:  3 pH:   2.000

Residue    0 State:  3 pH:   2.000

Residue    0 State:  3 pH:   2.500
```

You can see that the pH is changing between snapshots. In addition to generating pH-based trajectories, you also must generate pH-based `cpout` information that is stored in each `cpout` file. The replica pH is identified on each line of the `cpout` file to aid in constructing the pH-specific protonation state ensembles. To aid with this, the `cphstats` program, described in Subsection 27.7.5, has an option to provide a “prefix” that defines the new file names for the pH-specific `cpout` files that it builds from a list of *each cpout file from a single pH-REMD simulation*. For example:

```
cphstats --fix-remd 1AKI.cpout 1AKI.cpout.000 1AKI.cpout.001 \
                  1AKI.cpout.002 1AKI.cpout.003
```

Assuming you ran replicas at pH 2.0, 2.5, 3.0, and 3.5, this will generate files `1AKI.cpout.pH.2.00`, `1AKI.cpout.pH.2.50`, `1AKI.cpout.pH.3.00`, and `1AKI.cpout.pH.3.50`, with their respective ensembles. If you ran, for instance, 20 ns of simulation in chunks of 5 ns (so you ran 4 “chunks” after 3 restarts), you will need to run this command 4 times—once for each simulation segment. You should analyze the resulting protonation state distribution using these newly-generated `cpout` files.

26.3.7.2. A pH-REMD Example

Below is an example in which 4 replicas are run at pH values of 2.0, 2.5, 3.0, and 3.5 (see Subsection 26.3.3 for information about how you can use the `genremdinputs.py` tool to prepare your input files). The command

below shows an example of running this simulation on 8 processors (2 processors for each replica). Note that the command to run MPI programs may vary from computer to computer.

```
mpirun -np 8 sander.MPI -ng 4 -groupfile groupfile -rem 4 -remlog rem.log
```

Please notice that pH-REMD uses the option `-rem 4`. The groupfile in this example is shown below:

```
-O -i phremd.pH2.0.mdin -cpin cpin -p ASPREF.top -c ASPREF.rst7
-O -i phremd.pH2.5.mdin -cpin cpin -p ASPREF.top -c ASPREF.rst7
-O -i phremd.pH3.0.mdin -cpin cpin -p ASPREF.top -c ASPREF.rst7
-O -i phremd.pH3.5.mdin -cpin cpin -p ASPREF.top -c ASPREF.rst7
```

The suffixes “.000”, “.001”, “.002”, and “.003” will be added to the output files from each of the replicas in order to distinguish them from each other by default. This suffix can be changed using the “-suffix” flag for that replica. Any suffix provided this way will be applied to ALL output files, regardless of whether or not they are specified. This is only true for multi-sander and multi-pmemd simulations.

The resulting `rem.log` file looks like the following:

```
# Replica Exchange log file
# numexchg is          50
# REMD filenames:
#   remlog= rem.log
#   remtype= rem.type
# Rep#, N.prot, old.pH, new.pH, Success rate (i,i+1)
# exchange              1
   1          1    2.000    3.500    0.0000
   2          1    2.500    3.000    2.0000
   3          1    3.000    2.500    0.0000
   4          1    3.500    2.000    2.0000
# exchange              2
   1          1    3.500    3.000    1.0000
   2          1    3.000    3.500    1.0000
   3          1    2.500    2.000    1.0000
   4          1    2.000    2.500    1.0000
# exchange              3
   1          1    3.000    2.500    0.6667
   2          1    3.500    2.000    1.3333
   3          1    2.000    3.500    0.6667
   4          1    2.500    3.000    1.3333
# exchange              4
   1          1    2.500    2.000    1.0000
   2          1    2.000    2.500    1.0000
   3          1    3.500    3.000    1.0000
   4          1    3.000    3.500    1.0000
```

The columns are the current replica number (which never changes because pH is swapped between replicas), the total number of protons “active” on all of the titratable sites, the original solution pH, the new solution pH after exchange, and the success ratio (multiplied by 2 to account for swapping neighbors on each exchange attempt).

26.3.8. Redox Potential REMD

In Redox Potential Replica Exchange MD (E-REMD) [612], replicas attempt to exchange their Redox Potential values in a similar way as temperatures are exchanged in T-REMD or pH values are exchanged in pH-REMD. In E-REMD each replica has a different value of Redox Potential, given by `solve` in the mdin file. The exchange probability, shown in Eq. 26.20, is derived under the assumption that all replicas have the same temperature:

$$P_{i \rightarrow j} = \min \left\{ 1, \exp \left[\frac{F}{k_b T} (N_i^{e^-} - N_j^{e^-}) (E_i - E_j) \right] \right\} \quad (26.20)$$

Where F is the Faraday constant, and $N_i^{e^-}$ and E_i are respectively the current number of ‘active’ titratable electrons and the Redox Potential of replica i .

Before running E-REMD simulations, you should first be familiar with running constant Redox Potential MD described in Section 28, since it will help you set up each replica. Each replica should have the same topology file, and all mdin files should be identical except for the value of `solve` and the random seed `ig`, and each replica should be titrating the same residues.

26.3.8.1. Analyzing Output

Similarly to T-REMD and pH-REMD, some preprocessing is required before analysing E-REMD data. As the target Redox Potential of a single replica keeps changing when an exchange attempt is accepted, the trajectory of each replica contains chunks from the ensembles at all Redox Potential values. Therefore, `ptraj` or `cpptraj` must be used to extract the ensemble at a given Redox Potential. The T-REMD functionalities should be used in `ptraj/cpptraj` to extract desired ensembles. Please refer to Subsection 37.10.4 and make use of the `remdtrajtemp` command replacing temperature by Redox Potential values.

The `ceout` files have additional information added to them to indicate which redox states belong to which target Redox Potential ensembles. This is done by printing the Redox Potential next to each record, as shown below.

```
Redox potential:    0.8100000 V Temperature:   300.00 K
Monte Carlo step size:      5
Time step:          5
Time:      1000.008
Residue    0 State:  1 E:    0.8100000 V

Residue    0 State:  1 E:    0.8100000 V

Residue    0 State:  1 E:    0.8400000 V

Residue    0 State:  0 E:    0.8400000 V

Residue    0 State:  1 E:    0.7500000 V

Residue    0 State:  1 E:    0.7500000 V

Residue    0 State:  0 E:    0.7800000 V
```

The `cestats` program can be used to construct Redox Potential-based `ceout` files (see Section 28.6 and Subsection 27.7.5 for more details). One `ceout` file is generated for each target Redox Potential value. This is an example command to generate Redox Potential-based `ceout` files:

```
cestats --fix-remd reordered.ceout ceout.000 ceout.001 \
ceout.002 ceout.003
```

Assuming your E-REMD simulation had the following target Redox Potential values 0.75, 0.78, 0.81 and 0.84 V, this will generate the files `reordered.ceout.E_0.75000`, `reordered.ceout.E_0.78000`, `reordered.ceout.E_0.81000`, and `reordered.ceout.E_0.84000`. If you restarted your E-REMD simulation, you will need to run this command for each simulation segment. Also, you should only analyze the resulting redox state distribution using these newly-generated `ceout` files.

26.3.8.2. A E-REMD Example

Below is an example in which 4 replicas are run at Redox Potential values 0.75, 0.78, 0.81 and 0.84 V (see Subsection 26.3.3 for information about how you can use the `genremdinputs.py` tool to prepare your input files). The command below shows an example of running this simulation on 8 processors (2 processors for each replica). Note that the command to run MPI programs may vary from computer to computer depending on your MPI settings.

```
mpirun -np 8 sander.MPI -ng 4 -groupfile groupfile -rem 5 -remlog rem.log
```

Please notice that E-REMD uses the option `-rem 5`. The groupfile in this example is shown below:

```
-O -i eremd.E0.75.mdin -cein cein -p prmtop -c rst7
-O -i eremd.E0.78.mdin -cein cein -p prmtop -c rst7
-O -i eremd.E0.81.mdin -cein cein -p prmtop -c rst7
-O -i eremd.E0.84.mdin -cein cein -p prmtop -c rst7
```

The suffixes “.000”, “.001”, “.002”, and “.003” will be added to the output files not specified in the groupfile for each replica in order to distinguish them from each other. This suffix can be changed using the “-suffix” flag for that replica. Any suffix provided this way will be applied to ALL output files, regardless of whether or not they are specified. This is only true for multi-sander and multi-pmemd simulations.

The resulting `rem.log` file looks like the following:

```
# Replica Exchange log file
# numexchg is      50
# REMD filenames:
#   remlog= rem.log
#   remtype= rem.type
# Rep#, N_elec, old_E, new_E, Success rate (i,i+1)
# exchange      1
   1      0  0.750  0.840  0.0000
   2      0  0.780  0.810  2.0000
   3      0  0.810  0.780  0.0000
   4      0  0.840  0.750  2.0000
# exchange      2
   1      0  0.840  0.810  1.0000
   2      0  0.810  0.840  1.0000
   3      0  0.780  0.780  1.0000
   4      1  0.750  0.750  0.0000
# exchange      3
   1      0  0.810  0.780  0.6667
   2      0  0.840  0.840  0.6667
   3      0  0.780  0.810  1.3333
   4      1  0.750  0.750  0.0000
# exchange      4
   1      1  0.780  0.750  1.0000
   2      0  0.840  0.810  0.5000
   3      0  0.810  0.840  1.0000
   4      1  0.750  0.780  0.5000
```

The different columns contain information for the different replicas (as only Redox Potential values are swapped between replicas, the replica number always remains the same). Each column also contains the total number of “active” electrons on all of the titratable sites, the original Redox Potential, the new Redox Potential after exchange (if the exchange is rejected, the Redox Potential remains the same), and the success ratio (multiplied by 2 to account for swapping neighbors on each exchange attempt).

26.3.9. Replica Exchange with Arbitrary Degree of Freedom (REAF): a REST2-like enhanced sampling implementation

Here we describe our REST2-like implementation, Replica Exchange with Arbitrary Degree of Freedom (REAF). Although REAF is similar to REST2 in spirit, it provides more controls over how the weight functions behavior. A region to be enhanced sampled is designed as the REAF region. The REAF parameter, τ , similar to λ in alchemical transformation, is used to define the reduction of interactions involved in the REAF region. A set of τ -dependent weighting functions can be utilized to form the τ -dependent total potential energy $U(r^N; \tau)$ can be written as

$$U(r; \tau) = U_{rec}(r^N; W_{rec}(\tau) \cdot q^{RE}) + \sum_{t \neq rec} W_t^{RE}(\tau) \cdot U_t^{RE}(r^N; \tau) + \sum_{t \neq rec} W_t^{RE/Env}(\tau) \cdot U_t^{RE/Env}(r^N; \tau) \quad (26.21)$$

where $U(r^N; \tau)$ is the system total potential energy. The subscription "rec" represents the PME reciprocal term and "t" represents all other energy terms. The superscript "RE" represents the interactions within the REAF region while "RE/Env" the interactions between the REAF region and its surrounding environment. q^{RE} is the charge set of all atoms in the REAF region. In eq. 26.21, the τ -dependency of the PME reciprocal term is through the scaling of the charges of the REAF region atoms by the weight function $W_{rec}(\tau)$, and other terms through alchemical transformation-like weight functions. In the default linear dependency scheme, the weight functions are

$$W_{rec}(\tau) = W_t^{RE/Env}(\tau) = (1 - \tau) \quad (26.22)$$

$$W_t^{RE}(\tau) = (1 - \tau)^2 \quad (26.23)$$

$$0 \leq \tau, W_{rec}(\tau), W_t^{RE/Env}(\tau), W_t^{RE}(\tau) \leq 1.$$

It is clear that $\tau = 0$ corresponds to unscaled interactions while $\tau = 1$ the totally vanished interactions (similar to setting temperature as infinity). Table 26.4} shows the corresponding REST2 temperatures for different τ and weight functions.

τ	$W_t^{RE/Env}(\tau) = (1 - \tau)$	$W_t^{RE}(\tau) = (1 - \tau)^2$	REST2 temperature (K)
0	1	1	298.00
0.1	0.9	0.81	367.90
0.2	0.8	0.64	465.63
0.3	0.7	0.49	608.16
0.4	0.6	0.36	827.78
0.5	0.5	0.25	1192.00
0.6	0.4	0.16	1862.50
0.7	0.3	0.09	3311.11

Table 26.4.: The corresponding REST2 temperatures for different τ and weight functions, assuming that the simulation is perform at 298.0K

Furthermore, these weight functions can be turned on or off to accommodate different situations. Similar to the `gti_add_sc` input control for the alchemical transformation, the `gti_add_re` input control is used in REAF and shown in Table 26.5.

Weight Symbol	Energy Term Abbreviation	Region / Interaction	gti_add_re flag						
			1	2	3	4	5	6	7
W_{bond}^{RE}	bond	RE	P	P	P	P	P	P	P
W_{ang}^{RE}	ang	RE	P	P	P	P	P	P	S
W_{tor}^{RE}	tor	RE	P	P	P	S	S	S	S
W_{dir}^{RE}	dir	RE	P	S	S	P	S	S	S
W_{1-4Ele}^{RE}	1-4 Ele	RE	P	S	S	S	S	S	S
W_{LJ}^{RE}	LJ	RE	P	P	S	P	P	S	S
W_{1-4LJ}^{RE}	1-4 LJ	RE	P	P	S	S	S	S	S
$W_{dir}^{RE/Env}$	dir	RE/Env	S	S	S	S	S	S	S
$W_{1-4Ele}^{RE/Env}$	1-4 Ele	RE/Env	S	S	S	S	S	S	S
$W_{LJ}^{RE/Env}$	LJ	RE/Env	S	S	S	S	S	S	S
$W_{1-4LJ}^{RE/Env}$	1-4 LJ	RE/Env	S	S	S	S	S	S	S
W_{rec}	rec	all	S	S	S	S	S	S	S

Table 26.5.: The scaling behavior τ -dependence of the weight functions in eq. 26.22, controlled by the `gti_add_re` flag, for different energy terms and regions/interactions in the AMBER DD Boost package.

Energy terms are defined in text (and also Table S1). RE: the REAF region internal interactions, REAF/Env the interactions between the REAF region and the environment.

Flags:

S: Scaled with τ . The corresponding λ -dependent weight function will be used.

P: Not scaled with τ . The corresponding weight function is simply 1.

The REAF module can be enabled by setting the input control "ifreaf=1" and the REAF parameter τ is defined by the input "reaf_tau", e.g., reaf_tau=0.001, The REAF region is defined by the standard AMBER mask selection, e.g., "reaf_mask1='@1-8'". The "reaf_mask1" is to define the REAF region when are in a regular MD or at the $\lambda=0$ state in an alchemical transformation. When used together with alchemical transformation, "reaf_mask2" is used to define the REAF region at the state of $\lambda = 1$.

`ifreaf` Enable REAF:

= 0 (Default) REAF is disabled.

= 1 REAF is enabled.

`gti_add_re` Control the interactions between the REAF region and the environment according to Table 26.5:

`reaf_tau` The τ value in eq. 26.22.

`reaf_temp` The effective temperature in K. It will be converted to τ according to Table 26.4. When both reaf_tau and reaf_temp present, reaf_tau will be used.

`reaf_mask1` The REAF region for a plain MD simulation or the $\lambda=0$ state in an alchemical transformation.

`reaf_mask2` The REAF region for the $\lambda=1$ state in an alchemical transformation.

To perform a REAF run, one needs set up a set of H-REMD replicas with different reaf_tau values, e.g., $\tau = (0, 0.1, 0.2, 0.3)$ for a 4-replica H-REMD to perform enhanced sampling with effective temperatures of (298, 367.90, 465.53, 608.16).

26.3.10. Multi-dimensional Replica Exchange

Multi-dimensional replica exchange [446, 613] refers to an expanded ensemble technique in which each subensemble (i.e., each replica) is defined by multiple state parameters such as the temperature of the heat bath or the Hamiltonian. For such systems, an exchange attempt between two arbitrary replicas yields a complex equation for the exchange probability that must be derived and specified for each type of multi-dimensional exchange scheme. However, if exchange attempts between replica pairs are restricted to pairs that only differ in *one* state parameter, such as the temperature, then the exchange probability equation reduces to the one used for that particular type of replica exchange. Such an approach allows the existing exchange routines to be used in complex, multi-dimensional replica exchange simulations.

To implement the scheme described above, the entire expanded ensemble is subdivided into *dimensions* which are further divided into *groups*. Each dimension is defined by a particular type of exchange attempt—namely temperature exchange, Hamiltonian exchange, pH exchange, or Redox Potential exchange—and assigns every replica in the simulation to a particular *group*. Each group is like a one-dimensional REMD simulation by itself—replicas differ only by the one state parameter used to define that particular dimension. Exchange attempts occur only between nearest neighbors of a single replica group.

No two replicas should be in the same group in more than one dimension since, by definition, that would require the state parameters of those two replicas to differ in more than one dimension (which would preclude them from being part of the same group in *any* dimension according to the scheme described previously). It is easiest to understand this scheme in the context of a 2-dimensional replica exchange ensemble with replicas represented by elements of a matrix, as in Fig. 26.2.

26.3.10.1. Running multi-dimensional replica exchange simulations

Multi-dimensional REMD simulations require an extra input file provided on the command-line (*not* in the groupfile) that defines each replica group in each dimension (see Subsection 26.3.3 for information about how you can use the `genremdinputs.py` tool to prepare your input files). Every replica must be assigned to one and only one group in each dimension (failure to do so results in an error message). Furthermore, each group must consist of an even number of replicas. Dimensions are defined in the `&multirem` namelist in the Multi-dimensional REMD input file. Each `&multirem` namelist adds another dimension. The following variables may be specified in the `&multirem` namelist:

exch_type Defines the type of exchange that will be performed. Supported values (case-insensitive) are “temperature” (or “temp”), “Hamiltonian” (or “HREMD”), “pH”, and “Redox”.

group(:, :) 2-dimensional (Fortran-style) array defining the group (first dimension) and the position within that group (second dimension). See the description of the exchange routines above to see if the ordering within each group is important (for example, the ordering defines exchange partners in H-REMD while replicas are automatically sorted by target temperature in T-REMD). Indexes in this array start from 1, and index n corresponds to the n^{th} replica defined in the groupfile. The suggested syntax for assigning to this variable is shown in the example below.

desc Description that will be printed in the `rem.log` files. This is for documentation purposes only, and will have no effect on the simulation.

A sample Multi-dimensional REMD input file that performs alchemical Hamiltonian-REMD in one dimension and Temperature-REMD in another dimension is shown below. In this example, replicas 1 and 2 have the same Hamiltonian, and replicas 1 and 3 have the same temperature.

```
Temperature REMD
&multirem
  exch_type = 'TEMPERATURE',
  group(1, :) = 1, 2,
  group(2, :) = 3, 4,
  desc = 'Temperature exchange from 300K to 400K'
/
```

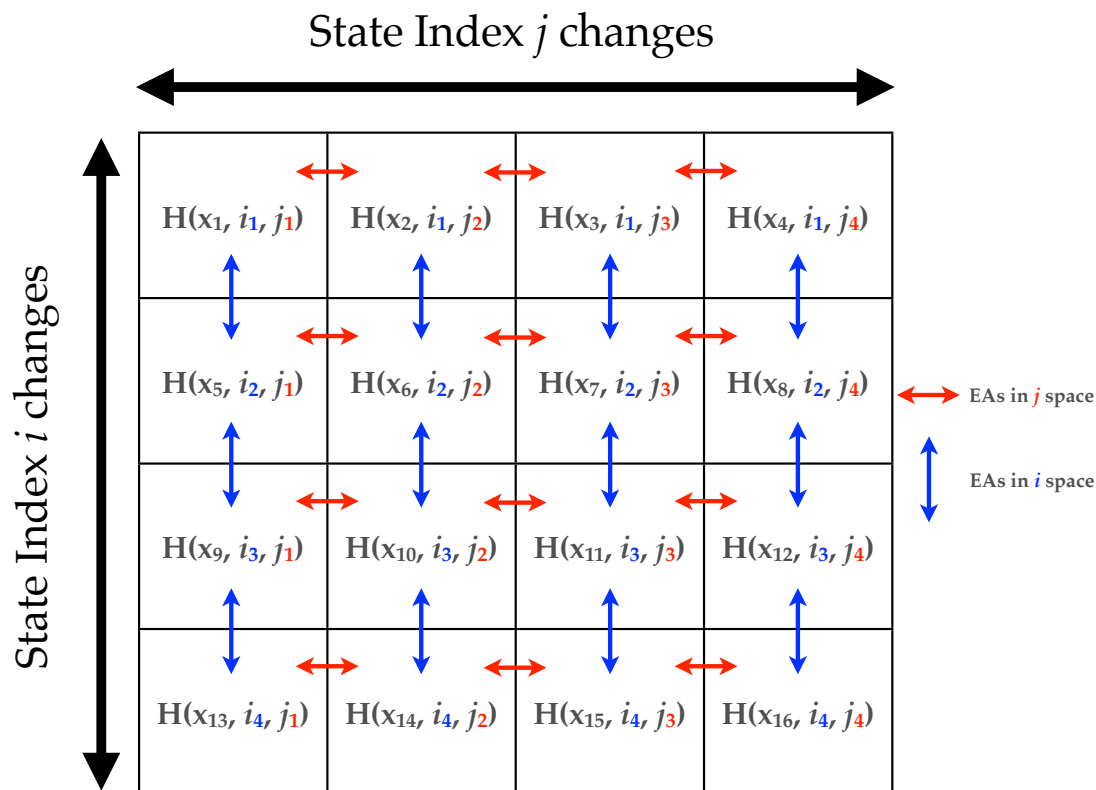


Figure 26.2.: Schematic showing exchange attempts (EAs) in multi-dimensional REMD simulations. Exchange attempts are indicated by the colored arrows, where red arrows indicate exchange attempts between replicas in a group of the dimension defined by the j state parameters. Blue arrows indicate exchange attempts between replicas in the dimension defined by the state parameter i . Figure taken from ref. 623.

```

Hamiltonian REMD
&multirem
  exch_type='HAMILTONIAN',
  group(1,:) = 1,3,
  group(2,:) = 2,4,
  desc = 'Protonated ASP to Deprotonated ASP mutation'
/

```

Running multi-dimensional REMD simulations differs from running them in a single dimension. First, restarts and trajectories *must* be written in the NetCDF format (ntxo=2 for restarts and ioutfm=1 for trajectories). These changes are applied by default for multi-dimensional REMD simulations. Next, the REMD input file is taken following the `-remd-file` flag, and `-rem` should not be specified (it is set to -1 internally when `-remd-file` is read). An example command-line corresponding to the 4-replica example input file is shown below:

```

mpirun -np 4 sander.MPI -ng 4 -groupfile groupfile \
      -remd-file remd.dim -remlog rem.log

```

The file just shown above is an example of the `remd.dim` file. The replica exchange information is stored in the `remlog` files written during the simulation. A separate `remlog` file is written for each dimension with the name `<prefix>.n` where n is the n^{th} dimension read from the REMD input file and `<prefix>` is the file name given on the command-line for the `-remlog` switch.

26.3.10.2. Restarting multi-dimensional replica exchange simulations

Some exchange types swap state parameters (e.g., temperature, pH and Redox Potential) while others swap coordinates (Hamiltonian), meaning that the ordering of the groupfile may change for restarts (see Fig. 26.3). To prevent requiring you to rewrite a new REMD file or groupfile each restart, the group number and replica position for each dimension is stored in the restart file. When `irest=1` (see page 398) and the restart files contain the replica position information, the position of each replica in each dimension is set to the values stored in the restart file. This allows the same groupfile and REMD file to be used for every subsequent restart. For general information on restarting a REMD simulation see Subsection 26.3.4.1.

Note, if the REMD index information is not present in the restart file *or* the REMD dimension information in the restart does not match what is defined in the REMD input file, the replica ordering will be assigned as it is defined in the REMD input file.

26.3.10.3. Analyzing multi-dimensional replica exchange simulations

The REMD log file for each dimension is further divided into the log messages for each group, as shown below.

```

# Replica Exchange log file
# numexchg is      100
# Dimension      1 of      2
# Description: Temperature exchange from 300K to 400K
# exchange_type = TEMPERATURE
# REMD filenames:
# remlog= rem.log.1
# remd dimension file= remd.dim
# Rep#, Velocity Scaling, T, Eptot, Temp0, NewTemp0, Success rate (i,i+1), ResStruct#
# exchange      1 REMD group      1
  1      -1.00      0.00      -40.69      300.00      300.00      0.00      0
  2      -1.00      0.00      -19.77      400.00      400.00      0.00      0
# exchange      1 REMD group      2
  1      -1.00      0.00      -66.78      300.00      300.00      0.00      0
  2      -1.00      0.00      -46.06      400.00      400.00      0.00      0
# exchange      3 REMD group      1
  1      -1.00      221.66      -34.00      300.00      300.00      0.00      0

```

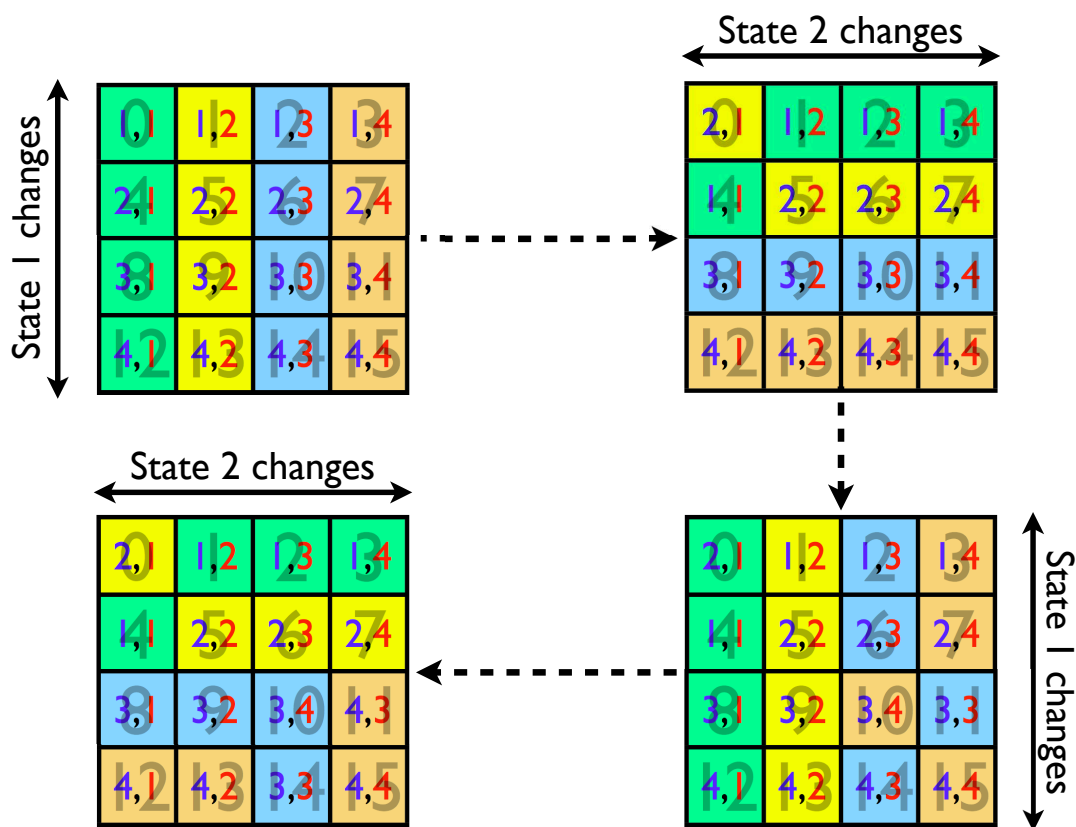



Figure 26.3.: Replica arrangement in multi-dimensional REMD simulations at multiple exchange steps following some successful state parameter exchanges. A large gray number in the background is the original placement in the REMD input file. A blue and red number pair is the group number and position in the group, respectively. Replicas with the same color are part of the same group. Figure taken from ref. 623.

26. Free energies

2	-1.00	347.52	-29.09	400.00	400.00	0.00	0
# exchange		3 REMD group		2			
1	-1.00	257.14	-63.10	300.00	300.00	0.00	0
2	-1.00	332.76	-26.73	400.00	400.00	0.00	0

The above example is shown for the first dimension (temperature) of the example REMD file shown in Sec. 26.3.10.1. The columns are the same as those used in the corresponding 1-dimensional REMD simulation for that exchange type.

To analyze structural properties, you must use *cpptraj* to properly snapshots into the appropriate replicas. For example, for a T,pH-REMD simulation the command inside *cpptraj* would be like:

```
trajin mdcrd.000 remdtraj remdtrajvalues 300.0,7.0
```

In this example, we would be reconstructing the trajectory for temperature 300 K and pH 7.0 See Chapter 37 and more specifically Subsection 37.10.4 for more details.

26.3.10.4. Reconstructing cpout and ceout files with fixremdcouts.py

If you perform any type of REMD simulation that contains the constant pH and/or constant Redox Potential options active, it becomes a problem to reconstruct your cpout or ceout files. This obviously happen in Multi-dimensional REMD simulations [446, 613] but this could even happen with one-dimensional REMD simulations, like, for example, T-REMD with constant pH active. In this example one would need to reconstruct the cpout files by temperature, in order to properly analyze the protonation states during the simulation.

fixremdcouts.py is a Python tool written by Vinícius Cruzeiro that allows the cpout or ceout files from any REMD to be reconstructed for posterior analysis in *cphstats* or *cestats* [613]. This tool generalizes for any REMD simulation what the *--fix-remd* option does in *cphstats* for pH-REMD or in *cestats* for E-REMD. You can access a list and description of all available command-line flags using the *--help* flag, whose output is shown below.

```
usage: fixremdcouts.py [Options]
optional arguments:
  -h, --help            show this help message and exit
  -v, --version          show the program's version and exit
  --author              show the program's author name and exit
  -O, --overwrite       Allow existing outputs to be overwritten. Default:
                        False
Required Arguments:
  -couts [FILE [FILE ...]]
                        AMBER CPOUT and/or CEOUT files
Non-required Arguments:
  -prefix STRING        Prefix of the reordered file names. Default: reordered
This program will reorder Replica Exchange CPOUT and/or CEOUT files. It can be
used even when pH or Redox Potential REMD are not used, for example to
reconstruct CPOUT files per temperature on a T-REMD simulation with constant
pH on. This tool can also be used with Multidimensional REMD CPOUT and/or
CEOUT files.
```

An example of the execution of the program is given below:

```
fixremdcouts.py -prefix reordered -couts [list all cpout and/or ceout files]
```

You may provide cpout and ceout files together to *-couts*. As an example, if you performed a T-REMD with CpHMD whose temperature replicas are 300 and 320 K, the generated files would be *reordered.cpout.T_300.00* and *reordered.cpout.T_320.00*. Another example, if you performed a pH,T-REMD whose replicas values are pH 7.0 and 7.5 and temperatures 300 and 320 K, the generated files would be *reordered.cpout.pH_7.000000.T_300.00*, *reordered.cpout.pH_7.000000.T_320.00*, *reordered.cpout.pH_7.500000.T_300.00*, and *reordered.cpout.pH_7.500000.T_320.00*.

26.4. Adaptively Biased MD, Steered MD, Umbrella Sampling with REMD and String Method

26.4.1. Overview

The following describes a suite of modules useful for the calculation of the free energy associated with a reaction coordinate $\sigma(\mathbf{r}_1, \dots, \mathbf{r}_N)$ (which is defined as a smooth function of the atomic positions $\mathbf{r}_1, \dots, \mathbf{r}_N$):

$$f(\xi) = -k_B T \ln \langle \delta[\xi - \sigma(\mathbf{r}_1, \dots, \mathbf{r}_N)] \rangle,$$

(the angular brackets denote an ensemble average, k_B is the Boltzmann constant and T is the temperature) that is also frequently referred to as the *potential of mean force*.

Specifically, new frameworks are provided for equilibrium umbrella sampling and steered molecular dynamics that enhance the functionality delivered by earlier implementations (described earlier in this manual), along with a new Adaptively Biased Molecular Dynamics (ABMD) method [624] that belongs to the general category of umbrella sampling methods with a time-dependent potential. Such methods were first introduced by Huber, Torda and van Gunsteren (the Local Elevation Method [625]) in the molecular dynamics (MD) context, and by Wang and Landau in the context of Monte Carlo simulations [626]. More recent approaches include the metadynamics method [627, 628]. All these methods estimate the free energy of a reaction coordinate from an evolving ensemble of realizations, and use that estimate to bias the system dynamics to flatten an effective free energy surface. Collectively, these methods may all be considered to be umbrella sampling methods with an evolving potential. The algorithms discussed here were developed by the group of Prof. Celeste Sagui (sagui@ncsu.edu) and Prof. Christopher Roland (cmroland@ncsu.edu); the current version was implemented by Dr. Volodymyr Babin.

The ABMD method grew out of attempts to speed up and streamline the metadynamics method for free energy calculations with a *controllable* accuracy. It is characterized by a favorable scaling in time, and only a few (two) control parameters. It is formulated in terms of the following equations:

$$m_a \frac{d^2 \mathbf{r}_a}{dt^2} = \mathbf{F}_a + \frac{\partial}{\partial \mathbf{r}_a} U[t|\sigma(\mathbf{r}_1, \dots, \mathbf{r}_N)],$$

$$\frac{\partial U(t|\xi)}{\partial t} = \frac{k_B T}{\tau_F} G[\xi - \sigma(\mathbf{r}_1, \dots, \mathbf{r}_N)],$$

where the first equation represents Newton's law that governs ordinary MD (temperature and pressure regulation terms are not shown) augmented with an additional force coming from the time dependent biasing potential $U(t|\xi)$ [$U(t=0|\xi) = 0$], whose time evolution is given by the second equation. $G(\xi)$ is a positive definite and symmetric kernel, which may be thought of as a smoothed Dirac delta function. For large enough τ_F (the flooding timescale) and small kernel width, the biasing potential $U(t|\xi)$ converges towards $-f(\xi)$ as $t \rightarrow \infty$.

Our numerical implementation of the ABMD method involves the use of a bi-weight kernel along with the use of cubic B-splines (or products thereof) to discretize the biasing potential $U(t|\xi)$ w.r.t. ξ , and an Euler-like scheme for time integration. ABMD admits two important extensions, which lead to a more uniform flattening of $U(t|\xi) + f(\xi)$ due to an improved sampling of the “evolving” canonical distribution. The first extension is identical in spirit to the *multiple walkers metadynamics* [629, 630]. It amounts to carrying out several different MD simulations biased by the same $U(t|\xi)$, which evolves via:

$$\frac{\partial U(t|\xi)}{\partial t} = \frac{k_B T}{\tau_F} \sum_{\alpha} G[\xi - \sigma(\mathbf{r}_1^{\alpha}, \dots, \mathbf{r}_N^{\alpha})],$$

where α labels different MD trajectories. A second extension is to gather several different MD trajectories, each bearing its own biasing potential and, if desired, its own distinct collective variable, into a generalized ensemble for “replica exchange” with modified “exchange” rules [631–633]. Both extensions are advantageous and lead to a more uniform flattening of $U(t|\xi) + f(\xi)$.

In order to assess and improve the accuracy of the free energies, the ABMD accumulations may need to be followed up with equilibrium umbrella sampling runs, which make use of the biasing potential $U(t|\xi)$ as is. Such

a procedure is very much in the spirit of adaptive umbrella sampling. With these runs, one calculates the biased probability density:

$$p^B(\xi) = \langle \delta[\xi - \sigma(\mathbf{r}_1, \dots, \mathbf{r}_N)] \rangle_B.$$

The idea here is that if, as a result of an ABMD run, $f(\xi) + U(t|\xi) = 0$ exactly, then the biased probability density $p^B(\xi)$ would be flat (constant). In practice, this is typically not the case, but one can use $p^B(\xi)$ to “correct” the free energy via:

$$f(\xi) = -U(\xi) - k_B T \ln p^B(\xi).$$

With the ABMD procedure, one can obtain accurate free energy curves and equilibrium properties. We note that to obtain ABMD free energies requires a (minor) amount of post-processing by means of the nfe-umbrella-slice utility freely available in AmberTools as described in Subsection 26.4.8. This methodology has been applied to a variety of biomolecular systems, including small peptides [624, 634, 635], sugar puckering [636], polyproline systems [637–639], guest-host systems [640, 641], polyglutamine systems [642, 643], and DNA systems [644–648]. In addition, SMD simulations (discussed below) have been used to examine transition pathways and mechanisms, to estimate free energy differences [638, 649], and to calculate transition rates [650–652].

While the above represents the basic ABMD implementation, AMBER20 introduced three additional algorithms – the Well-Tempered (WT) ABMD, a selection mechanism for multiple walker ABMD and Driven ABMD (D-ABMD)[653] – all of which enhance the stability and convergence of an ABMD simulation. Also implemented is the Swarms-of-Trajectories String Method (STSM) [654], which gives a way of exploring the Minimum Free Energy Path (MFEP) on free energy landscape. Current version of these codes were implemented by Dr. Mahmoud Moradi (moradi@uark.edu), Dr. Feng Pan (fpan3@ncsu.edu) and Ashkan Fakharzadeh (afakhar@ncsu.edu).

The Well-Tempered ABMD: An alternative to the follow-up equilibrium simulations for increased ABMD accuracy is provided by the WT-ABMD, which is implemented in the spirit of the WT-metadynamics [655]. In the original ABMD implementation, the history dependent biasing potential is built up at a fixed rate:

$$U(\xi, t) = U^0(\xi) + \int_0^t dt' \omega G(\xi - \xi'), \quad (26.24)$$

in which $U(\xi, t)$ is the biasing potential at time t , U^0 is an arbitrary function that typically represents the initial guess for the biasing potential (in the absence of a guess, this is assumed to be flat) and $\omega = k_B T / \tau_F$ is a constant, unbiased rate. As the simulation proceeds and reaches convergence, then $\langle U(\xi, t \rightarrow \infty) \rangle_a \approx U^s(\xi) + u(t)$, in which $\langle \cdot \rangle_a$ is the ensemble-average over the adaptive trajectories, the stationary term is $U^s \approx -F(\xi)$, and $u(t)$ is an additive time-dependent constant [655]. Unfortunately, updating the biasing potential at the same rate throughout the simulation may lead to a poorly converged result, since the biasing potential ends up fluctuating around $-F(\xi)$ with an amplitude that depends on ω .

One way to resolve this problem is to update the kernel at a non-uniform rate by means of a “well-tempered” ω :

$$U(\xi, t) = U^0(\xi) + \int_0^t dt' \omega(\xi', t') G(\xi - \xi'), \quad (26.25)$$

in which $\omega(\xi, t)$ is a time-dependent, non-uniform rate chosen to be $\omega_0 e^{-\beta' U(\xi, t)}$ ($1/\beta' = k_B T'$ where T' is a pseudo-temperature) that reduces to a constant ω_0 in the $\beta' \rightarrow 0$ limit (*i.e.*, resulting in conventional ABMD). With this choice, one can show that $\langle U(\xi, t \rightarrow \infty) \rangle_a \approx U^s(\xi) + u(t)$, ($u(t)$ is an additive constant) in which $U^s(\xi)$ and $F(\xi)$ are related via $U^s(\xi) = -(1 + \frac{\beta'}{\beta})^{-1} F(\xi)$ or $F(\xi) = -(1 + \frac{T}{T'}) U^s(\xi)$. This way of updating the biasing potential leads to a considerably smoother convergence to the desired free energy and more stable ABMD simulations.

Multiple walker selection algorithm: The ABMD multiple walker algorithm can be improved by allowing for periodic interactions between the different walkers and “resampling” on-the-fly. The rationale behind this is that not all walkers are equally effective in sampling the configuration space. A situation that is all too common is that different walkers end up being “bunched up” or clustered together in some local metastable region, because of hidden barriers that are oriented along orthogonal degrees of freedom to the reaction coordinate. To improve this situation, one would like to facilitate walkers that are sampling the undersampled regions of phase space, and force the walkers in the oversampled regions to move away and explore regions not yet covered. Such an algorithm has

previously been implemented via scripts in the NAMD code for the adaptive biasing force algorithm [656].

A resampling or selection algorithms for interacting multiple walkers requires a continual monitoring of the walkers by means of a periodic evaluation of a fitness function and a resampling of the walkers according to their fitness efficiency[656]. Efficient walkers that are wandering in the undersampled regions are enhanced by being cloned, while inefficient walkers found in the oversampled regions of phase space are correspondingly killed. This procedure is then repeated periodically during the simulation, thereby accelerating convergence to a more uniform distribution of walkers and flattening of the free energy landscape.

Our specific interacting/resampling/selection multiple-walker algorithm is implemented as follows. Each walker n is assigned a weight w_n , which is evaluated at the end of each resampling period of time τ . At the i^{th} resampling period, i.e., from time $t_{(i-1)} = (i-1)\tau$ to $t_i = i\tau$, walker n moves through configuration space building up its own trajectory $(r_1^n, \dots, r_N^n)$. The weights are then tested and updated every fixed time interval of length τ . Specifically, after the i^{th} time interval, weights are estimated by:

$$w_n = K^{-1} \exp \left(\int_{t_{i-1}}^{t_i} S(\xi_n^t) dt \right),$$

where ξ_n^t represents the collective variable evaluated at time t for trajectory n , $K = \sum_{n=1}^{N_w} w_n$ is the normalization factor, and

$$S(\xi) = C \nabla^2(\rho(\xi)) / \rho(\xi),$$

with $\rho(\xi)$ representing the density of microstates in the collective variable space and C a constant. The quantity $S(\xi)$ will be positive typically if the walker is found in the undersampled regions, which have a convex density function. Similarly, a negative $S(\xi)$ value indicates that the system is in the concave region of the density function, which typically is oversampled. In the context of ABMD implementation, the biasing potential is approximately proportional to the histogram of the collective variable by construction, and represents a good estimate for ρ . The implementation is therefore straightforward; the integral above is estimated for each trajectory independently by summing over $S(\xi_n^t)$ at every step from $t = t_{i-1}$ to $t = t_i$, in which Δt is the MD *timestep*. At the end of each period the walkers send their unnormalized weight estimates to the "master processor" to normalize them. A stochastic resampling method is then used to clone/kill the replicas based on their weight factors [656]. The number of copies present in the next period for walker n is determined by the integer number:

$$\begin{aligned} W_1 &= \lfloor \eta_1 + N_w w_1 \rfloor, \\ W_n &= \lfloor \eta_n + N_w \sum_{m=1}^n w_m \rfloor - \lfloor \eta_n + N_w \sum_{m=1}^{n-1} w_m \rfloor, \quad \text{for } n > 1. \end{aligned}$$

in which $0 < \eta_n < 1$ is drawn from a uniform distribution (using a random number generator). The atomic coordinates and velocities of the walkers with $N_n > 0$ are "sent" to N_n walkers. The resampling algorithm above guarantees $\sum_n W_n = N_w$.

In terms of an ABMD simulation, the selection algorithm is most beneficial during the initial and middle parts of the simulation when there are large variations in the biasing potential. In the latter parts, when the effective free energy is almost flat, the distribution of walkers should be roughly uniform. In that case, the selection mechanism is unnecessary and, if one wishes to continue the simulation, it is best to proceed with the non interacting multiple walker algorithm. It has been found that a convenient stopping mechanism may be based on the entropy of the weights. Defining $H = \sum_n w_n \log(w_n)$, the selection mechanism will be stopped if $E_w = H - \log(1/N_w)$ goes below $-\epsilon \log(1/N_w)$. Here, $\log(1/N_w)$ represents the entropy of uniform weights, and the stopping parameter ϵ varies between $0 \leq \epsilon \leq 1$. When $\epsilon = 0$, the algorithm never stops, while $\epsilon = 1$ forces a stop irrespective of the values of the weights.

In addition to ϵ , there are also two other user-defined variables in the selection algorithm, including the constant C and the interval time τ . While the physical interpretation of τ is straightforward, C represents a pseudo diffusion constant. One may think of the selection algorithm as an induced diffusion in the reaction coordinate space; The larger the value of C , the faster the system will diffuse along the reaction coordinate space. Therefore C determines the strength or aggressiveness of the resampling algorithm. The most efficient value for C is dependent on the

nature of the collective variable and the shape of its density ρ . Since the best choice of C for a given problem is somewhat of an art, we refer the interested reader to the ABMD tutorials on the AMBER webpage for insight into choosing this variable. Finally, we also note that the multiple walker selection mechanism can be invoked as is or in conjunction with the WT-ABMD for enhanced stability and convergence.

Driven ABMD: ABMD and SMD schemes are both powerful nonequilibrium sampling methods; however, each comes with its own practical limitations. For instance, SMD is often associated with a very slow convergence if used for free energy calculations. However it can be used to explore the transition paths, at least qualitatively; an advantage over ABMD, in which the system starting from one end of the configuration space (the reactant) may take a long time to visit the other end (the product). SMD and ABMD schemes, however can be integrated into a novel driven adaptive-bias 3 scheme, termed driven ABMD (D-ABMD) that takes advantage of both its driven and adaptive-bias components and is advantageous over both components in isolation. D-ABMD has an advantage over conventional (or well-tempered) ABMD in that it ensures the exploration of the transition pathway (from one end to the other) in the early stages of the simulation and gradually improves the estimate of the free energies almost uniformly along the reaction coordinate. D-ABMD has also an advantage over the conventional SMD in that the effective free energy surface gradually becomes smooth and flat such that the system can move along the reaction coordinate with progressively less amount of work. The D-ABMD method is similar to D-MetaD method, which was recently introduced in Ref.[653] as an example of driven, adaptive-bias schemes.

In order to combine the two schemes described above, we have developed a driven adaptive-bias scheme that adds an adaptive $U_a(\xi, t)$ and a driving $U_d(\xi, t)$ potential to the Hamiltonian. We use an iterative approach in which an independent simulation is performed from time $t = 0$ to $t = T$ in the n^{th} iteration ($n = 1, 2, \dots$), biased by the potential $U_d(\xi, t) + U_a^n(\xi, t)$ in which $U_d(\xi, t) = \frac{k}{2}(\xi - \eta(t))^2$ for all n ($\eta(t)$ is moving center of the SMD harmonic potential in the ξ space), and:

$$U_a^n(\xi, t) = U^{n-1}(\xi) + \int_0^t dt' \omega(\xi', t') K(\xi - \xi') e^{-\beta \omega'}$$

in which ω' is either defined as the accumulated work or the transferred work. The accumulated and transferred works are defined as $\omega'_{ac} = \int_0^t dt' \frac{\partial}{\partial t'} U_d(\xi', t')$ and $\omega'_{tr} = \omega'_{ac} - U_d(\xi', t)$. Theoretically the $e^{-\beta \omega'_{tr}}$ factor or “constant weight” is more accurate but for practical reasons the $e^{-\beta \omega'_{ac}}$ factor or “pulling weight” is preferred. Particularly, in our algorithm, the constant weight $e^{-\beta \omega'_{tr}} = e^{-\beta \omega'_{ac}} e^{\beta U_d(\xi', t')}$ may become unstable for large biasing potentials. To avoid the instability in either case a cutoff for ω' is used (i.e., the algorithm will not be applied if ω' is smaller than the cutoff). At the moment, Driven ABMD is only applicable to one-dimensional reaction coordinate.

If any of these modules prove to be useful, please consider quoting the following papers: V. Babin, C. Roland and C. Sagui, “Adaptively biased molecular dynamics for free energy calculations”, J. Chem. Phys. **128**, 134101 (2008); V. Babin, V. Karpusenka, M. Moradi, C. Roland and C. Sagui, “Adaptively biased molecular dynamics: an umbrella sampling method with a time-dependent potential”, Int. J. Quant. Chem. **109**, 3666 (2009).

From Amber16, we implement these modules from SANDER to PMEMD and the modules are GPU compatible. To keep the consistency in format, we do a series of changes and updates to the usage of these modules. One big change is that you must set **infe** = 1 in &cntrl to activate these modules. Also, the input format has been changed to namelist style and reaction coordinate variables will be read from separate files. For the details, please read Subsection 26.4.7

infe This variable controls the usage of the non-equilibrium free energy method. When *infe*=0, the ABMD and related methods are turned off; when *infe*=1, they are turned on and the blocks &smd, &pmmd, &abmd, &bbmd and &stsm will be recognized. The default value is 0. Note that use of these algorithms may require a (minor) amount of post-processing by means of the nfe-umbrella-slice utility freely available in AmberTools described in Subsection 26.4.8.

26.4.2. Reaction Coordinates

A reaction coordinate is defined in the `colvar` namelist in a separate file. (see Fig. 26.4). This section must contain a `cv_type` keyword along with a value of type `STRING` and a list of integers `cv_i` (the number of integers

```

&colvar
  cv_type = STRING
  cv_ni = N, cv_nr = M
  cv_i = i1, i2, ..., iN
  cv_r = r1, r2, ..., rM
/

```

Figure 26.4.: Syntax of reaction coordinate definition: *cv_type* is a *STRING*, *cv_i* is a list of integer numbers and *cv_r* is a list of real numbers.

is defined by *cv_ni*). For some types of reaction coordinates the *colvar* section must also contain a list of real numbers, *cv_r*, whose length is defined by *cv_nr*.

The following reaction coordinates (specified by *cv_type*) are currently implemented:

DISTANCE: distance (in Å) between two atoms whose indexes are read from the list *cv_i*.

COM_DISTANCE: distance between the center of mass of two atom groups. The *cv_i* list is interpreted as a list of indexes of participating atoms. Zeros separate the groups, the last zero is optional. eg: *cv_i* = *a1*, ..., *aN*, 0, *b1*, ..., *bM*, 0.

DF_COM_DISTANCE: difference of distances between the center of mass of first two atom groups and second two atom groups. The *cv_i* list is interpreted as a list of indexes of participating atoms. Zeros separate the groups, the last zero is optional. eg: *cv_i* = *a1*, ..., *aN*, 0, *b1*, ..., *bM*, 0, *c1*, ..., *cL*, 0, *d1*, ..., *dK*, 0, **DF_COM_DISTANCE** is **COM_DISTANCE**(*a1*, ..., *aN*, 0, *b1*, ..., *bM*) - **COM_DISTANCE**(*c1*, ..., *cL*, 0, *d1*, ..., *dK*).

LCOD: linear combination of distances (in Å) between pairs of atoms listed in *cv_i* with the coefficients read from *cv_r* list. For example, *i* = 1, 2, 3, 4 and *r* = 1.0, -1.0 define the difference between 1-2 and 3-4 distances, i.e. **LCOD** = *r1**distance(1, 2) + *r2**distance(3, 4).

ANGLE: angle (in radians) between the lines joining atoms with indexes *i1* and *i2* and atoms with indexes *i2* and *i3*.

COM_ANGLE: angle (in radians) formed by the center of mass of three atom groups. The *cv_i* list is interpreted as a list of indexes of participating atoms. Zeros separate the groups, the last zero is optional. eg: *cv_i* = *a1*, ..., *aN*, 0, *b1*, ..., *bM*, 0, *c1*, ..., *cK*, 0.

TORSION: dihedral angle (in radians) formed by atoms with indexes *i1*, *i2*, *i3* and *i4*.

COM_TORSION: dihedral angle (in radians) formed by the center of mass of four atom groups. The *cv_i* list is interpreted as a list of indexes of participating atoms. Zeros separate the groups, the last zero is optional. eg: *cv_i* = *a1*, ..., *aN*, 0, *b1*, ..., *bM*, 0, *c1*, ..., *CK*, 0, *d1*, ..., *dL*, 0.

COS_OF_DIHEDRAL: sum of cosines of dihedral angles formed by atoms with indexes in the list *cv_i*. The number of atoms must be a multiple of four.

SIN_OF_DIHEDRAL: sum of sines of dihedral angles formed by atoms with indexes in the list *cv_i*. The number of atoms must be a multiple of four.

PAIR_DIHEDRAL: sum of cosines of a list of angles each formed by summing two neighboring dihedral angles from a list formed by atoms with indices *cv_i*. The number of atoms must be a multiple of four. For a list of dihedral angles such as $\{\alpha_1, \dots, \alpha_N\}$, **PAIR_DIHEDRAL** is $\sum_{i=1}^{N-1} \cos(\alpha_i + \alpha_{i+1})$ which ranges between $-N + 1$ and $N - 1$.

PATTERN_DIHEDRAL: a particular pattern-recognizing function defined on a list of dihedral angles formed by atoms with indices `cv_i`. The number of atoms must be a multiple of four. The definition is particularly relevant for the dihedral angles with a binary-like behavior of being either around 0 or 180 (e.g., ω backbone dihedral angle). For a list of dihedral angles such as $\{\alpha_1, \dots, \alpha_N\}$, **PATTERN_DIHEDRAL** is $\sum_{i=1}^N \cos^2(\alpha_i/2) 2^{i-1}$ which ranges between 0 and $2^N - 1$.

R_OF_GYRATION: radius of gyration (in Å) of atoms with indexes given in the `cv_i` list (mass weighted).

```
&colvar
  cv_type = 'MULTI_RMSD'
  cv_ni = 9, cv_nr = 21,
  cv_i = 1, 2, 3, 4, 0, 3, 4, 5, 0 ! the last zero is optional
  cv_r = 1.0, 1.0, 1.0, ! group #1, atom 1
        2.0, 2.0, 2.0, ! group #1, atom 2
        3.0, 3.0, 3.0, ! group #1, atom 3
        4.0, 4.0, 4.0, ! group #1, atom 4
        23.0, 23.0, 23.0, ! group #2, atom 3
        4.0, 4.0, 4.0, ! group #2, atom 4
        5.0, 5.0, 5.0 ! group #2, atom 5
/
```

Figure 26.5.: An example of *MULTI_RMSD* variable definition.

MULTI_RMSD: RMS (in Å, mass weighted) of RMSDs of several groups of atoms w.r.t. reference positions provided in the `cv_r` list. The `cv_i` list is interpreted as a list of indexes of participating atoms. Zeros separate the groups. An atom may enter several groups simultaneously. The `cv_r` array is expected to contain the reference positions (without zero sentinels). The implementation uses the method (and the code) introduced in Ref. [657]. An example of variable of this type is presented in Fig. 26.5. Two groups are defined here: one comprises the atoms with indexes 1, 2, 3, 4 (line 3 in Fig. 26.5, numbers prior to the first zero) and another one of atoms with indexes 3, 4, 5. The code will first compute the (mass weighted) RMSD (R_1) of atoms belonging to the first group w.r.t. reference coordinates provided in the `cv_r` array (first 12 = 4×3 real numbers of it; lines 4, 5, 6, 7 in Fig. 26.5). Next, the (mass weighted) RMSD (R_2) of atoms of the second group w.r.t. the corresponding reference coordinates (last 9 = 3×3 elements of the `cv_r` array in Fig. 26.5) will be computed. Finally, the code will compute the value of the variable as follows:

$$\text{value} = \sqrt{\frac{M_1}{M_1 + M_2} R_1^2 + \frac{M_2}{M_1 + M_2} R_2^2},$$

where M_1 and M_2 are the total masses of atoms in the corresponding groups.

N_OF_BONDS:

$$\text{value} = \sum_p \frac{1 - (r_p/r_0)^6}{1 - (r_p/r_0)^{12}},$$

where the sum runs over pairs of atoms p , r_p denotes distance between the atoms of pair p and r_0 is a parameter measured in Å. The `cv_r` array must contain exactly one element that is interpreted as r_0 . The `cv_i` array is expected to contain pairs of indexes of participating atoms. For example, if 1 and 2 are the indexes of Oxygen atoms and 3, 4, 5 are the indexes of Hydrogen atoms and one intends to count all possible O-H bonds, the `cv_i` list must be (1, 3, 1, 4, 1, 5, 2, 3, 2, 4, 2, 5), that is, it must explicitly list all the pairs to be counted.


```

&colvar
  cv_type = 'N_OF_STRUCTURES'
  cv_ni = 9, cv_nr = 23,
  cv_i = 1, 2, 3, 4, 0, 3, 4, 5, 0 ! the last zero is optional
  cv_r = 1.0, 1.0, 1.0, ! group #1, atom 1
        2.0, 2.0, 2.0, ! group #1, atom 2
        3.0, 3.0, 3.0, ! group #1, atom 3
        4.0, 4.0, 4.0, ! group #1, atom 4
        1.0,           ! R0 for group #1
  23.0, 23.0, 23.0, ! group #2, atom 3
        4.0, 4.0, 4.0, ! group #2, atom 4
        5.0, 5.0, 5.0, ! group #2, atom 5
        2.0           ! R0 for group #2
/

```

Figure 26.6.: An example of `N_OF_STRUCTURES` variable.**HANDEDNESS:**

$$\text{value} = \sum_a \frac{\mathbf{u}_{a,3} \cdot [\mathbf{u}_{a,1} \times \mathbf{u}_{a,2}]}{|\mathbf{u}_{a,1}| |\mathbf{u}_{a,2}| |\mathbf{u}_{a,3}|},$$

where

$$\begin{aligned} \mathbf{u}_{a,1} &= \mathbf{r}_{a+1} - \mathbf{r}_a \\ \mathbf{u}_{a,2} &= \mathbf{r}_{a+3} - \mathbf{r}_{a+2} \\ \mathbf{u}_{a,3} &= (1-w)(\mathbf{r}_{a+2} - \mathbf{r}_{a+1}) + w(\mathbf{r}_{a+3} - \mathbf{r}_a), \end{aligned}$$

and $\mathbf{r}_a$ denote the positions of participating atoms. The `cv_i` array is supposed to contain indexes of the atoms and the `cv_r` array may provide the value of w ($0 \leq w \leq 1$, the default is zero).

N_OF_STRUCTURES:

$$\text{value} = \sum_g \frac{1 - (R_g/R_{0,g})^6}{1 - (R_g/R_{0,g})^{12}},$$

where the sum runs over groups of atoms, R_g denotes the RMSD of the group g w.r.t. some reference coordinates and $R_{0,g}$ are positive parameters measured in Å. The `cv_i` array is expected to contain indexes of participating atoms with zeros separating different groups. The elements of the `cv_r` array are interpreted as the reference coordinates of the first group followed by their corresponding R_0 ; then followed by the reference coordinates of the atoms of the second group, followed by the second R_0 , and so forth. To make the presentation clearer, let us consider the example presented in Fig. 26.6. The atomic groups and reference coordinates are the same as the ones shown in Fig. 26.5. Lines 7 and 11 in Fig. 26.6 contain additional entries that set the values of the threshold distances $R_{0,1}$ and $R_{0,2}$. To compute the variable, the code first computes the mass weighted RMSD values R_1 and R_2 for both groups –much like in the `MULTI_RMSE` case– and then combines those in a manner similar to that used in the `N_OF_BONDS` variable.

$$\text{value} = \frac{1 - (R_1/R_{0,1})^6}{1 - (R_1/R_{0,1})^{12}} + \frac{1 - (R_2/R_{0,2})^6}{1 - (R_2/R_{0,2})^{12}}.$$

In other words, the variable “counts” the number of structures that match (stay close in RMSD sense) with the reference structures.

QUATERNIONS: Describing large-scale atomistic conformational changes in biomolecular systems requires one

to deal with orientational changes of atomistic domains with large numbers of atoms. While there are several ways of defining a collective variable that quantifies an orientation based conformational change, the orientation quaternion technique[658–661] has proven successful as a well-behaved, flexible method for defining system-specific CVs, specifically aimed at inducing interdomain orientational changes or restraining the orientation of certain domains. The CVs in the orientation quaternion class, are all derived from an ‘optimal rotation’ between a set of reference coordinates $\mathbf{X}_k$ ($1 \leq k \leq N$; where N is the number of atoms involved) and the set of target coordinates $\mathbf{Y}_k$. A ‘quaternion’ is introduced as a four-component vector that can be expressed as $q_0 + q_1\hat{i} + q_2\hat{j} + q_3\hat{k}$ where q_0 and $q_1\hat{i} + q_2\hat{j} + q_3\hat{k}$ are called scalar and vector parts respectively. The optimal rotation can be parametrized² by a unit quaternion, $\hat{q} = (q_0, q_1, q_2, q_3)$, that minimize $\langle \|\hat{q}\mathbf{X}_k\hat{q}^* - \mathbf{Y}_k\|^2 \rangle$ in which $\langle . \rangle$ denotes an average over k , q^* is the conjugate of q and $\|\mathbf{q}\|^2 = q^*q$ (see Ref.[658] for more details). The optimal rotation unit quaternion (or orientation quaternion) $\hat{q}$ can be written as $(\cos(\theta/2), \sin(\theta/2)\hat{u})$, where θ is the optimal rotation angle and $\hat{u}$ is a unit vector associated with the optimal axis of rotation. To deal with large atomistic conformational changes, a set of quaternion-based CVs has implemented in AMBER20. For the details of usage, send emails to Ashkan Fakharzadeh (afakhar@ncsu.edu), Dr. Feng Pan (fpan3@ncsu.edu), and Prof. Mahmoud Moradi (moradi@uark.edu). The specific quaternion-based CVs implemented are: ORIENTATION_ANGLE, ORIENTATION_PROJ, TILT, SPINANGLE, QUATERNION0, QUATERNION1, QUATERNION2, and QUATERNION3.

Orientation (QUATERNION0,...,QUATERNION3): These define the orientation of several atoms with respect to a set of reference coordinates in terms of a unit quaternion vector $\hat{q} = (q_0, q_1, q_2, q_3)$ according to the method introduced in Ref.[658, 659]. These variables return the best-fit rotation, also used in best-fit RMSD calculation procedures, to superimpose the coordinates $\mathbf{X}$ onto a set of reference coordinates $\mathbf{X}_0$. The unit quaternion $\hat{q} = (q_0, q_1, q_2, q_3)$ can be written as $(\cos(\theta/2), \sin(\theta/2)\hat{u})$, where θ is the rotation angle and $\hat{u}$ is a unit vector associated with the axis of rotation; for example, a rotation of 90° around the z axis $(0, 0, 1)$ is expressed as $(\cos(90^\circ/2), 0.0, 0.0, \sin(90^\circ/2)) = (\sqrt{2}/2, 0, 0, \sqrt{2}/2)$. The components of the unit quaternion (q_0, q_1, q_2, q_3) were implemented separately as QUATERNION0, QUATERNION1, QUATERNION2, and QUATERNION3 CVs. To find the orientation, all four CVs QUATERNION0,...,QUATERNION3 are being used. To calculate the quaternion CVs one needs to specify a list of participating atoms and also their reference coordinates. The reference coordinates may be passed to AMBER either via direct specification inside the CV call, or by passing the name of a reference coordinates file. It is recommended that if the set of participating atoms is small (say no larger than 15), then these are specified directly inside the CV call. Otherwise, the passing of information via filename is recommended since these lists may contain hundreds if not thousands of atoms. Relevant parameters pertaining to the input of this information are: *cv_ni* represents the number of participating atoms; *cv_i* represents the list of the indices of all participating atoms; *cv_r* represents the reference coordinates (when passed directly) and *refcrd_file* is the filename for the reference coordinates when they are to be read from file. The file *refcrd_file* should be an AMBER coordinates/restart file containing coordinates, velocities, etc. of all atoms. The list participating atoms, *cv_i*, and their reference coordinates (*cv_r* and or *refcrd_file*) must be the same for all QUATERNION0,...,QUATERNION3. The CVs are linked together using an attribute ‘*q_index*’. The ‘*q_index*’ accepts an integer between 1,...,100, where its default value is one. The Fig. 26.7 is an example of Quaternion CVs syntax. An example this type of CVs is presented in the Fig. 26.8. Two set of orientations are defined here: each set consists of QUATERNION0,..., QUATERNION3. The first set comprises 18 atoms with indexes 11,41,48,74,104,... and another one of 24 atoms with indexes 12,16,46,55,75,... A file, ‘inpcrd’ is used as an AMBER coordinate/restart file to read reference coordinates. There is no need to set ‘*q_index*’ for the first four quaternions since the default value is one, but it is set to be 2 for all quaternion CVs in the second set to link and normalize them. The returned value of each QUATERNION0,...,QUATERNION3 CVs is the corresponding component of the unit orientation vector $\hat{q} = (q_0, q_1, q_2, q_3)$.

ORIENTATION_ANGLE: The angle of rotation $\theta = 2\cos^{-1}(q_0)$ between the current and the reference positions. This angle is between 0° to 180° . The *cv_i* list is interpreted as a list of indexes of participating atoms.

$$\text{orientation angle: } \theta = 2\cos^{-1}(q_0)$$

²Assuming both sets have been already shifted to bring their barycenters to the origin (optimum translation).

```

&colvar
  cv_type = 'QUATERNION0'

  ! number of participating atoms
  cv_ni = ni

  ! index of participating atoms
  cv_i = a1, a2, ..., aN

  ! AMBER coordinate/restart file to read reference coordinates
  refcrd_file = 'refcrd_file'

  ! number of references which must be 3*ni; Should not be set if
  ! refcrd_file is being used
  cv_nr = nr

  ! reference coordinates of participating atoms; Should not be set if
  ! refcrd_file is being used
  cv_r = alx, aly, alz, a2x, a2y, a2z, a3x, a3y, a3z, ...

  ! an arbitrary integer between 1 to 100
  q_index = n
/

```

Figure 26.7.: Syntax of Quaternion reaction coordinates.

ORIENTATION_PROJ: The cosine of the angle of rotation θ between the current and the reference positions. While ORIENTATION_ANGLE diverges near $\theta = 0$, because of $\nabla_{\mathbf{x}} \theta$, ORIENTATION_PROJ might be used instead to apply forces. The range of ORIENTATION_PROJ is $[-1, 1]$. The cv_i array is supposed to contain indexes of the atoms.

$$\text{orientation proj: } 2q_0^2 - 1$$

SPINANGLE: Angle of rotation ϕ around a given unit axis $\hat{\mathbf{e}}$. The axis $\hat{\mathbf{e}}$ is being used to decompose a complete orientation rotation in two sub-rotations, spin ϕ and tilt ω . An advantage of this decomposition is ϕ and ω have the same values, regardless of which one is applied first (in comparison to Euler angles methods). The participating atoms with indexes are given in the cv_i . The 'axis' must provide three components of the axis³ $\hat{\mathbf{e}}$ in $\text{\AA}^\circ$. The default axis of rotation is (0.0, 0.0, 1.0). The range of SPINANGLE is between $[-180 : 180]$ degrees. The reference coordinates are specified either via cv_r or $refcrd_file$.

$$\text{spin angle: } \phi = 2 \tan^{-1}(\mathbf{q} \cdot \mathbf{e} / q_0)$$

where $\mathbf{q}$ is the vector part of quaternion, namely (q_1, q_2, q_3) . An example of SPINANGLE cv is presented in Fig. 26.9. The same atoms as example one are used, but the axis of rotation is set to be 'x-axis'. The reference coordinates are given by cv_nr , cv_r options.

TILT: Cosine of the rotation orthogonal to an unit given axis. The tilt angle ω , shows a rotation away from the direction $\hat{\mathbf{e}}$. The tilt combined with the 'spin' sub-rotation provides the complete orientation rotation of a group of atoms. Similar to ORIENTATION_PROJ, to avoid the discontinuity around 0° and 180° , the cosine of the tilt is implemented instead of the tilt angle itself, so that derivatives are continuous almost everywhere. The cv_i and 'axis' are the participating atoms with indexes and the given axis, respectively. The reference coordinates are specified either via cv_r or $refcrd_file$. The value of TILT is between -1 to 1 , where the value 1 represents an orientation fully parallel to $\hat{\mathbf{e}}$ ($\omega = 0^\circ$), and the value -1 represents an anti-parallel

³The axis is from the origin(0.0,0.0,0.0) to that point.

orientation.

$$\text{tilt: } t = \cos(\omega) = 2 \left(\frac{q_0}{\cos(\frac{\tan^{-1} \mathbf{q} \cdot \mathbf{e}}{q_0})} \right)^2 - 1$$

26.4.3. Steered Molecular Dynamics

The `&smd` namelist, if present in the MDIN file, activates the steered MD code (the method itself is extensively described in the literature: see for example Ref. [662] and references therein). The prefix NFE appears in several switches to do with steered MD: this stands for “Non-equilibrium Free Energy”.

The following is recognized within the `&smd` namelist:

output_file sets the output file name. Default is `'nfe-smd.txt'`.

output_freq sets the output frequency (in MD steps). Default is 50.

cv_file sets the collective variable file name. Default is `'nfe-smd-cv'`.

There must be at least one reaction coordinate defined (that is, there must be at least one `&colvar` namelist in the `cv_file`). The steered MD code requires that additional entries be present in the `&colvar` namelist:

path the steering path whose elements must be real numbers. The `path` must include at least two elements. The upper limit on the number of entries is 20000. The elements define Catmull-Rom spline used for steering.

npath sets the number of elements in `path`. Default is 0.

path_mode The way steering paths are constructed. There are two modes available. In `SPLINE` mode (default) the path is approximated by a spline that passes through the given points; in `LINES` mode the path is represented by the line segments joining the control points.

harm specifies the harmonic constant. If a single number is provided, e.g., `harm = 10.0`, then it is constant throughout the run. If two or more numbers are provided, e.g., `harm = 10.0, 20.0`, then the harmonic constant follows a Catmull-Rom spline built upon the provided values.

nharm sets the number of elements in `harm`. Default is 0.

harm_mode The way harmonical paths are constructed, similar with `path_mode`.

An example of MDIN file and CV.IN file for steered MD is shown in Fig. 26.10. The reaction coordinate is defined in `cv.in`. The spring constant is set constant throughout the run and the steering path is configured from 5.0 to 3.0. The values of the reaction coordinate, harmonic constant and the work performed on the system are requested to be dumped to the `smd.txt` file every 50 MD steps.

26.4.4. Umbrella sampling

To activate the umbrella sampling code, the `&pmd` namelist must be present in the MDIN file. `&pmd` is currently available to both SANDER and PMEMD, and also can be fully applied in GPU accelerated PMEMD. The `output_file`, `output_freq` and `cv_file` entries are recognized just as in the steered MD case presented earlier. The `cv_file` must contain at least one `&colvar` namelist section. For umbrella sampling, the `&colvar` section(s) must contain two additional entries:

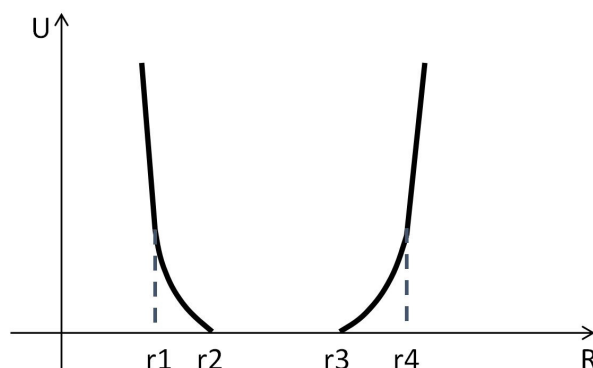
anchor_position: this consists of four real numbers (r_1, r_2, r_3, r_4) that determine the rectangle of the umbrella (harmonic) potential. The default value is that all of the r 's is set to zero.

anchor_strength: two non-negative real numbers (k_1, k_2) that set the harmonic constant for the umbrella (harmonic) potential. The default value is zero.

The umbrella (harmonic) potential U is determined by (supposing R is the value of reaction coordinate)

- $U = k_1 * (r_1 - r_2) * R \quad (R \leq r_1)$
- $U = 0.5 * k_1 * (R - r_2)^2 \quad (r_1 < R \leq r_2)$
- $U = 0 \quad (r_2 < R \leq r_3)$
- $U = 0.5 * k_2 * (R - r_3)^2 \quad (r_3 < R \leq r_4)$
- $U = k_2 * (r_4 - r_3) * R \quad (R > r_4)$

A plot of the umbrella potential is shown below



eg1: if $r_2 = r_3$, $r_1 \ll r_2$ and $r_4 \gg r_3$, then the generated U is simply the traditional harmonic potential.

eg2: if r_1 is slightly less than r_2 and r_4 is slightly larger than r_3 , also with very large k_1, k_2 , the reaction coordinate is restrained in the range (r_2, r_3) with no potential added.

An example of an MDIN file and CV.IN file for an umbrella sampling simulation is shown in Fig. 26.11. The first reaction coordinate here is the angle formed by the lines joining the 5th with 9th and 9th with 15th atoms. It is to be harmonically restrained near 1.0 rad (anchor_position entry) using the spring of strength 10.0 kcal/mol/rad² (anchor_strength entry). The second reaction coordinate requested in Fig. 26.11 is a dihedral angle (type = 'TORSION') formed by the 1st, 2nd, 3rd and 4th atoms (the cv_i array). It is to be restrained near zero with strength 23.8 kcal/mol/rad². The values of the reaction coordinate(s) are to be dumped every 50 MD steps to the pmd.txt file. Another example of restraining reaction coordinate in a specific range is shown in Fig. 26.12. The reaction coordinates here are φ and ψ angles of dialanine. φ is restrained between -2.0 rad and 2.0 rad, ψ is restrained between -1.8 rad and 1.8 rad.

The NFE implementation of umbrella sampling works correctly with the Amber standard replica-exchange MD described earlier in this manual (compatible with different types of REMD for different values of -rem flag in both SANDER and PMEMD). For example, the typical umbrella sampling with Hamiltonian Replica Exchange can be performed by setting -rem to 3. In this case, both anchor_position and anchor_strength may be different for different temperatures. Even the number and type of reaction coordinate(s) could vary for different replicas. The output files (set by the output_file keyword on a per-replica basis) are MDIN-bound, consistent with -rem.

26.4.5. Adaptively Biased Molecular Dynamics

The implementation has a very simple and intuitive interface: the code is activated if either an &abmd (both SANDER and PMEMD) or an &bbmd (both SANDER and PMEMD) namelist is present in the MDIN file (the difference between those “flavors” is purely technical and will become clear later). Unlike in the &smd and &pmd cases, the dimensionality of a reaction coordinate (the number of &colvar namelists in the cv_file) cannot exceed five (though three is already hardly useful due to statistical reasons).

As previously noted, in order to activate the ABMD and related algorithm, the variable *infe* in &cntrl must be set to unity (i.e. *infe* = 1; default value *infe* = 0).

In addition to the cv_file entry, the following entries are recognized within the &abmd (or &bbmd) namelist:

mode sets the execution mode. There are three modes available: 'ANALYSIS' | 'UMBRELLA' | 'FLOODING'. In ANALYSIS mode the dynamics is not altered. The only effect of this mode is that the value(s) of the reaction coordinate(s) is(are) dumped every `monitor_freq` to `monitor_file`. In UMBRELLA mode, biasing potential from the `umbrella_file` is used to bias the simulation ($\tau_F = \infty$, biasing potential does not change). In FLOODING mode the adaptive biasing is enabled.

monitor_file sets the name of the file to which value(s) of reaction coordinate(s) (along with the magnitude of biasing potential in FLOODING mode) are dumped.

monitor_freq the frequency of the output to the `monitor_file`.

timescale τ_F , the flooding timescale in picoseconds (only required in FLOODING mode).

umbrella_file biasing potential file name (the file must exist for the UMBRELLA mode).

In FLOODING mode, the following two entries are optional:

snapshots_basename sets the name of the file to which the biasing potential is dumped during the simulation for snapshot.

snapshots_freq the frequency of dumping snapshot biasing potential (in MD steps). If `snapshots_freq` is not specified, the snapshot biasing potential will not be dumped.

and the `&colvar` namelist for `&abmd` method must also contain the following entries:

cv_min smallest desired value of the reaction coordinate (required, unless the reaction coordinate is limited from below).

cv_max largest desired value of the reaction coordinate (required, unless the reaction coordinate is limited from above).

resolution the "spatial" resolution for the reaction coordinate.

To access the biasing potential files created in the course of FLOODING simulations, the `nfe-umbrella-slice` utility is provided (it prints a short description of itself if invoked with `--help` option).

The multiple-walker selection algorithm can improve the simulation by resampling between different walkers. The well-tempered ABMD can lead to a smoother convergence to the desired free energy. These two algorithm are implemented to SANDER and PMEMD from Amber16 onwards.

The multiple-walker selection algorithm currently works with `&abmd` only. The algorithm should be used only within the multiple-walker scheme (*i.e.*, when command-line `-rem` flag is set to zero). The following entries are recognized regarding with the selection algorithm (selection algorithm can work with FLOODING and UMBRELLA mode):

selection_freq positive integer number that sets the frequency of the resampling algorithm (in MD steps). If `selection_freq` is not specified, the selection algorithm will not be used.

selection_constant positive real number that sets the parameter C . if `selection_freq` is specified, specifying `selection_constant` is required (no default value). Parameter C is to determine how strong the selection mechanism is. If C is too large, all the walkers will be replaced with the most dominant one. If C is too small, there will be no killing/duplicating of walkers.

selection_epsilon positive real number (typically less than unity) that sets the stopping criterion parameter ϵ . Parameter ϵ determines the threshold for stopping the selection algorithm. If `selection_epsilon` is not specified, there will be no stop to the algorithm. If `selection_epsilon` is equal or larger than one, the algorithm will be stopped after the first attempt.

The well-tempered flavor can be used within either `&abmd` or `&bbmd` namelist. There are two entries relevant to the well-tempered feature:

wt_temperature positive real number that sets the pseudo-temperature T' . If this flag is not specified, conventional ABMD will be used (*i.e.*, $T' \rightarrow \infty$ or $\beta' \rightarrow 0$). The smaller the T' ; the smoother/slower the convergence.

wt_umbrella_file the file name of true biasing potential after modification by $1 + (T/T')$ in which T is the reference temperature of the system (`temp0`).

An example MDIN file and CV.IN file for the `&abmd` flavor of ABMD is shown in the Fig. 26.13.

In this example, the reaction coordinate is defined as the distance between the 5th and 9th atoms (more than one reaction coordinates might be requested by mere inclusion of additional `&colvar` subsections). The mode is set to `FLOODING` thus enabling the adaptive biasing with flooding timescale $\tau_F = 100ps$. The region of interest of the reaction coordinate is specified to be between $-1 \text{ \AA}$ and $10 \text{ \AA}$ and the resolution is set to $0.5 \text{ \AA}$. The lower bound ($-1 \text{ \AA}$) could have been omitted for `DISTANCE` variable: the default value of zero would be used in such case. The code will try to load the biasing potential from the `umbrella.nc` file and use it as the value of $U(t|\xi)$ at the beginning of the run. The biasing potential built in the course of simulation will be saved to the same file (`umbrella.nc`) every time the `RESTART` file is written. The selection algorithm is used with the frequency of selection defined as 10000 MD steps and selection constant defined as 0.001. The well-tempered algorithm is also used, with the pseudo-temperature defined as 10000 K in and the true biasing potential will be dumped as `wt_umbrella.nc` file. The `nfe-umbrella-slice` utility can then be used to access its content. An MDIN file for the follow up biased run at equilibrium would look much like the one shown in the Fig. 26.13, but with mode changed from `FLOODING` to `UMBRELLA`.

Driven ABMD can be performed using `&smd` block (for the SMD part of the algorithm) along with `&abmd` block (for the ABMD part of the algorithm). There is no additional flag for the `&smd` block relevant to the algorithm; however, there are two additional flags to ABMD relevant to the “driven” feature.

driven_weight string that sets the weighting scheme. The default option (*i.e.*, not using the flag) is `NONE` which indicates no reweighting is used (NOT RECOMMENDED if SMD is performed along ABMD). Other options include `CONSTANT` and `PULLING` for constant and pulling reweighting protocols.

driven_cutoff positive real number that sets a cutoff for work for applying the reweighting algorithm (default: 0.0). If the work (accumulated or transferred depending on the scheme) at any given time is lower than the cutoff, no reweighting is done at that particular time. If the cutoff is too small, it may result in instability of the algorithm.

For both SANDER and PMEMD since Amber18, the `&abmd` code works correctly with Amber replica-exchange similar with `&pmd` (that is, for `-rem` flag set to different values). If `-rem` is set to 3, ABMD with replica-exchange is carried out. In such case different replicas can have different temperatures, collective variables and even different mode. The monitor and umbrella files are MDIN-bound. If number of `sander` groups exceeds one (the flag `-ng` is greater than one) and `-rem` flag is set to zero, the code runs *multiple walkers* ABMD. In both cases the number and type(s) of variable(s) must be the same across all replicas.

Finally, the `&bbmd` flavor allows one to run replica-exchange (AB)MD with different reaction coordinates and different modes (`ANALYSIS`, `UMBRELLA` or `FLOODING`) in different replicas (along with different temperatures, if desired). This module is outdated since `&abmd` has been compatible with `-rem` equals 3. The only advantage of `&bbmd` is that the number of replicas can be odd numbers if desired by runs, while this cannot be achieved in any `-rem` types. To applying `&bbmd` module, the `-rem` flag must be set to zero and the `&bbmd` sections must be present in all MDIN files. The MDIN file for the replica of rank zero (first line in the group file) is expected to contain additional information as compared to `&abmd` case (an example of such MDIN file for replica zero is shown in Fig. 26.14). The MDIN files for all other replicas except zero do not need any additional information, and therefore take the same form as in the `&abmd` flavor (except that the namelist is changed from `&abmd` to `&bbmd`, thus activating a slightly different code path). Each MDIN file may define its own reaction coordinates, have different mode and temperature if desired.

Within the first replica `&bbmd` namelist the following additional entries are recognized:

exchange.freq number of MD steps between the exchange attempts.

exchange.log.file the name of the file to which exchange statistics is to be reported.

exchange.log.freq frequency of `exchange.log.file` updates.

mt19937.seed seed for the random generator (Mersenne twister [663]).

mt19937.file the name of the file to which the state of the Mersenne twister is dumped periodically (for restarts).

The MDOUT, MDCRD, RESTRT, `umbrella.file` and `monitor.file` files are MDIN-bound in course of the bbmd-enabled run. An example that uses this kind of replica exchange is presented in Ref. 636.

26.4.6. Swarms-of-Trajectories String Method

ABMD is a robust method for calculating free energy landscapes as a function of a small number of collective variables. Since the required computer time grows enormously with the number of collective variables, ABMD is best for exploring one- or two-dimensional phase spaces. However, rather than calculating full n -dimensional free energy maps, it is often fruitful to focus on the so-called Minimum Free Energy Path (MFEP) which the system is likely to take when transitioning between two minima. Calculating a MFEP in a complicated phase space is often difficult, and so-called "string methods"[664][654] represent one of the best approaches for finding the MFEP. Since sampling in string methods is essentially limited to regions around the MFEP, the cost of the method scales linearly with the length of the string or path, but only weakly on the number of collective variables. This results in considerable computational savings since the full free energy landscape is not calculated.

The swarms-of-trajectories string method (STSM)[654] is one of the most popular versions of the string method and has been implemented here by Dr. Moradi (moradi@uark.edu). The module is available in both SANDER and PMEMD from Amber18 onwards. It is a path-finding algorithm that refines a putative transition pathway iteratively until the path is deemed to have been converged. The string is defined by a number of nodes or images parameterized in a high-dimensional space of collective variables, whose position is updated iteratively. The center of each image is first used as a restraining center to generate representative conformations at the current center before allowing of a small change in this center for the next iteration. The change in the center of each image is estimated by averaging over the drifts of a swarm of short unbiased trajectories all starting at the current image position (generated using the constrained simulations). Thus, each iteration consists of a series of restrained and free simulations. In the current serial version of the code, these simulations are performed independently. In parallel versions -- which are more efficient -- a very large number of replicas is required which are run in parallel; this method is particularly efficient on large supercomputers.

To invoke the swarms-of-trajectories string method, the `&stsm` must be invoked in the MDIN file. For a string consisting of N_s nodes each requiring M copies $N_s \times M$ replicas will be required. The parallel implementation of the STSM method is based on iterative restrained and free MD simulations followed by a reparametrization of the image centers defined in a multidimensional collective variable space ξ . For the i^{th} iteration, first M copies of the n^{th} image are generated around the old center ξ_n^{i-1} by MD equilibration lasting τ_E timesteps. The generated M copies of the n^{th} image are expected to be close to ξ_n^{i-1} for time τ_E , assuming that the invoked harmonic constant k for the restraining potential is large enough. The parameters τ_E and k thus need to be appropriately chosen in order to ensure that all copies of each image will be close to the image center. The restraint is then released, and each copy (swarm) is allowed to drift for τ_R timesteps. The newly shifted center ξ_n^i for the n^{th} image is then determined by averaging over all drifted copies $\xi_{n,m}^i$ at time $t = \tau_E + \tau_R$. The resulting string of images is then smoothed using a linear interpolation protocol. A smoothing parameter ε with $0 \leq \varepsilon \leq 1$ determines the smoothness of the curve; it is recommended that ε be of the order of $1/(N_s - 1)$. The last setep is a reparameterization, which gain follows a linear interpolation protocol in order to generate N_s equidistant centers along the string. The two key parameters of the method are M and τ_R . Generally, the large the M and the shorter τ_R , the smoother (but slower) the evolution of the MFEP will be. These variables must be optimized empirically, but typically 10 - 30 copies and 5 - 20 ps are reasonable values. It is often advantageous to set $\tau_E = \tau_R$.

An improved sequential repeat version of the algorithm has also been implemented, which avoids the large number of copies and does not require a large number of processors to run. Here a new variable N_R is introduced,

as the number of repeat runs for each replica. Now for each copy, it will run around the old center ξ_n^{i-1} for N_R times sequentially. And each repeat run can be equally considered as a parallel run of a new copy around the old center. Namely, the new shifted center will be determined by averaging on $N_R \times M$ copies. So the number of processors needed will be reduced to $1/N_R$, while the running time will be multiplied by N_R .

The following are recognized within the `&stsm` namelist:

- image** positive integer number that sets the image id (between 1 and N). Default is 0.
- repeats** positive integer number that sets the number of repeat runs, should be the same for each image and each copy. Equal to parallel implementation when not set. Default is 1.
- equilibration** non-negative integer number that sets the number of MD steps specified for biased equilibration (restraining) at each iteration. Default is 0.
- release** Number of MD steps specified for the release (drift) at each iteration. Note: the total number of iterations is determined by the total simulation time (`nstlim` flag in `mdin` file) divided by total time for each iteration given by `equilibration+release`.
- smoothing** positive number that sets the smoothing parameter for reparametrization (between 0 and 1). Smoothing parameter should be, preferably, on the order of $1/(N_s - 1)$. If this flag is not used, no smoothing will be performed.
- report_centers** a string that determines if drifted and/or smoothed and/or reparametrized centers will be reported. The default value is `NONE` and other available options include `ALL`, `DRIIFT`, `SMOOTHED`, `REPARAMETRIZED`, `NO_DRIIFT`, `NO_SMOOTHED`, `NO_REPARAMETRIZED`.

The `output_file`, `output_freq` and `cv_file` entries are recognized just as `&smd` and `&pmd`, the information of reaction coordinates will be read from `cv_file`. The number of collective variables can not exceed five. (here be attention that the `anchor_position` and `anchor_strength` will be defined using the traditional harmonical potential, different with `&pmd`!). An example of `MDIN` file and `CV.IN` file for STSM in parallel case is shown in Fig. 26.15. Here we run 8 images along the path, with `I` defining the image ID. We run 980 MD steps for equilibration and 20 MD steps release at each iteration, so there are totally 1000 MD steps for each iteration. With `nstlim` set to 10000, 10 iterations will be carried out. The smoothing parameter is set to 0.1 and all the centers will be reported. For each image, 16 copies will be run in parallel, with `J` defining the copy ID. The evolution of reaction coordinate will recorded in the file `stsm.00I.J.txt`. For this run, at least $128(8 \times 16)$ processors are needed. Another example of `MDIN` file of equivalent sampling level in sequential case is shown in Fig. 26.16. Here we still have 8 images to run. We set the number of repeats to be 16, namely 16 repeat runs for each image to get the new drifted center. Therefore, 16000 MD steps are needed for one iteration, and so we set `nstlim` to 160000 to complete 10 iterations. For this run, 8 processors are needed at least.

Part of sample `MDOUT` file is shown in Fig. 26.17. The restoring restraint part will be only in sequential run, since the restraint needs to be restored after each repeat. The values of reaction coordinates before reporting centers are the averaged value over repeats for this copy and the instantaneous value. All the centers will be reported only in the `MDOUT` file of first copy of first image. The drifted centers are the averaged value over copies, and also the smoothed and reparametrized centers can be reported. Always the reparametrized centers will be extracted to draw the MFEP in the phase space.

26.4.7. Implementation in PMEMD

From Amber16 and onwards, the above features have been implemented in PMEMD. Thus, users can now apply GPUs to substantially improve the speed of free energy sampling calculations with `pmemd.cuda` and `pmem.cuda.MPI`. This is very important for studying systems in explicit solvent. With `pmemd` and `pmemd.MPI` the option to use CPUs is also kept, and the whole method has been tested successfully in both implicit and explicit environments. If you have questions with regards to the PMEMD implementation of ABMD and related algorithms, please contact Dr. Feng Pan <fpan3@ncsu.edu> and Dr. M. Moradi <moradi@uark.edu>.

Several changes have been made compared with the previous version before Amber16 in order to make the modules easier and more friendly to use, and some functionality has been modified. Here are all the changes:

- Naming was changed, all the input format is changed to namelist style. The reaction coordinate (collective variable) information is read from a separate file so it can be reusable by different runs.
- A new variable *infe* is added within the namelist &cntrl, to control the usage of the NFE method in a friendly way. To disable the NFE method, set *infe* to 0; to enable it, set *infe* to 1.
- The restraint potential contributions from different modules are shown in MDOUT file. And Amber replica-exchange methods with `-rem` set to different values are compatible. (through update.8 of Amber18)
- Two new entries have been added to the blocks &abmd and &bbmd to give snapshots of the biasing potential during the simulation:

snapshots_basename = STRING : sets the snapshots file name.

snapshots_freq = INTEGER : sets the snapshot frequency. (in MD steps)

- Two new algorithms have been added to both SANDER and PMEMD: (1) a selection algorithm for multiple-walker ABMD; (2) the well-tempered ABMD (WT-ABMD).
- The swarms-of-trajectories string method (STSM) has been added to SANDER and PMEMD (GPU compatible).
- Several new reaction coordinates are added, which include
 - type = SIN_OF_DIHEDRAL
 - type = PAIR_DIHEDRAL
 - type = PATTERN_DIHEDRAL
 - type = DF_COM_DISTANCE
- For customizing your own reaction coordinate (collective variable), please check the online tutorial <https://ambermd.org/tutorials/advanced/tutorial31/index.html>.

26.4.8. Post-processing of biasing potential

When you get the biasing potential (*.nc file), you can always use the `nfe-umbrella-slice` utility to access its content and get a friendly-written ASCII file from which one can obtain the free energy map. The output is the free energy value, which is the opposite of the biasing potential ($f = -U$ (units kcal/mol)). The `nfe-umbrella-slice` utility has been included in AmberTools.

Usage: `nfe-umbrella-slice [options] bias-potential.nc`

Options:

-h, --help Print out a usage summary

-p, --pretend Only print out the basic properties of source without biasing potential data (off by default)

-g, --gradient Print out the gradients (off by default)

-r, --reset Set the value of minimum to zero (off by default)

-t, --translate Translate the numerical value of biasing potential by a real number (0 by default)

-d, --dimensions Set the way of slice in different dimensions. The format is “D1:D2:...:Dn”, where n is the number of dimensions. Each D can only be set with one number or three numbers separated by commas. If only one number is set, the variable will be fixed at that value. If three numbers are set, the first two define the boundary of the slice and the last one defines the number of points.

Example:

```
• nfe-umbrella-slice -r -d ``-5.0,5.0,50'' 1d-bias.nc > FE.dat
```

This processes the 1-dimensional biasing potential file `1d-bias.nc` and prints out the results to `FE.dat`. The minimum of free energy will be set to zero. The variable will be taken from -5.0 to 5.0 using 50 points.

```
• nfe-umbrella-slice -g -t 50.0 -d ``1.0:-2.0,2.0,20'' 2d-bias.nc >
  FE.dat
```

This processes the 2-dimensional biasing potential file `2d-bias.nc` and prints out the results to `FE.dat`. All the free energy will be incremented by a constant 50.0. The gradients in both dimensions will be printed out. For the first dimension, the variable will be fixed at 1.0; for the second dimension, the variable will be taken from -2.0 to 2.0 using 20 points.

```
• nfe-umbrella-slice wt_umbrella.nc > wt_FE.dat
```

This processes the biasing potential after WT-ABMD and prints out the results to `wt_FE.dat`. The default dimensional information is obtained and used by the program from the biasing potential file.

26.5. Steered Molecular Dynamics (SMD) and the Jarzynski Relationship

26.5.1. Background

SMD applies an external force onto a physical system, and drives a change in coordinates within a certain time. Several applications have come from Klaus Schulten's group.^[665] An implementation where the coordinate in question changes in time at constant velocity is coded in this version of Amber. The present implementation has been done by the group of Prof. Dario Estrin in Buenos Aires <dario@q1.fcen.uba.ar> by Marcelo Marti <marcelomarti@yahoo.com> and Alejandro Crespo <alec@qi.fcen.uba.ar>, and in the group of Prof. Adrian Roitberg at the University of Florida <roitberg@ufl.edu>.^[666]

The method should be thought of as an umbrella sampling where the center of the restraint is time-dependent as in:

$$V_{rest}(t) = (1/2)k[x - x_0(t)]^2$$

where x could be a distance, an angle, or a torsion between atoms or groups of atoms.

This methodology can be used then to drive a physical process such as ion diffusion, conformational changes and many other applications. By integrating the force over time (or distance), a generalized work can be computed. This work can be used to compute free energy differences using the so-called Jarzynski relationship.^[667–669] This method states that the free energy difference between two states A and B (differing in their values of the generalized coordinate x) can be calculated as

$$\exp(-\Delta G/k_B T) = \langle \exp(-W/k_B T) \rangle_A \quad (26.26)$$

This means that by computing the work between the two states in question, and averaging over the initial state, equilibrium free energies can be extracted from non-equilibrium calculations. In order to make use of this feature, SMD calculations should be done, with different starting coordinates taken from equilibrium simulations. This can be done by running `sander` multiple times, or by running `multisander` (Section 22.12). There are examples of the various modes of action under the `test/jar` directories in the Amber distribution.

26.5.2. Implementation and usage

To set up a SMD run, set the `jar` variable in the `&cntrl` namelist to 1. The change in coordinates is performed from a starting to an end value in `nstlim` steps.

To specify the type and conditions of the restraint an additional ".RST" file is used as in *nmropt=1*. (Note that *jar=1* internally sets *nmropt=1*.) The restraint file is similar to that of NMR restraints (see Section 30.1), but fewer parameters are required. For instance, the following RST file could be used:

```
# Change distance between atoms 485 and 134 from 15 A to 20 A
&rst iat=485,134, r2=15., rk2 = 5000., r2a=20. /
```

Note that only *r2*, *r2a* and *rk2* are required; *rk3* and *r3* are set equal to these so that the harmonic restraint is always symmetric, and *r1* and *r4* are internally set so that the restraint is always operative. An SMD run changing an angle, would use three *iat* entries, and one changing a torsion needs four. As in the case of NMR restraints, group inputs can also be used, using *iat<0* and defining the corresponding groups using the *igr* flag.

The output file differs substantially from that used in the case of nmr restraints. It contains 4 columns: $x_0(t)$, x , force, work. Here work is computed as the integrated force over distances (or angle, or torsion). These files can be used for later processing in order to obtain the free energy along the selected reaction coordinate using Jarzynski's equality.

Example

The following example changes the distance between two atoms along 1000 steps:

```
Sample pulling input
&cntrl
nstlim=1000, cut=99.0, igb=1, saltcon=0.1,
ntpr=100, ntwr=100000, ntt=3, gamma_ln=5.0,
ntx=5, irest=1, ig = 256251,
ntc=2, ntf=2, tol=0.000001,
dt=0.002, ntb=0, tempi=300., temp0=300.,
jar=1,
/
&wt type='DUMPFREQ', istep=1, /
&wt type='END', /
DISANG=dist.RST
DUMPAVE=dist_vs_t
LISTIN=POUT
LISTOUT=POUT
```

Note that the flag *jar* is set to 1, and redirections to the *dist.RST* file are given. In this example the values in the output file *dist_vs_t* are written every *istep=1* steps.

The restraint file *dist.RST* in this example is:

```
# Change distance between atoms 485 and 134 from 15 A to 20.0 A
&rst iat=485,134, r2=15., rk2 = 5000., r2a=20.0, /
```

and the output *dist_vs_t* file might contain:

```
15.00000 15.12396 -1239.55482 0.00000
15.00500 14.75768 2470.68119 3.07782
15.01000 15.13490 -1246.46571 6.13835
15.01500 15.15041 -1350.03026 -0.35289
15.02000 14.77085 2481.56731 2.47596
15.02500 15.12423 -987.34073 6.21152
15.03000 15.18296 -1520.41603 -0.05787
15.03500 14.79016 2431.22399 2.21915
.....
19.97000 19.89329 4.60255 67.01305
19.97500 19.87926 4.78696 67.03652
19.98000 19.86629 4.54839 67.05986
19.98500 19.85980 3.75589 67.08062
19.99000 19.86077 2.58457 67.09647
19.99500 19.86732 1.27678 67.10612
```

In this example, the work of pulling from 15.0 to 20.0 (over 2 ps) was 67.1 kcal/mol. One would need to repeat this calculation many times, starting from different snapshots from an equilibrium trajectory constrained at the initial distance value. This could be done with a long MD or a REMD simulation, and postprocessing with ptraj to extract snapshots. Once the work is computed, it should be averaged using Eq. 26.26 to get the final estimate of the free energy difference. The number of simulations, the strength of the constraint, and the rate of change are all important factors. The user should read the appropriate literature before using this method. It is recommended that the width of the work distribution do not exceed 5-10% for faster convergence. In many cases, umbrella sampling may be a better way to estimate the free energy of a conformational change.

26.6. Absolute Free Energies using EMIL

As well as comparing two similar systems to find a free energy difference, thermodynamic integration techniques can be used to find the absolute free energy, integrating between an all-atom AMBER model and a simplified model for which the free energy can be directly written down. To find a chemical equilibrium, pairs or sets of absolute free energies must of course be compared to find free energy differences, but taking this “long way around” can be better if the direct integration path between the systems would involve a sharp energetic barrier or a large conformational change. The basic equation of EMIL is thus:

$$A = A_{ref} - \int_0^1 d\lambda \left\langle \frac{\partial \mathcal{H}}{\partial \lambda} \right\rangle_{|\lambda}$$

where A is the total free energy of a system, A_{ref} is the (analytically calculated) free energy of the associated EMIL Hamiltonian and $\mathcal{H}$ is the mixed Hamiltonian, which has the value of the normal AMBER Hamiltonian at $\lambda = 0$ and the EMIL Hamiltonian at $\lambda = 1$.

The method was introduced in the literature with demonstrations for example systems with short-range interactions [670, 671], and an example AMBER calculation for the B-Z conformational equilibrium of DNA also exists [672]. Some further discussion of accuracy and convergence of EMIL calculations using AMBER has also been made [673]. To call EMIL, set “*emil_do_calc*” = 1 in the main input file, and also prepare an EMIL-specific input file (by default called “emilParameters.in”).

It is advised to use a Langevin thermostat (*ntt*=3) (section 22.6.7) with a fairly high value of *gamma_ln*, (e.g. 1.0) because dynamics under the EMIL Hamiltonian can have little coupling between particles, therefore an external source of randomness is desirable in order to drive sampling. Use of generalized-Langevin thermostats (section 25.1) is consistent with EMIL, however no study has been made to ascertain the benefits of this approach. Use of SHAKE with EMIL can give unphysical results, so it is advised to turn this off (*ntc* = 1, *ntf* = 1, *dt* = 0.001).

The letters “EM” in EMIL refer to “Einstein Molecule”, the name given in the literature [674, 675] to this type of calculation. The use of EMIL is an alternative to other AMBER methods of finding the absolute free energy of molecules in implicit solvent, such as by combining a normal modes analysis (see section 37.12.12) and MM(PB/GB)SA (see Chapter 39). EMIL is quite likely to be more computationally expensive than this type of post-hoc estimate of the free energy carried out after a normal MD simulation, but is also in some ways simpler and is likely to be more accurate in the limit of a large amount of computation being available.

Periodic boundaries can be applied, although EMIL does not support non-rectilinear boxes.

When carrying out an EMIL integration, the AMBER part of the Hamiltonian is gradually turned off with increasing *lambda*. To help achieve this without artifacts, the *emil_sc* option is available (pmemd only) which allows mutual softcoring of all interatomic forces (see section 26.1, eqn 26.5). For EMIL softcoring there is no need to specify a softcoring mask or modified topology file, as all atoms are included in the process, however *icfe* = 1 and *ifsc* = 1 must be set if *emil_sc* = 1. The value of *clambda* must also be set, to whatever *lambda* is also specified in the *emil_paramfile*. When using *emil_sc* with the default value of *klambda* (=1) (eqn 26.4), there may be sharp changes in the generalized force near to *lambda*=*clambda*=1. In this case it is advised to have an integration point at *lambda*=*clambda*=0.99 or a similar value so that the endpoint behaviour is not over-weighted in the total calculation.

When *emil_sc*=0 (the default), a less sophisticated approach to the problem of discontinuities in the Lennard-Jones and Coulomb potentials is taken: a short-range repulsion is automatically added to the Hamiltonian for the intermediate stages of the integration $0 < \lambda < 1$ in order to prevent atoms from approaching within the

problematic regions of the scaled LJ and Coulomb interactions. This method is not always entirely effective, especially in explicit solvent calculations, and may require a cut in the timestep for some values of *lambda*.

EMIL is compatible with multisander and multiplmemd (section 22.12), however the only benefit currently is to collect together runs at multiple values of *lambda* for submission as a single job: H-REMD methods (sec 26.3.5) and other advanced uses of multisander and multiplmemd have not yet been implemented.

Based on benchmarking studies it is now recommended for explicit water calculations to use pmemd with ifsc=1. Optimal parameters were found in this case to be: *epsilonWell* 1., *epsilonTrap* 0.5, *rWell* 0.5, *rTrap* 5.0 in *emilParameters.in*, and *scalpha*=0.3, *scbeta*=16.0 in the *mdin*. Optimal parameters may be different for other systems, so if you are planning a large calculation you should first carry out short runs (1ps) with different parameter values comparing the variance and apparent smoothness of the generalised force measurements for a small change up and down of each free parameter. The EMIL calculations in the test set can be used as a starting point when setting up a new calculation, although real runs will be much longer and require many more *lambda* points.

26.6.1. EMIL Namelist Input

An EMIL-specific namelist of input and output filenames for EMIL should be provided in the main input file, of the form:

```
&emil_cntrl
emil_paramfile = "emilParameters.in",
emil_logfile   = "emil.log",
emil_model_infile = "wellsIn.dat",
emil_model_outfile = "wellsOut.dat",
/
```

The variables *emil_paramfile* and *emil_logfile* are paths to files for control data and logging specific to the EMIL calculation. The variable *emil_model_infile* gives the path to an initial specification for an analytically tractable model and *emil_model_outfile* points to a saved model state. If these variables are not set then an initial model will be automatically generated, and no output model will be saved.

26.6.2. EMIL parameter input

The “emilParameters.in” file contains setup info specific to the EMIL calculation. The file is formatted as a list of key-value pairs, one per line. Blank lines or those beginning with a “#” are ignored. The keys are case-insensitive. Providing that you are running at 300K with a fairly standard forcefield, only the *seed*, *lambda*, *liquidRes* and *solidRes* values should need to be changed.

The input keys which can be used are:

seed *integer* seed for EMIL’s random number generator

lambda *real* mixing parameter for the alchemical transformation. Must be equal to *sclambda* if *emil_sc*=1.

epsilonWell *real* Depth of harmonic restraints. This is in units of $k_B T$, so that the wells are automatically deeper if the temperature increases. The value of $\beta = 1/k_B T$ at the start of the simulation is printed in the *emil_logfile*. Harmonic restraints are assigned to atoms of residues in the *solidRes* list and have a potential of the form

$$V(\mathbf{r}) = \epsilon(r^2/r_{\text{well}}^2 - 1).$$

rWell *real* The radius of a harmonic restraint, such that the potential is zero.

epsilonTrap *real* Depth of ‘trap’ restraints (in units of $k_B T$). Trap restraints are assigned to atoms of residues in the *liquidRes* lists (if any) and have a potential which is harmonic on $0 \leq r \leq reqTrap$ and then has a constant force on $reqTrap < r < rTrap$. Beyond *rTrap* the force exerted by a trap well is zero.

reqTrap *real* The radius of the harmonic region of a trap well. Trap wells need to have (at least) a small harmonic region in order to increase the stability of the dynamics near to the bottom of the well.

rTrap *real* The total radius of a trap well.

wingForce *real* The force in the constant-force region of a trap well (in units of $k_B T / \text{\AA}$).

solidRes *string* The list of residues for which each atom is permanently assigned to a specific harmonic well.

liquidRes *string* A list of residues which are part of a fluid of chemically identical molecules, for which the chain-well assignment can be adjusted at each timestep by Monte Carlo sampling. Multiple liquids can be defined, in the case that different sets of indistinguishable chains are present in liquid or dissolved phases. Chains whose residues are in these lists are assigned to trap wells, but chains can exchange wells with their neighbours based on a Metropolis acceptance criterion. In each liquid chain only one atom (the heaviest atom is chosen automatically, so this would be the oxygen of a TIP3P water) interacts directly with the trap well; the remainder of the atoms in the chain have a harmonic well generated for them which holds them in an approximately constant relative position to the 'root' atom of the chain.

swapTriesPerChain *float* Monte Carlo attempt rate for moves that exchange the trap wells between particles in the *liquidRes* lists. The use of swap moves can greatly accelerate convergence, but can also create problems if the acceptance rate (printed in the *emil.logfile*) is zero or close to zero for any value of λ .

relocTriesPerChain *float* Monte Carlo attempt rate for moves that move particles in the *liquidRes* lists (typically solvent or salt molecules) into or out of their wells. Even if this value is nonzero, relocation moves are only applied if the AMBER Hamiltonian is fully mixed out.

saveWellsEvery *integer* Period with which to write the well positions.

printEvery *integer* Period with which to log the generalized force. The average over the previous non-printed timesteps is output.

Here is an example input file for a fairly standard EMIL run using *pmemd* and *emil.sc*:

```
##EMIL input configuration: this is a comment.

##emil has its own RNG
seed                2325

##set the Mixing parameter:
## you will need several values on the interval [0,1].
lambda              0.0

##Residue names associated with wells.
##This is the list of residues needed for duplex DNA
##you will have to extend/change it for your own system
solidRes    DC,DG,DA,DT,DA5,DT5,DA3,DT3,DG5,DG3,DC5,DC3
liquidRes   WAT
liquidRes   NA
liquidRes   CL

swapTriesPerChain 0.1

##timesteps between writing well positions
saveWellsEvery 100000

##timesteps between output of generalized force
printEvery     1000
```


26.6.3. EMIL generalized-force output

EMIL writes its output to the `emil_logfile`. This logfile contains some header information, and data to monitor the progress of the run, but the important lines are of the following format:

```
nstep: 25 soft_dHdL: 2.06419354e+04 molec_dHdL: ...
...6.13140526e+04 abstr_dHdL: -5.34856062e+02
```

The step number, *nstep*, indicates the timestep at which the printout was made. The *soft_dHdL* is the generalized force due to the weak and short-range repulsive term which is present in the mixed Hamiltonian for values of $0 < \lambda < 1$, but only if *emil_sc*=0. The *molec_dHdL* is the generalized force due to the AMBER Hamiltonian, and the *abstr_dHdL* is the generalized force due to the EMIL Hamiltonian. The gradient of the total Hamiltonian with respect to *lambda* is just the sum of these three terms. In order to make the most efficient use of information, EMIL accumulates a mean value of each generalized force term between printouts, so the value written is not an instantaneous “snapshot” but the average over a time window *printEvery* steps in length.

Although the EMIL Hamiltonian is specified in units of $k_B T$, the generalized force is output in units kcal/mol, so the strength of the restraints (and the size of the generalized force) will increase with temperature.

26.6.4. EMIL tractable model definition

The model defined by EMIL is currently very simple. Each atom of any residues in the list *solidRes* from “`emilParameters.in`” is restrained to a fixed position using a harmonic well of depth *epsilonSolid*, with the zero of the potential at distance *rWellSolid*. The position of the harmonic well minimum is fixed at whatever the atom position at the start of the run might be, unless the option *readStartWellFileName* is provided, in which case the positions are read in from the file.

Atoms defined by the *liquidRes* lists have wells with a finite range, and in order to have faster convergence for simulations including explicit solvent (where the particle-well distance can otherwise be very large at small λ) the particle-well assignment is shuffled at each timestep by Monte Carlo sampling. The MC method is not currently implemented in parallel, which can create limitations for EMIL calculations using large numbers of cores per value of λ : the optimal parallelisation strategy in this case is to make many runs on few cores each, at different values of λ .

Derivations and formulae for the free energy associated with each well type are available in the supplementary data of [672], however the calculated totals are also printed out at the start of the *emil_logfile*.

Use of thermostat synchronisation to reduce errorbars

A feature of the Langevin thermostat which can cause serious problems in other circumstances (discussed in [493]) is that simulations run with the same seed will come to resemble each other, even if the Hamiltonians and initial configurations are somewhat different. A surprising benefit of this is that, if EMIL is used to compare two or more dissimilar systems then the variance of the difference in the generalized forces at a given value of λ can be less than the sum of the variances of the individual measurements:

$$\text{VAR} \left[\frac{\partial \mathcal{H}(x_1, \lambda)}{\partial \lambda} - \frac{\partial \mathcal{H}(x_2, \lambda)}{\partial \lambda} \right] < \text{VAR} \left[\frac{\partial \mathcal{H}(x_1, \lambda)}{\partial \lambda} \right] + \text{VAR} \left[\frac{\partial \mathcal{H}(x_2, \lambda)}{\partial \lambda} \right] \quad (26.27)$$

which is to say that, although the means of the two generalized forces are estimated correctly, the covariance of the two generalized forces is greater than zero. Using this phenomenon it is possible to estimate the difference in free energies between two (or N) systems more cheaply than the free energies themselves [672, 673, 676].

While it is therefore beneficial to use the same seeds for a given value of λ across all systems, it is still necessary to use a new seed for each restart of the same trajectory, and to use different seeds for different values of λ . To maintain thermostat synchronization, the number of atoms in the different systems must be the same. This can be achieved if necessary by the addition of non-interacting dummy atoms to the smaller topology files using the *parmed* (sec. 15.2) utility.

Brief instructions for an EMIL calculation

To run an EMIL calculation, first equilibrate a single simulation of the system in question then follow the steps below:

1. If you started off at constant pressure, find the average box-size and scale the system to this size.
2. Prepare multiple "emilParameters.in" files (see section 26.6.2) which differ from each other only in the parameters *seed* and *lambda*. The values of *lambda* should be spread over the interval $0 \leq \lambda \leq 1$.
3. Put your "emilParameters.in" files into one directory each and run *pmemd* in each of the directories, setting *ntt* = 3, *ntp* = 0, *emil_do_calc*=1, *emil_sc*=1. If runs finish and are restarted, then the saved well positions written at the end of the old run will need to be loaded into the new one, as well as the normal AMBER restart files.
4. It may be necessary to set up restraints of some kind from within *pmemd* or *sander* if the free energy to be calculated is for only a subset of the available conformations of the molecule(s), or to speed up convergence at low values of λ , by preventing the solute molecule from drifting away from its restraint system (this drift is a particular problem for small systems, where the cumulative effect of the EMIL solute restraints, even over all atoms, is still weak at small λ).
5. Collect the converged time-average values of the generalized forces (or the differences in generalized forces if you are comparing several systems) at each value of λ . It is often worth looking at the different time series individually, in order to make the most efficient use of data by only throwing away the minimum number of equilibration points, and in order to target simulation effort to those values of λ which are taking the longest to give a small errorbar [672]).
6. Do a numerical integration of each of the three *dHdL* terms from the EMIL logfiles with respect to λ then subtract these totals from the free energy of the EMIL Hamiltonian, which is printed in the headers of the EMIL logfiles, to get the free energy of the system under the AMBER Hamiltonian. As well as taking time-averages of the (delta) generalized forces and then integrating these values, it may also be valuable to collect a time-series of the (delta) free energy values and examine this total for convergence.

A longer tutorial on the use of EMIL is available on the AMBER website, also the examples in the test suite might provide some help to get started.

26.7. Nonequilibrium sampling and Jarzynski's equation

It was shown by Jarzynski [667, 677] that an equilibrium change in free energy, $\Delta F = F(\lambda = 1) - F(\lambda = 0)$, can be expressed as an exponential average of the work, W , performed on the system during an irreversible transformation between two states. [678]

$$\exp(-\beta\Delta F) = \langle \exp(-\beta W) \rangle \quad (26.28)$$

$$W = \int_{t=0}^{\tau} \frac{\partial H(\mathbf{r}(t); \lambda(t))}{\partial \lambda} \frac{\partial \lambda}{\partial t} dt \quad (26.29)$$

The angle brackets denote an average taken over the starting state ensemble; i.e., the $\lambda = 0$ state. In other words, to estimate ΔF , one first simulates the $\lambda = 0$ state to obtain N statistically independent samples. For each sample, one performs a simulation of length τ that dynamically varies λ (from $\lambda_{\min}$ to $\lambda_{\max}$) to evaluate a work value, W . The N work values are exponentiated, and the free energy is calculated from the (minus) natural log of the averaged exponential.

In practice, the time integration is discretized $\tau = \Delta t \times \text{nstlim}$, where Δt is the time step and *nstlim* is the number of steps. If one assumes that the masses are independent of λ , then $H(\mathbf{r}(t); \lambda(t))$ can be replaced with $U(\mathbf{r}(t); \lambda(t))$,

26. Free energies

and the work can be expressed by the following summation. The time steps are indexed from 0 to $n_{\text{stlim}}-1$, such that the elapsed time at step n is $t = n\Delta t$.

$$W = \sum_{n=0}^{n_{\text{stlim}}-1} \frac{\partial U(\mathbf{r}(n\Delta t); \lambda(n\Delta t))}{\partial \lambda} [\lambda(n\Delta t + \Delta t) - \lambda(n\Delta t)] \quad (26.30)$$

The derivative is the thermodynamic gradient normally used to evaluate the free energy from thermodynamic integration; however, it only contributes to the nonequilibrium work when λ changes between consecutive time steps. In sander, the time dependence of λ is controlled with the `dynlmb` and `ntave` variables. Specifically, the value of λ is incremented by `dynlmb` once every `ntave` steps. If `nstlim` is an integer multiple of `ntave`, then λ will be incremented a total of $N_{\text{incr}} = n_{\text{stlim}}/\text{ntave}$ times, and each increment increases λ by $\text{dynlam} = (\lambda_{\text{max}} - \lambda_{\text{min}})/N_{\text{incr}}$. One can rewrite the summation to include only the subset of samples contributing a nonzero value to the work (i.e., the samples whose time step is a multiple of `ntave`).

$$W = \frac{\lambda_{\text{max}} - \lambda_{\text{min}}}{N_{\text{incr}}} \sum_{m=1}^{N_{\text{incr}}} \frac{\partial U(\mathbf{r}(m \times \text{ntave} \times \Delta t); \lambda(m \times \text{ntave} \times \Delta t))}{\partial \lambda} \quad (26.31)$$

The sander program has several options that one may find useful for performing nonequilibrium sampling. Many of these options have been previously discussed in the context of equilibrium sampling with thermodynamic integration (Sec. 26.1). When performing nonequilibrium sampling, it is recommended to set a new option: `ijarzynski=1`, as discussed below. Although it is possible to perform the nonequilibrium sampling and analysis without setting `ijarzynski=1`, the purpose of this option is to help eliminate mistakes one can easily make when setting the other input parameters. Specifically, setting `ijarzynski=1` will enable the following actions.

1. It will automatically choose an appropriate value of `dynlmb`, if `dynlmb` was unset.
2. It performs additional consistency checks between the input parameters; e.g., `ntave > 0`, `mod(nstlim, ntave) == 0`, `clambda + dynlmb * nstlim / ntave <= 1.0` (or 0.995), etc.
3. It will average the appropriate subset of DV/DL values and report the work at the end of the simulation.

The following is an overview for calculating a free energy using Jarzynski's equation with sander.

- Equilibrate the $\lambda = 0$ state and perform production sampling. The production sampling input file should set `ntwx > 0` and `ntwv=-1` to periodically save both the coordinates and velocities to the trajectory file.
- To perform nonequilibrium sampling, start by preparing 2 `mdin` files and a `groupfile` in the manner that one would normally use for equilibrium sampling with thermodynamic integration. Set `icfe=1` in both files to turn on the free energy simulation capabilities of multisander. For MM→QM/MM transformations, the two `mdin` files would only differ by their values of `ifqnt` (0 or 1). If the QM bond lengths are not constrained by shake, then the MM `mdin` file should include those atoms in `noshakemask`. For alchemical MM transformations, the two `mdin` files would only differ by their values of `crgmask` and/or `scmask`. If the transformation does not involve the use of softcore potentials (`ifsc=0`), then set `clambda=0`. Alternatively, set `clambda=0.005` if `ifsc=1` due to the limitations of sander's softcore potential. The value of `clambda` within the `mdin` file corresponds to λ_{min} .
- Modify each `mdin` file by setting the options `ijarzynski=1` and `ntave > 0`. By setting `ijarzynski=1`, an appropriate value of `dynlmb` will be automatically determined from the values of `nstlim`, `ntave`, `clambda`, and `ifsc`. If one followed the instructions as written, the value of `clambda` will scale from 0.0 to 1.0 (if `ifsc=0`) or from 0.005 to 0.995 (if `ifsc=1`). If you need `clambda` to vary according to a different schedule, then you will need to manually set the value of `dynlmb`.
- You have two options for running each of the N simulations with multisander.
 - Use `cpptraj` to extract each frame from the trajectory file and save them as restart files. Each simulation can then be run independently.

- Alternatively, set the `imin=6` option in the `&cntrl` section and tell multisander to read the trajectory with the `-y` command-line option (this flag is set on each line of the groupfile). Each frame of the trajectory file will effectively be treated as a restart file and a simulation will be performed for each frame. Because N may be very large, it may be better to use `cpptraj` to split the trajectory file into several smaller trajectory files. Each trajectory file can then be run with `imin=6`. This approach exploits the inherent parallelization of the nonequilibrium sampling while avoiding the need to manage several thousand restart and `mdout` files.
- After the simulation has completed, the following line will be printed (if `ijarzynski=1`). This is the final value of the work (in kcal/mol) calculated from equation 26.31. The collection of work values are used to estimate the free energy.

```
Final work value:          1.539753
```

```

&colvar
  cv_type = 'QUATERNION0'
  cv_ni = 18,
  cv_i = 11 , 41 , 48 , 74 , 104 , 111 , 137 , 167 , 174 , 199 , 229 , 236, 262 ,
          292 , 299 , 325 , 355 , 362 ,
  refcrd_file = 'inpcrd'
/
&colvar
  cv_type = 'QUATERNION1'
  cv_ni = 18,
  cv_i = 11 , 41 , 48 , 74 , 104 , 111 , 137 , 167 , 174 , 199 , 229 , 236, 262 ,
          292 , 299 , 325 , 355 , 362 ,
  refcrd_file = 'inpcrd'
/
&colvar
  cv_type = 'QUATERNION2'
  cv_ni = 18,
  cv_i = 11 , 41 , 48 , 74 , 104 , 111 , 137 , 167 , 174 , 199 , 229 , 236, 262 ,
          292 , 299 , 325 , 355 , 362 ,
  refcrd_file = 'inpcrd'
/
&colvar
  cv_type = 'QUATERNION3'
  cv_ni = 18,
  cv_i = 11 , 41 , 48 , 74 , 104 , 111 , 137 , 167 , 174 , 199 , 229 , 236, 262 ,
          292 , 299 , 325 , 355 , 362 ,
  refcrd_file = 'inpcrd'
/
&colvar
  cv_type = 'QUATERNION0'
  cv_ni = 24,
  cv_i = 12 , 16 , 46 , 55 , 75 , 79 , 109 , 118 , 138 , 142 , 172 , 181 , 200 ,
          204 , 234 , 243 , 263 , 267 , 297 , 306 , 326 , 330 , 360 , 369 ,
  refcrd_file = 'inpcrd',
  q_index = 2
/
&colvar
  cv_type = 'QUATERNION1'
  cv_ni = 24,
  cv_i = 12 , 16 , 46 , 55 , 75 , 79 , 109 , 118 , 138 , 142 , 172 , 181 , 200 ,
          204 , 234 , 243 , 263 , 267 , 297 , 306 , 326 , 330 , 360 , 369 ,
  refcrd_file = 'inpcrd',
  q_index = 2
/
&colvar
  cv_type = 'QUATERNION2'
  cv_ni = 24,
  cv_i = 12 , 16 , 46 , 55 , 75 , 79 , 109 , 118 , 138 , 142 , 172 , 181 , 200 ,
          204 , 234 , 243 , 263 , 267 , 297 , 306 , 326 , 330 , 360 , 369 ,
  refcrd_file = 'inpcrd',
  q_index = 2
/
&colvar
  cv_type = 'QUATERNION3'
  cv_ni = 24,
  cv_i = 12 , 16 , 46 , 55 , 75 , 79 , 109 , 118 , 138 , 142 , 172 , 181 , 200 ,
          204 , 234 , 243 , 263 , 267 , 297 , 306 , 326 , 330 , 360 , 369 ,
  refcrd_file = 'inpcrd',
  q_index = 2
/

```

```

&colvar
  cv_type = 'SPINANGLE'
  cv_ni = 18, cv_nr = 54,
  cv_i = 11 , 41 , 48 , 74 , 104 , 111 , 137 , 167 , 174 , 199 , 229 ,
          236 , 262 , 292 , 299 , 325 , 355 , 362 ,
  cv_r = 0.96 , -4.47 , -0.31 , 3.48 , -3.00 , 3.06 , 0.88 , 0.01 ,
          3.36 , 4.55 , -0.51 , 6.46 , 3.93 , 2.38 , 9.81 , 0.26 ,
          0.84 , 10.12 , 1.90 , 4.16 , 13.21 , -1.06 , 4.47 , 16.58 ,
          -0.71 , 0.52 , 16.88 , -0.96 , -4.47 , 17.21 , -3.48 ,
          -3.00 , 13.84 , -0.88 , 0.01 , 13.54 , -4.55 , -0.51 , 10.44 ,
          -3.93 , 2.38 , 7.09 , -0.26 , 0.84 , 6.78 , -1.90 , 4.16 ,
          3.69 , 1.06 , 4.47 , 0.32 , 0.71 , 0.52 , 0.02 ,
  axis = 1.0, 0.0, 0.0
/

```

Figure 26.9.: An example of *SPINANGLE* variable.

```

title line
&cntrl
..., infe = 1
/

&smd
  output_file = 'smd.txt'
  output_freq = 50
  cv_file = 'cv.in'
/

```

```

cv_file
&colvar
  cv_type = 'DISTANCE'
  cv_ni = 2
  cv_i = 5, 9
  npath = 2, path = 5.0, 3.0, path_mode = 'LINES',
  nharm = 1, harm = 10.0
/

```

Figure 26.10.: An example *MDIN* file and *CV.IN* file for steered MD. Only the relevant part is shown.

```

title line
&cntrl
..., infe = 1
/

&pmd
  output_file = 'pmd.txt'
  output_freq = 50
  cv_file = 'cv.in'
/

```

```

cv_file
&colvar ! first
  cv_type = 'ANGLE'
  cv_ni = 3, cv_i = 5, 9, 15
  anchor_position = -10.0,1.0,1.0,10.0
  anchor_strength = 10.0,10.0
/
&colvar ! second
  cv_type = 'TORSION'
  cv_ni = 4, cv_i = 1, 2, 3, 4
  anchor_position = -10.0,0.0,0.0,10.0
  anchor_strength = 23.8,23.8
/

```

Figure 26.11.: An example MDIN file and CV.IN file for umbrella sampling (only relevant part is presented in full).

```

cv_file
&colvar ! phi
  cv_type = 'TORSION'
  cv_ni = 4, cv_i = 5, 7, 9, 15
  anchor_position = -2.05,-2.0,2.0,2.05
  anchor_strength = 500.0,500.0
/
&colvar ! psi
  cv_type = 'TORSION'
  cv_ni = 4, cv_i = 7, 9, 15, 17
  anchor_position = -1.85,-1.8,1.8,1.85
  anchor_strength = 500.0,500.0
/

```

Figure 26.12.: An example CV.IN file to restrain the φ and ψ of dialanine.

```

title line
&cntrl
..., infe = 1
/

&abmd
  mode = 'FLOODING'

  monitor_file = 'abmd.txt'
  monitor_freq = 33
  cv_file = 'cv.in'

  umbrella_file = 'umbrella.nc'

  timescale = 100.0 ! in ps

  selection_freq = 10000
  selection_constant = 0.001

  wt_temperature = 10000.0
  wt_umbrella_file = 'wt_umbrella.nc'
/

```

```

cv_file
&colvar
  cv_type = 'DISTANCE'
  cv_ni = 2, cv_i = 5, 9
  cv_min = -1.0, cv_max = 10.0 ! min is not needed for DISTANCE
  resolution = 0.5 ! required for mode = FLOODING
/

```

Figure 26.13.: An example *MDIN* file and *CV.IN* file for ABMD (only the relevant part is presented in full).

```

title line
&cntrl
..., infe = 1
/

&bbmd

    ! 0th replica only

    exchange_freq = 100 ! try for exchange every 100 steps

    exchange_log_file = 'bbmd.log'
    exchange_log_freq = 25

    mt19937_seed = 123455 ! random generator seed
    mt19937_file = 'mt19937.nc' ! file to store/load the PRG

    ! not specific for 0th replica

    mode = 'ANALYSIS'

    monitorfile = 'bbmd.01.txt' ! it is wise to have different
                                ! names in different replicas

    monitor_freq = 123
    cv_file = 'cv.in'
/

cv_file
&colvar
    cv_type = 'DISTANCE'
    cv_ni = 2, cv_i = 5, 9
/

```

Figure 26.14.: An example *MDIN* file and *CV.IN* file for *&bbmd* flavor of *ABMD* (only the relevant part is presented in full).


```

title line
&cntrl
..., nstlim = 10000
..., infe = 1
/

&stsm      ! parallel case, I from 1 to 8, J from 1 to 16
  image = I
  equilibration = 980
  release = 20
  smoothing = 0.1
  report_centers = 'ALL'

  output_file = 'stsm.00I.J.txt'
  output_freq = 10
  cv_file = 'cv.I'
/

```

```

cv_file
&colvar ! phi
  cv_type = 'TORSION'
  cv_ni = 4, cv_i = 5, 7, 9, 15
  anchor_position = -3.00
  anchor_strength = 20.0
/
&colvar ! psi
  cv_type = 'TORSION'
  cv_ni = 4, cv_i = 7, 9, 15, 17
  anchor_position = 3.00
  anchor_strength = 20.0
/

```

Figure 26.15.: An example *MDIN* file and *CV.IN* file for *&stsm* in parallel case (only the relevant part is presented in full)

```

title line
&cntrl
..., nstlim = 160000
..., infe = 1
/

&stsm      ! sequential case, I from 1 to 8
  image = I
  repeats = 16
  equilibration = 980
  release = 20
  smoothing = 0.1
  report_centers = 'ALL'

  output_file = 'stsm.00I.txt'
  output_freq = 10
  cv_file = 'cv.I'
/

```

Figure 26.16.: An example MDIN file for &stsm in sequential case (only the relevant part is presented in full)

```

NFE : #   restoring restraint:
NFE : #   << colvar(1) = -3.000000 >>
NFE : #   << colvar(2) = 3.000000 >>
NFE : #   equilibration begins...
.....
NFE : #   << colvar(1) = -2.500688 -2.586429 >>
NFE : #   << colvar(2) = 2.782725 3.082205 >>
NFE : #   drifted center of image 1 :           8      -2.54041796      2.70644813
NFE : #   drifted center of image 2 :           8      -2.54963153      2.71715138
.....
NFE : #   drifted center of image 8 :           8       1.02191205      0.16837852
NFE : #   smoothed center of image 1 :           8      -2.54041796      2.70644813
NFE : #   smoothed center of image 2 :           8      -2.60416697      2.75924174
.....
NFE : #   smoothed center of image 8 :           8       1.02191205      0.16837852
NFE : #   reparametrized center of image 1 :           8      -2.54041796
2.70644813
NFE : #   reparametrized center of image 2 :           8      -2.06027108
2.47738701
.....
NFE : #   reparametrized center of image 8 :           8       1.02191205
0.16837852

```

Figure 26.17.: An example of MDOUT file for STSM run (only part is presented, and some centers are also omitted)

27. Constant pH calculations

A constant pH molecular dynamics method was developed by John Mongan for simulations run with the Generalized Born implicit solvent model [679] and Jason Swails for simulations with explicit solvent [680]. Using either constant pH method requires minor modifications to the process of generating the prmtop file and also requires a second input file describing the titrating residues.

27.1. Background

Traditionally, molecular dynamics simulations have employed constant protonation states for titratable residues. This approach has many drawbacks. First, assigning protonation states requires knowledge of pKa values for the protein's titratable groups. Second, if any of these pKa values are near the solvent pH there may be no single protonation state that adequately represents the ensemble of protonation states appropriate at that pH. Finally, since protonation states are constant, this approach decouples the dynamic dependence of pKa and protonation state on conformation.

The constant pH method implemented in *sander* and *pmemd* addresses these issues through Monte Carlo sampling of the Boltzmann distribution of discrete protonation states concurrent with the molecular dynamics simulation. The protonation state distribution is affected by solvent pH, which is set as an external parameter. Residue protonation states are changed by changing the partial charges on the atoms of the protonable residue.

27.2. Preparing a system for constant pH simulation

Amber provides definitions for titrating side chains of ASP, GLU, HIS, LYS, TYR, and CYS. See below if you need other titrating groups.

Begin by preparing your PDB file as you normally would for use with LEaP. Edit the PDB file, replacing all histidine residue names (HIS, HID, or HIE) with HIP. Change all ASP and ASH to AS4 and all GLU and GLH to GL4. The others—LYS, TYR, and CYS—have the same name. This ensures that the prmtop file will have a hydrogen defined at every possible point of protonation. Note that these changes should only be applied to residues that you wish to titrate.

Run LEaP with the *leaprc.constph* command file. This file loads all parameters that were used for the reference compounds. You can load this file with the following command:

```
source leaprc.constph
```

This loads the ff10 force field. In addition, it loads the special carboxylate residue libraries and force field modifications—*constph.lib* and *frmod.constph*—that defines a hydrogen atom at each protonable location (syn- and anti- for both oxygens) along with improper torsions to prevent them from rotating into each other. It also sets the GB solvation radii (PBradii) to mbondi2, which was the set used to parameterize the reference compounds. Now load your edited PDB file and proceed as usual to create the topology and coordinates files. Changing any of the above parameters should be closely checked by titrating the reference compounds and ensuring the predicted pKa matches.

Once you have the prmtop (topology) file, you need to generate a cpin file. The cpin file describes which residues should titrate, and defines the possible protonation states and their relative energies. A python script, *cpinutil.py*, is provided to generate this file. It takes a prmtop file as input, on the command line along with the GB model you wish to evaluate protonation transitions in, and writes the cpin file. Here is an example of generating the cpin file from your prmtop file, 'prmtop' using the igb=2 GB model:

```
cpinutil.py -p prmtop -igb 2 -o cpin
```

27. Constant pH calculations

The *cpinutil.py* program accepts a number of flags that modify its behavior. By default, all residues start in protonation state 0: deprotonated for ASP and GLU, protonated for LYS, TYR, and CYS, and doubly protonated for HIS (i.e. HIP). Initial protonation states can be specified using the `-states` flag followed by a comma and/or whitespace-delimited list of initial protonation states (see below for more about protonation state definitions) as follows:

```
cpinutil.py -p prmtop -igb 2 -states 1 3 0 0 0 1 -o cpin
```

Note that if a list of states is provided, it must match exactly the number of residues that *cpinutil* has found to titrate based on the restrictions put on the command line. The `-system` flag can be used to provide a name for the titrating system. This is purely cosmetic and has no effect on your simulations.

```
cpinutil.py -p prmtop -igb 2 -system HEWL -o cpin
```

A number of flags are available for filtering which residues are included in the *cpin* file. All residues in the *cpin* file, and only the residues in the *cpin* file, will be titrated. In general it is safe to exclude TYR and LYS for acidic simulations and GL4 and AS4 for basic simulations. HIP should be included in all except very acidic simulations. Note that there is currently no support for titrating N or C terminal residues. If you have an N or C terminal residue with a titratable sidechain, you should explicitly exclude it from the *cpin* file. The `-resnum` flag may be used to specify which residue numbers should be retained; all others are deleted. Conversely, the `-notresnum` flag can be used to specify which residue numbers are deleted; all others are retained. Residue number refers to the numbering in the PDB file, not the index number among titrating residues. Similarly, `-resname` and `-notresname` can be used to filter by residue type. For instance, `-notresname TYR,LYS` would eliminate basic residues from the *cpin* file. The `-minpKa` and `-maxpKa` flags can be used to filter out residues whose reference pKas do not satisfy that criteria. For example, `-minpKa 5.0` will exclude all AS4 and GL4 residues from titrating.

The *cpin* format has changed in Amber 18, but *cpin* files in the older format compatible with Amber 16 and older versions can still be generated using the `--old-format` argument. However, simulations with `temp0` other than 300 Kelvins will not work.

You can get a full list of all available titratable residues using the `--list` argument to *cpinutil.py*, and you can get a full description of reference energies and charge vectors for any residue using the `--describe` argument. The full usage statement for *cpinutil.py* (accessible via `-h/--help`) is shown on the next page.

```

usage: cpinutil.py [Options]
optional arguments:
  -h, --help            show this help message and exit
  -v, --version          show program's version number and exit
  -d, --debug            Enable verbose tracebacks to debug this program
  -oldfmt, --old-format  Print output file in a format compatible with AMBER 16
                        and older versions

Output files:
  -o FILE, --output FILE      Output file. Defaults to standard output
  -op FILE, --output-prmtop FILE
                              For explicit solvent simulations, a custom set of
                              radii are necessary to obtain reasonable results for
                              carboxylate pKas (e.g., AS4 and GL4 residues). If
                              specified, this file will be the prmtop compatible
                              with the reference energies in the printed cpin file.

Required Arguments:
  -p FILE                  Topology file to be used in constant pH simulation

Simulation Options:
  -igb IGB                 Generalized Born model which you intend to use to
                              evaluate dynamics (or protonation state swaps).
                              Default is 2.
  -intdiel DIEL            Internal dielectric constant to use in the evaluation
                              of the GB potential. Default 1.0.

Residue Selection Options:
  -resnames [RES [RES ...]]
                              Residue names to include in CPIN file
  -notresnames [RES [RES ...]]
                              Residue names to exclude from CPIN file
  -resnums [NUM [NUM ...]]
                              Residue numbers to include in CPIN file
  -notresnums [NUM [NUM ...]]
                              Residue numbers to exclude from CPIN file
  -minpKa pKa              Minimum reference pKa to include in CPIN file
  -maxpKa pKa              Maximum reference pKa to include in CPIN file

System Information:
  -states [NUM [NUM ...]]
                              List of default states to assign to titratable
                              residues
  -system <system name>
                              Name of system to titrate. No effect on simulation.

Residue Information:
  If any options here are used, no CPIN file will be written. These
  arguments take precedence and are mutually exclusive with each other.
  --describe [RESNAME [RESNAME ...]]
                              Print out the details of given residues
  -l, --list               List all titratable residues

This program will read a topology file and generate a cpin file for constant
pH simulations with sander or pmemd

```

27.3. Running at constant pH

27.3.1. Running at constant pH in implicit solvent

Running constant pH simulations in either *sander* or *pmemd* has few differences from normal operation. In the *mdin* file, you must set *icnstph*=1 to turn on constant pH in implicit solvent. *solvph* is used to set the solvent pH value. You must also specify the period for Monte Carlo steps, *ntcnstph* (the number of steps between protonation state change attempts). Note that only one residue is examined on each step, so you should decrease the step period as the number of titrating residues increases to maintain a constant effective step period for each residue. We have seen good results with fairly short periods, in the neighborhood of 100 fs effective period for each residue (e.g. *ntcnstph*=5, *dt*=0.002 with about 10 residues titrating).

Constant pH MD techniques employ a reference (model) compound to compute relative free energy differences between the various protonation states through a thermodynamic cycle (see Figure 27.1). The free energy of the protonation state change in the model compound that is necessary to yield the correct pK_a prediction. This quantity is pre-computed for each protonation state change. This so-called *reference energy* is printed to the *cpin* file by *cpinutil.py*. In order to obtain sensible results, you *must* run your simulations with the same potential energy function for your system that was used to derive these reference energies (or alternatively rederive the reference energies with the potential you wish to use).

The reference energies were derived using the following parameters:

```
cut=30.0, igb=#, saltcon=0.1, nrespa=1,
temp0=300.0, ntc=2, ntf=2
```

where # is the value passed to the *cpinutil.py* program. In particular, care should be taken when modifying the *igb*, *saltcon*, *nrespa*, or *temp0* parameters (*nrespa* should never be changed). The cutoff, 30 Å, is effectively infinite for the (very small) model compounds, so using any reasonable cutoff—including an infinite cutoff—is valid. The *ff99SB* force field was used to parametrize the model compounds. Using other force fields should be validated before you run simulations. If the charge scheme is the same as *ff99SB* (e.g., *ff14SB*), chances are good that the reference energies will still be valid. Other force fields (e.g., *ff03* and *ff13*) that have different charge definitions require recalculating the reference energies.

The model compounds have the sequence ACE-X-NME, where ACE is a neutral acetyl capping group, X is the titratable residue, and NME is a neutral methylamine capping group. Both ACE and NME are provided in the standard Amber residue libraries.

Additional command line flags have been added to *sander* and *pmemd* to support constant pH operation. The *cpin* file must be specified using the *-cpin* option. Additionally, a history of the protonation states sampled is written to the filename specified by *-cpout*. Finally, a constant pH restart file is written to the filename specified by *-cprestart*. This is used to ensure that titrating residues retain the same protonation state when the simulation is restarted. The constant pH restart file is a *cpin*-format file, and should be used as the *cpin* file when restarting the simulation. It will generally be longer than the original *cpin* file, as it contains some amount of zeroed data. The only difference between the *cprestart* file created at the end of a simulation and the *cpin* file used to start it will be the *RESSTATES* array. Note that due to compiler-dependence of the namelist implementation, *cprestart* files may differ from computer to computer.

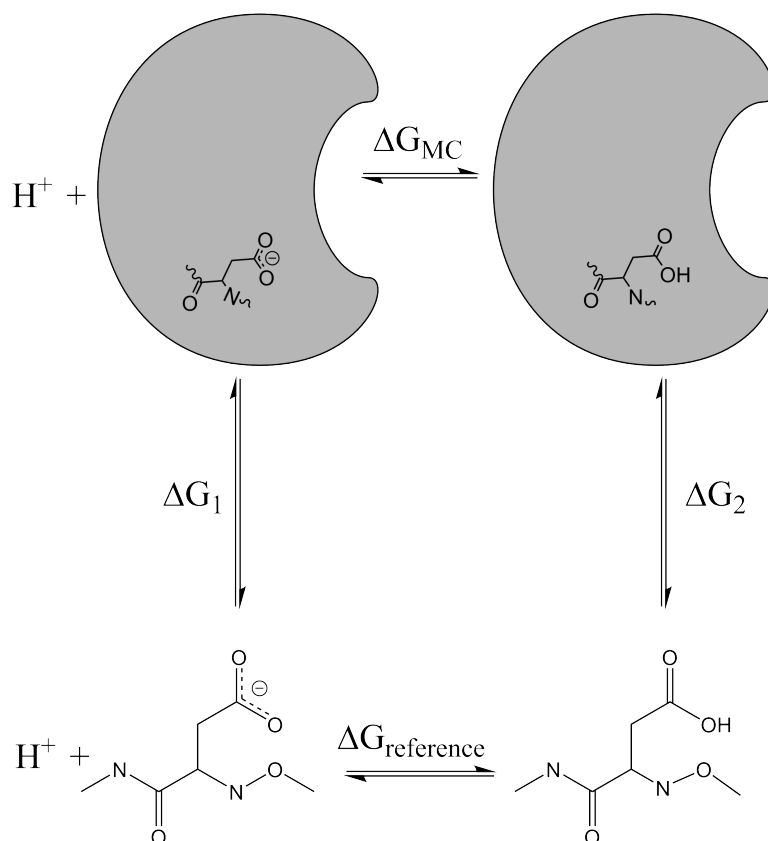


Figure 27.1.: Thermodynamic cycle used in CpHMD simulations. The energy difference between the two protonation states computed by sander is equal to the difference $\Delta G_1 - \Delta G_2$ and $\Delta G_{MC} = \Delta G_2 - \Delta G_1 - \Delta G_{reference}$ is used to evaluate the Metropolis Monte Carlo criteria for the proposed change in protonation state(s).

27.3.2. Running at constant pH in explicit solvent

The hybrid molecular dynamics/Monte Carlo technique used in the implicit solvent calculations will not work in explicit solvent because all protonation state changes will be opposed by the solvent orientation around the existing protonation states. To work around this limitation while allowing MD to be propagated in explicit solvent, protonation state changes are still attempted using a Generalized Born implicit solvent model. [680] The workflow, shown in Figure 27.2, involves running MD for `ntcnstph` steps, stripping the solvent and ions, attempting protonation state changes for each titratable residue in random order, and restoring the solvent for running solvent relaxation dynamics for `ntrelax` steps if any protonation states have changed before resuming MD.

The modifications needed to run explicit solvent simulations at constant pH are similar to the modifications needed to run implicit solvent simulations at constant pH, with some small differences highlighted here. We found that the existing GB radii defined for carboxylate oxygens is too large for the titratable residues AS4 and GL4. The reason is that the 4 hydrogen atoms in the carboxylate groups are all assigned an intrinsic solvent radius that contributes significantly to the effective radii of the carboxylate oxygens. To compensate, the intrinsic GB radii of AS4 and GL4 carboxylate oxygens must be reduced such that the effective radius is closer to the carboxylate oxygen atoms of an ASP or GLU residue. The `cpinutil.py` script that generates the `cpin` file has been modified to make the necessary changes to the topology file (which can be written with the new “-op” flag that was added for this purpose). An example command-line used to set up a constant pH simulation in explicit solvent for carboxylates is:

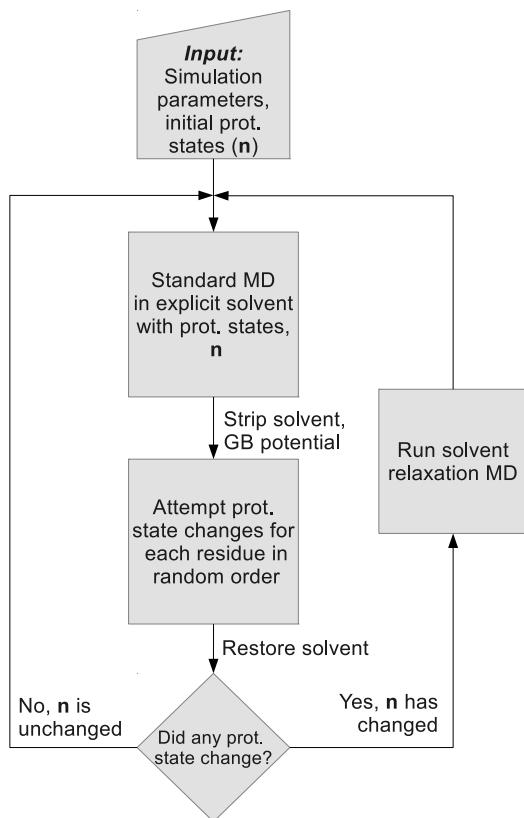


Figure 27.2.: Workflow for constant pH MD simulations in explicit solvent.

```
cpinutil.py -igb 2 -resnames AS4 GL4 -p <tleap.prmtop> -op <new.radii.prmtop>
```

In the above command, `new.radii.prmtop` is generated and must be used for constant pH simulations. In addition to the modified topology file you need for CpHMD in explicit solvent, there is an additional parameter, `ntrelax`, that defines the number of solvent relaxation steps that will be performed following successful protonation state changes. In general, we've found that while ca. 4 ps is required to generate a truly relaxed solvent distribution, 200 fs is sufficient to account for the bulk of the solvent relaxation.

Another difference with respect to implicit CpHMD simulations is that a protonation state change attempt is carried out for each residue in random order. This is done to allow protonation state change attempts to be done far less frequently to limit the amount of MD time that is consumed by the solvent relaxation dynamics. Here is an example of input variables to be used in your *sander* or *pmemd* input file.

```
icnstph=2, ntcnstph=100, ntrelax=200,  
solvph=6.4, saltcon=0.1, temp0=300.0,  
ntc=2, ntf=2
```

Notice that the value of `icnstph` is 2, which indicates that CpHMD should be run in explicit solvent. The `ntrelax` flag will run solvent relaxation dynamics (in which the non-solvent is held fixed) for 200 steps. The `saltcon` variable controls the salt concentration for the GB calculations. It has no effect on the dynamics, but is required for consistency with the reference energy of the model compound.

27.4. Analyzing constant pH simulations

As the simulation progresses, the protonation states that are sampled are written to the cpout file. A section of a cpout file from an implicit solvent simulation is included here:

```
Solvent pH: 2.00000
Monte Carlo step size: 2
Time step: 0
Time: 0.000
Residue 0 State: 1
Residue 1 State: 0
Residue 2 State: 1
Residue 3 State: 0
Residue 4 State: 1
Residue 5 State: 0

Residue 2 State: 0

Residue 4 State: 0

Residue 0 State: 3

Residue 1 State: 0

Residue 0 State: 0
```

One record is written on each Monte Carlo step. Each record is terminated by a blank line. There are two types of records: full records and delta records. Full records, like the one shown above, lists the solvent pH, MC step size, current time step, and current time before listing every residue in the system. Full records are written on the first step and every ntwx steps afterwards so as to coincide with the frames written to the trajectory. Delta records list only those residues that were titrated (single or double lines for implicit solvent or a list of every residue for explicit solvent). Note that in some cases, the protonation state for a delta record may be the same as that in an earlier record: this indicates that the Monte Carlo protonation move was rejected for that residue. The residue numbers in cpout are indices over the titrating residues included in the cpin file; cpout files must be analyzed in conjunction with the cpin to map these indices back to the original system.

The *cphstats* program can be used to perform several different analyses on the cpout files. It prints the fraction of protonated species, which can be used to compute the pK_a values of each titratable residue. The *cphstats* usage is described in Section 27.7.

27.5. Extending constant pH to additional titratable groups

There are two major components to defining a new titrating group for constant pH. First you must define the partial charges for each atom in the residue for each protonation state. Then you must set the relative energies of each state (this can be done using TI calculations or the *finddgreg.py* tool, see below).

Defining charge sets

Partial charges can be, in most cases, easily calculated using Antechamber and Gaussian. You must set up a model to calculate charges for each protonation state. If the titrating group you are defining is a polymer subunit (e.g. amino acid residue), you must adjust the charges on atoms that have bonded interactions (including 1-4) with atoms in neighboring residues. The charges on these atoms must be changed so they are the same in all protonation states - otherwise relative energies of protonation states become sequence dependent. For an amino acid, this means that all backbone atoms must have the same charges. For the residues defined here, we arbitrarily selected

the backbone charges of the protonated state to be used across all protonation states. The total charge difference between a protonated and a deprotonated state should remain 1.

Calculating relative energies

Relative energies are used to calibrate the method such that when a model compound is titrated at pH equal to its pK_a , the population of the protonated and deprotonated states are equal (e.g. fraction of protonated species equal to 50%). Relative energies of the different protonation states of a model compound can be computed using thermodynamic integration or the *finddgreg.py* tool (see Subsection 27.5.1 below). The model compound should be a small molecule that mimics the bonded environment of the titratable group of interest, and for which the experimental pK_a data is available. For instance, the model compound for an amino acid X is generally ACE-X-NME; the model compound for a ligand might be the free ligand. The thermodynamic integration or the *finddgreg.py* calculations must be performed using exactly the same parameters and force field as you plan to use in your constant pH simulations.

Testing the titratable group definitions

Prior to large scale use of your new titratable group definition, it's a good idea to test it by performing a constant pH simulation of your model compound, with pH set equal to the model pK_a . Doing this requires generation of a cpin file, and for this you need to define your titratable residue in *cpinutil.py*. These definitions are found in \$AMBERHOME/AmberTools/src/parmed/parmed/amber/titratable_residues.py. Your residue name must be added to the list `titratable_residues` at the top of this file. Add your residue definition to the bottom of the file, following the examples of the other residues (and make sure to execute the “check” function on that residue at the end as a way of checking your input). Don't forget to recompile *parmed* (or the whole AmberTools) so that your changes may take effect. It is also a good idea to use *cpinutil.py* with `--describe` to check that the charge vectors match what you meant to input—the output format using `--describe` is much easier to check than the input in *titratable_residues.py*. The definition of CYS is shown below as an example.

```
# Cysteine
refene1 = ReferenceEnergy(igb2=77.4666763, igb5=76.2588331, igb8=71.5804519)
refene1.solvent_energies(igb2=77.6041407, igb5=76.2827217)
refene1.dielc2_energies(igb2=38.090523, igb5=37.454637)
refene1.dielc2.solvent_energies(igb2=38.489170)
# Copying the reference energy to be printed on the old CPIN format
refene1_old = ReferenceEnergy(igb2=77.4666763, igb5=76.2588331, igb8=71.5804519)
refene1_old.solvent_energies(igb2=77.6041407, igb5=76.2827217)
refene1_old.dielc2_energies(igb2=38.090523, igb5=37.454637)
refene1_old.dielc2.solvent_energies(igb2=38.489170)
refene1_old.set_pKa(8.5, deprotonated=False)
refene2 = ReferenceEnergy(igb2=0, igb5=0, igb8=0)
refene2.solvent_energies(igb1=0, igb2=0, igb5=0, igb7=0, igb8=0)
refene2.dielc2_energies(igb2=0, igb5=0, igb8=0)
refene2.dielc2.solvent_energies(igb1=0, igb2=0, igb5=0, igb7=0, igb8=0)

CYS = TitratableResidue('CYS', ['N', 'H', 'CA', 'HA', 'CB', 'HB2', 'HB3', 'SG',
                                'HG', 'C', 'O'], pka=8.5, typ="ph")
CYS.add_state(protcnt=1, refene=refene1, refene_old=refene1_old, pka_corr=8.5, # protonated
              charges=[-0.4157, 0.2719, 0.0213, 0.1124, -0.1231, 0.1112, 0.1112,
                      -0.3119, 0.1933, 0.5973, -0.5679])
CYS.add_state(protcnt=0, refene=refene2, refene_old=refene2, pka_corr=0.0, # deprotonated
              charges=[-0.4157, 0.2719, 0.0213, 0.1124, -0.3593, 0.1122, 0.1122,
                      -0.8844, 0.0, 0.5973, -0.5679])

CYS.check()
```

Reference energies are those calculated from either TI (and adjusted if necessary to reproduce experimental pK_a s) or *finddgreg.py*. The reference energies should be calculated for all GB models you plan to support. If you use TI, you should always titrate your model compound to make sure that the calculated pK_a s match experiment. The reference energies obtained are typically sufficient, but some residues may require adjustments.

27.5.1. Finding reference energies with finddgreg.py

finddgreg.py is a Python tool written by Vinícius Cruzeiro that allows one to automatically compute the reference energy of a titratable residue for constant pH simulations. This is an alternative approach to TI calculations and works by directly titrating the model compound using CpHMD and making adjustments to the reference energy until a 50% fraction of protonated species is obtained at pH equal to the pK_a of the model compound. The script has two modes of execution: serial and parallel. The serial mode consists of a simple CpHMD simulation. The parallel mode consists of a pH-REMD simulation, therefore a short number of MD steps should be necessary to run the simulation. You can access a list and description of all available command-line flags using the `--help` flag, whose output is shown below.

```
usage: finddgreg.py [Options]
optional arguments:
  -h, --help            show this help message and exit
  -v, --version          show the program's version and exit
  --author              show the program's author information and exit
Required Arguments:
  -mdexec FILE          Path to the AMBER executable file. Example:
                        $AMBERHOME/bin/pmemd
Required Arguments - With Replica Exchange:
  -target FLOAT         Value of pH or Redox Potential (in Volts) that we
                        expect to obtain a converged fraction of protonated or
                        reduced species close to 50%. This is the target value
                        of the  $pK_a$  or Standard Redox Potential ( $E_o$ ) of the
                        system at the end of the execution. Default: None
Not-required Arguments:
  -do_parallel STRING   Command preceding mdexec for parallel execution. Used
                        only with Replica Exchange. Default: mpirun -np [-ng]
  -log FILE             When set, prints the log of the program execution to
                        an external file (-log FILENAME). If not set, print it
                        at the screen. Default: None
  -resnum INT           Number of the residue in which the fraction of
                        protonated or reduced species will be monitored.
                        (REQUIRED if the number of pH or Redox titratable
                        residues is larger than 1)
  -dgregfest FLOAT      Estimated value of Delta G reference. When this flag
                        is given, the program starts in the last phase of
                        execution, that is, on the phase of making more
                        accurate estimatives of Delta G reference. Note: if
                        the value of -dgregfest is not close enough to the true
                        value of Delta G reference, the execution will fail.
                        Default: None
  -dgreffrange FLOAT FLOAT
                        Range of values for Delta G reference. The desired
                        Delta G reference value has to be inside this range.
                        If -dgregfest and -dgreffrange are not given, the
                        program will try to find a range automatically.
                        Suggestion: choose one value in which the fraction of
                        protonated or reduced species is ~ 0 and the other
                        value in which it is ~ 1. Default: None
  -dginterval FLOAT     When the values of the argument -dgreffrange are to be
                        found automatically, dginterval is the interval of
```

```

trial values. Default: 100.0 kcal/mol
-maxsteps INT      Maximum number of AMBER executions. Default: 100
-fractthreshold FLOAT Fraction threshold. The fraction convergence criterium
is:  $0.5 - \text{fractthreshold}/2 \geq \text{frac} \geq 0.5 + \text{fractthreshold}/2$ . Default: 0.03
-noequi           If stated, the equilibration simulation for a new
DELTAGREF value will not be performed. Equilibration
runs for 10% the number of steps of the production
simulation. Default: False
-rmouts           If stated, at the end of the execution of the program,
erases all output files generated by AMBER (all files
not stated as REQUIRED at "AMBER Arguments" below).
Default: False
-bin-path FILE     Path to the AMBER bin directory. Used to locate
cphstats, cestats or fitpkao.py (Example:
$AMBERHOME/bin ; Default: not set).

AMBER Arguments - Without Replica Exchange:
These are the arguments to be executed together with mdexec.
-i FILE           AMBER mdin file (REQUIRED)
-p FILE           AMBER parmtop file (REQUIRED)
-c FILE           AMBER inpcrd (input coordinates) file (REQUIRED)
-x FILE           AMBER mdcrd (output coordinates) file
-inf FILE         AMBER mdinfo file
-o FILE           AMBER mdout (log) file
-r FILE           AMBER mdout file
-cpin FILE        AMBER cpin file (REQUIRED if cein file is not given)
-cpout FILE       AMBER cpout file
-cprestrt FILE    AMBER cprestrt file
-cein FILE        AMBER cein file (REQUIRED if cpin file is not given)
-ceout FILE       AMBER ceout file
-cerestrt FILE    AMBER cerestrt file
-ref FILE         AMBER ref file

AMBER Arguments - With Replica Exchange:
These are the arguments to be executed together with do_parallel and
mdexec.
-ng INT           Number of groups/replicas (REQUIRED)
-groupfile FILE   AMBER groupfile file (REQUIRED)

This program will perform constant pH or constant redox potential simulations
in order to find the value of Delta G reference (DELTAGREF) that gives around
50% fraction of protonated or reduced species for a given residue at a given
target pH or redox potential. In order to run the program, you need to replace
at least one of the values at the STATENE flag by DELTAGREF on your cpin or
cein file.

```

Serial mode (CpHMD)

The first step the input files as if you were to run a regular constant pH MD simulation of your model compound. In your input files, make sure of two things: 1) that the `solvph` variable inside your mdin file is equal to the pK_a of your model compound; 2) that the total number of MD steps is long enough to ensure convergence. This includes preparing a cpin file. For that, even without having the reference energies yet, you do need to change the source code so `cpinutil.py` in order to generate your cpin file (see Section 27.5 for instructions). At this point, the values you chose for the reference energies for your residue inside the source code are irrelevant. Once your cpin file is generated, you need to change the STATENE flag inside it. The STATENE flag contains information about the reference energies of each state. The PROTCNT flag contains the proton count for each state and will look something like this:

```
PROTCNT=0,1,1,1,1,
```

In this example, there are 5 different protonation states. The first one is a deprotonated state, and the other four are protonated states. Assuming there is only one titratable residue in your cpin file and that the protonated states are all equivalent (which is true for a GL4 residue for example), this is how the STATENE flag needs to look like for *finddgreg.py*:

```
STATENE=0.0, DELTAGREF, DELTAGREF, DELTAGREF, DELTAGREF,
```

finddgreg.py will return at the end of its execution the value of DELTAGREF that gives 50% fraction of protonated species for the pH you set in your mdin file (which should be equal to the pK_a of your titratable residue). The absolute values that appear on STATENE are irrelevant for constant pH simulations; it is only the energy differences between the different states that matters. The execution of *finddgreg.py* is very similar to the execution of *pmemd* or *sander*. This is an example of how a simple execution looks like:

```
finddgreg.py -mdexec pmemd.cuda -i mdin -p prmtop -c restrt -cpin cpin
```

Additional options are available (see the command description the `--help` flag). The `-mdexec` flag contains the location of the executable to be used for the constant pH simulation (in this example, *pmemd.cuda*). *finddgreg.py* will keep repeating automatically different simulations (the input files you provided will not be overwritten) for different values of DELTAGREF until the fraction of protonated species is equal to 50% within a convergence criterion. By default, *finddgreg.py* will finish its execution when it finds a fraction between 48.50% and 51.50%. This range can be changed by altering the `-fracthreshold` flag. After each CpHMD execution *finddgreg.py* will print an output message that looks like this:

```
AMBER execution #8: running 250000 MD steps of equilibration for DELTAGREF =
-14.887694 kcal/mol
  AMBER execution #8: running 2500000 MD steps of production      for DELTAGREF =
-14.887694 kcal/mol
  The fraction of protonated species is 66.10% for the Residue 'GL4 2'
```

At the beginning of *finddgreg.py*'s execution, unless a estimation of DELTAGREF is provided by the user, the total number of MD steps in each CpHMD execution will be reduced to a very small number just in order to find a rough estimation of DELTAGREF. When this rough estimation is found, *finddgreg.py* starts to perform more refined and accurate estimations of DELTAGREF with each CpHMD simulation having the total number of MD steps that was set by the user. Then, when a good value of DELTAGREF is found, *finddgreg.py* will print an output message like this:

```
The value of DELTAGREF that gives a converged fraction of protonated species for 9500000 MD
steps and for solvent pH = 4.400 equals to 49.40% is: DELTAGREF = -15.285781 kcal/mol
The execution of finddgreg.py ended with success.
```

At this point, the only thing left to do is to update Amber's source code (see Section 27.5 for instructions) with this value of DELTAGREF so that *cpinutil.py* can generate cpin files that contain your titratable residue with the correct reference energies.

Parallel mode (pH-REMD)

The first step is to prepare your input files as if you were to run a pH-REMD simulation of your model compound. You need to make sure that the total number of MD steps is long enough to ensure convergence, however it is not necessary that any of the `solvrph` variables inside the mdin file of each replica to be equal to the pK_a of your model compound. The fractions of protonated species for each replica (therefore, different pH values) can be used to extrapolate, from a fit based on the Henderson-Hasselbalch equation, the fraction of protonated species at the target pH (which should be set equal to the pK_a of your model compound). The cpin file should be prepared following the same instructions given above for the serial mode. The execution of *finddgreg.py* is very similar to the execution of *pmemd* or *sander* for replica exchange simulations. This is an example of how it looks like:

27. Constant pH calculations

```
finddgreg.py -do_parallel "mpirun -np 4" -mdexec pmemd.cuda.MPI -ng 4  
-groupfile groupfile -target 4.4
```

Additional options are available (see the command description using the `--help` flag). After each pH-REMD execution *finddgreg.py* will print an output message that looks like this:

```
AMBER execution #8: running 50000 MD steps (500 replica exchange attempts) of  
equilibration for DELTAGREF = -15.134900 kcal/mol  
AMBER execution #8: running 500000 MD steps (5000 replica exchange attempts) of  
production for DELTAGREF = -15.134900 kcal/mol  
The fraction of protonated species for pH = 3.500 is 89.30% for the Residue 'GL4  
The fraction of protonated species for pH = 4.000 is 73.30% for the Residue 'GL4  
The fraction of protonated species for pH = 4.500 is 44.30% for the Residue 'GL4  
The fraction of protonated species for pH = 5.000 is 19.30% for the Residue 'GL4  
Fitted values for Residue 'GL4 2': pKa = 4.408 and Hill coefficient = 1.047  
The computed fraction of protonated species at the target pH = 4.400 is 50.47% fo  
the Residue 'GL4 2'
```

Similarly to the serial mode, at the begging of *finddgreg.py*'s execution a rough estimation of DELTAGREF is done with a small total number of MD steps. Afterwards, more accurate DELTAGREF estimations are performed with each pH-REMD simulation having the total number of MD steps that was set by the user. Finally, when a good value of DELTAGREF is found, *finddgreg.py* will print an output message that looks like this:

```
The value of DELTAGREF that gives a converged fraction of protonated species for 500000 MD  
steps and for target solvent pH = 4.400 equals to 50.47% is: DELTAGREF = -15.134900 kcal/mol  
The execution of finddgreg.py ended with success.
```

Don't forget to update Amber's source code (see Section 27.5 for instructions) with this value of DELTAGREF so that *cpinutil.py* can generate cpin files that contain your titratable residue with the correct reference energies.

27.6. pH Replica Exchange MD

Running constant pH replica exchange simulations can be performed in either implicit or explicit solvent. There is no difference in the replica exchange setup between running in implicit or explicit solvent. This method is described in Section 26.3.7 above. We have found that pH-REMD dramatically improves protonation state and conformational state sampling, so we suggest using it whenever possible.

27.7. cphstats

cphstats is a C++ command-line program written by Jason Swails to compute protonation state statistics from constant pH simulations (in both implicit and explicit solvent). You can access a list and description of all available command-line flags using the `--help` flag, whose output is shown below.

```
Usage: cphstats [-O] [-V] [-h] [-i <cpin>] [-t] [-o FILE] [-R FILE -r INT]  
      [--chunk INT --chunk-out FILE] [--cumulative --cumulative-out FILE]  
      [-v INT] [-n INT] [-p|-d] [--calcpka|--no-calcpka] [--fix-remd]  
      [--population FILE] [-c CONDITION -c CONDITION -c ...]  
      [--conditional-output FILE] [--chunk-conditional FILE]  
      cpout1 [cpout2 [cpout3 ...] ]  
General Options:  
  -h, --help      Print this help and exit.
```

```

-V, --version  Print the version number and exit.
-O, --overwrite
                Allow existing outputs to be overwritten.
--debug        Print out information about the files that are
                being read in and used for the calculations.
--expert       I will consider you an expert user and NOT warn
                you if you try to compute statistics from REMD-based
                files before using --fix-remd [NOT default behavior]
--novice       I will warn you if you try to use REMD-based files
                to compute statistics. [Default behavior]

```

Input Files and Options:

```

-i FILE, --cpin FILE
                Input cpin file (from sander) with titrating residue
                information.
-t FLOAT, --time-step FLOAT
                This is the time step in ps you used in your simulations.
                It will be used to print data as a function of time.
                Default is 2 fs (0.002)

```

Output Files:

```

-o FILE, --calcpka-output FILE
                File to which the standard 'calcpka'-type statistics
                are written. Default is stdout
-R FILE, --running-avg-out FILE
                Output file where the running averages of time series
                data for each residue is printed (see [Output Options]
                below for details). Default is [running_avgs.dat]
--chunk-out FILE
                Output file where the time series data calculated
                over chunks of the simulation are printed (see
                [Output Options] below for details).
                Default is [chunk.dat]
--cumulative-out FILE
                Output file where the cumulative time series data
                is printed (see [Output Options] below for details).
                Default is [cumulative.dat]
--population FILE
                Output file where protonation state populations are
                printed for every state of every residue.
--conditional-output FILE
                Output file with requested conditional probabilities.
                Default is [conditional_prob.dat].
--chunk-conditional FILE
                Prints a time series of the conditional probabilities over
                a trajectory split up into chunks.

```

Output Options:

These options modify how the output files will appear

```

-v INT, --verbose INT
                Controls how much information is printed to the
                calcpka-style output file. Options are:
                (0) Just print fraction protonated. [Default]
                (1) Print everything calcpka prints.
-n INT, --interval INT
                An interval between which to print out time series data
                like 'chunks', 'cumulative' data, and running averages.
                It is also used as the 'window' of the conditional
                probability time series (--chunk-conditional).
                Default [1000]

```

```

-p, --protonated
    Print out protonation fraction instead of deprotonation
    fraction in time series data (Default behavior).
-d, --deprotonated
    Print out deprotonation fraction instead of protonation
    fraction in time series data.
-a, --pKa
    Print predicted pKas (via Henderson-Hasselbalch) in place
    of fraction (de)protonated. NOT default behavior.
Analysis Options:
These options control which analyses are done. By default, only
the original, calcpka-style analysis is done.
--calcpka      Triggers the calcpka-style output [On by default]
--no-calcpka   Turns off the calcpka-style output
-r WINDOW, --running-avg WINDOW
    Defines a window size for a moving, running average
    time series. <WINDOW> is the number of MD steps (NOT
    the number of MC exchange attempts).
--chunk WINDOW
    Computes the time series data over a chunk of the
    simulation of size <WINDOW> time steps. See above for
    details.
--cumulative   Computes the cumulative average time series data (see above
    for options) over the course of the trajectory.
--fix-remd PREFIX
    This option will trigger cphstats to reassemble the
    titration data into pH-specific ensembles. This
    is an exclusive mode of the program---no other
    analyses will be done.
-c CONDITIONAL, --conditional CONDITIONAL
    Evaluates conditional probabilities. CONDITIONAL should be a
    string of the format:
        <resid>:<state>,<resid>:<state>,...
    or
        <resid>:PROT,<resid>:DEPROT,...
    or
        <resid>:<state1>;<state2>,<resid>:PROT,...
    Where <resid> is the residue number in the prmtop (NOT the
    cpin) and <state> is either the state number or (p)rotonated
    or (d)eprotonated, case-insensitive
This program analyzes constant pH output files (cpout) from Amber.
These output files can be compressed using gzip compression. The
compression will be detected automatically by the file name extension.
You must have the gzip headers for this functionality to work.

```

27.7.1. Standard statistics

The standard output of cphstats is the same as that for the calcpka and calcpka.pl programs that came before. An example from a protein with 10 titratable residues is shown below.

```

Solvent pH is      4.000
GL4 7   : Offset -0.448 Pred 3.552 Frac Prot 0.263 Transitions 91163
HIP 15   : Offset  2.036 Pred 6.036 Frac Prot 0.991 Transitions  6194
AS4 18   : Offset -1.063 Pred 2.937 Frac Prot 0.080 Transitions 41490
GL4 35   : Offset  2.113 Pred 6.113 Frac Prot 0.992 Transitions  5458
AS4 48   : Offset -1.365 Pred 2.635 Frac Prot 0.041 Transitions 17596
AS4 52   : Offset -1.123 Pred 2.877 Frac Prot 0.070 Transitions 25306
AS4 66   : Offset -1.689 Pred 2.311 Frac Prot 0.020 Transitions  7334
AS4 87   : Offset -1.757 Pred 2.243 Frac Prot 0.017 Transitions  8976

```



```
AS4 101 : Offset  0.342  Pred  4.342  Frac Prot 0.687  Transitions    105377
AS4 119 : Offset -1.894  Pred  2.106  Frac Prot 0.013  Transitions     6700
```

```
Average total molecular protonation:    4.174
```

The external pH that was set in `sander` or `pmemd` is shown at the top, followed by each of the residues with their name and number as they appear in the topology file. The computed pK_a values printed by `cphstats` are computed by fitting to the Hendersen-Hasselbalch equation (Eq. 27.1). The values printed in the standard output are defined below:

Offset Difference in pK units that the predicted pK_a is from the solvent pH.

Pred The predicted pK_a computed from the fraction protonated and the pH in Eq. 27.1.

Frac Prot The total fraction of the simulation that the residue spent in its ‘protonated’ form.

Transitions The number of times that the total number of ‘active’ protons on the titratable residue changed following a protonation state change attempt. This does *not* count when the protonation state changed between two tautomers or protomers with the same number of protons. For instance, the switching from the HID to the HIE tautomers of histidine does not count. Nor does switching from the syn-O1-protonated to the syn-O2-protonated forms of any carboxylate residues.

$$pK_a = pH - \log \left(\frac{1 - f_p}{f_p} \right) \quad (27.1)$$

In Eq. 27.1, f_p is the total fraction protonated for a given residue. A more rigorous way of computing the pK_a of a titratable residue is to fit Eq. 27.2—the Hill equation—to the pHs and protonation fractions collected over a full titration curve to compute the best-fit values of the Hill coefficient (n) and computed pK_a . This requires post-processing the output from `cphstats` with your own script or program.

$$f_d = 1 - f_p = \frac{1}{1 + 10^{n(pK_a - pH)}} \quad (27.2)$$

Example You can analyze as many `cpout` files as you would like, provided that each `cpout` file was generated from a simulation run at the same pH as the others. For pH-REMD simulations, a pre-processing stage is initially required, as described in a section 27.7.5. You can direct `cphstats` to print the output to a file with the `-o` flag or have it printed to the screen (stdout) by default. You must provide a `cpin` file with the `-i` flag to calculate any protonation state statistics.

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout \
-o pH_4.dat
```

27.7.2. Cumulative, running, and “chunk” averages

These options provide a way of monitoring how the ensemble of protonation states evolve during the course of a simulation. Because MD yields insights into the dynamical behavior of molecules, it’s often advantageous to monitor the evolution of the protonation state fractions with geometric measurements of the system coordinates. Each option—cumulative, running, and “chunk” averages—can be output as a time series of fraction protonated, fraction deprotonated, or predicted pK_a using the `-p`, `-d`, and `-a` flags, respectively. The details of calculating each of these properties is described in the next sections. The output is printed to a file in the following format:

```
#Time step    GL4 7    HIP 15    AS4 18    GL4 35    Total Avg. Prot.
1000  0.30693  0.99505  0.00000  0.99505          3.900498
2000  0.24378  0.99751  0.00000  0.99751          3.962594
3000  0.23754  0.99834  0.05150  0.99834          4.014975
...
```

27. Constant pH calculations

The final column is always the total average protonation (note, a protonated histidine counts as ‘2’ protons and a protonated lysine counts as ‘3’, so only differences in total protonation fraction are meaningful). The time step corresponds to the actual MD time step, *not* the interval between protonation state changes.

Cumulative averages

A cumulative average is a time series whose values at time t are calculated according to

$$\langle A \rangle_t = \frac{\int_0^t A(t) dt}{t}$$

such that it represents the average value from time 0 to t . The final average should match the output printed in the standard statistics output of the previous section, which is an average over the entire ensemble. Cumulative averages can be misleading, however, as $\langle A \rangle_t$ changes rapidly when t is small and very slowly as t becomes large. It can give the impression that a property is converging to a particular value when in fact that property is fluctuating a lot.

Example To compute a cumulative average, you must use the `--cumulative` flag to indicate you wish to compute this value. You can control how frequently this quantity is sampled by setting the interval, in MD time steps (*not* protonation state change attempts), using the `-n` flag. The default interval is to print values every 1000 time steps. The longer your simulation is, the less frequently you have to sample points. Data is written to the file `cumulative.dat` unless a different name is provided with the `--cumulative-out` flag. An example usage is:

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout -n 10000 -p \
--no-calcpka --cumulative --cumulative-out pH_4.cumulative.dat
```

The `--no-calcpka` flag prevents the standard statistics (previous section) from being computed and printed. The cumulative protonated fraction will be printed to the file `pH_4.cumulative.dat` with values dumped every 10000 steps.

Running averages

A running average is a time series whose values at time t are calculated according to

$$\langle A \rangle_t = \frac{\int_{t-\sigma}^{t+\sigma} A(t) dt}{2\sigma}$$

such that it represents the average value from $t - \sigma$ to $t + \sigma$ —the value 2σ is referred to as the *window* size. The advantage of a running average over a cumulative average is that the shape of the curve at large values of t do not depend on the values near $t = 0$. If the interval is smaller than the window size, then adjacent values of $\langle A \rangle_t$ will be comprised of overlapping data points.

Example To compute a running average, you must specify a window size, in MD time steps, with the `-r` flag. The interval—specified with the `-n` flag—controls how frequently samples from the time series are saved to the output file. An example usage is

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout \
-n 2000 -r 10000 -R pH_4.runningavg.dat -d
```

This command will compute the running average of the deprotonation fraction (because of the `-d` flag) every 2000 time steps with a window size of 10000 time steps—the average will be computed from the 5000 steps before time t to 5000 steps after time t every 2000 steps. It may be important to note that any portion of the window that extends before $t = 0$ or after the last time step of the simulation is simply truncated. In this example, the running average data is printed to the file `pH_4.runningavg.dat` and the standard statistical output is printed to the screen.

“Chunk” averages

A “chunk” average is a time series in which the trajectory is segmented into separate chunks of specified size 2σ time steps. The average value is then calculated according to

$$\langle A \rangle_t = \frac{\int_{t-\sigma}^{t+\sigma} A(t) dt}{2\sigma}$$

Indeed, “chunk” averages are simply a special case of running averages in which the times for which $\langle A \rangle_t$ are computed are the center points of the time chunks. In this analysis, every point of the simulation is uniquely assigned to a single chunk (so no overlap is possible like there is for running averages with a window size larger than twice the interval).

Example Unlike the cumulative and running average analyses, the interval is not used for “chunk” averaging. The chunk size, in MD time steps, simultaneously specifies the size of the simulation to use in the average as well as the positions of the points in the generated time series. An example chunk analysis is

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout --pKa \
--no-calcpka --chunk 100000 --chunk-out pH_4.chunks.dat
```

This command will break the trajectory into 100,000 time step-chunks and compute the pK_a of each residue for each chunk according to Eq. 27.1. In this example, the “chunk” pK_a s are printed to the file pH_4.chunks.dat.

27.7.3. Populations

If you specify a file with the `--populations` flag, the population of every residue in every state computed over the whole trajectory will be printed to the specified file. An example command is shown below

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout \
--populations populations.dat
```

The populations.dat file will look something like the following:

Residue Number	State 0	State 1	State 2	State 3	State 4
Residue: GL4 7	0.642697 (0)	0.182143 (1)	0.003043 (1)	0.168653 (1)	0.003465 (1)
Residue: HIP 15	0.982722 (2)	0.017278 (1)	0.000000 (1)		
Residue: AS4 18	0.986247 (0)	0.006043 (1)	0.000075 (1)	0.007635 (1)	0.000000 (1)
Residue: GL4 35	0.052398 (0)	0.437664 (1)	0.025815 (1)	0.476629 (1)	0.007495 (1)
Residue: AS4 48	0.995087 (0)	0.001435 (1)	0.000085 (1)	0.003393 (1)	0.000000 (1)
Residue: AS4 52	0.973672 (0)	0.007520 (1)	0.002705 (1)	0.015465 (1)	0.000638 (1)
Residue: AS4 66	0.999612 (0)	0.000165 (1)	0.000000 (1)	0.000223 (1)	0.000000 (1)
Residue: AS4 87	0.948970 (0)	0.023918 (1)	0.001240 (1)	0.025235 (1)	0.000638 (1)
Residue: AS4 101	0.678859 (0)	0.160313 (1)	0.002295 (1)	0.153793 (1)	0.004740 (1)
Residue: AS4 119	0.971682 (0)	0.015043 (1)	0.000623 (1)	0.011888 (1)	0.000765 (1)

In the example output, each residue except for histidine 15 is a carboxylate that has 5 available states: deprotonated and protonated on the syn- or anti- positions of either carboxylate oxygen. You can use the `--describe` flag for `cpinutil.py` to get a detailed description of what each state is. The decimal numbers shown are the fraction of the time that the residue spent in the specified protonation state. The integer in parentheses next to the protonation state population is the number of titratable protons that are present in that state. Notice that histidine 15 has only 3 states. The first is the doubly-protonated state whereas the other two states are singly-protonated at the N^δ and N^ϵ positions, respectively. This output gives a more detailed view of *where* the protons are during the simulation.

27.7.4. Conditional probabilities

It is frequently the case that an enzyme's or ribozyme's catalytic mechanism depends on two titratable residues having specific protonation states to fulfill their role as either a proton donor or acceptor in the mechanism. When one residue is a proton acceptor and the other a proton donor, there is an 'optimum' pH at which the fraction of proteins that have the correct set of protonation states will be a maximum and the catalytic rate is lowered when the pH is either raised or reduced from this optimum value, since we typically assume that the only catalytically active state is the one with the catalytically 'correct' set of protonation states. By assuming that each residue titrates independently—that is that the protonation state of one does not affect the protonation state of the other—we can derive a pH-rate profile for the enzyme or ribozyme by using the pK_a of each residue to compute fraction of time each residue spends in its catalytically active protonation state and simply multiply those fractions for each residue to arrive at the conditional probability.

If, however, the catalytic residues titrate cooperatively or anticooperatively, the conditional probabilities cannot be computed as a simple product of the individual probabilities. By directly capturing the coupling between dynamics and titration of all titratable residues, CpHMD and pH-REMD are capable of probing this cooperativity. This section describes how to use `cphstats` to compute conditional probabilities directly from the protonation state data.

Conditional Probability Expressions

This section describes the syntax of the conditional probability expressions that you must use for `cphstats`. The format of the expression is a comma-delimited list of residue:state specifications shown below.

```
<residue 1>:<state specification>,<residue 2>:<state specification>
```

You can list as many residues as you want. A snapshot satisfies the conditional probability criteria if each of the specified residues is in the list of allowable states within the state specifications—unspecified residues can be in any state. The residue specifiers are the residue numbers in the topology file. The state specification can be either a semicolon-delimited list of state numbers or the description "protonated" or "deprotonated." The parser is case-insensitive and you only have to type up to one letter of either word to trigger recognition of "protonated" or "deprotonated." Example conditional probability expressions are shown below with accompanying descriptions of what conditional probabilities they define. Individual residue:state specifications are indicated by color.

```
``35:P,52:D''
```

This expression is satisfied if residue 35 is protonated at the same time that residue 52 is deprotonated.

```
``35:Prot,52:1;3,15:1''
```

This expression is satisfied if residue 35 is protonated and residue 52 is in either state 1 or 3 and residue 15 is in state 1. Other residues can be in any state.

Examples

You can specify conditional probability expressions following the `-c` flag on the command-line. You can specify as many expressions as you want, as long as each one is preceded by `-c` or `--conditional`. The fraction of states that satisfy each conditional probability is written to the conditional probability output file specified by `--conditional-output` (or `conditional_prob.dat` by default). An example command-line is shown below

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout \
-c ``35:P,52:D'' -c ``35:prot,52:1;3,15:1'' \
--conditional-output conditional.dat
```

Example output from `conditional.dat` is shown below.

Conditional Probabilities	Fraction
35:P,52:D	0.982922
35:prot,52:1;3,15:1	0.000290

For the sake of comparison, the standard statistics are shown below.

Solvent	pH	is							
GL4	7	:	Offset	-0.167	Pred	3.833	Frac Prot	0.405	Transitions 31378
HIP	15	:	Offset	1.391	Pred	5.391	Frac Prot	0.961	Transitions 5578
AS4	18	:	Offset	-1.851	Pred	2.149	Frac Prot	0.014	Transitions 1910
GL4	35	:	Offset	2.432	Pred	6.432	Frac Prot	0.996	Transitions 391
AS4	48	:	Offset	-2.282	Pred	1.718	Frac Prot	0.005	Transitions 570
AS4	52	:	Offset	-1.855	Pred	2.145	Frac Prot	0.014	Transitions 438
AS4	66	:	Offset	-1.998	Pred	2.002	Frac Prot	0.010	Transitions 698
AS4	87	:	Offset	-1.489	Pred	2.511	Frac Prot	0.031	Transitions 1239
AS4	101	:	Offset	-0.478	Pred	3.522	Frac Prot	0.250	Transitions 19924
AS4	119	:	Offset	-1.516	Pred	2.484	Frac Prot	0.030	Transitions 2408

Average total molecular protonation: 3.716

In this example, residue 35 is protonated while residue 52 is deprotonated 98.2922% of the time. Assuming that the two residues titrate independently, we would calculate this conditional probability to be $0.996 \times (1 - 0.014) \times 100\% = 98.2056\%$. This indicates that these residues titrate independently at pH 4.

Conditional probability “chunks”

cphstats currently supports creating time series of conditional probabilities by breaking the trajectory into equal-sized “chunks” and computing the conditional probability over that chunk (see Sec. 27.7.2 for details). To perform this analysis, specify a conditional “chunk” output file with the `--chunk-conditional` flag. The `-n` flag is used to define the size of the chunk (the same flag used to define the time series interval for cumulative and running averages). The format of the output file is the same as that shown in Sec. 27.7.2 for the other time series, but the columns are labeled with the conditional probability expression instead of the residue name and number. An example usage is shown below.

```
cphstats -i cpin pH_4.md1.cpout pH_4.md2.cpout pH_4.md3.cpout \
-c ``35:P,52:D`` -c ``35:prot,52:1;3,15:1`` -n 100000 \
--chunk-conditional chunk_conditional.dat
```

This example will break the simulation into 100,000-time-step chunks and compute the two conditional probabilities over each chunk, writing the results to chunk_conditional.dat.

27.7.5. Processing pH-REMD cpout files

Replica exchange in pH-space is performed by attempting to exchange solution pH between replicas. Like with temperature-based REMD, this means that individual replicas do not keep the same pH throughout the entire simulation. Since many of the analyses described in the previous sections pertain to ensembles at one pH, they are not readily applicable to the raw output from pH-REMD simulations and must often be pre-processed before being analyzed. This section describes only how to do the preprocessing. You are expected to be familiar with the content of Subsection 26.3.7 above.

Re-ordering cpout files

You can use cphstats to generate pH-specific cpout files from pH-REMD replica cpout files using the `--fix-remd` flag. To do so, you must provide a file name prefix to which the suffix `.pH_X` will be appended as well as the cpout files from every replica for a single simulation (and only a single simulation). If you ran the

27. Constant pH calculations

simulation in multiple steps, using restarts from the previous simulation to start the next, you will have to run the command described here for each segment of the total simulation separately. No cpin file is needed for this step. An example usage is shown below:

```
cphstats --fix-remd cpout pH_1.md1.cpout pH_2.md1.cpout pH_3.md1.cpout \  
pH_4.md1.cpout pH_5.md1.cpout pH_6.md1.cpout
```

This command will create the files cpout.pH_1.00, cpout.pH_2.00, cpout.pH_3.00, cpout.pH_4.00, cpout.pH_5.00, and cpout.pH_6.00 that can subsequently be used for the analyses described in the previous section.

If you attempt to use REMD cpout files without fixing them in the previous analyses, you will receive an error.

Analyzing replica statistics

There are times when you may want to analyze statistics, such as fraction protonated or deprotonated, from a single replica when looking at things like correlation times, for instance. You can disable the REMD file protection using the `--expert` flag, but note that any computed pK_a is based on Eq. 27.1 with a single pH, so they will be meaningless for REMD-based trajectories.

28. Constant Redox Potential calculations

The constant Redox Potential molecular dynamics method, developed in Amber by Vinícius Cruzeiro, is available for both implicit and explicit solvent simulations [612, 613, 681]. Its implementation is based on the constant pH molecular dynamics method (CpHMD, see chapter 27), and has a similar procedure to CpHMD for generating the input files and running simulations. This chapter assumes the reader already has some familiarity with constant pH simulations. Likewise CpHMD, constant Redox Potential simulations require minor modifications to the regular process of generating the prmtop file and also require an additional input file (cein) describing the redox-active titrating residues.

The constant Redox Potential method is implemented in both *sander* and *pmemd*, and, similarly to CpHMD, makes use of Monte Carlo sampling of the Boltzmann distribution of discrete redox states concurrent with the molecular dynamics simulation. The redox states distribution is affected by the solvent Redox Potential, which is set as an external parameter in the simulation. The redox states of a redox-active titrable residue are changed by modifying the partial charges on the atoms of this same residue.

The methodology in *sander* and *pmemd* allows constant Redox Potential simulations to be executed simultaneously with constant pH simulations; in this case, the simulation then becomes constant pH and Redox Potential MD. More recent improvements in the code now allow a given residue to be simultaneously pH- and redox-active, as presented in reference [681].

28.1. Preparing a system for constant Redox Potential simulation

Currently, Amber provides definitions for titrating a bis-histidine heme group, like the heme group in N-acetylmicroperoxidase-8 (NAcMP8) or in the horse heart cytochrome *c* (PDB code 1HRC). The iron atom, the porphyrin ring, along with the side chains of two histidines and two cysteines (that are attached to the heme) are considered as a single residue called HEH. HEH is the redox-active residue that changes its atomic charge distribution when a redox state change attempt is successful. Therefore, a successful redox state change affects the charge distribution of the histidines and cysteines as well. The two heme propionates are called PRN and are separate residues from HEH.

Begin by preparing your PDB file as you normally would for use with LEaP. Then the PDB file must be edited to replace the heme as well as the histidines and cysteines attached to it with the standard labeling defined in Amber. For example, the side chains of the two histidines and the two cysteines have to be part of the same HEH residue, thus the atom names of each side chain must be unique and match the names defined inside the force field. The backbone atoms have to be reassigned to new residues called HIO and CYO. For more information on how to prepare your PDB file, please refer to the constant Redox Potential tutorial at <https://ambermd.org/tutorials>.

Then, you should run LEaP with the *leaprc.conste* command file. This file loads all parameters to be used for HEH, PRN, HIO and CYO. Inside LEaP you can load this file with the following command:

```
source leaprc.conste
```

This loads the *ff10* force field. In addition, it loads the residue libraries and force field modifications—*conste.lib* and *frmod.conste*. It also sets the GB solvation radii (PBradii) to mbondi2, which was the set used to parameterize the reference compounds. Now load your edited PDB file and proceed as usual to create the topology and coordinates files.

Once you have the prmtop (topology) file, you need to generate a cein file. The cein file describes which redox-active residues should be titrated, and defines the possible redox states and their relative energies. A python tool called *ceinutil.py* is provided with AmberTools to generate this input file. It takes a prmtop file as input along with the GB model you wish to evaluate redox transitions in, and writes the cein file. Here is an example of how to generate the cein file from your prmtop file using the igb=2 GB model:

```
ceinutil.py -p prmtop -igb 2 -o cein
```

The *ceinutil.py* program accepts a number of flags that modify its behavior. This program is equivalent to *cpinutil.py* used for constant pH simulations and has similar functionalities to it. Please refer to Section 27.2 for more details on how to use it. As the heme propionates are pH-active residues, their information need to be present at the cpin file, so the *cpinutil.py* must be used for it.

Note: in order to deal with residues that are simultaneously pH- and redox-active, users must use the *cpeinutil.py* program instead. This software is very similar to *cpinutil.py* and *ceinutil.py*, and generates a cpein file that can be used in *pmemd* or *sander* instead of cpin and cein files. At time of writing, the only pH- and redox-active residue parametrized is the tyrosine discussed in reference [681], labeled as TYX. The CPEIN file format supports residues that are only pH-active or only redox-active.

You can get a full list of all available titratable residues using the `--list` argument in *ceinutil.py* (or in *cpeinutil.py*), and you can get a full description of reference energies and charge vectors for any residue using the `--describe` argument. The full usage statement for *ceinutil.py* (accessible via `-h/--help`) is shown below.

```
usage: ceinutil.py [Options]
optional arguments:
  -h, --help            show this help message and exit
  -v, --version          show program's version number and exit
  -d, --debug           Enable verbose tracebacks to debug this program
Output files:
  -o FILE, --output FILE
                        Output file. Defaults to standard output
Required Arguments:
  -p FILE               Topology file to be used in constant Redox Potential
                        simulation
Simulation Options:
  -igb IGB              Generalized Born model which you intend to use to
                        evaluate dynamics (or protonation state swaps).
                        Default is 2.
  -intdiel DIEL         Internal dielectric constant to use in the evaluation
                        of the GB potential. Default 1.0.
Residue Selection Options:
  -resnames [RES [RES ...]]
                        Residue names to include in CEIN file
  -notresnames [RES [RES ...]]
                        Residue names to exclude from CEIN file
  -resnums [NUM [NUM ...]]
                        Residue numbers to include in CEIN file
  -notresnums [NUM [NUM ...]]
                        Residue numbers to exclude from CEIN file
  -mineo Eo             Minimum reference standard Redox Potential (given in
                        Volts) to include in CEIN file
  -maxeo Eo             Maximum reference standard Redox Potential (given in
                        Volts) to include in CEIN file
System Information:
  -states [NUM [NUM ...]]
                        List of default states to assign to titratable
                        residues
  -system <system name>
                        Name of system to titrate. No effect on simulation.
Residue Information:
  If any options here are used, no CEIN file will be written. These
```



```
arguments take precedence and are mutually exclusive with each other.
--describe [RESNAME [RESNAME ...]]
                                Print out the details of given residues
-l, --list                      List all titratable residues
This program will read a topology file and generate a cein file for constant
Redox Potential simulations with sander or pmemd
```

28.2. Running at constant Redox Potential

28.2.1. Running at constant Redox Potential in implicit solvent

In the mdin file, you must set *icnst*=1 to turn on constant Redox Potential in implicit solvent. *solve* is the variable used to set the solvent Redox Potential and should be given in units of Volts. You must also specify the period for Monte Carlo redox state change attempts, *ntcnste*. In the implicit solvent implementation only one residue is examined on each MC step, so you should decrease *ntcnste* as the number of titrating residues increases to maintain a constant effective step period for each residue.

The constant Redox Potential approach makes use of a reference compound. The standard Redox Potential, E° , for this compound is known and the relative free energy differences between the various redox states are computed through a thermodynamic cycle equivalent to the one used for constant pH MD shown in Figure 27.1. The free energy of the redox state change in the reference compound is necessary to yield the correct E° prediction. This quantity is pre-computed for each redox state change. This *reference energy* is printed to the cein file by *ceinutil.py*. The reference energies in *ceinutil.py* were derived using the following parameters:

```
cut=1000.0, igb=#, saltcon=0.1, nrespa=1,
temp0=300.0, ntc=2, ntf=2
```

where # is the *igb* value passed to *ceinutil.py*. Changes in these parameters, specially *igb*, *saltcon*, *nrespa*, or *temp0*, might require a new reference energy computation. The *ff10* force field was used in the reference energy calculations. The use of other force fields should be validated before you run simulations and might require recalculating the reference energies.

Additional command line flags are available in *sander* and *pmemd* to support constant Redox Potential operation. The cein file must be specified using the *-cein* flag. Additionally, a history of the redox states sampled is written to the filename specified by *-ceout*. Finally, a constant Redox Potential restart file is written to the filename specified by *-cerestrt*. This is used to ensure that titrating residues retain the same redox state when the simulation is restarted.

28.2.2. Running at constant Redox Potential in explicit solvent

Likewise constant pH MD, in the explicit solvent implementation redox state changes are attempted using a Generalized Born implicit solvent model [612]. The procedure works as follows: MD is executed for *ntcnstph* steps and the simulation is halted. Then, the solvent and any eventual ions are stripped. After that, one redox state change attempt is performed for each redox-active residue in random order. The solvent is restored, and if any redox state have changed then a solvent relaxation dynamics is executed during *ntrelaxe* steps (200 fs are generally enough). This cycle is repeated until the end of the simulation is reached.

As each residue is visited in random order when the MD is halted, this allows redox state change attempts to be done far less frequently than in implicit solvent simulations. This is a good strategy because it reduces the amount of MD time that is consumed by the solvent relaxation dynamics, which then improves the computational performance of the calculation. Here is an example of input variables to be used in your *sander* or *pmemd* input file for explicit solvent constant Redox Potential simulations.

```
icnst=2, ntcnst=100, ntreaxe=200,
solve=-0.203, saltcon=0.1, temp0=300.0,
ntc=2, ntf=2
```

Notice that the value of `icnst` is 2, which indicates that constant Redox Potential MD should be run in explicit solvent. The `ntrelax` flag will run solvent relaxation dynamics (in which the non-solvent is held fixed) for 200 steps. The `saltcon` variable controls the salt concentration for the GB calculations. It has no effect on the dynamics, but is required for consistency with the reference energy of the model compound.

28.3. Analyzing constant Redox Potential simulations

As the simulation progresses, the redox states that are sampled are written to the `ceout` file. A section of a `ceout` file for an implicit solvent simulation is shown here:

```
Redox potential:  -0.2800000 V Temperature:  300.00 K
Monte Carlo step size:      2
Time step:         0
Time:             0.000
Residue    0 State:  0
Residue    1 State:  0

Residue    0 State:  1

Residue    1 State:  1

Residue    1 State:  0

Residue    1 State:  1

Residue    0 State:  0

Residue    1 State:  1
```

One record is written on each Monte Carlo step. Each record is terminated by a blank line. There are two types of records: full records and delta records. Full records list the Redox Potential, Temperature, MC step size, current time step, and current time before listing every residue in the system. Full records are written on the first step and every `ntwx` steps afterwards so as to coincide with the frames written to the trajectory. Delta records list only those redox-active residues that were titrated (single line for implicit solvent or a list of every residue for explicit solvent). Note that in some cases, the redox state for a delta record may be the same as that in an earlier record: this indicates that the Monte Carlo reduction move was rejected for that residue. The residue numbers in `ceout` are indices over the titrating residues included in the `cein` file; `ceout` files must be analyzed in conjunction with the `cein` to map these indices back to the original system.

The `cestats` program can be used to perform several different analyses on the `ceout` files. It prints the fraction of reduced species, which can be used to compute the E^o values of each redox-active titratable residue. The `cestats` usage is described in Section 28.6.

28.4. Extending constant Redox Potential to additional titratable groups

In order to do this, you must first define the partial charges of each atom in the redox-active residue for each redox state. Afterwards, you must set the relative energies of each redox state. This procedure is similar to the one described in Section 27.5 for CpHMD. Therefore, please refer to it for more information. The reference energies of redox-active titratable residues can be found by using TI calculations or by using the `finddgreg.py` tool (see Subsection 27.5.1 having in mind that in the `cein` file you should be looking to the `ELECCNT` flag instead of `PROTCNT`). To see an example of how a redox-active titratable residue is defined inside `$AMBERHOME/Am-`

berTools/src/parmed/parmed/amber/titratable_residues.py look for the definitions of the residue HEH inside this file.

28.5. Redox Potential Replica Exchange MD

Redox Potential Replica Exchange simulations can be performed in both implicit or explicit solvent. The procedure to setup the replica exchange simulation is the same in implicit or explicit solvent. The Redox Potential Replica Exchange MD method (E-REMD) is described in Section 26.3.8, thus please refer to this section for further information. E-REMD dramatically improves redox state and conformational state sampling, so we suggest using it whenever possible.

28.6. cestats

`cestats` was adapted by Vinícius Cruzeiro from the `cphstats` program written by Jason Swails in order to allow the computation of redox state statistics from constant Redox Potential simulations (in both implicit and explicit solvent). `cestats` contains all functionalities from `cphstats` and a very similar usage (a few flag names differ). Therefore, please refer to Section 27.7 for more information on how to use it. You can access a list and description of all available command-line flags using the `--help` flag, whose output is shown below.

```
Usage: cestats [-O] [-V] [-h] [-i <cein>] [-t] [-o FILE] [-R FILE -r INT]
      [--chunk INT --chunk-out FILE] [--cumulative --cumulative-out FILE]
      [-v INT] [-n INT] [-p|-d] [--calceo|--no-calceo] [--fix-remd]
      [--population FILE] [-c CONDITION -c CONDITION -c ...]
      [--conditional-output FILE] [--chunk-conditional FILE]
      ceout1 [ceout2 [ceout3 ...] ]

General Options:
  -h, --help          Print this help and exit.
  -V, --version       Print the version number and exit.
  -O, --overwrite          Allow existing outputs to be overwritten.
  --debug              Print out information about the files that are
                        being read in and used for the calculations.
  --expert             I will consider you an expert user and NOT warn
                        you if you try to compute statistics from REMD-based
                        files before using --fix-remd [NOT default behavior]
  --novice             I will warn you if you try to use REMD-based files
                        to compute statistics. [Default behavior]

Input Files and Options:
  -i FILE, --cein FILE
                        Input cein file (from pmemd or sander) with titrating residue
                        information.
  -t FLOAT, --time-step FLOAT
                        This is the time step in ps you used in your simulations.
                        It will be used to print data as a function of time.
                        Default is 2 fs (0.002)

Output Files:
  -o FILE, --calceo-output FILE
                        File to which the standard 'calceo'-type statistics
                        are written. Default is stdout
  -R FILE, --running-avg-out FILE
                        Output file where the running averages of time series
                        data for each residue is printed (see [Output Options]
                        below for details). Default is [running_avgs.dat]
  --chunk-out FILE
                        Output file where the time series data calculated
                        over chunks of the simulation are printed (see
```

```

        [Output Options] below for details).
        Default is [chunk.dat]
--cumulative-out FILE
        Output file where the cumulative time series data
        is printed (see [Output Options] below for details).
        Default is [cumulative.dat]
--population FILE
        Output file where reduction state populations are
        printed for every state of every residue.
--conditional-output FILE
        Output file with requested conditional probabilities.
        Default is [conditional_prob.dat].
--chunk-conditional FILE
        Prints a time series of the conditional probabilities over
        a trajectory split up into chunks.

Output Options:
These options modify how the output files will appear
-v INT, --verbose INT
        Controls how much information is printed to the
        calceo-style output file. Options are:
        (0) Just print fraction reduced. [Default]
        (1) Print everything calceo prints.
-n INT, --interval INT
        An interval between which to print out time series data
        like 'chunks', 'cumulative' data, and running averages.
        It is also used as the 'window' of the conditional
        probability time series (--chunk-conditional).
        Default [1000]
-p, --reduced
        Print out reduction fraction instead of oxidation
        fraction in time series data (Default behavior).
-d, --oxidized
        Print out oxidation fraction instead of reduction
        fraction in time series data.
-a, --Eo
        Print predicted Eos (via Nernst equation) in place
        of fraction reduced or oxidized. NOT default behavior.

Analysis Options:
These options control which analyses are done. By default, only
the original, calceo-style analysis is done.
--calceo
        Triggers the calceo-style output [On by default]
--no-calceo
        Turns off the calceo-style output
-r WINDOW, --running-avg WINDOW
        Defines a window size for a moving, running average
        time series. <WINDOW> is the number of MD steps (NOT
        the number of MC exchange attempts).
--chunk WINDOW
        Computes the time series data over a chunk of the
        simulation of size <WINDOW> time steps. See above for
        details.
--cumulative
        Computes the cumulative average time series data (see above
        for options) over the course of the trajectory.
--fix-remd PREFIX
        This option will trigger cestats to reassemble the
        titration data into Redox potential specific ensembles. This
        is an exclusive mode of the program---no other
        analyses will be done.
-c CONDITIONAL, --conditional CONDITIONAL

```

Evaluates conditional probabilities. CONDITIONAL should be a string of the format:

`<resid>:<state>,<resid>:<state>,...`

or

`<resid>:REDU,<resid>:OXID,...`

or

`<resid>:<state1>;<state2>,<resid>:REDU,...`

Where `<resid>` is the residue number in the prmtop (NOT the cein) and `<state>` is either the state number or p (reduced) or d (oxidized), case-insensitive

This program analyzes constant Redox potential output files (ceout) from Amber. These output files can be compressed using gzip compression. The compression will be detected automatically by the file name extension. You must have the gzip headers for this functionality to work.

29. Continuous constant pH molecular dynamics

Rooted in the λ -dynamics method for free energy calculations[682], the continuous constant pH MD[683, 684] is an alternative to the Monte-Carlo based constant pH MD methods described in section 27. λ particles with fictitious mass and coordinates bound between 0 (protonated) and 1 (deprotonated) are added to the system to represent protonation states of titratable sites. Protonation/deprotonation is simulated by propagating the underlying θ variables ($\lambda=\sin^2\theta$) with a Langevin integrator and the forces are calculated from the extended Hamiltonian. Titratable groups with two competitive protonation sites (e.g., His, Asp and Glu) are treated by adding an additional variable x to control the tautomeric states. These variables are integrated in the same fashion as the titration variables.

Continuous constant pH MD can be performed in three modes: implicit solvent (iphmd=1, only available in CPU),[684–686] hybrid solvent (iphmd=2),[687] and all-atom (iphmd=3). [688–690] In the implicit-solvent mode, both conformational and protonation state sampling is performed using a generalized Born (GB) model. In the hybrid-solvent mode, conformational sampling is performed in explicit solvent, while protonation state sampling is performed using the GB model (i.e., forces on λ particles are derived using the GB model) using the conformations in the explicit-solvent. A similar scheme is adopted in the explicit-solvent mode of the Monte-Carlo based CpHMD. In the all-atom mode, both conformational and protonation state sampling is performed in the explicit solvent and no GB calculations are conducted. We note, the hybrid-solvent mode (iphmd=2) is implemented only for CPUs and has not been extensively tested. The implicit-solvent mode makes use of the GBNeck2 model[29], and it is implemented for both CPUs[685] and GPUs[686], although the GPU version is currently only available through a downloadable patch (<https://gitlab.com/shenlab-amber-cphmd/cphmd-patches>). By contrast, the all-atom mode (iphmd=3) is implemented only for GPUs.[690]

The continuous constant pH MD simulations can be performed either at a single pH or at multiple pH conditions using the pH replica-exchange protocol.[687, 691] The latter significantly accelerates protonation state sampling and through the coupling between protonation and conformational dynamics also the conformational sampling. It has been shown that the use of pH replica exchange protocol can significantly accelerate the convergence of pK_a values in GB-based, hybrid-solvent, and all-atom continuous constant pH titration simulations.[686, 687, 690] pH replica-exchange simulations typically require 4 or more replicas. Thus, the traditional replica-exchange protocol may not be feasible for use on the GPUs, because it requires each replica to run on one GPU, e.g., 8 replicas require 8 GPUs on a node. In this case, we recommend the asynchronous replica-exchange Python code (https://gitlab.com/shenlab-amber-cphmd/async_ph_replica_exchange)[691], which allows replica exchange to be performed on one or any number of GPUs that is smaller than the number of replicas. This code controls Amber runs externally and is not the most efficient way. We will contribute a code that implements the asynchronous protocol within the Amber program in the future.

29.1. Implementation notes

To account for the changing protonation states, we attach a fictitious particle λ to each titratable site. These fictitious particles are propagated with a Langevin integrator at the same time as the spatial coordinates in an extended Lagrangian approach. For residues with a single titration site (e.g., lysine), $\lambda = 0$ represents the protonated state while 1 represents the deprotonated state. For residues with two different deprotonated states (due to tautomerisation) and a single protonated state (e.g., His), an additional fictitious particle X is introduced, where $X = 0$ and $\lambda = 0$ corresponds to one of the deprotonated tautomer states (HID) and $X = 1$ and $\lambda = 0$ the other (HIE). Once again, $\lambda = 0$ means that the site is protonated and 1 means that it is deprotonated. Similarly, for residues with one deprotonated and two protonated states (e.g., Asp and Glu), $X = 1$ and $\lambda = 0$ corresponds to one of the protonated

states, while $X = 0$ and $\lambda = 0$ to the other, and $\lambda = 1$ to the deprotonated state. We then interpolate between these different states by linearly scaling the charge on the titrating hydrogens and their van der Waals interactions with the surrounding atoms.

Since X and λ should vary between 0 and 1, direct integration is tricky. To circumvent this problem, we introduce auxiliary variables θ and θ_x , where $\lambda = \sin^2(\theta)$ and $x = \sin^2(\theta_x)$. These variables do not have hard barriers and can adopt any real value. Therefore, rather than computing $\partial U/\partial \lambda$ and $\partial U/\partial x$, we compute $\partial U/\partial \theta$ and $\partial U/\partial \theta_x$ and integrate θ and θ_x .

Because the method does not describe bond breakage and formation, the absolute pK_a cannot be calculated. Instead, the difference between the pK_a of the titratable site in the solute relative to the pK_a of a model compound or peptide in solution is calculated. The latter is called the model pK_a , which has been experimentally determined. Model compounds are blocked single amino acids and model peptides are pentapeptides AAXAA, where X is the titratable residue. The model potentials of mean force (PMF) in λ and if necessary also X ($U^{\text{mod}}(\lambda, X)$) are calculated, and these PMF's are subtracted from the potential energy, leading to an approximately flat PMF for the residue at its model pK_a . To describe the pH dependence of the protonation state, we add a pH-dependent free energy (U_{pH}), and to ensure that the system remains near the endpoints of λ and X we add a quadratic penalty potential centered in the middle point of lambda (U_{bar}) to the potential energy. More details of the continuous CpHMD formalism can be found in the original work[683, 684] as well as a recent tutorial.[692]

For the GB calculations, the intrinsic Born radius is, in principle, dependent on λ . However, the implementation would become very complicated. Thus, we made the following compromises. For Lys and His, the contributions from the dummy hydrogens are present in both the protonated and deprotonated state. For Asp and Glu, the contributions of the dummy hydrogens to the Born radii calculations are excluded for both the protonated and deprotonated states. Additionally, the intrinsic Born radius of the His hydrogens is reduced from the GBNeck2 (igb = 8, mbondi3) default value of 1.3 to 1.17 Å to reduce the salt-bridge formation.[685] Note, the GB parameters for Cys are modified to compensate for the overly small desolvation penalty. We set the scaling parameter (Sx) of sulfur to that of oxygen (1.061039) and increased the intrinsic Born radius from the default value of 1.8 to 2.0 Å.[693]

For Asp and Glu, only one proton is ever present on the side chain. Therefore, to avoid the interactions between the two dummy hydrogens, we used the parmed utility to add an explicit exclusion between these atoms.

29.2. Usage description

When setting up a continuous GB-CpHMD run in LEaP, a ff14SB force field should be loaded, and the PBradii should be set to mbondi3 (GBNeck2). Next, phmd.lib should be loaded to add the definitions of AS2 and GL2, which are ASP and GLU with two dummy hydrogens on the carboxylate oxygens. Also, frmod.phmd should be loaded to add some dihedrals necessary for these new residue types and to increase the barrier between the syn- and anti- configurations of the carboxylate oxygens in AS2 and GL2 such that the syn- configuration is maintained. The syn- configuration has been demonstrated to be more energetically favorable in quantum mechanical calculations[684], and since the carboxylate oxygens in the anti- configuration cannot turn back to the syn- configuration due to the nature of dummy hydrogens, the current continuous CpHMD methods maintain the syn- configuration through raising the dihedral barrier.[684] Note, it is conceivable that an anti- configuration might be stabilized in a particular protein environment; this is a topic of future study.

Note, the GB-CpHMD based on the GBNeck2 model and ff14SB has been extensively validated for titration of Asp, Glu, His, Cys, and Lys.[685, 686, 693, 694] If desired, other GB models or force fields may be used; however, new parameters for the model compounds/peptides would need to be derived. ASP and GLU residues should be identified as AS2 and GL2, and HIS residues as HIP so that the proper dummy hydrogens will be added. We provide scripts to assist the users in generating GB-CpHMD input files, including the Python [scripts](#) for running asynchronous replica-exchange simulations.

For performing PME-CpHMD simulations based on Amber force fields, the steps are similar as above, except that all GB-related settings should be removed and instead the user should add the TIP3P water box and ions as for typical all-atom MD simulations. When adding ions, calculate the system charge by treating titratable residue charges as they are under pH 7.0 condition. As a result, the number of positive and negative ions should be added manually in leap. For convenience, we have scripts and instructions to prepare [PME-CpHMD scripts](#). Note that

the PME-CpHMD version is only supported on the GPU platform. The parameters files for CHARMM22, AMBER ff14sb and ff19sb force fields can be found in the same repository under the Parameters folder.

Simulations with CHARMM22 force field are also supported for PME-CpHMD and the system can be built with charmm scripts and then be converted to Amber prmtop and crd files using charmber in ParmEd. Note that the Asp, Glu, His, Cys, and Lys residue names are ASP, GLU, HIP, CYS, and LYS, different from those for Amber force fields.

To use the GBNeck2 and PME based continuous CpHMD, set *iphmd*=1 and 3 respectively in the mdin file. The solution pH is set by *solvph*. When *iphmd*=1/3 pmemd takes several additional command line flags. First, a phmd input file with the namelist &phmdin as described below should be provided with the command-line flag -phmdin. Second, a phmd parameter file (which contains the PMFs of titrating model compounds/peptides) with the namelist &phmdparm should be provided with the command-line flag -phmdparm. Third, a phmd restart file with the namelist &phmdrst will be written to the path specified by the command-line flag -phmdrst. Fourth, a phmd λ file will be output to the path specified by the command-line flag -phmdout, and finally an optional restart file can be provided with the command-line flag -phmdstrt, which can be used to restart a previous run or to set the titration variables during model potential calculations.

The definitions and default values of the variables set in these namelists follow:

29.2.1. Variables in the &phmdin namelist

<code>nsolute</code>	Number of solute residues in the system (not used anymore).
<code>phmdcut</code>	The cutoff distance in angstroms to use in GB calculations in the constant pH simulation. This should in general be very large, if not large enough to encompass the whole system. Default is 1000 Å.
<code>qmass_phmd</code>	The mass of the virtual particles associated with the titration coordinates in amu. In general, this should be roughly as large as the largest masses in the system. Default is 10 amu.
<code>temp_phmd</code>	The temperature of the virtual particles associated with the titration coordinates in Kelvin. Default is 300 K.
<code>phbeta</code>	The friction constant for the Langevin integrator in ps ⁻¹ . Default is 5.0 ps ⁻¹ .
<code>iphfrq</code>	The number of steps between updates of the titration coordinates. Default is 1.
<code>qphmdstart</code>	Should the velocities of the virtual particles be regenerated? If true, these velocities are sampled from the Boltzmann distribution. Otherwise, they are read in from the start file. It should be set as false for thermodynamic integration (parameterization) simulations for CpHMD. Default is true.
<code>nprint_phmd</code>	How many steps there should be between prints to the λ file. Default is set to the same as the trajectory printing frequency.
<code>prlam</code>	Should the λ values be printed to the λ file? Default is false.
<code>prderiv</code>	Should the θ and $\partial U/\partial\theta$ information be output to the output file? Used for parameterization of new residues. Default is false.
<code>prnlev</code>	Determines what gets printed during continuous CpHMD. If it greater than or equal to 0, the header information in the output file is printed. If it is greater than 2, the full output file is generated. If it is greater than or equal to 5 more diagnostic data is printed to the mdout file. Default is 6.
<code>outu</code>	The unit for printing continuous CpHMD diagnostic information. Default is 6.
<code>phtest</code>	If equal to 1, θ and θ_x are fixed. Used for parameterization. Default is 0.
<code>masktitres</code>	The names of the titratable residues.
<code>masktitrrestypes</code>	The number of entries in masktitres.

29.2.2. Variables in the &phmdparm namelist

<code>ngt</code>	The number of titratable residue types defined in the parm file.
<code>numch</code>	An array of the numbers of atoms in the titratable residues defined in the parm file.
<code>res_name</code>	An array of the names of the titratable residue types in the parm file.
<code>res_type</code>	An array defining the residue types of the titratable residues in the parm file. • -2 – coions titrating with linked titratable residues to maintain constant charge, not currently used. • 0 – residues with a single titratable hydrogen (eg. lysine). • 2 – residues with two deprotonated states and a single protonated state with the two deprotonated states having different pK_a's (eg. histidine) • 4 – residues with two protonated states and a single deprotonated state where the two states have the same pK_a (eg. aspartic and glutamic acids).
<code>atom_name</code>	A two-dimensional array containing an array for each residue in <code>res_name</code> containing the names of the atoms in the force field.
<code>ch</code>	A two dimensional array containing an array for each residue in <code>res_name</code> containing the charges used in the dynamics of the titration coordinates. For residues of types 0 and -2 the charges of the protonated state are listed followed by the charges of the deprotonated state. For residues of type 2 the charges of the protonated state are followed by the charges of the two deprotonated states. For residues of type 4 the charges of the two protonated states are listed followed by the charges of the deprotonated state.
<code>ch_md</code>	The same as <code>ch</code> except that it contains the charges used for the calculation of the spatial forces.
<code>rad</code>	A two dimensional array containing an array for each residue in <code>res_name</code> containing flags identifying which atoms are disappearing during the calculation. Atoms which are going to disappear are identified with 1.0 in the deprotonated flags and 0.0 in the protonated flags. Atoms which are always present are identified as 0.0 in both sets of flags. For residues of type -2 or 0, first the flags corresponding to the protonated state are listed followed by those for the deprotonated state. For residues of type 2 the flags of the protonated state are followed by the flags for the two deprotonated states. For residues of type 4 the flags for the two protonated states are followed by those for the deprotonated state.
<code>model_pka</code>	A two dimensional array containing an array for each residue in <code>res_name</code> . For residues of type -2 or 0, the first entry in the array is the target pK_a of the residue. For residues of type 2 the two entries of the array correspond to the 2 pK_a 's of the two tautomers. For residues of type 4 the target pK_a is the first entry in the array.
<code>parameters</code>	A two dimensional array containing an array for each residue in <code>res_name</code> containing the parameters of the model potential for each residue. The first two entries in each array are A and B . For residues of types 2 and 4 the third, fourth, fifth, and sixth entries are A_0 , B_0 , A_{10} , and B_{10} . For residues of type 4 entries 7-12 are R_1 - R_6 .
<code>bar</code>	A two dimensional array containing an array for each residue in <code>res_name</code> containing the heights of the barriers in the model potentials. For residues of types -2 and 0 the first entry in this array is the barrier height in λ . For residues of type 2 and 4 the first entry in this array is the barrier height in X , and the second entry is the barrier height in λ .

29.2.3. Variables in the &phmdstrt namelist

ph_theta The θ and θ_X values of the titration coordinates.

vph_theta The velocities of the titration coordinates.

29.2.4. Example phmdin and phmdparm files

Here is an example of a phmdin file,

```
&phmdin
  NPrint_PHMD = 250,
  PrLam = .true.,
  MaskTitrRes(:) = 'AS2','GL2','HIP','CYS',
  MaskTitrResTypes = 4,
/
```

This file instructs the code to write λ to the λ files (prlam=true) every 250 steps. Residues named AS2, GL2, HIP, and CYS will be titrating.

Here is an example of a phmdparm file built for the systems set up with the procedure described above,

```
&phmdparm
  NGT = 5,
  NUMCH(:) = 14,17,18,22,11,
  RES_NAME(:) = 'AS2','GL2','HIP','LYS','CYS',
  RES_TYPE(:) = 4,4,2,0,0
  ATOM_NAME(1,:) = 'N','H','CA','HA','CB','HB2','HB3','CG','OD1','OD2',
    'HD2','C','O','HD1',
  ATOM_NAME(2,:) = 'N','H','CA','HA','CB','HB2','HB3','CG','HG2','HG3',
    'CD','OE1','OE2','HE2','C','O','HE1',
  ATOM_NAME(3,:) = 'N','H','CA','HA','CB','HB2','HB3','CG','ND1','HD1',
    'CE1','HE1','NE2','HE2','CD2','HD2','C','O',
  ATOM_NAME(4,:) = 'N','H','CA','HA','CB','HB2','HB3','CG','HG2','HG3',
    'CD','HD2','HD3','CE','HE2','HE3','NZ','HZ1','HZ2','HZ3','C','O',
  ATOM_NAME(5,:) = 'N','H','CA','HA','CB','HB2','HB3','SG','HG','C','O'
  CH(1,:) = -0.415700,0.271900,0.034100,0.086400,-0.031600,0.048800,
    0.048800,0.646200,-0.637600,-0.555400,0.00,0.597300,-0.567900,
    0.4747,-0.415700,0.271900,0.034100,0.086400,-0.031600,0.048800,
    0.048800,0.646200,-0.555400,-0.637600,0.474700,0.597300,-0.567900,
    0.00,-0.415700,0.271900,0.034100,0.086400,-0.178200,-0.012200,
    -0.012200,0.799400,-0.801400,-0.801400,0.00,0.597300,-0.567900,0.00,
  CH(2,:) = -0.415700,0.271900,0.014500,0.077900,-0.007100,0.025600,
    0.025600,-0.017400,0.043000,0.043000,0.680100,-0.651100,-0.583800,
    0.00,0.597300,-0.567900,0.4641,-0.415700,0.271900,0.014500,
    0.077900,-0.007100,0.025600,0.025600,-0.017400,0.043000,0.043000,
    0.680100,-0.583800,-0.651100,0.464100,0.597300,-0.567900,0.00,
    -0.415700,0.271900,0.014500,0.077900,-0.039800,-0.017300,-0.017300,
    0.013600,-0.042500,-0.042500,0.805400,-0.818800,-0.818800,0.00,0.597300,
    -0.567900,0.00,
  CH(3,:) = -0.347900,0.274700,-0.135400,0.121200,-0.041400,0.081000,
    0.081000,-0.001200,-0.151300,0.386600,-0.017000,0.268100,
    -0.171800,0.391100,-0.114100,0.231700,0.734100,-0.589400,-0.347900,
    0.274700,-0.135400,0.121200,-0.111000,0.040200,0.040200,-0.026600,
    -0.381100,0.364900,0.205700,0.139200,-0.572700,0.00,0.129200,
    0.114700,0.734100,-0.589400,-0.347900,0.274700,-0.135400,0.121200,
```

```

-0.101200,0.036700,0.036700,0.18680,-0.543200,0.00,0.163500,0.143500,
-0.279500,0.333900,-0.220700,0.186200,0.734100,-0.589400,
  CH(4,:) = -0.347900,0.274700,-0.240000,0.142600,-0.009400,0.036200,
    0.036200,0.018700,0.010300,0.010300,-0.047900,0.062100,0.062100,
-0.014300,0.113500,0.113500,-0.38540,0.340000,0.340000,0.340000,
    0.734100,-0.589400,-0.347900,0.274700,-0.240000,0.142600,-0.109600,
    0.034000,0.034000,0.066120,0.010410,0.010410,-0.037680,0.011550,
    0.011550,0.326040,-0.033580,-0.033580,-1.035810,0.00,0.386040,
    0.386040,0.734100,-0.589400,
  CH(5,:) = -0.415700,0.271900,0.021300,0.112400,-0.123100,0.111200,
    0.111200,-0.311900,0.193300,0.597300,-0.567900,-0.415700,0.271900,
    0.021300,0.112400,-0.3593,0.112200,0.112200,-0.884400,0.00,0.597300,
-0.567900
  CH_MD(1,:) = -0.415700,0.271900,0.034100,0.086400,-0.031600,0.048800,
    0.048800,0.646200,-0.637600,-0.555400,0.00,0.597300,-0.567900,0.4747,
-0.415700,0.271900,0.034100,0.086400,-0.031600,0.048800,0.048800,0.646200,
-0.555400,-0.637600,0.474700,0.597300,-0.567900,0.00,-0.516300,0.293600,
    0.038100,0.088000,-0.030300,-0.012200,-0.012200,0.799400,-0.801400,
-0.801400,0.00,0.536600,-0.581900,0.00,
  CH_MD(2,:) = -0.415700,0.271900,0.014500,0.077900,-0.007100,0.025600,
    0.025600,-0.017400,0.043000,0.043000,0.680100,-0.651100,-0.583800,
    0.00,0.597300,-0.567900,0.4641,-0.415700,0.271900,0.014500,0.077900,
-0.007100,0.025600,0.025600,-0.017400,0.043000,0.043000,0.680100,
-0.583800,-0.651100,0.464100,0.597300,-0.567900,0.00,-0.516300,
    0.293600,0.039700,0.110500,0.056000,-0.017300,-0.017300,0.013600,
-0.042500,-0.042500,0.805400,-0.818800,-0.818800,0.00,0.536600,-0.581900,
    0.00,
  CH_MD(3,:) = -0.347900,0.274700,-0.135400,0.121200,-0.041400,0.081000,
    0.081000,-0.001200,-0.151300,0.386600,-0.017000,0.268100,-0.171800,
    0.391100,-0.114100,0.231700,0.734100,-0.589400,-0.415700,0.271900,
    0.018800,0.088100,-0.046200,0.040200,0.040200,-0.026600,-0.381100,
    0.364900,0.205700,0.139200,-0.572700,0.00,0.129200,0.114700,0.597300,
-0.567900,-0.415700,0.271900,-0.058100,0.136000,-0.007400,0.036700,
    0.036700,0.18680,-0.543200,0.00,0.163500,0.143500,-0.279500,0.333900,
-0.220700,0.186200,0.597300,-0.567900,
  CH_MD(4,:) = -0.347900,0.274700,-0.240000,0.142600,-0.009400,0.036200,
    0.036200,0.018700,0.010300,0.010300,-0.047900,0.062100,0.062100,
-0.014300,0.113500,0.113500,-0.38540,0.340000,0.340000,0.340000,
    0.734100,-0.589400,-0.415700,0.271900,-0.072060,0.099400,-0.048450,
    0.034000,0.034000,0.066120,0.010410,0.010410,-0.037680,0.011550,
    0.011550,0.326040,-0.033580,-0.033580,-1.035810,0.00,0.386040,
    0.386040,0.597300,-0.567900,
  CH_MD(5,:) = -0.415700,0.271900,0.021300,0.112400,-0.123100,0.111200,
    0.111200,-0.311900,0.193300,0.597300,-0.567900,-0.415700,0.271900,
-0.035100,0.050800,-0.241300,0.112200,0.112200,-0.884400,0.00,0.597300,
-0.567900
  RAD(1,:) = 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,
    0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
    0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
  RAD(2,:) = 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
    0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
    1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
    0.0,0.0,0.0,0.0,0.0,

```

```

RAD(3,:) = 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,
0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,
RAD(4,:) = 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
RAD(5,:) = 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,
0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
MODEL_PKA(1,:) = 3.5,
MODEL_PKA(2,:) = 4.2,
MODEL_PKA(3,:) = 6.1,6.6,
MODEL_PKA(4,:) = 10.4,
MODEL_PKA(5,:) = 8.5,
BAR(1,:) = 2.5,2.5,
BAR(2,:) = 2.5,2.5,
BAR(3,:) = 2.5,2.5,
BAR(4,:) = 2.5,
BAR(5,:) = 2.5,
PARAMETERS(1,:) = -60.0334,0.305108,-59.876,0.304068,-21.555,
0.497647,-19.9303,41.6005,-21.5678,0.51803,-59.8132,0.304052,
PARAMETERS(2,:) = -52.4396,0.447635,-52.1586,0.447125,-23.1202,
0.499933,-21.5247,44.8027,-23.166,0.50406,-52.1457,0.447245,
PARAMETERS(3,:) = -47.5760265552,0.487738121,-45.3526708755,
0.4528747997,-38.6267713735,0.5382055867,
PARAMETERS(4,:) = -55.665,0.6655,
PARAMETERS(5,:) = -80.3161983,0.053018419,
/

```

29.3. Continuous constant pH MD with pH replica exchange

The basic procedure of running a pH replica exchange simulation with continuous CpHMD is identical to that used with the other constant pH methods in Amber except that the groupfile needs to be modified to include the appropriate command-line flags and the phmdin and phmdparm input files must be supplied. After running the replica exchange simulation trajectories with the frames sorted by pH can be extracted in the same manner using cpptraj. Analyzing the results of the simulations is different, however, as cphstats does not work with the output files from continuous CpHMD, a different method of analyzing the λ files must be used. First, the replica exchange log should be reformatted in a simpler form with the code walker_extraction. This program takes three arguments, the path to the replica exchange log file, the number of steps between replica exchange attempts, and the total number of replica swaps attempted. For example, if the replica exchange log file were saved as rem.log, swaps were attempted every 1000 steps, and the total simulation length was 500,000 steps (500 swaps), the command to process the log file would be

```
walker_extraction rem.log 1000 500
```

After the output of this command is saved to a file, the λ files produced from the replica exchange simulation have to be sorted into new λ files containing the results from a single pH. This sorting is performed by the program wrap_traj. It takes six arguments:

1. The path to the simplified replica exchange log produced in the previous step.
2. The path to a list of the pH values used in the replica exchange simulation. Eg., if the simulation were run with pH's ranging from 4-7 in increments of 0.5 units, this file would look like

```

4
4.5
5
5.5
6
6.5
7

```

3. The path to a file containing a list of the λ files produced by the replica exchange simulation arranged in order by pH.
4. The number of steps in the simulation (Eg. for a 2 fs time step 1 ns would correspond to 500000 steps).
5. The number of steps between replica exchange attempts.
6. The number of steps between prints to the λ files.

For example, if the simplified replica exchange log were saved as repwalk.dat, the list of pH's were saved as ph_list.dat, the list of λ files were saved as file_list.dat, the simulation was run for 1 ns with a 2 fs time step, replica exchanges were attempted every 1000 steps, and the λ values were printed every 1000 steps, the command would be

```
wrap_traj repwalk.txt ph_list.dat file_list.dat 500000 1000 1000
```

This command will produce a set of files called out.phPH.lambda, where PH is the pH of the λ file. These files now have to be processed to extract the pK_a 's of the residues. First, remove all files ending in .lambda from the directory except these out.phPH.lambda files. Now, call the script renameLamb.sh. This script takes two arguments, the first lambda value that you want to consider, and the final lambda value that you want to consider. For example, for the run described above there will be 500 lambda values in each file. If you wanted to include all of these values in your analysis you would run

```
renameLamb.sh 0 500
```

This script will rename the files as phPH.lambda and produce .sx files needed for the next step.

Finally, you need to fit the data to the Henderson-Hasselbalch equation to obtain the pK_a 's. This is done with the script getS_fitpKa_plot_taut.sh. This script calls xmgrace and gnuplot, which must be installed on the machine to perform the fit and plot the titration curves. It takes the string pH, a name string that will be used to name the resulting files, the lowest pH you used, the highest pH you used, an initial guess for the pK_a (6 tends to work well), and an initial guess for the Hill coefficient (1 works well). Eg.,

```
getS_fitpKa_plot_taut.sh ph BBL 4 7 6 1
```

This script will create several files, including a file NAME_all_pka.dat, which contains the pK_a values for all titratable residues in the protein and .png files showing the titration curves for all of the titratable residues. All the above mentioned analysis scripts in this section can be found in our [CpHMD-Analysis](#) repository. For CpHMD simulations run with asynchronous replica-exchange through our Python [script](#), the results can be analyzed using the [example jupyter-notebook](#). The main difference is that the lambda files are for individual pHs.

29.4. Obtaining parameters for a novel titratable group

The first step to obtaining parameters for a novel residue is to construct a model compound or peptide with an experimental pK_a . For the standard protein residues, we use a pentapeptide AAXAA, where X is the titratable residue. Next, a phmdparm file should be created with entries containing the atom names, charges, and flags for identifying which atoms are present in the van der Waals calculations in each state. The parameters can be left as zeros, as they will be determined once the thermodynamic integration (TI) simulations and fitting are done. Next, a series of TI simulations are run to obtain mean forces, $\langle \partial U / \partial \lambda \rangle$ and for double-site titratable residues also

29. Continuous constant pH molecular dynamics

$\langle \partial U / \partial X \rangle$, at a series of values of λ and X . These values should be set in the phmdstrt file, and the phmdin file should have the phtest and printderiv flags set to true and the printlam flag set to false. The resulting TI simulations will yield estimates of $\langle \partial U / \partial \lambda \rangle$ and if necessary also $\langle \partial U / \partial X \rangle$ which will be fit to the analytical function described in earlier publications.[684] The resulting parameters can then be added to the phmdparm file along with the model pK_a value. A test pH replica-exchange titration simulation should be run to verify that the model pK_a is obtained. An example is given on Gitlab [CpHMD thermodynamic integration](#) . More explanation is also given in the recent tutorial. [692]

30. NMR refinement

We find the *sander* module to be a flexible way of incorporating a variety of restraints into a optimization procedure that includes energy minimization and dynamical simulated annealing. The "standard" sorts of NMR restraints, derived from NOE and J-coupling data, can be entered in a way very similar to that of programs like DISGEO, DIANA or X-PLOR; an aliasing syntax allows for definitions of pseudo-atoms, connections with peak numbers in spectra, and the use of "ambiguous" constraints from incompletely-assigned spectra. More "advanced" features include the use of time-averaged constraints, use of multiple copies (LES) in conjunction with NMR refinement, and direct refinement against NOESY intensities, paramagnetic and diamagnetic chemical shifts, or residual dipolar couplings. In addition, a key strength of the program is its ability to carry out the refinements (usually near the final stages) using an explicit-solvent representation that incorporates force fields and simulation protocols that are known to give pretty accurate results in many cases for unconstrained simulations; this ability should improve predictions in regions of low constraint density and should help reduce the number of places where the force field and the NMR constraints are in "competition" with one another.

Since there is no generally-accepted "recipe" for obtaining solution structures from NMR data, the comments below are intended to provide a guide to some commonly-used procedures. Generally speaking, the programs that need to be run to obtain NMR structures can be divided into three parts:

1. *front-end* modules, which interact with NMR databases that provide information about assignments, chemical shifts, coupling constants, NOESY intensities, and so on. We have tried to make the general format of the input straightforward enough so that it could be interfaced to a variety of programs. We generally use the FELIX and NMRView codes, but the principles should be similar for other ways of keeping track of a database of NMR spectral information. As the flow-chart in Section 30.7 indicates, there are only a few files that need to be created for NMR restraints; these are indicated by the solid rectangles. The primary distance and torsion angle files have a fairly simple format that is largely compatible with the DIANA programs; if one wishes to use information from ambiguous or overlapped peaks, there is an additional "MAP" file that makes a translation from peak identifiers to ambiguous (or partial) assignments. Finally, there are some specialized (but still pretty straightforward) file formats for chemical shift or residual dipolar coupling restraints.

There are a variety of tools, besides the ones described below, that can assist in preparing input for structure refinement in Amber.

- The SANE (Structure Assisted NOE Evaluation) package, <https://ambermd.org/sane.zip>, is widely used at The Scripps Research Institute.[695]
 - If you use Bruce Johnson's NmrView package, you might also want to look at the additions to that: http://garbanzo.scripps.edu/nmrgrp/wisdom/pipe/tips_scripts.html. In particular, the *xpkTOupl* and *starTOupl* scripts there convert NmrView peak lists into the "7-column" needed for input to makeDIST_RST.
 - Users of the MARDIGRAS programs from UCSF can use the *mardi2amber* program to do conversion to Amber format: <http://picasso.ucsf.edu/mardihome.html>
2. *restrained molecular dynamics*, which is at the heart of the conformational searching procedures. This is the part that *sander* itself handles.
 3. *back-end* routines that do things like compare families of structures, generate statistics, simulate spectra, and the like. For many purposes, such as visualization, or the running of procheck-NMR, the "interface" to such programs is just the set of PDB files that contain the family of structures to be analyzed. These general-purpose structure analysis programs are available in many locations and are not discussed here. The

principal *sander*-specific tool is *sviol*, which prepares tables and statistics of energies, restraint violations, and the like.

30.1. Distance, angle and torsional restraints

Distance, angle, and other restraints are read from the DISANG file if *nmropt* > 0. Namelist *rst* ("&rst") contains the following variables; it is read repeatedly until a namelist &rst statement is found with IAT(1)=0, or until reaching the end of the DISANG file.

[In many cases, the user will not prepare this section of the input by hand, but will use the auxiliary programs *makeDIST_RST*, *makeANG_RST* and *makeCHIR_RST* to prepare input from simpler files. See also the programs *cyanarest_to_amberRST* and *nef_to_RST* if you have restraints in Cyana or NEF (NMR Exchange Format) formats.]

30.1.1. Variables in the &rst namelist:

iat(1) → *iat*(8)

- If *IRESID* = 0 (*normal operation*): The atoms defining the restraint. Type of restraint is determined (in order) by:

1. If IAT(3) = 0, this is a distance restraint.
2. If IAT(4) = 0, this is an angle restraint.
3. If IAT(5) = 0, this is a torsional (or J-coupling, if desired) restraint or a generalized distance restraint of 4 atoms, a type of restraint new as of Amber 10 (*sander* only, see below).
4. If IAT(6) = 0, this is a plane-point angle restraint, a second restraint new as of Amber 10 (*sander* only). The angle is measured between the normal of a plane defined by IAT(1)..IAT(4) and the vector from the center of mass of atoms IAT(1)..IAT(4) to the position of IAT(5). The normal is defined by $(r1 - r2) \times (r3 - r4)$, where *rn* is the position of IAT(*n*).
5. If IAT(7) = 0, this is a generalized distance restraint of 6 atoms (see below).
6. Otherwise, if IAT(1)..IAT(8) are all nonzero, this is a plane- plane angle restraint, a third new restraint type as of Amber 10 (*sander* only, or a generalized distance restraint of 8 atoms (see below). For the plane-plane restraint, the angle is measured between the two normals of the two planes, which are defined by $(r1 - r2) \times (r3 - r4)$ and $(r5 - r6) \times (r7 - r8)$. In the case of either planar restraint, the plane may be defined using three atoms instead of four simply by using one atom twice.

If any of IAT(*n*) are < 0, then a corresponding group of atoms is defined below, and the coordinate- averaged position of this group will be used in place of atom IAT(*n*). A new feature as of Amber 10, atom groups may be used not only in distance restraints, but also in angle, torsion, the new plane restraints, or the new generalized restraints. If this is a distance restraint, and IAT1 < 0, then a group of atoms is defined below, and the coordinate-averaged position of this group will be used in place of the coordinates of atom 1 [IAT(1)]. Similarly, if IAT(2) < 0, a group of atoms will be defined below whose coordinate-averaged position will be used in place of the coordinates for atom 2 [IAT(2)].

- If *IRESID*=1: IAT(1)..IAT(8) point to the *residues* containing the atoms comprising the internal. Residue numbers are the absolute in the entire system. In this case, the variables ATNAM(1)..ATNAM(8) must be specified and give the character names of the atoms within the respective residues. If any of IAT(*n*) are less than zero, then group input will still be read in place of the corresponding atom, as described below.
- Defaults for IAT(1)→IAT(8) are 0.

rstwt(1) → *rstwt*(4) New as of Amber 10 (*sander* only), users may now define a single restraint that is a function of multiple distance restraints, called a "generalized distance coordinate" restraint. The energy of such a restraint has the following form:

$$U = k(w_1|\mathbf{r}_1 - \mathbf{r}_2| + w_2|\mathbf{r}_3 - \mathbf{r}_4| + w_3|\mathbf{r}_5 - \mathbf{r}_6| + w_4|\mathbf{r}_7 - \mathbf{r}_8| - r_0)^2$$

where the weights w_n are given in `rstwt(1)..rstwt(4)` and the positions $\mathbf{r}_n$ are the positions of the atoms in `iat(1)..iat(8)`.

Generalized distance coordinate restraints must be defined with either 4, 6, or 8 atoms and 2, 3, or 4 corresponding nonzero weights in `rstwt(1)..rstwt(4)`. Weights may be any positive or negative real number.

If all the weights in `rstwt(1)..rstwt(4)` are zero and four atoms are given in `iat(1)..iat(4)` for the restraint, the restraint is a torsional or J-coupling restraint. If eight atoms are given in `iat(1)..iat(8)` and all weights are zero, the restraint is a plane-plane angle restraint. However, if the weights are nonzero, the restraint will be a generalized distance coordinate restraint.

Default for `rstwt(1)..rstwt(4)` is 0.0

`restraint` New as of Amber 10 (*sander* only), users may now use a "natural language" system to define restraints by using the `RESTRAINT` character variable. Valid restraints defined in this manner will begin with a "distance()", "angle()", "torsion()", or "coordinate()" keyword. Within the parentheses, the atoms that make up the restraint are specified. Atoms may be defined either with an explicit atom number or by using ambmask format, namely `:(residue#)@(atom name)`. Atoms may be separated by commas, spaces, or parentheses. Additionally, negative integers may be used if atom groupings are defined in other variables in the namelist as described below. In addition to the principle distance, angle, torsion, and coordinate keywords, Some keywords may be used within the principle keywords to define more complicated restraints. The keyword "plane()" may be used once or twice within the parentheses of the "angle()" keyword to define a planar restraint. Defining one plane grouping plus one other atom in this manner will create a plane-point angle restraint as described above. Defining two plane groupings will create a plane-plane angle restraint. The keyword "plane()" may only be used inside of "angle()", and is necessary to define either a plane-point or plane-plane restraint. Within the "coordinate()" keyword, the user must use 2 to 4 "distance()" keywords to define a generalized distance coordinate restraint. The "distance()" keyword functions just like it does when used to define a traditional distance restraint. The user may specify any two atom numbers, masks, or negative numbers corresponding to atom groups defined outside of `RESTRAINT`. Additionally, following each "distance()" keyword inside "coordinate()" the user must specify a real-number weight to be applied to each distance making up the generalized coordinate. The "com()" keyword may be used within any other keyword to define a center of mass grouping of atoms. Within the parenthesis, the user will enter a list of atom numbers or masks. Negative numbers, which correspond to externally-defined groups, may not be used. Any type of parenthetical character, i.e., (), [], or { }, may be used wherever parentheses have been used above.

The following are all examples of valid restraint definitions:

```
restraint = "distance( (45) (49) )"
= "angle ( :21@C5' :21@C4' 108 )"
= "torsion[-1,-1,-1, com(67, 68, 69)]"
= "angle( -1, plane(81, 85, 87, 90) )"
= "angle(plane(com(9,10), :5@CA, 31, 32), plane(14,15,15,16))"
= "coordinate(distance(:5@C3', :6@O5'), -1.0, distance(134,-1), 1.0) "
```

There is a 256 character limit on `RESTRAINT`, so if a particularly large atom grouping is desired, it is necessary to specify a negative number instead of "com()" and define the group as described below. `RESTRAINT` will only be parsed if `IAT(1) = 0`, otherwise the information in `IAT(1) .. IAT(8)` will define the restraint. *Default for restraint is ' '*.

30. NMR refinement

atnam	If IRESID = 1, then the character names of the atoms defining the internal are contained in ATNAM(1)→ATNAM(8). Residue IAT(1) is searched for atom name ATNAM(1); residue IAT(2) is searched for atom name ATNAM(2); etc. <i>Defaults for ATNAM(1)→ATNAM(8) are ' '</i> .
iresid	Indicates whether IAT(I) points to an atom # or a residue #. See descriptions of IAT() and ATNAM() above. If RESTRAINT is used to define the internal instead of IAT(), IRESID has no effect on how RESTRAINT is parsed. However, it will affect the behavior of atom group definitions as described below if negative numbers are specified within RESTRAINT. <i>Default = 0.</i>
nstep1, nstep2	This restraint is applied for steps/iterations NSTEP1 through NSTEP2. If NSTEP2 = 0, the restraint will be applied from NSTEP1 through the end of the run. Note that the first step/iteration is considered step zero (0). <i>Defaults for NSTEP1, NSTEP2 are both 0.</i>
irstyp	Normally, the restraint target values defined below (R1→R4) are used directly. If IRSTYP = 1, the values given for R1→R4 define relative displacements from the current value (value determined from the starting coordinates) of the restrained internal. For example, if IRSTYP=1, the current value of a restrained distance is 1.25, and R1 (below) is -0.20, then a value of R1=1.05 will be used. <i>Default is IRSTYP=0.</i>
ialtd	Determines what happens when a distance restraint gets very large. If IALTD=1, then the potential "flattens out", and there is no force for large violations; this allows for errors in constraint lists, but might tend to ignore constraints that <i>should</i> be included to pull a bad initial structure towards a more correct one. When IALTD=0 the penalty energy continues to rise for large violations. See below for the detailed functional forms that are used for distance restraints. Set IALTD=0 to recover the behavior of earlier versions of <i>sander</i> . Default value is 0, or the last value that was explicitly set in a previous restraint. This value is set to 1 if <i>makeDIST.RST</i> is called with the <i>-altdis</i> flag.
ifvari	If IFVARI > 0, then the force constants/positions of the restraint will vary with step number. Otherwise, they are constant throughout the run. If IFVARI >0, then the values R1A→R4A, RK2A, and RK3A must be specified (see below). <i>Default is IFVARI=0.</i>
ninc	If IFVARI > and NINC > 0, then the change in the target values of of R1→R4 and K2,K3 is applied as a step function, with NINC steps/ iterations between each change in the target values. If NINC = 0, the change is effected continuously (at every step). <i>Default for NINC is the value assigned to NINC in the most recent namelist where NINC was specified. If NINC has not been specified in any namelist, it defaults to 0.</i>
imult	If IMULT=0, and the values of force constants RK2 and RK3 are changing with step number, then the changes in the force constants will be linearly interpolated from rk2→rk2a and rk3→rk3a as the step number changes. If IMULT=1 and the force constants are changing with step number, then the changes in the force constants will be effected by a series of multiplicative scalings, using a single factor, R, for all scalings. <i>i.e.</i> rk2a = R**INCREMENTS * rk2 rk3a = R**INCREMENTS * rk3. INCREMENTS is the number of times the target value changes, which is determined by NSTEP1, NSTEP2, and NINC. <i>Default for IMULT is the value assigned to IMULT in the most recent namelist where IMULT was specified. If IMULT has not been specified in any namelist, it defaults to 0.</i>
r1→r4, rk2, rk3, r1a→r4a, rk2a, rk3a	If IALTD=0, the restraint is a well with a square bottom with parabolic sides out to a defined distance, and then linear sides beyond that. If R is the value of the restraint in question: • R < r1 Linear, with the slope of the "left-hand" parabola at the point R=r1.

- $r1 \leq R < r2$ Parabolic, with restraint energy $k_2(R - r_2)^2$.
- $r2 \leq R < r3$ $E = 0$.
- $r3 \leq R < r4$ Parabolic, with restraint energy $k_3(R - r_3)^2$.
- $r4 \leq R$ Linear, with the slope of the "right-hand" parabola at the point $R=r4$.

For torsional restraints, the value of the torsion is translated by $\pm n \times 360$, if necessary, so that it falls closest to the mean of $r2$ and $r3$. Specified distances are in Angstroms. Specified angles are in degrees. Force constants for distances are in kcal/mol-Å². Force constants for angles are in kcal/mol-rad². (Note that angle positions are specified in degrees, but force constants are in radians, consistent with typical reporting procedures in the literature).

If IALTD=1, distance restraints are interpreted in a slightly different fashion. Again, If R is the value of the restraint in question:

- $R < r2$ Parabolic, with restraint energy $k_2(R - r_2)^2$.
- $r2 \leq R < r3$ $E = 0$.
- $r3 \leq R < r4$ Parabolic, with restraint energy $k_3(R - r_3)^2$.
- $r4 \leq R$ Hyperbolic, with energy $k_3[b/(R - r_3) + a]$, where $a = 3(r_4 - r_3)^2$ and $b = -2(r_4 - r_3)^3$. This function matches smoothly to the parabola at $R = r_4$, and tends to an asymptote of ak_3 at large R . The functional form is adapted from that suggested by Michael Nilges, *Prot. Eng.* **2**, 27-38 (1988). Note that if *ialtd*=1, the value of $r1$ is ignored.

ifvari = 0 The values of $r1 \rightarrow r4$, $rk2$, and $rk3$ will remain constant throughout the run.

> 0 The values $r1a$, $r2a$, $r3a$, $r4a$, $r2ka$ and $r3ka$ are also used. These variables are defined as for $r1 \rightarrow r4$ and $rk2$, $rk3$, but correspond to the values appropriate for NSTEP = NSTEP2: e.g., if *IVARI* > 0, then the value of $r1$ will vary between NSTEP1 and NSTEP2, so that, e.g. $r1(\text{NSTEP1}) = r1$ and $r1(\text{NSTEP2}) = r1a$. Note that you *must* specify an explicit value for *nstep1* and *nstep2* if you use this option. Defaults for $r1 \rightarrow r4, rk2, rk3, r1a \rightarrow r4a, rk2a$ and $rk3a$ are the values assigned to them in the most recent namelist where they were specified. They should always be specified in the first &rst namelist.

r0, k0, r0a, k0a New as of Amber 10 (*sander* only), the user may more easily specify a large parabolic well if desired by using $R0$ and $K0$, and then $R0A$ and $K0A$ if *IFVARI* > 0. The parabolic well will have its zero at $R = R0$ and a force constant of $K0$. These variables simply map the desired parabolic well into $r1 \rightarrow r4$, $rk2$, $rk3$, $r1a \rightarrow r4a$, $rk2a$, and $rk3a$ in the following manner:

- $R1 = 0$ for distance, angle, and planar restraints, $R1 = R0 - 180$ for torsion restraints
- $R1A = 0$ for distance, angle, and planar restraints, $R1A = R0A - 180$ for torsion restraints
- $R2 = R0$; $R3 = R0$
- $R2A = R0A$; $R3A = R0A$
- $R4 = R0 + 500$ for distance restraints, $R4 = 180$ for angle and planar restraints, $R4 = R0 + 180$ for torsion restraints
- $RK2 = K0$; $RK3 = K0$
- $RK2A = K0A$; $RK3A = K0A$

rjcoef(1) $\rightarrow$ rjcoef(3) By default, 4-atom sequences specify torsional restraints. It is also possible to impose restraints on the vicinal 3 J-coupling value related to the underlying torsion. J is related to the torsion τ by the approximate Karplus relationship: $J = A \cos^2(\tau) + B \cos(\tau) + C$. If you specify a nonzero value for either *RJCOEF*(1) or *RJCOEF*(2), then a J-coupling restraint, rather than a torsional restraint, will be imposed. At every MD step, J will be calculated from the Karplus relationship with $A = \text{RJCOEF}(1)$, $B = \text{RJCOEF}(2)$ and $C = \text{RJCOEF}(3)$. In this case, the target values ($R1 \rightarrow R4$, $R1A \rightarrow R4A$) and force constants ($RK2$, $RK3$, $RK2A$, $RK3A$) refer to J-values

for this restraint. RJCOEF(1)->RJCOEF(3) must be set individually for each torsion for which you wish to apply a J-coupling restraint, and RJCOEF(1)->RJCOEF(3) may be different for each J-coupling restraint. With respect to other options and reporting, J-coupling restraints are treated identically to torsional restraints. This means that if time-averaging is requested for torsional restraints, it will apply to J-coupling restraints as well. The J-coupling restraint contribution to the energy is included in the "torsional" total. And changes in the relative weights of the torsional force constants also change the relative weights of the J-coupling restraint terms. Setting RJCOEF has no effect for distance and angle restraints. *Defaults for RJCOEF(1)->RJCOEF(3) are 0.0.*

igr1(i),i=1→200, igr2(i),i=1→200, ... igr8(i),i=1→200 If IAT(n) < 0, then IGRn() gives the atoms defining the group whose coordinate averaged position is used to define "atom n" in a restraint. Alternatively, if RESTRAINT is used to define the internal, then if the nth atom specified is a number less than zero, IGRn() gives the atoms defining the group whose coordinate averaged position is used to define "atom n" in a restraint. If IRESID = 0, absolute atom numbers are specified by the elements of IGRn(). If IRESID = 1, then IGRn(I) specifies the number of the residue containing atom I, and the name of atom I must be specified using GRNAMn(I). A maximum of 200 atoms (N # of atoms if using pmemd) are allowed in any group. Only specify those atoms that are needed. Default value for any unspecified element of IGRn(i) is 0.

fxyz If iat(3)=0 and igr1 and/or igr2 is defined then it is possible to weight the x, y, z components of the force in the restraint to 0 (no force) or 1 (full restraint force). Ex: fxyz=0, 0, 1. This sets no additional restraint force on the x component or y-component of the restraint force, and full z-component restraint force. Default fxyz=1,1,1. Note: When setting fxyz, the r1, r2, r3, r4 values should be set relative to a weighted distance $\sqrt{(w_x * d_x)^2 + (w_y * d_y)^2 + (w_z * d_z)^2}$, so if fxyz=0,0,1 then the only distance taken into account when comparing to r1,r2,r3,r4 is the z distance between the molecule and the center of mass. Note that the DUMPAVE value when outxyz=0 is also just the weighted distance.

outxyz If iat(3)=0 and igr1 and/or igr2 is defined then it is possible to output the x, y, z components of the force in the restraint if outxyz is set to 1. Default outxyz=0. When outxyz is set to 1, the components of the distance and total distance are outputted in DUMPAVE in the order of the x-component, y-component, z-component, total distance.

grnam1(i), i=1→200, grnam2(i),i=1→200, ... grnam8(i),i=1→200 If group input is being specified (IGRn(1) > 0), and IRESID = 1, then the character names of the atoms defining the group are contained in GRNAMn(i), as described above. In the case IAT(1) < 0, each residue IGR1(i) is searched for an atom name GRNAM1(i) and added to the first group list. In the case IAT(2) < 0, each residue IGR2(i) is searched for an atom name GRNAM2(i) and added to the second group list. *Defaults for GRNAMn(i) are ' '*.

ir6 If a group coordinate-averaged position is being used (see IGR1 and IGR2 above), the average position can be calculated in either of two manners: If IR6 = 0, center-of-mass averaging will be used. If IR6=1, the $\langle r^{-6} \rangle^{-1/6}$ average of all interaction distances to atoms of the group will be used. *Default for IR6 is the value assigned to IR6 in the most recent namelist where IR6 was specified. If IR6 has not been specified in any namelist, it defaults to 0.*

ifntyp If time-averaged restraints have been requested (see DISAVE/ANGAVE/TORAVE above), they are, by default, applied to all restraints of the class specified. Time-averaging can be overridden for specific internals of that class by setting IFNTYP for that internal to 1. IFNTYP has no effect if time-averaged restraint are not being used. *Default value is IFNTYP=0.*

ixpk, nxpk These are user-defined integers that can be set for each constraint. They are typically the "peak number" and "spectrum number" associated with the cross-peak that led to this particular distance restraint. Nothing is ever done with them except to print them out in the "violation summaries", so that NMR people can more easily go from a constraint violation to the corresponding peak in their spectral database. Default values are zero.

`iconstr` If `iconstr > 0`, (default is 0) a Lagrangian multiplier is also applied to the two-center internal coordinate defined by IAT(1) and IAT(2). The effect of this Lagrangian multiplier is to maintain the initial orientation of the internal coordinate. The rotation of the vector IAT(1)->IAT(2) is prohibited, though translation is allowed. For each defined two-center internal coordinate, a separate Lagrangian multiplier is used. Therefore, although one can use as many multipliers as needed, defining centers should NOT appear in more than one multiplier. This option is compatible with mass centers (i.e., negative IAT(1) or IAT(2)). ICONSTR can be used together with harmonic restraints. RK2 and RK3 should be set to 0.0 if the two-center internal coordinate is a simple Lagrangian multiplier. An example has been included in \$AMBERHOME/example/lagmul.

Namelist &rst is read for each restraint. Restraint input ends when a namelist statement with `iat(1) = 0` (or `iat(1)` not specified) is found. Note that comments can precede or follow any namelist statement, allowing comments and restraint definitions to be freely mixed.

30.2. NOESY volume restraints

After the previous section, NOESY volume restraints may be read. This data described in this section is only read if `NMROPT = 2`. The molecule may be broken in overlapping submolecules, in order to reduce time and space requirements. Input for each submolecule consists of namelist "&noexp", followed immediately by standard Amber "group" cards defining the atoms in the submolecule. In addition to the submolecule input ("&noexp"), you may also need to specify some additional variables in the `cntrl` namelist; see the "NMR variables" description in that section.

In many cases, the user will not prepare this section of the input by hand, but will use the auxiliary program `makeDIST_RST` to prepare input from simpler files.

Variables in the &noexp namelist:

For each submolecule, the namelist "&noexp" is read (either from *stdin* or from the NOESY redirection file) which contains the following variables. There are no effective defaults for `npeak`, `emix`, `ihp`, `jhp`, and `aexp`: you must specify these.

`npeak(imix)` Number of peaks for each of the "imix" mixing times; if the last mixing time is `mxmix`, set `NPEAK(mxmix+1) = -1`. End the input when `NPEAK(1) < 0`.

`emix(imix)` Mixing times (in seconds) for each mixing time.

`ihp(imix,ipeak)`, `jhp(imix,ipeak)` Atom numbers for the atoms involved in cross-peak "ipeak" at mixing time "imix"

`aexp(imix,ipeak)` Experimental target integrated intensity for this cross peak. If AEXP is negative, this cross peak is part of a set of overlapped peaks. The computed intensity is added to the peak that follows; the next time a peak with `AEXP > 0` is encountered, the running sum for the calculated peaks will be compared to the value of AEXP for that last peak in the list. In other words, a set of overlapped peaks is represented by one or more peaks with `AEXP < 0` followed by a peak with `AEXP > 0`. The computed total intensity for these peaks will be compared to the value of AEXP for the final peak.

`arange(imix,ipeak)` "Uncertainty" range for this peak: if the calculated value is within $\pm$ ARANGE of AEXP, then no penalty will be assessed. Default uncertainties are all zero.

`awt(imix,ipeak)` Relative weight for this cross peak. Note that this will be multiplied by the overall weight given by the NOESY weight change cards in the weight changes section (Section 1). Default values are 1.0, unless `INVWT1`, `INVWT2` are set (see below), in which case the input values of AWT are ignored.

30. NMR refinement

<code>invwt1, invwt2</code>	Lower and upper bounds on the weights for the peaks respectively, such that the relative weight for each peak is 1/intensity if 1/intensity lies between the lower and upper bounds. This is the intensity after being scaled by <i>oscale</i> . The inverse weighing scheme adopted by this option prevents placing too much influence on the strong peaks at the expense of weaker peaks and was previously invoked using the compilation flag "INVWGT". Default values are INVWGT1=INVWGT2=1.0, placing equal weights on all peaks.
<code>omega</code>	Spectrometer frequency, in Mhz. Default is 500. It is possible for different sub-molecules to have different frequencies, but omega will only change when it is explicitly re-set. Hence, if all of your data is at 600 Mhz, you need only set <i>omega</i> to 600. in the first submolecule.
<code>taurot</code>	Rotational tumbling time of the molecule, in nsec. Default is 1.0 nsec. Like <i>omega</i> , this value is "sticky", so that a value set in one submolecule will remain until it is explicitly reset.
<code>taumet</code>	Correlation time for methyl jump motion, in ns. This is only used in computing the intra-methyl contribution to the rate matrix. The ideas of Woessner are used, specifically as recommended by Kalk & Berendsen.[696] Default is 0.0001 ns, which is effectively the fast motion limit. The default is consistent with the way the rest of the rate matrix elements are determined (also in the fast motion limit,) but probably is not the best value to use, since methyl groups appear to have T1 values that are systematically shorter than other protons, and this is likely to arise from the fact that the methyl correlation time can be near to the inverse of the spectrometer frequency. A value of 0.02 - 0.05 ns is probably better than 0.0001, but this is still an active research area, and you are on your own here, and should consult the literature for further discussion.[697] As with <i>omega</i> , <i>taumet</i> can be different for different sub-molecules, but will only change when it is explicitly re-set.
<code>id2o</code>	Flag for determining if exchangeable protons are to be included in the spin-diffusion calculation. If ID2O=0 (default) then all protons are included. If ID2O=1, then all protons bonded to nitrogen or oxygen are assumed to not be present for the purposes of computing the relaxation matrix. No other options exist at present, but they could easily be added to the subroutine <i>indexn</i> . Alternatively, you can manually rename hydrogens in the <i>prmtop</i> file so that they do not begin with "H": such protons will not be included in the relaxation matrix. (Note: for technical reasons, the HOH proton of tyrosine must always be present, so setting ID2O=1 will not remove it; we hope that this limitation will be of minor importance to most users.) The <i>id2o</i> variable retains its value across namelist reads, <i>i.e.</i> its value will only change if it is explicitly reset.
<code>oscale</code>	overall scaling factor between experimental and computed volume units. The experimental intensities are multiplied by <i>oscale</i> before being compared to calculated intensities. This means that the weights WNOESY and AWT always refer to "theoretical" intensity scales rather than to the (arbitrary) experimental units. The <i>oscale</i> variable retains its value across namelist reads, <i>i.e.</i> its value will only change if it is explicitly reset. The initial (default) value is 1.0.

The atom numbers *ihp* and *jhp* are the absolute atom numbers. For methyl groups, use the number of the last proton of the group; for the delta and epsilon protons of aromatic rings, use the delta-2 or epsilon-2 atom numbers. Since this input requires you to know the absolute atom numbers assigned by Amber to each of the protons, you may wish to use the separate *makeDIST_RST* program which provides a facility for more turning human-readable input into the required file for *sander*.

Following the &noexp namelist, give the Amber "group" cards that identify this submolecule. This combination of "&noexp" and "group" cards can be repeated as often as needed for many submolecules, subject to the limits described in the *nmr.h* file. As mentioned above, this input section ends when NPEAK(1) < 0, or when and end-of-file is reached.

30.3. Chemical shift restraints

After reading NOESY restraints above (if any), read the chemical shift restraints in namelist *&shf*, or the pseudocontact restraints in namelist *&pcshift*. Reading this input is triggered by the presence of a SHIFTS line in the I/O redirection section. In many cases, the user will not prepare this section of the input by hand, but will use the auxiliary programs *shifts* or *fantasian* to prepare input from simpler files.

Variables in the &shf namelist.

(Defaults are only available for *shrang*, *wt*, *nter*, and *shcut*; you must specify the rest.)

<code>nring</code>	Number of rings in the system.
<code>natr(i)</code>	Number of atoms in the <i>i</i> -th ring.
<code>iatr(j,i)</code>	Absolute atom number for the <i>j</i> -th atom of the <i>i</i> -th ring.
<code>namr(i)</code>	Eight-character string that labels the <i>i</i> -th ring. The first three characters give the residue name (in caps); the next three characters contain the residue number (right justified); column 7 is blank; column 8 may optionally contain an extra letter to distinguish the two rings of trp, or the 5 or 8 rings of the heme group.
<code>str(i)</code>	Ring current intensity factor for the <i>i</i> -th ring. Older values are summarized by Cross and Wright; [698] more recent empirical parametrizations seem to give improved results. [699, 700]
<code>nprot</code>	Number of protons for which penalty functions are to be set up.
<code>iprot(i)</code>	Absolute atom number of the <i>i</i> -th proton whose shifts are to be evaluated. For equivalent protons, such as methyl groups or rapidly flipping phenylalanine rings, enter all two or three atom numbers in sequence; averaging will be controlled by the <i>wt</i> parameter, described below.
<code>obs(i)</code>	Observed secondary shift for the <i>i</i> -th proton. This is typically calculated as the observed value minus a random coil reference value.
<code>shrang(i)</code>	"Uncertainty" range for the observed shift: if the calculated shift is within $\pm$ SHRANG of the observed shift, then no penalty will be imposed. The default value is zero for all shifts.
<code>wt(i)</code>	Weight to be assigned to this penalty function. Note that this value will be multiplied by the overall weight (if any) given by the SHIFTS command in the assignment of weights (above). Default values are 1.0. For sets of equivalent protons, give a negative weight for all but the last proton in the group; the last proton gets a normal, positive value. The average computed shift of the group will be compared to <i>obs</i> entered for the last proton.
<code>shcut</code>	Values of calculated shifts will be printed only if the absolute error between calculated and observed shifts is greater than this value. <i>Default = 0.3 ppm.</i>
<code>nter</code>	Residue number of the N-terminus, for protein shift calculations; <i>default = 1.</i>
<code>cter</code>	Residue number of the C-terminus, for protein shift calculations. Believe it or not, the current code cannot figure this out for itself.

In typical usage, the *shifts* program (<http://casegroup.rutgers.edu/shifts.html>) would be used to create this file, with a typical command line:

```
shifts -readobs -sander ' ::H*' gcg10
```

Sample input and output files are in \$AMBERHOME/test/rdc.

30.4. Pseudocontact shift restraints

The PCSHIFT module allows the inclusion of pseudocontact shifts as constraints in energy minimization and molecular dynamics calculations on paramagnetic molecules. The pseudocontact shift depends on the magnetic susceptibility anisotropy of the metal ion and on the location of the resonating nucleus with respect to the axes of the magnetic susceptibility tensor. For the nucleus *i*, it is given by:

$$\delta_{pc}^i = \sum_j \frac{1}{12\pi r_{ij}^3} \left[\Delta\chi_{ax}^j (3n_{ij}^2 - 1) + (3/2)\Delta\chi_{rh}^j (l_{ij}^2 - m_{ij}^2) \right]$$

where l_{ij} , m_{ij} , and n_{ij} are the direction cosines of the position vector of atom *i* with respect to the *j*-th magnetic susceptibility tensor coordinate system, r_{ij} is the distance between the *j*-th paramagnetic center and the proton *i*, $\Delta\chi_{ax}$ and $\Delta\chi_{rh}$ are the axial and the equatorial (rhombic) anisotropies of the magnetic susceptibility tensor of the *j*-th paramagnetic center. For a discussion, see Ref. [701].

The PCSHIFT module to be used needs a namelist file which includes information on the magnetic susceptibility tensor and on the paramagnetic center, and a line of information for each nucleus. This module allows to include more than one paramagnetic center in the calculations. To include pseudocontact shifts as constraints in energy minimization and molecular dynamics calculations the NMROPT flag should be set to 2, and a *PCSHIFT=filename* statement entered in the I/O redirection section.

To perform molecular dynamics calculations it is necessary to eliminate the rotational and translational degree of freedom about the center of mass (this because during molecular dynamics calculations the relative orientation between the external reference coordinate system and the magnetic anisotropy tensor coordinate system has to be fixed). This option can be obtained with the NSCM flag of *sander*.

Variables in the pcsshift namelist

nprot	number of pseudocontact shift constraints.
nme	number of paramagnetic centers.
nmpmc	name of the paramagnetic atom
optphi(n), opttet(n), optomg(n), opta1(n), opta2(n)	the five parameters of the magnetic anisotropy tensor for each paramagnetic center.
optkon	force constant for the pseudocontact shift constraints

Following this, there is a line for each nucleus for which the pseudocontact shift information is given has to be added. Each line contains :

iproto(i)	atom number of the <i>i</i> -th proton whose shift is to be used as constraint.
obs(i)	observed pseudocontact shift value, in ppm
wt(i)	relative weight
tolpro(i)	relative tolerance ix mltpro
mltpro(i)	multiplicity of the NMR signal (for example the protons of a methyl group have mltpro(i)=3)

Example

Here is a &pcshf namelist example: a molecule with three paramagnetic centers and 205 pseudocontact shift constraints.


```

&pcshf
nprot=205,
nme=3,
nmpmc='FE ',
optphi(1)=-0.315416,
opttet(1)=0.407499,
optomg(1)=0.0251676,
opta1(1)=-71.233,
opta2(1)=1214.511,
optphi(2)=0.567127,
opttet(2)=-0.750526,
optomg(2)=0.355576,
opta1(2)=-60.390,
opta2(2)=377.459,
optphi(3)=0.451203,
opttet(3)=-0.0113097,
optomg(3)=0.334824,
opta1(3)=-8.657,
opta2(3)=704.786,
optkon=30,
iprot(1)=26, obs(1)=1.140, wt(1)=1.000, tolpro(1)=1.00, mltpro(1)=1,
iprot(2)=28, obs(2)=2.740, wt(2)=1.000, tolpro(2)=.500, mltpro(2)=1,
iprot(3)=30, obs(3)=1.170, wt(3)=1.000, tolpro(3)=.500, mltpro(3)=1,
iprot(4)=32, obs(4)=1.060, wt(4)=1.000, tolpro(4)=.500, mltpro(4)=3,
iprot(5)=33, obs(5)=1.060, wt(5)=1.000, tolpro(5)=.500, mltpro(5)=3,
iprot(6)=34, obs(6)=1.060, wt(6)=1.000, tolpro(6)=.500, mltpro(6)=3,
...
...
iprot(205)=1215, obs(205)=.730, wt(205)=1.000, tolpro(205)=.500,
mltpro(205)=1,
/

```

An *mdin* file that might go along with this, to perform a maximum of 5000 minimization cycles, starting with 500 cycles of steepest descent. PCSHIFT=./pcs.in redirects the input from the namelist "pcs.in" which contains the pseudocontact shift information.

```

Example of minimization including pseudocontact shift constraints
&cntrl
ibelly=0,imin=1,ntpr=100,
ntr=0,maxcyc=500,
ncyc=50,ntmin=1,dx0=0.0001,
drms=.1,cut=10.,
nmropt=2,pencut=0.1, ipnlty=2,
/
&wt type='REST', istep1=0,istep2=1,value1=0.,
value2=1.0, /
&wt type='END' /
DISANG=../noe.in
PCSHIFT=../pcs.in
LISTOUT=POUT

```

30.5. Direct dipolar coupling restraints

Energy restraints based on direct dipolar coupling constants are entered in this section. All variables are in the namelist &align; reading of this section is triggered by the presence of a DIPOLE line in the I/O redirection

section.

When dipolar coupling restraints are turned on, the five unique elements of the alignment tensor are treated as additional variables, and are optimized along with the structural parameters. Their effective masses are determined by the *scal*m parameter entered in the &cntrl namelist. Unlike some other programs, the variables used are the Cartesian components of the alignment tensor in the axis system defined by the molecule itself: e.g. $S_{mn} \equiv \langle (3 \cos \theta_m \cos \theta_n - \delta_{mn})/2 \rangle$, where $m, n = x, y, z$, and θ_x is the angle between the x axis and the spectrometer field.[702] The factor of 10^5 is just to make the values commensurate with atomic coordinates, since both the coordinates and the alignment tensor values will be updated during the refinement. The calculated dipolar splitting is then

$$D_{calc} = - \left(\frac{10^{-5} \gamma_i \gamma_j h}{2\pi^2 r_{ij}^3} \right) \sum_{m,n=xyz} \cos \phi_m \cdot S_{mn} \cdot \cos \phi_n$$

where ϕ_x is the angle between the internuclear vector and the x axis. Geometrically, the splitting is proportional to the transformation of the alignment tensor onto the internuclear axis. This is just Eqs. (5) and (13) of the above reference, with any internal motion corrections (which might be a part of S_{system}) set to unity. If there is an internal motion correction which is the same for all observations, this can be assimilated into the alignment tensor. The current code does not allow for variable corrections for internal motion. See Ref. [703] for a fuller discussion of these issues.

At the end of the calculation, the alignment tensor is diagonalized to obtain information about its principal components. This allows the alignment tensor to be written in terms of the "axial" and "rhombic" components that are often used to describe alignment.

Variables in the &align namelist.

- ndip** Number of observed dipolar couplings to be used as restraints.
- id, jd** Atom numbers of the two atoms involved in the dipolar coupling.
- dobsl, dobsu** Limiting values for the observed dipolar splitting, in Hz. If the calculated coupling is less than *dobsl*, the energy penalty is proportional to $(D_{calc} - D_{obs,l})^2$; if it is larger than *dobsu*, the penalty is proportional to $(D_{calc} - D_{obs,u})^2$. Calculated values between *dobsl* and *dobsu* are not penalized. Note that *dobsl* must be less than *dobsu*; for example, if the observed coupling is -6 Hz, and a 1 Hz "buffer" is desired, you could set *dobsl* to -7 and *dobsu* to -5.
- dwt** The relative weight of each observed value. Default is 1.0. The penalty function is thus:

$$E_{align}^i = D_{wt}^i (D_{calc}^i - D_{obs(u,l)}^i)^2$$
 where D_{wt} may vary from one observed value to the next. Note that the default value is arbitrary, and a smaller value may be required to avoid overfitting the dipolar coupling data.[703]
- dataset** Each dipolar peak can be associated with a "dataset", and a separate alignment tensor will be computed for each dataset. This is generally used if there are several sets of experiments, each with a different sample or temperature, etc., that would imply a different value for the alignment tensor. By default, there is one dataset to which each observed value is assigned.
- num_datasets** The number of datasets in the constraint list. Default is 1.
- s11, s12, s13, s22, s23** Initial values for the Cartesian components of the alignment tensor. The tensor is traceless, so S_{33} is calculated as $-(S_{11}+S_{22})$. In order to have the order of magnitude of the S values be roughly commensurate with coordinates in Angstroms, the alignment tensor values must be multiplied by 10^5 .
- gigj** Product of the nuclear "g" factors for this dipolar coupling restraint. These are related to the nuclear gyromagnetic ratios by $\gamma_N = g_N \beta_N / \hbar$. Common values are $^1\text{H} = 5.5856$, $^{13}\text{C} = 1.4048$, $^{15}\text{N} = -0.5663$, $^{31}\text{P} = 2.2632$.

<code>dij</code>	The internuclear distance for observed dipolar coupling. If a nonzero value is given, the distance is considered to be fixed at the given value. If a <i>dij</i> value is zero, its value is computed from the structure, and it is assumed to be a variable distance. For one-bond couplings, it is usually best to treat the bond distance as "fixed" to an effective zero-point vibration value. ^[704]
<code>dcut</code>	Controls printing of calculated and observed dipolar couplings. Only values where $\text{abs}(\text{dobs}(u,l) - \text{dcalc})$ is greater than <i>dcut</i> will be printed. Default is 0.1 Hz. Set to a negative value to print all dipolar restraint information.
<code>freezemol</code>	If this is set to <i>.true.</i> , the molecular coordinates are not allowed to vary during dynamics or minimization: only the elements of the alignment tensor will change. This is useful to fit just an alignment tensor to a given structure. Default is <i>.false.</i> .

30.6. Residual CSA or pseudo-CSA restraints

Resonance positions in partially aligned media will be shifted from their positions in isotropic media, and this can provide information that is very similar to residual dipolar coupling constraints. This section shows how to input these sorts of restraints. The entry of the alignment tensor is done as in Section 30.5, so you must have a DIPOLE file (with an `&align` namelist) even if you don't have any RDC restraints. Then, if there is a CSA line in I/O redirection section, that file will be read with the following inputs:

Variables in the `&csa` namelist.

<code>nrsa</code>	Number of observed residual CSA peaks to be used as restraints.
<code>icsa, jrsa, krsa</code>	Atom numbers for the csa of interest: <i>jrsa</i> is the atom whose $\Delta\sigma$ value has been measured; <i>icsa</i> and <i>krsa</i> are two atoms bonded to it, used to define the local axis frame for the CSA tensor. See <i>amber12/test/pcsa/RST.csa</i> for examples of how to set these.
<code>cobsl, cobsu</code>	Limiting values for the observed residual CSA, in Hz (not ppm or ppb!). If the calculated value of $\Delta\sigma$ is less than <i>cobsl</i> , the energy penalty is proportional to $(\Delta\sigma_{\text{calc}} - \Delta\sigma_{\text{obs},l})^2$; if it is larger than <i>cobsu</i> , the penalty is proportional to $(\Delta\sigma_{\text{calc}} - \Delta\sigma_{\text{obs},u})^2$. Calculated values between <i>cobsl</i> and <i>cobsu</i> are not penalized. Note that <i>cobsl</i> must be less than <i>cobsu</i> .
<code>cwt</code>	The relative weight of each observed value. Default is 1.0. The penalty function is thus: $E_{\text{rsa}}^i = C_{\text{wt}}^i (\Delta\sigma_{\text{calc}}^i - \Delta\sigma_{\text{obs}(u,l)}^i)^2$ where C_{wt} may vary from one observed value to the next. Note that the default value is arbitrary, and a smaller value may be required to avoid overfitting the data.
<code>datasetc</code>	Each residual CSA can be associated with a "dataset", and a separate alignment tensor will be computed for each dataset. This is generally used if there are several sets of experiments, each with a different sample or temperature, etc., that would imply a different value for the alignment tensor. By default, there is one dataset to which each observed value is assigned. The tensors themselves are entered for each dataset in the DIPOLE file.
<code>field</code>	Magnetic field (in MHz) for the residual CSA being considered here. This is indexed from 1 to <i>nrsa</i> , and is nucleus dependent. For example, if the proton frequency is 600 MHz, then <i>field</i> for ^{13}C would be 150, and that for ^{15}N would be 60.
<code>sigma11, sigma22, sigma12, sigma13, sigma23</code>	Values of the CSA tensor (in ppm) for atom <i>icsa</i> , in the local coordinate frame defined by atoms <i>icsa, jrsa</i> and <i>krsa</i> . See <i>\$AMBERHOME/test/pcsa/RST.csa</i> for examples of how to set these.

`ccut` Controls printing of calculated and observed residual CSAs. Only values where $\text{abs}(\text{cobs}(u,l) - \text{ccalc})$ is greater than `ccut` will be printed. Default is 0.1 Hz. Set to a negative value to print all information.

The residual CSA facility is new as of Amber 10, and has not been used as much as other parts of the NMR refinement package. You should study the example files listed above to see how things work. The residual CSA values should closely match those found by the RAMAH package (<http://www-personal.umich.edu/~hashimi/Software.html>), and testing this should be a first step in making sure you have entered the data correctly.

30.7. Preparing restraint files for Sander

Fig. 30.1 shows the general information flow for auxiliary programs that help prepare the restraint files. Once the restraint files are made, Fig. 30.2 shows a flow-chart of the general way in which *sander* refinements are carried out.

The basic ideas of this scheme owe a lot to the general experience of the NMR community over the past decade. Several papers outline procedures in the Scripps group, from which a lot of the NMR parts of *sander* are derived.[695, 705–709] They are by no means the only way to proceed. We hope that the flexibility incorporated into *sander* will encourage folks to experiment with refinement protocols.

30.7.1. Preparing distance restraints: makeDIST_RST

The *makeDIST_RST* program converts a simplified description of distance bounds into a detailed input for *sander*. A variety of input and output filenames may be specified on the command line:

input:

```
-upb <filename> 7-col file of upper distance bounds, OR
-ual <filename> 8-col file of upper and lower bounds, OR
-vol <filename> 7-col file of NOESY volumes
-pdb <filename> Brookhaven format file
-map <filename> MAP file (default:map.DG-AMBER)
-les <filename> LES atom mappings, made by addles
```

output:

```
-dgm <filename> DGEOM95 restraint format
-rst <filename> SANDER restraint format
-svf <filename> Sander Volume Format, for NOESY refinement
```

other options:

```
-help (gives you this explanation, overrides other parameters)
-report (gives you short runtime diagnostic output)
-nocorr (do not correct upper bound for r*-6 averaging)
-altdis (use alternative form for the distance restraints)
```

The 7/8 column distance bound file is essentially that used by the DIANA or DISGEO programs. It consists of one-line per restraint, which would typically look like the following:

```
23 ALA HA 52 VAL H 3.8 # comments go here
```

The first three columns identify the first proton, the next three the second proton, and the seventh column gives the upper bound. Only the first three letters of the residue name are used, so that DIANA files that contain residues like "ASP-" will be correctly interpreted. An alternate, 8-column, format has both upper and lower bounds as the seventh and eighth columns, respectively. A typical line might in an "8-col" file might look like this:

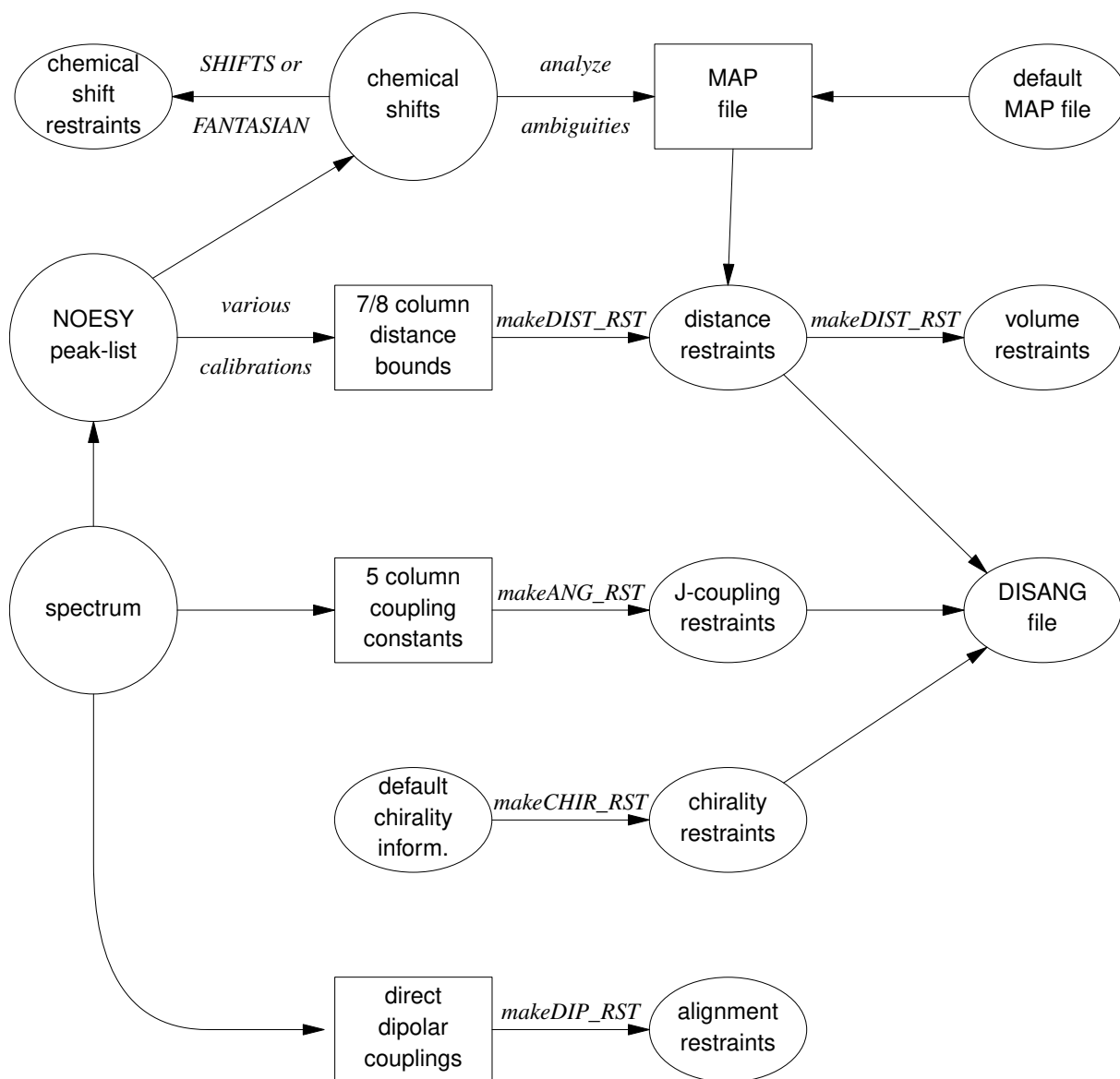


Figure 30.1.: Notation: circles represent logical information, whose format might differ from one project to the next; solid rectangles are in a specific format (largely compatible with DIANA and other programs), and are intended to be read and edited by the user; ellipses are specific to sander, and are generally not intended to be read or edited manually. The conversion of NOESY volumes to distance bounds can be carried out by a variety of programs such as mardigras or xpk2bound that are not included with Amber. Similarly, the analysis and partial assignment of ambiguous or overlapped peaks is a separate task; at TSRI, these are typically carried out using the programs xpkasgn and filter.pl

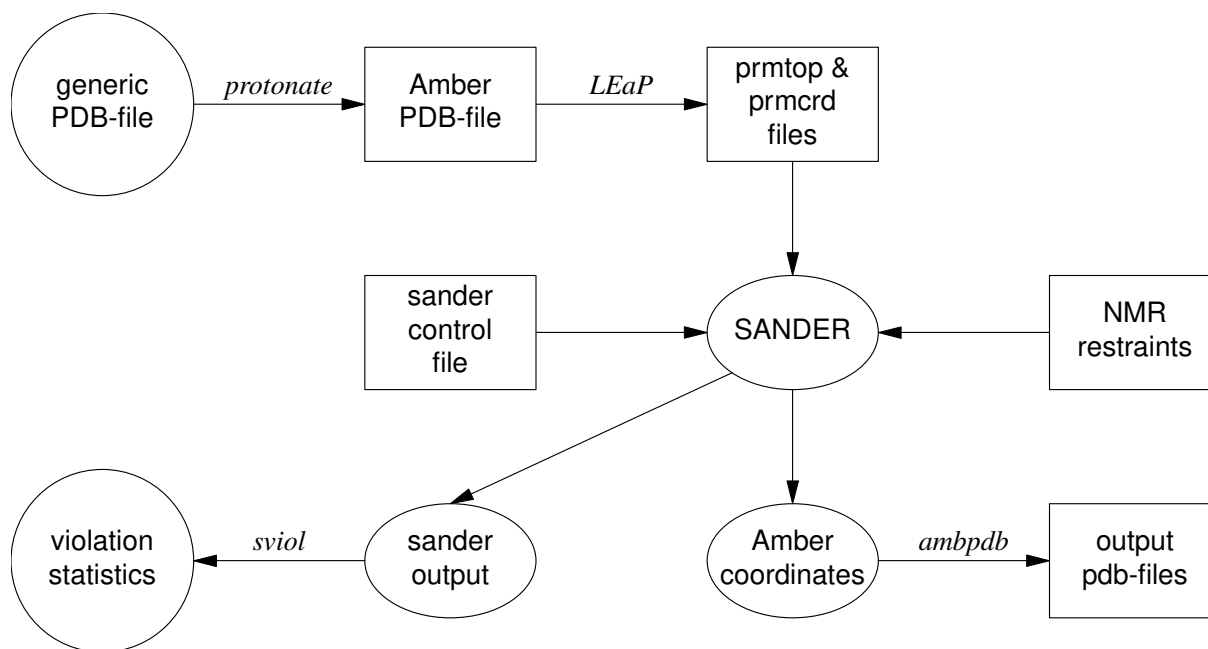


Figure 30.2.: General organization of NMR refinement calculations.

23 ALA HA 52 VAL H 3.2 3.8 # comments go here

Here the lower bound is 3.2 Å and the upper bound is 3.8 Å. Comments typically identify the spectrum and peak-number or other identification that allow cross-referencing back to the appropriate spectrum. If the comment contains the pattern "<integer>:<integer>", then the first integer is treated as a peak-identifier, and the second as a spectrum-identifier. These identifiers go into the *ixpk* and *n timer>* variables, and will later be printed out in *sander*, to facilitate going back to the original spectra to track down violations, etc.

The format for the *-vol* option is the same as for the *-upb* option except that the seventh column holds a peak intensity (volume) value, rather than a distance upper bound.

The input PDB file must exactly match the Amber *prmtop* file that will be used; use the *ambpdb -a atm* command to create this.

If all peaks involved just single protons, and were fully assigned, this is all that one would need. In general, though, some peaks (especially methyl groups or fast-rotating aromatic rings) represent contributions from more than one proton, and many other peaks may not be fully assigned. *Sander* handles both of these situations in the same way, through the notion of an "ambiguous" peak, that may correspond to several assignments. These peaks are given two types of special names in the 7/8-column format file:

1. Commonly-occurring ambiguities, like the lack of stereospecific assignments to two methylene protons, are given names defined in the default MAP file. These names, also more-or-less consistent with DIANA, are like the names of "pseudo-atoms" that have long been used to identify such partially assigned peaks, e.g. "QB" refers to the (HB2,HB3) combination in most residues, and "MG1" in valine refers collectively to the three methyl protons at position CG1, etc.
2. There are generally also molecule-specific ambiguities, arising from potential overlap in a NOESY spectrum. Here, the user assigns a unique name to each such ambiguity or overlap, and prepares a list of the potential assignments. The names are arbitrary, but might be constructed, for example, from the chemical shifts that identify the peak, e.g. "p-2.52" might identify the set of protons that could contribute to a peak at 2.52 ppm. The chemical shift list can be used to prepare a list of potential assignments, and these lists can often be pruned by comparison to approximate or initial structures.

The default and molecule-specific MAP files are combined into a single file, which is used, along with the 7-column restraint file, the the program *makeDIST_RST* to construct the actual *sander* input files. You should

consult the help file for `makeDIST_RST` for more information. For example, here are some lines added to the MAP file for a recent TSRI refinement:

```

AMBIG n2:68 = HE 86 HZ 86
AMBIG n2:72 = HE 24 HD 24 HZ 24
AMBIG n2:73 = HN 81 HZ 13 HE 13 HD 13 HZ 24
AMBIG n2:78 = HN 76 HZ 13 HE 13 HZ 24
AMBIG n2:83 = HN 96 HN 97 HD 97 HD 91
AMBIG n2:86 = HD1 66 HZ2 66
AMBIG n2:87 = HN 71 HH2 66 HZ3 66 HD1 66

```

Here the spectrum name and peak number were used to construct a label for each ambiguous peak. Then, an entry in the restraint file might look like this:

```
123 GLY HN 0 AMB n2:68 5.5
```

indicating a 5.5 Å upper bound between the amide proton of Gly 123 and a second proton, which might be either the HE or HZ protons of residue 86. (The "zero" residue number just serves as a placeholder, so that there will be the same number of columns as for non-ambiguous restraints.) If it is possible that the ambiguous list might not be exhaustive (e.g. if some protons have not been assigned), it is safest to set *ialtd*=1, which will allow "mistakes" to be present in the constraint list. On the other hand, if you want to be sure that every violation is "active", set *ialtd*=0.

If the *-les* flag is set, the program will prepare distance restraints for multiple copies (LES) simulations. In this case, the input PDB file is one *without* LES copies, i.e. with just a single copy of the molecule. The "lesfile" specified by this flag is created by the *addles* program, and contains a mapping from original atom numbers into the copy numbers used in the multiple-copies simulation.

The *-rst* and *-svf* flags specify outputs for *sander*, for distance restraints and NOESY restraints, respectively. In each case, you may need to hand-edit the outputs to add additional parameters. You should make it a habit to compare the outputs with the descriptions given earlier in this chapter to make sure that the restraints are what you want them to be.

It is common to run `makeDIST_RST` several times, with different inputs that correspond to different spectra, different mixing times, etc. It is then expected that you will manually edit the various output files to combine them into the single file required by *sander*.

30.7.2. Preparing torsion angle restraints: `makeANG_RST`

There are fewer "standards" for representing coupling constant information. We have followed the DIANA convention in the program *makeANG_RST*. This program takes as input a five-column torsion angle constraint file along with an Amber PDB file of the molecule. It creates as output (to standard out) a list of constraints in RST format that is readable by Amber.

```

Usage: makeANG_RST -help
makeANG_RST -pdb ambpdb_file [-con constraint] [-lib libfile]
[-les lesfile ]

```

The input torsion angle constraint file can be read from standard in or from a file specified by the *-con* option on the command line. The input constraint file should look something like this:

```

1 GUA PPA 111.5 144.0
2 CYT EPSILN 20.9 100.0
2 CYT PPA 115.9 134.2
3 THY ALPHA 20.4 35.6
4 ADE GAMMA 54.7 78.8
5 GLY PHI 30.5 60.3
6 ALA CHI 20.0 50.0
. . . .

```

30. NMR refinement

Lines beginning with "#" are ignored. The first column is the residue number; the second is the residue name (three letter code, or as defined in your personal torsion library file). Only the first three letters of the residue name are used, so that DIANA files that contain residues like "ASP-" will be correctly interpreted. Third is the angle name (taken from the torsion library described below). The fourth column contains the lower bound, and the fifth column specifies the upper bound. Additional material on the line is (presently) ignored.

Note: It is assumed that the lower bound and the upper bound define a region of allowed conformation on the unit circle that is swept out in a clockwise direction from *lb* $\rightarrow$ *ub*. If the number in the *lb* column is greater than the number in the *ub* column, 360° will successively be subtracted from the *lb* until *lb* < *ub*. This preserves the clockwise definition of the allowed conformation space, while also making the number that specifies the lower bound less than the number that specifies the upper bound, as is required by Amber. If this occurs, a warning message will be printed to *stderr* to notify the user that the data has been modified.

The angles that one can constrain in this manner are defined in the library file that can be optionally specified on the command line with the *-lib* flag, or the default library "tordef.lib" (written by Garry P. Gippert) will be used. If you wish to specify your own nomenclature, or add angles that are not already defined in the default file, you should make a copy of this file and modify it to suit your needs. The general format for an entry in the library is:

```
LEU PSI N CA C N+
```

where the first column is the residue name, the second column is the angle name that will appear in the input file when specifying this angle, and the last four columns are the atom names that define the torsion angle. When a torsion angle contains atom(s) from a preceding or succeeding residue in the structure, a "-" or "+" is appended to those atom names in the library, thereby specifying that this is the case. In the example above, the atoms that define PSI for LEU residues are the N, CA, and C atoms of that same LEU and the N atom of the residue after that LEU in the primary structure. Note that the order of atoms in the definition is important and should reflect that the torsion angle rotates about the two central atoms as well as the fact that the four atoms are bonded in the order that is specified in the definition.

If the first letter of the second field is "J", this torsion is assumed to be a J-coupling constraint. In that case, three additional floats are read at the end of the line, giving the A,B and C coefficients for the Karplus relation for this torsion. For example:

```
ALA JHNA H N CA HA 9.5 -1.4 0.3
```

will set up a J-coupling restraint for the HN-HA 3-bond coupling, assuming a Karplus relation with A,B, C as 9.5, -1.4 and 0.3. (These particular values are from Brüschweiler and Case, JACS 116: 11199 (1994).)

This program also supports pseudorotation phase angle constraints for prolines and nucleic acid sugars; each of these will generate restraints for the 5 component angles which correspond to the *lb* and *ub* values of the input pseudorotation constraint. In the torsion library, a pseudorotation definition looks like:

```
PSEUDO CYT PPA NU0 NU1 NU2 NU3 NU4
CYT NU0 C4' O4' C1' C2'
CYT NU1 O4' C1' C2' C3'
CYT NU2 C1' C2' C3' C4'
CYT NU3 C2' C3' C4' O4'
CYT NU4 C3' C4' O4' C1'
```

The first line describes that a PSEUDOrotation angle is to be defined for CYT that is called PPA and is made up of the five angles NU0-NU4. Then the definition for NU0-NU4 should also appear in the file in the same format as the example given above for LEU PSI.

PPA stands for Pseudorotation Phase Angle and is the angle that should appear in the input constraint file when using pseudorotation constraints. The program then uses the definition of that PPA angle in the library file to look for the 5 other angles (NU0-NU4 in this case) which it then generates restraints for. PPA for proline residues is included in the standard library as well as for the DNA nucleotides.

If the *-les* flag is set, the program will prepare torsion angle restraints for multiple copies (LES) simulations. In this case, the input PDB file is one *without* LES copies, i.e. with just a single copy of the molecule. The "lesfile" specified by this flag is created by the *addles* program, and contains a mapping from original atom numbers into the copy numbers used in the multiple-copies simulation.

Torsion angle constraints defined here cannot span two different copy sets, i.e., there cannot be some atoms of a particular torsion that are in one multiple copy set, and other atoms from the same torsion that are in other copy sets. It is OK to have some atoms with single copies, and others with multiple copies in the same torsion. The program will create as many duplicate torsions as there are copies.

A good alternative to interpreting J-coupling constants in terms of torsion angle restraints is to refine directly against the coupling constants themselves, using an appropriate Karplus relation. See the discussion of the variable `RJCOEF`, above.

30.7.3. Chirality restraints: `makeCHIR_RST`

Usage: `makeCHIR_RST <pdb-file> <output-constraint-file>`

We also find it useful to add chirality constraints and *trans*-peptide ω constraints (where appropriate) to prevent chirality inversions or peptide bond flips during the high-temperature portions of simulated annealing runs. The program `makeCHIR_RST` will create these constraints. Note that you may have to edit the output of this program to change *trans* peptide constraints to *cis*, as appropriate.

30.7.4. Direct dipolar coupling restraints: `makeDIP_RST`

For simulations with residual dipolar coupling restraints, the `makeDIP_RST.protein`, `makeDIP_RST.dna` and `makeDIP_RST.diana` are simple codes to prepare the input file. Use `-help` to obtain a more detailed description of the usage. For now, this code only handles backbone NH and C α H data. The header specifying values for various parameters needs to be manually added to the output of `makeDIP_RST`.

Use of residual dipolar coupling restraints is new both for Amber and for the general NMR community. Refinement against these data should be carried out with care, and the optimal values for the force constant, penalty function, and initial guesses for the alignment tensor components are still under investigation. Here are some suggestions from the experiences so far:

1. Beware of overfitting the dipolar coupling data in the expense of Amber force field energy. These dipolar coupling data are very sensitive to tiny changes in the structure. It is often possible to drastically improve the fitting by making small distortions in the backbone angles. We recommend inclusion of explicit angle restraints to enforce ideal backbone geometry, especially for those residues that have corresponding residual dipolar coupling data.
2. The initial values for the Cartesian components of the alignment tensor can influence the final structure and alignment if the structure is not fixed (`ibelly = 0`). For a fixed structure (`ibelly = 1`), these values do not matter. Therefore, the current "best" strategy is to fit the experimental data to the fixed starting structure, and use the alignment tensor[s] obtained from this fitting as the initial guesses for further refinement.
3. Amber is capable of simultaneously fitting more than one set of alignment data. This allows the use of individually obtained datasets with different alignment tensors. However, if the different sets of data have equal directions of alignment but different magnitudes, using an overall scaling factor for these data with a single alignment tensor could greatly reduce the number of fitting parameters.
4. Because the dipolar coupling splittings depend on the square root of the order parameters ($0 \leq S_2 \leq 1$), these order parameters describing internal motion of individual residues are often neglected (N. Tjandra and A. Bax, *Science* **278**, 1111-1113, 1997). However, the square root of a small number can still be noticeably smaller than 1, so this may introduce undesirable errors in the calculations.

30.7.5. Using NMR exchange format (NEF) files

The NMR community, in collaboration with the worldwide PDB, is developing a common format for encoding of NMR restraints, including all of the kinds discussed above. This format is not yet finalized, but we are including here a conversion script, `nef_to_RST`, that would convert these files to *sander* format. Because this format is so new, and is still subject to revisions, care should be taken in using this script: make sure that the

output files do what they should be doing. Here are the usage instructions (which you can also get by typing “nef_to_RST -help” at the command line:

```
# nef_to_RST
convert NEF restraints to Amber format
input:
  -nef <filename>: NEF file
  -pdb <filename>: PDBFILE using AMBER nomenclature and numbering
  -map <filename>: MAP file (default:map.NEF-AMBER)
output:
  -rst <filename>: SANDER restraint format
  -rdc <filename>: SANDER DIP format

other options:
  -nocorr (do not correct upper bound for r*-6 averaging)
  -altdis (use alternative form for the distance restraints)
  -help (gives you this explanation, overrides other parameters)
  -report (gives you short runtime diagnostic output)
errors come to stderr.
```

30.7.6. fantasian

A program to evaluate magnetic anisotropy tensor parameters

Ivano Bertini
 Depart. of Chemistry, Univ. of Florence, Florence, Italy
 e-mail: bertini@risc1.lrm.fi.cnr.it

INPUT FILES:

Observed shifts file (pcshifts.in):

```
1st column --> residue number
2nd column --> residue name
3rd column --> proton name
4th column --> observed pseudocontact shift value
5th column --> multiplicity of the NMR signal (for example it is 3 for of a methyl group)
6th column --> relative tolerance
7th column --> relative weight
```

Amber pdb file (parm.pdb): coordinates file in PDB format. If you need to use a solution NMR family of structures you have to superimpose the structures before to use them.

OUTPUT FILES:

Observed out file (obs.out): This file is built and read by the program itself, it reports the data read from the input files.

output file (res.out): The main output file. In this file the result of the fitting is reported. Using fantasian it is possible to define an internal reference system to visualize the orientation of the tensor axes. Then in this file you can find PDB format lines (ATOM) which can be included in a PDB file to visualize the internal reference system and the tensor axes. In the main output file all the three equivalent permutations of the tensor parameters with respect to the reference system are reported. The summary of the minimum and maximum errors and that of squared errors are also reported.

Example files: in the directory example there are all the files necessary to run a fantasian calculation:

```

fantasian.com --> run file
pcshifts.in --> observed shifts file
parm.pdb --> coordinate file in PDB format
obs.out --> data read from input files
res.out --> main output file ~

```

30.8. Getting summaries of NMR violations

If you specify LISTOUT=POUT when running *sander*, the output file will contain a lot of detailed information about the remaining restraint violations at the end of the run. When running a family of structures, it can be useful to process these output files with *sviol*, which takes a list of *sander* output files on the command line, and sends a summary of energies and violations to STDOUT. If you have more than 20 or so structures to analyze, the output from *sviol* becomes unwieldy. In this case you may also wish to use *sviol2*, which prints out somewhat less detailed information, but which can be used on larger families of structures. The *senergy* script gives a more detailed view of force-field energies from a series of structures. (We thank the TSRI NMR community for helping to put these scripts together, and for providing many useful suggestions.)

30.9. Time-averaged restraints

The model of the previous sections involves the "single-average-structure" idea, and tries to fit all constraints to a single model, with minimal deviations. A generalization of this model treats distance constraints arising from NOE crosspeaks (for example) as being the average distance determined from a trajectory, rather than as the single distance derived from an average structure.

Time-averaged bonds and angles are calculated as

$$\bar{r} = (1/C) \left\{ \int_0^t e^{(t'-t)/\tau} r(t')^{-i} dt' \right\}^{-1/i} \quad (30.1)$$

where

- $\bar{r}$ = time-averaged value of the internal coordinate (distance or angle)
- t = the current time
- τ = the exponential decay constant
- $r(t')$ = the value of the internal coordinate at time t'
- i = average is over internals to the inverse of i . Usually $i = 3$ or 6 for NOE distances, and -1 (linear averaging) for angles and torsions.
- C = a normalization integral.

Time-averaged torsions are calculated as

$$\langle \phi \rangle = \tan^{-1} (\langle \sin(\phi) \rangle / \langle \cos(\phi) \rangle)$$

where ϕ is the torsion, and $\langle \sin(\phi) \rangle$ and $\langle \cos(\phi) \rangle$ are calculated using the equation above with $\sin(\phi(t'))$ or $\cos(\phi(t'))$ substituted for $r(t')$.

Forces for time-averaged restraints can be calculated either of two ways. This option is chosen with the DISAVI / ANGAVI / TORAVI commands. In the first (the default),

$$\partial E / \partial x = (\partial E / \partial \bar{r}) (\partial \bar{r} / \partial r(t)) (\partial r(t) / \partial x) \quad (30.2)$$

(and analogously for y and z). The forces then correspond to the standard flat-bottomed well functional form, with the instantaneous value of the internal replaced by the time-averaged value. For example, when $r_3 < \bar{r} < r_4$,

$$E = k_3(\bar{r} - r_3)^2$$

and similarly for other ranges of $\bar{r}$.

When the second option for calculating forces is chosen (IINC = 1 on a DISAVI, ANGAVI or TORAVI card), forces are calculated as

$$\partial E / \partial x = (\partial E / \partial \bar{r})(\partial r(t) / \partial x) \quad (30.3)$$

For example, when $r_3 < \bar{r} < r_4$,

$$\partial E / \partial x = 2k_3(\bar{r} - r_3)(\partial r(t) / \partial x)$$

Integration of this equation does not give Eq. 30.2, but rather a non-intuitive expression for the energy (although one that still forces the bond to the target range). The reason that it may sometimes be preferable to use this second option is that the term $\partial \bar{r} / \partial r(t)$, which occurs in the exact expression [Eq. 30.2], varies as $(\bar{r} / r(t))^{1+i}$. When $i=3$, this means the forces can be varying with the fourth power the distance, which can possibly lead to very large transient forces and instabilities in the molecular dynamics trajectory. [Note that this will not be the case when linear scaling is performed, i.e. when $i = -1$, as is generally the case for valence and torsion angles. Thus, for linear scaling, the default (exact) force calculation should be used].

It should be noted that forces calculated using Eq. 30.3 are not conservative forces, and would cause the system to gradually heat up, if no velocity rescaling were performed. The temperature coupling algorithm should act to maintain the average temperature near the target value. At any rate, this heating tendency should not be a problem in simulations, such as fitting NMR data, where MD is being used to sample conformational space rather than to extract thermodynamic data.

This section has described the methods of time-averaged restraints. For more discussion, the interested user is urged to consult studies where this method has been used.[710–714]

30.10. Multiple copies refinement using LES

NMR restraints can be made compatible with the multiple copies (LES) facility; see the following chapter for more information about LES. To use NMR constraints with LES, you need to do two things:

(1) Add a line like "file wnmr name=(lesnmr) wovr" to your input to *addles*. The filename (lesnmr in this example) may be whatever you wish. This will cause *addles* to output an additional file that is needed at the next step.

(2) Add "-les lesnmr" to the command line arguments to *makeDIST.RST*. This will read in the file created by *addles* containing information about the copies. All NMR restraints will then be interpreted as "ambiguous" restraints, so that if any of the copies satisfies the restraint, the penalty goes to zero.

Note that although this scheme has worked well on small peptide test cases, we have yet not used it extensively for larger problems. This should be treated as an experimental option, and users should use caution in applying or interpreting the results.

30.11. Some sample input files

The next few pages contain excerpts from some sample NMR refinement files used at TSRI. The first example just sets up a simple (but often effective) simulated annealing run. You may have to adjust the length, temperature maximum, etc. somewhat to fit your problem, but these values work well for many "ordinary" NMR problems.

30.11.1. 1. Simulated annealing NMR refinement

```
15ps simulated annealing protocol
&cntrl
  nstlim=15000, ntt=1, !(time limit, temp. control)
```

```

ntpr=500, pencut=0.1, !(control of printout)
ipnlty=1, nmropt=1, !(NMR penalty function options)
vlimit=10, !(prevent bad temp. jumps)
ntb=0, !(non-periodic simulation)
igb=8, !(generalize Born solvent model)
/
#
# Simple simulated annealing algorithm:
#
# from steps 0 to 1000: raise target temperature 10-1200K
# from steps 1000 to 3000: leave at 1200K
# from steps 3000 to 15000: re-cool to low temperatures
#
&wt type='TEMP0', istep1=0,istep2=1000,value1=10.,
value2=1200., /
&wt type='TEMP0', istep1=1001, istep2=3000, value1=1200.,
value2=1200.0, /
&wt type='TEMP0', istep1=3001, istep2=15000, value1=0.,
value2=0.0, /
#
# Strength of temperature coupling:
# steps 0 to 3000: tight coupling for heating and equilibration
# steps 3000 to 11000: slow cooling phase
# steps 11000 to 13000: somewhat faster cooling
# steps 13000 to 15000: fast cooling, like a minimization
#
&wt type='TAUTP', istep1=0,istep2=3000,value1=0.2,value2=0.2, /
&wt type='TAUTP', istep1=3001,istep2=11000,value1=4.0,value2=2.0, /
&wt type='TAUTP', istep1=11001,istep2=13000,value1=1.0,value2=1.0, /
&wt type='TAUTP', istep1=13001,istep2=14000,value1=0.5,value2=0.5, /
&wt type='TAUTP', istep1=14001,istep2=15000,value1=0.05,value2=0.05, /
#
# "Ramp up" the restraints over the first 3000 steps:
#
&wt type='REST', istep1=0,istep2=3000,value1=0.1,value2=1.0, /
&wt type='REST', istep1=3001,istep2=15000,value1=1.0,value2=1.0, /
&wt type='END' /
LISTOUT=POUT (get restraint violation list)
DISANG=RST.f (file containing NMR restraints)

```

The next example just shows some parts of the actual RST file that *sander* would read. This file would ordinarily *not* be made or edited by hand; rather, run the programs *makeDIST_RST*, *makeANG_RST* and *makeCHIR_RST*, combining the three outputs together to construct the RST file.

30.11.2. Part of the RST.f file referred to above

```

# first, some distance constraints prepared by makeDIST_RST:
# (comment line is input to makeRST, &rst namelist is output)
#
#( proton 1 proton 2 upper bound)
#-----
#
# 2 ILE HA 3 ALA HN 4.00
#
&rst iat= 23, 40, r3= 4.00, r4= 4.50,
r1 = 1.3, r2 = 1.8, rk2=0.0, rk3=32.0, ir6=1, /

```

```

#
# 3 ALA HA 4 GLU HN 4.00
#
&rst iat= 42, 50, r3= 4.00, r4= 4.50, /
#
# 3 ALA HN 3 ALA MB 5.50
#
&rst iat= 40, -1, r3= 6.22, r4= 6.72,
igr1= 0, 0, 0, 0, igr2= 44, 45, 46, 0, /
#
# .....etc.....
#
# next, some dihedral angle constraints, from makeANG_RST:
#
&rst iat= 213, 215, 217, 233, r1=-190.0,
r2=-160.0, r3= -80.0, r4= -50.0, /
&rst iat= 233, 235, 237, 249, r1=-190.0,
r2=-160.0, r3= -80.0, r4= -50.0, /
# .....etc.....
#
# next, chirality and omega constraints prepared by makeCHIR_RST:
#
#
# chirality for residue 1 atoms: CA CG HB2 HB3
&rst iat= 3 , 8 , 6 , 7 ,
r1=10., r2=60., r3=80., r4=130., rk2 = 10., rk3=10., /
#
# chirality for residue 1 atoms: CB SD HG2 HG3
&rst iat= 5 , 11 , 9 , 10 , /
#
# chirality for residue 1 atoms: N C HA CB
&rst iat= 1 , 18 , 4 , 5 , /
#
# chirality for residue 2 atoms: CA CG2 CG1 HB
&rst iat= 22 , 26 , 30 , 25 , /
#
# .....etc.....
# trans-omega constraint for residue 2
&rst iat= 22 , 20 , 18 , 3 ,
r1=155., r2=175., r3=185., r4=205., rk2 = 80., rk3=80., /
#
# trans-omega constraint for residue 3
&rst iat= 41 , 39 , 37 , 22 , /
#
# trans-omega constraint for residue 4
&rst iat= 51 , 49 , 47 , 41 , /
#
# .....etc.....
#
The next example is an input file for volume-based NOE refinement. As with the distan

```

30.11.3. 3. Sample NOESY intensity input file

```
# A part of a NOESY intensity file:
&noeexp
id2o=1, (exchangeable protons removed)
oscale=6.21e-4, (scale between exp. and calc. intensity units)
taumet=0.04, (correlation time for methyl rotation, in ns.)
taurot=4.2, (protein tumbling time, in ns.)
NPEAK = 13*3, (three peaks, each with 13 mixing times)
EMIX = 2.0E-02, 3.0E-02, 4.0E-02, 5.0E-02, 6.0E-02,
8.0E-02, 0.1, 0.126, 0.175, 0.2, 0.25, 0.3, 0.35,
(mixing times, in sec.)
IHP(1,1) = 13*423, IHP(1,2) = 13*1029, IHP(1,3) = 13*421,
(number of the first proton)
JHP(1,1) = 78*568, JHP(1,2) = 65*1057, JHP(1,3) = 13*421,
(number of the second proton)
AEXP(1,1) = 5.7244, 7.6276, 7.7677, 9.3519,
10.733, 15.348, 18.601,
21.314, 26.999, 30.579,
33.57, 37.23, 40.011,
(intensities for the first cross-peak)
AEXP(1,2) = 8.067, 11.095, 13.127, 18.316,
22.19, 26.514, 30.748,
39.438, 44.065, 47.336,
54.467, 56.06, 60.113,
AEXP(1,3) = 7.708, 13.019, 15.943, 19.374,
25.322, 28.118, 35.118,
40.581, 49.054, 53.083,
56.297, 59.326, 62.174,
/
SUBMOL1
RES 27 27 29 29 39 41 57 57 70 70 72 72 82 82 (residues in this submol)
END END
```

Next, we illustrate the form of the file that holds residual dipolar coupling restraints. Again, this would generally be created from a human-readable input using the program *makeDIP_RST*.

30.11.4. Residual dipolar restraints, prepared by *makeDIP_RST*:

```
&align
ndip=91, dcut=-1.0, gigj = 37*-3.1631, 54*7.8467,
s11=3.883, s22=53.922, s12=33.855, s13=-4.508, s23=-0.559,
id(1)=188, jd(1)=189, dobsu(1)= 6.24, dobsl(1)= 6.24,
id(2)=208, jd(2)=209, dobsu(2)= -10.39, dobsl(1)= -10.39,
id(3)=243, jd(3)=244, dobsu(3)= -8.12, dobsl(1)= -8.12,
....
id(91)=1393, jd(91)=1394, dobsu(91)= -19.64, dobsl(91) = -19.64,
/
```

Finally, we show how the detailed input to *sander* could be used to generate a more complicated restraint. Here is where the user would have to understand the details of the RST file, since there are no "canned" programs to create this sort of restraint. This illustrates, though, the potential power of the program.

30.11.5. A more complicated constraint

```

# 1) Define two centers of mass. COM1 is defined by
# {C1 in residue 2; C1 in residue 3; N2 in residue 4; C1 in residue 5}.
# COM2 is defined by {C4 in residue 1; O4 in residue 1; N* in residue 1}.
# (These definitions are effected by the igr1/igr2 and grnam1/grnam2
# variables; You can use up to 200 atoms to define a center-of-mass
# group)
#
# 2) Set up a distance restraint between COM1 and COM2 which goes from a
# target value of 5.0A to 2.5A, with a force constant of 1.0, over steps 1-5000.
#
# 3) Set up a distance restraint between COM1 and COM2 which remains fixed
# at the value of 2.5A as the force slowly constant decreases from
# 1.0 to 0.01 over steps 5001-10000.
#
# 4) Sets up no distance restraint past step 10000, so that free (unrestrained)
# dynamics takes place past this step.
#
&rst iat=-1,-1, nstep1=1,nstep2=5000,
    iresid=1,irstyp=0,ifvari=1,ninc=0,imult=0,ir6=0,ifntyp=0,
    r1=0.00000E+00,r2=5.0000,r3=5.0000, r4=99.000,rk2=1.0000,rk3=1.0000,
    r1a=0.00000E+00,r2a=2.5000,r3a=2.5000, r4a=99.000,rk2a=1.0000,rk3a=1.0000,
    igr1 = 2,3,4,5,0, grnam1(1)='C1',grnam1(2)='C1',grnam1(3)='N2',
    grnam1(4)='C1', igr2 = 1,1,1,0, grnam2(1)='C4',grnam2(2)='O4',grnam2(3)='N*',
/
&rst iat=-1,-1, nstep1=5001,nstep2=10000,
    iresid=1,irstyp=0,ifvari=1,ninc=0,imult=0,ir6=0,ifntyp=0,
    r1=0.00000E+00,r2=2.5000,r3=2.5000, r4=99.000,rk2=1.0000,rk3=1.0000,
    r1a=0.00000E+00,r2a=2.5000,r3a=2.5000, r4a=99.000,rk2a=1.0000,rk3a=0.0100,
    igr1 = 2,3,4,5,0, grnam1(1)='C1',grnam1(2)='C1',grnam1(3)='N2',
    grnam1(4)='C1', igr2 = 1,1,1,0, grnam2(1)='C4',grnam2(2)='O4',grnam2(3)='N*',
/

```


31. Xray and cryoEM refinement

31.1. EMAP restraints for rigid and flexible fitting into EM maps

EMAP restrained simulation[517, 715] was developed to incorporate electron microscopy (EM) image information into macromolecular structure determination. Different from NMR and X-ray data, EM images have low resolutions (5~50Å). However, EM images of large molecular assemblies up to millions of atoms and in various biologically relevant environments are available. These low resolution images provide precious structural information that can help to determine structures of many molecular assemblies and machineries[715–725].

With EMAP restraints, Sander and PMEMD can be used to perform both rigid[715] and flexible[517] fitting of molecules into experimental maps of complexes to obtain both complex structures and conformations agreeing with experimental maps. In addition to experimental map information, homologous structural information can be used by EMAP to perform targeted conformational search (TCS) to induce simulation systems to form structures of interest.

If the restraint map or structure is very different from the starting conformation, SGLD is recommended to induce large conformational change by setting *isgld*=1. This is often used to simulate conformational transition between different states. See the Sampling and free energy search section 25.1 for details on running SGLD.

If domain motion is desired while domain structures need to be maintained, one can use an EMAP restraint generated from the initial coordinates for each domain and set *move*=1 to allow the restraint map to move with the domain, so that domains can search the conformational space without unfolding or changing shape.

Each EMAP restraint is defined by a map file and a selection of atoms, as well as related parameters. Multiple EMAP restraints can be defined. The map can be either input from an image file, or generated from a pdb structure or derived from the starting coordinates.

One application of EMAP restraint is using a map as a boundary for finite systems. A boundary map can be created around the simulation system or read in from a map file. A negative *resolution* must be set to define a boundary map. To create a boundary map, one can leave the *mapfile* empty and set a *resolution*<0. The boundary map will be created with a grid size of $|resolution|$ and numbers of grid points defined by *grids*(1:3) and the shape of the boundary defined by *grids*(4:6). The shape of the boundary can be rectangular (Figure 30.1(a)), elliptical (Figure 30.1(b)), or cylindrical (Figure 30.1(c)). The created boundary map can be outputted to an image file and reused in other simulations. A negative restraint constant must be used to keep the system within the boundary.

The definition of EMAP restraints are read in from the input file as “&emap” namelists. The following are variables in each &emap namelist.

mapfile The filename of a restraint map or structure. The restraint maps must be in “map”, “ccp4”, or “mrc” format. The structure must be in pdb format. The structure need not be the same as the simulation system. A resolution can be specified for the conversion to a density map. When a

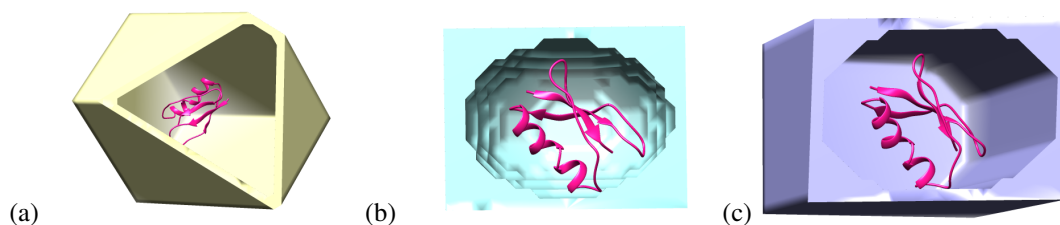


Figure 31.1.: Boundary maps for simulation of finite systems. (a) cubic boundary; (b) ellipse boundary; (c) cylinder boundary.

31. Xray and cryoEM refinement

	blank filename is specified, <i>mapfile</i> ="", the input coordinates of the masked atoms will be used to generate a restraint map (default="").
atmask	The atom mask for selecting atoms to be restrained (default=':*').
fcons	The restraining constant (default=0.05 kcal/g).
move	Allow the restraint map to move when <i>move</i> >0 (default=0).
resolution	The resolution used to convert an atomic structure to a map (default=2 Å). A negative resolution will create a boundary map with <i> resolution </i> defines the grid size and <i>grids</i> define the number of grid points in x, y, z directions.
ifit	Perform rigid fitting before simulation when <i>ifit</i> >0. One would do this when the initial coordinates don't match those of the map (default=0). When <i>ifit</i> =1, the map is transformed (by translation and rotation) to match the coordinates; the coordinates are not altered. EMAP allows output of the re-oriented map (<i>mapfit</i> =...) that matches the (final) simulation coordinates, and/or output of the coordinates (<i>molfit</i> =...) that would match the orientation of the original map. When <i>ifit</i> =2, the masked atoms will be transformed to fit the map and the transformed coordinates will be used for the following simulation. For periodic systems, <i>ifit</i> =2 may cause atoms to clash with periodic image atoms.
grids	Grid numbers in x,y,z,phi,psi,theta dimensions for grid-threading rigid fitting[715]. For example, <i>grids</i> =2,2,2,3,3,3 defines 2 grid points in each of x,y,z directions between the minimum and maximum coordinates, and 3 grid points in each of phi (0-360), psi(0-360), theta(0-180) angles. A search for local minimums starts from every grid point and the global minimum is identified from all the local minimums (default=1,1,1,1,1,1). When <i>resolution</i> <0 and <i>mapfile</i> ="", a boundary map will be created according to <i>grids</i> : <i>grids</i> (1:3) defines the grid numbers in x, y, z directions, <i>grids</i> (4:6) defines the boundary shape in x, y, z directions with 0 indicating an elliptical shape and a positive integer indicating a fractional shape. (20,30,25,1,1,1) would create a rectangular boundary enclosing 20, 30, and 25 grid points in x, y, z directions; (20,30,25,0,0,0) would create an elliptical boundary; and (20,30,25,0,1,0) would create an ellipse-cylindrical boundary with the cylinder along the y-direction.
mapfit	The filename for the final constraint map after rigid fitting and/or moving. The filename must have an extension of .map, .ccp4, or .mrc (default='', for no map output).
molfit	The filename for the final restrained atom coordinates after rigid fitting and/or simulation. The filename must have an extension of .pdb (default='', for no structure output).

Here is an example input file for an EMAP constrained SGLD simulation:

```
Map Constraint Self-Guided Langevin dynamics
&cntrl ntx=1, ntb=0, nstlim=100000, imin=0, maxcyc=1, ntc=2, ntf=2, cut=9.0,
ntpr=1000, ntwr=100000, ntwx=10000, ntt=3, gamma_ln=10.0, nscm=100, dt=0.001,
ntb=0, igb=0, ips=1, isgld=1, tsavg=1.0, sgft=1.0, (SGLD)
iemap=1, (turn on EMAP)
/
&emap (EMAP restraint 1)
mapfile='data/lgb1.ccp4', (map is input from a map file)
atmask=':1-20', (residues 1-20 are restrained)
fcons=0.1,
move=1, (restraint map can move)
ifit=1, (perform rigid fitting first)
mapfit='scratch/gb1n_1.ccp4', (final map)
molfit='scratch/gb1n_1.pdb', / (final restrained atoms related to initial map)
&emap (EMAP restraint 2)
mapfile='data/lgb1.pdb', (map is generated from a pdb file)
```

```

atmask=':22-37',          (residues 22-37 are restrained)
fcons=0.1,mov=0,          (restraint map is fixed)
ifit=1,                   (perform rigid fitting first)
mapfit='scratch/gblh_1.ccp4', (final map, same as initial)
molfit='scratch/gblh_1.pdb', / (final restrained atoms related to initial map)
&emap                     (EMAP restraint 3)
mapfile='',               (map is generated from initial coordinates)
atmask=':41-56',          (residues 41-56 are restrained)
fcons=0.1,mov=1,          (restraint map can move)
ifit=1,                   (perform rigid fitting first)
mapfit='scratch/gblc_1.ccp4', (final map)
molfit='scratch/gblc_1.pdb', / (final restrained atoms related to initial map)
&emap                     (create a boundary map for finite systems)
mapfile='',               (map is generated from initial coordinates)
atmask=':*', fcons=-1.0, (all atoms are restrained within the boundary)
resolution=-4, (create a boundary map with grid size of 4 )
grids=20,20,20,1,1,1, (number of grid points in x, y,z direction,rectangular shape)
mapfit='scratch/gbl_boundary.ccp4', (output boundary map for reused)
molfit='scratch/gbl_final.pdb', / (final conformation)

```

31.2. FRETrest: Förster Resonance Energy Transfer restraints

FRETrest is a set of helper scripts for generating FRET-restraints for Molecular Dynamics (MD) simulations performed with the AMBER Software Suite. FRETrest saves restraints in DISANG format, which is also used for NMR-based restraints (30.1). FRETrest implements FRET restraints for implicit dyes represented by pseudo atoms. Each pseudo atom represents the statistical mean position of the fluorescent dye as determined by Accessible Volume simulations [726]. These pseudo atoms are restrained with respect to the backbone atoms of the labeled residue and, optionally, adjacent residues. FRETrest can be used to incorporate experimental data obtained from different types of FRET experiments [726, 727] with various organic dyes and linkers. For MD simulations with explicit (as opposed to implicit) dyes see section 3.10.

Adding FRET restraints to an MD simulation takes two steps.

1) Add pseudo atoms to the topology file with placeAV.py:

```

placeAV.py [-p <parmtop>] [-o <output>]
           [-j <config>] [--chi2 <Xevaluator>]

-p <parmtop> Load <parmtop> as a topology file.
-o <output> Write output to file <output>.
-j <config> FRET configuration file.
--chi2 <Xevaluator> Set the  $\chi^2$ evaluator.

```

2) Tether pseudo atoms to the corresponding backbone residues and apply FRET restraints between pseudo atoms with FRETrest.py:

```

FRETrest.py [-t <parmtop>] [-r <restart>]
            [-j <config>] [--chi2 <Xevaluator>]
            [--fout <outfile>] [--restout <restfile>]
            [--force <mforce>] [--resoffset <resoff>]

-t <parmtop> Load <parmtop> as a topology file.
-r <restart> Read restart file from <restart>.
-j <config> FRET configuration file.
--chi2 <Xevaluator> Set the  $\chi^2$ evaluator.

```

-fout <outfile> Resulting restraints output file.
-restout <restfile> Resulting adjusted restart file.
-force <mforce> Maximum inter-dummy force in piconewton.
-resoffset <resoff> Integer number of shifted residues between t4l.fps.json and AMBER topology file. Should be provided if residue numbering starts from '1' in the t4l.fps.json, but in the .prmtop it starts from '0', as is usual. In general, the offset can be any integer number, positive or negativ.

First, this command will add dummy atoms to a PDB file of T4 lysozyme protein (PDB ID 148L) and save it to 148L.PA.pdb:

```
python3 placeAVmp.py -p 148l_noH.pdb -o 148l.PA.pdb -j t4l.fps.json --chi2 'C3  $\chi^2$ '
```

For the script to work, an input PDB file (148l_noH.pdb) and a FRET configuration file (t4l.fps.json) must be provided. The FRET configuration file can be generated Olga software [728]. C3 χ^2 is the name of the relevant χ^2 evaluator from t4l.fps.json. See the documentation of Olga for more details. Labeling positions that are present in t4l.fps.json, but are not relevant to the specified χ^2 will be omitted from the resulting PDB.

Second, the command will generate the AMBER restraint (DISANG) file:

```
python3 FRETrest.py -t 148l_watio_hmr.prmtop -r Equil/26_md.nvt_red.pme.11.restrt \
-j t4l.fps.json --chi2 'C3  $\chi^2$ ' --fout prod.0001.f --restout prod.0000.restrt \
--force 50 --resoffset -1
```

It will also generate an updated restart file, so that if there are any inconsistencies between the conformation of the macromolecule and dummy atom positions, positions of the dummy atoms are adjusted accordingly.

Comprehensive step by step usage examples are available at examples/T4L/t4l.sh and examples/hGBP1/hGBP1.sh. To use the scripts you would need a working installation of AmberTools, and python libraries LabelLib [726, 727] and mdtraj [729].

FRETrest was introduced here[728].

31.3. X-ray functionality and diffraction-based restraints for pmemd

New to Amber 20 and updated in Amber 22, the *pmemd* and *pmemd.cuda* programs include an experimental module dedicated to biomolecular crystallography. It is envisioned that in future Amber can be used as a platform to address various crystallography-related problems, e.g. to refine crystallographic structures of proteins and nucleic acids (similar to the existing capability in the area of biomolecular NMR). This module is intended for use with an MD simulation of the crystal unit cell or a “supercell”[730]. For information on how to set up a crystal simulation, including periodic boundary conditions to emulate crystalline lattice, see Chapter 21. It is expected that the crystal is solvated using an explicit (or implicit) solvent. While a number of crystallographic concepts are implemented in the new Amber module, many others have not yet been implemented. For example, an MD model of a crystal unit cell can naturally accommodate different side-chain conformations; however, the concept of alternate side-chain conformations, as employed in protein crystallography, is currently unavailable in Amber.

Although it is not a part of Amber, it is worth noting that the *phenix* crystallographic package now allows for X-ray refinement using Amber (or other) force fields[731]. This was accomplished by using the python API to *sander*, discussed in Section 22.14, and uses locally-enhanced sampling (see Chap. 32) to handle alternate conformations. It supports all of the X-ray related options in *phenix.refine*, but has limited options for molecular dynamics, and no GPU acceleration.

31.3.1. Structure factor calculations

For the crystal simulation, the program can calculate crystallographic structure factors (SFs) for individual MD frames. The calculations are conducted using direct summation formula[732]; a mask is available to define the subset of atoms included in these calculations (*atom_selection_mask*). For example, this mask could select the macromolecules, but not the solvent or the neutralizing ions present in the simulation. The B-factors used in the direct summation formula are supplied through a designated PDB file. The set of Miller indices for SF calculations is supplied as a part of the *reflection_infile*.

In principle, explicit solvent and ions can also be accounted for via the direct summation formula. However, an individual frame does not offer an adequate statistical sampling with regard to the positioning of water molecules (if desired, such statistical sampling can be obtained by modeling a very large supercell or otherwise by means of time averaging). As a commonly accepted alternative, Amber 22 offers two mask-based models of bulk solvent. The first one (*bulk_solvent_model* = ‘simple’) is a simple variant of flat mask bulk solvent model[733], which calculates the contribution from interstitial solvent into SFs using two generic parameters (*k_sol* and *b_sol*), as reported by Fokine and Urzhumtsev[734]. The implementation, including the scheme to build a solvent mask, is analogous to the one in the cctbx library[735]. The more advanced version (*bulk_solvent_model* = ‘afonine-2013’) employs the variable k_{mask} , which replaces *k_sol* and *b_sol*; this variable is automatically optimized over the individual resolution bins[736]. The iterative optimization procedure to determine k_{mask} also adjusts the overall scaling coefficient that is applied to calculated SFs. The implementation follows the one in the cctbx library with several minor modifications.

It is worth noting that crystal MD simulations in Amber (as described in Chap. 21) do not maintain a perfect space group symmetry. Therefore, strictly speaking, the calculated SFs correspond to P(1) space group with the unit cell that is identical to the simulation box. During the course of the simulation, the calculated SFs can be collected frame-by-frame at a specified interval (*ntwsf*) and stored in a form of special trajectory file (*sf_outfile*).

31.3.2. Structure-factor-based potential

The X-ray energy term E_{xray} is added to the total potential energy with the user-specified weight w_{xray} (controlled by *xray_weight_initial* / *xray_weight_final* variables):

$$E_{total} = E_{force-field} + w_{xray}E_{xray} \quad (31.1)$$

E_{xray} uses the set of target (i.e. experimentally observed) SFs, which are supplied via the input file *reflection_infile*. This file must also contain flags to divide all reflections into a “working” set and a “free” (test) set.

Currently *pmemd* offers two variants of the E_{xray} term. The first one is a very simple least squares objective function (*target* = ‘ls’) involving the amplitudes of calculated and observed structure factors:

$$E_{xray} = \frac{\sum_{h,k,l} (F_{calc}(h,k,l) - F_{obs}(h,k,l))^2}{\sum_{h,k,l} F_{obs}^2(h,k,l)} \quad (31.2)$$

The sum in this expression is over the working set of reflections.

The second option for E_{xray} is the Maximum Likelihood target function (*target* = ‘ml’)[737, 738]:

$$E_{xray} = \sum_{h,k,l} \left(-\ln \left(\frac{2F_{obs}(h,k,l)}{\epsilon\beta} \right) + \frac{F_{obs}^2(h,k,l)}{\epsilon\beta} + \frac{\alpha^2 F_{calc}^2(h,k,l)}{\epsilon\beta} - \ln I_0 \left(\frac{2\alpha F_{obs}(h,k,l)F_{calc}(h,k,l)}{\epsilon\beta} \right) \right) \quad (31.3)$$

where α and β are (resolution-shell-dependent) ML likelihood distribution parameters, ϵ is the symmetry coefficient ($\epsilon = 1$ for the space group P(1) at hand), $I_0(x)$ is the zeroth-order modified Bessel function of the first kind, and the summation is over the working set of reflections. For practical applications, we recommend using *target* = ‘ml’ along with the more advanced version of solvent, *bulk_solvent_model* = ‘afonine-2013’.

The expression for E_{xray} , along with the direct-summation formula for F_{calc} , provides a basis to evaluate forces. These “restraint” forces act like those used in NMR refinement, discussed in Chap.30, and are generally used to drive minimization or MD simulations that minimize E_{total} . The value of E_{xray} is reported in the mdout file, together with R_{work} and R_{free} .

We envisage that SF-based restraints can be used for a number of purposes. For example, they can be viewed as an empirical addition to the force fields, which can potentially remedy certain existing biases[739, 740]. Another promising application is refinement of crystallographic structures. Such an Amber-based protocol has been reported by Mikhailovskii et al.[741] (the web interface is available at <https://arx.bio-nmr.spbu.ru>). Ultimately, the entire process of crystallographic structure determination can be incorporated into Amber. This approach may be particularly valuable for lower-quality diffraction data sets and incomplete structural models (e.g. in the case of weak or missing electron density for mobile side chains, loops or terminal regions in protein molecules). In this situation, the state-of-the-art force field provides a natural solution to model the poorly resolved or unresolved elements of the structure. This is accomplished in a highly realistic manner, by using the explicit representation of the crystal unit cell (supercell), taking into consideration the effect of solvent, crystal contacts, etc.

31.3.3. Inputs and file formats

System setup follows the general procedures outlined in Chap. 21. For users with access to the *phenix* package of crystallographic analysis tools, the XrayPrep tool can prepare the system: inputs are simply a PDB file (*xxxx.pdb*, where *xxxx* is a PDB id) and the corresponding structure factor file (*xxxx-sf.cif*).

For those who will prepare their own inputs, one needs a PDB file, expanded to the unit cell (see Chap. 21) that contains the B-factors. The structure factors have to be listed in the *reflection_infile*, which is a human-readable ascii file containing the same information that can be normally found in .mtz files. The first line contains a total number of reflections followed by a zero. Subsequent lines list Miller indices *h*, *k* and *l*, followed by the respective SF values and their standard deviations, followed by an R-free flag (we adopt the convention that “1” indicates a member of the working set, and “0” a member of the test set). An example file is given below. Note that column spacing or number formatting is not critical, but each entry should be separated by at least one space.

41243	0				
-19	-6	1	13.86329	9.685285	0
-19	-6	2	46.38137	3.528763	1
-19	-5	1	9.675193	21.28529	1
...					
19	6	1	13.86329	9.685285	0
19	6	2	46.38137	3.528763	1

Input variables in the &xray namelist The X-ray functionalities are activated by adding the *&xray* namelist to the mdin file. User-assigned parameters that are not used by the algorithm are silently ignored (e.g. *k_sol* and *b_sol* in 'afonine-2013' bulk solvent model). The keywords in *&xray* namelist include the following:

File handling:

<i>pdb_infile</i>	name of the PDB input file containing B-factors
<i>pdb_read_coordinates</i>	if true, use coordinates from the PDB file, not <i>inpcrd</i> , as starting coordinates
<i>pdb_outfile</i>	name of PDB file to write the final atomic coordinates from the simulation. Currently writes back the input B-factors and occupancies as read from <i>pdb_infile</i>
<i>reflection_infile</i>	name of the input file containing experimental SFs and R-flags
<i>sf_outfile</i>	name of the trajectory file with calculated SFs
<i>ntwsf</i>	time interval to write calculated SFs to <i>sf_outfile</i>

Bulk solvent parameters:

<i>bulk_solvent_model</i>	the type of bulk solvent to use. Possible values: 'none' for disabled bulk solvent contribution, 'simple' or 'afonine-2013' (default)
<i>k_sol</i>	solvent electron density (default $0.35 \text{ e \AA}^{-3}$)
<i>b_sol</i>	determines the blurring of the boundary between the solvent region and the macromolecule (default $46 \text{ \AA}^2$)
<i>solvent_mask_adjustment</i>	increment to be added to atomic radii of the atoms selected by <i>atom_selection_mask</i> as a part of the algorithm to build bulk solvent mask (default $1.11 \text{ \AA}$)
<i>solvent_mask_probe_radius</i>	the radius of solvent probe to apply as a part of the algorithm to build bulk solvent mask (default $0.9 \text{ \AA}$)
<i>mask_update_period</i>	time interval for bulk solvent grid update (expressed as a multiple of integration step, default 100 steps)

Structure-factor-based potential:

<i>atom_selection_mask</i>	ambmask-format mask to specify the atoms that contribute to F_{calc} via direct summation formula (default '!@H=')
<i>scale_update_period</i>	time interval to re-scale F_{calc} to F_{obs} (expressed as a multiple of integration step, default 100 steps)
<i>target</i>	the type of crystallographic target function. Possible values: 'ls' for Least Squares, 'ml' for Maximum Likelihood (default)
<i>ml_update_period</i>	time interval to update ML parameters α and β (expressed as a multiple of integration step, default 100 steps)
<i>xray_weight_initial</i>	initial value that defines linear scaling of w_{xray} weight along the trajectory (default 1.0)
<i>xray_weight_final</i>	final value that defines linear scaling of w_{xray} weight along the trajectory (if unassigned, assumed to be equal to <i>xray_weight_initial</i>)

32. Locally-enhanced sampling

Locally-enhanced sampling (LES) is a method to allow for multiple local copies of regions within a larger biomolecule. An example would be to allow sidechains in a protein to be “disordered” (that is, to be described as a superposition of several configurations), while the backbone is represented as a single configuration. This is similar to the “alternate conformer” model often used by crystallographers to describe local disorder in proteins. As the method name implies, this method can achieve enhanced sampling compared to conventional MD. Explanations of the approach, along with key examples, can be found in early, seminal papers.[\[742–745\]](#)

The LES functionality for *sander* was written by Carlos Simmerling. It basically functions by modifying the *prmtop* file using the program *addles*. The modified *prmtop* file is then used with a slightly modified version of *sander* called *sander.LES*.

32.1. Preparing to use LES with Amber

The first decision that must be made is whether LES is an appropriate technique for the system that you are studying. For further guidance, you may wish to consult published articles to see where LES has proven useful in the past. Several examples will also be given at the end of this section in order to provide models that you may wish to follow.

There are three main issues to consider before running the ADDLES module of Amber.

1. What should be copied?
2. How many copies should be used?
3. How many regions should be defined?

A brief summary of my experience with LES follows.

1. You should make copies of flexible regions of interest. This sounds obvious, and in some cases it is. If you are interested in determining the conformation of a protein loop, copy the loop region. If you need to determine the position of a side chain in a protein after a single point mutation, copy that side chain. If the entire biomolecule needs refinement, then copy the entire molecule. Some other cases may not be obvious—you may need to decide how far away from a particular site structural changes may propagate, and how far to extend the LES region.
2. You should use as few copies as are necessary. While this doesn’t sound useful, it illustrates the general point—too few copies and you won’t get the full advantages of LES, and too many will not only increase your system size unnecessarily but will also flatten the energy surface to the point where minima are no longer well defined and a wide variety of structures become populated. In addition, remember that LES is an approximation, and more copies make it more approximate. Luckily, published articles that explore the sensitivity of the results to the number of copies show that 3-10 copies are usually reasonable and provide similar results, with 5 copies being a good place to start.
3. Placing the divisions between regions can be the most difficult choice when using LES. This is essentially a compromise between surface smoothing and copy independence. The most effective surface-smoothing in LES takes places between LES regions. This is because N_a copies in region A interact with all N_b copies in region B, resulting in $N_a \cdot N_b$ interactions, with each scaled by $1/(N_a \cdot N_b)$ compared to the original interaction. This is better both from the statistics of how many different versions of this interaction contribute to the LES average, and how much the barriers are reduced. Remember that since the copies of a given region do not interact with different copies of that same region, interactions inside a region are only scaled by $1/N$.

The other thing to consider is whether these enhanced statistics are actually helpful. For example, if the copies cannot move apart, you will obtain many copies of the same conformation—obviously not very helpful. This will also result in less effective reduction in barriers, since the average energy barriers will be very similar to the non-average barrier. The independence of the copies is also related to how the copies are attached. For example, different copies of an amino acid side chain are free to rotate independently (at least within restrictions imposed by the surroundings and intrinsic potential) and therefore each side chain in the sequence could be placed into a separate LES region. If you are interested in backbone motion, however, placing each amino acid into a separate region is not the best choice. Each copy of a given amino acid will be bonded to the neighbor residues on each side. This restriction means that the copies are not very independent, since the endpoints for each copy need to be in nearly the same places. A better choice is to use regions of 2-4 amino acids. As the regions get larger, each copy can start to have more variety in conformation- for example, one segment may have some copies in a helical conformation while others are more strand-like or turn-like. The general rule is that larger regions are more independent, though you need to consider what types of motions you expect to see.

The best way to approach the division of the atoms that you wish to copy into regions is to make sure that you have several LES regions (unless you are copying a very small region such as a short loop or a small ligand). This will ensure plenty of inter-copy averaging. Larger regions permit wider variations in structure, but result in less surface smoothing. A subtle point should be addressed here- the statistical improvement available with LES is not a benefit in all cases and care must be taken in the choice of regions. For example, consider a ligand exiting a protein cavity in which a side chain acts as a *gate* and needs to move before the ligand can escape. If we make multiple copies of the gate, and do not copy the ligand, the ligand will interact in an average way with the *gates*. If the gate was so large that even the softer copies can block the exit, then the ligand would have to wait until ALL of the gate copies opened in order to exit. This may be more statistically difficult than waiting for the original, single gate to open despite the reduced barriers. Another way to envision this is to consider the ligand trying to escape against a true probability distribution of the gate- if it was open 50% of the time and closed 50%, then the exit may still be completely blocked. Continuum representations are therefore not always the best choice.

Specific examples will be given later to illustrate how these decisions can be made for a particular system.

32.2. Using the ADDLES program

The ADDLES module of Amber is used to prepare input for simulations using LES. A non-LES prmtop and pmrcrd file are generated using a program such as LEaP. This prmtop file is then given to ADDLES and replaced by a new prmtop file corresponding to the LES system. All residues are left intact- copies of atoms are placed in the same residue as the original atom, so that analysis based on sequence is preserved. Atom numbering is changed, but atom names are unchanged, meaning that a given residue may have several atoms with the same name. A different program is available for taking this new topology file and splitting the copies apart into separate residues, if desired. All copies are given the same coordinates as in the input coordinate file for the non-LES system.

Using addles:

```
addles < inputfile > outputfile
```

SAMPLE INPUT FILE:

```
~ a line beginning with ~ is a comment line.
~ all commands are 4 letters.
~ the maximum line length is 80 characters;
~ a trailing hyphen, "-", is the line continuation token.
~ use 'file' to specify an input/output file, then the type of file
'rpm' means this is the file to read the prmtop
~ the 'read' means it is an input file
~
file rpm name=(solv200.topo) read
~
~ 'rcrd' reads the original coordinates- optional, only if you want
```

32. Locally-enhanced sampling

```
~ a set of coords for the new topology
~ you can also use 'rcvd' for coords+velocities, 'rcvb' for coords,
~ velos and box dimensions, 'rcbd' for coords and box dimensions.
~ use "pack=n" option to read in multiple sets of coordinates and
~ assign different coordinates to different copies.
file rcrd name=(501v200.coords) read
~ 'wprm' is the new topology file to be written. the 'wovr' means to
~ write over the file if it exists, 'writ' means don't write over.
file wprm name=(lesprm) wovr
~ 'wcrd' is for writing coords, it will automatically write velo and box
~ if they were read in by 'rcvd' or 'rcvb'
file wcrd name=(lescrd) wovr
~ now put 'action' before creating the subspaces
action
~ the default behavior is to scale masses by 1/N.
~ omas leaves all masses at the original values
omas
~ now we specify LES subspaces using the 'spac' keyword, followed
~ by the number of copies to make and then a pick command to tell which
~ atom to copy for this subspace
~ 3 copies of the fragment consisting of monomers (=residues) 1 and 2
spac numc=3 pick #mon 1 2 done
~ 3 copies of the fragment consisting of monomers 3 and 4
spac numc=3 pick #mon 3 4 done
~ 3 copies of the fragment consisting of residues 5 and 6
spac numc=3 pick #mon 5 6 done
~ 2 copies of the side chain on residue 1
~ note that this replaces each of the side chains ON EACH OF THE 3
~ COPIES MADE ABOVE with 2 copies - net 6 copies
~ each of the 3 copies of residue 1-2 has 2 side chain copies.
~ the '#sid' command picks all atoms in the residue except
~ C,O,CA,HA,N,H and HN.
spac numc=2 pick #sid 1 1 done
spac numc=2 pick #sid 2 2 done
spac numc=2 pick #sid 3 3 done
spac numc=2 pick #sid 4 4 done
spac numc=2 pick #sid 5 5 done
~ use the *EOD to end the input
*EOD
```

What this does: all of the force constants are scaled in the new prmtop file by $1/N$ for N copies, so that this scaling does not need to be done for each pair during the nonbond calculation. Charges and VDW epsilon values are also scaled. New bond, angle, torsion and atom types are created. Any of the original types that were not used are discarded. Since each LES copy should not interact with other copies of the SAME subspace, the other copies are placed in the exclusion list. If you define very large LES regions, the exclusion list will get large and you may have trouble with the fixed length for this entry in the prmtop file- currently 8 digits.

The coordinates are simply copied - that means that all of the LES copies initially occupy the same positions in space. In this setup, the potential energy should be identical to the original system- this is a good test to make sure everything is functioning properly. Do a single energy evaluation of the LES system and the original system, using the copied coordinate file. All terms should be nearly identical (to within machine precision and roundoff). With PME on non- neutral systems, all charges are slightly modified to neutralize the system. For LES, there are a different number of atoms than in the original system, and therefore this charge modification to each atom will differ from the non-LES system and electrostatic energies will not match perfectly.

IMPORTANT: After creating the LES system, the copies will all feel the same forces, and since the coordinates are identical, they will move together unless the initial velocities are different. If you are initializing velocities using `INIT=3` and `TEMPI>0`, this is not a problem. In order to circumvent this problem, addles slightly (and randomly) modifies the copy velocities if they were read from the coordinate input file. If the keyword "nomodv" is specified, the program will leave all of the velocities in the same values as the original file. If you do not read velocities, make sure to assign an initial nonzero temperature to the system. You should think about this and change the behavior to suit your needs. In addition, the program scales the velocities by $\sqrt{N}$ for N copies to maintain the correct thermal energy (mv^2), but only when the masses are scaled (not using `omas` option). Again, this requires some thought and you may want different behavior. Regardless of what options are used for the velocities, further equilibration should be carried out. These options are simple attempts to keep the system close to the original state.[746]

Sometimes it is critical that different copies can have different initial coordinates (NEB for example), this is why the option "pack" is added to command `rcrd(rcvd,rcvb,rcbd)`. To use this option, user need first concatenate different coordinates into a single file, and use "pack=n" to indicate how many sets of coordinates there are in the file, like the following example:

```
file rcrd name=(input.inpcrd) pack=4 read
```

Then addles will assign coordinates averagely. For example, if 4 sets coordinates exists in input file, and 20 copies are generated, then copy 1-5 will have coordinate set 1, copy 6-10 will have coordinates set 2, and so on. Note this option can't work with multiple copy regions now.

It is important to understand that each subsequent pick command acts on the ORIGINAL particle numbers. Making a copy of a given atom number also makes copies of all copies of that atom that were already created. This was the simplest way to be able to have a hierarchical LES setup, but you can't make extra copies of part of one of the copies already made. I'm not sure why you would want to, or if it is even correct to do so, but you should be warned. Copies can be anything -spanning residues, copies of fragments already copied, non-contiguous fragments, etc. Pay attention to the order in which you make the copies, and look carefully at the output to make sure you get what you had in mind. Addles will provide a list at the end of all atoms, the original parent atom, and how many copies were made.

There are array size limits in the file `SIZE.h`, I apologize in advance for the poor documentation on these. Mail carlos.simmerling@stonybrook.edu if you have any questions or problems.

32.3. More information on the ADDLES commands and options

<code>file:</code>	open a file, also use one of
<code>rcrd:</code>	read coords from this file
<code>rcvd:</code>	read coords + velo from file
<code>rcvb:</code>	read coords, velo and box from file
<code>wcrd:</code>	write coords (and more if rcvd, rcvb) to file
<code>wprm:</code>	write new topology file
<code>action:</code>	start run, all of the following options must come AFTER action
<code>nomodv:</code>	do NOT slightly randomize the velocities of the copies
<code>spac:</code>	add a new subspace definition, using a pick command (see below); follow with " <code>numc=#pickcmd</code> ", where # is the number of copies to make and <code>pickcmd</code> is a pick command that selects the group of atoms to copy.
<code>omas:</code>	leave all masses at original values (otherwise scale $1/N$)

32. Locally-enhanced sampling

`pimd:` write an `prmtop` file for PIMD simulation, which contains a much smaller non-bond exclusion list, atoms from other copy will not be included in this non-bond exclusion list.

Syntax for 'pick' commands

Currently, the syntax for picking atoms is somewhat limited. Simple Boolean logic is followed, but operations are carried out in order and parentheses are not allowed.

`#prt A B` picks the atom range from A to B by atom number

`#mon A B` picks the residue range from A to B by residue number

`#cca A B` picks the residue range from A to B by residue number, but dividing the residue between CA and C; the CO for A is included, and the CO for monomer B is not. See Simmerling and Elber, 1994 for an example of where this can be useful.

`chem prt c A` picks all atoms named A, case sensitive

`chem mono A` picks all residues named A, case sensitive

Completion wildcards are acceptable for names: `H*` picks H, HA, etc. Note that `H*2` will select all atoms starting with H and ignore the 2.

Boolean logic:

`|` or atoms in either group are selected

`&` and atoms must be in both groups to be selected

`!=` `not A != B` will pick all atoms in A that are NOT in B

The user should carefully check the output file to ensure that the proper atoms were selected.

Examples:

<i>pick command</i>	<i>atoms selected</i>
<code>pick #mon 4 19 done</code>	all atoms in residues 4 through 19
<code>pick #mon 1 50 & chem mono GLY done</code>	only GLY in residues 1 to 50
<code>pick chem mono LYS chem mono GLU done</code>	any GLU or LYS residue
<code>pick #mon 1 5 != #prt 1 3 done</code>	residues 1 to 5 but not atoms 1 to 3

so, a full command to add a new subspace (LES region) with 4 copies of atoms 15 to 35 is:

```
spac numc=4 pick #prt 15 35 done
```

32.4. Using the new topology/coordinate files with SANDER

These topology files are ready to use in Sander with one exception: all of the FF parameters have been scaled by $1/N$ for N copies. This is done to provide the energy of the new system as an average of the energies of the individual copies (note that it is an average energy or force, not the energy or force from an average copy coordinate). However, one additional correction is required for interactions between pairs of atoms in the same LES region. Sander will make these corrections for you, and this information is just to explain what is being done. For example, consider a system where you make 2 copies of a sidechain in a protein. Each charge is scaled by $1/2$. For these atoms interacting with the rest of the system, each interaction is scaled by $1/2$ and there are 2 such interactions. For a pair of particles inside the sub-space, however, the interaction is scaled by $1/2 * 1/2 = 1/4$, and since the copies do not interact, there are only 2 such interactions and the sum does not correspond to the correct average. Therefore, the interaction must be scaled up by a factor of N . When the PME technique is requested, this simple scaling cannot be used since the entire charge set is used in the construction of the PME grid and individual charges are not used in the reciprocal space calculation. Therefore, the intra-copy energies and forces are corrected in a separate step for PME calculations. Sander will print out the number of correction interactions that need to

be calculated, and very large amounts of these will make the calculation run more slowly. PME also needs to do a separate correction calculation for excluded atom pairs (atoms that should not have a nonbonded interaction, such as those that are connected by a bond). Large LES regions result in large numbers of excluded atoms, and these will result in a larger computational penalty for LES compared to non-LES simulations. For both of these reasons, it is more efficient computationally to use smaller LES regions- but see the discussion above for how region size affects simulation efficiency. These changes are included in the LES version of Sander (sander.LES). Each particle is assigned a LES 'type' (each new set of copies is a new type), and for each pair of types there is a scaling factor for the nonbond interactions between LES particles of those types. Most of the scaling factors are 1.0, but some are not - such as the diagonal terms which correspond to interactions inside a given subspace, and also off-diagonal terms where only some of the copies are in common. An example of this type is the side chain example given above- each of the 3 backbone copies has 2 sidechains, and while interactions inside the side chains need a factor of 6, interactions between the side chain and backbone need a factor of 3. This matrix of scaling factors is stored in the new topology file, along with the type for each atom, and the number of types. The changes made in sander relate to reading and using these scale factors.

32.5. Using LES with the Generalized Born solvation model

LES simulations can be performed using the GB solvent model, with some limitations. Compared to LES simulations in explicit water, using GB with LES provides several advantages. The most important is how each of the copies interacts with the solvent. With explicit water, the water is normally not copied and therefore interacts in an average way with all LES copies. This has important consequences for solvation of the copies. If the copies move apart, water cannot overlap any of them and therefore the water cavity will be that defined by the union of the space occupied by the copies. This has two consequences. First, moving the copies apart requires creation of a larger solvent cavity and therefore copies have a greater tendency to remain together, reducing the effectiveness of LES. Second, when the copies do move apart, each copy will not be individually solvated.

These effects arise because the water interacts with all of the copies; for each copy to be solvated independently of the other copies would require copying the water molecules. This is normally not a good idea, since copying all of the water would result in very significant computational expense. Copying only water near the solute would be tractable, but one would need to ensure that the copied waters did not exchange with non-LES bulk waters.

Using GB with LES largely overcomes these problems since each copy can be individually solvated with the continuum model. Thus when one copy moves, the solvation of the other copies are not affected. This results in a more reasonable solvation of each copy and also improves the independence of the copies. Of course the resulting simulations do retain all of the limitations that accompany the GB models.

The current code allows *igb* values of 1, 5 or 7 when using LES. Surface area calculations are not yet supported with LES. Only a single LES region is permitted for GB+LES simulations. A new namelist variable was introduced (RDT) in sander to control the compromise of speed and accuracy for GB+LES simulations. The article referenced below provides more detail on the function of this variable. RDT is the effective radii deviation threshold. When using GB+LES, non-LES atoms require multiple effective Born radii for an exact calculation. Using these multiple radii can significantly increase calculation time required for GB calculations. When the difference between the multiple radii for a non-LES atom is less than RDT, only a single effective radius will be used. A value of 0.01 has been found to provide a reasonable compromise between speed and accuracy, and is the default value. Before using this method, it is strongly recommended that the user read the article describing the derivation of the GB+LES approach.^[747]

32.6. Case studies: Examples of application of LES

32.6.1. Enhanced sampling for individual functional groups: Glucose

The first example will deal with enhancing sampling for small parts of a molecule, such as individual functional groups or protein side chains. In this case we wanted to carry out separate simulations of α and β (not converting between anomers, only for conversions involving rotations about bonds) glucose, but the 5 hydroxyl groups and the strong hydrogen bonds between neighboring hydroxyls make conversion between different

32. Locally-enhanced sampling

rotamers slow relative to affordable simulation times. The eventual goal was to carry out free energy simulations converting between anomers, but we need to ensure that each window during the Gibbs calculation would be able to sample all relevant orientations of hydroxyl groups in their proper Boltzmann-weighted populations. We were initially unsure how many different types of structures should be populated and carried out non-LES simulations starting from different conformations. We found that transitions between different conformations were separated by several hundred picoseconds, far too long to expect converged populations during each window of the free energy calculation. We therefore decided to enhance conformational sampling for each hydroxyl group by making 5 copies of each hydroxyl hydrogen and also 5 copies of the entire hydroxymethyl group. Since the hydroxyl rotamer for each copy should be relatively independent, we decided to place each group in a different LES region. This meant that each hydroxyl copy interacted with all copies of the neighboring groups, with a total of $5*5*5*5$ or 3125 structural combinations contributing to the LES average energy at each point in time. The input file is given below.

```
file rprm name=(parm.solv.top) read
file rcvb name=(glucose.solv.equ.crd) read
file wprm name=(les.prmtop) wovr
file wcrd name=(glucose.les.crd) wovr
action
omas
~ 5 copies of each hydroxyl hydrogen- copying oxygen will make no difference
~ since they will not be able to move significantly apart anyway
spac numc=5 pick chem prtc H01 done
spac numc=5 pick chem prtc H02 done
spac numc=5 pick chem prtc H03 done
spac numc=5 pick chem prtc H04 done
~ take the entire hydroxy methyl group
spac numc=5 pick #prt 20 24 done
*EOD
```

This worked quite well, with transitions now occurring every few ps and populations that were essentially independent of initial conformation.[\[744\]](#)

32.6.2. Enhanced sampling for a small region: Application of LES to a nucleic acid loop

In this example, we consider a biomolecule (in this case a single RNA strand) for which part of the structure is reliable and another part is potentially less accurate. This can be the case in a number of different modeling situations, such as with homologous proteins or when the experimental data is incomplete. In this case two different structures were available for the same RNA sequence. While both structures were hairpins with a tetraloop, the loop conformations differed, and one was more accurate. We tested whether MD would be able to show that one structure was not stable and would convert to the other on an affordable timescale.

Standard MD simulations of several ns were not able to undergo any conversion between these two structures (the initial structure was always retained). Since the stem portion of the RNA was considered to be accurate, LES was only applied to the tetraloop region. In this case, both of the ends of the LES region would be attached to the same locations in space, and there was no concern about copies diffusing too far apart to re-converge to the same positions after optimization. The issues that need to be addressed once again are the number of copies to use, and how to place the LES region(s). I usually start with the simplest choices and used 5 LES copies and only a single LES region consisting of the entire loop. If each half of the loop was copied, then it might become too *crowded* with copies near the base-pair hydrogen bonds and conformational changes that required moving a base through this regions could become even more difficult (see the background section for details). Therefore, one region was chosen, and the RNA stem, counterions and solvent were not copied. The ADDLES input file is given below.

```
file rprm name=(prm.top) read
file rcvb name=(rna.crd) read
```

```

file wprm name=(les.parm) wovr
file wcrd name=(les.crd) wovr
action
omas
~ copy the UUCG loop region- residues 5 to 8.
~ pick by atom number, though #mon 5 8 would work the same way
spac numc=5 pick #prt 131 255 done
*EOD

```

Subsequent LES simulations were able to reproducibly convert from what was known to be the incorrect structure to the correct one, and stay in the correct structure in simulations that started there. Different numbers of LES copies as well as slightly changing the size of the LES region (from 4 residues to 6, extending 1 residue beyond the loop on either side) were not found to affect the results. Fewer copies still converted between structures, but on a slower timescale, consistent with the barrier heights being reduced roughly proportional to the number of copies used. See Simmerling, Miller and Kollman, 1998, for further details.

32.6.3. Improving conformational sampling in a small peptide

In this example, we were interested not just in improving sampling of small functional groups or even individual atoms, but in the entire structure of a peptide. The peptide sequence is AVPA, with ACE and NME terminal groups. Copying just the side chains might be helpful, but would not dramatically reduce the barriers to backbone conformational changes, especially in this case with so little conformational variety inherent in the Ala and Pro residues. We therefore apply LES to all atoms. If we copied the entire peptide in 1 LES regions, the copies could float apart. While this would not be a disaster, it would make it difficult to bring all of the copies back together if we were searching for the global energy minimum, as described above. We therefore use more than one LES region, and need to decide where to place the boundaries between regions. A useful rule of thumb is that regions should be at least two amino acids in size, so we pick our two regions as Ace-Ala-Val and Pro-Ala-Nme. If we make five LES copies of each region and each copy does not interact with other copies of the same regions, each half the peptide will be represented by five potentially different conformations at each point in time. In addition, since each copy interacts with all copies of the rest of the system, there are 25 different combinations of the two halves of the peptide that contribute at each point in time. This statistical improvement alone is valuable, but the corresponding barriers are also reduced by approximately the same factors. When we place the peptide in a solvent box the solvent interacts in an average way with each of the copies. The input file is given below, and all of the related files can be found in the test directory for LES.

```

~ all file names are specified at the beginning, before "action"
~ specify input prmtop
file rprm name=(prmtop) read
~ specify input coordinates, velocities and box (this is a restart file)
file rcvb name=(md.solv.crd) read
~ specify LES prmtop
file wprm name=(LES.prmtop) wovr
~ specify LES coordinates (and velocities and box since they were input)
file wcrd name=(LES.crd) wovr
~ now the action command reads the files and tells addles to
~ process commands
action
~ do not scale masses of copied particles
omas
~ divide the peptide into 2 regions.
~ use the CCA option to place the division between carbonyl and
~ alpha carbon
~ use the "or" to make sure all atoms in the terminal residues
~ are included since the CCA option places the region division at C/CA

```

32. Locally-enhanced sampling

```
~ and we want all of the terminal residue included on each end
~
~ make 5 copies of each half
~ "spac" defines a LES subspace (or region)
spac numc=5 pick #cca 1 3 | #mon 1 1 done
spac numc=5 pick #cca 4 6 | #mon 6 6 done
~ the following line is required at the end
*EOD
```

This example brings up several important questions:

1. Should I make LES copies before or after adding solvent? Since LEaP is used to add solvent, and LEaP will not be able to load and understand a LES structure, you must run ADDLES after you have solvated the peptide in LEaP. ADDLES should be the last step before running SANDER.
2. Which structure should be used as input to ADDLES? If you will also be carrying out non-LES simulations, then you can equilibrate the non-LES simulation and carry out any amount of production simulation desired before taking the structure and running ADDLES. At the point you may switch to only LES simulations, or continue both LES and non-LES from the same point (using different versions of SANDER). Typically I equilibrate my system without LES to ensure that it has initial stability and that everything looks OK, then switch to LES afterward. This way I separate any potential problems from incorrect LES setup from those arising from problems with the non-LES setup, such as in initial coordinates, LEaP setup, solvent box dimensions and equilibration protocols.
3. How can I analyze the resulting LES simulation? This is probably the most difficult part of using LES. With all of the extra atoms, most programs will have difficulty. For example, a given amino acid with LES will have multiple phi and psi backbone dihedral angles. There are basically two options: first, you can process your trajectory such that you obtain a single structure (non-LES). This might be just extracting one of the copies, or it might be one by taking the average of the LES copies. After that, you can proceed to traditional analysis but must keep in mind that the average structure may be non-physical and may not represent any actual structure being sampled by the copies, especially if they move apart significantly. A better way is to use LES-friendly analysis tools, such as those developed in the group of Carlos Simmerling. The visualization program MOIL-View (<http://morita.chem.sunysb.edu/carlos/moil-view.html>) is one example of these programs, and has many analysis tools that are fully LES compatible. Read the program web page or manual for more details.

1.7. Unresolved issues with LES in Amber

1. Sander can't currently maintain groups of particles at different temperatures (important for dynamics, less so for optimization.)[743, 748] Users can set *temp0les* to maintain all LES atoms at a temperature that is different from that for the system as a whole, but all LES atoms are then coupled to the same bath.
2. Initial velocity issues as mentioned above- works properly, user must be careful.
3. Analysis programs may not be compatible. See <http://morita.chem.sunysb.edu/carlos/moil-view.html> for an LES-friendly analysis and visualization program.
4. Visualization can be difficult, especially with programs that use distance-based algorithms to determine bonds. See #3 above.
5. Water should not be copied- the fast water routines have not been modified. For most users this won't matter.
6. Copies should not span different 'molecules' for pressure coupling and periodic imaging issues. Copies of an entire 'molecule' should result in the copies being placed in new, separate molecules- currently this is not done. This would include copying things such as counterions and entire protein or nucleic acid chains.
7. Copies are placed into the same residue as the original atoms- this can make some residues much larger than others, and may result in less efficient parallelization with algorithms that assign nonbond workload based on residue numbers.

33. gem.pmemd

33.1. Introduction

The Amoeba force field is a multipolar/polarizable force field with parameters for water, univalent ions, small organic molecules, proteins, nucleic acids and ionic liquids.[18, 19, 518, 519, 749–755] Differences from the current amber force fields include more complex valence terms including anharmonic bond and angle corrections and bond angle and bond dihedral cross terms, and a two dimensional spline fit for the phi-psi bitorsional energy. The differences in the nonbond treatment include the use of atomic multipoles up to quadrupole order, induced dipoles using a Thol  screening model, and the use of the Halgren buffered 7-14 functional form for van der Waals interactions. The PME implementation used here, as well as a multigrid approach for atomic multipoles, is described in Ref. [525].

Right now, setting up the system is a bit complex: you need to set up the system in Tinker, then run the *tinker-to-amber* program to convert to Amber prmtop and coordinate files. Some examples are in `$AMBERHOME/Amber-Tools/src/gem.pmemd/build.amoeba`. For ionic liquid systems, a separate Tinker-style parameter file is provided in the same directory. We hope to provide a simpler path soon and shall post a notice on the Amber webpage when it is available.

Two executables are provided to perform AMOEBA simulations, *sander* and *gem.pmemd*. Both executables employ the same input parameters for the `&amoeba` namelist (see below). The *gem.pmemd* executable has full AMOEBA capabilities, and, in addition, can run MD simulations for an experimental implementation of the GEM* water potential. [756–759] Features such as replica exchange and vdw soft core capabilities are not supported in *gem.pmemd*. Also note that GEM* and the GEM implementation in *gem.pmemd* are **experimental**, the *gem.pmemd* executable is being provided primarily as an option for AMOEBA simulations, as it provides improved parallel performance. Three tests are provided for GEM* for a 2048 waters box involving the model published in Ref. [759], and the new parametrization involving separate exchange and new dispersion terms. The code and test for GEM* are provided *as is*, and methods to build other boxes/parameters are unavailable.

With the use of AMOEBA, minimization as well as usual methods of molecular dynamics can be used, including constant temperature and pressure simulations. In addition, with the AMOEBA implementation it is possible to use the Beeman dynamics integrator, which is helpful in making detailed comparisons to Tinker results. Note that the Amoeba forcefield is parametrized for fully flexible molecules. Thus, SHAKE is not used with this forcefield. In addition to these capabilities, *gem.pmemd* provides the ability to employ a Monte Carlo barostat for constant pressure simulations with AMOEBA.

The parameters *ew_coeff*, *nfft1*, *nfft2*, *nfft3*, and *order* from the `&ewald` section of input all relate to the accuracy of the PME method, which is used in the AMOEBA implementation in *sander*. Due to the use of atomic quadrupoles, *order* (i.e. the B-spline polynomial degree plus one) needs to be at least 5 since the B-spline needs 3 continuous derivatives. The *ew_coeff* together with the direct sum cutoff (see below) controls the accuracy in the Ewald direct sum, and *ew_coeff* together with the PME grid dimensions *nfft1,2,3* and *order* controls the accuracy in the reciprocal sum. Since AMOEBA atomic multipoles are typically dominated by the charges, experience gained in the usual use of PME is pertinent. Typical values we have used for a good cost vs. accuracy balance are *ew_coeff*=0.45, *order*=5, and *nfft1,2,3* approximately 1.25 times the cell length in the relevant direction.

33.2. Input variables

&cntrl Namelist input:

`iamoeba` It should be set to 1 to use the AMOEBA force field. To use GEM* set to 2. To use GEM set to 3. When AMOEBA is used, only an `&amoeba` namelist is required (see below). When GEM* or

33. *gem.pmemd*

GEM are used, both `&amoeba` and `&gem` namelists must be provided. The `&amoeba` section then serves to provide information needed to evaluate covalent and polarization terms of the forcefield.

`&amoeba` Namelist input:

`beeman_integrator` Setting this to be one turns on the Beeman integrator. This is the default integrator for AMOEBA in Tinker. In sander this integrator can be used for NVE simulations, or for NVT or NTP simulations using the Berendsen coupling scheme. (This means that you must set `ntt` to 0 or 1 if you use the Beeman integrator.) By default, `beeman_integrator=0`, and the usual velocity Verlet integration scheme is used instead.

`amoeba_verbose` In addition to the usual sander output, by setting `amoeba_verbose=1`, energy and virial components can be output. By default, `amoeba_verbose=0`.

`ee_dsum_cut` This is the ewald direct sum cutoff. In the amoeba implementation this is allowed to be different from the nonbond cutoff specified by `cut`. It should be less than or equal to the latter. (Note, this feature does not apply to the direct sum for standard amber force fields, which use the nonbond cutoff for the Ewald direct sum as well as van der Waals interactions. The default is 7.0 Angstroms, which is conservative for energy conservation with `ew_coff=0.45`).

`dipole_scf_tol` The induced dipoles in the amoeba force field are solutions to a set of linear equations (like the Applequist model but modified by Thol  damping for close dipole-dipole interactions). These equations are solved iteratively by the method of successive over-relaxation. `dipole_scf_tol` is the convergence criterion for the iterative solution to the linear equations. The iterations towards convergence stop when the RMS difference between successive sets of induced dipoles is less than this tolerance in Debye. The default is set to 0.01 Debye, which has been seen to give reasonable energetics and dynamics, but requires mild temperature restraints. Good energy conservation in NVE simulations requires a tolerance of about 10^{-6} Debye.

`sor_coefficient` This is the successive over-relaxation parameter. This can be adjusted to optimize the number of iterations needed to achieve convergence. Default value is 0.75. Productive values seem to be in the range 0.6-0.8. The optimal values seem to depend on the polarizabilities of the system atoms.

`dipole_scf_iter_max` This prevents infinite iterations when the polarization equations are somehow not converging. A possible reason for this is a bad `sor_coefficient`, exacerbated by a close contact. Default is 50. For comparison, with typical `sor_coefficient` values and an equilibrated system it should take 4-7 iterations to achieve 0.01 Debye convergence and 18-25 iterations to achieve 10^{-6} Debye.

`ee_damped_cut` This is used to cutoff the Thol  damping interactions. The default value is 4.5 Angstroms, which should work for the typical sized polarizabilities encountered, and the default Thol  screening parameter (0.39).

`do_vdw_taper` Amoeba uses a Halgren buffered 7-14 form for the van der Waals interactions. In the Tinker code these are typically evaluated out to 12 Angstroms, with a taper turned on and no long-range isotropic continuum corrections to the energy and virial. In the sander implementation, the usual nonbond cutoff from the `&cntrl` namelist is used for van der Waals interactions. The long range correction is available to allow for shorter cutoffs. Setting `do_vdw_taper` to one causes VDW interactions to be tapered to zero beginning at 0.9 times the van der waals cutoff. The taper is a 5th order polynomial switch on the energy term, which gets differentiated for the forces (atom based switching). It's turned on by default.

`do_vdw_longrange` Setting this to one causes the long-range isotropic continuum correction to be turned on. This adjusts the energy and virial, and in most cases will result in energies and virials that are fairly invariant to van der Waals cutoff, with or without the above taper function. The integrals involved in this correction are done numerically.

There are a lot of other `do_`-prefixed keywords that may be used in the `&amoeba` namelist; these are all used to turn on or off evaluation of various energy/force components in the Amoeba forcefield. These really were intended primarily for development test, but we mention them here as you may encounter them in some of the sample test cases.

In addition to the `&amoeba` namelist, `gem.pmemd` must have a `&gem` namelist which includes several options to deal with the Gaussian distributions.

&gem Namelist input:

`pme_auto_setup` Set to 1 to use the PME method

`regewald_auto_setup` Set to 1 to use the regular Ewald method

`ffp_auto_setup` Set to 1 to use the Fast Fourier Poisson (FFP) method

`nfft#_for_gridtype` *nfft1_for_gridtype*, *nfft2_for_gridtype*, *nfft3_for_gridtype* – These keywords determine the FFT grid count in the x, y, and z dimensions of the unit cell. Assuming that `coul_CD_split_exponent` is less than 0.3 (for the current GEM* fitted density), reasonable accuracy is obtained by specifying values that result in a grid density in the range of 1.3 to 1.5 grids per Å. If the `coul_CD_split_exponent` value is higher, which typically results in the inclusion of diffuse hermites in the system, then values of 2.0 grids per Å or higher are appropriate.

`coul_gaussian_extent_tol` Coulomb Hermite Gaussian extent error tolerance. The default is 1.d-08.

`exch_gaussian_extent_tol` Exchange Hermite Gaussian extent error tolerance. The default is 1.d-08.

`bspline_order_for_gridtype` A bspline order of 6 typically produces reasonable results.

`coul_CD_split_expon` GEM Hermites with an exponent of less than `coul_CD_split_expon` will be treated as diffuse GEM Hermites and all pairs involving Hermites below this exponent (diffuse–diffuse and compact–diffuse) will be evaluated in reciprocal space. The rest of the GEM Hermites will be treated as compact Hermites and evaluated only in direct space (compact–compact). Setting this value effectively determines the minimum cutoff required for direct space evaluation of GEM compact Hermites.

`exch_factor` The proportionality factor for exchange forces and energies. The current exchange factor for the fitted GEM* parameters is 6.6899; the current default is 1.d0, and not recommended.

`exch_cutoff` The current auxiliary fitting basis contains Gaussian Hermites that result in fairly compact Gaussian products, and it is possible to neglect a large number of overlap integrals for the Exchange repulsion term by using a cutoff distance. We currently recommend this value be set to 6.d0 Å; the current default is 10.d0, and is unnecessarily large.

`gem_verbose` Additional information about energy decomposition, etc. is available by setting `gem_verbose` to 1, 2, or 3, with information increasing in that order.

`gaussian_recip_tol` A reciprocal space tolerance only used for FFP. The current default is 1.d-08.

Namelist input reserved for use in GEM forcefield development: The following `&gem` namelist keywords are allowed input to `gem.pmemd`, but usage is discouraged, as usage of these keywords requires an intimate knowledge of GEM parameters as well as efficiency and accuracy considerations. Appropriate system–dependent values for these keywords are determined by the `*_auto_setup` keywords in the `&gem` Namelist. The experimental reserved keywords include:

`user_num_HC_prim_grids`, `user_num_HD_prim_grids`, `user_num_aux_Cprims`, `user_num_aux_Dprims`,
`user_num_prim_gridtypes`, `user_gridtype_for_MPOLES`, `user_gridtype_for_sumch`,
`user_gridtype_for_HC_prim_grid`, `user_gridtype_for_HD_prim_grid`, `user_HC_gridtype_idx_for_Caux`,
`user_HD_gridtype_idx_for_Daux`, `coulomb_use_recip`, `struc_fac_method_for_gridtype`

33. *gem.pmemd*

gem.pmemd limitations:

The current version of *gem.pmemd* supports constant T and constant P (only with the Monte Carlo barostat) simulations with GEM*. Only orthogonal unit cells are supported. *The appropriate options to meet these conditions should be selected under the &cntrl namelist.*

Extra *gem.pmemd* input files required for the GEM* test:

- gem_aux- File specifying the gem multipole and Hermite basis set/fitting coefficients. The example provided in the tests is Avg_A4_Analytic.
- gem_exch- File specifying the gem multipole and Hermite basis set/fitting *exchange* coefficients (if unspecified file from -gem_aux is used). The example provided in the tests is EXCHANGE.GEM.SNK.SMITH, *only for GEM2021 (iamoeba=3).*
- gem_lst- File specifying various details about the GEM* molecules, including the local frame descriptions.
- gem_crd- A redundant file specifying atom coordinates in a format suitable for earlier GEM code. While this input will be checked to see if it matches inpcrd, there is no requirement for it to match. An informational message will be printed if the files do not match. In our test cases, a totally incorrect file (but in the correct format) is provided. This has been retained for cross-checking capability, but is of no use to non-developers. The file we provide in tests is equil_216_wat_crd.

34. Artificial Intelligence / Machine Learning

There is a lot of interest in using “artificial intelligence” or “machine learning” techniques, such as neural networks or kernel machines, to increase accuracy in the calculation of energies and forces for biomolecule configurations. Various new standalone tools for this exist or are being developed by groups around the world. The classical treatment provided by an Amber forcefield is efficient and often accurate; however various quantum effects are not captured by the default atomistic model, arising for instance from collective motion of electrons, non-spherical shapes of electron distributions, and coupling of electronic and nuclear degrees of freedom. Machine learning methods seek to fit a potential energy surface with a more complicated form than the physically-motivated Amber Hamiltonian, allowing greater accuracy typically at the cost of a greater input of training data, and often also suffering a loss of transferability or an increased cost of evaluation.

The field of machine learning is large and currently evolving quickly, so it is not possible to closely integrate every new method with Amber, however such integration is ultimately very desirable in order to combine the efficiency of the standard classical Hamiltonian with the accuracy of machine learning techniques for a subset of the system, for example a drug molecule, or for selected interactions or degrees of freedom. The ANI neural network toolkit can be successfully run with Amber in this way however it is not currently supported or provided directly as an Amber component; interested users are encouraged to obtain the software from sources documented in the provided citation Smith *et al.* [760].

34.1. KMMD: Molecular Dynamics Using a Kernel Machine

With KMMD, a simple kernel machine is provided as a means to add information learned from *ab-initio* quantum simulations (or some other source) on top of a classical Amber forcefield evaluation, under the name Kernel Modified Molecular Dynamics (KMMD). It requires as input a set of molecular configurations $\{\vec{x}_i\}$ for the system of interest, each labelled with a reference energy E_i typically calculated using accurate *ab initio* methods. Labels must also then be generated for the energy of the configuration given the Amber forcefield which will be used for the simulation, and the KMMD will attempt to predict the difference for not-seen conformations between the forcefield-defined force+energy and the force+energy that would be calculated using the same *ab-initio* method which generated the training data. The functional form of the energy delta for a new configuration $\vec{x}$ is very simple:

$$\Delta E(\vec{x}) = \frac{1}{Z} \sum_i \omega_i \Delta E_i \exp(-r(\vec{x}, \vec{x}_i)^2 / \sigma^2) \quad (34.1)$$

Here Z is a normalisation factor, σ^2 is a smoothing parameter, and $r(\vec{x}, \vec{x}_i)$ is a distance calculated between the test point $\vec{x}$ and the i th training point $\vec{x}_i$ in the feature space of the kernel. Formally this is called a *Normalised Radial Basis Function* (NRBF) Network or a NRBF Kernel Machine [761], the architecture can be expressed equivalently either as a single-layer neural network or as a kernel machine. The KMMD software provided with Amber calculates the distance $r(\vec{x}, \vec{x}_i)$ as a standard Euclidean or l^2 norm in the space of sines and cosines of dihedral angles, thus information on bond lengths or three-point bond angles does not enter the calculation, however joint information about correlations of two or many dihedral angles can be retrieved. The dihedral angles to be treated in the calculation are defined by the user, loosely the number of training points should grow exponentially with the number of degrees of freedom to be corrected, so these should be selected carefully as relevant to the problem at hand.

As seen from equation 34.1, in the limit of σ^2 tending to zero, training point energies should be recovered exactly (although in this limit forces would change too sharply for stable MD simulation), while for larger σ^2 the perturbation to the energy is an average over the nearest few training points. In practice the choice of σ^2 seems not to be very important, within the range 0.01 to 1. Training points can be weighted with some ω_i , however the method seems in practice to function reasonably well with these values left to be equal for each point.

34.1.1. KMMD Intended Use

The KMMD method is not highly ambitious and cannot fully replace physics-driven simulation, however it is constructed such that in the limit of infinite training data then exactly correct dynamics, in the rather narrow sense of exactly correct potential surfaces in the space of dihedral angles, can be recovered. This approach is inspired by the CMAP correction [444, 445] which can already be fitted for AMBER forcefields (18.6); the purpose of CMAP is to define a correct 2-dimensional distribution over a pair of dihedral angles, KMMD extends this to an arbitrary number of dihedrals. An appropriate use would be to refine the classical treatment of a small but flexible drug molecule in the case that a correct conformational ensemble could not be recovered by simpler methods.

An example specific problem that KMMD could be used to fix is to compensate for the lack of an appropriately prepared forcefield for a given situation, in particular transfer of a molecule from aqueous to other or to no solvent. Partial charges on given atomic sites are likely to change depending on the electrostatic environment, and this is relatively easy to fix by modifying the partial charge set defined in the parmtop based on simple calculations (one set in water clusters, and one in vacuum, for example). The coupling of torsions to environment, mediated by complex electronic effects, is less easy to fix and amounts to derivation of an entire new forcefield. KMMD should be able to ameliorate problems of this sort when trained and applied on a case-by-case basis.

An application from the literature is to correct the stretching behaviour of a small DNA duplex (cite this paper if you use KMMD in a publication [762]). Because the spring constant for DNA depends on collective motion of multiple dihedrals, while having a low effective dimensionality overall, a KMMD correction is a sensible choice to augment the standard forcefield.

34.1.2. System Preparation for KMMD

The Amber input file should have the iextpot key set, and an extpot namelist defined, thus:

```
.
.
iextpot = 1,
/
&extpot
extprog='kmmd',           !select KMMD
json='../kmmd_test.json', !control file for KMMD
/
```

1. Prepare a parmtop, restart and pdb file for the system of interest. Atom order and names should be the same in the parmtop and the pdb, you can use *cpptraj* to generate a pdb with appropriate naming from the parmtop+restart.
2. For each training point (each configuration of the target system) evaluate a reference energy using the method of your choice. Save the training point in .xyz format, i.e.:
 - line 1: N_atoms (single integer)
 - line 2: #header (string)
 - lines 3 to N+2: Element x y z (4-column, atomic coordinates in columns 2,3,4).

Atom order should be the same as for the parmtop and pdb files.

3. Prepare a summary of the training points to be used, together with classical energies for each point calculated using the parmtop that you will use for the simulation. A python script for this task is provided without warranty in `AmberTools/src/KMMD/scripts` however the best method to do this depends on the type of simulation that you intend to apply the KMMD correction to. An example (very small) summary file is contained in `AmberTools/test/KMMD/MINI_KMMD_DB/cache_ffenes.txt`. Each line of the file follows the same format, with a minimum of 4 columns:

- Column 1: path to a reference .xyz file (either absolute, or relative to the directory in which you intend to run your calculation).
 - Column 2: forcefield potential energy, evaluated using the same parmtop and Amber input settings that you will use in your calculation.
 - Column 3: reference energy in units kcal/mol. The KMMD code will learn the difference between Columns 3 and 2.
 - Column 4: a weight. 1.0 is a good default weight. In the field of kernel machines it is common to optimise weights and give varying importance to different training points, however be aware that very large training sets are needed for such a procedure to be useful. Weights can also be phased to zero across a sequence of multiple parallel KMMD calculations, as a means to smoothly turn off the KMMD potential.
 - (optionally: Column 5): weight gradient. It is possible to measure the free energy of phasing a KMMD correction in or out, or the work to change from one KMMD correction to another, by systematically varying the weights for multiple KMMD calculations, and defining the gradient of each weight with respect to the fictitious coordinate of the integration. If weight gradients are defined, the KMMD code will track the generalised force which can then be integrated with respect to the coordinate. This TI method operates independently of the primary AMBER code and writes the generalised forces to their own log file, "KMMD_ti_gradients.txt". If set, usually the weight gradient should be equal to 1.0 (phasing in) or -1.0 (phasing out).
4. Prepare a .json-like file describing the degrees of freedom to be tracked in the KMMD calculation. An example such file for simulation of the DNA duplex GG-CC is provided in `AmberTools/test/KMMD/KMMD-test.json`. This file tells KMMD where to look for input data, and defines the torsion angles which KMMD will use to exert a force. Keys are given in double quotes, and separated from each value by a colon, something like "key": value, on each line. Lines are terminated by a newline, entries are terminated by a comma. Line-by-line, the file format is:

```
{
    #The file is contained in a pair of curly brackets

    "DB_fileList": "cache_ffenes.txt", #A path to the summary of training points.
    "ref_pdb": "GG.pdb",              #A path to the reference pdb file.
    "sigma2": 0.1,                    #Smoothing parameter, robust to a range of values.

    "dh_atnames": [                  #Define the dihedrals to be tracked,
        {"alpha" : "O3' P   O5' C5'"}, #in this case DNA sugar torsions nu0-nu4...
        {"nu0" : "C4' O4' C1' C2'"}, #and also the backbone torsion alpha,
        {"nu1" : "O4' C1' C2' C3'"}, # which crosses two residues.
        {"nu2" : "C1' C2' C3' C4'"},
        {"nu3" : "C2' C3' C4' O4'"},
        {"nu4" : "C3' C4' O4' C1'"}
    ],                                #Close the list of dihedrals

    "dh_resdel": [                  #For each treated dihedral,
        {"nu0" : "0 0 0 0"},        #any of the four atoms may be at an offset
        {"nu1" : "0 0 0 0"},        #to the "main" residue defined below.
        {"nu2" : "0 0 0 0"},
        {"nu3" : "0 0 0 0"},
        {"nu4" : "0 0 0 0"},
        {"alpha" : "-1 0 0 0"},      #this has a non-zero value because
                                    #alpha torsions cross two DNA residues.
    ],

    "dh_byres": [                  #Having defined dihedrals to treat, we can match the same ones
        {"nu0 1" : ""},             #in multiple residues, in this example we treat nu0-4
    ]
}
```

```

{"nu1 1" : ""},    #in residues 1 and 2, and alpha at the junction between them.
{"nu2 1" : ""},    #The {"key":"value"} format is abused here: "value" is empty
{"nu3 1" : ""},    #and only the presence of absence of a given dihedral matters.
{"nu4 1" : ""},
{"alpha 2" : ""},
{"nu0 2" : ""},
{"nu1 2" : ""},
{"nu2 2" : ""},
{"nu3 2" : ""},
{"nu4 2" : ""},
]
}#end the file.

```

Note that the json file format does not formally support comments, with the # symbol or anything else. Although the above seems to be processed adequately, for production code it is probably better not to comment json files.

34.1.3. KMMD Output

Changes made to the energy by KMMD are added to the RESTRAINT energy recorded in the Amber output file, forces are returned as a perturbation to the forces calculated by the primary Amber Hamiltonian. Further logging information about the setup of the calculation is sent to standard output and standard error, this can typically be disregarded if the calculation seems to be working well.

In the case that weight gradients were defined for the training points, generalised force information is accumulated and averages are written out to a special file created in whatever directory the calculation is running, called “KMMD_ti_gradients.txt”.

35. NAB, nabc and libsff

The NAB language compiler *nab2c* (which converts NAB source code to C, for subsequent compilation) was written in the 1990's by Tom Macke. The original design idea was to create a “molecular awk”: a scripting language for manipulation of (macro-)molecules that would be primarily used to create short scripts to carry out molecular manipulations. It was quickly realized that manipulations like force field minimization would be useful, and the Amber-compatible molecular mechanics routines were added by David Case as *sff*, a “simple force field”.

Over the years, *sff* evolved to keep pace with (and in many cases drive) Amber developments involving implicit force fields, including generalized Born, Poisson-Boltzmann and RISM approaches. In keeping with its original motivation, *sff* concentrated on implicit solvation, leaving explicit solvent and periodic simulations to the main Amber programs *sander* and *pmemd*. The *sff* routines were parallelized using both openmp and MPI, and second derivatives of the generalized Born model were added by Russ Brown.[763] Apart from the lack of a GPU implementation, the routines in *sff* are the most general and efficient ones in the Amber package. In particular, *sff* excels at generalized Born simulations on large systems, benefitting from an advanced nonbonded list builder, and from the hierarchical charge partition model.

Since relatively few people are using this (rather old code), we have moved it from *AmberTools* into a new repository: <https://github.com/dacase/nabc>. This repository has *nabc* (a C-language interface to *libsff*) and *libsff* itself, along with the older NAB compiler. Since this is all based on Amber file formats, etc., serious use of this package will require AmberTools to be installed as well. Full installation and usage documentation are now at on github.

Part V.

Analysis of simulations

36. mdout_analyzer.py and ambpdb

mdout_analyzer.py is a simple script designed to help you rapidly parse and analyze the energy components printed in the output files from *sander* and *pmemd*. It requires that the *numpy* and *matplotlib* packages be installed. The *scipy* Python package is also required when plotting smoothed histograms using kernel density estimates. You can use it as follows:

```
mdout_analyzer.py <mdout1> <mdout2> <mdout3> ... <mdoutN>
```

Where each mdout file is combined into a single data set. A GUI window will open up with buttons for every energy component parsed from the mdout file followed by a button for each type of graphical analysis you can do on the data shown below.

A second window has options to control how the graphs will appear. Help is available in the <Help> menu at the top of the main window. Note, mdout files must be from the same type of simulation (or at least have all of the same energy components printed inside) in order to be combined.

Right-clicking on each energy button brings up a little window describing what that energy term is.

36.1. ambpdb

NAME ambpdb - convert amber-format coordinate files to pdb format

SYNOPSIS

```
ambpdb [ -p prmtop-file ] < AmberRestartFile  
ambpdb [ -p prmtop-file ] -c coordinate-file
```

Additional Options:

```
[ -tit title ] [ -pqr|-mol2 ] [ -aatm ] [ -bres ] [ -noter ] [ -offset # ] [ -ext ]
```

ambpdb is a filter to take a coordinate "restart" file from an AMBER dynamics or minimization run and prepares a pdb-format file (on STDOUT). The program assumes that a *prmtop* file is available, from which it gets atom and residue names. Note: starting with AmberTools15, ambpdb can convert *any* coordinate file format that CPPTRAJ can read using the '*-c*' flag. Either an Amber restart file must be directed in via STDIN or a file with '*-c*' must be specified.

OPTIONS

- h* Print a usage summary to the screen.
- p* Specify the Amber topology file to use (if not specified will look for file named "prmtop").
- c* Instead of reading an Amber restart from STDIN, specifies file to read coordinates from; can be any format that CPPTRAJ can read.
- tit* The title, if given, will be output as a REMARK at the top of the file. It should be protected by quotes or double quotes if it contains spaces or special characters.

- pqr* If *-pqr* is set, output will be in the format needed for the electrostatics programs that need charge and radius information.
- mol2* creates a TRIPOS mol2 file with all of the residues and bond information present in the topology file.
- aatm* This switch controls whether the output atom names follow Amber or Brookhaven (PDB) formats. With the default (when this switch is not set), atom names will be placed into four columns following the rules used by the Protein Data Base in Version 3.
- bres* If *-bres* (Brookhaven-residue-names) is not set (the default), Amber-specific atom names (like CYX, HIE, RG5, etc.) will be kept in the pdb file; otherwise, these will be converted to PDB-standard names (CYS, HIS, G, in the above example). Note that setting *-bres* creates a naming ambiguity between protonated and unprotonated forms of amino acids.
- If you plan to re-read the pdb file back into Amber programs, you should use the default behavior; for programs that demand stricter conformance to Brookhaven standards, set *-bres*.
- noter* If *-noter* is set, the output PDB file not include TER cards between molecules. Otherwise, TER cards will be added whenever there is not bond between adjacent residues. Note that this means there will be a TER card between each water molecule, for example, unless *-noter* is set. The PDB is idiosyncratic about TER cards: they are generally present between separate protein chains, but generally not present between cofactors or solvent molecules. This behavior is not mimicked by *ambpdb*.
- offset* If a number is given here, it will be added to all residue numbers in the output pdb file. This is useful if you want the first residue (which is always "1" in an Amber prmtop file, to be a larger number, (say to more closely match a file from Brookhaven, where initial residues may be missing). Note that the number you provide is one less than what you want the first residue to have.
- Residue numbers greater than 9999 will not "fit" into the Brookhaven format; *ambpdb* actually prints `mod(resno,10000)`; that is, after 9999, the residue number re-cycles to 0.
- ext* This tells *ambpdb* to use any extended PDB info present in prmtop-file (from using e.g. the 'addPDB' command from *parmed*).

37. cpptraj

37.1. Introduction

Cpptraj[\[764\]](#) (the successor to *ptraj*) is the main program in Amber for processing coordinate trajectories and data files. *Cpptraj* has a wide range of functionality, and makes use of OpenMP/MPI to speed up many calculations, including processing ensembles of trajectories and/or conducting multiple analyses in parallel with MPI.[\[765\]](#)

Here are several notable features of *cpptraj*:

1. Trajectories with different topologies can be processed in the same run.
2. Several actions/analyses in *cpptraj* are OpenMP parallelized; see section [37.2.7.2](#) for more details.
3. Trajectory and ensemble reads can be MPI parallelized.
4. Almost any file read or written by *cpptraj* can be compressed (with the exception of the NetCDF trajectory format). So for example gzipped/bzipped topology files can be read, and data files can be written out as gzip/bzip2 files. Compression is detected automatically when reading, and is determined by the filename extension (.gz and .bz2 respectively) on writing.
5. The format of output data files can be specified by extension. For example, data files can be written in xmgrace format if the filename given has a '.agr' extension. A trajectory can be written in DCD format if the '.dcd' extension is used.
6. Multiple output trajectories can be specified, and can be written during action processing (as opposed to only after) via the **outtraj** command. In addition, output files can be directed to write only specific frames from the input trajectories.
7. Multiple reference structures can be specified. Specific frames from trajectories may be used as a reference structure.
8. The **rmsd** action allows specification of a separate mask for the reference structure. In addition, per-residue RMSD can be calculated easily.
9. Actions that modify coordinates and topology such as the **strip/closest** actions can often write an accompanying fully-functional stripped topology file.
10. Users usually are able to fine-tune the output format of data files declared in actions using the “**out**” keyword (for example, the precision of the numbers can be changed). In addition, users can control which data sets are written to which files (e.g. if two actions specify the same data file with the 'out' keyword, data from both actions will be written to that data file).
11. Users can manipulate data sets using mathematical expressions (with some limitations), see [37.5.2 on page 694](#) for details.
12. There is some support for creating internal loops over e.g. mask expressions and setting internal variables (see **for**, **set**, and **show** commands).

See the README.md file in the *cpptraj* home directory for information on how to build, authors, and so on.

37.1.1. Manual Syntax Format

The syntax presented in this manual uses the following conventions:

<> Denotes a variable.

[] Denotes something is optional.

{|} Denotes several choices separated by the '|' character; one of the choices must be specified.

... Denotes the preceding option can be repeated.

Everything else is as printed.

37.1.2. Installation

See instructions in the CPPTRAJ GitHub repository README.md file under 'Installation & Testing': <https://github.com/Amber-MD/cpptraj>

37.1.3. Examples

Some examples of running CPPTRAJ are available in the **examples** subdirectory. There are also many tests in the **test** subdirectory which can serve as simple examples.

37.2. Running Cpptraj

Cpptraj can be run in either "interactive mode" or in "batch mode".

37.2.1. Command Line Syntax

```
cpptraj [-p <Top0>] [-i <Input0>] [-y <trajin>] [-x <trajout>]
        [-ya <args>] [-xa <args>] [<file>]
        [-c <reference>] [-d <datain>] [-w <dataout>] [-o <output>]
        [-h | --help] [-V | --version] [--defines] [-debug <#>]
        [--interactive] [--log <logfile>] [-tl]
        [-ms <mask>] [-mr <mask>] [--mask <mask>] [--resmask <mask>]
        [--rng {marsaglia|stdlib|mt|pcg32|xol28}] [--charge <mask>]
```

* denotes a flag may be specified multiple times.

-p <Top0>* Load <Top0> as a topology file.

-i <Input0>* Read input from <Input0>.

-y <trajin>* Read from trajectory file <trajin>; same as input
'trajin <trajin>'.

-x <trajout>* Write trajectory file <trajout>; same as input
'trajout <trajout>'.

-ya <args>* Input trajectory file arguments.

-xa <args>* Output trajectory file arguments.

<file>* A topology, input trajectory, or file containing cpptraj
input.

-c <reference>* Read <reference> as reference coordinates; same as
input 'reference <reference>'.

-d <datain>* Read data in from file <datain> ('readdata <datain>').

```

-w <dataout> Write data from <datain> as file <dataout>
('writedata <dataout>').
-o <output> Write CPPTRAJ STDOUT output to file <output>.
-h | -help Print command line help and exit.
-V | -version Print version and exit.
-defines Print compiler defines and exit.
-debug <#> Set global debug level to <#>; same as input 'debug
<#>'.
-interactive Force interactive mode.
-log <logfile> Record commands to <logfile> (interactive mode only).
Default is 'cpptraj.log'.
-tl Print length of trajectories specified with '-y' to STDOUT. The
total number of frames is written out as 'Frames: <X>'.
-ms <mask> Print selected atom numbers to STDOUT. Selected atoms
are written out as 'Selected= 1 2 3 ...'.
-mr <mask> : Print selected residue numbers to STDOUT. Selected
residues are written out as 'Selected= 1 2 3 ...'.
-mask <mask> Print detailed atom selection to STDOUT.
-resmask <mask> Print detailed residue selection to STDOUT.
--rng <type> Change default random number generator.
--charge <mask> Print total charge (in e-) of atoms selected by
<mask> to STDOUT.

```

Note that unlike *ptraj*, in *cpptraj* it is not required that a topology file be specified on the command line as long as one is specified in the input file with the 'parm' keyword. Multiple topology/input files can be specified by use of multiple '-p' and '-i' flags. All topology and coordinate flags will be processed before any input flags.

37.2.2. Commands

Input to *cpptraj* is in the form of commands, which can be categorized in to 2 types: immediate and queued. Immediate commands are executed as soon as they are encountered. Queued commands are initialized when they are encountered, but are not executed until a Run is executed via a *run* or *go* command. Actions, Analyses, and Trajectory commands (except *reference*) are queued commands; however, they can also be run immediately via commands such as *crdaction*, *runanalysis*, *loadcrd*, etc. See [37.7 on page 702](#) for more details.

Commands fall into seven categories:

General (Immediate) These commands are executed immediately when entered.

System (Immediate) These are unix system commands (e.g. 'ls', 'pwd', etc).

Coords (Immediate) These commands are used to manipulate COORDS data sets; see [37.7 on page 702](#) for more details.

Trajectory (Queued) These commands prepare *cpptraj* for reading or writing trajectories during a Run.

Topology (Immediate) These commands are used to read, write, and modify topology information.

Action (Queued) These commands specify actions that will be performed on coordinate frames read in from trajectories during a Run.

Analysis (Queued) These commands specify analyses that will be performed on data that has been either generated from a Run or read in from an external source.

Control (Immediate) These commands set up control blocks that can be used to e.g. loop over a set of commands.

In addition to normal commands, *cpptraj* now has the ability to perform certain basic math operations, even on data sets. See [37.5.2 on page 694](#) for more details.

Commands in *cpptraj* can be read in from an input file or from the interactive command prompt. A '#' anywhere on a line denotes a comment; anything after '#' will be ignored no matter where it occurs. A '\' allows the continuation of one line to another. For example, the input:

```
# Sample input
trajin mdcrd # This is a trajectory
rms first out rmsd.dat \
:1-10
```

Translates to:

```
trajin mdcrd
rms first out rmsd.dat :1-10
```

37.2.3. Getting Help

If in interactive mode, the 'help' command can be used to list recognized commands and topics; topics (such as mask syntax) start with uppercase letters. 'help <command>' can be used to get the associated keywords as well as an abbreviated description of the command. Most commands have a corresponding test which also serves as an example of how to use the command. See \$AMBERHOME/AmberTools/test/cpptraj/README for more details.

37.2.4. Batch mode

In "batch" mode, *cpptraj* is executed from the command line with one or more input files containing commands to be processed or STDIN. The syntax of <input file> is similar to that of *ptraj*. Keywords specifying different commands are given one per line. Lines beginning with '#' are ignored as comments. Lines can also be continued through use of the '\' character. This is the only allowed mode for *cpptraj.MPI*.

37.2.5. Interactive mode

In "interactive mode" users can enter commands in a UNIX-like shell. Interactive mode is useful for running short and simple analyses or for trying out new kinds of analyses. If *cpptraj* is run with '-interactive', no arguments, or no specified input file:

```
cpptraj
cpptraj --interactive
cpptraj <parm file>
cpptraj -p <parm file>
```

this brings up the interactive interface. This interface supports command history (via the up and down arrows) and tab completion for commands and file names. If no log file name has been given (with '-log <logfile>'), all commands used in interactive mode will be logged to a file named 'cpptraj.log', which can subsequently be used as input if desired. When starting *cpptraj*, command histories will be read from any existing logs.

37.2.6. Trajectory Processing "Run"

Like *ptraj*, a trajectory processing "Run" is one of the main ways to run *cpptraj*. First the Run is set up via commands read in from an input file or the interactive prompt. Trajectories are then read in one frame at a time (or in the case of ensemble processing all frames from a given step are read). Actions are performed on the coordinates stored in the frame, after which any output coordinates are written. At the end of the run, any data sets generated are written, and any queued Analyses are performed.

37.2.6.1. Actions and multiple topologies

Since *cpptraj* supports multiple topology files, during a Run actions are set up every time the topology changes in order to recalculate things like what atoms are in a mask etc. Actions that are not valid for the current topology are skipped for that topology. So for example given two topology files with 100 residues, if the first topology file processed includes a ligand named MOL and the second one does not, the action:

```
distance :80 :MOL out D.80-to-MOL.dat
```

will be valid for the first topology but not for the second, so it will be skipped as long as the second topology is active.

37.2.7. Parallelization

Cpptraj has many levels of parallelization that can be enabled via the '-mpi', '-openmp', and/or '-cuda' configure flags for MPI, OpenMP, and CUDA parallelization respectively. At the highest level, trajectory and ensemble reads are parallelized with MPI. In addition, certain time consuming actions have been parallelized with OpenMP and/or CUDA.

Note that any combination of the '-openmp', '-cuda', and '-mpi' flags may be used to generate a hybrid MPI/OpenMP/CUDA binary; however this may require additional runtime setup (e.g. setting OMP_NUM_THREADS for OpenMP) to work properly and not oversubscribe cores.

37.2.7.1. MPI Trajectory Parallelization

Cpptraj has two levels of MPI parallelization for reading input trajectories. The first is for '*trajin*' trajectory input, where the trajectory read is divided as evenly as possible among all input frames (across-trajectory parallelism). For example, if given two trajectories of 1000 frames each and 4 MPI processes, process 0 reads frames 1-500 of trajectory 1, process 1 reads frames 501-1000 of trajectory 1, process 2 reads frames 1-500 of trajectory 2, and process 3 reads frames 501-1000 of trajectory 2. Most Actions will work with across-trajectory parallelization with the exception of the following: '*clusterdihedral*', '*contacts*', '*createreservoir*', '*lipidorder*', '*pairwise*', '*stfcdiffusion*', '*tordiff*', '*unwrap*', and '*xtalsymm*'. The '*diffusion*' Action will only work with across-trajectory parallelism if no imaging is to be performed.

In addition to across-trajectory parallelism, the '*gist*' command will also MPI-parallelize the entropy calculation that occurs after trajectory processing.

The second is for '*ensemble*' trajectory input, where the reading/processing/writing of each member of the ensemble is divided up among MPI processes. The number of MPI processes must be a multiple of the ensemble size. If the number of processes is greater than the ensemble size then the processing of each ensemble member will be divided among MPI processes (i.e. across-trajectory parallelism will be used). For example, given an ensemble of 4 trajectories and 8 processes, processes 0 and 1 are assigned to the first ensemble trajectory, processes 2 and 3 are assigned to the second ensemble trajectory, and so on. When using ensemble mode in parallel it is recommended that the *ensemblesize* command be used prior to any ensemble command as this will make set up far more efficient.

Note that most Analyses are not MPI-parallelized, with the exception of the *calcdiffusion* Analysis ([37.12.3 on page 849](#)).

In order to use the MPI version, Amber/*cpptraj* should be configured with the '-mpi' flag. You can tell if *cpptraj* has been compiled with MPI as it will print 'MPI' in the title, and/or by calling '*cpptraj* —defines' and looking for '-DMPI'.

37.2.7.2. OpenMP Parallelization

Some of the more time-consuming actions/analyses in *cpptraj* have been parallelized with OpenMP to take advantage of machines with multiple cores. In order to use OpenMP parallelization Amber/*cpptraj* should be configured with the '-openmp' flag. You can easily tell if *cpptraj* has been compiled with OpenMP as it will print 'OpenMP' in the title, and/or by calling '*cpptraj* —defines' and looking for '-D.OPENMP'. The following actions/analyses have been OpenMP parallelized:

```

2drms/rms2d
atomiccorr
calcdiffusion
checkstructure
closest
cluster (pair-wise distance calculation and sieved frame restore only)
diffusion
dssp/secstruct
energy
gist (non-bonded calculation)
hbond
kde
lipidscd
mask (distance-based masks only)
matrix (coordinate covariance matrices only)
minimage
radial
replicatecell
rotdif
rmsavgcorr
spam
surf
tordiff
unwrap
velocityautocorr
volmap
watershell
wavelet

```

By default OpenMP *cpptraj* will use all available cores. The number of OpenMP threads can be controlled by setting the OMP_NUM_THREADS environment variable.

37.2.7.3. CUDA Parallelization

Some time-consuming actions in *cpptraj* have been parallelized with CUDA to take advantage of machines with NVIDIA GPUs. In order to use CUDA parallelization Amber/*cpptraj* should be configured with the `'-cuda'` flag. You can easily tell if *cpptraj* has been compiled with CUDA as it will print 'CUDA' and details on the current graphics device in the title, and/or by calling `'cpptraj --defines'` and looking for `'-DCUDA'`. The following actions have been CUDA parallelized:

```

closest
watershell
gist
radial

```

37.3. General Concepts

37.3.1. Units

Cpptraj uses the AKMA system of units. The exception is time, which is typically expressed in ps (except where noted).

Variable	Unit
Length	Angstrom
Energy	kcal/mol
Mass	AMU
Charge	electron
Time	ps (typically)
Force	kcal/mol*Angstrom

37.3.2. Atom Mask Selection Syntax

The mask syntax is similar to *ptraj*. Note that the characters ':', '@', and '*' are reserved for masks and should not be used in output file or data set names. All masks are case-sensitive. Either names or numbers can be used. Masks can contain ranges (denoted with '-') and comma separated lists. The logical operands '&' (and), '|' (or), and '!' (not) are also supported.

The syntax for elementary selections is the following:

@{atom numlist} e.g. '@12,17', '@54-85', '@12,54-85,90'

@{atom namelist} e.g. '@CA', '@CA,C,O,N,H'

@%{atom type name} e.g. '@%CT'

@/{atom element name} e.g. '@/N'

:{residue numlist} e.g. ':1-10', ':1,3,5', ':1-3,5,7-9'

:{residue namelist} e.g. ':LYS', ':ARG,ALA,GLY'

::{chain id} e.g. '::B', '::A,D'. Requires chain ID information be present in the topology.

::{pdb residue number} e.g. '::2-4,8'. Requires a PDB loaded as topology, or Amber topology with embedded PDB information (see [15.2.2.5 on page 286](#)).

^{molecule numlist} e.g. '^1-10', ':23,84,111'

<mask><distance operator><distance> Selection by distance, see below.

Several wildcard characters are supported:

****** Zero or more characters.

'=' Same as '**'

'?' One character.

The wildcards can also be used with numbers or other mask characters, e.g. ':?0' means ":10,20,30,40,50,60,70,80,90", ':*' means all residues and '@*' means all atoms. If the atom name (or type name) contains a wildcard character like an asterisk, it can be explicitly selected by escaping (i.e. preceding) the wildcard character with a backslash '\'. So for example:

atoms @C?*

would select atoms named C5, C4*, C422, etc., but:

atoms @C?*

would only select C4* out of the above 3 atoms.

Compound expressions of the following type are allowed:

```
:{residue numlist | namelist}@{atom namelist | numlist}
```

and are processed as:

```
:{residue numlist | namelist} & @{atom namelist | numlist}
```

e.g. ':1-10@CA' is equivalent to ":1-10 & @CA".

More examples:

:ALA,TRP All alanine and tryptophan residues.

:5,10@CA CA carbon in residues 5 and 10.

:*&!@H= All non-hydrogen atoms (equivalent to "!@H=").

@CA,C,O,N,H All backbone atoms.

!@CA,C,O,N,H All non-backbone atoms (=sidechains for proteins only).

:1-500@O&!(:WAT|:LYS,ARG) All backbone oxygens in residues 1-500 but not in water, lysine or arginine residues.

^1-2:ASP All residues named 'ASP' in the first two molecules.

::A,D@CA All atoms named 'CA' in chains A and D.

Distance-based Masks

<mask><distance operator><distance>

<mask> Atoms to consider.

<distance operator> Distance operator. {<|>}{@|:|;|^}

< Distances less than <distance> will be selected.

> Distances greater than or equal to <distance> will be selected.

@ Any atom.

: Any atom within a residue.

; Residue geometric center.

^ Any atom within a molecule.

<distance> The distance criteria in Angstroms.

There are two very important things to keep in mind when using distance based masks:

1. Distance-based masks that update each frame are currently only supported by the *mask* action.
2. Selection by distance for everything but the *mask* action requires defining a reference frame with *reference*; distances are then calculated using the specified reference frame only. This reference frame can be changed using the *activeref* command.

The syntax for selection by distance is a **<mask>** expression followed by a **<distance operator>** followed by a **<distance>** (which is in Angstroms). The **<distance operator>** consists of 2 characters: '<' (within) or '>' (without) followed by either '^' (molecules), ':' (residues), ';' (residue centers), or '@' (atoms). For example, '<:3.0' means "residues within 3.0 Angstroms" etc. For ':' residue- and '^' molecule-based distance selection, if any atom in that residue/molecule meets the given distance criterion, the entire residue/molecule is selected. For ';' residue center, the geometric center of the residue must meet the given distance criterion in order to be selected.

In plain language, the entire distance mask can be read as "Select **<distance operator>** **<distance>** of **<mask>**". So for example, the mask expression:

37. *cpptraj*

```
:11-17<@2.4
```

Means “Select atoms within 2.4 Å distance of atoms selected by ‘:11-17’ (residues numbered 11 through 17)”. To strip everything outside 3.0 Å (i.e. without 3.0 Å) from residue 4 using specified reference coordinates:

```
reference mol.rst7
trajin mol.rst7
strip !(:4<:3.0)
```

37.3.3. Ranges

For several commands some arguments are ranges (e.g. ‘trajout onlyframes <range>’, ‘nastruct resrange <range>’, ‘rmsd perres range <range>’); **THESE ARE NOT ATOM MASKS**. They are simple number ranges using ‘-’ to specify a range and ‘,’ to separate different ranges. For example 1-2,4-6,9 specifies 1 to 2, 4 to 6, and 9, i.e. ‘1 2 4 5 6 9’.

37.3.4. Parameter/Reference Tagging

Parameter and reference files may be ‘tagged’ (i.e. given a nickname); these tags can then be used in place of the file name itself. A tag in *cpptraj* is recognized by being bounded by brackets (‘[’ and ‘]’). This can be particularly useful when reading in many parameter or reference files. For example, when reading in multiple reference structures:

```
trajin Test1.crd
reference 1LE1.NoWater.Xray.rst7 [xray]
reference Test1.crd lastframe [last]
reference Test2.crd 225 [open]
rms Xray ref [xray] :2-12@CA out rmsd.dat
rms Last ref [last] :2-12@CA out rmsd.dat
rms Open ref [open] :2-12@CA out rmsd.dat
```

This defines three reference structures and gives them tags [xray], [last], and [open]. These reference structures can then be referred to by their tags instead of their filenames by any action that uses reference structures (in this case the RMSD action).

Similarly, this can be useful when reading in multiple parameter files:

```
parm tz2.ff99sb.tip3p.truncoc.t.parm7 [tz2-water]
parm tz2.ff99sb.mbondi2.parm7 [tz2-nowater]
trajin tz2.run1.explicit.nc parm [tz2-water]
reference tz2.dry.rst7 parm [tz2-nowater] [tz2]
rms ref [tz2] !(:WAT) out rmsd.dat
```

This defines two parm files and gives them tags [tz2-water] and [tz2-nowater], then reads in a trajectory associated with one, and a reference structure associated with the other. Note that in the ‘reference’ command there are two tags; the first goes along with the ‘parm’ keyword and specifies what parameter file the reference should use, the second is the tag given to the reference itself (as in the previous example) and is referred to in the subsequent RMSD action.

37.4. Variables and Control Structures

As of version 18, CPPTRAJ has limited support for “script” variables and ‘for’ loops. Script variables are referred to by a dollar sign (\$) prefix and are replaced when they are processed. These are stored in the master data set list like other data and are assigned the type “string variable”. **Note that to use script variables in CPPTRAJ input that is inside another script (e.g. a BASH script), they must be escaped with the ‘\’ character, e.g.**

```
#!/bin/bash
TOP=MyTop.parm7
cpptraj <<EOF
set topname=$TOP # TOP is a BASH script variable
parm \ $topname  # topname is a CPPTRAJ script variable
EOF
```

Note that regular CPPTRAJ 1D Data Sets that contain a single value can be used as script variables (if the Data Set contains more than 1 value only the first value will be used).

Command	Description
for	Create a 'for' loop.
set	Set or update a script variable.
show	Show all current script variables and their values.

37.4.1. for

```
for { {atoms|residues|molecules|molfirstres|mollastres}
  <var> inmask <mask> [parm <name> | parmindex <#> | <#>] |
  <var> in <list> |
  <var> indata <data set name> |
  <var> oversets <list> |
  <var> datasetblocks <set> blocksize <#> [blockoffset <#>]
    [cumulative [firstblock <#>]] |
  <var>=<start>;[<var><end OP><end>;]<var><increment OP>[<value>] ... }
END KEYWORD: 'done'
```

Available 'end OP' : '<' '>' '<=' '>='

Available 'increment OP' : '++', '--', '+=', '-='

atoms|residues|molecules|molfirstres|mollastres <var> inmask <mask> Loop over atoms/residues/molecules/first residue in molecules/last residue in molecules selected by the given mask expression, set as script variable <var>.

parm <name> | parmindex <#> <#> Select topology that <mask> should be based on (default first topology).

<var> in <list> Loop over a comma-separated list of strings. File name wildcards can be used.

<var> in <data set name> Loop over elements of specified data set. Currently only 1D scalar sets and string sets can be specified.

<var> oversets <list> Loop over sets selected by comma-separated list of names. Data set wildcards can be used.

<var> datasetblocks <set> Loop over blocks in specified DataSet.

blocksize <#> Size of blocks to use.

[blockoffset <#>] Offset between blocks.

[cumulative] Instead of blocks of fixed size, use blocks of increasing size incremented by blocksize.

[firstblock <#>] When cumulative, the size of the first block (default is first data set element).

<var>=<start>;[<var><end OP><end>;]<var><increment OP>[<value>] Loop over integer script variable <var> starting from <start>, optionally ending at <end>, increment by <value>.

Data Sets Created (datasetblocks loops):

<var>[block]:<start idx> (Data set blocks only) Data set block of blocksize starting at <start idx>.

<var>[cumul]:<end idx> (Cumulative data set blocks only) Data set block starting at firstblock and ending at <end idx>.

Create a for loop using one or more mask expressions, integers, etc. Loops can be nested inside each other. Integer loops may be used without an end condition, but in that case at least one descriptor in the loop should have an end condition or refer to a mask. Loops are ended by the **done** keyword.

Note that non-integer variables (e.g. 'inmask' loops) are NOT incremented after the final loop iteration, i.e. these loop variables always retain their final value.

For example:

```
for atoms A0 inmask :1-3@CA i=1;i++
  distance d$i :TCS $A0 out $i.dat
done
```

This loops over all atoms in the mask expression ':1-3@CA' (all atoms named CA in residues 1 to 3) and creates a variable named 'i' that starts from 1 and is incremented by 1 each iteration. Inside the loop, the mask selection is referred to by \$A0 and the integer by \$i. This is equivalent to doing 3 distance commands like so:

```
distance d1 :TCS :1@CA out 1.dat
distance d2 :TCS :2@CA out 2.dat
distance d3 :TCS :3@CA out 3.dat
```

To loop over files named trajA*.nc and trajB*.nc:

```
for TRAJ in trajA*.nc,trajB*.nc
  trajin $TRAJ 1 last 10
done
```

37.4.2. set

```
set { <variable> <OP> <value> |
      <variable> <OP> {atoms|residues|molecules|atomnums|
                        resnums|oresnums|molnums|
                        charge|mass} inmask <mask>
      [parm <name> | crdset <set> | parmindex <#> | <#>]
      <variable> <OP> trajinframes }
      Available <OP> : '=' , '+='
```

<variable> <OP> <value> Set or append a script variable.

<variable> <OP> {atoms|residues|molecules|atomnums|resnums

|oresnums|molnums} inmask <mask> Set/append a script variable to/by the total number of atoms/residues/molecules in, a range expression of selected atom #s/residue #s/original residue #s/molecule #s in, or the total charge/mass of atoms selected by the given mask expression.

parm <name> | parmindex <#> | <#> Topology to which mask should correspond (default first).

<variable> <OP> trajinframes Set/append a script variable to/by the total number of frames in trajectories currently loaded by *trajin* commands.

Set (<OP> = '=') or append (<OP> = '+=') a script variable. Script variables are character strings, and are referred to in CPPTRAJ input by using a dollar sign '\$' prefix.

For example, the following input will load files my.parm7 and my.rst7:

```
set PREFIX = my
trajin $PREFIX.parm7
trajin $PREFIX.rst7
```

For example, the following input will print info for the last 10 atoms in a topology to 'last10.dat':

```
set Natom = atoms inmask *
last10 = $Natom - 10
show
atoms "@$last10 - $Natom" out last10.dat
```

The following input will put a range of residues selected by :LYS:

```
> set SELECTED1 = resnums inmask :1-183&:LYS
Using topology: FtuFabI.NAD.TCL.parm7
Variable 'SELECTED1' set to '7-8,18,26,44,49,71,79,128,135,151,163,183'
```

37.4.3. show

```
show [<var1> ...]
```

If no variable names specified, show all current script variables and their values. Otherwise, show the values of the specified script variables.

37.5. Data Sets and Data Files

In *cpptraj*, Actions and Analyses can generate one or more data sets which are available for further processing. For example, the *distance* command creates a data set containing distances vs time. The data set can be named by the user simply by specifying a non-keyword string as an additional argument. If no name is given, a default one will be generated based on the action name and data set number. For example:

```
distance d1-2 :1 :2 out d1-2.dat
```

will create a data set named "d1-2". If a name is not specified, e.g.:

```
distance :1 :2 out d1-2.dat
```

the data set will be named "Dis_00000".

Data files are created automatically by most commands, usually via the "out" keyword. Data files can also be explicitly created with the *write/writedata* and *create* commands. Data can also be read in from files via the *readdata* command. Cpptraj currently recognizes the formats listed in 37.1, although it cannot write in all formats. In addition, a data set must be valid for the data file format. For example, 3D data (such as a grid) can be written to an OpenDX format file but not a Grace format file.

The default file format is called 'Standard', which simply has data in columns, like *ptraj*, although multiple data sets can be directed to the same output file. The format of a file can be changed either by specifying a recognized keyword (either on the command line itself or later via a 'datafile' command) or by giving the file an extension corresponding to the format, so 'filename.agr' will output in Grace format, and 'filename.gnu' will output in Gnuplot contour, and so on. The xmgrace/gnuplot output is particularly nice for the secstruct sumout and rmsd perresout files. Additional options for data files can be found in 37.6 on page 697.

Any action using the "out" keyword will allow data sets from separate commands to be written into the same file. For example, the commands:

```
dihedral phi :1@C :2@N :2@CA :2@C out phipsi.dat
dihedral psi :2@N :2@CA :2@C :3@N out phipsi.dat
```

will assign the “phi” and “psi” data sets generated from each action to the standard data output file “phipsi.dat”:

```
#Frame phi psi
```

Note that when reading the Amber Prep and Amber OFF Library formats, a COORDS data set will be created for each unit present in these files.

37.5.1. Data Set Selection Syntax

Many analysis commands can be used to analyze multiple data sets. The general format for selecting data sets is:

```
<name> [<aspect>] : <index>
```

The ‘*’ character can be used as a wild-card for *entire* names (no partial matches).

- **<name>**: The data set name, usually specified in the action (e.g. in ‘distance d0 @1 @2’ the data set name is “d0”).
- **<aspect>**: Optional; this is set for certain data sets internally in order to easily select subsets of data. **The brackets are required.** For example, when using ‘hbond series’, both solute-solute and solute-solvent hydrogen bond time series may be generated. To select all solute-solute hydrogen bonds one would use the aspect “[solutehb]”; to select solute-solvent hydrogen bonds the aspect “[solventhb]” would be used. Aspects are hard-coded and are listed in the commands that use them.
- **<index>**: Optional; for actions that generate many data sets (such as ‘rmsd perres’) an index is used. Depending on the action, the index may correspond to atom #s, residue #s, etc. A number range (comma and/or dash separated) may be used.

For example: to select all data sets with aspect “[shear]” named NA_00000:

```
NA_00000 [shear]
```

To select all data sets with aspect “[stagger]” with any name, indices 1 and 3:

```
*[stagger] :1,3
```

In ensemble mode, data set selection has additional syntax:

```
<name> [<aspect>] : <index>%<member>
```

Where <member> is the ensemble member number starting from 0.

37.5.2. Data Set Math

As of version 15, *cpptraj* can perform basic math operations, even on data sets (with some limitations). Currently recognized operations are:

Operation	Symbol
Minus	-
Plus	+
Divide	/
Multiply	*
Power	^
Negate	-
Assign	=

Format	Filename Extensions	Keyword	Valid Dimensions	Notes
Standard	.dat	dat	1D, 2D, 3D	
Grace	.agr, .xmgr	grace	1D	
Gnuplot	.gnu	gnu	1D, 2D	
Xplor	.xplor, .grid	xplor	3D	
OpenDX	.dx	opendx	3D	
Amber REM log	.log	remlog	-	Read Only
Amber MDOUT	.mdout	mdout	-	Energy information, Read Only
Amber Energy File	.ene	amberene	1D	Read Only
Amber Evecs	.evecs	evecs	Modes data set only	
Amber Constant pH output	.cpout	cpout	pH data only	
Density Peaks	.peaks	peaks	3D density peaks (spam/volmap)	
Vector pseudo-traj	.vectraj	vectraj	Vector data set only.	Write Only
Gromacs XVG	.xvg	xvg	-	Read Only
CCP4	.ccp4	ccp4	3D	
Charmm REPD log	.exch	charmmrepd	-	Read Only
Charmm Output	.charmmout	charmmout	-	Energy information, Read Only
Pairwise Cache (binary)	.cmatrix	cmatrix	pairwise distances	Used for cluster analysis.
Pairwise Cache (NetCDF)	.nccmatrix	nccmatrix	pairwise distances	Used for cluster analysis.
NetCDF Data	.nc	netcdf	All data	Only state info saved for pH data.
Amber Prep File	.prepin	prepin	COORDS	Read Only
Amber OFF Library File	.off, .lib	off,lib	COORDS	Read Only

Table 37.1.: *DataFile formats recognized by cpptraj. 'Valid Dimensions' shows what dimensions the format is valid for (e.g. you cannot write a 1D data set with OpenDX format).*

Several functions are also supported:

Function	Form
Square Root	<code>sqrt()</code>
Exponential	<code>exp()</code>
Natural Logarithm	<code>ln()</code>
Absolute Value	<code>abs()</code>
Sine	<code>sin()</code>
Cosine	<code>cos()</code>
Tangent	<code>tan()</code>
Summation	<code>sum()</code>
Average	<code>avg()</code>
Standard Deviation	<code>stdev()</code>
Minimum	<code>min()</code>
Maximum	<code>max()</code>

Numbers can be expressed in scientific notation using “E” notation, e.g. $1\text{E-}5 = 0.00001$. The parser also recognizes PI as the number pi. Expressions can also be enclosed in parentheses. So for example, the following expression is valid:

```
> 1 - ln(sin(PI/4) * 2)^2
Result: 0.879887
```

Results of numerical calculations like the above can be assigned to a variable (essentially a data set of size 1) for use in subsequent calculations, e.g.

```
> R = 1 - ln(sin(PI/4) * 2)^2
Result stored in 'R'
> R + 1 Result: 1.879887
```

Data sets can be specified in expressions as well. Currently data sets in an expression must be of the same type and only 1D, 2D, and 3D data sets are supported. Functions are applied to each member of the data set. So for example, given two 1D data sets of the same size named D0 and D1, the following expression:

```
> D2 = sqrt( D0 ) + D1
```

would take the square root of each member of D0, add it to the corresponding member of D1, and assign the result to D2. The following table lists which operations are valid for data set types. If a type is not listed it is not supported:

Data Set Type	Supported Ops	Supported Funcs	Notes
1D (integer, double, float)	All	All	
1D (vector)	+, -, *, /, =	None	'*' is dot product
2D (matrices)	+, -, /, *, =	sum, avg, stdev, min, max	
3D (grids)	+, -, /, *, =	sum, avg, stdev, min, max	

37.6. Data File Options

Data file output can be handled multiple ways in *cpptraj*. Output data files can be created by Actions/Analyses/Commands, or can be explicitly created with *writedata* (37.8.32 on page 730) or *create* (37.8.4 on page 718) commands. Reading data from files is only done via the *readdata* command (37.8.22 on page 727).

In general, data files which have been declared with an 'out' keyword will recognize data file write keywords on the same command line. For example, the 'time' argument can be passed directly to the output from a *distance* command:

```
distance d0 :1 :2 out d0.agr time 0.001
```

The data file format can be changed from standard implicitly by using specific filename extensions or keywords. If the extension is not recognized or no keyword is given the default format is 'Standard'. Keywords and extensions for data file formats recognized by *cpptraj* are shown in 37.1. Note that the use of certain options may be restricted for certain data file formats. These options can also be passed to data files via the *datafile* command (37.8.6 on page 718).

```
[<format keyword>]
[{xlabel|ylabel|zlabel} <label>] [{xmin|ymin|zmin} <min>] [sort]
[{xstep|ystep|zstep} <step>] [time <dt>] [prec <width>[.<precision>]]
[xprec <width>[.<precision>]] [xfmt {double|scientific|general}]
[noensextension]
```

{xlabel | ylabel | zlabel} <label> Set the x-axis label for the specified datafile to <label>. For regular data files this is the header for the first column of data. If the data is at least 2-dimensional 'datafile ylabel <label>' will likewise set the y-axis label.

{xmin | ymin | zmin} <min> Set the starting X coordinate value to <min>. If the data is at least 2-dimensional 'datafile ymin <min>' will likewise set the starting Y coordinate value.

sort Sort data sets prior to write. Ordering is by name, aspect, then index (all descending).

{xstep | ystep | zstep} <step> Multiply each frame number by <step> (x coordinates). If the data is at least 2-dimensional 'datafile ystep <step>' will likewise multiply y coordinates by <step>.

time <dt> Equivalent to the *ptraj* argument 'time' that could be specified with many actions. Multiplies frame numbers (x-axis) by <dt>.

prec <width>[.<precision>] Change the output format width (and optionally precision) of all sets subsequently added to the data file (i.e. does not change the precision of any data sets currently in the file). For example,

```
prec 12.4
prec 10
```

xprec <width>[.<precision>] Change output ordinate width and precision.

xfmt {double|scientific|general} Change output ordinate format.

[noensextension] Omit ensemble extension in ensemble processing mode.

NOTE: THIS OPTION HAS NOT BEEN FULLY TESTED IN PARALLEL.

37.6.1. Standard Data File Options

Write

```
[invert] [noxcol] [groupby <type>] [noheader] [square2d|nosquare2d]
[nosparsed|sparse [cut <cutoff>]]
```

invert Normally, data is written out with X-values pertaining to frames (i.e. data over all trajectories is printed in columns). This command flips that behavior so that X-values pertain to data sets (i.e. data over all trajectories is printed in rows).

groupby <type> (1D) group data sets by <type>:

name Group by name.

aspect Group by aspect.

idx Group by index.

ens Group by ensemble number.

dim Group by dimension.

xcoll Write indices for the specified datafile. This is usually the default behavior.

noxcol Prevent printing of indices (i.e. the #Frame column in most datafiles) for the specified datafile. Useful e.g. if one would like a 2D plot such as phi vs psi. For example, given the input:

```
dihedral phi :1@C :2@N :2@CA :2@C out phipsi.dat
dihedral psi :2@N :2@CA :2@C :3@N out phipsi.dat
datafile phipsi.dat noxcol
```

Cpptraj will write a 2 column datafile containing only phi and psi, no frame numbers will be written.

header Write header line at beginning of data file. This is usually the default behavior.

noheader Prevent printing of header line (e.g. '#Frame D1') at the beginning of data file.

square2d Write 2D data as a square matrix, e.g.:

```
<1,1> <2,1> <3,1>
<1,2> <2,2> <3,2>
```

nosquare2d Write 2D data in 3 columns as:

```
<X> <Y> <Value>
```

sparse Only write 3D grid voxels with value > cutoff (default 0).

cut<cut> Cutoff for 'sparse'; default 0.

nosparsed Write all 3D voxels (default).

Read

```
[prec {flt|dbl}]
{[read1d [index <col>] [onlycols <range>] [floatcols <range>]
 [intcols <range>] [stringcols <range>]] |
[read2d [{square2d|nosquare2d}]] |
```

```

[vector [magnitude]] |
[mat3x3] |
[read3d [dims <nx>,<ny>,<nz>] [origin <ox>,<oy>,<oz>]
      [delta <dx>,<dy>,<dz>] [prec {dbl|flt}] [bin {center|corner} ]
]
}

prec {flt|dbl} Read 2d/3d data as single (flt) or double (dbl, default)
precision.

read1d Read data as 1D data sets (default).
  index <col> Use column <col> (starting from 1) as index column
  (1D data only).
  onlycols <range> Only read columns in range.
  floatcols <range> Force specified columns to be read as
  single-precision floats.
  intcols <range> Force specified columns to be read as integers.
  stringcols <range> Force specified columns to be read as strings.

read2d Read data as 2D matrix.
  square2d Read data as square matrix (default).
  nosquare2d Read data as XYZ matrix (i.e. each line contains
  '<column> <row> <data>').

vector Read data as vector. If indices are present they will be
skipped. Assume first 3 columns after the index column are
vector X, Y, and Z, and (if present) the next 3 columns contain
vector origin X, Y, and Z.
  magnitude If specified, assume vector file last column contains
  vector magnitudes; read these into a set with Aspect 'Mag'.

mat3x3 Read data as 3x3 matrix. If indices are present they will
be skipped. Assume matrices are in row major order on each
line, i.e. M(1,1) M(1,2) ... M(3,2) M(3,3).

read3d Read data as 3D grid. If no dimension data in file must also
specify 'dims'.
  dims <dx>,<dy>,<dz> Grid dimensions.
  origin <ox>,<oy>,<oz> Grid origins (default 0,0,0).
  delta <dx>,<dy>,<dz> Grid spacings (default 1,1,1).
  prec {dbl|flt} Grid precision, double or float (default float).
  bin {center|corner} Coords specify bin centers or corners (default
  corners).

```

By default, standard data files are assumed to contain 1D data in columns. Data set legends will be read in if the file has a header line (denoted by '#'). Columns labeled '#Frame' are automatically considered the 'index' column and skipped. Data sets are stored as <name>:<idx> where <name> is the given data set name (the file name if not specified) and <idx> corresponds to the column the data was read from starting from 1. *Cpptraj* assumes the data increases monotonically and will automatically attempt to determine the dimensions of the data set(s); a warning will be printed if this is not successful.

If a file contains the header:

```
#F1 F2 <name>
```

CPPTRAJ will assume the file contains pairwise distances for clustering, where the F1 and F2 columns contain the frame numbers, and the <name> column contains the distance.

37.6.2. Grace Data File Options

For more information on Grace see <http://plasma-gate.weizmann.ac.il/Grace/>.

Write

```
[{invert|noinvert}] [{xydy|noxydy}] [<label set>]
```

invert Normally, data is written out with X-values pertaining to frames (i.e. data over all trajectories is printed in columns). This command flips that behavior so that X-values pertain to data sets.

noinvert Do not flip X-Y axes (default).

xydy Combine consecutive pairs of sets into XYDY sets.

noxydy Do not combine consecutive pairs of sets into XYDY sets (default).

<label set> If a string dataset is specified, assume it has data point labels.

If a single string data set is specified when writing Grace format, it is assumed they are data point labels.

Read

Cpptraj will read set legends from grace files, and data sets are stored as <name>:<idx> where <name> is the given data set name (the file name if not specified) and <idx> corresponds to the set number the data was read from starting from 0.

37.6.3. Gnuplot Data File Options

For more information on these options it helps to look at the PM3D options in the Gnuplot manual (see <http://www.gnuplot.info/>).

Write

```
[{nolabels|labels}] [{usemap|pm3d|nopm3d}] [title <title>]
[jpeg] [noheader] [{xlabels|ylabels|zlabels} <labellist>]
```

nolabels Do not print axis labels.

labels Print axis labels.

usemap pm3d output with 1 extra empty row/col (may improve look).

pm3d Normal pm3d map output.

nopm3d Turn off pm3d

jpeg Plot will write to a JPEG file when used with gnuplot.

title <title> Set plot title (default is file name).

binary Plot will be written in binary format.

header Format the plot so it can be directly processed by gnuplot.
This is usually the default behavior.

noheader Do not format plot; data output only.

palette <arg> Change gnuplot pm3d palette to <arg>:

'rgb' Red, yellow, green, cyan, blue, magenta, red.

'kbvwy' Black, blue, violet, yellow, white.

'bgyr' Blue, green, yellow, red.

'gray' Grayscale.

xlabels|ylabels|zlabels <labellist> Set x, y, or z axis labels with comma-separated list, e.g. **'xlabels X1,X2,X3'**.

37.6.4. Amber REM Log Options

Note that multiple REM logs can be specified in a single *readdata* command. See [37.12.30 on page 880](#) for more on replica log analysis.

Read

[nosearch] [dimfile <file>] [crdidx <crd indices>]

[nosearch] If specified do not automatically search for MREMD dimension logs.

[dimfile <file>] remd.dim file for processing MREMD logs.

[crdidx <crd indices>] Use comma-separated list of indices as the initial coordinate indices (H-REMD only). For example (4 replicas):

crdidx 4,2,3,1

37.6.5. Amber MDOUT Options

Note that multiple MDOUT files can be specified in a single *readdata* command.

37.6.6. Evecs File Options

Read

[ibeg <firstmode>] [iend <lastmode>]

ibeg <firstmode> Number of the first mode (or principal component) to read from evecs file. Default 1.

iend <lastmode> Number of the last mode (or principal component) to read from evecs file. Default is to read all for newer evecs files (generated by *cpptraj* version > 12), 50 for older evecs files.

37.6.7. Vector psuedo-traj Options

This can be used to write out a representation of a vector data set which can then be visualized. See [37.11.91 on page 840](#) for more on generating vector data sets.

Write

[trajfmt <format>] [parmout <file>] [noorigin]

trajfmt <format> Output pseudo-trajectory format. See [37.10 on page 742](#) for trajectory format keywords.

parmout <file> File to write pseudo-trajectory topology to.

[noorigin] Do not write vector origin coordinates.

37.6.8. OpenDX file options

Read

`[type {float|double}]`
`type {float|double}` Precision to read in 3D grid (default float).

Write

`[bincenter] [gridwrap] [gridext]`
bincenter Center grid points on bin centers instead of corners.
gridwrap Like 'bincenter', but also wrap grid density. Useful when grid encompasses unit cell.
gridext Like 'bincenter', but also print extra layer of empty bins.

37.6.9. CCP4 file options

Write

`[title <title>]`
`[title <title>]` Set CCP4 output title.

37.6.10. Charmm REPD log options

Read

`[nrep <#>] [crdidx <crd indices>]`
nrep <#> Total number of replicas.
crdidx <crd indices> Comma-separated list of indices to use as initial coordinate indices.

37.6.11. Amber Constant pH Out options

Read

`cpin <file>`
cpin <file> Constant pH input (CPIN) file name.

Note that when reading in constant pH data the data set aspect will be set to the residue name and the index will be set to the residue number. When reading in constant pH REMD data the data is unsorted, and `sortensembldata` should be used to create sorted constant pH data sets (see [37.8.30 on page 729](#)).

37.7. Coordinates (COORDS) Data Set Commands

Coordinate I/O tends to be the most time-consuming part of trajectory analysis. In addition, many types of analyses (for example two-dimensional RMSD and cluster analysis) require using coordinate frames multiple times. To simplify this, trajectory coordinates may be saved as a separate data set via the *loadcrd* command or *createcrd* action. Any action can then be performed on the COORDS data set with the *crdaction* command. The *crdout* command can be used to write coordinates to an output trajectory (similar to *trajout*).

Although COORDS data sets store everything internally with single-precision, they can still use a large amount of memory. Because of this there is a specialized type of COORDS data set called a TRAJ data set (trajectory),

which functions exactly like a COORDS data set except all data is stored on disk. TRAJ data sets can be created with the **loadtraj** command. *TRAJ data sets cannot be modified.*

There are several analyses that can be performed using COORDS data sets, either as part of the normal analysis list or via the **runanalysis** command. Note that while these analyses can be run on specified COORDS data sets, if one is not specified a default COORDS data set will be created, made up of frames from **trajin** commands.

As an example of where this might be useful is in the calculation of atomic positional fluctuations. Previously this required two steps: one to generate an average structure, then a second to rms-fit to that average structure prior to calculating the fluctuations. This can now be done in one pass with the following input:

```
parm topology.parm7
loadcrd mdcrd.nc
# Generate average structure PDB, @CA only
crdaction mdcrd.nc average avg.pdb @CA
# Load average structure PDB as reference
parm avg.pdb
reference avg.pdb parm avg.pdb
# RMS-fit to average structure PDB
crdaction mdcrd.nc rms reference @CA
# Calculate atomic fluctuations for @CA only
crdaction mdcrd.nc atomicfluct out fluct.dat bfactor @CA
```

The following COORDS data set commands are available:

Command	Description
catcrd	Concatenate two or more COORDS sets.
combinecrd	Combine two or more COORDS sets.
crdaction	Run a single Action on a COORDS set.
crdout	Write a COORDS set to a file.
crdtransform	Transform a COORDS set in one of several ways.
createcrd	(Action) Create a COORDS set during a Run.
emin	Run simple energy minimization on a frame of a COORDS set.
extendedcomp	Calculate extended comparison similarity values for each frame in COORDS set.
graft	Graft part of one COORDS set onto another COORDS set.
loadcrd	Create or append to a COORDS set from a file.
loadtraj	Create special COORDS set where frames remain on disk.
permutedihedrals	Rotate specified dihedral(s) in given COORDS set by specific interval or to random values.
prepareforleap	Prepare a structure (usually loaded from a PDB) for processing with LEaP from Amber.
reference	Load a single trajectory frame as a reference.
rotatedihedral	Rotate specified dihedral to specified value or by given increment.
sequence	Create a new molecule from a sequence of COORDS sets.
splitcoords	Split molecules in a COORDS set into a trajectory.
zmatrix	Apply Z-matrix to a COORDS set or calculate Z-matrix for a molecule/frame in a COORDS set.

37.7.1. catcrd

```
catcrd <crd1> <crd2> ... name <name>
<crdX> COORDS data sets to concatenate, specify 2 or more.
name <name> New COORDS set name
```

Concatenate two or more COORDS data sets into a single COORDS data set. The topologies must have the same number of atoms for this to work. If the topologies differ in other ways, the topology of the first COORDS set takes priority.

37.7.2. combinecrd

```
combinecrd <crd1> <crd2> ... [parmname <topname>] [crdname <crdname>]
<crdX> COORDS data sets to combine, specify 2 or more.
[parmname <topname>] Name of combined Topology.
[crdname <crdname>] Name of combined COORDS data set.
```

Combined two or more COORDS data sets into a single COORDS data set. Note that the resulting topology will most likely **not** be usable for MD simulations. Box information will be retained - the largest box dimensions will be used.

For example, to load two MOL2 files as COORDS data sets, combine them, and write them out as a single MOL2:

```
loadcrd Tyr.mol2 CRD1
loadcrd Pry.mol2 CRD2
combinedcrd CRD1 CRD2 parmname Parm-1-2 crdname CRD-1-2
crdout CRD-1-2 Tyr.Pry.mol2
```

37.7.3. crdaction

```
crdaction <crd set> <actioncmd> [<action args>] [crdframes <start>,<stop>,<offset>]
```

Perform action <actioncmd> on COORDS data set <crd set>. A subset of frames in the COORDS data set can be specified with 'crdframes'.

For example, to calculate RMSD for a previously created COORDS data set named crd1 using frames 1 to the last, skipping every 10:

```
crdaction crd1 rmsd first @CA out rmsd-ca.agr crdframes 1,last,10
```

37.7.4. crdout

```
crdout <crd set> <filename> [<trajout args>] [crdframes <start>,<stop>,<offset>]
```

Write COORDS data set <crd set> to trajectory named <filename>. A subset of frames in the COORDS data set can be specified with 'crdframes'.

For example, to write frames 1 to 10 from a previously created COORDS data set named "crd1" to separate PDB files:

```
crdout crd1 crd1.pdb multi crdframes 1,10
```

37.7.5. crdtransform

```
crdtransform <input crd set> [name <output crd set>]
{ rmsrefine [mask <mask>] [mass] [rmstol <tolerance>] |
  normcoords |
  trim [metric <metric>] [{ntrimmed <#>|cutoff <val>}]
  [criterion {comp|medoid}]]
}
```

<input crd set> COORDS set to transform.

[name <output crd set>] COORDS set to create; if not specified <input crd set> will be modified.

rmsrefine Do iterative RMS refinement.

[mask <mask>] Mask of atoms to fit during refinement.

[mass] Mass-weight the refinement.

[rmstol <tolerance>] Tolerance (in Ang.) below which RMS-refinement will stop.

normcoords Normalize coordinates between 0.0 and 1.0 using the minimum and maximum coordinate values.

trim Remove trajectory frames using extended similarity metrics.
[metric <metric>] Metric to use; default MSD.
[{ntrimmed <#>|cutoff <val>}] # of frames or fraction of trajectory to trim.
[criterion {comp|medoid}] Trim frames by comparative similarity (i.e. trim most dissimilar frames) or comparison to medoid (i.e. trim most dissimilar to medoid frame).

Transform a COORDS set in one of several ways. Does not yet work with TRAJ data sets. The iterative RMS refinement is similar to the procedure outlined by Klem et al. (J. Chem. Theory Comput. 2022, 18, 3218–3230). The extended similarity metrics are those defined by Racz et al. (J. Comp.-Aid. Mol. Design, 2022, 36, 157-173).

37.7.6. createcrd

This command is actually an Action that can be used to create COORDS data sets during trajectory processing, see [37.11.21 on page 769](#).

37.7.7. emin

emin crdset <name> [trajoutname <name>] [rmstol <tol>] [nsteps <#>] [<mask>] [frame <#>] [dx0 <step0>] [out <file>] [name <setname>] [{nonbond|openmm}] [<potential options>]
crdset <name> COORDS set to use.
[trajoutname <name>] Optional output trajectory for minimization steps.
[rmstol <tol>] Minimum RMS tolerance (default 1E-4).
[nsteps <#>] Number of minimization steps (default 1).
[<mask>] Atoms to minimize (default all).
[frame <#>] Frame from COORDS set to minimize (default 1).
[dx0 <step0>] Size of initial minimization step (default 0.01).
[out <file>] File to write energies to.
[name <setname>] If specified, create an energy per step data set.
[nonbond] If specified, use simple nonbonded potential term in additon to bonded terms.
[openmm] If specified and if CPPTRAJ was compiled with OpenMM support, use OpenMM to calculate the forces.
<potential options>
cut <cutoff> Set nonbonded interaction cutoff in Ang. (electrostatics and vdW). Default 8.0.
cutee <cutoff> Set electrostatics interaction cutoff in Ang.
cutnb <cutoff> Set vdW interaction cutoff in Ang.
scaleee <factor> Scaling factor to multiply 1-4 electrostatics interactions by.
scaleb <factor> Scaling factor to multiply 1-4 vdW interactions by.
shake <type> Use SHAKE constraints of <type>:
 'hydrogen' Constrain bonds to hydrogen.

'all' Constrain all bonds.

nexclude <#> Number of bonded atoms within which nonbonded interactions are excluded. Default 4.

qfac <factor> Factor to use in electrostatic calculation.

DataSets Created

<setname>[Energy] Total energy at each minimization step.

THIS COMMAND IS STILL IN DEVELOPMENT AS OF VERSION 5.0.2.

Perform steepest descent minimization on a frame in a COORDS set using a very basic force field (bonds, angles, dihedrals). A simple nonbonded term can be added as well if desired.

37.7.8. extendedcomp

extendedcomp <input crd set> [name <output data set>]
[metric <metric>] [out <file>]
<metric> = msd bub fai gle ja jt rt rr sm ss1 ss2

<input crd set> Input COORDS set.

[name <output data set>] Output data set name containing values.

[metric <metric>] Metric to use.

[out ,file>] File to write values to.

DataSets generated:

<output set name> Set containing similarity values for each COORDS frame.

Calculate extended comparison similarity values for each frame in a COORDS set. The extended similarity metrics are those defined by Racz et al. (J. Comp.-Aid. Mol. Design, 2022, 36, 157-173). The metrics are as follows:

Keyword	Metric
msd	Mean-squared deviation.
bub	Bhattacharyya's U coefficient
fai	Faiman's coefficient
gle	Gleason's coefficient
ja	Jaccard's coefficient
jt	Jaccard-Tanimoto coefficient
rt	Rogers-Tanimoto coefficient
rr	Russell-Rao coefficient
sm	Simpson's coefficient
ss1	Sokal-Sneath 1 coefficient
ss2	Sokal-Sneath 2 coefficient

37.7.9. graft

graft src <source COORDS> [srcframe <#>] [srcmask <srcmask> [srccharge <srccharge>]
tgt <target COORDS> [tgtframe <#>] [tgtmask <tgtmask> [tgtcharge <tgtcharge>]]

```

    {ic | [srcfitmask <srcmask>] [tgtfitmask <tgtmask>]}
    name <output COORDS> [bond <tgt>,<src> ...]
src <source COORDS> Source coordinates.
[srcframe <#>] Frame # from source coordinates to use (default 1).
[srcmask <mask>] Atoms to keep from source (default all).
    [srccharge <charge>] If trimming atoms from source, ensure sum of
        charges on remaining atoms equals <charge> via scaling.
tgt <target COORDS> Target coordinates that will be grafted onto.
[tgtframe <#>] Frame # from target coordinates to use (default 1).
[tgtmask <mask>] Atoms to keep from target (default all).
    [tgtcharge <charge>] If trimming atoms from target, ensure sum of
        charges on remaining atoms equals <charge> via scaling.
[ic] Connect source and target using internal coordinates.
[srcfitmask <mask>] Atoms from source to use if RMS-fitting source
    onto target.
[tgtfitmask <mask>] Atoms from target to use if RMS-fitting source
    onto target.
name <output COORDS> Name of output COORDS set containing source
    grafted onto target.
[bond <tgt>,<src>] Create a bond between target atom selected by
    <tgt> and source atoms selected by <src> in the final
    structure. Must be specified only once if connecting via
    internal coordinates, otherwise may be specified multiple
    times.

```

Graft one COORDS set onto another. If **srcfitmask** and/or **tgtfitmask** is specified, the source coordinates will be RMS best-fit onto target using the specified atoms. Only the atoms specified by **srcmask** and **tgtmask** will be kept. The **bond** keyword can be used to create bonds between target and source in the final structure. If using internal coordinates to connect the units, exactly one bond must be specified.

37.7.10. loadcrd

```

loadcrd <filename> [parm <parm> | parmindex<#>] [<trajin args>] [name <name>]
    [prec {single|double}]
<filename> Trajectory file to load.
[parm <parmfile/tag>] Topology filename/tag to associate with
    trajectory (default first topology).
[parmindex <#>] Index of Topology to associate with trajectory
    (default 0, first topology).
[<trajin args>] Additional 'trajin' args; see 37.10.4 on page 745.
[name <name>] Name of the COORDS set.
[prec {single|double}] Load as either a single-precision COORDS set
    (the default) or a double-precision FRAMES set (which will use
    much more memory).

```


Immediately load trajectory <filename> as a COORDS data set named <name> (default base name of <filename>). If <name> is already present the coordinates will be appended to the existing data set.

For example, to load frames from trajectories named 'traj1.nc' and 'traj2.nc' into a COORDS data set named Crd1:

```
loadcrd traj1.nc name Crd1
loadcrd traj2.nc name Crd2
```

37.7.11. loadtraj

```
loadtraj name <setname> [<filename>]
name <setname> Name of the TRAJ set.
[<filename>] If specified, trajectory to add to the TRAJ set.
```

This command functions in two ways. If <filename> is not provided, all currently loaded input trajectories (from *trajin* commands) are added to TRAJ data set named <setname>. **Note that if the input trajectory list is cleared (via 'clear trajin') this will invalidate the TRAJ data set.** In addition, currently all trajectories must have the same number of atoms. Otherwise add trajectory <filename> to TRAJ data set <setname>.

TRAJ data sets cannot be modified.

37.7.12. permutedihedrals

```
permutedihedrals crdset <COORDS set> resrange <range> [{interval | random}]
                        [outtraj <filename> [<outfmt>]] [crdout <output COORDS>]
                        [<dihedral types>]
Options for 'random':
[rseed <rseed>] [out <# problems file> [<set name>]]
[check [cutoff <cutoff>] [rescutoff <rescutoff>] [maxfactor <max.factor>]
[backtrack <backtrack> [checkallresidues] [increment <increment>]] ]
Options for 'interval':
<interval deg>
<dihedral types> = alpha beta gamma delta epsilon zeta nu1 nu2 h1p c2p chin
                  phi psi chip omega

crdset <COORDS set> COORDS data set to operate on.
resrange <range> Residue range to search for dihedrals.
interval Rotate found dihedrals by <interval>. This is done in an
ordered fashion so that every combination of dihedral rotations
is sampled at least once.
random Rotate each found dihedral randomly.
[outtraj <filename>] Trajectory file to write coordinates to.
[<outfmt>] Trajectory file format.
[crdout <output COORDS>] COORDS data set to write coordinates to.
<dihedral type> One or more dihedral types to search for.

Options for 'interval':
<interval deg> Amount to rotate dihedral by each step.
Options for 'random':
[rseed <rseed>] Random number seed.
```

[out <# problems file>] File to write number of problems (clashes) each frame to.

[<set name>] Number of problems data set name.

[check] Check randomly rotated structure for clashes.

[cutoff <cutoff>] Atom cutoff for checking for clashes (default 0.8 Å).

[rescutoff <cutoff>] Residue cutoff for checking for clashes (default 10.0 Å).

[maxfactor <max_factor>] The maximum number of total attempted rotations will be <max_factor> * <total # of dihedrals> (default 2).

[backtrack <backtrack>] (No longer recommended as of version 5.1.0). If a clash is encountered at dihedral N and cannot be resolved, go to dihedral N-<backtrack> to try and resolve the clash (default is no backtracking).

[checkallresidues] If specified all residues checked for clashes, otherwise only residues up to the currently rotated dihedral check.

[increment <increment>] If a clash is encountered, first attempt to rotate dihedral by increment to resolve it; if it cannot be resolved by a full rotation the calculation will backtrack (default 1).

Create a trajectory by rotating specified dihedrals in a structure by regular intervals (**interval**), or create 1 structure by randomly rotating specified dihedrals (**random**). When randomly rotating dihedrals steric clashes will be checked if **check** is specified; in such cases the algorithm will attempt to resolve the clash as best it can. If clashes are not being resolved you can increase the number of rotation attempts *cpptraj* will make by increasing **maxfactor**.

For example, to rotate all backbone dihedrals in a protein with coordinates in a file named tz2.rst7 in -120 degree intervals and write the resulting trajectory in Amber format to rotations.mdcrd:

```
reference tz2.rst7 [TZ2]
permutedihedrals crdset [TZ2] interval -120 outtraj rotations.mdcrd phi psi
```

To randomly rotate backbone dihedrals for the same structure and write to file random.mol2 in MOL2 format:

```
reference tz2.rst7 [TZ2]
permutedihedrals crdset [TZ2] random rseed 1 check maxfactor 10 phi psi \
outtraj random.mol2 multi
```

37.7.13. prepareforleap

```
prepareforleap crdset <coords set> [frame <#>] name <out coords set>
[psdbout <psdbfile> [terbymol]]
[leapunitname <unit>] [out <leap input file> [runleap <ff file>]
[skiperrors]
[nowat [watermask <watermask>] [noh]
[keepaltloc {<alt loc ID>|highestocc}]
[stripmask <stripmask>] [solventresname <solventresname>]
[molmask <molmask> ...] [determinemolmask <mask>]
[{nohisdetect |
[nd1 <nd1>] [ne2 <ne2>] [hisname <his>] [hienname <hie>]
```

```

    [hidname <hid>] [hipname <hip>]]
  [{nodelsulfides |
    existingdisulfides |
    [cysmask <cysmask>] [disulfidecut <cut>] [newcysname <name>]
  }{nosugars |
    sugarmask <sugarmask> [noclsearch] [nosplitres]
    [rescut <residue cutoff>] [bondoffset <offset>]
    [resmapfile <file>]
    [hasglycam] [determinesugarsby {geometry|name}]
  }]
}

```

crdset <coords set> COORDS data set containing coordinates and topology to prepare.

[frame <#>] Frame to use from COORDS set (default first).

name <out coords set> Output COORDS set containing prepared topology/coordinates.

[pdbout <pdbfile>] Output PDB name.

[terbymol] If specified, base TER cards on molecules instead of PDB chains.

[leapunitname <unit>] LEaP unit name to use when writing to <leap input file> (i.e. the LEaP input file will contain '**<unit> = loadpdb <pdbfile>**').

[out <leap input file>] File containing LEaP input needed to read in the prepared system (loadpdb, bond commands for disulfides, etc).

[runleap <ff file>] If specified, CPPTRAJ will attempt to run LEaP directly to generate a topology and coordinates; <ff file> should contain the appropriate 'source' commands for loading the desired force field parameters. Will attempt to produce topology <unit>.parm7 and coordinates <unit>.rst7.

[skiperrors] If specified, the command will try to ignore any errors encountered. Can be useful for debugging.

[nowat] If specified, remove waters from the system.

[watermask <watermask>] Mask selecting waters to remove (default '**:<solventresname>**').

[noh] If specified, strip all hydrogen atoms from the system (recommended).

[keepaltloc {<alt loc ID>|highestocc}] LEaP cannot handle alternate atom locations, so the command will choose location 'A' by default. This can be changed to either <alt loc id> or the location with the highest occupancy if 'highestocc' is specified.

[stripmask <stripmask>] Mask of atoms to remove from the system.

[solventresname <solventresname>] Solvent residue name (default 'HOH').

[molmask <mask>] If specified, atoms in <mask> will be considered all part of one molecule. May be specified multiple times.

[determinemolmask <mask>] If specified, determine if atoms selected in <mask> are in the same molecule via bonds.

Histidine Detection:

[nohisdetect] Disable renaming of histidine residues based on existing hydrogens.

[nd1 <nd1>] Delta nitrogen atom name (default 'ND1').

[ne2 <ne2>] Epsilon nitrogen atom name (default 'NE2').

[hisname <his>] Histidine residue name (default 'HIS').

[hienname <hie>] Epsilon-protonated histidine name (default 'HIE').

[hidname <hid>] Delta-protonated histidine name (default 'HID').

[hipname <hip>] Doubly-protonated histidine name (default 'HIP').

Disulfide Handling:

[nodisulfides] Disable handling of disulfides.

[existingdisulfides] Only handle disulfides already present; do not search for additional disulfides.

[cysmask <cysmask>] Mask for selecting cysteine residues (default 'CYS').

[disulfidecut <cut>] Sulfur to sulfur atom distance cutoff for forming a disulfide (default 2.5 Ang).

[newcysname <name>] Name to change cysteine residues that participate in a disulfide bond to (default 'CYX').

Sugar Handling:

[nosugars] Disable handling of sugars.

[sugarmask <sugarmask>] Mask selecting sugars to be handled. If not specified the default is all residues defined in resmapfile.

[noc1search] If specified, disable search for missing linkages to sugar C1 atom bonds.

[nosplitres] If specified, do not attempt to split off functional groups from sugars into separate residues.

[rescut <residue cutoff>] Initial distance cutoff (default 8 Ang.) for residue center to residue center distance when looking for missing sugar linkages.

[bondoffset <offset>] Offset (default 0.2 Ang.) to add to 'ideal' bond distances when looking for missing sugar linkages. Can be increased to accommodate distorted structures.

[resmapfile <file>] File containing sugar residue/atom name mapping. Default is '\$CPPTRAJHOME/dat/Carbohydrate.PDB_Glycam.Names.txt'.

[hasglycam] If specified, assume sugars already have GLYCAM residue names; just check sugar anomer type/configuration/linkage.

[determinesugarsby {geometry|name}] Determine whether sugar anomer type/configuration should be chosen based on sugar geometry (default) or the residue name. CPPTRAJ will report when a mismatch is detected between the sugar anomer type/configuration based on geometry and anomer type/configuration based on the residue name.

This command will prepare a structure (usually from a PDB) for processing with the Amber program LEaP to generate topology and coordinates files for MD simulations.^[766] It will handle things like choosing alternate atom locations, removing waters/hydrogen atoms from the structure, renaming residues and generating 'bond' commands

for disulfide bonds, change histidine names based on any existing protonation, and renaming residues/atoms and generating 'bond' commands for carbohydrates. The command can also call LEaP directly to generate the parameters once the structure is prepared.

If hydrogen atoms are present in the structure, the command will attempt a simple and straightforward determination of the protonation state of any histidine residues based on where hydrogens are bonded, and assign the appropriate residue name. The command will also identify any existing disulfide bonds as well as potential disulfide bonds and generate the corresponding LEaP 'bond' commands which can be applied after the structure is loaded in LEaP. Potential disulfide bonding atoms can be identified via a user-specifiable mask expression.

By default, sugars will have their residue names changed to those compatible with the GLYCAM force field based on their anomer type (alpha/beta), configuration (D/L), and linkages (glycosidic and covalent sugar to non-sugar). Any recognized functional groups that are part of sugar residues (hydroxyl, acetyl, sulfate, etc) will be split into separate residues as required by GLYCAM. If this happens and 'runleap' has not been specified, CPPTRAJ will warn about any residues/atoms that require charge to be adjusted. If 'runleap' has not been specified the command will warn about any atoms that need to have their charges adjusted after LEaP is run.

The command will try to report any potential problems that LEaP might encounter. These include residue names that may be unrecognized (and therefore may not have parameters), mismatches between detected sugar anomer type/configuration and anomer type/configuration based on the sugar residue name, unrecognized sugar linkages, and so on.

For example, the following input prepares PDB 4zzw for processing with PDB, putting the proper leap commands in leap.4zzw.in, writing the prepared PDB to 4zzw.cpptraj.pdb, removing waters and hydrogen atoms, and keeping alternate atom locations with the highest occupancy:

```
parm 4zzw.pdb
loadcrd 4zzw.pdb name MyCrd
prepareforleap crdset MyCrd name Final out leap.4zzw.in leapunitname m \
  pdbout 4zzw.cpptraj.pdb nowat noh keepaltloc highestocc
```

Sugar Residue/Atom Name Mapping File

This file controls how CPPTRAJ will name sugars based on sugar form/chirality linkage. It consists of three sections separated by a blank line. The first section defines sugar PDB residue names and how they are mapped to GLYCAM residue characters:

```
Format: <ResName> <GlycamCode> <Anomer> <Config> <RingType> "<Name>"
Anomer: A=alpha, B=beta
Config: D/L
RingType: P=pyranose, F=furanose
Example: 64K A A D P "alpha-D-arabinopyranose"
```

The second section contains PDB to GLYCAM atom name maps for residues:

```
Format: <GLYCAM residue codes> <PDB atom name>,<GLYCAM atom name>[,<anomer>] ...
If <anomer> (A=alpha, B=beta) is specified, the atom name map is only valid for tha
Example: V,W,Y C7,C2N O7,O2N C8,CME
```

The third section contains PDB to GLYCAM linkage residue (i.e. non-sugar residues bonded to sugars) name maps:

```
Format: <PDB residue name> <GLYCAM residue name>
Example: SER OLS
```

37.7.14. reference

Reference coordinates can now be used and manipulated like COORDS data sets. See [37.10.3 on page 744](#) for command syntax.

37.7.15. rotatedihedral

```

rotatedihedral crdset <COORDS set> [frame <#>] [name <output set name>]
    {value <value> | increment <increment>}
    { <mask1> <mask2> <mask3> <mask4> |
      res <#> type <dih type> }
<dih type> = alpha beta gamma delta epsilon zeta nu1 nu2 h1p c2p chin
            phi psi chi omega
crdset <COORDS set> Coordinates data set to work on.  If a TRAJ data
set is specified, name must also be specified.
[frame <#>] Frame of the COORDS set to work on.
[name <output set name>] Output COORDS set.  If not specified the
input COORDS set will be modified.
value <value> Set specified dihedral to given value in degrees.
increment <increment> Increment specified dihedral by increment in
degrees.
<mask1> <mask2> <mask3> <mask4> Define dihedral by atom masks.
Each mask should only select one atom.
res <#> Rotate dihedral specified by type in residue number <#>.
type <dih type> Dihedral type to rotate in specified residue.

```

Rotate the specified dihedral in given COORDS set to a target value or by given increment. For example, to set the protein chi dihedral in residue 8 to 35 degrees and write out to a mol2 file:

```

parm ../tz2.parm7
loadcrd ../tz2.nc 1 1 name TZ2
rotatedihedral crdset TZ2 value 35 res 8 type chip
crdout TZ2 tz2.rotate.1.mol2

```

37.7.16. sequence

```

sequence name <output set name> <unit0> <unit1> ...
    [{libset <libsetname>} ...]
name <output set name> Name of final molecule.
<unit0> <unit1> Name of COORDS set (with connection info).
[{libset <libsetname>} ...] One or more set name prefixes of data sets
(libraries) containing units.
Data sets created:
<output set name> COORDS set containing final molecule.

```

Connect units in different COORDS sets together to form a single molecule. Internal coordinates are used to try to determine the correct geometry around connection sites. The COORDS sets must have connection information set, either from reading in an Amber OFF library file or set manually via **dataset connect** (see 37.8.8 on page 719).

For example, the following reads in two Mol2 files as COORDS sets, sets up connection atoms, then creates a molecule via *sequence*:

```

parm MOC.mol2
loadcrd MOC.mol2 parm MOC.mol2 name MOC
dataset connect MOC tailmask @O5

```

```

parm CNALA.mol2
loadcrd CNALA.mol2 parm CNALA.mol2 name CNALA
dataset connect CNALA headmask @N
sequence MOC CNALA name Mol
crdout Mol Mol.mol2

```

37.7.17. splitcoords

```

splitcoords <crd set> name <output set name>
<crd set> COORDS set to split.
name <output set name> Name of new set to create.

```

Split trajectory specified by <crd set> by molecule into a new COORDS set. All molecules in <crd set> must be the same size. For example, if there are 10 molecules and 10 frames in COORDS set "Set0", the following would create a new COORDS set with 100 frames (original molecules 1-10 frame 1, original molecules 1-10 frame 2, etc):

```
splitcoords Set0 name Set0Split
```

37.7.18. zmatrix

```

zmatrix <COORDS set name> [name <output set name>]
{ zset <input zmatrix set> [parm <top>|parmindex <#>] |
  [molnum <mol#>] [frame <frame#>] [out <zmatrix file>] }
<COORDS set name> COORDS set to calculate Z-matrix from or use as
topology for applied Z-matrix.
[name <output set name>] Name of output COORDS set (if 'zset') or
output Z-matrix set.
zset <input zmatrix set> Name of Z-matrix set to use to generate
coordinates.
parm <top> Use specified topology name as topology for
generated coordinates.
parmindex <#> Use topology index as topology for generated
coordinates.
[molnum <mol#>] Calculate Z-matrix from specified molecule (default
first molecule).
[frame <frame#>] Calculate Z-matrix from specified molecule in
specified frame (default first frame).
[out <zmatrix file>] File to write calculated Z-matrix to.
Data sets created:
<output set name> If 'zset', COORDS set containing final
coordinates. Otherwise contains Z-matrix data.

```

Command for working with Z-matrices. If 'zset' is specified, generate coordinates from the specified Z-matrix data set and topology. Otherwise, calculate a Z-matrix for a single molecule from the specified frame of given COORDS data set.

37.8. General Commands

The following general commands are available:

Command	Description
activeref	Select the reference for distance-based masks.
calc	Evaluate the given mathematical expression.
clear	Clear various objects from the cpptraj state.
create	Create (but do not yet write) a data file.
createset	Create a dataset from a simple mathematical expression.
datafile	Used to manipulate data files.
datafilter	Filter data sets based on given criteria.
dataset	Use to manipulate data sets.
debug prnlev	Set debug level. Higher levels give more info.
ensexextension	Enable/disable ensemble number extension for files in ensemble mode.
exit quit	Quit cpptraj.
flatten	Distribute elements of 2d matrix across 1d array.
go run	Start a trajectory processing Run.
help	Provide help for commands.
list	List various objects in the cpptraj state.
noexitonerror	Attempt to continue even if errors are encountered.
noprogress	Do not print a progress bar during a Run.
parallelanalysis	(MPI only) Divide current Analyses among MPI processes.
parsedata	Parse timing data from CPPTRAJ output.
precision	Change the output precision of data sets.
printdata	Print data set to screen.
random	Change default random number generator, create random sets.
readdata	Read data sets from files.
readensembledata	Read data files in ensemble mode.
readinput	Read cpptraj input from a file.
removedata	Remove specified data set(s).
rst	Generate Amber-style distance/angle/torsion restraints.
runanalysis	Run an analysis immediately or run all queued analyses.
select	Print the results of an atom mask expression.
selectds	Print the results of a data set selection expression.
silenceactions	Prevent Actions from writing information to STDOUT.
sortensembledata	Sort data sets using replica information (currently constant pH only).
usediskcache	Turn caching of data sets to disk on or off.
write writedata	Immediately write data to a file or write to all current data files.

37.8.1. **activeref**

activeref <#>

Set which reference structure should be used when setting up distance-based masks for everything but the 'mask' action. Numbering starts from 0, so 'activeref 0' selects the first reference structure read in, 'activeref 1' selects the second, and so on.

37.8.2. **calc**

calc <expression>
[**prec** <width>.<precision>] [**format** {double|general|scientific}]
<expression> Mathematical expression to evaluate. See [37.5.2 on page 694](#) for details.
prec <width>.<precision> Set the width and precision of the result.
format {double|general|scientific} Set the format of the result.

Evaluate the given mathematical expression. This version gives more control over the format of the output.

37.8.3. **clear**

clear [{all | <type>}]
(<type> = actions, trajin, trajout, ref, parm, analysis, datafile, dataset)

Clear list of indicated type, or all lists if 'all' specified. Note that when clearing actions or analyses, associated data sets and data files are not cleared and vice versa.

37.8.4. **create**

create <filename> <datasetname0> [<datasetname1> ...] [<DataFile Options>]

Add specified data sets to the data file named <filename>; if the file does not exist, it will be added to the DataFileList. Data files created in this way are only written at the end of coordinate processing, analyses, or via the '*writedata*' command. See [37.6 on page 697](#) for more data file format options.

37.8.5. **createset**

createset <expression> [**xmin** <min>] **xstep** <step> **nx** <nxvals>
expression Simple mathematical expression, must contain equals sign, can contain X (e.g. Y=2*X). If not enclosed in quotes must not contain whitespace.
xmin <min> Minimum X value.
xstep <step> X step.
nx <nxvals> Number of X values.

Generate a data set from a simple mathematical expression.

37.8.6. **datafile**

datafile <filename> <datafile arg>

Pass <datafile arg> to data file <filename>. See [37.6 on page 697](#) for more details.

37.8.7. datafilter

```

datafilter {<dataset arg> min <min> max <max> ...} [out <file>] [name <setname>]
           {[multi] | [filterset <set> [newset <newname>]] [countout <countfile>]}
<dataset arg> min <min> max <max> Data set name and min/max cutoffs
to use; can specify more than one.
[out <file>] Write out to file named <file>.
[name <setname>] Name of filter data set containing 1 when cutoffs
satisfied, 0 otherwise.
[multi] Filter each set separately instead of all together (creates
filter set for each input set). Cannot be used with
'filterset'.
[filterset <set>] If specified, <set> will be filtered to only contain
data that satisfies cutoffs. Cannot be used with 'multi'.
[newset <newname>] If specified a new set will be created from
'filterset' instead of replacing 'filterset'.
[countout <count>] If specified, write number of elements passed and
filtered to <countfile>. Cannot be used with 'multi'.

Sets Created (not 'multi')
<setname> For each input element contains 1 for elements that
'passed', 0 otherwise.
<setname>[npassed] Number of elements that passed.
<setname>[nfiltered] Number of elements filtered out.

Sets Created ('multi')
<setname>:<idx> For each input set (number with <idx>, starting
from 0) contains 1 for elements that 'passed', 0 otherwise.

```

Create a data set (optionally named <setname>) containing 1 for data within given <min> and <max> criteria for each specified data set. There must be at least one <min> and <max> argument, and can be as many as there are specified data sets. If 'multi' is specified then only filter data sets will be created for each data set instead. If 'filterset' is specified, the specified <set> will be modified to only contain '1' frames; cannot be used with 'multi'. If 'newset' is also specified, a new set will be created containing the '1' frames instead. The 'filterset' functionality only works for 1D scalar sets. If 'countout' is specified, the final number of elements passed and filtered out will be written to <countfile>.

For example, to read in data from two separate files (d1.dat and a1.dat) and generate a filter data set named FILTER having 1 when d1 is between 0.0 and 3.0 and a1 is between 135.0 and 180.0:

```

readdata a1.dat name a1
readdata d1.dat name d1
datafilter d1 min 0.0 max 3.0 a1 min 135.0 max 180.0 out filter.dat name FILTER

```

Note that a similar command that can be used with data generated by Actions during trajectory processing is *filter* (see page 780).

37.8.8. dataset

```

dataset { legend <legend> <set> |
          makexy <Xset> <Yset> [name <name>] |
          vectorcoord {X|Y|Z} <set> [name <name>] |

```

```

cat <set0> <set1> ... [name <name>] [nooffset] |
  make2d <1D set> cols <ncols> rows <nrows> [name <name>] |
  {drop|keep}points {range <range arg> | [start <#>] [stop <#>] [offset <#>]
    [name <output set>] <set arg1> ... |
  remove <criterion> <select> <value> [and <value2>] [<set selection>] |
  connect {[head <head atom>] [tail <tail atom>] [headmask <headmask>] [tail
  dim {xdim|ydim|zdim|ndim <#>} [label <label>] [min <min>] [step <step>] |
  outformat {double|scientific|general} <set arg1> [<set arg 2> ...] |
  invert <set arg0> ... name <new name> [legendset <set>] |
  shift [above <value> by <offset>] [below <value> by <offset>] <set arg0>
[mode <mode>] [type <type>] <set arg1> [<set arg 2> ...]
}
<mode>: 'distance' 'angle' 'torsion' 'pucker' 'rms' 'matrix' 'vector'
<type>: 'alpha' 'beta' 'gamma' 'delta' 'epsilon' 'zeta' 'nu0' 'nu1' 'nu2' 'nu3'
       'nu4' 'h1p' 'c2p' 'chin' 'phi' 'psi' 'chip' 'omega' 'chi2' 'chi3' 'chi4'
       'chi5' 'pucker' 'noe' 'distance' 'covariance' 'mass-weighted covariance'
       'correlation' 'distance covariance' 'IDEA' 'IRED' 'dihedral covariance'
Options for 'type noe':
  [bound <lower> bound <upper>] [rexp <expected>] [noe_strong] [noe_medium]
  [noe_weak]

[name <name>] New data set name for
  makexy/vectorcoord/cat/make2d/droppoints/keepoints.
legend <legend> <set> Set the legend for data set <set> to
  <legend>.
makexy <Xset> <Yset> Create a new data set (optionally named
  <name>) with X values from <Xset> and Y values from <Yset>.
vectorcoord {X|Y|Z} <set> Extract X/Y/Z coordinates from vector data
  set into a new 1D data set.
cat <set0> <set1> ... Concatenate two or more data sets into a new
  data set (optionally named <name>). Only works for scalar 1D
  and string sets.
make2d <1D set> cols <ncols> rows <nrows> Convert 1D data set into
  row-major 2D data set with specified number of rows and
  columns.
{drop|keep}points <set arg1> ... Drop or keep specified points from data
  set(s), optionally creating a new data set.
  range <range arg> Range of points to drop/keep.
  [start <#>] [stop <#>] [offset <#>] Start/stop/offset values of
  points to drop/keep.
remove <criterion> <select> <value> [and <value2>] [<set selection>] Remove
  data sets from <set selection> according to specified
  criterion and selection.

  <criterion>: 'ifaverage' 'ifsize' 'ifmode' 'iftype'
  <select>    : 'equal' '==' 'notequal' '!=' 'lessthan' '<'
               'greaterthan' '>' 'between' 'outside'

connect <set args> Add/change connect atom information (used by the
  sequence command, 37.7.16 on page 714) to COORDS set(s). Head
  atoms connect to previous residue, tail atoms connect to next
  residue. Can use either absolute atom numbers or atom mask
  expressions.

```

[head <head atom>] [tail <tail atom>] Specify the head/tail atoms by atom number.

[headmask <headmask>] [tailmask <tailmask>] Specify the head/tail atoms by atom mask.

dim {xdim|ydim|zdim|ndim <#>} Change specified dimension in set(s).

label <label> Change dimension label to <label>

min <min> Change dimension minimum to <min>.

step <step> Change dimension step to <step>.

invert <set arg0> ... name <new name> [legendset <set>]
<set arg0> ... Specify sets to invert.
name <new name> Inverted output set name.
[legendset <set>] String data set containing legends

shift

[above <value> by <offset>] Values in set(s) above <value> will be shifted by <offset>.

[below <value> by <offset>] Values in set(s) below <value> will be shifted by <offset>.

<set arg0> ... Set(s) to shift.

[mode <mode>] Set data set(s) mode to <mode>.

[type <type>] Set data set(s) type to 'type', useful for e.g. analysis with *statistics*. Note this can also be done with 'type <type>' for certain commands (*distance*, *dihedral*, *pucker* etc). Note that not every <type> is compatible with a given <mode>.

Options for 'type noe' only:

[bound <lower> bound <upper>] Lower and upper bounds for NOE (in Angstroms); must specify both.

[rexp <expected>] Expected value for NOE (in Angstroms); if not given '(<lower> + <upper>)' / 2.0 is used.

[noe_strong] Set lower and upper bounds to 1.8 and 2.9 Å respectively.

[noe_medium] Set lower and upper bounds to 2.9 and 3.5 Å respectively.

[noe_weak] Set lower and upper bounds to 3.5 and 5.0 Å respectively.

Either set the legend for a single data set, create a new set with X values from one set and Y values from another, concatenate 2 or more sets, make a 2D set from 1D set, remove sets according to a certain criterion, or change the mode/type for one or more data sets.

Setting the mode/type can be useful for cases where the data set is being read in from a file; for example when reading in a dihedral data set the type can be set to 'dihedral' so that various Analysis routines like *statistics* know to treat it as periodic. A brief description of possible modes and types follows:

Mode	Type	Description
distance	noe	NOE distance.
angle		Angle.
torsion	alpha	Nucleic acid alpha.
	beta	Nucleic acid beta.
	gamma	Nucleic acid gamma.
	delta	Nucleic acid delta.
	epsilon	Nucleic acid epsilon.
	zeta	Nucleic acid zeta.
	nu1	Nucleic pucker (O4').
	nu2	Nucleic pucker (C4').
	h1p	Nucleic acid H1'.
	c2p	Nucleic acid C2'.
	chin	Nucleic acid chi.
	phi	Protein Phi.
	psi	Protein psi.
	chip	Protein chi.
	omega	Protein omega.
pucker	pucker	Sugar pucker.
rms		RMSD.
matrix	distance	Distance matrix.
	covariance	Cartesian covariance matrix.
	'mass-weighted covariance'	Mass weighted Cartesian covariance matrix.
	correlation	Dynamic cross correlation matrix.
	'distance covariance'	Distance covariance matrix.
	IDEA	IDEA matrix.
	IREDD	IREDD matrix.
	'dihedral covariance'	Dihedral covariance matrix.
vector	IREDD	IREDD vector.

The invert mode takes a group of M 1D data sets of size N and create N new "inverted" data sets of size M. This is similar to the invert keyword already available for standard and Grace data writes, but operates directly on data sets. For example, given the following two data sets:

```

D0 D1
 1  4
 2  5
 3  6

```

The new data sets will be laid out like so:

```
NO N1 N2
  1  2  3
  4  5  6
```

The dataset invert command can be useful if you want to easily view output from multiple analysis commands in a single graph. For example, to view state counts from two different simulations side by side:

```
calcstate name Sim1 state bound1,dist1,0.0,2.0
calcstate name Sim2 state bound1,dist1,0.0,2.0
runanalysis dataset invert Sim*[Count] name Inverted legendset Sim1[Name]
dataset dim xdim label Simulation min 1 step 1 Inverted*
writedata statecount.agr Inverted*
```

The dataset shift command can be used for wrapping circular values, such as torsions. For example, to ensure a pucker has a range from 0 to 360 instead of -180 to 180:

```
pucker Furanoid @C2 @C3 @C4 @C5 @O2 cremer out CremerF.dat amplitude
run
dataset shift Furanoid below 0 by 360
```

37.8.9. debug | prnlev

```
debug [<type>] <#>
      (<type> = actions, trajin, trajout, ref, parm, analysis, datafile, dataset)
```

Set the level of debug information to print. In general the higher the <#> the more information that is printed. If <type> is specified only set the debug level for a specific area of *cpptraj*.

37.8.10. ensexextension

```
ensexextension {on|off}
```

Turn printing of ensemble member number filename extensions on or off. By default ensemble extensions are printed in parallel and not in serial.

NOTE: THE 'ensexextension off' OPTION HAS NOT BEEN FULLY TESTED IN PARALLEL AND IS NOT CURRENTLY RECOMMENDED.

37.8.11. exit | quit

Exit normally.

37.8.12. flatten

```
flatten name <output set name> [mode {sum|avg}] <input set args>
name <output set name> Name of ``flattened`` 1D output set(s).
mode {sum|avg} If sum, matrix elements will be summed. If avg,
matrix elements will be averaged.
<input set args> Specify matrices to ``flatten``.
DataSets Created
<output set name> Flattened 1D set if only one input matrix.
```

<output set name>:<idx> Flattened 1D sets when more than one input matrix; index starts from 1.

Flatten 1 or more matrices into 1D array(s) by summing or averaging elements. For example, given a matrix with values like this:

```
X Y Value
1 3 5.0
1 4 4.0
2 3 2.0
```

The “flattened” 1D array with mode SUM would be determined as follows:

```
Element 1 = (5.0/2) + (4.0/2) = 4.5
Element 2 = (2.0/2) = 1.0
Element 3 = (5.0/2) + (2.0/2) = 3.5
Element 4 = (4.0/2) = 2.0
```

And the final 1D array would look like so:

```
Index Value
1      4.5
2      1.0
3      3.5
4      2.0
```

37.8.13. *go* | *run*

Begin trajectory processing, followed by analysis and datafile write.

37.8.14. *help*

```
help [ { All |
      <cmd> |
      <command category> |
      Form[ats] [{read|write}] |
      Form[ats] [{trajin|trajout|readdata|writedata|parm|parmwrite} [<fmt key>]] |
      Mask      } ]
Command Categories: Gen[eral] Sys[tem] Coor[ds] Traj[ectory] Top[ology]
                   Act[ion] Ana[lysis] Con[trol]
All                : Print all known commands.
<cmd>              : Print help for command <cmd>.
<command category> : Print all commands in specified category.
Form[ats]          : Help for file formats.
Mask              : Help for mask syntax.
```

If ‘All’ is specified, list all commands known to *cpptraj*. If given with a command, print help for that command. Otherwise, list all commands of a certain category (General, System, Coords, Trajectory, Topology, Action, Analysis, or Control), help for various file formats, or help with atom mask syntax.

37.8.15. *list*

```
list <type>
(<type> = actions, trajin, trajout, ref, parm, analysis, datafile, dataset)
```

List the currently loaded objects of <type>. If no type is given then list all loaded objects.

37.8.16. noexitonerror

noexitonerror

Normally *cpptraj* will exit if actions fail to initialize properly. If *noexitonerror* is specified, *cpptraj* will attempt to continue past such errors. This is the default if in interactive mode.

37.8.17. noprogress

noprogress

Do not display read progress during trajectory processing.

37.8.18. parallelanalysis

parallelanalysis [**sync**]

MPI only. Divide all currently set up analyses as evenly as possible among available MPI processes and execute. Each analysis will get a single MPI process. If **sync** is specified all data will be synced back to the master process (for e.g. subsequent analysis). For an example of how to use the parallelanalysis command, see [37.12.16 on page 868](#).

37.8.19. parsedata

```
parsetiming <filename args> ... [out <file>] [name <setname>]
      [sortby {time|cores|filename}] [includebad] [showdetails]
      [type {trajproc|trajread|actframe}] [reverse]
      [groupout <file>] [grouptype {prefix|name|kind}]
```

<filename args> Files containing CPPTRAJ output to get timing data from.

[out <file>] Write total sorted timing sets to <file>.

[name <setname>] Set name for timing data sets.

[sortby {time|cores|filename}] Sort timing data sets by either time, number of cores (MPI processes * OpenMP threads), or file name.

[includebad] If specified, include run output for which timing data cannot be extracted (e.g. an incomplete/failed run).

[showdetails] If specified, details about each run will be printed to STDOUT.

[type {trajproc|trajread|actframe}] If specified, report time other than the total time (which is the default):

trajproc Total trajectory processing time (trajectory I/O plus action frame time).

trajread Trajectory read time (requires compiling with '-timer' configure option/-DTIMER compiler define).

actframe Action frame time (requires compiling with '-timer' configure option/-DTIMER compiler define).

[reverse] Instead of longest time to shortest time, sort shortest time to longest time.

[groupout <file>] Group run output by a property and write to a file. Additional details will be written like speedup and efficiency (relative to the slowest run).

[grouptype {prefix|name|kind}] Property to group runs by in group output file.

prefix Group by directory prefix.

name Group by run type name (see `<setname>[name]` below).

kind Group by run type name; OpenMP runs are separated by number of OpenMP threads.

DataSets Created:

`<setname>` Set containing sorted run times.

`<setname>[name]` Set containing shortedhand run names. Consists of a prefix (S for serial, O for OpenMP, M for MPI, H for hybrid MPI/OpenMP) followed by numbers indicating number of MPI processes 'x' OpenMP threads; e.g., H16x4 means a hybrid run consisting of 16 MPI processes and 4 OpenMP threads per process. If CUDA is active, the name will be wrapped in 'G()', e.g. G(H16x4).

`<setname>[dir]` Set containing output directory prefixes. If no directory prefix, just contains output file name.

The *parsedata* command can be used to extract timing data from CPPTRAJ output.

37.8.20. precision

precision {<filename> | <dataset arg>} [<width>] [<precision>]

Set the precision for all data sets in data file `<filename>` or data set(s) specified by `<dataset arg>` to *width.precision*, where width is the column width and precision is the number of digits after the decimal point. Note that the `<precision>` argument only applies to floating-point data sets.

For example, if one wanted to set the precision of the output of an Rmsd calculation to 8.3, the input could be:

```
trajin ../run0.nc
rms first :10-260 out prec.dat
precision prec.dat 8 3
```

and the output would look like:

```
#Frame RMSD_00000
1 0.000
2 0.630
```

37.8.21. random

random [setdefault {marsaglia|stdlib|mt|pcg32|xol128}]

[createset <name> count <#> [seed <#>]

settype {int|float01|gauss [mean <mean>] [sd <SD>]]]

setdefault If specified, change the default random number generator (RNG).

marsaglia Use the Marsaglia RNG that is used in the Amber MD programs sander/pmemd.

stdlib Use the C standard library RNG.

mt Use the C++11 implementation of the Mersenne twister (mt19937); only available with C++11 support.

pcg32 Use the 32 bit version of the Permuted Congruential Generator. [\[767\]](#)

xo128 Use the Xoshiro128++ RNG. [\[768\]](#)

createset If specified, create a 1D data set filled with random numbers of the specified type.

<name> Name of created set.

count <#> The number of elements to put into the set.

settype {int|float01|gauss} Type of numbers to use; integer, floating point between 0 and 1, Gaussian distribution.

mean <mean> Mean of distribution for 'gauss'.

sd <SD> Standard deviation of distribution for 'gauss'.

seed <#> Optional seed for the RNG.

This command can be used to set the default random number generator used in CPPTRAJ, and/or create a 1D data set filled with random values.

37.8.22. readdata

readdata <filename> [name <dsname>] [as <fmt>] [separate] [<format options>]

name <dsname> Name for read-in data set(s). Default is <filename>.

as <fmt> Force <filename> to be read as a specific format using given format keyword.

separate Read each file specified into separate data sets indexed from 0.

Read data from file <filename> and store as data sets. For more information on formats currently recognized by cpptraj see [37.1 on page 695](#). For format-specific options see [37.6](#). For example, given the file calc.dat:

```
#Frame  R0   D1
1       1.7 2.22
```

The command 'readdata calc.dat' would read data into two data sets, calc.dat:2 (legend set to "R0") and calc.dat:3 (legend set to "D1").

37.8.23. readensembledata

readensembledata <filename> [filenames <additional files>] [<readdata args>]

<filename> Lowest replica file name.

filenames <additional files> Specified additional members of the ensemble. If not specified ensemble members will be search for using numerical extensions.

<readdata args> Additional data file arguments.

Read data sets as an ensemble, i.e. each file is a different member of an ensemble. This command is MPI-aware.

If one filename is given, it is assumed it is the "lowest" member of an ensemble with a numerical extension, e.g. 'file.001' and the remaining files are searched for automatically. Otherwise all other members of the ensemble can be specified with 'filenames' and a comma-separated list e.g. 'file.001 filenames file.002,file.003,file.004. For additional 'readdata' arguments that can be passed in see [37.6 on page 697](#).

For example, to read in data files named cpout.001 to cpout.006 automatically:

37. *cpptraj*

```
readensembledata cpout.001 cpin cpin name PH
```

Or specified:

```
readensembledata cpout.001 \  
    filenames cpout.002,cpout.003,cpout.004,cpout.005,cpout.006 \  
    cpin cpin name PH
```

37.8.24. readinput

```
readinput <filename>
```

Read *cpptraj* commands from file <filename>.

37.8.25. removedata

```
removedata <arg>
```

Remove data set corresponding to <arg>.

37.8.26. rst

```
rst <mask1> <mask2> [<mask3>] [<mask4>]  
    r1 <r1> r2 <r2> r3 <r3> r4 <r4> rk2 <rk2> rk3 <rk3>  
    {[parm <parmfile / tag> | parmindex <#>]}  
    [{ref <refname> | refindex <#> | reference} [offset <off>] [width <width>]]  
    [out <outfile>]
```

<mask1> (Required) First atom mask.

<mask2> (Required) Second atom mask. If only two masks assume distance restraint.

[<mask3>] (Optional) Third atom mask. If 3 atom masks assume angle restraint.

[<mask4>] (Optional) Fourth atom mask. If 4 atom masks assume dihedral restraint.

rX <rX> Value of RX (X=1-4, default 0.0)

rk2 <rk2> Value of RK2 (force constant to be applied when R is R1 <= R < R2)

rk3 <rk3> Value of RK3 (force constant to be applied when R is R3 <= R < R4)

[parm <parmfile / tag> | parmindex <#>] Topology to be used for atom masks.

{ref <refname> | refindex <#> | reference} Use distance/angle/dihedral in reference structure to determine values for r1, r2, r3, and r4. The value of r2 is set to <r2> + <off>, r3 = r2 - <width>, r4 = r3 + <width>.

[offset <off>] (Reference only) Value to offset distance/angle/torsion in reference by (default 0.0).

[width <width>] (Reference only) Width between r1 and r2, r3 and r4 (default 0.5).

[out <outfile>] Write restraints to outfile. If not specified, write to STDOUT.

Generate Amber-style distance restraints for use with nmropt=1. This is particularly useful for generating distance restraints based off of reference coordinates. For example to generate a distance restraint between two C5' atoms using the current distance between them in a reference structure, offsetting the distance by 1.0 Ang.:

```
parm 30bp-longbox-tip3p-na.parm7
reference 30bp-longbox.rst7
rst :1@C5' :31@C5' reference offset 1.0 rk2 10.0 rk3 10.0 out output
```

37.8.27. runanalysis

```
runanalysis [<analysiscmd> [<analysis args>]]
```

Run given analysis command immediately and write any data generated. If no command is given run any analysis currently set up. NOTE: When 'runanalysis' is specified alone, data is not automatically written; to write data generated with 'runanalysis' use the 'writedata' command (this allows multiple analysis runs between output if desired).

37.8.28. select

```
select <mask>
```

Prints the number of selected atoms corresponding to the given mask, as well as the atom numbers with format:

```
Selected= <#atom1> <#atom2> ...
```

This does not affect the state in any way, but is intended for use in scripts etc. for testing the results of a mask expression.

37.8.29. selectds

```
selectds <dataset arg>
```

Show the results of a data set selection. Data set selection has the format:

```
<name> [<aspect>] : <index>
```

Either the [<aspect>] or the <index> arguments may be omitted. A '*' can be used in place of <name> or [<aspect>] as a wildcard. The <index> argument can be a single number or a range separated by '-' and ','.

This command does not affect the state in any way, but is particularly useful in interactive mode for determining the results of a dataset argument.

37.8.30. sortensembledata

```
sortensembledata <dset arg0> [<dset arg1> ...]
<dset arg0> [<dset arg1> ...] Data set(s) to sort.
```

Sort unsorted data sets. Currently only works for constant pH REMD data.

37.8.31. usediskcache

```
usediskcache {on|off}
```

If on, CPPTRAJ will attempt to cache data sets to disk if possible. This currently only works for integer data sets (e.g. *hbond series* data sets, etc).

37.8.32. write | writedata

```
write [<filename> <datasetname0> [<datasetname1> ...]] [<DataFile Options>]
```

With no arguments, write all files currently in the data file list. Otherwise, write specified data set(s) to <filename>. This is like the 'create' command except a data file is not added to the data file list; it is written immediately. See [37.6 on page 697](#) for more data file format options.

37.8.33. System Commands

These commands call the equivalent external system commands.

gnuplot <args> Call `gnuplot` (if it is installed on your system) with the given arguments.

head <args> Call `head`, which lists the first few lines of a file.

less <args> Call `less`, which can be used to view the contents of a file.

ls <args> List the contents of a directory.

pwd <args> Print the current working directory.

xmgrace <args> Call `xmgrace` (if it is installed on your system) with the given arguments.

37.9. Topology File Commands

These commands control the reading and writing of topology files. Cpptraj supports the following topology file formats:

Format	Keyword	Extension	Notes
Amber Topology	amber	.parm7	Only fully-supported format for write.
PDB	pdb	.pdb	Read Only
Mol2	mol2	.mol2	Read Only
CIF	cif	.cif	Read Only
Charmm PSF	psf	.psf	Limited Write
Gromacs Topology	gromacs	.top	Read only
SDF	sdf	.sdf	Read Only
Tinker ARC	arc	.arc	Read Only

For most commands that require a topology one can be specified via two keywords:

parm [<name>] Select topology corresponding to given file name, tag, or name.

parmindex [<#>] Select topology by order in which it was loaded, starting from 0.

The following topology related commands are available:

Command	Description
angleinfo, angles, printangles	Print angle info for selected atoms.
atominfo, atoms, printatoms	Print details for selected atoms.
bondinfo, bonds, printbonds	Print bond info for selected atoms.
bondparminfo	Print the bond parameter table.
change	Change specified parts of a topology.
charge	Print total charge for selected atoms.
comparatop	Compare two topologies and report differences.
dihedralinfo, dihedrals, printdihedrals	Print dihedral info for selected atoms.
hmassrepartition	Perform hydrogen mass repartitioning.
improperinfo, impropers, printimprovers	Print improper info for selected atoms.
mass	Print total mass for selected atoms.
molinfo	Print molecule info for selected atoms.
parm	Load a topology file.
parmbox	Modify box info for a loaded topology.
parminfo	Print details for selected topology.
parmstrip	Remove selected atoms from topology.
parmwrite	Write selected topology to file.
printub, ubinfo	Print Urey-Bradley info for selected atoms.
resinfo	Print residue info for selected atoms.
scaledihedralk	Scale selected dihedral force constants.
solvent	Change which molecules are considered solvent.
updateparameters	Update/add parameters in/to a topology.

37.9.1. angleinfo | angles | printangles

```
angleinfo [parm <name> | parmindex <#> | <#>] [<mask1>] [<mask2> <mask3>]
[out <file>]
```

[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.
Default is first loaded topology.

[<mask1>] Mask to print angle info for.

[<mask2> <mask3>] If specified, angles must match all masks.

[out <file>] File to print to (default STDOUT).

Print angle information of atoms in <mask> for selected topology (first loaded topology by default) with format:

```
# Angle Kthet degrees atom names (numbers)
```

Where *Angle* is the internal angle index, *Kthet* is the angle force constant, *degrees* is the angle equilibrium value, *atom names* shows the atoms involved in the angle with format :<residue num>@<atom name>, and (*numbers*) shows the atom indices involved in a comma-separated list. Atom types will be shown in the last column.

If 3 masks are given instead of 1, print info for angles with first atom in <mask1>, second atom in <mask2>, and third atom in <mask3>.

37.9.2. **atominfo | atoms | printatoms**

```
atominfo [parm <name> | parmindex <#> | <#>] <mask> [out <file>]
[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.
    Default is first loaded topology.
<mask> Mask selecting atoms to print info for.
[out <file>] File to print to (default STDOUT).
```

Print information on atoms in <mask> for selected topology (first loaded topology by default) with format:

```
#Atom Name #Res Name #Mol Type Charge Mass GBradius El [rVDW] [eVDW]
```

where #Atom is the internal atom index, the first Name column is the atom name, #Res is the atom's residue number, the second Name column is residue name, #Mol is the atom's molecule number, Type is the atom's type (certain topologies only), Charge is the atom charge (in units of electron charge), Mass is the atom's mass (in amu), GBradius is the generalized Born radius of the atom (Amber topologies only), and El is the 2 character element string. The final two columns are only shown if the topology contains non-bonded parameters: rVDW is the atom's Lennard-Jones radius and eVDW is the atom's Lennard-Jones epsilon.

37.9.3. **bondinfo | bonds | printbonds**

```
bondinfo [parm <name> | parmindex <#> | <#>]
    [<mask1>] [<mask2>] [out <file>] [nointrares]
[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.
    Default is first loaded topology.
[<mask1>] Mask to print bond info for.
[<mask2>] If specified, bonds must match both masks.
[out <file>] File to print to (default STDOUT).
[nointrares] Do not print intra-residue bonds.
```

Print bond information for atoms in <mask> for selected topology (first loaded topology by default) with format:

```
# Bond Kb Req atom names (numbers)
```

where Bond is the internal bond index, Kb is the bond force constant, Req is the bond equilibrium value (in Angstroms), *atom names* shows both atom names with format :<residue num>@<atom name>, and (*numbers*) shows both atom numbers in a comma-separated list. Atom types will be shown in the last column.

If 2 masks are given instead of 1, print info for bonds with first atom in <mask1> and second atom in <mask2>.

37.9.4. bondparminfo

```
bondparminfo [parm <name> | crdset <set> | parmindex <#> | <#>] [out <file>]
[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.
    Default is first loaded topology.
[out <file>] File to print to (default STDOUT).
```

Print the bond parameter table with format:

```
#Idx Rk Req
```

Where Idx is the internal bond parameter index, Rk is the bond force constant (in kcal/mol*Ang²), and Req is the bond equilibrium value (in Ang.).

37.9.5. change

```
change [parm <name> | parmindex <#> | <#> |
        crdset <COORDS set> ]
{ resname from <mask> to <value> |
  chainid of <mask> to <value> |
  oresnums of <mask> min <range min> max <range max> |
  icode of <mask> min <char min> max <char max> resnum <#> |
  atomname from <mask> to <value> |
    addbond <mask1> <mask2> [req <length> <rk> <force constant>]
    removebonds <mask1> [<mask2>] [out <file>] |
    bondparm <mask1> [<mask2>] {setrk|scalerk|setreq|scalereq} <value> |
    {mass|charge} [of <mask>] {to <value> |by <offset> |
      byfac <factor> |fromset <data set>} |
    mergeres firstres <start res#> lastres <stop res#>
  }
```

parm <name> | parmindex <#> | <#> | crdset <COORDS set> Topology to change.

resname from <mask> to <value> Change residue names for residues in <mask> to the given <value>.

chainid of <mask> to <value> Change the chain ID of residues in <mask> to given <value>.

oresnums of <mask> min <range min> max <range max> Change original residue numbers (to e.g. original PDB numbers) of residues in <mask> to a range starting from <min> and ending with <max>.

icode of <mask> min <char min> max <char max> <resnum> <#> Change residue insertion codes of residues in <mask> to a range of characters starting from <min> and ending with <max>; set the original residue number to <resnum>.

atomname from <mask> to <value> Change atom names for atoms in <mask> to the given <value>.

addbond <mask1> <mask2> Add bond between atom specified by <mask1> and atom specified by <mask2>.

[req <length>] The equilibrium bond length in Angstroms.

[rk <force constant>] The bond force constant in kcal/mol*Angstrom.

removebonds <mask1> [<mask2>] Remove bonds from atoms in <mask1>. If <mask2> also given, remove bonds between atoms in <mask1> and atoms in <mask2>.

[out <file>] If specified, write removed bonds to <file> with format '<residue name> <residue num> <atom name> <atom num>'.

bondparm <mask1> [<mask2>] {setrk|scalerk|setreq|scalereq} <value> Modify bond parameters in bonds selected by <mask1> (and <mask2> if specified) by specified <value>.

setrk Set bond force constants to <value>.

scalerk Scale bond force constants by <value>.

setreq Set bond equilibrium lengths to <value>.

scalereq Scale bond equilibrium lengths by <value>.

mass|charge Change mass or charge in specified topology.

of <mask> Atoms to change mass/charge of.

to <value> Value to change mass/charge to.

by <offset> Value to offset masses/charges by.

byfac <factor> Value to multiply masses/charges by.

fromset <data set> Use values in <data set> for mass/charge; must have the same number of values as atoms selected by <mask>.

mergeres Merge consecutive residues.

firsres <start res#> Index (starting from 1) of first residue to merge.

lastres <stop res#> Index (starting from 1) of last residue to merge. Should be greater than firstres.

Change specified parts of the specified topology. For example, to change atoms named 'HN' to 'H' in topology 0:

```
change parmindex 0 atomname from @HN to H
```

37.9.6. charge

charge [parm <name> | parmindex <#> | <#>] <mask> [out <file>] [name <set>]

parm <name> | **parmindex** <#> Topology to calculate charge from.

<mask> Atom(s) to calculate total charge for (default all).

[out <file>] File to write total charge to.

[name <set>] If specified, a data set named <set> will be created containing total charge.

Print the total charge of atoms in <mask> (in units of electron charge) for selected topology (first loaded topology by default).

37.9.7. comparetop

```
comparetop {parm <name> | parmindex <#>} {parm <name> | parmindex <#>} [out <file>]
           [atype] [lj] [bnd] [ang] [dih] [atoms]
```

parm <name> | **parmindex** <#> Topologies to compare.
out <file> Print results to file instead of screen.
[atype] Only report atom type differences.
[lj] Only report differences in Lennard-Jones parameters.
[bnd] Only report differences in bond parameters.
[ang] Only report differences in angle parameters.
[dih] Only report differences in dihedral parameters.
[atoms] Only report differences in atom properties.

Compare and report differences in atoms/parameters between two topologies. Differences are reported in standard 'diff' format, with '<' prefix indicating the parameter is from the first topology and '>' prefix indicating the parameter is from the second topology.

37.9.8. dihedralinfo | dihedrals | printdihedrals

dihedralinfo [**parm** <name> | **parmindex** <#> | <#>] [**<mask1>**] [**<mask2>** <mask3> <mask4>] [**out** <file>]
[parm <name> | **parmindex** <#> | <#>] Name/tag or index of topology.
 Default is first loaded topology.
[<mask1>] Mask to print dihedral info for.
[<mask2> <mask3> <mask4>] If specified, dihedrals must match all masks.
[out <file>] File to print to (default STDOUT).

Print dihedral information of atoms in <mask> for selected topology (first loaded topology by default) with format:

```
#Dihedral pk phase pn atoms
```

where #Dihedral is the internal dihedral index, pk is the dihedral force constant, phase is the dihedral phase, pn is the dihedral periodicity, and atoms shows the names of the atoms involved in the angle with format :<residue num>@<atom name>, followed by the atom indices involved in a comma-separated list. In addition if the dihedral is an end dihedral, improper dihedral, or both it will be prefaced with an E, I, or B respectively. Atom types will be shown in the last column.

If 4 masks are given instead of 1, print info for dihedrals with first atom in <mask1>, second atom in <mask2>, third atom in <mask3>, and fourth atom in <mask4>.

37.9.9. hmassrepartition

hmassrepartition [**parm** <name> | **crdset** <set> | **parmindex** <#> | <#>] [**<mask>**] [**hmass** <hydrogen new mass>] [**dowater**]
parm <name> Modify topology selected by name.
crdset <set> Modify topology of COORDS set.
parmindex <#> | <#> Modify topology selected by index <#> (starting from 0).
<mask> Atoms to modify (all solute atoms by default).
hmass <hydrogen new mass> Mass to change hydrogens to (3.024 u by default).

dowater If specified, modify water hydrogen mass as well.

Perform hydrogen mass repartitioning on the specified topology. Hydrogen mass repartitioning means that for a given heavy atom, the mass of all bonded hydrogens are increased (to 3.024 u by default) and the mass of that heavy atom is decreased so as to maintain the same overall mass. The main use case is to allow longer time steps for molecular dynamics integration due to reduced frequency of vibration of bonds to hydrogen atoms.

37.9.10. improperinfo | impropers | printimpropers

```
improperinfo [parm <name> | parmindex <#> | <#>] [<mask1>] [<mask2> <mask3> <mask4>]
[out <file>]
```

[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.

Default is first loaded topology.

[<mask1>] Mask to print improper info for.

[<mask2> <mask3> <mask4>] If specified, impropers must match all masks.

[out <file>] File to print to (default STDOUT).

For specified topology (first by default) either print CHARMM improper info for all atoms in <mask1>, or print info for dihedrals with first atom in <mask1>, second atom in <mask2>, third atom in <mask3>, and fourth atom in <mask4>.

37.9.11. mass

```
[<parmindex>] [parm <name> | parmindex <#> | <#>] <mask> [out <file>] [name <set>]
```

parm <name> | parmindex <#> Topology to calculate mass from.

<mask> Atom(s) to calculate total mass for (default all).

[out <file>] File to write total mass to.

[name <set>] If specified, a data set named <set> will be created containing total mass.

Print the total mass of atoms in <mask> (in amu) for selected topology (first loaded topology by default).

37.9.12. molinfo

```
molinfo [parm <name> | parmindex <#> | <#>] <mask> [out <file>]
```

[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.

Default is first loaded topology.

<mask> Mask selecting molecules to print info for.

[out <file>] File to print to (default STDOUT).

Print molecule information for atoms in <mask> for selected topology (first loaded topology by default) with format:

```
#Mol  Natom  #Res Name C [SOLVENT]
```

where #Mol is the molecule number, Natom is the number of atoms in the molecule, #Res and Name are the residue number and residue name of the first residue in the molecule respectively, and C is the chain ID of the first residue. If the molecule is composed on non-consecutive fragments, #Res, Name, and C will be printed for each fragment. SOLVENT will be printed if the molecule is currently considered a solvent molecule.

37.9.13. parm

```
parm <filename> [{[TAG] | name <setname>}]
    [{ nobondsearch |
      [bondsearch <offset>] [searchtype {grid|pairlist}]
    }] [nomolsearch] [renumresidues]
```

<filename> Parameter file to read in; format is auto-detected.

'[TAG]' Optional tag (bounded in brackets) which can be referred to in place of the topology file name in order to simplify references to it (see [37.3.4 on page 690](#) for examples of how to use tags).

[name <setname>] Optional name that can be used to refer to the topology in place of the file name.

[bondsearch <offset>] Optional; when searching for bonds via geometry search (default for Topologies without bond information) add <offset> to distances (default 0.2 Å). Increase this if your system includes unusually long bonds.

[searchtype {grid|pairlist}] Change search algorithm from the default search between residues algorithm:

- grid Uses a grid when searching for bonds between residues. This can find bonds between residues that are not sequential (e.g. disulfide bonds).
- pairlist Uses a pair list to search for bonds between atoms. This can potentially find bonds across periodic boundaries, but is the more experimental of the two.

Advanced Options - Not recommended for general use

[nobondsearch] If specified do not search for bonds via geometry if Topology does not include bond information. May cause some Actions to fail.

[nomolsearch] If specified do not search for molecule information. May cause some Actions to fail.

[renumresidues] If specified, ensure that any residue cannot be part of more than 1 molecule (can occur with e.g. alternate sites). Residues will be renumbered according to molecule information in that case.

Read in parameter file. The file format will be auto-detected. Current formats recognized by cpptraj are listed on [page 730](#). If the file does not contain bond information, cpptraj will attempt to assign bonds based on a simple distance search of atoms within and between residues. The distance cutoff for determining bonds between atoms depends on the elements of the two atoms in question, augmented by <offset>. Molecule information is then determined from bond information.

37.9.13.1. PDB format:

```
[pqr] [readbox] [conect] [noconect] [link] [nolink] [keepaltloc <char>]
```

[pqr] Read charge and radius information from the occupancy and B-factor columns.

[readbox] Read unit cell information from CRYST1 record if present.

[conect] Read CONECT records if present (default).

[noconnect] Do not read in CONECT records from PDB file.
[link] Read LINK records if present.
[nolink] Do not read LINK records if present (default).
[keepaltloc <char>] If specified, only keep alternate atom location IDs matching the specified character <char>.

IMPORTANT NOTES FOR PDB FILES Sometimes PDB files can contain alternate coordinates for the same atom in a residue, e.g.:

ATOM	806	CA	ACYS	A	105	6.460	-34.012	-21.801	0.49	32.23
ATOM	807	CB	ACYS	A	105	6.054	-33.502	-20.415	0.49	35.28
ATOM	808	CA	BCYS	A	105	6.468	-34.015	-21.815	0.51	32.42
ATOM	809	CB	BCYS	A	105	6.025	-33.499	-20.452	0.51	35.38

If this is the case *cpptraj* will print a warning about alternate location IDs being present but will take no other action. Both residues are considered 'CYS' and the mask ':CYS@CA' would select both atom 806 and 808. If desired, a specific location ID can be kept via the **keepaltloc** keyword. If **keepaltloc** is specified, it should also be specified for any **trajin** commands (see 37.10.4.3 on page 747). Residue insertion codes are read in but also not used by the mask parser.

37.9.13.2. Charmm PSF:

[param <file>]
[param <file>] Read CHARMM parameters from given file. Can do multiple times.

37.9.13.3. Gromacs Top

By default *cpptraj* will look for Gromacs topology data (that is not in the same directory) in the directory defined by the GMXDATA environment variable; specifically, it expects things to be in the "\$GMXDATA/top" directory.

37.9.14. parmbbox

parmbbox [**parm** <name> | **parmindex** <#> | <#>] [**nobox**] [**truncoc**]
 [x <xval>] [y <yval>] [z <zval>] [alpha <a>] [beta] [gamma <g>]
[parm <name> | parmindex <#> | <#>] Name/tag or index of topology to modify. Default is first loaded topology.
[nobox] Remove box information.
[truncoc] Set truncated octahedon angles with lengths equal to <xval>.
[x <xval>] Box X length.
[y <yval>] Box Y length.
[z <zval>] Box Z length.
[alpha <a>] Box alpha angle.
[beta] Box beta angle.
[gamma <g>] Box gamma angle.

Modify the box information for specified topology. Overwrites any box information if present with specified values; any that are not specified will remain unchanged. Note that unlike the **box** action this command affect box information immediately. This can be useful for e.g. removing box information from a parm when stripping solvent:

```
parm mol.water.parm7
parmstrip :WAT
parmbbox nobox
parmwrite out strip.mol.nobox.parm7
```

37.9.15. parminfo

```
parminfo [parm <name> | parmindex <#> | <#>] [<mask>]
```

Print a summary of information contained in the specified topology (first loaded topology by default) .

37.9.16. parmstrip

```
parmstrip <mask> [parm <name> | parmindex <#> | <#>]
```

Strip atoms in <mask> from specified topology (by default the first topology loaded). Note that unlike the *strip* Action, this permanently modifies the topology for as long as *cpptraj* is running, so this should not be used if the topology is being used to read or write a trajectory via *trajin/trajout*. This command can be used to quickly created stripped Amber topology files. For example, to strip all residues name WAT from a topology and write a new topology:

```
parm mol.water.parm7
parmstrip :WAT
parmwrite out strip.mol.parm7
```

37.9.17. parmwrite

```
parmwrite out <filename> [{parm <name> | parmindex <#> | <#> | crdset <setname>}
    [<fmt>] [nochamber]
```

<filename> File to write to.

[parm <name> | parmindex <#> | <#>] Topology to write out.

[crdset <setname>] Write topology from specified COORDS data set.

[<fmt>] Format keyword. If not specified the file name extension will be used. Default is Amber Topology.

[nochamber] (Amber topology only) Remove any CHAMBER information from the topology.

Write out specified topology (first topology loaded by default) to <filename> with format <fmt> (Amber topology if not specified). Note that the Amber topology format is the only fully supported format for topology writes.

37.9.17.1. Amber Topology

```
[nochamber] [writeempty] [nopdbinfo]
```

[nochamber] Do not write CHAMBER information to topology (useful for e.g. using topology for visualization with older versions of VMD).

[writeempty] Write Amber tree, join, and rotate info even if not present.

[nopdbinfo] Do not write "PDB" info (e.g. chain IDs, original res #s, etc).

37.9.17.2. Charmm PSF

[oldpsf] [ext]

[oldpsf] Write atom type indices instead of type names (not recommended).

[ext] Use extended format.

37.9.18. printub | ubinfo

printub [parm <name> | parmindex <#> | <#>] [<mask1>] [<mask2>] [out <file>]

[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.

Default is first loaded topology.

[<mask1>] Atoms to print UB info for.

[<mask2>] If specified, UB info must match both masks.

[out <file>] File to print to (default STDOUT).

For specified topology (first by default) either print CHARMM Urey-Bradley info for all atoms in <mask1>, or print info for bonds with first atom in <mask1> and second atom in <mask2>.

37.9.19. resinfo

resinfo [parm <name> | parmindex <#> | <#>] <mask> [short [maxwidth <#res>]]
[out <file>]

[parm <name> | parmindex <#> | <#>] Name/tag or index of topology.

Default is first loaded topology.

<mask> Mask selecting residues to print info for.

[short] Use a short 1 character residue name format

[maxwidth <#res>] Max # of residues to print in one line (default 50).

[out <file>] File to print to (default STDOUT).

Print residue information for atoms in <mask> for selected topology (first loaded topology by default) with format:

```
#Res  Name First  Last Natom #Orig #Mol C
```

where #Res is the residue number, Name is the residue name, First and Last are the first and last atom numbers of the residue, Natom is the total number of atoms in the residue, #Orig is the original residue number (in PDB files), #Mol is the molecule number, and C is the chain ID.

If **short** is specified then residues will be printed out in a condensed format. Each residue name will be shortened to 1 character, and residues are printed out in groups of 10, 5 groups to a line, with each line beginning with a residue number, e.g.

```
> resinfo short 4
1      MGFLAGKKIL ITGLLSNCSI AYGIAMKAMHR EGAELAFTYV GQFKDRVEKL
51     CAEFNPAAVL PCDVISDQEI KDLFVELGKV WDGLDAIVHS IAFAPRDQLE
```

If the 1 character name for a residue is unknown it will be shown as the first letter of the residue name in lower-case.

37.9.20. scaledihedralk

```
scaledihedralk [parm <name> | parmindex <#>] <scale factor> [<mask> [useall]]
```

Scale dihedral force constants for dihedrals selected by <mask> for specified topology. If **useall** is specified all atoms in <mask> must be present to select a dihedral, otherwise any atom in <mask> will select a dihedral.

37.9.21. solvent

```
solvent [parm <name> | parmindex <#> | <#>] { <mask> | none }
```

Set solvent for selected topology (first loaded topology by default) based on <mask>, or set nothing as solvent if **none** is specified.

37.9.22. updateparameters

```
parm <name> | parmindex <#> setname <parm set>
```

```
parm <name> | parmindex <#> Topology to update.
```

```
setname <parm set> Topology or parameter data set containing  
parameters to use.
```

NOTE: This command is provided for convenience only. For editing topology files, ParmEd is a much better alternative.

Update parameters in specified topology with those from <parm set>. <parm set> can either be a parameter set or a topology. If a parameter from <parm set> does not exist in the topology it will be added.

For example, to modify parameters in a topology file named lys.parm7 with those from parameter file kcx.str:

```
# Read Topology to modify
parm lys.parm7
# Read CHARMM parameters
readdata kcx.str as charmmrtfprm name MyParm
# Update parameters in Topology with those from kcx.str
updateparameters parmindex 0 setname MyParm
# Write out the updated Topology
parmwrite out lys.kcx.parm7
```

37.10. Trajectory File Commands

These commands control the reading and writing of trajectory files. There are three trajectory types in *cpptraj*: input, output, and reference. In *cpptraj*, trajectories are always associated with a topology file. If a topology file is not specified, a trajectory file will be associated with the first topology file loaded by default (this is true for both input and output trajectories).

Cpptraj currently understands the following trajectory file formats:

Format	Keyword(s)	Extension	Notes
Amber Trajectory	crd	.crd	Default format if keywords/extensions not recognized.
Amber NetCDF	cdf, netcdf	.nc	No compression.
Amber Restart	restart	.rst7, .rst	
Amber NetCDF Restart	ncrestart, restartnc	.ncrst	
Charmm “DCD” Trajectory	dcd, charmm	.dcd	
Charmm COORdinateS	cor	.cor	
Charmm Restart	charmmres	.res	Read Only
PDB	pdb	.pdb	
Mol2	mol2	.mol2	
Scripps Binpos	binpos	.binpos	
Gromacs TRR	trr	.trr	
Gromacs GRO	gro	.gro	Read Only
Gromacs XTC	xtc	.xtc	
Gromacs TNG	tng	.tng	Read Only
CIF	cif	.cif	Read Only
Tinker ARC	arc	.arc	Read Only
SQM Input	sqm	.sqm	Write Only
SDF	sdf	.sdf	Read Only
XYZ	xyz	.xyz	
Desmond DTR (Anton)	dtr	.dtr	Read Only
LMOD Conflib	conflib	.conflib	Read Only, Detection by extension
MDTraj H5	h5	.h5	Read Only
MDAnalysis H5MD	h5md	.h5md	Read Only

The following trajectory-related commands are available:

Command	Description
ensemble	Set up a trajectory ensemble for reading during a run.
ensemblesize	(MPI only) specify number of members expected in subsequent <i>ensemble</i> commands.
reference	Read in a reference structure.
trajin	Set up a trajectory for reading during a Run.
trajout	Set up an output trajectory or ensemble for writing during a Run.

37.10.1. ensemble

```
ensemble <file0> {[<start>] [<stop> | last] [offset]} | lastframe
    [parm <parmfile / tag> | parminindex <#>]
    [trajnames <file1>,<file2>,...,<fileN>
    [{nosort |
    bycrdidx |
    remlog <remlogfile> [nstlim <nstlim> ntwx <ntwx>]}]]
```

<file0> Lowest replica filename.

[<start>] Frame to begin reading ensemble at (default 1).

[<stop> | last] Frame to stop reading ensemble at; if not specified or 'last' specified, end of trajectories.

[<offset>] Offset for reading in trajectory frames (default 1).

[lastframe] Select only the final frame of the trajectories.

[parm <parmfile>] Topology filename/tag to associate with trajectories (default first topology).

[parminindex <#>] Index of Topology to associate with trajectories (default 0, first topology).

[trajnames <file1>,...,<fileN>] Do not automatically search for additional replica trajectories; use comma-separated list of trajectory names.

[nosort] Do not attempt to sort trajectories. Useful for H-REMD trajectories which are already sorted by replica/Hamiltonian, or collections of MD trajectories.

[bycrdidx] For H-REMD trajectories, sort by coordinate indices stored in trajectory files. This is preferred over sorting via 'remlog'.

[remlog <remlogfile>] For H-REMD trajectories only, use specified REMD log file to sort trajectories by coordinate index (instead of by replica/Hamiltonian).

[nstlim <nstlim> ntwx <ntwx>] If trajectory and REMD log were not written at the same rate, these are the values for nstlim (steps between each exchange) and ntwx (steps between trajectory write) used in the REMD simulation.

Read in and process trajectories as an ensemble. Similar to '*trajin* remdtraj', except instead of processing one frame at a target temperature, process all frames. This means that action and trajout commands apply to the entire ensemble; note however that not all actions currently function in '*ensemble*' mode. For example, to read in a replica ensemble, convert it to temperature trajectories, and calculate a distance at each temperature:

```
parm ala2.99sb.mbondi2.parm7
ensemble rem.crd.000 trajnames rem.crd.001,rem.crd.002,rem.crd.003
trajout temp.crd
distance d1 out d1.ensemble.dat @1 @21
```

This will output 4 temperature trajectories named 'temp.crd.X', where X ranges from 0 to 3 with 0 corresponding to the lowest temperature, and 'd1.ensemble.dat' containing 4 columns, each corresponding to a temperature. If run with MPI, data will be written to separate files named 'd1.ensemble.dat.X', similar to the output trajectories.

Note that in parallel (i.e. MPI) users should specify the *ensemblesize* command prior to *ensemble* in order to improve set up efficiency.

H-REMD trajectories which are typically already sorted by replica/Hamiltonian can be sorted by coordinate index instead with 'bycrdidx' (if the trajectory contains coordinate indices) or by REMD log data specified with the 'remlog' keyword. For example, to sort by coordinate index using a REMD log and write sorted trajectories:

```
parm ../tz2.nhe.parm7
ensemble size 4
ensemble rem.crd.001 remlog rem.log nstlim 1000 ntwx 1000
trajout sorted.remlog.nc
```

To sort by coordinate index when trajectories contain coordinate indices:

```
parm ../tz2.nhe.parm7
ensemble size 4
ensemble rem.crd.001 bycrdidx
trajout sorted.crdidx.nc
```

37.10.2. ensemble size

```
ensemble size <#>
```

This command is MPI only. It is used to set the expected number of members in any subsequent *ensemble* command, which dramatically improves set up efficiency.

37.10.3. reference

```
reference <name> [<frame#>] [<mask>] ([tag]) [lastframe] [crdset]
[parm <parmfile / tag> | parminindex <#>]
```

<name> File name (or COORDS set name if 'crdset' specified) to read in as reference; any trajectory recognized by 'trajin' can be used.

[<frame#>] Frame number to use (default 1).

[<mask>] Only load atoms corresponding to <mask> from reference.

([tag]) Tag to give this reference file, e.g. ``[MyRef]``; BRACKETS MUST BE INCLUDED.

[lastframe] Use last frame of reference.

[crdset] Use for COORDS data set named <name> instead of file.

[parm <parmfile/tag>] Topology filename/tag to associate with reference (default first topology).

[parminindex <#>] Index of Topology to associate with reference (default 0, first topology).

Use specified trajectory as reference coordinates. For trajectories with multiple frames, the first frame is used if a specific frame is not specified. An optional tag can be given (bounded in brackets) which can then be used in place of the name (see [37.3.4 on page 690](#) for examples of how to use tags). If desired, an atom mask can be used to read in only specified atoms from a reference.

Reference coordinates are now considered COORDS data sets and can be used anywhere a COORDS data set could, which allows reference structures to be manipulated once they are loaded. For example, a reference structure could be centered on the origin like so:

```
reference tz2.rst7 [MyRef]
crdaction [MyRef] center origin
```

Note that the 'average' keyword has been deprecated for reference. If desired, an averaged reference COORDS data set can be created from a trajectory using the 'average' command like so:

```
parm myparm.parm7
trajin mytraj.nc
rms first :1-12
average crdset RefAvg
run
rms ToAvg reference :1-12 out ToAvg.dat
```

37.10.4. trajin

```
trajin <filename> {[<start> [<stop> | last] [<offset>]]} | lastframe
[parm <parmfile / tag> | parmindex <#>]
[mdvel <velocities>] [mdfrc <forces>]
[as <format keyword>] [ <Format Options> ]
[ remdtraj {remdtrajtemp <Temperature> | remdtrajidx <idx1,idx2,...>
| remdtrajvalues <value1,value2,...>}
[trajnames <file1>,<file2>,...,<fileN>] ]
```

<filename> Trajectory file to read in.

[<start>] Frame to begin reading at (default 1). If a negative value is given it means ``<start> frames before <stop>''.

[<stop> | last] Frame to stop reading at; if not specified or 'last' specified, end of trajectory.

[<offset>] Offset for reading in trajectory frames (default 1).

[lastframe] Select only the final frame of the trajectory.

[parm <parmfile/tag>] Topology filename/tag to associate with trajectory (default first topology).

[parmindex <#>] Index of Topology to associate with trajectory (default 0, first topology).

[mdvel <velocities>] Use velocities from specified file.

[mdfrc <forces>] Use forces from specified file.

[as <format keyword>] Force file to be read as specified format; overrides file autodetection.

[<Format Options>] See below.

[remdtraj] Read <filename> as the first replica in a group of replica trajectories.

remdtrajtemp <Temperature> | remdtrajidx <idx1,idx2,...> Use frames at <Temperature> (for temperature replica trajectories) or index <idx1,idx2,...> (for Hamiltonian replica trajectories); For Multidimensional REMD simulations, multiple values are comma-separated.

remdtrajvalues <value1,value2,...> Use frames at <value1,value2,...> (for Multidimensional REMD trajectories). Each value may correspond to either temperature, pH, Redox Potential or Hamiltonian index. The values need to be entered in the same order as the dimensions in the Multidimensional REMD simulation. For example, for T,pH-REMD value1 would correspond to a temperature and value2 to a pH. In the command, the values are comma-separated.

[trajnames <file1>,...,<fileN>] Do not automatically search for additional replica trajectories; use comma-separated list of trajectory names.

Read in trajectory specified by filename. See page 742 for currently recognized trajectory file formats. If just the <start> argument is given, all frames from <start> to the last frame of the trajectory will be read. To read in a trajectory with offsets where the last frame # is not known, specify the **last** keyword instead of a <stop> argument, e.g.

```
trajin Test1.crd 10 last 2
```

This will process Test1.crd from frame 10 to the last frame, skipping by 2 frames. To explicitly select only the last frame, specify the **lastframe** keyword:

```
trajin Test1.crd lastframe
```

Here is an example of loading in multiple trajectories which have different topology files:

```
parm top0.parm7
parm top1.parm7
parm top2.parm7 [top2]
parm top3.parm7
trajin Test0.crd
trajin Test1.crd parm top1.parm7
trajin Test2.crd parm [top2]
trajin Test3.crd parminde 3
```

Test0.crd is associated with top0.parm7; since no parm was specified it defaulted to the first parm read in. Test1.crd was associated with top1.parm7 by filename, Test2.crd was associated with top2.parm7 by its tag, and finally Test3.crd was associated with top3.parm7 by its index (based on the order it was read in).

Replica Trajectory Processing

If the **remdtraj** keyword is specified the trajectory is treated as belonging to the lowest # replica of a group of REMD trajectories. The remaining replicas can be either automatically detected by following a naming convention of <REMDFILENAME>.X, where X is the replica number, or explicitly specified in a comma-separated list following the **trajnames** keyword. All trajectories will be processed at the same time, but only frames with a temperature matching the one specified by **remdtrajtemp** or **remdtrajidx** will be processed. For example, to process replica trajectories rem.001, rem.002, rem.003, and rem.004, grabbing only the frames at temperature 300.0 (assuming that this is a temperature in the ensemble):

```
trajin rem.001 remdtraj remdtrajtemp 300
```

or

```
trajin rem.001 remdtraj remdtrajtemp 300 trajnames rem.002,rem.003,rem.004
```

Note that the **remdout** keyword is deprecated. For this functionality see the **ensemble** keyword.

37.10.4.1. Options for Amber NetCDF, Amber NC Restart, Amber Restart:

```
[usevelascoords] [usefrcascoords]
```

usevelascoords Read in velocities in place of coordinates if present.

usefrcascoords Read in forces in place of coordinates if present.

37.10.4.2. Options for CHARMM DCD:

```
[{shape | namdcell | charmmcell}]
shape Force reading of box info as CHARMM shape matrix (XX XY YY XZ
  YZ ZZ) .
namdcell Force reading of box info as NAMD unit cell (X cos(g) Y
  cos(b) cos(a) Z) .
charmmcell Force reading of box info as old CHARMM unit cell (X Y Z
  a b g) .
```

Note that CHARMM trajectories can have unit cell data stored in one of two ways. Older versions (<22) of charmm store the 3x lengths (X Y Z, in Angstroms) and 3x angles (alpha beta gamma, in degrees). Newer versions (>=22) store elements of the symmetric shape matrix (XX, XY, YY, XZ, YZ, ZZ). CPPTRAJ will attempt to automatically detect which type of parameters are present, but this can be overridden with the **shape** and **charmmcell** keywords. The **namdcell** keyword is provided for compatibility with DCD trajectories produced by VMD/NAMD, but note these trajectories only store a version of the shape matrix that is correct if the unit cell is also X-aligned.

37.10.4.3. Options for PDB files:

```
[keepaltloc <char>]
[keepaltloc <char>] If specified, only keep alternate atom location
  IDs matching the specified character <char> .
```

Note that if **keepaltloc** is specified, the associated topology should not have alternate location IDs, i.e. if the topology is from a PDB the **keepaltloc** keyword may need to be used with the **parm** command (see [37.9.13.1 on page 737](#)).

37.10.5. trajout

```
trajout <filename> [<format>] [append] [nobox] [novelocity]
  [notemperature] [notime] [noforce] [noreplicadim]
  [parm <parmfile> | parminindex <#>] [onlyframes <range>] [title <title>]
  [onlymembers <memberlist>]
  [start <start>] [stop <stop>] [offset <offset>]
  [ <Format Options> ]
```

<filename> Trajectory file to write to.

[<format>] Keyword specifying output format (see Table on [page 742](#)).
If not specified format will be determined from extension,
otherwise default to Amber trajectory.

[append] If <filename> exists, frames will be appended to
<filename>.

[nobox] Do not write box coordinates to trajectory.

[novelocity] Do not write velocities to trajectory.

[notemperature] Do not write temperature to trajectory.

[notime] Do not write time to trajectory.

[noreplicadim] Do not write replica dimensions to trajectory.

[parm <parmfile>] Topology filename/tag to associate with trajectory
(default first topology).

[parminindex <#>] Index of Topology to associate with trajectory
(default 0, first topology).

[onlyframes <range>] Write only the specified input frames to <filename>.

[title <title>] Output trajectory title.

[onlymembers <memberlist>] Ensemble processing only; only write from specified members (starting from 0).

[start <start>] Begin output at frame <start> (1 by default).

[stop <stop>] End output at frame <stop> (last frame by default).

[offset <offset>] Skip <offset> frames between each output (1 by default).

During a run, write frames to trajectory specified by filename in specified file format (Amber trajectory if none specified) after all Action processing has occurred. To write out trajectories within the Action queue see the [outraj Action \(37.11.59 on page 818\)](#). See [page 742](#) for currently recognized output trajectory formats and their associated keyword(s). Note that now the file type can be determined from the output extension if not specified by a keyword. Multiple output trajectories of any format can be specified.

Frames will be written to the output trajectory when the parameter file being processed matches the parameter file the output trajectory was set up with. So given the input:

```
parm top0.parm7
parm top1.parm7 [top1]
trajin input0.crd
trajin input1.crd parm [top1]
trajout output.crd parm [top1]
```

only frames read in from input1.crd (which is associated with top1.parm7) will be written to output.crd. The trajectory input0.crd is associated with top0.parm7; since no output trajectory is associated with top0.parm7 no frames will be written when processing top0.parm7/input0.crd.

If **onlyframes** is specified, only input frames matching the specified range will be written out. For example, given the input:

```
trajin input.crd 1 10
trajout output.crd onlyframes 2,5-7
```

only frames 2, 5, 6, and 7 from input.crd will be written to output.crd.

Cell not X-aligned Warning

Certain Actions (e.g. *align*, *rms*, *principal*, etc.) can rotate the unit cell vectors (i.e. the box) if they are present. Some trajectory formats do not support writing out box coordinates if the unit cell is not “X-aligned”; in other words, if the unit cell “A” vector is not aligned with the coordinate X-axis and the “B” vector is not in the X-Y plane. If this is the case, the following warnings may appear:

```
Warning: Unit cell is not X-aligned. Box cannot be properly stored as <format>.
Warning: Set <#>; unit cell is not X-aligned. Box cannot be properly stored as <format>.
```

This means that the frame will be written with the X-aligned unit cell instead of the actual unit cell. Imaging will not be possible with a trajectory written this way. Currently the only trajectory formats that support writing non-X-aligned cells are the Gromacs TRR and XTC formats.

If unit cell information is no longer needed, it can be removed (via e.g. the *box* action, the *strip* action with the ‘*noibox*’ keyword, etc.) to prevent these warnings from triggering.

37.10.5.1. Options for pdb format

```

[dumpq | parse | vdw] [pdbres] [pdbatom]
[pdbv3] [teradvance] [terbyres | pdbter | noter]
[model | multi] [chainid <ID>] [sg <group>]
[include_ep] [conect] [conectmode <m>] [keepext] [usecol21]
[bfacdefault <#>] [occddefault <#>]
[bfacdata <set>] [occddata <set>] [bfacbyres] [occbbyres]
[bfacscale] [occscale] [bfacmax <max>] [occmx <max>]
[adpdata <set>]

```

dumpq PQR format; write charges (in units of e-) and GB radii to occupancy and B-factor columns respectively.

parse PQR format; write charges and PARSE radii to occupancy/B-factor columns.

vdw PQR format; write charges and vdW radii to occupancy/B-factor columns.

pdbres Use PDB V3 residue names. Will write a default chain ID ('Z') for each residue if the corresponding topology does not have chain ID information.

pdbatom Use PDB V3 atom names.

pdbv3 Use PDB V3 residue/atom names. Same as specifying 'pdbres' and 'pdbatom'.

topresnum Use topology residue numbers; otherwise use original residue numbers.

teradvance Increment record (atom) number for TER records (not done by default).

terbyres Print TER cards based on residue sequence instead of molecules.

pdbter Print TER cards according to original PDB TER (if available).

noter Do not write TER cards.

model (Default) Frames will be written to a single PDB file separated by MODEL/ENDMDL keywords.

multi Each frame will be written to a separate file with the frame # appended to <filename>.

chainid <ID> Write PDB file with chain ID <ID>.

sg <group> Space group for CRYST1 record; only used if box coordinates written.

include_ep Include extra points.

conect Write CONECT records for all bonds.

conectmode <m> Write CONECT records for <m>='all' (all bonds), 'het' (HETATM only), 'none' (no CONECT).

keepext Keep filename extension; write '<name>.<num>.<ext>' instead (implies 'multi').

usecol21 Use column 21 for 4-letter residue names.

bfacdefault <#> Default value to use in B-factor column (default 0.0).

occdefault <#> Default value to use in occupancy column (default 1.0).

bfacdata <set> Use data in <set> for B-factor column.

occddata <set> Use data in <set> for occupancy column.

bfacbyres If specified assume X values in B-factor data set are residue numbers.

occbyses If specified assume X values in occupancy data set are residue numbers.

bfacscale If specified scale values in B-factor column between 0 and <bfacmax>.

occscale If specified scale values in occupancy column between 0 and <occmx>.

bfacmax <max> Max value for bfacscale.

occmx <max> Max value for occscale.

adpdata <set> Use data in <set> (e.g. from the *atomicfluct* command, on page 759) for anisotropic B-factors.

37.10.5.2. Options for Amber ASCII format:

[remdtraj] [highprecision] [mdvel|mdfrc]

remdtraj Write REMD header to trajectory that includes temperature: 'REMD <Replica> <Step> <TotalSteps> <Temperature>'. Since *cpptraj* has no concept of replica number, 0 is printed for <Replica>. <Step> and <TotalSteps> are set to the current frame #.

highprecision (EXPERT USE ONLY) Write with 8.6 precision instead of 8.3. Note that since the width does not change, the precision of large coords may be lower than 6.

mdvel Write velocities instead of coordinates.

mdfrc Write forces instead of coordinates.

37.10.5.3. Options for Amber NetCDF format:

[remdtraj] [mdvel] [mdfrc] [mdcrd]

remdtraj Write replica temperature to trajectory.

mdvel Write only velocity information in trajectory.

mdfrc Write only force information in trajectory.

mdcrd Write coordinates to trajectory (only required with mdvel/mdfrc).

hdf5 Create file as NetCDF4/HDF5 instead of NetCDF4 (classic).

compress Use compression in NetCDF4/HDF5 file.

icompress Use lossy compression in NetCDF4/HDF5 file via conversion to integers. [769]

37.10.5.4. Options for Amber Restart/NetCDF Restart format:

```
[remdtraj] [novelocity] [notime] [time0 <initial time>] [dt <timestep>] [keepext]
```

remdtraj Write replica temperature to restart. Note that this will automatically include time in the restart file (see the **time0** keyword).

time0 <initial time> Time for first frame (default 1.0).

dt <timestep> Time step between frames (default 1.0). Time is calculated as $t = (\text{time0} + \text{frame}) * \text{dt}$.

keepext Keep filename extension; write '**<name>.<num>.<ext>**' instead.

37.10.5.5. Options for CHARMM COORdinateS:

```
[keepext] [ext] [segid <segid>] [segmask <mask> <segid> ...]
```

keepext Keep filename extension; write '**<name>.<num>.<ext>**'

ext Use 'extended' format (default when > 99999 atoms).

segid <segid> Use <segid> as segment ID for all atoms.

segmask <mask> <segid> Use <segid> as segment ID for atoms selected by <mask>. Can be specified more than once.

37.10.5.6. Options for CHARMM DCD:

```
[x64] [ucell] [veltraj] [{shape | namdcell | charmmcell}]
```

x64 Use 8 byte block size (default 4 bytes).

veltraj Write velocity trajectory instead of coordinates.

dt Set trajectory time step in ps.

nstep # steps between frames.

step0 Initial step.

shape Force writing box info as CHARMM shape matrix (XX XY YY XZ YZ ZZ).

namdcell Force writing box info as NAMD unit cell (X cos(g) Y cos(b) cos(a) Z).

charmmcell Force writing box info as old CHARMM unit cell (X Y Z a b g).

Note that by default CPPTRAJ will try to write the symmetric shape matrix if box information is present. If this is not possible, CPPTRAJ will fall back to writing unit cell parameters (lengths and angles) as long as the cell is X-aligned. For more information see [37.10.4.2 on page 747](#).

37.10.5.7. Options for GROMACS TRX/XTC format:

```
[dt <time step>]
```

dt Time step to multiply set numbers by (default 1.0). Ignored if time already present.

Note: these formats can write rotated (i.e. non-X-aligned) unit cells.

37.10.5.8. Options for mol2 format:

```
[single | multi] [sybyltype] [sybylatom <file>] [sybylbond <file>] [keepext]
```

single (Default) Frames will be written to a single Mol2 file separated by MOLECULE keywords.

multi Each frame will be written to a separate file with the frame # appended to <filename>.

sybyltype Convert Amber atom types (if present) to SYBYL types. Requires \$AMBERHOME is set.

sybylatom File containing Amber to SYBYL atom type correspondance (optional).

sybylbond File containing Amber to SYBYL bond type correspondance (optional).

keepext Keep filename extension; write '<name>.<num>.<ext>' instead (implies 'multi').

37.10.5.9. Options for SQM input format:

```
[charge <c>]
```

charge <c> Set total integer charge. If not specified it will be calculated from atomic charges.

37.10.5.10. Options for XYZ format:

```
[ftype {namexyz|atomxyz|xyz}] [titletype {none|single|perframe}] [width <#>] [prec <#>]
```

ftype {atomxyz|xyz} Choose either 'NAME X Y Z' (default), 'ATOM X Y Z', or 'X Y Z' output format. 'namexyz' format is the standard XYZ format, where each frame is preceded by the number of atoms and a comment. The comment written by CPPTRAJ will include the set number and box information (if present).

titletype {none|single|perframe} No title, one title (default), or title before every frame. Only applies if not 'namexyz'.

width <#> Output format width.

prec <#> Output format precision.

37.11. Action Commands

Actions in *cpptraj* operate on frames read in by the *trajin* or *ensemble* commands one at a time and extract derived data, modify the coordinates/topology in some way, or both. Most Actions in *cpptraj* function exactly the way they do in *ptraj* and are backwards-compatible. Some Action commands in *cpptraj* have extra functionality compared to *ptraj* (such as the per-residue RMSD function of the *rmsd* Action, or the ability to write out stripped topologies for visualization in the *strip* Action), while other Actions produce slightly different output (like the *hbond/secstruct* Actions).

Unlike some other command types, when an Action command is issued it is by default added to the Action queue and is not executed until trajectory processing is started (e.g. by a *run* or *go* command). However, Actions can be executed immediately on COORDS data sets via the *crdaction* command ([37.7.3 on page 705](#)).

When a frame is modified by an Action, it is modified for every Action that follows them during trajectory processing. For example, given a solvated system with water residues named WAT and the following Action commands:

```
rmsd R1 first :WAT out water-rmsd.dat
strip :WAT
rmsd R2 first :WAT out water-rmsd-2.dat
```

the first *rms* command will be valid, but the second *rms* command will not since all residues named WAT are removed from the state by the *strip* command.

Note that for commands which can use a reference mask as well as a target mask (e.g. *rms*, *drmsd*, *symmrmsd*, etc.) there must be a 1 to 1 correspondence between the atoms in each mask, i.e. the *number of atoms and the ordering of selected atoms must be the same*.

The following Actions are available. If an Action may modify coordinate/topology information for subsequent Actions it is denoted with an X in the **Mod** column.

Command	Description	Mod
addatom	Temporarily add an atom to the system.	X
align	Align structure to a reference.	X
angle	Calculate the angle between three points.	
areapermol	Calculate area per molecule for molecules in a specified plane.	
atomiccorr	Calculate average correlation between motions of specified atoms.	
atomicfluct, rmsf	Calculate root mean square fluctuation of specified atoms/residues.	
atommap	Attempt to create a map between atoms in molecules with different atom ordering.	X
autoimage	Automatically re-image coordinates.	X
average	Calculate average structure.	
avgbox	Calculate average unit cell (box), primarily for unwrapping NPT trajectories.	
bounds	Calculate the min/max coordinates for specified atoms. Can be used to create grid data sets.	
box	Set or overwrite box information for frames.	
center	Center specified coordinates to box center or onto reference structure.	X
check, checkoverlap, checkstructure	Check for bad atomic overlaps or bond lengths. Can be used to skip corrupted frames.	
checkchirality	Report chirality around alpha carbons in amino acids (L, D).	
closest, closestwaters	Retain only the specified number of solvent molecules closest to specified solute.	X
clusterdihedral	Assign frames into clusters based on binning of backbone dihedral angles in amino acids.	
contacts	Older version of <i>nativecontacts</i> , retained for backwards compatibility.	

createcrd	Create a COORDS data set from input frames.	
createreervoir	Create a structure reservoir for use with reservoir REMD simulations.	
density	Calculate density along a coordinate.	
diffusion	Calculate translational diffusion of molecules.	
dihedral	Calculate the dihedral angle using four points.	
dihrms dihedralrms	Calculate the RMSD of dihedrals to dihedrals in a reference structure.	
dipole	Bin dipoles of solvent molecules in 3D grid. Not well tested, may be obsolete.	
distance	Calculate the distance between two points.	
drms, drmsd	Calculate the RMSD of distance pairs within selected atoms.	
dssp, secstruct	Calculate secondary structure content using the DSSP algorithm	
enedecomp	Perform per-atom energy decomposition.	
energy	Calculate simple bond, angle, dihedral, and non-bonded energy terms (no PME).	
esander	Calculate energies using via SANDER; requires compilation with the SANDER API.	
filter	Filter frames for subsequent Actions using data sets and user defined criteria.	
fixatomorder	Fix atom ordering so that all atoms in molecules are sequential.	X
fiximagedbonds	Fix bonds which have been split across periodic boundaries by imaging.	
gist	Perform grid inhomogenous solvation theory.	
grid	Bin selected atoms on a 3D grid.	
hbond	Calculate hydrogen bonds using geometric criteria.	
image	Re-image coordinates. The <i>autoimage</i> command typically provides better results.	X
jcoupling	Calculate J-coupling values from specified dihedral angles.	
keep	Keep specified atoms in system.	X
lessplit	Split/average frames from LES trajectories.	
lie	Calculate linear interaction energy between user-specified ligand and surroundings.	
lipidorder	Calculate order parameters for lipids in planar membranes.	
lipidscd	Calculate lipid order parameters SCD ($ \langle P_2 \rangle $) for lipid chains. Automatically identifies lipids.	
makestructure	Modify structure by applying dihedral values to specified residues.	X
mask	Print the results of selection by specified atom mask. Good for distance-based masks.	

matrix	Calculate a matrix of the specified type from input coordinates.	
mindist/maxdist	Calculate the minimum or maximum distance between pairs of atoms/residues/molecules.	
minimage	Calculate minimum non-self imaged distance between atoms in specified masks.	
molsurf	Calculate Connolly surface area of specified atoms. Cannot do partial surface areas.	
multidihedral	Calculate multiple dihedral angles of specified/given types.	
multivector	Calculate multiple vectors between specified atoms.	
nastruct	Perform nucelic acid structure analysis.	
nativecontacts	Calculate native contacts within a region or between two regions using a given reference. Can also be used to get min/max distances between groups of atoms.	
outtraj	Write frames to a trajectory file within a list of Actions.	
pairdist	Calculate pair distribution function.	
pairwise	Calculate pair-wise non-bonded energies.	
principal	Calculate and optionally align system along principal axes.	X
projection	Project coordinates along given eigenvectors.	
pucker	Calculate ring pucker using five or six points.	
radgyr, rog	Calculate radius of gyration (and optionally tensor) for specified atoms.	
radial, rdf	Calculate radial distribution function.	
randomizeions	Swap specified ions with randomly selected solvent molecules.	X
remap	Re-map atoms according to a given data set.	X
replicatecell	Replicate unit cell in specified (or all) directions for specfied atoms and write to trajectory.	
rms, rmsd	Perform best fit of coordinates to reference and calculate coordinate RMSD. Fitting can be disabled.	X
rotate	Rotate the system around X/Y/Z axes, a specified axis, or via given rotation matrices.	X
runavg, runningaverage	Calculate the running average of coordinates over specified window size.	X
scale	Scale coordinates in X/Y/Z directions by specified factors.	X
setvelocity	Set velocities for specified atoms using Maxwellian distribution based on given temperature.	
spam	SPAM method for estimating relative free energies of waters in hydration shell around proteins.	X

stfcdiffusion	Alternative translational diffusion calculation which can calculate diffusion in specified regions.	
strip	Remove specified atoms from the system.	X
surf	Calculate the LCPO surface area of specified atoms. Can do partial surface areas.	
symmrmsd	Calculate symmetry-corrected RMSD.	X
temperature	Calculate system temperature using velocities of specified atoms.	
time	Add/remove/modify time information in frames.	X
tordiff	Calculate diffusion using the toroidal-view-preserving scheme.	
trans, translate	Translate specified atoms by specified amounts in X/Y/Z directions.	X
unstrip	Undo all previous <i>strip</i> Action commands.	
unwrap	Reverse of <i>image</i> ; unwrap selected atoms so they have continuous trajectories.	X
vector	Calculate various types of vector quantities.	
velocityautocorr	Calculate velocity autocorrelation function.	
volmap	Create volumetric map for specified coordinates; similar to <i>grid</i> but takes into account atomic radii. Similar to VMD <i>volmap</i> .	
volume	Calculate unit cell volume.	
watershell	Calculate the number of waters in the first and second solvation shells based on distance criteria.	
xtalsymm	Re-image coordinates based on crystal space group symmetry operations and asymmetric unit volume.	X

37.11.1. addatom

```
addatom aname <name> [elt <element>] [rname <res name>]
      [xyz <X> <Y> <Z>] [mass <mass>] [charge <charge>]
      [outprefix <prefix>] [nobox] [parmout <filename>]
      [parmopts <comma-separated-list>]
```

aname <name> New atom name. Required.

[elt <element>] One or two character element string (e.g. 'H' is hydrogen etc.). Default is 'H'.

[rname <res name>] New residue name for the new atom. Default is 'TMP'.

[xyz <X> <Y> <Z>] Cartesian XYZ coordinates of the new atom. Default is 0 0 0.

[mass <mass>] Mass of the new atom. Default is based off of the element type.

[charge <charge>] Charge of the new atom. Default is 0.

[outprefix <prefix>] Write out modified topology file with name '**<prefix>.<Original Topology Name>**'.
[nobox] Remove any box information from the modified topology.
[parmout <file>] Write modified topology to file with name **<file>**.
[parmopts <list>] Options for writing modified topology file.

Temporarily add an atom to the system. This is mostly useful for adding a placeholder “dummy” atom for subsequent actions to use. For example, this can be used in conjunction with the '*mask*' command in order to get a list of atoms within a certain distance of a specified point in space. For example, the following input gets a list of atoms within 3.0 Angstroms of coordinates 1, 1, 1:

```
parm tz2.pdb
trajin tz2.pdb 1 1
addatom aname TEMP rname TMP elt H xyz 1 1 1
mask :TMP<@3.0&!:TMP out tz2.mask.dat name TZ2
```

37.11.2. align

```
align <mask> [<refmask>] [move <mask>] [mass]
  [ first | reference | ref <name> | reindex <#> | previous |
    reftraj <name> [parm <name> | parmindex <#>] ]
```

<mask> Target atoms to fit.
[<refmask>] Reference atoms to fit (default is target mask).
[move <mask>] Atoms to move when aligning (default is target mask).
[mass] Mass-weight the fit.

Reference keywords:

first Use the first trajectory frame processed as reference.
reference Use the first previously read in reference structure (reindex 0).
ref <name> Use previously read in reference structure specified by filename/tag.
reindex <#> Use previously read in reference structure specified by <#> (based on order read in).
previous Use frame prior to current frame as reference.
reftraj <name> Use frames from COORDS set <name> or read in from trajectory file <name> as references. Each frame from <name> is used in turn, so that frame 1 is compared to frame 1 from <name>, frame 2 is compared to frame 2 from <name> and so on. If <trajname> runs out of frames before processing is complete, the last frame of <trajname> continues to be used as the reference.
parm <parmname> | parmindex <#> If reftraj specifies a trajectory file, associate it with specified topology; if not specified the first topology is used.

Align structure using specified <mask> onto reference. If 'move' is specified, only move atoms in the move mask.

37.11.3. angle

```
angle [<dataset name>] <mask1> <mask2> <mask3> [out <filename>] [mass]
```

[<dataset name>] Output data set name.

<maskX> Three atom masks selecting atom(s) to calculate angle for.

[out <filename>] Output file name.

[mass] Use center of mass of atoms in <maskX> instead of geometric center.

Calculate angle (in degrees) between atoms in <mask1>, <mask2>, and <mask3>. For example, to calculate the angle between the first three atoms in the system:

```
angle A123 @1 @2 @3 out A123.agr
```

37.11.4. areapermol

```
areapermol [<name>] {[<mask1>] [nlayers <#>] | nmols <#>} [out <filename>]
           [{xy | xz | yz}]
```

[<name>] Data set name.

[<mask1>] Atom mask for selecting molecules. If any atom in a molecule is selected the whole molecule is selected.

[nlayers <#>] Number of layers of molecules. Total number of molecules used will be # molecules divided by # layers.

[nmols <#>] If <mask1> is not specified, the number of molecules to use when calculating area per molecule.

[out <filename>] Output file name.

[{xy|xz|yz}] Cross-section of box to calculate area of. Default is X-Y.

Calculate area per molecule as Area / # molecules. The area is determined from the specified cross-section of the box (X-Y by default). Currently the calculation is only guaranteed to work properly with orthorhombic unit cells. For example, to get the area per molecule of residues named "OL" which are arranged in 2 layers:

```
areapermol OL.area :OL nlayers 2 out apm.dat
```

37.11.5. atomiccorr

```
atomiccorr [<mask>] out <filename> [cut <cutoff>] [min <min spacing>]
           [byatom | byres]
```

<mask> Atoms to calculate motion vectors for.

out <filename> File to write results to.

cut <cutoff> Only print correlations with absolute value greater than <cutoff>.

min <min spacing> Only calculate correlations for motion vectors spaced <min spacing> apart.

byatom Default; calculate atomic motion vectors.

byres Calculate motion vectors for entire residues (selected atoms in residues only).

Calculate average correlations between the motion of atoms in <mask>. For each frame, a motion vector is calculated for each selected atom from its previous position to its current position. For each pair of motion vectors V_a and V_b , the average correlation between those vectors is calculated as the average of the dot product of those vectors over all N frames.

$$\text{AvgCorr}(a,b) = \frac{\sum V_a(i) \cdot V_b(i)}{N}$$

The value of AvgCorr can range from 1.0 (correlated) to 0.0 (no correlation) to -1.0 (anti-correlated). For example, to calculate the correlation of motion vectors between residues 1 to 13, writing to a Gnuplot-readable formatted file:

```
atomiccorr :1-13 out acorr.gnu byres
```

37.11.6. atomicfluct | rmsf

```
atomicfluct [<name>] [out <filename>] [<mask>] [byres [pdbres] | byatom | bymask]
           [bfactor] [calcadp [adpout <file>]]
           [start <start>] [stop <stop>] [offset <offset>]
```

<name> Output data set name.

out <filename> Write data to file named <filename>

[<mask>] Calculate fluctuations for atoms in <mask> (all if not specified).

byres [pdbres] Output the average (mass-weighted) fluctuation by residue. If 'pdbres' is specified, the original residue numbering will be used.

bymask Output the average (mass-weighted) fluctuation for all atoms in <mask>.

byatom (default) Output the fluctuation by atom.

[bfactor] Calculate atomic positional fluctuations squared and weight by $\frac{8}{3}\pi^2$; this is similar but not necessarily equivalent to the calculation of crystallographic B-factors.

[calcadp [adpout <file>]] Calculate anisotropic displacement parameters and optionally output them to <file>.

[<start>] Frame to begin calculation at (default 1).

[<stop>] Frame to end calculation at (default last).

[<offset>] Frames to skip between calculations (default 1).

DataSets created

<name> Hold atomic fluctuations.

<name>[ADP] Hold anisotropic displacement parameters if 'calcadp' specified.

Compute the atomic positional fluctuations (also referred to as root-mean-square fluctuations, RMSF) for atoms specified in the <mask>. The RMSF of a given atom i is calculated as:

$$\text{RMSF}_i = \sqrt{\langle (x_i - \langle x_i \rangle)^2 \rangle}$$

where x denotes atomic positions and the averages are over all input frames.

Note that RMS fitting is not done implicitly. If you want fluctuations without rotations or translations (for example to the average structure), perform an RMS fit to the average structure (best) or the first structure (see *rmsd*) prior to this calculation. The units are (Å) for RMSF or $\text{\AA}^2 \times \frac{8}{3}\pi^2$ if **bfactor** is specified.

If **byres** or **bymask** are specified, the mass-weighted average of atomic fluctuations of each atom for either each residue or the entire mask will be calculated respectively:

$$\langle Fluct \rangle = \frac{\sum AtomFluct_i * Mass_i}{\sum Mass_i}$$

If **calcadp** is specified, anisotropic displacement factors for atoms will be calculated and written to the file specified by **adpout** (or STDOUT if not specified) using PDB ANISOU record format. The displacement factors will be saved to a data set. Note that **calcadp** automatically implies **bfactor**.

With *cpptraj* it is possible to perform coordinate averaging, the fit to average coordinates, and the atomic fluctuation calculation in a single execution like so:

```
parm myparm.parm7
trajin mytrajectory.crd
rms first
average crdset MyAvg
run
rms ref MyAvg
atomicfluct out fluct.agr
```

To write the mass-weighted B-factors for the protein backbone atoms C, CA, and N, averaged by residue use the command:

```
atomicfluct out back.agr @C,CA,N byres bfactor
```

To write the RMSF or atomic positional fluctuations of the same atoms, use the command:

```
atomicfluct out backbone-atoms.agr @C,CA,N
```

To write a PDB of averaged coordinates (after fitting to the first frame) with both B-factors and anisotropic temperature factors:

```
parm myparm.parm7
trajin mytraj.nc
rms first
average crdset MyAvg
atomicfluct MyFluct calcadp
run
crdout MyAvg mypdb.pdb adpdata MyFluct[ADP] bfacdata MyFluct
```

37.11.7. atommap

```
atommap <target> <reference> [mapout <filename>] [maponly]
[rmsfit [ rmsout <rmsout> ]]
[changenames] [chiralimpcut <cut>]
```

<target> Reference structure whose atoms will be remapped.

<reference> Reference structure that **<target>** should be mapped to.

mapout <filename> Write atom map to **<filename>** with format:

```
TargetAtomNumber TargetAtomName ReferenceAtomNumber
ReferenceAtomName
```

Target atoms that cannot be mapped to a reference atom are denoted ``|''.

maponly Write atom map but do not reorder atoms.

rmsfit Any input frames using the same topology as <target> will be RMS fit to <reference> using whatever atoms could be mapped.

rmsout <rmsout> If rmsfit specified, write resulting RMSDs to <rmsout>.

changenames If specified, change names of mapped atoms in <target> to match those in <reference>.

chiralimpcut <cut> sets the improper dihedral angle cutoff for determining mapping via chirality (default 10 deg.).

Attempt to map the atoms of <target> to those of <reference> based on structural similarity. This is useful e.g. when there are two files containing the same structure but with different atom names or atom ordering. Both <target> and <reference> need to have been read in with a previous *reference* command. The state will then be modified so that any trajectory read in with the same parameter file as <target> will have its atoms mapped (i.e. reordered) to match those of <reference>. If the number of atoms that can be mapped in <target> are less than those in <reference>, the reference structure specified by <reference> will be modified to include only mapped atoms; this is useful if for example the reference structure is protonated with respect to the target. The **rmsfit** keyword is useful in cases where the atom mapping will not be complete (e.g. two ligands with the same scaffold but different substituents). If **changenames** is specified, in addition to remapping, the target atom names will be changed to match the reference atom names.

Part of the mapping process involves mapping atoms bonded to chiral centers, which in turn involves calculating so-called “improper” dihedral angles to determine the orientation of the bonded atoms. By default the code assumes the orientation of the atoms around the chiral centers are fairly close, hence there is a small improper dihedral angle cutoff of 10 degrees. However, this can be increased via the **chiralimpcut** keyword to handle cases where one structure is distorted with respect to the other.

For example, say you have the same ligand structure in two files, Ref.mol2 and Lig.mol2, but the atom ordering in each file is different. To map the atoms in Lig.mol2 onto those of Ref.mol2 so that Lig.mol2 has the same ordering as Ref.mol2:

```
parm Lig.mol2
reference Lig.mol2
parm Ref.mol2
reference Ref.mol2 parminindex 1
atommap Lig.mol2 Ref.mol2 mapout atommap.dat
trajin Lig.mol2
trajout Lig.reordered.mol2 mol2
```

37.11.8. autoimage

```
autoimage [{<mask> | anchor <mask> [fixed <mask>] [mobile <mask>]]}
          [origin] [firstatom] [{familiar|triclinic}] [moveanchor]
          [mode {bydist|byvec}]
```

[<mask> | anchor <mask>] Atoms to image around; this is the region that will be centered. Default is the entire first molecule.

[fixed <mask>] Molecules that should remain ‘fixed’ to the anchor region; default is all non-ion/non-solvent molecules.

[mobile <mask>] Molecules that can be freely imaged; default is all ion/solvent molecules.

[origin] Center anchor region at the origin; if not specified, center at box center.

[firstatom] Image based on molecule first atom; default is to image by molecule center of mass.

[familiar] Image to familiar truncated-octahedral shape; this is on by default if the original cell is truncated octahedron.

[triclinic] Force general triclinic imaging.

[moveanchor] When treating ``fixed`` molecules, the anchor point will be set to the previous "fixed" molecule; this is only expected to work well when "fixed" molecules that are sequential are also geometrically close. Most useful in more condensed systems like those containing membranes.

[mode {bydist|byvec}] How to treat 'fixed' molecules.

bydist The default. 'Fixed' molecules will use the image closest to the "anchor" molecule.

byvec Fixed molecules will use the image closest to their orientation with respect to the anchor in the first frame. May work better than 'bydist' for systems containing membranes.

Automatically center and image (by molecule) a trajectory with periodic boundaries. For most cases just specifying **'autoimage'** alone is sufficient. The atoms of the **'anchor'** region (default the entire first molecule) will be centered; all **'fixed'** molecules will be imaged only if imaging brings them closer to the **'anchor'** molecule (default for **'fixed'** molecules is all non-solvent non-ion molecules). All other molecules (referred to as **'mobile'**) will be imaged freely.

The autoimage command works for the majority of systems; however, for very densely packed systems the default anchor (entire first molecule) may not be appropriate. In these cases, it is recommended to choose as the anchor a small region which should lie near the center of your system. For example, in a protein dimer system one could choose a single residue that is near the center of the interface between the two monomers. The **'moveanchor'** and **'mode byvec'** options may also help in cases where the system is more condensed, such as those containing membranes.

37.11.9. average

```
average {crdset <set name> | <filename>} [<mask>]
      [start <start>] [stop <stop>] [offset <offset>]
      [Trajout Args]
```

<filename> If specified, write averaged coordinates to **<filename>** (not compatible with **crdset**).

crdset <set name> If specified, save averaged coordinates to **COORDS set <set name>** (not compatible with **<filename>**).

[<mask>] Average coordinates in **<mask>** (all atoms if not specified).

[<start>] Frame to begin calculation at (default 1).

[<stop>] Frame to end calculation at (default last).

[<offset>] Frames to skip between calculations (default 1).

[Trajout args] Output trajectory format argument(s) (default Amber Trajectory).

Calculate the average of input coordinates and write out to file named **<filename>** or save to **COORDS** set named **<set name>** in any trajectory format *cpptraj* recognizes (Amber Trajectory if not specified). If the number of atoms in **<mask>** are less than the total number of atoms, the topology will be stripped to match **<mask>**.

Note that since coordinates are being averaged over many frames, resulting structures may appear distorted. For example, if one averages the coordinates of a freely rotating methyl group the average position of the hydrogen atoms will be close to the center of rotation. Also note that typically one will want to remove global rotational and translation movement prior to this command by using e.g. the *rms* (37.11.70 on page 825) command.

Any arguments that are valid for the *trajout* command (37.10.5 on page 747) can be passed to this command in order to control the format of the output coordinates. For example, to write out a PDB file containing the averaged coordinates over all frames:

```
average test.pdb pdb
```

To write out a mol2 file containing only the averaged coordinates of residues 1 to 10 for frames 1 to 100:

```
average test.mol2 mol2 start 1 stop 100 :1-10
```

To create an average structure of atoms named CA and then use it as a reference for an rms command in a subsequent run:

```
trajin Input.nc
average crdset MyAvg @CA
run
rms ref MyAvg @CA out RmsToAvg.dat
run
```

37.11.10. avgbox

```
avgbox [name <setname>] [out <file>]
[name <setname>] Average unit cell data set name.
[out <file>] File to write average unit cell data to.
DataSets created:
<setname>[avg] Hold average unit cell as 3x3 matrix data.
```

Calculate the average unit cell vectors for incoming frames and store them in a 3x3 matrix data set, The average unit cell is particularly useful when unwrapping trajectories from NPT simulations where the box size fluctuates (see the *unwrap* command, 37.11.90 on page 839).

If writing to a .dat file, the output will look something like:

```
#Frame
1 42.433046075 0.000000000 0.000000000 -14.144347550 40.006259905 0.000000000
```

Where the first 3 floating point numbers are the average X unit cell vector, the next 3 floating point numbers are the average Y unit cell vector, and the last 3 floating point numbers are the average Z unit cell vector.

37.11.11. avgcoord

This command is deprecated. Use 'vector center' (optionally with keyword 'magnitude') instead.

37.11.12. bounds

```
bounds [<mask>] [out <filename>]
[dx <dx> [dy <dy>] [dz <dz>] name <gridname> [offset <bin offset>]]
[<mask>] Mask of atoms to determine bounds of.
[out <filename>] File to write bounds to (default STDOUT if not
specified).
```

[dx <dx> [dy <dy>] [dz <dz>]] Triggers creation of a grid data set from bounds. Spacings of generated grid in the X, Y and Z directions. If only dx is specified <dx> will be used for <dy> and <dz> as well.

[name <gridname>] Name of generated data sets.

[offset <bin offset>] Number of bins to add/subtract in each direction to generated grid.

DataSets Generated

<gridname> The 3D grid (only if 'dx' etc specified).

<gridname>[xmin] The minimum x coordinate encountered.

<gridname>[xmax] The maximum x coordinate encountered.

<gridname>[ymin] The minimum y coordinate encountered.

<gridname>[ymax] The maximum y coordinate encountered.

<gridname>[zmin] The minimum z coordinate encountered.

<gridname>[zmax] The maximum z coordinate encountered.

Calculate the boundaries (i.e. the max/min X/Y/Z coordinates) of atoms in <mask> and write to <filename> (STDOUT if not specified). Useful for determining dimensions for the *grid* command, and can be used to generate a grid data set that can be used by *grid* (see 37.11.39 on page 792).

37.11.13. box

```
box {[x <xval>] [y <yval>] [z <zval>] {[alpha <a>] [beta <b>] [gamma <g>] |
  [truncoc] [x <length>] |
  nobox |
  auto [offset <offset>] [radii {vdw|gb|parse|none}] |
  getbox {ucell|frac|shape} [name <setname>] [out <file>]
}
```

[x <xval>] [y <yval>] [z <zval>] Change box length(s) to specified value(s).

[alpha <a>] [beta] [gamma <g>] Change box angle(s) to specified value(s).

[truncoc] [x <length>] Set box angles (and optionally a length) to truncated octahedron.

[nobox] Remove any existing box information.

auto Set an orthogonal bounding box enclosing all atoms by the specified radii and an optional offset.

offset <offset> Offset in Angstroms to add to each box length (both + and -).

radii {vdw|gb|parse|none} Radii to use for each atom: van der Waals, generalized Born, PARSE, or no radii.

getbox Save existing box information to a 3x3 matrix data set.

{ucell|frac|shape} Specify the kind of box information to save: ucell saves the unit cell vectors, frac saves the fractional unit cell vectors, and shape saves the symmetric shape matrix unit cell vectors.

name <setname> The name of the 3x3 matrix data set.
out <file> File to write the 3x3 matrix data to.

Modify box information during trajectory processing. Note that this will permanently modify the box information for topology files during trajectory processing as well. It is possible to modify any number of the box parameters (e.g. only the Z length can be modified if desired while leaving all other parameters intact). If no box is present, an orthogonal bounding box enclosing all atoms can be created with the **auto** keyword. If 'getbox' is specified, the existing box information will be saved to a data set.

37.11.14. center

```
center [<mask>] [origin] [mass]
      [ reference | ref <name> | reindex <#> [<refmask>]]
```

[<mask>] Center based on atoms in mask; default is all atoms.
[origin] Center to origin (0, 0, 0); default is center to box center (X/2, Y/2, Z/2).
[mass] Use center of mass instead of geometric center.
[reference | ref <name> | reindex <#> [<refmask>]] Center using coordinates in specified reference structure selected by <refmask> (<mask> if not specified).

Move all atoms so that the center of the atoms in <mask> is centered at the specified location: box center (default), coordinate origin, or reference coordinates.

For example, to move all coordinates so that the center of mass of residue 1 is at the center of the box:

```
center :1 mass
```

37.11.15. check | checkoverlap | checkstructure

```
check [<mask>] [around <mask2>] [reportfile <report>] [noimage]
      [skipbadframes] [offset <offset>] [minoffset <minoffset>]
      [cut <cut>] [nobondcheck] [silent] [plcut <cut>]
```

[<mask>] Check structure of atoms in <mask> (all if not specified).
[around <mask2>] If specified, only check for problems between atoms in <mask> and atoms in <mask2>.
[reportfile <report>] Write any problems found to <report> (STDOUT if not specified).
[noimage] Do not image distances.
[skipbadframes] If errors are encountered for a frame, subsequent actions/trajectory output will be skipped.
[offset <offset>] Report bond lengths greater than the equilibrium value plus <offset> (default 1.15 Å).
[minoffset <minoffset>] Report bond lengths less than the equilibrium value minus <minoffset> (default 0.5 Å).
[cut <cut>] Report atoms closer than <cut> (default 0.8 Å).
[nobondcheck] Check overlaps only.
[silent] Do not print information for bad frames - useful in conjunction with the skipbadframes option.

[plcut <cut>] Pair list cutoff (default 4.0 Å); only matters if box is present.

Check the structure and report problems related to atomic overlap/unusual bond length. Problems are reported when any two atoms in <mask> (or between <mask> and <mask2> if using 'around') are closer than <cut>; atoms that are bonded to each other are ignored (except if using the 'around' mask). If bonds are being checked then bond lengths greater than their equilibrium value plus <offset> and less than their equilibrium value minus <minoffset> are reported as well. If box information is present and not using the 'around' mask, a pairlist will be used to speed up the calculation.

This command can also be used to skip corrupted frames in a trajectory during processing. For example, if this message is encountered:

Warning: Frame 10 coords 1 & 2 overlap at origin; may be corrupt.

One could use *check* so that e.g. a subsequent *distance* command is not processed for bad frames:

```
check @1,2 skipbadframes silent
distance d1 :1 :10
```

Usually frame corruption can be detected using only a few atoms, but this may not catch all types of corruption. The more atoms that are used the better the corruption detection will be, but the slower it will be to process the command. Typically a good procedure to follow when corruption is suspected is to run *check* using all important atoms (e.g. all solute heavy atoms) with the *skipbadframes* keyword followed by a *trajout* command to write all non-corrupt frames, for example:

```
trajin corrupted.crd
check :1-13 skipbadframes silent
trajout fixed.corrupted.nc
```

37.11.16. checkchirality

checkchirality [<name>] [<mask>] [out <filename>]

[<name>] Data set name.

[<mask>] Atoms to check.

[out <filename>] File to write results to.

DataSet Aspects:

[L] Number of frames 'L' for each residue.

[D] Number of frames 'D' for each residue.

Check the chirality around the alpha carbon in amino acid residues selected by <mask>. Note that cpptraj expects atom names to correspond to the PDB V3 standard: N, CA, C, CB. For each residue, the number of frames in which the amino acid is 'L' or 'D' will be recorded. For example, to check the chirality of all amino acids in a system and write to a file named *chiral.dat* with data set name *DPDP*:

```
checkchirality DPDP out chiral.dat
```

Output will have format similar to:

```
#Res      DPDP [L]  DPDP [D]
  2.000      100       0
```

So in this example residue 2 was 'L' for 100 frames and 'D' for 0 frames.

37.11.17. **closest** | **closestwaters**

```
closest <# to keep> <mask> [solventmask <solvent mask>] [noimage]
    [first | oxygen] [center] [closestout <filename> [name <setname>]]
    [outprefix <prefix>] [nobox] [parmout <filename>]
    [parmopts <comma-separated-list>]

<# to keep> Number of solvent molecules to keep around <mask>
<mask> Mask of atoms to search for closest waters around.
[solventmask <solvent mask>] Optional mask for selecting solvent
atoms. If not specified, atoms in all molecules marked as
``solvent`` will be used.
[noimage] Do not perform imaging; only recommended if trajectory has
previously been imaged.
[first | oxygen] Calculate distances between all atoms in <mask> and
the first atom of solvent only (recommended for standard water
models as it will increase speed of calculation).
[center] Search for waters closest to geometric center of <mask>
instead of each atom in <mask>.
[closestout <filename>] Write information on the closest solvent
molecules to <filename>.
[outprefix <prefix>] Write corresponding topology to file with name
prefix <prefix>.
[nobox] Remove any box information from the topology.
[parmout <file>] Write corresponding topology to file with name
<file>.
[parmopts <list>] Comma-separated list of options for writing the
topology file.

DataSet Aspects:
[Frame] Frame number.
[Mol] Original solvent molecule number.
[Dist] Solvent molecule distance in Å.
[FirstAtm] First atom number of original solvent molecule.
```

Similar to the *strip* command, but modify coordinate frame and topology by keeping only the specified number of closest solvent molecules to the region specified by the given mask. Solvent molecules can be determined automatically by *cpptraj* (by default residues named WAT, HOH, or TIP3), can be specified prior via the *solvent* command ([37.9.21 on page 741](#)), or can be selected by *solventmask*.

The format of the **closestout** file is:

Frame	Molecule	Distance	FirstAtom#
-------	----------	----------	------------

For example, to obtain the 10 closest waters to residues 1-268 by distance to the first atom of the waters, write out which waters were closest for each frame to a file called “closestmols.dat”, and write out the stripped topology with prefix “closest” containing only the solute and 10 waters:

```
closest 10 :1-268 first closestout closestmols.dat outprefix closest
```

As of version 17 this command is CUDA-enabled in CUDA versions of CPPTRAJ.

37.11.18. cluster

Although the **'cluster'** command can still be specified as an action, it is now considered an analysis. See [37.12.5 on page 852](#).

37.11.19. clusterdihedral

```
clusterdihedral [phibins <N>] [psibins <M>] [out <outfile>]
                [dihedralfile <dfile> | <mask>]
                [framefile <framefile>] [clusterinfo <infofile>]
                [clustervtime <cvtfiler>] [cut <CUT>]
```

Cluster frames in a trajectory using dihedral angles. To define which dihedral angles will be used for clustering either an atom mask or an input file specified by the **dihedralfile** keyword should be used. If dihedral file is used, each line in the file should contain a dihedral to be binned with format:

```
ATOM#1 ATOM#2 ATOM#3 ATOM#4 #BINS
```

where the ATOM arguments are the atom numbers (starting from 1) defining the dihedral and #BINS is the number of bins to be used (so if #BINS=10 the width of each bin will be 36°). If an atom mask is specified, only protein backbone dihedrals (Phi and Psi defined using atom names C-N-CA-C and N-CA-C-N) within the mask will be used, with the bin sizes specified by the phibins and psibins keywords (default for each is 10 bins).

Output will either be written to STDOUT or the file specified by the **out** keyword. First, information about which dihedrals were clustered will be printed. Then the number of clusters will be printed, followed by detailed information of each cluster. The clusters are sorted from most populated to least populated. Each cluster line has format

```
Cluster CLUSTERNUM CLUSTERPOP [ dihedral1bin, dihedral2bin ... dihedralNbin ]
```

followed by a list of frame numbers that belong to that cluster. If a cutoff is specified by **cut**, only clusters with population greater than CUT will be printed.

If specified by the **clustervtime** keyword, the number of clusters for each frame will be printed to <cvtfiler>. If specified by the **framefile** keyword, a file containing cluster information for each frame will be written with format

```
Frame CLUSTERNUM CLUSTERSIZE DIHEDRALBINID
```

where DIHEDRALBINID is a number that identifies the unique combination of dihedral bins this cluster belongs to (specifically it is a 3*number-of-dihedral-characters long number composed of the individual dihedral bins).

If specified by the **clusterinfo** keyword, a file containing information on each dihedral and each cluster will be printed. This file can be read by SANDER for use with REMD with a structure reservoir (-remd=3). The file, which is essentially a simplified version of the main output file, has the following format:

```
#DIHEDRALS
dihedral1_atom1 dihedral1_atom2 dihedral1_atom3 dihedral1_atom4
...
#CLUSTERS
CLUSTERNUM1 CLUSTERSIZE1 DIHEDRALBINID1
...
```

37.11.20. contacts

```
contacts [ first | reference | ref <ref> | refindex <#> ] [byresidue]
         [out <filename>] [time <interval>] [distance <cutoff>] [<mask>]
```

NOTE: Users are encouraged to try the *nativecontacts* command (on page 815), an update version of this command.

For each atom given in *mask*, calculate the number of other atoms (contacts) within the distance *cutoff*. The default cutoff is 7.0 Å. Only atoms in *mask* are potential interaction partners (e.g., a mask @CA will evaluate only contacts between CA atoms). The results are dumped to *filename* if the keyword “out” is specified. Thereby, the time between snapshots is taken to be *interval*. In addition to the number of overall contacts, the number of native contacts is also determined. Native contacts are those that have been found either in the first snapshot of the trajectory (if the keyword “first” is specified) or in a reference structure (if the keyword “reference” is specified). Finally, if the keyword “byresidue” is provided, results are output on a per-residue basis for each snapshot, whereby the number of native contacts is written to *filename.native*.

37.11.21. createcrd

```
createcrd [<name>] [ parm <name> | parmindex <#> ]
```

This command creates a COORDS data set named <name> using trajectory frames that are associated with the specified topology.

For example, to save frames that have been previously RMS-fit to a reference structure into a COORDS set named MyCrd you would use the input:

```
rms reference :1-12@CA
createcrd MyCrd
strip :6-8
```

Note that here the *strip* command will have no effect on the coordinates saved in MyCrd since it occurs after the *createcrd* command.

37.11.22. createreservoir

```
createreservoir <filename> ene <energy data set> [bin <cluster bin data set>]
temp0 <temp0> iseed <iseed> [velocity]
[parm <parmfile> | parmindex <#>] [title <title>]
```

<filename> File name of the reservoir to create.

ene <energy data set> Data set with energies corresponding to frames.

[bin <cluster bin data set>] Data set with bin numbers (for RREMD=3).

temp0 <temp0> Reservoir temperature.

iseed <iseed> Reservoir random number seed.

[velocity] Include velocities in the reservoir.

[parm <parmfile> | parmindex <#>] Associated topology.

[title <title>] Reservoir title.

Create structure reservoir for use with reservoir REMD simulations using energies in <energy data set>, temperature <temp0> and random seed <iseed> Include velocities if [velocity] is specified. If <cluster bin data set> is specified from e.g. a previous 'clusterdihedral' command, the reservoir can be used for non-Boltzmann reservoir REMD (rremd==3).

37.11.23. density

```
density [out <filename>] [name <set name>]
[ <mask1> ... <maskN> [delta <resolution>] [{x|y|z}]
[{number|mass|charge|electron}] [{bincenter|binedge}]
[restrict {cylinder|square} cutoff <cut>] ]
```

[out <filename>] Output file for histogram(s) (relative distances vs. densities for each mask) or total density.

[name <set name>] Output data set name.

<mask1> ... <maskN> Arbitrary number of masks for atom selection; a dataset is created and the output will contain entries for each mask.

[delta <resolution>] Resolution, i.e. determines number of slices (i.e. histogram bins). (default 0.25 Å)

[{x|y|z}] Coordinate (axis) for density calculation. (default z)

[{number|mass|charge|electron}] Number, mass, partial charge (q) or electron (Ne - q) density. Electron density will be converted to e-/Å³ by dividing the average area spanned by the other two dimensions. (default number)

[{bincenter|binedge}] Determine whether histogram bin coordinates will be based on bin center (default) or bin edges.

[restrict {cylinder|square}] If 'restrict' is specified, only calculate the density that is within a cylinder or square shape from the specified axis as defined by a distance cutoff.

cutoff <cut> The distance cutoff for 'restrict'.

DataSets Created if masks specified:

<set name>[avg]:<idx> Average density over coordinate for mask number <idx>.

<set name>[sd]:<idx> Standard deviation of density over coordinate for mask number <idx>.

DataSets Created if no masks specified:

<set name> Total system density each frame.

If no atom masks are specified, calculate the total system density. Otherwise, calculate specified density along the given axis for atoms in specified mask(s). Defaults are shown in parentheses above. The format of the file is as follows. Comments are lines starting with '#' or empty lines. All other lines must contain the atom type followed by an integer number for the electron number. Entries must be separated by spaces or '='. Example input:

```
density out number_density.dat number delta 0.25 ":POPC@P1" ":POPC@N" \
":POPC@C2" ":POPC"
density out mass_density.dat mass delta 0.25 ":POPC@P1" ":POPC@N" \
":POPC@C2" ":POPC"
density out charge_density.dat charge delta 0.25 ":POPC@P1" ":POPC@N" \
":POPC@C2" ":POPC"
density out electron_density.dat electron delta 0.25 efile Nelec.in \
":POPC@P1" ":POPC@N" ":POPC@C2" ":POPC" ":TIP3" \
":POPC | :TIP3" "*"
density out ion_density.dat number delta 0.25 ":SOD" ":CLA"
```

See also \$AMBERHOME/AmberTools/test/cpptraj/Test_Density.

It can be useful to write out the average and standard deviation as an XYDY set to a Grace data file, e.g.

```
density :WAT@O out wato.agr xydy
```

37.11.24. diffusion

*Note that although the syntax for **diffusion** has changed as of version 16, the old syntax is still supported.*

```
diffusion [{out <filename>|separateout <suffix>}] [time <time per frame>] [noimage]
[<mask>] [<set name>] [individual] [diffout <filename>] [nocalc]
[avgucell <avg ucell set>]
[allowmultipleorigins]
```

[out <filename>] Write mean-square displacement (MSD) data set output to file specified by <filename>.

[separateout <suffix>] Write each MSD data set type to files with suffix <suffix>; see description below.

[time <time_per_frame>] Time in-between each coordinate frame in ps; default is 1.0.

[noimage] If specified do not perform imaging. If this is specified coordinates should be unwrapped prior to this command.

[<mask>] Mask of atoms to calculate diffusion for; default all atoms.

[<set name>] MSD data set name.

[individual] Write diffusion for each individual atom as well as average diffusion for atoms in mask.

[diffout <filename>] Write diffusion constants calculated from fits of MSD data sets to <filename>.

[nocalc] Do not calculate diffusion constants.

[avgucell] Remove periodic box fluctuations from imaged NPT trajectories using average unit cell vectors.

[allowmultipleorigins] (MPI only). For imaged trajectories in parallel, calculate diffusion by averaging over multiple time origins. Should be used with caution.

DataSet Aspects:

[X] MSD(s) in the X direction.

[Y] MSD(s) in the Y direction.

[Z] MSD(s) in the Z direction.

[R] Overall MSD(s).

[A] Overall displacement(s).

[D] Diffusion constants ($1 \times 10^{-5} \text{ cm}^2/\text{s}$).

[Label] Diffusion constant labels.

[Slope] Linear regression slopes.

[Intercept] Linear regression Y-intercepts.

[Corr] Linear regression correlation coefficients.

[aX]:<atomN> (individual only) Atom <N> MSD in the X direction.

[aY]:<atomN> (individual only) Atom <N> MSD in the Y direction.

[aZ]:<atomN> (individual only) Atom <N> MSD in the Z direction.

[aR]:<atomN> (individual only) Atom <N> overall MSD.

[aA]:<atomN> (individual only) Atom <N> overall displacement.

Compute mean-squared displacement (MSD, in Angstroms squared) plots (using distance traveled from initial position) for the atoms in **<mask>**. By default only the diffusion averaged over all atoms in **<mask>** is calculated; if **individual** is specified diffusion for individual atoms is calculated as well.

In order to correctly calculate diffusion molecules should take continuous paths, so imaging of atoms is automatically performed. If the trajectory is already unwrapped (or the **unwrap** command is used prior to this command) the **noimage** keyword can be used. To remove the “noise” caused by box fluctuations in NPT trajectories, the average unit cell vectors describing the average box can be provided with the **avgucell** keyword; see the **avgbox** command (37.11.10 on page 763). Alternatively, the trajectory can be unwrapped prior using the **unwrap** command, 37.11.90 on page 839. If the trajectory is unwrapped, the **noimage** keyword should be specified.

Note that in parallel, imaging becomes difficult because there is no way to correct for any wrapping that has been done on preceding MPI ranks. Therefore this command will not work on imaged trajectories in parallel by default. There are two workarounds. 1) Unwrap the trajectory prior to **diffusion** with the **unwrap** command, then run the diffusion calculation with the **noimage** keyword (this is the recommended way), or 2) specify the **allowmultipleorigins** keyword to calculate MSD separately on each MPI rank, then averaging over all MSD plots. This means the maximum length of any given MSD plot will be **<# frames> / <# MPI ranks>**, and the calculated diffusion constants will not be as accurate.

The following types of displacements are calculated. If **separateout** is specified the following files will be created:

x_<suffix> Mean square displacement(s) in the X direction (in Å²/ps).

y_<suffix> Mean square displacement(s) in the Y direction (in Å²/ps).

z_<suffix> Mean square displacement(s) in the Z direction (in Å²/ps).

r_<suffix> Overall mean square displacement(s) (in Å²/ps).

a_<suffix> Total distance traveled (in Å/ps).

The diffusion coefficient **D** can be calculated using the Einstein relation:

$$2nD = \lim_{t \rightarrow \infty} \frac{MSD}{t}$$

Where **n** is the number of dimensions; for overall MSD **n** = 3, for single dimension MSD (e.g. X) **n** = 1, etc. Unless **nocalc** is specified, the diffusion constant is calculated automatically from MSD data sets (and written to the file specified by **diffout**) in the following manner. The slope the plot of MSD versus time is obtained via linear regression. To convert from units of Å²/ps to 1x10⁻⁵ cm²/s, the slope is multiplied by 10.0/(2n). Both the calculated diffusion constants as well as the results of the fit are reported.

Due to the fact that diffusion is currently calculated from initial positions only, diffusion calculated for small numbers of atoms will be inherently stochastic, so the results are most sensible when averaged over many atoms; for example, the diffusion of water should be calculated using all waters in the system. If more averaging is needed, the **calcdiffusion** Analysis command (37.12.3 on page 849) can be used to calculate diffusion from multiple time origins.

For example, to calculate the diffusion of water in a system:

```
diffusion :WAT@O out WAT.O.agr WAT.O diffout DC.dat
```

37.11.25. dihedral

```
dihedral [<name>] <mask1> <mask2> <mask3> <mask4> [out <filename>] [mass]
         [type {alpha|beta|gamma|delta|epsilon|zeta|chi|c2p|h1p|phi|psi|omega|pchi}]
         [range360]

[<name>] Output data set name.
```


<maskX> Four atom masks selecting atom(s) to calculate dihedral for.

[out <filename>] Output file name.

[mass] Use center of mass of atoms in <maskX>; default is geometric center.

[range360] Output dihedral angle values from 0 to 360 degrees instead of -180 to 180 degrees.

[type <type>] Label dihedral as <type> for use with *statistics* analysis; note 'chi' is nucleic acid chi and 'pchi' is protein chi.

Calculate dihedral angle (in degrees) between the planes defined by atoms in <mask1>, <mask2>, <mask3> and <mask2>, <mask3>, <mask4>. To calculate multiple dihedral angles see the *multidihedral* command on page 808.

37.11.26. dihedrals | dihrms

```
dihedrals [ <name> ] <dihedral types> [ out <file> ]
      [ first | reference | ref <name> | refindex <#> | previous |
        reftraj <name> [ parm <name> | parmindex <#> ] ]
      [ dihtype <name>:<a0>:<a1>:<a2>:<a3>[:<offset>] ... ]
      [ tgtrange <range> [ refrange <range> ] ]
```

[<name>] Output data set name.

<dihedral types> Dihedral types to look for. Note that chip is 'protein chi', chin is 'nucleic chi'.

[out <filename>] Output file name.

[dihtype <name>:<a0>:<a1>:<a2>:<a3>[:<offset>]] Search for a custom dihedral type called <name> using atom names <a0>, <a1>, <a2>, and <a3>.

Offset: -2=<a0><a1> in previous res, -1=<a0> in previous res, 0=All <aX> in single res, 1=<a3> in next res, 2=<a2><a3> in next res.

[tgtrange <range>] Residue range to look for target dihedrals in. Default is all solute residues.

[refrange <range>] Residues range to look for reference dihedrals in. If not specified, use target range.

Calculate RMSD of selected dihedrals to dihedrals in a reference structure. See the *multidihedral* command syntax on page 808 for a list of all available dihedral types.

37.11.27. dihedralscan

This command has been replaced by *permutedihedrals*; see 37.7.12 on page 709.

37.11.28. dipole

```
dipole [ out <filename> ]
      { data <dsname> | boxref <ref name/tag> <nx> <ny> <nz> |
        <nx> <dx> <ny> <dy> <nz> <dz>
        [ { gridcenter <cx> <cy> <cz> |
```

```

    boxcenter |
    maskcenter <mask> |
    rmsfit <mask> [noxalign]} ]
[box|origin|center <mask>] [negative] [name <gridname>]
    <mask1> [max <maxpercent>]
[out <filename>] File to write out grid to. Use ``.grid'' or
    ``.xplor'' extension for XPLOR format, ``.dx'' for OpenDX
    format.

Options for setting up grid:

data <dsname> Use previously calculated/loaded grid data set named
    <dsname>. When using this option there is no need to specify
    grid bins/spacing/center.

boxref <ref name/tag> <nx> <ny> <nz> Set up grid using box
    information from a previously loaded reference structure.
    Currently the only way to set up non-orthogonal grids.

<nx> <dx> <ny> <dy> <nz> <dz> Number of grid bins and spacing in
    the X/Y/Z directions.

[gridcenter <cx> <cy> <cz>] Location of grid center, default is
    origin (0.0, 0.0, 0.0).

[boxcenter] Center grid on box center.

[maskcenter <mask>] Center the grid on the atoms selected by
    <mask>.

[rmsfit <mask>] Perform a best-fit rotation of the grid using the
    coordinates selected by <mask>.

[noxalign] If specified, grid will not be re-oriented to align
    with Cartesian axes once binning is finished. Will affect
    file formats that do not store full unit cell vectors (like
    Xplor).

Options for offset during grid binning (must center grid at
    origin):

[box] Offset each point by location of box center prior to gridding.
    Cannot be used with 'gridcenter'.

[origin] No offset (default)

[center <mask>] Offset each point by center of atoms in <mask>
    prior to gridding. Cannot be used with 'gridcenter'.

Other options:

[negative] Grid negative density instead of positive density.

[name <gridname>] Grid data set name.

<mask1> Mask selecting solvent atoms to bin.

[max <max percent>] Only keep density >= to <maxpercent> of the
    maximum density.

```

NOTE: This command is not well-tested and may be obsolete.

Similar to **grid** (see [37.11.39 on page 792](#) below) except that dipoles of the solvent molecules are binned. The output file format is for Chris Bayly's discern delegate program that comes with Midas/Plus. Consult the code in Action_Dipole.cpp for more information.

37.11.29. distance

```
distance [<name>] <mask1> [<mask2>] [point <X> <Y> <Z>]
      [ reference | ref <name> | reindex <#> ]
      [out <filename>] [geom] [noimage] [type noe]
Options for 'type noe':
      [bound <lower> bound <upper>] [rexp <expected>] [noe_strong] [noe_medium] [noe_weak]
```

[<name>] Output data set name

<mask1> Atom mask selecting atom(s) to calculate distance between.

<mask2> If specified, second atom mask selection atom(s) to calculate distance from <mask1>.

point <X> <Y> <Z> If specified instead of second mask, calculate distance between <mask1> and specified XYZ coordinates.

reference | ref <name> | reindex <#> If specified, calculate distance between <mask1> in each input frame and <mask2> in the specified reference.

[out <filename>] Output filename.

[geom] Use geometric center of atoms in <mask1>/<mask2>; default is to use center of mass.

[noimage] Do not image distances across periodic boundaries.

[type noe] Mark distance as 'noe' for use with *statistics* analysis.

 [bound <lower> bound <upper>] Lower and upper bounds for NOE (in Angstroms); must specify both.

 [rexp <expected>] Expected value for NOE (in Angstroms); if not given '(<lower> + <upper>)' / 2.0 is used.

 [noe_strong] Set lower and upper bounds to 1.8 and 2.9 Å respectively.

 [noe_medium] Set lower and upper bounds to 2.9 and 3.5 Å respectively.

 [noe_weak] Set lower and upper bounds to 3.5 and 5.0 Å respectively.

Calculate distance between the center of mass of atoms in <mask1> to atoms in <mask2>, between atoms in <mask1> from each input frame and atoms in <mask2> in specified reference, or atoms in <mask1> and the specified point. If **geom** is specified use the geometric center instead. For periodic systems imaging is turned on by default; the **noimage** keyword disables imaging.

A distance can be labeled using 'type noe' for further analysis as an NOE using the '*statistics*' analysis command ([37.12.37 on page 886](#)).

37.11.30. drms | drmsd (distance RMSD)

```
drmsd [<dataset name>] [<mask> [<refmask>]] [out <filename>]
      [ first | ref <refname> | reindex <#> |
      reftraj <trajname> [parm <trajparm> | parmindex <parm#>] ]
```

[<dataset name>] Output data set name.

[<mask>] Atoms to calculate DRMSD for.

[<refmask>] Mask corresponding to atoms in reference; if not specified, <mask> is used.

[out <filename>] Output file name.

[first] Use the first trajectory frame processed as reference.

[reference] Use the first previously read in reference structure.

[ref <refname>] Use previously read in reference structure specified by <refname>.

[refindex <#>] Use previously read in reference structure specified by <#> (based on order read in).

previous Use frame prior to current frame as reference.

reftraj <name> Use frames from COORDS set <name> or read in from trajectory file <name> as references. Each frame from <name> is used in turn, so that frame 1 is compared to frame 1 from <name>, frame 2 is compared to frame 2 from <name> and so on. If <trajname> runs out of frames before processing is complete, the last frame of <trajname> continues to be used as the reference.

parm <parmname> | parminx <#> If reftraj specifies a file associate trajectory <name> with specified topology; if not specified the first topology is used.

Calculate the distance RMSD (i.e. the RMSD of all pairs of internal distances) between atoms in the frame defined by <mask> (all if no <mask> specified) to atoms in a reference defined by <refmask> (<mask> if <refmask> not specified). Both <mask> and <refmask> must specify the same number of atoms, otherwise an error will occur.

Because this method compares pairs of internal distances and not absolute coordinates, it is not sensitive to translations and rotations the way that a no-fit RMSD calculation is. It can be more time consuming however, as $(N^2-N)/2$ distances must be calculated and compared for both the target and reference structures.

For example, to get the DRMSD of a residue named LIG to its structure in the first frame read in:

```
drmsd :LIG first out drmsd.dat
```

37.11.31. dssp

See [37.11.78 on page 830](#).

37.11.32. enedecomp

```
enedecomp [<name>] [<mask>] [out <filename>] [savecomponents]
          [ pme [cut <cutoff>] [dsumtol <dtol>] [ewcoeff <coeff>]
            [erfcdx <dx>] [skinnb <skinnb>] [ljswidth <width>]
            [order <order>] [nfft <nfft1>,<nfft2>,<nfft3>]
```

[<name>] Data set name.

[<mask>] Mask of atoms to calculate energy for.

[out <filename>] File to write results to.

[savecomponents] If specified, also save the individual components of the total energy (bond, angle, dihedral, etc.).

[pme] Use particle mesh Ewald for electrostatics; van der Waals energy will be calculated using a long-range correction for periodicity.

cut <cutoff> Direct space cutoff in Angstroms (default 8.0).

dsumtol <dtol> Direct sum tolerance (default 0.00001). Used to determine Ewald coefficient.

ewcoeff <coeff> Ewald coefficient in 1/Ang.

erfcdx <dx> Spacing to use for the ERFC splines (default 0.0002 Ang.).

skinnb Used to determine pairlist atoms (added to cut, so pairlist cutoff is cut + skinnb); included in order to maintain consistency with results from sander.

ljswidth <width> If specified, use a force-switching form for the Lennard-Jones calculation from <cutoff>-<width> to <cutoff>.

order <order> Spline order for charges.

nfft <nfft1>,<nfft2>,<nfft3> Explicitly set the number of FFT grid points in each dimension. Will be determined automatically from unit cell dimensions if not specified.

DataSets created:

<name> Set containing atom index and the corresponding average total energy over frames.

If 'savecomponents' is specified:

<name>[bond] Set containing atom index and average bond energy over frames.

<name>[angle] Set containing atom index and average angle energy over frames.

<name>[dih] Set containing atom index and average dihedral energy over frames.

<name>[vdw14] Set containing atom index and average 1-4 van der Waals energy over frames.

<name>[elec14] Set containing atom index and average 1-4 electrostatic energy over frames.

<name>[elec] Set containing atom index and average electrostatic energy over frames.

<name>[vdw] Set containing atom index and average van der Waals energy over frames.

Perform per-atom energy decomposition for selected atoms. The energy is calculated for the entire system but only the energies for selected atoms will be reported. The energy is composed of the regular bond, angle, torsion, 1-4 nonbonded, and nonbonded terms. If 'savecomponents' is specified, each of the energy components will be saved in addition to the total energy for each atom. If 'pme' is specified the non-bonded terms will use PME for electrostatics and a long-range periodic correction for van der Waals, otherwise a simple model with no cutoff will be used.

37.11.33. energy

```
energy [<name>] [<mask1>] [out <filename>] [nobondstoh] [openmm [<mdopts>]]
    [bond] [angle] [dihedral] {[nb14]|[e14]|[v14]} {[nonbond]|[elec] [vdw]}
    [{nokinetic|kinetic [ketype {vel|vv}] [dt <dt>]]}
    [ etype { simple |
              directsum [npoints <N>] |
```

```

ewald [cut <cutoff>] [dsumtol <dtol>] [ewcoeff <coeff>]
      [erfcdx <dx>] [skinnb <skinnb>] [ljswidth <width>]
      [rsumtol <rtol>] [maxexp <max>] [mlimits <X>,<Y>,<Z>] |
pme [cut <cutoff>] [dsumtol <dtol>] [ewcoeff <coeff>]
    [erfcdx <dx>] [skinnb <skinnb>] [ljswidth <width>]
    [order <order>] [nfft <nfft1>,<nfft2>,<nfft3>]
    [ljpme [ewcoefflj <ljcoeff>]]
} 1

```

[<name>] Data set name.

[<mask1>] Mask of atoms to calculate energy for.

[out <filename>] File to write results to.

[nobondstoh] Skip calculating the energy of bonds to hydrogen.

[openmm] If specified and CPPTRAJ is compiled with OpenMM support, use OpenMM to calculate energy. Note this will only calculate total energy; any keywords pertaining to individual energy components are not available. For a list of potential options that can be used with 'openmm', see [37.7.7 on page 706](#).

[bond] Calculate bond energy.

[angle] Calculate angle energy.

[dihedral] Calculate dihedral energy.

[nb14] Calculate nonbonded 1-4 energy.

[e14] Calculate 1-4 electrostatics.

[v14] Calculate 1-4 van der Waals.

[nonbond] Calculate nonbonded energy (electrostatics and van der Waals).

[elec] Calculate electrostatic energy (Coulomb potential).

[vdw] Calculate van der Waals energy (Lennard-Jones 6-12 potential).

[nokinetic] Do not calculate kinetic energy even if velocity/force information present.

[kinetic] Attempt to calculate kinetic energy. Requires force and/or velocity information.

ketype {vel|vv} Specify kinetic energy type. If not specified, if velocity and force information use a velocity verlet-type calculation (vv), i.e. assume velocities are a half-step ahead of the forces. If only velocity information is present, calculate from on-step velocities (vel).

dt <dt> Time step for vv calculation in ps.

[etype <type>] Calculate electrostatics via specified type.

[simple] Use simple Coulomb term for electrostatics, no cutoff.

[directsum] Use direct summation method for electrostatics.

[npoints <N>] Number of cells in each direction to calculate the direct sum.

[ewald] Use Ewald summation for electrostatics. If van der Waals energy will be calculated a long-range correction for periodicity will be applied.

cut <cutoff> Direct space cutoff in Angstroms (default 8.0).

dsumtol <dtol> Direct sum tolerance (default 0.00001). Used to determine Ewald coefficient.

ewcoeff <coeff> Ewald coefficient in 1/Ang.

erfcdx <dx> Spacing to use for the ERFC splines (default 0.0002 Ang.).

skinnb Used to determine pairlist atoms (added to cut, so pairlist cutoff is cut + skinnb); included in order to maintain consistency with results from sander.

ljswidth <width> If specified, use a force-switching form for the Lennard-Jones calculation from <cutoff>-<width> to <cutoff>.

rsumtol <rtol> Reciprocal sum tolerance (default 0.00005). Used to determine number of reciprocal space vectors.

mlimits <X>,<Y>,<Z> Explicitly set the number of reciprocal space vectors in each dimension. Will be determined automatically if not specified.

[pme] Use particle mesh Ewald for electrostatics. If van der Waals energy will be calculated a long-range correction for periodicity will be applied.

cut <cutoff> Direct space cutoff in Angstroms (default 8.0).

dsumtol <dtol> Direct sum tolerance (default 0.00001). Used to determine Ewald coefficient.

ewcoeff <coeff> Ewald coefficient in 1/Ang.

erfcdx <dx> Spacing to use for the ERFC splines (default 0.0002 Ang.).

skinnb Used to determine pairlist atoms (added to cut, so pairlist cutoff is cut + skinnb); included in order to maintain consistency with results from sander.

ljswidth <width> If specified, use a force-switching form for the Lennard-Jones calculation from <cutoff>-<width> to <cutoff>.

order <order> Spline order for charges.

nfft <nfft1>,<nfft2>,<nfft3> Explicitly set the number of FFT grid points in each dimension. Will be determined automatically from unit cell dimensions if not specified.

ljpme If specified use particle mesh Ewald for calculating Lennard-Jones interactions.

ewcoefflj Ewald coefficient for Lennard-Jones PME (implies ljpme).

DataSet Aspects:

[bond] Bond energy.

[angle] Angle energy.

[dih] Dihedral energy.

[vdw14] 1-4 van der Waals energy.

[elec14] 1-4 electrostatic energy.

[vdw] van der Waals energy.

[elec] Electrostatic energy.

[kinetic] Kinetic energy.

[total] Total energy.

Calculate the energy for atoms in <mask>. If no terms are specified, all terms are calculated. Note that the non-bonded energy terms for **'simple'** do not take into account periodicity and there is no distance cut-off. Electrostatics can also be determined via the direct sum, Ewald, or particle-mesh Ewald summation procedures. The particle mesh Ewald functionality requires that CPPTRAJ be compiled with FFTW and a C++11 compliant compiler.

Calculation of energy terms requires that the associated topology file have parameters for any of the calculated terms, so for example angle calculations are not possible when using a PDB file as a topology, etc. All nonbonded calculations methods other than **simple** require unit cell parameters.

For example, to calculate all energy terms and write to a Grace-format file:

```
parm DPDP.parm7
trajin DPDP.nc
energy DPDP out ene.agr
```

37.11.34. esander

```
esander [<name>] [out <filename>] [saveforces] [parmname <file>] [keepfiles]
      [<namelist vars> ...]
```

[<name>] Data set name.

[out <filename>] File to write results to.

[saveforces] If specified, save forces to frames. Requires writing frames in NetCDF format.

[parmname <file>] Name of temporary topology file (default: 'CpptrajEsander.parm7').

[keepfiles] Keep temporary topology file after program execution.

[<namelist vars>] Namelist variables supported by the sander API in format 'var <value>'; see below.

Calculate energies for input frames using the sander API. It requires compilation with the SANDER API (sander-lib). This can be considered as a faster alternative to energy post-processing with sander (imin = 5). Currently the following sander namelist variables are supported: **extidel**, **intdiel**, **rgbmax**, **saltcon**, **cut**, **dielc**, **igb**, **alpb**, **gbsa**, **lj1264**, **ipb**, **inp**, **vdwmeth**, **ew_type**, **ntb**, **ntf**, **ntc**. See [22 on page 392](#) for details.

If ntb/cut/igb are not specified cpptraj will attempt to pick reasonable values based on the input system. The defaults for a non-periodic system are ntb=0, cut=9999.0, igb=1. The defaults for a periodic system are ntb=1, cut=8.0, igb=0. This currently requires writing a temporary Amber topology, the name of which can be set by **parmname**. If **keepfiles** is specified this temporary topology will not be deleted after execution.

For example, to calculate energies for a non-periodic system using igb=1 (the default) with GB surface area turned on (gbsa=1):

```
parm DPDP.parm7
trajin DPDP.nc
esander DPDP out Edpdp.dat gbsa 1
```

37.11.35. filter

```
filter {<dataset arg> min <min> max <max> ...} [out <file>] [name <setname>]
      {[multi] | [filterset <set> [newset <newname>]] [countout <countfile>]}
```

<dataset arg> Data set name(s) to use for filtering

min <min> Allow values greater than <min> in dataset(s).

max <max> Allow values greater than <max> in dataset(s).

[out <file>] File containing 1 for frames that were allowed, 0 for frames that were filtered.

[name <setname>] Filtered data set name containing 1 for allowed frames, 0 for filtered frames.

[multi] Filter each set separately instead of all together (creates filter set for each input set). Cannot be used with 'filterset'.

[filterset <set>] If specified, <set> will be filtered to only contain data that satisfies cutoffs. Cannot be used with 'multi'.

[newset <newname>] If specified a new set will be created from 'filterset' instead of replacing 'filterset'.

[countout <count>] If specified, write number of elements passed and filtered to <countfile>. Cannot be used with 'multi'.

Sets Created (not 'multi')

<setname> For each input element contains 1 for elements that ``passed'', 0 otherwise.

<setname>[npassed] Number of elements that passed.

<setname>[nfiltered] Number of elements filtered out.

Sets Created ('multi')

<setname>:<idx> For each input set (number with <idx>, starting from 0) contains 1 for elements that ``passed'', 0 otherwise.

For all following actions, only include frames that are between <min> and <max> of data sets in <dataset arg>. There must be at least one <min> and <max> argument, and there must be as many <min>/<max> arguments as there are specified data sets. If 'multi' is specified then only filter data sets will be created for each data set instead. If 'filterset' is specified, the specified <set> will be modified to only contain '1' frames; cannot be used with 'multi'. If 'newset' is also specified, a new set will be created containing the '1' frames instead. The 'filterset' functionality only works for 1D scalar sets. If 'countout' is specified, the final number of elements passed and filtered out will be written to <countfile>.

For example, to write only frames in-between an RMSD of 0.7-0.8 Angstroms for a given input trajectory:

```
trajin ../tz2.truncocct.nc
rms R1 first :2-11
filter R1 min 0.7 max 0.8 out filter.dat
outtraj maxmin.crd
```

The output trajectory will only contain frames that meet the RMSD requirement, and the *filter.dat* file can be used to see which frames those were that were output.

A similar command that can be used with data that already exists (e.g. it has been read in with *readdata*) is *datafilter* (see page 719).

37.11.36. fixatomorder

```
fixatomorder [outprefix <prefix>] [nobox] [parmout <filename>]
             [parmopts <comma-separated-list>]
             [pdborder [hetatm <mask>]] (EXPERIMENTAL)

outprefix <prefix> Write re-ordered topology to
                 <prefix>.<originalname>
```

[nobox] Remove any box information from the re-ordered topology.
parmout <filename> Write re-ordered topology to <filename>
parmopts <list> Options for writing topology file
pdborder (EXPERIMENTAL) Try to reorder atoms according to PDB information.
hetatm <mask> Mark atoms in mask as HETATM, order them after other atoms.

Cpptraj (and most of Amber) expects that atom indices in molecules to increase monotonically. However, occasionally atom indices in molecules can become disordered or non-sequential, in which case cpptraj will print an error message such as the following:

Error: Atom 45 was assigned a lower molecule # (1) than previous atom (2).

and:

Error: Could not determine molecule information for <topology file>.

. This command fixes atom ordering so that all atoms in molecules are sequential. The **outprefix** keyword will write out the re-ordered topology with name <name>.<original name>.

For example, given an out of order topology named 'outoforder.parm7' and a corresponding trajectory 'min1.crd', the following will produce a reordered topology named 'reorder.outoforder.parm7' and a reordered trajectory named 'reorder.mdcrd':

```
parm outoforder.parm7
trajin min1.crd 1 10
fixatomorder outprefix reorder
trajout reorder.mdcrd
```

If '**pdborder**' is specified, attempt to organize atoms by PDB information (i.e. Chain ID, original residue numbering, and insertion codes). Atoms optionally specified by '**hetatm** <mask>' will be placed after all other atoms. *Note that the 'pdborder' keyword is still experimental, and requires that the Topology have PDB-type information present.*

37.11.37. fiximagedbonds

fiximagedbonds [<mask>]
 <mask> Mask expression of atoms to check.

Fix bonds that have been split across periodic boundary conditions by imaging. It may be desirable to reimage the coordinates after this with *autoimage*.

37.11.38. gist (Grid Inhomogeneous Solvation Theory)

```
gist [name <dataset name>] [doorder [nop1] [plcut <plcut>]]
[doei] [skipE] [skipS] [refdens <rdval>] [temp <tval>]
[noimage] [gridcntr <xval> <yval> <zval>]
[rmsfit <fitmask>]
[griddim <nz> <ny> <nx>] [gridspacn <spaceval>] [neighborcut <ncut>]
[prefix <filename prefix>] [ext <grid extension>] [out <output suffix>]
[floatfmt {double|scientific|general}] [floatwidth <fw>] [floatprec <fp>]
[intwidth <iw>]
```

[info <info suffix>]
 [nopme|pme [cut <cutoff>] [dsumtol <dtol>] [ewcoeff <coeff>]
 [erfcdx <dx>] [skinnb <skinnb>] [ljswidth <width>]
 [order <order>] [nfft <nfft1>,<nfft2>,<nfft3>]]
 [name <dataset name>] Name for output data sets.
 [doorder] Calculate the water order parameter [770] for each voxel.
 [nopl] If specified, do not use the pair list for the order calculation (may be much slower).
 [plcut <plcut>] Pair list cutoff for order calculation (default 10.0 Ang.).
 [doeij] Calculate the triangular matrix representing the water-water interactions between pairs of voxels (see below).
 [skipE] Skip all energy calculations (cannot be specified with 'doeij').
 [skipS] Skip all entropy calculations.
 [refdens rdval>] Reference density of bulk water, used in computing g_O , g_H , and the translational entropy. Default is 0.0334 molecules/Å³.
 [temp <tval>] Temperature of the input trajectory.
 [noimage] Disable distance imaging in energy calculation.
 [gridcntr <xval> <yval> <zval>] Coordinates (Å) of the center of the grid (default 0.0, 0.0, 0.0).
 [rmsfit <fitmask>] If specified, grid will be centered and rotated to follow atoms selected by <fitmask>.
 [griddim <nx> <ny> <nz>] Grid dimensions (number of bins/voxels) along each coordinate axis (default 40, 40, 40).
 [gridspacn <spaceval>] Grid spacing (linear dimension of each voxel) in Angstroms. Values greater than 0.75 Å are not recommended (default 0.5 Å).
 [neighborcut <ncut>] Cutoff in Å for determining solvent O-O neighbors (default 3.5 Å).
 [prefix <filename prefix>] Output file name prefix (default ``gist``).
 [ext <grid extension>] Output grid file name extension (default ``.dx``).
 [out <output suffix>] Suffix for main GIST output file name. If not specified, output file will be set to '<prefix>-output.dat'.
 [floatfmt {double|scientific|general}] Format for floating point values in GIST output file: double (regular fixed decimal point), scientific, or general (default, chooses fixed or scientific, whichever fits better).
 [floatwidth <fw>] Changes width of floating point values in GIST output file. Default is no width restriction.
 [floatprec <fp>] Changes precision of floating point values in GIST output file. Default is to use whatever the system default is.
 [intwidth <iw>] Changes width of integer values in GIST output file. Default is no width restriction.

- [info <info suffix>] Suffix for main GIST info file name. If not specified, info will be written to standard output.
- [oldnnvolume] Use the old reference volume for the nearest neighbor entropy, instead of the more precise new implementation.
- [nnsearchlayers <nlayers>] Number of layers of neighboring voxels that should be used when searching for nearest neighbors. This has to be at least 1 to obtain the correct entropy. Higher values can help to obtain better convergence of the translational and 6D entropy with little sampling or fine grid spacings, but increase the calculation time (default 1).
- [solute <mask>] Selection mask for the solute. All other molecules will be solvent. If this is omitted, the standard solute/solvent assignment will be used.
- [solventmols <MOLS>] Comma-separated list of names of solvent molecules. Energies will be computed per solvent molecule. For the entropy, only the main solvent (the first one) will be used. Use, e.g., solventmols WAT,NA,CL for a GIST calculation including ions. This needs to be specified if there is more than one solvent species.
- [nocom] Do not use the center of mass to define the molecular position. Instead, use the first atom in rigidatoms. Use this flag to restore the behavior of old GIST runs.
- [rigidatoms <CENTRAL> <SUBST1> <SUBST2>] Specifies how to define the molecular orientation for the entropy. By default, a simple heuristic will be used. This works for water, but not for all solvents. The atoms should be representative of the molecular orientation and should not be collinear. Note that the central atom goes first. For water, the default is equivalent to rigidatoms O H1 H2, corresponding to H1-O-H2 as the rigid substructure.
- [nopme] Do not use particle mesh Ewald for the non-bonded calculation (default).
- [pme] Use particle mesh Ewald for the non-bonded electrostatics calculation. The van der Waals energy will be calculated using a long-range correction for periodicity. Does not support doiij.
- cut <cutoff> Direct space cutoff in Angstroms (default 8.0).
- dsumtol <dtol> Direct sum tolerance (default 0.00001). Used to determine Ewald coefficient.
- ewcoeff <coeff> Ewald coefficient in 1/Ang.
- erfcdx <dx> Spacing to use for the ERFC splines (default 0.0002 Ang.).
- skinnb Used to determine pairlist atoms (added to cut, so pairlist cutoff is cut + skinnb); included in order to maintain consistency with results from sander.
- ljswidth <width> If specified, use a force-switching form for the Lennard-Jones calculation from <cutoff>--<width> to <cutoff>.
- order <order> Spline order for charges.

nfft <nfft1>,<nfft2>,<nfft3> Explicitly set the number of FFT grid points in each dimension. Will be determined automatically if not specified.

DataSet Aspects:

[gO] Number density of oxygen centers found in the voxel, in units of the bulk density.

[gH] Number density of hydrogen centers found in the voxel in units of the reference bulk density.

[Esw] Mean solute-water interaction energy density.

[Eww] Mean water-water interaction energy density.

[dTStrans] First order translational entropy density.

[dTSorient] First order orientational entropy density.

[dTSSix] First order six-dimensional entropy density.

[neighbor] Mean number of waters neighboring the water molecules found in this voxel multiplied by the voxel number density.

[dipole] Magnitude of mean dipole moment (polarization).

[order] Average Tetrahedral Order Parameter.

[dipolex] x-component of the mean water dipole moment density

[dipoley] y-component of the mean water dipole moment density

[dipolez] z-component of the mean water dipole moment density

[Eij] Water-water interaction matrix.

[PME] (pme only) Mean water energy on the GIST grid.

[U_PME] (pme only) Mean solute energy on the GIST grid.

DataSets if the main solvent is not water:

[gELEM] For every element **ELEM** in the main solvent, the atomic density relative to rho0 (e.g., gC and gH for benzene).

DataSets if there are multiple solvents:

[g_mol.NAME] for every solvent species **NAME** (e.g., g_mol.WAT and g_mol.NA if solventmols WAT,NA was specified).

[Esw_mol.NAME] for every solvent species **NAME** (e.g., Esw_mol.WAT and Esw_mol.NA if solventmols WAT,NA was specified).

[Eww_mol.NAME] for every solvent species **NAME** (e.g., Eww_mol.WAT and Eww_mol.NA if solventmols WAT,NA was specified).

Grid Inhomogeneous Solvation Theory [771, 772] (GIST) is a method for analyzing the structure and thermodynamics of solvent in the vicinity of a solute molecule. The current implementation works for only water, but the method can be generalized to other solvents whose molecules are rigid like water, such as chloroform or dimethylsulfoxide (DMSO). GIST post-processes explicit solvent simulation data to create a three-dimensional mapping of water density and thermodynamic properties within a region of interest, which is defined by a user-specified 3D rectangular grid. The small grid boxes are referred to as voxels, and each voxel is associated with solvent properties. (See Fig. 37.1.) The GIST implementation incorporated into AmberTools *cpptraj* also calculates a number of other local water properties, as listed below. GIST works for the nonpolarizable water models currently supported by AMBER.

In order to carry out a GIST calculation, you must have a trajectory file generated with explicit water, as well as the corresponding topology file. To generate the most readily interpretable results, it is recommended that the solute (e.g., a protein) be restrained into essentially one conformation. GIST will then provide information about

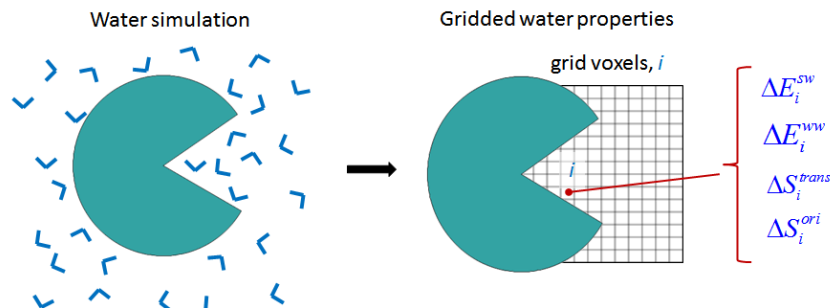


Figure 37.1.: Diagram, in 2D, of GIST's gridded water properties in a binding site.

the structure and thermodynamics of the solvent for that conformation. For a room-temperature simulation of a solvent-exposed binding site, and a grid-spacing of 0.5 Å, it is recommended that the simulation be at least 10-20 ns in duration, and it is also a good idea to check for convergence of the GIST properties you are interested in by loading and then processing successively more frames of your trajectory file. Because GIST assumes that the solute of interest comprises all molecules in the simulation that are not waters, it is a good idea to remove all counterions and cosolutes with cpptraj's strip command before running GIST. A sample series of cpptraj commands for running GIST is provided below.

Although it is not mandatory to supply values of **gridentr**, **griddim** and **gridspcn**, these parameters should be carefully chosen, because they determine the region to be analyzed (**gridentr** and **griddim**) and the spatial resolution and convergence properties of the results (**gridspcn**). In particular, although smaller grid spacings will give finer spatial resolution, longer simulation times will be needed to converge the properties in the smaller voxels that result. A larger grid spacing will allow earlier convergence, but will smooth the spatial distributions. When computing the sum over voxel values in a larger region, the result is independent of the grid spacing as long as **nlayers** is high enough for the given sampling.

The reference density of water (**rdval**) is taken by default to be the experimental number density of pure water at 300 K and 1 atm. However, different water models may yield slightly different bulk densities under these conditions, and the density also depends on T and P. If you know that the bulk density of the water model you are using, at the T and P of your simulation, deviates significantly from 0.0334 water molecules/Å³, it would be advisable to supply the actual value with the **refdens** keyword, instead of allowing GIST to supply the default value.

For GIST, a GPU accelerated version is available, in which the interaction energy is calculated using CUDA. When using the GPU accelerated version of GIST, the **doeij** keyword is not available. It is recommended to use a grid covering the entire box, when using the GPU implementation. You may also choose a smaller grid, but all interaction energies, i.e., each atom with each atom, will always be calculated independent of the chosen grid. This ensures optimum performance when calculating the interaction energies. Thus, the additional time required to calculate the order parameters (**doorder**) is negligible.

The nonbonded energy calculation can also be accelerated using particle mesh Ewald via the **pme** keyword (CPU only).^[773]

GIST Output

GIST generates a main output file and a collection of grid data files that by default are in Data Explorer format (.dx); this can be changed via the **ext** keyword. These grid files enable visualization of the various gridded quantities, such as with the program VMD ^[774]. If the **doeij** keyword is provided, GIST also writes out a matrix of water-water interactions between pairs of voxels. In addition, run details are written to stdout, which can be

redirected into a log file.

Note that a number of quantities are written out as both densities and normalized quantities. For example, the output file includes both the solute-water energy density and the normalized (per water) solute-water energy. In all cases, the normalized quantity at voxel i , $X_{i,norm}$ is related to the corresponding density, $X_{i,dens}$, by the relationship $X_{i,norm} = \rho_i X_{i,dens}$, where ρ_i is the number density of water in the voxel. The normalized quantity provides information regarding the nature of the water found in the voxel. The density has the property that, if the grid extended over the entire simulation volume, the total system quantity would be given by $X_{tot} = V_{voxel} \sum_i X_{i,dens}$, where V_{voxel} is the volume of one grid voxel.

The main output file takes the form of a space-delimited-variable file, where each row corresponds to one voxel of the grid. This file can easily be opened with and manipulated with spreadsheet programs like Excel and LibreOffice Calc. The columns are as follows.

- **index** - A unique, sequential integer assigned to each voxel
- **xcoord** - x coordinate of the center of the voxel (Å)
- **ycoord** - y coordinate of the center of the voxel (Å)
- **zcoord** - z coordinate of the center of the voxel (Å)
- **population** - Number of water molecule, n_i , found in the voxel over the entire simulation. A water molecule is deemed to populate a voxel if its oxygen coordinates are inside the voxel. The expectation value of this quantity increases in proportion to the length of the simulation.
- **g_O** - Number density of oxygen centers found in the voxel, in units of the bulk density (rdval). Thus, the expectation value of **g_O** for a neat water system is unity.
- **g_H** - Number density of hydrogen centers found in the voxel in units of the reference bulk density ($2 \times \text{rdval}$). Thus, the expectation value of **g_H** for a neat water system would be unity.
- **g_ELEM** - (if the main solvent is not water) Density of every further element **ELEM** in the main solvent. Scaled such that the expectation value in pure solvent is unity.
- **g_mol_NAME** - (if there is more than one solvent) Density of every solvent species **NAME** specified in solventmols. Scaled by **rho0**.
- **dTStrans-dens** - First order translational entropy density (kcal/mole/Å³), referenced to the translational entropy of bulk water, based on the value rdval.
- **dTStrans-norm** - First order translational entropy per water molecule (kcal/mole/molecule), referenced to the translational entropy of bulk water, based on the value rdval. The quantity **dTStrans-norm** equals **dTStrans-dens** divided by the number density of the voxel in units of number/Å³.
- **dTSorient-dens** - First order orientational entropy density (kcal/mole/Å³), referenced to bulk solvent (see below).
- **dTSorient-norm** - First order orientational entropy per water molecule (kcal/mole/water), referenced to bulk solvent (see below). This quantity equals **dTSorient-dens** divided by the number density of the voxel.
- **Esw-dens** - Mean solute-water interaction energy density (kcal/mole/Å³). This is the interaction of the solvent in a given voxel with the entire solute. Both Lennard-Jones and electrostatic interactions are computed without any cutoff, within the minimum image convention but without Ewald summation. This quantity is referenced to bulk, in the trivial sense that the solute-solvent interaction energy is zero in bulk.
- **Esw-norm** - Mean solute-water interaction energy per water molecule (kcal/mole/molecule). This equals **Esw-dens** divided by the number density of the voxel.

- **Eww-dens** - Mean water-water interaction energy density, scaled by $\frac{1}{2}$ to prevent double-counting, and not referenced to the corresponding bulk value of this quantity (see below). This quantity is one half of the mean interaction energy of the water in a given voxel with all other waters in the system, both on and off the GIST grid, divided by the volume of the voxel (kcal/mole/Å³). Unless PME is used, both Lennard-Jones and electrostatic interactions are computed without any cutoff, within the minimum image convention.
- **Esw_mol_NAME-dens** and **Esw_mol_NAME-norm** - (if there are multiple solvent species) Mean solute-solvent energy per molecule of species **NAME**, for each solvent specified in **solventmols**. Follows the same conventions as **Esw**.
- **Eww_mol_NAME-dens** and **Eww_mol_NAME-norm** - (if there are multiple solvent species) Mean solvent-solvent energy per molecule of species **NAME**, for each solvent specified in **solventmols**. Follows the same conventions as **Eww**.
- **PME-norm** - (Only if PME was used) Mean PME solvent energy per water molecule (kcal/mole/molecule). This equals **PME-dens** divided by the number density of the voxel.
- **PME-dens** - (Only if PME was used) Mean PME solvent energy density (kcal/mole/Å³). This corresponds roughly to **Eww-norm** plus one half of **Esw-norm** in a non-PME GIST calculation.
- **Eww-norm** - Mean water-water interaction energy, normalized to the mean number of water molecules in the voxel (kcal/mole/water). See prior column definition for details.
- **Dipole.x-dens** - x-component of the mean water dipole moment density (Debye/Å³).
- **Dipole.y-dens** - y-component of the mean water dipole moment density (Debye/Å³).
- **Dipole.z-dens** - z-component of the mean water dipole moment density (Debye/Å³).
- **Dipole-dens** - Magnitude of mean dipole moment (polarization) (Debye/Å³).
- **Neighbor-dens** - Mean number of waters neighboring the water molecules found in this voxel multiplied by the voxel number density. Two waters are considered neighbors if their oxygens are within 3.5 angstroms of each other. For any given frame, the contribution to the average is set to zero if no water is found in the voxel (units of number/Å³).
- **Neighbor-norm** - Mean number of neighboring water molecules, per water molecule found in the voxel (units of number per water).
- **Order-norm** - Average Tetrahedral Order Parameter [770], q_{tet} , for water molecules found in the voxel, normalized by the number of waters in the voxel. The order parameter for water i in a given frame is given by: $q_{tet}(i) = 1 - \frac{3}{8} \sum_{j=1}^3 \sum_{k=j+1}^4 (\cos\phi_{ijk} + \frac{1}{3})^2$ where j and k index the 4 closest water neighbors to water i , and ϕ_{ijk} is the angle formed by water i , j , and k . If the **doorder** keyword is not provided or is set to FALSE, then this calculation will not be done, and the entries in this column will be set to zero.

Grid files are provided for all computed quantities listed above, except that the normalized quantities are not included. The filenames are as follows: gist-gO.dx, gist-gH.dx, gist-dTStrans-dens.dx, gist-dTSorient-dens.dx, gist-Esw-dens.dx, gist-Eww-dens.dx, gist-dipolex-dens.dx, gist-dipoley-dens.dx, gist-dipolez-dens.dx, gist-dipole-dens.dx, gist-neighbor-dens.dx, gist-neighbor-norm.dx, gist-order-norm.dx. If the **doorder** keyword is not provided, then the data in gist-order-norm.dx will all be zeroes. Note that the file of voxel water densities, gist-gO.dx, can be used as input to the program Placevent [775], in order to define spherical hydration sites based on the density distribution. More detailed descriptions of the files follows:

- **gist-dipole-dens.dx** - Magnitude of mean dipole moment (polarization) (Debye/Å³).
- **gist-dipolex-dens.dx** - X-component of the mean water dipole moment density (Debye/Å³).
- **gist-dipoley-dens.dx** - Y-component of the mean water dipole moment density (Debye/Å³).

- **gist-dipolez-dens.dx** - Z-component of the mean water dipole moment density (Debye/Å³).
- **gist-dTSorient-dens.dx** - Density weighted first order orientational entropy (kcal/mole/Å³). The more negative the isovalue, the more restricted or unfavorable the orientation of the water is. The value at bulk density is expected to be zero, most values will fall between 0 and -1.
- **gist-dTStrans-dens.dx** - Density weighted first order translational entropy (kcal/mole/Å³). The more negative isovalue, the more restricted the water is positionally. The value at bulk density is expected to be zero, most values will fall between 0 and -1.
- **gist-Esw-dens.dx** - Density weighted Solute-Water interaction energy (kcal/mole/Å³). This is the interaction of the water with the entire solute. Both Lennard-Jones and electrostatic interactions are computed without any cutoff, within the minimum image convention. The more negative the number, the more favorable the interaction between the water and solute is. Isovalues will be negative numbers.
- **gist-Eww-dens.dx** - Density weighted Water-Water interaction energy (kcal/mole/Å³). This is the interaction of the water with the entire solvent. Both Lennard-Jones and electrostatic interactions are computed without any cutoff, within the minimum image convention. The more negative the number, the more favorable the interaction between water within this voxel and all other water molecules is. Isovalues will be negative numbers.
- **gist-gH.dx** - Number density of hydrogen centers found in the voxel, in units of the bulk density. The expectation value of g_H for a neat water system is unity. The units of this dx file are the density/bulk density. Therefore an isovalue of 1 represents every voxel at or greater than bulk density.
- **gist-gO.dx** - Number density of oxygen centers found in the voxel, in units of the bulk density. The expectation value of g_O for a neat water system is unity. The units of this dx file are the density/bulk density. Therefore an isovalue of 1 represents every voxel at or greater than bulk density.
- **gist-neighbor-norm.dx** - Mean number of neighboring water molecules, per water molecule found in the voxel (units of number per water). Two waters are considered neighbors if their oxygens are within 3.5 Angstroms of each other.
- **gist-order-norm.dx** - Average Tetrahedral Order Parameter for water molecules found in the voxel, normalized by the number of waters in the voxel. **doorder** must be declared in the command line for this file to be produced.

Similar grid files with other computed quantities can be generated by reading the gist.out file into a spreadsheet program, processing the numbers to generate a new column of voxel data of interest, and writing this column to an ascii text file. Then the Perl script write_dx_file.pl, which should be available on the GIST tutorial web-site, may be used to read in the column of data and create the corresponding dx file. The input format, and an example, are as follows:

```
./write_dx_file.pl [filename] [x-dimension y-dimension z-dimension]
[x-origin y-origin z-origin] [grid spacing]
./write_dx_file.pl file.dat 40 40 40 13.0 13.0 13.0 0.75
```

If the **doij** keyword is provided, GIST also writes a large file, Eww_ij.dat, containing the mean water-water interaction energies between pairs of voxels, scaled by ½. (See below.) This file has three columns. The first two columns are voxel indexes, *i*, *j*, where *j* > *i*, so that no pair appears more than once, and the third column is the mean interaction energy (kcal/mole) of water in voxels *i* and *j*, scaled by ½. If the occupancy of either voxel is 0, such as for voxels covered by solute atoms, then the interaction energy is zero. In order to save space, such interactions are omitted from the file.

Sample cpptraj input file to run GIST

Water Model	Mean Energy (E _{ww-norm}) (kcal/mol/water)	Number Density (Å ⁻³)
TIP3P	-9.533	0.0329
TIP4PEW	-11.036	0.0332
TIP4P	-9.856	0.0332
TIP5P	-9.596	0.0329
Tip3PFW	-11.369	0.0334
SPCE	-11.123	0.0333
SPCFW	-11.873	0.0329

Table 37.3.: Water model energy and density.

The following input file, `gist.in`, causes `cpptraj` to read a parameter file named `topology.top`; read in the first 5000 frames of the trajectory file named `trajectoryfile.mdcrd`; strip out all Na and Cl ions; and carry out a GIST run which computes order parameters, uses a 41x41x45 grid centered at (25.0, 31.0, 30.0) with a spacing of 0.5 Å, uses the default bulk water density of 0.0334 molecules/Å³, and generates the main output file `gist.out`.

```
parm topology.top
trajin trajectoryfile.mdcrd 1 5000
strip @Na
strip @Cl
gist doorder doeij gridcntr 25.0 31.0 30.0 griddim 41 41 45
    gridspacn 0.50 out gist.out
go
```

To execute this run in the background, use

```
cpptraj<gist.in>gist.log& or cpptraj {i gist.in}>gist.log&
```

Referencing GIST results to unperturbed (bulk) water

Inhomogeneous fluid solvation theory, which is the basis of GIST, is designed to provide information on how water structure and thermodynamics around a solute molecule, such as a protein, are changed relative to the structure and thermodynamics of unperturbed (bulk) water. Accordingly, the quantities reported by GIST are most informative when the results are referenced to the corresponding bulk water properties. For the orientational entropy, the reference value is the same regardless of water model or conditions, because the first order orientational distribution of water in the bulk is always uniform. Therefore, the GIST results for orientational entropies are already referenced to bulk. However, `cpptraj` reports unreferenced values for those GIST quantities whose reference values depend upon the water model and the simulation conditions; i.e., the energies. The translational entropy as well as the number densities will be referenced to bulk using the input referenced density or the default density value of 0.0334. The table below provides useful reference values for these quantities, computed for various water models at P=1atm, T=300K, using GIST in order to ensure a consistent minimum image treatment of periodic boundary conditions.

Users running calculations under significantly different conditions, or with different water models, should consider generating their own reference quantities by applying GIST to a simulation of pure water under their conditions of interest. The quantities of interest can then be obtained in their most precise available form by averaging over voxels, for the pure water simulation. If the quantity of interest is Q , then its average reference value is $Q_{reference} = \frac{\sum n_i Q_i}{\sum n_i}$, where Q_i and n_i are, respectively, GIST's reported values of the quantity and the population in

voxel i . The densities, ρ_i , are referenced to the corresponding bulk densities, ρ^o , as $g_i = \rho_i/\rho^o$, while the energy and entropy terms are referenced by subtracting their bulk values.

Note that the Eww reference needs to be subtracted from the normalized water-water energy **Eww-norm**. A referenced **Eww-dens** can be obtained by multiplying the referenced **Eww-norm** by the solvent number density $\rho = g_o\rho^o = \frac{N_w}{N_f V_{vox}}$, where N_w is the number of water molecules in a voxel (**population**), N_f is the number of frames, and V_{vox} is the voxel volume.

Interpreting GIST results

GIST provides access to the first order entropies and the first- and second-order energies of inhomogeneous fluid solvation theory. Non-zero higher-order entropies exist but are not yet computationally accessible. However, for a pairwise additive force-field, such as those listed in the Table above, the energy is fully described at the second order provided by GIST.

GIST is a research tool, and its applications (to, for example, protein-ligand binding and protein function) are still being explored. The following general comments may be helpful to users studying GIST results.

1. The water in voxels near a solute (e.g., a protein) almost always has unfavorable water-water interaction energies, relative to bulk, simply because the solute displaces water, resulting in fewer proximal water-water interactions.

2. The unfavorable water-water energies mentioned in [771] may be balanced by favorable water-solute interactions. If they are not, as may occur especially for voxels in small, hydrophobic pockets, then the net energy of the water in the voxel may be unfavorable relative to bulk, in which case a ligand which displaces water from the voxel into bulk may get a boost in affinity.

3. Because the first order orientational distribution of bulk water is uniform, and a nonuniform distribution always has lower entropy than a uniform one, the solute can only lower the orientational entropy of water, relative to bulk. Thus, this term always opposes solvation, and displacing oriented water into the bulk is always favorable from the standpoint of orientational entropy.

4. Localized water, which corresponds to voxels with high water density, has a low first order translational entropy, and the translational entropy around a solute is lower than that in bulk, as a nonuniform translational distribution takes the place of the uniform translational distribution of bulk water.

5. The displacement of highly oriented (low orientational entropy) and localized (low translational entropy) water into bulk leads to a favorable increase in these entropy terms.

6. However, highly oriented and localized water is often the consequence of strongly favorable polar interactions, such as hydrogen-bonding, between water and the solute. As a consequence, the net favorability of displacing such water is frequently a balance between favorable entropic consequences and unfavorable energetic consequences.

7. The water-water energy associated with a given voxel accounts for the interactions of the waters in this voxel with all other waters in the system, including waters in other voxels. This quantity is multiplied by $\frac{1}{2}$, so that, in a pure-water system where the GIST grid covers the entire simulation box, the sum over all voxels equals the correct mean water-water interaction energy. Note that Reference [772] does not include this factor of $\frac{1}{2}$.

8. For a typical GIST application, in which the grid occupies only part of the simulation box, the energy bookkeeping can become complicated, as discussed in Section II.B.3 (page 044101-6) of Reference [772]. That section also explains how one can compute the water-water energy associated with a region R defined by a set of voxels, E_{WW}^R . The regional water-water energy, on a normalized (per water) basis, is given by $E_{WW}^R = 2(\sum_{i \in R} E_{i,WW} - \sum_{i \in R} \sum_{j \in R, j > i} E_{i,j,WW})$ where $i \in R$ means that voxel i is in region R , $E_{i,WW}$ is the value of Eww-norm for voxel i , and $E_{i,j,WW}$ is the value of the water-water interaction energy between voxels i and j , taken from the file Eww_ij.dat. The extra factor of 2 in the present formula, relative to that in the paper, results from application of an extra factor of $\frac{1}{2}$ to the reported water-water interaction energies here.

9. If the GIST grid contains the entire solute and the calculation is sufficiently converged, the energy and first-order entropy of hydration can be calculated by numerical integration. E.g., the energy of hydration is $\Delta E_{hyd} = \sum_{voxels} (E_{sw}^{dens} + E_{ww}^{dens}) \times V_{vox}$. For this, Eww has to be referenced carefully, since numerical inaccuracies can add up quickly. It can be advisable to omit all voxels above a certain distance to the solute to obtain more stable results.

37.11.39. grid

```

grid [out <filename>]
  { data <dsname> | boxref <ref name/tag> <nx> <ny> <nz> |
    <nx> <dx> <ny> <dy> <nz> <dz>
    [ { gridcenter <cx> <cy> <cz> |
      boxcenter |
      maskcenter <mask> |
      rmsfit <mask> [noxalign]} ]
    [box|origin|center <mask>] [negative] [name <gridname>]
    <mask> [normframe | normdensity [density <density>]]
    [pdb <pdbout> [max <fraction>]] [{byres|mymol}]
    [[smoothdensity <value>] [invert]] [madura <madura>]
[out <filename>] File to write out grid to. Use ``.grid'' or
  ``.xplor'' extension for XPLOR format, ``.dx'' for OpenDX
  format.

```

Options for setting up grid:

data <dsname> Use previously calculated/loaded grid data set named <dsname>. When using this option there is no need to specify grid bins/spacing/center.

boxref <ref name/tag> <nx> <ny> <nz> Set up grid using box information from a previously loaded reference structure. Currently the only way to set up non-orthogonal grids.

<nx> <dx> <ny> <dy> <nz> <dz> Number of grid bins and spacing in the X/Y/Z directions.

[gridcenter <cx> <cy> <cz>] Location of grid center, default is origin (0.0, 0.0, 0.0).

[boxcenter] Center grid on box center.

[maskcenter <mask>] Center the grid on the atoms selected by <mask>.

[rmsfit <mask>] Perform a best-fit rotation of the grid using the coordinates selected by <mask>.

[noxalign] If specified, grid will not be re-oriented to align with Cartesian axes once binning is finished. Will affect file formats that do not store full unit cell vectors (like Xplor).

Options for offset during grid binning (must center grid at origin):

[box] Offset each point by location of box center prior to gridding. Cannot be used with 'gridcenter'.

[origin] No offset (default)

[center <mask>] Offset each point by center of atoms in <mask> prior to gridding. Cannot be used with 'gridcenter'.

Other options:

[negative] Grid negative density instead of positive density.

[name <gridname>] Grid data set name.

<mask> Mask of atoms to grid.

[normframe] Normalize grid bins by the number of frames.

[normdensity [density <density>]] Normalize grid bins by density:
 $\text{GridBin} = \text{GridBin} / (\text{Nframes} * \text{BinVolume} * \text{density})$. Default particle density (molecules/Ang³) for water based on 1.0 g/mL.

[pdb <pdbout> [max <fraction>]] Write a pseudo-PDB of grid points that have density greater than <fraction> (default 0.80) of the grid max value.

[{byres|bymol}] Grid the centers of mass of residues or molecules selected by <mask>.

Less common options:

[smoothdensity <smooth>] Used to smooth density. The smoothing takes the form of $\text{GridBin} = 0$ if $\text{GridBin} < \text{smooth}$, otherwise $\text{GridBin} = \text{GridBin} - (\text{GridBin} * \exp[-(\text{GridBin} - \text{smooth})^2 / (0.2 * \text{smooth}^2)])$.

[invert] (Only used if smoothdensity also used) Do inverse smoothing (i.e. if $\text{GridBin} > \text{smooth}$).

[madura <madura>] Grid values lower than <madura> become flipped in sign, exposes low density.

Data Sets Created:

<dsname> Grid data set.

Create a grid representing the histogram of atoms in *mask1* on the 3D grid that is "*nx* * *x-spacing* by *ny* * *y-spacing* by *nz* * *z-spacing* angstroms (cubed). By default the grid is centered at the origin unless **gridcenter** is specified. Grid points can be offset by either the box center (using **box**) or the center of specified atoms (using **center** <mask>); if either of these options are used the grid must be centered at the origin. Note that the *bounds* command (on page 763) can be very useful for determining grid dimensions.

Note that when calculating grid densities for things like solvent/ions, the solute of interest (about which the atomic densities are binned) should be rms fit, centered and imaged prior to the **grid** call in order to provide any meaningful representation of the density. If the optional keyword **negative** is also specified, then these density will be stored as negative numbers. Output can be in the XPLOR or OpenDX data formats.

Examples

Example 1: Grid water density around a solute. The solute is imaged to the origin and rms fit to the first frame. The grid will be centered on the origin as well.

```
trajin tz2.truncocct.nc
autoimage origin
rms first :1-13
# Create average of solute to view with grid.
average avg.mol2 :1-13
grid out.dx 20 0.5 20 0.5 20 0.5 :WAT@O
```

Example 2: Grid water density around a solute. The grid is centered on the solute.

```
trajin tz2.truncocct.nc
autoimage
grid out.dx 20 0.5 20 0.5 20 0.5 :WAT@O maskcenter :1-13
```

Example 3: Grid water density around a solute. The grid is centered on the solute and rms-fit. The density obtained should be equivalent to the first example.

```

trajin tz2.truncocct.nc
image :WAT
grid out.dx 20 0.5 20 0.5 20 0.5 :WAT@O rmsfit :1-13

```

Example 4: Generate grid from bounds command.

```

trajin tz2.ortho.nc
autoimage
rms first :1-13&!@H= mass
bounds :1-13 dx .5 name MyGrid out bounds.dat
average bounds.mol2 :1-13
# Save coordinates for second pass.
createcrd MyCoords
run
# Grid using grid data set from bounds command.
crdaction MyCoords grid bounds.xplor data MyGrid :WAT@O

```

Example 5: Create non-orthogonal grid based on the box.

```

trajin tz2.truncocct.nc
reference ../tz2.truncocct.nc [REF]
autoimage triclinic
grid nonortho.dx boxref [REF] 50 50 50 :WAT@O pdb nonortho.pdb

```

37.11.40. hbond

```

hbond [<dsname>] [out <filename>] [<mask>] [angle <acut>] [dist <dcut>]
[donormask <dmask> [donorhmask <dhmask>]] [acceptormask <amask>]
[avgout <filename>] [printatomnum] [nointramol] [image]
[solventdonor <sdmask>] [solventacceptor <samask>]
[solvout <filename>] [bridgeout <filename>] [bridgebyatom]
[series [uuseries <filename>] [uvseries <filename>]]
[bseries [bseriesfile <filename>]]
[uuresmatrix [uuresmatrixnorm {none|frames}] [uuresmatrixout <file>]]
[splitframe <comma-separated-list>]

```

[<dsname>] Data set name.

[out <filename>] Write # of solute-solute hydrogen bonds (aspect [UU]) vs time to this file. If searching for solute-solvent hydrogen bonds, write # of solute-solvent hydrogen bonds (aspect [UV]) and # of bridging solvent molecules (aspect [Bridge]), as well as the residue # of the bridging solvent and the solute residues being bridged with format '<solvent resnum>(<solute res1>+<solute res2>+...+),... ' (aspect [ID]).

[<mask>] Atoms to search for solute hydrogen bond donors/acceptors.

[angle <acut>] Angle cutoff for hydrogen bonds (default 135°). Can be disabled by specifying -1.

[dist <dcut>] Distance cutoff for hydrogen bonds (acceptor to donor heavy atom, default 3.0 Å).

[donormask <dmask>] Use atoms in <dmask> as solute donor heavy atoms. If 'donorhmask' not specified only atoms bonded to hydrogen will be considered donors.

- [donorhmask <dhmask>] Use atoms in <dhmask> as solute donor hydrogen atoms. Should only be specified if 'donormask' is. Should be a 1 to 1 correspondence between donormask and donorhmask.
- [acceptormask <amask>] Use atoms in <amask> as solute acceptor atoms.
- [avgout <filename>] Write solute-solute hydrogen bond averages to <filename>.
- [printatomnum] Add atom numbers to the output, in addition to residue name, residue number and atom name.
- [nointramol] Ignore intramolecular hydrogen bonds.
- [image] Turn on imaging of distances/angles.
- [solventdonor <sdmask>] Use atoms in <sdmask> as solvent donors. Can specify ions as well.
- [solventacceptor <samask>] Use atoms in <samask> as solvent acceptors. Can specify ions as well.
- [solvout <filename>] Write solute-solvent hydrogen bond averages to <filename>. If not specified and 'avgout' is, solute-solvent hydrogen bonds averages will be written to that file.
- [bridgeout <filename>] Write information on detected solvent bridges to <filename>. If not specified, will be written to same place as 'solvout'.
- [bridgebyatom] Report bridging results by atom instead of by residue.
- [series] Save hydrogen bond formed (1.0) or not formed (0.0) per frame for any detected hydrogen bond. Solute-solute hydrogen bonds are saved with aspect [solutehb], solute-solvent hydrogen bonds are saved with aspect [solventhb].
- [uuseries <filename>] File to write solute-solute hbond time series data to.
- [uvseries <filename>] File to write solute-solvent hbond time series data to.
- [bseries] Save bridge formed (1.0) or not formed (0.0) per frame for any detected bridge. Bridges are saved with aspect [bridge-<indexlist>], where <indexlist> is an underscore ('_') delimited list of bridged atom/residue numbers (depending on bridgebyatom).
- [bseriesfile <filename>] File to write bridge time series data to.
- [uuresmatrix] If specified, create a matrix with aspect [UUresmat] containing # of hydrogen bonds between each possible solute residue pair.
- [uuresmatrixnorm {none|frames}] Control how matrix is normalized: none=no normalization, frames=normalize by total # frames.
- [uuresmatrixout <file>] If specified, write matrix data to specified file.
- [splitframe <comma-separated-list>] If specified, split the average hydrogen bond (avgout, solvout, bridgeout) analysis into sections delimited by the frame numbers. For example, 'splitframe 250,500,1000' will divide analysis into frames 1-249, 250-499, 500-999, and 1000 to end.

Data Sets Created:

- <dsname>[UU] Number of solute-solute hydrogen bonds.
- <dsname>[UV] (only for solventdonor/solventacceptor) Number of solute-solvent hydrogen bonds.
- <dsname>[Bridge] (only for solventdonor/solventacceptor) Number of bridging solvent molecules.
- <dsname>[ID] (only for solventdonor/solventacceptor) String identifying bridging solvent residues and the solute residues they bridge.
- <dsname>[solutehb] (series only) Time series for solute-solute hydrogen bonds; 1 for present, 0 for not present.
- <dsname>[solventhb] (series only) Time series for solute-solvent hydrogen bonds; 1 for present, 0 for not present.
- <dsname>[bridge_<indexlist>] (bseries only) Time series for bridge; 1 for present, 0 for not present. The <indexlist> is an underscore ('_') delimited list of bridged atom/residue numbers (depending on bridgebyatom).
- <dsname>[UUresmatr] (uuresmatrix only) Solute residue hydrogen bond matrix.

*Note that **series** data sets are not generated until hydrogen bonds are actually determined (i.e. **run** is called).*

Determine hydrogen bonds in each coordinate frame using simple geometric criteria. A hydrogen bond is defined as being between an acceptor heavy atom A, a donor hydrogen atom H, and a donor heavy atom D. If the A to D distance is less than or equal to the distance cutoff and the A-H-D angle is greater than or equal to the angle cutoff a hydrogen bond is considered formed. Imaging of distances/angles is not performed by default, but can be turned on using the **image** keyword.

Potential hydrogen bond donor/acceptor atoms are searched for as follows:

1. If just <mask> is specified donors and acceptors will be automatically determined from <mask>.
2. If **donormask** is specified donors will be determined from <dmask> (only atoms bonded to hydrogen will be considered valid). Optionally, **donorhmask** can be used in conjunction with **donormask** to explicitly specify the hydrogen atoms bonded to donor atoms. Acceptors will be automatically determined from <mask>.
3. If **acceptormask** is specified acceptors will be determined from <amask>. Donors will be automatically determined from <mask>.
4. If both **acceptormask** and **donormask** are specified only <amask> and <dmask> will be used; no searching will occur in <mask>.

Automatic determination of hydrogen bond donors/acceptors uses the simplistic criterion that “hydrogen bonds are FON”, i.e., hydrogens bonded to F, O, and N atoms are considered donors, and F, O, and N atoms are considered acceptors. Intra-molecular hydrogen bonds can be ignored using the **nointramol** keyword.

The number of hydrogen bonds present at each frame will be determined and written to the file specified by **out**. If desired, the bridge [ID] data can be used in conjunction with the **keep** command to generate structures that only contain bridging solvent (37.11.43 on page 799). If the **series** keyword is specified the time series for each hydrogen bond (1 for present, 0 for not present) will also be saved for subsequent analysis (e.g. with **lifetime**, see on page 872); solute-solute hydrogen bonds will be saved to '<dataset name>[solutehb]' and solute-solvent hydrogen bonds will be saved to '<dataset name>[solventhb]'. The data set legends are set with the residues and atoms involved in the hydrogen bonds. In the case of solute to non-specific solvent hydrogen bonds, a V is used in place of solvent.

If **avgout** is specified the average of each solute-solute hydrogen bond (sorted by population) formed over the course of the trajectory is printed with the format:

Acceptor DonorH Donor Frames Frac AvgDist AvgAng

where *Acceptor*, *DonorH*, and *Donor* are the residue and atom name of the atoms involved in the hydrogen bond, *Frames* is the number of frames the bond is present, *Frac* is the fraction of frames the bond is present, *AvgDist* is the average distance of the bond when present, and *AvgAng* is the average angle of the bond when present. The **printatomnum** keyword can be used to print atom numbers as well.

Solute to non-specific solvent hydrogen bonds can be tracked by using the **solventdonor** and/or **solventacceptor** keywords. The number of solute-solvent hydrogen bonds and number of “bridging” solvent molecules (i.e. solvent that is hydrogen bonded to two or more different solute residues at the same time) will also be written to the file specified by **out**. These keywords can also be used to track non-specific interactions with ions. If **avgout** or **solvavg** is specified the average of each solute solvent hydrogen bond will be printed with the format:

Acceptor DonorH Donor Count Frac AvgDist AvgAng

where *Acceptor*, *DonorH*, and *Donor* are either the residue and atom name of the solute atoms or “SolventAcc”/“SolventH”/“SolventDnr” representing solvent, *Count* is the total number of interactions between solute and solvent (note this can be greater than the total number of frames since for any given frame more than one solvent molecule can hydrogen bond to the same place on solute and vice versa), *AvgDist* is the average distance of the bond when present, and *AvgAng* is the average angle of the bond when present. If **avgout** or **bridgeout** is specified information on residues that were bridged by a solvent molecule over the course of the trajectory will be written to <bfilename> with format:

Bridge Res <N0:RES0> <N1:RES1> ... , <X> frames.

where ‘<N0:RES0> ...’ is a list of residues that were bridged (residue # followed by residue name) and <X> is the number of frames the residues were bridged.

hbond Examples

To search for all hydrogen bonds within residues 1-22, writing the number of hydrogen bonds per frame to “nhb.dat” and information on each hydrogen bond found to “avghb.dat”:

```
hbond :1-22 out nhb.dat avgout avghb.dat
```

To search for all hydrogen bonds formed between donors in residue 1 and acceptors in residue 2:

```
hbond donormask :1 acceptormask :2 out nhb.dat avgout avghb.dat
```

To search for all intermolecular hydrogen bonds only and solute-solvent hydrogen bonds, saving time series data to HB:

```
hbond HB out nhb.dat avgout solute_avg.dat \
  solventacceptor :WAT@O solventdonor :WAT \
  solvout solvent_avg.dat bridgeout bridge.dat \
  series uuseries uuhbonds.agr uvseries uvhbonds.agr
```

To search for non-specific hydrogen bonds between solute and ions named Na+:

```
hbond HB-Ion out nhb.agr avgout ion_avg.dat \
  solventacceptor :Na+ solventdonor :Na+
```

37.11.41. image

```
image [origin] [center] [triclinic | familiar [com <commask>]] [<mask>]
  [ bymol | byres | byatom ] [xoffset <x>] [yoffset <y>] [zoffset <z>]
```

[origin] Image to coordinate origin (0.0, 0.0, 0.0); default is to image to box center.

[center] For bymol/byres, image by center of mass; default is to image by first atom position.

[triclinic] Force imaging with triclinic code. This is the default for non-orthorhombic cells.

[familiar [com <commask>]] Image to truncated octahedron shape. If 'com <commask>' is given, image with respect to the center of mass of atoms in <commask>.

[<mask>] Image atoms/residues/molecules in mask.

[bymol] Image by molecule (default).

[byres] Image by residue.

[byatom] Image by atom.

[xoffset <x>] Shift atoms by a factor of <x> in the X-direction.

[yoffset <y>] Shift atoms by a factor of <y> in the Y-direction.

[zoffset <z>] Shift atoms by a factor of <z> in the Z-direction.

Note this command is intended for advanced use; for most cases the *autoimage* command should be sufficient.

For periodic systems only, image molecules/residues/atoms that are outside of the box back into the box. Currently both orthorhombic and non-orthorhombic boxes are supported. A typical use of *image* is to move molecules back into the box after performing *center*. For example, the following commands move all atoms so that the center of residue 1 is at the center of the box, then image so that all molecules that are outside the box after centering are wrapped back inside:

```
center :1
image
```

The xoffset etc. keywords can be used to shift the entire unit cell in a certain direction by the given factor, which can be useful for visualizing trajectories with periodic boundary conditions. For example, to generate a trajectory that is offset by 1.0 box length in the X direction, one could use:

```
image xoffset 1.0
trajout traj.offsetx1.nc
```

37.11.42. jcoupling

```
jcoupling <mask> [outfile <filename>] [kfile <param file>] [out <filename>]
[name <dsname>]
```

<mask> Atom mask in which to search for dihedrals within.

[outfile <filename>] File to write j-coupling values to with fixed format.

[kfile <param file>] File containing Karplus parameters. If not specified will check CPPTRAJHOME, AMBERHOME, and KARPLUS environment variables (see below).

[out <filename>] File to write data set output to.

[name <dsname>] Data set name.

Note data sets are not generated until **run** is called.

Calculate J-coupling values for all dihedrals found within **<mask>** (all atoms if no mask given). In order to use this function, Karplus parameters for all dihedrals which will be calculated must be loaded. By default *cpptraj* will use the data found in either \$CPPTRAJHOME/dat/Karplus.txt or \$AMBERHOME/dat/Karplus.txt; if this is not found *cpptraj* will look for the file specified by the \$KARPLUS environment variable.

In the Karplus parameter file each parameter set consists of two lines for each dihedral with the format:

```
[<Type>] <Name1><Name2><Name3><Name4><A><B><C> [<D>]
<Resname1> [<Resname2> . . .]
```

The first line defines the parameter set for a dihedral. **<Type>** is optional; if not given the form for calculating the J-coupling will be as described by Chou et al.[776]; if 'C' the form will be as described by Perez et al.[777]. The **<NameX>** parameters define the four atoms involved in the dihedral. Each **<NameX>** parameter is 5 characters wide, starting with a plus '+', minus '-' or space ' ' character indicating the atom belongs to the next, previous, or current residue. The remaining 4 characters are the atom name. The parameters **<A>**, ****, **<C>**, and **<D>** are floating point values 6 characters wide describing the Karplus parameters. For the 'C' form A, B, and C correspond to C0, C1, and C2; D is unused and should not be specified. The second line is a list of residue names (4 characters each) to which the dihedral applies. For example:

```
C HA   CA   CB   HB   5.40 -1.37  3.61
ILE VAL
```

Describes a dihedral between atoms HA-CA-CB-HB using the Perez et al. form with constants C0=5.40, C1=-1.37, C2=3.61 applied to ILE and VAL residues.

Output can be in both a fixed format (**outfile <filename>**) and using *cpptraj* data set/data file formatting (**out <filename>**). The fixed format has each dihedral that is defined from **<mask1>** printed along with its calculated J-coupling value for each frame, e.g.:

```
#Frame 1
1 SER HA CA CB HB2 45.334742 4.024759
1 SER HA CA CB HB3 -69.437134 1.829510
...
```

First the frame number is printed, then for each dihedral: Residue number, residue name, atom names 1-4 in the dihedral, the value of the dihedral, the J-coupling value.

In *cpptraj* format, only the J-coupling value is written.

37.11.43. keep

```
keep [ bridgedata <bridge data set> [nbridge <#>] [nobridgewarn]
      [bridgeresname <res name>] bridgeresonly <resrange>] ]
      [keepmask <atoms to keep>] [charge <new charge>]
      [outprefix <prefix>] [nobox] [parmout <filename>]
      [parmopts <comma-separated-list>]

bridgedata <bridge data set> Data set containing bridge ID strings from
the hbond command (37.11.40 on page 794).
nbridge <#> Number of bridging residues to keep (default 1).
nobridgewarn If specified, suppress warnings for when active #
bridges does not equal requested number.
bridgeresname <res name> Name of bridging residues (default
'WAT').
bridgeresonly <range> If specified, only keep bridges that bridge
residues in the <resrange> list.
keepmask <atoms to keep> Mask of atoms to keep.
```

charge <new charge> Scale charges so total charge of remaining atoms matches the specified <new charge>.

outprefix <prefix> Write modified topology to <prefix>.<originalname>

[nobox] Remove any box information from the modified topology.

parmout <filename> Write modified topology to <filename>.

parmopts <list> Options for writing topology file.

Keep only specified atoms (opposite of *strip*). This can also be used in conjunction with output from the *hbond* command to retain solute and only bridging residues (e.g. bridging waters). For example, the following run generates bridging data with the '*hbond*' command in a first pass, then uses the bridge ID data to retain only 1 single bridging water between residues 10 and 11:

```
parm tz2.ortho.parm7
trajin tz2.ortho.nc
# First pass, generate bridge time series
hbond hb solventacceptor :WAT@O solventdonor :WAT out hb.dat
run
# Second pass, retain only frames where the bridge is present
# for residues 10 and 11.
keep bridgedata hb[ID] nbridge 1 bridgeresonly 10,11 parmout keep.parm7
# Write trajectory
trajout keep.nc
run
```

This run reads in bridge ID data from a previous *hbond* run and uses it to keep only residues 10, 11, and a bridging water:

```
parm tz2.ortho.parm7
trajin tz2.ortho.nc
readdata hb.dat
keep keepmask :10,11 bridgedata hb.dat:5 nbridge 1 bridgesonly 10,11 \
  parmout keep.10.11.parm7
trajout keep.10.11.nc
run
```

37.11.44. lessplit

lessplit [out <filename prefix>] [average <avg filename>] <trajout args>

[out <filename prefix>] Write split LES trajectories to <filename prefix>.<X>, where X is an integer.

[average <avg filename>] Write trajectory of averaged LES regions to <avg filename>.

<trajout args> Arguments for output trajectories.

Split and/or average LES trajectory. At least one of '*out*' or '*average*' must be specified. If both are specified they share <trajout args>.

37.11.45. **lie**

```
lie [<name>] <Ligand mask> [<Surroundings mask>] [out <filename>] [nopbc]
    [noelec] [novdw] [cutvdw <cutoff>] [cutelec <cutoff>] [diel <dielc>]
DataSet Aspects:
```

```
[EELEC] Electrostatic energy (kcal/mol).
```

```
[EVDW] van der Waals energy (kcal/mol).
```

For each frame, calculate the non-bonded interactions between all atoms in <Ligand mask> with all atoms in <Surroundings mask>. Electrostatic and van der Waals interactions will be calculated for all atom pairs. A separate electrostatic and van der Waals cutoff can be applied, the default is 12.0 Angstroms for both. <dielc> is an optional dielectric constant. Either the electrostatic or van der Waals calculations can be suppressed via the keywords noelec and novdw, respectively. Periodic boundary conditions (and the minimum image convention) can be abandoned with the “nopbc” keyword. Note, however, that no prior imaging is performed if the frames contain periodic boundaries. This may be useful for instances when you are simulating a microscopic droplets.

The electrostatic interactions are calculated according to a simple shifting function shown below. The data file will contain two data sets—one for electrostatic interactions and one for van der Waals interactions. Periodic topologies and trajectories are required (i.e., explicit solvent is necessary). The minimum image convention is followed.

$$E_{elec} = k \frac{q_i q_j}{r_{ij}} \left(1 - \frac{r_{ij}^2}{r_{cut}^2} \right)^2$$

37.11.46. **lipidorder**

```
order out <filename> [x|y|z] [scd] [unsat <mask>]
    [taildist <filename> [delta <resolution>] tailstart <mask>
    tailend <mask>] <mask0> ... <maskN>
```

out Output file for order parameters: Sx, Sy, Sz (each succeeded by the standard deviation), and two estimates for the deuterium-order parameter |SCD| = 0.5Sz and |SCD| = -(2Sx + Sy)/3. If scd is set then the order parameter directly computed from the C-H vectors is output.

x|y|z Reference axis. (z)

unsat Mask for unsaturated bonds. Sz is calculated for vector Cn-Cn+1. This is only relevant if scd (below) is not set, i.e. order parameters are calculated from carbon position only.

scd Calculate the deuterium-order parameter |SCD| directly from the C-H vectors (masks must contain C-H-H triplets, see below). Otherwise the order parameter is estimated from carbon positions only (masks must contain only relevant carbons).
(false)

taildist Optional output file for end-to-end distances.

delta Optional resolution for taildist. (0.1)

tailstart Mask for the start of the tail. Must be given if taildist.

tailend Mask for the end of the tail. Must be given if taildist.

mask0 ... maskN Masks for each group in the lipid chain.

The order parameters S_x , S_y , S_z and $|SCD|$ are calculated. Carbons must be given in bonding order. If **scd** the masks must be made up of C-H-H triples, hence hydrogens to double bonds must be enumerated twice while methyl groups require an additional mask which will also create two entries in the output.

S_z is the vector joining carbons C_{n-1} and C_{n+1} , S_x the vector normal to the $C_{n-1} - C_n$ and $C_n - C_{n+1}$ plane and S_y is the third axis in the molecular coordinate system. The order parameter is then calculated from $S_c = 0.5 < 3\cos(2\theta) > -1$, where θ is the angle to the chosen reference axis. See example input file.

Example input (all atom names according to CHARMM27 force field for POPC).

sn1 chain: order parameters S_x , S_y , S_z and $|SCD| = 0.5 \times S_z$ and $|SCD| = -(2S_x + S_y)/3$

```
lipidorder out sn1.dat z taildist e2e_sn1.dat delta 0.1 \
tailstart ":POPC@C32" tailend ":POPC@C316" \
":POPC@C32" ":POPC@C33" ":POPC@C34" ":POPC@C35" \
":POPC@C36" ":POPC@C37" ":POPC@C38" ":POPC@C39" \
":POPC@C310" ":POPC@C311" ":POPC@C312" ":POPC@C313" \
":POPC@C314" ":POPC@C315" ":POPC@C316"
```

See also \$AMBERHOME/AmberTools/test/cpptraj/Test_LipidOrder.

37.11.47. lipidscd

```
lipidscd [<name>] [<mask>] [{x|y|z}] [out <file>] [p2]
```

<name> Output data set name.

<mask> Atom mask specifying where to search for lipids.

x|y|z Axis to calculate order parameters with respect to (default z).

out <file> File to write order parameters to.

p2 If specified, report raw <P2> values.

DataSets Generated:

<name>[H1]:<idx> Hold lipid order parameters for each C-H1. Each lipid type will have a different <idx> starting from 0.

<name>[H2]:<idx> Hold lipid order parameters for each C-H2. If no H2, the C-H1 value will be used.

<name>[H3]:<idx> Hold lipid order parameters for each C-H3. If no H3, the C-H2/C-H1 value will be used.

<name>[SDHX]:<idx> Hold standard deviation of lipid order parameters for each C-HX.

Calculate lipid order parameters SCD ($|<P2>|$) for lipid chains in mask <mask>. Lipid chains are identified by carboxyl groups, i.e. O-(C=O)-C1-...-CN, where C1 is the first carbon in the acyl chain and CN is the last. Order parameters will be determined for each hydrogen bonded to each carbon. If 'p2' is specified the raw <P2> values will be reported.

37.11.48. makestructure

```
makestructure <List of Args>
```

Apply dihedrals to specified residues using arguments found in <List of Args>, where an argument is 1 or more of the following arg types:

<sstype keyword>:<res range>

Apply secondary structure type (via phi/psi backbone angles) to residues in given range. If the secondary structure type is a turn, the residue range must correspond to a multiple of 2 residues.

Keyword	phi, psi (deg.)	# residues
alpha	-57.8, -47.0	1
left	-57.8, 47.0	1
pp2	-75.0, 145.0	1
hairpin	-100.0, 130.0	1
extended	-150.0, 155.0	1
typeI	-60.0, -30.0 -90.0, 0.0	2
typeII	-60.0, 120.0 80.0, 0.0	2
typeVIII	-60.0, -30.0 -120.0, 120.0	2
typeI'	60.0, 30.0 90.0, 0.0	2
typeII'	60.0, -120.0 -80.0, 0.0	2
typeVIa1	-60.0, 120.0 -90.0, 0.0	2
typeVIa2	-120.0, 120.0 -60.0, 0.0	2
typeVIb	-135.0, 135.0 -75.0, 160.0	2

<custom ss name>:<res range>[:<phi>:<psi>]

If <phi> and <psi> are given, define a custom secondary structure conformation named <custom.ss> and apply to residues in range. If <custom.ss> has been previously defined then apply it to residues in range.

<custom turn name>:<res range>[:<phi1>:<psi1>:<phi2>:<psi2>]

If <phi1>, <psi1>, <phi2>, and <psi2> are given, defined a custom turn conformation named <custom.turn> and apply to residues in range (range must correspond to a multiple of 2 residues). If <custom.turn> has been previously defined then apply it to residues in range.

<custom dih name>:<res range>[:<dih type>:<angle>]

**<dih type> = alpha beta gamma delta epsilon zeta nu0 nu1 nu2 nu3 nu4
h1p c2p chin phi psi chip omega chi2 chi3 chi4 chi5**

If <dih type> and <angle> are given, apply <angle> to selected dihedrals of type in range. If <custom dih> has been previously defined then apply it to residues in range.

<custom dih name>:<res range>[:<at0>:<at1>:<at2>:<at3>:<angle>[:<offset>]]

Apply <angle> to dihedral defined by atoms <at1>, <at2>, <at3>, and <at4>, or use previously defined <custom.dih>.

<offset>	Description
-2	<at0> and <at1> in previous residue.
-1	<at0> in previous residue.
0	All atoms in single residue.
1	<at3> in next residue.
2	<at2> and <at3> in next residue.

ref:<range>:<refname>[:<ref range>[:<dih types>]] [refvalsout <file>] [founddihout <file>]

Apply dihedrals from residues <ref_range> in previously loaded reference structure <refname> to dihedrals in <range>. If <ref range> is specified, use those residues from reference. The dihedral types to be used (see <dih_type> above) can be specified in a comma-separated list; default is phi/psi. Note that in order to specify <dih types>, <ref range> must be specified. The 'refvalsout' and 'founddihout' keywords can be used to print dihedrals found in the reference and target structures respectively to files.

Examples

Assign polyproline II structure to residues 1 through 13:

```
makestructure pp2:1-13
```

Make residues 1 and 12 'extended', residues 6 and 7 a type I' turn, and two custom assignments, one (custom1) for residues 2-5, the other (custom2) for residues 8-11:

```
makestructure extended:1,12 \  
      custom1:2-5:-80.0:130.0:-130.0:140.0 \  
      typeI':6-7 \  
      custom2:8-11:-140.0:170.0:-100.0:140.0
```

Assign residue 5 phi 90 degrees, residues 6 and 7 phi=-70 and psi=60 degrees:

```
makestructure customdih:5:phi:90 custom:6,7:-70:60
```

Create a new dihedral named chi1 and assign it a value of 35 degrees in residue 8:

```
makestructure chi1:8:N:CA:CB:CG:35
```

Assign 'extended' structure to residues 1 and 12, a custom turn to residues 2-5 and 8-11, and a typeI' turn to residues 6-7:

```
makestructure extended:1,12 \  
      custom1:2-5:-80.0:130.0:-130.0:140.0 \  
      typeI':6-7 \  
      custom1:8-11
```

Assign secondary structure from reference structure:

```
parm ../tz2.parm7  
reference ../tz2.rst7  
trajin pp2.rst7.save  
makestructure "ref:1-13:tz2.rst7" rmsd reference  
trajout fromref.pdb multi
```


37.11.49. mask

```
mask <mask> [maskout <filename>] [out <filename>] [nselectedout <filename>]
    [name <setname>] [ {maskpdb <filename> | maskmol2 <filename>}
    [trajargs <comma-separated args>] ]
```

<mask> Atom mask to process.

maskout <filename> Write information on atoms in <mask> to <filename>.

out <filename> Write the frame, atom number, atom name, residue number, residue name, and molecule number for each selected atom to file.

nselectedout <filename> Write the total number of selected atoms to file.

name <setname> Name for output data sets.

maskpdb <filename> Write PDB of atoms in <mask> to <name>.X.

maskmol2 <filename> Write Mol2 of atoms in <mask> to <name>.X.

trajargs <comma-separated args> When writing output PDB/Mol2, additional trajectory arguments to pass to the output trajectory.

DataSets Created

<name> Number of atoms selected each frame.

<name>[Frm] Frame number for each selected atom.

<name>[AtNum] Atom number for each selected atom.

<name>[Aname] Atom name for each selected atom.

<name>[Rnum] Residue number for each selected atom.

<name>[Rname] Residue name for each selected atom.

<name>[Mnum] Molecule number for each selected atom.

For each frame determine all atoms that correspond to <mask>. This is most useful when using distance-based masks, since the atoms in the mask are updated for every frame read in. If **maskout** is specified information on all atoms in <mask> will be written to <filename> with format:

```
#Frame AtomNum Atom ResNum Res MolNum
```

where #Frame is the frame number, AtomNum is the number of the selected atom, Atom is the name of the selected atom, ResNum is the residue number of the selected atom, Res is the residue name, and MolNum is the molecule number of the selected atom.

If **maskpdb** or **maskmol2** are specified a PDB/Mol2 file corresponding to <mask> will be written out every frame with name "<name>.frame#".

For example, to write out all residues within 3.0 Angstroms of residue 195 that are named WAT to "Res195WAT.dat", as well as write out corresponding PDB files:

```
mask `(:195<:3.0)&:WAT` maskout Res195WAT.dat maskpdb Res195WAT.pdb
```

To write all out atoms outside of 5.0 Angstroms of residues named ARG to PDB files with a chain ID of 'B':

```
mask :ARG>@5.0 maskpdb Outside5Arg.pdb trajargs ``chainid 'B'``
```

37.11.50. matrix

```

matrix [out <filename>] [start <#>] [stop|end <#>] [offset <#>]
      [name <name>] [ byatom | byres [mass] | bymask [mass] ]
      [ ired [order <#>] ]
      [ {distcovar | idea} <mask1> ]
      [ {dist | correl | covar | mwcovar} <mask1> [<mask2>] ]
      [ dihcovar dihedrals <dataset arg> ]

[out <filename>] If specified, write matrix to <filename>.
[start <#>] [stop|end <#>] [offset <#>] Start, stop, and offset frames to
use (as a subset of all frames read in).
[name <name>] Name of the matrix dataset (for referral in
subsequent analysis).
byatom Write results by atom (default). This is the sole option
for covar, mwcovar, and ired.
byres Write results by calculating an average for each residue (mass
weighted if mass is specified).
bymask Write average over <mask1>, and if <mask2> is specified
<mask1> x <mask2> and <mask2> as well (mass weighted if mass
is specified).

```

Calculate matrix of the specified type from input coordinate frames:

dist <mask1> [<mask2>] Distance matrix (default).

correl <mask1> [<mask2>] Correlation matrix (aka dynamic cross correlation[778]).

covar <mask1> [<mask2>] Coordinate covariance matrix.

mwcovar <mask1> [<mask2>] Mass-weighted coordinate covariance matrix.

distcovar <mask1> Distance covariance matrix.

idea <mask1> Isotropically Distributed Ensemble Analysis matrix.[779]

ired [order <#>] Isotropic Reorientational Eigenmode Dynamics matrix[780] with Legendre polynomials of specified order (default 1). IRED vectors must have been specified previously with '*vector ired*' (see 37.11.91 on page 840).

dihcovar dihedrals <dataset arg> Dihedral covariance matrix. Dihedral data sets must have been previously defined with e.g. *dihedral* or *multidihedral* commands or read in externally with *readdata* and marked as dihedrals.

Matrix dimensions will be of the order of $N \times M$ for **dist**, **correl**, **idea**, and **ired**, $2N \times 2N$ for **dihcovar**, $3N \times 3M$ for **covar** and **mwcovar**, and $N(N-1) \times N(N-1) / 4$ for **distcovar** (with N being the number of data sets in the case of **ired** and **dihcovar** and the number of atoms in <mask1> otherwise, and M being the number of atoms in <mask2> if specified or <mask1> otherwise). No mask is required for **ired**; the matrix will be made up of previously defined IRED vectors (see the *vector* command on page 840). Similarly no mask is required for dihcovar; dihedral data sets must have been previously defined. Only one mask can be used with **distcovar** and **idea** matrices (i.e. they can be symmetric only), otherwise one or two masks can be used (for symmetric and full matrices respectively). If two masks are specified the number of atoms covered by *mask1* must be greater than or equal to the number of atoms covered by *mask2*, and on output <mask1> corresponds to columns while <mask2> corresponds to rows.

Note that for backwards compatibility, output files written with '**out <filename>**' will have the options '**noheader noxcol square2d**' applied to them (see 37.6 on page 697 for more details). To prevent any of these from taking effect, simply specify '**header**', '**xcoll**', and/or '**nosquare2d**' after '**out <filename>**'.

As a simple example, a distance matrix of all CA atoms is generated and output to 'distmat.dat'.

```
matrix dist @CA out distmat.dat
```

37.11.51. mindist/maxdist

```
{min|max}dist mask1 <mask1> [mask2 <mask2>] [{byatom|byres|bymol}]  
[noimage] [name <setname>] [out <file>] [resoffset <#>]
```

mask1 <mask1> First mask for selecting atoms.

[mask2 <mask2>] Optional second mask for selecting atoms.

[{byatom|byres|bymol}]

byatom Report the minimum or maximum distance between atoms in <mask1> or between atoms in <mask1> and atoms in <mask2>.

byres Report the minimum or maximum distance between all residue pairs selected by <mask1>, or pairs of residues selected by <mask1> and residues selected by <mask2> (excluding certain pairs, see resoffset).

bymol Report the minimum or maximum distance between all molecule pairs selected by <mask1>, or pairs of molecules selected by <mask1> and molecules selected by <mask2>.

[noimage] Do not use the minimum image convention for distances.

[name <setname>] Output data set name.

[out <file>] Write data to <file>.

[resoffset <#>] For byres, ignore residue pairs if the difference in residue numbers is greater than the cutoff (default 1).

Data Sets Created:

<name> For byatom, a set containing the minimum or maximum distance for each frame.

<name>[<#>_<#>] For byres/bymol, a set containing the minimum or maximum distance between the residue/molecule pair specified by the numbers in the aspect, e.g. '**<name>[1.3]**' for byres would be between residues 1 and 3.

Calculate the minimum or maximum distance in Angstroms between atoms or residue/molecule pairs.

37.11.52. minimage

```
minimage [<name>] <mask1> <mask2> [out <filename>] [geom] [maskcenter]
```

<name> Data set name.

<mask1> First atom mask.

<mask2> Second atom mask.

out <filename> File to write to.

geom (maskcenter only) If specified, use geometric center instead of center of mass.

maskcenter Calculate distance from center of masks instead of between each atom.

Data Sets Created:

<name> Minimum distance to an image in Ang.

<name>[A1] Atom number in mask 1 involved in minimum distance.

<name>[A2] Atom number in mask 2 involved in minimum distance.

Calculate the shortest distance to an image, i.e. the distance to a neighboring unit cell, as well as the numbers of the atoms involved in the distance. By default the distance between each atom in <mask1> and <mask2> is considered; if **maskcenter** is specified the center of the masks is used. By convention, the lower atom number is saved as A1 and the higher is saved as A2.

37.11.53. molsurf

```
molsurf [<name>] [<mask>] [out filename] [probe <probe_rad>]
        [radii {gb | parse | vdw}] [offset <rad_offset>]
```

[<name>] Name of surface area data set.

[<mask>] Atoms to calculate surface area of.

[out <filename>] File to write values to.

[probe <probe_rad>] Probe radius (default 1.4 Angstrom).

[offset <rad_offset>] Add <rad_offset> to each atom radius (default 0.0).

[radii {gb|parse|vdw}] Specify radii to use:

gb GB radii (default).

parse PARSE radii.

vdw van der Waals radii.

Calculate the Connolly surface area^[781] of atoms in <mask> (default all atoms if no mask specified) using routines from molsurf (originally developed by Paul Beroza) using the probe radius specified by **probe** (1.4 Å if not specified). Note that if GB/VDW radii are not present in the topology file (e.g. for PDB files), then PARSE^[217] radii can be used. Also note that this routine only calculate absolute surface areas, i.e. it cannot be used to get the contribution of a subset of atoms to overall surface area; if such functionality is needed try the *surf* command (37.11.83 on page 835).

37.11.54. multidihedral

```
multidihedral [<name>] <dihedral types> [resrange <range/mask>] [out <filename>] [
    [dihtype <name>:<a0>:<a1>:<a2>:<a3>[:<offset>] ...]
    Offset -2=<at0><at1> in previous res, -1=<at0> in previous res,
    0=All <atX> in single res,
    1=<at3> in next res, 2=<at2><at3> in next res.
    <dihedral types> = alpha beta gamma delta epsilon zeta
    nu0 nu1 nu2 nu3 nu4 hlp c2p chin
    phi psi chip omega chi2 chi3 chi4 chi5
```

[<name>] Output data set name.

<dihedral types> Dihedral types to look for. Note that chip is 'protein chi', chin is 'nucleic chi'.

[resrange <range/mask>] Residue range to look for dihedrals in.

Default is all solute residues. If a mask expression is given, use residues selected by the mask expression; if any part of a residue is selected it will be used.

[out <filename>] Output file name.

[range360] Wrap torsion values from 0.0 to 360.0 (default is -180.0 to 180.0).

[dihtype <name>:<a0>:<a1>:<a2>:<a3>[:<offset>]] Search for a custom dihedral type called <name> using atom names <a0>, <a1>, <a2>, and <a3>.

Offset: -2=<a0><a1> in previous res, -1=<a0> in previous res, 0=All <aX> in single res, 1=<a3> in next res, 2=<a2><a3> in next res.

DataSets Generated:

<name>[<dihedral type>]:<#> Aspect corresponds to the dihedral type name (e.g. [phi], [psi], etc). The index is the residue number.

*Note data sets are not generated until **run** is called.*

Calculate specified dihedral angle types for residues in given range/mask. By default, dihedral angles are identified based on standard Amber atom names. The resulting data sets will have aspect equal to [<dihedral type>] and index equal to residue #. To differentiate the chi angle, chip is used for proteins and chin for nucleic acids. For example, to calculate all phi/psi dihedrals for residues 6 to 9:

```
multidihedral MyTorsions phi psi resrange 6-9 out PhiPsi_6-9.dat
```

This will generate data sets named MyTorsions[phi]:6, MyTorsions[psi]:6, MyTorsions[phi]:7, etc. Dihedrals other than those defined in <dihedral types> can be searched for using **dihtype**. For example to create a custom dihedral type called chi1 using atoms N, CA, CB, and CG (all in the same residue), then search for and calculate the dihedral in all residues:

```
multidihedral dihtype chi1:N:CA:CB:CG out custom.dat
```

37.11.55. multipucker

```
multipucker [<name>] [<pucker types>] [out <filename>] [resrange <range>]
      [altona|cremer] [puckertype <name>:<a0>:<a1>:<a2>:<a3>:<a4>[:<a5>]]
      [amplitude [ampout <ampfile>]] [theta [thetaout <thetofile>]]
      [range360] [offset <offset>]
      <pucker types> = nucleic furanose pyranose
```

[<name>] Output data set name.

<pucker types> Pucker types to look for.

[out <filename>] Output file name to write pucker data to.

[resrange <range>] Residue range to look for puckers in. Default is all solute residues.

[puckertype <name>:<a0>:<a1>:<a2>:<a3>:<a4>[:<a5>]] Search for a custom pucker type called <name> using atom names <a0>, <a1>, <a2>, <a3>, and <a4> (also <a5> for 6 atom puckers).

[altona] Use method of Altona & Sundaralingam (5 atoms only). This is the default when pucker has 5 atoms.

[cremer] Use method of Cremer and Pople (5 or 6 atoms). This is the default when pucker has 6 atoms.

[amplitude] Also calculate amplitude (in degrees).

ampout <ampfile> File to write amplitude sets to.

[theta] (Valid for 6 atoms only) Also calculate theta (in degrees).
thetaout <thetofile> File to write theta sets to.

[range360] Wrap pucker values from 0.0 to 360.0 (default is -180.0 to 180.0).

[offset <offset>] Add <offset> to pucker values.

DataSets Generated:

<name>[<pucker type>]:<#> Aspect corresponds to the pucker type name (e.g. [nucleic], [furanose], etc). The index is the residue number.

<name>[<pucker type>Amp]:<#> amplitude only. Data set for pucker amplitude.

<name>[<pucker type>Theta]:<#> theta only. Data set for pucker theta.

*Note data sets are not generated until **run** is called.*

Calculate specified pucker types for residues in given range. By default, puckers are identified based on standard Amber atom names. The resulting data sets will have aspect equal to [<pucker type>] and index equal to residue #. In order to be identified as a pucker, all consecutive atoms in the pucker must be bonded, and the last atom of the pucker must be bonded to the first.

For example, to calculate all nucleic acid ribose puckers for residues 6 to 9:

```
multipucker MyPuckers nucleic resrange 6-9 out Pucker_6-9.dat
```

This will generate data sets named MyPuckers[nucleic]:6, MyPuckers[nucleic]:7, etc. Puckers other than those defined in <pucker types> can be searched for using **puckertype**. For example to create a custom pucker type called furanoid using atoms C2, C3, C4, C5, and O2, then search for and calculate that pucker (with amplitudes) using the method of Cremer and Pople in all residues:

```
multipucker Furanoid puckertype furanoid:C2:C3:C4:C5:O2 cremer \  
out furanoid.dat amplitude ampout furanoid.dat
```

37.11.56. multivector

```
multivector [<name>] [resrange <range>] name1 <name1> name2 <name2> [out <filename>  
ired]
```

[<name>] Data set name.

[resrange <range>] Range of residues to look for vectors in.

name1 <name1> Name of first atom in each residue.

name2 <name2> Name of second atom in each residue.

[out <filename>] File to write results to.

Search for and calculate atomic vectors between atoms named <name1> and <name2> in residues specified by the given <range>; each one is equivalent to the command '**vector** <name1> <name2>'. For example, to calculate all vectors between atoms named 'N' and atoms named 'H' in residues 5-20, storing the results in data sets named NH and writing to NH.dat:

```
multivector NH name1 N name2 H ired out NH.dat resrange 5-20
```

37.11.57. **nastruct**

```

nastruct [<dataset name>] [resrange <range>] [sscalc] [naout <suffix>]
  [noheader] [resmap <ResName>:{A,C,G,T,U} ...] [calcnohb]
  [noframespaces] [baseref <file>] ...
  [bpmode {3dna|babcock}] [allhb]
  [hbcut <hbcut>] [origincut <origincut>] [altona | cremer]
  [zcut <zcut>] [zanglecut <zanglecut>] [groovecalc {simple | 3dna}]
  [axesout <file>] [axesoutarg <arg> ...] [axesparmout <file>]]
  [bpaxesout <file>] [bpaxesoutarg <arg> ...] [bpaxesparmout <file>]]
  [stepaxesout <file>] [stepaxesoutarg <arg> ...] [stepaxesparmout <file>]]
  [axisnameo <name>] [axisnamex <name>] [axisnamey <name>] [axisnamez <name>]
  [{ first | reference | ref <name> | refindex <#> |
    allframes |
    specifiedbp pairs <b1>-<b2>,... }]

```

[<dataset name>] Output data set name.

[resrange <range>] Residue range to search for nucleic acids in (default all).

[sscalc] Calculate parameters between consecutive bases in strands.

[naout <suffix>] File name suffix for output files; BP.<suffix> for base pair parameters, BPstep.<suffix> for base pair step parameters, and Helix.<suffix> for base pair step helical parameters. If sscalc is specified, also SS.<suffix> for parameters of consecutive bases in strands.

[noheader] Do not print header to naout file.

[resmap <ResName>:{A,C,G,T,U}] Attempt to treat residues named <ResName> as if it were A, C, G, T, or U; useful for residues with modifications or non-standard residue names. This will only work if enough reference atoms are present in <ResName>.

[calcnohb] Calculate parameters between bases in base pairs even if no hydrogen bonds present between them.

[noframespaces] If specified there will be no spaces between frames in the naout files.

[baseref <file>] Specify a custom nucleic acid base reference. One file per custom residue; multiple 'baseref' keywords may be present. See below for details.

[bpmode {3dna|babcock}] Specify axis conventions for calculating base pair parameters. If '3dna' (default), use conventions of 3DNA[782]; flip Y and Z of complimentary base for antiparallel. If 'babcock', use conventions of Babcock et al.[783] ; flip Y and Z of complimentary base for antiparallel, flip X and Y for parallel.

[allhb] Report the total number of hydrogen bonds detected instead of just the number of Watson-Crick-Franklin hydrogen bonds.

[hbcut <hbcut>] Distance cutoff (in Angstroms) for determining hydrogen bonds between bases (default 3.5).

[origincut <origincut>] Distance cutoff (in Angstroms) between base pair axis origins for determining which bases are eligible for base-pairing (default 2.5).

[**altona**] Use method of Altona & Sundaralingam to calculate sugar pucker (default, see *pucker* command).

[**cremer**] Use method of Cremer and Pople to calculate sugar pucker (see *pucker* command).

[**zcut**] Distance cutoff (in Angstroms) between base reference axes along the Z axis (i.e. stagger) for determining base pairing (default 2).

[**zanglecut**] Angle cutoff (in degrees) between base reference Z axes for determining base pairing (default 65).

[**groovecalc**] Groove width calculation method:
 simple Use P-P distance for major groove, O4-O4 distance for minor groove. Output to 'BP.<suffix>'.
 3dna Use groove width calculation of El Hassan and Calladine[784]. Output to 'BPstep.<suffix>'.
 [**axesout <file>**] Trajectory file to write base axes to.
 [**axesoutarg <arg>**] Trajectory argument to pass to base axes trajectory file (can specify more than once).
 [**axesparmout <file>**] Topology file to write base axes pseudo topology to.

[**bpaxesout <file>**] Trajectory file to write base pair axes to.
 [**bpaxesoutarg <arg>**] Trajectory argument to pass to base pair axes trajectory file (can specify more than once).
 [**bpaxesparmout <file>**] Topology file to write base pair axes pseudo topology to.

[**stepaxesout <file>**] Trajectory file to write base pair step axes to.
 [**stepaxesoutarg <arg>**] Trajectory argument to pass to base pair step axes trajectory file (can specify more than once).
 [**stepaxesparmout <file>**] Topology file to write base pair step axes pseudo topology to.

[**axisnameo <name>**] Change name of axis origin pseudo atom (default 'Orig').

[**axisnamex <name>**] Change name of axis origin pseudo atom (default 'X').

[**axisnamey <name>**] Change name of axis origin pseudo atom (default 'Y').

[**axisnamez <name>**] Change name of axis origin pseudo atom (default 'Z').

How to determine base pairing:

[**first**] Use first frame to determine base pairing (default).

[**reference | refindex <#> | ref <name>**] Reference structure to use to determine base pairing.

[**allframes**] If specified determine base pairing each frame.

[**specifiedbp pairs <b1>-<b2>,...**] User specified base pairing. Base pairs are specified in a comma-separated list after the 'pairs' keyword as <b1>-<b2>, where <b1> and <b2> are the residue numbers of bases in the base pair, e.g. 'pairs 1-16,2-15,3-14,4-13'. Can specify 'pairs' multiple times.

DataSets Created:

<name>[pucker]:X Base X (residue number) sugar pucker.

Base pairs:

<name>[shear]:X Base pair X (starting from 1) shear.

<name>[stretch]:X Base pair stretch.

<name>[stagger]:X Base pair stagger.

<name>[buckle]:X Base pair buckle.

<name>[prop]:X Base pair propeller.

<name>[open]:X Base pair opening.

<name>[hb]:X Number of WC hydrogen bonds between bases in base pair.

<name>[bp]:X Contain 1 if bases are base paired, 0 otherwise.

<name>[major]:X (If groovecalc simple) Major groove width calculated between P atoms of each base.

<name>[minor]:X (If groovecalc simple) Minor groove width calculated between O4 atoms of each base.

Base pair steps:

<name>[shift]:X Base pair step X (starting from 1) shift.

<name>[slide]:X Base pair step slide.

<name>[rise]:X Base pair step rise.

<name>[tilt]:X Base pair step tilt.

<name>[roll]:X Base pair step roll.

<name>[twist]:X Base pair step twist.

<name>[zp]:X Base pair step Zp value.

<name>[major]:X (If groovecalc 3dna) Major groove width, El Hassan and Calladine.

<name>[minor]:X (If groovecalc 3dna) Minor groove width, El Hassan and Calladine.

Helical steps:

<name>[xdisp]:X Helical step X (starting from 1) X displacement.

<name>[ydisp]:X Helical Y displacement.

<name>[hrise]:X Helical rise.

<name>[incl]:X Helical inclination.

<name>[tip]:X Helical tip.

<name>[htwist]:X Helical twist.

Strands (sscalc only):

<name>[dx]:X Strand pair X (starting from 1) X displacement.

<name>[dy]:X Y displacement.

<name>[dz]:X Z displacement.

<name>[rx]:X Relative rotation around X axis.

<name>[ry]:X Relative rotation around Y axis.

<name>[rz]:X Relative rotation around Z axis.

Note that base pair data sets are not created until base pairing is determined.

Calculate basic nucleic acid (NA) structure parameters for all residues in the range specified by **resrange** (or all NA residues if no range specified). Residue names are recognized with the following priority: standard Amber residue names DA, DG, DC, DT, RA, RG, RC, and RU; 3 letter residue names ADE, GUA, CYT, THY, and URA; and finally 1 letter residue names A, G, C, T, and U. Non-standard/modified NA bases can be recognized by using the **resmap** keyword. For example, to make *cpptraj* recognize all 8-oxoguanine residues named '8OG' as a guanine-based residue:

```
nastruct naout nastruct.dat resrange 274-305 resmap 8OG:G
```

The **resmap** keyword can be specified multiple times, but only one mapping per unique residue name is allowed. Note that **resmap** may fail if the residue is missing heavy atoms normally present in the specified base type.

Base pairs are determined either once from the first frame or from a reference structure, or can be determined each frame if **allframes** is specified. Base pairing can also be specified via the **specifiedbp** and **pairs** keywords. Base pairing is determined first by base reference axis origin distance, then by stagger, then by angle between base Z axes, then finally by hydrogen bonding (at least one hydrogen bond must be present). Base pair parameters will only be written for determined base pairs. Both Watson-Crick and other types of base pairing can be detected. Note that although all possible hydrogen bonds are searched for, only WC hydrogen bonds are reported in the BP.<suffix> file.

The procedure used to calculate NA structural parameters is the same as 3DNA[782], with algorithms adapted from Babcock et al.[783] and reference frame coordinates from Olson et al.[785]. Given the same base pairs are determined, *cpptraj* **nastruct** should give the exact same numbers as 3DNA. One notable exception are parameters for G-quadruplex structures.

Calculated NA structure parameters are written to three separate files, the suffix of which is specified by **naout**. Base pair parameters (shear, stretch, stagger, buckle, propeller twist, opening, # WC hydrogen bonds, base pairing, and simple groove widths) are written to BP.<suffix>, along with the number of WC hydrogen bonds detected. Base pair step parameters (shift, slide, rise, tilt, roll, twist, Zp, and El Hassan and Calladine groove widths) are written to BPstep.<suffix>, and helical parameters (X-displacement, Y-displacement, rise, inclination, tip, and twist) are written to Helix.<suffix>. If **noheader** is specified a header will not be written to the output files. Note that although base puckering is calculated, it is not written to an output file by default. You can output pucker to a file via the create or write/writedata commands after the data has been generated, e.g.:

```
nastruct NA naout nastruct.dat resrange 1-3,28-30
run
writedata NApucker.dat NA[pucker]
```

Note that while the underlying procedure is geared towards calculating parameters for base pairs, the code can be made to calculate parameters between consecutive bases in single strands by specifying **sscalc**.

Base axes, base pair axes, and base pair step axes can be written to trajectory files using the **axesout**, **bpaxesout**, and **stepaxesout** and related keywords. The axes are written using 4 points: an origin, and X Y and Z which are bonded to the origin. The names of these pseudo atoms can be changed using the **axisnameo**, **axisnamex**, **axisnamey**, and **axisnamez** keywords.

Custom Nucleic Acid Base References

Users can now specify **baseref** <file> to load a custom nucleic acid base reference. The base reference files are white-space delimited, begin with the line NASTRUCT REFERENCE, and have the following format:

```
NASTRUCT REFERENCE
<base character> <res name 0> [<res name 1> ...]
<atom name> <X> <Y> <Z> <HB type> <RMS fit>
...
```

There is a line for each reference atom. Lines beginning with '#' are ignored as comments.

<base character> Used to identify the underlying base type: A G C T or U. If none of these, it will be considered an unknown residue (which just means WC hydrogen bonding will not be identified).

<res name X> Specifies what residue names this reference corresponds to. There must be at least one residue name. There can be any number of these specified.

<atom name> A reference atom name.

<X> <Y> <Z> The X Y and Z coordinates of the reference atom.

<HB type> Denotes if and how the atom participates in hydrogen bonding. Can be 'd'onor, 'a'cceptor, or 'n'one (or the numbers 1, 2, 0 respectively). Only the first character of the word actually matters.

<RMS fit> Denotes whether the atom is involved in RMS-fitting.

Here is an example for GUA:

```

NASTRUCT REFERENCE
G G G5 G3
# Modified into format readable by cpptraj nastruct
C1' -2.477  5.399  0.000 0      0
N9  -1.289  4.551  0.000 0      1
C8   0.023  4.962  0.000 0      1
N7   0.870  3.969  0.000 accept 1
C5   0.071  2.833  0.000 0      1
C6   0.424  1.460  0.000 0      1
O6   1.554  0.955  0.000 accept 0
N1  -0.700  0.641  0.000 donor  1
C2  -1.999  1.087  0.000 0      1
N2  -2.949  0.139 -0.001 donor  0
N3  -2.342  2.364  0.001 accept 1
C4  -1.265  3.177  0.000 0      1

```

37.11.58. nativecontacts

```

nativecontacts [<mask1> [<mask2>]] [writecontacts <outfile>] [resout <resfile>]
[noimage] [distance <cut>] [out <filename>] [includesolvent]
[ first | reference | ref <name> | refindex <#> ]
[resoffset <n>] [contactpdb <file>] [pdbcut <cut>] [mindist] [maxdist]
[name <dsname>] [byresidue] [map [mapout <mapfile>]]
[series [seriesout <file>]]
[savenonnative [seriesnnout <file>] [nncontactpdb <file>]]
[resseries { present | sum } [resseriesout <file>]] [skipnative]

```

<mask1> First mask to calculate contacts for.

[<mask2>] (Optional) Second mask to calculate contacts for.

[writecontacts <outfile>] Write information on native contacts to
 <outfile> (STDOUT if not specified).

[resout <resfile>] File to write contact residue pairs to.

[noimage] Do not image distances.

[distance <cut>] Distance cutoff for determining native contacts in
 Angstroms (default 7.0 Ang).

[out <filename>] File to write number of native contacts and
 non-native contacts.

[includesolvent] By default solvent molecules are ignored; this will
 explicitly include solvent molecules.

[first | reference | ref <name> | reindex <#>] Reference structure to use for determining native contacts.

[resoffset <n>] (byresidue only) Ignore contacts between residues spaced less than <n> residues apart in sequence.

[contactpdb <file>] Write PDB with B-factor column containing relative contact strength for native contacts (strongest is 100.0).

[pdbcut <cut>] If writing contactpdb, only write contacts with relative contact strength greater than <cut>.

[mindist] If specified, determine the minimum distance between any atoms in the mask(s).

[maxdist] If specified, determine the maximum distance between any atoms in the mask(s).

[name <dsname>] Data set name.

[byresidue] Write out the contact map by residue instead of by atom.

[map] Calculate matrices of native contacts ([nativemap]) and non-native contacts ([nonnatmap]). These matrices are normalized by the total number of frames, so that a value of 1.0 means ``contact always present''. If byresidue specified, the values for each individual atom pair are summed over the residues they belong to (this means for byresidue values greater than 1.0 are possible).

[mapout <mapfile>] Write native/non-native matrices to 'native.<mapfile>' and 'nonnative.<mapfile>' respectively.

[series] Calculate native contact time series data, 1 for contact present and 0 otherwise.

[seriesout <file>] Write native contact time series data to file.

[savenonnative] Save non-native contacts; series must also be specified. This is enabled by default if skipnative specified.

[seriesnnout <file>] Write non-native contact time series data to file.

[nncontactpdb <file>] Write PDB with B-factor column containing relative contact strength for non-native contacts (strongest is 100.0).

[resseries {present | sum}] Create contacts time series by residue; series must also be specified.

 present Record a 1 if any contact is present and 0 if no contact is present for the residue pair.

 sum The sum of all individual contacts is recorded for the residue pair.

[resseriesout <file>] Write residue time series data to <file>.

[skipnative] If specified, skip native contacts determination, i.e. treat all sonctacts as non-native contacts. Implies savenonnative.

Data Sets Created:

<dsname>[native] Number of native contacts.

<dsname>[nonnative] Number of non-native contacts.

```

<dsname>[mindist] (mindist only) Minimum observed distance each
frame.
<dsname>[maxdist] (maxdist only) Maximum observed distance each
frame.
<dsname>[nativemap] (map only) Native contacts matrix (2D).
<dsname>[nonnatmap] Non-native contacts matrix (2D).
<dsname>[NC] Native contacts time series.
<dsname>[NN] Non-native contacts time series.
<dsname>[NCRES] Residue native contacts time series.
<dsname>[NNRES] Residue non-native contacts time series.

```

Define and track “native” contacts as determined by a simple distance cut-off, i.e. any atoms which are closer than **<cut>** in the specified reference frame (the first frame if no reference specified) are considered a native contact. If one mask is provided, contacts are looked for within **<mask1>**; if two masks are provided, only contacts between atoms in **<mask1>** and atoms in **<mask2>** are looked for (useful for determining intermolecular contacts). By default only native contacts are tracked. This can be changed by specifying the **savenonnative** keyword or by specifying **skipnative**. The time series for contacts can be saved using the **series** keyword; these can be further consolidated by residue using the **resseries** keyword. When using **<resseries>** the data set index is calculated as $(r2 * nres) + r1$ so that indices can be matched between native/non-native contact pairs. Non-native residue contact legends have an **nn_** prefix.

Native contacts that are found are written to the file specified by **writecontacts** (or **STDOUT**) with format:

```
# Contact Nframes Frac. Avg Stdev
```

Where **Contact** takes the form **'<residue1 num>@<atom name>.:<residue2 num>@<atom name>**, **Nframes** is the number of frames the contact is present, **Frac.** is the total fraction of frames the contact is present, **Avg** is the average distance of the contact when present, and **Stdev** is the standard deviation of the contact distance when present. If **resout** is specified the total fraction of contacts is printed for all residue pairs having native contacts with format:

```
#Res1 #Res2 TotalFrac Contacts
```

Where **#Res1** is the first residue number, **#Res2** is the second residue number, **TotalFrac** is the total fraction of contacts for the residue pair, and **Contacts** is the total number of native contacts involved with the residue pair. Since **TotalFrac** is calculated for each pair as the sum of each contact involving that pair divided by the total number of frames, it is possible to have **TotalFrac** values greater than 1 if the residue pair includes more than 1 native contact.

During trajectory processing, non-native contacts (i.e. any pair satisfying the distance cut-off which is not already a native contact) are also searched for. The time series for native contacts can be saved as well, with 1 for contact present and 0 otherwise (similar to the **hbond** command). This data can be subsequently analyzed using e.g. [37.12.21 on page 872](#).

Contact maps (matrices) are generated for native and non-native contacts. If **byresidue** is specified, contact maps are summed over residues, and contacts between residues spaced **<resoffset>** residues apart in sequence are ignored.

If **contactpdb** is specified a PDB is generated containing relative contact strengths in the B-factor column. The relative contact strength is normalized so that a value of 100 means that atom participated in the most contacts with other atoms.

Example command looking for contacts between residues 210 to 260 and residue named NDP, using reference structure 'FtuFabI.WT.pdb' to define native contacts:

```

parm FtuFabI.parm7
trajin FtuFabI.nc
reference FtuFabI.WT.pdb

```

```
nativecontacts name NC1 :210-260&!@H= :NDP&!@H= \
  byresidue out nc.all.res.dat mindist maxdist \
  distance 3.0 reference map mapout resmap.gnu \
  contactpdb Loop-NDP.pdb \
  series seriesout native.dat
```

37.11.59. outtraj

```
outtraj <filename> [ trajout args ]
      [maxmin <dataset> min <min> max <max>] ...
```

<filename> Output trajectory file name.

[trajout args] Output trajectory arguments (see [37.10.5 on page 747](#)).

[maxmin <dataset> min <min> max <max>] Only write frames to **<filename>** if values in **<dataset>** for those frames are between **<min>** and **<max>**. Can be specified for one or more data sets.

The outtraj command is similar in function to *trajout*, and takes all of the same arguments. However, instead of writing a trajectory frame after all actions are complete outtraj writes the trajectory frame at its position in the Action queue. For example, given the input:

```
trajin mdcrd.crd
trajout output.crd
outtraj BeforeRmsd.crd
rms R1 first :1-20@CA out rmsd.dat
outtraj AfterRmsd.crd
```

three trajectories will be written: output.crd, BeforeRmsd.crd, and AfterRmsd.crd. The output.crd and AfterRmsd.crd trajectories will be identical, but the BeforeRmsd.crd trajectory will contain the coordinates of mdcrd.crd before they are RMS-fit.

The maxmin keyword can be used to restrict output using one more more data sets. For example, to only write frames for which the RMSD value is between 0.7 and 0.8:

```
trajin tz2.truncocct.nc
rms R1 first :2-11
outtraj maxmin.crd maxmin R1 min 0.7 max 0.8
```

37.11.60. pairdist

```
pairdist out <filename> mask <mask> [delta <resolution>]
```

Calculate pair distribution function. In the following, defaults are given in parentheses. The **out** keyword specifies output file for histogram: distance, $P(r)$, $s(P(r))$. The **mask** option specifies atoms for which distances should be computed. The **delta** option specifies resolution. (0.1 Å)

37.11.61. pairwise

```
pairwise [<name>] [<mask>] [out <filename>] [cuteelec <ecut>] [cutevdw <vcut>]
      [ reference | ref <name> | refindex <#> ] [cutout <cut mol2 prefix>]
      [vmapout <vdw map>] [emapout <elec map>] [avgout <avg file>]
      [eout <eout file>] [pdbout <pdb file>] [scalepdb] [printmode {only|or|and|}]
```

[<name>] Data set name; van der Waals energy will get aspect [EVDW] and electrostatic energy will get aspect [EELEC].

[<mask>] Atoms to calculate energy for.

[out <filename>] File to write total EELEC and EVDW to.

[eout <eout file>] File to write individual EELEC and EVDW interactions to.

[reference | ref <name> | refindex <#>] Specify a reference to compare frames to (i.e. calculate $E_{\text{ref}} - E_{\text{frame}}$).

[cuteelec <cut>] Only report interaction EELEC (or delta EELEC) if absolute value is greater than <ecut> (default 1.0 kcal/mol).

[cutevdw <cutv>] Only report interaction EVDW (or delta EVDW) if absolute value is greater than <vcut> (default 1.0 kcal/mol).

[cutout <cut mol2 prefix>] Write out mol2 containing only atom pairs which satisfy <ecut> and <vcut>.

[vmapout <vdw map>] Write out interaction EVDW (or delta EVDW) matrix to file <vdw map>.

[emapout <elec map>] Write out interaction EELEC (or delta EELEC) matrix to file <elec map>.

[avgout <avg file>] Print average interaction EVDW|EELEC (or average delta EVDW|EELC) to <avg file>.

[pdbout <pdb file>] Write PDB with EVDW|EELEC in occupancy|B-factor columns to <pdb file>.

[scalepdb] Scale energies written to PDB from 0 to 100.

[printmode {only|or|and}] Control when/how average energies are written

Data Sets Created:

<name>[EELEC] Electrostatic energy in (kcal/mol).

<name>[EVDW] van der Waals energy in (kcal/mol).

<name>[VMAP] van der Waals energy matrix.

<name>[EMAP] Electrostatic energy matrix.

This action has two related functions: 1) Calculate pairwise (i.e. non-bonded) energy (in kcal/mol) for atoms in <mask>, or 2) Compare pairwise energy of frames to a reference frame. This calculation does use an exclusion list but is not periodic.

When comparing to a reference frame, the **eout** file will contain the differences for each individual interaction (i.e. $E_{\text{ref}} - E_{\text{frame}}$), otherwise the **eout** file will contain the absolute value of each individual interaction. The **cutelec** and **cutevdw** keywords can be used to restrict printing of individual interactions to those for which the absolute value is above a cutoff. The VMAP and EMAP matrix elements will contain these values as well (differences for reference, absolute value otherwise) averaged over all frames. The **avgout** file will contain only these values averaged over all frames that satisfy the cutoffs. The **printmode** keyword controls when the average energies are written: **only** means only average energy components that satisfy cutoffs will be printed, **or** means that both energy components will be printed if either satisfy a cutoff, and **and** means that both energy components will be written only if both satisfy the cutoffs.

The **cutout** keyword can be used to write out MOL2 files each frame named '**<cut mol2 prefix>.evdw.mol2.X'** and '**<cut mol2 prefix>.eelec.mol2.X'** (where X is the frame number) containing only atoms with energies that satisfy the cutoffs. Similarly, the **pdbout** keyword can be used to write out a PDB file (with 1 MODEL per frame). The occupancy and B-factor columns will contain the total van der Waals and electrostatic energy for each atom if cutoffs are satisfied, or 0.0 otherwise.

37.11.62. principal

```
principal [<mask>] [dorotation] [out <filename>] [name <dsname>]
```

[<mask>] Mask of atoms used to determine principal axes (default all).

[dorotation] Align coordinates along principal axes.

[out <filename>] Write resulting eigenvalues/eigenvectors to <filename>.

[name <dsname>] Data set name (3x3 matrices).

Data Sets Created (name keyword only):

<dsname>[evect] Eigenvectors (3x3 matrix, row-major).

<dsname>[eval] Eigenvalues (vector).

Determine principal axes of each frame determined by diagonalization of the inertial matrix from the coordinates of the specified atoms. At least one of **dorotation**, **out**, or **name** must be specified. The resulting eigenvectors are sorted from largest eigenvalue to smallest, and the corresponding axes labelled using the *cpptraj* convention of X > Y > Z (similar to '**vector principal**'). If out is specified the eigenvectors and eigenvalues will be written for each frame N with format:

```
<N> EIGENVALUES: <EX> <EY> <EZ>
<N> EIGENVECTOR 0: <Xx> <Xy> <Xz>
<N> EIGENVECTOR 1: <Yx> <Yy> <Yz>
<N> EIGENVECTOR 2: <Zx> <Zy> <Zz>
```

NOTE: The eigenvector 3x3 matrix data set could subsequently be used e.g. with the **rotate** action.

Example: Align system (residues 1-76) along principle axes:

```
parm myparm.parm7
trajin protein.nc
principal :1-76 dorotation out principal.dat
```

37.11.63. projection

```
projection [<name>] evects <dataset name> [out <outfile>] [beg <beg>] [end <end>]
{[<mask>] | [dihedrals <dataset arg>] | [data <dataset arg> ...]}
[start <start>] [stop <stop>] [offset <offset>]
```

[<name>] Output data set name.

evects <dataset name> Data set containing eigenvectors (modes).

[out <outfile>] Write projections to <outfile>.

[beg <beg>] First eigenvector/mode to use (default 1).

[end <end>] Final eigenvector/mode to use (default 2).

[<mask>] (Not dihedral covariance) Mask of atoms to use in projection; MUST CORRESPOND TO HOW EIGENVECTORS WERE GENERATED.

[dihedrals <dataset arg>] (Dihedral covariance only) Dihedral data sets to use in projection; MUST CORRESPOND TO HOW EIGENVECTORS WERE GENERATED.

[data <dataset arg>] (Data covariance only, e.g. from TICA). 1D data sets to use in projection; MUST CORRESPOND TO HOW EIGENVECTORS WERE GENERATED.

[start <start>] Frame to start calculating projection.

[stop <stop>] Frame to stop calculating projection.

[offset <offset>] Frames to skip between projection calculations.

Data Sets Created:

DataSet indices correspond to mode #.

<name> (All except IDEA) Projection data set.

<name>[X] X component of mode (IDEA modes only).

<name>[Y] Y component of mode (IDEA modes only).

<name>[Z] Z component of mode (IDEA modes only).

<name>[R] Magnitude of mode (IDEA modes only).

Projects snapshots onto eigenvectors obtained by diagonalizing covariance or mass-weighted covariance matrices. Eigenvectors are taken from previously generated (e.g. with *diagmatrix/tica*) or previously read-in (e.g. with *readdata*) eigenvectors with name <dataset name>. The user has to make sure that the atoms selected by <mask> agree with the ones used to calculate the modes (i.e., if mask = '@CA' was used in the "*matrix*" command, mask = '@CA' needs to be set here as well). See [37.13 on page 893](#) for examples using the *projection* command. If only 1D data sets need to be projected, see the *projectdata* analysis command, [37.12.28 on page 879](#).

37.11.64. pucker

```
pucker [<name>] <mask1> <mask2> <mask3> <mask4> <mask5> [<mask6>] [geom]
      [out <filename>] [altona | cremer] [amplitude] [theta]
      [range360] [offset <offset>]
```

<name> Output data set name.

<maskX> Five (optionally six) atom masks selecting atom(s) to calculate pucker for.

[geom] Use geometric center of atoms in <maskX> (default is center of mass).

[out <filename>] Output file name.

[altona] Use method of Altona & Sundaralingam (5 masks only).

[cremer] Use method of Cremer and Pople (5 or 6 masks). This is the default when 6 masks are specified.

[amplitude] Also calculate amplitude.

[theta] (6 masks only) Also calculate theta.

[range360] Wrap pucker values from 0.0 to 360.0 (default is -180.0 to 180.0).

[offset <offset>] Add <offset> to pucker values.

Data Sets Created:

<name> Pucker in degrees.

<name>[Amp] Amplitude (if amplitude was specified).

<name>[Theta] Theta (if theta and 6 masks were specified).

Calculate the pucker (in degrees) for atoms in <mask1>, <mask2>, <mask3>, <mask4>, <mask5> using the method of Altona & Sundarlingam[786, 787] (default for 5 masks, or if **altona** specified), or the method of Cremer & Pople[788] (default for 6 masks, or if **cremer** is specified). If the **amplitude** or **theta** keywords are given, amplitudes/thetas (also in degrees) will be calculated in addition to pucker. The results from **pucker** can be further analyzed with the **statistics** analysis.

By default, pucker values are wrapped to range from -180 to 180 degrees. If the **range360** keyword is specified values will be wrapped to range from 0 to 360 degrees. Note that the Cremer & Pople convention is offset from Altona & Sundarlingam convention (with nucleic acids) by +90.0 degrees; the **offset** keyword will add an offset to the final value and so can be used to convert between the two. For example, to convert from Cremer to Altona specify “**offset 90**”.

To calculate nucleic acid pucker specify C1' first, followed by C2', C3', C4' and O4'. For example, to calculate the sugar pucker for nucleic acid residues 1 and 2 using the method of Altona & Sundarlingam, with final pseudorotation values ranging from 0 to 360:

```
pucker p1 :1@C1' :1@C2' :1@C3' :1@C4' :1@O4' range360 out pucker.dat
pucker p2 :2@C1' :2@C2' :2@C3' :2@C4' :2@O4' range360 out pucker.dat
```

37.11.65. radgyr | rog

```
radgyr [name>] [<mask>] [out <filename>] [mass] [nomax] [tensor]
[<name>] Data set name.
[<mask>] Atoms to calculate radius of gyration for; default all
         atoms.
[out <filename>] Write data to <filename>.
[mass] Mass-weight radius of gyration.
[nomax] Do not calculate maximum radius of gyration.
[tensor] Calculate radius of gyration tensor, output format 'XX YY ZZ
         XY XZ YZ'.
```

Data Sets Created:

```
<name> Radius of gyration in Ang.
<name>[Max] Max radius of gyration in Ang.
<name>[Tensor] Radius of gyration tensor; format 'XX YY ZZ XY XZ
         YZ'.
```

Calculate the radius of gyration of specified atoms. For example, to calculate only the mass-weighted radius of gyration (not the maximum) of the non-hydrogen atoms of residues 4 to 10 and print the results to “RoG.dat”:

```
radgyr :4-10&!(@H=) out RoG.dat mass nomax
```

37.11.66. radial | rdf

```
radial [out <outfilename>] <spacing> <maximum> <solvent mask1> [<solute mask2>]
         [noimage]
         [density <density> | volume] [<dataset name>] [intrdf <file>] [rawrdf <file>]
         [{center1|center2|nointramol|toxyz <x>,<y>,<z>} |
         [byres1] [byres2] [bymol1] [bymol2]]]
[out <outfilename>] File to write RDF to.
<spacing> Bin spacing, required.
```

<maximum> Max bin value, required.

<solvent mask1> Atoms to calculate RDF for, required.

[<solute mask2>] (Optional) If specified calculate RDF of all atoms in **<solvent mask1>** to each atom in **<solute mask2>**.

[noimage] Do not image distances.

[density <density>] Use density value of **<density>** for normalization (default 0.033456 molecules Å⁻³).

[volume] Determine density for normalization from average volume of input frames.

[<dataset name>] Name of output data sets.

[intrdf <file>] Calculate integral of RDF bin values (averaged over # of frames but otherwise not normalized) and write to **<file>** (can be same as **<output filename>**).

[rawrdf <file>] Write raw (non-normalized) RDF values to **<file>**.

[center1] Calculate RDF from geometric center of atoms in **<solvent mask1>** to all atoms in **<solute mask2>**.

[center2] Calculate RDF from geometric center of atoms in **<solute mask2>** to all atoms in **<solvent mask1>**.

[nointramol] Ignore intra-molecular distances.

[toxyz <x>,<y>,<z>] Calculate RDF from center of atoms in **<solvent mask1>** to point specified by **<x>** **<y>** and **<z>** (in Ang.).

[byres1] Calculate using the centers of mass of each residue in the first mask.

[bymol1] Calculate using the centers of mass of each molecule in the first mask.

[byres2] Calculate using the centers of mass of each residue in the second mask.

[bymol2] Calculate using the centers of mass of each molecule in the second mask.

DataSet Aspects:

<setname> The radial distribution function.

<setname>[int] (intrdf only) Integral of RDF bin values.

<setname>[raw] (rawrdf only) Raw (non-normalized) RDF values.

Calculate the radial distribution function (RDF, aka pair correlation function) of atoms in **<solvent mask1>** (note that this mask does not need to be solvent, but this nomenclature is used for clarity). If an optional second mask (**<solute mask2>**) is given, calculate the RDF of ALL atoms in **<solvent mask1>** to EACH atom in **<solute mask2>**. If desired, the geometric center of atoms in **<solvent mask1>** or **<solute mask2>** can be used by specifying the **center1** or **center2** keywords respectively, or alternatively intra-molecular distances can be ignored by specifying the **nointramol** keyword.

The RDF is calculated from the histogram of the number of particles found as a function of distance R, normalized by the expected number of particles at that distance. The normalization is calculated from:

$$Density * \frac{4\pi}{3} \left((R + dR)^3 - R^3 \right)$$

where dR is equal to the bin spacing. Some care is required by the user in order to normalize the RDF correctly. The default density value is $0.033456 \text{ molecules } \text{\AA}^{-3}$, which corresponds to a density of water approximately equal to 1.0 g mL^{-1} . To convert a standard density in g mL^{-1} , multiply the density by $\frac{0.6022}{M_r}$, where M_r is the mass of the molecule in atomic mass units. Alternatively, if the **volume** keyword is specified the density is determined from the average volume of the system over all Frames.

Note that correct normalization of the RDF depends on the number of atoms in each mask; if multiple topology files are being processed that result in changes in the number of atoms in each mask, the normalization will be off.

The basic (i.e. no center1/center2/byres1/byres2/bymol1/bymol2) RDF calculations are now CUDA parallelized. However, the calculation is done in single-precision on GPUs so the resulting histograms may differ slightly from the CPU (on the order of 0.0002 - 0.0004).

37.11.67. randomizeions

```
randomizeions <mask> [around <aroundmask> by <distance>] [{allowoverlap|overlap <value>}]
[noimage] [seed <value>] [originalalgorithm]
```

<mask> Mask of ions to randomize.

around <mask> by <distance> Ensure ions come no closer than
<distance> Ang. to atoms in **<aroundmask>**.

allowoverlap No restrictions on how close ions can be to each other.

overlap <value> Ions in **<mask>** can be no closer than **<value>** Ang.
to each other.

[noimage] Do not image distances.

[seed <value>] Seed for the random number generator.

[originalalgorithm] Use the original, slower algorithm (from versions
before 5.1.0).

This can be used to randomly swap the positions of solvent and single atom ions. The “overlap” specifies the minimum distance between ions, and the “around” keyword can be used to specify a solute (or set of atoms) around which the ions can get no closer than the distance specified. The optional keywords “noimage” disable imaging and “seed” update the random number seed. An example usage is

```
randomizeions @NA around :1-20 by 5.0 overlap 3.0
```

The above will swap Na^+ ions with water getting no closer than $5.0 \text{ \AA}$ from residues 1 – 20 and no closer than $3.0 \text{ \AA}$ from any other Na^+ ion.

37.11.68. remap

```
remap data <setname>
[outprefix <prefix>] [nobox] [parmout <filename>]
[parmopts <comma-separated-list>]

data <setname> Data set to use for remapping; should be a 1D integer
data set with X= reference (old) atom index, Y = target (new)
atom index.

outprefix <prefix> Write remapped topology to
<prefix>.<originalname>

[nobox] Remove any box information from the remapped topology.

parmout <filename> Write remapped topology to <filename>

parmopts <list> Options for writing topology file
```

Re-map atoms according to the given reference data set which is of the format:

Reference[Target]

with atom numbering starting from 1. E.g. Reference[1] = 10 would mean remap atom 10 in target to position 1.

37.11.69. replicatecell

```
replicatecell [out <traj filename>] [name <dsname>]
               { all | dir <XYZ> [dir <XYZ> ...] } [<mask>]
               [outprefix <prefix>] [nobox] [parmout <filename>]
               [parmopts <comma-separated-list>]

out <traj filename> Write replicated cell to output trajectory file.
name <dsname> If specified save replicated cell to COORDS data
               set.
all Replicate cell once in all possible directions.
dir <XYZ> Replicate cell once in specified directions. <XYZ>
           should consist of 3 numbers with no spaces in between them and
           are restricted to values of -1, 1, and 0. May be specified
           more than once.
<mask> Mask of atoms to replicate.
outprefix <prefix> Write replicated topology to
                 <prefix>.<originalname>
[nobox] Remove any box information from the replicated topology.
parmout <filename> Write replicated topology to <filename>
parmopts <list> Options for writing topology file
```

Create a trajectory where the unit cell is replicated in 1 or more directions (up to 27). The resulting coordinates and topology can be written to a trajectory/topology file. They can also be saved as a COORDS data set for subsequent processing. Currently replication is only allowed 1 axis length in either direction. The **all** keyword will replicate the cell once in all directions. The **dir** keyword can be used to restrict replication to specific directions, e.g. 'dir 10-1' would replicate the cell once in the +X, -Z directions.

For example, to replicate a cell in all directions, writing out to NetCDF trajectory cell.nc:

```
parm ../tz2.truncocct.parm7
trajin ../tz2.truncocct.nc
replicatecell out cell.nc parmout cell.parm7 all
```

37.11.70. rms | rmsd

```
rmsd [<name>] <mask> [<refmask>] [out <filename>] [mass]
      [nofit | norotate | nomod]
      [savematrices [matricesout <file>]]
      [savevectors {combined|separate} [vecsout <file>]]
      [ first | reference | ref <name> | reindex <#> | previous |
        reftraj <name> [parm <name> | parmindex <#>] ]
      [perres perresout <filename> [perresavg <avgfile>]
      [range <resRange>] [refrange <refRange>]
      [perresmask <additional mask>] [perrescenter] [perresinvert]

[<name>] Output data set name.
```

[<mask>] Mask of atoms to calculate RMSD for; if not specified, calculate for all atoms.

[<refmask>] Reference mask; if not specified, use <mask>.

[out <filename>] Output data file name.

[mass] Mass-weight the RMSD calculation.

[nofit] Do not perform best-fit RMSD.

[norotate] If calculating best-fit RMSD, translate but do not rotate coordinates.

[nomod] If calculating best-fit RMSD, do not modify coordinates.

[savematrices] If specified save rotation matrices to data set with aspect [RM].
 matricesout <file> Write rotation matrices to specified file.

[savevectors {combined|separate}] If specified save translation vectors: combined means save target-to-origin plus the origin-to-reference translation vectors, separate means save target-to-origin as Vx, Vy, Vz and save origin-to-reference as Ox Oy Oz in the output vector data set.
 vecsout <file> Output translation vector data set to <file>.

Reference keywords:

first Use the first trajectory frame processed as reference.

reference Use the first previously read in reference structure (refindex 0).

ref <name> Use previously read in reference structure specified by filename/tag.

refindex <#> Use previously read in reference structure specified by <#> (based on order read in).

previous Use frame prior to current frame as reference.

reftraj <name> Use frames from COORDS set <name> or read in from trajectory file <name> as references. Each frame from <name> is used in turn, so that frame 1 is compared to frame 1 from <name>, frame 2 is compared to frame 2 from <name> and so on. If <trajname> runs out of frames before processing is complete, the last frame of <trajname> continues to be used as the reference.

parm <parmname> | parminindex <#> If reftraj specifies a trajectory file, associate it with specified topology; if not specified the first topology is used.

Per-residue RMSD keywords:

perres Activate per-residue no-fit RMSD calculation.

perresout <perresfile> Write per-residue RMSD to <perresfile>.

perresavg <avgfile> Write average per-residue RMSDs to <avgfile>.

range <res range> Calculate per-residue RMSDs for residues in <res range> (default all solute residues).

refrange <ref range> Calculate per-residue RMSDs to reference residues in <ref range> (use <res range> if not specified).

perresmask <additional mask> By default residues are selected using the mask ':X' where X is residue number; this appends <additional mask> to the mask expression.

perrescenter Translate residues to a common center of mass prior to calculating RMSD.

perresinvert Make X-axis residue number instead of frame number.

Data Sets Created:

<name> RMSD of atoms in mask to reference.

<name>[RM] (savematrices only) Rotation matrices of target to reference.

<name>[TV] (savevectors only) Translation vector.

<name>[res] (perres only) Per-residue RMSDs; index is residue number.

<name>[Avg] (perres only) Average per-residue RMSD for each residue.

<name>[Stdev] (perres only) Standard deviation of RMSD for each residue.

*Note that **perres** data sets are not generated until **run** is called.*

Calculate the coordinate RMSD of input frames to a reference frame (or reference trajectory). Both <mask> and <refmask> must specify the same number of atoms, otherwise an error will occur.

For example, say you have a trajectory and you want to calculate RMSD to two separate reference structures. To calculate the best-fit RMSD of the C, CA, and N atoms of residues 1 to 20 in each frame to the C, CA, and N atoms of residues 3 to 23 in StructX.crd, and then calculate the no-fit RMSD of residue 7 to residue 7 in another structure named Struct-begin.rst7, writing both results to Grace-format file "rmsd1.agr":

```
reference StructX.crd [structX]
reference md.begin.rst7 [struct0]
rmsd BB :1-20@C,CA,N ref [structX] :3-23@C,CA,N out rmsd1.agr
rmsd Res7 :7 ref [struct0] out rmsd1.agr nofit
```

Per-residue RMSD calculation

If the **perres** keyword is specified, after the initial RMSD calculation the no-fit RMSD of specified residues is also calculated. So for example:

```
rmsd :10-260 reference perres perresout PRMS.dat range 190-211 perresmask &!(@H=)
```

will first perform a best-fit RMSD calculation to the first specified reference structure using residues 10 to 260, then calculate the no-fit RMSD of residues 190 to 211 (excluding any hydrogen atoms), writing the results to PRMS.dat. Two additional recommendations for the 'perres' option: 1) try not including backbone atoms by using the 'perresmask' keyword, e.g. "perresmask &!(@H,N,CA,HA,C,O)", and 2) try using the 'perrescenter' keyword, which centers each residue prior to the 'nofit' calculation; this is useful for isolating changes in residue conformation.

37.11.71. rms2d | 2drms

Although the '**rms2d**' command can still be specified as an action, it is now considered an analysis. See [37.12.31 on page 881](#).

37.11.72. rmsavgccorr

Although the '*rmsavgccorr*' command can still be specified as an action, it is now considered an analysis. See [37.12.32 on page 882](#).

37.11.73. rmsf | atomicfluct

See [37.11.6 on page 759](#).

37.11.74. rotate

```
rotate [<mask>] { [x <xdeg>] [y <ydeg>] [z <zdeg>] |
                  axis0 <mask0> axis1 <mask1> <deg> |
                  usedata <set name> [inverse] |
                  calcfrom <set name> [name <output set name>] [out <file>]
                }
```

[<mask>] Rotate atoms in <mask> (default all).

[x <xdeg>] Degrees to rotate around the X axis.

[y <ydeg>] Degrees to rotate around the Y axis.

[z <zdeg>] Degrees to rotate around the Z axis.

axis0 <mask0> Mask defining the beginning of a user-defined axis.

axis1 <mask1> Mask defining the end of a user-defined axis.

<deg> Value in degrees to rotate around user defined axis.

usedata <set name> If specified, use 3x3 rotation matrices in specified data set to rotate coordinates.

[inverse] Perform inverse rotation from input rotation matrices.

calcfrom <set name> Instead of rotating coordinates, calculate rotations around the X Y and Z axes (as well as total rotation) in degrees from existing rotation matrices specified by <set name>.

[name <output set name>] Output set name.

[out <file>] File to write output sets to.

DataSets Created:

<output set name>[TX] (calcfrom only) Rotation around the X axis in degrees.

<output set name>[TY] (calcfrom only) Rotation around the Y axis in degrees.

<output set name>[TZ] (calcfrom only) Rotation around the Z axis in degrees.

<output set name>[T] (calcfrom only) Total rotation in degrees.

Rotate specified atoms around the X, Y, and/or Z axes by the specified amounts, around a user-defined axis (specified by <mask0> and <mask1>), or use a previously read in or generated data set of 3x3 matrices to perform rotations.

For example, to rotate the entire system 90 degrees around the X axis:

```
rotate x 90
```

To rotate residue 270 90 degrees around the axis defined between atoms C1, C2, C3, C4, C5, and C6 in residue 270 and atoms C7, C8, C9, C10, C11, and C12 in residue 270:


```
rotate :270 axis0 :270@C1,C2,C3,C4,C5,C6 axis1 :270@C7,C8,C9,C10,C11,C12 90.0
```

To rotate the system with rotation matrices read in from rmatrices.dat:

```
trajin tz2.norotate.crd
readdata rmatrices.dat name RM mat3x3
rotate usedata RM
```

To calculate rotations from rotation matrices generated by a previous RMSD calculation:

```
parm ../tz2.parm7
reference tz2.separate.rotate.rst7.save name REF
trajin ../tz2.nc
rms R0 reference savematrices matricesout matrices.dat
rotate calcfrom R0[RM] name Rot out rotations.dat
```

37.11.75. rotdif

The *'rotdif'* command is now an analysis (see [37.12.33 on page 883](#)), and requires that rotation matrices be generated via an *rmsd* action. For example:

```
reference avgstruct.pdb
trajin tz2.nc
rms R0 reference @CA,C,N,O savematrices
rotdif rmatrix R0[RM] rseed 1 nvecs 10 dt 0.002 tf 0.190 \
      itmax 500 tol 0.000001 d0 0.03 order 2 rvecout rvecs.dat \
      rmout matrices.dat deffout deffs.dat outfile rotdif.out
```

37.11.76. runavg | runningaverage

```
runavg [window <window_size>]
```

Note that for backwards compatibility with ptraj "runningaverage" is also accepted.

Replaces the current frame with a running average over a number of frames specified by **window** <window_size> (5 if not specified). This means that in order to build up the correct number of frames to calculate the average, the first <window_size> minus one frames will not be processed by subsequent actions. So for example given the input:

```
runavg window 3
rms first out rmsd.dat
```

the rms command will not take effect until frame 3 since that is the first time 3 frames are available for averaging (1, 2, and 3). The next frame processed would be an average of frames 2, 3, and 4, etc.

37.11.77. scale

```
scale x <sx> y <sy> z <sz> <mask>
```

Scale the X|Y|Z coordinates of atoms in <mask> by <sx>|<sy>|<sz>.

37.11.78. secstruct

```
secstruct [<name>] [out <filename>] [<mask>] [sumout <filename>]
[assignout <filename>] [totalout <filename>] [ptrajformat]
[betadetail]
[namen <N name>] [nameh <H name>] [nameca <CA name>]
[namec <C name>] [nameo <O name>] [namesg <sulfur name>]
```

[<name>] Output data set name.

[out <filename>] Output file name for secondary structure vs time.

[<mask>] Atom mask in which residues should be looked for.

[sumout <sumfilename>] Write average secondary structure values for each residue to <sumfilename>; if not specified <filename>.sum is used.

[assignout <filename>] Write overall secondary structure assignment (based on dominant secondary structure type for each residue) to file.

[ptrajformat] Write secondary structure as a string of characters for each frame, similar to ptraj output.

[betadetail] Record anti-parallel beta and parallel beta in place of extended and bridge secondary structure. If a residue could be both only anti-parallel is reported.

[namen <N name>] Backbone amide nitrogen atom name (default 'N').

[nameh <H name>] Backbone amide hydrogen atom name (default 'H').

[nameca <CA name>] Backbone alpha carbon atom name (default 'CA').

[namec <C name>] Backbone carbonyl carbon atom name (default 'C').

[nameo <O name>] Backbone carbonyl oxygen atom name (default 'O').

[namesg <SG name>] Cysteine sulfur atom name, used to ignore disulfide connectivity (default 'SG').

Data Sets Created:

<name>[res] Residue secondary structure per frame; index corresponds to residue number. If ptrajformat specified these will be characters, otherwise integers (see table below).

<name>[avgss] Average of each type of secondary structure; index corresponds to secondary structure type (see table below; no index for 'None').

<name>[None] Total fraction of residues with no structure vs time.

<name>[Para] Total fraction of residues with parallel beta structure vs time.

<name>[Anti] Total fraction of residues with anti-parallel beta structure vs time.

<name>[3-10] Total fraction of 3-10 helical structure vs time.

<name>[Alpha] Total fraction of alpha helical structure vs time.

<name>[Pi] Total fraction of Pi helical structure vs time.

<name>[Turn] Total fraction of turn structure vs time.

<name>[Bend] Total fraction of bend structure vs time.

As of version 4.18.0, this command now produces output that better conforms with the original definitions in Kabsch and Sander 1983; namely that Extended beta (i.e. 2 or more consecutive beta bridges of the same type) and beta Bridge (i.e. an isolated beta bridge) are now reported instead of anti-parallel and parallel beta. To restore the original behavior the 'betadetail' keyword must be specified.

*Note that the residue and [avgss] data sets are not generated until **run** is called.*

Calculate secondary structural propensities for residues in **<mask>** (or all solute residues if no mask given) using the DSSP method of Kabsch and Sander[789], which assigns secondary structure types for residues based on backbone amide (N-H) and carbonyl (C=O) atom positions. By default *cpptraj* assumes these atoms are named "N", "H", "C", and "O" respectively. If a different naming scheme is used (e.g. amide hydrogens are named "HN") the backbone atom names can be customized with the **nameX** keywords (e.g. 'nameH HN'). Note that it is expected that some residues will not have all of these atoms (such as proline); in this case *cpptraj* will print an informational message but the calculation will proceed normally. If a residue has no atoms selected it will be skipped. When determining residue connectivity, disulfide bonds will be ignored; *cpptraj* identifies such bonds based on the **namesg** atom name (default "SG").

Results will be written to filename specified by **out** with format:

<#Frame> <ResX SS> <ResX+1 SS> ... <ResN SS>

where **<#Frame>** is the frame number and **<ResX SS>** is an integer representing the calculated secondary structure type for residue X. If the keyword **ptrajformat** is specified, the output format will instead be:

<#Frame> STRING

where **STRING** is a string of characters (one for each residue) where each character represents a different structural type (this format is similar to what *ptraj* had outputted and is retained for backwards compatibility). The various secondary structure types and their corresponding integer/character are listed below. If 'betadetail' is specified what is reported and the characters used change slightly.

STRING (betadetail)	Integer	DSSP	SS type (betadetail)
0	0	' '	None
E (b)	1	'E'	Extended beta (parallel beta)
B	2	'B'	Isolated beta (anti-parallel beta)
G	3	'G'	3-10 helix
H	4	'H'	Alpha helix
I	5	'I'	Pi (3-14) helix
T	6	'T'	Turn
S	7	'S'	Bend

Average structural propensities over all frames for each residue will be written to the file specified by **sumout** (or "**<filename>.sum**" if **sumout** is not specified). The total structural propensity over all residues for each secondary structure type will be written to the file specified by **totalout**. If **assignout** is specified, the overall secondary structure assignment for each residue will be printed in two line chunks of 50 residues, with the first line containing the residue number the line starts with and one character residue names, and the second line containing secondary structure assignment using DSSP-style characters, like so:

```
1 KCNTATCATQ RLANFLVHSS NNFGAILSST NVGSNTRn
   SSS   TH HHHTTSEEEE TTTEEEE SS       S
```

The output of `secstruct` command is amenable to visualization with `gnuplot`. To generate a 2D map-style plot of secondary structure vs time, with each residue on the Y axis simply give the output file a “.gnu” extension. For example, to generate a 2D map of secondary structure vs time, with different colors representing different secondary structure types for residues 1-22:

```
secstruct :1-22 out dssp.gnu
```

The resulting file can be visualized with `gnuplot`:

```
gnuplot dssp.gnu
```

Similarly, the `sumout` file can be nicely visualized using `xmgrace` (use “.agr” extension).

```
secstruct :1-22 out dssp.gnu sumout dssp.agr
xmgrace dssp.agr
C <X> <Y> <Z> <Density>
```

Values of `dgbulk` and `dhbulk` for different water models can be calculated from pure water simulations with the `purewater` keyword.

37.11.79. setvelocity

```
setvelocity [<mask>]
  [{ tempi <temperature> |
    scale [factor <fac>] [sx <xfac>] [sy <yfac>] [sz <zfac>] |
    add [value <val>] [vx <xval>] [vy <yval>] [vz <zval>] |
    none |
    modify}]
  [[ntc <#>]] [[dt <time>] [epsilon <eps>]]
  [zeromomentum] [ig <random seed>]
```

<mask> Mask of atoms to assign velocities to.

tempi <temperature> Assign velocities at specified temperature (default 300.0 K).

scale Scale existing velocities

[factor <fac>] Factor to scale velocities by.

[sx <xfac>] Factor to scale X component of velocities by.

[sy <yfac>] Factor to scale Y component of velocities by.

[sz <zfac>] Factor to scale Z component of velocities by.

add Add to existing velocities

[value <val>] Value to add to velocities.

[vx <xval>] Value to add to X component of velocities.

[vy <yval>] Value to add to Y component of velocities.

[vz <zval>] Value to add to Z component of velocities.

none Remove any velocities.

modify If specified, do not set, just modify any existing velocities (via ‘ntc’ or ‘zeromomentum’).

ig <random seed> Random seed to use to generate velocity distribution.

ntc <#> Correct set velocities for SHAKE constraints. Numbers match sander/pmemd: 1 = no SHAKE, 2 = SHAKE on hydrogens, 3 = SHAKE on all atoms.

dt <time> Time step for SHAKE correction.
epsilon <eps> Epsilon for SHAKE correction
zeromomentum If specified adjust velocities so the total momentum of atoms in <mask> is zero.

Set velocities in frame for atoms in <mask> using Maxwellian distribution based on given temperature, optionally adjusted for SHAKE constraints. Can also be used to modify existing velocity information or remove it entirely. The total momentum of the system can be set to zero as well, which can be useful for NVE simulations.

37.11.80. spam

```
spam [name <name>] [out <datafile>] [cut <cut>] [solv <solvname>]
{ purewater |
  <peaksname> [reorder] [info <infofile>] [summary <summary>]
  [site_size <size>] [sphere] [temperature <T>]
  [dgbulk <dgbulk>] [dhbulk <dhbulk>] }
```

name <name> Output data sets name.

out <datafile> Data file with all SPAM energies for each snapshot.

cut <cut> Non-bonded cutoff for energy evaluation

solv <solvname> Name of the solvent residues.

[purewater] The system is pure water. Used to parametrize the bulk values. If this is specified, none of the below options are relevant.

<peaksname> Data set or file (XYZ- format: see below) with the peak locations present .

[reorder] The solvent should be re-ordered so the same solvent molecule is always in the same site.

info <infofile> File with stats about which sites are occupied when.

summary <summary> File with the summary of all SPAM results. If not specified, no SPAM energies will be calculated.

site_size <size> Size of the water site around each density peak (sphere diameter/box edge length) in Ang.

[sphere] Treat each site like a sphere.

temperature <T> Temperature at which SPAM calculation was run.

dgbulk <dgbulk> SPAM free energy of the bulk solvent in kcal/mol; default is -30.3 kcal/mol (SPC/E water).

dhbulk <dhbulk> SPAM enthalpy of the bulk solvent in kcal/mol; default is -22.2 kcal/mol (SPC/E water).

Data Sets Created for 'purewater':

<name> Energies for each water at each frame.

Data Sets Created otherwise:

<name>:<#> SPAM energies for peak <#> starting from 1.

<name>[DG] SPAM delta G values for valid peaks.

<name>[DH] SPAM delta H values for valid peaks.

<name>[-TDS] SPAM -T * delta S values for valid peaks.

Perform profiling of bound water molecules via SPAM analysis[790]. Briefly, this method identifies and estimates the free energy profiles of bound waters via calculation of the distribution of interaction energies between the water and it's environment from explicit solvent MD trajectories. The interaction energies are calculated using a force- and energy-shifted electrostatic term with a hard cutoff. For a given peak, SPAM energies will only be calculated for peaks where the peak is singly-occupied (i.e. a multiple-occupied peak is not considered valid).

Prior to this command, the *volmap* command should be run with the **peakfile** keyword (see 37.11.93 on page 842) to generate the peaks file. If not using peaks from the *volmap* command, the peaks file should have one line per peak with format:

```
<# of peaks>
C      <X>      <Y>      <Z>      <Peak Density>
...
```

With a 'C' line for each peak.

37.11.81. stfcdiffusion

```
stfcdiffusion mask <mask> [out <file>] [time <time per frame>]
                    [mask2 <mask> [lower <distance>] [upper <distance>]]
                    [nwout <file>]] [avout <file>] [distances] [com]
                    [x|y|z|xy|xz|yz|xyz]
```

mask Atoms for which MSDs will be computed.

out Output file: time vs. MSD.

time Time step in the trajectory. (1.0 ps)

mask2 Compute MSDs only within the lower and upper limit of mask2.

IMPORTANT: may be very slow!!!

lower Smaller distance from reference point(s). (0.01 Å)

upper Larger distance from reference point(s). (3.5 Å)

nwout Output file containing number of water molecules in the chosen region, see mask2. (off)

avout Output file containing average distances. (off)

x|y|z|xy|xz|yz|xyz Computation of the mean square displacement in the chosen dimension. (xyz)

distances Dump un-imaged distances. By default only averages are output. (off)

com Calculate MSD for centre of mass. (off)

Calculate diffusion for selected atoms using code based on the 'diffusion' routine developed by Hannes Loeffler at STFC (<http://www.stfc.ac.uk/CSE>).

37.11.82. strip

```
strip <mask> [charge <new charge>]
          [outprefix <prefix>] [nobox] [parmout <filename>]
          [parmopts <comma-separated-list>]
```

<mask> Remove atoms specified by mask from the system.

charge <new charge> Scale charges so total charge of remaining atoms matches the specified <new charge>.

[outprefix <prefix>] Write out stripped topology file with name '**<prefix>.<Original Topology Name>**'.

[nobox] Remove any box information from the stripped topology.
[parmout <file>] Write stripped topology to file with name <file>.
[parmopts <list>] Options for writing topology file.

Strip all atoms specified by <mask> from the frame and modify the topology to match for any subsequent Actions. The **outprefix** keyword can be used to write stripped topologies; stripped Amber topologies are fully-functional. Available options for <parmopts> can be determined by running the *help Formats parmwrite* command.

Note that stripping a system rennumbers all atoms and residues, so for example after this command:

```
strip :1
```

residue 1 will be gone, and the former second residue will now be the first, and so on.

For example, to strip all residues named WAT from each topology/coordinate frame:

```
strip :WAT
```

The next example uses a distance-based mask to strip atoms in a single frame. Note that with the exception of the *mask* command, distance-based masks do not update on a per-frame basis. To strip all residues outside of 6.0 from any atom in residues 1 to 14 and write out the stripped topology and coordinates, both with no box information:

```
parm parm7
trajin frame1000.rst.1
reference frame1000.rst.1
strip !(1-14<:6.0) outprefix f1.1 nobox
trajout f1.1.x restart nobox
```

37.11.83. surf

```
surf [<name>] [<mask1>] [out <filename>] [solutemask <mask>]
[offset <offset>] [nbrcut <cut>]
<name> Output data set name.
<mask1> Atoms to calculate surface area for.
out <filename> File to write surface area to.
solutemask <mask> If specified, calculate the contribution of
<mask1> to <mask>.
offset <offset> Increment van der Waals radii by <offset>; 1.4 Ang.
is the default (as used by Amber).
nbrcut <cut> Only atoms with van der Waals radii greater than <cut>
are considered to have neighbors (2.5 Ang Amber default).
```

Calculate the surface area in Å² of atoms in <mask> (if no mask specified, all atoms not marked as 'solvent' that are part of a molecule > 1 atom in size) using the LCPO algorithm of Weiser et al.[185]. In order for this to work, the topology needs to have bond information and atom type information.

Note that even if <mask> does not include all solute atoms, the neighbor list is still calculated for all solute atoms so the surface area calculated reflects the contribution of atoms in <mask> to the overall surface area, not the surface area of <mask> as an isolated system. As a result, it may be possible to obtain a negative surface area if only a small fraction of the solute is selected.

For example, to calculate the overall surface area of all solute atoms, as well as the contribution of residue 1 to the overall surface area, writing both results to "surf.dat":

```
surf out surf.dat
surf :1 out surf.dat
```

37.11.84. symmrmsd

```
symmrmsd [<name>] [<mask>] [<refmask>] [out <filename>] [nofit] [mass] [remap]
[ first | reference | ref <name> | refindex <#> | previous |
  reftraj <name> [parm <parmname> | parmindex <#>] ]
```

[<name>] Output data set name.

[<mask>] Mask of atoms to calculate RMSD for; if not specified, calculate for all atoms.

[<refmask>] Reference mask; if not specified, use <mask>.

[out <filename>] Output data file name.

[nofit] Do not perform best-fit RMSD (not recommended).

[mass] Mass-weight the RMSD calculation.

[remap] Re-arrange atoms according to symmetry. See below for more details.

Reference keywords:

first Use the first trajectory frame processed as reference.

reference Use the first previously read in reference structure (refindex 0).

ref <name> Use previously read in reference structure specified by filename/tag.

refindex <#> Use previously read in reference structure specified by <#> (based on order read in).

previous Use frame prior to current frame as reference.

reftraj <name> Use frames from COORDS set <name> or read in from trajectory file <name> as references. Each frame from <name> is used in turn, so that frame 1 is compared to frame 1 from <name>, frame 2 is compared to frame 2 from <name> and so on. If <trajname> runs out of frames before processing is complete, the last frame of <trajname> continues to be used as the reference.

parm <parmname> | parmindex <#> If reftraj specifies a file associate trajectory <name> with specified topology; if not specified the first topology is used.

Perform symmetry-corrected RMSD calculation. This is done by identifying potential symmetric atoms in each residue, performing an initial best-fit, then determining which configuration of symmetric atoms will give the lowest RMSD using atomic distance to reference atoms.

Note that when re-mapping, all atoms in the residues of interest should be selected to prevent cases where selected symmetric atoms are swapped but the atoms they are bonded to are not. Also, occasionally larger symmetric structures (e.g. 6 membered rings) may become distorted due to only part of the residue being corrected for symmetry. This appears to happen about 4% of the time but does not overly inflate the RMSD. The '*check*' command can be used after *symmrmsd* to look for such distortions.

Warning: the symmetry correction is generally robust enough to account for symmetries in the standard amino and nucleic acid residues, but has not been extensively tested on residues with more extended types of symmetry.

37.11.85. temperature

```
temperature [<name>] [out <filename>]
           { frame |
             [<mask>] [ntc <#>] [update] [remove {trans|rot|both}]
           }
```

[<name>] Data set name.

[out <filename>] File to write values to.

frame Do not calculate temperature; use existing frame temperature.

[<mask>] Atoms to calculate temperature for.

[ntc <#>] Value of SHAKE bond constraint: 1 - none, 2 - bonds to H,
3- all bonds (equivalent to SANDER/PMEMD).

[update] Update temperature in Frames with calculated temperatures.

[remove {trans|rot|both}] Correct for removed translational, rotational,
or both kinds of degrees of freedom.

Calculate temperature in frame based on velocity information. If 'update' is specified, update frame temperature too. If 'frame' is specified just use frame temperature (e.g. read in from a REMD trajectory).

The 'ntc' keyword can be used to correct for lost degrees of freedom due to SHAKE constraints (2 = bonds to hydrogen, 3 = all bonds). The 'remove' keyword can be used to account for removed translational and/or rotational degrees of freedom.

For example, if using a trajectory that has been generated with SHAKE on hydrogens, no periodic boundary conditions (i.e. no box), and has had the center of mass periodically removed:

```
temperature T1 ntc 2 remove both out T1.dat
```

If using a trajectory that has been generated with SHAKE on hydrogens, periodic boundary conditions (i.e. with a box), and has had the center of mass periodically removed:

```
temperature T1 ntc 2 remove trans out T1.dat
```

If using a trajectory that has been generated with SHAKE on all bonds, periodic boundary conditions, and no center of mass motion removal:

```
temperature T1 ntc 3 out T1.dat
```

37.11.86. time

```
time {time0 <initial time> dt <step> [update] | remove}
```

time0 <initial time> Time of the first frame (ps).

dt <step> Time step between frames (ps).

[update] If specified, modify any existing time info.

remove Remove any time info from frame.

Either add time information to frames, modify existing time information in frames, or remove existing time information from frames. Note that currently COORDS data sets do not store time information, so using this command with the *crdaction* command will have no effect.

37.11.87. tordiff

```
tordiff [<set name>] [<mask>] [mass] [out <file>] [diffout <file>] [time <dt>]
```

[<set name>] Output mean-squared-displacement data sets name.

[<mask>] Mask of molecules to calculate diffusion for.

[mass] Use center of mass of molecules instead of geometric center.

[out <file>] File to write mean-squared-displacement sets to.

[diffout <file>] Write diffusion constants calculated from fits of mean-squared-displacement data sets to <filename>.

[time <dt>] Time between frames in ps.

DataSet Aspects:

[X] MSD(s) in the X direction.

[Y] MSD(s) in the Y direction.

[Z] MSD(s) in the Z direction.

[R] Overall MSD(s).

[A] Overall displacement(s) in Å.

[D] Diffusion constants (1×10^{-5} cm²/s).

[Label] Diffusion constant labels.

[Slope] Linear regression slopes.

[Intercept] Linear regression Y-intercepts.

[Corr] Linear regression correlation coefficients.

Calculate the diffusion via mean-squared displacement (in Å²/ps) of specified molecules using the toroidal-view-preserving (TOR) scheme of Hummer et al. **Note that currently this only works for orthogonal boxes.** (<https://arxiv.org/abs/2303.09418>). Unlike the *diffusion* (37.11.24 on page 771) and *unwrap* (37.11.90 on the facing page) commands which correct for box fluctuations via fractional coordinates, the TOR scheme corrects for box fluctuations by tracking the displacement of each molecule with respect to its position in the previous frame in Cartesian space.

Diffusion constants are calculated in the same manner as the *diffusion* command; for more details see 37.11.24 on page 771.

For example, to calculate the diffusion (with data set name TOR) of waters (name WAT) in an orthogonal box using the toroidal scheme:

```
parm tz2.ortho.parm7
trajin tz2.ortho.nc
tordiff TOR :WAT@O out tor.msd.dat diffout tor.diff.dat
```

37.11.88. trans | translate

```
translate [<mask>] {[x <dx>] [y <dy>] [z <dz>] | topoint <x>,<y>,<z> [mass]}
```

<mask> Mask of atoms to translate (all atoms if not specified).

x<dx> Translation (delta) in the X direction (Å).

y<dy> Translation (delta) in the Y direction (Å).

z<dz> Translation (delta) in the Z direction (Å).

topoint <x>,<y>,<z> If specified, translate center of specified atoms to a specific point defined by <x>, <y>, and <z> in the given comma-separated list instead of by deltas.
mass If specified, translate center of mass of specified atoms (topoint only).

Translate atoms in <mask> (all atoms if no mask specified) <dx> Å in the X direction, <dy> Å in the Y direction, and <dz> Å in the Z direction. If 'topoint' is specified, translate atoms in <mask> to the specified coordinates (also in Å).

37.11.89. unstrip

unstrip

Requests that the original topology and frame be used for all following actions. This has the effect of undoing any command that modifies the state (such as strip). For example, the following code takes a solvated complex and uses a combination of strip, unstrip, and outtraj commands to write out separate dry complex, receptor, and ligand files:

```
parm Complex.WAT.pdb
trajin Complex.WAT.pdb
# Remove water, write complex
strip :WAT
outtraj Complex.pdb pdb
# Reset to solvated Complex
unstrip
# Remove water and ligand, write receptor
strip :WAT,LIG
outtraj Receptor.pdb pdb
# Reset to solvated Complex
unstrip
# Remove water and receptor, write ligand
strip :WAT
strip !(:LIG)
outtraj Ligand.pdb pdb
```

37.11.90. unwrap

```
unwrap [center] [{byatom | byres | bymol}] [avgucell <avg ucell set>]
[ reference | ref <name> | reindex <#> ] [<mask>]
[scheme {frac|tor}]
```

[center] Unwrap by center of mass; otherwise unwrap by first atom position (byres and bymol).

byatom Unwrap by atom (default).

byres Unwrap by residue.

bymol Unwrap by molecule.

[avgucell <avg ucell set>] Average unit cell data set; useful when unwrapping NPT trajectories.

[reference | ref <name> | reindex <#>] Reference structure to use in unwrapping.

[<mask>] Selection to unwrap.

[scheme frac|tor]] Unwrap using either fractional coordinates (default) or toroidal-view-preserving scheme (orthogonal boxes only) .

Under periodic boundary conditions, MD trajectories are not continuous if molecules are wrapped (imaged) into the central unit cell. Especially, in sander, with *iwrap*=1, molecular trajectories become discontinuous when a molecule crosses the boundary of the unit cell. This command, **unwrap** processes the trajectories to force the *masked* molecules continuous by translating the molecules into the neighboring unit cells. It is the opposite function of **image**, but this command can also be used to place molecules side by side, for example, two strands of a DNA duplex. However, this command may fail if the *masked* entities travel more than half of the box size within a single frame. Note that unwrapping by atom (the default behavior) is slower than unwrapping by residue or molecule, but is usually the safer method, especially if it is unknown if the original imaging was done by atom, by residue, or by molecule. If the optional reference arguments are specified, then the first frame is unwrapped according to the reference structure. Otherwise, the first frame is not modified.

To remove the “noise” caused by box fluctuations in NPT trajectories, the average unit cell vectors describing the average box can be provided with the **avgucell** keyword; see the **avgbox** command (37.11.10 on page 763).

By default coordinates are unwrapped using fractional coordinates. To unwrap via the toroidal-view-preserving scheme (<https://arxiv.org/abs/2303.09418>) specify **scheme tor**. However, note that this currently **only works for orthogonal boxes** and may result in unrealistic trajectories (particularly at long times when molecules have had the chance to traverse multiple box lengths) and is only currently recommended for calculating diffusion.

As an example, assume that :1-10 is the first strand of a DNA duplex and :11-20 is the other strand of the duplex. Then the following commands could be used to create system where the two strands are not separated artificially:

```
unwrap :1-20
center :1-20 mass origin
image origin center familiar
```

To unwrap an NPT trajectory by using the average unit cell (box), the calculate diffusion from the unwrapped trajectory:

```
parm tz2.ortho.parm7
trajin tz2.ortho.nc
# Create average box data
avgbox MyBox
run
# Unwrap using average box data
unwrap bymol avgucell MyBox[avg]
# Calculate diffusion
diffusion Water :WAT@O out tz2.ortho.wato.dat
run
```

37.11.91. vector

```
vector [<name>] <Type> [out <filename> [ptrajoutput]] [<mask1>] [<mask2>]
[magnitude] [geom] [ired] [gridset <grid>] [debye]
<Type> = { mask      | minimage | dipole | center  | corplane |
          box       | boxcenter | ucellx | ucelly  | ucellz  |
          momentum | principal [x|y|z] | velocity | force }
```

[<name>] Vector data set name.

<Type> Vector type; see below.

[out <filename>] Write vector data to <filename> with format 'Vx Vy Vz Ox Oy Oz' where V denotes vector coordinates and 'O' denotes origin coordinates.

[ptrajoutput] Write vector data in ptraj style (Vx Vy Vz Ox Oy Oz Vx+Ox Vy+Oy Vz+Oz). This prevents additional formatting of <filename> and is not compatible with 'magnitude'.

[<mask1>] Atom mask, required for all types except 'box'.

[<mask2>] Second atom mask, only required for type 'mask'.

[magnitude] Store the magnitude of the vector with aspect [Mag].

[geom] If specified, use geometric centers instead of centers of mass.

[ired] Mark this vector for subsequent IRED analysis with commands 'matrix ired' and 'ired'.

[gridset <grid>] Name of grid data set to get box info from instead of frame for box, boxcenter, and ucell[x|y|z].

[debye] ('dipole' vector only) Report dipole vector in units of Debye instead of e-*Ang.

Data Sets Created:

<name> Vector data set.

<name>[Mag] (magnitude only) Vector magnitude.

This command will keep track of a vector value (and its origin) over the trajectory; the data can be referenced for later use based on the *name* (which must be unique). The types of vectors that can be calculated are:

mask (Default) Store vector from center of mass of atoms in <mask1> to atoms in <mask2>.

minimage Store minimum-imaged vector from center of mass of atoms in <mask1> to atoms in <mask2>.

dipole Store the dipole and center of mass of the atoms specified in <mask1>. The dipole vector has units of e-*Ang unless 'debye' is specified for units of Debye. The center is always stored as simply Ang (since it is just coordinates). Note that the value may not be well-defined if the atoms in the mask are not overall charge neutral.

center Store the center of mass of atoms in <mask1>. The reference point is the origin (0.0, 0.0, 0.0).

corrplane This defines a vector perpendicular to the (least-squares best) plane through the atoms in <mask1>. The reference point is the center of mass of atoms in <mask1>.

box (No mask needed) Store the box lengths of the trajectory. The reference point is the origin (0.0, 0.0, 0.0).

boxcenter (No mask needed) Store the center of the box as a vector.

ucell{x|y|z}: (No mask needed) Store specified unit cell (i.e. box) vector.

momentum Store momentum of atoms selected by <mask1> (requires velocities).

principal [x|y|z] Store one of the principal axis vectors determined by diagonalization of the inertial matrix from the coordinates of the atoms specified by <mask1>. The eigenvector with the largest eigenvalue is considered "x" (i.e., the hardest axis to rotate around) and the eigenvector with the smallest eigenvalue is considered "z". If none of x or y or z are specified, then the "x" principal axis is stored. The reference point is the center of mass of atoms in <mask1>.

velocity Store velocity of atoms in <mask1> (requires velocities).

force Store force of atoms in <mask1> (requires forces).

Cpptraj supports writing out vector data in a pseudo-trajectory format for easy visualization. Once a vector data set has been generated the writedata command can be used with the vectraj keyword (see [37.6 on page 697](#) for more details) to write a pseudo trajectory consisting of two atoms, one for the vector origin and one for the vector from the origin (i.e. V+O). For example, to create a MOL2 containing a pseudo-trajectory of the minimum-imaged vector from residue 4 to residue 11:

```
trajin tz2.nc
vector v8 minimage out v8.dat :4 :11
run
writedata v8.mol2 vectraj v8 trajfmt mol2
```

Auto-correlation or cross-correlation functions can be calculated subsequently for vectors using either the *corr* analysis command or the *timecorr* analysis command (to calculate via spherical harmonic theory).

37.11.92. velocityautocorr

```
velocityautocorr [<set name>] [<mask>] [usevelocity] [out <filename>] [diffout <file>]
                  [maxlag <frames>] [tstep <timestep>] [direct] [norm]
```

[<set name>] Data set name.

[<mask>] Atoms(s) to calculate velocity autocorrelation (VAC) function for.

[usevelocity] Use velocity information in frame if present. This will only give sensible results if the velocities are recorded close to the order of the simulation time step.

[out <filename>] Write VAC function to <filename>.

[diffout <file>] File to write diffusion constants to.

[maxlag <frames>] Maximum lag in frames to calculate VAC function for. Default is half the total number of frames.

[tstep <timestep>] Time between frames in ps (default 1.0).

[direct] Calculate VAC function directly instead of via FFT (will be much slower).

[norm] Normalize resulting VAC function to 1.0.

DataSet Aspects:

[D] Diffusion constant calculated from integral over VAC function in $1 \times 10^{-5} \text{ cm}^2/\text{s}$.

Calculate the velocity autocorrelation (VAC) function averaged over the atoms in <mask>. Pseudo-velocities are calculated using coordinates and the specified time step. As with all time correlation functions the statistical noise will increase if the maximum lag is greater than half the total number of frames. In addition to calculating the velocity autocorrelation function, the self-diffusion coefficient will be reported in the output, calculated from the integral over the VAC function.

37.11.93. volmap

```
volmap [out <filename>] <mask> [radscale <factor>] [stepfac <fac>]
      [sphere] [radii {vdw | element}] [splinedx <spacing>]
      [calcpeaks] [peakcut <cutoff>] [peakfile <xyzfile>]
```

```
{ data <existing set> |
  name <setname> <dx> [<dy> <dz>]
    { size <x,y,z> [center <x,y,z>] |
      centermask <mask> [buffer <buffer>] |
      boxref <reference> } }
```

out <filename> The name of the output file with the grid density.

<mask> The atom selection from which to calculate the number density.

radscale <factor> Factor by which to scale radii (by division). To match the atomic radius of Oxygen used by the VMD volmap tool, a scaling factor of 1.36 should be used. Default 1.0.

stepfac <factor> Factor for determining how many voxels to smear Gaussian (default 4.1, 1.0 for sphere).

sphere When smearing Gaussian, skip voxels farther than radii/2.

radii {vdw|element} Specify either van der Waals radii (default) or elemental radii.

splinedx <spacing> Spacing to use for cubic spline interpolation (default 0.01 Ang.).

calcpeaks If specified, peaks in the grid density will be calculated and saved to set <setname> with aspect ``peaks``.

peakcut <cutoff> The minimum density required to consider a local maximum a 'density peak' in the outputted peak file (default 0.05).

peakfile <xyzfile> A file in XYZ-format that contains a carbon atom centered at the grid point of every local density maximum. This file is necessary input to the spam action command.

data <setname> Name of existing grid data set to use.

name <setname> Name of grid set that will be created (size/center or centermask/buffer keywords).

dx,dy,dz The grid spacing (Angstroms) in the X-, Y-, and Z-dimensions, respectively.

size <x,y,z> Specify the size of the grid in the X-, Y-, and Z-dimensions. Must be used alongside the center argument.

center <x,y,z> Specify the grid center explicitly. Note, the size argument must be present in this case. Default is the origin.

centermask <mask> The mask around which the grid should be centered (via geometric center). If this is omitted and the center and size are not specified, the default <mask> entered (see above) is used in its place.

buffer <buffer> A buffer distance, in Angstroms, by which the edges of the grid should clear every atom of the centermask (or default mask if centermask is omitted) in every direction. The default value is 3. The buffer is ignored if the center and size are specified (see below).

boxref <reference> Set up the grid using the unit cell info in the specified reference.

Data Sets Created:**<setname>** The 3D grid.**<setname>[peaks]** The density peaks if **calcpeaks** specified.

Grid data as a volumetric map, similar to the 'volmap' command in VMD. The density is calculated by treating each atom as a 3-dimensional Gaussian function whose standard deviation is equal to the van der Waals radius. The density calculated is the number density averaged over the entire simulation. The grid can be specified in one of three ways:

1. An existing grid data set (from e.g. bounds), specified with the **data** keyword.
2. Via the sizes and center specified by the **size** and **center** keywords (comma-separated strings, e.g. '20,20,20').
3. Centered on the atoms in the mask given by **centermask** with an additional buffer in each direction specified by **buffer**.

The calculation is sped up by using cubic splines to interpolate the exponential function when calculating the Gaussians.^[791]

37.11.94. volume**volume** [**<name>**] [**out <filename>**]**<name>** Data set name.**out <filename>** Output file name.

Calculate unit cell volume.

37.11.95. watershell

```
watershell <solutemask> [out <filename>] [lower <lower cut>] [upper <upper cut>]
           [noimage] [<solventmask>]
```

<solutemask> Atom mask corresponding to solute of interest (required).

[out <filename>] Output file name.

[lower <lower cut>] Cutoff for the first water shell (default 3.4 Angstroms).

[upper <upper cut>] Cutoff for the second water shell (default 5.0 Angstroms).

[noimage] Do not image distances.

[<solventmask>] Optional atom mask corresponding to solvent.

DataSet Aspects:

[lower] Number of solvent molecules in first solvent shell.

[upper] Number of solvent molecules in second solvent shell.

This option will count the number of waters within a certain distance of the atoms in the **<solutemask>** in order to represent the first and second solvation shells. The optional **<solventmask>** can be used to consider other atoms as the solvent; the default is ":WAT".

This action is often used prior to the *closest* command in order to determine how many waters around a solute should be retained to maintain the first and/or second water shells.

As of version 17 this command is CUDA-enabled in CUDA versions of CPPTRAJ.

37.11.96. xtalsymm

```
xtalsymm <mask> group <space group> [collect [centroid]]
      [ first | reference | ref <name> | reindex <#> ]
      [na <na>] [nb <nb>] [nc <nc>]
```

<mask> Atom mask defining the asymmetric unit within the larger system (required).

group <space group> The space group to which the system belongs. Omit spaces in the name. Example: `''P22(1)2(1)''`.

[collect] Optional flag to have all solvent particles, not just the asymmetric units, re-imaged. This will trigger `cpptraj` to compute the unit cell volume that constitutes the asymmetric unit and thereby classify all particles for re-imaging.

[centroid] If specified along with `collect`, re-image solvent molecules by centroids, not individual atom coordinates. This is useful for keeping water molecules intact.

[first | reference | ref <name> | reindex <#>] Reference structure to use for determining crystal symmetry.

[na <na>] [nb <nb>] [nc <nc>] The number of times the crystal unit cell is replicated along the `\a,` `\b,` or `\c` axes (for orthorhombic unit cells, these are the `x`, `y`, and `z` axes) of the simulation; default is 1. Many crystal unit cells are too small in one or more dimensions for our simulation cutoffs, and replicating the unit cell is an effective way to counter imaging artifacts even for larger unit cells.

Calculate the optimal approach for superimposing symmetry-related subunits of the simulation back onto one another. The calculation assumes that the system is a simulation of an X-ray structure in its native crystal lattice, finds all copies of the asymmetric unit among the entire system, and devises plans for re-imagining their coordinates to superimpose them back on the original asymmetric unit. The space group information can be found in a PDB X-ray structure used as the initial coordinates for a simulation. All 230 space groups are supported, and a scan of the PDB was made to ensure that common variants of the names are included (P2(1)22(1) is the same as P22(1)2(1), but with different axis conventions). If your space group is not understood, contact the Amber mailing list. This command is compute intensive, especially for simulations that are “supercells” containing many crystallographic unit cells.

This command will cause `cpptraj` to locate all asymmetric units from within the topology, then determine what wrapping, if any, has occurred in order to bring about an optimal re-alignment based on the space group symmetry operations. The user need not worry about wrapping or drift of the simulation over time—the asymmetric units will be re-imaged frame by frame. Coordinate modifications due to this action are permanent and will affect the results of subsequent actions and analyses.

37.12. Analysis Commands

Analyses in `cpptraj` operate on data sets which have been generated by Actions in a prior Run or read in with a `readdata` command ([37.8.22 on page 727](#)). Unlike `ptraj`, Analysis commands in `cpptraj` do not need to be prefaced with `'analysis'`. The exception to this is `'analyze matrix'` in order to differentiate it from the `matrix` Action command; users are encouraged to use the new command `diagmatrix` instead.

Like Actions, when an Analysis command is issued it is by default added to the Analysis queue and is not executed until after trajectory processing is completed; a complete list of data sets available for analysis is shown after trajectory processing (prefaced by `'DATASETS'`) or can be shown with the `'list dataset'` command. Analyses can also be executed immediately via the `runanalysis` command ([37.8.27 on page 729](#)).

Note that for Analysis commands that use COORDS data sets, if no COORDS data set is specified then a default one will be automatically created from frames read in by *trajin* commands.

Command	Description	Set Type(s)
autocorr	Calculate autocorrelation function for multiple data sets.	N 1D scalar
avg	Calculate average, standard deviation, min, and max for (or over) data sets.	N 1D scalar
calcdiffusion	Calculate diffusion using multiple time origins.	COORDS
calcstate	Calculate states based on given data sets and criteria.	N 1D scalar
cluster	Perform cluster analysis.	COORDS, N 1D scalar
corr, correlationcoe	Calculate auto or cross correlation for 1 or 2 data sets.	1D scalar, vector
cphstats	Calculate statistics for constant pH data sets.	pH data sets
crank, crankshaft	Calculate crankshaft motion between two data sets.	2 1D scalar
crdfluct	Calculate atomic fluctuations (RMSF) for atoms over time blocks.	COORDS
crosscorr	Calculate a matrix of Pearson product-moment coefficients between given data sets.	N 1D scalar
curvefit	Perform non-linear curve fitting on given data set.	1D scalar
diagmatrix	Calculate eigenvectors and eigenvalues from given symmetric matrix.	symmetric matrix
divergence	Calculate Kullback-Leibler divergence between two data sets.	2 1D scalar
evalplateau	Evaluate whether the data in a 1D set has reached a single exponential plateau.	
FFT	Perform a fast Fourier transform on data sets.	N 1D scalar
hausdorff	Calculate the Hausdorff distance for given matrix data set(s).	N 2D matrices
hist, histogram	Calculate N-dimensional histogram for N given data sets.	N 1D scalar
integrate	Perform integration on each of the given data sets.	N 1D scalar
ired	Perform isotropic reorientational eigenmode dynamics analysis using given IRED vectors.	N IRED vectors
kde	Calculate 1D histogram from given data set using a kernel density estimator. Also time-dependent Kullback-Leibler divergence analysis with another set.	1 or 2 1D scalar

lifetime	Perform lifetime analysis on given data sets.	N 1D scalar
lowestcurve	For each given data set, calculate a curve that traces the lowest N points over specified bins.	N 1D scalar
meltcurve	Calculate a melting curve from given data sets assuming simple 2 state kinetics.	N 1D scalar
modes	Perform various analyses on eigenmodes (from e.g. <i>diagmatrix</i>).	eigenmodes
multicurve	Perform non-linear curve fitting for multiple input data sets.	N 1D scalar
multihist	Calculate 1D histograms (optionally with a kernel density estimator) from multiple input data sets.	N 1D scalar
phipsi	Calculate and plot the average phi and psi values from input dihedral data sets.	N phi/psi dihedrals
projectdata	Project data along eigenmodes.	eigenmodes, N 1D scalar
regress	Perform linear regression on multiple input data sets.	N 1D scalar
remlog	Calculate various statistics from a replica log data set.	replica log
rms2d, 2drms	Calculate 2D RMSD between frames in 1 or 2 COORDS data sets.	1 or 2 COORDS
rmsavgcorr	Calculate RMS average correlation curve for a COORDS data set.	COORDS
rotdif	Calculate rotational diffusion using given rotation matrices (from e.g. <i>rms</i>).	rotation matrices
runningavg	Calculate running average for given data sets using given window size.	N 1D scalar
spline	Calculate cubic splines for given data sets.	N 1D scalar
stat, statistics	Calculate various statistics for given data sets.	N 1D scalar
ti	Perform Gaussian quadrature integration for given DV/DL data sets.	N 1D scalar
tica	Perform time-independent correlation analysis.	COORDS, N 1D scalar
timecorr	Calculate auto/cross-correlation functions for given vector(s) using spherical harmonics.	1 or 2 vector
vectormath	Perform math on given vector data sets.	2 vector
wavelet	Perform wavelet analysis on coordinates from given COORDS set.	COORDS

37.12.1. autocorr

```
autocorr [name <dsetname>] <dsetarg0> [<dsetarg1> ...] [out <filename>]
        [lagmax <lag>] [nocovar] [direct]
<dsetarg0> [dsetarg1> ...] Argument(s) specifying datasets to be used.
[name <dsetname>] Store results in dataset(s) named <dsetname>:X.
[out <filename>] Write results to file named <filename>.
[lagmax] Maximum lag to calculate for. If not specified all frames
        are used.
[nocovar] Do not calculate covariance.
[direct] Do not use FFTs to calculate correlation; this will be much
        slower.
```

This is for integer/double/float datasets only; for vectors see the 'timecorr' command.

Calculate auto-correlation (actually auto-covariance by default) function for datasets specified by one or more dataset arguments. The datasets must have the same # of data points.

37.12.2. avg

```
avg <dset0> [<dset1> ...] [torsion] [out <file>] [oversets]
    [name <name>] [nostdout]
<dsetX> Data set(s) to calculate the average for.
[torsion] If the data sets are not already marked periodic (e.g. if
        read in via 'readdata'), treat them as periodic torsion.
[out <file>] File to write results to.
[oversets] If specified, calculate the average over all input sets
        instead of each input set.
[name <name>] Output data set name.
[nostdout] If 'nostdout' specified do not write averages to STDOUT
        when 'out' not specified.
DataSets Created (not oversets):
<name>[avg] Average of each set.
<name>[sd] Standard deviation of each set.
<name>[ymin] Y minimum of each set.
<name>[ymax] Y maximum of each set.
<name>[yminidx] Index of minimum Y value.
<name>[ymaxidx] Index of maximum Y value.
<name>[names] Name of each set.
DataSets Created (oversets)
<name> Average over all input sets for each frame.
<name>[SD] Standard deviation over all input sets for each frame.
```

Calculate the average, standard deviation, min, and max of given 1D data sets. Alternatively, if **oversets** is specified the average over each set for each point is calculated; this requires all input sets be the same size.

For example, to read in data from a file named perres.peptide.dat and calculate the averages etc for all the input sets:

```
readdata perres.peptide.dat
avg perres.peptide.dat out output.dat name V
```

37.12.3. calcdiffusion

```
calcdiffusion [crdset <coords set>] [maxlag <maxlag>] [<mask>] [time <dt>]
               [<name>] [out <file>] [diffout <file>]
```

[crdset <coords set>] Input COORDS set to calculate diffusion for.

[maxlag <maxlag>] Maximum lag to calculate diffusion for in frames.
Defaults to half the number of input frames if not specified.

[<mask>] Mask of atoms to calculate diffusion for.

[time <dt>] Time between frames in ps.

[<name>] Output MSD data sets name.

[out <file>] File to write MSD data sets to.

[diffout <file>] Write diffusion constants calculated from fits of MSD
data sets to <filename>.

DataSet Aspects:

[X] MSD(s) in the X direction.

[Y] MSD(s) in the Y direction.

[Z] MSD(s) in the Z direction.

[R] Overall MSD(s).

[A] Overall displacement(s) in Å.

[D] Diffusion constants (1×10^{-5} cm²/s).

[Label] Diffusion constant labels.

[Slope] Linear regression slopes.

[Intercept] Linear regression Y-intercepts.

[Corr] Linear regression correlation coefficients.

Calculate diffusion via mean-squared-displacements (MSD) of specified atoms. Unlike the *diffusion* Action ([37.11.24 on page 771](#)), which calculates MSD from a single time origin, the *calcdiffusion* command calculates diffusion from multiple time origins up to a user-specified lag (in frames). Note that no imaging is performed for this command, so any unwrapping should be performed prior to this command (see [37.11.90 on page 839](#)). Diffusion constants are calculated in the same manner as the *diffusion* command; for more details see [37.11.24 on page 771](#).

This command is both OpenMP- and MPI-parallelized; either or both can be active. In order to keep per-process memory requirements low, it is recommended that TRAJ (i.e. on-disk) data sets be used with MPI instead of CRD (in-memory) sets (see [37.7.11 on page 709](#)).

For example, to calculate diffusion from multiple time origins (with a maximum lag of half the number of trajectory frames) from an input trajectory in memory (unwrapping first):

```
parm tz2.ortho.parm7
loadcrd tz2.ortho.nc name TZ2
crdaction TZ2 unwrap bymol
runanalysis calcdiffusion crdset TZ2 out tz2.diff.dat :WAT@O
```

To do the same calculation using MPI parallelism, it will first be necessary to unwrap the trajectory (non-MPI parallel, but OpenMP parallelism can be used):

```
parm tz2.ortho.parm7
trajin tz2.ortho.nc
unwrap bymol
trajout unwrap.dcd
```

Then diffusion can be calculated using MPI (and/or OpenMP):

```
parm tz2.ortho.parm7
loadtraj unwrap.dcd name TZ2
runanalysis calcdiffusion crdset TZ2 out tz2.traj.diff.dat :WAT@O
```

37.12.4. calcstate

```
calcstate {state <ID>,<dataset>,<min>,<max>[,<dataset1>,<min1>,<max1>]} ...
[out <state v time file>] [name <setname>]
[curveout <curve file>] [stateout <states file>]
[transout <transitions file>] [countout <count file>]
```

state <ID>,<dataset>,<min>,<max> Define a state according to given data set and criteria. Multiple states can be given, and each state can have multiple criteria. If multiple criteria are specified, each one must be satisfied in order to assign the state. If the same state is defined multiple times, the state will be assigned if either criteria match.

<ID> Name to give each state index. State indices start at 0. -1 means ``undefined state``.

<dataset> Data set to use.

<min>,<max> Frames with data set value above <min> and below <max> will be assigned <ID>.

[out <state v time file>] File to write state index vs frame to.

[name <setname>] Data set name.

[curveout <curve file>] File to write state lifetime and transition curves to.

[stateout <states file>] File to write state lifetime data to.

[transout <transitions file>] File to write state transition data to.

[countout <state count file>] File to write state counts (i.e. how many frames each state was observed) to.

DataSets Created:

<setname> State index vs frame.

<setname>[Count] Number of frames each state was observed.

<setname>[Frac] Fraction of time each state was observed

<setname>[Nlifetimes] Number of times each state was reached.

<setname>[Avglife] Average lifetime length for each state.

<setname>[Maxlife] Maximum lifetime of each state.

<setname>[Name] Name (<ID>) of each state.

<setname>[Xlifetimes] Number of times each state transitioned to each other state.

<setname>[Xavglife] Average lifetime of each state before transitioning to each other state.

<setname>[Xmaxlife] Maximum lifetime of each state before transitioning to each other state.

<setname>[Xname] Name of each transition, format
 ``StateA->StateB``.

<setname>[sCurve]:X State curves; lifetime curve for transitions
 from given state to any other state.

<setname>[tCurve]:X Transition curves; lifetime curve for
 transitions from given state to other specific state.

Data for the specified data set(s) that matches the given criteria will be assigned a state index. State indices start from 0 and match the order in which **state** keywords were given. The -1 state index is reserved for “undefined state”. For example, the following input:

```
parm DPDP.parm7
trajin DPDP.nc
distance d1 :19@O :12@N
angle a1 :19@O :12@H :12@N
calcstate state D,d1,3.0,4.0 state A,a1,100,120 out state.dat curveout curve.agr \
stateout States.dat transout States.dat name d1_a1
run
```

Defines two states. State index 0 is defined as a state named “D” based on the distance from ‘:19@O’ to ‘:12@N’ (data set d1) being between 3 and 4 Angstroms. State index 1 is defined as a state named “A” based on the angle between ‘:19@O’, ‘:12@H’, and ‘:12@N’ (data set a1) being between 100 and 120 degrees. The output in state.dat might look like:

#Frame	d1_a1
1	-1
2	0
3	0
4	0
5	-1
6	1
7	-1
8	-1
9	0
10	-1

where the values in column d1_a1 refer to state index: -1 is undefined, 0 is state “D”, and 1 is state “A”.

To define a state State1 as having a distance named “dist” between 2.5 and 5.0 Ang. and an angle named “ang” between 30 and 60 degrees OR having a distance named “distA” between 0.0 and 3.0 Ang.:

```
calcstate state State1,dist,2.5,5.0,ang,30,60 \
state State1,distA,0.0,3.0
```

Lifetime curves (see [37.12.21 on page 872](#) for further explanation) are calculated for transitions from each state to any other state (aspect [sCurve]) and each state to each other state (aspect [tCurve]). In this case there will be 3 sCurves and 4 tCurves:

```
d1_a1[sCurve]:0 "Undefined" (double), size is 10
d1_a1[sCurve]:1 "D" (double), size is 3
d1_a1[sCurve]:2 "A" (double), size is 1
d1_a1[tCurve]:0 "Undefined->D" (double), size is 10
d1_a1[tCurve]:1 "D->Undefined" (double), size is 3
d1_a1[tCurve]:2 "Undefined->A" (double), size is 1
d1_a1[tCurve]:3 "A->Undefined" (double), size is 1
```

Lifetime analysis from each state to any other state is directed to the file specified by **stateout** and has format:

#Index N Average Max State

Where #Index is the state index, N is the number of lifetimes in that state, Average is the average lifetime while in that state (in frames), Max is the maximum lifetime while in that state (in frames) and State is the name of the state.

Finally, lifetime analysis of transitions from each state to each other state is directory to the file specified by transout and has format:

#N Average Max Transition

Where #N is the number of transitions, Average is the average lifetime (in frames) in the first state before transitioning to the second state, Max is the max lifetime (in frames) before transitioning to the second state, and Transition is the name of the transition.

37.12.5. cluster

```
cluster [crdset <crd set>] [data <dset0>[,<dset1>...]] [nocoords]
[<name>] [<Algorithm>] [<Metric>] [<Pairwise>] [<Sieve>] [<BestRep>]
[<Output>] [<Coord. Output>] [<Graph>]
  [readinfo {infofile <info file> | cnvtset <dataset>}]
  [useframesincache]
  Algorithm Args: [{hieragglo|dbscan|kmeans|dpeaks}]
  [hieragglo [epsilon <e>] [clusters <n>] [linkage|averagelinkage|complete]
    [epsilonplot <file>]]
  [dbscan minpoints <n> epsilon <e> [kdist <k> [kfile <prefix>]]]
  [kmeans clusters <n> [randompoint [kseed <seed>]] [maxit <iterations>]]
  [dpeaks epsilon <e> [noise] [dvdf file <density vs dist file>]
    [choosepoints {manual | auto}]
    [distancecut <distcut>] [densitycut <densitycut>]
    [runavg <runavg file>] [deltafile <file>] [gauss]]
  Metric Args:
  [{dme|rms|srmsd|qrmsd} [mass] [nofit] [<mask>]] [{euclid|manhattan}] [wgt <list>]
  Pairwise Args:
  [pairdist <name> [pairdistfile <file>]] [pwrecalc]
  [loadpairdist] [savepairdist] [pairwisecache {mem|disk|none}]
  Sieve Args:
  [sieve <#> [sieveseed <#>] [random] [includesieveincalc] [includesieved.cdist]
    [{sieveto frame|sieveto centroid|closest centroid}] [repsilon <restore epsilon>]]
  BestRep Args:
  [bestrep {cumulative|centroid|cumulative_nosieve}] [savenreps <#>]
  Output Args:
  [out <cnumvtime> [gracecolor]] [noinfo|info <file>] [summary <file>]
  [summarysplit <splitfile>] [splitframe <comma-separated frame list>]
  [clustersvtime <file> [cvtwindow <#>]] [sil <prefix> [silidx {idx|frm}]]
  [metricstats <file>] [cpopvtime <file> [{normpop|normframe}]] [lifetime]
  Coordinate Output Args:
  [clusterout <trajfileprefix> [clusterfmt <trajformat>]]
  [singlerepout <trajfilename> [singlerepfmt <trajformat>]]
  [repout <repprefix> [repfmt <trajformat>] [repframe]]
  [avgout <avgprefix> [avgfmt <trajformat>]]
  [assignrefs [refcut <rms>] [refmask <mask>]]
  Graph Args:
  [{drawgraph|drawgraph3d} [drawtol <tolerance>] [drawmaxit <iterations>]]
```


- [**crdset** <crd set>] Name of COORDS data set to cluster on and/or use for coordinate output. If not specified the default COORDS set will be generated and used unless **nocoords** has been specified.
- [**data** <dset0>[,<dset1>,...]] Distance between frames calculated using specified data set(s). Currently 1D scalar sets and COORDS sets are supported.
- [**nocoords**] Do not use a COORDS data set; distance metrics that require coordinates and coordinate output will be disabled.
- [<name>] Data set Name for generated cluster data sets.
- [**readinfo**] Use previous cluster results to set up initial clusters. Clustering will continue if possible (i.e. this can be used to restart clustering).
- infofile** <file> Cluster info file to read clusters from.
- cvtset** <dataset> Cluster number vs time data set to use to generate initial clusters.
- [**useframesincache**] If a pairwise cache is specified, cluster on the frames stored in the cache.

Algorithms:

- hieragglo** (Default) Use hierarchical agglomerative (bottom-up) approach.
- [**epsilon** <e>] Finish clustering when minimum distance between clusters is greater than <e>.
- [**clusters** <n>] Finish clustering when <n> clusters remain.
- [**linkage**] Single-linkage; use the shortest distance between members of two clusters.
- [**averagelinkage**] Average-linkage (default); use the average distance between members of two clusters.
- [**complete**] Complete-linkage; use the maximum distance between members of two clusters.
- [**epsilonplot** <file>] Write number of clusters vs epsilon to <file>.
- dbscan** Use DBSCAN clustering algorithm of Ester et al.[792]
- minpoints** <n> Minimum number of points required to form a cluster.
- epsilon** <e> Distance cutoff between points for forming a cluster.
- [**kdist** <k>] Generate K-dist plot for help in determining DBSCAN parameters (see below).
- [**kfile** <prefix>] Prefix for K-dist plot file.
- dpeaks** Use the density peaks algorithm of Rodriguez and Laio[793]
- epsilon** <e> Cutoff for determining local density in Angstroms.
- [**noise**] If specified, treat all points within epsilon of another cluster as noise.
- [**dvdfile** <density_vs_dist_file>] File to write density versus minimum distance to point with next highest density. This can be used to determine appropriate cutoffs for distance and density in a subsequent step with **choosepoints** manual.

[choosepoints {manual | auto}] Specify whether clusters will be chosen based on specified distance/density cutoffs, or automatically. If not specified only the density vs distance file will be written and no clustering will be performed. Currently manual is recommended.

[distancecut <distcut>] [densitycut <densitycut>] If choosepoints manual, points with minimum distance greater than or equal to <distcut> and density greater than or equal to <densitycut> will be chosen.

[runavg <runavg file>] If choosepoints automatic, the calculated running average of density versus distance will be written to <runavg file>.

[deltafile <file>] If choosepoints automatic, distance minus the running average for each point will be written to this file.

[gauss] Calculate density with Gaussian kernels instead of using discrete density.

kmeans Use K-means clustering algorithm.

clusters <n> Finish clustering when number of clusters is <n>.

[randompoint] Randomize initial set of points used (recommended).

[kseed <seed>] Random number generator seed for randompoint.

[maxit <iteration>] Algorithm will run until frames no longer change clusters of <iteration> iterations are reached (default 100).

Distance Metric Options:

[{rms | srmsd} [<mask>]] (Default rms) For COORDS data, distance between coordinate frames calculated via best-fit coordinate RMSD using atoms in <mask>. If srmsd specified use symmetry-corrected RMSD (see [37.11.84 on page 836](#)).

[mass] Mass-weight the RMSD.

[nofit] Do not fit structures onto each other prior to calculating RMSD.

qrmsd [<mask>] For COORDS data, distance between coordinate frames calculated using best-fit quaternion RMSD (can be 15-20% faster than regular RMSD) using atoms in <mask>.

[mass] Mass-weight the RMSD.

dme [<mask>] For COORDS data, distance between coordinate frames calculated using distance-RMSD (aka DME, *distrmsd*) using atoms in <mask>.

euclid Use Euclidean distance ($\sqrt{\text{SUM}(\text{distance}^2)}$) when more than one data set has been specified (default).

manhatttan Use Manhattan distance ($\text{SUM}(\text{distance})$) when more than one data set has been specified.

wgt <list> Factor to multiply distances from each metric by in a comma-separated list. Can be used to adjust the contribution from each metric. Default is 1 for each metric. Output from the metricstats keyword can be used to determine the relative contribution of each metric to the distance.

Pairwise Distance Matrix Options:

[pairdist <name>] Pairwise cache DataSet/File name to use for loading/saving pairwise distances.

[pairdistfile <file>] File name to use for pairwise cache; if not specified and 'pairdist' specified, uses 'pairdist'.

[pwrecalc] If the loaded pairwise distance matrix does not match the current setup, force recalculation.

[loadpairdist] Load pairwise distances from file specified by pairdist (CpptrajPairDist if pairdist not specified).

[savepairdist] Save pairwise distances to file specified by pairdist (CpptrajPairDist if pairdist not specified). NOTE: If sieving was performed only the calculated distances are saved.

[pairwisecache {mem | disk | none}] Cache pairwise distance data in memory (default), to disk, or disable pairwise caching. No caching will save memory but be extremely slow. Caching to disk will likely be slow unless writing to a fast storage device (e.g. SSD) - data is saved to a file named 'CpptrajPairwiseCache'.

Sieving Options:

[sieve <#>] Perform clustering only for every <#> frame. After clustering, all other frames will be added to clusters.

[random] When sieve is specified, select initial frames to cluster randomly.

[sieve seed <#>] Seed for random sieving; if not set the wallclock time will be used.

[includesieved cdist] Include sieved frames in final cluster distance calculation (may be very slow).

[includesieveincalc] Include sieved frames when calculating within-cluster average (may be very slow).

[sieveto frame] When restoring sieved frames, compare frame to every frame in a cluster using a cutoff of <restore epsilon> (default is algorithm epsilon when using DPeaks/DBscan) instead of the centroid; slower but more accurate.

[sieveto centroid] When restoring sieved frames, compare frame to cluster centroid using a cutoff of <restore epsilon> (default is algorithm epsilon when using DPeaks/DBscan). Default method for DPeaks/DBSCAN.

[closest centroid] When restoring sieved frames, add each frame to its closest centroid. Default method for hieragglo/kmeans.

[repsilon <restore epsilon>] Epsilon to use for sieveto frame/sieveto centroid (default is algorithm epsilon when using DPeaks/DBscan).

Best Representative Options:

[bestrep {cumulative|centroid|cumulative_nosieve}] Method for choosing cluster representative frames.

cumulative Choose by lowest cumulative distance to all other frames in cluster. Default when not sieving.

centroid Choose by lowest distance to cluster centroid. Default when sieving.

cumulative_nosieve Choose by lowest cumulative distance to all other frames, ignoring sieved frames.

[savenreps <#>] Number of best representative frames to choose (default 1).

Output Options:

[out <cnumvtime>] Write cluster # vs frame to <cnumvtime>.
Algorithms that calculate noise (e.g. DBSCAN) will assign noise points a value of -1.

[gracecolor] Instead of cluster # vs frame, write cluster# + 1 (corresponding to colors used by XMGRACE) vs frame. Cluster #s larger than 15 are given the same color. Algorithms that calculate noise (e.g. DBSCAN) will assign noise points a color of 0 (blank).

[summary <summaryfile>] Summarize each cluster with format '#Cluster Frames Frac AvgDist Stdev Centroid AvgCDist':
#Cluster Cluster number starting from 0 (0 is most populated).
Frames # of frames in cluster.
Frac Size of cluster as fraction of total trajectory.
AvgDist Average distance between points in the cluster.
Stdev Standard deviation of points in the cluster.
Centroid Frame # of structure in cluster that has the lowest cumulative distance to every other point. If multiple representatives are being saved this column is replaced with two columns for each representative, 'Rep' (representative frame #) and 'RepScore' (score according to current best representative metric).
AvgCDist Average distance of this cluster to every other cluster.

[info <infofile>] Write ptraj-like cluster information to <infofile>.
This file has format:
#Clustering: <X> clusters <N> frames
#Cluster <I> has average-distance-to-centroid <AVG>
...
#DBI: <DBI>
#pSF: <PSF>
#SSR/SST: <SSR/SST>
#Algorithm: <algorithm-specific info>
<Line for cluster 0>
...
#Representative frames: <representative frame list>
Where <X> is the number of clusters, <N> is the number of frames clustered, <I> ranges from 0 to <X>-1, <AVG> is the average distance of all frames in that cluster to the centroid, <DBI> is the Davies-Bouldin Index, <pSF> is the pseudo-F statistic, <SSR/SST> is the SSR/SST ratio, and <representative frame list> contains the frame # of the representative frame (i.e. closest to the centroid) for each cluster. Each cluster has a line made up of characters (one for each frame) where '.' means 'not in cluster' and 'X' means 'in cluster'.

[noinfo] Suppress printing of cluster info.

[summarysplit <splitfile>] Summarize each cluster based on which of its frames fall in portions of the trajectory specified by splitframe with format '#Cluster Total Frac C# Color NumInX ... FracX ... FirstX ... RepX':

- #Cluster** Cluster number starting from 0 (0 is most populated).
- Total #** of frames in cluster.
- Frac** Size of cluster as a fraction of the total trajectory.
- C#** Grace color number.
- Color** Text description of the color (based on standard XMGRACE coloring).
- NumInX** Number of frames in Xth portion of the trajectory.
- FracX** Fraction of frames in Xth portion of the trajectory.
- FirstX** Frame in the Xth portion of the trajectory where the cluster is first observed.
- RepX** Best representative frame in the Xth portion of the trajectory for that cluster.

[splitframe <frame>] For summarysplit, frame or comma-separated list of frames to split the trajectory at, e.g. '100,200,300'.

[clustersvtime <filename>] Write number of unique clusters observed in a given time window to <filename>.

[cvtwindow <>windowsize>] Window size for clustersvtime output.

[sil <prefix>] Write average cluster silhouette value for each cluster to '<prefix>.cluster.dat' and cluster silhouette value for each individual frame to '<prefix>.frame.dat'.

[silidx {idx|frm}] Choose what indices to write to the cluster silhouette frame file: idx (the default) specifies the sorted index (starting from 0), frm specifies the actual frame number.

[metricstats <file>] When more than one metric in use, print the fraction contribution of each metric to the total distance. This information can be used in conjunction with the wgt keyword to adjust the contribution of each metric to the total distance. It is written to <file> with format:

```
#Metric FracAv FracSD Avg SD Min Max Description
```

Where #Metric is the metric number, FracAv and FracSD are the average and standard deviation of the fraction contribution of that metric to the total distance (taking into account distance type and weights), Avg, SD, Min, and Max are the average, standard deviation, minimum, and maximum of the unmodified distance contribution from that metric, and Description is the metric description. This may be slow for large numbers of frames, so it is advisable to run this on a smaller (potentially sieved) number of frames.

[cpopvtime <file> [normpop | normframe]] Write cluster population vs time to <file>; if normpop specified normalize each cluster to 1.0; if normframe specified normalize cluster populations by number of frames.

[lifetime] Create a DataSet with aspect [Lifetime] for each cluster; for each frame, have 1 if the cluster is present and 0

otherwise. Can be used with *lifetime* analysis ([37.12.21 on page 872](#)).

Coordinate Output Options:

clusterout <trajfileprefix> Write frames in each cluster to files named <trajfileprefix>.cX, where X is the cluster number.

clusterfmt <trajformat> Format keyword for clusterout (default Amber Trajectory).

singlerepout <trajfilename> Write all representative frames to single trajectory named <trajfilename>.

singlerepfmt <trajformat> Format keyword for singlerepout (default Amber Trajectory).

repout <repprefix> Write representative frames to separate files named <repprefix>.X.<ext>, where X is the cluster number and <ext> is a format-specific filename extension.

repfmt <trajformat> Format keyword for repout (default Amber Trajectory).

repframe Include representative frame number in repout filename.

avgout <avgprefix> Write average structure for each cluster to separate files named <avgprefix>.X.<ext>, where X is the cluster number and <ext> is a format-specific filename extension.

avgfmt <trajformat> Format keyword for avgout.

assignrefs In summary/summarysplit, assign clusters to loaded reference structures if RMSD to that reference is less than specified cutoff. This will be printed in summary and summarysplit files as 2 extra columns: 'Name' (reference name) and 'RMS' (RMS to cluster centroid).

[refcut <rms>] RMSD cutoff in Angstroms.

[refmask <mask>] Mask to use for RMSD calculation. If not specified the default mask is all heavy atoms.

DataSets Created:

<name> Cluster number vs time (color number if gracecolor specified).

<name>[DBI] Hold final Davies-Bouldin index.

<name>[PSF] Hold final pseudo-F value.

<name>[SSRSST] Hold final SSR/SST value.

<name>[NCVT] (clustersvtime only). Number of unique clusters observed over time.

<name>[Pop]:<X> Cluster X population vs time; index X corresponds to cluster number.

<name>[Lifetime]:<X> (lifetime only). For each cluster X, contain 1 if cluster present that frame, 0 otherwise.

Note cluster population vs time data sets are not generated until the analysis has been run.

Cluster input frames using the specified input data sets (can be any combination of coordinates/COORDS and/or 1D scalar data) with the specified clustering algorithm. For COORDS sets, the distance metric can be RMSD, symmetry-corrected RMSD, or DME. When multiple data sets are present, the total distance can be determined either via the Euclidean (default) or Manhattan method.

In order to speed up clustering of large trajectories, the **sieve** keyword can be used. In addition, subsequent clustering calculations can be sped up by writing/reading calculated pair distances between each frame to/from a file specified by **pairedist** (or “CpptrajPairDist” if **pairedist** not specified).

Example: cluster on a specific distance:

```
distance endToEnd :1 :255
cluster data endToEnd clusters 10 epsilon 3.0 summary summary.dat info info.dat
```

Example: two clustering commands on the CA atoms of residues 2-10 using average-linkage, stopping when either 3 clusters are reached or the minimum distance between clusters is 4.0 for the first, and 8 clusters or minimum distance 2.0 for the second. The first command will write the cluster number vs time to “cnumvtime.dat” and a summary of each cluster to “avg.summary.dat”. The second clustering command will use the pairwise distance matrix from the first to speed things up:

```
cluster C1 :2-10 clusters 3 epsilon 4.0 info C1.info out cnumvtime.dat summary avg.s
cluster C2 :2-10 clusters 8 epsilon 2.0 info C2.info pairedist PW
```

Clustering Success Metrics

The Davies-Bouldin Index (DBI, reported in the **info** file) measures sum over all clusters of the within cluster scatter to the between cluster separation; **the smaller the DBI, the better**. The DBI is defined as the average, for all clusters X, of $\text{fred}(X) = \max, \text{across other clusters } Y, \text{ of } (C_x + C_y)/d_{XY}$. Here C_x is the average distance from points in X to the centroid, similarly C_y , and d_{XY} is the distance between cluster centroids.

The pseudo-F statistic (pSF, reported in the **info** file) is another measure of clustering goodness. It is intended to capture the ‘tightness’ of clusters, and is in essence a ratio of the mean sum of squares between groups to the mean sum of squares within group. **Higher values of pseudo-F are good**. Generally, one selects a cluster-count that gives a peak in the pseudo-f statistic. Formula: A/B , where $A = (T - P)/(G - 1)$, and $B = P / (n - G)$. Here n is the number of points, G is the number of clusters, T is the total distance from the all-data centroid, and P is the sum (for all clusters) of the distances from the cluster centroid.

The SSR/SST (reported in the **info** file) is the ratio of the sum of squares regression (SSR or between sum of squares) and the total sum of squares (SST). The SSR is calculated via the sum of the squared distances of all points within a given cluster to its centroid, and summed together for all clusters. The total sum of squares is the sum of squared distances for all frames to the overall mean. The ratio lies between 0 and 1 and is supposed to give the fraction of explained variance by the data. The ratio should increase with cluster count. **There should be a point at which adding more clusters does not substantially increase SSR/SST**, i.e. the point where increasing the cluster count does not add new information and should not increase further.

The cluster silhouette (**sil/silidx** keywords) is a measure of how well each point fits within a cluster. Values of 1 indicate the point is very similar to other points in the cluster, i.e. it is well-clustered. Values of -1 indicate the point is dissimilar and may fit better in a neighboring cluster. Values of 0 indicate the point is on a border between two clusters. The **sil** <prefix> keyword will write two files. The first, <prefix>.cluster.dat, which has the format:

```
#Cluster <Si> StdDev
```

Where #Cluster is the cluster number, <Si> is the average silhouette value for all frames in the cluster, and StdDev is the standard deviation of the silhouette value for all frames in the cluster. The second, <prefix>.frame.dat, will contain the silhouette value for each frame grouped by cluster, with indices controlled by the **silidx** keyword (default sorted by ascending silhouette value), e.g.:

```
#C0 Silhouette
#C0      Silhouette
      0 -0.135988
      1 -0.0266746
      2 -0.0167628
```

```

3 0.0609673
4 0.0649603
5 0.0835595
#C1      Silhouette
6 0.319039
7 0.319785
8 0.348833
9 0.358286
0 0.1376
9 0.1376

```

The last two lines will contain the overall average silhouette value twice, one at the lowest index and one at the highest. The file is formatted in this way to make it easy to visualize each cluster silhouette relative to the average value in e.g. the XMGRACE plotting program. If the clustering results in one or more cluster with silhouette values completely below the average line, the clustering is likely poor.

Hints for setting DBSCAN parameters with 'kdist'

It is not always obvious what parameters to set for DBSCAN. You can get a rough idea of what to set 'mindist' and 'epsilon' to by generating a so-called "K-dist" plot with the 'kdist <k>' option. The K-dist plot shows for each point (X axis) the Kth farthest distance (Y axis), sorted by decreasing distance. You supply the same distance metric and sieve parameters you want to use for the actual clustering, but nothing else. For example:

```
cluster C0 dbscan kdist 4 rms :1-4@CA sieve 10 loadpairdist pairdist CpptrajPairDist
```

The K-dist plot will be named <prefix>.<k>.dat, with the default prefix being 'Kdist' (in this case the file name would be Kdist.4.dat). The K-dist plot usually looks like a curve with an initially steep slope that gradually decreases. Around where the initial part of the curve starts to flatten out (indicating an increase in density) is around where epsilon should be set; minpoints is set to whatever <k> was. It has been suggested that the shape of the K-dist curve doesn't change too much after Kdist=4, but users are encouraged to experiment.

Using 'dpeaks' clustering

The 'dpeaks' (density peaks) algorithm attempts to find clusters by identifying points in high density regions which are far from other points of high density[793]. There are two ways these points can be chosen. The first and recommended way is manually. In this method, clustering is first run with **choosepoints** not specified to generate a plot containing density versus minimum distance to point with next highest density (the decision graph). Appropriate cut offs for distance and density can then be chosen based on visual inspection; cutoffs should be chosen so that they select points that have both a high density and a high distance to point with next highest density. Clustering can then be run again with **distancecut** and **densitycut** set.

The second way is automatically; *cpptraj* will attempt to identify outliers in the density vs distance plot based on distance from the running average. Although this only requires a single pass, this method of choosing points is not well-tested and currently not recommended.

The Binary pairwise matrix file format

When NetCDF is not present, the pairwise matrix file will be written in binary. The exact format depends on what version of *cpptraj* generated the file (since earlier versions had no concept of 'sieve'). The *CpptrajPairDist* file starts with a 4 byte header containing the characters 'C' 'T' 'M' followed by the version number. A quick way to figure out the version is to use the linux 'od' command to output the first 4 bytes as hexadecimal, e.g.:

```
$ od -t x1 -N 4 CpptrajPairDist 0000000 43 54 4d 02
```

So the *CpptrajPairDist* file version in the above example is 2.

The next few numbers describe the matrix size and depend on the version.

Version 0: Two 4-byte integers: # of rows and # of elements.

Version 1: Two 8-byte unsigned integers (equivalent to size_t on most systems): # of rows and # of elements.

Version 2: Three 8 byte unsigned integers: original # of rows, actual # of rows, and sieve value.

This is followed by the actual matrix data, stored as a single array of floats (4 bytes). For versions 1 and 2 the number of elements is explicitly stored. For version 2, to calculate the number of matrix elements you need to read:

$$\text{Elements} = (\text{actual_rows} * (\text{actual_rows} - 1)) / 2$$

The cluster pair-distance matrix is an upper-right triangle matrix without the diagonal (in row-major order), so the first element is the distance between elements 0 and 1, the second is between elements 0 and 2, etc.

In version 2 files, if the sieve value is greater than 1 that means original_rows > actual_rows and there is an additional array of characters original_nrows long, with 'T' if the row is being ignored (i.e. it was sieved out) and 'F' if the row is active (i.e. is active in the actual pairwise-distance matrix).

The code that cpptraj uses to read in CpptrajPairDist files is in ClusterMatrix::LoadFile() (ClusterMatrix.cpp).

The NetCDF pairwise matrix format

The default way to write pairwise matrix files as of version 6.0.0 is with NetCDF. This will be set up with the following parameters:

Attributes:

Conventions ``CPPTRAJ\_CMATRIX``

Version <version string>

MetricDescription <description of the distance metric used to create matrix>

Dimensions:

n.original_frames Number of frames originally in the set (i.e. before sieving).

n.rows Number of rows in the upper-triangle pairwise matrix.

msize Actual size of the matrix; should be $(nRows - 1) * (nRows - 1) / 2$;

Variables:

sieve (integer, no dimension). Sieve value.

matrix (float, dimension msize). The pairwise matrix, flattened to 1 dimension. Index calc is: $i1 = i + 1$; $index = ((nRows - 1) * i - ((i1 * i) / 2)) + j - i1$;

actual frames (integer, dimension n.rows). The actual frame numbers for which pairwise distances were calculated.

37.12.6. cphstats

```
cphstats <pH sets> [name <name>] [statsout <statsfile>] [deprot]
        [fracplot [fracplotout <file>]]
```

<pH sets> Previously read in pH data sets.

name <name> Output set name.

statsout <statsfile> Write pH statistics to <statsfile>

deprot If specified, calculate fraction deprotonated instead of protonated.

fracplot If specified, calculate fraction protonated/deprotonated vs pH.

fracplotout <file> File to write fraction plots to.

Data Sets Generated

<name>[Frac]:<idx> Fraction protonated/deprotonated for residue <idx>.

Calculate statistics for constant pH simulation data previously read in with *readdata* (see [37.6.11 on page 702](#)). Statistics are calculated for each residue at each input pH. Output format is as follows:

```
Solvent pH is <pH>
<res name> <res num> : Offset <off> Pred <pred> Frac Prot <frac> Transitions <#tr>
...
Average total molecular protonation: <avg>
```

Where <off> is offset from predicted, <pred> is predicted pH, and <#trans> is the number of transitions. A line is printed for each residue. This functionality is similar to the *cphstats* utility that comes with Amber (see [27.7 on page 602](#)).

Note that data from constant pH REMD must be sorted prior to use with *cphstats*. See the *readensembledata* ([37.8.23 on page 727](#)) and *sortensembledata* ([37.8.30 on page 729](#)) commands for more details.

For example, to read in constant pH data from constant pH REMD, sort and analyze:

```
readensembledata ExplicitRemd/cpout.001 cpin ExplicitRemd/cpin name PH
sortensembledata PH
runanalysis cphstats PH[*] statsout stats.dat fracplot fracplotout frac.agr deprot
```

37.12.7. corr | correlationcoe

```
corr out <outfilename> <dataset1> [<dataset2>]
    [lagmax <lag>] [nocovar] [direct]

out <outfilename> Write results to file named <outfilename>. The
    datasets must have the same # of data points.

<dataset1> [<dataset2>] Data set(s) to calculate correlation for. If
    one dataset or the same dataset is given twice, the
    auto-correlation will be calculated, otherwise
    cross-correlation.

[lagmax] Maximum lag to calculate for. If not specified all frames
    are used.

[nocovar] Do not calculate covariance.

[direct] Do not use FFTs to calculate correlation; this will be much
    slower.

DataSet Aspects:
[<dataset1>] (Auto-correlation) The aspect will be the name of each
    of the input data set.
[<dataset1>-<dataset2>] (Cross-correlation) The aspect will be the
    names of each of the input data sets joined by a dash ('-').

DataSet Aspects:
```

[coeff] Correlation coefficient.

Calculate the auto-correlation function for data set named <dataset1> or the cross-correlation function for data sets named <dataset1> and <dataset2> up to <lagmax> frames (all if **lagmax** not specified), writing the result to file specified by **out**. The two datasets must have the same # of datapoints.

37.12.8. crank | crankshaft

```
crank {angle | distance} <dsetname1> <dsetname2> info <string>
      [out <filename>] [results <resultsfile>]
```

angle Analyze angle data sets.

distance Analyze distance data sets.

<dsetname1> Data set to analyze.

<dsetname2> Data set to analyze.

info <string> Title the analysis <string>.

[out <filename>] Write frame-vs-bin to <filename>.

[results <resultsfile>] Write results to <resultsfile>.

Calculate crankshaft motion between two data sets.

37.12.9. crdfloct

```
[crdset <crd set>] [<mask>] [out <filename>] [window <size>] [bfactor]
```

Calculate atomic positional fluctuations for atoms in <mask> over windows of size <size>. If **bfactor** is specified, the fluctuations are weighted by $\frac{8}{3}\pi^2$ (similar but not necessarily equivalent to crystallographic B-factor calculation). Units are Å, or Å² × $\frac{8}{3}\pi^2$ if **bfactor** specified.

37.12.10. crosscorr

```
crosscorr [name <dsetname>] <dsetarg0> [<dsetarg1> ...] [out <filename>]
```

[name <dsetname>] The resulting upper-triangle matrix is stored with name <dsetname>.

<dsetarg0> [<dsetarg1> ...] Argument(s) specifying datasets to be used.

[out <filename>] Write results to file named <filename>.

Calculate the Pearson product-moment correlation coefficients between all specified datasets.

37.12.11. curvefit

```
curvefit <dset> { <equation> |
                  name <dsname> {gauss | nexp <m> [form {mexp|mexpk|mexpk.penalty}}
                  [AX=<value> ...] [out <outfile>] [resultsout <results>]
                  [maxit <max iterations>] [tol <tolerance>]
                  [outxbins <NX> outxmin <xmin> outxmax <xmax>]
```

<dset> Data set to fit.

<equation> Equation to fit of form **<Variable> = <Equation>**. See [37.5.2 on page 694](#) for more details on equations *cpptraj* understands.

name <dsname> Final data set name (required if using *nexp* or *gauss*).

gauss Fit to Gaussian of form $A0 * \exp(-(X - A1)^2 / (2 * A2^2))$

nexp <m> Fit to specified number of exponentials.

form <type> Fit to specified exponential form:

mexp Multi-exponential, $SUM(m) [An * \exp(An+1 * X)]$

mexpk Multi-exponential plus constant, $A0 + SUM(m) [An * \exp(An+1 * X)]$

mexpk_penalty Same as *mexpk* except sum of prefactors constrained to 1.0 and exponential constants constrained to < 0.0 .

AX=<value> Value of any constants in specified equation with X starting from 0 (can specify more than one).

out <outfile> Write resulting fit curve to **<outfile>**.

resultsout <results> Write details of the fit to **<results>** (default *STDOUT*).

maxit <max iterations> Number of iterations to run curve fitting algorithm (default 50).

tol <tolerance> Curve-fitting tolerance (default $1E-4$).

outxbins <NX> Number of points to use when generating final curve (default same number of points as input data set).

outxmin <xmin> Minimum X value to use for final curve (default same number of points as input data set).

outxmax <xmax> Maximum X value to use for final curve (default same number of points as input data set).

Perform non-linear curve fitting for the specified data set using the Levenberg-Marquardt algorithm. Any equation form that *cpptraj* understands (see [37.5.2 on page 694](#)) can be used, or several preset forms can be used. Similar to Grace (<http://plasma-gate.weizmann.ac.il/Grace/>), an equation can contain constants for curve fitting termed AX (with X being a numerical digit, one for each constant), and is assigned to a variable which then becomes a data set. For example, to fit a curve to data from a file named *Data.dat* to a data set named 'FitY':

```
readdata Data.dat
runanalysis curvefit Data.dat \
  "FitY = (A0 * exp(X * A1)) + (A2 * exp(X * A3))" \
  A0=1 A1=-1 A2=1 A3=-1 \
  out curve.dat tol 0.0001 maxit 50
```

To perform the same fit but to a multi-exponential curve with two exponentials:

```
readdata Data.dat
runanalysis curvefit Data.dat nexp 2 name FitY \
  A0=1 A1=-1 A2=1 A3=-1 \
  out curve1.dat tol 0.0001 maxit 50
```

37.12.12. diagmatrix

```
diagmatrix <matrix name> [out <filename>] [thermo [outthermo <filename>]]
[vecs <#>] [name <modesname>] [reduce]
[nmwiz [nmwizvecs <#>] [nmwizfile <filename>]]
```

<matrix name> Name of symmetric matrix to diagonalize.

[out <filename>] Write results to <filename>.

[thermo [outthermo <filename>]] Mass-weighted covariance (mwcovar) matrix only. Calculate entropy, heat capacity, and internal energy from the structure of a molecule (average coordinates, see above) and its vibrational frequencies using standard statistical mechanical formulas for an ideal gas. Results are written to <filename> if specified, otherwise results are written to STDOUT. Note that this converts the units of the calculated eigenvalues to frequencies (cm^{-1}).

[vecs <#>] Number of eigenvectors to calculate. Default is 0, which is only allowed when 'thermo' is specified.

[name <modesname>] Store resulting modes data set with name <modesname>.

[reduce] Covariance (covar/mwcovar/distcovar) matrices only. For coordinate covariance (covar/mwcovar) matrices, each eigenvector element is reduced via $E_i = E_{ix}^2 + E_{iy}^2 + E_{iz}^2$. For distance covariance (distcovar) the eigenvectors are reduced by taking the sum of the squares of each row. See Abseher & Nilges, JMB 1998, 279, 911-920 for further details. They may be used to compare results from PCA in distance space with those from PCA in cartesian-coordinate space.

[nmwiz] Generate output in .nmd format file for viewing with NMWiz[794]. See http://prody.csb.pitt.edu/tutorials/nmwiz_tutorial/ for further details.

[nmwizvecs <#>] Number of vectors to write out for nmwiz output, starting with the lowest frequency mode (default 20).

[nmwizfile <filename>] Name of nmwiz file to write to (default 'out.nmd').

[nmwizmask <mask>] Mask of atoms corresponding to eigenvectors - should be the same one used to generate the matrix.

Calculate eigenvectors and eigenvalues for the specified symmetric matrix. This is followed by Principal Component Analysis (in cartesian coordinate space in the case of a covariance matrix or in distance space in the case of a distance-covariance matrix), or Quasiharmonic Analysis (in the case of a mass-weighted covariance matrix). Diagonalization of distance, correlation, idea, and ired matrices are also possible. Eigenvalues are given in cm^{-1} in the case of a mass-weighted covariance matrix and in the units of the matrix elements in all other cases. In the case of a mass-weighted covariance matrix, the eigenvectors are mass-weighted.

For quasi-harmonic analysis the input must be a mass-weighted covariance matrix. Thermodynamic quantities are calculated based on statistical mechanical formulae that assume the input system is oscillating in a single energy well: see Statistical Thermodynamics by D. A. McQuarrie, particularly chapters 4, 5, and 6 for more details.[795] For an in-depth discussion of the accuracy of thermodynamic parameters obtained via quasi-harmonic analysis see Chang et al..[796]

Note that the maximum number of non-zero eigenvalues obtainable depends on the number of frames used to generate the input matrix; the number of frames should be equal to or greater than the number of columns in the matrix in order to obtain all eigenmodes.

Results may include average coordinates (in the case of covar, mwcovar, correl), average distances (in the case of distcovar), main diagonal elements (in the case of idea and ired), eigenvalues, and eigenvectors.

For example, in the following a mass-weighted covariance matrix of all atoms is generated and stored internally with the name mwcvmat; the matrix itself is written to mwcvmat.dat. Subsequently, the first 20 eigenmodes of the matrix are calculated and written to evecs.dat, and quasi-harmonic analysis is performed at 300.0 K, with the results written to thermo.dat.

```
matrix mwcovar name mwcvmat out mwcvmat.dat
diagramatrix mwcvmat out evecs.dat vecs 20 \
    thermo outthermo thermo.dat temp 300.0
```

Output Format

The “modes” or “evecs” output file is a text file with the following format:

```
[Reduced] Eigenvector file: <Type> nmodes <#> width <width>
  <# Avg Coords> <Eigenvector Size>
<Average Coordinates>
```

Where <Type> is a string identifying what kind of matrix the eigenvectors/eigenvalues were determined from, nmodes is how many eigenvectors are in the file, and <Average Coordinates> are in lines 7 columns wide, with each element having width specified by <width>. Then for each eigenvector:

```
****
<Eigenvector#> <Eigenvalue>
<Eigenvector Coordinates>
...
```

Where <Eigenvector Coordinates> are in lines 7 columns wide, with each element having width specified by <width>.

37.12.13. divergence

```
divergence ds1 <ds1> ds2 <ds2>
```

Calculate Kullback-Leibler divergence between specified data sets.

37.12.14. evalplateau

```
evalplateau [name <set out name>] [tol <tol>] [valacut <valacut>]
    [initpct <initial pct>] [finalpct <final pct>]
    [chisqcut <chisqcut>] [slopecut <slopecut>] [maxit <maxit>]
    [out <outfile>] [resultsout <resultsfile>] [statsout <statsfile>]
    <input set args> ...
```

name <set out name>] Name for output data sets.

tol <tol> Curve fitting tolerance. Default 0.00001.

valacut <valacut> (``Value A cutoff'') Cutoff for last half average vs estimated long term value. Default 0.01.

initpct <initial pct> The initial percentage of data to use for the initial density guess. Default 1%.

finalpct <final pct> The final percentatge of data to use for the final density guess. Default 50%.

chisqcut <chisqcut> Curve fit chi-squared cutoff. Defualt 0.5.

slopecut <slopecut> Final slope of fitted curve cutoff. Default 0.000001.

maxit <maxit> Maximum number of iterations to perform during curve fit.

out <outfile> File to write data and fitted curve to.

resultsout <resultsfile> File to write plateau results to.

statsout <statsfile> File to write curve fitting stats to.

<input set args> Data sets to evaluate plateau for.

Data Sets Created

<name>[A0] The A0 (initial density) values.

<name>[A1] The A1 (rate constant) values.

<name>[A2] The A2 (final density) values.

<name>[OneA1] One over the A1 (rate constant) values.

<name>[corr] Curve fit correlation.

<name>[vala] Difference between last half average of data vs final density (A2).

<name>[chisq] Chi-squared of the curve fit.

<name>[pltime] Plateau time (time at which all cutoffs satisfied).

<name>[fslope] Final slope of fitted curve.

<name>[name] Input set legend.

<name>[result] Final result: yes, no, err (error).

Attempt to determine if data has “plateaued”, i.e. stopped changing significantly by the end of the set. Currently the defaults are set up to evaluate density data as part of the system preparation protocol described by Roe & Brooks.^[797]

37.12.15. fft

fft <dset0> [<dset1> ...] [out <outfile>] [name <outsetname>] [dt <samp_int>]

<dset0> [<dset1 ...] Argument(s) specifying datasets to be used.

[out <outfile>] Write results to file named <outfile>.

[name <outsetname>] The resulting transform will be stored with name <outsetname>.

[dt <samp_int>] Set the sampling interval (default is 1.0).

Perform fast Fourier transform (FFT) on specified data set(s). If more than 1 data set, they must all have the same size.

37.12.16. hausdorff

```

hausdorff <set arg0> [<set arg1> ...]
           [outtype {basic|trimatrix nrows <#>|fullmatrix nrows <#> [ncols <#>]}]
           [name <output set name>] [out <file>] [outab <file>] [outba <file>]
<set arg0> ... Input matrix data set(s) to calculate Hausdorff
distance(s) for.
[outtype] Specify the output type.
    basic Output the Hausdorff distance for each input matrix as
    scalar 1D data.
    trimatrix nrows <#> Output Hausdorff distances for each input
    matrix as a 2D upper-triangular matrix with the given number
    of rows. Must have (nrows * (nrows-1)) / 2 input sets.
    fullmatrix nrows <#> ncols <#> Output Hausdorff distances for each
    input matrix as a full matrix with the given number of
    columns and rows. If ncols is not given, use nrows. Must
    have nrows * ncols input sets.
[name <output set name>] Name of output data sets.
[out <file>] File to write Hausdorff distances to.
[outab <file>] File to write directed A->B Hausdorff distances to.
[outba <file>] File to write directed B->A Hausdorff distances to.

```

Calculate the symmetric Hausdorff distance for one or more matrices. The results can be saved as an array or as a full or upper-triangular matrix with the specified dimensions. The Hausdorff distance H is determined from:

$$H = \max\{dH(A, B), dH(B, A)\}$$

Where $dH(A, B)$ is the directed Hausdorff distance between sets A and B , etc. Colloquially speaking, the directed Hausdorff distance between A and B is determined as follows:

1. What is the closest approach (distance) of each point in A to any point in B ?
2. Choose the largest distance from among those distances.

If desired, the output can be formed into a matrix, which can be useful e.g. when doing multiple 2D rms calculations on different regions of a trajectory. For example, the following input divides a 100 frame trajectory into 10 frame chunks, calculates the 2D RMS matrix for each chunk, then performs Hausdorff analysis on the resulting matrices and forms a full output matrix.

```

parm ../DPDP.parm7
for beg=1; beg<100; beg+=10 end=10; end+=10 i=1; i++
  loadcrd ../DPDP.nc \ $beg \ $end name Chunk \ $i
done
# Do the 2drms in chunks
for i=1; i<11; i++
  for j=1; j<11; j++
    2drms crdset Chunk \ $i reftraj Chunk \ $j M \ $i. \ $j
  done
done
hausdorff M* out hausdorff.fullmatrix.gnu title hausdorff.matrix.gnu \
  outtype fullmatrix nrows 10
runanalysis

```

This type of calculation lends itself well to parallelization. The **parallelanalysis** command can be used to run all the **2drms** calculations in parallel with MPI-enabled cpptraj:


```

parm ../DPDP.parm7
for beg=1;beg<100;beg+=10 end=10;end+=10 i=1;i++
  loadcrd ../DPDP.nc \$beg \$end name Chunk\$i
done
# Do the 2drms in chunks
for i=1;i<11;i++
  for j=1;j<11;j++
    2drms crdset Chunk\$i reftraj Chunk\$j M\$i.\$j
  done
done
parallelanalysis sync
runanalysis hausdorff M* out hausdorff.fullmatrix.gnu title hausdorff.matrix.gnu \
  outtype fullmatrix nrows 10

```

37.12.17. hist | histogram

```

hist <dataset_name>[,<min>,<max>,<step>,<bins>] ...
[free <temperature>] [norm | normint] [gnu] [circular] out <filename>
[amd <amdboost_data>] [name <outputset name>]
  [traj3d <file> [trajfmt <format>] [parmout <file>]]
[min <min>] [max <max>] [step <step>] [bins <bins>] [nativeout]

```

<dataset_name>[,<min>,<max>,<step>,<bins>] Dataset(s) to be histogrammed. Optionally, the min, max, step, and/or number of bins can be specified for this dimension after the dataset name separated by commas. It is only necessary to specify the step or number of bins, an asterisk '*' indicates the value should be calculated from available data.

[free <temperature>] If specified, estimate free energy from bin populations using $G_i = -k_B T \ln\left(\frac{N_i}{N_{Max}}\right)$, where K_B is Boltzmann's constant, T is the temperature specified by <temperature>, N_i is the population of bin i and N_{Max} is the population of the most populated bin. Bins with no population are given an artificial barrier equivalent to a population of 0.5.

[norm] If specified, normalize bin populations so the sum over all bins equals 1.0.

[normint] Normalize bin populations so the integral over them is 1.0.

[gnu] Internal output only; data will be gnuplot-readable, i.e. a space will be printed after the highest order coordinate cycles.

[circular] Internal output only; data will wrap, i.e. an extra bin will be printed before min and after max in each direction. Useful for e.g. dihedral angles.

out <filename> Write results to file named <filename>.

[amd <amdboost_data>] Reweight bins using AMD boost energies in data set <amdboost_data> (in KT).

[name <outputset name>] Output histogram data set name.

[traj3d <file> [trajfmt <format>]] (3D histograms only) Write a pseudo-trajectory of the 3 data sets (1 atom) to <file> with format <format>.

[parmout <file>] (3D histograms only) Write a topology corresponding to the pseudo-trajectory to <file>.

[min <min>] Default minimum to bin if not specified.

[max <max>] Default max to use if not specified.

[step <step>] Default step size to use if not specified.

[bins <bins>] Default bin size to use if not specified.

[nativeout] Do not use cpptraj data file framework; only necessary for writing out histograms with > 3 dimensions.

Create an N-dimensional histogram, where N is the number of datasets specified. For 1-dimensional histograms the xmgrace '.agr' file format is recommended; for 2-dimensional hisograms the gnuplot '.gnu' file format is recommended; for all other dimensions plot formatting is disabled and the routine uses its own internal output format; this is also enabled if **gnu** or **circular** is specified.

For example, to create a two dimensional histogram of two datasets 'phi' and 'psi':

```
dihedral phi :2@C :3@N :3@CA :3@C
dihedral psi :3@N :3@CA :3@C :4@N
hist phi,-180,180,*,72 psi,-180,180,*,72 out hist.gnu
```

In this case the number of bins (72) has been specified for each dimension and '*' has been given for the step size, indicating it should be calculated based on min/max/bins. The following 'hist' command is equivalent:

```
hist phi psi min -180 max 180 bins 72 out hist.gnu
```

37.12.18. integrate

```
integrate <dset0> [<dset1> ...] [out <outfile>] [intout <intfile>]
      [name <name>]
```

<dset0> [<dset1> ...] Data set(s) to integrate.

[out <outfile>] If specified, write cumulative sum curves to <outfile>.

[intout <intfile>] If specified, write final integral values to <intfile>.

[name <name>] Output data set(s) name.

DataSets Created:

<name> Final integral values, 1 for each input data set (indexed from 0).

<name>[Sum]:<idx> Cumulative sum curves if out was specified, 1 for each input data set (indexed from 0).

Integrate specified data set(s) using trapezoid integration. If 'out' is specified write cumulative sum curves to <outfile>. If 'intout' is specified write final integral values for each set to <intfile>.

37.12.19. ired

```
ired [relax freq <MHz> [NHdist <distnh>] [noefile <noefilename>]]
      [order <order>] [orderparamfile <orderfilename>]
      tstep <tstep> tcorr <tcorr> out <filename> [norm] [drct]
      modes <modesname> [name <output sets name>] [ds2matrix <file>]
```

[relax freq <MHz>] Should only be used when ired vectors represent N-H bonds; calculate correlation times τ_m for each eigenmode and relaxation rates and NOEs for each N-H vector. 'freq <MHz>' (required) is the Lamor frequency of the measurement.

[NHdist <distnh>] Specifies the length of the NH bond in Angstroms (default is 1.02).

[noefile <noefilename>] File to write the T1, T2, and NOE data to.

[order <order>] Order of the Legendre polynomials to use when calculating spherical harmonics (default 2).

[orderparamfile <orderfilename>] File to write the S2 data to.

[tstep <tstep>] Time between snapshots in ps (default 1.0).

[tcorr <tcorr>] Maximum time to calculate correlation functions for in ps (default 10000.0).

[out <filename>] Name of file to write plateau and TauM data. Also the prefix for the .cmt and .cjt files (see below).

[norm] Normalize all correlation functions, i.e., $C_l(t=0) = P_l(t=0) = 1.0$.

[drct] Use the direct method to calculate correlations instead of FFT; this will be much slower.

modes <modesname> Name of previously calculated eigenmodes corresponding to IRED vectors.

[name <name>] Output data set name.

[ds2matrix <file>] If specified, write full $\delta \cdot S^2$ matrix (# IRED vector rows by # eigenmodes columns) to <file>.

DataSets Created:

<name>[S2] S2 order parameters for each vector.

<name>[Plateau] Plateau values for each vector.

<name>[TauM] TauM values for each vector.

<name>[dS2] Full $\delta \cdot S^2$ matrix.

<name>[T1] T1 relaxation values for each vector.

<name>[T2] T2 relaxation values for each vector.

<name>[NOE] NOEs for each vector.

<name>[Cm(t)]:X Cm(t) function for vector X.

<name>[Cj(t)]:X Cj(t) function for vector X.

Perform IRED[780] analysis on previously defined IRED vectors (see vector ired) using eigenmodes calculated from those vectors with a previous 'diagmatrix' command. The number of defined IRED vectors should match the number of eigenmodes calculated. Autocorrelation functions for each mode and the corresponding correlation time τ_m will be written to <filename>.cmt. Autocorrelation functions for each vector will be written to <filename>.cjt. Relaxation rates and NOEs for each N-H vector will be written to <filename> or added to the end of the standard output. For the calculation of τ_m the normalized correlation functions and only the first third of the analyzed time steps will be used. For further information on the convergence of correlation functions see [Schneider, Brünger, Nilges, *J. Mol. Biol.* **285**, 727 (1999)].

Example of IRED in Cpptraj

In *cpptraj*, IRED analysis[780] can now be performed in one pass (as opposed to the two passes previously required in *ptraj*). First, IRED vectors are defined (in this case for N-H bonds) and an IRED matrix is calculated and analyzed. The IRED vectors are then projected onto the calculated IRED eigenvectors in the *ired* analysis command to calculate the time correlation functions. If the parameter *order* is specified, order parameters based on IRED are calculated. By specifying the *relax* parameter, relaxation rates and NOEs can be obtained for each N-H vector. Note that the order of the IRED matrix should be the same as the one specified for IRED analysis.

```
# Define N-H IRED vectors
vector v0 @5 ired @6
vector v1 @7 ired @8
...
vector v5 @15 ired @16
vector v6 @17 ired @18`
# Define IRED matrix using all previous IRED vectors
matrix ired name matired order 2
# Diagonalize IRED matrix
diagmatrix matired vecs 6 out ired.vec name ired.vec
# Perform IRED analysis
ired relax NHdist 1.02 freq 500.0 tstep 1.0 tcorr 100.0 out v0.out \
    noefile noe order 2
```

37.12.20. kde

```
kde <dataset> [bandwidth <bw>] [out <file>] [name <dsname>]
    [min <min>] [max <max>] [step <step>] [bins <bins>] [free]
    [kldiv <dsname2> [klout <outfile>]] [amd <amdboost_data>]

[bandwidth <bw>] Bandwidth to use for KDE; if not specified
    bandwidth will be estimated using the normal distribution
    approximation.

[out <file>] Output file name.

[name <dsname>] Output data set name.

[min <min>] Minimum bin.

[max <max>] Maximum bin.

[step <step>] Bin step.

[bins <bins>] Number of bins.

[free] Calculate free energy from bin population.

[kldiv <dsname2> [klout <outfile>]] Calculate Kullback-Leibler divergence
    over time of <dataset> distribution to <dsname2>
    distribution. Output to <outfile> if klout specified.

[amd <amdboost_data>] Reweight histogram using AMD boost data from
    data set <amdboost_data> (in KT).
```

Histogram 1D data set using a Gaussian kernel density estimator.

37.12.21. lifetime

```
lifetime [out <filename>] <dsetarg0> [ <dsetarg1> ... ]
    [window <>window size>] [name <setname>]] [averageonly]
    [cumulative] [delta] [cut <cutoff>] [greater | less] [rawcurve]
    [fuzz <fuzzcut>] [nosort]
```

[out <filename>] Write results to file named <filename>, and lifetime curves to 'crv.<filename>'. If performing windowed lifetime analysis, <filename> contains the fraction present over time windows, and 2 additional files are written: 'max.<filename>', containing max lifetime over windows, and 'avg.<filename>', containing average lifetime over windows.

<dsetarg0> [<dsetarg1> ...] Argument(s) specifying datasets to be used.

[window <>window size>] Size of window (in frames) over which to calculate lifetimes/averages. If not specified lifetime/average will be calculated over all frames.

[name <setname>] Store results in data sets with name <setname>.

[averageonly] Just calculate averages (no lifetime analysis).

[cumulative] Calculate cumulative lifetimes/averages over windows.

[delta] Calculate difference from previous window average.

[cut <cutoff>] Cutoff to use when determining if data is 'present' (default 0.5).

[greater] Data is considered present when above the cutoff (default).

[less] Data is considered present when below the cutoff.

[rawcurve] Do not normalize lifetime curves to 1.0.

[fuzz <fuzzcut>] Ignore changes in lifetime state that are less than <fuzzcut> frames.

[nosort] Do not sort data sets by name.

Data Sets Created:

<setname> Number of lifetimes for each set, or if window specified fraction present over time windows.

<setname>[max] Maximum lifetime for each set, or if window specified maximum lifetime over time windows.

<setname>[avg] Average lifetime for each set, or if window specified average lifetime over time windows.

<setname>[curve] Lifetime curves.

The following are created only if window not specified:

<setname>[frames] Total number of frames lifetime present for each set.

<setname>[name] Name of each set.

Perform lifetime analysis for specified data sets. Lifetime data can either be determined for the entire set, or for time windows of specified size within the set if **window** specified.

A "lifetime" is defined as the length of time something remains 'present'; data is considered present when above or below a certain cutoff (the default is greater than 0.5, useful for analysis of *hbond* time series data). For example, in the case of a hydrogen bond **'series'** data set, if a hydrogen bond is present during a frame the value is 1, otherwise it is 0. Given the *hbond* time series data set {1 1 1 0 1 0 0 1 1}, the overall fraction present is 0.6. However, there are 3 lifetimes of lengths 3, 1, and 2 ({1 1 1}, {1}, and {1 1}). The maximum lifetime is 3 and the average lifetime is 2.0, i.e. $(3 + 1 + 2) / 3 \text{ lifetimes} = 2.0$. One can also construct a "lifetime curve", which is constructed as the sum of all individual lifetimes. By default these curves are normalized to 1.0, but the raw curve can be obtained using the **rawcurve** keyword. For the example data set here the raw lifetime curve would be 3 frames long:

```

      1 1 1
      1
      1 1
Curve: 3 2 1

```

By default data sets are sorted by name unless **nosort** is specified. The lifetime command can calculate lifetimes over specific time windows by using the **window** keyword. This can be particularly useful if one wants to get a sense for how lifetimes are changing over the course of very long time series data. In addition, averages can be calculated instead of lifetimes by specifying **averageonly**. Cumulative averages over windows can be obtained using the **cumulative** keyword, or the change from the average value in the previous window can be obtained using the **delta** keyword.

The **fuzz** keyword can be used to try and smooth the input data by ignoring changes in state that occur for fewer frames than `<fuzzcut>`. For example, in the above example hbond time series data set there is a one frame change in state between the first and second lifetimes which could be interpreted as a transient breaking of the hydrogen bond. Using a `<fuzzcut>` value of 1, this one frame change in state would be ignored, and the data set would effectively appear to lifetime as {1 1 1 1 1 0 0 1 1}. The state change between the second and third lifetimes is longer than `<fuzzcut>` (3 frames) and so it would remain.

If **window** is not specified, two files are output: `<filename>` and `crv.<filename>`. The file `<filename>` contains overall lifetime stats for each set with format:

```
#Set <setname> <setname>[max] <setname>[avg] <setname>[frames] <setname>[name]
```

where `<setname>` denotes the total number of lifetimes, `<setname>[max]` denotes the maximum lifetime, `<setname>[avg]` denotes the average lifetime, `<setname>[frames]` denotes the total number of frames present in all lifetimes, and `<setname>[name]` is the data set name. The file `crv.<filename>` contains the lifetime curves for each set.

If **window** is specified, four files are output: `<filename>`, `max.<filename>`, `avg.<filename>`, and `crv.<filename>`. `<filename>` contains the fraction “present” over each time window for each set, `max.<filename>` contains the maximum lifetime in each time window for each set, `avg.<filename>` contains the average lifetime over each window for each set, and `crv.<filename>` contains the overall lifetime curves for each set. For window output, Gnuplot format is recommended.

Example: hbond lifetime analysis

```

parm DPDP.parm7
trajin DPDP.nc
hbond HB out hbond.dat @N,H,C,O series uuseries solutehb.agr \
  avgout hbavg.dat printatomnum
# 'run' is used here to process the trajectory and generate hbond data
run
# Perform lifetime analysis
runanalysis lifetime HB[solutehb] out lifehb.dat

```

Calculate ion lifetimes from hbond over windows of size 100 frames:

```

hbond ION out ion.dat solventdonor :WAT solventacceptor :WAT@O series
run
lifetime HB[solventhb] out ion.lifetime.100.gnu window 100

```

37.12.22. lowestcurve

```

lowestcurve points <# lowest> [step <stepsize>] <dset0> [<dset1> ...]
               [out <file>] [name <setname>]

```

<# lowest> Number of lowest points in each bin to average over.
[step <stepsize>] Bin step size
<dset0> [<dset1> ...] Data set(s) to use.
[out <file>] File to write lowest curve to.
[name <setname>] Output lowest curve set name.

Calculate a curve of the average of the # lowest points in bins of stepsize. Essentially each input data set is binned over bins of stepsize, then the lowest <#> points are averaged over for each bin.

37.12.23. meltcurve

meltcurve <dset0> [<dset1> ...] [out <outfile>] [name <outsetname>] cut <cut>

Calculate melting curve from input data sets (i.e. fraction 'folded' for each data set) assuming a simple 2-state transition model, using data below <cut> as 'folded' and data above <cut> as 'unfolded'.

37.12.24. modes

modes {fluct|displ|corr|eigenval|trajout|rmsip} name <modesname> [name2 <modesname>]
[beg <beg>] [end <end>] [bose] [factor <factor>] [calcall]
[out <outfile>] [setname <name>]
 Options for 'trajout': (Generate pseudo-trajectory)
[trajout <name> parm <name> | parminindex <#>
[trajoutfmt <format>] [trajoutmask <mask>]
[pcmin <pcmin>] [pcmax <pcmax>] [tmode <mode>]]
 Options for 'corr': (Calculate dipole correlation)
{ maskp <mask1> <mask2> [...] | mask1 <mask> mask2 <mask> }
parm <name> | parminindex <#>

Types of Calculations:

fluct RMS fluctuations (X, Y, Z, and total) for each atom across specified normal modes.

displ Displacement of cartesian coordinates in the X, Y and Z directions for each atom across specified normal modes.

corr Dipole-dipole correlation functions. Must also specify maskp (see below).

eigenval Calculate eigenvalue fractions.

trajout Create a pseudo-trajectory along the given mode from the average structure.

rmsip Calculate the root-mean-square inner product between modes specified by name and name2.

Options:

name <modesname> Previously read-in or generated Modes data set name.

[beg <beg>][end <end>] If modes taken from datafile, beginning and end modes to read. Default for beg is 7 (which skips the first 6 zero-frequency modes in the case of a normal mode analysis); for end it is 50.

[bose] Use quantum (Bose) statistics in populating the modes.

[factor <factor>] multiplicative constant on the amplitude of displacement/pseudo-trajectory, default 1.0.

[calcall] If specified use all eigenvectors; otherwise eigenvectors associated with zero or negative eigenvalues will be skipped.

[out <outfile>] File to write data results to. If not given results are written to STDOUT.

[setname <name>] Output data set name.

Options for 'trajout':

<name> Output trajectory file name.

[parm <parmfile/tag>|parmindex <#>] Topology file to use (default first Topology loaded).

[trajoutfmt <format>] Output trajectory format.

[trajoutmask <mask>] Mask of atoms that correspond to how modes were originally generated.

[pcmin <pcmin>] Lowest principal component projection value to use for output trajectory.

[pcmax <pcmax>] Highest principal component projection value to use for output trajectory.

[tmode <mode>] Mode to generate pseudo-trajectory for.

Options for 'corr':

[maskp <mask1> <mask2> [...]] If corr, pairs of atom masks (*mask1*, *mask2*; each pair preceded by ``maskp'' and each mask defining only a single atom) have to be given that specify the atoms for which the correlation functions are desired.

mask1 <mask> mask2 <mask> Instead of maskp, specify two masks; atoms from the first mask will be paired up with atoms from the second mask.

DataSets Created (fluct)

<name>[rmsX] RMS fluctuations in the X direction.

<name>[rmsY] RMS fluctuations in the Y direction.

<name>[rmsZ] RMS fluctuations in the Z direction.

<name>[rms] Total RMS fluctuations.

DataSets Created (displ)

<name>[displX] Displacement in X direction.

<name>[displY] Displacement in Y direction.

<name>[displZ] Displacement in Z direction.

DataSets Created (eigenval)

<name>[Frac] Fraction eigenvalue contributes to overall motion.

<name>[Cumulative] Cumulative fraction.

<name>[Eigenval] Value of eigenvalue.

DataSets Created (rmsip)

<name> Result of RMSIP calculation.

Analyze previously calculated eigenmodes obtained from principal component analyses (of covariance matrices) or quasiharmonic analyses (diagmatrix analysis command). Modes are taken from a previously generated data set (i.e. from *diagmatrix*) or read in from a data file with *readdata*. By default, classical (Boltzmann) statistics are used in populating the modes. A possible series of commands would be “**matrix covar | mwcovar ...**” to generate the matrix, “**diagmatrix ...**” to calculate the modes, and, finally, “**modes ...**”.

For example, to calculate the RMS fluctuations or displacements of the first 3 eigenmodes calculated from a mass-weighted covariance matrix:

```
matrix mwcovar name mwcvmat out mwcvmat.dat
diagmatrix mwcvmat name evecs vecs 5
modes fluct out rmsfluct.dat name evecs beg 1 end 3
modes displ out resdispl.dat name evecs beg 1 end 3
```

Additionally, dipole-dipole correlation functions for modes obtained from principle component analysis or quasiharmonic analysis can be computed.

```
modes corr out cffromvec.dat name evecs beg 1 end 3 \
maskp @1 @2 maskp @3 @4 maskp @5 @6
```

or

```
mode corr out cffromvec.dat name evecs beg 1 end 3 mask1 @1,3,5 mask2 @2,4,6
```

If **eigenval** is specified, the fraction contribution of each eigenvector to the total motion is calculated and output with format:

```
#Mode Frac. Cumulative Eigenval
```

where #Mode is the eigenvector number, Frac. is the eigenvalue over the sum of all eigenvalues, Cumulative is the cumulative sum of Frac., and Eigenval is the eigenvalue itself. Note that in order to get an idea for how much each eigenvector contributes to all motion, this is best used when all possible eigenvectors have been determined for a system.

In order to visualize eigenvectors, pseudo-trajectories along eigenvectors can be created using average coordinates with the **trajout** keyword. For example, to write a pseudo-trajectory of the first principal component from principal component value of -100 to 100 for a previously calculated Modes data set corresponding to heavy atoms (no hydrogens) for residues 1 to 36:

```
parm ../GAAC.nowat.parm7
readdata evecs.dat
runanalysis modes name evecs.dat trajout test.nc trajoutfmt netcdf \
trajoutmask :1-36&!@H= pccmin -100 pccmax 100 tmode 1
```

37.12.25. multicurve

```
multicurve set <dset> [set <dset> ...]
<dset> { <equation> |
        name <dsname> nexp <m> [form {mexp|mexpk|mexpk_penalty} }
[AX=<value> ...] [out <outfile>] [resultsout <results>]
[maxit <max iterations>] [tol <tolerance>]
[outxbins <NX> outxmin <xmin> outxmax <xmax>]

set <dset> [set <dset> ...] Data set(s) to fit.

<equation> Equation to fit of form <Variable> = <Equation>. See
37.5.2 on page 694 for more details on equations ccpptraj
understands.
```

name <dsname> Name of output data sets (required if using **nexp**).
nexp <m> Fit to specified number of exponentials.
form <type> Fit to specified exponential form:
 mexp Multi-exponential, $\text{SUM}(m) [A_n * \exp(A_{n+1} * X)]$
 mexpk Multi-exponential plus constant, $A_0 + \text{SUM}(m) [A_n * \exp(A_{n+1} * X)]$
 mexpk_penalty Same as **mexpk** except sum of prefactors constrained to 1.0 and exponential constants constrained to < 0.0 .
AX=<value> Value of any constants in specified equation with X starting from 0 (can specify more than one).
out <outfile> Write resulting fit curve to <outfile>.
resultsout <results> Write details of the fit to <results> (default STDOUT).
maxit <max iterations> Number of iterations to run curve fitting algorithm (default 50).
tol <tolerance> Curve-fitting tolerance (default 1E-4).
outxbins <NX> Number of points to use when generating final curve (default same number of points as input data set).
outxmin <xmin> Minimum X value to use for final curve (default same number of points as input data set).
outxmax <xmax> Maximum X value to use for final curve (default same number of points as input data set).

Fit each input data set <dset> to <equation>. See the *curvefit* command on page 863 for more details.

37.12.26. multihist

multihist [out <filename>] [name <dsname>] [norm | normint] [kde]
 [min <min>] [max <max>] [step <step>] [bins <bins>] [free <T>]
 <dsetarg0> [<dsetarg1> ...]
out <filename> Output file.
name <dsname> Name for resulting histogram data sets.
norm (Only used if not **kde**) Normalize so that max bin is 1.0.
normint (Default for **kde**) Normalize integral over histogram to 1.0.
kde Use kernel density estimator to construct histogram.
min <min> Histogram minimum (default data set minimum).
max <max> Histogram maximum (default data set maximum).
step <step> Histogram step.
bins <bins> Number of histogram bins.
free <T> Calculate free energy from bin populations as $G = -R * \langle T \rangle * \ln(N_i / N_{\max})$.
<dsetargX> Data set argument - may specify more than one.

Histogram each data set separately in 1D. Must specify at least **bins** or **step**.

37.12.27. phipsi

```
phipsi <dsarg0> [<dsarg1> ...] resrange <range> [out <file>]
<dsargX> Argument selecting data sets. Can specify more than 1.
resrange <range> Residue range to use (actually uses data set
index).
[out <file>] Output file.
```

Calculate the average and standard deviation of [phi] and [psi] data set pairs, write to <file> with format:

```
#Phi Psi SD(Phi) SD(Psi) Legend
```

Where Phi is the average value of phi, Psi is the average value of psi, SD(Phi) is the standard deviation of phi, SD(psi) is the standard deviation of psi, and Legend contains text describing the phi and psi data sets used in the calculation. Periodicity is taken into account during averaging. The data sets must have been internally labeled as type 'phi'/'psi' and must have a data set index set (actions like dihedral and multidihedral do this automatically). For example:

```
parm ../DPDP.parm7
trajin ../DPDP.nc
multidihedral DPDP phi psi
run
phipsi DPDP[phi] DPDP[psi] out phipsi.dat resrange 1-22
```

37.12.28. projectdata

```
projectdata evects <evects dataset> [name <name>] [out <outfile>] [beg <beg>] [end
{[dihedrals <dataset arg>] | [data <dataset arg> ...]}]
evects <dataset name> Data set containing eigenvectors (modes).
[name <name>] Output data set name.
[out <outfile>] Write projections to <outfile>.
[beg <beg>] First eigenvector/mode to use (default 1).
[end <end>] Final eigenvector/mode to use (default 2).
[dihedrals <dataset arg>] (Dihedral covariance only) Dihedral data sets
to use in projection; MUST CORRESPOND TO HOW EIGENVECTORS WERE
GENERATED.
[data <dataset arg>] (Data covariance only, e.g. from TICA). 1D data
sets to use in projection; MUST CORRESPOND TO HOW EIGENVECTORS
WERE GENERATED.
Data Sets Created:
<name>:<#> Projection data set for mode <#>.
```

Project data along previously generated eigenmodes from e.g. PCA or TICA. This is a faster alternative to the *projection* command ([37.11.63 on page 820](#)) if only 1D data sets need to be projected.

37.12.29. regress

```
regress <dset0> [<dset1> ...] [name <name>] [nx <nxvals>]
[out <filename>] [statsout <filename>]
dsetX Data set(s) to perform linear regression for.
```

name <name> Data set name for resulting linear fits.

nx <nxvals> Number of X values to use in output data set(s) (ranging from input set min to max X). If not specified, input X values used.

out <filename> File to write fit lines to.

statsout <filename> File to write fit statistics to.

DataSets Generated:

<name>:<idx> Output fit line(s) (indexed by input set order if more than one input set).

<name>[slope]:<idx> Output fit line slope(s).

<name>[intercept]:<idx> Output fit line intercept(s).

Perform linear regression on the specified data set(s). The fit line is calculated using either the input X values or <nxvals> values ranging from the input set minimum to maximum X. Statistics for the fit(s) are saved to the file specified by **statsout** or reported to STDOUT.

For example, to fit data read in from a file and then create a set using the fit parameters:

```
readdata esurf_vs_rmsd.dat.txt index 1 name XY
runanalysis regress XY name FitXY statsout statsout.dat
createset "Y = FitXY[slope] * X + FitXY[intercept]" xstep .2 nx 100
writedata Y.dat Y
```

37.12.30. remlog

```
remlog {<remlog dataset> | <remlog filename>} [out <filename>] [crdidx | repidx]
[stats [statsout <file>] [printtrips] [reptime <file>]] [lifetime <file>]
[reptimeslope <n> reptimeslopeout <file>] [acceptout <file>] [name <setname>]
[edata [edataout <file>]]
```

<remlog dataset> Previously read-in REM log data.

<remlog filename> REM log file name to read in.

[out <filename>] Write replica/coordinate index versus time to <filename>.

crdidx Print coordinate index vs exchange; output sets contain replica indices.

repidx Print replica index vs exchange; output sets contain coordinate indices.

stats [statsout <file>] Calculate round-trip statistics and optionally write to <file>.

printtrips Print details of each individual round trip.

[reptime <file>] Write time spent at each replica to <file>.

[lifetime <file>] Print lifetime data at each replica to <file>.

[reptimeslope <n>] Calculate the slope of time spent at each replica every <n> exchanges.

[reptimeslopeout <file>] File to write reptimeslope output to.

[acceptout <file>] Write overall exchange acceptances to <file>.

[name <setname>] Output data set name.

[edata [edataout <file>]] Extract energy data from replica log, optionally write to file.

DataSets created:

<setname>:<idx> Replica/coordinate index vs exchange.

<setname>[E]:<idx> If 'edata' specified, energy data from replica log.

Analyze previously read in (via *readdata*) M-REMD/T-REMD/H-REMD replica log data. Statistics calculated include round-trip time, which is the time needed for a coordinate set to travel from the lowest replica to the highest and back, and the number of exchanges each coordinate spent at each replica. For example, to read in REM log data from an Amber M-REMD run and analyze it:

```
readdata rem.log.1.save rem.log.2.save dimfile remd.dim as remlog nosearch
remlog rem.log.1.save stats reptime mremdreptime.dat
```

For an example of *remlog* analysis applied to actual REMD data, see Roe et al.[\[798\]](#).

37.12.31. rms2d | 2drms

```
rms2d [crdset <crd set>] [<name>] [<mask>] [out <filename>]
      [{dme | nofit | srmsd | qrmsd}] [mass]
      [reftraj <traj>] [parm <parmname> | parminde <parm#>] [<refmask>]]
      [corr <corrfilename>]
```

[crdset <crd set>] Name of previously generated COORDS DataSet. If not specified the default COORDS set will be used.

[<mask>] Mask of atoms to calculate 2D-RMSD for. Default is all atoms.

[out <filename>] Write results to <filename>.

[dme] Calculate distance RMSD instead of coordinate RMSD; this is substantially slower.

[nofit] Calculate RMSD without fitting.

[srmsd] Calculate symmetry-corrected RMSD (see [37.11.84 on page 836](#)).

[qrmsd] Use quaternion RMSD calculation (can be 15-20% faster).

[mass] Mass-weight RMSD.

[reftraj <traj>] Calculate 2D RMSD to frames in trajectory <traj> instead (can also be another COORDS set).

[parm <parmname> | parminde <#>] Topology to use for <traj>; only useful in conjunction with reftraj.

[<refmask>] Mask of atoms in reference; only useful in conjunction with reftraj.

[corr <corrfilename>] Calculate pseudo-auto-correlation C for 2D-RMSD as $C(i) = \frac{\sum_{j=0}^{N-i} \exp(-\text{RMSD}(j, j+i))}{N-i}$, where i is the lag, j is the frame #, and N is the total number of frames. An exponential is used to weight the RMSD since 0.0 RMSD is equivalent to correlation of 1.0. This can only be done if reftraj is not used.

DataSet Aspects:

[Corr] (corr only) Pseudo-auto-correlation.

Note: For backwards compatibility with ptraj the command '2drms' will also work.

Calculate the best-fit RMSD of each frame in <crd set> (the default COORDS set if none specified) to each other frame. This creates an upper-triangle matrix named <name> (or a full matrix if **reftraj** specified). The output of the rms2d command can be best-viewed using gnuplot; a gnuplot-formatted file can be produced by giving <filename> a '.gnu' extension. For example, to calculate the RMSD of non-hydrogen atoms of each frame in trajectory "test.nc" to each other frame, writing to a gnuplot-viewable file "test.2drms.gnu":

```
trajin test.nc
rms2d !(@H=) out test.2drms.gnu
```

To calculate the RMSD of atoms named CA of each frame in trajectory "test.nc" to each frame in "ref.nc" (assuming test.nc and ref.nc are using the default topology file):

```
trajin test.nc
rms2d @CA out test.2drms.gnu reftraj ref.nc
```

37.12.32. rmsavgccorr

```
rmsavgccorr [crdset <crd set>] [<name>] [<mask>] [out <filename>] [mass]
           [stop <maxwindow>] [offset <offset>]
           {reference <ref file> parm <parmfile> | first}
```

[crdset <crd set>] COORDS data set to use (if not specified the default COORDS set will be used).

[<name>] Output data set name.

[<mask>] Atoms to calculate RMS average correlation for.

[out <filename>] Output filename.

[mass] Mass weight the RMSD calculation.

[stop <maxwindow>] Only calculate RMS average correlation up to <maxwindow>.

[offset <offset>] Skip every <offset> windows in calculation.

[first] Use first averaged frame as reference for each window (default).

[reference <ref file> [parm <parmfile>] Use reference file (with specified parm) as reference for each window.

The RMS average correlation^[764] (RAC) is calculated as the average RMSD of running-averaged coordinates over increasing window sizes (or lag). Output has format:

```
<WindowSize> <RAC>
```

The first entry has a window size of 1, and so is just the average RMSD of all frames to the specified reference structure. The second entry has a window size of two, so it is the average RMSD of all frames averaged over two adjacent windows to the specified reference, and so on. The RAC will be calculated up to the number of frames minus 1 or the value specified by **stop**, whichever is lower. The offset can be used to speed up the calculation by skipping window sizes. To calculate mass-weighted RMSD specify **mass**. Note that to reduce memory costs it can be useful to strip all coordinates not involved in the RMS fit from the system prior to specifying 'rmsavgccorr'. For example, to calculate the correlation of C-alpha RMSD of residues 2 to 12:

```
strip !(:2-12@CA)
rmsavgccorr out rmscorr.dat
```

The curve generated by RAC decays towards zero due to the way RAC is defined. By the time the "lag" is $N-1$ (where N is the total number of frames) you have only two averaged coordinates: call them Avg1 (averaged over 1 through $N-1$ frames) and Avg2 (averaged over 2 through N frames). Barring any extraordinary circumstances the RMSD between Avg1 and Avg2 will almost certainly be quite low.

The RAC is a way to probe the time scales of interesting events. Any deviation from a smoothly decaying curve is an indication that there are some significant structural differences occurring over that time interval. RAC curves can be particularly useful when comparing independent simulations of the same system.

One thing to keep in mind that since the underlying metric is RMSD, it can be sensitive to the reference frame you choose. It may be useful to try looking at both RAC from the first frame, as well as an averaged reference frame. For an example of use see Galindo-Murillo et al.[799], in particular Figure 2.

37.12.33. rotdif

```
rotdif [outfile <outfilename>] [usefft]
  Options for generating random vectors:
[nvecs <nvecs>] [rvecin <randvecIn>] [rseed <random seed>]
[rvecout <randvecOut>] [rmatrix <set name>] [rmout <rmOut>]]
  Options for calculating vector time correlation functions:
[order <olegendre>] [ncorr <ncorr>] [corrout <corrOut>]
  *** The options below only apply if 'usefft' IS NOT specified. ***
  Options for calculating local effective D, small anisotropy:
[deffout <deffOut>] [itmax <itmax>] [tol <tolerance>] [d0 <d0>]
[nmesh <NmeshPoints>] dt <tfac> [ti <ti>] tf <tf>
  Options for calculating D with full anisotropy:
[amoeba_tol <tolerance>] [amoeba_itmax <iterations>]
[amoeba_nsearch <n>] [scalesimplex <scale>] [gridsearch]
  *** The options below only apply if 'usefft' IS specified. ***
  Options for curve-fitting:
[fit_tol <tolerance>] [fit_itmax <max # iterations>]
```

outfile <outfilename> File to write all output from rotdif command to.

Options for generating random vectors:

nvecs <nvecs> Number of random vectors to generate (default 1000).

rvecin <randvecIn> File to read random vectors from (format is 1 per line, 4 columns, <#> <VX> <VY> <VZ>).

rseed <random seed> Seed for random number generator (default 80531). Specify -1 to use wallclock time.

rvecout <randvecOut> File to write random vectors to (format is 1 per line, 4 columns, <#> <VX> <VY> <VZ>).

rmatrix <set name> Data set to read rotation matrices from. Rotation matrices will be used to rotate random vectors.

rmout <rmOut> Write rotation matrices to file, 1 per line, frame # followed by matrix in row-major order.

Options for calculating vector time correlation functions:

order <olegendre> The order of Legendre polynomials to use when calculating vector time correlation functions (default 2).

ncorr <ncorr> Maximum length of time correlation functions in frames. If this is not specified it will be set to $(tf - ti) / dt$ (recommended).

corrout <corrOut> If specified write vector time correlation functions to <corrOut>.X with format: <Time> <Px>

Options for calculating local effective D, small anisotropy:

deffout <deffOut> File to write out local effective diffusion constants determined in the limit of small anisotropy.

itmax <itmax> Maximum number of iterations to determine each local effective diffusion constant (small anisotropy) assuming fit to single exponential form (default 500).

tol <tolerance> Tolerance for determining local effective diffusion constant (small anisotropy) assuming fit to single exponential form (default 1E-6).

d0 <d0> Initial guess for small anisotropy diffusion constant in radians²/ns (default 0.03).

nmesh <NmeshPoints> Number of points per frame to use when creating cubic-splined-smoothed forms of vector time correlation curves (default 2).

dt <tfac> Time interval between frames (used in integrating vector time correlation curves) in ns.

ti <ti> Initial time value in ns for integrating the time correlation functions (default 0.0).

tf <tf> Final time value in ns for integrating the time correlation functions. It is recommended this be less than the maximum simulation time since the tails of time correlation functions tend to be noisy.

Options for calculating D with full anisotropy:

amoeba_tol <tolerance> Tolerance for downhill-simplex minimizer (default 1E-7).

amoeba_itmax <iterations> Number of iterations to run downhill-simplex minimizer (default 10000).

amoeba_nsearch <n> Number of searches to perform with downhill-simplex minimizer (default 1).

scalesimplex <scale> Factor to use when scaling simplexes (default 0.5).

gridsearch If specified, perform a brute-force grid search to attempt to find a better solution for diffusion tensor with full anisotropy (may be expensive).

Evaluate rotational diffusion properties of a molecule over a trajectory according to an expanded version of the procedure laid out by Wong & Case[800]. Briefly, random vectors (representing the orientation of the molecule) are rotated according to rotation matrices obtained from an RMS fit to a reference structure (typically an averaged structure). For each random vector the time correlation function of the rotated vector is calculated using Legendre polynomials of the specified order. The integral over this time correlation function (which may be smoothed using cubic splines to improve the integration) is then used to find the effective diffusion constant (D) in the limit of small anisotropy. Then, using each calculated D, the diffusion tensor is determined with full anisotropy. Finally, a downhill simplex minimizer is used to optimize D with full anisotropy; (this last step is not described in the original paper).

Rotation matrices are generated via an RMS fit to a reference structure (see 37.11.70 on page 825). It is recommended that the RMS fit be done to an average structure (see 37.11.9 on page 762). These rotation matrices are

used to rotate each random vector M times (where M is the total number of frames), which creates a time series for each random vector. The time correlation functions are calculated for each random vector time series using Legendre polynomials of the specified order (default 2). Calculation of time correlation functions can be sped up by using the OpenMP version of CPPTRAJ. The maximum length of the correlation function (or lag) can be specified by **ncorr** (in frames). If **ncorr** is not specified it will be set internally based on the specified values of **ti**, **tf**, and **dt**; this is recommended. Note that if **ncorr** is specified it should be set to a number less than the total number of frames since noise in time correlation functions increases as **ncorr** approaches the # of frames. The integration over the correlation function is from **ti** (in whatever units are used of **dt**, generally ns; 0.0 ns if not specified) to **tf** (same units as **ti**), with the time between frames specified by **dt**; the final time should be less than the total simulation time (see example below). The relative size of the mesh used with cubic spline interpolation for integration is controlled by **nmesh** (size of the mesh is **ncorr** points * **nmesh**); **nmesh** = 1 means no interpolation, default is 2. Note that if the integral of the correlation function for a vector is negative, that vector will be skipped in subsequent calculations (since it would imply a negative value for effective diffusion).

The iterative solver for effective value of the diffusion constant from the correlation functions is controlled by **itmax**, **tol**, and **d0**, where **itmax** specifies the number of iterations to perform (default 500), **tol** specifies the tolerance (default 1E-6), and **d0** specifies the initial guess for the diffusion constant in $\text{radians}^2 / \text{ns}$ (default 0.03). Effective diffusion constants for each random vector can be written out to a file specified by **deffout**. Results are printed to the file specified by **outfile**. Details on the Q and D tensors are given, as well as observed and calculated tau for each random vector. First, results are printed for analysis in the limit of small anisotropy. Next, results are printed for analysis with full anisotropy. The results of the full anisotropic calculation are first given using results from the small anisotropic analysis as an initial guess, followed by the final results after minimization using the downhill simplex (amoeba) minimizer.

Example

There are two important things to keep in mind when using rotdif analysis:

1. When calculating any kind of diffusive property it is best to simulate in the microcanonical (NVE) ensemble with a shorter time step and increased SHAKE tolerance; thermostats and barostats will effect diffusion calculations.
2. Time correlation functions become noisier as the length of the function approaches the maximum. Therefore in general one should choose parameters for the time correlation function that are much shorter than the total simulation length.

For example, given a trajectory 'mdcrd.nc' containing 10000 frames with a total simulation time of 200 ns (so the time between frames is 0.02 ns), to calculate rotational diffusion using 100 vectors using rotation matrices generated via an RMS fit to 'avgstruct.pdb', computing and integrating the time correlation function for each vector from 0 to 5 ns (1/40th of the simulation), and writing out the effective diffusion constants and results to 'deffs.dat' and 'rotdif.out' respectively:

```
reference avgstruct.pdb [avg]
rms R0 @CA,C,N,O ref [avg] savematrices
trajin mdcrd.nc
rotdif nvecs 100 rmatrix R0[RM] \
    ti 0.0 tf 5.0 dt 0.02 deffout deffs.dat \
    outfile rotdif.out
```

37.12.34. runningavg

```
runningavg <dset1> [<dset2> ...] [name <dsetname>] [out <filename>]
           [ [cumulative] | [window <window>] ]
<dset1> [<dset2> ...] Data set(s) to calculate running average for.
```

[name <dsetname>] Output running average data set name.
 [out <filename>] File to write results to.
 [cumulative] Calculate cumulative running average instead.
 [window <window>] Size in frames of window over which to calculate running average.

Calculate running average over windows of given size for data in selected data set(s).

37.12.35. slope

```
slope <dset0> [<dset1> ...] [out <outfile>] [name <name>]
      [type {forward|backward|central}]
```

<dset0> [<dset1> ...] Data set(s) to calculate finite difference for.
 [out <outfile>] File to write finite difference curves to.
 [name <name>] Output data set(s) name.
 [type {forward|backward|central}] Specify type of finite difference to calculate (default forward).
 DataSets generated:
 <name>:<idx> Output finite difference curves for each input data set (indexed from 0).

Calculate finite differences for each input data set.

37.12.36. spline

```
spline <dset0> [<dset1> ...] [out <outfile>] [meshsize <n> | meshfactor <x>]
      [meshmin <mmin>] [meshmax <mmax>]
```

<dsetX> Data set(s) to perform splining on.
 [out <outfile>] Write splined data to <outfile>.
 [meshsize<n>] Size of the mesh to use for splining.
 [meshfactor <x>] If meshsize is not given, use a mesh of data set size * <x>.
 [meshmin <mmin>] Mesh X minimum value.
 [meshmax <mmax>] Mesh X maximum value.

Apply cubic splines to the given input data sets to create new data sets.

37.12.37. statistics | stat

```
stat {<name> | ALL} [shift <value>] [out <filename>] [noeout <filename>]
      [ignoreenv] [name <noe setname>]
```

<name> Name of data set to analyze.
 ALL analyze all data sets.
 shift <value> Subtract <value> from all elements in each data set.
 [out <filename>] Write analysis results to <filename> (STDOUT if not specified).

[noeout <filename>] (Type 'noe' only) Write summary of NOE results to <filename>.

[ignorenv] (Type 'noe' only) Ignore negative NOE violations (i.e. shorter-than-expected distances).

[name <noe setname>] (Type 'noe' only) Name for output NOE data sets.

DataSet Aspects for type 'noe' output:

[R6] Averaged $1/r^6$ distance for each set.

[NViolations] Number of violations based on given bounds for each set.

[AvgViolation] $1/r^6$ averaged distance minus expected distance for each set.

[NOE names] Name of each set.

Analyze angles, dihedrals, distances, and/or puckers and calculate various properties. More specific analyses can be obtained by labelling distances/dihedrals/puckers (from e.g. the *distance*, *dihedral*, *pucker* commands or with the *dataset* command) with the 'type <label>' keyword:

dihedral type labels: alpha, beta, gamma, delta, epsilon, zeta, chi, c2p h1p, phi, psi, omega, pchi

distance type labels: noe

pucker type labels: pucker

For each input data set, the average, standard deviation, initial and final values will be reported. The cyclic nature of dihedral/pucker data sets is taken into consideration when averaging.

37.12.37.1. Torsion Analysis

A table will be written in ASCII format showing the distribution of torsion values for each data set. More specific information may be printed based on the set type. Values in the output marked SNB are from those defined by Schneider, Neidle, and Berman.[801] For more information on nucleic acid torsion as pertains to RNA see further work by Schneider et al..[802]

For example, to perform in-depth analysis on some nucleic acid dihedral angles:

```
dihedral g0 out dihedrals.dat :1@O5' :1@C5' :1@C4' :1@C3' type gamma
dihedral d0 out dihedrals.dat :1@C5' :1@C4' :1@C3' :1@O3' type delta
dihedral c0 out dihedrals.dat :1@O4' :1@C1' :1@N9 :1@C4 type chi
analyze statistics all out stat.dat
```

37.12.37.2. Distance Analysis

A table will be written in ASCII format showing the distribution of distance values < 6.5. If a distance is labeled as 'type noe' a compact time series will be printed in ASCII format showing the NOE as strong, medium, or weak. In addition the $\langle r^{-6} \rangle^{(-1/6)}$ averaged value will be reported, as well as the number of upper/lower bound violations. If 'noeout' is specified, a summary of these results will be written with format:

```
<#NOE> <R6> <Nviolation> <AvgViolation> <Name>
```

Where <#NOE> is an index, <R6> is the $\langle r^{-6} \rangle^{(-1/6)}$ averaged distance, <Nviolation> is the total number of bounds violations, <AvgViolation> is the average difference from expected distance *Rexp* when the distance is violated (note that if not explicitly set, *Rexp* is set to the upper bound when the lower bound is 0.0, or the average of upper and lower bounds otherwise), and <Name> is the data set legend.

For example, the following input could be used to check certain distances for NOE violations:

```

distance :3@HB= :10@HG= type noe noe_medium
distance :3@HE= :10@HG= type noe noe_strong
distance :3@HA :12@HA type noe noe_medium
distance :3@HD= :12@HG= type noe noe_medium
distance :3@HE= :12@HA type noe noe_strong
analyze statistics all out dpdp.noe.dat noeout noe_graph.dat name Res3_NOE

```

37.12.37.3. Pucker Analysis

A table will be written in ASCII format showing the distribution of pucker phases for each data set.

37.12.38. ti

```

ti <dset0> [<dset1> ...] {nq <n quad pts> | xvals <x values>}
[name <set name>] [out <file>] [curveout <ti curve file>]
[nskip <#s to skip>]
[avgincrement <#>] [avgmax <#>] [avgskip <#>]]
[bs_samples <samples>] [bs_points <points>] [bs_seed <#>]
[bs_fac <factor>]]
<dset0> [<dset1> ...] Data set arguments specifying input DV/DL
values.
nq <n quad pts> Number of points for Gaussian quadrature
integration. Expect one data set per point.
xvals <x values> Comma-separated list of X values for integration.
Expect one data set per value.
name <set name> Output data set name.
out <file> File to write results of integration to.
curveout <ti curve file> File to write TI curves to.
nskip <#s to skip> Comma separated list of number of points to skip.
For each number given, the TI integration will be repeated.
avgincrement <#> [avgmax <#>] [avgskip <#>] Starting from point
'avgskip' (default 0), repeat the TI integration calculation in
increments of <#> up to 'avgmax' (default all points), so
'avgincrement 10' will do points 0-10, 0-20, etc.
bs_samples <samples> [bs_points <points>] [bs_seed<#>] [bs_fac <factor>]
Estimate error via bootstrap analysis, repeating the TI
integration <samples> times using <points> points or
<factor> times the total number of points. Randomize with
given seed.

DataSet Aspects:
[TIcurve] Raw TI curve. If 'nskip' index is number of points
skipped. If bootstrapping, index is sample index. If
'avgincrement' the index is the number of points.
[SD] For bootstrap analysis, standard deviation of average free
energy over samples.

```

Calculate free energy using DV/DL energies from thermodynamic integration. The results of integration of the DV/DL curve will be written to <file>, while the curves themselves will be written to <ti curve file>. Use **nq** to specify number of Gaussian quadrature points; otherwise the lambda values should be specified by **xvals**, where <x values> is a comma-separated list.

For example, to perform Gaussian quadrature integration using data sets named 'Tldata', repeating the calculation for various number of skipped data points:

```
ti Tldata nq 9 name Curve out skip.agr curveout curve.agr nskip 0,5,10,15,20,30,40,5
```

37.12.39. tica

```
tica { crdset <COORDS set name> [mask <mask>] |
      data <input set arg1> ... }
[lag <time lag>] [map {kinetic|commute|none}]
[name <output set name>] [out <file>] [cumvarout <file>]
```

crdset <COORDS set name> Input coordinates (COORDS) data set.

mask <mask> Selected atoms in input coordinates (COORDS) data set.

data <input set arg1> Input 1D data set name(s), may specify more than once. If any data set is periodic, all need to be periodic.

lag <time lag> TICA lag time in frames.

map {kinetic|commute|none} How to transform the resulting eigenvectors.

kinetic (default) Scale eigenvectors by eigenvalues so that distances in the transformed data approximate kinetic distances; particularly useful if using the projections to cluster.

commute Scale eigenvectors by regularized time scales, $\sqrt{\text{timescale}_i / 2}$, so that distances in the transformed data will approximate commute distances. Timescales smaller than the lag time are dampened. [803]

none Do not scale eigenvectors.

name <output set name> Output data set name.

out <file> File to write TICA modes to.

cumvarout <file> File to write eigenvalue cumulative variance to.

Perform time-independent correlation analysis (TICA). Similar to principal component analysis (PCA), TICA calculates eigenvectors/eigenvalues (i.e. eigenmodes) from input data sets (either coordinates or a combination of other 1D data sets). Whereas the eigenvectors from PCA describe the variance in the input data, the eigenvectors from TICA describe the maximal autocorrelation in the input data at the given lag time. [804] The analysis can be performed on either coordinates or 1D data sets; the data sets can either be all periodic (in which case they will be converted to cos/sin form) or not.

37.12.40. timecorr

```
timecorr vec1 <vecname1> [vec2 <vecname2>] out <filename> [name <dsname>]
[order <order>] [tstep <tstep>] [tcrr <tcrr>]
[dplr] [norm] [drct] [dplrout <dplrfile>] [ptrajformat]
```

vec1 <vecname1> [vec2 <vecname2>] Vector(s) on which to operate. By default the auto-correlation function will be calculated if one vector is specified, and the cross-correlation function will be calculated if two vectors are specified.

out <filename> Name of file to write output to.

[name <dsname>] Name of output vector data sets.

[order <order>] Order of Legendre polynomials to use; default 2.

[tstep <tstep>] Time between snapshots (default 1.0).

[tcorr <tcorr>] Maximum time to calculate correlation functions for (default 10000.0).

[dplr] Output correlation functions $C_l \equiv \langle P_l(r(0)^3 r(\tau)^3) \rangle$ and $\langle 1/(r(0)^3 r(\tau)^3) \rangle$ in addition to the P_l correlation function.

[norm] Normalize all correlation functions, i.e., $C_l(t=0) = P_l(t=0) = 1.0$.

[drct] Use the direct method to calculate correlations instead of FFT; this will be much slower.

[dplROUT] (dplr only) Write extra information for each vector related to dplr option to <dplrfile>.

[ptrajformat] Write output in ptraj style (prevents use of data formatting options).

DataSet Aspects:

[P] $P<order>$ correlation function.

[C] $C<order>$ correlation function (dplr only).

DataSet Aspects for dplr only:

[R3R3] $\langle 1/(r(0)^3 r(t)^3) \rangle$ correlation function.

[R] Average magnitude ($\langle R \rangle$).

[RRIG] $\text{Sqrt}(\langle R^2 \rangle)$.

[R3] $\langle 1/R^3 \rangle$.

[R6] $\langle 1/R^6 \rangle$.

[Name] Vector name.

Calculate time auto/cross-correlation functions for vectors using spherical harmonics theory. NOTE: To calculate direct correlation functions for vectors just use the **corr** analysis command. The **norm** keyword will normalize the resulting correlation functions. Note that if **dplr** is specified, several additional data sets with aspects [R], [RRIG], [R3], [R6], and [Name] will be created containing either 1 or 2 values depending on how many vectors were specified.

Examples

Vectors between atoms 5 and 6 as well as 7 and 8 are calculated below, for which auto and cross time correlation functions are obtained.

```
vector v0 @5 @6
vector v1 @7 @8
timecorr vec1 v0 tstep 1.0 tcorr 100.0 out v0.out order 2
timecorr vec1 v1 tstep 1.0 tcorr 100.0 out v1.out order 2
timecorr vec1 v0 vec2 v1 tstep 1.0 tcorr 100.0 out v0_v1.out order 2
```

Similarly, a vector perpendicular to the plane through atoms 18, 19, and 20 is obtained and further analyzed.

```
vector v2 @18,@19,@20 corplane
timecorr vec1 v3 tstep 1.0 tcorr 100.0 out v2.out order 2
```

37.12.41. vectormath

```

vectormath vec1 <vecname1> vec2 <vecname2> [out <filename>] [norm] [name <setname>]
[ dotproduct | dotangle | crossproduct | magnitude ]

vec1 <vecname1> vec2 <vecname2> Vector(s) on which to operate.
[out <filename>] Name of file to write output to.
[dotproduct] (Default) Calculate the dot-product of the two vectors.
[dotangle] Calculate angle from dot-product between the two vectors;
vectors will be normalized.
[crossproduct] Calculate cross-product of the two vectors.
[magnitude] Calculate the magnitude of the vectors selected by vec1
(no need to specify vec2).
[norm] Normalize the vectors; this will affect any subsequent
calculations with the vectors. This is turned on automatically
if dotangle/magnitude specified.

```

Calculate dot product, angle from dot product (degrees), or cross product for specified vectors. Note that **norm** normalizes the vectors themselves; the vectors will remain normalized for subsequent calculations or output. Either **vec1** or **vec2** can be of size 1; in that case each vector in the set with N frames operates on the single vector. For example, if **vec1** is size N and **vec2** is size 1, then each frame of **vec1** is operated on the single vector from **vec2**.

For example, to get the angles between two previously calculated vectors **v1** and **v2**:

```

vectormath vec1 v1 vec2 v2 dotangle out dotproduct.dat name acos(|V1|*|V2|)

```

37.12.42. wavelet

```

wavelet [crdset <set name>] nb <n scaling vals> [s0 <s0>] [ds <ds>]
[correction <correction>] [chival <chival>] [type <wavelet>]
[out <filename>] [name <setname>]
[cluster [minpoints <#>] [epsilon <value>] [clusterout <file>]
[clustermmapout <file>] [cmapdetail] [kdist] [cprefix <PDB prefix>]
[overlay <trajfile>] [overlayparm <parmfile>]]

[crdset <set name>] COORDS data set to use
nb <n scaling vals> Number of scales. The smaller the number the
better resolution, but slower to plot.
[s0 <s0>] The smallest scale of the wavelet function (default 2dt
where dt is time between snapshots in ps )
[ds <ds>] Spacing between discrete scales. (Default is 0.25.
Smaller value of ds gives finer resolution. The largest values
that give adequate sampling in scale for Morlet and Paul are
0.5 and 1.5, respectively)
[correction <correction>] The scale-to-wavelength parameter (1.01 for
Morlet, 1.389 for Paul). Automatically set based on wavelet if
not otherwise specified.
[chival <chival>] The value of  $\chi^2_2$  at a particular confidence level
[type <wavelet>] Type of wavelet function to use <morlet> or <paul>

```

[out <filename>] Write results to file named <filename>
 [name <setname>] Store results in data set with name <setname>
 [cluster] Perform wavelet clustering i.e. wavelet feature extraction analysis.
 [minpoints <#>] Minimum number of points necessary to form a region of interest.
 [epsilon <value>] Minimum region of interest size.
 [clusterout <file>] Output for clustering (see below).
 [clustermmapout <file>] Output cluster map (recommended gnuplot format, see below).
 [cmapdetail] Instead of the map being smoothed to cluster regions, show full detail.
 [kdist] Can be used to determine minpoints and epsilon - see below.
 [cprefix <PDB prefix>] Output cluster region PDBs (only containing from minimum to maximum atom and minimum to maximum frame) with given prefix.
 [overlay <trafile>] Create a trajectory that can be overlaid with the original trajectory to highlight atoms of interest. Atoms in cluster regions will get their normal coordinates - all others are set to the common center of mass.
 [overlayparm <parmfile>] Topology that can be used with the overlay trajectory.
 <wavelet>: morlet, paul

Perform the wavelet analysis using fast Fourier transform (FFT) algorithm on specified trajectory and write out to a gnuplot-formatted file named <name.gnu>. The created Wavelet map provides a clear picture of the significant motions which are characterized both in time and space. Note that typically the trajectory in question should have rotational and translational movement removed (via e.g. the *rms* command); otherwise these will be reflected in the wavelet analysis results.

Wavelet analysis contains two main steps which performs continuous wavelet transform (CWT) and statistical significance testing as proposed by Torrence and Compo[805]. Analysis is executed on one dimensional (1-D) coordinate which is defined as the displacement from the starting position. For each atom, CWT is calculated over a specified range of scales from S_0 up to $S_0 2^{(nb-1)ds}$. To obtain the CWT of the trajectory the Fourier transform of atom's displacement and wavelets which scaled by S (S is calculated from: $S = S_0 2^{jds}$; $j = 0, 1, 2, \dots, nb-1$) is computed and then the inverse Fourier transform of the product of Fourier transforms will be calculated as the CWT. After calculating the wavelet coordinates for all atoms, a significance testing is performed to determine the significance of each wavelet coordinate. For doing this test we need to have an appropriate background spectrum to consider as a mean or expected spectrum and compare our wavelet coordinates against this background. In order to calculate the background spectrum since wavelet spectrum (according to the convolution theorem) follows the Fourier spectrum, the Fourier coefficients over every atom's displacement is calculated using the following formula and a model (μ_k) is constructed on average which Fourier coefficients fit (X_n) is the time series which is the atom's displacement and k is the frequency index[806].

$$f_{k=\frac{1}{N}} = \sum_{n=0}^{N-1} \exp\left(\frac{-2\pi i k n}{N}\right) X_n$$

This test is implemented based on the null hypothesis that the assumption is that Fourier coordinates normally distributed around the expected value, then the wavelet coordinates should also be normally distributed. Assuming the expected background spectrum and since the square of a normally distributed variable is chi-square distributed, then the distribution for the square of the absolute values of wavelet coordinates ($|W_{i,k}|^2$ is as follows (σ^2 is the variance of the atom's displacement)).

$$\sigma^2 \mu_k \chi^2 / 2$$

Then choosing a confidence level we can determine the minimum acceptable value for $|W_{i,k}|^2$ to be considered as a significant coordinates at that certain confidence level. In the final map the scales of only those wavelet coordinates which are significantly above the expected distribution are stored.

For example, to perform wavelet analysis on residues 1 to 17 with 40 scaling values starting from scaling of 0.2 with a spacing of 0.25 using the Morlet wavelet:

```
parm nowat.withions.parm7
trajin nowat.image.nc
rms :1-17@C*,N*,O*,P* first mass
wavelet nb 40 s0 0.2 ds 0.25 correction 1.01 chival 1.6094 type morlet \
      :1-17 out wavelet.gnu usemap
```

Wavelet Analysis Feature Extraction

Wavelet analysis feature extraction (WAFEX)[807] uses a density-based clustering algorithm (a modified version of the DBSCAN algorithm) to highlight physical and temporal regions that have significant motions from wavelet maps and can extract the specific atoms and frames involved in these motions for further analysis. Cluster regions shown in the map will be smoother by default for easier visualization (unless **cmapdetail** is specified). Details of the clustering are provided via the **clusterout** keyword with format:

```
#Cluster [points] [minatm] [maxatm] [minfrm] [maxfrm] [avgval]
#Cluster Cluster region number.
points Number of points in the cluster.
minatm Starting atom of the region.
maxatm End atom of the region.
minfrm Starting frame of the region.
maxfrm End frame of the region.
avgval Average value of points in the region.
```

For example, to create a 2D gnuplot map highlight regions of interest called 'cluster.gnu' one could use the following input.

```
parm ../DPDP.parm7
trajin ../DPDP.nc
rms @C,CA,N first
wavelet nb 10 s0 2 ds 0.25 type morlet correction 1.01 chival 0.25 \
      :1-22 name DPDP \
      cluster clustermapout cluster.gnu clusterout cluster.dat \
      minpoints 66 epsilon 10.0
datafile cluster.gnu usemap palette kbvyw
```

Some experimentation with **kdist** may be required to obtain reasonable values for **minpoints** and **epsilon**. See [37.12.5 on page 860](#) as well as the Heidari et al paper for further discussion.

37.13. Analysis Examples

Please note that typically for principal component analysis (PCA) the trajectory needs to be aligned against a reference structure to remove overall global and translation motion. Use the **rms** command for this.

37.13.1. Cartesian covariance matrix calculation and projection (PCA)

After calculating modes, snapshots can be projected onto these in an additional pass through the trajectory. It is very important that the snapshots used when projecting are exactly the same as those used to generate the original covariance matrix. This example takes advantage of the COORDS data set functionality in cpptraj to save snapshots for the purposes of projection.

```
# Step one. Generate average structure.
# RMS-Fit to first frame to remove global translation/rotation.
parm myparm.parm7
trajin mytraj.nc
rms first !@H=
average crdset AVG
run
# Step two. RMS-Fit to average structure. Calculate covariance matrix.
# Save the fit coordinates.
rms ref AVG !@H=
matrix covar name MyMatrix !@H=
createcrd CRD1
run
# Step three. Diagonalize matrix.
runanalysis diagmatrix MyMatrix vecs 2 name MyEvecs
# Step four. Project saved fit coordinates along eigenvectors 1 and 2
crdaction CRD1 projection evecs MyEvecs !@H= out project.dat beg 1 end 2
```

37.13.2. Dihedral covariance matrix calculation and projection for backbone phi/psi (PCA)

```
parm ../lrrb_vac.prmtop
trajin ../lrrb_vac.mdcrd
# Generation of phi/psi dihedral data
multidihedral BB phi psi resrange 2
run
# Calculate dihedral covariance matrix and obtain eigenvectors
matrix dihcovar dihedrals BB[*] out dihcovar.dat name DIH
diagmatrix DIH vecs 4 out modes.dihcovar.dat name DIHMODES
run
# Project along eigenvectors
projection evecs DIHMODES out dih.project.dat beg 1 end 4 dihedrals BB[*]
run
```

38. pytraj

38.1. Introduction

pytraj [808] is Python front end of *cpptraj*. It is written to introduce more flexibility in data analysis by combining with Python's rich ecosystems (such as *numpy*, *scipy*, *pandas*, *scikit-learn*, *ipython-notebook*, etc.). It is aimed at users who are familiar with Python and want to combine *cpptraj*'s functionality with the flexibility of Python. It is still very new, and in active development, and users should be aware that some of the syntax (i.e., the *API*) may change in future versions.

This project is not intended to replace *cpptraj*, but rather to extend its functionality by placing allowing seamless and efficient data interchange between *cpptraj* and Python. Therefore, this project is aimed at users who are either comfortable and familiar with the Python programming language or wish to become so. You should be familiar with basic programming concepts (like conditionals, loops, and arrays) and preferably Python syntax before trying to use *pytraj*. **Note:** there is no program called *pytraj*; that term is rather a shorthand for using "import *pytraj*" within a python driver script.

38.2. Development

If you are interested in contributing to the development of *pytraj*, or you want to build the source code directly, either fork or clone the repository from Github at <https://github.com/amber-md/pytraj>. Note that this method of installation is more complex, as you will need to obtain and build an updated version of *libcpptraj* (instructions can be found in the *pytraj* Github project). Also, you can check developer guide http://amber-md.github.io/pytraj/latest/developer_guide.html.

38.3. Documentation and examples

Useful links are listed below.

- The *pytraj* Github repository: <https://github.com/amber-md/pytraj>
- Comprehensive documents and tutorials can be found in <https://amber-md.github.io/pytraj>
- Example scripts: <https://github.com/amber-md/pytraj/tree/master/examples>
- We highly suggest user to use Jupyter notebook for interactive computing: <http://jupyter.org/>

38.3.1. Minimal examples

Only several highlight features of *pytraj* are shown here.

38.3.1.1. Loading trajectories to memory

```
import pytraj as pt

# load all frames into memory if filesize is small
traj = pt.load('tz2.nc', top='tz2.parm7')

# load but skip every 10 frames
```

```

traj = pt.load('tz2.nc', top='tz2.parm7', stride=10)

# load specific frame numbers
traj = pt.load('tz2.nc', top='tz2.parm7', frame_indices=[0, 8, 10, 20])

# load with given mask
traj = pt.load('tz2.nc', top='tz2.parm7', mask='@CA')

```

38.3.1.2. Lazy loading trajectories

Sometimes the trajectories are too large to load to memory, users can use `pytraj.iterload` method.

```

import pytraj as pt

traj = pt.iterload('tz2.nc', 'tz2.parm7')

# load several files
traj = pt.iterload('tz2.*.nc', 'tz2.parm7')

# load several files by explicitly giving filenames
traj = pt.iterload(['tz2.0.nc', 'tz2.1.nc'], 'tz2.parm7')

```

Please check http://amber-md.github.io/pytraj/latest/trajectory_exercise.html for further information.

38.3.1.3. Perform calculation

```

# radgyr
data = pt.radgyr(traj, mask='@CA')

# rmsd to 1st frame, mask='@CA'
data = pt.rmsd(traj, ref=0, mask='@CA')

# rmsd to specific reference
ref = traj[3]
data = pt.rmsd(traj, ref=ref, mask='@CA')

# compute distance
pt.distance(traj, ':1-3 :5-8')

# load pdb from RCSB website then perform hbond calculation
traj = pt.fetch_pdb('1l2y')
hbond_data = pt.hbond(traj)
print(hbond_data.donor_acceptor)

```

More analysis commands can be found in http://amber-md.github.io/pytraj/latest/_api/pytraj.all_actions.html.

38.3.1.4. Writing trajectory

```

import pytraj as pt

# write whole trajectory
pt.write_traj('output.nc', traj, overwrite=True)

```

```
# write specific frames
pt.write_traj('output.nc', traj, frame_indices=[0, 8, 9], overwrite=True)

# user can use save method
traj.save('output.nc')

# if you are using Ipython/Jupyter notebook (or interactive terminal), you can get help by
pt.write_traj?
# the hit <Enter>

# converting netcdf format to DCD
traj = pt.iterload('tz2.nc', 'tz2.parm7')
traj.save('tz2.dcd')
```

38.3.1.5. Combine pytraj with pysander for energy evaluation

```
import pytraj as pt
traj = pt.iterload('tz2.nc', 'tz2.parm7')

# compute energies (potential, bond, angle, dihedral, GB, ...), use igb=8 model
energy_dict = pt.energy_decomposition(traj, igb=8)

# get energy for different component
energy_dict['tot']
energy_dict['gb']
```

38.3.1.6. Parallel calculation

pytraj supports parallel calculation through multiprocessing (python) or MPI (requires mpi4py)

```
import pytraj as pt

# for parallel calculation, pytraj only supports `iterload` method.
traj = pt.iterload('tz2.nc', top='tz2.parm7')

# serial version
data = pt.radgyr(traj, '@CA')

# multiprocessing
data = pt.pmap(pt.radgyr, traj, '@CA', n_cores=6)
energy_dict = pt.pmap(pt.energy_decomposition, traj, igb=8, n_cores=6)

# user can chain a series of cpptraj commands for parallel calculation too
data = pt.pmap(['rms', 'radgyr @CA nomax', 'surf @CA'], traj, n_cores=8)

# mpi
data = pt.pmap_mpi(pt.radgyr, traj, '@CA')
```

Please check: http://amber-md.github.io/pytraj/latest/tutorials/tutorial_parallel.html

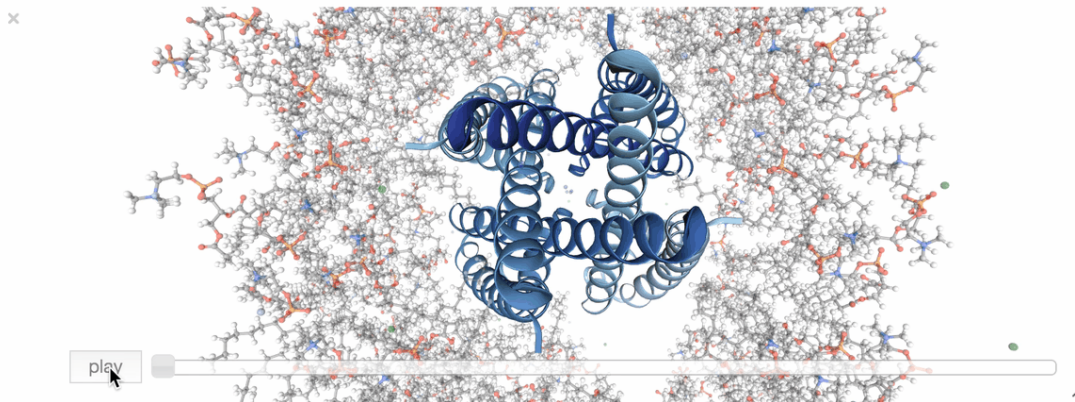
38.3.1.7. Simplify Principal Component Analysis (PCA) calculation

```
traj = pt.load('tz2.nc', 'tz2.parm7')
data = pt.pca(traj, mask='@CA', n_vecs=2)
```

38. pytraj

```
In [1]: import pytraj as pt
import nglview as nv
```

```
In [2]: traj = pt.load('sim.nc', top='sim.prmtop')
traj.strip(":TIP3")
view = nv.show_pytraj(traj)
view
```



```
In [3]: view.clear()
view.add_cartoon('protein')
view.add_ball_and_stick('not protein', opacity=0.5)
```

Figure 38.1.: Example of trajectory viewer in Jupyter notebook

```
# get document for this method
print(pt.pca.__doc__)
```

Please also check: <http://amber-md.github.io/pytraj/latest/>.

38.3.1.8. Fancy indexing of trajectory

```
# get new Trajectory by skipping every 2 frames
traj[::2]

# get new Trajectory but only keeping coordinates of CA atoms
traj['@CA']

# get new Trajectory with given frame numbers
framelist = [0, 3, 7]
traj[framelist]
```

Please see also: http://amber-md.github.io/pytraj/latest/trajectory_slice.html

38.3.1.9. Trajectory Viewer

Trajectory can be quickly viewed in Jupyter notebook by using pytraj and nglview^[809] (<https://github.com/arose/nglview>).

39. MMPBSA.py

Note: Historically, Amber has supported several scripts to carry out MM-PBSA-like calculations. The one described here (the “python” version) is more recent, generally simpler to use, and has a more active support community for answering questions. An older, “perl”, version is still in the AmberTools21 distribution, but we have removed information about it in this Reference Manual. If you have need to run this older, perl-based version, please consult the [Amber 2019 Reference Manual](#).

Neither of these should be considered as a “black-box”, and users should be familiar with Amber before attempting these sorts of calculations. These scripts automate a series of calculations, and cannot trap all the types of errors that might occur. ***You should be sure that you know how to carry out an MM-PBSA calculation “by hand” (i.e., without using the scripts);*** if you don’t understand in detail what is going on, you will have no good reason to trust the results. Also, if something goes awry (and this is not all that uncommon), you will need to run and examine the individual steps to carry out useful debugging.

39.1. Introduction

This section describes the use of the python script MMPBSA.py [810] to perform Molecular Mechanics / Poisson Boltzmann (or Generalized Born) Surface Area (MM/PB(GB)SA) calculations. This is a post-processing method in which representative snapshots from an ensemble of conformations are used to calculate the free energy change between two states (typically a bound and free state of a receptor and ligand). Free energy differences are calculated by combining the so-called gas phase energy contributions that are independent of the chosen solvent model as well as solvation free energy components (both polar and non-polar) calculated from an implicit solvent model for each species. Entropy contributions to the total free energy may be added as a further refinement. The entropy calculations can be done in either a HCT Generalized Born solvation model [200, 211] or in the gas phase using a *mmpbsa.py_nabnmode* program written in the *nab* programming language, or via the quasi-harmonic approximation in *ptraj*.

The gas phase free energy contributions are calculated by *sander* within the Amber program suite or *mmpbsa.py_energy* within the AmberTools package according to the force field with which the topology files were created. The solvation free energy contributions may be further decomposed into an electrostatic and hydrophobic contribution. The electrostatic portion is calculated using the Poisson Boltzmann (PB) equation, the Generalized Born method, or the Reference Interaction Site Model (RISM). The PB equation is solved numerically by either the *pbsa* program included with AmberTools or by the Adaptive Poisson Boltzmann Solver (APBS) program through the iAPBS interface[538] with Amber (for more information, see <http://www.poissonboltzmann.org/apbs>). The hydrophobic contribution is approximated by the LCPO method [185] implemented within *sander* or the *molsurf* method as implemented in *cpptraj*.

MM/PB(GB)SA typically employs the approximation that the configurational space explored by the systems are very similar between the bound and unbound states, so every snapshot for each species is extracted from the same trajectory file, although MMPBSA.py will accept separate trajectory files for each species. Furthermore, explicit solvent and ions are stripped from the trajectory file(s) to hasten convergence by preventing solvent-solvent interactions from dominating the energy terms. A more detailed explanation of the theory can be found in Srinivasan, et. al.[811] You may also wish to refer to reviews summarizing many of the applications of this model,[812–814] as well as to papers describing some of its applications.[815–819]

39.2. Preparing for an MM/PB(GB)SA calculation

MM/PB(GB)SA is often a very useful tool for obtaining relative free energies of binding when comparing ligands. Perhaps its biggest advantage is that it is very computationally inexpensive compared to other free energy

calculations, such as TI or FEP. Following the advice given below before any MD simulations are run will make running MMPBSA.py successfully much easier.

39.2.1. Building Topology Files

MMPBSA.py requires at least three, usually four, compatible topology files. If you plan on running MD in explicit water, you will need a solvated topology file of the entire complex, and you will always need a topology for the entire complex, one for just the receptor, and a final one for just the ligand. Moreover, they must be compatible with one another (i.e., each must have the same charges for the same atoms, the same force field must be used for all three of the required prmtops, and they must have the same PBRadii set, see LEaP for description of pbradii). Thus, it is strongly advised that all prmtop files are created with the same script. We run through a typical example here, though leave some of the details to other sections and other tutorials. We will start with a system that is a large protein binding a small, one-residue ligand. We will assume that a docked structure has already been obtained as a PDB and that two separate PDBs have been constructed, receptor.pdb and LIG.pdb. We will also assume that a MOL2 file was created from LIG.pdb, residue name 'LIG', was built with charges already derived (either through antechamber or some other method), and an frcmod file for 'LIG' that contains all missing parameters have already been created. Furthermore, we will use the FF14SB force field for this example. A sample script file called, for instance, mmpbsa_leap.in, is shown below

```
source leaprc.protein.ff14SB
source leaprc.water.tip3p
loadAmberParams LIG.frcmod
LIG = loadMol2 LIG.mol2
receptor = loadPDB receptor.pdb
complex = combine {receptor LIG}
set default PBRadii mbondi2

saveAmberParm LIG lig.top lig.crd
saveAmberParm receptor rec.top rec.crd
saveAmberParm complex com.top com.crd

solvateOct complex TIP3PBOX 15.0
saveAmberParm complex com_solvated.top com_solvated.crd
quit
```

The above script, when executed using the command

```
tLeap -f mmpbsa_leap.in
```

should produce four prmtop files, lig.top, rec.top, com.top, and com_solvated.top. Topology files created in this manner will make running MMPBSA.py far easier. This is, of course, the simplest case, but we briefly describe some other examples. MMPBSA.py will guess the mask for both the receptor and ligand inside the complex topology file as long as the ligand residues appear continuously in the complex topology file. Therefore, if you're adding two ligands, combine them consecutively in the complex (rather than one residue at the beginning and one at the end, for instance). If you have done this, you should allow MMPBSA.py to guess the masks since it provides a good error check.

39.2.2. Using ante-MMPBSA.py

ante-MMPBSA.py is a python utility that allows you to create compatible complex, receptor, and ligand topology files from a solvated topology file, or compatible receptor and ligand topology files from a complex topology file. The usage statement for ante-MMPBSA.py is

```
Usage: ante-MMPBSA.py [options]
Options:
```



```

-h, --help          show this help message and exit
-p PRMTOPTOP, --prmtop=PRMTOPTOP
                    Input "dry" complex topology or solvated complex
                    topology
-c COMPLEX, --complex-prmtop=COMPLEX
                    Complex topology file created by stripping PRMTOPTOP of
                    solvent
-r RECEPTOR, --receptor-prmtop=RECEPTOR
                    Receptor topology file created by stripping COMPLEX of
                    ligand
-l LIGAND, --ligand-prmtop=LIGAND
                    Ligand topology file created by stripping COMPLEX of
                    receptor
-s STRIP_MASK, --strip-mask=STRIP_MASK
                    Amber mask of atoms needed to be stripped from PRMTOPTOP
                    to make the COMPLEX topology file
-m RECEPTOR_MASK, --receptor-mask=RECEPTOR_MASK
                    Amber mask of atoms needed to be stripped from COMPLEX
                    to create RECEPTOR. Cannot specify with -n/--ligand-
                    mask
-n LIGAND_MASK, --ligand-mask=LIGAND_MASK
                    Amber mask of atoms needed to be stripped from COMPLEX
                    to create LIGAND. Cannot specify with -m/--receptor-
                    mask
--radii=RADIUS_SET  PB/GB Radius set to set in the generated topology
                    files. This is equivalent to "set PBRadii <radius>" in
                    LEaP. Options are bondi, mbondi2, mbondi3, amber6, and
                    mbondi and the default is to use the existing radii.

```

The input prmtop is required. It can either be a solvated, complex topology file or a complex topology file with no solvent present. If a strip_mask is given, you must also provide a complex topology file, and that complex topology file will be created by stripping strip_mask from the input prmtop. If you wish to create receptor and ligand topology files (you must create both or neither), provide BOTH a -receptor-prmtop and a -ligand-prmtop file name, as well as only ONE of either -receptor-mask or -ligand-mask. Whichever mask you do NOT define will be defined as the negated mask that you DID provide.

In short, you can use ante-MMPBSA.py to strip solvent from your prmtop for 3 applications.

1. Strip solvent from a solvated topology file and write out a non-solvated topology file.
2. Create ligand and receptor topologies from a complex topology by removing a given ligand or receptor mask.
3. A combination of 1 and 2 in the same command.

39.2.3. Running Molecular Dynamics

Not many details will be given here because MM/PB(GB)SA is a post-processing trajectory analysis technique. Molecular dynamics are run to generate an ensemble of snapshots upon which to calculate the binding energy. This technique is most effective when the structures are not correlated, which means that the simulated time between extracted snapshots should be sufficiently large to avoid such correlation.

There are two techniques that can be employed when running these simulations with respect to MMPBSA.py. The first is what's called the "single trajectory protocol" and the second of which is called the "multiple trajectory protocol". The first method will extract the snapshots for the complex, receptor, and ligand from the same trajectory. This is a faster method because it requires the simulation of only a single system, but makes the assumption that the configurational space explored by the receptor and ligand is unchanged between the bound and unbound

states. The latter method eliminates this assumption at the cost of more simulations. MMPBSA.py requires a complex trajectory, but will accept a receptor and/or ligand trajectory as well. Any trajectory not given to the script will be extracted from the complex trajectory.

39.3. Running MMPBSA.py

39.3.1. The input file

The input file was designed to be as syntactically similar to other programs in Amber as possible. The input file has the same namelist structure as both *sander* and *pmemd*. The allowed namelists are `&general`, `&gb`, `&pb`, `&ris`, `&alanine_scanning`, `&nmode`, and `&decomp`. The input variables recognized in each namelist are described below, but those in `&general` are typically variables that apply to all aspects of the calculation. The `&gb` namelist is unique to Generalized Born calculations, `&pb` is unique to Poisson Boltzmann calculations, `&ris` is unique to 3D-RISM calculations, `&alanine_scanning` is unique to alanine scanning calculations, `&nmode` is unique to the normal mode calculations used to approximate vibrational entropies, and `&decomp` is unique to the decomposition scheme. All of the input variables are described below according to their respective namelists. Integers and floating point variables should be typed as-is while strings should be put in either single- or double-quotes. All variables should be set with “variable = value” and separated by commas. See the examples below. Variables will usually be matched to the minimum number of characters required to uniquely identify that variable within that namelist. Variables require at least 4 characters to be matched unless that variable name has fewer than 4 characters (in which case the whole variable name is required). For example, “star” in `&general` will match “startframe”. However, “stare” and “sta” will match nothing.

&general namelist variables

debug_printlevel MMPBSA.py prints errors by raising exceptions, and not catching fatal errors. If `debug_printlevel` is set to 0, then detailed tracebacks (effectively the call stack showing exactly where in the program the error occurred) is suppressed, so only the error message is printed. If `debug_printlevel` is set to 1 or higher, all tracebacks are printed, which aids in debugging of issues. Default: 0. (Advanced Option)

endframe The frame from which to stop extracting snapshots from the full, concatenated trajectory comprised of every trajectory file supplied on the command-line. (Default = 9999999)

entropy Specifies whether or not a quasi-harmonic entropy approximation is made with *ptraj*. Allowed values are 0: Don't. 1: Do (Default = 0)

interval The offset from which to choose frames from each trajectory file. For example, an interval of 2 will pull every 2nd frame beginning at `startframe` and ending less than or equal to `endframe`. (Default = 1)

keep_files The variable that specifies which temporary files are kept. All temporary files have the prefix “_MMPBSA.” prepended to them (unless you change the prefix on the command-line—see subsection Subsection 39.3.2 for details). Allowed values are 0, 1, and 2.

0: Keep no temporary files

1: Keep all generated trajectory files and mdout files created by *sander* simulations

2: Keep all temporary files. Temporary files are only deleted if MMPBSA.py completes successfully

(Default = 1) A verbose level of 1 is sufficient to use `-rewrite-output` and recreate the output file without rerunning any simulations.

ligand_mask The mask that specifies the ligand residues within the complex *prmtop* (NOT the solvated *prmtop* if there is one). The default guess is generally sufficient and will only fail as stated above. You should use the default mask assignment if possible because it provides a good error catch. This follows the same description as the `receptor_mask` above.

netcdf Specifies whether or not to use NetCDF trajectories internally rather than writing temporary ASCII trajectory files. NOTE: NetCDF trajectories can be used as input for MMPBSA.py regardless of what this variable

is set to, but NetCDF trajectories are faster to write and read. For very large trajectories, this could offer significant speedups, and requires less temporary space. However, this option is incompatible with alanine scanning. Default value is 0.

0: Do NOT use temporary NetCDF trajectories

1: Use temporary NetCDF trajectories

receptor_mask The mask that specifies the receptor residues within the complex prmtop (NOT the solvated prmtop if there is one). The default guess is generally sufficient and will only fail if the ligand residues are not found in succession within the complex prmtop. You should use the default mask assignment if possible because it provides a good error catch. It uses the “Amber mask” syntax described elsewhere in this manual. This will be replaced with the default receptor_mask if ligand_mask (below) is not also set.

search_path Advanced option. By default, MMPBSA.py will only search for executables in \$AMBERHOME/bin. To enable it to search for binaries in your full PATH if they can't be found in \$AMBERHOME/bin, set search_path to 1. Default 0 (do not search through the PATH). This is particularly useful if you are using an older version of *sander* that is not in AMBERHOME.

startframe The frame from which to begin extracting snapshots from the full, concatenated trajectory comprised of every trajectory file placed on the command-line. This is always the first frame read. (Default = 1)

strip_mask The variable that specifies which atoms are stripped from the trajectory file if a *solvated_prmtop* is provided on the command-line. See 39.3.2. (Default = “:WAT:CL:CIO:CS:IB:K:LI:MG:NA:RB”)

use_sander Forces MMPBSA.py to use *sander* for energy calculations, even when *mmpbsa_py_energy* will suffice (Default 0)

0 - Use *mmpbsa_py_energy* when possible

1 - Always use *sander*

full_traj This variable is for calculations performed in parallel to control whether complete trajectories are made of the complex, receptor, and ligand. In parallel calculations, a different trajectory is made for each processor to analyze only the selected frames for that processor. A value of 0 will only create the intermediate trajectories analyzed by each processor, while a value of 1 will additionally combine those trajectories to make a single trajectory of all frames analyzed across all processors for the complex, receptor, and ligand. (Default = 0)

verbose The variable that specifies how much output is printed in the output file. There are three allowed values: 0, 1, and 2. A value of 0 will simply print difference terms, 1 will print all complex, receptor, and ligand terms, and 2 will also print bonded terms if one trajectory is used. (Default = 1)

&gb namelist variables

ifqnt Specifies whether a part of the system is treated with quantum mechanics. 1: Use QM/MM, 0: Potential function is strictly classical (Default = 0). This functionality requires *sander*

igb Generalized Born method to use (seeSection 4). Allowed values are 1, 2, 5, 7, 8 and 66. (Default = 66) All models are now available with both *mmpbsa_py_energy* and *sander*. A new generalized Born model is now available (see section 5). The corresponding value is 66.

qm_residues Comma- or semicolon-delimited list of complex residues to treat with quantum mechanics. All whitespace is ignored. All residues treated with quantum mechanics in the complex must be treated with quantum mechanics in the receptor or ligand to obtain meaningful results. If the default masks are used, then MMPBSA.py will figure out which residues should be treated with QM in the receptor and ligand. Otherwise, skeleton mdin files will be created and you will have to manually enter qmmask in the ligand and receptor topology files. There is no default, this must be specified.

qm_theory Which semi-empirical Hamiltonian should be used for the quantum calculation. No default, this must be specified. See its description in the QM/MM section of the manual for options.

qmcharge.com The charge of the quantum section for the complex. (Default = 0)

qmcharge.lig The charge of the quantum section of the ligand. (Default = 0)

qmcharge.rec The charge of the quantum section for the receptor. (Default = 0)

qmcut The cutoff for the qm/mm charge interactions. (Default = 9999.0)

saltcon Salt concentration in Molarity. It is converted to mM for GBNSR6. (Default = 0.0)

surfoff Offset to correct (by addition) the value of the non-polar contribution to the solvation free energy term (Default 0.0)

surften Surface tension value (Default = 0.0072). Units in $kcal/mol/\text{\AA}^2$

molsurf When set to 1, use the molsurf algorithm to calculate the surface area for the nonpolar solvation term. When set to 0, use LCPO (Linear Combination of Pairwise Overlaps). (Default 0)

probe Radius of the probe molecule (supposed to be the size of a solvent molecule), in Angstroms, to use when determining the molecular surface (only applicable when molsurf is set to 1). Default is 1.4.

msoffset Offset to apply to the individual atomic radii in the system when calculating the molsurf surface. See the description of the molsurf action command in *cpptraj*. Default is 0.

&pb namelist variables

inp Nonpolar optimization method (Default = 2).

cavity_offset Offset value used to correct non-polar free energy contribution (Default = -0.5692) This is not used for APBS.

cavity_surften Surface tension. (Default = 0.0378 kcal/mol Angstrom²). Unit conversion to *kJ* done automatically for APBS.

exdi External dielectric constant (Default = 80.0).

indi Internal dielectric constant (Default = 1.0).

fillratio The ratio between the longest dimension of the rectangular finite-difference grid and that of the solute (Default = 4.0).

scale Resolution of the Poisson Boltzmann grid. It is equal to the reciprocal of the grid spacing. (Default = 2.0)

istrng Ionic strength in Molarity. It is converted to mM for PBSA and kept as M for APBS. (Default = 0.0)

linit Maximum number of iterations of the linear Poisson Boltzmann equation to try (Default = 1000)

prbrad Solvent probe radius in Angstroms. Allowed values are 1.4 and 1.6 (Default = 1.4)

radiopt The option to set up atomic radii according to 0: the prmtop, or 1: pre-computed values (see Amber manual for more complete description). (Default = 1)

sander_apbs Option to use APBS for PB calculation instead of the built-in PBSA solver. This will work only through the iAPBS interface[538] built into sander.APBS. Instructions for this can be found online at the iAPBS/APBS websites. Allowed values are 0: Don't use APBS, or 1: Use sander.APBS. (Default = 0)

memopt Turn on membrane protein support (Default = 0).

emem Membrane dielectric constant (Default = 1.0).

mthick Membrane thickness (Default = 40.0).

mctrdz Absolute membrane center in the z-direction (Default=0.0, use protein center as the membrane center).

poretype Turn on the automatic membrane channel/pore finding method (Default=1).

A more thorough description of these and other options can be found in Chapter 6. Please also note that the default options have changed over time. For a detailed discussion of all related options on the quality of the MM/PBSA calculations, please refer to our recent publication [258].

&alanine_scanning namelist variables

mutant_only Option to perform specified calculations only for the mutants. Allowed values are 0: Do mutant and original or 1: Do mutant only (Default = 0)

Note that all calculation details are controlled in the other namelists, though for alanine scanning to be performed, the namelist must be included (blank if desired)

&nmode namelist variables

dielc Distance-dependent dielectric constant (Default = 1.0)

drms Convergence criteria for minimized energy gradient. (Default = 0.001)

maxcyc Maximum number of minimization cycles to use per snapshot in sander. (Default = 10000)

nminterval* Offset from which to choose frames to perform nmode calculations on (Default = 1)

nmendframe* Frame number to stop performing nmode calculations on (Default = 1000000)

nmode_igb Value for Generalized Born model to be used in calculations. Options are 0: Vacuum, 1: HCT GB model [200, 211] (Default 1)

nmode_istrng Ionic strength to use in nmode calculations. Units are Molarity. Non-zero values are ignored if *nmode_igb* is 0 above. (Default = 0.0)

nmstartframe* Frame number to begin performing nmode calculations on (Default = 1)

* These variables will choose a subset of the frames chosen from the variables in the &general namelist. Thus, the “trajectory” from which snapshots will be chosen for nmode calculations will be the collection of snapshots upon which the other calculations were performed.

&decomp namelist variables

csv_format Print the decomposition output in a Comma-Separated-Variable (CSV) file. CSV files open natively in most spreadsheets. If set to 1, this variable will cause the data to be written out in a CSV file, and standard error of the mean will be calculated and included for all data. If set to 0, the standard, ASCII format will be used for the output file. Default is 1 (CSV-formatted output file)

dec_verbose Set the level of output to print in the decmop_output file.

0 - DELTA energy, total contribution only

1 - DELTA energy, total, sidechain, and backbone contributions

2 - Complex, Receptor, Ligand, and DELTA energies, total contribution only

3 - Complex, Receptor, Ligand, and DELTA energies, total, sidechain, and backbone contributions

Note: If the values 0 or 2 are chosen, only the Total contributions are required, so only those will be printed to the mdout files to cut down on the size of the mdout files and the time required to parse them. However, this means that -rewrite-output cannot be used to change the default verbosity to print out sidechain and/or backbone energies, but it can be used to reduce the amount of information printed to the final output. The parser will extract as much information from the mdout files as it can, but will complain and quit if it cannot find everything it's being asked for.

Default = 0

idecomp Energy decomposition scheme to use:

- 1 - Per-residue decomp with 1-4 terms added to internal potential terms
 - 2 - Per-residue decomp with 1-4 EEL added to EEL and 1-4 VDW added to VDW potential terms.
 - 3 - Pairwise decomp with 1-4 terms added to internal potential terms
 - 4 - Pairwise decomp with 1-4 EEL added to EEL and 1-4 VDW added to VDW potential terms
- (No default. This must be specified!) This functionality requires *sander*.

print.res Select residues from the complex prmtop to print. The receptor/ligand residues will be automatically figured out if the default mask assignments are used. If you specify your own masks, you will need to modify the mdin files created by MMPBSA.py and rerun MMPBSA.py with the `-use-mdins` flag. Note that the DELTAs will not be computed in this case. This variable accepts a sequence of individual residues and/or ranges. The different fields must be either comma- or semicolon-delimited. For example: `print.res = "1, 3-10, 15, 100"`, or `print.res = "1; 3-10; 15; 100"`. Both of these will print residues 1, 3 through 10, 15, and 100 from the complex prmtop and the corresponding residues in either the ligand and/or receptor prmtops. (Default: print all residues)*

* Please note: Using `idecomp=3` or `4` (pairwise) with a very large number of printed residues and a large number of frames can quickly create very, very large temporary mdout files. Large print selections also demand a large amount of memory to parse the mdout files and write decomposition output file (~500 MB for just 250 residues, since that's 62500 pairs!) It is not unusual for the output file to take a significant amount of time to print if you have a lot of data. This is most applicable to pairwise decomp, since the amount of data scales as $O(N^2)$.

&rism namelist variables*

buffer Minimum distance between solute and edge of solvation box. Specify this with `grdspc` below. Mutually exclusive with `ng` and `solvbox`. Set `buffer < 0` if you wish to use `ng` and `solvbox`. (Default = 14 Å)

closure The approximation to the closure relation. Allowed choices are *kh* (Kovalenko-Hirata), *hmc* (Hypernetted-chain), or *pse**n* (Partial Series Expansion of order-*n*) where "*n*" is a positive integer (e.g., "*pse3*"). (Default = 'kh')

closureorder (Deprecated) The order at which the PSE-*n* closure is truncated if closure is specified as "*pse*" or "*pse**n*" (no integers). (Default = 1)

grdspc Grid spacing of the solvation box. Specify this with `buffer` above. Mutually exclusive with `ng` and `solvbox`. (Default = 0.5 Å)

ng Number of grid points to use in the x, y, and z directions. Used only if `buffer < 0`. Mutually exclusive with `buffer` and `grdspc` above, and paired with `solvbox` below. No default, this must be set if `buffer < 0`. Define like "`ng=1000,1000,1000`"

polardecomp Decompose the solvation free energy into polar and non-polar contributions. Note that this will increase computation time by roughly 80%. 0: Don't decompose solvation free energy. 1: Decompose solvation free energy. (Default = 0)

rism_verbose Level of output in temporary RISM output files. May be helpful for debugging or following convergence. Allowed values are 0 (just print the final result), 1 (additionally prints the total number of iterations for each solution), and 2 (additionally prints the residual for each iteration and details of the MDIIS solver). (Default = 0)

solvbox Length of the solvation box in the x, y, and z dimensions. Used only if `buffer < 0`. Mutually exclusive with `buffer` and `grdspc` above, and paired with `ng` above. No default, this must be set if `buffer < 0`. Define like "`solvbox=20,20,20`"

solvcut Cutoff used for solute-solvent interactions. The default is the value of `buffer`. Therefore, if you set `buffer < 0` and specify `ng` and `solvbox` instead, you must set `solvcut` to a nonzero value or the program will quit in error. (Default = `buffer`)

thermo Which thermodynamic equation you want to use to calculate solvation properties. Options are “std”, “gf”, or “both” (case-INsensitive). “std” uses the standard closure relation, “gf” uses the Gaussian Fluctuation approximation, and “both” will print out separate sections for both. (Default = “std”). Note that all data are printed out for each RISM simulation, so no choice is any more computationally demanding than another. Also, you can change this option and use the -rewrite-output flag to obtain a different printout after-the-fact.

tolerance Upper bound of the precision requirement used to determine convergence of the self-consistent solution. This has a strong effect on the cost of 3D-RISM calculations. (Default = 1e-5).

* 3D-RISM calculations are performed with the rism3d.snglpnt program built with AmberTools, written by Tyler Luchko. It is the most expensive, yet most statistically mechanically rigorous solvation model available in MMPBSA.py. See Chapter 8 for a more thorough description of options and theory. A list of references can be found there, too. One advantage of 3D-RISM is that an arbitrary solvent can be chosen; you just need to change the xvfile specified on the command line (see 39.3.2).

Sample input files

Sample input file for GB and PB calculation

```
&general
  startframe=5, endframe=100, interval=5,
  verbose=2, keep_files=0,
/
&gb
  igb=5, saltcon=0.150,
/
&pb
  istrng=0.15, fillratio=4.0
/
```

Sample input file for Alanine scanning

```
&general
  verbose=2,
/
&gb
  igb=2, saltcon=0.10
/
&alanine_scanning
/
```

Sample input file with nmode analysis

```
&general
  startframe=5, endframe=100, interval=5,
  verbose=2, keep_files=2,
/
&gb
  igb=5, saltcon=0.150,
/
&nmode
  nmstartframe=2, nmendframe=20, nminterval=2,
  maxcyc=50000, drms=0.0001,
/
```

Sample input file with decomposition analysis

```

&general
    startframe=5, endframe=100, interval=5,
/
&gb
    igb=5, saltcon=0.150,
/
&decomp
    idecomp=2, dec_verbose=3,
    print_res="20, 40-80, 200"
/
-----
Sample input file for QM/MMGBSA
&general
    startframe=5, endframe=100, interval=5,
/
&gb
    igb=5, saltcon=0.100, ifqnt=1, qmcharge=0,
    qm_residues="100-105, 200", qm.theory="PM3"
/
-----
Sample input file for MM/3D-RISM
&general
    startframe=5, endframe=100, interval=5,
/
&rism
    polardecomp=1, thermo="gf"
/
-----
Sample input file for MMPBSA with membrane proteins
&general
    use_sander=1,
    startframe=1, endframe=100, interval=1,
    keep_files=0, debug.printlevel=2
/
&pb
    radiopt=0, indi=20.0, istrng=0.150,
    fillratio=1.25, ipb=1, nfocus=1,
    bcopt=10, eneo=1, cutfd=7.0, cutnb=99.0,
    npbverb=1, solvopt=2, inp=1,
    memopt=1, emem=7.0, mctrdz=-10.383, mthick=36.086, poretype=1,
    maxarcdot=15000
/

```

A few important notes about input files. Comments are allowed by placing a # at the beginning of the line (whitespace is ignored). Variable initialization may span multiple lines. In-line comments (i.e., putting a # for a comment after a variable is initialized in the same line) is not allowed and will result in an input error. Variable declarations must be comma-delimited, though all whitespace is ignored. Finally, all lines between namelists are ignored, so comments may be put before each namelist without using #.

39.3.2. Calling MMPBSA.py from the command-line

MMPBSA.py is invoked through the command line as follows:

Usage: `MMPBSA.py [Options]`

Options:

```

--help, -h, --h, -H
    show this help message and exit
-O
    Overwrite existing output files
-i
    input_file
    MM/PBSA input file
-o
    output_file
    Final MM/PBSA statistics file. Default
    FINAL_RESULTS_MMPBSA.dat
-sp
    solvated_prmtop
    Solvated complex topology file
-cp
    complex_prmtop
    Complex topology file. Default ``complex_prmtop``
-rp
    receptor_prmtop
    Receptor topology file
-lp
    ligand_prmtop
    Ligand topology file
-y
    mdcrd1,mdcrd2,...,mdcrdN
    Input trajectories to analyze. Default mdcrd
-do
    decompout
    Decomposition statistics summary file. Default
    FINAL_DECOMP_MMPBSA.dat
-eo
    energyout
    CSV-format output of all energy terms for every frame in
    every calculation. File name forced to end in .csv
-deo
    dec_energies
    CSV-format output of all decomposition energy terms for
    every frame. File name forced to end in .csv
-yr
    receptor_mdcrd1,receptor_mdcrd2,...,receptor_mdcrdN
    Receptor trajectory file for multiple trajectory approach
-yl
    ligand_mdcrd1,ligand_mdcrd2,...,ligand_mdcrdN
    Ligand trajectory file for multiple trajectory approach
-mc
    mutant_complex_prmtop
    Alanine scanning mutant complex topology file
-ml
    mutant_ligand_prmtop
    Alanine scanning mutant ligand topology file
-mr
    mutant_receptor_prmtop
    Alanine scanning mutant receptor topology file
-slp
    solvated_ligand_prmtop
    Solvated ligand topology file
-srp
    solvated_receptor_prmtop
    Solvated receptor topology file
-xvfile
    xvfile
    XVV file for 3D-RISM. Default
    $AMBERHOME/dat/mmpbsa/spc.xvv
-prefix
    prefix
    Beginning of every intermediate file name generated
-make-mdins
    Create the Input files for each calculation and quit
-use-mdins
    Use existing input files for each calculation
-rewrite-output

```

```

        Don't rerun any calculations, just parse existing output
        files
--clean
        Clean temporary files from previous run
-A memhead
        Choose which membrane head group atom (memhead) to use in the
        automatic setting of membrane thickness and location. Only effective
        when memopt is set to be larger than 0 in the input. Default to 'P'.
        Three choices are available: 'P', 'N', and 'NP'. 'P' means phospho
        atoms (Amber mask P31). 'N' means nitrogen atoms (Amber mask N31).
        'NP' means to use both nitrogen and phosphorus atoms. If the input is
        'False' or 'mctrdz' and 'mthick' are defined in the input, this
        option does not take effect.

```

`-make-mdins` and `-use-mdins` are intended to give added flexibility to user input. If the MM/PBSA input file does not expose a variable you require, you may use the `-make-mdins` flag to generate the MDIN files and then quit. Then, edit those MDIN files, changing the variables you need to, then running MMPBSA.py with `-use-mdins` to use those modified files.

`--clean` will remove all temporary files created by MMPBSA.py in a previous calculation.

`--version` will display the program version and exit.

39.3.3. Running MMPBSA.py

39.3.3.1. Serial version

This version is installed with Amber during the serial install of AmberTools. `AMBERHOME` must be set, or it will quit on error. If any changes are made to the modules, MMPBSA.py must be remade so the updated modules are found by MMPBSA.py. An example command-line call is shown below:

```
MMPBSA.py -O -i mmpbsa.in -cp com.top -rp rec.top -lp lig.top -y traj.crd
```

The tests, found in `${AMBERHOME}/test/mmpbsa.py` provide good examples for running MMPBSA.py calculations.

39.3.3.2. Parallel (MPI) version

This version is installed with Amber during the parallel install. The python package `mpi4py` is included with the MMPBSA.py source code and must be successfully installed in order to run the MPI version of MMPBSA.py. It is run in the same way that the serial version is above, except MPI directions must be given on the command line as well. Note, if `mpi4py` does not install correctly, you must install it yourself in order to use `MMPBSA.py.MPI`. One note: at a certain level, running RISM in parallel may actually hurt performance, since previous solutions are used as an initial guess for the next frame, hastening convergence. Running in parallel loses this advantage. Also, due to the overhead involved in which each thread is required to load every topology file when calculating energies, parallel scaling will begin to fall off as the number of threads reaches the number of frames. A usage example is shown below:

```
mpirun -np 2 MMPBSA.py.MPI -O -i mmpbsa.in -cp com.top -rp rec.top \
      -lp lig.top -y traj.crd
```

39.3.4. Types of calculations you can do

There are many different options for running MMPBSA.py. Among the types of calculations you can do are:

1. Normal binding free energies, with either PB or GB implicit solvent models. Each can be done with either 1, 2, or 3 different trajectories, but the complex, receptor, and ligand topology files must all be defined. The complex mdcrd must always be provided. Whichever trajectories of the receptor and/or ligand that are NOT specified will be extracted from the complex trajectory. This allows a 1-, 2-, or 3-trajectory analysis. All PB calculations and GB models can be performed with just AmberTools via the *mmpbsa.py_energy* program installed with MMPBSA.py.
2. Stability calculations with any calculation type. If you only specify the complex prmtop (and leave receptor and ligand prmtop options blank), then a “stability” calculation will be performed, and you will get statistics based on only a single system. Any additional receptor or ligand information given will be ignored, but note that if receptor and/or ligand topologies are given, it will no longer be considered a stability calculation. The previous statement refers principally to mutated receptor/ligand files or extra ligand/receptor trajectory files.
3. Alanine scanning with either PB or GB implicit solvent models. All trajectories will be mutated to match the mutated topology files, and whichever calculations that would be carried out for the normal systems are also carried out for the mutated systems. Note that only 1 mutation is allowed per simulation, and it must be to an alanine. If *mutant_only* is not set to 1, differences resulting from the mutations are calculated. This option is incompatible with intermediate NetCDF trajectories (see the *netcdf = 1* option above). This has the same program requirements as option 1 above.
4. Entropy corrections. An entropy term can be added to the free energies calculated above using either the quasi-harmonic approximation or the normal mode approximation. Calculations will be done for the normal and mutated systems (alanine scanning) as requested. Normal mode calculations are done with the *mmpbsa.py_nabnmode* program included with AmberTools.
5. Decomposition schemes. The energy terms will be decomposed according to the decomposition scheme outlined in the *idecomp* variable description. This should work with all of the above, though entropy terms cannot be decomposed. APBS energies cannot be decomposed, either. Neither can PBSA surface area terms. This functionality requires *sander* from the Amber 11 (or later) package.
6. QM/MMGBSA. This is a binding free energy (or stability calculation) using the Generalized Born solvent model allowing you to treat part of your system with a quantum mechanical Hamiltonian. See “Advanced Options” for tips about optimizing this option. This functionality requires *sander* from the Amber package.
7. MM/3D-RISM. This is a binding free energy (or stability calculation) using the 3D-RISM solvation model. This functionality is performed with *rism3d.snglpnt* built with AmberTools.
8. Membrane Protein MMPBSA. Calculate the MMPBSA binding free energy for a ligand bound to a protein that is embedded into a membrane. Only *use_sander=1* is supported.

39.3.5. The Output File

The header of the output file will contain information about the calculation. It will show a copy of the input file as well as the names of all files that were used in the calculation (topology files and coordinate file(s)). If the masks were not specified, it prints its best guess so that you can verify its accuracy, along with the residue name of the ligand (if it is only a single residue).

The energy and entropy contributions are broken up into their components as they are in *sander* and *nmode* or *ptraj*. The contributions are further broken into G_{gas} and G_{solv} . The polar and non-polar contributions are EGB (or EPB) and ESURF (or ECAVITY / ENPOLAR), respectively for GB (or PB) calculations.

By default, bonded terms are not printed for any one-trajectory simulation. They are computed and their differences calculated, however. They are not shown (nor included in the total) unless specifically asked for because they should cancel completely. A single trajectory does not produce any differences between bond lengths, angles, or dihedrals between the complex and receptor/ligand structures. Thus, when subtracted they cancel completely. This includes the BOND, ANGLE, DIHED, and 1-4 interactions. If inconsistencies are found, these values are displayed and inconsistency warnings are printed. When this occurs the results are generally useless. Of course this does not hold for the multiple trajectory protocol, and so all energy components are printed in this case.

Finally, all warnings generated during the calculation that do not result in fatal errors are printed after calculation details but before any results.

39.3.6. Temporary Files

MMPBSA.py creates working files during the execution of the script beginning with the prefix `_MMPBSA_`. The variable “keep_files” controls how many of these files are kept after the script finishes successfully. If the script quits in error, all files will be kept. You can clean all temporary files from a directory by running `MMPBSA -clean` described above.

If MMPBSA.py does not finish successfully, several of these files may be helpful in diagnosing the problem. For that reason, every temporary file is described below. In general, one should begin ones problem investigation by examining the output files of the first failed calculation. Note that not every temporary file is generated in every simulation. At the end of each description, the lowest value of “keep_files” that will retain this file will be shown in parentheses.

`_MMPBSA_gb.mdin` Input file that controls the GB calculation done in *sander*. (2)

`_MMPBSA_pb.mdin` Input file that controls the PB calculation done in *sander*. (2)

`_MMPBSA_gb_decomp_com.mdin` Input file that controls the GB decomp calculation for the complex done in *sander*. (2)

`_MMPBSA_gb_decomp_rec.mdin` Input file that controls the GB decomp calculation for the receptor done in *sander*. (2)

`_MMPBSA_gb_decomp_lig.mdin` Input file that controls the GB decomp calculation for the ligand done in *sander*. (2)

`_MMPBSA_pb_decomp_com.mdin` Input file that controls the PB decomp calculation for the complex done in *sander*. (2)

`_MMPBSA_pb_decomp_rec.mdin` Input file that controls the PB decomp calculation for the receptor done in *sander*. (2)

`_MMPBSA_pb_decomp_lig.mdin` Input file that controls the PB decomp calculation for the ligand done in *sander*. (2)

`_MMPBSA_gb_qmmm_com.mdin` Input file that controls the GB QM/MM calculation for the complex done in *sander*. (2)

`_MMPBSA_gb_qmmm_rec.mdin` Input file that controls the GB QM/MM calculation for the receptor done in *sander*. (2)

`_MMPBSA_gb_qmmm_lig.mdin` Input file that controls the GB QM/MM calculation for the ligand done in *sander*. (2)

`_MMPBSA_complex.mdcrd.#` Trajectory file(s) that contains only those complex snapshots that will be processed by MMPBSA.py. (1)

`_MMPBSA_ligand.mdcrd.#` Trajectory file(s) that contains only those ligand snapshots that will be processed by MMPBSA.py. (1)

`_MMPBSA_receptor.mdcrd.#` Trajectory file(s) that contains only those receptor snapshots that will be processed by MMPBSA.py. (1)

`_MMPBSA_complex.nc.#` Same as `_MMPBSA_complex.mdcrd.#`, except in the NetCDF format. (1)

`_MMPBSA_receptor.nc.#` Same as `_MMPBSA_receptor.mdcrd.#`, except in the NetCDF format. (1)

`_MMPBSA_ligand.nc`.# Same as `_MMPBSA_ligand.mdcrd`., except in the NetCDF format. (1)

`_MMPBSA_dummycomplex.inpcrd` Dummy inpcrd file generated by `_MMPBSA_complexinpcrd.in` for use with `imin=5` functionality in *sander*. (1)

`_MMPBSA_dummyreceptor.inpcrd` Same as above, but for the receptor. (1)

`_MMPBSA_dummyligand.inpcrd` Same as above, but for the ligand. (1)

`_MMPBSA_complex.pdb` Dummy PDB file of the complex required to set molecule up in nab programs

`_MMPBSA_receptor.pdb` Dummy PDB file of the receptor required to set molecule up in nab programs

`_MMPBSA_ligand.pdb` Dummy PDB file of the ligand required to set molecule up in nab programs

`_MMPBSA_complex_nm.mdcrd`.# Trajectory file(s) for each thread with snapshots used for normal mode calculations on the complex. (1)

`_MMPBSA_receptor_nm.mdcrd`.# Trajectory file for each thread with snapshots used for normal mode calculations on the receptor. (1)

`_MMPBSA_ligand_nm.mdcrd`.# Trajectory file for each thread with snapshots used for normal mode calculations on the ligand. (1)

`_MMPBSA_ptrajentropy.in` Input file that calculates the entropy via the quasi-harmonic approximation. This file is processed by *ptraj*. (2)

`_MMPBSA_avgcomplex.pdb` PDB file containing the average positions of all complex conformations processed by `_MMPBSA_cenptraj.in`. It is used as the reference for the `_MMPBSA_ptrajentropy.in` file above. (1)

`_MMPBSA_complex_entropy.out` File into which the entropy results from `_MMPBSA_ptrajentropy.in` analysis on the complex are dumped. (1)

`_MMPBSA_receptor_entropy.out` Same as above, but for the receptor. (1)

`_MMPBSA_ligand_entropy.out` Same as above, but for the ligand. (1)

`_MMPBSA_ptraj_entropy.out` Output from running *ptraj* using `_MMPBSA_ptrajentropy.in`. (1)

`_MMPBSA_complex_gb.mdout`.# *sander* output file containing energy components of all complex snapshots done in GB. (1)

`_MMPBSA_receptor_gb.mdout`.# *sander* output file containing energy components of all receptor snapshots done in GB. (1)

`_MMPBSA_ligand_gb.mdout`.# *sander* output file containing energy components of all ligand snapshots done in GB. (1)

`_MMPBSA_complex_pb.mdout`.# *sander* output file containing energy components of all complex snapshots done in PB. (1)

`_MMPBSA_receptor_pb.mdout`.# *sander* output file containing energy components of all receptor snapshots done in PB. (1)

`_MMPBSA_ligand_pb.mdout`.# *sander* output file containing energy components of all ligand snapshots done in PB. (1)

`_MMPBSA_complex_rism.out`.# *rism3d.snglpnt* output file containing energy components of all complex snapshots done with 3D-RISM (1)

39. MMPBSA.py

`_MMPBSA_receptor_rism.out`.# *rism3d.snglpnt* output file containing energy components of all receptor snapshots done with 3D-RISM (1)

`_MMPBSA_ligand_rism.out`.# *rism3d.snglpnt* output file containing energy components of all ligand snapshots done with 3D-RISM (1)

`_MMPBSA_pbsanderoutput.junk`.# File containing the information dumped by sander.APBS to STDOUT. (1)

`_MMPBSA_ligand_nm.out`.# Output file from *mmpbsa.py_nabnmode* that contains the entropy data for the ligand for all snapshots. (1)

`_MMPBSA_receptor_nm.out`.# Output file from *mmpbsa.py_nabnmode* that contains the entropy data for the receptor for all snapshots. (1)

`_MMPBSA_complex_nm.out`.# Output file from *mmpbsa.py_nabnmode* that contains the entropy data for the complex for all snapshots. (1)

`_MMPBSA_mutant...` These files are analogs of the files that only start with `_MMPBSA_` described above, but instead refer to the mutant system of alanine scanning calculations.

`_MMPBSA_*out`.# These files are thread-specific files. For serial simulations, only `#=0` files are created. For parallel, `#=0` through `NUM_PROC - 1` are created.

39.3.7. Advanced Options

The default values for the various parameters as well as the inclusion of some variables over others in the general MMPBSA.py input file were chosen to cover the majority of all MM/PB(GB)SA calculations that would be attempted while maintaining maximum simplicity. However, there are situations in which MMPBSA.py may appear to be restrictive and ill-equipped to address. Attempts were made to maintain the simplicity described above while easily providing users with the ability to modify most aspects of the calculation easily and without editing the source code.

-make-mdins This flag will create all of the mdin and input files used by sander and nmode so that additional control can be granted to the user beyond the variables detailed in the input file section above. The files created are `_MMPBSA_gb.mdin` which controls GB calculation; `_MMPBSA_pb.mdin` which controls the PB calculation; `_MMPBSA_sander_nm_min.mdin` which controls the sander minimization of snapshots to be prepared for nmode calculations; and `_MMPBSA_nmode.in` which controls the nmode calculation. If no input file is specified, all files above are created with default values, and `_MMPBSA_pb.mdin` is created for AmberTools's *pbsa*. If you wish to create a file for sander.APBS, you must include an input file with "sander_apbs=1" specified to generate the desired input file. Note that if an input file is specified, only those mdin files pertinent to the calculation described therein will be created!

-use-mdins This flag will prevent MMPBSA.py from creating the input files that control the various calculations (`_MMPBSA_gb.mdin`, `_MMPBSA_pb.mdin`, `_MMPBSA_sander_nm_min.mdin`, and `_MMPBSA_nmode.in`). It will instead attempt to use existing input files (though they must have those names above!) in their place. In this way, the user has full control over the calculations performed, however care must be taken. The mdin files created by MMPBSA.py have been tested and are (generally) known to be consistent. Modifying certain variables (such as `imin=5`) may prevent the script from working, so this should only be done with care. It is recommended that users start with the existing mdin files (generated by the `-make-mdins` flag above), and add and/or modify parameters from there.

strip.mask This input variable allows users to control which atoms are stripped from the trajectory files associated with `solvated_prmtop`. In general, counterions and water molecules are stripped, and the complex is centered and imaged (so that if `iwrap` caused the ligand to "jump" to the other side of the periodic box, it is replaced inside the active site). If there is a specific metal ion that you wish to include in the calculation, you can prevent `ptraj` from stripping this ion by NOT specifying it in `strip.mask`. Note that `strip.mask` does nothing if no `solvated_prmtop` is provided.

QM/MMGBSA There are a lot of options for QM/MM calculations in *sander*, but not all of those options were made available via options in the MMPBSA.py input file. In order to take advantage of these other options, you'll have to make use of the `-make-mdins` and `-use-mdins` flags as detailed above and change the resulting `_MMPBSA_gb_qmmm.com/rec/lig.mdin` files to fit your desired calculation. Additionally, MMPBSA.py suffers all shortcomings of *sander*, one of those being that PB and QM/MM are incompatible. Therefore, only QM/MMGBSA is a valid option right now.

39.4. Python API

The aim of the MMPBSA.py API is to provide you with direct access to the raw data produced during a MMPBSA.py calculation. By default, MMPBSA.py calculates an average, standard deviation, and standard error of the mean for all of the generated data sets, but does not support custom analyses. The API reads an `_MMPBSA_info` file, from which it will determine what kind of calculation you performed, then automatically parse the output files and load the data into arrays.

The `keep_files` variable in the `&general` section must be set to 1 or 2 in order to keep enough files for the API to work. It currently does NOT load decomposition data into available data structures. The topology files you used in the MMPBSA.py calculation must also be available in the location specified in the `_MMPBSA_info` file.

Using the API

The function `load_mmpbsa_info` takes the name of an MMPBSA.py info file (typically `_MMPBSA_info`) and returns a populated `mmpbsa_data` instance with all of the parsed data. An example code snippet that creates a `mmpbsa_data` instance from the information in `_MMPBSA_info` is shown below.

```
from MMPBSA.mods import API as MMPBSA_API
data = MMPBSA_API.load_mmpbsa_info("_MMPBSA_info")
```

Properties of mmpbsa_data

The `mmpbsa_data` class is a nested dictionary structure (`mmpbsa_data` is actually derived from `dict`). The various attributes of `mmpbsa_data` are described below followed by the defined operators.

Attributes

If the `numpy` package is installed and available, all data arrays will be `numpy.ndarray` instances. Otherwise, all data arrays will be `array.array` instances with the `'d'` data type specifier (for a double precision float). The data is organized in an `mmpbsa_data` instance in the following manner:

```
mmpbsa_data_instance[calc_key][system_component][energy_term]
```

In this example, `calc_key` is a `dict` key that is paired to another `dict` (`mmpbsa_data_instance` is the first-level `dict`, in this case). The keys of these second-level `dict` instances (`system_component`) pair to another `dict`. The keys of these inner-most (third-level) `dict` instances are paired with the data arrays for that energy term. The various dictionary keys are listed below for each level. If alanine scanning was performed, the `mmpbsa_data_instance` also has a `"mutant"` attribute that contains the same dictionary structure as `mmpbsa_data` does for the normal system. The only difference is that the data is accessed as follows:

```
mmpbsa_data_instance.mutant[calc_key][system_component][energy_term]
```

Note, all keys are case-sensitive, and if a space appears in the key, it must be present in your program. Also, if polar/non-polar decomposition is not performed for 3D-RISM, then the `'POLAR SOLV'` and `'APOLAR SOLV'` keys are replaced with the single key `'ERISM'`

Table 39.1.: List and description of `calc_key` dict keys that may be present in instances of the `mmpbsa_data` class.

Dictionary Key (<code>calc_key</code>)	Calculation Type
'gb'	Generalized Born Results
'pb'	Poisson-Boltzmann Results
'rism gf'	Gaussian Fluctuation 3D-RISM Results
'rism std'	Standard 3D-RISM Results
'nmode'	Normal Mode Analysis Results
'qh'	Quasi-harmonic Approximation Results

Table 39.2.: List and description of `system_component` keys that may be present in instances of the `mmpbsa_data` class.

Dictionary Key (<code>system_component</code>)	Description
'complex'	Data sets for the complex. (Stability & Binding)
'receptor'	Data sets for the receptor. (Binding only)
'ligand'	Data sets for the ligand. (Binding only)

Defined operators

In-place addition: It extends all of the arrays that are common to both `mmpbsa_data` instances. This is useful if, for instance, you run two MMPBSA.py calculations, and you use `-prefix <new_prefix>` for the second simulation. Assuming that `<new_prefix>` is `_MMPBSA2_` for the second MMPBSA.py calculation, the following pseudo-code will generate an `mmpbsa_data` instance with all of the data in concatenated arrays. The pseudo-code assumes `MMPBSA_mods.API` was imported as demonstrated in Subsection 39.4.

```
data = MMPBSA_API.load_mmpbsa_info("MMPBSA_info")
data += MMPBSA_API.load_mmpbsa_info("MMPBSA2_info")
```

Example API Usage

In many cases, the autocorrelation function of the energy can aid in the analysis of MM/PBSA data, since it provides a way of determining the statistical independence of your data points. For example, 1000 correlated snapshots provide less information, and therefore less statistical certainty, than 1000 uncorrelated snapshots. The standard error of the mean calculation performed by MMPBSA.py assumes a completely uncorrelated set of snapshots, which means that it is a lower bound of the *true* standard error of the mean, and a plot of the autocorrelation function may help determine the actual value.

The example program below will calculate the autocorrelation function of the total energy (complex only for both the normal and alanine mutant systems) from a GB calculation and plot the resulting code using `matplotlib`.

```
import os
import sys
# append AMBERHOME/bin to sys.path
sys.path.append(os.path.join(os.getenv('AMBERHOME'), 'bin'))
# Now import the MMPBSA API
from MMPBSA_mods import API as MMPBSA_API
import matplotlib.pyplot as plt
```


Table 39.3.: List and description of `energy_term` keys that may be present in instances of the `mmpbsa_data` class. The allowed values of `energy_term` depend on the value of `calc_key` above in Table 39.1. The `energy_term` keys are listed for each `calc_key` enumerated above, accompanied by a description. The RISM keys are the same for both `'rism gf'` and `'rism std'` although the value of `'POLAR SOLV'` and `'APOLAR SOLV'` will differ depending on the method chosen. Those keys marked with * are specific to the CHARMM force field used through chamber. Those arrays are all 0 for normal Amber topology files.

Description	'gb'	'pb'	RISM
Bond energy	'BOND'	'BOND'	'BOND'
Angle energy	'ANGLE'	'ANGLE'	'ANGLE'
Dihedral Energy	'DIHED'	'DIHED'	'DIHED'
Urey-Bradley*	'UB'	'UB'	—
Improper Dihedrals*	'IMP'	'IMP'	—
Correction Map*	'CMAP'	'CMAP'	—
1-4 van der Waals energy	'1-4 VDW'	'1-4 VDW'	'1-4 VDW'
1-4 Electrostatic energy	'1-4 EEL'	'1-4 EEL'	'1-4 EEL'
van der Waals energy	'VDWAALS'	'VDWAALS'	'VDWAALS'
Electrostatic energy	'EEL'	'EEL'	'EEL'
Polar solvation energy	'EGB'	'EPB'	'POLAR SOLV'
Non-polar solvation energy	'ESURF'	'ENPOLAR'	'APOLAR SOLV'
Total solvation free energy	'G solv'	'G solv'	'G solv'
Total gas phase free energy	'G gas'	'G gas'	'G gas'
Total energy	'TOTAL'	'TOTAL'	'TOTAL'

Table 39.4.: Same as Table 39.3 for the entropy data.

Description	'nmode'	'qh'
Translational entropy	'Translational'	'Translational'
Rotational entropy	'Rotational'	'Rotational'
Vibrational entropy	'Vibrational'	'Vibrational'
Total entropy	'Total'	'Total'

```

import numpy as np

data = MMPBSA_API.load_mmpbsa_info('_MMPBSA_info')
total = data['gb']['complex']['TOTAL'].copy()

data = MMPBSA_API.load_mmpbsa_info('_MMPBSA_info')
total_mut = data.mutant['gb']['complex']['TOTAL'].copy()

# Create a second copy of the data set. The np.correlate function does not
# normalize the correlation function, so we modify total and total2 to get
# that effect
total -= total.mean()
total /= total.std()
total2 = total.copy() / len(total)
acor = np.correlate(total, total2, 'full')

total_mut -= total_mut.mean()
total_mut /= total_mut.std()
total2_mut = total_mut.copy() / len(total_mut)
acor_mut = np.correlate(total_mut, total2_mut, 'full')

# Now generate the 'lag' axis
xdata = np.arange(0, len(total))

# The acor data set is symmetric about the origin, so only accept the
# positive lag times. Graph the result
plt.plot(xdata, acor[len(acor)//2:], xdata, acor_mut[len(acor)//2:])
plt.show()

```

Decomposition Data

When performing decomposition analysis, the various decomp data is stored in a separate tree of dicts referenced with the 'decomp' key. The key sequence is similar to the sequence for the 'normal' data described above, where decomp is followed by the solvent model (GB or PB), followed by the species (complex, receptor, or ligand), followed by the decomposition components (total, backbone, or sidechain), followed by the residue number (or residue pair for pairwise decomposition), finally followed by the contribution (internal, van der Waals, electrostatics, etc.) The available keys are shown in Figure 39.1 on page 919 (and each key is described afterwards).

Decomp Key Descriptions

gb All Generalized Born results

pb All Poisson-Boltzmann results

complex All results from the complex trajectory

receptor All results from the receptor trajectory

ligand All results from the ligand trajectory

TDC All results from the total decomposition

SDC All results from the sidechain decomposition

BDC All results from the backbone decomposition

All data from residue number “#” in per-residue decomposition (same residue numbering scheme as in each respective topology file)

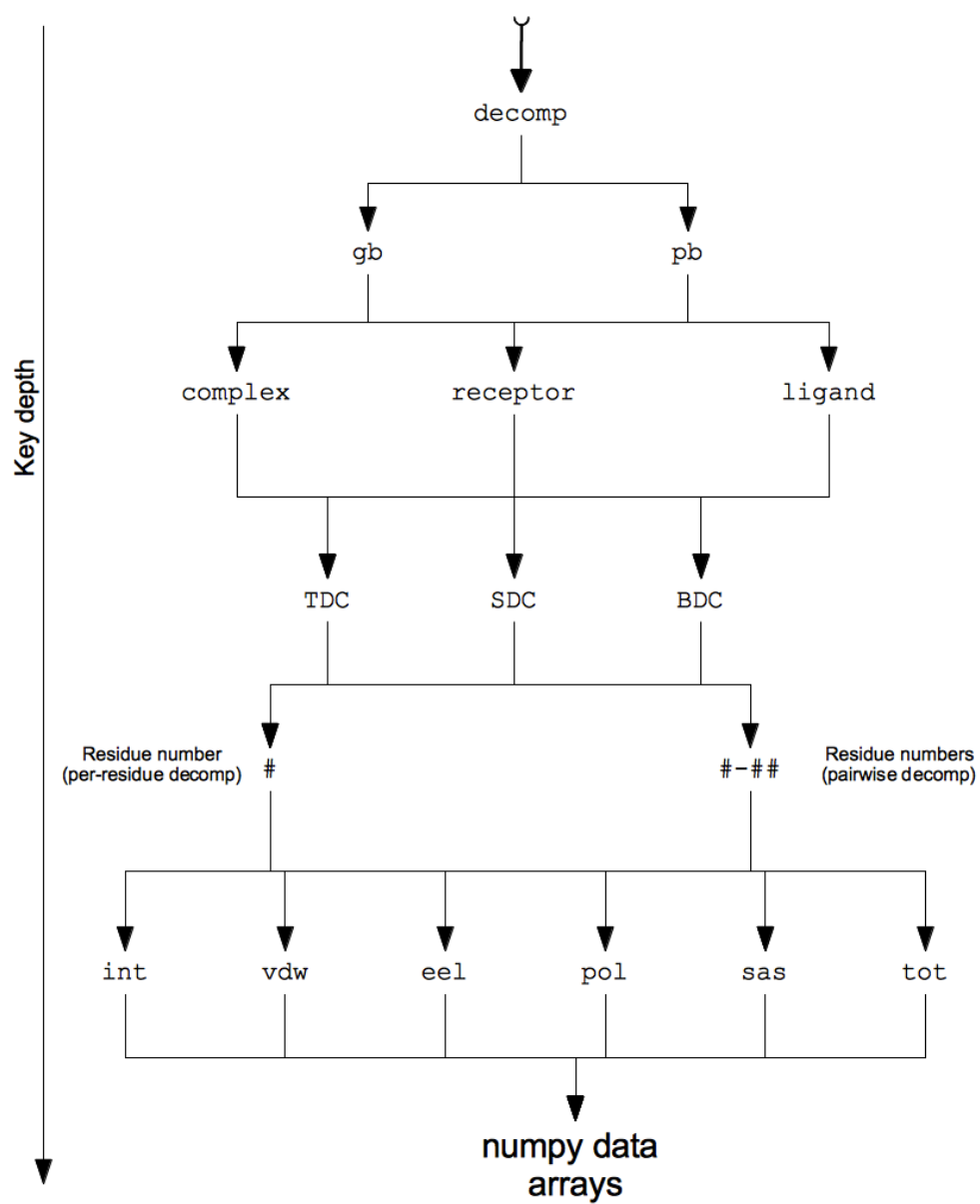


Figure 39.1.: *Tree of dict keys following the ‘decomp’ key in a `mmpbsa_data` instance.*

39. *MMPBSA.py*

#-## All interaction energies between residues “#” and “##” (same residue numbering scheme as in each respective topology file)

int Internal energy contributions (see the *idecomp* variable description above)

vdw van der Waals energy contributions

eel Electrostatic energy contributions

pol Polar solvation free energy contributions

sas Non-polar solvation free energy contributions

tot Total free energy contributions (sum of previous 5).

40. FEW

The Free Energy Workflow (FEW) is a tool for automated calculation of the binding free energy of a *set of ligands binding to the same receptor* using modules provided in the AMBER suite of programs. Prerequisite for calculations with FEW is the existence of 3D complex structures of a receptor and ligands. Generally, the more accurate the complex structures are the more accurate results can be expected.

FEW provides functions for setup of three types of binding free energy calculations: implicit solvent calculations by the MM-PBSA or MM-GBSA methods, linear interaction energy analyses (LIE), and thermodynamic integration (TI) calculations. These three binding free energy calculation approaches are available via three program modules provided in FEW:

- WAMM: Workflow for automated MM-PBSA & MM-GBSA
- LIEW: Linear interaction energy workflow
- TIW: Thermodynamic integration workflow

40.1. Installation

The program FEW consists of the main Perl script “FEW.pl” and a set of Perl modules stored in the folder “libs” provided in the main FEW directory.

A perl installation (version 5.10 or newer) needs to be available on the system where FEW shall be executed. For running the program some additional Perl modules are needed (Table 40.1), which are provided under the terms of the respective license in the folder “additional_libs”. Please ensure that the “additional_libs” folder is located in the same directory in which the FEW.pl script resides.

FEW can be used with Amber and AmberTools. To enable access of the program FEW to AmberTools, the tools need to be executable on the system by just calling their names, e.g., “antechamber” should invoke the *antechamber* program. The following tools and programs are used by FEW directly: *ambpdb*, *tleap*, *antechamber*, *cpptraj*, *parmchk*, *mm_pbsa.pl* and *Babel* (in case SDF-input files are provided). In addition, the AMBER programs *sander* and/or *PMEMD* are required, and if charges shall be calculated by the RESP procedure also access to the program *Gaussian03* is needed. The later programs can be installed on a different system or a compute cluster.

Table 40.1.: *Perl modules from CPAN used by FEW.*

Module name ¹⁾	Functionality
PerlMol	Read and manage atom information
FreezeThaw	Interconversion between Perl structures and strings
File::ReadBackwards	Read file line by line from end of file
Statistics::Normality Statistics::PointEstimation Statistics::Descriptive Statistics::Smoother Statistics::Distributions	Modules for statistical analysis

¹⁾ Modules are provided with FEW under the terms of the respective license.

Basic program call

```
perl FEW.pl <procedure> <command-file>
```

Table 40.2.: *Overview of procedures and corresponding modules available in FEW.*

Procedure name	Program module used	Key phrase in command file ¹⁾
MMPBSA or MMGBSA ²⁾	WAMM	@WAMM
LIE	LIEW	@LIEW
TI	TIW	@TIW

¹⁾ Expression that needs to be provided in the first line of the command file to ensure that the requested procedure and the provided command file match.

²⁾ Either MMPBSA or MMGBSA can be specified.

	MM-PBSA & MM-GBSA	LIE	TI
Workflow module	WAMM	LIEW	TIW
MD setup procedure	1- & 3-trajectory	3-trajectory (ligand & complex)	3-trajectory (ligand & complex)
Prepared free energy calculations	Implicit solvent free energy calculations	Molecular mechanics energy calculations in explicit solvent	Thermodynamic integration simulations in explicit solvent
Calculated energy	$\Delta G_{\text{effective}}$	ΔE_{elec} & ΔE_{vdW}	$\Delta \Delta G_{\text{binding}}$

Figure 40.1.: Overview of program modules and functionality provided in FEW. All three free energy calculation workflows available in FEW have a MD setup step in common.

The procedures that can be chosen are listed in Table 40.2, and an overview of the functionality provided in the individual free energy calculation modules is shown in Figure 40.1. Example command files can be found in the folder `$AMBERHOME/AmberTools/src/FEW/examples/command_files`. Please ensure that in each command file the program module that shall be used for calculation is specified via a key phrase in the first line (Table 40.2).

In addition, template files, e.g., input files with parameters for MD simulations, are available under `examples/input_info`. It is strongly recommended that non-experts use these template files for analysis and make only those system and/or computing resource specific modifications that are requested below.

A complete example analysis corresponding to the show case example presented in ref. [820] including all input files for setup and the final result files with the computed binding free energies can be obtained from <http://cpclab.uni-duesseldorf.de/software>. The current version of FEW uses per default the ff12SB force field of AMBER. Earlier FEW versions, as the one used for the generation of the case study data, employed the ff99SB force field. For backwards compatibility with previous FEW versions set the flag `backwards` to 1.

40.2. Overview of workflow steps and minimal input

A detailed description of FEW and its functionality is provided in ref. [820]. We strongly encourage the user to run the FEW tutorial first that is available at <https://ambermd.org/tutorials>.

For the setup of free energy calculations with FEW a 3D receptor structure in PDB format and 3D ligand structures with coordinates of the ligand bound position in mol2 format are required (see section 40.3.1). FEW provides besides the general setup functionality a lot of additional system / computing architecture specific and expert options that can be requested by setting parameters / flags in the command file. All available options are described in the following sections, where essential parameters are marked in bold, while optional additional parameters are shown in normal writing. For a typical system it is usually sufficient to define the essential flags. Example files containing only those flags that are commonly needed can be found under `$AMBERHOME/AmberTools/src/FEW/examples/command_files/minimalistic_files`. Please use these files only if your ligands are available as single structure mol2 files and if the receptor contains only standard residues defined in the ff12SB force field.

The setup of free energy calculations with FEW is conducted in a multi-step procedure, i.e., FEW is called several times using a command file with the parameters for the respective setup step. The Table 40.3 shows the minimum number of FEW calls required for preparation and analysis of the individual free energy calculations. The individual setup steps can be further divided into individual tasks, such that each setup task can

be tracked and checked. The later is generally recommended if any problems are encountered in the setup procedure. In this case it should also be thoroughly checked, whether additional parameters might need to be specified for the the specific system. Example command files of the individual setup steps of the different setup procedures containing all available parameters can be found in the procedure specific folders under `$AMBERHOME/AmberTools/src/FEW/examples/command_files`

Table 40.3.: Overview of steps required for setup, execution, and analysis of MM-PB(GB)SA, LIE, and TI calculations with FEW¹⁾.

Call ²⁾	MM-PB(GB)SA (Section 40.4)	LIE (Section 40.5)
MD simulations (Section 40.3)		
RESP charges		AM1-BCC charges
X	Preparation of Gaussian input files (40.3.2)	Charge calculation & setup of MD simulations (40.3.2)
	Calculation of ESP with Gaussian	
X	Charge calculation & setup of MD simulations (40.3.2)	
	Running MD simulations	
Free energy calculations		
X	Setup of MM-PB(GB)SA calculations (40.4)	Setup of LIE analysis (40.5)
	Running MM-PB(GB)SA calculations	Running LIE calculations
	Preparation of MM-PB(GB)SA results for analysis (40.4)	Preparation of LIE results for analysis (40.5)

40.3. Common setup of molecular dynamics simulations

The setup of molecular dynamics (MD) simulations with FEW can be used in connection with all three available free energy calculation procedures (cf. Figure 40.1).

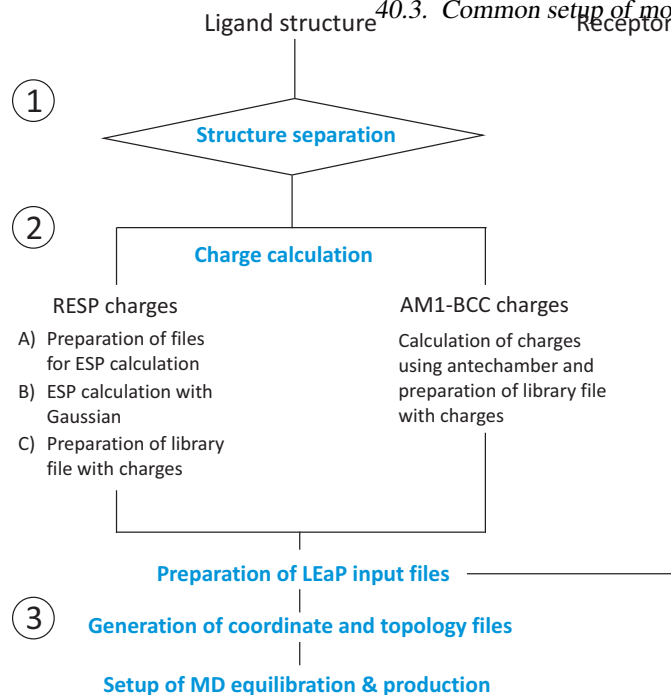


Figure 40.2.: Graphical illustration of the steps conducted for setup of MD simulations. Steps that can be executed independently by separate calls of *FEW.pl* are indicated by numbers. However, the individual steps can be combined, so that the whole MD simulation setup can be conducted in 1 or 2 steps for the AM1-BCC and RESP charge option, respectively (see 40.3).

MM-PB/GBSA and LIE calculations require the existence of MD trajectories from which snapshots can be extracted, so that a MD setup is needed. For TI calculations it is recommended to use structures pre-equilibrated with the common MD setup functionality of FEW as input structures. Expert users may also provide structures directly, i.e. without using the MD equilibration preparation functionality of FEW. In the later case the structures for TI input must be preprocessed using the structure preparation workflow available in the MD setup procedure (see Section 40.3.2). As the MD setup functionality requires the same input for all three procedures, it is discussed here separately from the procedure specific features. The setup of MD simulations is conducted in 3 consecutive steps (see Figure 40.2), which can be initiated by a minimum of 1 or 2 FEW calls in the case of a setup of MD simulations with AM1-BCC charges or RESP charges for the ligands, respectively (cf. Table 40.3).

40.3.1. Input structures

Ligand structures: 3D coordinates of ligand structures in the bound position and with the correct protonation state need to be provided in one of the following formats:

- A) SDF file containing multiple ligand structures (requires the program Babel)
- B) mol2 file with multiple ligand structures
- C) mol2 files with one structure per file

In the case of A) and B) a structure separation needs to be requested using the flag `structure_separation` in the command file. This will result in a set of structures in format C), which is required for MD setup and all further calculations. Ligands must consist of no more than one residue, and mol2 files must obey the formatting rules defined by TRIPOS (see <http://www.tripos.com/data/support/mol2.pdf>). In addition to the information obligatory according to these rules for the entries in the ATOM section of mol2 files, FEW requires the substructure ID and the substructure name, i.e., the residue ID and name. As residue names will be shortened to three characters, it is recommended to use ligand residue names that consist of three characters only. Residue names can consist of letters and numbers, but should not start with a number nor contain special characters.

Receptor structure: A structure of the receptor in PDB format with all atoms that shall be considered in the calculation, i.e., including protons, is required. This structure can contain crystal water and / or non-standard residues. The residues of the receptor need to be consecutively numbered starting from residue number 1. To ensure that the atom names of the PDB structure can be recognized by LEaP, it is recommended to load the prepared PDB file first into LEaP and then re-save it. By this the residues are also automatically re-numbered according to the requirements of FEW. If there are different chains or missing residues in the receptor structure, those parts of the structure that are not directly connected need to be separated by a TER card in the PDB file (see http://deposit.rcsb.org/adit/docs/pdb_atom_format.html). The residue name of all atoms that belong to water molecules must be either "WAT" or "HOH".

40.3.2. Flags for MD setup

The following flags are available for MD setup. Flags and corresponding options are given. Essential flags are marked in bold and optional ones are shown in normal writing. Statements in "<" and ">" brackets denote place holders. For example input files see `$AMBERHOME/AmberTools/src/FEW/examples/command_files/commonMDsetup`.

MD simulations are setup with a cubic water box extending at least 11 Å in each direction from the solute. Truncated octahedrons are currently not supported. The normal file extensions of MD input and output files are shortened: *.inpcrd to *.crd and *.prmtop to *.top. An overview of the folder structure created upon MD setup is shown in Figure 40.3.

Specification of input / output directories and formats:

lig_struct_path <path>	Path to folder containing the ligand structures. For ligands provided in format C) a folder containing exclusively all ligand structures that shall be regarded needs to be manually created and specified under <code>lig_struct_path</code> . If ligand structures are provided in input format A) or B) and a separation is requested a folder called <code>structs</code> containing the separated structures is created in the basic output directory. This folder needs to be specified in all subsequent setup steps.
output_path <path>	Path to main output directory in which all new folders will be generated.
rec_structure <structure>	Full path and name of receptor structure file in PDB format.
lig_format_sdf 0 1	Set to 1, if multi-ligand file in sdf-format is provided; format A).
lig_format_mol2 0 1	Set to 1, if ligand structure files are provided in format B) or C).
water_in_rec 0 1	Optional: 1: Crystal water present in receptor structure. Water molecules need to be provided after the solute and should carry the residue name "WAT" or "HOH". 0: PDB structure of the receptor contains only the solute and no crystal water molecules.
multi_structure_lig_file <name>	Only relevant for ligands in input formats A) or B): Basic name of ligand input file if multi-structure file is provided in input formats A) or B). File extension can be omitted.
bound_rec_structure <structure>	Optional: Absolute path and name of the receptor PDB structure in the bound state, in case two different receptor structures shall be used for setup of complex and receptor in the 3-trajectory approach.

membrane_file <structure>

Optional: Absolute path and name of a PDB file containing lipids, ions, and water molecules. This file is only required if a MD simulation with an explicit membrane shall be performed. The file needs to be generated using external tools, e.g. the CHARMM-GUI Membrane Builder (<http://www.charmm-gui.org/?doc=input/membrane>) [821–824]. It is recommended to use the latter tool for preparing a PDB file of the membrane, water, and ions, if the Lipid14 force field [103] shall be used for the MD simulations. The files generated with the CHARMM-GUI Membrane Builder can be converted with the `charmm_lipid2amber.py` script provided with AMBER in order to obtain the required Lipid14 specific lipid naming scheme. If the file containing lipids, ions, and water is generated with another program, the user needs to ensure that the file formatting and lipid naming scheme is consistent with AMBER and the force fields that shall be used.

Structure separation

structure_separation 0 | 1

Only relevant if ligands are in input formats A) or B): Set to 1 in case of ligand input format A) or B). If set to 1, structure separation is conducted, and the resulting single structure files are stored in mol2 format in a folder called `structs` under `<output_path>`. Default = 0.

Generation of files for setup of system with LEaP

prepare_leap_input 0 | 1

The parameters in this section will only be regarded if this flag is set to 1. If the flag is switched on, the files needed for the preparation of the system with LEaP are generated.

non_neutral_ligands 0 | 1

Set to 1, if the total charge of at least one ligand molecule is not equal to zero. In this case the total charge of each non-neutral ligand molecule needs to be defined in a separate file `lig_charge_file`.

lig_charge_file <file>

If the total charge of at least one ligand molecule is not equal to zero, specify the full path and name of a file in which the names, the total charge, and the multiplicity of the non-neutral ligands is stored in tab-separated format; see `examples/input-info/charge.txt`.

am1_lig_charges 0 | 1

Set to 1 if ligand charges shall be calculated according to the AM1-BCC method [449, 450]. Please note: Only one charge calculation method can be used at a time.

resp_lig_charges 0 | 1

Set to 1 if ligand charges shall be calculated according to the "Restraint electrostatic potential fit" (RESP) method [454]. Please note: Only one charge calculation method can be used at a time.

resp_setup_step1 0 | 1

Request step one of the RESP charge calculation. The RESP charges are calculated in two steps. First, the files needed for ligand structure optimization and the calculation of the electrostatic potential with the program *Gaussian* are generated. If this step is carried out, a folder called "gauss" containing all input files for the *Gaussian* calculation is generated in the `<output_path>` directory. This folder can be copied to a compute cluster, where the program *Gaussian* is available. It is then possible to run the *Gaussian* jobs for all ligands at the same time.

resp_setup_step2 0 1	Request step two of the RESP charge calculation, in which the atomic charges are calculated based on the ESP computed with <i>Gaussian</i> . If this flag is set to 1, the <i>Gaussian</i> output files need to be available in the folder <code><output_path>/gauss</code> .
gauss_batch_file 0 1	Optional: Request setup of batch scripts for <i>Gaussian</i> jobs. Default = 0.
gauss_batch_template <file>	In case <code>resp_lig_charges=1</code> , <code>resp_setup_step1=1</code> , and <code>gauss_batch_file=1</code> , then the full path and name of the template file for the generation of the <i>Gaussian</i> batch-script needs to be specified here. Example template file: <code>examples/input_info/gaussian.pbs</code> . Please adapt the file according to the needs of your queuing system, but keep the variables and the format in the section "Fix variables" and ensure that the line for job naming ends with "-N".
gauss_batch_path <path>	If the basis working directory for the <i>Gaussian</i> calculations differs from the <code><output_path></code> directory the new basis directory can be specified here. For example, this might be the case if the calculations shall be run on a compute cluster.
average_charges <file>	Optional: If the charges of two enantiomers shall be averaged, such that the two molecules obtain the same atomic charges, a file in which the enantiomer pairs are defined needs to be specified here. Pre-requisite: The atom order and naming in the input mol2-files of the ligand isomers is identical. An example file can be found under <code>examples/input_info/isomer_pairs.txt</code>
calc_charges 0 1	Optional: This flag determines whether charges are calculated. If set to 0, only LEaP input files that do not require charge calculation are generated. Default = 1.
prepare_membrane 0 1	Optional: Request setup of MD simulation with explicit membrane. Only if <code>prepare_membrane=1</code> the lipids, ions, and water molecules specified in the <code>membrane_file</code> will be considered. Default = 0.
ligand_water_cutoff <no.>	Optional: Relevant only if <code>prepare_membrane=1</code> . Cutoff distance from the ligand within which all water molecules will be removed upon ligand insertion in order to avoid clashes between the ligand and water molecules. Default = 1 Å.

Setup of MD simulations

setup_MDsimulations 0 1	Request generation of input files for MD simulations by setting this flag to 1. All other flags in this section are only taken into account if <code>setup_MDsimulations=1</code> .
traj_setup_method 1 3	Specify whether simulations shall be setup according to the 1-trajectory or the 3-trajectory protocol for MM-PBSA or MM-GBSA. For LIE analyses, only the 3-trajectory setup, i.e., separate simulations for the ligand bound to the complex and for the ligand free in solution, works. For the TI approach preparation of an equilibration according to the 3-trajectory setup can be performed.
MD_am1 0 1	Set to 1 if MD simulation setup shall be conducted using previously calculated AM1-BCC charges.

MD_resp 0 | 1

Set to 1 if setup of MD simulations shall be carried out using previously calculated RESP charges.

SSbond_file <file>

If your receptor contains disulfide bridges the S-S bond connectivities need to be defined in a separate file. The full path and name of the file containing the disulfide bridge definitions should be provided here. In this file the numbers of those residues involved in S-S bonds should be specified in tab-separated format. Please note, all cysteine residues involved in S-S bonds should be named CYX in the provided receptor PDB structure. For an example S-S connectivity file see `examples/input_info/SSbridges.txt`

total_MDequil_time <time>

Total equilibration time in [ps]. The simulation time requested in all template files provided for equilibration needs to sum up to the time provided here. In case the files provided in the example `MDequil_template_folder` are used this keyword does not need to be specified. Default = 400 ps.

MDequil_batch_template <file>

Absolute path and name of the batch template file for the equilibration. This file should contain calls for all equilibration steps. For an example template file see `examples/input_info/equi.pbs`. Please adapt this file according to your needs, but keep the variables and the format in the section "Fix variables" and ensure that the line for job naming ends with "-N".

total_MDprod_time <time>

Total simulation time of MD production in [ns].

MD_prod_batch_template <file>

Absolute path and name of the batch template file for MD production. For an example template file see `examples/input_info/prod.pbs`. Please adapt this file according to the needs of your queuing system, but do not change anything from the section "Fix variables" up to the section "Re-queue" and ensure that the line for job naming ends with "-N".

no_of_rec_residues <no.>

Actual number of residues in the receptor structure when all residues in the receptor are consecutively numbered starting from 1. Structurally bound ions should be treated as part of the receptor.

restart_file_for_MDprod <file>

Basename of restart file from equilibration that shall be used as initial file for MD production.

additional_library <library file>

Absolute path and name of additional library file. If your receptor structure contains non-standard residues or ions, an AMBER library file for these residues / ions needs to be provided here.

additional_frmod <file>

Absolute path and name of additional parameter file. If your receptor structure contains residues or ions for which no parameters are available in the ff12SB force field, a parameter file in which the missing parameters are defined needs to be provided here.

MD_batch_path <path>

If the simulations need to be conducted on another system / machine than the one used for setup, the <output_path> during the simulations may differ from the one used for setup. If this is

	the case, please specify here the basis directory for the MD simulations. If no path is defined, it is assumed that the path is equal to <code><output_path></code> .
<code>MDequil_template_folder</code> <code><folder></code>	Absolute path to the folder containing the template files for equilibration. All files provided in this folder will be considered for equilibration setup. Example equilibration files that will be used per default can be found under <code>examples/input_info/equi</code> . If you change the template files or create additional files, please keep the format for the definition of the residues that shall be restraint.
<code>MDprod_template</code> <code><file></code>	Please specify the absolute path and name of the template file for production run. In this file all the flags you would like to use in your MD simulation should be set according to the <i>sander</i> <i>PME</i> MD definitions. The assignment should have the form flag = <code><value></code> , and individual flags should be separated by commas. For an example file see <code>examples/input_info/MDprod.in</code> . Per default this file will be used as template if no template file is specified.
<code>water_model</code> <code>TIP3P</code> <code>OPC</code>	Water model that shall be used for the MD simulations. Currently the water models TIP3P and OPC are available.

Additional parameters for setup of MD simulations with explicit membrane

<code>prepare_membrane</code> <code>0</code> <code>1</code>	Optional: Request setup of MD simulation with explicit membrane. Default = 0.
<code>use_lipid14_ff</code> <code>0</code> <code>1</code>	If set to 1, the Lipid14 force field will be used for the lipids in the explicit membrane simulation. In case a setup of a MD simulation with explicit membrane is requested (<code>prepare_membrane=1</code>) although <code>use_lipid14_ff</code> is not specified or <code>use_lipid14_ff=0</code> and <code>use_gaff_lipid_ff=0</code> , then <code>use_lipid14_ff</code> is set to 1 per default.
<code>use_gaff_lipid_ff</code> <code>0</code> <code>1</code>	If <code>use_gaff_lipid_ff=1</code> parameters from the GaffLipid force field will be used. Please note that in this case library and parameter files for the lipids need to be provided under <code>additional_library</code> and <code>additional_frcmod</code> file (see above). These files can be obtained from the Lipidbook repository at http://lipidbook.bioch.ox.ac.uk [825].
<code>restrain_membrane_residues</code> <code><no.></code>	Membrane residues that shall be restrained during the equilibration phase of the MD simulations. This parameter needs only to be provided if <code>prepare_membrane=1</code> . Attention: The number of membrane residues differs from the number of lipids if the Lipid14 force field is used. In this case usually residue number = $3 \times \text{lipid number}$. Default = 0.

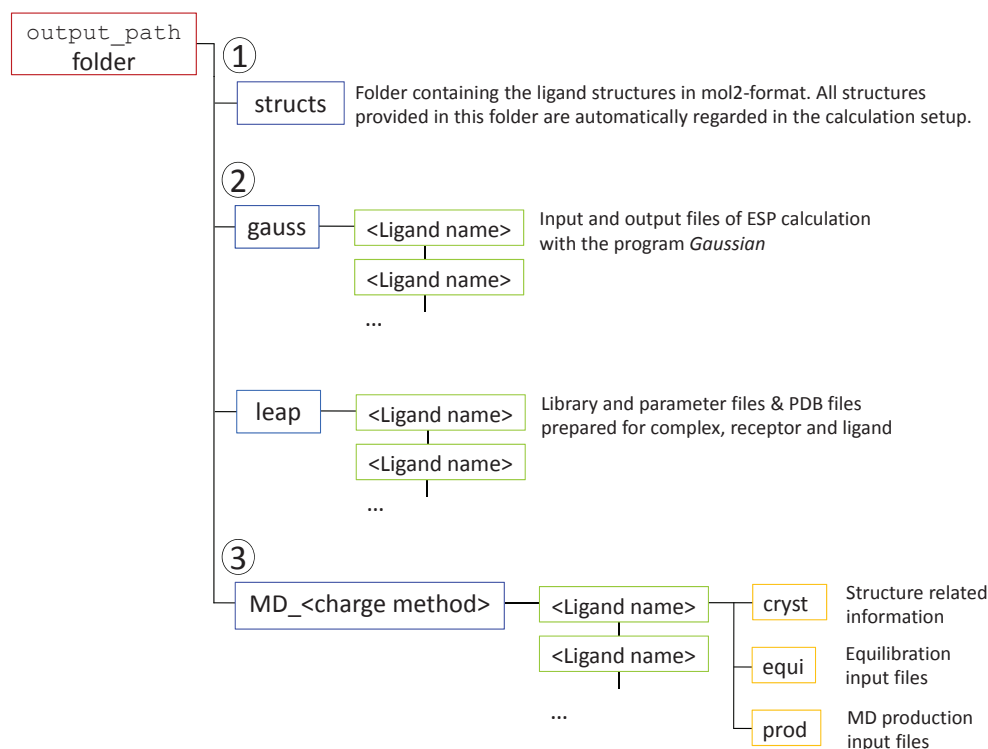


Figure 40.3.: Directory structure and files created during the common MD setup step of FEW.

40.4. Workflow for automated MM-PBSA & MM-GBSA calculations (WAMM)

The module WAMM allows to calculate binding free energies of ligands according to four flavors of the Molecular Mechanics Poisson-Boltzmann Surface Area (MM-PBSA) approach and three types of the Molecular Mechanics Generalized Born Surface Area (MM-GBSA) approach. All energies are calculated based on conformational ensembles generated by MD simulations that have been conducted using the common MD setup functionality of FEW (Section 40.3). An overview of the available binding free energy calculation options is given in Table 40.4. All binding free energy calculation methods except method PB2 can be applied to the 1- and the 3-trajectory approach. PB2 can only be used in conjunction with the 3-trajectory approach. Residue-wise and pair-wise decomposition of the effective energy (`decomposition` keyword) is currently only possible with PB=4 & GB=1. The solvent accessible surface area is calculated according to the ICOSA method in this case.

The availability of trajectories from MD productions for the complex (1-trajectory approach) or for complex, receptor, and ligand (3-trajectory approach) is prerequisite for the setup of free energy calculations according to the MM-PBSA / MM-GBSA method. These trajectories should be prepared with the MD setup functionality of FEW (cf. section 40.3). Example files for WAMM analysis setup can be found in `$AMBERHOME/AmberTools/src/FEW/examples/command.files/MMPBSA`. Besides the common section for input / output directories and format definitions, the WAMM module considers several specific flags (see below). An overview of the folder structure created by the MM-PB(GB)SA workflow is shown in Figure 40.4.

Table 40.4.: Overview of flavors of MM-PBSA and MM-GBSA calculation procedures available in the WAMM module.

Alias	Radii ¹⁾	Calculation of polar solvation energy	Method for calculation of the non-polar solvation energy			
			SASA ²⁾	E _{nonpolar} ³⁾	γ ⁴⁾	b ⁵⁾
GB1	mbondi [213]	GB ^{HTC} [200, 211, 213]	LCPO [185]	γ SASA + b	0.00720	0.0000
GB2	mbondi2 [197]	GB ^{OBC} model I [197]	LCPO [185]	γ SASA + b	0.00500	0.0000
GB5	mbondi2 [197]	GB ^{OBC} model II [197]	LCPO [185]	γ SASA + b	0.00500	0.0000
GB6	bondi	GBNSR6 [826], 5.1	GBNSR6	γ SASA + b	0.00500	0.0000
PB1	Tan&Luo + mbondi ⁶⁾ [213, 260]	PBSA ⁷⁾	PBSA ⁸⁾ [243]	γ SASA + b + E _{dispersion} .	0.03780	- 0.5692
PB2	Tan&Luo + mbondi ⁶⁾ [213, 260]	Hybrid PBSA ⁹⁾ [827]	Molsurf [781] + PBSA	γ MSA + b + E _{vdW} ¹⁰⁾	0.06900	0.0000
PB3	Parse [217]	PBSA ⁷⁾	Molsurf [781]	γ SASA + b	0.00542	0.9200
PB4	mbondi [213]	PBSA ⁷⁾	Molsurf [781]	γ SASA + b	0.00720	0.0000
dec ¹¹⁾	mbondi [213]	PBSA ⁷⁾ + GB ^{HTC} [200, 211, 213]	ICOSA ¹²⁾	γ SASA + b	0.00720	0.0000

¹⁾ Radii used for the calculation of the polar solvation free energy.

²⁾ Program or method used for the calculation of the solvent accessible surface area

³⁾ Equation used for the calculation of the nonpolar part of the solvation free energy

⁴⁾ Surface tension (SURFTEN) term in MM-PBSA / MM-GBSA calculations

⁵⁾ Offset (SURFOFF) term in MM-PBSA / MM-GBSA calculations

⁶⁾ Tan&Luo radii for the protein and mbondi radii for the ligand (per default). Radii optimized according to Tan&Luo [260] can be provided in the topology file and will then be regarded in the calculation setup.

⁷⁾ Calculations are conducted with the PBSA module using the “Modified Incomplete Choleski Conjugate Gradient” Poisson-Boltzmann solver.

⁸⁾ E_{dispersion} is calculated by a numerical determination of the solvent accessible surface area.

⁹⁾ Hybrid solvent MM-PBSA calculation according to Metz & Gohlke 2006. Please refer to the respective mm_pbsa.pl execution example provided in Amber 14 for a detailed explanation of the results and their correct interpretation.

¹⁰⁾ The nonpolar solvation free energy is calculated as the sum of the cavity free energy γ MSA + b (where MSA = molecular surface area) and the van der Waals interaction energy between solute and solvent atoms.

¹¹⁾ Decomposition of effective binding free energies requested by the `decomposition` option.

¹²⁾ SASA is calculated by a recursive approximation of a sphere around an atom, starting from an icosahedron.

Specification of input / output directories and formats

lig_struct_path <path> Path to folder containing the ligand structures. All ligand structures should now be available in mol2 format, since the conversion should have been carried out in the MD simulation preparation step.

output_path <path> Path to the basic output directory. This path should be identical to the <output_path> specified in the common MD setup step.

water_in_rec 0 | 1 Set to 1 if crystal water molecules were present in the receptor structure used for MD simulation setup.

General parameters for MM-PBSA / MM-GBSA calculation setup

mmpbsa_calc 0 1	Request setup of files for MM-PBSA or MM-GBSA calculations.
1_or_3_traj 1 3	Specification of the method that shall be used for calculation setup. <i>1-trajectory approach:</i> Requires complex trajectories prepared using <code>traj_setup_method=1</code> in the MD setup step. <i>3-trajectory approach:</i> Requires trajectories of ligand, receptor, and complex prepared with <code>traj_setup_method=3</code> in the MD setup step.
charge_method am1 resp	Charge method that shall be used for the calculations. MD trajectories in which the corresponding charge method was employed for the ligand need to be available. See section 40.3 on how to setup the MD simulations.
additional_library <file>	Optional: Absolute path and name of additional library file. Such a library file is only required if the receptor structure contains non-standard residues.
additional_frcmod <file>	Optional: Absolute path and name of additional parameter file. Such a file is only needed, if not all parameters required to describe the receptor are available in the ff12SB force field.
mmpbsa.pl <file>	Absolute path and name of <code>mm_pbsa.pl</code> executable that shall be used for the calculations. Also a path relative to the AMBERHOME directory can be specified. Note that in the latter case the AMBERHOME variable needs to be set in the <code>mmpbsa_batch.template</code> batch template script. Per default it is assumed that <code>mm_pbsa.pl</code> can be called by <code>\$AMBERHOME/bin/mm_pbsa.pl</code>

Snapshot extraction

extract_snapshots 0 1	Request coordinate extraction.
first_snapshot <number>	Number of the first structure that shall be extracted. Please consider that <number> is equivalent to the sum of the number of the structure in the corresponding trajectory and the number of structures present in all trajectories read in before.
last_snapshot <number>	Number of last structure that shall be extracted. Please consider that <number> is equivalent to the sum of the number of the structure in the corresponding trajectory and the number of structures present in all trajectories read in before.
offset_snapshots <number>	Frequency of snapshot extraction. Every <number> th structure will be extracted from the trajectory.
trajectory_files all <file>	Trajectory that shall be considered in snapshot extraction. For a consistent numbering and addressing of the snapshots request consideration of all trajectories by specifying <code>all</code> . The interval from which snapshots shall be extracted can be defined via the flags <code>first_snapshot</code> , <code>last_snapshot</code> , and <code>offset_snapshots</code> . If individual trajectories shall be used, specify each trajectory file in a separate line starting with the flag <code>trajectory_files</code> . Default = <code>all</code> .
snap_extract_template <file>	Optional: Absolute path and name of input-file for <code>mm_pbsa.pl</code> that shall be used for coordinate extraction. If no file is specified, it is assumed that the default file <code>examples/input_info/extract_snaps.in</code> shall be used.

image_trajectories 1 | 0

If set to 1, snapshots of the specified trajectories will be imaged to the origin before coordinate extraction. It is strongly recommended to use this option for all MM-PBSA / MM-GBSA calculations. Attention: Imaging may require a large amount of additional disk space. Default = 1.

use_imaged_trajectories 1 | 0

If imaged trajectories were generated in a previous FEW run, then these will be re-used for snapshot extraction if `use_imaged_trajectories=1`. In case imaged trajectories already exist and `use_imaged_trajectories=0` the existing trajectories will be renamed and new imaged trajectories will be generated from which then snapshots are extracted. Default = 1.

image_mass_origin 1 | 0

Optional: If set to 1, the receptor is imaged relative to the mass origin instead of the coordinate origin. Switching this flag on ensures compatibility of the imaging procedure with the one of the Amber FEW version. Default = 0.

MM-PBSA / MM-GBSA Analysis

PB 0 | 1 | 2 | 3 | 4

Type of Poisson-Boltzmann calculation (cf. Table 40.4 for an overview of the available calculation options). Please consider that only PB and GB methods requiring the same radii can be run together, i.e. `PB=4` and `GB=1`. All other PB methods can only be run with `GB=0`.

GB 0 | 1 | 2 | 5 | 6

Type of generalized Born calculation (cf. Table 40.4 for an overview of the available calculation options).

decomposition 0 | 1 | 2 | 3 | 4

If larger than 0 energy decomposition of the specified type is performed (cf. `idecomp` in Chapter 22 for decomposition options). Decomposition only works with `PB=4` and `GB=1`.

no_of_rec_residues <number>

Actual number of residues in the receptor structure.

total_no_of_intervals <number>

Total number of intervals that shall be analyzed. Please note that specifying more than one interval is only reasonable, if different offsets between structures shall be considered. Otherwise the energies for subsets of the analyzed snapshots can be calculated using the `mm_pbsa_statistics.pl` script provided in AMBER. Default = 1.

first_PB_snapshot <number>

Number of the first structure to be considered in the analysis.

last_PB_snapshot <number>

Number of the last structure to be considered in the analysis.

offset_PB_snapshots <number>

Offset between structures that shall be considered in the MM-PBSA / MM-GBSA analysis. Every <number>th snapshot will be taken into account.

mmpbsa_batch_template <file>

Absolute path and name of batch template file for the MM-PBSA / MM-GBSA calculations. Example file: `examples/input_info/MMPBSA.pbs`. Please adjust the template according to your computing environment, but keep everything from the section "Prepare calculation" onward and ensure that the line for job naming ends with "-N". The files generated during the calculation will be temporarily stored in the `/tmp` folder of the machine used for the calculation. Thus, not more than one node should be used per calculation.

40.4. Workflow for automated MM-PBSA & MM-GBSA calculations (WAMM)

<code>mmpbsa_batch_path</code> <path>	Optional: If the calculations shall be conducted using a different path than the one used for setup, this path can be specified here. In case no path is given the <output_path> will be used.
<code>mmpbsa_sander_exe</code> <file>	Optional: Absolute path and name of sander executable that shall be used instead of the default executable in \$AMBERHOME/bin
<code>parallel_mmpbsa_calc</code> <number>	Number of processors to use in parallel run. This flag sets the PARALLEL flag in the mmpbsa.in file, i.e. <number> of threads will be run. Default = 1 (serial).
<code>mmpbsa_template</code> <file>	Optional: Absolute path and name of the input file for mm.pbsa.pl that shall be used for the MM-PBSA / MM-GBSA calculations. If no file is specified, the default file located under <code>examples/input.info/mmpbsa.in</code> is taken. The default file can be modified by expert users, but only the following parameters may be changed: VERBOSE, DIELC, INDI, EXDI, SCALE, LINIT, ISTRNG, SALTCON, INTDIEL, and/or EXTDIEL.

Parameters for MM-PBSA calculations with implicit membrane

Implicit membrane MM-PBSA calculations are currently only possible if the Adaptive Poisson-Boltzmann Solver APBS [828–832] is installed on the system where the calculations shall be performed. Furthermore exclusively the combination `PB=3`, i.e. Poisson-Boltzmann calculation with Parse radii [217], and `GB=0`, i.e. no generalized Born calculation, is available (see options for `PB` and `GB` above). In addition, in order to avoid path inconsistencies, the setup of the calculations should be conducted with FEW on the same system where the calculations shall be run. The MM-PBSA calculations with implicit membrane are carried out with the Perl script `$AMBERHOME/AmberTools/src/FEW/miscellaneous/mmpbsa-FEWmem.pl`. For the calculations also the files `apbs_mem_dummy.in` and `apbs_mem_solv.in` or `apbs_mem_dummy_focus.in` and `apbs_mem_solv_focus.in` provided in the `miscellaneous` directory are required. Therefore the path of the FEW version used for the setup of the calculations should not differ from the path under which FEW can be found during the calculations. The parameters for the implicit membrane can be selected and tested with APBSmem (<http://apbsmem.sourceforge.net>) [833]. If you use the implicit solvent, implicit membrane MM-PBSA calculation functionality of FEW, please cite APBS [828] as well as `draw_membrane2` [834] and the extension of FEW for handling membrane systems [835].

<code>membrane_residue_no</code> <number>	Number of residues in the explicit membrane present in the MD simulation that serves as basis for the MM-PBSA calculation. Please consider all residues that are part of the membrane and not only the number of lipids. In the Lipid14 force field for example the lipids are split into head and tail groups, which are treated as separate residues.
<code>implicit_membrane</code> 1 0	If set to 1, an implicit membrane is considered in the MM-PBSA calculation, i.e. the system is embedded in an membrane slab with a lower dielectric constant than water.
<code>apbs_executable</code> 1 0	Full path to APBS executable, e.g. <code>/home/Software/iAPBS/bin/apbs</code> .
<code>epsilon_solute</code> 1 0	Dielectric constant of the solute, i.e. the protein and the ligand, in the MM-PBSA calculation. Please note, that the variable <code>DIELC</code> in the template input script for <code>mm.pbsa.pl</code> specified under <code>mmpbsa_template</code> needs to be set to the same dielec-

		tric constant to ensure that the calculated molecular mechanics electrostatic energies are scaled by the same constant.
<code>bottom_membrane_boundary</code>	<code><no.></code>	Lower boundary of the membrane slab relative to the coordinate origin in [Å]. If more than one slab region is defined please give the lower boundary of the slab that is farthest away from the origin, see Figure 40.5. Default = -18 Å.
<code>membrane_thickness</code>	<code><no.></code>	Thickness of the implicit membrane slab in [Å]. If a membrane slab with different slab regions is defined, please specify the thickness of the complete slab including all sub-slabs, see Figure 40.5. Default = 36 Å.
<code>membrane_dielc</code>	<code><no.></code>	Dielectric constant of the implicit membrane slab. If a multi-slab membrane is constructed, this is the dielectric constant of the central membrane slab closest to the coordinate origin, see Figure 40.5. Default = 2.
<code>second_slab_thickness</code>	<code><no.></code>	Optional: Thickness of a second slab region flanking the central slab on both sides. This slab can e.g. be used to model the properties in the region of or close to the lipid head groups. Please note that the thickness of the central slab defined under <code>membrane_thickness</code> decreases by $2 \times \text{second_slab_thickness}$, see Figure 40.5.
<code>second_slab_dielc</code>	<code><no.></code>	Optional: Dielectric constant of the two second implicit membrane slab regions above and below the central membrane slab (Figure 40.5). This dielectric constant is usually larger than <code>membrane_dielc</code> to describe the properties in the region of or close to the lipid head groups. For a discussion of the complex electrostatic properties of a lipid bilayer see e.g. [836][837].
<code>third_slab_thickness</code>	<code><no.></code>	Optional: Thickness of a third slab region located between the central slab and the second slab on both sides of the central slab (Figure 40.5). Please note that the thickness of the central slab defined under <code>membrane_thickness</code> decreases by $(2 \times \text{second_slab_thickness}) + (2 \times \text{third_slab_thickness})$, see Figure 40.5.
<code>third_slab_dielc</code>	<code><no.></code>	Optional: Dielectric constant of the third implicit membrane slab region sandwiched between the second slab region and the central slab on both sides of the central slab (Figure 40.5). This dielectric constant is usually larger than <code>membrane_dielc</code> to describe the properties of the membrane region close to the membrane surface. For a discussion of the complex electrostatic properties of a lipid bilayer see e.g. [836][837].
<code>ion_concentration</code>	<code><no.></code>	Concentration of ions, i.e. salt, that shall be considered in the Poisson-Boltzmann calculation. Default = 0.15 M.
<code>upper_exclusion_radius</code>	<code><no.></code>	Upper exclusion radius in [Å]. See [833] and Figure 40.5.
<code>lower_exclusion_radius</code>	<code><no.></code>	Lower exclusion radius in [Å]. See [833] and Figure 40.5.
<code>do_focussing</code>	<code>1 0</code>	Perform a three step APBS focussing calculation. In such a calculation three successive calculations are performed starting from a large grid followed by focussing using smaller grids, see http://www.poissonboltzmann.org. Default: 0.

<code>size_large_grid</code> <no.>	Optional: Size of the largest grid in the focussing calculation in [Å]. This is only considered if <code>do_focussing=1</code> . Please choose the size of the grids such that even the smallest grid (<code>size_small_grid</code>) completely comprises the membrane slab in the direction orthogonal to the plane of the membrane slab. Default = 300 Å.
<code>size_medium_grid</code> <no.>	Optional: Size of medium grid in the focussing calculation in [Å]. This is only considered if <code>do_focussing=1</code> . Please choose the size of the grid such that even the smallest grid (<code>size_small_grid</code>) completely comprises the membrane slab in the direction orthogonal to the plane of the membrane slab. Default = 200 Å.
<code>size_small_grid</code> <no.>	Size of the grid, if <code>do_focussing=0</code> , or size of the smallest grid, if <code>do_focussing=1</code> , in [Å]. Please choose the size of the grids such that it completely comprises the membrane slab in the direction orthogonal to the plane of the membrane slab. Default = 100 Å.
<code>grid_dimensions</code> <no.>	Number of grid points in each dimension, i.e. x, y, and z directions, of the grid. Valid values are 97, 129, and 161. Defaults: If <code>do_focussing=0</code> then <code>grid_dimensions=161</code> and if <code>do_focussing=1</code> then <code>grid_dimensions=97</code> .

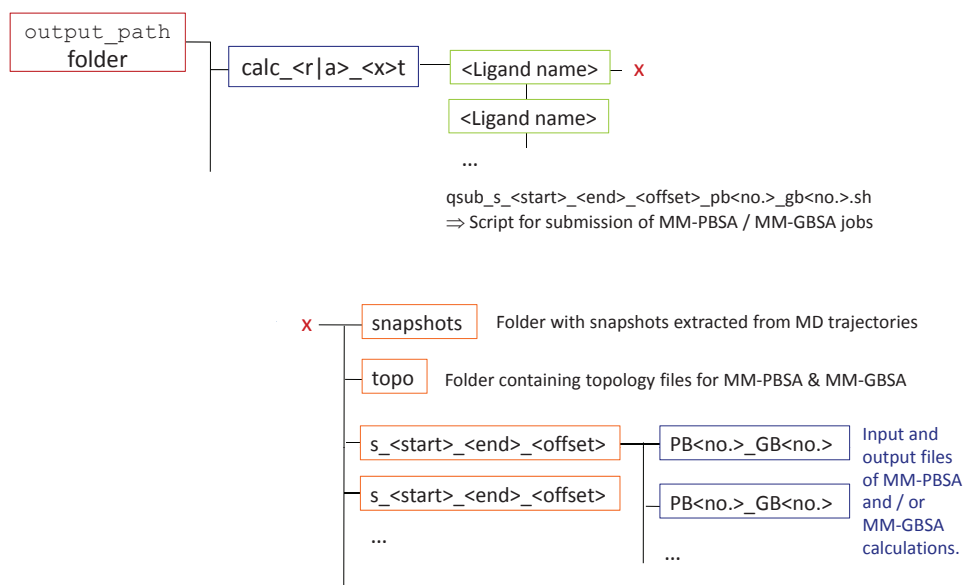


Figure 40.4.: Folder structure and files created during setup of MM-PB/GBSA calculations.

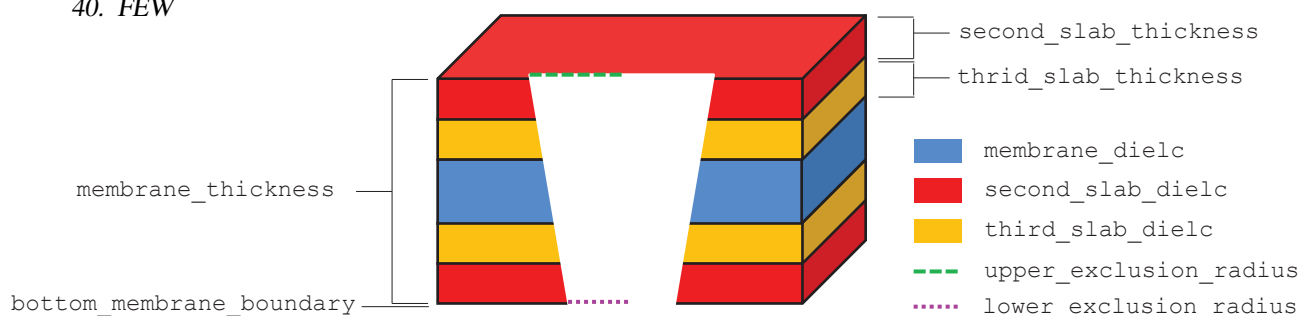
40. *FEW*

Figure 40.5.: Parameters for definition of implicit membrane in MM-PBSA calculations.

MM-PBSA calculation of a protein-protein complex in the membrane

The protein-protein option is an extension of the implicit membrane MM-PBSA functionality in FEW, which allows the identification of important residues in protein-protein interactions for membrane proteins[838]. MM-PBSA energies can be calculated between two proteins/peptides on a global or per-residue basis from MD trajectories of the protein-protein complex in an explicit membrane. Currently, only post-processing of existing trajectories is supported (`mmpbsa_calc=1`) in a 1-trajectory approach (`1_or_3_traj=1`). Requirements for system preparation and input parameter choice are the same as for general MM-PBSA calculations with implicit membrane in FEW (see above). The input trajectories have to be specified explicitly with `trajectory_files` and results are saved in a new directory “`calc.p_1t`” within the directory specified by `output_path`. Per-residue decomposition of energies can be requested with the `decomposition` option (1 or 2 available only). Since the nonpolar part of the solvation free energy is proportional to the SASA in the model used here, this assumption is not true for residues located inside the membrane. To account for this, it is recommended to use the option `nonpolar_solv=1`, which treats all residues located in the implicit membrane as having a SASA of 0 Å². It is recommended to use the default APBS input templates provided with FEW with `do_focussing=1` and to check if the grid dimensions are large enough to account for the protein-protein complex.

protein_protein	1 0	If set to 1, do MM-PBSA calculations of a protein-protein complex in a membrane system.
protein_protein_com	<file>	Path to pdb file of protein-protein complex in explicit membrane needed for protein-protein MMPBSA. This should be the pdb file which was created during system setup for MD simulation with LEaP. It is supposed to contain the proteins, lipids and water.
protein1_res_range	<no.-no.>	Starting and ending residue numbers of the first protein in the protein-protein complex.
protein2_res_range	<no.-no.>	Starting and ending residue numbers of the second protein in the protein-protein complex.
nonpolar_solv	1 0	If set to 1, change SASA to 0 Å ² for all residues inside the membrane. Membrane is defined with "membrane.thickness" and "bottom_membrane_boundary".

Postprocessing:

If MM-PBSA or MM-GBSA calculations without decomposition were conducted for several ligands, the binding free energies and important energetic contributions can be extracted from the `<ligand>.statistics.out` files created by `mm_pbsa.pl` using the script `.../FEW/miscellaneous/extract_WAMMEnergies.pl`.

Usage:

```
perl extract.WAMMEnergies.pl <structure file> <path> pb<no.>_gb<no.> <Start>_<Stop>_<Offset>
```

structure file	Text file containing names of ligands for which energies shall be extracted; one name per line.
path	Path to directory containing MM-PBSA or MM-GBSA results, e.g., /home/<user>/work_dir/calc_r.lt.
pb<no.>_gb<no.>	FEW internal number of type of MM-PBSA / MM-GBSA calculation; see Table 40.4. The script can be used for all types of implicit solvent calculations available in FEW, except the hybrid model (PB2) and decomposition (dec).
<Start>_<Stop>_<Offset>	Snapshots taken into account in the MM-PBSA / MM-GBSA calculations; see flags <code>first_PB_snapshot</code> , <code>last_PB_snapshot</code> , and <code>offset_PB_snapshots</code> in the “MM-PBSA / MM-GBSA Analysis” section above.

A file called `pb<no.>_gb<no.>.txt` will be created in the current working directory. In this file the electrostatic (ELE), van der Waals (VDW), nonpolar solvation (NP_SOLV), and polar solvation (P_SOLV) energy contributions to binding as well as the total binding free energy (ETOT) are listed for each ligand.

40.5. Linear interaction energy workflow (LIEW)

The LIE workflow enables energy calculations according to the linear interaction energy approach introduced by Åqvist et al. [839] and was applied in numerous ligand binding affinity studies [840–842]. In this approach the changes upon complex formation in the electrostatic and the van der Waals interaction energy between a ligand and its surrounding environment are calculated based on MD simulations of the receptor bound ligand and of the ligand in solution. The binding free energy is estimated by combining differences in the electrostatic and van der Waals interaction energies in a linear equation with the coefficients α and β and possibly a constant term γ .

$$\Delta E^{LIE} = \beta \left(E_{bound}^{ele} - E_{free}^{ele} \right) + \alpha \left(E_{bound}^{vdW} - E_{free}^{vdW} \right) + \gamma$$

Commonly β is set to 0.5. However, several alternative strategies for selecting the coefficients and γ exist [840, 843, 844]. Furthermore it has been proposed to consider the difference in solvent accessible surface area between the bound and the free state of the ligand in the calculation of the binding free energy [845, 846].

$$\Delta E^{LIE} = \beta \left(E_{bound}^{ele} - E_{free}^{ele} \right) + \alpha \left(E_{bound}^{vdW} - E_{free}^{vdW} \right) + \gamma (SASA_{bound} - SASA_{free})$$

With the LIE workflow it is possible to setup the required MD simulations and to calculate the electrostatic and van der Waals interaction energy contributions as well as the solvent accessible surface area based on snapshots from the MD simulations by an automated procedure. This enables a fast calculation of the energy components needed for a LIE analysis, making energetic calculations for multiple ligands feasible. The computed energies can be used to construct a LIE model employing a (multiple) linear regression analysis.

The MD simulations can be conducted with *sander* or *PME*MD of Amber. Electrostatic and van der Waals interaction energies of the ligand based on snapshots from the MD simulations are calculated with *sander*.

MD simulations for LIE analysis can be prepared using the common MD setup functionality of FEW described in section 40.3. Only MD setups according to the 3-trajectory approach are possible when the LIE procedure is requested. The receptor part of the 3-trajectory approach will automatically be neglected such that only files for the two simulations required for LIE analysis are generated. Thus, internally a 2-trajectory approach is prepared.

The availability of output/trajectory-files of simulations of the receptor bound ligand and of the ligand free in solution in the folders created by the MD setup procedure of FEW is a prerequisite for the energetic calculations. As for all FEW setup procedures, the command file for the energetic calculations according to the LIE approach needs to contain the flags specifying the input and output directories and formats (see section 40.3 "Common setup of molecular dynamics simulations" for a detailed explanation) as well as procedure specific flags. Example command files for LIE calculation setup are provided in `$AMBERHOME/AmberTools/src/FEW/examples/command_files/LIE`. An overview of the folder structure created by the LIE workflow is shown in Figure 40.6.

Specification of input / output directories and formats

lig_struct_path <path>	Path to folder containing the ligand structures. All ligand structures should be available as single structure mol2 files, because the format conversion should have been carried out in the preparatory step.
output_path <path>	Path to the basis output directory. This path needs to be identical to the <output_path> specified in the common MD setup step.
water_in_rec 0 1	Set to 1 if crystal water molecules were present in the receptor structure used for MD simulation setup.

General parameters for LIE calculations

lie_calc 0 1	Request setup of LIE calculations.
charge_method am1 resp	Charge method that shall be considered for LIE analyses. Trajectories of MD simulations with corresponding atomic charges for the ligand need to be available. The generation of the files required for these simulations is described in section 40.3.
no_of_rec_residues <number>	Actual number of residues in the receptor structure.
additional_library <file>	Optional: Absolute path and name of additional library file. Such a library file is only required if the receptor structure contains non-standard residues.
additional_frcmod <file>	Optional: Absolute path and name of additional parameter file. Such a parameter file is only required if not all parameters that are needed to describe the receptor are available in the ff12SB force field.
lie_executable <executable>	Optional: Absolute path and name of the LIE.pl program for calculation of interaction energies according to the LIE approach, which is distributed with FEW. If no executable is specified, it is assumed that the LIE program can be found under the default path and name at <code>\$AMBERHOME/AmberTools/src/FEW/miscellaneous/LIE.pl</code>
lie_batch_template <file>	Absolute path and name of batch file for LIE analysis. An example file can be found under <code>examples/input_info/lie.pbs</code> . Please adapt the batch file according to the requirements of your queuing system, but do not change anything from the "Prepare calculation" section onward and ensure that the line for job naming ends with "-N".
lie_batch_path <path>	Optional: Path that shall be used instead of the <output_path> for the setup of batch file. This information is only required if the LIE analysis shall be run under a different path than the setup.

Snapshot extraction

snaps_per_trajectory	<number>	Number of snapshots per trajectory. If more than one trajectory file is provided, all trajectory files need to contain the same number of snapshots.
image_trajectories	1 0	If set to 1, the structures will be imaged to the origin before coordinates are extracted. This is strongly recommended. However, please regard that imaging may consume a large amount of disk space, since new trajectories with imaged structures are created. Default=1.
trajectory_files	all <file>	Trajectory files that shall be regarded. For a consistent numbering of the snapshots it is strongly recommended to consider all trajectories that have been generated by specifying <code>all</code> . Subsets of snapshots that shall be considered in the energy calculation can be selected by the parameters <code>first_lie_snapshot</code> , <code>last_lie_snapshot</code> , and <code>offset_lie_snapshots</code> . Individual trajectory files can be selected by specifying their file name (without the path). Each file that shall be considered must be specified in a separate line starting with the keyword <code>trajectory_files</code> . Default = <code>all</code> .

LIE Analysis

first_lie_snapshot	<number>	No. of first snapshot that shall be regarded in the energy calculation.
last_lie_snapshot	<number>	No. of last snapshot that shall be regarded in the energy calculation.
offset_lie_snapshots	<number>	Offset between snapshots that shall be regarded in the energy calculation. Every <code><number></code> th snapshot will be considered.
calc_sasa	0 1	Request calculation of solvent accessible surface area. Default = 0.
sander_executable	<executable>	Optional: Absolute path and name of <i>sander</i> executable that shall be used for the energy calculation if not the default application under \$AMBERHOME/bin shall be employed.
parallel_lie_call	<call>	The calculations can be conducted using a parallel version of <i>sander</i> . If you would like to start a parallel job, please specify the call required for starting a parallel execution of <i>sander</i> on your system here, e.g.: <code>mpirun -np 2</code> . Prerequisite for parallel execution: Parallel version of <i>sander</i> available.
delete_lie_trajectories	0 1	As storing the coordinates of the structures in a form specifically required for LIE analyzes can consume a large amount of disk space, it can be advantageous to only temporarily create them. If <code>delete_lie_trajectories</code> is set to 1, the trajectories for LIE analyzes are deleted directly after the energy calculations.

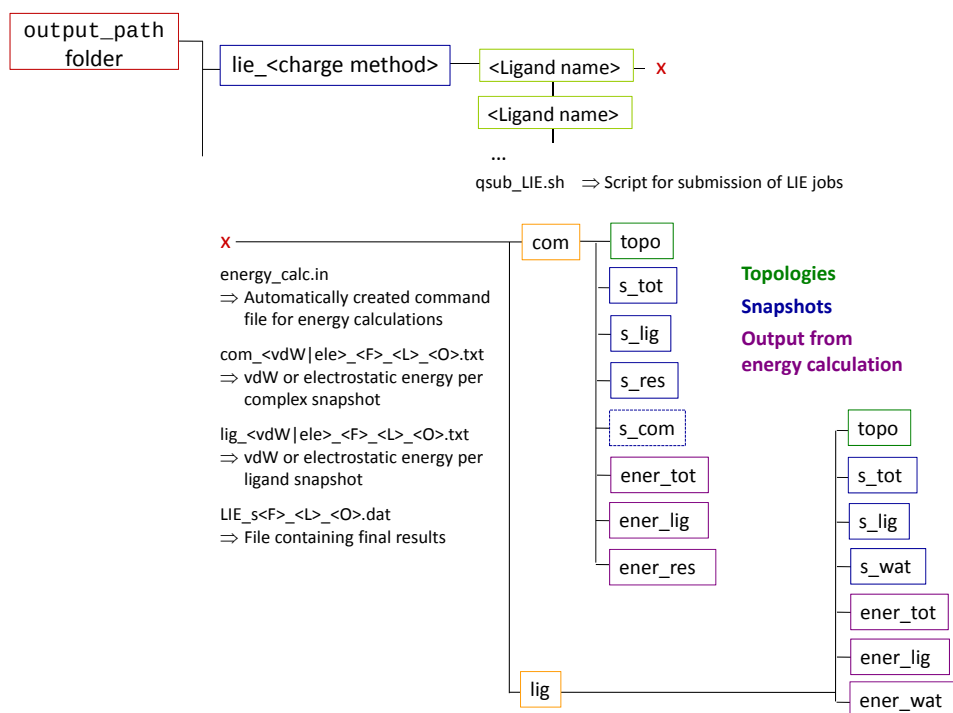


Figure 40.6.: Folder structure and files created during the setup of LIE calculations. For reasons of clarity *first_lie_snapshot*, *last_lie_snapshot*, and *offset_lie_snapshots* were replaced by aliases *F*, *L*, and *O*.

Postprocessing:

If LIE analyzes were conducted for several ligands, the differences in electrostatic and vdW interaction energies can be extracted from the `LIE.s<first>_<last>_<offset>.txt` files using the script `$AMBERHOME/AmberTools/src/FEW/miscellaneous/extract_LIEenergies.pl`.

Usage:

```
perl extract_LIEenergies.pl <structure file> <path> <name of LIE output file>
```

structure file

Text file containing the names of the ligands that shall be considered (one ligand per line) and experimentally measured IC₅₀ or K_i or binding free energies in tab-separated format.

Example:

```
#Ligands dG
Lig_5    -0.5394
Lig_17   -1.3409
```

path

Path to directory containing the LIE results, e.g. `/home/<user>/work_dir/lie_aml`.

name of LIE output file

Name of the final result file of the LIE calculations, i.e. LIE.<first>_<last>_<offset>.txt, where <first>, <last>, and <offset> are equivalent to the values selected for the corresponding <X>_lie_snapshot(s) keywords described above.

A file called LIE.results.txt will be created in the current working directory. In this file, besides the ligand name and the binding affinity value provided in the <structure file>, the differences in electrostatic (ELE) and van der Waals (VDW) interaction energies and the difference in solvent accessible surface area between the bound and the free state are listed. The file can be used directly to derive a linear model by a (multiple) linear regression analysis.

40.6. Thermodynamic integration workflow (TIW)

The TI workflow enables a fast setup of transformation simulations between two ligands for the determination of the difference in free energy of binding according to the thermodynamic integration approach. Transformation simulations are prepared employing the one step, soft core option provided in AMBER. For a detailed description of the method see Section 26.1. Prerequisite for conducting the TI calculation setup with FEW is a parallel installation of the program *sander* of AMBER.

Transformation simulations can either be started from provided structures or from structures that have been pre-equilibrated with FEW. Equilibrated structures of complex and ligand can be prepared using the common MD setup functionality of FEW. See section 40.3 for details on how to prepare the files for minimization and equilibration. Alternatively, the TI setup can be requested based on coordinate and topology files generated from a crystal structure or from other sources. This option can be valuable in cases where high resolution crystal structures are available for the receptor bound state of both the initial (*V0*) and the final (*V1*) ligand and these show only marginal differences with respect to the receptor structure. In case user provided structures shall be employed directly it is necessary to run the common MD setup procedure without providing the flag `MDequil_template_folder`, in order to prepare the files required for the TI calculations. Figure 40.7 illustrates the two setup options and the corresponding workflows.

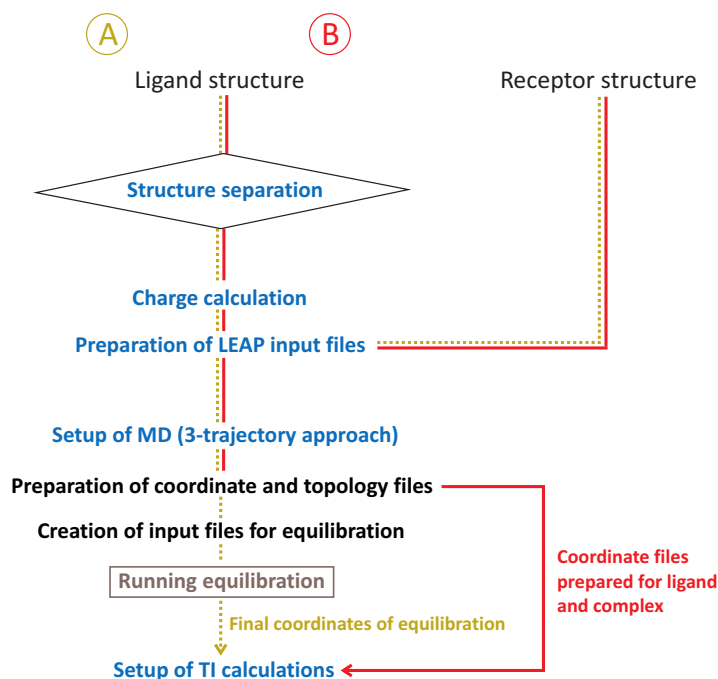


Figure 40.7.: *TI workflow options: TI based on (A) a structure equilibrated using the MD setup functionality of FEW or (B) a user provided structure, e.g., a crystal structure.*

The TI simulations are separated into a TI equilibration and a TI production phase. Input files for the latter can only be prepared when the equilibration simulations have been completed. In the equilibration phase the transformation simulations are conducted sequentially for all λ values in ascending order (Figure 40.8), i.e., the final coordinate file of the equilibration at the smallest λ value serves as input file for the next larger λ value, and so on. Thus, only one batch-job for the equilibration needs to be submitted per system.

1) Structure preparation:

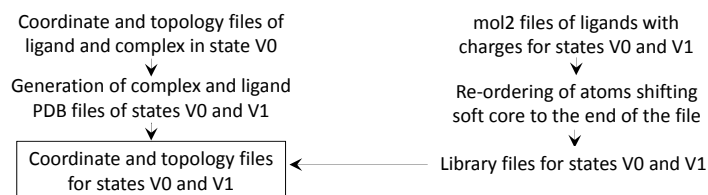
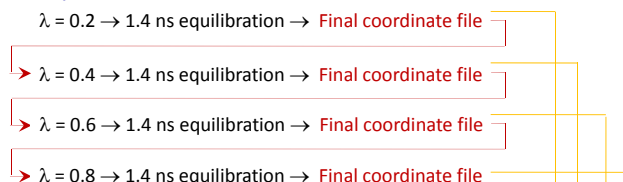
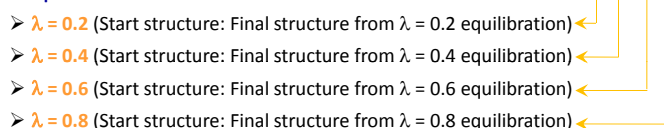
2A) Sequential equilibration with different λ 's:2B) Parallel production simulations with different λ 's:

Figure 40.8.: Internal TI workflow of FEW consisting of structure preparation, equilibration simulations, and production simulations.

Production simulations are started from equilibrated structures, i.e., from coordinate files obtained in the equilibration phase. Prior to the setup of the production simulations it is checked whether the systems are thoroughly equilibrated employing a reverse cumulative averaging procedure [847]. The production simulations, which are prepared when the equilibration check is complete, can be conducted in parallel for all λ values. For each λ value a separate batch script is generated. Production simulations are run either until a convergence measure, calculated after each production step, falls below a specified limit or the total runtime defined in the command file is reached. Two alternative convergence criteria are available: (I) The difference between the current standard error in $dV/d\lambda$, determined according to [597], and the one calculated in the previous step. (II) The precision of $dV/d\lambda$, i.e., the expected deviation of the true mean from the sample mean determined based on a student's distribution at a confidence level of 95%. How often the convergence is checked depends on the simulation time specified in the provided template file for TI production. A convergence analysis is performed after each production run, and if the termination criterion is not reached, the next round of TI production is started. Since the calculation of the convergence measures requires the determination of the autocorrelation time in $dV/d\lambda$, the number of $dV/d\lambda$ values that are written to the *sander* output file should be 10 times larger than the autocorrelation time. As the autocorrelation time is typically in the range of 1 ps [597], it is recommended to request writing of at least 20 $dV/d\lambda$ values in the template production file when a recording interval of 1 ps is used. If the number of $dV/d\lambda$ values is not larger than 10 times the autocorrelation time, the simulation procedure is terminated after the first production step. The convergence analysis is handled by the batch script and does not require user intervention.

When the transformation simulations have been completed (cf. Figure 40.9 for created folder structure), the TIW module of FEW can be used to calculate the difference in free energy of binding between the two studied ligands. The free energy difference ΔG is calculated by numerical integration over the average $dV/d\lambda$ values obtained from the simulations at the individual λ 's, employing the trapezoidal rule. The user can choose whether the commonly applied linear interpolation to $\lambda=0$ and $\lambda=1$ shall be conducted (eq. E1) or the boundary area of the $dV/d\lambda$ curve shall be neglected (eq. E2).

$$\Delta G = \int_0^1 \langle dV(\lambda)/d\lambda \rangle d\lambda \quad (\text{E1})$$

$$\Delta G = \sum \langle dV(\lambda)/d\lambda \rangle \Delta \lambda \quad (\text{E2})$$

Finally the difference in free energy of binding $\Delta\Delta G$ is calculated by subtracting ΔG_{ligand} calculated based on the transformation of the ligand free in solution from $\Delta G_{\text{complex}}$ derived from the transformation within the complex (eq. E3).

$$\Delta\Delta G = \Delta G_{\text{complex}} - \Delta G_{\text{ligand}} \quad (\text{E3})$$

The existence of files created according to the MD setup for the 3-trajectory approach with FEW is a prerequisite for the execution of the TI workflow. Example command files for TI calculation setup are provided in `$AMBERHOME/AmberTools/src/FEW/examples/command_files/TI`.

Specification of input / output directories and formats

lig_struct_path <path>	Path to folder containing the ligand structures. All ligand structures should now be available as single structure mol2 files because the conversion should have been carried out in the preparatory step.
output_path <path>	Path to the basis output directory. This path needs to be identical to the <output_path> specified in the common MD setup step.

TI simulations

Parameters that have to be specified and need to be identical in all subsequent TI setup runs for one system:

ti_simulation_setup 0 1	Request setup of files for TI simulation.
charge_method am1 resp	Charge method that shall be used for the calculations. MD setup files or equilibrated structures generated with the corresponding charge method need to be available. See section 40.3 on how to generate these files.
lig_name_v0_struct <name>	Name of ligand in start state (V0). The name needs to be identical to the name used for the corresponding structure in the MD setup step, i.e. basename of mol2 file.
lig_name_v1_struct <name>	Name of ligand in end state (V1). The name needs to be identical to the name used for the corresponding structure in the MD setup step, i.e. basename of mol2 file.
lig_alias_v0 <alias>	Alias that shall be used for the ligand in the start state (V0). The alias serves, e.g., as ligand residue name and identifier for the TI simulation files and must consist of 3 characters.
lig_alias_v1 <alias>	Alias that shall be used for the ligand in the end state (V1). The alias serves, e.g., as ligand residue name and identifier for the TI simulation files and must consist of 3 characters.
softcore_mask_v0 <mask>	Soft core mask for state V0 to be used for AMBER "scmask" definition. Format: <V0_alias>@<atom>,<atom>,... For details about the format see Section 26.1.

softcore_mask.v1 <mask>	Soft core mask for V1 to be used for AMBER "scmask" definition. Format: <V1.alias>@<atom>,<atom>,... For details about the format see Section 26.1.
use_pmemd 0 1	This parameter specifies with which program TI transformation simulations shall be performed. If not provided or set to zero, input files for Sander are prepared, whereas when set to 1, input files for PMEMD are generated. Input files for PMEMD can be used to run TI calculations with PMEMD on CPUs or GPUs. Please consider that in the later case the batch template script needs to be adjusted so that calculations are started on GPUs and the CUDA version of PMEMD is used.
The following three steps are done by three consecutive calls of FEW according to Figure 40.3.	
1. Creation of coordinate and topology files	
prepare_match_list 0 1	Request setup of match list with atom correspondence information for none soft-core part of states V0 and V1. The list contains the atom names of corresponding atoms in the two states, in tab-separated format. In case the automatic matching fails, the list can also be created manually.
prepare_inpcrd_prmtop 0 1	Request setup of coordinate and topology files. The steps needed for preparation of coordinate and topology files for start and end states are only carried out if this flag is set to 1.
lig_inpcrd.v0 <file>	Coordinate file of solvated ligand start structure in <i>coordinate</i> or <i>restart</i> (<i>inpcrd</i> , <i>restrt</i>) format. Either the end structure of an equilibration simulation or a crystal / model structure can be provided. Please regard that in the later case the structure will be directly subjected to an equilibration MD, without previous minimization, heating and density adjustment. A significant longer equilibration run will be necessary in this case. <i>Attention:</i> The coordinate file must have been prepared with the common MD setup functionality of FEW.
com_inpcrd.v0 <file>	Coordinate file of the solvated complex start structure either in <i>coordinate</i> or <i>restart</i> format (cf. <i>lig_inpcrd.v0</i> flag).
lig_prmtop.v0 <file>	Topology file of the solvated ligand corresponding to the coordinate file specified under <i>lig_inpcrd.v0</i> .
com_prmtop.v0 <file>	Topology file of the solvated complex corresponding to the coordinate file specified under <i>com_inpcrd.v0</i> .
match_list_file <file>	Absolute path and name of a file containing atom correspondence information between states V0 and V1. An example match-file can be found in <i>examples/input_info/match_list.txt</i> . This information must only be provided if the automated generation of the atom correspondence list (<i>prepare_match_list=1</i>) was not successful and the list was created manually.
SSbond_file <file>	Absolute path and name of file containing disulfide bridge definitions for the receptor. For an example file see <i>examples/input_info/SSbridges.txt</i>
chain_termini <no.>,<no.>,...	Numbers of terminal residues of chains in receptor structure, e.g., if a chain ends at residue 234 and a new chain starts with residue 235, the number 234 needs to be specified as <no.>.

<code>create_sybyl_mol2</code>	0 1	Optional: Request generation of mol2 ligand files for V0 and V1 with sybyl atom types. As most molecule visualization programs support this format, the created files allow an easy comparison of atom names of start and end structures to check the correctness of the atom matching step.
<code>additional_library</code>	<file>	Optional: Absolute path and name of additional library file containing information about non-standard residues or ions.
<code>additional_frcmod</code>	<file>	Optional: Absolute path and name of additional parameter file. Such a file is only required if parameters necessary for the description of the receptor are missing in the ff12SB force field.

2. General parameters for preparation of TI transformation simulations

<code>ti_batch_path</code>	<path>	Optional: If the simulations shall be run under a different path than the setup, a new <output_path> for the batch file generation can be specified.
<code>ti_prod_template</code>	<file>	Optional: Template file for TI production simulations. Per default the example file under <code>examples/input_info/MD_prod_TI.in</code> will be used as a template. Please adapt the file according to your needs but keep the format of the lines containing the flags "t", "scmask", and "clambda". If decomposition is requested, please also use the format shown in the example file for the specification of "RES" and "LRES".
<code>no_shake</code>	0 1	Optional: Request calculation without AMBER shake option. In this case ensure that shake is also switched off in the <code>ti_prod_template</code> file. For an example file see <code>examples/input_info/MD_prod_noShake_TI.in</code> . It is recommended to conduct transformations not involving exchanges of atoms in rings or exchanges of single hydrogen atoms with shake (<code>no_shake=0</code>) on hydrogens (<code>ntc=2, ntf=2</code>) to be able to increase the integration step size to 2 fs. Default = 0.

A) Setup of scripts for TI equilibration

<code>ti_equil</code>	0 1	Request setup of files for TI equilibration.
<code>ti_equil_batch_template</code>	<file>	Template batch file for the submission of the equilibration phase job to a queuing system. An example file can be found under <code>examples/input_info/equi_TI.pbs</code> . Please adapt the file according to the needs of your queuing system, but keep everything from the section entitled "Fix variables" up to the section "Re-queue" and ensure that the line for job naming ends with "-N".
<code>ti_equil_lambda</code>	<no.>,<no.>,...	λ values for which TI equilibration calculations shall be prepared in ascending order. Please specify only the decimal digits, e.g. 1 for lambda 0.1, 05 for lambda 0.05. λ values can be in the range 0.01 – 0.99, i.e., 01 – 99 in the FEW internal nomenclature. For equilibration only equi-distant λ values can be used, i.e., $\Delta\lambda$ needs to be equal for all successive λ 's.
<code>ti_equil_template</code>	<file>	Template file for equilibration part of the equilibration phase. For an example file see <code>examples/input_info/equi_TI.in</code> . The equilibration part is followed by a 1 ns free MD simulation for finishing equilibration of the system. For setup of this later

part the template file specified under `ti_prod_template` will be used.

B) Setup scripts for TI production simulations

ti_production 0 1	Request setup of scripts for TI production simulations. Please note that this option requires the presence of the results of the TI equilibration calculations in the corresponding "equi" folder.
ti_prod_lambda <no.>,<no.>,...	λ values for which TI production calculations shall be prepared in ascending order. Please specify only the decimal digits, e.g., 1 for lambda 0.1, 05 for lambda 0.05. λ values can be in the range 0.01 - 0.99, i.e. 01 - 99 in the FEW internal nomenclature.
total_ti_prod_time <time>	Total simulation time per λ value in [ns]. The number of cyclic runs required will be calculated based on the definitions in the <code>ti_prod_template</code> . Please ensure that the MD total simulation time is a multitude of the MD simulation time specified in the production template file. The requested total simulation time will only be reached, if the error limit for simulation termination is not reached before.
ti_prod_batch_template <file>	Template batch file for the submission of the production phase job to a queuing system. An example file can be found under <code>examples/input_info/prod_TI.pbs</code> . Please adapt the file according to your queuing system, but keep everything from the section entitled "Fix variables" up to the section "Re-queue" and ensure that the line for job naming ends with "-N".
converge_check_script <file>	Optional: Absolute path and name of Perl-script used for convergence checking after each production step. If the location of the script is not provided it will be assumed that the script is located under the default location at <code>.../FEW/miscellaneous/convergenceCheck.pl</code>
converge_check_method 1 2	Optional: Method that shall be used for convergence analysis. 1: Difference in standard error of $dV/d\lambda$ between consecutive production runs; 2: Precision of $dV/d\lambda$ determined employing student's distribution. For a detailed explanation refer to the introduction section of the TI calculation module (Section 40.6). Default = 1.
converge_error_limit <limit>	Optional: Error limit that shall be used as termination criterion for the TI production simulations. Default: 0.01 kcal/mol (method 1); 0.2 kcal/mol (method 2). As long as the convergence measure is larger than this limit and the total simulation time has not been reached, the simulation will go on.

3. Calculation of the difference in free energy of binding

$\Delta\Delta G_{\text{binding}}$ can be calculated using a command file containing the following parameters (in addition to the section specifying input / output directories and formats).

ti_ddG 0 1	Request calculation of the difference in free energy of binding between start (V0) and end (V1) ligands.
---------------------	--

charge_method	am1 resp	Charge method (see above).
lig_name_v0_struct	<name>	Name of ligand in the start state (V0). The name needs to be identical to the name used in the setup of the simulations (see above).
lig_name_v1_struct	<name>	Name of ligand in the end state (V1). The name needs to be identical to the name used in the setup of the simulations (see above).
lig_alias_v0	<alias>	Alias that shall be used for the ligand in the start state (V0). The alias must be identical with the alias used for the setup of the TI simulations (see above).
lig_alias_v1	<alias>	Alias that shall be used for the ligand in the end state (V1). The alias must be identical with the alias used for the setup of the TI simulations (see above).
dVdL_calc_source	<no.>-<no.>	Range of files from the production phase of the TI simulations that shall be considered in the calculation of the difference in free energy of binding. If set to "0", all recorded files will be considered. If only files in a certain range shall be regarded, specify the range, e.g., the range "3-5" will result in considering of the files xxx_prod03_v1.out, xxx_prod04_v1.out and xxx_prod05_v1.out from the production run of the TI simulations. In case all files from a certain time point onward shall be regarded, provide a range that ends with zero, e.g. "4-0".
ddG_calc_method	0 1	Method that shall be used for the calculation of $\Delta\Delta G_{\text{binding}}$. If "1" is specified, the common procedure with linear interpolation to $\lambda=0$ and $\lambda=1$ is used. In case of "2", no linear interpolation is conducted. The later calculation method can only be used, if the production simulations were run with equi-distant λ values, i.e., $\Delta\lambda$ was of the same size for all successive λ 's.

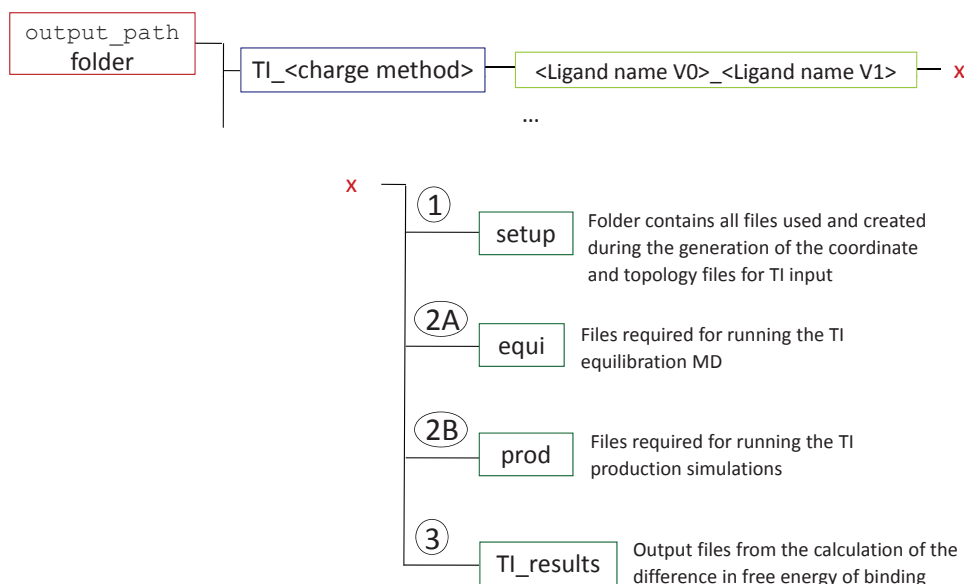


Figure 40.9.: Folder structure and files generated during TI calculation setup with FEW. Numbering according to steps shown in Figure 40.8.

Script for the identification of “optimal” transformations

If several ligands shall be studied by thermodynamic integration the shortest path algorithm of Kruskal [848] can be used to determine the “optimal” transformations between the ligands, i.e. those that require overall the smallest structural changes. In this way, relative binding free energies can be computed between those ligand pairs that show overall the highest similarity. A script called `identify_transformations.pl` provided in the miscellaneous folder of FEW can be used to identify the “optimal” transformations. This script employs Kruskal’s algorithm to determine those transformations that lead to the smallest overall score based on a matrix of similarity scores. Such matrix of similarity scores can for example be obtained by a pairwise ligand comparison employing the TanimotoCombo score of ROCS [849, 850]. The script uses the Perl module `Graph::Kruskal`, which needs to be downloaded from CPAN (<http://www.cpan.org/modules/index.html>) and installed as part of the local Perl installation before the script `identify_transformations.pl` can be used.

Usage:

```
perl identify_transformations.pl <number of structures> <score matrix file>
```

number of structures

Integer number specifying the number of structures that shall be regarded in the search for “optimal” transformations.

score matrix file

Absolute path and name of file containing the score matrix based on which the “optimal” transformations are determined. This matrix file should be in tab-separated text format. The similarity matrix needs to comprise $N \times N$ score values, where N is the number of ligands that shall be regarded. It is assumed that smaller scores correspond to a higher similarity between ligands. The first line and the first column should contain the names of the ligands.

Example - section of score matrix file:

40. FEW

	L01	L02	L03	L04	...
L01	0	217	284	199	...
L02	217	0	285	427	...
L03	284	285	0	118	...
L04	199	427	118	0	...
...	...	...	...	...	...

41. BAR/PBSA

41.1. Introduction

Alchemical simulations in standard additive force fields are unable to handle electronic polarization effects upon ligand transfer from water to the protein interior, leading to inaccurate prediction of binding affinities for charged molecules/binding pockets. The BAR/PBSA method has been developed to address this issue. It can be used to post-process alchemical simulation trajectories to incorporate the polarization effects in a continuum manner.

This is based on the theory that the protein dielectric constant in Poisson-Boltzmann models correlates with the strength of electronic polarization in the protein environment. In typical PBSA calculations, the default `epsin` value of 1 is used for Amber force fields as this is how the force fields are designed and calibrated for molecular dynamics. In doing so, no electronic polarization is included in the PBSA calculations. The consequence in the use of default `epsin` value of 1 is that Coulombic interactions are not screened by electronic polarization, resulting in exaggerated electrostatic interactions even if this is consistent with the force field design. In general, the overestimation can be alleviated by increasing the solute `epsin` value to imitate the effect of electronic polarization that screens electrostatic interactions.

The next question is how to set the solute dielectric constant. The additive force fields are developed with effective partial charges to model electrostatics and include polarization responses to the environment (mostly in water), though only in an averaged, mean-field manner. They are not fully compatible with the theoretical dielectric constant of 2 because polarization is already partially accounted for in the effective partial charges. Thus, a further scanning procedure to find the optimal solute dielectrics is necessary. This is not a satisfactory solution until a polarizable force field can be used in routine alchemical simulations.

Before the dielectric scanning procedure, pre-optimization of the ligand and protein atomic radii are often necessary to achieve the highest accuracy in binding free energy prediction as the atomic radii assigned by the `prmtop` building process usually do not reproduce the solvation free energies from alchemical simulations in a given solvent model.

In summary, to post-process alchemical simulations with BAR/PBSA, ligand radii are first scaled by the `radiscale` PBSA keyword that is optimized to minimize the absolute deviation between PBSA and explicit-solvent electrostatic free energies for the ligand alchemical simulations. Next given the optimized `radiscale` value, the protein radii are then scaled by the `protscale` PBSA keyword that is optimized to minimize the absolute deviation between PBSA and explicit-solvent electrostatic free energies for the complex alchemical simulations. Both steps can be realized by using the BAR/PBSA method presented below. Following calibration of the atomic radii, i.e. `radiscale` and `protscale` optimized from the previous steps, the BAR/PBSA method can then be utilized for the investigation of electronic polarization by varying the solute dielectric constant, `epsin`.^[851] Further details can be found in our recent publication.^[852]

41.2. Usage

`bar-pbsa.py` is a python script that automates the preparation of trajectories from the decharging step of alchemical simulations for BAR/PBSA analysis of binding free energies.^[852] The script is separated into four stages:

1. Stripping solvent and ions from trajectories
2. Preparing *sander* PBSA input files with target surface area and `epsin` parameters
3. Running *sander* in parallel

41. BAR/PBSA

4. Calculating final decharging energies through BAR

Setup parameters are input with YAML formatted files, examples and descriptions of the arguments are provided below.

41.2.1. Stripping solvent and ions from explicit solvent trajectories

Prepare explicit solvent trajectories for *sander* PBSA by stripping water molecules and ions, concatenating replicate trajectories, and autoimaging/RMSD aligning snapshots to the first frame. Keep dummy counter-ions for decharging and ligand atoms through Amber mask selection. Setup can ignore the first half frames from each lambda window to avoid unequilibrated snapshots.

```
dest_path: '1C5X'
complex_paths:
  - '/home/bar_pbsa_demo/1C5X/t1/complex'
  - '/home/bar_pbsa_demo/1C5X/t2/complex'
ligand_paths:
  - '/home/bar_pbsa_demo/1C5X/t1/ligands'
  - '/home/bar_pbsa_demo/1C5X/t2/ligands'
ligand_mask: ':DRG'
ion_decharge: True
last_half_frames: True
stride: 1
```

dest_path	Location to output stripped trajectories.
complex_paths	Paths to explicit solvent trajectories for complex decharging, multiple replicates (t1/t2 above) should be used to improve convergence of free energy simulations.
ligand_paths	Paths to explicit solvent trajectories for ligand decharging.
ligand_mask	Amber mask to select ligand atoms.
ion_decharge	Boolean to indicate whether counter-ion decharge should be performed to maintain charge neutrality. The decharged counter-ion is automatically identified by its lack of charge.
last_half_frames	Boolean to indicate whether only frames from the last half of the trajectory should be kept. This allows removal of unequilibrated data.
stride	Step to subsample frames and remove correlated data from sequential snapshots, higher values minimize the amount of processing necessary.

The source folders for the complex and ligand trajectories should follow the listed folder structure:

```
. `-- 1C5X
   |-- t1
   |   |-- complex
   |   |   |-- 0.000
   |   |   |   |-- ti.parm7
   |   |   |   |-- ti001.nc
   |   |   |-- 0.200
   |   |   |   |-- ti.parm7
   |   |   |   |-- ti001.nc
   |   |   |-- 0.400
   |   |   |   |-- ti.parm7
   |   |   |   |-- ti001.nc
   |   |   |-- 0.600
```

```

| | |-- ti.parm7
| | |-- ti001.nc
| |-- 0.800
| | |-- ti.parm7
| | |-- ti001.nc
| |-- 1.000
| |-- ti.parm7
| |-- ti001.nc
|-- ligands
|-- 0.000
| |-- ti.parm7
| |-- ti001.nc
|-- 0.200
| |-- ti.parm7
| |-- ti001.nc
|-- 0.400
| |-- ti.parm7
| |-- ti001.nc
|-- 0.600
| |-- ti.parm7
| |-- ti001.nc
|-- 0.800
| |-- ti.parm7
| |-- ti001.nc
|-- 1.000
|-- ti.parm7
|-- ti001.nc

```

41.2.2. Preparing *sander* PBSA input files

Write *sander* PBSA input files with target radii scaling factors (`radiscale`, `protscale`) and solute dielectric (`epsin`) for post-processing each trajectory. This is carried out for both the ligand and complex paths. A new folder with copies of the stripped trajectories and input file will be created.

```

dest_path: '1C5X'
ligand_res: 'DRG'
istrng: 150
epsin: 1.0
radiscale: 1.0
protscale: 1.0

```

dest_path	Path to stripped trajectories.
ligand_res	Ligand residue name, used to apply <code>radiscale</code> .
istrng	Salt concentration in mM.
epsin	Protein dielectric constant, higher values more strongly screen charged interactions.
radiscale	Ligand atoms radii scaling factor.
protscale	Protein atoms radii scaling factor.

41.2.3. Running parallel *sander* post-processing trajectories

Run BAR/PBSA *sander* calculations for neighboring lambdas with multiprocessing. This stage is carried out separately for the complex and ligand paths due to memory limitations. In this way both sets of trajectories do not have to be stored together.

```
dest_path: '1C5X'
epsin: 1.0
radiscale: 1.0
protscale: 1.0
ligcom: 'complex'
del_traj: True
```

dest_path	Path to stripped trajectories.
epsin	Protein dielectric constant, higher values more strongly screen charged interactions.
radiscale	Ligand atoms radii scaling factor.
protscale	Protein atoms radii scaling factor.
ligcom	Option to indicate whether the setup is being performed for the 'ligand' or 'complex' trajectories.
del_traj	Boolean to delete trajectories after post-processing to save memory space. The trajectories are copied for each new combination of <code>radiscale</code> , <code>protscale</code> , and <code>epsin</code> and are redundant.

41.2.4. Calculating total decharging energies

Run BAR to calculate decharging energies at each lambda window and aggregate data to obtain the decharging energy for the full process. Perform individually for the complex and ligand paths.

The input YAML to calculate decharging energies is identical to the input for the run stage.

41.3. Example for *bar_pbsa.py*

An example run for processing a single ligand trajectory and a single complex trajectory can be found in `$AMBERHOME/AmberTools/test/bar_pbsa`.

- Strip source trajectories.

```
python bar_pbsa.py strip strip_input.yaml
```

- Prepare *sander* input files with selected `radiscale`, `protscale`, and `epsin`.

```
python bar_pbsa.py prep prep_input.yaml
```

- Run *sander* in parallel, the ligand and complex are run separately. Select the max number cores for parallel jobs with `-n`.

```
python bar_pbsa.py run lig_input.yaml -n 8
python bar_pbsa.py run com_input.yaml -n 8
```

- Calculate final decharging energies, uses same input files as run step. Log with the final energies will be saved in the directory of the run.

```
python bar_pbsa.py calc lig_input.yaml
python bar_pbsa.py calc com_input.yaml
```


42. edgembarr

42.1. edgembarr

The edgembarr program [853] calculates the free energy of an alchemical transformation in 1- or 2-environments using the MBAR[621] and thermodynamic integration methods. The input is an XML file describing the available data, and the output is a python script. Execution of the python script on the command line produces an HTML report of the transformation, including error estimates, time series analysis, convergence properties, and decomposition of the relative free energy into components. Multiple python output files can be used with the edgembarr-WriteGraphHtml.py script (42.3) to perform network analysis. The edgembarr program is a part of the FE-ToolKit library. The most recent development version, extended documentation, and examples can be found [online](#).

It is recommended that one configure Amber with the -DOPENMP cmake option to compile edgembarr.OMP, which parallelizes the calculation with OpenMP.

Usage: edgembarr [options] inpfiler

The input is an XML-formatted file with the following structure.

```
<edge name="LIG1~LIG2">
<env name="target">
<stage name="STAGENAME">
<trial name="TRIALNAME">
<dir>path/to/data</dir>
<ene>State0</ene>
...
</trial>
</stage>
</env>
<env name="reference">
<stage name="STAGENAME">
<trial name="TRIALNAME">
<dir>path/to/data</dir>
<ene>State0</ene>
...
</trial>
</stage>
</env>
</edge>
```

The XML file should contain only 1 edge. It may contain 1 or 2 environments. If there is only 1 environment, then it must be named “target”. If there are 2 environments, the other is called “reference”. The net free energy of the edge is the difference between the 2 environments; i.e., $\Delta\Delta G = \Delta G_{target} - \Delta G_{ref}$. Each environment consists of 1-or-more stages. For example, a “3-stage” strategy might consist of “decharge”, “soft-core van der Waals”, and “recharge” stages; however, more advanced single-stage transformations are possible using the “smooth-step” soft-core potentials implemented in the CUDA-version of pmemd (see 26.1.7.1). The transformation free energy in an environment is the sum of stage free energies; i.e., $\Delta G_{env} = \sum_{stage} \Delta G_{env,stage}$. Each stage consists of 1-or-more trials. A trial is an independent sets of simulations that may be repeated to better estimate the statistics. The transformation free energy of a stage is the average of the trials; i.e., $\Delta G_{env,stage} = N^{-1} \sum_{trial} \Delta G_{env,stage,trial}$. A trial is a series of alchemical free energy simulations that connect two end-states ($\lambda = 0$ and $\lambda = 1$).

The `<dir>` node is a path to a directory containing a series of files named `efep_tlam_elam.dat` and `dvd1_tlam.dat`. Each of these data files should consist of 2 columns. The first column is the simulation time (in ps), and the second column is an energy (in kcal/mol). The rows are the samples to be analyzed. The placeholders “tlam” and “elam” correspond to the value of lambda used to generate the trajectory and evaluate the energy, respectively. The data within the efep files are the potential energies of lambda state elam (the energy lambda) evaluated at the samples generated from the simulation of state tlam (the trajectory lambda). The dvd1 files are the TI derivatives DVDL produced by state tlam.

The `<ene>` nodes list the values of tlam. Note that these are interpreted as character strings that denote the thermodynamic states. The labels listed in the XML must match the labels appearing in the filenames. The free energy of the trial is the difference in free energy between the two end-states: $\Delta G_{env,stage,trial} = G_{env,stage,trial}(\lambda = 1) - G_{env,stage,trial}(\lambda = 0)$. The label for the $\lambda = 0$ state is the first occurrence of `<ene>` within a trial, and the label for the $\lambda = 1$ state is the last occurrence of `<ene>` within a trial. Therefore, to reverse the sign of the trial free energy, one can simply reverse the order of the `<ene>` nodes appearing within a trial. The efep and dvd1 files can be generated by processing the mdout files with `edgembar-amber2dats.py` (see 42.2). This script reads the DVDL and MBAR Analysis output (see 26.1.9) listed in the mdout file(s) and writes the efep and dvd1 data files to a directory for analysis.

The available options are listed below:

- `--out, -o` Output python file. If not specified, then '.py' is appended to the input filename
- `--temp, -t` Temperature (K) [Default: 298]
- `--tol, -T` Optimization relative tolerance [Default: 1.e-13]
- `--btol, -B` Optimization relative tolerance during bootstrap calculations [Default: 1.e-7]
- `--ptol, -p` The p-value (significance level) used in the Welch T-test for determining the production region of each simulation. Large values ($p \sim 1.0$) will cause more of the simulation to be discarded; small values ($p \sim 0.0$) will include more samples within the production region. [Default: 0.05]. p-values less than ptol implies that the difference between two means is significant. If $ptol \leq 0$, then T-tests are not performed; instead, it checks if the difference in means is less than 1 standard error. The default value is $1 - 0.68270473138$, where 0.6827... is a 68% confidence interval (1 standard error), such that the T-test is very similar to accepting two means as being the same if $|\mu_1 - \mu_2| < 1.0 * \sqrt{err_1^2 + err_2^2}$.
- `--dtol` If the first and last half means are within dtol of each other, then ignore the T-test and assign $p=1.0$ [Default: 0.1 kcal/mol]
- `--nboot, -b` Number of bootstrap samples to estimate errors [Default: 20] (Recommended: 20 or higher)
- `--verbosity, -v` Verbosity level. No printing is ≤ 0 . [Default: 0] (0 is recommended if openmp parallel.)
- `--null, -G` If present, then the initial guess free energies are 0. The default is to use exponential averaging. Use the `--null` option if exponential averaging is numerically unstable.
- `--ncon, -c` Create a spline of the objective function using $2n+1$ points centered about the unconstrained solution. [Default: 2, resulting in a 5-point spline] Setting this value to 0 will skip the spline generation. Precomputation of the objective function is necessary to later perform network analysis.
- `--dcon, -C` The maximum displacement of the generated spline in kcal/mol. [Default: 2.0]
- `--ntimes, -A` The number of time series calculations. [Default: 4, which schedules: 0.25, 0.50, 0.75, 1.00]
- `--halves, -H` Perform first- and last-half analysis of the correlated samples after excluding some portion of equilibration. This form of time series analysis splits the samples into two sections: proposed

equilibration period to be excluded and the proposed production region. It then splits the production region in to two halves: the first half of the proposed production region and the second half of the proposed production region. It then calculates the free energies of each half. Ideally, the two halves of the production region would produce the same free energies. This process is repeated N times, based on the value of the `--ntimes` option. Each evaluation discards a different amount of sampling as equilibration.

- `--fwdrev, -F` Perform forward and reverse time series analysis of the correlated samples. This form of analysis calculates the free energies using the samples from the first $X\%$ of the simulations and compares it to the free energies computed from the last $X\%$ of the simulations. If there is a linear drift in the free energy values or a large gap between the forward and reverse values, then it is an indicator that the samples contain unequilibrated data. The values of X are chosen from the `--ntimes` option.
- `--fstart` Skip the first $\langle f \rangle$ fraction of samples when reading the data files. This option is applied to all trajectories. [Default: 0.0]
- `--fstop` Ignore the last $\langle f \rangle$ fraction of samples when reading the data files. This option is applied to all trajectories. [Default: 1.0]
- `--fmaxeq` Do not discard more than the first $\langle f \rangle$ fraction of the simulations even if the auto-equilibration detection algorithm believes it should be discarded [Default: 0.5]
- `--ferreq` Flag a simulation as being unequilibrated if the detection algorithm believes more the first $\langle f \rangle$ fraction of the samples should be discarded. This value should be greater-than-or-equal-to `fmaxeq`. The default value (-1.0) causes the `ferreq` to adopt the same value as `fmaxeq`. That is, you only need to set `ferreq` if you want it to have a different value than `fmaxeq`. [Default: -1.0]
- `--stride` Stride through the data when reading the data files. [Default: 1] This option is applied to all trajectories. It is useful when the user has generated an excessive amount of data that needs to be reduced in order to improve performance.
- `--no-auto` The default behavior is to perform an automatic equilibration procedure that identifies the production region of each simulation and extract the statistically independent samples. In other words, the program tries to automatically choose values of `--fstop` and `--stride` for each trajectory that you provide. If `--no-auto` is used, then all samples from `--fstart` to `--fstop` are analyzed and the errors are estimated from block bootstrap analysis, where the block size is chosen from the autocorrelation in the data.
- `--mode` Defines the objective function and the expected filenames.
 - `--mode=AUTO` (default) : Automatically detect based on the existing filenames.
 - `--mode=MBAR` : A full matrix of trajectories and energies
 - `--mode=MBAREXP0` : MBAR between windows $[2, N]$ and exponential averaging from $2 \rightarrow 1$
 - `--mode=MBAREXP1` : MBAR between windows $[1, N-1]$ and exponential averaging from $N-1 \rightarrow N$
 - `--mode=MBAREXP` MBAR between windows $[2, N-1]$ and exponential averaging from $N-1 \rightarrow N$ and $2 \rightarrow 1$
 - `--mode=BAR` $N-1$ pairs of MBAR objectives connecting adjacent states $[i, i+1]$
 - `--mode=BAREXP0` BAR between all adjacent pairs of states in the range $[2, N]$, and exponential averaging from $2 \rightarrow 1$
 - `--mode=BAREXP1` BAR between all adjacent pairs of states in the range $[1, N-1]$, and exponential averaging from $N-1 \rightarrow N$
 - `--mode=BAREXP` BAR between windows $[2, N-1]$ and exponential averaging from $N-1 \rightarrow N$ and exponential averaging from $2 \rightarrow 1$

42.2. edgembar-amber2dats.py

Extracts the DVDL and MBAR data from 1-or-more mdout files and writes a series of data files using the naming convention expected by the edgembar program.

Usage: `edgembar-amber2dats.py [options] mdout [mdout ...]`

More than 1 mdout file can be listed. A typical usage case is: `edgembar-amber2dats.py --odir ./dats $(ls *.mdout)`

The available optional argument are listed below:

`-h, --help` show this help message and exit

`-o, --odir` Output directory. If empty, then same dir as mdout

`--nmax` Maximum number of samples to extract from an mdout file [Default: 10000]

42.3. edgembar-WriteGraphHtml.py

Performs free energy analysis of an alchemical transformation network. [853]

Usage: `edgembar-WriteGraphHtml.py [options] edge.py [edge.py ...]`

The inputs are the python output files produced by edgembar for each edge in the transformation network. The output is an HTML file.

The available optional argument are listed below:

`-h, --help` show this help message and exit

`-o, --out` Output HTML filename

`-s, --solver` The method for determining the graph solution. This is either “linear”, “mixed”, or “nonlinear”. If unspecified, a linear solution is obtained (this is the recommended solver).

`--exclude` Exclude the specified edge. This is a string, e.g., “LIG1~LIG2”. This option can be used multiple times.

`--refnode` Name of node to select as the reference. If unspecified, the node with the lowest free energy is selected.

`-x, --expt` Filename containing the node names and experimental/reference free energies, if available. This file should have two columns. The first column is a string: the name of the node. The second column is the reference free energy of the node in kcal/mol.

`-c, --constrain` Constrain the specified edge. The constraint value is the difference between the experimental/reference node free energies listed in the --expt file. The value of --constrain is a string, e.g., “LIG1~LIG2”. This option can be used multiple times.

42.4. edgembar-bookend2dats.py

Similar to edgembar-amber2dats.py; however, instead of extracting MBAR and TI data from a pmemd alchemical free energy simulation, it instead extracts MBAR and TI data from a sander simulation that linearly transforms between a MM and QM/MM Hamiltonian. If the sander simulation used the `ijarzynski=1` option (see 26.7), then the edgembar-bookend2dats.py script will extract the nonequilibrium work values, which edgembar analyzes in the same manner as exponential averaging/Zwanzig's equation.

Usage: `edgembar-bookend2dats.py [options] mdout [mdout ...]`

The inputs are sander (linear transformation) alchemical free energy simulation mdout files. The output is a collection of `efep_*.dat` time series files used by edgembar to perform free energy analysis.

The available optional argument are listed below:

`-h, --help` show this help message and exit

`-o, {odir` the output directory [Default: `./`]

42.5. fetkutils-tischedule.py

Optimizes a schedule of alchemical states from the analysis of short burn-in simulations performed with a larger number of states. The new schedule is chosen such that adjacent states equalize a property. The possible properties are: predicted replica exchange acceptance ratio (AR), phase space overlap (PSO), and symmetrized Kullback-Leibler divergence (KL). The theoretical description of the approach is provided in reference [854].

Usage: `fetkutils-tischedule.py [options] directory`

The directory is expected to contain a complete matrix of `efep_slam_elam.dat` files, where `slam` and `elam` are lambda states used in the burn-in simulations. Specifically, `slam` is the lambda state used to perform the burn-in sampling, and `elam` is the lambda state whose potential energy is stored within the file.

The available optional argument are listed below:

`-h, --help` show this help message and exit

`--opt` Optimize a schedule of OPT states. If this option is not used, then `--read` must be used. The value of OPT is the integer number lambda values in the schedule. If `--varlam-minval > 0`, then the value of `--opt` is the minimum allowable number of lambda values in the schedule. The `varlam` options are only used if you want the script to dynamically vary the number of lambda states.

`--varlam-minval` If `--varlam-minval > 0`, then optimize a schedule whose size is given by `--opt`. If the minimum PSO/AR/KL is less than the given minimum value, then increase the schedule by `--varlam-multiple`. The value of `--opt` must be divisible by `--varlam-multiple` on input. [Default: 0]

`--varlam-multiple` The output schedule size produced from `--varlam-minval` will be an integer multiple of `--varlam-multiple` [Default: 4]

`--read` Read a schedule from the file READ. If this option is not used, then `--opt` must be used. If `--ssc` is not used, then the value of READ is a filename. The file should contain 1 column; the rows are the lambda values. If `--ssc` is used, then the value of READ is an integer: the number of lambda values. See also: `--alpha0` and `--alpha1`.

`--o, --out` Write the schedule to a file named OUT.

`--pso` Optimize a schedule to minimize the variance in neighbor phase space overlaps.

42. edgembar

<code>--ar</code>	Optimize a schedule to minimize the variance in neighbor acceptance ratios.
<code>--kl</code>	Optimize a schedule to minimize the variance in neighbor Kullback-Leibler divergences.
<code>--sym</code>	Constrain the optimized schedule to be symmetric about $\lambda=0.5$.
<code>--ssc</code>	Constrain the optimized schedule to adhere to a 1 or 2 parameter analytic mapping function. If <code>--ssc</code> is used, then the mapping function contains 2 parameters (α_0 and α_1) yielding an asymmetric schedule. If <code>--ssc</code> and <code>--sym</code> are used, then the mapping function contains 1 parameter (α_0) to produce a symmetric schedule. The mapping is a uniform discretization of a linear combination of smoothstep softcore polynomials.
<code>--maxgap</code>	Search for a schedule whose gaps between adjacent states are less than or equal to MAXGAP [Default: 1.0]
<code>--spen</code>	Apply an optimization penalty if the property is less than SPEN to avoid having the optimization get trapped in an unwanted minimum [Default: 0.1]
<code>--digits</code>	Round the schedule to this number of digits to the right of the decimal [Default: 5]
<code>--start</code>	Exclude this percentage of simulation as equilibration [Default: 0.]
<code>--stop</code>	Exclude all samples after this percentage [Default: 1.]
<code>-T, --temp</code>	Temperature [Default: 298]
<code>--plot</code>	Write heatmap and path projection to an image file named PLOT.

43. SAXS

43.1. Introduction and theory

Small angle X-ray scattering (SAXS) is a solution based technique that is conventionally used to probe the shape and structure of (bio)molecules. It has long been recognized that the solvent shell around the molecule significantly impacts the shape of the measured SAXS profile. Experimentally, X-ray scattering on biomolecules compare the scattering intensity from the sample of interest to a "blank" with just solvent present, and report the difference, or "excess" intensity:

$$I(\mathbf{q}) = \langle |A(\mathbf{q})|^2 \rangle_t - \langle |B(\mathbf{q})|^2 \rangle_t$$

where the $\langle \rangle_t$ bracket indicates the intensities are averaged over the measurement time and volume. $A(\mathbf{q})$ and $B(\mathbf{q})$ are Fourier transforms of the scattering amplitudes for the sample and blank, respectively:

$$\langle |A(\mathbf{q})|^2 \rangle = \int \langle \tilde{A}(\mathbf{r}) \tilde{A}(\mathbf{r}') \rangle e^{-i\mathbf{q} \cdot (\mathbf{r} - \mathbf{r}')} d\mathbf{r} d\mathbf{r}'$$

with $\tilde{A}(\mathbf{r})$ is the electron density in the system. It has been shown that the total intensity can be approximately (though usefully) rewritten as:[855, 856]

$$I(\mathbf{q}) = [\langle A_1(\mathbf{q}) \rangle - \langle B_1(\mathbf{q}) \rangle]^2 + \left[\langle |A_1(\mathbf{q})|^2 \rangle - |\langle A_1(\mathbf{q}) \rangle|^2 \right] - \left[\langle |B_1(\mathbf{q})|^2 \rangle - |\langle B_1(\mathbf{q}) \rangle|^2 \right] \quad (43.1)$$

where $A_1(\mathbf{q})$ and $B_1(\mathbf{q})$ are Fourier transforms for the sample and blank but here only considering regions where there is excess/deficit electron density relative to the bulk value. In RISM, the second and the third terms vanish, leading to:

$$I(\mathbf{q}) = [\langle A_1(\mathbf{q}) \rangle - \langle B_1(\mathbf{q}) \rangle]^2 \quad (43.2)$$

There are now two SAXS programs in Amber: `saxs_rism` for calculating SAXS from distribution function of solvent in grid format (dx files) from 3D-RISM, another one is `saxs_md` which takes input as two sets of coordinates extracted from snapshots of "sample" and "blank" MD simulations (the "sample" MD contains the biomolecule plus water and ions, while the "blank" MD only has pure water + salt).

43.1.1. saxs_rism

Intensity is calculated based on eq. 43.2, neglecting the time-correlation of solvent density. The total excess amplitude is calculated by summing up amplitudes from the biomolecule and the solvent (including ions):

$$A_1(\mathbf{q}) - B_1(\mathbf{q}) = F(\mathbf{q}) = F_{solu}(\mathbf{q}) + F_{grid}(\mathbf{q})$$

where the solute form factor is $F_{solu}(\mathbf{q}) = \sum_j f_j(q) \exp\left(-\frac{B_j q^2}{16\pi^2}\right) \exp(-i\mathbf{q} \cdot \mathbf{r}_j)$ (with $f_j(q)$ is the atomic scattering factor and B_j is the B-factor) and the contribution from the solvent is $F_{grid}(\mathbf{q}) = \sum_j^{N_{grid}} f_j(\mathbf{q}) \exp(-i\mathbf{q} \cdot \mathbf{r}_j)$.

The angle averaging is then performed by using Lebedev quadrature to obtain the total intensity:

$$I(q) = \frac{1}{4\pi} \int I(\mathbf{q}) d\Omega$$

This approach was shown valid up to angles corresponding to $q \simeq 1.5 \text{ \AA}^{-1}$. (For more details, see [856]).

43.1.2. saxs_md

The intensity is calculated based on eq. 43.1, which can be rewritten as:[855]

$$I(\mathbf{q}) = |a(\mathbf{q}) - b(\mathbf{q})|^2 + \frac{1}{N} \sum_i |A_1^{(i)}(\mathbf{q}) - a(\mathbf{q})|^2 - \frac{N' + 1}{N'(N' - 1)} \sum_j |B_1^{(j)}(\mathbf{q}) - b(\mathbf{q})|^2$$

where $A_1^{(i)}(\mathbf{q})$ and $B_1^{(j)}(\mathbf{q})$ are the scattering amplitudes of the each snapshot from the “sample” and “blank”, respectively, and are computed by:

$$A_1(\mathbf{q}) = \sum_n f_n(\mathbf{q}) e^{-i\mathbf{q} \cdot \mathbf{r}_n}$$

$a(\mathbf{q})$ and $b(\mathbf{q})$ are the averaged amplitudes for the total N and N' snapshots, respectively (with each weight w_i)

$$a(\mathbf{q}) = \frac{\sum_i^N w_i A_1^{(i)}(\mathbf{q})}{\sum_i^N w_i}$$

$$b(\mathbf{q}) = \frac{\sum_j^{N'} w_j B_1^{(j)}(\mathbf{q})}{\sum_j^{N'} w_j}$$

The angle averaging is then performed by Lebedev quadrature, as in `saxs_rism`.

$$I(q) = \frac{1}{4\pi} \int I(\mathbf{q}) d\Omega$$

43.2. Usage

43.2.1. saxs_rism

The program requires solvent distribution in dx format (as output of 3D-RISM) and a pdb file of the biomolecule to compute SAXS signal. If run without input, `saxs_rism` prints the usage and default settings for all parameters.

```
--grid_dir  Location of the folder where all the 3D-RISM outputs found. All files in this folder starting with
              guv will be considered by the program. Atom or ion names must be present in the file name in
              order for the program to recognize. Currently supporting O, H1 (for water), Li+, Na+, K+, Rb+,
              Cs+, Mg2+, Sr2+, F-, Cl-, Br-, I-. For example these file names are valid: guv.Cl-dx, guvfileRb+,
              guvO. The following file names are NOT valid: abc.O.dx, guvNa.dx, guv.H.dx

--solute     pdb file of the solute. Currently only supporting the following atoms: H, O, C, N, P, S, Fe

--conc_ion   concentration of salt [mol.l-1]. This is the concentration of the cation. Concentration of the anion
              will be automatically computed (2x in Mg2+ and Sr2+ cases)

--conc_wat   concentration of water [mol.l-1]. Default is 55.34

--qcut       momentum transfer q cutoff [Å-1]. Default is 0.5

--dq         q spacing [Å-1]. Default is 0.01

--cutoff     real space cutoff [Å]. Only considering grid points within cutoff distance to the nearest solute
              atom. Default is 20

--off_cutoff using all grid points for calculating SAXS, ignoring cutoff value

--expli      flag for using explicit H atoms in pdb file to calculate intensity. Default is to merge H atoms into
              heavier atoms

--anom_f     f' for anomalous scattering. Currently only applied to Rb+, Sr2+ or Br- grids. Default is 0
```



```

--decomp    flag for decomposing total intensity into site contributions (usually this leads to 2-5x in computational time)
--exper     provide the experimental data to read q from. This will override dq and qcut
--exclV     flag for merging those contribution of the grid points inside the excluded volume of the solute into the solute
--phase     turn on this flag will output the phase and error analysis
--tight     flag for using tighter convergence criteria for Lebedev quadrature (also leads to more time)
--bfactor   flag for using B factor (Debye–Waller factor) in the PDB file to compute intensity
--output    output file
--ncpus     (need to compile with OpenMP to use this flag) specify the number of threads used. Default is to use all available threads

```

Example

The following example first run 3D-RISM to calculate the distribution function of water around lysozyme (lys.pdb). The output (guv.O.dx and guv.H1.dx) will then be used to compute SAXS intensity

- Run 3D-RISM to obtain the distribution function around the solute

```
$AMBERHOME/bin/rism3d.snglpnt --pdb lys.pdb --prmtop prmtop --xvv rism.xvv --guv
```

- Run saxs_rism

```
$AMBERHOME/bin/saxs_rism --grid.dir . --solute lys.pdb --expli --decomp \
--bfactor --output saxs.out
```

43.2.2. saxs_md

The program requires two sets of coordinates (both in PDB formats) of the “sample” (biomolecule + solvent) and “blank” (pure solvent) systems. Each snapshot starts with “MODEL”, following by “ATOM” or “HETATM” and ends with “ENDMDL” or “END” (for the last snapshot). These pdb files can be generated directly from the trajectory by using ptraj/cpptraj as following:

```

parm prmtop
trajin md.nc
autoimage
trajout rep.pdb pdb

```

Additionally, you can assign the weight for each snapshot by using “WEIGHT”. This is useful if you want to use only representative snapshots for SAXS calculation. For example, the following is a valid pdb file which assign a weight of 342 for the first snapshot and 148 for the second.

```

MODEL      0
WEIGHT     342
ATOM       1 HO5' DG5      1      14.902  29.822  29.924  1.00  0.00      H
ATOM       2 O5'  DG5      1      15.380  29.001  30.064  1.00  0.00      O
...
ATOM    72772 H1  WAT    3867      40.377  65.382  83.718  1.00  0.00      H
ATOM    72773 H2  WAT    3867      40.942  64.499  82.466  1.00  0.00      H

```

43. SAXS

```

ENDMDL
MODEL      1
WEIGHT    148
ATOM       3  C5'  DG5      1      16.744  29.215  29.653  1.00  0.00      C
ATOM       4  H5'  DG5      1      16.808  29.389  28.579  1.00  0.00      H
  ....
END

```

If run without input, `saxs_md` prints the usage and default settings for all parameters.

```

--system      pdb file for the solute system
--solvent     pdb file for the solvent
--qcut        momentum transfer q cutoff [ $\text{\AA}^{-1}$ ]. Default is 1.0
--dq          q spacing [ $\text{\AA}^{-1}$ ]. Default is 0.01
--cutoff      distance cutoff to the solute, keep only waters and ions within cutoff distance from the nearest
               solute atom. Default is 5.0
--tight       flag for using tighter convergence criteria for Lebedev quadrature (leads to more computational
               time)
--anom_f      f' for anomalous scattering. Currently only applied to  $\text{Rb}^+$ ,  $\text{Sr}^{2+}$  or  $\text{Br}^-$ . Default is 0
--expli       flag for using explicit H atoms in pdb files to calculate SAXS. Default is to merge H atoms into
               heavier atoms.
--output      output file
--ncpus       (need to compile with OpenMP to use this flag) specify the number of threads used. Default is to
               use all available threads

```

Example

The following example use two pdb files (sample.pdb and solvent.pdb) to compute SAXS.

```

$AMBERHOME/bin/saxs_md --system sample.pdb --solvent solvent.pdb \
  --cutoff 10 --expli --output saxs.out

```

44. MoFT: analysis of volumetric data

MoFT¹ is a series of computational programs and libraries for analysis of volumetric data generated by theoretical models (MD, MC simulations, 3D-RISM, NLPB) or derived from experimental measurement (e.g X-ray crystallography, cryo-EM). `metatwist` is an application that provides a low level access to most of the functionalities available in MoFT and is supported by `metaFFT`, a templated interface to FFTW library v3², that supports discrete Fourier transforms, correlations, convolutions on 1 or 3-dimensional data of float, double or complex datatypes.

Examples of MoFT usage and how to cite. The development of the functionalities available in MoFT has been driven by applied work which has been reported in the references bellow. Consider including these publications in your reference list when using MoFT:

1. "Ion counting from explicit-solvent simulations and 3D-RISM" GM Giambaşu, T Luchko, D Herschlag, DM York, DA Case **Biophysical Journal** 106 (4), 883-894 doi:10.1016/j.bpj.2014.01.021
2. "Competitive interaction of monovalent cations with DNA from 3D-RISM" GM Giambaşu, MK Gebala, MT Panteva, T Luchko, DA Case, DM York **Nucleic Acids Research** 43 (17), 8405-8415 doi:10.1093/nar/gkv830
3. "Predicting site-binding modes of ions and water to nucleic acids using molecular solvation theory" GM Giambaşu, DA Case, DM York **Journal of the American Chemical Society** doi:10.1021/jacs.8b11474

44.1. Usage

Most of the functionalities available in MoFT are exposed through the `metatwist` application, and include:

1. Reading, converting and writing plain or compressed (gz, bz2) *.dx (OpenDX³), mrc⁴, ccp4⁵ volumetric data formats.
2. Dimensionality reduction of volumetric data:
 - a) radial distribution functions using cylindrical and spherical frames of references (3D -> 1D).
 - b) projection of 3D-data on x,y or z coordinates (3D -> 1D).
 - c) worm plots (3D -> 1D), useful to characterize how density changes along curvilinear paths (such as channels) which are represented as B-splines and whose pivot points are provided by the user. The abscissa is the result of integrating the 3D density within a tube of specified radius around the curvilinear path. See [151] for examples of how worm plots can be used to analyze water and ion distribution in ion channels and G-quadruplexes.

¹Origin of name MoFT: most of the included tools and libraries use template meta-programming approaches in C++ and hence when using meta-programming too often to write software you may get MoFT. Incidentally *moft* is a also word in Romanian, the main developer's native language, see this link for possible translations: <https://translate.google.com/#ro/en/moft>.

²<http://www.fftw.org/>

³antiquated format that is still widely used by most molecular graphics programs, see https://en.wikipedia.org/wiki/IBM_OpenDX.

⁴a common binary format in use by X-ray crystallography and cryo-EM, see https://www.ccpem.ac.uk/mrc_format/mrc2014.php

⁵another common, but older, binary format in use by X-ray crystallography and cryo-EM, see <http://www.ccp4.ac.uk/html/maplib.html>

44. MoFT: analysis of volumetric data

- d) twisted, untwisted maps (3D -> 2D), meant to map the density of ions and water in an average plane of nucleic acid basepairs that are part of helical regions. Twisted maps are simply average densities in a plane perpendicular to the helical axis. Untwisted maps deconvolute this information with a mobile frame of reference that moves against the natural twist of the helical motif. See [291, 857] for examples of untwisted maps usage.
3. Convolutions of volumetric data with several kernels, including Gaussian, sinc, box, Laplacian of a Gaussian, Butterworth filter for reduction of resolution range in the reciprocal space, crystallographic atomic form factors and densities to obtain to corresponding electron densities.
4. Transformations, including numerical derivatives (finite difference Laplacian), logarithm operators to compute potentials of mean force from equilibrium distributions.
5. Water and ion placement using Laplacian mapping.

metatwist has the following command line options:

<code>--help</code>	Produces help message.
<code>--dx</code>	Input density file(s): *.dx(gz,bz2) *.mrc *.ccp4.
<code>--ldx</code>	Input Laplacian file (*.dx *.ccp4) for use with “-- map blobs(per)”.
<code>--odx</code>	Output density file. File type is determined by extension: *.dx, *.mrc or *.ccp4.
<code>--map</code>	Mapping type: ~ cylindrical (1D): cylindrical RDF along z-axis. ~ twist (2D): twisted helical map along z-axis. ~ untwist (2D): untwisted helical map along z-axis. ~ spherical (1D): spherical RDF. ~ projxyz: (1D) project 3D-map on x,y,z axes. ~ excess: excess number of particles. ~ blobs: Laplacian blob analysis. ~ blobsper: Laplacian blob analysis on a periodic 3D-map. ~ rhoel (3D) : Electron density using atomic form factors. ~ rhoelreal (3D): Electron density using atomic densities. ~ cutresol (3D): Cut 3D-map resolution range.
<code>--bin</code>	Bin size for re-sampling (Å) .
<code>-- (x y z r) max</code>	Extent in the x,y, z or r directions (Å).
<code>--utrate</code>	Untwisting rate for use with “--map twist”. Untwisting rate: 0.18587 rad/Å - BDNA 0.16870 rad/Å - TDNA 0.25590 rad/Å - ARNA (rad/Å).
<code>--com</code>	COM coordinates .
<code>--resolution</code>	Min and max resolution thresholds (in Å) for use with “ -- map cutresol” (default “1.0 10.0”, Å).
<code>--bulkdens</code>	Bulk density (M, mol/L, molar).
<code>--species</code>	Chemical species: atom, e.g. “N”, or atom & residue, e.g. “O WAT”, useful for water and ions placement as well as for computing electron densities.
<code>--sigma</code>	Convolution kernel width, sigma (in Å).
<code>--threshold</code>	Laplacian threshold. Sometimes not all the locally concentrated regions might be interest- ing. The threshold limits the region of interest to min(L[rho]) to threshold*min(L[rho]).

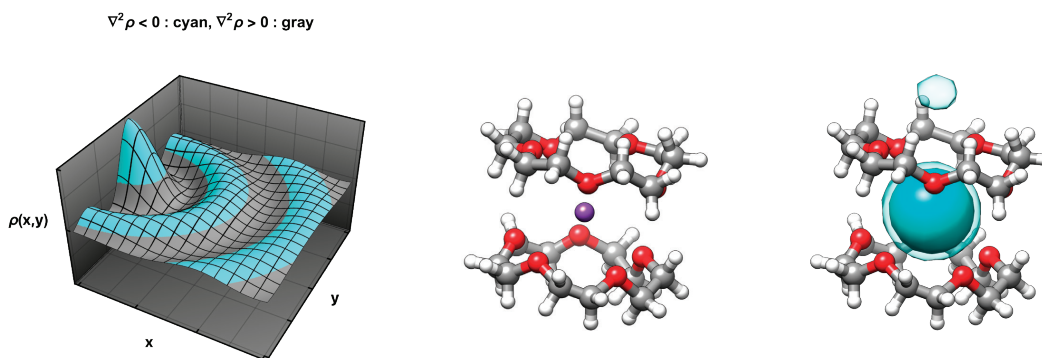


Figure 44.1.: (Left) The negative Laplacian (cyan) can be used to map locally concentrated regions of a density distribution, ρ . (Middle) A bis-crown ether (shown as CPK) binding mode to K^+ (violet sphere) determined using density distributions obtained using RISM and located using Laplacian mapping in MoFT. (Right) Two level sets of the Laplacian of the K^+ distribution, one using a threshold (see documentation) of 0.1 (solid cyan) and the other using a threshold of 0.01 (semi-transparent cyan).

<code>--convolve</code>	Convolution type: (1) Gaussian, (2) box, (3) sinc, (4) Laplacian of Gaussian.
<code>--nlog</code>	Take the negative natural logarithm of the input density.
<code>--laplacian</code>	Compute Laplacian of the input density using finite difference.
<code>--average</code>	Average volumetric data when multiple datasets have been loaded. Otherwise, data will be accumulated.

44.2. Examples

All files relevant for these examples are available in `$AMBERHOME/Ambertools/src/moft/examples/`.

Water and ion placement using Laplacian mapping

We will use MoFT to locate and map tightly bound solution particles to a solute molecule of interest using molecular distribution functions obtained from 3D-RISM. Specifically, we will try to locate K^+ binding mode(s) to a small molecule ionophore - a crown ether.

Generally, molecular density distributions of ions and water have alternating regions where they are highly concentrated and others where they are locally depleted. While these complex topologies are a benefit of models that include particle-particle correlations (such as explicit solvent MD, RISM) they make determination of boundaries of “binding modes” a complex task. Our solution is to demarcate these binding modes using the Laplacian of the solvent distributions. When applied to 3D distributions, the Laplacian measures the difference between the local particle density and the average of the density in a small neighborhood of that point. Hence, where the Laplacian is positive the local particle density is *locally depleted*, while for the regions with negative values of the Laplacian the particle density is *locally concentrated*. Experience shows that a pre-conditioning using kernels that smooth out small local variations in the density can help eliminate false positives. Here we will apply the Laplacian mapping on the density convolution with a 3D Gaussian which can be carried out in a single step by a convolution with a Laplacian of a Gaussian kernel.

(1) Generate density distributions. Solvent density distributions can be determined by several means, but here RISM is used (See 8 for how to run RISM). When running RISM, make sure to specify the “`--guv`” keyword to have the solution components density distributions outputted:

```
rism3d.snglpnt --prmtop bc5-k.parm7 --xvv KCl-aq-0.2M-pse3.xvv \
--closure pse1,pse2,pse3 --tolerance 1e-03,1e-06 \
--ng 192,192,192 --solvbox 96,96,96 --buffer -1 \
--mdiis_del 0.5 --mdiis_nvec 10 \
--verbose 2 --npropagate 0 --guv g > rism.out
```

(2) Compute the Laplacian map. First, one has to take the Laplacian of the distribution, using the “convolve” option:

```
metatwist --dx g.K+.1.dx.bz2 --odx lp-K+.dx --species K+ K+ --bulkdens 0.2 \
--convolve 4 --sigma 1.0
```

Here, --dx specifies the input density, --odx the root of the output file containing the Laplacian density. Option “--convolve 4” specifies the type of convolution that leads to the Laplacian; here we have chosen to obtain the Laplacian using a convolution with the Laplacian of a Gaussian, in this case of width 1.0, specified using “--sigma 1.0”. This step produces a “convolution-lp-K+.dx” file that can be visualized in your molecular graphics application and will be used in the next step.

(3) Solvent Placement. Second, using the determined Laplacian, we can proceed to the actual analysis:

```
metatwist --dx g.K+.1.dx.bz2 --ldx convolution-lp-K+.dx --species K+ K+ \
--bulkdens 0.2 --map blobs --thresh 0.1
```

Here, --ldx specifies the input Laplacian, “--map blobs” asks for solvent placement analysis to be carried out (you can think about solvent binding modes as blobs) using a Laplacian threshold of 0.1. While all the regions of space having a negative Laplacian can be considered as “locally concentrated”, often a tighter (more negative) threshold can simplify the analysis. Lastly, --bulkdens specifies the concentration of the solution particle; in this case K+ has a bulk concentration of 0.2M. With these settings, a pdb file named “g.K+.1-convolution-lp-K+-blobs-centroid.pdb” is produced that contains the coordinates of the centroid of each solvation binding mode (in this case only one mode has been found), its occupancy and temperature factor.

```
ATOM      1  K+      K+ C      1      10.926  12.084   4.026   0.10  92.73      K+
```

Converting particle density distributions to electron densities

It is often necessary to convert particle density distributions to electron densities to directly compare against experimentally derived data, such as that obtained from X-ray crystallography. To illustrate this functionality, we will use the aforementioned RISM calculation on the crown ether immersed in a KCl aqueous solution which produced density distributions for K+, Cl-, water H and water O. In the first step, each of the particle densities is converted to their corresponding electron densities using model atomic factors used in crystallography. In a second stage, all the electron densities are accumulated into a resulting total electron density. Note the use of “--species” option to guide the choice of model density based on the ionization or oxidation number of each atom as well as the “--map rhoel” option to ask for computation of the electron density map. A similar option “--map rhoelreal” could be used which instead of atomic factors will use reference atomic densities to compute the overall electron density.

```
# (1) convert each particle density to electron densities :
metatwist --dx g.K+.1.dx.bz2 --species K+ --odx rho.K+.1.dx --map rhoel --bulkdens 0.2
metatwist --dx g.Cl-.1.dx.bz2 --species Cl- --odx rho.Cl-.1.dx --map rhoel --bulkdens 0.2
metatwist --dx g.O.1.dx.bz2 --species O2- --odx rho.O.1.dx --map rhoel --bulkdens 0.2
# (2) assembly of all densities into rho.dx :
metatwist --dx rho.Cl-.1.dx rho.K+.1.dx rho.O.1.dx --odx rho.dx --species none
```

45. ndfes

45.1. ndfes

The ndfes program [858] reads an input file that describes the biased umbrella sampling, performs the variational free energy profile (vFEP) [858–860], multistate Bennett acceptance ratio (MBAR) [621], weighted thermodynamic perturbation theory (wTP), [861–863], or generalized weighted thermodynamic perturbation theory (gwTP) [863] methods to obtain an unbiased free energy surface (FES). The FES is output as an XML file (a ndfes “checkpoint file”). The ndfes program is a part of the [FE-ToolKit](#) library. The most recent development version, extended documentation, and examples can be found [online](#).

It is recommended that one configure Amber with the `-DOPENMP` cmake option to compile ndfes.OMP, which parallelizes the calculation with OpenMP.

The ndfes input file, referred to as a “metafile”, lists the “Hamiltonian index” (an integer representing the unbiased potential energy function used to produce the sampling), the temperature, the umbrella potential center(s) and force constant(s), and a dumpave filename containing the samples to be analyzed. For a N-dimensional surface generated from M biased simulations, the metafile contains $3+(2*N)$ columns and M rows.

The first column is an integer (the 0-based Hamiltonian index).

The second column is the temperature, a float, in Kelvin. The temperatures listed in the metafile correspond to the target temperatures used during the simulation, whereas the `--temp` command line option to the ndfes is the temperature of the predicted free energy surface. One must currently run all simulations and calculate the FES at the same temperature; the mixing of different temperatures is an experimental feature that is not well-tested.

The 3rd column of the metafile is a filename. This is the name of the dumpave file containing the samples.

The 4th column is the position of the umbrella potential center (either in Angstroms or Degrees).

The 5th column is $1/2$ the harmonic force constant (in kcal/mol/A² or kcal/mol/Deg²), such that the biasing potential is $U_{bias}(x; x_0, k) = k(x - x_0)^2$. This definition of the biasing potential is consistent with the NMR restraints specified in Amber’s `disang` files (the value of $k = rk2 = rk3$); however, note that the “disang” files list angular force constants in kcal/mol/Radian². For these restraints, $k = (\frac{\pi}{180})^2 rk2 = (\frac{\pi}{180})^2 rk3$.

The remaining columns similarly list pairs of umbrella potential centers and (half the) force constants for the other dimensions.

The dumpave files read by ndfes are almost the same as the dumpave files produced by Amber. To analyze a N-dimensional surface with vFEP or MBAR, the dumpave file is expected to contain N+1 columns. The first column is the time (in ps). The remaining columns are the values of the reaction coordinates. The wTP and gwTP methods allow one to predict the unbiased FES of a target potential energy function from the biased sampling produced by one-or-more reference potential energy functions (which may or may not include the target potential energy function). In this case, the dumpave file contains H extra columns, where H is the number of unique potential energy functions (either target or reference), and the rows are the unbiased potential energy values of the samples.

The construction of the metafile and the preparation of the corresponding dumpave files is aided by the use of the `ndfes-PrepareAmberData.py` script (45.2).

For vFEP:

```
Usage:      ndfes --vfep --order=5 {nquad=5 -c out.chk [options] metafile
```

For MBAR:

```
Usage:      ndfes --mbar -c out.chk [options] metafile
```

For wTP and gwTP:

Usage: `ndfes --mbar --nham=N [--sdos=0.2] -c out.chk [options] metafile`

The available options are listed below:

`--chkpt, -c` This is the output filename. It is an XML file, referred to as a checkpoint file.

`--width, -w` Bin width size (a float). Use this option multiple times to define the bin width for each dimension. If only one instance is used, then all dimensions share the same bin width. Recommended values: 0.15 Angstroms and 8.0 degrees. Using small bin widths will result in a large histogram and a noisy free energy surface. Large bin widths will decrease the resolution of the free energy surface, and it will underestimate barrier heights.

vFEP-Specific Options:

`--nquad, -q` Integer number of Gauss-Legendre quadrature points used to integrate the partition function within each bin. This value is applied to each dimension. [Default: 5]

`--order, -o` The B-spline order (an integer). A value of 3 is required to yield a smooth FES. Increasing the order may produce polynomial artifacts from incomplete sampling. [Default: 5]

MBAR-Specific Options:

`--nham, -G` The integer number of unbiased potential energies to expect in each dumpave file; i.e., this is the number of extra columns in the dumpave file. This is also the number of free energy surfaces written to the checkpoint file. [Default: 0, which does not read columns of potential energies, and it will output a single free energy surface.]

`--sdos` A float, dU , in reduced energy units (values of kT). If $dU > 0$, then it will smooth the MBAR density-of-states using the method described in Hu (2020) [862]. The value used in that paper was 0.2 kT. [Default: -1. (no smoothing)] One would not normally use this option, but if the free energy surface contains excessive noise, then one can use this option to see if it makes it more smooth.

`--sepsdos` If present, then perform density-of-states smoothing from the samples of the individual reference potentials rather than the combined sampling from all potentials. It is arguably more correct to not use this option, but one may find it yields more stable results in some circumstances.

Options Applicable to both methods:

`--temp, -t` Temperature. [Default 298.] This must be the same temperature as listed in the metafile. All simulations must be at the same temperature, and the FES prediction must be at that temperature as well. The infrastructure for mixing temperatures is in-place, but it is not yet well-tested.

`--tol, -T` Optimization stopping criteria. [Default: 1.e-9]

`--maxit, -n` Perform a maximum of maxit iterations. [Default: 99999]

`--periodic, -p` Integer, which flags a dimension as a periodic coordinate ranging from 0 to 360. The option can be used multiple times for multiple periodic coordinates. This is a 1-based index, such that `--periodic=1` causes the first reaction coordinate to be treated as periodic. If a periodic value is outside the range [0,360), it will be wrapped to this range.

`--start, -s` A float from [0.0,1.0) representing a time as a fraction of the samples. [Default: 0.0] This discards all samples before “start” as equilibration.

- `--stop, -S` A float from (0.0,1.0] representing a time as a fraction of the samples. [Default: 1.0] This discards the samples after “stop” from the analysis. This is useful when performing convergence tests on the sampling.
- `--uniq, -u` If present, then prune the inputs to keep only the statistically independent samples. The default is to keep all samples and estimate errors from block bootstrap analysis, where the block size is chosen from the statistical inefficiency of the biasing energy.
- `--maxuniq, -U` Integer number of statistically independent samples to use from each simulation. To use this option, you must also use `--uniq`. [Default: 0, which analyzes all statistically unique samples.] This option is only useful when trying to determine the smallest subset of data that reproduces the results from the full set of sampling.
- `--expert, -X` Don’t terminate abnormally if the target and simulation temperatures disagree. It is recommended that all of your simulations are run at the same temperature and you analyze the FES with that same temperature. The support for mixing different temperatures is still experimental and likely won’t give you good results. You shouldn’t trust any result you obtain while using this option.
- `--nboot, -M` Number of bootstrap resamples to estimate errors. [Default: 0] If you want to estimate uncertainties from bootstrap analysis, this should be 20 or larger. Each bootstrap estimate reperforms the calculation with a subset of the input data using a procedure called “sampling with replacement”. The error estimate is the standard deviation of the bootstrap calculations. If the biasing potential time series is correlated, then the error is estimated from a circular block bootstrap procedure, where the block size is chosen from the biasing potential’s “statistical inefficiency”. In practice, the bootstrap error estimates are likely an underestimate of the true uncertainty. A better estimate of the error can be made by performing independent sets (trials) of the simulations with different thermostat random number seeds and/or different starting coordinates, estimate the free energy surface from each independent trial, and looking at the variation of the free energy values between the trials. This can be performed with the `ndfes-AvgFESs.py` script (45.6).
- `--btol, -B` Optimization tolerance for bootstrap solutions. [Default: 1.e-5] This allows you to reduce the convergence tolerance when reperforming the calculations during the bootstrap procedure. When MBAR is used, a value ≤ 0 will estimate the solution of the MBAR/UWHAM equations by performing 1 iteration of the MBAR self-consistent equations, rather than a full nonlinear optimization.

45.2. ndfes-PrepareAmberData.py

Prepares the `ndfes` metafile and dumpave files for several situations, as described below. This script creates a new dumpave file for a single simulation, and it outputs one line of the metafile to standard output, such that one would use it in a loop. The primary distinction between `ndfes-PrepareAmberData.py` and `ndfes-CheckEquil.py` is that `ndfes-PrepareAmberData.py` manipulates the columns of a dumpave file, whereas `ndfes-CheckEquil.py` manipulates the rows of a dumpave file. A typical loop structure is shown here.

```
if [ ! -d analysis/dumpaves ]; then
    mkdir -p analysis/dumpaves
fi
for disang in *.disang; do
    idump=${disang%.disang}.dumpave
    odump=analysis/dumpaves/${idump}
    if [ ! -e "${idump}" ]; then
        echo "Missing ${idump}" 1>&2
        continue;
    fi
done
```

45. ndfes

```
fi
ndfes-PrepareAmberData.py --prefix=analysis/ --disang=${disang} \
--inp=${idump} --out=${odump} [options]
done > analysis/metafile
cd analysis
[run ndfes using this metafile input]
```

The exact usage of `ndfes-PrepareAmberData.py` within the loop depends on what you are trying to accomplish. The following examples describe several usages.

Example: A 1D FES umbrella window for vFEP or MBAR analysis. There is only 1 collective variable (the first variable in the `disang` file). All other restraints are ignored. In other words, in this usage, one is merely writing a new dumpave that extracts a certain column from the Amber output.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang -i rc_0.00.dumpave \
-o dumpaves/rc_0.00.dumpave -r 1
```

The output file will contain 2 columns:

1. The simulation step.
2. The value of the first collective variable.

One would analyze the resulting files with: "`ndfes --mbar`" or "`ndfes --vfep`".

Example: A 2D FES umbrella window for vFEP or MBAR analysis. There are 2 collective variable (the first two variables in the `disang` file). All other restraints are ignored. In this usage, one is merely writing a new dumpave that extracts 2 particular columns from the Amber output.

```
ndfes-PrepareAmberData.py -d rc_0.00_0.00.disang \
-i rc_0.00_0.00.dumpave -o dumpaves/rc_0.00_0.00.dumpave \
-r 1 2
```

The output file will contain 3 columns:

1. The simulation step .
2. The value of the first collective variable.
3. The value of the second collective variable.

One would analyze the resulting files with: "`ndfes --mbar`" or "`ndfes --vfep`".

Example: A 1D FES umbrella window for wTP analysis, in which the effect of a second biasing potential is removed.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang -i rc_0.00.dumpave \
-o dumpaves/rc_0.00.dumpave -r 1 -b 2
```

The output file will contain 4 columns:

1. The simulation step.
2. The value of the first collective variable.
3. The bias energy (kcal/mol) caused by the 2nd collective variable.
4. The potential energy without the bias (which is safely chosen to be 0.0).

One would analyze the resulting files with "`ndfes --mbar --nham=2`". The biased FES (the surface which removes the bias from reaction coordinate 1, but not coordinate 2) would correspond to "model 0" in the checkpoint file (because it is the first extra column of energies in the dumpave file), and the unbiased FES would be "model 1" (because it is the second extra column of energies in the dumpave file). The free energies of "model 1" will only be accurate, in practice, if the biasing potential introduced by the 2nd reaction coordinate is small (roughly 2 kcal/mol or less).

Example: A 1D FES umbrella window for wTP analysis, in which a low level Hamiltonian is transformed to a high level Hamiltonian.

If the simulation was performed with the low level Hamiltonian, then `--hamidx` is set to 0 to indicate that the corresponding low level energies are stored in the first instance of `--ene`.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang \
  --hamidx 0 -i LL_0.00.nc \
  -o dumpaves/rc_0.00.dumpave -r 1 \
  --ene LL_0.00.mdout --ene HL_0.00.mdout
```

whereas, if the high-level Hamiltonian produced the trajectory, then `--hamidx=1` because the high level energies are stored in the second instance of `--ene`.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang \
  --hamidx 1 -i HL_0.00.nc \
  -o dumpaves/rc_0.00.dumpave -r 1 \
  --ene LL_0.00.mdout --ene HL_0.00.mdout
```

The output file will contain 4 columns:

1. The simulation step.
2. The value of the first collective variable.
3. The low level potential energy.
4. The high level potential energy.

One would analyze the resulting files with “`ndfes --mbar --nham=2`”. The low level FES would correspond to "model 0" and the high level FES would be "model 1".

The mdout files should contain the unbiased potential energies of each sample stored within the trajectory file. One generates these mdout files by reanalyzing the trajectory with sander using the `-y` command line argument and the following options set in the `&cntrl` section: “`irest=0, imin=6, nstlim=0, nmropt=0, ntp=1, ntwx=0, ntwf=0`”.

Example: This the same as the previous example; however, the low-level simulations were restart several times to yield several trajectory and reanalysis files. The individual restarts are combined by including multiple filenames within a single instance of an option.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang \
  --hamidx 0 -i LL_0.00.nc LL_0.00_rst1.nc LL_0.00_rst2.nc \
  -o dumpaves/rc_0.00.dumpave -r 1 \
  --ene LL_0.00.mdout LL_0.00_rst1.mdout LL_0.00_rst2.mdout \
  --ene HL_0.00.mdout HL_0.00_rst1.mdout HL_0.00_rst2.mdout
```

The output file will contain 4 columns:

1. The simulation step.
2. The value of the first collective variable.
3. The low level potential energy.
4. The high level potential energy.

One would analyze the resulting files with “`ndfes --mbar --nham=2`”. The low level FES would correspond to "model 0" and the high level FES would be "model 1".

Example: This is similar to the previous wTP examples; however, an additional biasing potential is removed from the 1D FES.

```
ndfes-PrepareAmberData.py -d rc_0.00.disang \
  --hamidx 1 -i HL_0.00.nc \
  -o dumpaves/rc_0.00.dumpave -r 1 -b 2 \
  --ene LL_0.00.mdout --ene HL_0.00.mdout
```

The output file will contain 6 columns:

1. The simulation step.
2. The value of the first collective variable.
3. The biased low level potential energy.
4. The biased high level potential energy.
5. The unbiased low level potential energy.
6. The unbiased high level potential energy.

One would analyze the resulting files with “*ndfes --mbar --nham=4*”. The biased low level FES would correspond to “model 0”. The biased high level FES would be “model 1”. The unbiased low level FES would be “model 2”. The unbiased high level FES would be “model 3”.

The available optional arguments are provided below.

- `-d, --disang` The disang file defining the restraint energies.
- `-T, --temp` Temperature [Default: 298.]
- `-i, --inp` The input dumpave file(s) or netcdf trajectory file(s). Multiple files can be listed for each instance of `--inp`, which are interpreted as a series of restarts. The output will aggregate the samples from each restart.
- `-o, --out` The output dumpave file to write.
- `-r, --rxn` The column indexes of the desired reaction coordinates. The number of columns defines the dimensionality of the free energy surface. This is a 1-based integer, such that `-r 1` corresponds to the first reaction coordinate. The output dumpave file will only contain the reaction coordinates specified by this option.
- `-b, --bias` The column indexes of the biased coordinates. This is a 1-based integer such that `-b 1` corresponds to the first reaction coordinate. The reaction coordinates specified by this option will not appear in the output dumpave file. Instead, extra columns of energies will be written so that the biased and unbiased free energies will be computed in the reduced dimensional space of coordinates specified by the `-r` option.
- `-e, --ene` The mdout file(s) containing the unbiased potential energy of each frame. Multiple files can be listed for each instance of `--ene`, which are interpreted as a series of restarts. Each instance of `--ene` begins a list of mdout files for a new Hamiltonian. The ordering of the mdouts must be consistent with the ordering of the `--inp` files. This option is only used when performing wTP or gwTP analysis.
- `-H, --hamidx` The index of the Hamiltonian that produced the trajectory. [Default: 0] The value listed here will be written as the first column in the metafile.
- `-s, --sym` If present, negate the observed samples of the specified reaction coordinate. This is only useful for 1D free energy surfaces of symmetric systems, and it should not be used in any other circumstance. It can be used to symmetrize the sampling to ensure that the free energy surface is symmetric about the origin.

- `-w, --width` The histogram bin width. Use this option multiple times for multiple dimensions. If used only once, then it assumes each dimension uses the same width. If used multiple times, it must be used the same number of times as the `--rxn` option. If this option is used, then the output samples will be shifted by $+1.e-6$ if it would lie exactly on the border of two bins. This is only useful when creating dumpave files for systems that will use the `--sym` option, because negating a sample precisely on the border between two bins does not round the bin-index 'away from zero' in a manner required to perfectly enforce the desired symmetry.
- `-P, --prefix` Remove the leading string from the output dumpave filename when writing the metafile. If one is trying to store the dumpaves in `analysis/dumpaves/*.dumpave` and write a metafile called `analysis/metafile`, such that the contents of the metafile refer to the dumpaves with the prefix `dumpaves/*.dumpave`, then set `--prefix="analysis/"` to strip the leading directory when writing the dumpave filenames to the metafile.

45.3. ndfes-CheckEquil.py

This script performs forward and reverse analysis of the biasing potential time series to detect the equilibrated region of a trajectory. It optionally uses `cpptraj` to write a new trajectory containing the statistically independent samples from the equilibrated region. One would normally either read an input NetCDF trajectory file and output a new trajectory file with fewer frames, or read an input dumpave file and output a new dumpave file with fewer rows. One would typically use this script before using `ndfes-PrepareAmberData.py` (45.2).

Examples that analyze NetCDF trajectory files The biasing potential is evaluated using the coordinates read from the trajectory file and the restraints defined in the `disang` file, but only for those restraints listed with the `-r` option.

Example: Exclude the first 25% of a simulation and keep all remaining samples.

```
ndfes-CheckEquil.py -p sys.parm7 -i inp.nc -o out.nc \
  --start=0.25 --skipg --skipsearch -d inp.disang -r 1
```

Example: Automatically detect equilibration from forward and reverse analysis, keeping all samples in the production region.

```
ndfes-CheckEquil.py -p sys.parm7 -i inp.nc -o out.nc \
  --skipg -d inp.disang -r 1
```

Example: Automatically detect equilibration from forward and reverse analysis, keeping only the statistically independent samples within the production region.

```
ndfes-CheckEquil.py -p sys.parm7 -i inp.nc -o out.nc \
  -d inp.disang -r 1
```

Example: Automatically detect equilibration but skip the writing of a new trajectory.

```
ndfes-CheckEquil.py -i inp.nc -d inp.disang -r 1
```

Note: If the simulations were restart several times, then one can specify multiple input trajectories to reconstruct the time series; e.g., `-i init.nc rst1.nc rst2.nc [...]`

Examples that analyze dumpave files The biasing potential is evaluated from the disang file and the samples listed in the input dumpave file for those reaction coordinates listed with the -r option.

Example: Exclude the first 25% of a simulation and keep all remaining samples.

```
ndfes-CheckEquil.py -i inp.dumpave -o out.dumpave \
  --start=0.25 --skipg --skipsearch \
  -d inp.disang -r 1
```

Example: Automatically detect the production region from forward and reverse analysis, but keep all remaining samples.

```
ndfes-CheckEquil.py -i inp.dumpave -o out.dumpave \
  --skipg -d inp.disang -r 1
```

Example: Automatically detect equilibration from forward and reverse analysis, and keep only the statistically independent samples.

```
ndfes-CheckEquil.py -i inp.dumpave -o out.dumpave \
  -d inp.disang -r 1
```

Example: Automatically detect equilibration but skip the writing of a new dumpave.

```
ndfes-CheckEquil.py -i inp.dumpave -d inp.disang -r 1
```

Note: If the simulations were restart several times, then one can specify multiple input dumpaves to reconstruct the timeseries; e.g., -i init.dumpave rst1.dumpave [...]

The standard output will show several columns of quantities with the following meanings.

1. The first column is the name of the input dumpave (or trajectory) file.
2. The second column is the number of input samples.
3. The number of output samples.
4. The percentage of samples to be excluded as the equilibration region.
5. The index of the first sample to be included in the production region.
6. The stride used to extract the statistically independent samples from the production region. (This will be 1 if --skipg is specified.) The values from columns 5 and 6 correspond to the “start” and “offset” arguments of the “trajout” command in cpptraj to extract a new, petite trajectory file.
7. The average value of the biasing potential in the first half of the production region.
8. The standard error of the biasing potential values in the first half of the production region.
9. The average value of the biasing potential in the second half of the production region.
10. The standard error of the biasing potential values in the second half of the production region.
11. The last, optional, column displays warnings that may require your attention. “UNCONVERGED” means that a production region could not be identified. “Teq > 70” means that more than 70% of the simulation should be thrown away as equilibration. “N<50” means that there are fewer than 50 samples being output.

The available optional arguments are provided below.

- `-d, --disang` The disang file defining the restraint energies.
- `-i, --inp` INP The input netcdf trajectory (or disang) file(s). Multiple files can be listed for each instance of `--inp`, which are interpreted as a series of restarts.
- `-p, --parm` Amber parm7 (only needed if `--out` is a netcdf file).
- `-o, --out` The output trajectory (or disang) file.
- `-r, --rxn` The indexes of the desired reaction coordinates. This is a 1-based index, such the `-r 1` refers to the first restraint defined in the disang file.
- `--start` Exclude the first fraction of samples. This is a float from 0. to 1. [Default: 0.0]
- `--stop` Ignore the last fraction of samples. This is a float from 0. to 1. [Default: 1.0]
- `-m, --maxeq` The maximum fraction of samples to exclude as equilibration [Default: 0.5]
- `-g, --skipg` If present, keep all samples after identifying the production region (it assumes $g=1$, where g is the statistical inefficiency.)
- `-S, --skipsearch` Skip the auto-detection of the production region. Instead, extract the samples from `--start` to `--stop`.
- `-C, --chunks` The integer number of chunks to split the data into to perform time series analysis [Default: 20]. In other words, the timeseries is split into N chunks, and the detection algorithm tries to figure out how many of the first chunks should be discarded as equilibration.
- `--ptol` The p-value threshold when performing statistical tests. Larger values exclude more samples. Some useful values to note: 0.21135667595 (79% confidence, 1.25 sigma) 0.31729526862 (68% confidence, 1.00 sigma, default) 0.45310971043 (55% confidence, 0.75 sigma) 0.61678615958 (38% confidence, 0.50 sigma). [Default: 0.05]
- `--dtol` If the first and last half means are less than `dtol`, then it passes the delta test [Default: 0.1 kcal/mol]
- `--prune` Try to find a minimal number of samples to represent the production region that reproduces the average value of the production region by adjusting the stride. Only available if `-S` is not used.
- `--minsize` The minimum allowable number of samples after pruning. [Default: 75]
- `--maxsize` The maximum allowable number of samples after pruning. [Default: 125]

45.4. ndfes-CombineMetafiles.py

Creates a new metafile from one-or-more metafiles. This is primarily used to analyze the aggregate sampling from multiple, independent trials; or multiple iterations of the finite temperature string method. It can also be used to move a metafile to another directory without needing to move the dumpave files that it refers to (the script will adjust the relative path of the filenames in the output metafile).

Usage: `ndfes-CombineMetafiles.py -o output input [inputs ...]`

45.5. ndfes-PrintFES.py

Prints the free energies defined in a ndfes checkpoint at the histogram bin centers, for those bins occupied by at least 1 sample.

Usage: `ndfes-PrintFES.py [-m MODEL] [-c] chkpt`

The output consists of N+5 columns:

1. Columns 1-to-N are the reaction coordinate values of the bin center.
2. Column N+1 is the free energy value (kcal/mol).
3. Column N+2 is the standard error of the free energy value (kcal/mol). If -c is used as an option, then this value is a 95% confidence interval (the standard error scaled by 1.96).
4. Column N+3 is the reweighting entropy. The entropy is a measure of the uniformity of MBAR weights within the bin, such that it is 1.0 if the weights are uniform and 0.0 if the weights are dominated by only a few samples. Entropies below 0.6 likely mean that the free energy estimate for this bin is untrustworthy. The entropy values are not computed when performing vFEP.
5. Column N+4 is the integer number of samples within the bin.

The available optional arguments are provided below.

`-m, --model` The zero-based index of the model to print. [Default: -1, which causes all models within the checkpoint file to be printed.]

`-c, --ci` If present, write the 95% confidence intervals rather than standard errors (the 95% confidence interval is $1.96 \times \text{standard error}$).

45.6. ndfes-AvgFESs.py

Averages 1-or-more models from the supplied checkpoint file(s) and writes a new checkpoint file. If N input surfaces are supplied, the output checkpoint file will contain N+1 surfaces. The first surface will be the averaged free energy, and the remaining N surfaces will be the same as the input surfaces, but uniformly shifted to minimize the weighted least square fit to the average.

Example: To estimate the uncertainty in the free energy values, one often performs multiple sets of independent simulations (trials), and one can use ndfes to analyze each trial. Furthermore, one can use ndfes-CombineMetafiles.py to aggregate the sampling and make a best estimate of the free energies from all available sampling. One then uses ndfes-AvgFESs.py to update the uncertainty of the best estimate by considering the variations between the independent trials. This example is the recommended usage.

```
cd Trial1 && ndfes.OMP --mbar -w 0.15 --nboot=20 \
-c chkpt metafile && cd ..
cd Trial2 && ndfes.OMP --mbar -w 0.15 --nboot=20 \
-c chkpt metafile && cd ..
cd Trial3 && ndfes.OMP --mbar -w 0.15 --nboot=20 \
-c chkpt metafile && cd ..
ndfes-CombineMetafiles.py -o Aggregate/metafile Trial1/metafile \
Trial2/metafile Trial3/metafile
cd Aggregate && ndfes.OMP --mbar -w 0.15 --nboot=20 \
-c chkpt metafile && cd ..
ndfes-AvgFESs.py --fes Trial1/chkpt --fes Trial2/chkpt \
--fes Trial3/chkpt --ene Aggregate/chkpt \
-o Aggregate/chkpt.BetterErrorEstimates
```


The available optional arguments are provided below.

<code>--fes</code>	One or more models to average; e.g., <code>chkptfile=0</code> will read model 0 from the checkpoint file names 'chkptfile'. If the trailing '=idx' is missing, then '=0' is assumed.
<code>--ene</code>	If present, then read the energies from this checkpoint file and use the <code>--fes</code> surfaces only to update the error estimate. The only difference between this option and <code>--ref</code> is: this option reports the reference energy rather than the average energy; i.e., the output model with <code>--ene</code> only modifies the standard errors. This option also uses the <code>chkptfile=i</code> syntax. You should always be using this option. If you have multiple free energy surfaces, you will make a better estimate of the free energy values by analyzing the aggregate sampling.
<code>--ref</code>	If present, then shift the <code>--fes</code> surfaces to minimize the error with respect to this reference surface, but report the average energy instead of the reference energy. This option also uses the <code>chkptfile=i</code> syntax. The <code>--ene</code> option should be preferred over the use of <code>--ref</code> .
<code>-o, --out</code>	The output checkpoint file name.

45.7. ndfes-path

This script has 3 use cases:

1. Evaluate the free energy of a multidimensional surface along a parametric curve.
2. Optimize a minimum free energy pathway through a fixed, multidimensional surface.
3. Prepare a new set of simulation inputs for the finite temperature string method. To better understand how to use `ndfes-path` to perform the finite temperature string method, you should refer to the FE-ToolKit tutorial available [online](#).

It is recommended that one configure Amber with the `-DOPENMP` cmake option to compile `ndfes-path.OMP`, which parallelizes the calculations with OpenMP.

Example: Generate the next directory of simulations using modified string method described in Rosta (2011) [864]. This method does not read a free energy surface from a `ndfes` checkpoint file. This is the recommended string method when you have more than 5 reaction coordinates.

```
ndfes-path -d DISANG -i MDIN --curit CURIT --maxit=0 --msm --no-smooth --line
```

You can also choose a spline representation, either: `--akima` or `--linear`. By setting `--maxit=0`, the new spline will be constructed from the observed reaction coordinate averages obtained from simulation (by averaging the appropriate columns in the amber dumpave output file). The discretization of the path must be the same as the number of simulations. It is recommended to use the `--linear` spline representation of the path when performing the modified string method.

Example: Generating the next directory of simulations using the modified string method accelerated with “synthetic simulations”. This is similar to the modified string method, but the minimum free energy path is optimized on the free energy surface. This version of the modified string method reads a free energy surface from a `ndfes` checkpoint file to perform the optimization; therefore, it is only appropriate when you have a few reaction coordinates.

```
ndfes-path -d DISANG -i MDIN --curit CURIT --maxit=300 --msm
```

By settings `--maxit>0`, the new spline will be constructed by performing minimizations on the free energy surface in the reduced space of reaction coordinates. In this manner, the output path will be the minimum free energy pathway along the current estimate of the unbiased free energy

surface. The 'synthetic modified string method' described here has not been previously published; in part because our experience has been that 'grid based string method' is a superior approach. The discretization of the path does not have to be the same as the number of simulations. You can set this using, for example, `--npathpts=100`. For this reason, one may prefer to use `--akima` rather than `--linear` to represent the path.

Example: Generate the next directory of simulations using the "surface accelerated string method" (unpublished, in prep.). This is the recommended string method when you have 5 or fewer reaction coordinates.

```
ndfes-path -d DISANG -i MDIN --curit CURIT --sasm
```

The discretization of the path does not have to be the same as the number of simulations. You can set this using, for example, `--npathpts=100`. For this reason, one may prefer to use `--akima` rather than `--linear` to represent the path.

Example: Print a 1D projection through a multidimensional FES.

```
ndfes-path --ipath IPATH --chk CHK --opath OPATH --maxit=0 \
  [--nsplpts NSP]
```

Example: Optimize a path on a fixed FES without generating a new directory of simulations.

```
ndfes-path --ipath IPATH --chk CHK --opath OPATH --maxit MAXIT \
  [--npathpts NPTS] [--nsplpts NSP]
```

Options controlling the creation of new simulation inputs

- `-d, --disang` Template disang file. A disang file is considered a template when some of the `r2` and `r3` values are replaced with a placeholder string; e.g., `r2=RC1`, `r3=RC1`, which denote the first, second, etc. reaction coordinates.
- `-i, --mdin` Template mdin file. It is expected that the mdin file sets `nmropt=1`. When writing mdin files from this template, it replaces the `DUMPAVE=` and `DISANG=` lines to refer to the appropriate restraint file.
- `--curit` Index of the current iteration. The current directory is assumed to be `itXXX`, where `XXX` is a zero-padded integer, specified by `--curit`. If `--curit=0`, then the directory name is assumed to be "init".
- `--odir` Output directory. If unspecified, it is `itXXX`, where `XXX` is 0-padded integer corresponding to `--curit + 1`. The program will not create the directory; the directory must exist.
- `--pad` The width of the file and directory name 0-padding. [Default: 3]
- `--dry-run` Do not write any output files to disk
- `--neqit` Remove the first `N` string iterations as equilibration in later iterations. [Default: -1]. The value of `N` is the last iteration to be excluded, such that `N=0` would only exclude the simulation data in the init directory. This option only effects the simulation restart files that can be copied to the next directory. You will need to set the `--neqit` option in `ndfes-FTSM-PrepareAnalysis.py` to effect the FES.
- `--cp-nearest` If present, then the restart of the next iteration image is copied from the image whose centroid was closest to the new window center. If the grid-based string method is not being used, then the default behavior is to copy the restarts from the corresponding image.

- `--prefix` The subdirectory to read and write mdin and rst7 files. By default, these files are written to itXXX/, but setting a `--prefix=SUBDIR` will write them to itXXX/SUBDIR/. This is useful when optimizing several paths on a common FES.
- `--extra-prefix` Extra subdirectories to search for rst7 files. This is only useful when trying to optimize several paths at the same time. By using this option, one can copy the restart file from any of the paths. This option can be used more than once.

Options for path analysis and optimization without creating new simulation inputs

- `--chk` Read the free energy surface from the specified ndfes checkpoint file. This option must be used with `--ipath` and `--opath`, and it cannot be used with `--curit` `--mdin` and `--disang`.
- `--ipath` Read the umbrella centers and force constants from the specified metafile. IPATH can also be a table whose first 2 columns are ignored. Columns 3-to-3+Ndim are assumed to be umbrella centers. All remaining columns are ignored. In this case, all force constants are assumed to be 200 kcal/mol/Ang**2. IPATH can also be a ndfes path XML file, from which it will read the last path. This option must be used with `--chk` and `--opath`, and it cannot be used with `--curit` `--mdin` and `--disang`.
- `--opath` Write the output path to OPATH. This file will contain 2+2*Ndim columns: i, t[i], rc[i][dim], and fc[i][dim]. Two additional files will be written: OPATH.xml (the path iterations) and OPATH.interp.model.dat (A file containing the path properties). The columns of OPATH.interp.model.dat correspond to: i t[i] rc[i][dim] F[i] dF[i] RE[i] N[i], where t is the progress variable, rc is the reaction coordinate, F is the free energy, Err is the standard error in F, RE is the reweighting entropy, and N is the number of samples in the bin that this point lies within. This option must be used with `--ipath` and `--opath`, and it cannot be used with `--curit` `--mdin` and `--disang`.

Options for reading a free energy surface

- `-m, --model` The model index within the checkpoint to read. [Default: 0]
- `--minsize` If a FES bin has fewer than MINSIZE samples, then discard it. [Default: 0, Recommend: 10]

Options for representing the free energy surface

- `--wavg` Represent the FES by a weighted average of nearby bin values using B-splines. This argument requires an integer, which is the B-spline order used to perform the averaging. [Default: 0]
- `--wavg-niter` Number of iterative corrections to the wavg histogram values such that the interpolated values match the values at the bin centers. This will correct artifacts introduced from the averaging procedure, but it will also reproduce noise in the data. [Default: 0].
- `--rbf` Represent the FES with Radial Basis Function interpolation. Valid for both vFEP and MBAR
- `--shape` The RBF shape parameter used when `--rbf` is specified. Small values tend to yield larger oscillations. [Default: 100.0]
- `--arbf` Represent the FES with an approximate radial basis function parametrized from the central bin and its nearest neighbors. This option requires an integer, which is the number of nearby layers to include in the RBF solutions. [Default: -1, which disables the arbf]. A value of 1 constructs a RBF from the central bin and first layer of nearby neighbors. This would be fast, but noticeable artifacts would be present in the FES. A value of 2 reduces the artifacts. Setting ARBF >= 3 is not recommended because it would likely be more expensive than simply using `--rbf`.

<code>--bspl</code>	Cardinal B-spline interpolation. Valid only for vFEP, which is the default when the surface was obtained with the vFEP method.
<code>--hist</code>	No interpolation; only use the histogram values. Path optimizations will not be possible with this option.

Recommended options for representing the free energy surface

- To smooth out a noisy FES (this is the recommended set of options for representing a FES): `--wavg=4 --wavg-niter=1 --minsize=10`
- To better reproduce the noise: `--wavg=4 --wavg-niter=5 --minsize=10`
- To exactly reproduce the noise: `--rbf --minsize=10`

Options for representing the parametric curve

<code>--no-smooth</code>	Do not smooth the spline control points when generating a new path. The default is to apply smoothing procedure.
<code>--linear</code>	Use piecewise linear splines. This is the default when using <code>--msm --maxit=0</code> .
<code>--akima</code>	Use Akima splines. This is usually the default, except when <code>--msm --maxit=0</code> .
<code>--fix0</code>	If present, then don't update the position of $t=0$ (the beginning of the path).
<code>--fix1</code>	If present, then don't update the position of $t=1$ (the end of the path).
<code>--npathpts</code>	Reconstruct a new initial guess from IPATH using N points. This is useful when making a simple guess but you want to optimize a path with more points. The default is to optimize with the same number of points as found in the IPATH file. If this option is unused and one is creating a new directory of simulations with <code>--gbsm</code> , then it will default to 100.
<code>--nsplpts</code>	Given a spline of the final path, interpolate the spline with N points to write OPATH.interp.model.dat. To print the free energy values at the input points, set: <code>--maxit=0</code> and do not use <code>--nsplpts</code> nor <code>--npathpts</code> .

Options for optimizing the path

<code>--maxit</code>	The maximum number of synthetic iterations used to find the minimum free energy path on the fixed surface. If one is generating a new directory of simulations, then the default is 300. If one is analyzing a 1D projection, then the default is 0.
<code>--ptol</code>	T-test p-value tolerance used to terminate the optimization of the synthetic path optimization on the free energy surface. Values close to (but less than) 1.0 represent a strict tolerance requiring many iterations to achieve convergence. A value ≥ 1 will never converge. A value ≤ 0 will prematurely terminate the optimization. [Default: 0.95]
<code>--acc-oobk</code>	The force constant used to push the path toward the nearest occupied bin, if it is out-of-bounds. [Default: 300]
<code>--acc-maxit</code>	The number of nonlinear synthetic optimizations steps. [Default: 100]
<code>--distol</code>	FES-FTSM slope termination tolerance for distance coordinates [Default: 0.05 Ang].
<code>--angtol</code>	ANGTOL FES-FTSM slope termination tolerance for angle coordinates [Default: 0.5 Deg].
<code>--mlen</code>	The number of iterations to consider when calculating the slope of the path with respect to iteration [Default: 5].

Options for altering the predicted force constants

- `--maxfc` Maximum allowable force constants for distance restraints. This value is the largest force constant that you feel comfortable simulating. [Default: 300 kcal/mol/A²]
- `--minfc` Minimum allowable force constants for distance restraints. This value is the smallest force constant that you feel comfortable simulating. [Default: 50 kcal/mol/A²]
- `--maxfc-ang` Maximum allowable force constants for angle restraints. This value is the largest force constant that you feel comfortable simulating. [Default: 0.5 kcal/mol/deg²]
- `--minfc-ang` Minimum allowable force constants for angle restraints. This value is the smallest force constant that you feel comfortable simulating. [Default: 0.1 kcal/mol/deg²]
- `--predfc` Adjust the simulation force constants and progress variable values to place simulations along the path in a manner such that the expected centroid means are evenly spaced (rather than spacing the umbrella window centers). By default, the force constants are not adjusted and the simulations are chosen to uniformly space the biasing potential centers. This option should not be used when performing the grid-based string method.
- `--smoothfc` If present, apply smoothing to the predicted force constants. This option should not be used when performing the grid-based string method.
- `--tdispfc` Adjust the force constants to achieve the target displacement. Only used if `--pred-fc` is present. A value of 1 will scale the force constants such that the predicted centroids will displace along the path by the same amount as the nearest neighbors of a uniformly discretized set of windows. A value of 0.5 will adjust the force constants so that the centroid does not displace by more than 1/2 the distance between nearest neighbors. Setting `tdisp` to small values will generally cause the force constants to increase. [Default: 0.75] This option should not be used when performing the grid-based string method.

Misc options

- `-T, --temp` Temperature, K [Default: 298.]
- `--print-freq` Output frequency during optimization [Default: 50]

45.8. ndfes-path-prepguess.py

This script helps to create an initial guess from which one can start the finite temperature string method. It is presumed that the user has prepared and equilibrated one-or-more structures that approximate stationary point(s) on the reduced dimensional free energy surface. One needs to have prepared at least one structure (typically either the reactant or product structure). By “reactant” and “product” we mean the two ends of a minimum free energy pathway. Furthermore, it is presumed that the user has prepared two-or-more `disang` files that the initial path should pass through. One needs at least 2 `disang` files to represent the two ends of the path. Given these `disang` and restart files, the script writes a series of NSIM output `mdin` and output `disang` files that connect the input `disang` files via linear interpolation. Furthermore, one-or-more shell scripts will be created that can be used to generate initial starting configurations for each of the NSIM states via a sequential series of short simulations from the nearest input restart file.

The available options are listed below.

- `-d, --disang` A template `disang` file.
- `-i, --mdin` A template `mdin` file.

45. ndfes

`--min` Either a `disang` defining a proposed stationary point on the free energy surface. The script will check to see if there is a corresponding restart file by replacing the `'disang'` suffix with a `'.rst7'` suffix. This option must be used at least twice (once for the reactant and product, respectively), and a corresponding restart file (with velocities) must exist for at least one state.

`-n, --nsim` Number of windows in the string.

`--pad` Length of the filename padding. [Default: 3] The files will be named `imgXXX.mdin` and `imgXXX.disang`, where `XXX` is a zero-padded integer.

`-q, --dry-run` Don't prepare the outputs; only show what would be done.

`-o, --odir` Output directory to write the files.

45.9. ndfes-path-analyzesims.py

This script is used during the finite temperature string method to prepare the latest set of simulations for analysis. A typical string iteration involves the following steps:

1. Perform the simulations.
2. Prepare the sampling for analysis using `ndfes-path-analyzesims.py`. This effectively acts as a convenience wrapper to `ndfes-CheckEquil.py`, `ndfes-PrepareAmberData.py`, and `ndfes-CombineMetafiles.py`.
3. Run `ndfes` to obtain the current estimate of the free energy surface from the aggregate sampling obtained from the current and previous string iterations.
4. Run `ndfes-path` to optimize a minimum free energy path on the current estimate of the free energy surface and write a new directory of simulation inputs.

The available options are listed below.

`-d, --disang` Template `disang` file

`--prefix` The subdirectory within the iteration directory containing the simulation data. The default prefix is `"`, which will search for `itXXX/img*.dumpave`. If `--prefix=SUBDIR` is set, then it will search for `itXXX/SUBDIR/img*.dumpave`.

`--curit` Index of the current iteration. The current directory is assumed to be `itXXX`, where `XXX` is a zero-padded integer, specified by `--curit`. If `--curit=0`, then the directory name is assumed to be `"init"`.

`--pad` Length of the directory name zero-padding. [Default: 3]

`--nprev` The number of previous iterations to include in the estimation of the free energy surface. A value of 0 means only the sampling from the current iteration contributes the calculation of the free energy surface. A value of 1000 will include all previous iterations. [Default: 1000]

`-T, --temp` Temperature, K [Default: 298.] This is the temperature value written to the metafile. It should be the same temperature used when running `ndfes`.

`--start` Exclude the first percentage of samples [Default: 0.0]

`--stop` Ignore the samples after this percentage [Default: 1.0]

`--maxeq` Maximum fraction of samples to exclude as equilibration [Default: 0.7]

`-g, --skipg` If present, keep all samples after identifying the production region (assume `g=1`, where `g` is the statistical inefficiency).

- S, --skipsearch If present, skip the forward/reverse analysis search for a production region and instead extract the region specified by -start and -stop.
- neqit Phase-out the first neqit iterations from the sampling when generating metafile.all as curit increases. [Default: -1, which does not phase-out any early iterations.]

Bibliography

- [1] D. A. Pearlman; D. A. Case; J. W. Caldwell; W. S. Ross; T. E. Cheatham, III; S. DeBolt; D. Ferguson; G. Seibel; P. Kollman. AMBER, a package of computer programs for applying molecular mechanics, normal mode analysis, molecular dynamics and free energy calculations to simulate the structural and energetic properties of molecules. *Comp. Phys. Commun.*, **1995**, *91*, 1–41.
- [2] D. A. Case; T. E. Cheatham, III; T. Darden; H. Gohlke; R. Luo; K. M. Merz, Jr.; A. Onufriev; C. Simmerling; B. Wang; R. Woods. The Amber biomolecular simulation programs. *J. Computat. Chem.*, **2005**, *26*, 1668–1688.
- [3] R. Salomon-Ferrer; D. A. Case; R. C. Walker. An overview of the Amber biomolecular simulation package. *WIREs Comput. Mol. Sci.*, **2013**, *3*, 198–210.
- [4] D. Case; H. Aktulga; K. Belfon; D. Cerutti; G. Cisneros; V. Cruzeiro; N. Forouzes; T. Giese; A. Götz; H. Gohlke; S. Izadi; K. Kasavajhala; M. Kaymak; E. King; T. Kurtzman; T. Lee; P. Li; J. Liu; T. Luchko; R. Luo; M. Manathunga; M. Machado; H. Nguyen; K. O’Hearn; A. Onufriev; F. Pan; S. Pantano; R. Qi; A. Rahnamoun; A. Risheh; S. Schott-Verdugo; A. Shajan; J. Swails; J. Wang; H. Wei; X. Wu; Y. Wu; S. Zhang; S. Zhao; Q. Zhu; T. Cheatham; D. Roe; A. Roitberg; C. Simmerling; D. York; M. Nagan; K. Merz, Jr. AmberTools. *J. Chem. Inf. Model.*, **2023**, *63*, 6183–6191.
- [5] J. W. Ponder; D. A. Case. Force fields for protein simulations. *Adv. Prot. Chem.*, **2003**, *66*, 27–85.
- [6] T. E. Cheatham, III; D. A. Case. Twenty-five years of nucleic acid simulations. *Biopolymers*, **2013**, *99*, 969–977.
- [7] S. Harvey; J. A. McCammon. *Dynamics of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1987.
- [8] A. R. Leach. *Molecular Modelling. Principles and Applications, Second Edition*. Prentice-Hall, Harlow, England, 2001.
- [9] C. J. Cramer. *Essentials of Computational Chemistry: Theories and Models*. John Wiley & Sons, New York, 2002.
- [10] M. P. Allen; D. J. Tildesley. *Computer Simulation of Liquids*. Clarendon Press, Oxford, 1987.
- [11] D. Frenkel; B. Smit. *Understanding Molecular Simulation: From Algorithms to Applications. Second edition*. Academic Press, San Diego, 2002.
- [12] M. E. Tuckerman. *Statistical Mechanics: Theory and Molecular Simulation*. Oxford University Press, Oxford, 2010.
- [13] W. F. van Gunsteren; P. K. Weiner; A. J. Wilkinson, eds. *Computer Simulations of Biomolecular Systems, Vol. 3*. ESCOM Science Publishers, Leiden, 1997.
- [14] L. R. Pratt; G. Hummer, eds. *Simulation and Theory of Electrostatic Interactions in Solution*. American Institute of Physics, Melville, NY, 1999.
- [15] O. Becker; A. D. MacKerell; B. Roux; M. Watanabe, eds. *Computational Biochemistry and Biophysics*. Marcel Dekker, New York, 2001.
- [16] C. Chipot; A. Pohorille, eds. *Free energy calculations. Theory and Applications in Chemistry and Biology*. Springer, Berlin, 2007.

- [17] M. Griebel; S. Knapek; G. Zumbusch. *Numerical Simulation in Molecular Dynamics. Numerical Algorithms, Parallelization, Applications*. Springer-Verlag, Berlin, 2010.
- [18] J. W. Ponder; C. Wu; P. Ren; V. S. Pande; J. D. Chodera; M. J. Schieders; I. Haque; D. L. Mobley; D. S. Lambrecht; R. A. DiStasio, Jr.; M. Head-Gordon; G. N. I. Clark; M. E. Johnson; T. Head-Gordon. Current status of the AMOEBA polarizable force field. *J. Phys. Chem. B*, **2010**, *114*, 2549–2564.
- [19] G. A. Cisneros. Application of gaussian electrostatic model (gem) distributed multipoles in the amoeba force field. *J. Chem. Theo. Comput.*, **2012**, *12*, 5072–5080.
- [20] S. Khuvis; K. Tomko; S. R. Brozell; C.-C. Chen; H. Subramoni; D. K. Panda. Optimizing Amber for Device-to-Device GPU Communication. in *PEARC '23: Practice and Experience in Advanced Research Computing*, pp 200–205. ACM, 2023. <https://doi.org/10.1145/3569951.3597553>.
- [21] A. V. Onufriev; S. Izadi. Water models for biomolecular simulations. *WIREs Computational Molecular Science*, **2018**, *8*, e1347.
- [22] X. Wang; D. Xiong; Y. Zhang; J. Zhai; Y. Gu; X. He. The evolution of the Amber additive protein force field: History, current status, and future. *J. Chem. Phys.*, **2025**, *162*, 030901.
- [23] C. Tian; K. Kasavajhala; K. Belfon; L. Raguette; H. Huang; A. Migués; J. Bickel; Y. Wang; J. Pin-cay; Q. Wu; C. Simmerling. ff19SB: Amino-Acid-Specific Protein Backbone Parameters Trained against Quantum Mechanics Energy Surfaces in Solution. *J. Chem. Theory Comput.*, **2020**, *16*, 528–552.
- [24] S. Izadi; R. Anandakrishnan; A. V. Onufriev. Building Water Models: A Different Approach. *J. Phys. Chem. Lett.*, **2014**, *5*, 3863–3871.
- [25] J. A. Maier; C. Martinez; K. Kasavajhala; L. Wickstrom; K. E. Hauser; C. Simmerling. ff14SB: Improving the Accuracy of Protein Side Chain and Backbone Parameters from ff99SB. *J. Chem. Theory Comput.*, **2015**, *11*, 3696–3713.
- [26] V. Hornak; R. Abel; A. Okur; B. Strockbine; A. Roitberg; C. Simmerling. Comparison of multiple Amber force fields and development of improved protein backbone parameters. *Proteins*, **2006**, *65*, 712–725.
- [27] J. Graf; P. H. Nguyen; G. Stock; H. Schwalbe. Structure and Dynamics of the Homologous Series of Alanine Peptides: A Joint Molecular Dynamics/NMR Study. *J. Am. Chem. Soc.*, **2007**, *129*, 1179–1189.
- [28] L. Wickstrom; A. Okur; C. Simmerling. Evaluating the performance of the ff99SB force field based on NMR scalar coupling data. *Biophys. J.*, **2009**, *97*, 853–856.
- [29] H. Nguyen; D. R. Roe; C. Simmerling. Improved Generalized Born Solvent Model Parameters for Protein Simulations. *J. Chem. Theory Comput.*, **2013**, *9*, 2020–2034.
- [30] K. T. Debiec; D. S. Cerutti; L. R. Baker; A. M. Gronenborn; D. A. Case; L. T. Chong. Further along the Road Less Traveled: AMBER ff15ipq, an Original Protein Force Field Built on a Self-Consistent Physical Model. *J. Chem. Theory Comput.*, **2016**, *12*, 3926–3947.
- [31] D. S. Cerutti; J. E. Rice; W. C. Swope; D. A. Case. Derivation of fixed partial charges for amino acids accommodating a specific water model and implicit polarization. *J. Phys. Chem. B*, **2013**, *117*, 2328–2338.
- [32] D. S. Cerutti; W. C. Swope; J. E. Rice; D. A. Case. ff14ipq: A Self-Consistent Force Field for Condensed-Phase Simulations of Proteins. *J. Chem. Theory Comput.*, **2014**, *10*, 4515–4534.
- [33] K. Takemura; A. Kitao. Water Model Tuning for Improved Reproduction of Rotational Diffusion and NMR Spectral Density. *J. Phys. Chem. B*, **2012**, *116*, 6279–6287.
- [34] W. D. Cornell; P. Cieplak; C. I. Bayly; I. R. Gould; K. M. Merz, Jr.; D. M. Ferguson; D. C. Spellmeyer; T. Fox; J. W. Caldwell; P. A. Kollman. A second generation force field for the simulation of proteins, nucleic acids, and organic molecules. *J. Am. Chem. Soc.*, **1995**, *117*, 5179–5197.

BIBLIOGRAPHY

- [35] A. T. Bogetti; H. E. Piston; J. M. G. Leung; C. C. Cabalreja; D. T. Yang; A. J. Degrave; K. T. Debiec; D. S. Cerutti; W. S. Case, D. A. Horne; L. T. Chong. A twist in the road less traveled: The AMBER ff15ipq-m force field for protein mimetics. *J. Chem. Phys.*, **2020**, *153*, 064101.
- [36] D. T. Yang; A. M. Gronenborn; L. T. Chong. Development and Validation of Fluorinated, Aromatic Amino Acid Parameters for Use with the AMBER ff15ipq Protein Force Field. *J. Phys. Chem. A*, **2022**, *126*, 2286–2297.
- [37] L.-P. Wang; T. J. Martinez; V. S. Pande. Building force fields: An automatic, systematic and reproducible approach. *J. Phys. Chem. Lett.*, **2014**, *5*, 1885–1891.
- [38] L. Wang; K. A. McKiernan; J. Gomes; K. A. Beauchamp; T. Head-Gordon; J. E. Rice; W. C. Swope; T. J. Martíáñez; V. S. Pande. Building a More Predictive Protein Force Field: A Systematic and Reproducible Route to AMBER-FB15. *J. Phys. Chem. B*, **2017**, *121*, 4023–4039.
- [39] Y. Duan; C. Wu; S. Chowdhury; M. C. Lee; G. Xiong; W. Zhang; R. Yang; P. Cieplak; R. Luo; T. Lee. A point-charge force field for molecular mechanics simulations of proteins based on condensed-phase quantum mechanical calculations. *J. Comput. Chem.*, **2003**, *24*, 1999–2012.
- [40] M. C. Lee; Y. Duan. Distinguish protein decoys by using a scoring function based on a new Amber force field, short molecular dynamics simulations, and the generalized Born solvent model. *Proteins*, **2004**, *55*, 620–634.
- [41] L. Yang; C. Tan; M.-J. Hsieh; J. Wang; Y. Duan; P. Cieplak; J. Caldwell; P. A. Kollman; R. Luo. New-generation Amber united-atom force field. *J. Phys. Chem. B*, **2006**, *110*, 13166–13176.
- [42] V. N. Uversky; C. J. Oldfield; A. K. Dunker. Intrinsically disordered proteins in human diseases: Introducing the d2 concept. *Annual Review of Biophysics*, **2008**, *37*, 215–246.
- [43] S. Piana; A. G. Donchev; P. Robustelli; D. E. Shaw. Water dispersion interactions strongly influence simulated structural properties of disordered protein states. *The Journal of Physical Chemistry B*, **2015**, *119*, 5113–5123.
- [44] V. T. Duong; Z. Chen; M. T. Thapa; R. Luo. Computational studies of intrinsically disordered proteins. *The Journal of Physical Chemistry B*, **2018**, *122*, 10455–10469.
- [45] D. Song; W. Wang; W. Ye; D. Ji; R. Luo; H.-F. Chen. ff14idps force field improving the conformation sampling of intrinsically disordered proteins. *Chemical Biology & Drug Design*, **2017**, *89*, 5–15.
- [46] W. Ye; D. Ji; W. Wang; R. Luo; H.-F. Chen. Test and evaluation of ff99idps force field for intrinsically disordered proteins. *Journal of Chemical Information and Modeling*, **2015**, *55*, 1021–1029.
- [47] J. Huang; S. Rauscher; G. Nawrocki; T. Ran; M. Feig; B. L. de Groot; H. Grubmüller; A. D. MacKerell Jr. Charmm36m: an improved force field for folded and intrinsically disordered proteins. *Nat Meth*, **2017**, *14*, 71–73.
- [48] P. S. Shabane; S. Izadi; A. V. Onufriev. General purpose water model can improve atomistic simulations of intrinsically disordered proteins. *Journal of Chemical Theory and Computation*, **2019**, *15*, 2620–2634.
- [49] T. E. Cheatham, III; M. A. Young. Molecular dynamics simulation of nucleic acids: Successes, limitations and promise. *Biopolymers*, **2001**, *56*, 232–256.
- [50] T. E. Cheatham, III. Simulation and modeling of nucleic acid structure, dynamics and interactions. *Curr. Opin. Struct. Biol.*, **2004**, *14*, 360–367.
- [51] P. Varnai; D. Djuranovic; R. Lavery; B. Hartmann. α/γ Transitions in the B-DNA backbone. *Nucl. Acids Res.*, **2002**, *30*, 5398–5406.

- [52] N. Spackova; T. E. Cheatham; F. Ryjacek; F. Lankas; L. vanMeervelt; P. Hobza; J. Sponer. Molecular Dynamics Simulations and Thermodynamics Analysis of DNA-Drug Complexes. Minor Groove Binding between 4',6-Diamidino-2-phenylindole and DNA Duplexes in Solution. *J. Am. Chem. Soc.*, **2003**, *125*, 1759–1769.
- [53] A. Perez; I. Marchan; D. Svozil; J. Sponer; T. E. Cheatham, III; C. A. Laughton; M. Orozco. Refinement of the AMBER Force Field for Nucleic Acids: Improving the Description of alpha/gamma Conformers. *Biophys. J.*, **2007**, *92*, 3817–3829.
- [54] D. Svozil; J. E. Sponer; I. Marchan; A. Perez; T. E. Cheatham; F. Forti; F. J. Luque; M. Orozco; J. Sponer. Geometrical and electronic structure variability of the sugar-phosphate backbone in nucleic acids. *J. Phys. Chem. B*, **2008**, *112*, 8188–8197.
- [55] M. Zgarbova; M. Otyepka; J. Sponer; A. Mladek; P. Banas; T. E. Cheatham, III; P. Jurecka. Refinement of the Cornell et al. Nucleic Acids Force Field Based on Reference Quantum Chemical Calculations of Glycosidic Torsion Profiles. *J. Chem. Theory Comput.*, **2011**, *7*, 2886–2902.
- [56] T. Steinbrecher; J. Latzer; D. A. Case. Revised AMBER Parameters for Bioorganic Phosphates. *J. Chem. Theory Comput.*, **2012**, *8*, 4405–4412.
- [57] C. Bergonzo; T. E. Cheatham, III. Improved force field parameters lead to a better description of RNA structure. *J. Chem. Theory Comput.*, **2015**, *11*, 3969–3972.
- [58] I. Yildirim; H. A. Stern; S. D. Kennedy; J. D. Tubbs; D. H. Turner. Reparameterization of RNA chi Torsion Parameters for the AMBER Force Field and Comparison to NMR Spectra for Cytidine and Uridine. *J. Chem. Theory Comput.*, **2010**, *6*, 1520–1531.
- [59] A. H. Aytenfisu; A. Spasic; A. Grossfield; H. A. Stern; D. H. Mathews. Revised RNA Dihedral Parameters for the Amber Force Field Improve RNA Molecular Dynamics. *J. Chem. Theory Comput.*, **2017**, *13*, 900–915.
- [60] D. Tan; S. Piana; R. Dirks; D. Shaw. RNA force field with accuracy comparable to state-of-the-art protein force fields. *Proc. Natl. Acad. Sci. USA*, **2018**, *115*, E1346–E1355.
- [61] R. Aduri; B. T. Psciuk; P. Saro; H. Taniga; H. B. Schlegel; J. SantaLucia, Jr. AMBER force field parameters for the naturally occurring modified nucleosides in RNA. *J. Chem. Theory Comput.*, **2007**, *3*, 1465–1475.
- [62] P. Banáš; D. Hollas; M. Zgarbová; P. Jurecka; M. Orozco; T. E. Cheatham, III; J. Šponer; M. Otyepka. Performance of molecular mechanics force fields for RNA simulations: Stability of UUCG and GNRA hairpins. *J. Chem. Theory. Comput.*, **2010**, *6*, 3836–3849.
- [63] I. Yildirim; H. A. Stern; J. D. Tubbs; S. D. Kennedy; D. H. Turner. Benchmarking AMBER Force Fields for RNA: Comparisons to NMR Spectra for Single-Stranded r(GACC) Are Improved by Revised chi Torsions. *J. Phys. Chem. B*, **2011**, *115*, 9261–9270.
- [64] I. Yildirim; S. D. Kennedy; H. A. Stern; J. M. Hart; R. Kierzek; D. H. Turner. Revision of AMBER Torsional Parameters for RNA Improves Free Energy Predictions for Tetramer Duplexes with GC and iGiC Base Pairs. *J. Chem. Theory Comput.*, **2012**, *8*, 172–181.
- [65] M. Krepl; M. Zgarbova; P. Stadlbauer; M. Otyepka; P. Banas; J. Koca; T. E. Cheatham, III; J. Sponer. Reference simulations of noncanonical nucleic acids with different chi variants of the AMBER force field: Quadruplex DNA, quadruplex RNA, and Z-DNA. *J. Chem. Theory Comp.*, **2012**, *8*, 2506–2520.
- [66] M. Zgarbová; F. J. Luque; J. Šponer; T. E. Cheatham, III; M. Otyepka; P. Jurečka. Toward improved description of dna backbone: Revisiting epsilon and zeta torsion force field parameters. *J. Chem. Theory Comput.*, **2013**, *9*, 2339–2354.

BIBLIOGRAPHY

- [67] M. Zgarbová; J. Sponer; M. Otyepka; T. E. Cheatham, III; R. Galindo-Murillo; P. Jurečka. Refinement of the Sugar-Phosphate Backbone Torsion Beta for AMBER Force Fields Improves the Description of Z- and B-DNA. *J. Chem. Theor. and Comp.*, **2015**, *12*, 5723–5736.
- [68] R. Galindo-Murillo; J. C. Robertson; M. Zgarbovic; J. Sponer; M. Otyepka; P. Jureska; T. E. Cheatham. Assessing the Current State of Amber Force Field Modifications for DNA. *J. Chem. Theory Comput.*, **2016**, *12*, 4114–4127.
- [69] I. Ivani; P. D. Dans; A. Noy; A. Pérez; I. Faustino; A. Hopsital; J. Walther; P. Andrió; R. Goni; A. Balaceanu; G. Portella; F. Battistini; J. L. Gelpá; C. González; M. Vendruscolo; C. A. Laughton; S. Harris; D. A. Case; M. Orozco. Parmbsc1: A refined force field for DNA simulations. *Nature Meth.*, **2016**, *13*, 55–58.
- [70] K. Liebl; M. Zacharias. Tumuc1: A New Accurate DNA Force Field Consistent with High-Level Quantum Chemistry. *J. Chem. Theory Comput.*, **2021**, *17*, 7096–7105.
- [71] K. Liebl; M. Zacharias. Toward Force fields with Improved Base Stacking Descriptions. *J. Chem. Theory Comput.*, **2023**, *19*, 1529–1536.
- [72] M. Zgarbová; J. Sponer; P. Jurecka. Z-DNA as a Touchstone for Additive Empirical Force Fields and a Refinement of the Alpha/Gamma DNA Torsions for AMBER. *J. Chem. Theory Comput.*, **2021**, *17*, 6292–6301.
- [73] M. Zgarbová; J. Šponer; P. Jurečka. Refinement of the Sugar Puckering Torsion Potential in the AMBER DNA Force Field. *J. Chem. Theory Comput.*, **2025**, *21*, 833–846.
- [74] O. Love; R. Galindo-Murillo; M. Zgarbová; J. Sponer; P. Jurecka; T. Cheatham, III. Assessing the Current State of Amber Force Field Modifications for DNA-2023 Edition. *J. Chem. Theory Comput.*, **2023**, *19*, 4299–4307.
- [75] K. Liebl; M. Zacharias. The development of nucleic acids force fields: From an unchallenged past to a competitive future. *J. Chem. Theory Comput.*, **2023**, *122*, 2841–2851.
- [76] B. Knappeová; V. Mlýnský; M. Pykal; J. Šponer; P. Banáš; M. Otyepka; M. Krepl. Comprehensive Assessment of Force-Field Performance in Molecular Dynamics Simulations of DNA/RNA Hybrid Duplexes. *J. Chem. Theory Comput.*, **2024**, *20*, 6917–6929.
- [77] N. Homeyer; A. H. C. Horn; H. Lanig; H. Sticht. AMBER force-field parameters for phosphorylated amino acids in different protonation states: phosphoserine, phosphothreonine, phosphotyrosine, and phosphohistidine. *J. Mol. Model.*, **2006**, *12*, 281–289.
- [78] L. E. Raguette; A. E. Cuomo; K. A. A. Belfon; C. Tian; V. Hazoglou; G. Witek; S. M. Telehany; Q. Wu; C. Simmerling. phosaa14SB and phosaa19SB: Updated Amber Force Field Parameters for Phosphorylated Amino Acids. *J. Chem. Theory Comput.*, **2024**, *20*, 7199–7209.
- [79] K. Belfon; L. Raguette; Q. Wu; C. Simmerling. Application of RAGTAG: modified amino acids for comparing MD simulations with FRET/EPR experiments. *In Prep*, **2020**.
- [80] K. Belfon; C. Tian; J. Maier; L. Raguette; Q. Wu; C. Simmerling. RAGTAG: Rapid Amber Gpu Torsion pArameter Generator. . *In Prep*, **2020**.
- [81] O. Love; R. Galindo-Murillo; D. R. Roe; P. D. Dans; T. E. Cheatham, III; C. Bergonzo. modXNA: A Modular Approach to Parametrization of Modified Nucleic Acids for Use with Amber Force Fields. *J. Chem. Theory Comput.*, **2024**, *20*, 9354–9363.
- [82] K. N. Kirschner; A. B. Yongye; S. M. Tschampel; J. González-Outeiriño; C. R. Daniels; B. L. Foley; R. J. Woods. GLYCAM06: A generalizable biomolecular force field. Carbohydrates. *J. Comput. Chem.*, **2008**, *29*, 622–655.

- [83] M. L. DeMarco; R. J. Woods. Atomic-resolution conformational analysis of the G(M3) ganglioside in a lipid bilayer and its implications for ganglioside-protein recognition at membrane surfaces. *Glycobiology*, **2009**, *19*, 344–355.
- [84] M. L. DeMarco; R. J. Woods; J. H. Prestegard; F. Tian. Presentation of Membrane-Anchored Glycosphingolipids Determined from Molecular Dynamics Simulations and NMR Paramagnetic Relaxation Rate Enhancement. *J. Am. Chem. Soc.*, **2010**, *132*, 1334–1338.
- [85] R. Kadirvelraj; O. C. Grant; I. J. Goldstein; H. C. Winter; H. Tateno; E. Fadda; R. J. Woods. Structure and binding analysis of Polyporus squamosus lectin in complex with the Neu5Ac α 2-6Gal β 1-4GlcNAc human-type influenza receptor. *Glycobiology*, **2011**, *21*, 973–984.
- [86] M. L. DeMarco; R. J. Woods. From agonist to antagonist: Structure and dynamics of innate immune glycoprotein MD-2 upon recognition of variably acylated bacterial endotoxins. *Mol. Immunol.*, **2011**, *49*, 124–133.
- [87] B. L. Foley; M. B. Tessier; R. J. Woods. Carbohydrate force fields. *WIREs Comput. Mol. Sci.*, **2012**, *2*, 652–697.
- [88] E. Ficko-Blean; C. P. Stuart; M. D. Suits; M. Cid; M. Tessier; R. J. Woods; A. B. Boraston. Carbohydrate Recognition by an Architecturally Complex α -N-Acetylglucosaminidase from Clostridium perfringens. *PLoS ONE*, **2012**, *7*, e33524.
- [89] M. B. Tessier; M. L. DeMarco; A. B. Yongye; R. J. Woods. Extension of the GLYCAM06 biomolecular force field to lipids, lipid bilayers and glycolipids. *Mol. Simul.*, **2008**, *34*, 349–363.
- [90] R. J. Woods. Restrained electrostatic potential charges for condensed phase simulations of carbohydrates. *J. Mol. Struct (Theochem)*, **2000**, *527*, 149–156.
- [91] R. J. Woods. Derivation of net atomic charges from molecular electrostatic potentials. *J. Comput. Chem.*, **1990**, *11*, 29–310.
- [92] M. Basma; S. Sundara; D. Calgan; T. Venali; R. J. Woods. Solvated ensemble averaging in the calculation of partial atomic charges. *J. Comput. Chem.*, **2001**, *22*, 1125–1137.
- [93] S. M. Tschampel; M. R. Kennerty; R. J. Woods. TIP5P-consistent treatment of electrostatics for biomolecular simulations. *J. Chem. Theory Comput.*, **2007**, *3*, 1721–1733.
- [94] M. L. DeMarco; R. J. Woods. Bridging computational biology and glycobiology: A game of snakes and ladders. *Glycobiology*, **2008**, *18*, 426–440.
- [95] S. E. Feller. Molecular dynamics simulations of lipid bilayers. *Curr. Opin. Colloid Interface Sci.*, **2000**, *5*, 217–223.
- [96] L. Saiz; M. L. Klein. Computer Simulation Studies of Model Biological Membranes. *Acc. Chem. Res.*, **2002**, *35*, 482–489.
- [97] A. Lomize; I. Pogozheva; M. Lomize; H. Mosberg. The role of hydrophobic interactions in positioning of peripheral proteins in membranes. *BMC Struct. Biol.*, **2007**, *7*, 44.
- [98] M. L. Lundstrom, K. H.; Chiu. *G Protein-Coupled Receptors in Drug Discovery*. Taylor & Francis, London, 2005.
- [99] M. F. Crowley; M. J. Williamson; R. C. Walker. CHAMBER: Comprehensive support for CHARMM force fields within the AMBER software. *Int. J. Quant. Chem.*, **2009**, *109*, 3767–3772.
- [100] L. Rosso; I. R. Gould. Structure and dynamics of phospholipid bilayers using recently developed general all-atom force fields. *J. Comput. Chem.*, **2008**, *29*, 24–37.

BIBLIOGRAPHY

- [101] Å. Skjevik; B. D. Madej; R. C. Walker; K. Teigen. Lipid11: A modular framework for lipid simulations using amber. *J. Phys. Chem. B*, **2012**, *116*, 11124–11136.
- [102] C. J. Dickson; L. Rosso; R. M. Betz; R. C. Walker; I. R. Gould. GAFFlipid: a General Amber Force Field for the accurate molecular dynamics simulation of phospholipid. *Soft Matter*, **2012**, *8*, 9617.
- [103] C. J. Dickson; B. D. Madej; A. A. Skjevik; R. M. Betz; K. Teigen; I. R. Gould; R. C. Walker. Lipid14: The Amber Lipid Force Field. *J. Chem. Theory Comput.*, **2014**, *10*, 865–879.
- [104] Å. Skjevik; B. D. Madej; C. J. Dickson; K. Teigen; R. C. Walker; I. R. Gould. All-atom lipid bilayer self-assembly with the amber and charmm lipid force fields. *Chem. Commun.*, **2015**, *51*, 4402–4405.
- [105] Å. Skjevik; B. D. Madej; C. J. Dickson; C. Lin; K. Teigen; R. C. Walker; I. R. Gould. Simulations of lipid bilayer self-assembly using all-atom lipid force fields. *Phys. Chem. Chem. Phys.*, **2016**, *18*, 10573–10584.
- [106] B. D. Madej; I. R. Gould; R. C. Walker. A Parameterization of Cholesterol for Mixed Lipid Bilayer Simulation within the Amber Lipid14 Force Field. *J Phys Chem B*, **2015**, *119*, 12424–12435.
- [107] C. J. Dickson; R. C. Walker; I. R. Gould. Lipid21: Complex Lipid Membrane Simulations with AMBER. *J. Chem. Theory Comput.*, **2022**.
- [108] Y. Gomez; A. Natale; J. Lincoff; C. Wolgemuth; J. osenberg; M. Grabe. Taking the Monte-Carlo gamble: How not to buckle under the pressure! *J. Comput. Chem.*, **2022**, *43*, 431–434.
- [109] W. L. Jorgensen; J. Chandrasekhar; J. Madura; M. L. Klein. Comparison of simple potential functions for simulating liquid water. *J. Chem. Phys.*, **1983**, *79*, 926–935.
- [110] D. J. Price; C. L. Brooks. A modified TIP3P water potential for simulation with Ewald summation. *J. Chem. Phys.*, **2004**, *121*, 10096–10103.
- [111] W. L. Jorgensen; J. D. Madura. Temperature and size dependence for Monte Carlo simulations of TIP4P water. *Mol. Phys.*, **1985**, *56*, 1381–1392.
- [112] H. W. Horn; W. C. Swope; J. W. Pitera; J. D. Madura; T. J. Dick; G. L. Hura; T. Head-Gordon. Development of an improved four-site water model for biomolecular simulations: TIP4P-Ew. *J. Chem. Phys.*, **2004**, *120*, 9665–9678.
- [113] H. W. Horn; W. C. Swope; J. W. Pitera. Characterization of the TIP4P-Ew water model: Vapor pressure and boiling point. *J. Chem. Phys.*, **2005**, *123*, 194504.
- [114] M. W. Mahoney; W. L. Jorgensen. A five-site model for liquid water and the reproduction of the density anomaly by rigid, nonpolarizable potential functions. *J. Chem. Phys.*, **2000**, *112*, 8910–8922.
- [115] S. Izadi; A. V. Onufriev. Accuracy limit of rigid 3-point water models. *J. Chem. Phys.*, **2016**, *145*, 074501.
- [116] Y. Xiong; S. Izadi; A. V. Onufriev. Fast polarizable water model for atomistic simulations. *Journal of Chemical Theory and Computation*, **2022**, *18*, 6324–6333. PMID: 36190318.
- [117] J. W. Caldwell; P. A. Kollman. Structure and properties of neat liquids using nonadditive molecular dynamics: Water, methanol and N-methylacetamide. *J. Phys. Chem.*, **1995**, *99*, 6208–6219.
- [118] H. J. C. Berendsen; J. R. Grigera; T. P. Straatsma. The missing term in effective pair potentials. *J. Phys. Chem.*, **1987**, *91*, 6269–6271.
- [119] Y. Wu; H. L. Tepper; G. A. Voth. Flexible simple point-charge water model with improved liquid-state properties. *J. Chem. Phys.*, **2006**, *124*, 024503.
- [120] F. Paesani; W. Zhang; D. A. Case; T. E. Cheatham, III; G. A. Voth. An accurate and simple quantum model for liquid water. *J. Chem. Phys.*, **2006**, *125*, 184507.

- [121] P. Cieplak; J. Caldwell; P. Kollman. Molecular mechanical models for organic and biological systems going beyond the atom centered two body additive approximation: Aqueous solution free energies of methanol and N-methyl acetamide, nucleic acid base, and amide hydrogen bonding and chloroform/water partition coefficients of the nucleic acid bases. *J. Comput. Chem.*, **2001**, 22, 1048–1057.
- [122] R. Anandakrishnan; C. Baker; S. Izadi; A. V. Onufriev. Point charges optimally placed to represent the multipole expansion of charge distributions. *PloS one*, **2013**, 8, e67715.
- [123] S. Niu; M. L. Tan; T. Ichiye. The large quadrupole of water molecules. *J. Chem. Phys.*, **2011**, 134, 134501+.
- [124] A. Mukhopadhyay; A. T. Fenley; I. S. Tolokh; A. V. Onufriev. Charge hydration asymmetry: the basic principle and how to use it to test and improve water models. *J. Phys. Chem. B*, **2012**, 116, 9776–9783.
- [125] D. Dans; D. Gallego; A. Balaceanu; L. Darre; H. Gomez; M. Orozco. Modeling, simulations, and bioinformatics at the service of rna structure. *Chem*, **2019**, 5, 51 – 73.
- [126] P. Kuhrova; V. Mlynsky; M. Zgarbova; M. Krepl; G. Bussi; R. B. Best; M. Otyepka; J. Sponer; P. Banas. Improving the performance of the Amber RNA force field by tuning the hydrogen-bonding interactions. *bioRxiv*, **2019**.
- [127] A. Bochicchio; M. Krepl; F. Yang; G. Varani; J. Sponer; P. Carloni. Molecular basis for the increased affinity of an RNA recognition motif with re-engineered specificity: A molecular dynamics and enhanced sampling simulations study. *PLOS Computat. Biol.*, **2018**, 14, 1–27.
- [128] N. M. Kumbhar; J. S. Gopal. Structural significance of hypermodified nucleoside 5-carboxymethylaminomethyluridine (cmnm5U) from wobble (34th) position of mitochondrial tRNAs: Molecular modeling and Markov state model studies. *J. Molec. Graph. Model.*, **2019**, 86, 66 – 83.
- [129] F. Leonarski; M. Jasinski; J. Trylska. Thermodynamics of the fourU RNA thermal switch derived from molecular dynamics simulations and spectroscopic techniques. *Biochimie*, **2019**, 156, 22 – 32.
- [130] C. Yang; M. Kulkarni; M. Lim; Y. Pak. Insilico direct folding of thrombin-binding aptamer G-quadruplex at all-atom level. *Nucl. Acids Res.*, **2017**, 45, 12648–12656.
- [131] K. Gao; J. Yin; N. M. Henriksen; A. T. Fenley; M. K. Gilson. Binding enthalpy calculations for a neutral host-guest pair yield widely divergent salt effects across water models. *J. of Chem. Theory Comput.*, **2015**, 11, 4555–4564.
- [132] O. H. S. Ollila; H. A. Heikkinen; H. Iwamatsu. Rotational dynamics of proteins from spin relaxation times and molecular dynamics simulations. *The Journal of Physical Chemistry B*, **2018**, 122, 6559–6569. PMID: 29812937.
- [133] M. Javanainen; A. Lamberg; L. Cwiklik; I. Vattulainen; O. H. S. Ollila. Atomistic model for nearly quantitative simulations of langmuir monolayers. *Langmuir*, **2018**, 34, 2565–2572. PMID: 28945973.
- [134] D. Pantoja-Uceda; J. L. Neira; L. M. Contreras; C. A. Manton; D. R. Welch; B. Rizzuti. The isolated C-terminal nuclear localization sequence of the breast cancer metastasis suppressor 1 is disordered. *Arch. Biochem. Biophys.*, **2019**, 664, 95 – 101.
- [135] Z. Li; S. F. Lin; P. Li; K. M. Merz, Jr. Systematic Parametrization of Divalent Metal Ions for the OPC3, OPC, TIP3P-FB, and TIP4P-FB Water Models. *J. Chem. Theory Comput.*, **2020**, 16, 4429–4442.
- [136] A. Sengupta; Z. Li; S. F. Lin; P. Li; K. M. Merz, Jr. Parameterization of Monovalent Ions for the OPC3, OPC, TIP3P-FB, and TIP4P-FB Water Models. *J. Chem. Inf. Model.*, **2021**, 61, 869–880.
- [137] Z. Li; S. F. Lin; P. Li; K. M. Merz, Jr. Parametrization of Trivalent and Tetravalent Metal Ions for the OPC3, OPC, TIP3P-FB, and TIP4P-FB Water Models. *J. Chem. Theory Comput.*, **2021**, 17, 2342–2354.

BIBLIOGRAPHY

- [138] M. Kulkarni; C. Yang; Y. Pak. Refined alkali metal ion parameters for the opc water model. *Bulletin of the Korean Chemical Society*, **2018**, 39, 931–935.
- [139] I. S. Joung; T. E. Cheatham, III. Molecular dynamics simulations of the dynamic and energetic properties of alkali and halide ions using water-model-specific ion parameters. *J. Phys. Chem. B*, **2009**, 113, 13279–13290.
- [140] J. A. Lemkul; J. Huang; B. Roux; A. D. MacKerell. An empirical polarizable force field based on the classical drude oscillator model: Development history and recent applications. *Chem. Rev.*, **2016**, 116, 4983–5013. PMID: 26815602.
- [141] Y. Xiong; A. V. Onufriev. Exploring optimization strategies for improving explicit water models: Rigid n-point model and polarizable model based on Drude oscillator. *PLoS ONE*, **2019**, 14.
- [142] C. W. Hopkins; S. Le Grand; R. C. Walker; A. E. Roitberg. Long-Time-Step Molecular Dynamics through Hydrogen Mass Repartitioning. *J. Chem. Theory Comput.*, **2015**, 11, 1864–1874.
- [143] S. Joung; T. E. Cheatham, III. Determination of alkali and halide monovalent ion parameters for use in explicitly solvated biomolecular simulations. *J. Phys. Chem. B*, **2008**, 112, 9020–9041.
- [144] P. Li; B. P. Roberts; D. K. Chakravorty; K. M. Merz, Jr. Rational Design of Particle Mesh Ewald Compatible Lennard-Jones Parameters for +2 Metal Cations in Explicit Solvent. *J. Chem. Theory Comput.*, **2013**, 9, 2733–2748.
- [145] P. Li; K. M. Merz, Jr. Taking into Account the Ion-Induced Dipole Interaction in the Nonbonded Model of Ions. *J. Chem. Theory Comput.*, **2014**, 10, 289–297.
- [146] P. Li; L. F. Song; K. M. Merz, Jr. Parameterization of Highly Charged Metal Ions Using the 12-6-4 LJ-Type Nonbonded Model in Explicit Water. *J. Phys. Chem. B*, **2015**, 119, 883–895.
- [147] P. Li; L. F. Song; K. M. Merz, Jr. Systematic Parameterization of Monovalent Ions Employing the Nonbonded Model. *J. Chem. Theory Comput.*, **2015**, 11, 1645–1657.
- [148] E. S. Kolesnikov; I. Y. Gushchin; P. A. Zhilyaev; A. V. Onufriev. Similarities and Differences between Na⁺ and K⁺ Distributions around DNA Obtained with Three Popular Water Models. *J. Chem. Theory Comput.*, **2021**, 17, 7246–7259.
- [149] M. T. Panteva; G. M. Giambasu; D. M. York. Comparison of Structural, Thermodynamic, Kinetic and Mass Transport Properties of Mg²⁺ Models Commonly Used in Biomolecular Simulations. *J. Comput. Chem.*, **2015**, 36, 970–982.
- [150] M. T. Panteva; G. M. Giambasu; D. M. York. Force Field for Mg²⁺, Mn²⁺, Zn²⁺ and Cd²⁺ Ions That Have Balanced Interactions with Nucleic Acids. *J. Phys. Chem. B*, **2015**, 119, 15460–15470.
- [151] G. M. Giambasu; D. A. Case; D. M. York. Predicting Site-Binding Modes of Ions and Water to Nucleic Acids Using Molecular Solvation Theory. *J. Am. Chem. Soc.*, **2019**, 141, 2435–2445.
- [152] A. Sengupta; A. Seitz; K. M. Merz, Jr. Simulating the Chelate Effect. *J. Am. Chem. Soc.*, **2018**, 140, 15166–15169.
- [153] S. F. Lin; A. Sengupta; K. M. Merz, Jr. Thermodynamics of Transition Metal Ion Binding to Proteins. *J. Am. Chem. Soc.*, **2020**, 142, 6365–6374.
- [154] P. Li. Bridging the 12-6-4 Model and the Fluctuating Charge Model. *Front. Chem.*, **2021**, 9, 721960.
- [155] A. M. Wollacott; K. M. Merz, Jr. Development of a parameterized force field to reproduce semiempirical geometries. *J. Chem. Theory Comput.*, **2006**, 2, 1070–1077.

- [156] S. N. Steinmann; R. Ferreira de Moraes; A. W. Götz; P. Fleurat-Lessard; M. Iannuzzi; P. Sautet; C. Michel. Force field for water over pt(111): Development, assessment, and comparison. *J. Chem. Theory Comput.*, **2018**, *14*, 3258–3251.
- [157] S. N. Steinmann; P. Fleurat-Lessard; A. W. Götz; C. Michel; R. Ferreira de Moraes; P. Sautet. Molecular mechanics models for the image charge, a comment on “including image charge effects in the molecular dynamics simulations of molecules on metal surfaces”. *J. Comput. Chem.*, **2017**, *38*, 2127–2129.
- [158] T. Graen; M. Hoefling; H. Grubmueller. Amber-dyes: Characterization of charge fluctuations and force field parameterization of fluorescent dyes for molecular dynamics simulations. *Journal of Chemical Theory and Computation*, **2014**, *10*, 5505–5512.
- [159] B. Schepers; H. Gohlke. Amber-dyes in amber: Implementation of fluorophore and linker parameters into ambertools. *Journal of Chemical Physics*, **2020**, *152*.
- [160] F. Meng; M. M. Bellaiche; J.-Y. Kim; G. H. Zerze; R. B. Best; H. S. Chung. Highly disordered amyloid- β monomer probes by single-molecule fret and md simulation. *Biophysical Journal*, **2018**, *114*, 870–884.
- [161] C. Gebhardt; M. Lehmann; M. M. Reif; M. Zacharias; T. Cordes. Molecular and spectroscopic characterization of green and red cyanine fluorophores from the alexa fluor and af series. *bioRxiv*, **2020**.
- [162] K. L. Breyfogle; D. L. Blood; A. M. Rosnik; B. P. Krueger. Molecular dynamics parameters for the GFP chromophore and some of its analogues. *J. Phys. Chem. B*, **2023**, *127*, 5772–5788.
- [163] P. D. Dans; A. Zeida; M. R. Machado; S. Pantano. A coarse grained model for atomic-detailed dna simulations with explicit electrostatics. *J. Chem. Theory Comput.*, **2010**, *6*, 1711–1725. PMID: 26615701.
- [164] E. E. Barrera; M. R. Machado; S. Pantano. Fat sirah: Coarse-grained phospholipids to explore membrane-protein dynamics. *J. Chem. Theory Comput.*, **2019**, *15*, 5674–5688. PMID: 31433946.
- [165] M. R. Machado; E. E. Barrera; F. Klein; M. Sonora; S. Silva; S. Pantano. The SIRAH 2.0 Force Field: Altius, Fortius, Citius. *J. Chem. Theory Comput.*, **2019**, *15*, 2719–2733. PMID: 30810317.
- [166] P. G. Garay; E. E. Barrera; S. Pantano. Post-translational modifications at the coarse-grained level with the sirah force field. *J. Chem. Inform. Model.*, **2020**, *60*, 964–973. PMID: 31840995.
- [167] F. Klein; D. Cáceres; M. A. Carrasco; J. C. Tapia; J. Caballero; J. Alzate-Morales; S. Pantano. Coarse-Grained Parameters for Divalent Cations within the SIRAH Force Field. *J. Chem. Inf. Model.*, **2020**, *60*, 3935–3943.
- [168] P. G. Garay; M. R. Machado; H. Verli; S. Pantano. Sirah late harvest: Coarse-grained models for protein glycosylation. *J. Chem. Theory Comput.*, **2024**, *20*, 963–976.
- [169] L. Darré; M. R. Machado; P. D. Dans; F. E. Herrera; S. Pantano. Another coarse grain model for aqueous solvation: Wat four? *J. Chem. Theory Comput.*, **2010**, *6*, 3793–3807.
- [170] M. R. Machado; S. Pantano. SIRAH tools: mapping, backmapping and visualization of coarse-grained models. *Bioinformatics*, **2016**, *32*, 1568–1570.
- [171] A. Zeida; M. R. Machado; P. D. Dans; S. Pantano. Breathing, bubbling, and bending: DNA flexibility from multimicrosecond simulations. *Phys. Rev. E*, **2012**, *86*, 021903.
- [172] M. R. Machado; H. C. González; S. Pantano. Md simulations of viruslike particles with supra cg solvation affordable to desktop computers. *J. Chem. Theory Comput.*, **2017**, *13*, 5106–5116. PMID: 28876928.
- [173] M. Soñora; L. Martínez; S. Pantano; M. R. Machado. Wrapping up viruses at multiscale resolution : Optimizing packmol and sirah execution for simulating the Zika virus. *J. Chem. Inform. Model.*, **2021**, *61*.
- [174] H. C. Gonzalez; L. Darré; S. Pantano. Transferable mixing of atomistic and coarse-grained water models. *J. Phys. Chem. B*, **2013**, *117*, 14438–14448. PMID: 24219057.

BIBLIOGRAPHY

- [175] M. R. Machado; P. D. Dans; S. Pantano. A hybrid all-atom/coarse grain model for multiscale simulations of dna. *Phys. Chem. Chem. Phys.*, **2011**, *13*, 18134–18144.
- [176] M. R. Machado; S. Pantano. Exploring LacI–DNA Dynamics by Multiscale Simulations Using the SIRAH Force Field. *J. Chem. Theory Comput.*, **2015**, *11*, 5012–5023. PMID: 26574286.
- [177] M. R. Machado; A. Zeida; L. Darré; S. Pantano. From quantum to subcellular scales: multi-scale simulation approaches and the sirah force field. *Interface Focus*, **2019**, *9*.
- [178] J. Cantero; A. Ballesteros-Casallas; L. H. S. Santos; M. Paulino; S. Pantano. Pouring sirah on namd. *J. Phys. Chem. B*, **2024**, *128*.
- [179] L. David; R. Luo; M. K. Gilson. Comparison of generalized born and poisson models: Energetics and dynamics of hiv protease. *J. Comput. Chem.*, **2000**, *21*, 295–309.
- [180] M. Feig. Kinetics from Implicit Solvent Simulations of Biomolecules as a Function of Viscosity. *J. Chem. Theory Comput.*, **2007**, *3*, 1734–1748.
- [181] R. E. Amaro; X. Cheng; I. Ivanov; D. Xu; A. J. Mccammon. Characterizing Loop Dynamics and Ligand Recognition in Human- and Avian-Type Influenza Neuraminidases via Generalized Born Molecular Dynamics and End-Point Free Energy Calculations. *J. Am. Chem. Soc.*, **2009**, *131*, 4702–4709.
- [182] B. Zagrovic; V. Pande. Solvent viscosity dependence of the folding rate of a small protein: Distributed computing study. *J. Comput. Chem.*, **2003**, *24*, 1432–1436.
- [183] R. Anandakrishnan; A. Drozdetski; R. C. Walker; A. V. Onufriev. Speed of Conformational Change: Comparing Explicit and Implicit Solvent Molecular Dynamics Simulations. *Biophysical Journal*, **2015**, *108*, 1153–1164.
- [184] A. V. Onufriev; D. A. Case. Generalized Born implicit solvent models for biomolecules. *Annu. Rev. Biophys.*, **2019**, *48*, 275–296.
- [185] J. Weiser; P. S. Shenkin; W. C. Still. Approximate Atomic Surfaces from Linear Combinations of Pairwise Overlaps (LCPO). *J. Comput. Chem.*, **1999**, *20*, 217–230.
- [186] W. C. Still; A. Tempczyk; R. C. Hawley; T. Hendrickson. Semianalytical treatment of solvation for molecular mechanics and dynamics. *J. Am. Chem. Soc.*, **1990**, *112*, 6127–6129.
- [187] M. Schaefer; M. Karplus. A comprehensive analytical treatment of continuum electrostatics. *J. Phys. Chem.*, **1996**, *100*, 1578–1599.
- [188] S. R. Edinger; C. Cortis; P. S. Shenkin; R. A. Friesner. Solvation free energies of peptides: Comparison of approximate continuum solvation models with accurate solution of the Poisson-Boltzmann equation. *J. Phys. Chem. B*, **1997**, *101*, 1190–1197.
- [189] B. Jayaram; D. Sprous; D. L. Beveridge. Solvation free energy of biomacromolecules: Parameters for a modified generalized Born model consistent with the AMBER force field. *J. Phys. Chem. B*, **1998**, *102*, 9571–9576.
- [190] C. J. Cramer; D. G. Truhlar. Implicit solvation models: Equilibria, structure, spectra, and dynamics. *Chem. Rev.*, **1999**, *99*, 2161–2200.
- [191] D. Bashford; D. A. Case. Generalized Born models of macromolecular solvation effects. *Annu. Rev. Phys. Chem.*, **2000**, *51*, 129–152.
- [192] A. Onufriev; D. Bashford; D. A. Case. Modification of the generalized Born model suitable for macromolecules. *J. Phys. Chem. B*, **2000**, *104*, 3712–3720.
- [193] M. S. Lee; F. R. Salsbury, Jr.; C. L. Brooks, III. Novel generalized Born methods. *J. Chem. Phys.*, **2002**, *116*, 10606–10614.

- [194] B. N. Dominy; C. L. Brooks, III. Development of a generalized Born model parameterization for proteins and nucleic acids. *J. Phys. Chem. B*, **1999**, *103*, 3765–3773.
- [195] V. Tsui; D. A. Case. Molecular dynamics simulations of nucleic acids using a generalized Born solvation model. *J. Am. Chem. Soc.*, **2000**, *122*, 2489–2498.
- [196] N. Calimet; M. Schaefer; T. Simonson. Protein molecular dynamics with the generalized Born/ACE solvent model. *Proteins*, **2001**, *45*, 144–158.
- [197] A. Onufriev; D. Bashford; D. A. Case. Exploring protein native states and large-scale conformational changes with a modified generalized Born model. *Proteins*, **2004**, *55*, 383–394.
- [198] J. Srinivasan; M. W. Trevathan; P. Beroza; D. A. Case. Application of a pairwise generalized Born model to proteins and nucleic acids: inclusion of salt effects. *Theor. Chem. Acc.*, **1999**, *101*, 426–434.
- [199] A. Onufriev; D. A. Case; D. Bashford. Effective Born radii in the generalized Born approximation: The importance of being perfect. *J. Comput. Chem.*, **2002**, *23*, 1297–1304.
- [200] G. D. Hawkins; C. J. Cramer; D. G. Truhlar. Parametrized models of aqueous free energies of solvation based on pairwise descreening of solute atomic charges from a dielectric medium. *J. Phys. Chem.*, **1996**, *100*, 19824–19839.
- [201] F. M. Richards. Areas, volumes, packing, and protein structure. *Ann. Rev. Biophys. Bioeng.*, **1977**, *6*, 151–176.
- [202] M. Schaefer; C. Froemmel. A precise analytical method for calculating the electrostatic energy of macromolecules in aqueous solution. *J. Mol. Biol.*, **1990**, *216*, 1045–1066.
- [203] M. Feig; A. Onufriev; M. Lee; W. Im; D. A. Case; C. L. Brooks, III. Performance comparison of the generalized Born and Poisson methods in the calculation of the electrostatic solvation energies for protein structures. *J. Comput. Chem.*, **2004**, *25*, 265–284.
- [204] R. Geney; M. Layten; R. Gomperts; C. Simmerling. Investigation of salt bridge stability in a generalized Born solvent model. *J. Chem. Theory Comput.*, **2006**, *2*, 115–127.
- [205] A. Okur; L. Wickstrom; C. Simmerling. Evaluation of salt bridge structure and energetics in peptides using explicit, implicit and hybrid solvation models. *J. Chem. Theory Comput.*, **2008**, *4*, 488–498.
- [206] A. Okur; L. Wickstrom; M. Layten; R. Geney; K. Song; V. Hornak; C. Simmerling. Improved efficiency of replica exchange simulations through use of a hybrid explicit/implicit solvation model. *J. Chem. Theory Comput.*, **2006**, *2*, 420–433.
- [207] D. R. Roe; A. Okur; L. Wickstrom; V. Hornak; C. Simmerling. Secondary Structure Bias in Generalized Born Solvent Models: Comparison of Conformational Ensembles and Free Energy of Solvent Polarization from Explicit and Implicit Solvation. *J. Phys. Chem. B*, **2007**, *111*, 1846–1857.
- [208] H. Nguyen; J. Maier; H. Huang; V. Perrone; C. Simmerling. Folding simulations for proteins with diverse topologies are accessible in days with a physics-based force field and implicit solvent. *J. Am. Chem. Soc.*, **2014**, *136*, 13959–13962. PMID: 25255057.
- [209] H. Nguyen; A. Pérez; S. Bermeo; C. Simmerling. Refinement of Generalized Born Implicit Solvation Parameters for Nucleic Acids and Their Complexes with Proteins. *J. Chem. Theory Comput.*, **2015**, *11*, 3714–3728.
- [210] A. Onufriev. in *Modeling Solvent Environments*, M. Feig, Ed., (Wiley, USA). pp 127–165. 2010.
- [211] G. D. Hawkins; C. J. Cramer; D. G. Truhlar. Pairwise solute descreening of solute charges from a dielectric medium. *Chem. Phys. Lett.*, **1995**, *246*, 122–129.
- [212] M. Schaefer; H. W. T. Van Vlijmen; M. Karplus. Electrostatic contributions to molecular free energies in solution. *Adv. Protein Chem.*, **1998**, *51*, 1–57.

BIBLIOGRAPHY

- [213] V. Tsui; D. A. Case. Theory and applications of the generalized Born solvation model in macromolecular simulations. *Biopolymers (Nucl. Acid. Sci.)*, **2001**, 56, 275–291.
- [214] C. P. Sosa; T. Hewitt; M. S. Lee; D. A. Case. Vectorization of the generalized Born model for molecular dynamics on shared-memory computers. *J. Mol. Struct. (Theochem)*, **2001**, 549, 193–201.
- [215] J. Mongan; C. Simmerling; J. A. McCammon; D. A. Case; A. Onufriev. Generalized Born with a simple, robust molecular volume correction. *J. Chem. Theory Comput.*, **2007**, 3, 156–169.
- [216] H. Huang; C. Simmerling. Fast Pairwise Approximation of Solvent Accessible Surface Area for Implicit Solvent Simulations of Proteins on CPUs and GPUs. *J. Chem. Theory Comput.*, **2018**, 14, 5797–5814.
- [217] D. Sitkoff; K. A. Sharp; B. Honig. Accurate calculation of hydration free energies using macroscopic solvent models. *J. Phys. Chem.*, **1994**, 98, 1978–1988.
- [218] I. S. Tolokh; D. G. Thomas; A. V. Onufriev. Explicit ions/implicit water generalized born model for nucleic acids. *J. Chem. Phys.*, **2018**, 148, 195101.
- [219] E. S. Kolesnikov; Y. Xiong; A. V. Onufriev. Implicit Solvent with Explicit Ions Generalized Born Model in Molecular Dynamics: Application to DNA. *J. Chem. Theory Comput.*, **2024**, 20, 8724–8739.
- [220] M. Auton; D. W. Bolen. Predicting the energetics of osmolyte-induced protein folding/unfolding. *Proc. Natl. Acad. Sci.*, **2005**, 102, 15065–15068.
- [221] A. Arsiccio; J.-E. Shea. Protein cold denaturation in implicit solvent simulations: A transfer free energy approach. *J. Phys. Chem. B*, **2021**, 125, 5222–5232.
- [222] A. Arsiccio; J.-E. Shea. Pressure unfolding of proteins: New insights into the role of bound water. *J. Phys. Chem. B*, **2021**, 125, 8431–8442.
- [223] A. Arsiccio; P. Ganguly; J.-E. Shea. A transfer free energy based implicit solvent model for protein simulations in solvent mixtures: Urea-induced denaturation as a case study. *J. Phys. Chem. B*, **2022**, 126, 4472–4482.
- [224] A. Arsiccio; X. Liu; P. Ganguly; S. K. Buratto; M. T. Bowers; J.-E. Shea. Effect of cosolutes on the aggregation of a tau fragment: A combined experimental and simulation approach. *J. Phys. Chem. B*, **2023**, 127, 4022–4031.
- [225] G. Sigalov; P. Scheffel; A. Onufriev. Incorporating variable environments into the generalized Born model. *J. Chem. Phys.*, **2005**, 122, 094511.
- [226] G. Sigalov; A. Fenley; A. Onufriev. Analytical electrostatics for biomolecules: Beyond the generalized Born approximation. *J. Chem. Phys.*, **2006**, 124, 124902.
- [227] B. Aguilar; A. V. Onufriev. Efficient computation of the total solvation energy of small molecules via the r6 generalized born model. *J. Chem. Theory Comput.*, **2012**, 8, 2404–2411.
- [228] B. Aguilar; R. Shadrach; A. V. Onufriev. Reducing the secondary structure bias in the generalized born model via r6 effective radii. *J. Chem. Theory Comput.*, **2010**, 6, 3613–3630.
- [229] N. Forouzesh; S. Izadi; A. V. Onufriev. Grid-Based Surface Generalized Born Model for Calculation of Electrostatic Binding Free Energies. *J. Chem. Inf. Model.*, **2017**, 57, 2505–2513.
- [230] S. Izadi; R. C. Harris; M. O. Fenley; A. V. Onufriev. Accuracy comparison of generalized born models in the calculation of electrostatic binding free energies. *J. Chem. Theory Comput.*, **2018**, 14, 1656–1670.
- [231] A. Mukhopadhyay; B. H. Aguilar; I. S. Tolokh; A. V. Onufriev. Introducing charge hydration asymmetry into the generalized born model. *J. Chem. Theory Comput.*, **2014**, 10, 1788–1794.
- [232] Q. Cai; X. Ye; J. Wang; R. Luo. On-the-Fly Numerical Surface Integration for Finite-Difference Poisson-Boltzmann Methods. *J. Chem. Theory Comput.*, **2011**, 7, 3608–3619.

- [233] R. Luo; L. David; M. K. Gilson. Accelerated Poisson-Boltzmann calculations for static and dynamic systems. *J. Comput. Chem.*, **2002**, 23, 1244–1253.
- [234] J. Wang; R. Luo. Assessment of Linear Finite-Difference Poisson-Boltzmann solvers. *J. Comput. Chem.*, **2010**, 31, 1689–1698.
- [235] Q. Cai; M.-J. Hsieh; J. Wang; R. Luo. Performance of Nonlinear Finite-Difference Poisson-Boltzmann Solvers. *J. Chem. Theory Comput.*, **2010**, 6, 203.
- [236] R. Qi; W. Botello-Smith; R. Luo. Acceleration of Linear Finite-Difference Poisson-Boltzmann Methods on Graphics Processing Units. *J. Chem. Theory Comput.*, **2017**, 13, 3378–3387.
- [237] R. Qi; R. Luo. Robustness and Efficiency of Poisson-Boltzmann Modeling on Graphics Processing Units. *J. Chem. Inf. Model.*, **2019**, 59, 409–420.
- [238] B. Honig; A. Nicholls. Classical electrostatics in biology and chemistry. *Science*, **1995**, 268, 1144–1149.
- [239] Q. Lu; R. Luo. A Poisson-Boltzmann dynamics method with nonperiodic boundary condition. *J. Chem. Phys.*, **2003**, 119, 11035–11047.
- [240] M. K. Gilson; K. A. Sharp; B. H. Honig. Calculating the electrostatic potential of molecules in solution: method. *J. Comput. Chem.*, **1988**, 9, 327–35.
- [241] J. Warwicker; H. C. Watson. Calculation of the electric potential in the active site cleft due to. *J. Mol. Biol.*, **1982**, 157, 671–679.
- [242] I. Klapper; R. Hagstrom; R. Fine; K. Sharp; B. Honig. Focussing of electric fields in the active stie of Cu, Zn superoxide dismutase. *Proteins*, **1986**, 1, 47–59.
- [243] C. H. Tan; Y. H. Tan; R. Luo. Implicit nonpolar solvent models. *J. Phys. Chem. B*, **2007**, 111, 12263–12274.
- [244] E. Gallicchio; M. M. Kubo; R. M. Levy. Enthalpy-entropy and cavity decomposition of alkane hydration free energies: Numerical results and implications for theories of hydrophobic solvation. *J. Phys. Chem.*, **2000**, 104, 6271–6285.
- [245] F. Floris; J. Tomasi. Evaluation of the dispersion contribution to the solvation energy. A simple computational model in the continuum approximation. *J. Comput. Chem.*, **1989**, 10, 616–627.
- [246] M. E. Davis; J. A. McCammon. Solving the finite-difference linearized Poisson-Boltzmann equation – a comparison of relaxation and conjugate gradient methods. *J. Comput. Chem.*, **1989**, 10, 386–391.
- [247] A. Nicholls; B. Honig. A rapid finite difference algorithm, utilizing successive over-relaxation to solve the Poisson-Boltzmann equation. *J. Comput. Chem.*, **1991**, 12, 435–445.
- [248] D. Bashford. An object-oriented programming suite for electrostatic effects in biological molecules. *Lect. Notes Comput. Sci.*, **1997**, 1343, 233–240.
- [249] J. Wang; Q. Cai; Z. Li; H. Zhao; R. Luo. Achieving Energy Conservation in Poisson-Boltzmann Molecular Dynamics: Accuracy and Precision with Finite-difference Algorithms. *Chem. Phys. Lett.*, **2009**, 468, 112.
- [250] Z. Li; I. K. *The Immersed Interface Method: Numerical Solutions of PDEs Involving Interfaces and Irregular Domains*. SIAM Frontiers in Applied Mathematics, Philadelphia, 2006.
- [251] B. A. Luty; M. E. Davis; J. A. McCammon. Electrostatic energy calculations by a finite-difference method: Rapid calculation of charge-solvent interaction energies. *J. Comput. Chem.*, **1992**, 13, 768–771.
- [252] Q. Cai; J. Wang; H. Zhao; R. Luo. On removal of charge singularity in Poisson-Boltzmann equation. *J. Chem. Phys.*, **2009**, 130, 145101.

BIBLIOGRAPHY

- [253] Q. Cai; X. Ye; J. Wang; R. Luo. Dielectric boundary force in numerical Poisson-Boltzmann methods: Theory and numerical strategies. *Chem. Phys. Lett.*, **2011**, 514, 368.
- [254] M. E. Davis; J. A. McCammon. Dielectric boundary smoothing in finite difference solutions of the Poisson equation: An approach to improve accuracy and convergence. *J. Comput. Chem.*, **1991**, 12, 909–912.
- [255] J. Wang; Q. Cai; Y. Xiang; R. Luo. Reducing Grid Dependence in Finite-Difference Poisson-Boltzmann Calculations. *J. Chem. Theory Comput.*, **2012**, 8, 2741–2751.
- [256] H. Wei; R. Luo; R. Qi. An Efficient Second-Order Poisson-Boltzmann Method. *J. Comput. Chem.*, **2019**, 40, 1257.
- [257] H. Wei; A. Luo; T. Qiu; R. Luo; R. Qi. Improved Poisson-Boltzmann Methods for High-Performance Computing. *J. Chem. Theory Comput.*, **2019**, 15, 6190.
- [258] C. Wang; P. Nguyen; K. Pham; D. Huynh; T. Le; H. Wang; P. Ren; R. Luo. Calculating protein-ligand binding affinities with MMPBSA: Method and error analysis. *J. Comput. Chem.*, **2016**, 37, 2436–2446.
- [259] M. E. Davis; J. A. McCammon. Electrostatics in biomolecular structure and dynamics. *Chem. Rev.*, **1990**, 90, 509–521.
- [260] C. H. Tan; L. J. Yang; R. Luo. How well does Poisson-Boltzmann implicit solvent agree with explicit solvent? A quantitative analysis. *J. Phys. Chem. B*, **2006**, 110, 18680–18687.
- [261] X. Ye; J. Wang; R. Luo. A Revised Density Function for Molecular Surface Calculation in Continuum Solvent Models. *J. Chem. Theory Comput.*, **2010**, 6, 1157–1169.
- [262] W. M. Botello-Smith; X. Liu; Q. Cai; Z. Li; H. Zhao; R. Luo. Numerical Poisson-Boltzmann Model for Continuum Membrane Systems. *Chem. Phys. Lett.*, **2013**, 555, 274.
- [263] D. Greene; R. Qi; R. Nguyen; T. Qiu; R. Luo. Heterogeneous dielectric implicit membrane model for the calculation of mmpbsa binding free energies. *J. Chem. Infom. Model.*, **2019**, 59, 3041.
- [264] L. Xiao; J. Diao; D. Greene; J. Wang; R. Luo. A Continuum Poisson-Boltzmann Model for Membrane Channel Proteins. *J. Chem. Theory Comput.*, **2017**, 13, 3398.
- [265] Q. Cai; X. Ye; R. Luo. Dielectric Pressure in Continuum Electrostatic Solvation of Biomolecules. *Phys. Chem. Chem. Phys.*, **2012**, 14, 15917–15925.
- [266] Y. Wu; H. Wei; Q. Zhu; R. Luo. Grid-robust efficient neural interface model for universal molecule surface construction from point clouds. *J. Phys. Chem. Lett.*, **2023**, 14, 9034–9041.
- [267] M.-J. Hsieh; R. Luo. Exploring a coarse-grained distributive strategy for finite-difference poisson-boltzmann calculations. *J. Molec. Model.*, **2011**.
- [268] A. Paszke; S. Gross; S. Chintala; G. Chanan; E. Yang; Z. DeVito; Z. Lin; A. Desmaison; L. Antiga; A. Lerer. Automatic differentiation in pytorch. *NIPS-W*, **2017**.
- [269] M. K. Gilson; M. Davis; B. A. Luty; J. A. McCammon. Computation of electrostatic forces on solvated molecules using the Poisson-Boltzmann equation. *J Phys Chem*, **1993**, 97, 3591–3600.
- [270] T. Luchko; S. Gusarov; D. R. Roe; C. Simmerling; D. A. Case; J. Tuszynski; A. Kovalenko. Three-dimensional molecular theory of solvation coupled with molecular dynamics in Amber. *J. Chem. Theory Comput.*, **2010**, 6, 607–624.
- [271] D. Chandler; H. C. Andersen. Optimized cluster expansions for classical fluids. ii. theory of molecular liquids. *J. Chem. Phys.*, **1972**, 57, 1930–1937.
- [272] F. Hirata; P. J. Rossky. An extended RISM equation for molecular polar fluids. *Chem. Phys. Lett.*, **1981**, pp 329–334.

- [273] F. Hirata; B. M. Pettitt; P. J. Rossky. Application of an extended rism equation to dipolar and quadrupolar fluids. *J. Chem. Phys.*, **1982**, 77, 509–520.
- [274] F. Hirata; P. J. Rossky; B. M. Pettitt. The interionic potential of mean force in a molecular polar solvent from an extended rism equation. *J. Chem. Phys.*, **1983**, 78, 4133–4144.
- [275] D. Chandler; J. McCoy; S. Singer. Density functional theory of nonuniform polyatomic systems. i. general formulation. *J. Chem. Phys.*, **1986**, 85, 5971–5976.
- [276] D. Chandler; J. McCoy; S. Singer. Density functional theory of nonuniform polyatomic systems. ii. rational closures for integral equations. *J. Chem. Phys.*, **1986**, 85, 5977–5982.
- [277] D. Beglov; B. Roux. Numerical solution of the hypernetted chain equation for a solute of arbitrary geometry in three dimensions. *J. Chem. Phys.*, **1995**, 103, 360–364.
- [278] D. Beglov; B. Roux. An integral equation to describe the solvation of polar molecules in liquid water. *J. Phys. Chem. B*, **1997**, 101, 7821–7826.
- [279] A. Kovalenko; F. Hirata. Three-dimensional density profiles of water in contact with a solute of arbitrary shape: a RISM approach. *Chem. Phys. Lett.*, **1998**, 290, 237–244.
- [280] A. Kovalenko; F. Hirata. Self-consistent description of a metal–water interface by the Kohn–Sham density functional theory and the three-dimensional reference interaction site model. *J. Chem. Phys.*, **1999**, 110, 10095–10112.
- [281] A. Kovalenko. In Hirata [1092], chapter 4.
- [282] A. Kovalenko; F. Hirata. Potentials of mean force of simple ions in ambient aqueous solution. i: Three-dimensional reference interaction site model approach. *J. Chem. Phys.*, **2000**, 112, 10391–10402.
- [283] A. Kovalenko; F. Hirata. Potentials of mean force of simple ions in ambient aqueous solution. ii: Solvation structure from the three-dimensional reference interaction site model approach, and comparison with simulations. *J. Chem. Phys.*, **2000**, 112, 10403–10417.
- [284] L. Wilson; R. Krasny; T. Luchko. Accelerating the 3D reference interaction site model theory of molecular solvation with treecode summation and cut-offs. *Journal of Computational Chemistry*, **2022**, 43, 1251–1270.
- [285] J. G. Gray; G. M. Giambasu; D. A. Case; T. Luchko. Integral equation models for solvent in macromolecular crystals. *The Journal of Chemical Physics*, **2022**, 156, 014801.
- [286] J.-P. Hansen; I. R. McDonald. *Theory of simple liquids*. Academic Press, London, 1990.
- [287] F. Hirata. In *Molecular Theory of Solvation* [1092], chapter 1.
- [288] J. S. Perkyns; B. M. Pettitt. A site-site theory for finite concentration saline solutions. *J. Chem. Phys.*, **1992**, 97, 7656–7666.
- [289] A. Kovalenko; S. Ten-No; F. Hirata. Solution of three-dimensional reference interaction site model and hypernetted chain equations for simple point charge water by modified method of direct inversion in iterative subspace. *J. Comput. Chem.*, **1999**, 20, 928–936.
- [290] I. S. Joung; T. Luchko; D. A. Case. Simple electrolyte solutions: Comparison of DRISM and molecular dynamics results for alkali halide solutions. *J Chem Phys*, **2013**, 138, 044103.
- [291] G. M. Giambasu; T. Luchko; D. Herschlag; D. M. York; D. A. Case. Ion counting from explicit-solvent simulations and 3d-RISM. *Biophys J*, **2014**, 106, 883–894.
- [292] J. W. Kaminski; S. Gusarov; T. A. Wesolowski; A. Kovalenko. Modeling solvatochromic shifts using the orbital-free embedding potential at statistically mechanically averaged solvent density. *J. Phys. Chem. A*, **2010**, 114, 6082–6096.

BIBLIOGRAPHY

- [293] M. Frigo; S. G. Johnson. FFTW: An adaptive software architecture for the FFT. in *Proc. 1998 IEEE Intl. Conf. Acoustics Speech and Signal Processing*, volume 3, pp 1381–1384. IEEE, 1998.
- [294] M. Frigo. A fast Fourier transform compiler. in *Proc. 1999 ACM SIGPLAN Conf. on Programming Language Design and Implementation*, volume 34, pp 169–180. ACM, 1999.
- [295] S. J. Singer; D. Chandler. Free energy functions in the extended RISM approximation. *Mol. Phys.*, **1985**, 55, 621–625.
- [296] B. M. Pettitt; P. J. Rossky. Alkali halides in water: Ion-solvent correlations and ion-ion potentials of mean force at infinite dilution. *J. Chem. Phys.*, **1986**, 15, 5836–5844.
- [297] S. M. Kast. Free energies from integral equation theories: Enforcing path independence. *Phys. Rev. E*, **2003**, 67, 041203.
- [298] G. Schmeer; A. Maurer. Development of thermodynamic properties of electrolyte solutions with the help of RISM-calculations at the Born-Oppenheimer level. *Phys. Chem. Chem. Phys.*, **2010**, 12, 2407–2417.
- [299] S. Gusarov; T. Ziegler; A. Kovalenko. Self-consistent combination of the three-dimensional RISM theory of molecular solvation with analytical gradients and the amsterdam density functional package. *J. Phys. Chem. A*, **2006**, 110, 6083–6090.
- [300] T. Miyata; F. Hirata. Combination of molecular dynamics method and 3D-RISM theory for conformational sampling of large flexible molecules in solution. *J. Comput. Chem.*, **2007**, 29, 871–882.
- [301] S. M. Kast; T. Kloss. Closed-form expressions of the chemical potential for integral equation closures with certain bridge functions. *J. Chem. Phys.*, **2008**, 129, 236101.
- [302] D. Chandler; Y. Singh; D. M. Richardson. Excess electrons in simple fluids. I. General equilibrium theory for classical hard sphere solvents. *J. Chem. Phys.*, **1984**, 81, 1975–1982.
- [303] T. Ichiye; D. Chandler. Hypernetted chain closure reference interaction site method theory of structure and thermodynamics for alkanes in water. *J. Phys. Chem.*, **1988**, 92, 5257–5261.
- [304] P. H. Lee; G. M. Maggiora. Solvation thermodynamics of polar molecules in aqueous solution by the XRISM method. *J. Phys. Chem.*, **1993**, 97, 10175–10185.
- [305] S. Genheden; T. Luchko; S. Gusarov; A. Kovalenko; U. Ryde. An MM/3D-RISM approach for ligand-binding affinities. *J. Phys. Chem.*, **2010**. Accepted.
- [306] H.-A. Yu; B. Roux; M. Karplus. Solvation thermodynamics: An approach from analytic temperature derivatives. *J. Chem. Phys.*, **1990**, 92, 5020–5033.
- [307] J. Johnson; D. A. Case; T. Yamazaki; S. Gusarov; A. Kovalenko; T. Luchko. Small molecule solvation energy and entropy from 3D-RISM. *J. Phys. Condens. Mat.*, **2016**.
- [308] T. Yamazaki; N. Blinov; D. Wishart; A. Kovalenko. Hydration effects on the HET-s prion and amyloid- β 2 fibrilous aggregates, studied with three-dimensional molecular theory of solvation. *Biophys. J.*, **2008**, 95, 4540–4548.
- [309] T. Yamazaki; A. Kovalenko; V. V. Murashov; G. N. Patey. Ion solvation in a water-urea mixture. *J. Phys. Chem. B*, **2010**, 114, 613–619.
- [310] H. A. Boateng; R. Krasny. Comparison of treecodes for computing electrostatic potentials in charged particle systems with disjoint targets and sources. *J. Computat. Chem.*, **2013**, 34, 2159–2167.
- [311] Z.-H. Duan; R. Krasny. An adaptive treecode for computing nonbonded potential energy in classical molecular systems. *J. Computat. Chem.*, **2001**, 22, 184–195.
- [312] P. Li; H. Johnston; R. Krasny. A Cartesian treecode for screened Coulomb interactions. *J. Computat. Phys.*, **2009**, 228, 3858–3868.

- [313] C. Nguyen; T. Yamazaki; A. Kovalenko; D. A. Case; M. K. Gilson; T. Kurtzman; T. Luchko. A molecular reconstruction approach to site-based 3D-RISM and comparison to GIST hydration thermodynamic maps in an enzyme active site. *PLoS One*, **2019**, *14*, e0219743.
- [314] D. S. Palmer; A. I. Frolov; E. L. Ratkova; M. V. Fedorov. Towards a universal method for calculating hydration free energies: a 3D reference interaction site model with partial molar volume correction. *J. Phys.: Condens. Matter*, **2010**, *22*, 492101.
- [315] J.-F. Truchon; B. M. Pettitt; P. Labute. A cavity corrected 3D-RISM functional for accurate solvation free energies. *J. Chem. Theory Comput.*, **2014**, *10*, 934–941.
- [316] V. Sergiievskiy; G. Jeanmairet; M. Levesque; D. Borgis. Solvation free-energy pressure corrections in the three dimensional reference interaction site model. *J. Chem. Phys.*, **2015**, *143*, 184116.
- [317] T. Luchko; N. Blinov; G. C. Linon; K. P. Joyce; A. Kovalenko. SAMPL5: 3D-RISM partition coefficient calculations with partial molar volume corrections and solute conformational sampling. *J. Comput. Aided Mol. Design*, **2016**, *30*, 1115–1127.
- [318] I. Omelyan; A. Kovalenko. MTS-MD of biomolecules steered with 3D-RISM-KH mean solvation forces accelerated with generalized solvation force extrapolation. *J. Chem. Theory Comput.*, **2015**, *11*, 1875–1895.
- [319] M. E. Tuckerman; B. J. Berne; G. J. Martyna. Molecular dynamics algorithm for multiple time scales: Systems with long range forces. *J. Chem. Phys.*, **1991**, *94*, 6811–6815.
- [320] M. Tuckerman; B. J. Berne; G. J. Martyna. Reversible multiple time scale molecular dynamics. *J. Chem. Phys.*, **1992**, *97*, 1990–2001.
- [321] H. Grubmüller; H. Heller; A. Windemuth; K. Schulten. Generalized Verlet algorithm for efficient molecular dynamics simulations with long-range interactions. *Mol. Simulat.*, **1991**, *6*, 121–142.
- [322] T. Schlick. *Molecular modeling and simulation: an interdisciplinary guide*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2002.
- [323] I. Omelyan; A. Kovalenko. Multiple time step molecular dynamics in the optimized isokinetic ensemble steered with the molecular theory of solvation: Accelerating with advanced extrapolation of effective solvation forces. *J. Chem. Phys.*, **2013**, *139*, 244106.
- [324] T. Morita; K. Hiroike. A New Approach to the Theory of Classical Fluids. I. *Progress of Theoretical Physics*, **1960**, *23*, 1003–1027.
- [325] M. Misin; M. V. Fedorov; D. S. Palmer. Communication: Accurate hydration free energies at a wide range of temperatures from 3D-RISM. *The Journal of Chemical Physics*, **2015**, *142*, 091105.
- [326] I. Omelyan; A. Kovalenko. MTS-MD of Biomolecules Steered with 3D-RISM-KH Mean Solvation Forces Accelerated with Generalized Solvation Force Extrapolation. *J. Chem. Theor. and Comp.*, **2014**, *11*, 1875–1895.
- [327] R. C. Walker; M. F. Crowley; D. A. Case. The implementation of a fast and accurate QM/MM potential method in Amber. *J. Comput. Chem.*, **2008**, *29*, 1019–1031.
- [328] G. M. Seabra; R. C. Walker; M. Elstner; D. A. Case; A. E. Roitberg. Implementation of the SCC-DFTB Method for Hybrid QM/MM Simulations within the Amber Molecular Dynamics Package. *J. Phys. Chem. A.*, **2007**, *20*, 5655–5664.
- [329] M. Elstner; D. Porezag; G. Jungnickel; J. Elsner; M. Haugk; T. Frauenheim; S. Suhai; G. Seifert. Self-consistent charge density functional tight-binding method for simulation of complex material properties. *Phys. Rev. B*, **1998**, *58*, 7260.
- [330] T. Kruger; M. Elstner; P. Schiffels; T. Frauenheim. Validation of the density-functional based tight-binding approximation. *J. Chem. Phys.*, **2005**, *122*, 114110.

BIBLIOGRAPHY

- [331] T. J. Giese; D. M. York. Charge-dependent model for many-body polarization, exchange, and dispersion interactions in hybrid quantum mechanical/molecular mechanical calculations. *J. Chem. Phys.*, **2007**, *127*, 194101–194111.
- [332] J. J. P. Stewart. Optimization of parameters for semiempirical methods I. Method. *J. Comput. Chem.*, **1989**, *10*, 209–220.
- [333] M. J. S. Dewar; E. G. Zoebisch; E. F. Healy; J. J. P. Stewart. AM1: A new general purpose quantum mechanical molecular model. *J. Am. Chem. Soc.*, **1985**, *107*, 3902–3909.
- [334] G. B. Rocha; R. O. Freire; A. M. Simas; J. J. P. Stewart. RM1: A Reparameterization of AM1 for H, C, N, O, P, S, F, Cl, Br and I. *J. Comp. Chem.*, **2006**, *27*, 1101–1111.
- [335] M. J. S. Dewar; W. Thiel. Ground states of molecules. 38. The MNDO method, approximations and parameters. *J. Am. Chem. Soc.*, **1977**, *99*, 4899–4907.
- [336] M. P. Repasky; J. Chandrasekhar; W. L. Jorgensen. PDDG/PM3 and PDDG/MNDO: Improved semiempirical methods. *J. Comput. Chem.*, **2002**, *23*, 1601–1622.
- [337] J. P. McNamara; A. M. Muslim; H. Abdel-Aal; H. Wang; M. Mohr; I. H. Hillier; R. A. Bryce. Towards a quantum mechanical force field for carbohydrates: A reparameterized semiempirical MO approach. *Chem. Phys. Lett.*, **2004**, *394*, 429–436.
- [338] M. I. Bernal-Uruchurtu; M. F. Ruiz-López. Basic ideas for the correction of semiempirical methods describing H-bonded systems. *Chem. Phys. Lett.*, **2000**, *330*, 118–124.
- [339] O. I. Arillo-Flores; M. F. Ruiz-López; M. I. Bernal-Uruchurtu. Can semi-empirical models describe HCl dissociation in water? *Theoret. Chem. Acc.*, **2007**, *118*, 425–435.
- [340] W. Thiel; A. A. Voityuk. Extension of the MNDO formalism to d orbitals: Integral approximations and preliminary numerical results. *Theoret. Chim. Acta*, **1992**, *81*, 391–404.
- [341] W. Thiel; A. A. Voityuk. Extension of the MNDO formalism to d orbitals: Integral approximations and preliminary numerical results. *Theoret. Chim. Acta*, **1996**, *93*, 315.
- [342] W. Thiel; A. A. Voityuk. Erratum: Extension of MNDO to d orbitals: Parameters and results for the second-row elements and for the zinc group. *J. Phys. Chem.*, **1996**, *100*, 616–626.
- [343] P. Imhof; F. Noé; S. Fischer; J. C. Smith. AM1/d Parameters for Magnesium in Metalloenzymes. *J. Chem. Theory Comput.*, **2006**, *2*, 1050–1056.
- [344] K. Nam; Q. Cui; J. Gao; D. M. York. Specific Reaction Parametrization of the AM1/d Hamiltonian for Phosphoryl Transfer Reactions: H, O, and P Atoms. *J. Chem. Theory Comput.*, **2007**, *3*, 486–504.
- [345] J. J. P. Stewart. Optimization of parameters for semiempirical methods V: Modification of NDDO approximations and application to 70 elements. *J. Mol. Mod.*, **2007**, *13*, 1173–1213.
- [346] D. Porezag; T. Frauenheim; T. Kohler; G. Seifert; R. Kaschner. Construction of tight-binding-like potentials on the basis of density-functional-theory: Applications to carbon. *Phys. Rev. B*, **1995**, *51*, 12947.
- [347] G. Seifert; D. Porezag; T. Frauenheim. Calculations of molecules, clusters and solids with a simplified LCAO-DFT-LDA scheme. *Int. J. Quantum Chem.*, **1996**, *58*, 185.
- [348] M. Gaus; Q. Cui; M. Elstner. DFTB3: Extension of the self-consistent-charge density-functional tight-binding method (SCC-DFTB). *J. Chem. Theory Comput.*, **2011**, *7*, 931–948.
- [349] M. Elstner; P. Hobza; T. Frauenheim; S. Suhai; E. Kaxiras. Hydrogen bonding and stacking interactions of nucleic acid base pairs: a density-functional-theory based treatment. *J. Chem. Phys.*, **2001**, *114*, 5149.

- [350] J. A. Kalinowski; B. Lesyng; J. D. Thompson; C. J. Cramer; D. G. Truhlar. Class IV charge model for the self-consistent charge density-functional tight-binding method. *J. Phys. Chem. A*, **2004**, *108*, 2545–2549.
- [351] Y. Yang; H. Yu; D. M. York; Q. Cui; M. Elstner. Extension of the self-consistent charge density-functional tight-binding method: Third-order expansion of the density functional theory total energy and introduction of a modified effective Coulomb interaction. *J. Phys. Chem. A*, **2007**, *111*, 10861–10873.
- [352] M. Korth. Third-generation hydrogen-bonding corrections for semiempirical qm methods and force fields. *J. Chem. Theory Comput.*, **2010**, *6*, 3808.
- [353] P. Jurecka; J. Cerný; P. Hobza; D. R. Salahub. Density functional theory augmented with an empirical dispersion term. Interaction energies and geometries of 80 noncovalent complexes compared with ab initio quantum mechanics calculations. *J. Comp. Chem.*, **2007**, *28*, 555–569.
- [354] A. Bondi. van der Waals volumes and radii. *J. Phys. Chem.*, **1964**, *68*, 441–451.
- [355] M. Korth; M. Pitonak; J. Rezac; P. Hobza. A transferable h-bonding correction for semiempirical quantum-chemical methods. *J. Chem. Theory Comput.*, **2010**, *6*, 344–352.
- [356] M. Manathunga; K. A. O’Hearn; A. Shajan; J. Smith; Y. Miao; X. He; K. Ayers; E. Brothers; A. W. Goetz; K. M. Merz. QUICK-24.03. University of California San Diego, CA and Michigan State University, East Lansing, MI. 2024.
- [357] Y. Miao; K. M. Merz. Acceleration of electron repulsion integral evaluation on graphics processing units via use of recurrence relations. *J. Chem. Theory Comput.*, **2013**, *9*, 965–976.
- [358] Y. Miao; K. M. Merz. Acceleration of High Angular Momentum Electron Repulsion Integrals and Integral Derivatives on Graphics Processing Units. *J. Chem. Theory Comput.*, **2015**, *11*, 1449–1462.
- [359] M. Manathunga; Y. Miao; D. Mu; A. W. Götz; K. M. Merz. Parallel Implementation of Density Functional Theory Methods in the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.*, **2020**, *16*, 4315–4326.
- [360] M. Manathunga; C. Jin; V. W. D. Cruzeiro; Y. Miao; D. Mu; K. Arumugam; K. Keipert; H. M. Aktulga; J. Merz, Kenneth M.; A. W. Götz. Harnessing the Power of Multi-GPU Acceleration into the Quantum Interaction Computational Kernel Program. *J. Chem. Theory Comput.*, **2021**, *17*, 3955–3966.
- [361] M. Manathunga; H. M. Aktulga; A. W. Götz; J. Merz, Kenneth M. Quantum Mechanics/Molecular Mechanics Simulations on NVIDIA and AMD Graphics Processing Units. *J. Chem. Inf. Model.*, **2023**, *61*, 2109–2115.
- [362] A. Shajan; M. Manathunga; A. W. Götz; J. Merz, Kenneth M. Geometry optimization: A comparison of different open-source geometry optimizers. *J. Chem. Theory Comput.*, **2023**, *19*, 7533–7541.
- [363] E. Pellegrini; M. J. Field. A generalized-Born solvation model for macromolecular hybrid-potential calculations. *J. Phys. Chem. A*, **2002**, *106*, 1316–1326.
- [364] K. Nam; J. Gao; D. York. An efficient linear-scaling Ewald method for long-range electrostatic interactions in combined QM/MM calculations. *J. Chem. Theory Comput.*, **2005**, *1*, 2–13.
- [365] A. W. Götz; M. A. Clark; R. C. Walker. An extensible interface for QM/MM molecular dynamics simulations with AMBER. *J. Comput. Chem.*, **2014**, *35*, 95–108.
- [366] V. W. D. Cruzeiro; M. Manathunga; J. Merz, Kenneth M.; A. W. Götz. Open-Source Multi-GPU-Accelerated QM/MM Simulations with AMBER and QUICK. *J. Chem. Inf. Model.*, **2021**, *61*, 2109–2115.
- [367] Q. T. Wang; R. A. Bryce. Improved hydrogen bonding at the NDDO-type semiempirical quantum mechanical/molecular mechanical interface. *J. Chem. Theory Comput.*, **2009**, *5*, 2206–2211.
- [368] G. te Velde; F. M. Bickelhaupt; E. J. Baerends; C. F. Guerra; S. J. A. van Gisbergen; J. G. Snijders; T. Ziegler. Chemistry with ADF. *J. Comp. Chem.*, **2001**, *22*, 931–967.

BIBLIOGRAPHY

- [369] ADF2011, SCM, Theoretical Chemistry, Vrije Universiteit, Amsterdam, The Netherlands, <http://www.scm.com>, 2012.
- [370] M. W. Schmidt; K. K. Baldridge; J. A. Boatz; S. T. Elbert; M. S. Gordon; J. H. Jensen; S. Koseki; N. Matsunaga; K. A. Nguyen; S. Su; T. L. Windus; M. Dupuis; J. A. Montgomery, Jr. General atomic and molecular electronic structure system. *J. Comp. Chem.*, **1993**, *14*, 1347–1363.
- [371] M. S. Gordon; M. W. Schmidt. in *Theory and Applications of Computational Chemistry, the first forty years*, C. E. Dykstra; G. Frenking; K. S. Kim; G. E. Scuseria, Eds., chapter 41, pp 1167–1189. Elsevier, Amsterdam, 2005.
- [372] M. Valiev; E. J. Bylaska; N. Govind; K. Kowalski; T. P. Straatsma; H. J. J. van Dam; D. Wang; J. Niepolcha; E. Apra; T. L. Windus; W. A. de Jong. Nwchem: a comprehensive and scalable open-source solution for large scale molecular simulations. *Comput. Phys. Commun.*, **2010**, *181*, 1477.
- [373] M. J. Frisch; G. W. Trucks; H. B. Schlegel; G. E. Scuseria; M. A. Robb; J. R. Cheeseman; G. Scalmani; V. Barone; B. Mennucci; G. A. Petersson; H. Nakatsuji; M. Caricato; X. Li; H. P. Hratchian; A. F. Izmaylov; J. Bloino; G. Zheng; J. L. Sonnenberg; M. Hada; M. Ehara; K. Toyota; R. Fukuda; J. Hasegawa; M. Ishida; T. Nakajima; Y. Honda; O. Kitao; H. Nakai; T. Vreven; J. A. Montgomery, Jr.; J. E. Peralta; F. Ogliaro; M. Bearpark; J. J. Heyd; E. Brothers; K. N. Kudin; V. N. Staroverov; R. Kobayashi; J. Normand; K. Raghavachari; A. Rendell; J. C. Burant; S. S. Iyengar; J. Tomasi; M. Cossi; N. Rega; J. M. Millam; M. Klene; J. E. Knox; J. B. Cross; V. Bakken; C. Adamo; J. Jaramillo; R. Gomperts; R. E. Stratmann; O. Yazyev; A. J. Austin; R. Cammi; C. Pomelli; J. W. Ochterski; R. L. Martin; K. Morokuma; V. G. Zakrzewski; G. A. Voth; P. Salvador; J. J. Dannenberg; S. Dapprich; A. D. Daniels; O. Farkas; J. B. Foresman; J. V. Ortiz; J. Cioslowski; D. J. Fox. Gaussian 09 Revision A.1. Gaussian Inc. Wallingford CT 2009.
- [374] F. Neese. ORCA - an ab initio, Density Functional and Semiempirical program package, Version 2.8.0, University of Bonn, 2010.
- [375] Y. Shao; L. Fusti-Molnar; Y. Jung; J. Kussmann; C. Ochsenfeld; S. T. Brown; A. T. B. Gilbert; L. V. Slipchenko; S. V. Levchenko; D. P. O'Neill; R. A. DiStasio, Jr.; R. C. Lochan; T. Wang; G. J. O. Beran; N. A. Besley; J. M. Herbert; C. Y. Lin; T. V. Voorhis; S. H. Chien; A. Sodt; R. P. Steele; V. A. Rassolov; P. E. Maslen; P. P. Korambath; R. D. Adamson; B. Austin; J. Baker; E. F. C. Byrd; H. Daschel; R. J. Doerksen; A. Dreuw; B. D. Dunietz; A. D. Dutoi; T. R. Furlani; S. R. Gwaltney; A. Heyden; S. Hirata; C.-P. Hsu; G. Kedziora; R. Z. Khaliullin; P. Klunzinger; A. M. Lee; M. S. Lee; W. Liang; I. Lotan; N. Nair; B. Peters; E. I. Proynov; P. A. Pieniazek; Y. M. Rhee; J. Ritchie; E. Rosta; C. D. Sherrill; A. C. Simmonett; J. E. Subotnik; H. L. Woodcock, III; W. Zhang; A. T. Bell; A. K. Chakraborty; D. M. Chipman; F. J. Keil; A. Warshel; W. J. Hehre; H. F. Schaefer, III; J. Kong; A. I. Krylov; P. M. W. Gill; M. Head-Gordon. Advances in methods and algorithms in a modern quantum chemistry program package. *Phys. Chem. Chem. Phys.*, **2006**, *8*, 3172–3191.
- [376] I. S. Ufimtsev; T. J. Martinez. Quantum chemistry on graphical processing units. 3. Analytical energy gradients, geometry optimization, and first principles molecular dynamics. *J. Chem. Theory Comput.*, **2009**, *5*, 2619–2628.
- [377] M. Kállay; Z. Rolik; J. Csontos; I. Ladjánszki; L. Szegedy; B. Ladoćzki; G. Samu; K. Petrov; M. Farkas; P. Nagy; D. Mester; B. Hegely. Mrcc, a quantum chemical program suite. www.mrcc.hu.
- [378] Z. Rolik; L. Szegedy; I. Ladjánszki; B. Ladoćzki; M. Kállay. An efficient linear-scaling CCSD(T) method based on local natural orbitals. *J. Chem. Phys.*, **2013**, *139*, 094105.
- [379] J. P. Lewis; P. Jelínek; J. Ortega; A. A. Demkov; D. G. Trabada; B. Haycock; H. Wang; G. Adams; J. K. Tomfohr; E. Abad; H. Wang; D. A. Drabold. Advances and applications in the FIREBALL ab initio tight-binding molecular-dynamics formalism. *Phys. Status Solidi B*, **2011**, *248*, 1989–2007.
- [380] J. Torras; Y. He; C. Cao; K. Muralidharan; E. Deumens; H. Cheng; S. Trickey. PUPIL: A systematic approach to software integration in multi-scale simulations. *Comput. Phys. Comm.*, **2007**, *177*, 265–279.
- [381] J. Torras; G. Seabra; E. Deumens; S. B. Trickey; A. E. Roitberg. A versatile AMBER-Gaussian QM/MM interface through PUPIL. *J. Comput. Chem.*, **2008**, *29*, 1564–1573.

- [382] B. Hégely; P. R. Nagy; G. G. Ferenczy; M. Kállay. Exact density functional and wave function embedding schemes based on orbital localization. *J. Chem. Phys.*, **2016**, *145*, 064107.
- [383] C. Bannwarth; E. Caldeweyher; S. Ehlert; A. Hansen; P. Pracht; J. Seibert; S. Spicher; S. Grimme. Extended tight-binding quantum chemistry methods. *WIREs Comput. Mol. Sci.*, **2021**, *11*, e01493.
- [384] S. Grimme; C. Bannwarth; P. Shushkov. A Robust and Accurate Tight-Binding Quantum Chemical Method for Structures, Vibrational Frequencies, and Noncovalent Interactions of Large Molecular Systems Parametrized for All spd-Block Elements ($Z = 1-86$). *J. Chem. Theory Comput.*, **2017**, *13*, 1989–2009.
- [385] C. Bannwarth; S. Ehlert; S. Grimme. GFN2-xTB – An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method with Multipole Electrostatics and Density-Dependent Dispersion Contributions. *J. Chem. Theory Comput.*, **2019**, *15*, 1652–1671.
- [386] B. Hourahine; B. Aradi; V. Blum; F. Bonafé; A. Buccheri; C. Camacho; C. Cevallos; M. Y. Deshayé; T. Dumitrică; A. Dominguez; S. Ehlert; M. Elstner; T. van der Heide; J. Hermann; S. Irle; J. J. Kranz; C. Köhler; T. Kowalczyk; T. Kubař; I. S. Lee; V. Lutsker; R. J. Maurer; S. K. Min; I. Mitchell; C. Negre; T. A. Niehaus; A. M. N. Niklasson; A. J. Page; A. Pecchia; G. Penazzi; M. P. Persson; J. Řezáč; C. G. Sánchez; M. Sternberg; M. Stöhr; F. Stuckenberg; A. Tkatchenko; V. W. Yu; T. Frauenheim. DFTB+, a software package for efficient approximate density functional theory based atomistic simulations. *J. Chem. Phys.*, **2020**, *152*, 124101.
- [387] S. Grimme; S. Ehrlich; L. Goerigk. Effect of the damping function in dispersion corrected density functional theory. *J. Comput. Chem.*, **2011**, *32*, 1456–1465.
- [388] S. Grimme; J. Antony; S. Ehrlich; H. Krieg. A consistent and accurate ab initio parametrization of density functional dispersion correction (DFT-D) for the 94 elements H-Pu. *J. Chem. Phys.*, **2010**, *132*, 154104–154123.
- [389] J. Řezáč. Empirical Self-Consistent Correction for the Description of Hydrogen Bonds in DFTB3. *J. Chem. Theory Comput.*, **2017**, *13*, 4804–4817.
- [390] E. Caldeweyher; S. Ehlert; A. Hansen; H. Neugebauer; S. Spicher; C. Bannwarth; S. Grimme. A generally applicable atomic-charge dependent London dispersion correction. *J. Chem. Phys.*, **2019**, *150*, 154122.
- [391] J. Zeng; D. Zhang; D. Lu; P. Mo; Z. Li; Y. Chen; M. Rynik; L. Huang; Z. Li; S. Shi; Y. Wang;; H. Ye; P. Tuo; J. Yang; Y. Ding; Y. Li; D. Tisi; Q. Zeng; H. Bao; Y. Xia; J. Huang; K. Muraoka; Y. Wang; J. Chang; F. Yuan; S. L. Bore; C. Cai; Y. Lin; B. Wang; J. Xu; J.-X. Zhu; C. Luo; Y. Zhang; R. E. A. Goodall; W. Liang; A. K. Singh; S. Yao; J. Zhang; R. Wentzcovitch; J. Han; J. Liu; W. Jia; D. M. York; W. E. R. Car; L. Zhang; H. Wang. Deepmd-kit v2: A software package for deep potential models. *J. Chem. Phys.*, **2023**, *159*, 054801.
- [392] J. Zeng; T. J. Giese; Solen Ekesan; D. M. York. Development of Range-Corrected Deep Learning Potentials for Fast, Accurate Quantum Mechanical/Molecular Mechanical Simulations of Chemical Reactions in Solution. *J. Chem. Theory Comput.*, **2021**, *17*, 6993–7009.
- [393] T. J. Giese; J. Zeng; Solen Ekesan; D. M. York. Combined QM/MM, Machine Learning Path Integral Approach to Compute Free Energy Profiles and Kinetic Isotope Effects in RNA Cleavage Reactions. *J. Chem. Theory Comput.*, **2022**, *18*, 4304–4317.
- [394] R. E. Buló; C. Michel; P. Fleurat-Lessard; P. Sautet. Multiscale modeling of chemistry in water: Are we there yet? *J. Chem. Theory Comput.*, **2013**, *9*, 5567–5577.
- [395] R. E. Buló; B. Ensing; J. Sikkema; L. Visscher. Toward a practical method for adaptive qm/mm simulations. *J. Chem. Theory Comput.*, **2009**, *9*, 2212–2221.
- [396] K. Park; A. W. Götz; R. C. Walker; F. Paesani. Application of adaptive qm/mm methods to molecular dynamics simulations of aqueous systems. *J. Chem. Theory Comput.*, **2012**, *8*, 2868–2877.
- [397] N. Bernstein; C. Várnai; I. Solt; S. A. Winfield; M. C. Payne; I. Simon; M. Fuxreiter; G. Csányi. *Phys. Chem. Chem. Phys.*, **2011**, *14*, 646–656.

BIBLIOGRAPHY

- [398] C. Várnai; N. Bernstein; L. Mones; G. Csányi. Tests of an adaptive qm/mm calculation on free energy profiles of chemical reactions in solution. *J. Phys. Chem. B*, **2013**, *117*, 12202–12211.
- [399] G. Csányi; T. Albaret; G. Moras; M. C. Payne; A. D. Vita. Multiscale hybrid simulation methods for material systems. *J. Phys. Condens. Matt.*, **2005**, *17*, R691.
- [400] N. Bernstein; J. R. Kermode; G. Csányi. Hybrid atomistic simulation methods for materials systems. *Rep. Prog. Phys.*, **2009**, *72*, 026501.
- [401] A. Jones; B. Leimkuhler. Adaptive stochastic methods for sampling driven molecular systems. *J. Chem. Phys.*, **2011**, *135*, 084125.
- [402] T. Kerdcharoen; B. M. Rode. A QM/MM simulation method applied to the solution of Li⁺ in liquid ammonia. *Chem. Phys.*, **1996**, *211*, 313–323.
- [403] S. L. Dixon; K. M. Merz, Jr. Semiempirical molecular orbital calculations with linear system size scaling. *J. Chem. Phys.*, **1996**, *104*, 6643–6649.
- [404] S. L. Dixon; K. M. Merz, Jr. Fast, accurate semiempirical molecular orbital calculations for macromolecules. *J. Chem. Phys.*, **1997**, *107*, 879–893.
- [405] A. Marion; H. Gockan; G. Monard. SemiEmpirical Born-Oppenheimer Molecular Dynamics (SEBOMD) Within the Amber Biomolecular Package. *J. Chem. Inf. Model.*, **2019**, *59*, 206–214.
- [406] G. Monard; M. I. Bernal-Uruchurtu; A. Van Der Vaart; K. M. Merz, Jr.; M. F. Ruiz-López. Simulation of liquid water using semiempirical Hamiltonians and the divide and conquer approach. *J. Phys. Chem. A*, **2005**, *109*, 3425–3432.
- [407] A. Marion; G. Monard; M. F. Ruiz-López; F. Ingrosso. Water interactions with hydrophobic groups: assessment and recalibration of semiempirical molecular orbital methods. *J. Chem. Phys.*, **2014**, *141*, 034106.
- [408] M. I. Bernal-Uruchurtu; M. T. C. Martins-costa; C. Millot; M. F. Ruiz-López. Improving Description of Hydrogen Bonds at the Semiempirical Level : Water - Water Interactions as Test Case. *J. Comput. Chem.*, **2000**, *21*, 572–581.
- [409] W. Harb; M. I. Bernal-Uruchurtu; M. F. Ruiz-López. An improved semiempirical method for hydrated systems. *Theor. Chem. Acc.*, **2004**, *112*, 204–216.
- [410] E. Thiriote; G. Monard. Combining a genetic algorithm with a linear scaling semiempirical method for protein-ligand docking. *J. Mol. Struct. Theochem*, **2009**, *898*, 31–41.
- [411] O. Ludwig; H. Schinke; W. Brandt. Reparametrisation of Force Constants in MOPAC 6.0/7.0 for Better Description of the Activation Barrier of Peptide Bond Rotations. *J. Molec. Model.*, **1996**, *2*, 341–350.
- [412] H. M. Aktulga; J. C. Fogarty; S. A. Pandit; A. Y. Grama. Parallel reactive molecular dynamics: Numerical methods and algorithmic techniques. *Parallel Computing*, **2012**, *38*, 245–259.
- [413] S. B. Kylasa; H. M. Aktulga; A. Y. Grama. Puremd-gpu: A reactive molecular dynamics simulation package for gpus. *J. Comput. Phys.*, **2014**, *272*, 343–359.
- [414] A. C. Van Duin; S. Dasgupta; F. Lorant; W. A. Goddard. Reaxff: a reactive force field for hydrocarbons. *J. Phys. Chem. A*, **2001**, *105*, 9396–9409.
- [415] A. K. Rappe; W. A. Goddard III. Charge equilibration for molecular dynamics simulations. *J. Phys. Chem.*, **1991**, *95*, 3358–3363.
- [416] W. J. Mortier; S. K. Ghosh; S. Shankar. Electronegativity-equalization method for the calculation of atomic charges in molecules. *J. Am. Chem. Soc.*, **1986**, *108*, 4315–4320.
- [417] M. Samieegohar; F. Sha; A. Z. Clayborne; T. Wei. Reaxff md simulations of peptide-grafted gold nanoparticles. *Langmuir*, **2019**, *35*, 5029–5036.

- [418] S. Yang; T. Zhao; L. Zou; X. Wang; Y. Zhang. Reaxff-based molecular dynamics simulation of dna molecules destruction in cancer cells by plasma ros. *Phys. Plasmas*, **2019**, 26, 083504.
- [419] P. O. Hubin; D. Jacquemin; L. Leherste; D. P. Vercauteren. Parameterization of the reaxff reactive force field for a proline-catalyzed aldol reaction. *J. Comput. Chem.*, **2016**, 37, 2564–2572.
- [420] S. Monti; J. Jose; A. Sahajan; N. Kalarikkal; S. Thomas. Structure and dynamics of gold nanoparticles decorated with chitosan–gentamicin conjugates: Reaxff molecular dynamics simulations to disclose drug delivery. *Phys. Chem. Chem. Phys.*, **2019**, 21, 13099–13108.
- [421] T. Trnka; I. Tvaroska; J. Koca. Automated training of reaxff reactive force fields for energetics of enzymatic reactions. *J. Chem. Theory Comput.*, **2018**, 14, 291–302.
- [422] A. Rahnamoun; A. Van Duin. Reactive molecular dynamics simulation on the disintegration of kapton, polyimide, amorphous silica, and teflon during atomic oxygen impact using the reaxff reactive force-field method. *J. Phys. Chem. A*, **2014**, 118, 2780–2787.
- [423] D. Raymand; A. C. van Duin; M. Baudin; K. Hermansson. A reactive force field (reaxff) for zinc oxide. *Surf. Sci.*, **2008**, 602, 1020–1031.
- [424] K. Chenoweth; S. Cheung; A. C. Van Duin; W. A. Goddard; E. M. Kober. Simulations on the thermal decomposition of a poly (dimethylsiloxane) polymer using the reaxff reactive force field. *J. Am. Chem. Soc.*, **2005**, 127, 7192–7202.
- [425] M. F. Russo Jr; R. Li; M. Mench; A. C. Van Duin. Molecular dynamic simulation of aluminum–water reactions using the reaxff reactive force field. *Int. J. Hydrog. Ener.*, **2011**, 36, 5828–5835.
- [426] J. Ludwig; D. G. Vlachos; A. C. Van Duin; W. A. Goddard. Dynamics of the dissociation of hydrogen on stepped platinum surfaces using the reaxff reactive force field. *J. Phys. Chem. B*, **2006**, 110, 4274–4282.
- [427] K. Yoon; A. Rahnamoun; J. L. Swett; V. Iberi; D. A. Cullen; I. V. Vlassiuk; A. Belianinov; S. Jesse; X. Sang; O. S. Ovchinnikova et al. Atomistic-scale simulations of defect formation in graphene under noble gas ion irradiation. *ACS nano*, **2016**, 10, 8376–8384.
- [428] A. Rahnamoun; A. Van Duin. Study of thermal conductivity of ice clusters after impact deposition on the silica surfaces using the reaxff reactive force field. *Phys. Chem. Chem. Phys.*, **2016**, 18, 1587–1594.
- [429] C. Zou; A. Van Duin. Investigation of complex iron surface catalytic chemistry using the reaxff reactive force field method. *Jom*, **2012**, 64, 1426–1437.
- [430] Y. K. Shin; H. Kwak; A. V. Vasenkov; D. Sengupta; A. C. van Duin. Development of a ReaxFF reactive force field for Fe/Cr/O/S and application to oxidation of butane over a pyrite-covered Cr₂O₃ catalyst. *ACS Catalysis*, **2015**, 5, 7226–7236.
- [431] T. P. Senftle; S. Hong; M. M. Islam; S. B. Kylasa; Y. Zheng; Y. K. Shin; C. Junkermeier; R. Engel-Herbert; M. J. Janik; H. M. Aktulga et al. The reaxff reactive force-field: development, applications and future directions. *npj Computational Materials*, **2016**, 2, 1–14.
- [432] T. J. Dolinsky; P. Czodrowski; H. Li; J. E. Nielsen; J. H. Jensen; G. Klebe; N. A. Baker. Pdb2pqr: expanding and upgrading automated preparation of biomolecular structures for molecular simulations. *Nucleic acids research*, **2007**, 35, W522–W525.
- [433] M. H. Olsson; C. R. S ndergaard; M. Rostkowski; J. H. Jensen. Propka3: consistent treatment of internal and surface residues in empirical p k a predictions. *Journal of chemical theory and computation*, **2011**, 7, 525–537.
- [434] T. Nugent; D. T. Jones. Membrane protein orientation and refinement using a knowledge-based statistical potential. *BMC Bioinformatics*, **2013**, 14, 276.
- [435] B. Ho. pdbremix. <https://github.com/boscoh/pdbremix>, 2018.

BIBLIOGRAPHY

- [436] J. M. Martínez; L. Martínez. Packing optimization for automated generation of complex system's initial configurations for molecular dynamics and docking. *J. Computat. Chem.*, **2003**, 24, 819–825.
- [437] L. Martínez; R. Andrade; E. G. Birgin; J. M. Martínez. PACKMOL: A package for building initial configurations for molecular dynamics simulations. *J. Comput. Chem.*, **2009**, 30, 2157–2164.
- [438] S. Schott-Verdugo; H. Gohlke. PACKMOL-Memgen: A simple-to-use generalized workflow for membrane-protein/lipid-bilayer system building. *J. Chem. Inf. Model.*, **2019**, 59, 2522–2528.
- [439] A. Lomize; S. Todd; I. Pogozheva. Spatial arrangement of proteins in planar and curved membranes by PPM 3.0. *Protein Science*, **2022**, 31, 209–220.
- [440] J. Schmit; N. Kariyawasam; V. Needham; P. Smith. SLTCAP: A Simple Method for Calculating the Number of Ions Needed for MD Simulation. *J. Chem. Theory Comput.*, **2018**, 14, 1823–1827.
- [441] M. Machado; S. Pantano. Split the Charge Difference in Two! A Rule of Thumb for Adding Proper Amounts of Ions in MD Simulations. *J. Chem. Theory Comput.*, **2020**, 16, 1367–1372.
- [442] A. D. MacKerell Jr.; D. Bashford; M. Bellott; R. L. Dunbrack; J. D. Evanseck; M. J. Field; S. Fischer; J. Gao; H. Guo; S. Ha; D. Joseph-McCarthy; L. Kuchnir; K. Kuczera; F. T. K. Lau; C. Mattos; S. Michnick; T. Ngo; D. T. Nguyen; B. Prodhom; W. E. Reiher; B. Roux; M. Schlenkrich; J. C. Smith; R. Stote; J. Straub; M. Watanabe; J. Wierkiewicz-Kuczera; D. Yin; M. Karplus. All-Atom Empirical Potential for Molecular Modeling and Dynamics Studies of Proteins. *J. Phys. Chem. B*, **1998**, 102, 3586–3616.
- [443] A. D. MacKerell Jr.; N. Banavali; N. Foloppe. Development and current status of the CHARMM force field for nucleic acids. *Biopolymers*, **2000**, 56, 257–265.
- [444] A. D. MacKerell, Jr.; M. Feig; C. L. Brooks III. Improved Treatment of the Protein Backbone in Empirical Force Fields. *J. Am. Chem. Soc.*, **2004**, 126, 698–699.
- [445] A. D. MacKerell, Jr.; M. Feig; C. L. Brooks III. Extending the treatment of backbone energetics in protein force fields: Limitations of gas-phase quantum mechanics in reproducing protein conformational distributions in molecular dynamics simulations. *J. Computat. Chem.*, **2004**, 25, 1400–1415.
- [446] C. Bergonzo; N. M. Henriksen; D. R. Roe; J. M. Swails; A. E. Roitberg; T. E. Cheatham, III. Multidimensional Replica Exchange Molecular Dynamics Yields a Converged Ensemble of an RNA Tetranucleotide. *J. Chem. Theory Comput.*, **2013**, 10, 492–499.
- [447] J. Wang; R. M. Wolf; J. W. Caldwell; P. A. Kollman; D. A. Case. Development and testing of a general Amber force field. *J. Comput. Chem.*, **2004**, 25, 1157–1174.
- [448] J. Wang; W. Wang; P. A. Kollman; D. A. Case. Automatic atom type and bond type perception in molecular mechanical calculations. *J. Mol. Graphics Model.*, **2006**, 25, 247–260.
- [449] A. Jakalian; B. L. Bush; D. B. Jack; C. I. Bayly. Fast, efficient generation of high-quality atomic charges. AM1-BCC model: I. Method. *J. Comput. Chem.*, **2000**, 21, 132–146.
- [450] A. Jakalian; D. B. Jack; C. I. Bayly. Fast, efficient generation of high-quality atomic charges. AM1-BCC model: II. Parameterization and Validation. *J. Comput. Chem.*, **2002**, 23, 1623–1641.
- [451] X. He; V. Man; W. Wang; T. Lee; J. Wang. A fast and high-quality charge model for the next generation general AMBER force field. *J. Chem. Phys.*, **2020**, 153, 114502.
- [452] Y. Sun; X. He; T. Hou; L. Cai; V. Man; J. Wang. Development and test of highly accurate endpoint free energy methods. 1: Evaluation of ABCG2 charge model on solvation free energy prediction and optimization of atom radii suitable for more accurate solvation free energy prediction by the PBSA method. *J. Comput. Chem.*, **2023**, 44, 1334–1346.

- [453] X. He; V. H. Man; W. Yang; T. Lee; J. Wang. ABCG2: A Milestone Charge Model for Accurate Solvation Free Energy Calculation. *J. Chem. Theory Comput.*, **2025**, 21, 3032–3043.
- [454] C. I. Bayly; P. Cieplak; W. D. Cornell; P. A. Kollman. A well-Behaved electrostatic potential based method using charge restraints for determining atom-centered charges: The RESP model. *J. Phys. Chem.*, **1993**, 97, 10269–10280.
- [455] J. Wang; P. A. Kollman. Automatic parameterization of force field by systematic search and genetic algorithms. *J. Comput. Chem.*, **2001**, 22, 1219–1228.
- [456] A. P. Graves; D. M. Shivakumar; S. E. Boyce; M. P. Jacobson; D. A. Case; B. K. Shoichet. Rescoring docking hit lists for model cavity sites: Predictions and experimental testing. *J. Mol. Biol.*, **2008**, 377, 914–934.
- [457] B. Jojart; T. A. Martinek. Performance of the general amber force field in modeling aqueous POPC membrane bilayers. *J. Comput. Chem.*, **2007**, 28, 2051–2058.
- [458] J. Wang; T. Hou. Application of Molecular Dynamics Simulations in Molecular Property Prediction. 1. Density and Heat of Vaporization. *J. Chem. Theory Comput.*, **2011**, 7, 2151–2165.
- [459] X. He; B. Walker; W. Man; P. Ren; J. Wang. Recent progress in general force fields of small molecules. *Curr. Opin. Struc. Biol.*, **2022**, 72, 187–193.
- [460] P. Li; K. M. Merz, Jr. Metal Ion Modeling Using Classical Mechanics. *Chem. Rev.*, **2017**, 117, 1564–1686.
- [461] P. Li; K. M. Merz, Jr. MCPB.py: A Python Based Metal Center Parameter Builder. *J. Chem. Inf. Model.*, **2016**, 56, 599–604.
- [462] M. B. Peters; Y. Yang; B. Wang; L. Fusti-Molnar; M. N. Weaver; K. M. Merz, Jr. Structural Survey of Zinc-Containing Proteins and Development of the Zinc AMBER Force Field (ZAFF). *J. Chem. Theor. Comput.*, **2010**, 6, 2935–2947.
- [463] P. Li; K. M. Merz, Jr. in *Methods Mol. Biol.*, (Humana Press, New York, NY). volume 2199. pp 257–275. 2021.
- [464] P. Eastman; V. S. Pande. OpenMM: A Hardware-Independent Framework for Molecular Simulations. *Computing in Science and Engineering*, **2010**, 12, 34–39.
- [465] P. Eastman; M. S. Friedrichs; J. D. Chodera; R. J. Radmer; C. M. Bruns; J. P. Ku; K. A. Beauchamp; T. J. Lane; L. Wang; D. Shukla; T. Tye; M. Houston; T. Stich; C. Klein; M. R. Shirts; V. S. Pande. OpenMM 4: A Reusable, Extensible, Hardware Independent Library for High Performance Molecular Simulation. *J. Chem. Theory Comput.*, **2013**, 9, 461–469.
- [466] Z. Yu; P. Li; K. M. Merz, Jr. Extended Zinc AMBER Force Field (EZAFF). *J. Chem. Theory Comput.*, **2018**, 14, 242–254.
- [467] J. M. Seminario. Calculation of Intramolecular Force Fields from Second-Derivative Tensors. *Int. J. Quantum Chem.*, **1996**, 30, 1271–1277.
- [468] A. E. A. Allen; M. C. Payne; D. J. Cole. Harmonic Force Constants for Molecular Mechanics Force Fields via Hessian Matrix Projection. *J. Chem. Theory Comput.*, **2018**, 14, 274–281.
- [469] L. Landry; P. Li. Development of a Fluctuating Charge Model for Zinc-Containing Metalloproteins. *J. Chem. Inf. Model.*, **2024**, 64, 812–824.
- [470] L. Landry; P. Li. Parameterization of a Fluctuating Charge Model for Complexes Containing 3d Transition Metals. *J. Phys. Chem. B*, **2024**, 128, 10329–10338.
- [471] S. Zhao; H. Wei; P. Cieplak; Y. Duan; R. Luo. Pyresp: A program for electrostatic parameterizations of additive and induced dipole polarizable force fields. *J. Chem. Theory Comput.*, **2022**, 18, 3654–3670.

BIBLIOGRAPHY

- [472] W. D. Cornell; P. Cieplak; C. I. Bayly; P. A. Kollman. Application of RESP charges to calculate conformational energies, hydrogen bond energies and free energies of solvation. *J. Am. Chem. Soc.*, **1993**, *115*, 9620–9631.
- [473] J. Wang; P. Cieplak; J. Li; T. Hou; R. Luo; Y. Duan. Development of Polarizable Models for Molecular Mechanical Calculations I: Parameterization of Atomic Polarizability. *J. Phys. Chem. B*, **2011**, *115*, 3091–3099.
- [474] J. Wang; P. Cieplak; J. Li; J. Wang; Q. Cai; M. Hsieh; H. Lei; R. Luo; Y. Duan. Development of Polarizable Models for Molecular Mechanical Calculations II: Induced Dipole Models Significantly Improve Accuracy of Intermolecular Interaction Energies. *J. Phys. Chem. B*, **2011**, *115*, 3100–3111.
- [475] J. Wang; P. Cieplak; Q. Cai; M. Hsieh; J. Wang; Y. Duan; R. Luo. Development of Polarizable Models for Molecular Mechanical Calculations. 3. Polarizable Water Models Conforming to Thole Polarization Screening Schemes. *J. Phys. Chem. B*, **2012**, *116*, 7999–8008.
- [476] J. Wang; P. Cieplak; J. Li; Q. Cai; M. Hsieh; R. Luo; Y. Duan. Development of Polarizable Models for Molecular Mechanical Calculations. 4. van der Waals Parametrization. *J. Phys. Chem. B*, **2012**, *116*, 7088–7101.
- [477] H. Wei; R. Qi; J. Wang; P. Cieplak; Y. Duan; R. Luo. Efficient Formulation of polarizable Gaussian Multipole Electrostatics for Biomolecular Simulations. *J. Chem. Phys.*, **2020**, *153*, 114116.
- [478] H. Wei; P. Cieplak; Y. Duan; R. Luo. Stress tensor and constant pressure simulation for polarizable gaussian multipole model. *J. Chem. Phys.*, **2022**, *156*, 114114.
- [479] S. Zhao; H. Wei; P. Cieplak; Y. Duan; R. Luo. Accurate reproduction of quantum mechanical many-body interactions in peptide main-chain hydrogen-bonding oligomers by the polarizable gaussian multipole model. *J. Chem. Theory Comput.*, **2022**, *18*, 6172–6188.
- [480] S. Zhao; P. Cieplak; Y. Duan; R. Luo. Transferability of the electrostatic parameters of the polarizable gaussian multipole model. *J. Chem. Theory Comput.*, **2023**, *19*, 924–941.
- [481] J. Wang; P. Cieplak; R. Luo; Y. Duan. Development of polarizable gaussian model for molecular mechanical calculations i: Atomic polarizability parameterization to reproduce ab initio anisotropy. *Journal of chemical theory and computation*, **2019**, *15*, 1146–1158.
- [482] G. Landrum. Rdkit documentation. *Release*, **2013**, *1*, 4.
- [483] M. Lapsien; M. Bonus; L. Gahan; A. Raguin; H. Gohlke. *Pype_{resp} : A tool to facilitate and standardize derivation of resp charges*. *bioRxiv*, **2025**.
- [484] D. S. Cerutti; D. A. Case. Molecular dynamics simulations of macromolecular crystals. *Wires Comput. Mol. Sci.*, **2018**, *8*, e1402.
- [485] H. Kopitz; A. Zivkovic; J. W. Engels; H. Gohlke. Determinants of the unexpected stability of RNA fluorobenzene self pairs. *ChemBioChem*, **2008**, *9*, 2619–2622.
- [486] T. Morishita. Fluctuation formulas in molecular-dynamics simulations with the weak coupling heat bath. *J. Chem. Phys.*, **2000**, *113*, 2976.
- [487] A. Mudi; C. Chakravarty. Effect of the Berendsen thermostat on the dynamical properties of water. *Mol. Phys.*, **2004**, *102*, 681–685.
- [488] H. J. C. Berendsen; J. P. M. Postma; W. F. van Gunsteren; A. DiNola; J. R. Haak. Molecular dynamics with coupling to an external bath. *J. Chem. Phys.*, **1984**, *81*, 3684–3690.
- [489] S. C. Harvey; R. K. Tan; T. E. Cheatham, III. The flying ice cube: Velocity rescaling in molecular dynamics leads to violation of energy equipartition. *J. Comput. Chem.*, **1998**, *19*, 726–740.
- [490] T. A. Andrea; W. C. Swope; H. C. Andersen. The role of long ranged forces in determining the structure and properties of liquid water. *J. Chem. Phys.*, **1983**, *79*, 4576–4584.

- [491] H. C. Andersen. Molecular dynamics simulations at constant pressure and/or temperature. *J. Chem. Phys.*, **1980**, *72*, 2384–2393.
- [492] B. P. Uberuaga; M. Anghel; A. F. Voter. Synchronization of trajectories in canonical molecular-dynamics simulations: Observation, explanation, and exploitation. *J. Chem. Phys.*, **2004**, *120*, 6363–6374.
- [493] D. J. Sindhikara; S. Kim; A. F. Voter; A. E. Roitberg. Bad seeds sprout perilous dynamics: Stochastic thermostat induced trajectory synchronization in biomolecules. *J. Chem. Theory Comput.*, **2009**, *5*, 1624–1631.
- [494] I. Omelyan; A. Kovalenko. Generalized canonical-isokinetic ensemble: Speeding up multiscale molecular dynamics and coupling with 3d molecular theory of solvation. *Mol. Sim.*, **2013**, *39*, 25–48.
- [495] B. Leimkuhler; D. T. Margul; M. E. Tuckerman. Stochastic, Resonance-Free Multiple Time-Step Algorithm for Molecular Dynamics with Very Large Time Steps. *Mol. Phys.*, **2013**, *111*, 3579–3594.
- [496] G. Bussi; D. Donadio; M. Parrinello. Canonical sampling through velocity rescaling. *The Journal of chemical physics*, **2007**, *126*, 014101.
- [497] R. W. Pastor; B. R. Brooks; A. Szabo. An analysis of the accuracy of Langevin and molecular dynamics algorithms. *Mol. Phys.*, **1988**, *65*, 1409–1419.
- [498] R. J. Loncharich; B. R. Brooks; R. W. Pastor. Langevin dynamics of peptides: The frictional dependence of isomerization rates of N-actylananyl-N'-methylamide. *Biopolymers*, **1992**, *32*, 523–535.
- [499] Y. M. Rhee; V. S. Pande. Solvent viscosity dependence of the protein folding dynamics. *J. Phys. Chem. B*, **2008**, *112*, 6221–6227. PMID: 18229911.
- [500] J. A. Izaguirre; D. P. Catarello; J. M. Wozniak; R. D. Skeel. Langevin stabilization of molecular dynamics. *J. Chem. Phys.*, **2001**, *114*, 2090–2098.
- [501] Y. Xiong; P. S. Shabane; A. V. Onufriev. Melting points of opc and opc3 water models. *ACS Omega*, **2020**, *5*, 25087–25094. PMID: 33043187.
- [502] R. G. Fernández; J. L. F. Abascal; C. Vega. The melting point of ice Ih for common water models calculated from direct coexistence of the solid-liquid interface. *The Journal of Chemical Physics*, **2006**, *124*, 144506.
- [503] Y. Zhang; S. E. Feller; B. R. Brooks; R. W. Pastor. Computer simulation of liquid/liquid interfaces. I. Theory and application to octane/water. *J. Chem. Phys.*, **1995**, *103*, 10252–10266.
- [504] J.-P. Ryckaert; G. Ciccotti; H. J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: Molecular dynamics of n-alkanes. *J. Comput. Phys.*, **1977**, *23*, 327–341.
- [505] S. Miyamoto; P. A. Kollman. SETTLE: An analytical version of the SHAKE and RATTLE algorithm for rigid water models. *J. Comput. Chem.*, **1992**, *13*, 952–962.
- [506] Z. Zhang; X. Liu; Z. Chen; H. Zheng; K. Yan; J. Liu. A unified thermostat scheme for efficient configurational sampling for classical/quantum canonical ensembles via molecular dynamics. *J. Chem. Phys.*, **2017**, *147*, 034109.
- [507] J. Liu; D. Li; X. Liu. A simple and accurate algorithm for path integral molecular dynamics with the Langevin thermostat. *J. Chem. Phys.*, **2016**, *145*, 024103.
- [508] D. Li; X. Han; Y. Chai; C. Wang; Z. Zhang; Z. Chen; J. Liu; J. Shao. Stationary state distribution and efficiency analysis of the Langevin equation via real or virtual dynamics. *J. Chem. Phys.*, **2017**, *147*, 184104.
- [509] D. Li; Z. Chen; Z. Zhang; J. Liu. Understanding molecular dynamics with stochastic processes via real or virtual dynamics. *Chin. J. Chem. Phys.*, **2017**, *30*, 735–760.
- [510] Z. Zhang; K. Yan; X. Liu; J. Liu. A leap-frog algorithm-based efficient unified thermostat scheme for molecular dynamics. *Chinese Science Bulletin*, **2018**, *63*, 3467–3483.

BIBLIOGRAPHY

- [511] Z. Zhang; X. Liu; K. Yan; M. Tuckerman; J. Liu. Unified efficient thermostat scheme for the canonical ensemble with holonomic or isokinetic constraints via molecular dynamics. *J. Phys. Chem. A*, **2019**, *123*, 6056–6079.
- [512] B. Leimkuhler; C. Matthews. Rational construction of stochastic numerical methods for molecular sampling. *Appl. Math. Res. eXpress*, **2013**, *2013*, 34–56.
- [513] N. Grønbech-Jensen; O. Farago. A simple and effective Verlet-type algorithm for simulating Langevin dynamics. *Mol. Phys.*, **2013**, *111*, 983–991.
- [514] X. Liu; J. Liu. Critical role of quantum dynamical effects in the Raman spectroscopy of liquid water. *Mol. Phys.*, **2018**, *116*, 755–779.
- [515] H. C. Andersen. RATTLE: A "velocity" version of the shake algorithm for molecular dynamics calculations. *J. Computat. Phys.*, **1983**, *52*, 24 – 34.
- [516] Z. Sun; P. Kalhor; Y. Xu; J. Liu. Extensive numerical tests of leapfrog integrator in middle thermostat scheme in molecular simulations. *Chin. J. Chem. Phys.*, **2021**, *34*, 932–948.
- [517] X. Wu; S. Subramaniam; D. A. Case; K. Wu; B. R. Brooks. Targeted conformational search with map-restrained self-guided langevin dynamics: application to flexible fitting into electron microscopic density maps. *J. Struct. Biology*, **2013**, *183*, 429–440.
- [518] P. Ren; J. W. Ponder. Consistent treatment of inter- and intramolecular polarization in molecular mechanics calculations. *J. Comput. Chem.*, **2002**, *23*, 1497–1506.
- [519] P. Ren; J. W. Ponder. Temperature and pressure dependence of the AMOEBA water model. *J. Phys. Chem. B*, **2004**, *108*, 13427–13437.
- [520] T. Darden; D. York; L. Pedersen. Particle mesh Ewald—an Nlog(N) method for Ewald sums in large systems. *J. Chem. Phys.*, **1993**, *98*, 10089–10092.
- [521] U. Essmann; L. Perera; M. L. Berkowitz; T. Darden; H. Lee; L. G. Pedersen. A smooth particle mesh Ewald method. *J. Chem. Phys.*, **1995**, *103*, 8577–8593.
- [522] M. F. Crowley; T. A. Darden; T. E. Cheatham, III; D. W. Deerfield, II. Adventures in improving the scaling and accuracy of a parallel molecular dynamics program. *J. Supercomput.*, **1997**, *11*, 255–278.
- [523] C. Sagui; T. A. Darden. in *Simulation and Theory of Electrostatic Interactions in Solution*, L. R. Pratt; G. Hummer, Eds., pp 104–113. American Institute of Physics, Melville, NY, 1999.
- [524] A. Toukmaji; C. Sagui; J. Board; T. Darden. Efficient particle-mesh Ewald based approach to fixed and induced dipolar interactions. *J. Chem. Phys.*, **2000**, *113*, 10913–10927.
- [525] C. Sagui; L. G. Pedersen; T. A. Darden. Towards an accurate representation of electrostatics in classical force fields: Efficient implementation of multipolar interactions in biomolecular simulations. *J. Chem. Phys.*, **2004**, *120*, 73–87.
- [526] X. Wu; B. R. Brooks. Isotropic periodic sum: A method for the calculation of long-range interactions. *J. Chem. Phys.*, **2005**, *122*, 044107.
- [527] J. B. Klauda; X. Wu; R. W. Pastor; B. R. Brooks. Long-Range Lennard-Jones and Electrostatic Interactions in Interfaces. *J. Phys. Chem. B*, **2007**, *111*, 4393–4400.
- [528] K. Takahashi; K. Yasuoka; T. Narumi. Cutoff radius effect of isotropic periodic sum method for transport. *J. Chem. Phys.*, **2007**, *127*, 114511.
- [529] X. Wu; B. R. Brooks. Using the Isotropic Periodic Sum Method to Calculate Long-Range Interactions of Heterogeneous Systems. *J. Chem. Phys.*, **2008**, *129*, 154115.

- [530] X. Wu; B. R. Brooks. Isotropic periodic sum of electrostatic interactions for polar systems. *J. Chem. Phys.*, **2009**, *131*, 024107.
- [531] R. M. Venable; L. E. Chen; R. W. Pastor. Comparison of the Extended Isotropic Periodic Sum and Particle Mesh Ewald Methods for Simulations of Lipid Bilayers and Monolayers. *J. Phys. Chem. B*, **2009**, *113*, 5855–5862.
- [532] Y. Duan; T. Niu; J. Wang; P. Cieplak; R. Luo. Pcmresp: A method for polarizable force field parameter development and transferability of the polarizable gaussian multipole models across multiple solvents. *J. Chem. Theory Computat.*, **2024**, *20*, 2820–2829.
- [533] Y. Duan; J. Wang; P. Cieplak; R. Luo. Refinement of atomic polarizabilities for a polarizable gaussian multipole force field with simultaneous considerations of both molecular polarizability tensors and in-solution electrostatic potentials. *J. Chem. Inform. Model.*, **2025**.
- [534] Z. Huang; S. Zhao; P. Cieplak; Y. Duan; R. Luo; H. Wei. Optimal scheme to achieve energy conservation in induced dipole models. *J. Chem. Theory Computat.*, **2023**, *19*, 5047–5057.
- [535] Z. Huang; X. Wu; R. Luo. Isotropic periodic sum for polarizable gaussian multipole model. *J. Chem. Theory Computat.*, **2025**.
- [536] Z. Huang; Y. Wu; Y. Duan; R. Luo. Performance tuning of polarizable gaussian multipole model in molecular dynamics simulations. *J. Chem. Theory Computat.*, **2025**.
- [537] Y. Wu; Q. Zhu; Z. Huang; P. Cieplak; Y. Duan; R. Luo. Automated refinement of property-specific polarizable gaussian multipole water models using bayesian black-box optimization. *J. Chem. Theory Computat.*, **2025**.
- [538] R. Konecny; N. A. Baker; J. A. McCammon. iAPBS: a programming interface to the adaptive Poisson–Boltzmann solver. *Comput. Sci. Disc.*, **2012**, *5*, 15005–15013.
- [539] A. W. Goetz; M. J. Williamson; D. Xu; D. Poole; S. L. Grand; R. C. Walker. Routine microsecond molecular dynamics simulations with AMBER - Part I: Generalized Born. *J. Chem. Theory Comput.*, **2012**, *8*, 1542–1555.
- [540] R. Salomon-Ferrer; A. W. Goetz; D. Poole; S. L. Grand; R. C. Walker. Routine microsecond molecular dynamics simulations with AMBER - Part 2: Explicit Solvent Particle Mesh Ewald . *J. Chem. Theory Comput.*, **2013**, *9*, 3878–3888.
- [541] S. Le Grand; A. W. Goetz; R. C. Walker. SPFP: Speed without compromise—A mixed precision model for GPU accelerated molecular dynamics simulations. *Comput. Phys. Commun.*, **2013**, *184*, 374–380.
- [542] R. M. Betz; N. A. DeBardeleben; R. C. Walker. An Investigation of the effects of hard and soft errors on graphics processing unit-accelerated molecular dynamics simulations. *Concurrency and Computation: Practice and Experience*, **2014**, *26*, 2134.
- [543] X. Wu; B. R. Brooks. Self-guided Langevin dynamics simulation method. *Chem. Phys. Lett.*, **2003**, *381*, 512–518.
- [544] X. Wu; A. Damjanovic; B. R. Brooks. Efficient and unbiased sampling of biomolecular systems in the canonical ensemble: a review of self-guided langevin dynamics. *Adv. Chem. Phys.*, **2012**, *150*, 255–326.
- [545] X. Yu; X. Wu; G. A. Bermejo; B. R. Brooks; J. W. Taraska. Accurate high-throughput structure mapping and prediction with transition metal ion fret. *Structure*, **2013**, *21*, 9–19.
- [546] X. Wu; B. R. Brooks. Self-guided langevin dynamics via generalized langevin equation. *J. Comput. Chem.*, **2016**, *37*, 595–601.
- [547] X. Wu; B. R. Brooks. Reformulation of the self-guided molecular simulation method. *J. Chem. Phys.*, **2020**, *153*, 094112.
- [548] X. Wu; B. R. Brooks. Toward canonical ensemble distribution from self-guided Langevin dynamics simulation. *J. Chem. Phys.*, **2011**, *134*, 134108.

BIBLIOGRAPHY

- [549] X. Wu; B. R. Brooks. Force-momentum-based self-guided Langevin dynamics: a rapid sampling method that approaches the canonical ensemble . *J. Chem. Phys.*, **2011**, *135*, 204101.
- [550] X. Wu; M. Hodoscek; B. R. Brooks. Replica exchanging self-guided Langevin dynamics for efficient and accurate conformational sampling. *J. Chem. Phys.*, **2012**, *137*, 044106.
- [551] D. Hamelberg; J. Mongan; J. A. McCammon. Accelerated molecular dynamics: A promising and efficient simulation method for biomolecules. *J. Chem. Phys.*, **2004**, *120*, 11919–11929.
- [552] D. Hamelberg; C. A. F. de Oliveira; J. McCammon. Sampling of slow diffusive conformational transitions with accelerated molecular dynamics. *J. Chem. Phys.*, **2007**, *127*, 155102–155109.
- [553] B. J. Grant; A. A. Gorfe; J. A. McCammon. Ras conformational switching: Simulating nucleotide-dependent conformational transitions with accelerated molecular dynamics. *PLoS Computat. Biol.*, **2009**, *5*, e1000325.
- [554] C. A. F. de Oliveira; B. J. Grant; M. Zhou; J. A. McCammon. Large-scale conformational changes of trypanosoma cruzi proline racemase predicted by accelerated molecular dynamics simulation. *PLoS Computat. Biol.*, **2011**, *7*, e1002178.
- [555] L. C. T. Pierce; R. Salomon-Ferrer; C. A. F. de Oliveira; J. A. McCammon; R. C. Walker. Routine access to milli-second time scales with accelerated molecular dynamics. *J. Chem. Theory Comput.*, **2012**, *8*, 2997–3002.
- [556] U. Doshi; D. Hamelberg. Reoptimization of the amber forcefield for peptide bond (omega) torsions using accelerated molecular dynamics. *J. Chem. Phys. B*, **2009**, *113*, 16590–16595.
- [557] Y. Miao; V. A. Feher; J. A. McCammon. Gaussian Accelerated Molecular Dynamics: Unconstrained Enhanced Sampling and Free Energy Calculation. *J. Chem. Theory Comput.*, **2015**, *11*, 3584–3595.
- [558] Y. Miao; W. Sinko; L. Pierce; D. Bucher; R. C. Walker; J. A. McCammon. Improved Reweighting of Accelerated Molecular Dynamics Simulations for Free Energy Calculation. *J. Chem. Theory Comput.*, **2014**, *10*, 2677–2689.
- [559] Y. Miao; A. Bhattarai; J. Wang. Ligand Gaussian accelerated molecular dynamics (LiGaMD): Characterization of ligand binding thermodynamics and kinetics. *J. Chem. Theory Comput.*, **2020**, *16*, 5526–5547.
- [560] J. Wang; Y. Miao. Peptide Gaussian accelerated molecular dynamics (Pep-GaMD): Enhanced sampling and free energy and kinetics calculations of peptide binding. *J. Chem. Phys.*, **2020**, *153*, 154109.
- [561] J. Wang; Y. Miao. Protein-Protein Interaction-Gaussian Accelerated Molecular Dynamics (PPI-GaMD): Characterization of Protein Binding Thermodynamics and Kinetics. *J. Chem. Theory Comput.*, **2022**, *18*, 1275–1285.
- [562] G. Mills; H. Jönsson. Quantum and thermal effects in H₂ dissociative adsorption: Evaluation of free energy barriers in multidimensional quantum systems. *Phys. Rev. Lett.*, **1994**, *72*, 1124–1127.
- [563] H. Jönsson; G. Mills; K. W. Jacobsen. in *Classical and Quantum Dynamics in Condensed Phase Simulations*, B. J. Berne; G. Ciccoti; D. F. Coker, Eds., pp 385–404. World Scientific, Singapore, 1998.
- [564] R. Elber; M. Karplus M. A method for determining reaction paths in large molecules: Application to myoglobin. *Chem. Phys. Lett.*, **1987**, *139*, 375–380.
- [565] G. Henkelman; H. Jönsson. Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points. *J. Chem. Phys.*, **2000**, *113*, 9978–9985.
- [566] G. Henkelman; B. P. Uberuaga; H. Jönsson. A climbing image nudged elastic band method for finding saddle points and minimum energy paths. *J. Chem. Phys.*, **2000**, *113*, 9901–9904.
- [567] J. Chu; B. L. Trout; B. R. Brooks. A super-linear minimization scheme for the nudged elastic band method. *J. Chem. Phys.*, **2003**, *119*, 12708–12717.
- [568] C. Bergonzo; A. J. Campbell; R. C. Walker; C. Simmerling. A Partial Nudged Elastic Band Implementation for Use with Large or Explicitly Solvated Systems. *Int J Quantum Chem*, **2009**, *109*, 3781–3790.

- [569] D. H. Mathews; D. A. Case. Nudged Elastic Band calculation of minimal energy pathways for the conformational change of a GG mismatch. *J. Mol. Biol.*, **2006**, 357, 1683–1693.
- [570] K. Zinovjev; I. Tunón. Adaptive finite temperature string method in collective variables. *J. Phys. Chem. A*, **2017**, 121, 9764–9772. PMID: 29190105.
- [571] L. Maragliano; E. Vanden-Eijnden. On-the-fly string method for minimum free energy paths calculation. *Chem. Phys. Lett.*, **2007**, 446, 182–190.
- [572] K. Zinovjev; S. Martí; I. Tunón. A collective coordinate to obtain free energy profiles for complex reactions in condensed phases. *J. Chem. Theory Comput.*, **2012**, 8, 1795–1801. PMID: 26593670.
- [573] D. Branduardi; F. L. Gervasio; M. Parrinello. From A to B in free energy space. *J. Chem. Phys.*, **2007**, 126, 054103.
- [574] J. Kästner; W. Thiel. Bridging the gap between thermodynamic integration and umbrella sampling provides a novel analysis method: “Umbrella integration”. *J. Chem. Phys.*, **2005**, 123, 144104.
- [575] K. Zinovjev. Adaptive string method tutorial. <https://github.com/kzinovjev/string-amber/blob/master/tutorial.md>, 2021.
- [576] C. A. Ramos-Guzmán. Obtention of the reaction mechanism along a complex reaction coordinate. <https://carlosramosg.com/string-method>, 2021.
- [577] C. A. Ramos-Guzmán; J. J. Ruiz-Pernía; I. Tunón. Inhibition mechanism of sars-cov-2 main protease with ketone-based inhibitors unveiled by multiscale simulations: Insights for improved designs. *Angew. Chemie*, **2021**, 60, 25933–25941.
- [578] R. F. Grote; J. T. Hynes. The stable states picture of chemical reactions. II. Rate constants for condensed and gas phase reaction models. *J. Chem. Phys.*, **1980**, 73, 2715–2732.
- [579] I. Kolossváry; W. C. Guida. Low mode search. An efficient, automated computational method for conformational analysis: Application to cyclic and acyclic alkanes and cyclic peptides. *J. Am. Chem. Soc.*, **1996**, 118, 5011–5019.
- [580] I. Kolossváry; W. C. Guida. Low-mode conformational search elucidated: Application to C₃₉H₈₀ and flexible docking of 9-deazaguanine inhibitors into PNP. *J. Comput. Chem.*, **1999**, 20, 1671–1684.
- [581] I. Kolossváry; G. M. Keserü. Hessian-free low-mode conformational search for large-scale protein loop optimization: Application to c-jun N-terminal kinase JNK3. *J. Comput. Chem.*, **2001**, 22, 21–30.
- [582] G. M. Keserü; I. Kolossváry. Fully flexible low-mode docking: Application to induced fit in HIV integrase. *J. Am. Chem. Soc.*, **2001**, 123, 12708–12709.
- [583] I. Kolossváry. A Fresh Look at the Normal Mode Analysis of Proteins: Introducing Allosteric Co-Vibrational Modes. *JACS Au*, **2024**, 4, 1303–1309. <https://doi.org/10.1021/jacsau.4c00109>.
- [584] W. H. Press; B. P. Flannery; S. A. Teukolsky; W. T. Vetterling. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, New York, 1989.
- [585] D. C. Liu; J. Nocedal. On the limited memory method for large scale optimization. *Math. Programming B*, **1989**, 45, 503–528.
- [586] J. Nocedal; J. L. Morales. Automatic preconditioning by limited memory quasi-Newton updating. *SIAM J. Opt.*, **2000**, 10, 1079–1096.
- [587] P. Kollman. Free energy calculations: Applications to chemical and biochemical phenomena. *Chem. Rev.*, **1993**, 93, 2395–2417.
- [588] T. Simonson. in *Computational Biochemistry and Biophysics*, O. Becker; A. D. MacKerell; B. Roux; M. Watanabe, Eds. Marcel Dekker, New York, 2001.

BIBLIOGRAPHY

- [589] T. Steinbrecher; D. A. Case; A. Labahn. A multistep approach to structure-based drug design: Studying ligand binding at the human neutrophil elastase. *J. Med. Chem.*, **2006**, *49*, 1837–1844.
- [590] T. Steinbrecher; A. Hrenn; K. Dormann; I. Merfort; A. Labahn. Bornyl (3,4,5-trihydroxy)-cinnamate - An optimized human neutrophil elastase inhibitor designed by free energy calculations. *Bioorg. Med. Chem.*, **2008**, *16*, 2385–2390.
- [591] J. Kaus; L. C. T. Pierce; R. C. Walker; J. A. McCammon. Improving the Efficiency of Free Energy Calculations in the Amber Molecular Dynamics Package. *J. Chem. Theory Comput.*, **2013**, *9*, 4131–4139.
- [592] T.-S. Lee; Y. Hu; B. Sherborne; Z. Guo; D. M. York. Toward fast and accurate binding affinity prediction with pmemdgti: An efficient implementation of GPU-accelerated thermodynamic integration. *J. Chem. Theory Comput.*, **2017**, *13*, 3077–3084.
- [593] T.-S. Lee; D. S. Cerutti; D. Mermelstein; C. Lin; S. LeGrand; T. J. Giese; A. Roitberg; D. A. Case; R. C. Walker; D. M. York. Gpu-accelerated molecular dynamics and free energy methods in amber18: Performance enhancements and new features. *J. Chem. Inf. Model.*, **2018**, *58*, 2043–2050.
- [594] L. F. Song; T.-S. Lee; C. Zhu; D. M. York; K. M. Merz Jr. Using AMBER18 for Relative Free Energy Calculations. *J. Chem. Inf. Model.*, **2019**, *59*, 3128–3135.
- [595] G. Hummer; A. Szabo. Calculation of free-energy differences from computer simulations of initial and final states. *J. Chem. Phys.*, **1996**, *105*, 2004–2010.
- [596] T. Steinbrecher; D. L. Mobley; D. A. Case. Non-linear scaling schemes for Lennard-Jones interactions in free energy calculations. *J. Chem. Phys.*, **2007**, *127*, 214108.
- [597] T. Steinbrecher; I. Joung; D. A. Case. Soft-core potentials in thermodynamic integration: Comparing one- and two-step transformations. *J. Comp. Chem.*, **2011**, *32*, 3253–3263.
- [598] T.-S. Lee; Z. Lin; B. K. Allen; C. Lin; B. K. Radak; Y. Tao; H.-C. Tsai; W. Sherman; D. M. York. Improved alchemical free energy calculations with optimized smoothstep softcore potentials. *J. Chem. Theory Comput.*, **2020**, *16*, 5512–5525.
- [599] T.-S. Lee; H.-C. Tsai; A. Ganguly; T. J. Giese; D. M. York. in *Free Energy Methods in Drug Discovery: Current State and Future Directions*. chapter 7, pp 161–204. 2021.
- [600] H.-C. Tsai; T.-S. Lee; A. Ganguly; T. J. Giese; M. C. Ebert; P. Labute; K. M. Merz Jr; D. M. York. AMBER Free Energy Tools: A New Framework for the Design of Optimized Alchemical Transformation Pathways. *J. Chem. Theory Comput.*, **2023**, *19*, 640–658.
- [601] H.-C. Tsai; Y. Tao; T.-S. Lee; K. M. Merz; D. M. York. Validation of free energy methods in amber. *J. Chem. Inf. Model.*, **2020**, *60*, 5296–5300.
- [602] T.-S. Lee; H.-C. Tsai; A. Ganguly; D. M. York. ACES: Optimized Alchemically Enhanced Sampling. *J. Chem. Theory Comput.*, **2023**, *19*, 472–487.
- [603] S. Boresch; M. Karplus. The role of bonded terms in free energy simulations. 1. theoretical analysis. *J. Phys. Chem. A*, **1999**, *103*, 103–118.
- [604] S. Boresch; M. Karplus. The role of bonded terms in free energy simulations. 2. calculation of their influence on free energy differences of solvation. *J. Phys. Chem. A*, **1999**, *103*, 119–136.
- [605] S. Boresch. The role of bonded energy terms in free energy simulations - insights from analytical results. *Mol. Simul.*, **2002**, *28*, 13–37.
- [606] A. Mitsutake; Y. Sugita; Y. Okamoto. Generalized-ensemble algorithms for molecular simulations of biopolymers. *Biopolymers*, **2001**, *60*, 96–123.

- [607] H. Nymeyer; S. Gnanakaran; A. García. Atomic simulations of protein folding using the replica exchange algorithm. *Meth. Enzymol.*, **2004**, 383, 119–149.
- [608] X. Cheng; G. Cui; V. Hornak; C. Simmerling. Modified replica exchange simulation methods for local structure refinement. *J. Phys. Chem. B*, **2005**, 109, 8220–8230.
- [609] Y. Meng; D. Sabri Dashti; A. E. Roitberg. Computing Alchemical Free Energy Differences with Hamiltonian Replica Exchange Molecular Dynamics (H-REMD) Simulations. *J. Chem. Theory Comput.*, **2011**, 7, 2721–2727.
- [610] S. G. Itoh; A. Damjanovic; B. R. Brooks. pH replica-exchange method based on discrete protonation states. *Proteins*, **2011**, 79, 3420–3436.
- [611] J. M. Swails; A. E. Roitberg. Enhancing Conformation and Protonation State Sampling of Hen Egg White Lysozyme Using pH Replica Exchange Molecular Dynamics. *J. Chem. Theory Comput.*, **2012**, 8, 4393–4404.
- [612] V. W. D. Cruzeiro; M. S. Amaral; A. E. Roitberg. Redox Potential Replica Exchange Molecular Dynamics at constant pH in AMBER: Implementation, Validation and Application. *J. Chem. Phys.*, **2018**, 149, 072338.
- [613] V. W. D. Cruzeiro; A. E. Roitberg. Multidimensional Replica Exchange Simulations for Efficient Constant pH and Redox Potential Molecular Dynamics. *J. Chem. Theory Comput.*, **2019**.
- [614] A. Patriksson; D. van der Spoel. A temperature predictor for parallel tempering simulations. *Phys. Chem. Chem. Phys.*, **2008**, 10, 2073–2077.
- [615] A. Okur; D. R. Roe; G. Cui; V. Hornak; C. Simmerling. Improving convergence of replica-exchange simulations through coupling to a high-temperature structure reservoir. *J. Chem. Theory comput.*, **2007**, 3, 557–568.
- [616] A. E. Roitberg; A. Okur; C. Simmerling. Coupling of replica exchange simulations to a non-Boltzmann structure reservoir. *J. Phys. Chem. B*, **2007**, 111, 2415–2418.
- [617] D. Sabri Dashti; A. E. Roitberg. Calculating the pKa Shift of Titratable Group at Position 66 of Staphylococcal Nuclease Mutant with the Replica Exchange Free Energy Perturbation method (REFEP). *In preparation*, **2012**.
- [618] D. Sabri Dashti; A. E. Roitberg. Optimization of Umbrella Sampling Replica Exchange Molecular Dynamics by Replica Positioning. *J. Chem. Theory Comput.*, **2013**, 9, 4692–4699.
- [619] M. Fajer; D. Hamelberg; J. A. McCammon. Replica-Exchange Accelerated Molecular Dynamics (REXAMD) Applied to Thermodynamic Integration. *J. Chem. Theory Comput.*, **2008**, 4, 1565–1569.
- [620] M. Arrar; C. A. F. de Oliveira; M. Fajer; W. Sinko; J. A. McCammon. w-REXAMD: A Hamiltonian Replica Exchange Approach to Improve Free Energy Calculations for Systems with Kinetically Trapped Conformations. *J. Chem. Theory Comput.*, **2013**, 9, 18–23.
- [621] M. R. Shirts; J. D. Chodera. Statistically optimal analysis of samples from multiple equilibrium states. *J. Chem. Phys.*, **2008**, 129, 124105–124105–10.
- [622] A. Pohorille; C. Jarzynski; C. Chipot. Good practices in Free-Energy calculations. *J. Phys. Chem. B*, **2010**, 114, 10235–10253.
- [623] J. M. Swails. *Free Energy Simulations of Complex Biological Systems at Constant pH*. PhD thesis, University of Florida, 2013.
- [624] V. Babin; C. Roland; C. Sagui. Adaptively biased molecular dynamics for free energy calculations. *J. Chem. Phys.*, **2008**, 128, 134101.
- [625] T. Huber; A. E. Torda; W. F. van Gunsteren. Local elevation: a method for improving the searching properties of molecular dynamics simulation. *J. Comput. Aided. Mol. Des.*, **1994**, 8, 695–708.
- [626] F. Wang; D. P. Landau. Efficient, multiple-range random walk algorithm to calculate the density of states. *Phys. Rev. Lett.*, **2001**, 86, 2050–2053.

BIBLIOGRAPHY

- [627] A. Laio; M. Parrinello. Escaping free-energy minima. *Proc. Natl. Acad. Sci.*, **2002**, 99, 12562–12566.
- [628] M. Iannuzzi; A. Laio; M. Parrinello. Efficient exploration of reactive potential energy surfaces using car-parrinello molecular dynamics. *Phys. Rev. Lett.*, **2003**, 90, 238302–1.
- [629] T. Lelièvre; M. Rousset; G. Stoltz. Computation of free energy profiles with parallel adaptive dynamics. *J. Chem. Phys.*, **2007**, 126, 134111.
- [630] P. Raiteri; A. Laio; F. L. Gervasio; C. Micheletti; M. Parrinello. Efficient reconstruction of complex free energy landscapes by multiple walkers metadynamics. *J. Phys. Chem.*, **2006**, 110, 3533–3539.
- [631] Y. Sugita; A. Kitao; Y. Okamoto. Multidimensional replica-exchange method for free-energy calculations. *J. Chem. Phys.*, **2000**, 113, 6042–6051.
- [632] G. Bussi; F. L. Gervasio; A. Laio; M. Parrinello. Free-energy landscape for β hairpin folding from combined parallel tempering and metadynamics. *J. Am. Chem. Soc.*, **2006**, 128, 13435–13441.
- [633] S. Piana; A. Laio. A bias-exchange approach to protein folding. *J. Phys. Chem. B*, **2007**, 111, 4553–4559.
- [634] V. Babin; C. Roland; T. A. Darden; C. Sagui. The free energy landscape of small peptides as obtained from metadynamics with umbrella sampling corrections. *J. Chem. Phys.*, **2006**, 125, 2049096.
- [635] V. Babin; V. Karpusenko; M. Moradi; C. Roland; C. Sagui. Adaptively biased molecular dynamics: an umbrella sampling method with a time dependent potential. *Inter. J. Quantum Chem.*, **2009**, 109, 3666–3678.
- [636] V. Babin; C. Sagui. Conformational free energies of methyl- α -l-iduronic and methyl- β -d-glucuronic acids in water. *J. Chem. Phys.*, **2010**, 132, 104108.
- [637] M. Moradi; V. Babin; C. Roland; T. Darden; C. Sagui. Conformations and free energy landscapes of polyproline peptides. *Proc. Natl. Aca. Sci. USA*, **2009**, 106, 20746.
- [638] M. Moradi; V. Babin; C. Roland; C. Sagui. A classical molecular dynamics investigation of the free energy and structure of short polyproline conformers. *J. Chem. Phys.*, **2010**, 133, 125104.
- [639] M. Moradi; V. Babin; C. Sagui; C. Roland. in *Proline: Biosynthesis, Regulation and Health Benefits*, B. Nédjimi, Ed., pp 67–110. Nova Publishers, 2013.
- [640] M. Moradi; V. Babin; C. Sagui; C. Roland. A statistical analysis of the PPII propensity of amino acid guests in proline-rich peptides. *Biophysical J.*, **2011**, 100, 1083 – 1093.
- [641] M. Moradi; V. Babin; C. Sagui; C. Roland. PPII propensity of multiple-guest amino acids in a proline-rich environment. *J. Phys. Chem. B.*, **2011**, 115, 8645–8656.
- [642] V. Babin; C. Roland; C. Sagui. The alpha-sheet: A missing-in-action secondary structure? *Proteins –Structure Function and Bioinformatics*, **2011**, 79, 937–946.
- [643] M. Moradi; V. Babin; C. Roland; C. Sagui. Are long-range structural correlations behind the aggregation phenomena of polyglutamine diseases? *PLoS Comput. Biol.*, **2012**, 8, e1002501.
- [644] M. Moradi; V. Babin; C. Roland; C. Sagui. Reaction path ensemble of the B-Z-DNA transition: a comprehensive atomistic study. *Nucleic Acids Research*, **2013**, 41, 33–43.
- [645] F. Pan; V. H. Man; C. Roland; C. Sagui. Structure and Dynamics of DNA and RNA Double Helices of CAG and GAC Trinucleotide Repeats. *Biophys. J.*, **2017**, 113, 19–36.
- [646] F. Pan; Y. Zhang; V. H. Man; C. Roland; C. Sagui. E-motif formed by extrahelical cytosine bases in DNA homoduplexes of trinucleotide and hexanucleotide repeats. *Nucl. Acids Res.*, **2018**, 46, 942–955.
- [647] F. Pan; V. H. Man; C. Roland; C. Sagui. Structure and Dynamics of DNA and RNA Double Helices Obtained From the CCG and GGC Trinucleotide Repeats. *J. Phys. Chem. B*, **2018**, 122, 4491–4512.

- [648] P. Xu; F. Pan; C. Roland; C. Sagui; K. Weninger. Dynamics of strand slippage in DNA hairpins formed by CAG repeats: roles of sequence parity and trinucleotide interrupts. *Nucl. Acids Res.*, **2020**, 48, 2232–2245.
- [649] M. Moradi; J.-G. Lee; V. Babin; C. Roland; C. Sagui. Free energy and structure of polyproline peptides: an ab initio and classical molecular dynamics investigation. *Int. J. Quantum Chem.*, **2010**, 110, 2865–2879.
- [650] M. Moradi; C. Sagui; C. Roland. Calculating relative transition rates with driven nonequilibrium simulations. *Chem. Phys. Lett.*, **2011**, 518, 109.
- [651] M. Moradi; C. Sagui; C. Roland. Investigating rare events with nonequilibrium work measurements: I. Nonequilibrium transition paths. *J. Chem. Phys.*, **2014**, 140, 034114.
- [652] M. Moradi; C. Sagui; C. Roland. Investigating rare events with nonequilibrium work measurements: II. Transition and reaction rates. *J. Chem. Phys.*, **2014**, 140, 034115.
- [653] M. Moradi; E. Tajkhorshid. Driven Metadynamics: Reconstructing equilibrium free energies from driven adaptive-bias simulations. *J. Phys. Chem. Lett.*, **2013**, 4, 1882.
- [654] A. C. Pan; D. Sezer; B. Roux. Finding transition pathways using the string method with swarms of trajectories. *J. Phys. Chem. B*, **2008**, 112, 3432–3440.
- [655] A. Barducci and G. Bussi and M. Parrinello. Well-tempered metadynamics: a smoothly converging and tunable free energy method. *Phys. Rev. Lett.*, **2008**, 100, 020603.
- [656] K. Minoukadeh and Ch. Chipot and T. Lelievre. Potential of Mean Force Calculations: A multiple-walker adaptive biasing force technique. *J. Chem. Theor. and Comput.*, **2010**, 6, 1008.
- [657] E. A. Coutsiias; C. Seok; K. A. Dill. Using quaternions to calculate RMSD. *J. Comput. Chem.*, **2004**, 25, 1849–1857.
- [658] D. K. Coutsiias EA, Seok C. Using quaternions to calculate rmsd. *J Comput Chem*, **2004 Nov 30**, 25, 1849–57.
- [659] G. Fiorin; M. L. Klein; J. Hénin. Using collective variables to drive molecular dynamics simulations. *Molecular Physics*, **2013**, 111, 3345–3362.
- [660] M. Moradi; E. Tajkhorshid. Mechanistic picture for conformational transition of a membrane transporter at atomic resolution. *Proc. Natl. Acad. Sci. U.S.A.*, **2013**, 110, 18916–18921.
- [661] M. Moradi; E. Tajkhorshid. Computational Recipe for Efficient Description of Large-Scale Conformational Changes in Biomolecular Systems. *J Chem Theory Comput*, **2014**, 10, 2866–2880.
- [662] S. Park; F. Khalili-Araghi; E. Tajkhorshid; K. Schulten. Free energy calculation from steered molecular dynamics simulations using Jarzynski's equality. *J. Chem. Phys.*, **2003**, 119, 3559–3566.
- [663] M. Matsumoto; T. Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Trans. Model. Comput. Simul.*, **1998**, 8, 3–30.
- [664] L. Maragliano; A. Fischer; E. Vanden-Eijnden; G. Ciccotti. String method in collective variables: Minimum free energy paths and isocommittor surfaces. *J. Chem. Phys.*, **2006**, 125, 024106.
- [665] M. O. Jensen; S. Park; E. d; K. Schulten. Energetics of glycerol conduction through aquaglyceroporin GlpF. *Proc. Natl. Acad. Sci. USA*, **2002**, 99, 6731–6736.
- [666] A. Crespo; M. A. Marti; D. A. Estrin; A. E. Roitberg. Multiple-steering QM-MM calculation of the free energy profile in chorismate mutase. *J. Am. Chem. Soc.*, **2005**, 127, 6940–6941.
- [667] C. Jarzynski. Nonequilibrium equality for free energy differences. *Phys. Rev. Lett.*, **1997**, 78, 2690–2693.
- [668] G. Hummer; A. Szabo. Free energy reconstruction from nonequilibrium single-molecule pulling experiments. *Proc. Natl. Acad. Sci. USA*, **2001**, 98, 3658.

BIBLIOGRAPHY

- [669] G. Hummer; A. Szabo. Kinetics from nonequilibrium single-molecule pulling experiments. *Biophys. J.*, **2003**, 85, 5–15.
- [670] T. Schilling; F. Schmid. Computing absolute free energies of disordered structures by molecular simulation. *J. Chem. Phys.*, **2009**, 131, 231102.
- [671] F. Schmid; T. Schilling. A method to compute absolute free energies or enthalpies of fluids. *Physics Procedia*, **2010**, 4, 131–143.
- [672] J. T. Berryman; T. Schilling. Free Energies by Thermodynamic Integration Relative to an Exact Solution, Used to Find the Handedness-Switching Salt Concentration for DNA. *J. Chem. Theory Comput.*, **2013**, 9, 679–686.
- [673] J. T. Berryman; T. Schilling. Absolute Free Energies for Biomolecules in Implicit or Explicit Solvent. *Physics Procedia*, **2014**, 57, 7–15.
- [674] D. Frenkel; A. J. C. Ladd. New Monte Carlo method to compute the free energy of arbitrary solids. Application to the fcc and hcp phases of hard spheres. *J. Chem. Phys.*, **1984**, 81, 3188–3193.
- [675] C. Vega; E. G. Noya. Revisiting the Frenkel-Ladd method to compute the free energy of solids: the Einstein molecule approach. *J. Chem. Phys.*, **2007**, 127, 154113.
- [676] R. Assaraf; M. Caffarel; A. C. Kollias. Chaotic versus nonchaotic stochastic dynamics in monte carlo simulations: A route for accurate energy differences in n-body systems. *Phys. Rev. Lett.*, **2011**, 106, 150601.
- [677] C. Jarzynski. Equilibrium free-energy differences from nonequilibrium measurements: A master-equation approach. *Phys. Rev. E*, **1997**, 56, 5018–5035.
- [678] M. Wang; Y. Mei; U. Ryde. Predicting relative binding affinity using nonequilibrium qm/mm simulations. *J. Chem. Theory Comput.*, **2018**, 14, 6613–6622.
- [679] J. Mongan; D. A. Case; J. A. McCammon. Constant pH molecular dynamics in generalized Born implicit solvent. *J. Comput. Chem.*, **2004**, 25, 2038–2048.
- [680] J. M. Swails; D. M. York; A. E. Roitberg. Constant pH replica exchange molecular dynamics in explicit solvent using discrete protonation states: implementation, testing, and validation. *J. Chem. Theory Comput.*, **2014**, 10, 1341–1352.
- [681] V. W. D. Cruzeiro; G. T. Feliciano; A. E. Roitberg. Exploring Coupled Redox and pH Processes with a Force-Field-Based Approach: Applications to Five Different Systems. *J. Am. Chem. Soc.*, **2020**, 142, 3823–3835.
- [682] X. Kong; C. L. Brooks, III. Lambda-dynamics: A new approach to free energy calculations. *J. Chem. Phys.*, **1996**, 105, 10.
- [683] M. S. Lee; F. R. Salsbury; C. L. Brooks, III. Constant-pH molecular dynamics using continuous titration coordinates. *Proteins*, **2004**, 56, 738–752.
- [684] J. Khandogin; C. L. Brooks, III. Constant pH molecular dynamics with proton tautomerism. *Biophys. J.*, **2005**, 89, 141–157.
- [685] Y. Huang; R. C. Harris; J. Shen. Generalized Born based continuous constant pH molecular dynamics in Amber: implementation, benchmarking, and analysis. *J. Chem. Inform. Model.*, **2018**, 58, 1372–1383.
- [686] R. C. Harris; J. Shen. GPU-Accelerated Implementation of Continuous Constant pH Molecular Dynamics in Amber: pKa Predictions with Single-pH Simulations. *J. Chem. Inform. Model.*, **2019**, 59, 4821–4832.
- [687] J. A. Wallace; J. K. Shen. Continuous constant pH molecular dynamics in explicit solvent with pH-based replica exchange. *J. Chem. Theory Comput.*, **2011**, 7, 2617–2629.
- [688] J. A. Wallace; J. K. Shen. Charge-leveling and proper treatment of long-range electrostatics in all-atom molecular dynamics at constant pH. *J. Chem. Phys.*, **2012**, 137, 184105.

- [689] Y. Huang; W. Chen; J. A. Wallace; J. Shen. All-Atom continuous constant pH molecular dynamics with particle mesh Ewald and titratable water. *J. Chem. Theory Comput.*, **2016**, *12*, 5411–5421.
- [690] J. A. Harris; R. Liu; V. M. de Oliveira; E. A. Vázquez-Montelongo; J. A. Henderson; J. Shen. GPU-Accelerated All-Atom Particle-Mesh Ewald Continuous Constant pH Molecular Dynamics in Amber. *J. Chem. Theory Comput.*, **2022**, *18*, 7510–7527.
- [691] J. A. Henderson; N. Verma; R. C. Harris; R. Liu; J. Shen. Assessment of proton-coupled conformational dynamics of SARS and MERS coronavirus papain-like proteases: Implication for designing broad-spectrum antiviral inhibitors. *J. Chem. Phys.*, **2020**, *153*, 115101.
- [692] J. A. Henderson; R. Lui; J. A. Harris; Y. Huang; V. M. de Oliveira; J. Shen. A Guide to the Continuous Constant pH Molecular Dynamics Methods in Amber and CHARMM [Article v1.0]. *Liv. J. Comput. Mol. Sci.*, **2022**, *4*, 1563.
- [693] R. Liu; Z. Yue; C.-C. Tsai; J. Shen. Assessing Lysine and Cysteine Reactivities for Designing Targeted Covalent Kinase Inhibitors. *J. Am. Chem. Soc.*, **2019**, *141*, 6553–6560.
- [694] R. C. Harris; R. Liu; J. Shen. Predicting Reactive Cysteines with Implicit-Solvent-Based Continuous Constant pH Molecular Dynamics in Amber. *J. Chem. Theory Comput.*, **2020**, *16*, 3689–3698.
- [695] B. M. Duggan; G. B. Legge; H. J. Dyson; P. E. Wright. SANE (Structure Assisted NOE Evaluation): An automated model-based approach for NOE assignment. *J. Biomol. NMR*, **2001**, *19*, 321–329.
- [696] A. Kalk; H. J. C. Berendsen. Proton magnetic relaxation and spin diffusion in proteins. *J. Magn. Reson.*, **1976**, *24*, 343–366.
- [697] E. T. Olejniczak; M. A. Weiss. Are methyl groups relaxation sinks in small proteins? *J. Magn. Reson.*, **1990**, *86*, 148–155.
- [698] K. J. Cross; P. E. Wright. Calibration of ring-current models for the heme ring. *J. Magn. Reson.*, **1985**, *64*, 220–231.
- [699] K. Ösapay; D. A. Case. A new analysis of proton chemical shifts in proteins. *J. Am. Chem. Soc.*, **1991**, *113*, 9436–9444.
- [700] D. A. Case. Calibration of ring-current effects in proteins and nucleic acids. *J. Biomol. NMR*, **1995**, *6*, 341–346.
- [701] L. Banci; I. Bertini; G. Gori-Savellini; A. Romagnoli; P. Turano; M. A. Cremonini; C. Luchinat; H. B. Gray. Pseudocontact shifts as constraints for energy minimization and molecular dynamics calculations on solution structures of paramagnetic metalloproteins. *Proteins*, **1997**, *29*, 68.
- [702] C. R. Sanders, II; B. J. Hare; K. P. Howard; J. H. Prestegard. Magnetically-oriented phospholipid micelles as a tool for the study of membrane-associated molecules. *Prog. NMR Spectr.*, **1994**, *26*, 421–444.
- [703] V. Tsui; L. Zhu; T. H. Huang; P. E. Wright; D. A. Case. Assessment of zinc finger orientations by residual dipolar coupling constants. *J. Biomol. NMR*, **2000**, *16*, 9–21.
- [704] D. A. Case. Calculations of NMR dipolar coupling strengths in model peptides. *J. Biomol. NMR*, **1999**, *15*, 95–102.
- [705] G. P. Gippert; P. F. Yip; P. E. Wright; D. A. Case. Computational methods for determining protein structures from NMR data. *Biochem. Pharm.*, **1990**, *40*, 15–22.
- [706] D. A. Case; P. E. Wright. in *NMR in Proteins*, G. M. Clore; A. Gronenborn, Eds., pp 53–91. MacMillan, New York, 1993.
- [707] D. A. Case; H. J. Dyson; P. E. Wright. Use of chemical shifts and coupling constants in nuclear magnetic resonance structural studies on peptides and proteins. *Meth. Enzymol.*, **1994**, *239*, 392–416.

BIBLIOGRAPHY

- [708] R. Brüschweiler; D. A. Case. Characterization of biomolecular structure and dynamics by NMR cross-relaxation. *Prog. NMR Spectr.*, **1994**, 26, 27–58.
- [709] D. A. Case. The use of chemical shifts and their anisotropies in biomolecular structure determination. *Curr. Opin. Struct. Biol.*, **1998**, 8, 624–630.
- [710] A. E. Torda; R. M. Scheek; W. F. VanGunsteren. Time-dependent distance restraints in molecular dynamics simulations. *Chem. Phys. Lett.*, **1989**, 157, 289–294.
- [711] D. A. Pearlman; P. A. Kollman. Are time-averaged restraints necessary for nuclear magnetic resonance refinement? A model study for DNA. *J. Mol. Biol.*, **1991**, 220, 457–479.
- [712] A. E. Torda; R. M. Brunne; T. Huber; H. Kessler; W. F. van Gunsteren. Structure refinement using time-averaged J-coupling constant restraints. *J. Biomol. NMR*, **1993**, 3, 55–66.
- [713] D. A. Pearlman. How well to time-averaged J-coupling restraints work? *J. Biomol. NMR*, **1994**, 4, 279–299.
- [714] D. A. Pearlman. How is an NMR structure best defined? An analysis of molecular dynamics distance-based approaches. *J. Biomol. NMR*, **1994**, 4, 1–16.
- [715] X. Wu; J. L. Milne; M. J. Borgnia; A. V. Rostapshov; S. Subramaniam; B. R. Brooks. A core-weighted fitting method for docking atomic structures into low-resolution maps: application to cryo-electron microscopy. *J Struct Biol*, **2003**, 141, 63–76.
- [716] J. L. Milne; X. Wu; M. J. Borgnia; J. S. Lengyel; B. R. Brooks; D. Shi; R. N. Perham; S. Subramaniam. Molecular structure of a 9-MDa icosahedral pyruvate dehydrogenase subcomplex containing the E2 and E3 enzymes using cryoelectron microscopy. *J Biol Chem*, **2006**, 281, 4364–70.
- [717] X. Wu; B. R. Brooks. Modeling of Macromolecular assemblies with map objects. *Proc. 2007 Int. Conf. Bioinform. Comput. Biol.*, **2007**, II, 411–417.
- [718] C. M. Khursigara; X. Wu; P. Zhang; J. Lefman; S. Subramaniam. Role of HAMP domains in chemotaxis signaling by bacterial chemoreceptors. *PNAS*, **2008**, 105, 16555–60.
- [719] J. S. Lengyel; K. M. Stott; X. Wu; A. Brooks, B. R. and Balbo; P. Schuck; R. N. Perham; S. Subramaniam; J. L. Milne. Extended polypeptide linkers establish the spatial architecture of a pyruvate dehydrogenase multienzyme complex. *Structure*, **2008**, 16, 93–103.
- [720] C. M. Khursigara; X. Wu; ; S. Subramaniam. Chemoreceptors in *Caulobacter crescentus*: trimers of receptor dimers in a partially ordered hexagonally packed array. *J Bacteriol*, **2008**, 190, 6805–10.
- [721] J. Elegheert; A. Desfosses; A. V. Shkumatov; X. Wu; N. Bracke; K. Verstraete; K. Van Craenenbroeck; B. R. Brooks; D. I. Svergun; B. Vergauwen; I. Gutsche; S. N. Savvides. Extracellular complexes of the hematopoietic human and mouse CSF-1 receptor are driven by common assembly principles . *Structure*, **2011**, 19, 1762–72.
- [722] C. M. Khursigara; G. Lan; S. Neumann; X. Wu; S. Ravindran; M. J. Borgnia; V. Sourjik; J. Milne; Y. Tu; S. Subramaniam. Lateral density of receptor arrays in the membrane plane influences sensitivity of the *E. coli* chemotaxis response. *Embo J*, **2011**, 30, 1719–29.
- [723] X. Wu; B. R. Brooks. in *Microscopy: advances in scientific research and education*, A. Mendez-Vilas, Ed., pp 39–47. Formatex Research Center, Spain, 2014.
- [724] X. Wu; B. R. Brooks. in *Modern electron microscopy in physical and life science*, M. Janacek, Ed., chapter 12, pp 243–262. InTech, 2016.
- [725] A. Bartesaghi; A. Merk; S. Banerjee; D. Matthies; X. Wu; J. L. S. Milne; S. Subramaniam. 2.2 Å resolution cryo-em structure of beta-galactosidase in complex with a cell-permeant inhibitor. *Science*, **2015**, 348, 1147–1151.

- [726] S. Kalinin; T. Peulen; S. Sindbert; P. J. Rothwell; S. Berger; T. Restle; R. S. Goody; H. Gohlke; C. A. Seidel. A toolkit and benchmark study for fret-restrained high-precision structural modeling. *Nature Methods*, **2012**, 9, 1218–1225.
- [727] M. Dimura; T. O. Peulen; C. A. Hanke; A. Prakash; H. Gohlke; C. A. Seidel. Quantitative fret studies and integrative modeling unravel the structure and dynamics of biomolecular systems. *Curr. Opin. Struct. Biol.*, **2016**, 40, 163–185.
- [728] M. Dimura; T. O. Peulen; H. Sanabria; D. Rodnin; K. Hemmen; C. A. Seidel; H. Gohlke. Automated and optimally fret-assisted structural modeling. *Nature Commun.*, **2019**, 11, 5394.
- [729] R. T. McGibbon; K. A. Beauchamp; M. P. Harrigan; C. Klein; J. M. Swails; C. X. Hernandez; C. R. Schwantes; L.-P. Wang; T. J. Lane; V. S. Pande. Mdtraj: A modern open library for the analysis of molecular dynamics trajectories. *Biophys. J.*, **2015**, 109, 1528–1532.
- [730] P. A. Janowski; D. S. Cerutti; J. M. Holton; D. A. Case. Peptide crystal simulations reveal hidden dynamics. *J. Am. Chem. Soc.*, **2013**, 135, 7938–7948.
- [731] N. Moriarty; P. Janowski; J. Swails; H. Nguyen; J. Richardson; D. Case; P. Adams. Improved chemistry restraints for crystallographic refinement by integrating the Amber force field into Phenix. *Acta Cryst. D*, **2020**, 76, 51–62.
- [732] P. Afonine; A. Urzhumtsev. On a fast calculation of structure factors at a subatomic resolution. *Acta Cryst. A*, **2004**, 60, 19–32.
- [733] J. Jiang; A. Brünger. Protein hydration observed by X-ray diffraction: solvation properties of penicillopepsin and neuraminidase crystal structures. *J. Mol. Biol.*, **1994**, 243, 100–115.
- [734] A. Fokine; A. Urzhumtsev. Flat bulk-solvent model: obtaining optimal parameters. *Acta Cryst. D*, **2002**, 58, 1387–1392.
- [735] R. Grosse-Kunstleve; N. Sauter; N. Moriarty; P. Adams. The Computational Crystallography Toolbox: crystallographic algorithms in a reusable software framework. *J. Appl. Crystallogr.*, **2002**, 35, 126–136.
- [736] P. Afonine; R. Grosse-Kunstleve; P. Adams; A. Urzhumtsev. Bulk-solvent and overall scaling revisited: faster calculations, improved results. *Acta Cryst. D*, **2013**, 69, 625–634.
- [737] V. Lunin; T. Skovoroda. R-free likelihood-based estimates of errors for phases calculated from atomic models. *Acta Cryst. A*, **1995**, 51, 880–887.
- [738] P. V. Afonine; R. W. Grosse-Kunstleve; P. D. Adams. A robust bulk-solvent correction and anisotropic scaling procedure. *Acta Cryst. D*, **2005**, 61, 850–855.
- [739] Y. Xue; N. Skrynnikov. Ensemble MD simulations restrained via crystallographic data: accurate structure leads to accurate dynamics. *Prot. Sci.*, **2014**, 23, 488–507.
- [740] A. Raval; S. Piana; M. Eastwood; R. Dror; D. Shaw. Refinement of protein structure homology models via long, all-atom molecular dynamics simulations. *Proteins*, **2012**, 80, 2071–2079.
- [741] O. Mikhailovskii; Y. Xue; N. R. Skrynnikov. Modeling a unit cell: crystallographic refinement procedure using the biomolecular MD simulation platform Amber. *IUCrJ*, **2022**, 9, 114–133.
- [742] A. Roitberg; R. Elber. Modeling side chains in peptides and proteins: Application of the locally enhanced sampling and the simulated annealing methods to find minimum energy conformations. *J. Chem. Phys.*, **1991**, 95, 9277.
- [743] A. Ulitsky; R. Elber. The thermal equilibrium aspects of the time-dependent Hartree and the locally enhanced sampling approximations: Formal properties, a correction, and computational examples for rare gas clusters. *J. Chem. Phys.*, **1993**, 98, 3380.
- [744] C. Simmerling; T. Fox; P. A. Kollman. Use of Locally Enhanced Sampling in Free Energy Calculations: Testing and Application of the alpha to beta Anomerization of Glucose. *J. Am. Chem. Soc.*, **1998**, 120, 5771–5782.

BIBLIOGRAPHY

- [745] C. Simmerling; J. L. Miller; P. A. Kollman. Combined locally enhanced sampling and particle mesh Ewald as a strategy to locate the experimental structure of a nonhelical nucleic acid. *J. Am. Chem. Soc.*, **1998**, *120*, 7149–7155.
- [746] A. Miranker; M. Karplus. Functionality maps of binding sites: A multiple copy simultaneous search method. *Proteins: Str. Funct. Gen.*, **1991**, *11*, 29–34.
- [747] X. Cheng; V. Hornak; C. Simmerling. Improved conformational sampling through an efficient combination of mean-field simulation approaches. *J. Phys. Chem. B*, **2004**, *108*.
- [748] J. E. Straub; M. Karplus. Enery partitioning in the classical time-dependent Hartree approximation. *J. Chem. Phys.*, **1991**, *94*, 6737.
- [749] P. Y. Ren; J. W. Ponder. Polarizable atomic multipole water model for molecular mechanics simulation. *J. Phys. Chem. B*, **2003**, *107*, 5933–5947.
- [750] P. Y. Ren; J. W. Ponder. Tinker polarizable atomic multipole force field for proteins. *to be published.*, **2006**.
- [751] O. N. Starovoytov; H. Torabifard; G. A. Cisneros. Development of amoeba force field for 1,3-dimethylimidazolium based ionic liquids. *J. Phys. Chem. B*, **2014**, *118*, 7156–7166.
- [752] H. Torabifard; O. N. Starovoytov; P. Ren; G. A. Cisneros. Development of an amoeba water model using gem distributed multipoles. *Theo. Chem. Acc.*, **2015**, *134*, 1–10.
- [753] H. Torabifard; L. Reed; M. T. Berry; J. E. Hein; E. Menke; G. A. Cisneros. Computational and experimental characterization of a pyrrolidinium-based ionic liquid for electrolyte applications. *J. Chem. Phys.*, **2017**, *147*, 161731.
- [754] Y.-J. Tu; M. J. Allen; G. A. Cisneros. Simulations of the water exchange dynamics of lanthanide ions in 1-ethyl-3-methylimidazolium ethyl sulfate ([emim][etso4]) and water. *Phys. Chem. Chem. Phys.*, **2016**, *18*, 30323–30333.
- [755] Y.-J. Tu; Z. Lin; M. J. Allen; G. A. Cisneros. Molecular dynamics investigation of water-exchange reactions on lanthanide ions in water/1-ethyl-3-methylimidazolium trifluoromethylsulfate ([emim][otf]). *J. Chem. Phys.*, **2018**, *148*, 024503.
- [756] G. A. Cisneros; J.-P. Piquemal; T. A. Darden. Generalization of the gaussian electrostatic model: extension to arbitrary angular momentum, distributed multipoles and computational speedup with reciprocal space methods. *J. Chem. Phys.*, **2006**, *125*, 184101.
- [757] J.-P. Piquemal; G. A. Cisneros; P. Reinhardt; N. Gresh; T. A. Darden. Towards a force field based on density fitting. *J. Chem. Phys.*, **2006**, *124*, 104101.
- [758] J.-P. Piquemal; G. Cisneros. in *Many-body effects and electrostatics in multi-scale computations of Biomolecules*, Q. Cui; P. Ren; M. Meuwly, Eds., pp 269–300. Pan Stanford Publishing, 2015.
- [759] R. E. Duke; O. N. Starovoytov; J.-. Piquemal; G. A. Cisneros. Gem*: A molecular electronic density-based force field for molecular dynamics simulations. *J. Chem. Theo. Comput.*, **2014**, *10*, 1361–1365.
- [760] J. S. Smith; B. T. Nebgen; R. Zubatyuk; N. Lubbers; C. Devereux; K. Barros; S. Tretiak; O. Isayev; A. E. Roitberg. Approaching coupled cluster accuracy with a general-purpose neural network potential through transfer learning. *Nat. Commun.*, **2019**, *10*.
- [761] J. Moody; C. J. Darken. Fast Learning in Networks of Locally-Tuned Processing Units. *Neural Comput.*, **1989**, *1*, 281–294.
- [762] J. T. Berryman; A. Taghavi; F. Mazur; A. Tkatchenko. Quantum Machine Learning Corrects Classical Force Fields: Stretching DNA Base Pairs in Explicit Solvent. *J. Chem. Phys.*, **2022**. Draft with JCP at time of writing, preprint ArXiv id is: 2203.15525.
- [763] R. Brown; D. Case. Second derivatives in generalized Born theory. *J. Comput. Chem.*, **2006**, *27*, 1662–1675.

- [764] D. R. Roe; T. E. Cheatham, III. PTRAJ and CPPTRAJ: Software for Processing and Analysis of Molecular Dynamics Trajectory Data. *J. Chem. Theory Comput.*, **2013**, 9, 3084–3095.
- [765] D. R. Roe; T. E. Cheatham, III. Parallelization of CPPTRAJ enables large scale analysis of molecular dynamics trajectory data. *J. Computat. Chem.*, **2018**, 39, 2110–2117.
- [766] D. R. Roe; C. Bergonzo. PrepareForLeap: An Automated Tool for Fast PDB-to-Parameter Generation. *J. Comput. Chem.*, **2022**, 43, 930–935.
- [767] M. E. O'Neill. PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, Claremont, CA, 2014.
- [768] S. Vigna. Further scramblings of Marsaglia's xorshift generators. *J. Comput. Appl. Math.*, **2017**, 315, 175–181.
- [769] D. R. Roe; B. R. Brooks. Quantifying the Effects of Lossy Compression on Energies Calculated from Molecular Dynamics Trajectories. *Protein Science*, **2022**.
- [770] S. Chatterjee; P. G. Debenedetti; F. H. Stillinger; R. M. Lynden-Bell. A Computational Investigation of Thermodynamics, Structure, Dynamics and Solvation Behavior in Modified Water Models. *J. Chem. Phys.*, **2008**, 128, 124511.
- [771] T. Lazaridis. Inhomogeneous Fluid Approach to Solvation Thermodynamics. 1 Theory. *J. Phys. Chem. B*, **1998**, 102, 3531–3541.
- [772] C. N. Nguyen; T. Kurtzman Young; M. K. Gilson. Grid Inhomogeneous Solvation Theory: Hydration Structure and Thermodynamics of the Miniature Receptor cucurbit[7]uril. *J. Chem. Phys.*, **2012**, 137, 044101.
- [773] L. Chen; A. Cruz; D. R. Roe; A. C. Simmonett; L. Wickstrom; N. Deng; T. Kurtzman. Thermodynamic Decomposition of Solvation Free Energies with Particle Mesh Ewald and Long-Range Lennard-Jones Interactions in Grid Inhomogeneous Solvation Theory. *J. Chem. Theory Comput.*, **2021**, 17, 2714–2724.
- [774] W. Humphrey; A. Dalke; K. Schulten. VMD Visual Molecular Dynamics. *J. Molec. Graph.*, **1996**, 14, 33–38.
- [775] D. J. Sindhikara; N. Yoshida; F. Hirata. Placevent: An Algorithm for Prediction of Explicit Solvent Atom Distribution-Application to HIV-1 Protease and F-ATP Synthase. *J. Comput. Chem.*, **2012**, 33, 1536–1543.
- [776] J. J. Chou; D. A. Case; A. Bax. Insights into the mobility of methyl-bearing side chains in proteins. *J. Am. Chem. Soc.*, **2003**, 125, 8959–8966.
- [777] C. Perez; F. Lohr; H. Ruterjans; J. M. Schmidt. Self-Consistent Karplus Parameterization of (^3J) couplings depending on the polypeptide side-chain torsion $\chi(1)$. *J. Am. Chem. Soc.*, **2001**, 123, 7081–7093.
- [778] P. H. Hunenberger; A. E. Mark; W. F. van Gunsteren. Fluctuation and Cross-correlation Analysis of Protein Motions Observed in Nanosecond Molecular Dynamics Simulations. *J. Mol. Biol.*, **1995**, 252, 492–503.
- [779] J. J. Prompers; R. Brüschweiler. Dynamic and structural analysis of isotropically distributed molecular ensembles. *Proteins*, **2002**, 46, 177–189.
- [780] J. J. Prompers; R. Brüschweiler. General framework for studying the dynamics of folded and nonfolded proteins by NMR relaxation spectroscopy and MD simulation. *J. Am. Chem. Soc.*, **2002**, 124, 4522–4534.
- [781] M. L. Connolly. Analytical molecular surface calculation. *J. Appl. Cryst.*, **1983**, 16, 548–558.
- [782] X. Lu; W. Olson. 3dna: a software package for the analysis, rebuilding and visualization of three-dimensional nucleic acid structures. *NUCLEIC ACIDS RESEARCH*, **2003**, 31, 5108–5121.
- [783] M. S. Babcock; E. P. D. Pednault; W. K. Olson. Nucleic Acid Structure Analysis. *J. Mol. Biol.*, **1994**, 237, 125–156.

BIBLIOGRAPHY

- [784] M. A. E. Hassan; C. R. Calladine. Two distinct modes of protein-induced bending in dna. *J. Mol. Biol.*, **1998**, 282, 331–343.
- [785] W. K. Olson; M. Bansal; S. K. Burley; R. E. Dickerson; M. Gerstein; S. C. Harvey; U. Heinemann; X.-J. Lu; S. Neidle; Z. Shakked; H. Sklenar; M. Suzuki; C.-S. Tung; E. Westhof; C. Wolberger; H. M. Berman. A standard reference frame for the description of nucleic acid base-pair geometry. *J. Mol. Biol.*, **2001**, 313, 229–237.
- [786] C. Altona; M. Sundaralingam. Conformational analysis of the sugar ring in nucleosides and nucleotides. a new description using the concept of pseudorotation. *J Am Chem Soc*, **1972**, 94, 8205–8212.
- [787] S. Harvey; M. Prabhakaran. Ribose puckering - structure, dynamics, energetics, and the pseudorotation cycle. *J Am Chem Soc*, **1986**, 108, 6128–6136.
- [788] D. Cremer; J. Pople. A general definition of ring puckering coordinates. *J Am Chem Soc*, **1975**, 97, 1354–1358.
- [789] W. Kabsch; C. Sander. Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, **1983**, 22, 2577–2637.
- [790] G. Cui; J. M. Swails; E. S. Manas. SPAM: A Simple Approach for Profiling Bound Water Molecules. *J. Chem. Theory Comput.*, **2013**, 9, 5539–5549.
- [791] D. R. Roe; B. R. Brooks. Improving the Speed of Volumetric Density Map Generation via Cubic Spline Interpolation. *J. Mol. Graphics Model.*, **2021**, 104, 107832.
- [792] M. Ester; H. Kriegel; J. Sander; X. Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. *Proc. Second Int. Conf. Knowledge Disc. Data Mining (KDD-96)*, **1996**, pp 226–231.
- [793] A. Rodriguez; A. Laio. Clustering by fast search and find of density peaks. *Science*, **2014**, 344, 1492–1496.
- [794] A. Bakan; L. M. Meireles; I. Bahar. ProDy: Protein Dynamics Inferred from Theory and Experiments. *Bioinformatics*, **2011**, 27, 1575–1577.
- [795] D. A. McQuarrie. *Statistical Thermodynamics*. Harper and Row, New York, 1973.
- [796] C.-E. Chang; W. Chen; M. K. Gilson. Evaluating the Accuracy of the Quasiharmonic Approximation. *J. Chem. Theory Comput.*, **2005**, 1, 1017–1028. PMID: 26641917.
- [797] D. R. Roe; B. R. Brooks. A protocol for preparing explicitly solvated systems for stable molecular dynamics simulations. *J. Chem. Phys.*, **2020**, 153, 054123.
- [798] D. R. Roe; C. Bergonzo; T. E. Cheatham, III. Evaluation of enhanced sampling provided by accelerated molecular dynamics with hamiltonian replica exchange methods. *J. Phys. Chem. B*, **2014**, 118, 3543–3552.
- [799] R. Galindo-Murillo; D. R. Roe; T. E. Cheatham, III. Convergence and reproducibility in molecular dynamics simulations of the DNA duplex d(GCACGAACGAACGAACGC). *Biochim. Biophys. Acta*, **2015**, 1850, 1041–1058.
- [800] V. Wong; D. A. Case. Evaluating rotational diffusion from protein md simulations. *J. Phys. Chem. B*, **2008**, 112, 6013–6024.
- [801] B. Schneider; S. Neidle; H. M. Berman. Conformations of the sugar-phosphate backbone in helical dna crystal structures. *Biopolymers*, **1997**, 42, 113–124.
- [802] B. Schneider; Z. Moravsek; H. M. Berman. Rna conformational classes. *Nucleic Acids Res.*, **2004**, 32, 1666–1677.
- [803] F. Noé; R. Banisch; C. Clementi. Commute maps: Separating slowly mixing molecular configurations for kinetic modeling. *J. Chem. Theory Comput.*, **2016**, 12, 5620–5630.
- [804] G. Pérez-Hernández; F. Paul; T. Giorgino; G. De Fabritiis; F. Noé. Identification of slow molecular order parameters for Markov model construction. *J. Chem. Phys.*, **2013**, 139, 015102.

- [805] C. Torrence; G. P. Compo. A practical guide to wavelet analysis. *Bull. Am. Meteorol. Soc.*, **1998**, 79, 61–78.
- [806] N. C. Benson; V. Dagget. Wavelet analysis of protein motion. *Int. J. Wavelets Multi.*, **2012**, 10.
- [807] Z. Heidari; D. R. Roe; R. Galindo-Murillo; J. B. Ghasemi; T. E. Cheatham, III. Using Wavelet Analysis To Assist in Identification of Significant Events in Molecular Dynamics Simulations. *J. Chem. Inf. Model.*, **2016**, 56, 1282–1291.
- [808] H. Nguyen; D. R. Roe; J. M. Swails; D. A. Case. PYTRAJ: Interactive data analysis for molecular dynamics simulations. *Manuscript in preparation*, **2016**.
- [809] H. Nguyen; D. A. Case; A. S. Rose. Nglview—interactive molecular graphics for jupyter notebooks. *Bioinformatics*, **2017**, 34, 1241–1242.
- [810] B. R. Miller; T. D. McGee; J. M. Swails; N. Homeyer; H. Gohlke; A. E. Roitberg. MMPBSA.py: An Efficient Program for End-State Free Energy Calculations. *J. Chem. Theory Comput.*, **2012**, 8, 3314–3321.
- [811] J. Srinivasan; T. E. Cheatham, III; P. Cieplak; P. Kollman; D. A. Case. Continuum solvent studies of the stability of DNA, RNA, and phosphoramidate–DNA helices. *J. Am. Chem. Soc.*, **1998**, 120, 9401–9409.
- [812] P. A. Kollman; I. Massova; C. Reyes; B. Kuhn; S. Huo; L. Chong; M. Lee; T. Lee; Y. Duan; W. Wang; O. Donini; P. Cieplak; J. Srinivasan; D. A. Case; T. E. Cheatham, III. Calculating structures and free energies of complex molecules: Combining molecular mechanics and continuum models. *Accts. Chem. Res.*, **2000**, 33, 889–897.
- [813] N. Homeyer; H. Gohlke. Free energy calculations by the molecular mechanics poisson-boltzmann surface area method. *Mol. Informatics*, **2012**, DOI: 10.1002/minf.201100135.
- [814] E. Wang; H. Sun; J. Wang; Z. Wang; H. Liu; J. Zhang; T. Hou. End-Point Binding Free Energy Calculation with MM/PBSA and MM/GBSA: Strategies and Applications in Drug Design. *Chem. Rev.*, **2019**, 119, 9478–9508.
- [815] W. Wang; P. Kollman. Free energy calculations on dimer stability of the HIV protease using molecular dynamics and a continuum solvent model. *J. Mol. Biol.*, **2000**, 303, 567.
- [816] C. Reyes; P. Kollman. Structure and thermodynamics of RNA-protein binding: Using molecular dynamics and free energy analyses to calculate the free energies of binding and conformational change. *J. Mol. Biol.*, **2000**, 297, 1145–1158.
- [817] M. R. Lee; Y. Duan; P. A. Kollman. Use of MM-PB/SA in estimating the free energies of proteins: Application to native, intermediates, and unfolded vilin headpiece. *Proteins*, **2000**, 39, 309–316.
- [818] J. Wang; P. Morin; W. Wang; P. A. Kollman. Use of MM-PBSA in reproducing the binding free energies to HIV-1 RT of TIBO derivatives and predicting the binding mode to HIV-1 RT of efavirenz by docking and MM-PBSA. *J. Am. Chem. Soc.*, **2001**, 123, 5221–5230.
- [819] L. Marinelli; S. Cosconati; T. Steinbrecher; V. Limongelli; A. Bertamino; E. Novellino; D. A. Case. Homology Modeling of NR2B Modulatory Domain of NMDA Receptor and Analysis of Ifenprodil Binding. *ChemMedChem*, **2007**, 2, 1498–1510.
- [820] N. Homeyer; H. Gohlke. FEW - A workflow tool for free energy calculations of ligand binding. *J. Comput. Chem.*, **2013**, 34, 965–973.
- [821] S. Jo; T. Kim; W. Im. Automated builder and database of protein/membrane complexes for molecular dynamics simulations. *PLoS One*, **2007**, 2, e880.
- [822] S. Jo; T. Kim; V. G. Iyer; I. W. CHARMM-GUI: a web-based graphical user interface for CHARMM. *J. Comput. Chem.*, **2008**, 29, 1859–1865.
- [823] S. Jo; J. B. Lim; J. B. Klauda; W. Im. CHARMM-GUI Membrane Builder for mixed bilayers and its application to yeast membranes. *Biophys. J.*, **2009**, 97, 50–58.

BIBLIOGRAPHY

- [824] E. L. Wu; X. Cheng; S. Jo; H. Rui; K. C. Song; E. M. Davila-Contreras; Y. Qi; J. Lee; V. Monje-Galvan; R. M. Venable; J. B. Klauda; I. W. CHARM-GUI Membrane Builder toward realistic biological membrane simulations. *J. Comput. Chem.*, **2014**, *35*, 1997–2004.
- [825] J. Domanski; P. Stansfeld; M. S. P. Sansom; O. Beckstein. Lipidbook: A Public Repository for Force Field Parameters Used in Membrane Simulations. *J. Membrane Biol.*, **2010**, *236*, 255–258.
- [826] S. Izadi; B. Aguilar; A. V. Onufriev. Protein-ligand electrostatic binding free energies from explicit and implicit solvation. *J. Chem. Theory Comput.*, **2015**, *11*, 4450–4459.
- [827] A. Metz. *Goethe University (Frankfurt am Main)*, **2006**.
- [828] N. A. Baker; D. Sept; J. Simpson; M. J. Holst; M. J. A. Electrostatics of nanosystems: application to microtubules and the ribosome. *Proc. Natl. Acad. Sci. U. S. A.*, **2001**, *98*, 10037–10041.
- [829] M. Holst; F. Saied. Multigrid solution of the Poisson-Boltzmann equation. *J. Comput. Chem.*, **1993**, *14*, 105–113.
- [830] M. Holst; F. Saied. Numerical solution of the nonlinear Poisson-Boltzmann equation: Developing more robust and efficient methods. *J. Comput. Chem.*, **1995**, *16*, 337–364.
- [831] M. Holst. Adaptive numerical treatment of elliptic systems on manifolds. *Adv. Comput. Math.*, **2001**, *15*, 139–191.
- [832] R. Bank; M. Holst. A New Paradigm for Parallel Adaptive Meshing Algorithms. *SIAM Review*, **2003**, *45*, 291–323.
- [833] K. M. Callenberg; O. P. Choudhary; G. L. de Forest; D. W. Gohara; N. A. Baker; M. Grabe. APBSmem: A graphical interface for electrostatic calculations at the membrane. *PLoS One*, **2010**, *5*, e12722.
- [834] M. Grabe; H. Lecar; Y. N. Jan; J. L. Y. A quantitative assessment of models for voltage-dependent gating of ion channels. *Proc. Natl. Acad. Sci. U. S. A.*, **2004**, *101*, 17640–17645.
- [835] N. Homeyer; H. Gohlke. Extension of the free energy workflow FEW towards implicit solvent/implicit membrane MM-PBSA calculations. *BBA - Gen. Subjects*, **2015**, *1850*, 972–982.
- [836] H. Nymeyer; H. X. Zhou. A method to determine dielectric constants in nonhomogeneous systems: application to biological membranes. *Biophys. J.*, **2008**, *94*, 1185–1193.
- [837] H. A. Stern; S. E. Feller. Calculation of the dielectric permittivity profile for a nonuniform system: application to a lipid bilayer simulation. *J. Chem. Phys.*, **2003**, *118*, 3401–3412.
- [838] L. Waeschenbach; C. G. W. Gertzen; V. Keitel; H. Gohlke. Dimerization energetics of the G-protein coupled bile acid receptor TGR5 from all-atom simulations. *J. Comput. Chem.*, **2019**, *41*.
- [839] J. Åqvist; C. Medina; J. E. Samuelsson. A new method for predicting binding affinity in computer-aided drug design. *Protein Eng.*, **1994**, *7*, 385–391.
- [840] J. Åqvist; V. B. Luzhkov; B. O. Brandsdal. Ligand binding affinities from MD simulations. *Acc. Chem. Res.*, **2002**, *35*, 358–365.
- [841] M. M. van Lipzig; A. M. ter Laak; A. Jongejan; N. P. Vermeulen; M. Wamelink; D. Geerke; J. H. Meerman. Prediction of ligand binding affinity and orientation of xenoestrogens to the estrogen receptor by molecular dynamics simulations and the linear interaction energy method. *J. Med. Chem.*, **2004**, *47*, 1018–1030.
- [842] H. G. Wallnoefer; K. R. Liedl; T. Fox. A challenging system: free energy prediction for factor Xa. *J. Comput. Chem.*, **2011**, *32*, 1743–1752.
- [843] B. O. Brandsdal; F. Österberg; M. Almlöf; I. Feierberg; V. B. Luzhkov; Åqvist, J. Free energy calculations and ligand binding. *Adv. Protein Chem.*, **2003**, *66*, 123–158.
- [844] W. Wang; J. Wang; P. A. Kollman. What determines the van der Waals coefficient beta in the LIE (linear interaction energy) method to estimate binding free energies using molecular dynamics simulations? *Proteins: Struct., Funct., Genet.*, **1999**, *34*, 395–402.

- [845] D. K. Jones-Hertzog; W. L. Jorgensen. Binding affinities for sulfonamide inhibitors with human thrombin using Monte Carlo simulations with a linear response method. *J. Med. Chem.*, **1997**, *40*, 1539–1549.
- [846] M. L. Lamb; J. Tirado-Rives; W. L. Jorgensen. Estimation of the binding affinities of FKBP12 inhibitors using a linear response method. *Bioorg. Med. Chem.*, **1999**, *7*, 851–860.
- [847] W. Yang; R. Bitetti-Putzer; K. M. Free energy simulations: Use of reverse cumulative averaging to determine the equilibrated region and the time required for convergence. *J. Chem. Phys.*, **2004**, *120*, 2618–2628.
- [848] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proc. Amer. Math. Soc.*, **1956**, *7*, 48–50.
- [849] *ROCS, OpenEye Scientific Software, Santa Fe*, <http://www.eyesopen.com>.
- [850] P. C. D. Hawkins; A. G. Skillman; A. Nicholls. Comparison of shape-matching and docking as virtual screening tools. *J. Med. Chem.*, **2007**, *50*, 74–82.
- [851] C. Wang; P. Ren; R. Luo. Ionic solution: What goes right and wrong with continuum solvation modeling. *J. Phys. Chem. B*, **2017**, *121*, 11169.
- [852] E. King; R. Qi; H. Li; R. Luo; E. Aitchison. Estimating the roles of protonation and electronic polarization in absolute binding affinity simulations. *J. Chem. Theory Comput.*, **2021**, *17*, 0000.
- [853] T. J. Giese; D. M. York. Variational Method for Networkwide Analysis of Relative Ligand Binding Free Energies with Loop Closure and Experimental Constraints. *J. Chem. Theory Comput.*, **2021**, *17*, 1326–1336.
- [854] S. Zhang; T. J. Giese; T.-S. Lee; D. M. York. Alchemical Enhanced Sampling with Optimized Phase Space Overlap. *J. Chem. Theory Comput.*, **2024**, *20*, 3935–3953.
- [855] S. Park; J. P. Bardhan; B. Roux; L. Makowski. Simulated X-ray scattering of protein solutions using explicit-solvent models. *J. Chem. Phys.*, **2009**, *130*, 134114.
- [856] H. T. Nguyen; S. A. Pabit; S. P. Meisburger; L. Pollack; D. A. Case. Accurate small and wide angle X-ray scattering profiles from atomic models of proteins and nucleic acids. *J. Chem. Phys.*, **2014**, *141*, 22D508.
- [857] G. M. Giambasu; M. K. Gebala; M. T. Panteva; T. Luchko; D. A. Case; D. M. York. Competitive Interaction of Monovalent Cations with DNA from 3D-RISM. *Nucleic Acids Res.*, **2015**, *43*, 8405–8415.
- [858] T. J. Giese; Solen Ekesan; D. M. York. Extension of the Variational Free Energy Profile and Multistate Bennett Acceptance Ratio Methods for High-Dimensional Potential of Mean Force Profile Analysis. *J. Phys. Chem. A*, **2021**, *125*, 4216–4232.
- [859] T.-S. Lee; B. K. Radak; A. Pabis; D. M. York. A new maximum likelihood approach for free energy profile construction from molecular simulations. *J. Chem. Theory Comput.*, **2013**, *9*, 153–164.
- [860] T.-S. Lee; B. K. Radak; M. Huang; K.-Y. Wong; D. M. York. Roadmaps through free energy landscapes calculated using the multidimensional vFEP approach. *J. Chem. Theory Comput.*, **2014**, *10*, 24–34.
- [861] P. Li; X. Jia; X. Pan; Y. Shao; Y. Mei. Accelerated Computation of Free Energy Profile at ab Initio Quantum Mechanical/Molecular Mechanics Accuracy via a Semi-Empirical Reference Potential. I. Weighted Thermodynamics Perturbation. *J. Chem. Theory Comput.*, **2018**, *14*, 5583–5596.
- [862] W. Hu; P. Li; J.-N. Wang; Y. Xue; Y. Mo; J. Zheng; X. Pan; Y. Shao; Y. Mei. Accelerated Computation of Free Energy Profile at Ab Initio Quantum Mechanical/Molecular Mechanics Accuracy via a Semiempirical Reference Potential. 3. Gaussian Smoothing on Density-of-States. *J. Chem. Theory Comput.*, **2020**, *16*, 6814–6822.
- [863] T. J. Giese; J. Zeng; D. M. York. Multireference Generalization of the Weighted Thermodynamic Perturbation Method. *J. Phys. Chem. A*, **2022**, *126*, 8519–8533.

BIBLIOGRAPHY

- [864] E. Rosta; M. Nowotny; W. Yang; G. Hummer. Catalytic Mechanism of RNA Backbone Cleavage by Ribonuclease H from Quantum Mechanics/Molecular mechanics simulations. *J. Am. Chem. Soc.*, **2011**, 133, 8934–8941.
- [865] J. I. Mendieta-Moreno; R. C. Walker; J. P. Lewis; P. Gómez-Puertas; J. Mendieta; J. Ortega. FIREBALL/AMBER: An efficient local-orbital DFT QM/MM method for biomolecular systems. *J. Chem. Theory Comput.*, **2014**, 10, 2185–2193.
- [866] B. E. Hingerty; S. Figueroa; T. L. Hayden; S. Broyde. Prediction of DNA Structure from Sequence: A Build-up Technique. *Biopolymers*, **1989**, 28, 1195–1222.
- [867] D. A. Erie; K. J. Breslauer; W. K. Olson. A Monte Carlo Method for Generating Structures of Short Single-Stranded DNA Sequences. *Biopolymers*, **1993**, 33, 75–105.
- [868] D. A. Pearlman; S.-H. Kim. Conformational Studies of Nucleic Acids I. A Rapid and Direct Method for Generating Coordinates from the Pseudorotation Angle. *J. Biomol. Struct. Dyn.*, **1985**, 3, 85–98.
- [869] D. A. Pearlman; S.-H. Kim. Conformational Studies of Nucleic Acids II. The Conformational Energetics of Commonly Occurring Nucleosides. *J. Biomol. Struct. Dyn.*, **1985**, 3, 99–125.
- [870] D. A. Pearlman; S.-H. Kim. Conformational Studies of Nucleic Acids: III. Empirical Multiple Correlation Functions for Nucleic Acid Torsion Angles. *J. Biomol. Struct. Dyn.*, **1986**, 4, 49–67.
- [871] D. A. Pearlman; S.-H. Kim. Conformational Studies of Nucleic Acids: IV. The Conformational Energetics of Oligonucleotides: d(ApApApA) and ApApApA. *J. Biomol. Struct. Dyn.*, **1986**, 4, 69–98.
- [872] D. A. Pearlman; S.-H. Kim. Conformational Studies of Nucleic Acids. V. Sequence Specificities of in the Conformational Energetics of Oligonucleotides: The Homo-Tetramers. *Biopolymers*, **1988**, 27, 59–77.
- [873] T. Schlick. A Modular Strategy for Generating Starting Conformations and Data Structures of Polynucleotide Helices for Potential Energy Calculations. *J. Comput. Chem.*, **1988**, 9, 861–889.
- [874] J. Gabarro-Arpa; J. A. H. Cognet; M. Le Bret. Object Command Language: a formalism to build molecule models and to analyze structural parameters in macromolecules, with applications to nucleic acids. *J. Mol. Graph.*, **1992**, 10, 166–173.
- [875] R. Lavery. in *Unusual DNA Structures*, R. D. Wells; S. C. Harvey, Eds. Springer-Verlag, New York, 1988.
- [876] L. Shen; I. Tinoco. The Structure of an RNA Pseudoknot that Causes Efficient Frameshift in Mouse Mammary Tumor Virus. *J. Mol. Biol.*, **1995**, 247, 963–978.
- [877] J. M. Hubbard; J. E. Hearst. Predicting the Three-Dimensional Folding of Transfer RNA with a Computer Modeling Protocol. *Biochemistry*, **1991**, 30, 5458–5465.
- [878] S.-H. Chou; L. Zhu; B. R. Reid. The Unusual Structure of the Human Centromere (GGA)₂ Motif. *J. Mol. Biol.*, **1994**, 244, 259–268.
- [879] M. Levitt. Detailed Molecular Model for Transfer Ribonucleic Acid. *Nature*, **1969**, 224, 759–763.
- [880] J. M. Hubbard; J. E. Hearst. Computer Modeling 16S Ribosomal RNA. *J. Mol. Biol.*, **1991**, 221, 889–907.
- [881] F. Major; M. Turcotte; D. Gautheret; G. Lapalme; E. Fillon; R. Cedergren. The Combination of Symbolic and Numerical Computation for Three-Dimensional Modeling of RNA. *Science*, **1991**, 253, 1255–1260.
- [882] T. Schlick; W. K. Olson. Supercoiled DNA Energetics and Dynamics by Computer Simulation. *J. Mol. Biol.*, **1992**, 223, 1089–1119.
- [883] R. E. Dickerson. Definitions and Nomenclature of Nucleic Acid Structure Parameters. *J. Biomol. Struct. Dyn.*, **1989**, 6, 627–634.

- [884] S. R. Holbrook; J. L. Sussman; R. W. Warrant; S.-H. Kim. Crystal Structure of Yeast Phenylalanine Transfer RNA II. Structural Features and and Functional Implications. *J. Mol. Biol.*, **1978**, 123, 631–660.
- [885] B. N. Conner; C. Yoon; J. Dickerson; R. E. Dickerson. Helix Geometry and Hydration in an A-DNA Tetramer: C-C-G-G. *J. Mol. Biol.*, **1984**, 174, 663–695.
- [886] M. Le Bret; J. Gabarro-Arpa; J. C. Gilbert; C. Lemarechal. MORCAD an object-oriented molecular modeling package. *J. Chim. Phys.*, **1991**, 88, 2489–2496.
- [887] V. B. Zhurkin; Y. P. Lysov; V. I. Ivanov. Different Families of Double Stranded Conformations of DNA as Revealed by Computer Calculations. *Biopolymers*, **1978**, 17, 277–312.
- [888] A. T. Brünger. *X-PLOR: A System for Crystallography and NMR, Version 3.1*. Yale University, New Haven, CT, 1992.
- [889] J. R. Wyatt; J. D. Puglisi; I. Tinoco Jr. RNA Pseudoknots. Stability and Loop Size Requirements. *J. Mol. Biol.*, **1990**, 214, 455–470.
- [890] J. D. Puglisi; J. R. Wyatt; I. Tinoco Jr. Conformation of an RNA Pseudoknot. *J. Mol. Biol.*, **1990**, 214, 437–453.
- [891] G. Kuila; J. A. Fee; J. R. Schoonover; W. H. Woodruff. Resonance Raman Spectra of the [2Fe-2S] Clusters of the Rieske Protein from Thermus and Phthalate Dioxygenase from Pseudomonas. *J. Am. Chem. Soc.*, **1987**, 109, 1559–1561.
- [892] B. Lewin. in *Genes IV*, pp 409–425. Cell Press, Cambridge, Mass., 1990.
- [893] W. H. Press; S. A. Teukolsky; W. T. Vetterling; B. P. Flannery. in *Numerical Recipes in C*, pp 113–117. Cambridge, New York, 1992.
- [894] V. B. Zhurkin; G. Raghunathan; N. B. Ulyanov; R. D. Camerini-Otero; R. L. Jernigan. A Parallel DNA Triplex as a Model for the Intermediate in Homologous Recombination. *J. Mol. Biol.*, **1994**, 239, 181–200.
- [895] W. Saenger. in *Principles of Nucleic Acid Structure*, p 120. Springer-Verlag, New York, 1984.
- [896] M. Turcotte; G. Lapalme; F. Major. Exploring the conformations of nucleic acids. *J. Funct. Program.*, **1995**, 5, 443–460.
- [897] C.-S. Tung; E. S. Carter, II. Nucleic acid modeling tool (NAMOT): an interactive graphic tool for modeling nucleic acid structures. *CABIOS*, **1994**, 10, 427–433.
- [898] E. S. Carter, II; C.-S. Tung. NAMOT2—a redesigned nucleic acid modeling tool: construction of non-canonical DNA structures. *CABIOS*, **1996**, 12, 25–30.
- [899] R. Lavery; K. Zakrzewska; H. Skelnar. JUMNA (junction minimisation of nucleic acids). *Comp. Phys. Commun.*, **1995**, 91, 135–158.
- [900] G. M. Crippen; T. F. Havel. *Distance Geometry and Molecular Conformation*. Research Studies Press, Taunton, England, 1988.
- [901] D. C. Spellmeyer; A. K. Wong; M. J. Bower; J. M. Blaney. Conformational analysis using distance geometry methods. *J. Mol. Graph. Model.*, **1997**, 15, 18–36.
- [902] M. E. Hodsdon; J. W. Ponder; D. P. Cistola. The NMR solution structure of intestinal fatty acid-binding protein complexed with palmitate: Application of a novel distance geometry algorithm. *J. Mol. Biol.*, **1996**, 264, 585–602.
- [903] T. Macke; S.-M. Chen; W. J. Chazin. in *Structure and Function, Volume 1: Nucleic Acids*, R. H. Sarma; M. H. Sarma, Eds., pp 213–227. Adenine Press, Albany, 1992.
- [904] B. C. M. Potts; J. Smith; M. Akke; T. J. Macke; K. Okazaki; H. Hidaka; D. A. Case; W. J. Chazin. The structure of calycyclin reveals a novel homodimeric fold S100 Ca²⁺-binding proteins. *Nature Struct. Biol.*, **1995**, 2, 790–796.

BIBLIOGRAPHY

- [905] J. J. Love; X. Li; D. A. Case; K. Giese; R. Grosschedl; P. E. Wright. DNA recognition and bending by the architectural transcription factor LEF-1: NMR structure of the HMG domain complexed with DNA. *Nature*, **1995**, 376, 791–795.
- [906] R. J. Gurbel; P. E. Doan; G. T. Gassner; T. J. Macke; D. A. Case; T. Ohnishi; J. A. Fee; D. P. Ballou; B. M. Hoffman. Active site structure of Rieske-type proteins: Electron nuclear double resonance studies of isotopically labeled phthalate dioxygenase from *Pseudomonas cepacia* and Rieske protein from *Rhodobacter capsulatus* and molecular modeling studies of a Rieske center. *Biochemistry*, **1996**, 35, 7834–7845.
- [907] T. J. Macke. *NAB, a Language for Molecular Manipulation*. Ph.D. thesis, The Scripps Research Institute, 1996.
- [908] D. Gautheret; F. Major; R. Cedergren. Modeling the three-dimensional structure of RNA using discrete nucleotide conformational sets. *J. Mol. Biol.*, **1993**, 229, 1049–1064.
- [909] R. Tan; S. Harvey. Molecular Mechanics Model of Supercoiled DNA. *J. Mol. Biol.*, **1989**, 205, 573–591.
- [910] T. F. Havel. An evaluation of computational strategies for use in the determination of protein structure from distance constraints obtained by nuclear magnetic resonance. *Prog. Biophys. Mol. Biol.*, **1991**, 56, 43–78.
- [911] J. Kuszewski; M. Nilges; A. T. Brünger. Sampling and efficiency of metric matrix distance geometry: A novel partial metrization algorithm. *J. Biomolec. NMR*, **1992**, 2, 33–56.
- [912] B. L. deGroot; D. M. F. van Aalten; R. M. Scheek; A. Amadei; G. Vriend; H. J. C. Berendsen. Prediction of protein conformational freedom from distance constraints. *Proteins*, **1997**, 29, 240–251.
- [913] T. F. Havel; I. D. Kuntz; G. M. Crippen. The theory and practice of distance geometry. *Bull. Math. Biol.*, **1983**, 45, 665–720.
- [914] D. K. Agrafiotis. Stochastic Proximity Embedding. *J. Comput. Chem.*, **2003**, 24, 1215–1221.
- [915] C. Brooks; A. Brünger; M. Karplus. Active site dynamics in protein molecules: A stochastic boundary molecular-dynamics approach. *Biopolymers*, **1985**, 24, 843–865.
- [916] D. T. Nguyen; D. A. Case. On finding stationary states on large-molecule potential energy surfaces. *J. Phys. Chem.*, **1985**, 89, 4020–4026.
- [917] W. F. van Gunsteren; P. K. Weiner; A. J. Wilkinson, eds. *Computer Simulations of Biomolecular Systems*, Vol. 2. ESCOM Science Publishers, Leiden, 1993.
- [918] J. Wang; P. Cieplak; P. A. Kollman. How well does a restrained electrostatic potential (RESP) model perform in calculating conformational energies of organic and biological molecules? *J. Comput. Chem.*, **2000**, 21, 1049–1074.
- [919] R. W. Dixon; P. A. Kollman. Advancing beyond the atom-centered model in additive and nonadditive molecular mechanics. *J. Comput. Chem.*, **1997**, 18, 1632–1646.
- [920] M. D. Beachy; R. A. Friesner. Accurate ab initio quantum chemical determination of the relative energies of peptide conformations and assessment of empirical force fields. *J. Am. Chem. Soc.*, **1997**, 119, 5908–5920.
- [921] P. A. Kollman; R. Dixon; W. Cornell; T. Fox; C. Chipot; A. Pohorille. in *Computer Simulation of Biomolecular Systems*, Vol. 3, A. Wilkinson; P. Weiner; W. F. van Gunsteren, Eds., pp 83–96. Elsevier, 1997.
- [922] J. Higo; N. Ito; M. Kuroda; S. Ono; N. Nakajima; H. Nakamura. Energy landscape of a peptide consisting of α -helix, 3_{10} helix, β -turn, β -hairpin and other disordered conformations. *Prot. Sci.*, **2001**, 10, 1160–1171.
- [923] L. Wang; Y. Duan; R. Shortle; B. Imperiali; P. A. Kollman. Study of the stability and unfolding mechanism of BBA1 by molecular dynamics simulations at different temperatures. *Prot. Sci.*, **1999**, 8, 1292–1304.
- [924] T. E. Cheatham, III; P. Cieplak; P. A. Kollman. A modified version of the Cornell et al. force field with improved sugar pucker phases and helical repeat. *J. Biomol. Struct. Dyn.*, **1999**, 16, 845–862.

- [925] S. J. Weiner; P. A. Kollman; D. A. Case; U. C. Singh; C. Ghio; G. Alagona; S. Profeta, Jr.; P. Weiner. A new force field for molecular mechanical simulation of nucleic acids and proteins. *J. Am. Chem. Soc.*, **1984**, *106*, 765–784.
- [926] S. J. Weiner; P. A. Kollman; D. T. Nguyen; D. A. Case. An all-atom force field for simulations of proteins and nucleic acids. *J. Comput. Chem.*, **1986**, *7*, 230–252.
- [927] J. Åqvist. Ion-water interaction potentials derived from free energy perturbation simulations. *J. Phys. Chem.*, **1990**, *94*, 8021–8024.
- [928] J. Åqvist; A. Warshel. Computer simulation of the initial proton-transfer step in human carbonic anhydrase-I. *J. Mol. Biol.*, **1992**, *224*, 7–14.
- [929] H. J. C. Berendsen; J. P. M. Postma; W. F. van Gunsteren; A. DiNola; J. R. Haak. Molecular dynamics with coupling to an external bath. *J. Chem. Phys.*, **1984**, *81*, 3684–3690.
- [930] D. S. Wishart; D. A. Case. Use of chemical shifts in macromolecular structure determination. *Meth. Enzymol.*, **2001**, *338*, 3–34.
- [931] L. T. Chong; Y. Duan; L. Wang; I. Massova; P. A. Kollman. Molecular dynamics and free-energy calculations applied to affinity maturation in antibody 48G7. *Proc. Natl. Acad. Sci. USA*, **1999**, *96*, 14330–14335.
- [932] R. Elber; M. Karplus. Enhanced sampling in molecular dynamics. Use of the time-dependent Hartree approximation for a simulation of carbon monoxide diffusion through myoglobin. *J. Am. Chem. Soc.*, **1990**, *112*, 9161–9175.
- [933] C. Simmerling; M. R. Lee; A. R. Ortiz; A. Kolinski; J. Skolnick; P. A. Kollman. Combining MONSTER and LES/PME to Predict Protein Structure from Amino Acid Sequence: Application to the Small Protein CMTI-1. *J. Am. Chem. Soc.*, **2000**, *122*, 8392–8402.
- [934] C. Simmerling; R. Elber. Hydrophobic "collapse" in a cyclic hexapeptide: Computer simulations of CHDLFC and CAAAAC in water. *J. Am. Chem. Soc.*, **1994**, *116*, 2534–2547.
- [935] W. S. Ross; C. C. Hardin. Ion-induced stabilization of the G-DNA quadruplex: Free energy perturbation studies. *J. Am. Chem. Soc.*, **1994**, *116*, 6070–6080.
- [936] A. Vedani; D. W. Huhta. A new force field for modeling metalloproteins. *J. Am. Chem. Soc.*, **1990**, *112*, 4759–4767.
- [937] D. L. Veenstra; D. M. Ferguson; P. A. Kollman. How transferable are hydrogen parameters in molecular mechanics calculations? *J. Comput. Chem.*, **1992**, *13*, 971–978.
- [938] F. H. Allen; O. Kennard; D. G. Watson; L. Brammer; A. G. Orpen; R. Taylor. *J. Chem. Soc. Perkin Trans. II*, **1987**, pp S1–S19.
- [939] M. D. Harmony; R. W. Laurie; R. L. Kuczkowski; R. H. Schwendemann; D. A. Ramsay; F. J. Lovas; W. J. Lafferty; A. G. Maki. *J. Phys. Chem. Ref. Data*, **1979**, *8*, 619.
- [940] A. J. Hopfinger; R. A. Pearlstein. Molecular mechanics force-field parameterization procedures. *J. Comput. Chem.*, **1985**, *5*, 486–499.
- [941] J. F. Cannon. AMBER force-field parameters for guanosine triphosphate and its imido and methylene analogs. *J. Comput. Chem.*, **1993**, *14*, 995–1005.
- [942] P. Cieplak; W. D. Cornell; C. Bayly; P. A. Kollman. Application of the multimolecule and multiconformational RESP methodology to biopolymers: Charge derivation for DNA, RNA and proteins. *J. Comput. Chem.*, **1995**, *16*, 1357–1377.
- [943] A. E. Howard; P. Cieplak; P. A. Kollman. A molecular mechanical model that reproduces the relative energies for chair and twist-boat conformations of 1,3-dioxanes. *J. Comp. Chem.*, **1995**, *16*, 243–261.

BIBLIOGRAPHY

- [944] A. St.-Amant; W. D. Cornell; P. A. Kollman; T. A. Halgren. Calculation of molecular geometries, relative conformational energies, dipole moments, and molecular electrostatic potential fitted charges of small organic molecules of biochemical interest by density functional theory. *J. Comput. Chem.*, **1995**, *16*, 1483–1506.
- [945] T. A. Halgren. Merck Molecular Force Field (MMFF94). Part I-V. *J. Comput. Chem.*, **1996**, *17*, 490–641.
- [946] E. Meng; P. Cieplak; J. W. Caldwell; P. A. Kollman. Accurate solvation free energies of acetate and methylammonium ions calculated with a polarizable water model. *J. Am. Chem. Soc.*, **1994**, *116*, 12061–12062.
- [947] D. L. Beveridge; F. M. DiCapua. Free energy simulation via molecular simulations: Applications to chemical and biomolecular systems. *Annu. Rev. Biophys. Biophys. Chem.*, **1989**, *18*, 431–492.
- [948] C. Chipot; P. A. Kollman; D. A. Pearlman. Alternative approaches to potential of mean force calculations: free energy perturbation versus thermodynamics integration. Case study of some representative nonpolar interactions. *J. Comput. Chem.*, **1996**, *17*, 1112–1131.
- [949] D. A. Pearlman; P. A. Kollman. The overlooked bond-stretching contribution in free energy perturbation calculations. *J. Chem. Phys.*, **1991**, *94*, 4532–4545.
- [950] D. A. Pearlman. Determining the contributions of constraints in free energy calculations: Development, characterization, and recommendations. *J. Chem. Phys.*, **1993**, *98*, 8946–8957.
- [951] D. A. Pearlman. Free energy derivatives: A new method for probing the convergence problem in free energy calculations. *J. Comput. Chem.*, **1994**, *15*, 105–123.
- [952] D. A. Pearlman. A comparison of alternative approaches to free energy calculations. *J. Phys. Chem.*, **1994**, *98*, 1487–1493.
- [953] D. A. Pearlman; B. G. Rao. in *Encyclopedia of Computational Chemistry*, P. von R. Schleyer; N. L. Allinger; T. Clark; J. Gasteiger; P. A. Kollman; I. H. F. Schaefer, Eds., pp 1036–1061. John Wiley, Chichester, 1998.
- [954] R. J. Radmer; P. A. Kollman. Free energy calculation methods: A theoretical and empirical comparison of numerical errors and a new method for qualitative estimates of free energy changes. *J. Comput. Chem.*, **1997**, *18*, 902–919.
- [955] D. A. Pearlman; P. A. Kollman. A new method for carrying out free energy perturbation calculations: dynamically modified windows. *J. Chem. Phys.*, **1989**, *90*, 2460–2470.
- [956] H.-A. Yu; M. Karplus. A thermodynamic analysis of solvation. *J. Chem. Phys.*, **1988**, *89*, 2366–2379.
- [957] G. Hummer. Fast-growth thermodynamic integration: Error and efficiency analysis. *J. Chem. Phys.*, **2001**, *114*, 7330–7337.
- [958] S. H. Fleischman; C. L. Brooks, III. Thermodynamic calculations on biological systems: Solution properties of alcohols and alkanes. *J. Chem. Phys.*, **1988**, *87*, 221–234.
- [959] J. J. Vincent; K. M. Merz, Jr. A highly portable parallel implementation of AMBER4 using the message passing interface standard. *J. Comput. Chem.*, **1995**, *16*, 1420–1427.
- [960] R. Radmer; P. Kollman. The application of three approximate free energy calculations methods to structure based ligand design: Trypsin and its complex with inhibitors. *J. Comput.-Aided Mol. Design*, **1998**, *12*, 215–228.
- [961] S. R. Niketic; K. Rasmussen. *The Consistent Force Field: A Documentation*. Springer-Verlag, New York, 1977.
- [962] C. Cerjan; W. H. Miller. On finding transition states. *J. Chem. Phys.*, **1981**, *75*, 2800.
- [963] D. T. Nguyen; D. A. Case. On finding stationary states on large-molecule potential energy surfaces. *J. Phys. Chem.*, **1985**, *89*, 4020–4026.
- [964] G. Lamm; A. Szabo. Langevin modes of macromolecules. *J. Chem. Phys.*, **1986**, *85*, 7334–7348.

- [965] J. Kottalam; D. A. Case. Langevin modes of macromolecules: application to crambin and DNA hexamers. *Biopolymers*, **1990**, 29, 1409–1421.
- [966] S. Huo; I. Massova; P. A. Kollman. Computational Alanine Scanning of the 1:1 Human Growth Hormone-Receptor Complex. *J. Comput. Chem.*, **2002**, 23, 15–27.
- [967] T. Darden; D. Pearlman; L. G. Pedersen. Ionic charging free energies: Spherical versus periodic boundary conditions. *J. Chem. Phys.*, **1998**, 109, 10921–10935.
- [968] R. M. Levy; M. Karplus; J. Kushick; D. Perahia. Evaluation of the configurational entropy for proteins: Application to molecular dynamics simulations of an α -helix. *Macromolecules*, **1984**, 17, 1370–1374.
- [969] S. Arnott; P. J. Campbell-Smith; R. Chandrasekaran. in *Handbook of Biochemistry and Molecular Biology*, 3rd ed. *Nucleic*, G. P. Fasman, Ed., pp 411–422. CRC Press, Cleveland, 1976.
- [970] T. E. Cheatham, III; B. R. Brooks; P. A. Kollman. in *Current Protocols in Nucleic Acid Chemistry*, pp Sections 7.5, 7.8, 7.9, 7.10. Wiley, New York, 1999.
- [971] J. P. Valleau; G. M. Torrie. in *Modern Theoretical Chemistry, Vol. 5: Statistical Mechanics, Part A*, B. J. Berne, Ed. Plenum Press, New York, 1977.
- [972] S. Kumar; D. Bouzida; R. H. Swendsen; P. A. Kollman; J. M. Rosenberg. The weighted histogram analysis method for free-energy calculations on biomolecules. I. The method. *J. Comput. Chem.*, **1992**, 13, 1011–1021.
- [973] S. Kumar; J. M. Rosenberg; D. Bouzida; R. H. Swendsen; P. A. Kollman. Multidimensional free-energy calculations using the weighted histogram analysis method. *J. Comput. Chem.*, **1995**, 16, 1339–1350.
- [974] J. Kottalam; D. A. Case. Dynamics of ligand escape from the heme pocket of myoglobin. *J. Am. Chem. Soc.*, **1988**, 110, 7690–7697.
- [975] B. Roux. The calculation of the potential of mean force using computer simulations. *Comput. Phys. Comm.*, **1995**, 91, 275–282.
- [976] W. H. Press; B. P. Flannery; S. A. Teukolsky; W. T. Vetterling. in *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, New York, 1989.
- [977] P. Beroza; D. A. Case. Calculations of proton-binding thermodynamics in proteins. *Meth. Enzymol.*, **1998**, 295, 170–189.
- [978] J. D. Madura; M. E. Davis; M. K. Gilson; R. C. Wade; B. A. Luty; J. A. McCammon. Biological applications of electrostatic calculations and brownian dynamics simulations. *Rev. Computat. Chem.*, **1994**, 5, 229–267.
- [979] M. K. Gilson. Theory of electrostatic interactions in macromolecules. *Curr. Opin. Struct. Biol.*, **1995**, 5, 216–23.
- [980] M. Scarsi; J. Apostolakis; A. Caflisch. Continuum electrostatic energies of macromolecules in aqueous solutions. *J. Phys. Chem. A*, **1997**, 101, 8098–8106.
- [981] D. Elking; T. Darden; R. J. Woods. Gaussian induced dipole polarization model. *J. Computat. Chem.*, **2007**, 28, 1261–1274.
- [982] T. Simonson. Electrostatics and dynamics of proteins. *Rep. Prog. Phys.*, **2003**, 66, 737–787.
- [983] D. Bashford; M. Karplus. pK sub a's of ionizable groups in proteins: Atomic detail from a continuum electrostatic model. *Biochemistry*, **1990**, 29, 10219–10225.
- [984] A. Ghosh; C. S. Rapp; R. A. Friesner. Generalized Born model based on a surface integral formulation. *J. Phys. Chem. B*, **1998**, 102, 10983–10990.
- [985] J. D. Jackson. *Classical Electrodynamics*. Wiley and Sons, New York, 1975.

BIBLIOGRAPHY

- [986] M. Feig; J. Karanicolas; C. L. Brooks, III. MMTSB Tool Set: Enhanced sampling and multiscale modeling methods for application in structural biology. *J. Mol. Graphics Mod.*, **2004**, 22, 377–395.
- [987] C. Simmerling; B. Strockbine; A. E. Roitberg. All-atom structure prediction and folding simulations of a stable protein. *J. Am. Chem. Soc.*, **2002**, 124, 11258–11259.
- [988] A. E. García; K. Y. Sanbonmatsu. α -helical stabilization by side chain shielding of backbone hydrogen bonds. *Proc. Natl. Acad. Sci. USA*, **2002**, 99, 2782–2787.
- [989] K. N. Kirschner; R. J. Woods. Solvent interactions determine carbohydrate conformation. *Proc. Natl. Acad. Sci. USA*, **2001**, 98, 10541–10545.
- [990] K. N. Kirschner; R. J. Woods. Quantum mechanical study of the nonbonded forces in water-methanol complexes. *J. Phys. Chem. A*, **2001**, 105, 4150–4155.
- [991] K. A. Sharp; B. Honig. Electrostatic interactions in macromolecules: Theory and experiment. *Annu. Rev. Biophys. Biophys. Chem.*, **1990**, 19, 301–332.
- [992] U. C. Singh; S. J. Weiner; P. A. Kollman. Molecular dynamics simulations of d(C-G-C-G-A).d(T-C-G-C-G) with and without "hydrated" counterions. *Proc. Nat. Acad. Sci.*, **1985**, 82, 755–759.
- [993] J. Gao. Absolute free energy of solvation from Monte Carlo simulations using combined quantum and molecular mechanical potentials. *J. Phys. Chem.*, **1992**, 96, 537–540.
- [994] A. Warshel; M. Levitt. Theoretical studies of enzymic reactions: Dielectric, electrostatic and steric stabilization of the carbonium ion in the reaction of lysozyme. *J. Mol. Biol.*, **1976**, 103, 227–249.
- [995] M. J. Field; P. A. Bash; M. Karplus. A combined quantum mechanical and molecular mechanical potential for molecular dynamics simulations. *J. Comput. Chem.*, **1990**, 11, 700–733.
- [996] R. V. Stanton; D. S. Hartsough; K. M. Merz, Jr. An examination of a density functional/molecular mechanical coupled potential. *J. Comput. Chem.*, **1994**, 16, 113–128.
- [997] R. V. Stanton; L. R. Little; K. M. Merz, Jr. An examination of a Hartree-Fock/molecular mechanical coupled potential. *J. Phys. Chem.*, **1995**, 99, 17344–17348.
- [998] R. V. Stanton; D. S. Hartsough; K. M. Merz, Jr. Calculations of solvation free energies using a density functional/molecular dynamics coupled potential. *J. Phys. Chem.*, **1993**, 97, 11868–11870.
- [999] W. Yang; T.-S. Lee. A density-matrix divide-and-conquer approach for electronic structure calculations of large molecules. *J. Chem. Phys.*, **1995**, 103, 5674–5678.
- [1000] J. Nocedal; S. J. Wright. *Numerical Optimization*. Springer-Verlag, New York, 1999.
- [1001] S. G. Nash. A survey of truncated-Newton methods. *J. of Computational and Applied Mathematics*, **2000**, 124, 45–59.
- [1002] E. J. Sorin; V. S. Pande. Exploring the helix-coil transition via all-atom equilibrium ensemble simulations. *Biophys. J.*, **2005**, 88, 2472–2493.
- [1003] B. Wang; K. M. Merz, Jr. A fast QM/MM (quantum mechanical/molecular mechanical) approach to calculate nuclear magnetic resonance chemical shifts for macromolecules. *J. Chem. Theory Comput.*, **2006**, 2, 209–215.
- [1004] R. C. Rizzo; T. Aynechi; D. A. Case; I. D. Kuntz. Estimation of absolute free energies of hydration using continuum methods: Accuracy of partial charge models and optimization of nonpolar contributions. *J. Chem. Theory Comput.*, **2006**, 2, 128–139.
- [1005] Z.-X. Wang; W. Zhang; C. Wu; H. Lei; P. Cieplak; Y. Duan. Strike a Balance: Optimization of backbone torsion parameters of AMBER polarizable force field for simulations of proteins and peptides. *J. Comput. Chem.*, **2006**, 27, 781–790.

- [1006] R. P. Feynman; A. R. Hibbs. *Quantum Mechanics and Path Integrals*. McGraw-Hill, New York, 1965.
- [1007] R. P. Feynman. *Statistical Mechanics*. Benjamin, Reading, MA, 1972.
- [1008] H. Kleinert. *Path Integrals in Quantum Mechanics, Statistics, and Polymer Physics*. World Scientific, Singapore, 1995.
- [1009] L. S. Schulman. *Techniques and Applications of Path Integration*. Wiley & Sons, New York, 1996.
- [1010] A. Messiah. *Quantum Mechanics*. Wiley & Sons, New York, 1958.
- [1011] D. Chandler; P. G. Wolynes. Exploiting the isomorphism between quantum theory and classical statistical mechanics of polyatomic fluids. *J. Chem. Phys.*, **1981**, 74, 4078–4095.
- [1012] D. M. Ceperley. Path integrals in the theory of condensed helium. *Rev. Mod. Phys.*, **1995**, 67, 279–355.
- [1013] J. Cao; B. J. Berne. On energy estimators in path integral Monte Carlo simulations: Dependence of accuracy on algorithm. *J. Chem. Phys.*, **1989**, 91, 6359–6366.
- [1014] J. W. Storer; D. J. Giesen; C. J. Cramer; D. G. Truhlar. Class IV charge models: A new semiempirical approach in quantum chemistry. *J. Comput.-Aided Mol. Design*, **1995**, 9, 87–110.
- [1015] J. Li; C. J. Cramer; D. G. Truhlar. New class IV charge model for extracting accurate partial charges from Wave Functions. *J. Phys. Chem. A*, **1998**, 102, 1820–1831.
- [1016] A. van der Vaart; K. M. Merz, Jr. Divide and conquer interaction energy decomposition. *J. Phys. Chem. A*, **1999**, 103, 3321–3329.
- [1017] A. V. Mitin. The dynamic level shift method for improving the convergence of the SCF procedure. *J. Comput. Chem.*, **1988**, 9, 107–110.
- [1018] M. D. Ermolaeva; A. van der Vaart; K. M. Merz, Jr. Implementation and testing of a frozen density matrix - divide and conquer algorithm. *J. Phys. Chem.*, **1999**, 103, 1868–1875.
- [1019] B. Wang; E. N. Brothers; A. van der Vaart; K. M. Merz Jr. Fast semiempirical calculations for nuclear magnetic resonance chemical shifts: A divide-and-conquer approach. *J. Chem. Phys.*, **2004**, 120, 11392–11400.
- [1020] B. Wang; K. Raha; K. M. Merz Jr. Pose scoring by NMR. *J. Am. Chem. Soc.*, **2004**, 126, 11430–11431.
- [1021] K. Raha; A. van der Vaart; K. E. Riley; M. B. Peters; L. M. Westerhoff; H. Kim; K. M. Merz Jr. Pairwise decomposition of residue interaction energies using semiempirical quantum mechanical methods in studies of protein-ligand interaction. *J. Am. Chem. Soc.*, **2005**, 127, 6583–6594.
- [1022] A. Luzhkov; A. Warshel. Microscopic models for quantum-mechanical calculations of chemical processes in solutions - Ld/Ampac and Scaas/Ampac calculations of solvation energies. *J. Comp. Chem.*, **1992**, 13, 199–213.
- [1023] U. C. Singh; P. A. Kollman. A combined Ab initio quantum-mechanical and molecular mechanical method for carrying out simulations on complex molecular systems - Applications to the $\text{CH}_3\text{Cl} + \text{Cl}^-$ exchange-reaction and gas-phase protonation of polyethers. *J. Comp. Chem.*, **1986**, 7, 718–730.
- [1024] I. B. Bersuker; M. K. Leong; J. E. Boggs; R. S. Pearlman. A method of combined quantum mechanical (QM) molecular mechanics (MM) treatment of large polyatomic systems with charge transfer between the QM and MM fragments. *Int. J. Quant. Chem.*, **1997**, 63, 1051–1063.
- [1025] F. Maseras; K. Morokuma. Imomm - a new integrated ab-initio plus molecular geometry optimization scheme of equilibrium structures and transition-states. *J. Comp. Chem.*, **1995**, 16, 1170–1179.
- [1026] Y. K. Zhang; T. S. Lee; W. T. Yang. A pseudobond approach to combining quantum mechanical and molecular mechanical methods. *J. Chem. Phys.*, **1999**, 110, 46–54.

BIBLIOGRAPHY

- [1027] J. L. Gao; P. Amara; C. Alhambra; M. J. Field. A generalized hybrid orbital (GHO) method for the treatment of boundary atoms in combined QM/MM calculations. *J Phys Chem A*, **1998**, *102*, 4714–4721.
- [1028] D. M. Philipp; R. A. Friesner. Mixed ab initio QM/MM modeling using frozen orbitals and tests with alanine dipeptide and tetrapeptide. *J. Comp. Chem.*, **1999**, *20*, 1468–1494.
- [1029] M. J. Field; M. Albe; C. Bret; F. Proust-De Martin; A. Thomas. The Dynamo library for molecular simulations using hybrid quantum mechanical and molecular mechanical potentials. *J. Comp. Chem.*, **2000**, *21*, 1088–1100.
- [1030] R. M. Levy; E. Gallicchio. Computer simulations with explicit solvent: recent progress in the thermodynamic decomposition of free energies and in modeling electrostatic effects. *Annu. Rev. Phys. Chem.*, **1999**, *49*, 531–567.
- [1031] V. Hornak; A. Okur; R. Rizzo; C. Simmerling. HIV-1 protease flaps spontaneously open and reclose in molecular dynamics simulations. *Proc. Nat. Acad. Sci. USA*, **2006**, *103*, 915–920.
- [1032] V. Hornak; A. Okur; R. Rizzo; C. Simmerling. HIV-1 protease flaps spontaneously close when an inhibitor binds to the open state. *J. Am. Chem. Soc.*, **2006**, *128*, 2812–2813.
- [1033] J. Kästner; W. Thiel. Bridging the gap between thermodynamic integration and umbrella sampling provides a novel analysis method: “Umbrella integration”. *J. Chem. Phys.*, **2005**, *123*, 144104.
- [1034] A. Warshel. *Computer Modeling of Chemical Reactions in Enzymes and Solutions*. John Wiley and Sons, New York, 1991.
- [1035] S. R. Billeter; S. P. Webb; T. Iordanov; P. K. Agarwal; S. Hammes-Schiffer. Hybrid approach for including electronic and nuclear quantum effects in molecular dynamics simulations of hydrogen transfer reactions in enzymes. *J. Chem. Phys.*, **2001**, *114*, 6925.
- [1036] C. Simmerling; R. Elber. Hydrophobic “collapse” in a cyclic hexapeptide: Computer simulations of CHDLFC and CAAAAC in water. *J. Am. Chem. Soc.*, **1994**, *116*, 2534–2547.
- [1037] Y. Deng; B. Roux. Calculation of standard binding free energies: Aromatic molecules in the T4 lysozyme L99A mutant. *J. Chem. Theor. Comput.*, **2006**, *2*, 1255–1273.
- [1038] H. B. Schlegel; J. L. Sonnenberg. Empirical valence-bond models for reactive potential energy surfaces using distributed Gaussians. *J. Chem. Theory Comput.*, **2006**, *2*, 905.
- [1039] J. L. Sonnenberg; H. B. Schlegel. Empirical valence bond models for reactive potential energy surfaces. II. Intramolecular proton transfer in pyridone and the Claisen reaction of allyl vinyl ether. *Mol. Phys.*, **2007**, *105*, 2719.
- [1040] Y. Saad; M. H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, **1986**, *7*, 856.
- [1041] P. Pulay. Convergence acceleration of iterative sequences. The case of SCF iteration. *Chem. Phys. Lett.*, **1980**, *73*, 393.
- [1042] P. Pulay. Improved SCF convergence acceleration. *J. Comput. Chem.*, **1982**, *3*, 556.
- [1043] A. K. Rappe; C. J. Casewit; K. S. Colwell; W. A. Goddard III; W. M. Skiff. UFF, a Full Periodic Table Force Field for Molecular Mechanics and Molecular Dynamics Simulations. *J. Am. Chem. Soc.*, **1992**, *114*, 10024–10035.
- [1044] G. A. Voth; D. Chandler; W. H. Miller. Rigorous Formulation of Quantum Transition State Theory and Its Dynamical Corrections. *J. Chem. Phys.*, **1989**, *91*, 7749–7760.
- [1045] G. J. Martyna; M. L. Klein; M. Tuckerman. Nosé-Hoover chains: The canonical ensemble via continuous dynamics. *J. Chem. Phys.*, **1992**, *97*, 2635.
- [1046] G. J. Martyna; A. Hughes; M. E. Tuckerman. Molecular dynamics algorithms for path integrals at constant pressure. *J. Chem. Phys.*, **1999**, *110*, 3275.

- [1047] B. J. Berne; D. Thirumalai. On the simulation of quantum systems: path integral methods. *Annu. Rev. Phys. Chem.*, **1986**, 37, 401.
- [1048] G. A. Voth. Path-integral centroid methods in quantum statistical mechanics and dynamics. *Adv. Chem. Phys.*, **1996**, 93, 135.
- [1049] I. R. Craig; D. E. Manolopoulos. Quantum statistics and classical mechanics: Real time correlation functions from ring polymer molecular dynamics. *J. Chem. Phys.*, **2004**, 121, 3368.
- [1050] T. F. Miller; D. E. Manolopoulos. Quantum diffusion in liquid water from ring polymer molecular dynamics. *J. Chem. Phys.*, **2005**, 123, 154504.
- [1051] J. Cao; G. A. Voth. The formulation of quantum statistical mechanics based on the Feynman path centroid density. IV. Algorithms for centroid molecular dynamics. *J. Chem. Phys.*, **1994**, 101, 6168.
- [1052] J. Vaníček; W. H. Miller; J. F. Castillo; F. J. Aoiz. Quantum-instanton evaluation of the kinetic isotope effects. *J. Chem. Phys.*, **2005**, 123, 054108.
- [1053] J. Vaníček; W. H. Miller. Efficient estimators for quantum instanton evaluation of the kinetic isotope effects: application to the intramolecular hydrogen transfer in pentadiene. *J. Chem. Phys.*, **2007**, 127, 114309.
- [1054] W. H. Miller; Y. Zhao; M. Ceotto; S. Yang. Quantum instanton approximation for thermal rate constants of chemical. *J. Chem. Phys.*, **2003**, 119, 1329–1342.
- [1055] W. H. Miller. Semiclassical limit of quantum mechanical transition state theory for nonseparable systems. *J. Chem. Phys.*, **1975**, 62, 1899.
- [1056] T. Yamamoto; W. H. Miller. On the efficient path integral evaluation of thermal rate constants with the quantum instanton approximation. *J. Chem. Phys.*, **2004**, 120, 3086–3099.
- [1057] T. Yamamoto; W. H. Miller. Path integral evaluation of the quantum instanton rate constant for proton transfer in a polar solvent. *J. Chem. Phys.*, **2005**, 122, 044106.
- [1058] W. H. Miller; S. D. Schwartz; J. W. Tromp. Quantum mechanical rate constants for bimolecular reactions. *J. Chem. Phys.*, **1983**, 79, 4889–4898.
- [1059] A. T. Brünger; P. D. Adams; G. M. Clore; W. L. Delano; P. Gros; R. W. Grosse-Kunstleve; J.-S. Jiang; J. Kuszewski; M. Nilges; N. S. Pannu; R. J. Read; L. M. Rice; T. Simonson; G. L. Warren. Crystallography and NMR system (CNS): A new software system for macromolecular structure determination. *Acta Cryst. D*, **1998**, 54, 905–921.
- [1060] N. Yu; H. P. Yennawar; K. M. Merz, Jr. Refinement of protein crystal structures using energy restraints derived from linear-scaling quantum mechanics. *Acta Cryst. D*, **2005**, 61, 322–332.
- [1061] N. Yu; X. Li; G. Cui; S. Hayik; K. M. Merz, Jr. Critical assessment of quantum mechanics based energy restraints in protein crystal structure refinement. *Prot. Sci.*, **2006**, 15, 2773–2784.
- [1062] S. Fulle; H. Gohlke. Analyzing the flexibility of RNA structures by constraint counting. *Biophys. J.*, **2008**, DOI:10.1529/biophysj.107.113415.
- [1063] H. Gohlke; L. A. Kuhn; D. A. Case. Change in protein flexibility upon complex formation: Analysis of Ras-Raf using molecular dynamics and a molecular framework approach. *Proteins*, **2004**, 56, 322–327.
- [1064] A. Ahmed; H. Gohlke. Multiscale modeling of macromolecular conformational changes combining concepts from rigidity and elastic network theory. *Proteins*, **2006**, 63, 1038–1051.
- [1065] E. F. Pettersen; T. D. Goddard; C. C. Huang; G. S. Couch; D. M. Greenblatt; E. C. Meng; T. E. Ferrin. UCSF Chimera - A visualization system for exploratory research and analysis. *J. Comput. Chem.*, **2004**, 25, 1605–1612.

BIBLIOGRAPHY

- [1066] H. Sasaki; N. Ochi; A. Del; M. Fukuda. Site-specific glycosylation of human recombinant erythropoietin: Analysis of glycopeptides or peptides at each glycosylation site by fast atom bombardment mass spectrometry. *Biochemistry*, **1988**, 27, 8618–8626.
- [1067] S. Dube; J. W. Fisher; J. S. Powell. Glycosylation at specific sites of erythropoietin is essential for biosynthesis, secretion, and biological function. *J. Biol. Chem.*, **1988**, 263, 17516–17521.
- [1068] R. J. Darling; U. Kuchibhotla; W. Glaesner; R. Micanovic; D. R. Witcher; J. M. Beals. Glycosylation of erythropoietin effects receptor binding kinetics: Role of electrostatic interactions. *Biochemistry*, **2002**, 41, 14524–14531.
- [1069] J. C. Cheetham; D. M. Smith; K. H. Aoki; J. L. Stevenson; T. J. Hoeffel; R. S. Syed; J. Egrie; T. S. Harvey. NMR structure of human erythropoietin and a comparison with its receptor bound conformation. *Nat. Struct. Biol.*, **1998**, 5, 861–866.
- [1070] K. L. Dormann; R. Brueckner. Variable Synthesis of the Optically Active Thiotetronic Acid Antibiotics Thiolactomycin, Thiotetromycin, and 834-B1. *Angew. Chem. Int. Ed.*, **2007**, 46, 1160–1163.
- [1071] E. Darve; A. Pohorille. Calculating free energies using average force. *J. Chem. Phys.*, **2001**, 115, 9169–9183.
- [1072] L. Dang. Mechanism and thermodynamics of ion selectivity in aqueous solutions of 18-crown-6 ether: A molecular dynamics study. *J. Am. Chem. Soc.*, **1995**, 117, 6954–6960.
- [1073] J. Shao; S. W. Tanner; N. Thompson; T. E. Cheatham, III. Clustering molecular dynamics trajectories: 1. Characterizing the performance of different clustering algorithms. *J. Chem. Theory Comput.*, **2007**, 3, 2312–2334.
- [1074] P. Auffinger; T. E. Cheatham, III; A. C. Vaiana. Spontaneous formation of KCl aggregates in biomolecular simulations: a force field issue? *J. Chem. Theory Comput.*, **2007**, 3, 1851–1859.
- [1075] P. Cieplak; F.-Y. Dupradeau; Y. Duan; J. Wang. Polarization effects in molecular mechanical force fields. *J. Phys.: Condens. Matter*, **2009**, 21, 333102.
- [1076] Z. J. Shi; J. Shen. New inexact line search method for unconstrained optimization. *J. Optim. Theory Appl.*, **2005**, 127, 425–446.
- [1077] B. R. Brooks; R. E. Bruccoleri; D. J. Olafson; D. J. States; S. Swaminathan; M. Karplus. CHARMM: A Program for Macromolecular Energy, Minimization, and Dynamics Calculations. *J. Computat. Chem.*, **1983**, 4, 187–217.
- [1078] B. R. Brooks; C. L. Brooks; A. D. Mackerell; L. Nilsson; R. J. Petrella; B. Roux; Y. Won; G. Archontis; C. Bartels; S. Boresch; A. Caflisch; L. Caves; Q. Cui; A. R. Dinner; M. Feig; S. Fischer; J. Gao; M. Hodoscek; W. Im; K. Kuczera; T. Lazaridis; J. Ma; V. Ovchinnikov; E. Paci; R. W. Pastor; C. B. Post; J. Z. Pu; M. Schaefer; B. Tidor; R. M. Venable; H. L. Woodcock; X. Wu; W. Yang; D. M. York; M. Karplus. CHARMM: the biomolecular simulation program. *J. Comput. Chem.*, **2009**, 30, 1545–1614.
- [1079] B. J. Berne; G. D. Harp. *Adv. Chem. Phys.*, **1970**, 17, 63.
- [1080] R. Kubo; M. Toda; N. Hashitsume. *Statistical Physics II: Nonequilibrium Statistical Mechanics*, 2nd ed. Springer-Verlag, Heidelberg, 1991.
- [1081] W. H. Miller. *Adv. Chem. Phys.*, **1974**, 25, 69.
- [1082] W. H. Miller. Including quantum effects in the dynamics of complex (i.e., large) molecular systems. *J. Chem. Phys.*, **2006**, 125, 132305.
- [1083] H. Wang; X. Sun; W. H. Miller. Semiclassical approximations for the calculation of thermal rate constants for chemical reactions in complex molecular systems. *J. Chem. Phys.*, **1998**, 108, 9726.
- [1084] X. Sun; H. Wang; W. H. Miller. Semiclassical theory of electronically nonadiabatic dynamics: Results of a linearized approximation to the initial value representation. *J. Chem. Phys.*, **1998**, 109, 7064.

- [1085] J. Liu; W. H. Miller. A simple model for the treatment of imaginary frequencies in chemical reaction rates and molecular liquids. *J. Chem. Phys.*, **2009**, *131*, 074113.
- [1086] J. Liu; W. H. Miller; F. Paesani; W. Zhang; D. A. Case. Quantum dynamical effects in liquid water: A semiclassical study on the diffusion and the infrared absorption spectrum. *J. Chem. Phys.*, **2009**, *131*, 164509.
- [1087] J. Liu. Recent advances in the linearized semiclassical initial value representation/classical wigner model for the thermal correlation function. *International Journal of Quantum Chemistry*, **2015**, *115*, 657–670.
- [1088] J. Liu; W. H. Miller. Real time correlation function in a single phase space integral beyond the linearized semiclassical initial value representation. *J. Chem. Phys.*, **2007**, *126*, 234110.
- [1089] J. Liu; W. H. Miller. Test of the consistency of various linearized semiclassical initial value time correlation functions in application to inelastic neutron scattering from liquid para-hydrogen. *J. Chem. Phys.*, **2008**, *128*, 144511.
- [1090] Q. Shi; E. Giva. *J. Chem. Phys. A*, **2003**, *107*, 9059.
- [1091] M.-J. Hsieh; R. Luo. Balancing simulation accuracy and efficiency with the Amber united atom force field. *J. Phys. Chem. B*, **2010**, *114*, 2886–2893.
- [1092] F. Hirata, Ed. *Molecular Theory of Solvation*. Kluwer Academic Publishers, 2003.
- [1093] D. W. Li; R. Brüschweiler. NMR-based protein potentials. *Angew. Chem. Int. Ed.*, **2010**, *49*, 6778–6780.
- [1094] K. Lindorff-Larsen; S. Piana; K. Palmo; P. Maragakis; J. Klepeis; R. O. Dror; D. E. Shaw. Improved side-chain torsion potentials for the Amber ff99SB protein force field. *Proteins*, **2010**, *78*, 1950–1958.
- [1095] R. Anandakrishnan; A. V. Onufriev. An N log N approximation based on the natural organization of biomolecules for speeding up the computation of long range interactions. *J. Comput. Chem.*, **2010**, *31*, 691–706.
- [1096] R. Anandakrishnan; M. Daga; A. V. Onufriev. An n log n Generalized Born Approximation. *J. Chem. Theory Comput.*, **2011**, *7*, 544–559.
- [1097] D. S. Cerutti; R. E. Duke; T. A. Darden; T. P. Lybrand. Staggered Mesh Ewald: An Extension of the Smooth Particle-Mesh Ewald Method Adding Great Versatility. *J. Chem. Theory Comput.*, **2009**, *5*, 2322–2338.
- [1098] D. S. Cerutti; D. A. Case. Multi-Level Ewald: A Hybrid Multigrid/Fast Fourier Transform Approach to the Electrostatic Particle-Mesh Problem. *J. Chem. Theory Comput.*, **2010**, *6*, 443–458.
- [1099] D. S. Cerutti; P. L. Freddolino; R. E. Duke, Jr.; D. A. Case. Simulations of a Protein Crystal with a High Resolution X-ray Structure: Evaluation of Force Fields and Water Models. *J. Phys. Chem. B*, **2010**, pp 12811–12824.
- [1100] T. Gaillard; D. A. Case. Evaluation of DNA Force Fields in Implicit Solvation. *J. Chem. Theory Comput.*, **2011**, *7*, 3181–3198.
- [1101] Y. Meng; A. E. Roitberg. Constant pH replica exchange molecular dynamics in biomolecules using a discrete protonation model. *J. Chem. Theory Comput.*, **2010**, *6*, 1401–1412.
- [1102] D. Sabri Dashti; Y. Meng; A. E. Roitberg. pH-Replica Exchange Molecular Dynamics in Proteins Using a Discrete Protonation Method. *J. Phys. Chem. B*, **2012**, *116*, 8805–8811.
- [1103] D. L. Mobley; C. I. Bayly; M. D. Cooper; M. R. Shirts; K. A. Dill. Small Molecule Hydration Free Energies in Explicit Solvent: An Extensive Test of Fixed-Charge Atomistic Simulations. *J. Chem. Theory Comput.*, **2009**, *5*, 350–358.
- [1104] X. Wu; B. R. Brooks. A virtual mixture approach to the study of multistate equilibrium: application to constant pH simulation in explicit water. *PLOS Computational Biology*, **2015**, *11*, e1004480.

BIBLIOGRAPHY

- [1105] P. G. Karamertzanis; P. Raiteri; A. Galindo. The use of anisotropic potentials in modeling water and free energies of hydration. *J. Chem. Theory Comput.*, **2010**, 6, 3153–3161.
- [1106] B. T. Thole. Molecular polarizabilities calculated with a modified dipole interaction. *Chem. Phys.*, **1981**, 59, 341–350.
- [1107] R. Bosque; J. Sales. Polarizabilities of solvents from the chemical composition. *J. Chem. Inf. Comput. Sci.*, **2002**, 42, 1154–1163.
- [1108] Z. X. Wang; C. Wu; H. X. Lei; Y. Duan. Accurate ab initio study on the hydrogen-bond pairs in protein secondary structures. *J. Chem. Theory Comput.*, **2007**, 3, 1527–1537.
- [1109] J. Wang; T. Hou. Application of Molecular Dynamics Simulations in Molecular Property Prediction. II. Diffusion coefficient. *J. Comput. Chem.*, **2011**, 32, 3509–3519.
- [1110] C. F. Fu; S. X. Tian. A Comparative Study for Molecular Dynamics Simulations of Liquid Benzene. *J. Chem. Theory Comput.*, **2011**, 7, 2240–2252.
- [1111] S. Tsuzuki; T. Uchimaru; K. Tanabe; S. Kuwajima. Refinement of Nonbonding Interaction Potential Parameters for Methane on the Basis of the Pair Potential Obtained by Mp3/6-311g(3d,3p)-Level Ab-Initio Molecular-Orbital Calculations - the Anisotropy of H/H Interaction. *J. Phys. Chem.*, **1994**, 98, 1830–1833.
- [1112] G. A. Kaminski; R. A. Friesner; J. Tirado-Rives; W. L. Jorgensen. Evaluation and Reparametrization of the OPLS-AA Force Field for Proteins via Comparison with Accurate Quantum Chemical Calculations on Peptides. *J. Phys. Chem. B*, **2001**, 105, 6474–6487.
- [1113] I. J. Chen; D. Yin; A. D. MacKerell. Combined Ab initio/Empirical Approach for Optimization of Lennard-Jones Parameters for Polar-Neutral Compounds. *J. Comput. Chem.*, **2002**, 23, 199–213.
- [1114] F.-Y. Dupradeau; A. Pigache; T. Zaffran; C. Savineau; R. Lelong; N. Grivel; D. Lelong; W. Rosanskia; P. Cieplak. The R. E. D. tools: advances in RESP and ESP charge derivation and force field library building. *PhysChem-ChemPhys*, **2010**, 12, 7821–7839.
- [1115] D. E. Warschawski; P. F. Devaux. Order parameters of unsaturated phospholipids in membranes and the effect of cholesterol: a 1H-13C solid-state NMR study at natural abundance. *Eur. Biophys. J.*, **2005**, 34, 987–996.
- [1116] S. A. Showalter; R. Brüschweiler. Validation of molecular dynamics simulations of biomolecules using NMR spin relaxation as benchmarks: Application to the Amber99SB force field. *J. Chem. Theory Comput.*, **2007**, 3, 961–975.
- [1117] R. B. Best; N.-V. Buchete; G. Hummer. Are Current Molecular Dynamics Force Fields too Helical? *Biophys. J.*, **2008**, 95, L07–L09; 4494.
- [1118] R. B. Best; G. Hummer. Optimized Molecular Dynamics Force Fields Applied to the Helix-Coil Transition of Polypeptides. *J. Phys. Chem. B*, **2009**, 113, 9904–9015.
- [1119] R. B. Best; J. Mittal. Free-energy landscape of the GB1 hairpin in all-atom explicit solvent simulations with different force fields: Similarities and differences. *Proteins*, **2011**, 79, 1318–1328.
- [1120] K. K. Patapati; N. M. Glykos. Three force fields views of the 3-10 helix. *Biophys. J.*, **2011**, 101, 1766–1771.
- [1121] PDB Current Holdings Breakdown. **2013**.
- [1122] A. Ganguly; B. P. Weissman; T. J. Giese; N.-A. Li; S. Hoshika; S. Rao; A. A. Benner; J. A. Piccirilli; D. M. York. Confluence of theory and experiment reveals the catalytic mechanism of the Varkud satellite ribozyme. *Nat. Chem.*, **2020**, 12, 192–201.
- [1123] C. N. Nguyen; T. Kurtzman Young; M. K. Gilson. Grid Inhomogeneous solvation theory: Hydration structure and thermodynamics of the miniature receptor cucurbit[7]uril. *J. Chem. Phys.*, **2012**, 137, 044101–044118.

- [1124] D. P. Fernandez; A. R. H. Goodwin; E. W. Lemmon; J. M. H. Levelt Sengers; R. C. Williams. A formulation for the static permittivity of water and steam at temperatures from 238 k to 873 k at pressures up to 1200 mpa, including derivatives and debye huckel coefficients. *J. Phys. Chem. Ref. Data*, **1997**, 26, 1125–1166.
- [1125] R. Mills. Self-diffusion in normal and heavy water in the range 1–45. deg. *J. Phys. Chem.*, **1973**, 77, 685–688.
- [1126] W. Wagner; A. Pruss. The iapws formulation 1995 for the thermodynamic properties of ordinary water substance for general and scientific use. *J. Phys. Chem. Ref. Data*, **2002**, 31, 387–535.
- [1127] L. B. Skinner; C. Huang; D. Schlesinger; L. G. M. Pettersson; A. Nilsson; C. J. Benmore. Benchmark oxygen-oxygen pair-distribution function of ambient water from x-ray diffraction measurements with a wide q-range. *J. Chem. Phys.*, **2013**, 138, 074506+.
- [1128] G. S. Kell. Precise representation of volume properties of water at one atmosphere. *J. Chem. Eng. Data*, **1967**, 12, 66–69.
- [1129] C. Vega; J. L. F. Abascal. Simulating water with rigid non-polarizable models: a general perspective. *Phys Chem Chem Phys*, **2011**, 13, 19663–19688.
- [1130] J. Z. Ruscio; D. Kumar; M. Shukla; M. G. Prisant; T. M. Murali; A. V. Onufriev. Atomic level computational identification of ligand migration pathways between solvent and binding site in myoglobin. *Proc. Nat. Acad. Sci. USA*, **2008**, 105, 9204–9209.
- [1131] S. Izadi; R. Anandakrishnan; A. V. Onufriev. Implicit solvent model for Million-Atom atomistic simulations: Insights into the organization of 30-nm chromatin fiber. *J. Chem. Theory Comput.*, **2016**, 12, 5946–5959.
- [1132] D. S. Cerutti; K. T. Debiec; D. A. Case; L. T. Chong. Links between the charge model and bonded parameter force constants in biomolecular force fields. *J. Chem. Phys.*, **2017**, 147, 161730.
- [1133] J. Khandogin; C. L. Brooks, III. Toward the accurate first-principles prediction of ionization equilibria in proteins. *Biochemistry*, **2006**, 45, 9363–9373.
- [1134] W. Chen; J. A. Wallace; Z. Yue; J. K. Shen. Introducing titratable water to all-atom molecular dynamics at constant pH. *Biophys. J.*, **2013**, 105, L15–L17.
- [1135] J.-P. Ryckaert; G. Ciccotti; H. J. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. *J. of Computat. Phys.*, **1977**, 23, 327 – 341.
- [1136] S. Myamoto; P. A. Kollman. Settle: An analytical version of the SHAKE and RATTLE algorithm for rigid water models. *J. Computat. Chem.*, **1992**, 13, 952–962.
- [1137] H. C. Andersen. Molecular dynamics simulations at constant pressure and/or temperature. *J. Chem. Phys.*, **1980**, 72, 2384–2393.
- [1138] M. Havrila; M. Otyepka; P. Stadlbauer; J. Sponer; J. L. Mergny; P. Banas; P. Kuhrova. Structural dynamics of propeller loop: towards folding of RNA G-quadruplex. *Nucl. Acids Res.*, **2018**, 46, 8754–8771.
- [1139] R. Read; A. McCoy. A log-likelihood-gain intensity target for crystallographic phasing that accounts for experimental error. *Acta Cryst. D*, **2016**, 72, 375–387.
- [1140] S. Meisburger; D. Case; N. Ando. Correlated motions in a protein crystal. *Nature Commun.*, **2020**, 11, 1271.
- [1141] S. Meisburger; N. Ando. Correlated Motions from Crystallography beyond Diffraction. *Acc. Chem. Res.*, **2017**, 50, 580–583.
- [1142] T. J. Giese; D. M. York. A GPU-Accelerated Parameter Interpolation Thermodynamic Integration Free Energy Method. *J. Chem. Theory Comput.*, **2018**, 14, 1564–1582.
- [1143] E. W. Weisstein. Euler angles From MathWorld—A Wolfram Web Resource.

BIBLIOGRAPHY

- [1144] D. C. Rapaport. *The Art of Molecular Dynamics Simulation*. Cambridge University Press, 2010.
- [1145] X. Wu; B. R. Brooks. Origin of pka shifts of internal lysine residues in snase studied via equal-molar vmms simulations in explicit water. *J. Phys. Chem. B*, **2017**, *121*, 3318–3330.
- [1146] X. Wu; B. R. Brooks. Hydronium ions accompanying buried acidic residues lead to high apparent dielectric constants in the interior of proteins. *J. Phys. Chem. B*, **2018**, *122*, 6215–6223.
- [1147] X. Wu; B. R. Brooks. The homogeneity condition: A simple way to derive isotropic periodic sum potentials for efficient calculation of long-range interactions in molecular simulation. *J. Chem. Phys.*, **2019**, *150*, 214109.
- [1148] X. Wu; B. R. Brooks. A double exponential potential for van der waals interaction. *AIP Advances*, **2019**, *9*, 065304.
- [1149] V. Man; X. Wu; X. He; X.-Q. Xie; B. Brooks; J. Wang. Determination of van der waals parameters using a double exponential potential for divalent metal cations in tip3p solvent. *J. Chem. Theory Comput.*, **2021**, *17*, 1086–1097.
- [1150] J. Kästner; J. M. Carr; T. W. Keal; W. Thiel; A. Wander; P. Sherwood. DL-FIND: An open-source geometry optimizer for atomistic simulations. *J. Phys. Chem. A*, **2009**, *113*, 11856–11865.
- [1151] F. Klein; M. Soñora; L. Helene Santos; E. Nazareno Frigini; A. Ballesteros – Casallas; M. Rodrigo Machado et al. The sirah force field: A suite for simulation of complex biological systems at the coarse-grained and multiscale levels. *J. Struct. Biol.*, **2023**, *215*, 107985.

Index

Symbols

--entropy, [137](#)
--exchem, [137](#)
--potUV, [137](#)
--solvene, [137](#)

Numbers

1D-RISM, [111f](#), [123](#)
3D-RISM, [111](#), [113](#), [126](#)

A

AMBER-DYES in AMBER, [57](#)
ANI, [672](#)
abfqmmm, [206](#), [443](#)
accept, [86](#)
accuracy, [191](#)
acdoctor, [325](#)
active, [510](#)
activeref, [717](#)
adbcor, [126](#)
add, [248](#)
addAtomTypes, [249](#)
addAtomicNumber, [284](#)
addC4Pairwise, [249](#)
addC4Type, [249](#)
addDihedral, [284](#)
addExclusions, [285](#)
addIons, [249](#)
addIons2, [249](#)
addIonsRand, [250](#)
addLJType, [285](#)
addPDB, [285](#)
addPath, [250](#)
addPdbAtomMap, [250](#)
addPdbResMap, [250](#)
add_12_6_4, [286](#)
add_pdb, [285](#)
adjust_q, [167](#), [443](#)
aexp, [632](#)
alias, [251](#)
allow_shake, [512](#)
alpb, [73](#), [439](#)
alpha, [520](#)
am1bcc, [321](#)
amb2chm_par.py, [308](#)

amb2chm_psf_crd.py, [308](#)
amb2gro_top_gro.py, [308](#)
amoeba_verbose, [669](#)
anchor_postion, [567](#)
anchor_strength, [567](#)
angle, [757](#)
antechamber, [313](#)
apply_rism_force, [133](#)
arad, [73](#)
arange, [632](#)
arcres, [85](#)
arnoldi_dimension, [508](#)
asyp, [137](#)
asypKSpaceTolerance, [116](#), [119](#), [131](#), [138](#)
asymptcorr, [130](#)
atmask, [653](#)
atnam, [629](#)
atom_selection_mask, [658](#)
atomicfluct, [758](#)
atommap, [759](#)
atomn, [433](#)
atoms, [502](#)
atomtype, [321](#)
auxbasis, [178](#)
average, [761](#)
avg, [197](#)
awt, [632](#)

B

b_sol, [658](#)
bar_intervall, [530](#)
bar_l_incr, [530](#)
bar_l_max, [530](#)
bar_l_min, [530](#)
baroscalingdir, [404](#)
barostat, [404](#)
basis, [173f](#), [176ff](#), [180](#), [182](#), [185](#), [188](#)
basiscutoff, [185](#)
bcopt, [87](#)
beckegrid, [173](#)
beeman_integrator, [669](#)
bellymask, [399](#)
bond, [251](#)
bondByDistance, [251](#)
bondtype, [322](#)

INDEX

box, 502
bridge function, 112
bspline_order_for_gridtype, 670
buffer, 119, 131, 138, 905
buffer_iqmatoms, 443
buffer_size, 503
buffercharge, 206, 442
buffermask, 206, 444
bulk_solvent_model, 658

C
CEMD, 610
COLVAR_type, 501
CV_file, 502
calc, 180
calc_wbk, 202
car_to_files.py, 368
CartHess2FC.py, 365
cavity_offset, 903
cavity_surften, 903
cbasis, 177
ccut, 639
center, 764
center_type, 207
centering, 132, 140
centermask, 208, 444
cestats, 614
chamber, 287
change, 294
changeLJ14Pair, 294
changeLJPair, 294
changeLJSingleType, 294
changeProtState, 295
changeRadii, 295
changeRedoxState, 295
char_const_contig_end, 219
char_const_contig_start, 219
char_const_contig_value, 219
char_const_custom_atom_index, 219
char_const_custom_coeff, 219
char_const_custom_count, 219
char_const_custom_rhs, 219
charge, 211
charge density, 128
charge distribution, 128
charge_analysis, 189
charge_method, 218
charge_out, 213
check, 252, 764
checkValidity, 295
chelpg, 175
chgdist, 137
chkvir, 433
chnghmask, 425
cis, 189
cisnumstates, 189
cistarget, 189
clambda, 516, 576
closest, 766
closure, 121, 124, 129, 137, 905
closureorder, 905
cluster, 851
clusterdihedral, 767
cobsl, 638
cofactormask, 421
column_fft, 424
combine, 252
comp, 404
conflib_filename, 508
conflib_size, 508
control, 218
convkey, 177
convthre, 189
coords2, 513
copy, 252
core, 173
core_iqmatoms, 443
corecharge, 206, 442
coremask, 206, 444
corembd, 180
corembdatoms, 181
corr, 861
correlation, 179
coul_CD_split_expon, 670
coul_gaussian_extent_tol, 670
CpHMD, 590
cphstats, 601
crdrep, 510
create, 717
createAtom, 253
createResidue, 253
createUnit, 253
crgmask, 521
csurften, 405
csv_format, 904
cter, 634
cut, 418, 439
cut_bond_list_file, 208
cutcap, 416
cutfd, 88
cutnb, 88
cuv, 128, 137
cv_file, 567
cv_i, 561
cv_max, 569
cv_min, 569

cv_ni, 562
 cv_nr, 562
 cv_r, 562
 cv_type, 561
 cwt, 638

D

d3a1, 194
 d3a2, 194
 d3s8, 195
 DX, 145
 damp, 441
 datafile, 717
 dataset, 637
 datasetc, 638
 dbuff1, 211
 dbuff2, 211
 dcut, 638
 debug, 180, 186, 189, 192, 194, 722
 debug_printlevel, 901
 dec_verbose, 904
 decompmask, 421
 decompopt, 89
 defineSolvent, 295
 deleteBond, 253, 295
 deleteDihedral, 295
 deletePDB, 296
 delta, 511
 denserms, 185
 density, 125
 density_predict, 443
 desc, 253, 553
 dft, 180
 dftb_3rd_order, 150, 166, 444
 dftb_chg, 150, 443
 dftb_disper, 150, 443
 dftb_maxiter, 150, 443
 dftb_slko_path, 150
 dftb_telec, 150, 441
 dftb_telec_step, 441
 dftb_chg, 166
 dftb_maxiter, 166
 dftb_telec, 166
 dftd, 188
 dftgrid, 189
 diag_routine, 151, 166, 443
 dielc, 418, 439, 904
 dieps, 125
 dihedral, 771
 dij, 638
 dimer, 513
 dimer_maxrot, 513
 dimer_mode, 513

dimer_tolrot, 513
 dipmass, 426
 dipole, 174ff, 178f, 189
 dipole_scf_tol, 428
 dipole_solv_opt, 428
 dipole_scf_iter_max, 669
 dipole_scf_tol, 669
 diptau, 426
 diptol, 425
 dir, 502
 direct correlation function, 111
 disang, 512
 distance, 774
 do_dipole, 180
 do_parallel, 185
 dobsl, 637
 do_debugf, 433
 do_vdw_longrange, 669
 do_vdw_taper, 669
 dpmax, 212
 dprob, 85
 dr, 124
 driven_cutoff, 570
 driven_weight, 570
 drms, 400, 507, 904
 drmsd, 774
 dsum_tol, 422
 dt, 400
 dtfb_disper, 166
 dump_pathCV_def, 503
 dumpfrc, 433
 dvbips, 425
 dvdl_norest, 520
 dwt, 637
 dx0, 400
 dynlmb, 521

E

E-REMD, 548
 EMIL, 576
 ee_dsum_cut, 428
 ee_damped_cut, 669
 eedmeth, 423
 ee_dsum_cut, 669
 eedtb dns, 423
 effreq, 420
 efphase, 420
 efx, 419f
 efy, 420
 efz, 420
 embed, 180
 embedatoms, 180
 emem, 903

INDEX

emilParameters, 577
emil_do_calc, 576
emil_logfile, 577, 579
emil_model_infile, 577
emil_model_outfile, 577
emil_paramfile, 577
emil_sc, 576
emix, 632
endframe, 901
ene_avg_sampling, 463
eneopt, 87
energy, 296
energy_window, 508
entropicDecomp, 124, 135, 140
entropy, 128, 901
epsilonTrap, 577
epsilonWell, 577
epsin, 84
epsmem, 84
epsout, 84
equilibration, 572
errconv, 153, 441
es_cutoff, 462
espgen, 323
espgen.py, 367
ew_type, 422, 439
ew_coeff, 423
exactdensity, 174
excess chemical potential, 114
exch_type, 553
exchange, 179
exchange_freq, 571
exchange_log_file, 571
exchange_log_freq, 571
exch_cutoff, 670
exchem, 128
exch_factor, 670
exch_gaussian_extent_tol, 670
exdi, 903
executable, 176, 185, 189
explored_low_modes, 508
export, 186
ext_buffermask_subset, 208
ext_coremask_subset, 208
ext_qmmask_subset, 208
extdiel, 67, 438
extra_precision, 125

F

FCE, 126
fcap, 416
fcecrrd, 133
fceanormsw, 133

fceifreq, 134
fcenbase, 133
fcentbasis, 133
fcentfrcor, 135
fcesort, 133
fcestride, 133
fctrans, 134
fcweigh, 133
fcons, 653
ffield, 218
ffp_auto_setup, 670
fft_grids_per_ang, 463
fillratio, 87, 903
finalgrid, 177
finddgrf.py, 598
fit_type, 173
fix_atom_list, 207
fix_ends, 504
fixremdcouts.py, 557
fock_predict, 443
fockp_d1, 441
fockp_d2, 441
fockp_d3, 441
fockp_d4, 441
force_constant, 504
force_constant_d, 503
force_gamma, 504
force_kappa, 504
frameon, 425
frcopt, 88
freezemol, 638
frequency_eigenvector_recalc, 508
frequency_ligand_rottrans, 508
fscale, 87
fswitch, 419
full_traj, 902
fullscf, 213
fxyz, 631

G

gamma, 504
gamma_ln, 207
gamma_ln_qm, 207
gamma_ten, 405
gamma_ln, 403
gammamap, 417
Gaussian fluctuation, 114, 140
gaussian_recip_tol, 670
gbsa, 68, 439
gem_verbose, 670
genremdinputs.py, 533
gfCorrection, 129
gh_period, 503

gigj, 637
 gist, 781
 gms_version, 175
 go, 297
 gpu, 469
 gpuids, 189
 gradcutoff, 185
 grdspc, 129, 131, 138, 905
 grend_acyc, 532
 grid, 177
 gridips, 425
 grids, 653
 grms_tol, 152
 grnam1, 631
 gromber, 297
 grote_hynes, 503
 group, 553
 groupSelectedAtoms, 254
 gti_add_re, 552
 gti_add_sc, 525
 gti_bat_sc, 525
 gti_chg_keep, 527
 gti_cpu_output, 527
 gti_cut, 527
 gti_cut_sc, 524
 gti_cut_sc_off, 524
 gti_cut_sc_on, 524
 gti_ele_exp, 524
 gti_ele_sc, 523
 gti_lam_sch, 523
 gti_output, 527
 gti_scale_beta, 524
 gti_syn_mass, 526
 gti_vdw_exp, 524
 gti_vdw_sc, 524
 guess, 179, 188
 guess_file, 502
 guv, 128, 137

H

HMassRepartition, 298
 HNC, 112, 114
 hamiltonian, 211
 harm, 567
 harm_mode, 567
 hbond, 793
 hbondcut, 218
 hcorrection, 194
 help, 723
 hess_iseg, 513
 hess_nseg, 512
 hess_out, 512
 hessini, 511

hessupd, 511
 hist, 868
 history, 298
 host, 188
 hot_spot, 209, 443
 huv, 128, 137
 hybridgb, 541
 hypernetted-chain approximation, 112

I

IPMach.py, 362
 ialtd, 629
 iamoeba_, 668
 iamoeba, 419
 iat, 627
 iatr, 634
 ibelly, 399
 icfe, 516, 576
 iconstr, 632
 icsa, 638
 id, 637
 id2o, 633
 idecomp, 399, 517, 905
 idftd3, 182
 idistr, 403
 idprc, 196
 iemap, 417
 ievb, 419
 iextpot, 673
 ifit, 653
 ifmbar, 530
 ifntyp, 631
 ifqnt, 419, 439, 902
 ifreaf, 552
 ifsc, 520, 576
 ifvari, 629f
 ig, 402
 igb, 65, 296, 300, 419, 439, 902
 igr1, 631
 ihp, 632
 image, 572, 796
 imin, 83, 395
 impose, 255
 imult, 629
 include_polarization_energy, 218
 indi, 903
 indmeth, 425
 ineb, 500
 infe, 561
 initial_selection_type, 207
 inp, 84, 903
 intcutoff, 185
 intdiel, 67, 438

INDEX

integration, 173
interfile, 196
interpolate, 298
interval, 901
intrafile, 196
intramolecular pair correlation matrix, 112
invwt1, 633
ionstepvelocities, 398
ioutfm, 398
ipb, 83, 419, 439
ipgm_, 427
ipgm, 419
ipnlty, 417
ipol, 419
ipolyn, 213
iprob, 85
iprot, 634f
ips, 424
iqmatoms, 163, 441
ir6, 631
iresid, 629
irest, 397
irism, 129
irism, 419
irstdip, 426
irstyp, 629
iscale, 417
ischeme, 410
isgend, 481
isgld, 480
isgsta, 481
istrng, 84, 903
itgtmd, 495
ithermostat, 410
itrmax, 152, 166, 442
ivcap, 416
iwrap, 398
iwrtcharges, 182
iwrteigen, 182
ixpk, 631

J

jbasis, 177
jcoupling, 797
jfastw, 405, 439

K

KH, 112, 114
KMMD, 672
k_sol, 658
kappa, 441
keep_files, 901
keep_scr, 188

Kernel Modified Molecular Dynamics, 672
keywords, 186
klambda, 517, 576
kmaxqx, 164, 443
kmaxqy, 443
kmaxqz, 443
Kovalenko-Hirata, 112
ksave, 125
ksqmaxq, 164
ksqmaxsq, 443

L

lambda, 213, 576
lambda-scheduling, 522
lbfgsmem, 512
lbfgs_memory_depth, 507
lie, 800
ligand_mask, 901
ligcent_list, 510
ligmask, 421
ligstart_list, 510
linit, 903
link_atomic_no, 442
list, 257
listParms, 298
lj1264, 419, 439
ljTolerance, 119, 131, 138
lmod_job_title, 508
lmod_minimize_grms, 508
lmod_relax_grms, 508
lmod_restart_frequency, 508
lmod_step_size_max, 508
lmod_step_size_min, 508
lmod_trajectory_filename, 509
lmod_verbosity, 509
lnk_dis, 441
lnk_method, 442
lnk_atomic_no, 167
lnk_dis, 166
lnk_method, 166
loadAmberParams, 257
loadAmberPrep, 257
loadCoordinates, 299
loadMol2, 257
loadOff, 257
loadPdb, 258
loadPdbUsingSeq, 258
loadRestrtr, 299
logFile, 258
logdvdl, 520
long-range asymptotics, 128
longrange, 212

M

MCPB.py, 357
MDIIS, 113, 122
MDL, 140
MTS, 126
mapfile, 652
mapfit, 653
mask, 196, 804
mask_update_period, 658
match, 326
match_atomname, 327
matrix_vector_product_method, 507
Mav_damp, 504
max_bonds_per_atom, 208
max_scf_iterations, 182
maxcore, 177
maxcyc, 152, 400, 507, 904
maxcycle, 512
maxit, 175, 189
maxiter, 177, 192, 194, 426
maxitn, 86
maxsph, 90
maxstep, 125, 132, 139, 512
maxupdate, 511
mbar_lambda, 530
mbar_states, 530
mcbarint, 404
mcligshift, 420
mctrdz, 86, 904
mcwat, 420
mcwatmask, 420
mdiis_del, 122, 125, 132, 139
mdiis_method, 132
mdiis_nvec, 122, 125, 132, 139
mdiis_restart, 122, 125, 132, 139
mdinfo_flush_interval, 462
mdout_flush_interval, 462
mean solvation force, 114
measureGeom, 258
mem, 176, 180
membraneopt, 86, 92
memopt, 903
merge_residues, 512
metalpdb2mol2.py, 369
method, 175ff, 179, 185, 188, 211
midpoint, 422
min_heavy_mass, 209, 441
minimize, 299, 503
minstep, 512
mipso, 425
mipsx, 424
mixer, 194
ml_update_period, 658
mlimit, 423
mlses_opt, 90
mltpro, 635
mmhardness, 192
mode, 569
model, 125
modif, 212
modified direct inversion of the iterative subspace, 113
mol2rtf.py, 369
molReconstruction, 135
molfit, 653
molsurf, 807, 903
mom_cons_region, 207
mom_cons_type, 207
monitor_file, 569
monitor_freq, 569
monte_carlo_method, 509
move, 653
mprob, 86
msoffset, 903
mt19937_file, 571
mt19937_seed, 571
mthick, 86, 92, 903
mtmdforce, 497
mtmdform, 496
mtmdmask, 497
mtmdmult, 497
mtmdninc, 497
mtmdrmsd, 497
mtmdstep1, 497
mtmdvari, 497
Multidimensional REMD, 553
multipmemd, 435
multisander, 435
mutant_only, 904
mwords, 175
mxsub, 417

N

NCCL, 468
n_max_recursive, 208
n_partition, 202
namr, 634
nastruct, 810
natr, 634
nbflag, 423
nbins, 504
nbrcut, 218
nbtell, 423
nbuffer, 87
nchain, 207
ncore, 211
ncyc, 400

INDEX

ndiis_attempts, 442
ndiis_matrices, 442
ndiis_attempts, 153
ndiis_matrices, 153
ndip, 637f
nearest_qm_solvent, 201
nearest_qm_solvent_center_id, 201
nearest_qm_solvent_fq, 201
nearest_qm_solvent_resname, 201
neb, 513
neb_cart, 514
neb_climb_test, 513
neb_free_test, 513
neb_nimage, 513
nebk, 513
neglgdel, 433
netCharge, 300
netcdf, 901
netfrc, 423
neural network, 672
nfe_abmd, 568
nfe_bbmd, 570
nfe_pmd, 567
nfe_smd, 567
nfe_stsm, 571
nfft3, 422
nfft_for_gridtype, 670
nfocus, 87
ng, 138, 905
ng3, 131
ngpus, 189
nharm, 567
ninc, 629
ninterface, 405
nkija, 403
nkout, 125
nleb, 175
nmc, 420
nmd, 420
nme, 635
nmendframe, 904
nminterval, 904
nmo_corembd, 181
nmo_embed, 180
nmode_igb, 904
nmode_istrng, 904
nmpmc, 635
nmropt, 397
nmstartframe, 904
noasymptcorr, 138
node, 503
nodes, 503
noeskp, 417
noexitonerror, 724
no_intermolecular_bonds, 463
noprogess, 724
noshakemask, 519
noshakemask, 406
npath, 567
npbgrid, 87
npbopt, 86
npbverb, 89
npeak, 632
nprintlog, 180
npropagate, 118, 132, 139
nprot, 634f
nr, 124
nrad, 175
nranatm, 433
nrespa, 401
nring, 634
nrout, 124
nscm, 400
nsnb, 419
nsnba, 88
nsp, 125
nstep1, 629
nstlim, 400
ntave, 397
ntb, 418, 439
ntc, 405, 440
nter, 634
ntf, 418, 439
ntmin, 400, 507
ntp, 403
ntpr, 152, 174ff, 178ff, 189, 397
ntr, 399
ntt, 207, 401
ntwc, 213
ntwd, 420
ntwe, 398
ntwf, 398
ntwdrst, 209
ntwpdb, 209
ntwprt, 398
ntwr, 397
ntwrism, 135
ntwsf, 658
ntwv, 398
ntwx, 398
ntx, 83, 397
ntxo, 397
num_mpi_procs, 179
num_threads, 173, 175f, 178f
number_free_rottrans_modes, 509
number_ligand_rottrans, 509

number_ligands, 509
 number_lmod_iterations, 509
 number_lmod_moves, 509
 num_datasets, 637
 numthreads, 218
 numwatkeep, 541
 nvec, 117
 nxpk, 631

O

OMP_NUM_THREADS, 330
 obs, 634f
 offset, 68, 90
 omega, 633
 only_PMF, 503
 OpenMM, 300
 OptC4.py, 364
 optalg, 511
 optkon, 635
 optphi, 635
 order, 422
 Ornstein-Zernike, 111
 oscale, 633
 outCIF, 301
 outfprefix, 186
 outlist, 124
 outlvlset, 89
 outmlvlset, 89
 outparm, 302
 output_file, 567
 output_freq, 567
 output_period, 503
 outtraj, 817
 outxyz, 631
 oxidation_number_list_file, 208

P

PBRadii, 261
 PC+/3D-RISM, 129, 140
 pH-REMD, 546
 PSE-n, 112, 114
 packmol-memgen, 231
 pair-distribution function, 111
 parameter_file, 166
 parameterfile, 152
 paramfit, 329
 params_file, 504
 parm, 302, 736
 ParmEd, 283
 parmbox, 737
 parmcals, 325
 parmchk2, 316
 parminfo, 738

parmout, 302
 parmresinfo, 739
 parmstrip, 738
 parmwrite, 738
 path, 567
 path_mode, 567
 pbtemp, 84
 pdb, 137
 PdbSearcher.py, 364
 pdb_file, 209
 pdb_infile, 658
 pdb_outfile, 658
 pdb_read_coordinates, 658
 pencut, 417
 peptcorr, 213
 peptide_corr, 152, 166, 442
 peptk, 214
 periodic, 129, 138
 phiform, 88, 95
 phiout, 95
 plj1264, 419, 439
 pme_auto_setup, 670
 points_extra, 504
 points_per_node, 504
 pol_gauss_ips -, 428
 pol_gauss_verbose -, 427
 polarDecomp, 120, 135, 140
 polardecomp, 118, 905
 poreradius, 86
 poretype, 86, 904
 port, 188
 position_gamma, 504
 potUVroot, 128
 prbrad, 903
 precision, 188, 725
 preparation_steps, 503
 prepgen, 322
 pres0, 404
 printAngles, 303
 printBonds, 303
 printDetails, 303
 printDihedrals, 303
 printFlags, 303
 printInfo, 303
 printLJMatrix, 303
 printLJTypes, 303
 printPointers, 304
 print_eigenvalues, 152, 442
 print_qm_coords, 202
 print_res, 905
 printbondorders, 442
 printcharges, 152, 166, 442
 printdipole, 165, 442

INDEX

prmtop, 137
ProScrs.py, 367
probe, 903
profile_mpi, 426
progress, 125, 135
proteinmask, 421
pseudo.diag, 442
pseudo.diag_criteria, 441
pseudo.diag, 151, 166
pseudo.diag_criteria, 151, 166
pucker, 820

Q

qm_center_atom_id, 201
qm_ewald, 443
qm_level, 191, 194
qm_pme, 443
qm_residues, 902
qm_theory, 444, 902
qmcharge, 150, 166, 206, 442, 903
qmcut, 163, 441, 903
qm_ewald, 164
qmgb, 164, 442
qmmask, 163, 206, 443
qmmm_int, 443
qmmm_switch, 443
qmmm_int, 153, 165
qmmmrij_incore, 442
qmmm_switch, 164
qm_pme, 164
mqm_erep_incore, 442
mqmmdx, 150, 166, 442
qmshake, 165, 442
qm_theory, 149, 165f
quit, 304
quv, 137
qxd, 152, 166

R

r0, 630
RA, 202
REAF, 550
REX_period, 503
RISM, 111, 126
RISMnRESPA, 126
RT, 202
rTrap, 577
rWell, 577
r_buffer_in, 207
r_core_in, 206
r_core_out, 206f
r_qm_in, 206
r_switch_hi, 441

r_switch_lo, 441
radgyr, 821
radial, 821
radiopt, 85, 903
raips, 424
ramdboost, 421
ramdboostfreq, 421
ramdboostrate, 421
ramdint, 421
ramdligmask, 421
ramdmaxdist, 421
ramdprotmask, 421
random_seed, 509
ranseed, 433
rbornstat, 68
rcut, 196
rdt, 68, 439
read_M, 504
read_idrst_file, 209
read_pathCV_def, 503
readdata, 726
readinput, 727
reaf_mask1, 552
reaf_mask2, 552
reaf_tau, 552
reaf_temp, 552
receptor_mask, 902
recycleinitguess, 189
refcrd, 510
reference, 743
refin, 496
reflection_infile, 658
reg_ewald_auto_setup, 670
release, 572
remove, 259
repeats, 572
report_centers, 572
residuegen, 325
resolution, 569, 653
respgen, 323
restart_pool_size, 509
restraint, 628
restraintmask, 399
restraint_wt, 399
reuse_dmx, 185
reweight_z, 504
rgbmax, 67, 439
rhow_effect, 89
rismld, 111
rismld, 120f, 123
rism3d.snlpnt, 137
rism_verbose, 905
rismnrespa, 133

rjcoef, 630
 rms2d, 880
 rmsavgcorr, 881
 rmsd, 824
 rmsfrc, 433
 rotmin_list, 510
 rst, 137
 Rst7, 456
 rstwt, 627
 rsum_tol, 423
 r_switch_hi, 164
 r_switch_lo, 164
 rtemperature, 509
 runavg, 828

S

s11, 637
 saltcon, 67, 296, 300, 439, 903
 sander, 126
 sander_apbs, 903
 saopt, 85
 sasopt, 85
 saveAmberParm, 259
 saveMol2, 259
 saveOff, 260
 savePdb, 260
 sc_bond_mask1, 525f
 scaldip, 426
 scale, 304, 903
 scale_update_period, 658
 scalec, 88
 scalestep, 512
 scalm, 417
 scalpha, 577
 scbeta, 524, 577
 scee, 304
 scf_cg_niter_, 428
 scf_conv, 173, 175f, 179
 scf_cyc, 185
 scf_iter, 173
 scf_sor_coefficient_, 428
 scf_sor_niter_, 428
 scfconv, 151, 166, 177, 441
 scftol, 194
 scmask, 521
 scmask1, 521
 scmask2, 521
 scnb, 304
 scndir, 188
 screen, 213
 search_path, 902
 secstruct, 829
 select, 728
 selection_constant, 569
 selection_epsilon, 569
 selection_freq, 569
 selection_type, 207
 selftest, 125
 sequence, 260
 set, 260
 set container, 261
 set default, 260
 setAngle, 304
 setBond, 304
 setBox, 262
 setMolecules, 305
 setOverwrite, 305
 sf_outfile, 658
 sgff, 481
 sgft, 481
 sgmask, 481
 sgsize, 481
 sgtype, 481
 shcut, 634
 shrang, 634
 sigmatol, 182
 silent, 194
 sinrtau, 403
 sirahff, 58
 skin_permit, 423
 skinnb, 423
 skmax, 500
 skmin, 500
 smear, 126
 smoothing, 572
 smoothopt, 84
 Smoothstep, 522
 snapshots_basename, 569
 snapshots_freq, 569
 solvateBox, 263
 solvateCap, 263
 solvateOct, 263
 solvateShell, 264
 solvation, 111, 114
 solvation free energy, 114
 solvbox, 119, 131, 138, 905
solvcut, 129
 solvcut, 138, 905
 solvene, 128
 solvent_atom_number, 208
 solvent_mask_adjustment, 658
 solvent_mask_probe_radius, 658
 solvmaxit, 218
 solvopt, 86
 solvprecond, 218
 solvtol, 218

INDEX

sor_coefficient, 669
source, 305
space, 87
spin, 150, 166, 442
sprob, 89
startframe, 902
step, 503
str, 634
string_move, 504
string_move_period, 503
strip, 305, 833
strip_mask, 902
summary, 305
surf, 834
surfoff, 903
surften, 68, 90, 903

T

t, 400
T-REMD, 537
target, 658
taumet, 633
taup, 404
taurot, 633
tausw, 417
tautp, 402
tcfile, 188
temp0, 402
temp0les, 402
temperature, 125
tempi, 402
tempsg, 481
termi, 191
tfermi, 194
tgtfitmask, 496, 500
tgtmdfrc, 496
tgtrmsd, 496
tgtrmsmask, 496, 500
thbcut, 218
theory, 124
therm_par, 410
thermo, 906
thermodynamics, 114
threall, 188
tiMerge, 306
ti_vdw_mask, 527
tight_p_conv, 151, 166, 443
timask1, 517
timask2, 517
timescale, 569
tishake, 519f
tishake, 517, 526
tmode, 500

tol, 405, 512
tole, 512
tolerance, 119, 121, 125, 131, 138, 906
tolpro, 635
total correlation function, 111
total_low_modes, 509
trajin, 744
trajout, 746
transform, 255, 264
translate, 264
treeCoulomb, 130, 139
treeCoulombMAC, 130, 139
treeCoulombN0, 130, 139
treeCoulombOrder, 130, 139
treeDCF, 130, 138
treeDCFMAC, 130, 139
treeDCFN0, 130, 139
treeDCFOrder, 130, 139
treeTCF, 130, 139
treeTCFMAC, 130, 139
treeTCFN0, 130, 139
treeTCFOrder, 130, 139
triopt, 85
trmin_list, 510
trustrad, 511
tsgavg, 481

U

uccoeff, 129
Universal Correction, 140
unstrip, 838
use_dftb, 173
use_sander, 902
use_template, 174ff, 178ff, 186, 189
use_axis_opt, 463
use_rmin, 89
use_sav, 90
uuv, 137

V

vdw_cutoff, 462
vdwmeth, 423, 439
verbose, 135, 140, 422, 902
verbosity, 151, 166, 180, 201f, 265, 442, 512
vfac, 500
vlimit, 403
volfmt, 135, 137
vprob, 89
vrand, 403
vshift, 153, 441
vsolv, 166, 443
vv, 500

W

writeFrcmod, 306
writeOFF, 306
write_M, 504
write_idrst_file, 209
write_thermo, 135
writepdb, 166
wt, 634f
wt_temperature, 570
wt_umbrella_file, 570
wtmask, 512
wtmm, 512

X

XVV, 141
xc, 173
xccutoff, 185
xmax, 91
xmin, 91
xmin_method, 507
xmin_verbosity, 507
xray_weight_final, 658
xray_weight_initial, 658
xvv, 128, 137

Y

ymax, 91
ymin, 91

Z

zMatrix, 265
z_bias, 503
zcap, 416
zerochg, 433
zerodip, 433
zerofrc, 132
zerovdw, 433
zlmfit, 173
zmax, 91
zmin, 91